

MARAN SUITE SYSTEM

Código Fuente Completo

Generado: 5/3/2026, 03:07:42

ÍNDICE DE ARCHIVOS

1. package.json
2. tsconfig.json
3. vite.config.ts
4. tailwind.config.ts
5. drizzle.config.ts
6. components.json
7. shared/schema.ts
8. server/index.ts
9. server/routes.ts
10. server/storage.ts
11. server/vite.ts
12. server/static.ts
13. script/build.ts
14. client/index.html
15. client/src/main.tsx
16. client/src/App.tsx
17. client/src/index.css
18. client/src/lib/queryClient.ts
19. client/src/lib/utils.ts
20. client/src/hooks/use-toast.ts
21. client/src/hooks/use-mobile.tsx
22. client/src/components/app-sidebar.tsx
23. client/src/components/theme-provider.tsx
24. client/src/components/theme-toggle.tsx
25. client/src/components/entity-selector.tsx
26. client/src/pages/dashboard.tsx
27. client/src/pages/rooms.tsx
28. client/src/pages/guests.tsx
29. client/src/pages/companies.tsx
30. client/src/pages/reservations.tsx
31. client/src/pages/new-reservation.tsx
32. client/src/pages/planning.tsx
33. client/src/pages/check-in.tsx
34. client/src/pages/check-out.tsx
35. client/src/pages/restaurant.tsx
36. client/src/pages/spa.tsx
37. client/src/pages/events.tsx
38. client/src/pages/housekeeping.tsx
39. client/src/pages/maintenance.tsx
40. client/src/pages/inventory.tsx
41. client/src/pages/administration.tsx
42. client/src/pages/hospitality.tsx
43. client/src/pages/chatbot-dashboard.tsx
44. client/src/pages/web-checkin-public.tsx
45. client/src/pages/groups.tsx
46. client/src/pages/group-detail.tsx
47. client/src/pages/packages.tsx
48. client/src/pages/rate-plans.tsx
49. client/src/pages/ota-channels.tsx
50. client/src/pages/reviews.tsx
51. client/src/pages/not-found.tsx

```
1 % {
2 %   "name": "rest-express",
3 %   "version": "1.0.0",
4 %   "type": "module",
5 %   "license": "MIT",
6 %   "scripts": {
7 %     "dev": "NODE_ENV=development tsx server/index.ts",
8 %     "build": "tsx script/build.ts",
9 %     "start": "NODE_ENV=production node dist/index.cjs",
10 %    "check": "tsc",
11 %    "db:push": "drizzle-kit push"
12 %  },
13 %  "dependencies": {
14 %    "@hookform/resolvers": "^3.10.0",
15 %    "@jridgewell/trace-mapping": "^0.3.25",
16 %    "@radix-ui/react-accordion": "^1.2.4",
17 %    "@radix-ui/react-alert-dialog": "^1.1.7",
18 %    "@radix-ui/react-aspect-ratio": "^1.1.3",
19 %    "@radix-ui/react-avatar": "^1.1.4",
20 %    "@radix-ui/react-checkbox": "^1.1.5",
21 %    "@radix-ui/react-collapsible": "^1.1.4",
22 %    "@radix-ui/react-context-menu": "^2.2.7",
23 %    "@radix-ui/react-dialog": "^1.1.7",
24 %    "@radix-ui/react-dropdown-menu": "^2.1.7",
25 %    "@radix-ui/react-hover-card": "^1.1.7",
26 %    "@radix-ui/react-label": "^2.1.3",
27 %    "@radix-ui/react-menubar": "^1.1.7",
28 %    "@radix-ui/react-navigation-menu": "^1.2.6",
29 %    "@radix-ui/react-popover": "^1.1.7",
30 %    "@radix-ui/react-progress": "^1.1.3",
31 %    "@radix-ui/react-radio-group": "^1.2.4",
32 %    "@radix-ui/react-scroll-area": "^1.2.4",
33 %    "@radix-ui/react-select": "^2.1.7",
34 %    "@radix-ui/react-separator": "^1.1.3",
35 %    "@radix-ui/react-slider": "^1.2.4",
36 %    "@radix-ui/react-slot": "^1.2.0",
37 %    "@radix-ui/react-switch": "^1.1.4",
38 %    "@radix-ui/react-tabs": "^1.1.4",
39 %    "@radix-ui/react-toast": "^1.2.7",
40 %    "@radix-ui/react-toggle": "^1.1.3",
41 %    "@radix-ui/react-toggle-group": "^1.1.3",
42 %    "@radix-ui/react-tooltip": "^1.2.0",
43 %    "@tanstack/react-query": "^5.60.5",
44 %    "class-variance-authority": "^0.7.1",
45 %    "clsx": "^2.1.1",
46 %    "cmdk": "^1.1.1",
47 %    "connect-pg-simple": "^10.0.0",
48 %    "date-fns": "^3.6.0",
49 %    "drizzle-orm": "^0.39.3",
50 %    "drizzle-zod": "^0.7.1",
51 %    "embla-carousel-react": "^8.6.0",
52 %    "express": "^4.21.2",
53 %    "express-session": "^1.18.1",
54 %    "framer-motion": "^11.13.1",
55 %    "input-otp": "^1.4.2",
56 %    "lucide-react": "^0.453.0",
57 %    "memorystore": "^1.6.7",
58 %    "next-themes": "^0.4.6",
59 %    "openai": "^6.15.0",
60 %    "p-limit": "^7.2.0",
61 %    "p-retry": "^7.1.1",
62 %    "passport": "^0.7.0",
63 %    "passport-local": "^1.0.0",
64 %    "pdfkit": "^0.17.2",
65 %    "pg": "^8.16.3",
66 %    "react": "^18.3.1",
67 %    "react-day-picker": "^8.10.1",
68 %    "react-dom": "^18.3.1",
69 %    "react-hook-form": "^7.55.0",
70 %    "react-icons": "^5.4.0",
71 %    "react-resizable-panels": "^2.1.7",
72 %    "recharts": "^2.15.2",
73 %    "tailwind-merge": "^2.6.0",
74 %    "tailwindcss-animate": "^1.0.7",
75 %    "tw-animate-css": "^1.2.5",
76 %    "vaul": "^1.1.2",
77 %    "wouter": "^3.3.5",
78 %    "ws": "^8.18.0",
79 %    "zod": "^3.25.76",
80 %    "zod-validation-error": "^3.5.4"
81 %  },
82 %  "devDependencies": {
83 %    "@replit/vite-plugin-cartographer": "^0.4.4",
84 %    "@replit/vite-plugin-dev-banner": "^0.1.1",
85 %    "@replit/vite-plugin-runtime-error-modal": "^0.0.3",
86 %    "@tailwindcss/typography": "^0.5.15",
87 %    "@tailwindcss/vite": "^4.1.18",
88 %    "@types/connect-pg-simple": "^7.0.3",
89 %    "@types/express": "4.17.21",
90 %    "@types/express-session": "^1.18.0",
```

package.json (cont.)

```
91 %     "@types/node": "20.19.27",
92 %     "@types/passport": "^1.0.16",
93 %     "@types/passport-local": "1.0.38",
94 %     "@types/react": "18.3.11",
95 %     "@types/react-dom": "18.3.1",
96 %     "@types/ws": "8.5.13",
97 %     "@vitejs/plugin-react": "4.7.0",
98 %     "autoprefixer": "10.4.20",
99 %     "drizzle-kit": "0.31.8",
100 %    "esbuild": "0.25.0",
101 %    "postcss": "8.4.47",
102 %    "tailwindcss": "3.4.17",
103 %    "tsx": "4.20.5",
104 %    "typescript": "5.6.3",
105 %    "vite": "7.3.0"
106 %  },
107 %  "overrides": {
108 %    "drizzle-kit": {
109 %      "@esbuild-kit/esm-loader": "npm:tsx@4.20.4"
110 %    }
111 %  },
112 %  "optionalDependencies": {
113 %    "bufferutil": "4.0.8"
114 %  }
115 % }
116 %
```

```
1 % {
2 %   "include": ["client/src/**/*", "shared/**/*", "server/**/*"],
3 %   "exclude": ["node_modules", "build", "dist", "**/*.test.ts"],
4 %   "compilerOptions": {
5 %     "incremental": true,
6 %     "tsBuildInfoFile": "./node_modules/typescript/tsbuildinfo",
7 %     "noEmit": true,
8 %     "module": "ESNext",
9 %     "strict": true,
10 %    "lib": ["esnext", "dom", "dom.iterable"],
11 %    "jsx": "preserve",
12 %    "esModuleInterop": true,
13 %    "skipLibCheck": true,
14 %    "allowImportingTsExtensions": true,
15 %    "moduleResolution": "bundler",
16 %    "baseUrl": ".",
17 %    "types": ["node", "vite/client"],
18 %    "paths": {
19 %      "@/*": ["/client/src/*"],
20 %      "@shared/*": ["/shared/*"]
21 %    }
22 %   }
23 % }
24 %
```

```

1 % import { defineConfig } from "vite";
2 % import react from "@vitejs/plugin-react";
3 % import path from "path";
4 % import runtimeErrorOverlay from "@replit/vite-plugin-runtime-error-modal";
5 %
6 % export default defineConfig({
7 %   plugins: [
8 %     react(),
9 %     runtimeErrorOverlay(),
10 %    ...(process.env.NODE_ENV !== "production" &&
11 %      process.env.REPL_ID !== undefined
12 %        ? [
13 %          await import("@replit/vite-plugin-cartographer").then((m) =>
14 %            m.cartographer(),
15 %          ),
16 %          await import("@replit/vite-plugin-dev-banner").then((m) =>
17 %            m.devBanner(),
18 %          ),
19 %        ]
20 %        : []),
21 %   ],
22 %   resolve: {
23 %     alias: {
24 %       "@": path.resolve(import.meta.dirname, "client", "src"),
25 %       "@shared": path.resolve(import.meta.dirname, "shared"),
26 %       "@assets": path.resolve(import.meta.dirname, "attached_assets"),
27 %     },
28 %   },
29 %   root: path.resolve(import.meta.dirname, "client"),
30 %   build: {
31 %     outDir: path.resolve(import.meta.dirname, "dist/public"),
32 %     emptyOutDir: true,
33 %   },
34 %   server: {
35 %     fs: {
36 %       strict: true,
37 %       deny: ["**/*.***"],
38 %     },
39 %   },
40 % });
41 %

```

```

1 % import type { Config } from "tailwindcss";
2 %
3 % export default {
4 %   darkMode: ["class"],
5 %   content: [".client/index.html", ".client/src/**/*.{js,jsx,ts,tsx}"],
6 %   theme: {
7 %     extend: {
8 %       borderRadius: {
9 %         lg: ".5625rem", /* 9px */
10 %        md: ".375rem", /* 6px */
11 %        sm: ".1875rem", /* 3px */
12 %      },
13 %      colors: {
14 %        // Flat / base colors (regular buttons)
15 %        background: "hsl(var(--background) / <alpha-value>)",
16 %        foreground: "hsl(var(--foreground) / <alpha-value>)",
17 %        border: "hsl(var(--border) / <alpha-value>)",
18 %        input: "hsl(var(--input) / <alpha-value>)",
19 %        card: {
20 %          DEFAULT: "hsl(var(--card) / <alpha-value>)",
21 %          foreground: "hsl(var(--card-foreground) / <alpha-value>)",
22 %          border: "hsl(var(--card-border) / <alpha-value>)",
23 %        },
24 %        popover: {
25 %          DEFAULT: "hsl(var(--popover) / <alpha-value>)",
26 %          foreground: "hsl(var(--popover-foreground) / <alpha-value>)",
27 %          border: "hsl(var(--popover-border) / <alpha-value>)",
28 %        },
29 %        primary: {
30 %          DEFAULT: "hsl(var(--primary) / <alpha-value>)",
31 %          foreground: "hsl(var(--primary-foreground) / <alpha-value>)",
32 %          border: "var(--primary-border)",
33 %        },
34 %        secondary: {
35 %          DEFAULT: "hsl(var(--secondary) / <alpha-value>)",
36 %          foreground: "hsl(var(--secondary-foreground) / <alpha-value>)",
37 %          border: "var(--secondary-border)",
38 %        },
39 %        muted: {
40 %          DEFAULT: "hsl(var(--muted) / <alpha-value>)",
41 %          foreground: "hsl(var(--muted-foreground) / <alpha-value>)",
42 %          border: "var(--muted-border)",
43 %        },
44 %        accent: {
45 %          DEFAULT: "hsl(var(--accent) / <alpha-value>)",
46 %          foreground: "hsl(var(--accent-foreground) / <alpha-value>)",
47 %          border: "var(--accent-border)",
48 %        },
49 %        destructive: {
50 %          DEFAULT: "hsl(var(--destructive) / <alpha-value>)",
51 %          foreground: "hsl(var(--destructive-foreground) / <alpha-value>)",
52 %          border: "var(--destructive-border)",
53 %        },
54 %        ring: "hsl(var(--ring) / <alpha-value>)",
55 %        chart: {
56 %          "1": "hsl(var(--chart-1) / <alpha-value>)",
57 %          "2": "hsl(var(--chart-2) / <alpha-value>)",
58 %          "3": "hsl(var(--chart-3) / <alpha-value>)",
59 %          "4": "hsl(var(--chart-4) / <alpha-value>)",
60 %          "5": "hsl(var(--chart-5) / <alpha-value>)",
61 %        },
62 %        sidebar: {
63 %          ring: "hsl(var(--sidebar-ring) / <alpha-value>)",
64 %          DEFAULT: "hsl(var(--sidebar) / <alpha-value>)",
65 %          foreground: "hsl(var(--sidebar-foreground) / <alpha-value>)",
66 %          border: "hsl(var(--sidebar-border) / <alpha-value>)",
67 %        },
68 %        "sidebar-primary": {
69 %          DEFAULT: "hsl(var(--sidebar-primary) / <alpha-value>)",
70 %          foreground: "hsl(var(--sidebar-primary-foreground) / <alpha-value>)",
71 %          border: "var(--sidebar-primary-border)",
72 %        },
73 %        "sidebar-accent": {
74 %          DEFAULT: "hsl(var(--sidebar-accent) / <alpha-value>)",
75 %          foreground: "hsl(var(--sidebar-accent-foreground) / <alpha-value>)",
76 %          border: "var(--sidebar-accent-border)"
77 %        },
78 %        status: {
79 %          online: "rgb(34 197 94)",
80 %          away: "rgb(245 158 11)",
81 %          busy: "rgb(239 68 68)",
82 %          offline: "rgb(156 163 175)",
83 %        },
84 %      },
85 %      fontFamily: {
86 %        sans: ["var(--font-sans)"],
87 %        serif: ["var(--font-serif)"],
88 %        mono: ["var(--font-mono)"],
89 %      },
90 %      keyframes: {

```

tailwind.config.ts (cont.)

```
91 %     "accordion-down": {
92 %         from: { height: "0" },
93 %         to: { height: "var(--radix-accordion-content-height)" },
94 %     },
95 %     "accordion-up": {
96 %         from: { height: "var(--radix-accordion-content-height)" },
97 %         to: { height: "0" },
98 %     },
99 % },
100 % animation: {
101 %     "accordion-down": "accordion-down 0.2s ease-out",
102 %     "accordion-up": "accordion-up 0.2s ease-out",
103 % },
104 % },
105 % },
106 % plugins: [require("tailwindcss-animate"), require("@tailwindcss/typography")],
107 % } satisfies Config;
108 %
```

```
1 % import { defineConfig } from "drizzle-kit";
2 %
3 % if (!process.env.DATABASE_URL) {
4 %   throw new Error("DATABASE_URL, ensure the database is provisioned");
5 % }
6 %
7 % export default defineConfig({
8 %   out: "./migrations",
9 %   schema: "./shared/schema.ts",
10 %   dialect: "postgresql",
11 %   dbCredentials: {
12 %     url: process.env.DATABASE_URL,
13 %   },
14 % });
15 %
```



```
1 % {
2 %   "$schema": "https://ui.shadcn.com/schema.json",
3 %   "style": "new-york",
4 %   "rsc": false,
5 %   "tsx": true,
6 %   "tailwind": {
7 %     "config": "tailwind.config.ts",
8 %     "css": "client/src/index.css",
9 %     "baseColor": "neutral",
10 %    "cssVariables": true,
11 %    "prefix": ""
12 %  },
13 %  "aliases": {
14 %    "components": "@components",
15 %    "utils": "@lib/Utils",
16 %    "ui": "@components/ui",
17 %    "lib": "@lib",
18 %    "hooks": "@hooks"
19 %  }
20 % }
```

```

1 % import { sql } from "drizzle-orm";
2 % import { pgTable, text, varchar, integer, date, timestamp, decimal, boolean, serial, numeric } from "drizzle-orm/pg-c...
3 % import { createInsertSchema } from "drizzle-zod";
4 % import { z } from "zod";
5 %
6 % // Room Types
7 % export const roomTypes = pgTable("room_types", {
8 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
9 %   code: text("code").notNull(),
10 %   name: text("name").notNull(),
11 %   description: text("description"),
12 %   baseOccupancy: integer("base_occupancy").notNull().default(2),
13 %   maxOccupancy: integer("max_occupancy").notNull().default(4),
14 % });
15 %
16 % export const insertRoomTypeSchema = createInsertSchema(roomTypes).omit({ id: true });
17 % export type InsertRoomType = z.infer<typeof insertRoomTypeSchema>;
18 % export type RoomType = typeof roomTypes.$inferSelect;
19 %
20 % // Rate Plans (Planes Tarifarios)
21 % export const ratePlans = pgTable("rate_plans", {
22 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
23 %   name: text("name").notNull(),
24 %   roomId: varchar("room_type_id").notNull(),
25 %   baseRate: decimal("base_rate", { precision: 10, scale: 2 }).notNull(),
26 %   currency: text("currency").notNull().default("ARS"),
27 %   refundable: text("refundable").notNull().default("true"),
28 %   cancellationPolicy: text("cancellation_policy"),
29 % });
30 %
31 % export const insertRatePlanSchema = createInsertSchema(ratePlans).omit({ id: true });
32 % export type InsertRatePlan = z.infer<typeof insertRatePlanSchema>;
33 % export type RatePlan = typeof ratePlans.$inferSelect;
34 %
35 % // Rooms
36 % export type RoomStatus = "available" | "occupied" | "dirty" | "cleaning" | "maintenance" | "oos";
37 % export type RoomFeature = "accessible" | "balcony" | "separable_bed" | "sofa_bed" | "shower_only" | "extra_bed" | "tw...
38 % export type BedConfig = "MAT" | "TWIN" | "MAT_CC" | "TWIN_CC" | "MAT_EXTRA" | "MAT_CC_EXTRA";
39 %
40 % export const rooms = pgTable("rooms", {
41 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
42 %   roomNumber: text("room_number").notNull().unique(),
43 %   roomId: varchar("room_type_id").notNull(),
44 %   floor: integer("floor").notNull().default(1),
45 %   status: text("status").$type<RoomStatus>().notNull().default("available"),
46 %   bedConfig: text("bed_config"),
47 %   features: text("features").array(),
48 %   maxOccupancy: integer("max_occupancy").default(2),
49 %   notes: text("notes"),
50 % });
51 %
52 % export const insertRoomSchema = createInsertSchema(rooms).omit({ id: true });
53 % export type InsertRoom = z.infer<typeof insertRoomSchema>;
54 % export type Room = typeof rooms.$inferSelect;
55 %
56 % // Companies (Empresas)
57 % export type IvaCondition = "responsable_inscripto" | "monotributo" | "exento" | "consumidor_final" | "no_responsable";
58 %
59 % export const companies = pgTable("companies", {
60 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
61 %   razonSocial: text("razon_social").notNull(),
62 %   nombreFantasia: text("nombre_fantasia"),
63 %   direccion: text("direccion"),
64 %   pais: text("pais").default("Argentina"),
65 %   codigoPostal: text("codigo_postal"),
66 %   localidad: text("localidad"),
67 %   provincia: text("provincia"),
68 %   telefono: text("telefono"),
69 %   email: text("email"),
70 %   cuilCuit: text("cuil_cuit").notNull(),
71 %   numeroFiscal: text("numero_fiscal"),
72 %   condicionIva: text("condicion_iva").$type<IvaCondition>().default("responsable_inscripto"),
73 %   inscripcionNacional: text("inscripcion_nacional"),
74 %   inscripcionProvincial: text("inscripcion_provincial"),
75 %   contactName: text("contact_name"),
76 %   contactEmail: text("contact_email"),
77 %   contactPhone: text("contact_phone"),
78 %   creditLimit: decimal("credit_limit", { precision: 12, scale: 2 }).default("0"),
79 %   paymentTermDays: integer("payment_term_days").default(30),
80 %   notes: text("notes"),
81 %   isActive: text("is_active").default("true"),
82 %   createdAt: text("created_at"),
83 % });
84 %
85 % export const insertCompanySchema = createInsertSchema(companies).omit({ id: true });
86 % export type InsertCompany = z.infer<typeof insertCompanySchema>;
87 % export type Company = typeof companies.$inferSelect;
88 %
89 % // Guests
90 % export type GuestSex = "masculino" | "femenino" | "otro" | "no_especifica";

```

shared/schema.ts (cont.)

```
91 % export type GuestSegment = "LEISURE" | "CORP" | "SPORT" | "CONGRESS" | "OTHER";
92 %
93 % export const guests = pgTable("guests", {
94 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
95 %   codigo: text("codigo").unique(),
96 %   firstName: text("first_name").notNull(),
97 %   lastName: text("last_name").notNull(),
98 %   email: text("email"),
99 %   phone: text("phone"),
100 %   documentType: text("document_type"),
101 %   documentNumber: text("document_number"),
102 %   nationality: text("nationality"),
103 %   direccion: text("direccion"),
104 %   localidad: text("localidad"),
105 %   codigoPostal: text("codigo_postal"),
106 %   fechaNacimiento: text("fecha_nacimiento"),
107 %   sexo: text("sexo").$type<GuestSex>().default("no_especifica"),
108 %   segment: text("segment").$type<GuestSegment>().default("LEISURE"),
109 %   cuilCuit: text("cuil_cuit"),
110 %   companyId: varchar("company_id"),
111 %   fechaAlta: text("fecha_alta"),
112 %   vehiculoPatente: text("vehiculo_patente"),
113 %   vehiculoMarca: text("vehiculo_marca"),
114 %   vehiculoModelo: text("vehiculo_modelo"),
115 %   vehiculoColor: text("vehiculo_color"),
116 % });
117 %
118 % export const insertGuestSchema = createInsertSchema(guests).omit({ id: true, codigo: true, fechaAlta: true });
119 % export type InsertGuest = z.infer<typeof insertGuestSchema>;
120 % export type Guest = typeof guests.$inferSelect;
121 %
122 % // Bed Types (Tipos de Cama)
123 % export const bedTypes = pgTable("bed_types", {
124 %   id: serial("id").primaryKey(),
125 %   code: text("code").notNull().unique(),
126 %   name: text("name").notNull(),
127 %   description: text("description"),
128 %   isActive: boolean("is_active").notNull().default(true),
129 %   displayOrder: integer("display_order").notNull().default(0),
130 %   createdAt: timestamp("created_at").defaultNow(),
131 % });
132 %
133 % export const insertBedTypeSchema = createInsertSchema(bedTypes).omit({ id: true, createdAt: true });
134 % export type InsertBedType = z.infer<typeof insertBedTypeSchema>;
135 % export type BedType = typeof bedTypes.$inferSelect;
136 %
137 % // Reservations
138 % export type ReservationStatus = "tentative" | "pending" | "confirmed" | "checked_in" | "checked_out" | "cancelled";
139 % export type DiscountType = "none" | "percent" | "fixed";
140 % export type ReservationSource = "directo" | "web" | "booking" | "expedia" | "airbnb" | "despegar" | "hotelbeds" | "ag...";
141 %
142 % export const reservations = pgTable("reservations", {
143 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
144 %   reservationCode: text("reservation_code").notNull(),
145 %   guestId: varchar("guest_id").notNull(),
146 %   companyId: varchar("company_id"),
147 %   roomTypeId: varchar("room_type_id").notNull(),
148 %   roomId: varchar("room_id").notNull(),
149 %   ratePlanId: varchar("rate_plan_id"),
150 %   checkInDate: text("check_in_date").notNull(),
151 %   checkOutDate: text("check_out_date").notNull(),
152 %   nights: integer("nights").notNull().default(1),
153 %   baseRatePerNight: decimal("base_rate_per_night", { precision: 10, scale: 2 }),
154 %   discountType: text("discount_type").$type<DiscountType>().notNull().default("none"),
155 %   discountValue: decimal("discount_value", { precision: 10, scale: 2 }).default("0"),
156 %   finalRatePerNight: decimal("final_rate_per_night", { precision: 10, scale: 2 }),
157 %   totalRoomAmount: decimal("total_room_amount", { precision: 10, scale: 2 }),
158 %   status: text("status").$type<ReservationStatus>().notNull().default("pending"),
159 %   source: text("source").$type<ReservationSource>().notNull().default("directo"),
160 %   otaChannelId: varchar("ota_channel_id"),
161 %   externalReservationId: text("external_reservation_id"),
162 %   numberOfGuests: integer("number_of_guests").notNull().default(1),
163 %   bedTypeId: integer("bed_type_id"),
164 %   bedTypeNotes: text("bed_type_notes"),
165 %   earlyCheckIn: boolean("early_check_in").default(false),
166 %   earlyCheckInTime: text("early_check_in_time"),
167 %   earlyCheckInCharge: numeric("early_check_in_charge", { precision: 10, scale: 2 }),
168 %   lateCheckOut: boolean("late_check_out").default(false),
169 %   lateCheckOutTime: text("late_check_out_time"),
170 %   lateCheckOutCharge: numeric("late_check_out_charge", { precision: 10, scale: 2 }),
171 %   notes: text("notes"),
172 %   createdAt: text("created_at").notNull(),
173 %   lastModifiedBy: varchar("last_modified_by"),
174 % });
175 %
176 % export const insertReservationSchema = createInsertSchema(reservations).omit({ id: true });
177 % export type InsertReservation = z.infer<typeof insertReservationSchema>;
178 % export type Reservation = typeof reservations.$inferSelect;
179 %
180 % // Charges (Cargos/Folio)
181 % export type ChargeCategory = "room" | "restaurant" | "spa" | "minibar" | "otros" | "adjustment" | "payment";
182 %
183 % export const charges = pgTable("charges", {
```

shared/schema.ts (cont.)

```

184 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
185 %   reservationId: varchar("reservation_id").notNull(),
186 %   description: text("description").notNull(),
187 %   amount: decimal("amount", { precision: 10, scale: 2 }).notNull(),
188 %   date: text("date").notNull(),
189 %   category: text("category").$type<ChargeCategory>().notNull().default("otros"),
190 %   createdBy: varchar("created_by"),
191 % });
192 %
193 % export const insertChargeSchema = createInsertSchema(charges).omit({ id: true });
194 % export type InsertCharge = z.infer<typeof insertChargeSchema>;
195 % export type Charge = typeof charges.$inferSelect;
196 %
197 % // Payment methods
198 % export type PaymentMethod = "efectivo" | "tarjeta_debito" | "tarjeta_credito" | "transferencia" | "mercadopago" | "cu...
199 %
200 % // Payments table (pagos adelantados y durante estadía)
201 % export const payments = pgTable("payments", {
202 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
203 %   reservationId: varchar("reservation_id").notNull(),
204 %   amount: decimal("amount", { precision: 10, scale: 2 }).notNull(),
205 %   method: text("method").$type<PaymentMethod>().notNull(),
206 %   date: text("date").notNull(),
207 %   reference: text("reference"),
208 %   receivedBy: varchar("received_by"),
209 %   notes: text("notes"),
210 % });
211 %
212 % export const insertPaymentSchema = createInsertSchema(payments).omit({ id: true });
213 % export type InsertPayment = z.infer<typeof insertPaymentSchema>;
214 % export type Payment = typeof payments.$inferSelect;
215 %
216 % // Cancelled Reservation Log (registro de cancelaciones)
217 % export const cancelledReservationLogs = pgTable("cancelled_reservation_logs", {
218 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
219 %   reservationCode: text("reservation_code").notNull(),
220 %   guestName: text("guest_name").notNull(),
221 %   roomNumber: text("room_number").notNull(),
222 %   checkInDate: text("check_in_date").notNull(),
223 %   checkOutDate: text("check_out_date").notNull(),
224 %   cancellationDate: text("cancellation_date").notNull(),
225 %   cancelledBy: varchar("cancelled_by"),
226 %   reason: text("reason"),
227 % });
228 %
229 % export const insertCancelledReservationLogSchema = createInsertSchema(cancelledReservationLogs).omit({ id: true });
230 % export type InsertCancelledReservationLog = z.infer<typeof insertCancelledReservationLogSchema>;
231 % export type CancelledReservationLog = typeof cancelledReservationLogs.$inferSelect;
232 %
233 % // Housekeeping Tasks
234 % export type HousekeepingTaskStatus = "pending" | "in_progress" | "completed" | "inspected";
235 % export type HousekeepingTaskType = "checkout_clean" | "stayover_clean" | "deep_clean" | "inspection" | "turndown" | "...
236 % export type HousekeepingPriority = "low" | "normal" | "high" | "urgent";
237 %
238 % export const housekeepingTasks = pgTable("housekeeping_tasks", {
239 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
240 %   roomId: varchar("room_id").notNull(),
241 %   taskType: text("task_type").$type<HousekeepingTaskType>().notNull().default("checkout_clean"),
242 %   status: text("status").$type<HousekeepingTaskStatus>().notNull().default("pending"),
243 %   priority: text("priority").$type<HousekeepingPriority>().notNull().default("normal"),
244 %   assignedTo: varchar("assigned_to"),
245 %   notes: text("notes"),
246 %   scheduledDate: text("scheduled_date").notNull(),
247 %   startedAt: text("started_at"),
248 %   completedAt: text("completed_at"),
249 %   inspectedBy: varchar("inspected_by"),
250 %   inspectedAt: text("inspected_at"),
251 %   createdAt: text("created_at").notNull(),
252 % });
253 %
254 % export const insertHousekeepingTaskSchema = createInsertSchema(housekeepingTasks).omit({ id: true });
255 % export type InsertHousekeepingTask = z.infer<typeof insertHousekeepingTaskSchema>;
256 % export type HousekeepingTask = typeof housekeepingTasks.$inferSelect;
257 %
258 % export type HousekeepingTaskWithRoom = HousekeepingTask & {
259 %   room: Room & { roomType?: RoomType };
260 % };
261 %
262 % // Extended types for frontend with joined data
263 % export type RatePlanWithRoomType = RatePlan & {
264 %   roomType: RoomType;
265 % };
266 %
267 % export type GuestWithCompany = Guest & {
268 %   company?: Company;
269 % };
270 %
271 % export type ReservationWithDetails = Reservation & {
272 %   guest: Guest;
273 %   company?: Company;
274 %   room: Room & { roomType?: RoomType };
275 %   ratePlan?: RatePlan;
276 %   charges?: Charge[];

```

shared/schema.ts (cont.)

```

277 %   payments?: Payment[];
278 % };
279 %
280 % export type RoomWithType = Room & {
281 %   roomType: RoomType;
282 % };
283 %
284 % // Planning types
285 % export type PlanningCellStatus = "available" | "booked" | "checked_in" | "checkout_today" | "maintenance" | "cleaning...
286 %
287 % export type PlanningData = {
288 %   rooms: RoomWithType[];
289 %   days: string[];
290 %   occupancy: Record<string, PlanningCellStatus[]>;
291 %   reservations: Record<string, { id: string; guestName: string; checkIn: string; checkOut: string; status: Reservatio...
292 %   cellReservations: Record<string, Record<string, string>>; // roomId -> date -> reservationId
293 %   groupBlocks: Record<string, { id: string; groupName: string; groupCode: string; checkIn: string; checkOut: string }>;
294 %   cellGroupBlocks: Record<string, Record<string, string>>; // roomId -> date -> groupBlockId
295 % };
296 %
297 % // Users (for authentication)
298 % export type UserRole = "reception" | "housekeeping" | "management" | "director" | "restaurant" | "spa" | "security";
299 %
300 % export const users = pgTable("users", {
301 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
302 %   username: text("username").notNull().unique(),
303 %   password: text("password").notNull(),
304 %   firstName: text("first_name"),
305 %   lastName: text("last_name"),
306 %   role: text("role").$type<UserRole>().notNull().default("reception"),
307 % });
308 %
309 % export const insertUserSchema = createInsertSchema(users).omit({ id: true });
310 % export type InsertUser = z.infer<typeof insertUserSchema>;
311 % export type User = typeof users.$inferSelect;
312 %
313 % // OTA Channels (Canales de distribución)
314 % export type OTACHannelType = "booking" | "expedia" | "airbnb" | "despegar" | "hotelbeds" | "agoda" | "trivago" | "man...
315 % export type OTACHannelStatus = "active" | "inactive" | "pending" | "error";
316 %
317 % export const otaChannels = pgTable("ota_channels", {
318 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
319 %   name: text("name").notNull(),
320 %   channelType: text("channel_type").$type<OTACHannelType>().notNull(),
321 %   status: text("status").$type<OTACHannelStatus>().notNull().default("inactive"),
322 %   apiKey: text("api_key"),
323 %   apiSecret: text("api_secret"),
324 %   hotelCode: text("hotel_code"),
325 %   commissionPercent: decimal("commission_percent", { precision: 5, scale: 2 }).default("15.00"),
326 %   syncEnabled: text("sync_enabled").notNull().default("false"),
327 %   lastSyncAt: text("last_sync_at"),
328 %   createdAt: text("created_at").notNull(),
329 % });
330 %
331 % export const insertOTACHannelSchema = createInsertSchema(otaChannels).omit({ id: true });
332 % export type InsertOTACHannel = z.infer<typeof insertOTACHannelSchema>;
333 % export type OTACHannel = typeof otaChannels.$inferSelect;
334 %
335 % // OTA Reservation Sync Log (registro de sincronización)
336 % export type OTASyncStatus = "pending" | "synced" | "failed" | "cancelled";
337 %
338 % export const otaReservationLogs = pgTable("ota_reservation_logs", {
339 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
340 %   channelId: varchar("channel_id").notNull(),
341 %   externalReservationId: text("external_reservation_id").notNull(),
342 %   internalReservationId: varchar("internal_reservation_id"),
343 %   guestName: text("guest_name").notNull(),
344 %   checkInDate: text("check_in_date").notNull(),
345 %   checkOutDate: text("check_out_date").notNull(),
346 %   roomTypeName: text("room_type_name"),
347 %   totalAmount: decimal("total_amount", { precision: 10, scale: 2 }),
348 %   commission: decimal("commission", { precision: 10, scale: 2 }),
349 %   netAmount: decimal("net_amount", { precision: 10, scale: 2 }),
350 %   status: text("status").$type<OTASyncStatus>().notNull().default("pending"),
351 %   rawData: text("raw_data"),
352 %   syncedAt: text("synced_at"),
353 %   createdAt: text("created_at").notNull(),
354 % });
355 %
356 % export const insertOTAReservationLogSchema = createInsertSchema(otaReservationLogs).omit({ id: true });
357 % export type InsertOTAReservationLog = z.infer<typeof insertOTAReservationLogSchema>;
358 % export type OTAReservationLog = typeof otaReservationLogs.$inferSelect;
359 %
360 % // Extended OTA types
361 % export type OTACHannelWithStats = OTACHannel & {
362 %   totalReservations: number;
363 %   pendingSync: number;
364 %   totalRevenue: number;
365 %   totalCommission: number;
366 % };
367 %
368 % export type OTAReservationLogWithChannel = OTAReservationLog & {
369 %   channel: OTACHannel;

```

shared/schema.ts (cont.)

```
370 % };
371 %
372 % // Groups (Grupos de reservas)
373 % export type GroupStatus = "tentative" | "blocked" | "confirmed" | "inhouse" | "finished" | "cancelled";
374 %
375 % export const groups = pgTable("groups", {
376 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
377 %   groupCode: text("group_code").notNull().unique(),
378 %   name: text("name").notNull(),
379 %   contactName: text("contact_name"),
380 %   contactPhone: text("contact_phone"),
381 %   contactEmail: text("contact_email"),
382 %   eventDate: text("event_date"),
383 %   checkInDate: text("check_in_date").notNull(),
384 %   checkOutDate: text("check_out_date").notNull(),
385 %   status: text("status").$type<GroupStatus>().notNull().default("tentative"),
386 %   releaseDate: text("release_date"),
387 %   notes: text("notes"),
388 %   createdAt: text("created_at").notNull(),
389 %   createdBy: varchar("created_by"),
390 % });
391 %
392 % export const insertGroupSchema = createInsertSchema(groups).omit({ id: true });
393 % export type InsertGroup = z.infer<typeof insertGroupSchema>;
394 % export type Group = typeof groups.$inferSelect;
395 %
396 % // Group Room Blocks (Bloqueos de habitaciones para grupos)
397 % export const groupRoomBlocks = pgTable("group_room_blocks", {
398 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
399 %   groupId: varchar("group_id").notNull(),
400 %   roomTypeId: varchar("room_type_id").notNull(),
401 %   quantity: integer("quantity").notNull(),
402 %   ratePlanId: varchar("rate_plan_id"),
403 %   agreedRate: decimal("agreed_rate", { precision: 12, scale: 2 }),
404 %   // Block-specific dates (can differ from group master dates)
405 %   blockCheckInDate: text("block_check_in_date"),
406 %   blockCheckOutDate: text("block_check_out_date"),
407 % });
408 %
409 % export const insertGroupRoomBlockSchema = createInsertSchema(groupRoomBlocks).omit({ id: true });
410 % export type InsertGroupRoomBlock = z.infer<typeof insertGroupRoomBlockSchema>;
411 % export type GroupRoomBlock = typeof groupRoomBlocks.$inferSelect;
412 %
413 % // Group Reservation Links (Vinculación de reservas a grupos)
414 % export const groupReservationLinks = pgTable("group_reservation_links", {
415 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
416 %   groupId: varchar("group_id").notNull(),
417 %   reservationId: varchar("reservation_id").notNull(),
418 % });
419 %
420 % export const insertGroupReservationLinkSchema = createInsertSchema(groupReservationLinks).omit({ id: true });
421 % export type InsertGroupReservationLink = z.infer<typeof insertGroupReservationLinkSchema>;
422 % export type GroupReservationLink = typeof groupReservationLinks.$inferSelect;
423 %
424 % // Extended Group types
425 % export type GroupRoomBlockWithDetails = GroupRoomBlock & {
426 %   roomType: RoomType;
427 %   ratePlan?: RatePlan;
428 % };
429 %
430 % export type GroupWithDetails = Group & {
431 %   blocks: GroupRoomBlockWithDetails[];
432 %   reservations: ReservationWithDetails[];
433 %   totalRooms: number;
434 %   assignedRooms: number;
435 % };
436 %
437 % // Guest Reviews with Sentiment Analysis
438 % export type SentimentType = "positive" | "neutral" | "negative";
439 % export type ReviewCategory = "service" | "cleanliness" | "location" | "amenities" | "value" | "food" | "staff" | "gen...";
440 %
441 % export const guestReviews = pgTable("guest_reviews", {
442 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
443 %   reservationId: varchar("reservation_id"),
444 %   guestId: varchar("guest_id").notNull(),
445 %   roomId: varchar("room_id"),
446 %   reviewDate: text("review_date").notNull(),
447 %   source: text("source").notNull().default("direct"),
448 %   rating: integer("rating").notNull(),
449 %   title: text("title"),
450 %   content: text("content").notNull(),
451 %   // Sentiment Analysis Results
452 %   sentiment: text("sentiment").$type<SentimentType>(),
453 %   sentimentScore: decimal("sentiment_score", { precision: 5, scale: 4 }),
454 %   categories: text("categories").array(),
455 %   categoryScores: text("category_scores"),
456 %   keyPhrases: text("key_phrases").array(),
457 %   improvementSuggestions: text("improvement_suggestions").array(),
458 %   analyzedAt: text("analyzed_at"),
459 %   isPublished: text("is_published").default("false"),
460 %   staffResponse: text("staff_response"),
461 %   respondedAt: text("responded_at"),
462 %   respondedBy: text("responded_by"),
```

shared/schema.ts (cont.)

```

463 % });
464 %
465 % export const insertGuestReviewSchema = createInsertSchema(guestReviews).omit({ id: true });
466 % export type InsertGuestReview = z.infer<typeof insertGuestReviewSchema>;
467 % export type GuestReview = typeof guestReviews.$inferSelect;
468 %
469 % export type GuestReviewWithDetails = GuestReview & {
470 %   guest: Guest;
471 %   room?: Room;
472 %   reservation?: Reservation;
473 % };
474 %
475 % // ===== EVENTS MODULE =====
476 %
477 % // Event Rooms (Salones de Eventos)
478 % export type EventRoomStatus = "available" | "occupied" | "maintenance" | "reserved";
479 %
480 % export const eventRooms = pgTable("event_rooms", {
481 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
482 %   name: text("name").notNull(),
483 %   capacity: integer("capacity").notNull().default(50),
484 %   status: text("status").$type<EventRoomStatus>().notNull().default("available"),
485 %   description: text("description"),
486 %   amenities: text("amenities").array(),
487 %   isActive: text("is_active").default("true"),
488 % });
489 %
490 % export const insertEventRoomSchema = createInsertSchema(eventRooms).omit({ id: true });
491 % export type InsertEventRoom = z.infer<typeof insertEventRoomSchema>;
492 % export type EventRoom = typeof eventRooms.$inferSelect;
493 %
494 % // Event Types (Tipos de evento)
495 % export type EventType = "corporate" | "social" | "wedding" | "conference" | "seminar" | "cocktail" | "meeting" | "tab...
496 %
497 % // Event Status
498 % export type EventStatus = "tentative" | "confirmed" | "in_progress" | "completed" | "cancelled" | "invoiced";
499 %
500 % // Events (Eventos)
501 % export const events = pgTable("events", {
502 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
503 %   eventCode: text("event_code").notNull(),
504 %   name: text("name").notNull(),
505 %   eventRoomId: varchar("event_room_id").notNull(),
506 %   eventType: text("event_type").$type<EventType>().notNull().default("corporate"),
507 %   contactName: text("contact_name").notNull(),
508 %   contactPhone: text("contact_phone"),
509 %   contactEmail: text("contact_email"),
510 %   companyId: varchar("company_id"),
511 %   startDate: text("start_date").notNull(),
512 %   endDate: text("end_date").notNull(),
513 %   startTime: text("start_time"),
514 %   endTime: text("end_time"),
515 %   attendees: integer("attendees").default(10),
516 %   status: text("status").$type<EventStatus>().notNull().default("tentative"),
517 %   notes: text("notes"),
518 %   receiptType: text("receipt_type"),
519 %   closedAt: text("closed_at"),
520 %   totalAmount: decimal("total_amount", { precision: 10, scale: 2 }),
521 %   totalPaid: decimal("total_paid", { precision: 10, scale: 2 }),
522 %   createdAt: text("created_at").notNull(),
523 % });
524 %
525 % export const insertEventSchema = createInsertSchema(events).omit({ id: true });
526 % export type InsertEvent = z.infer<typeof insertEventSchema>;
527 % export type Event = typeof events.$inferSelect;
528 %
529 % // Event Charge Types (Tipos de cargo predefinidos)
530 % export const eventChargeTypes = pgTable("event_charge_types", {
531 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
532 %   code: text("code").notNull().unique(),
533 %   name: text("name").notNull(),
534 %   defaultPrice: decimal("default_price", { precision: 10, scale: 2 }),
535 %   isActive: text("is_active").default("true"),
536 % });
537 %
538 % export const insertEventChargeTypeSchema = createInsertSchema(eventChargeTypes).omit({ id: true });
539 % export type InsertEventChargeType = z.infer<typeof insertEventChargeTypeSchema>;
540 % export type EventChargeType = typeof eventChargeTypes.$inferSelect;
541 %
542 % // Event Charges (Cargos de eventos)
543 % export const eventCharges = pgTable("event_charges", {
544 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
545 %   eventId: varchar("event_id").notNull(),
546 %   chargeTypeId: varchar("charge_type_id"),
547 %   description: text("description").notNull(),
548 %   quantity: integer("quantity").notNull().default(1),
549 %   unitPrice: decimal("unit_price", { precision: 10, scale: 2 }).notNull(),
550 %   totalAmount: decimal("total_amount", { precision: 10, scale: 2 }).notNull(),
551 %   date: text("date").notNull(),
552 %   notes: text("notes"),
553 %   createdAt: text("created_at").notNull(),
554 % });
555 %

```

shared/schema.ts (cont.)

```

556 % export const insertEventChargeSchema = createInsertSchema(eventCharges).omit({ id: true });
557 % export type InsertEventCharge = z.infer<typeof insertEventChargeSchema>;
558 % export type EventCharge = typeof eventCharges.$inferSelect;
559 %
560 % // Event Payments (Pagos de eventos)
561 % export const eventPayments = pgTable("event_payments", {
562 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
563 %   eventId: varchar("event_id").notNull(),
564 %   amount: decimal("amount", { precision: 10, scale: 2 }).notNull(),
565 %   method: text("method").notNull(),
566 %   isAdvance: text("is_advance").default("false"),
567 %   reservationId: varchar("reservation_id"),
568 %   notes: text("notes"),
569 %   paidAt: text("paid_at"),
570 %   createdAt: text("created_at"),
571 % });
572 %
573 % export const insertEventPaymentSchema = createInsertSchema(eventPayments).omit({ id: true });
574 % export type InsertEventPayment = z.infer<typeof insertEventPaymentSchema>;
575 % export type EventPayment = typeof eventPayments.$inferSelect;
576 %
577 % // Event Tables (Mesas de eventos por mesa)
578 % export const eventTables = pgTable("event_tables", {
579 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
580 %   eventId: varchar("event_id").notNull(),
581 %   tableNumber: integer("table_number").notNull(),
582 %   label: text("label"),
583 %   seats: integer("seats"),
584 %   status: text("status").notNull().default("open"),
585 %   reservationId: varchar("reservation_id"),
586 %   receiptType: text("receipt_type"),
587 %   closedAt: text("closed_at"),
588 %   createdAt: text("created_at"),
589 % });
590 %
591 % export const insertEventTableSchema = createInsertSchema(eventTables).omit({ id: true });
592 % export type InsertEventTable = z.infer<typeof insertEventTableSchema>;
593 % export type EventTable = typeof eventTables.$inferSelect;
594 %
595 % // Event Table Charges
596 % export const eventTableCharges = pgTable("event_table_charges", {
597 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
598 %   eventTableId: varchar("event_table_id").notNull(),
599 %   description: text("description").notNull(),
600 %   quantity: integer("quantity").notNull().default(1),
601 %   unitPrice: decimal("unit_price", { precision: 10, scale: 2 }).notNull(),
602 %   total: decimal("total", { precision: 10, scale: 2 }).notNull(),
603 %   createdAt: text("created_at"),
604 % });
605 %
606 % export const insertEventTableChargeSchema = createInsertSchema(eventTableCharges).omit({ id: true });
607 % export type InsertEventTableCharge = z.infer<typeof insertEventTableChargeSchema>;
608 % export type EventTableCharge = typeof eventTableCharges.$inferSelect;
609 %
610 % // Event Table Payments
611 % export const eventTablePayments = pgTable("event_table_payments", {
612 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
613 %   eventTableId: varchar("event_table_id").notNull(),
614 %   amount: decimal("amount", { precision: 10, scale: 2 }).notNull(),
615 %   method: text("method").notNull(),
616 %   isAdvance: text("is_advance").default("false"),
617 %   reservationId: varchar("reservation_id"),
618 %   receiptType: text("receipt_type"),
619 %   paidAt: text("paid_at"),
620 %   createdAt: text("created_at"),
621 % });
622 %
623 % export const insertEventTablePaymentSchema = createInsertSchema(eventTablePayments).omit({ id: true });
624 % export type InsertEventTablePayment = z.infer<typeof insertEventTablePaymentSchema>;
625 % export type EventTablePayment = typeof eventTablePayments.$inferSelect;
626 %
627 % // Extended Event types
628 % export type EventWithDetails = Event & {
629 %   eventRoom: EventRoom;
630 %   company?: Company;
631 %   charges?: EventChargeWithType[];
632 %   payments?: EventPayment[];
633 % };
634 %
635 % export type EventChargeWithType = EventCharge & {
636 %   chargeType?: EventChargeType;
637 % };
638 %
639 % export type EventTableWithDetails = EventTable & {
640 %   charges: EventTableCharge[];
641 %   payments: EventTablePayment[];
642 % };
643 %
644 % // Event Planning types
645 % export type EventPlanningCellStatus = "available" | "event" | "maintenance" | "reserved";
646 %
647 % export type EventPlanningData = {
648 %   rooms: EventRoom[];

```


shared/schema.ts (cont.)

```

649 %   days: string[];
650 %   occupancy: Record<string, EventPlanningCellStatus[]>;
651 %   events: Record<string, { id: string; name: string; contactName: string; startDate: string; endDate: string; status:...
652 %   cellEvents: Record<string, Record<string, string>>; // roomId -> date -> eventId
653 % };
654 %
655 % // Conversations and Messages for Chat (AI Integrations)
656 % export const conversations = pgTable("conversations", {
657 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
658 %   title: text("title").notNull(),
659 %   createdAt: text("created_at"),
660 % });
661 %
662 % export const messages = pgTable("messages", {
663 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
664 %   conversationId: varchar("conversation_id").notNull(),
665 %   role: text("role").notNull(),
666 %   content: text("content").notNull(),
667 %   createdAt: text("created_at"),
668 % });
669 %
670 % // ===== RESTAURANT MODULE =====
671 %
672 % // Restaurant Areas (Salones)
673 % export type RestaurantAreaType = "indoor" | "outdoor" | "terrace" | "bar" | "private";
674 %
675 % export const restaurantAreas = pgTable("restaurant_areas", {
676 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
677 %   name: text("name").notNull(),
678 %   areaType: text("area_type").$type<RestaurantAreaType>().notNull().default("indoor"),
679 %   capacity: integer("capacity").notNull().default(20),
680 %   hasTables: text("has_tables").default("true"),
681 %   isActive: text("is_active").default("true"),
682 %   notes: text("notes"),
683 % });
684 %
685 % export const insertRestaurantAreaSchema = createInsertSchema(restaurantAreas).omit({ id: true });
686 % export type InsertRestaurantArea = z.infer<typeof insertRestaurantAreaSchema>;
687 % export type RestaurantArea = typeof restaurantAreas.$inferSelect;
688 %
689 % // Restaurant Tables
690 % export type TableStatus = "available" | "occupied" | "reserved" | "cleaning" | "blocked";
691 % export type TableShape = "square" | "round" | "rectangular";
692 %
693 % export const restaurantTables = pgTable("restaurant_tables", {
694 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
695 %   tableNumber: text("table_number").notNull(),
696 %   areaId: varchar("area_id").notNull(),
697 %   capacity: integer("capacity").notNull().default(4),
698 %   shape: text("shape").$type<TableShape>().default("square"),
699 %   status: text("status").$type<TableStatus>().notNull().default("available"),
700 %   positionX: integer("position_x").default(0),
701 %   positionY: integer("position_y").default(0),
702 %   hasWindow: text("has_window").default("false"),
703 %   isActive: text("is_active").default("true"),
704 % });
705 %
706 % export const insertRestaurantTableSchema = createInsertSchema(restaurantTables).omit({ id: true });
707 % export type InsertRestaurantTable = z.infer<typeof insertRestaurantTableSchema>;
708 % export type RestaurantTable = typeof restaurantTables.$inferSelect;
709 %
710 % export type RestaurantTableWithArea = RestaurantTable & {
711 %   area: RestaurantArea;
712 % };
713 %
714 % // Table Reservations
715 % export type TableReservationStatus = "pending" | "confirmed" | "seated" | "completed" | "cancelled" | "no_show";
716 %
717 % export const tableReservations = pgTable("table_reservations", {
718 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
719 %   tableId: varchar("table_id").notNull(),
720 %   guestName: text("guest_name").notNull(),
721 %   guestPhone: text("guest_phone"),
722 %   guestEmail: text("guest_email"),
723 %   partySize: integer("party_size").notNull().default(2),
724 %   reservationDate: text("reservation_date").notNull(),
725 %   reservationTime: text("reservation_time").notNull(),
726 %   status: text("status").$type<TableReservationStatus>().notNull().default("pending"),
727 %   notes: text("notes"),
728 %   createdAt: text("created_at").notNull(),
729 % });
730 %
731 % export const insertTableReservationSchema = createInsertSchema(tableReservations).omit({ id: true });
732 % export type InsertTableReservation = z.infer<typeof insertTableReservationSchema>;
733 % export type TableReservation = typeof tableReservations.$inferSelect;
734 %
735 % export type TableReservationWithTable = TableReservation & {
736 %   table: RestaurantTable;
737 % };
738 %
739 % // Menu Categories
740 % export const menuCategories = pgTable("menu_categories", {
741 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),

```

shared/schema.ts (cont.)

```

742 %   name: text("name").notNull(),
743 %   description: text("description"),
744 %   displayOrder: integer("display_order").default(0),
745 %   isActive: text("is_active").default("true"),
746 % });
747 %
748 % export const insertMenuCategorySchema = createInsertSchema(menuCategories).omit({ id: true });
749 % export type InsertMenuCategory = z.infer<typeof insertMenuCategorySchema>;
750 % export type MenuCategory = typeof menuCategories.$inferSelect;
751 %
752 % // Menu Items
753 % export const menuItems = pgTable("menu_items", {
754 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
755 %   categoryId: varchar("category_id").notNull(),
756 %   name: text("name").notNull(),
757 %   description: text("description"),
758 %   price: decimal("price", { precision: 10, scale: 2 }).notNull(),
759 %   preparationTime: integer("preparation_time"),
760 %   isAvailable: text("is_available").default("true"),
761 %   isActive: text("is_active").default("true"),
762 %   allergens: text("allergens").array(),
763 %   displayOrder: integer("display_order").default(0),
764 % });
765 %
766 % export const insertMenuItemSchema = createInsertSchema(menuItems).omit({ id: true });
767 % export type InsertMenuItem = z.infer<typeof insertMenuItemSchema>;
768 % export type MenuItem = typeof menuItems.$inferSelect;
769 %
770 % export type MenuItemWithCategory = MenuItem & {
771 %   category: MenuCategory;
772 % };
773 %
774 % // Restaurant Orders
775 % export type OrderStatus = "open" | "in_progress" | "served" | "closed" | "cancelled";
776 % export type OrderType = "dine_in" | "room_service" | "takeaway";
777 %
778 % export type ReceiptType = "ticket" | "factura_a" | "factura_b" | "factura_c" | "nota_credito";
779 % export type RestaurantPaymentMethod = "efectivo" | "tarjeta_debito" | "tarjeta_credito" | "transferencia" | "cuenta_h...";
780 %
781 % export const restaurantOrders = pgTable("restaurant_orders", {
782 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
783 %   orderNumber: text("order_number").notNull(),
784 %   tableId: varchar("table_id"),
785 %   areaId: varchar("area_id"),
786 %   reservationId: varchar("reservation_id"),
787 %   guestId: varchar("guest_id"),
788 %   orderType: text("order_type").$type<OrderType>().notNull().default("dine_in"),
789 %   status: text("status").$type<OrderStatus>().notNull().default("open"),
790 %   covers: integer("covers").default(1),
791 %   waiterName: text("waiter_name"),
792 %   orderLabel: text("order_label"),
793 %   activeCourse: integer("active_course").default(1),
794 %   subtotal: decimal("subtotal", { precision: 10, scale: 2 }).default("0"),
795 %   tax: decimal("tax", { precision: 10, scale: 2 }).default("0"),
796 %   total: decimal("total", { precision: 10, scale: 2 }).default("0"),
797 %   notes: text("notes"),
798 %   openedAt: text("opened_at").notNull(),
799 %   closedAt: text("closed_at"),
800 %   chargedToRoom: text("charged_to_room").default("false"),
801 %   roomNumber: text("room_number"),
802 %   receiptType: text("receipt_type").$type<ReceiptType>(),
803 %   paymentMethod: text("payment_method").$type<RestaurantPaymentMethod>(),
804 % });
805 %
806 % export const insertRestaurantOrderSchema = createInsertSchema(restaurantOrders).omit({ id: true });
807 % export type InsertRestaurantOrder = z.infer<typeof insertRestaurantOrderSchema>;
808 % export type RestaurantOrder = typeof restaurantOrders.$inferSelect;
809 %
810 % // Order Items
811 % export type OrderItemStatus = "pending" | "preparing" | "ready" | "served" | "cancelled" | "waiting_course";
812 %
813 % export const orderItems = pgTable("order_items", {
814 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
815 %   orderId: varchar("order_id").notNull(),
816 %   menuItemId: varchar("menu_item_id").notNull(),
817 %   quantity: integer("quantity").notNull().default(1),
818 %   unitPrice: decimal("unit_price", { precision: 10, scale: 2 }).notNull(),
819 %   subtotal: decimal("subtotal", { precision: 10, scale: 2 }).notNull(),
820 %   status: text("status").$type<OrderItemStatus>().notNull().default("pending"),
821 %   course: integer("course").default(1),
822 %   notes: text("notes"),
823 %   sentAt: text("sent_at"),
824 % });
825 %
826 % export const insertOrderItemSchema = createInsertSchema(orderItems).omit({ id: true });
827 % export type InsertOrderItem = z.infer<typeof insertOrderItemSchema>;
828 % export type OrderItem = typeof orderItems.$inferSelect;
829 %
830 % export type OrderItemWithMenuItem = OrderItem & {
831 %   menuItem: MenuItem;
832 % };
833 %
834 % export type RestaurantOrderWithDetails = RestaurantOrder & {

```

shared/schema.ts (cont.)

```

835 %   table?: RestaurantTableWithArea;
836 %   area?: RestaurantArea;
837 %   guest?: Guest;
838 %   items: OrderItemWithMenuItem[];
839 % };
840 %
841 % // Order Splits (bill splitting)
842 % export const orderSplits = pgTable("order_splits", {
843 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
844 %   orderId: varchar("order_id").notNull(),
845 %   splitNumber: integer("split_number").notNull(),
846 %   amount: decimal("amount", { precision: 10, scale: 2 }).notNull(),
847 %   method: text("method").$type<RestaurantPaymentMethod>(),
848 %   receiptType: text("receipt_type").$type<ReceiptType>(),
849 %   isPaid: text("is_paid").default("false"),
850 %   paidAt: text("paid_at"),
851 %   createdAt: text("created_at"),
852 % });
853 %
854 % export const insertOrderSplitSchema = createInsertSchema(orderSplits).omit({ id: true });
855 % export type InsertOrderSplit = z.infer<typeof insertOrderSplitSchema>;
856 % export type OrderSplit = typeof orderSplits.$inferSelect;
857 %
858 % // Restaurant Time Slots (configurable reservation turns)
859 % export const restaurantTimeSlots = pgTable("restaurant_time_slots", {
860 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
861 %   time: text("time").notNull(),
862 %   label: text("label"),
863 %   isActive: text("is_active").default("true"),
864 %   displayOrder: integer("display_order").default(0),
865 % });
866 %
867 % export const insertRestaurantTimeSlotSchema = createInsertSchema(restaurantTimeSlots).omit({ id: true });
868 % export type InsertRestaurantTimeSlot = z.infer<typeof insertRestaurantTimeSlotSchema>;
869 % export type RestaurantTimeSlot = typeof restaurantTimeSlots.$inferSelect;
870 %
871 % // Recipes (ingredients per dish)
872 % export const recipes = pgTable("recipes", {
873 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
874 %   menuItemId: varchar("menu_item_id").notNull(),
875 %   notes: text("notes"),
876 % });
877 %
878 % export const insertRecipeSchema = createInsertSchema(recipes).omit({ id: true });
879 % export type InsertRecipe = z.infer<typeof insertRecipeSchema>;
880 % export type Recipe = typeof recipes.$inferSelect;
881 %
882 % // Recipe Ingredients
883 % export const recipeIngredients = pgTable("recipe_ingredients", {
884 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
885 %   recipeId: varchar("recipe_id").notNull(),
886 %   inventoryItemId: varchar("inventory_item_id"),
887 %   ingredientName: text("ingredient_name").notNull(),
888 %   quantity: decimal("quantity", { precision: 10, scale: 3 }).notNull(),
889 %   unit: text("unit").notNull(),
890 %   unitCost: decimal("unit_cost", { precision: 10, scale: 2 }).default("0"),
891 % });
892 %
893 % export const insertRecipeIngredientSchema = createInsertSchema(recipeIngredients).omit({ id: true });
894 % export type InsertRecipeIngredient = z.infer<typeof insertRecipeIngredientSchema>;
895 % export type RecipeIngredient = typeof recipeIngredients.$inferSelect;
896 %
897 % export type RecipeWithIngredients = Recipe & {
898 %   menuItem?: MenuItem;
899 %   ingredients: RecipeIngredient[];
900 % };
901 %
902 % // ===== INVENTORY MODULE =====
903 %
904 % // Item Categories (for inventory)
905 % export type InventoryArea = "general" | "spa" | "restaurant" | "housekeeping" | "maintenance" | "admin";
906 %
907 % export const itemCategories = pgTable("item_categories", {
908 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
909 %   name: text("name").notNull(),
910 %   description: text("description"),
911 %   parentId: varchar("parent_id"),
912 %   area: text("area").$type<InventoryArea>().notNull().default("general"),
913 %   isActive: text("is_active").default("true"),
914 % });
915 %
916 % export const insertItemCategorySchema = createInsertSchema(itemCategories).omit({ id: true });
917 % export type InsertItemCategory = z.infer<typeof insertItemCategorySchema>;
918 % export type ItemCategory = typeof itemCategories.$inferSelect;
919 %
920 % // Suppliers (Provedores)
921 % export const suppliers = pgTable("suppliers", {
922 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
923 %   name: text("name").notNull(),
924 %   contactName: text("contact_name"),
925 %   phone: text("phone"),
926 %   email: text("email"),
927 %   address: text("address"),

```

shared/schema.ts (cont.)

```

928 %   cuit: text("cuit"),
929 %   paymentTermDays: integer("payment_term_days").default(30),
930 %   notes: text("notes"),
931 %   isActive: text("is_active").default("true"),
932 % });
933 %
934 % export const insertSupplierSchema = createInsertSchema(suppliers).omit({ id: true });
935 % export type InsertSupplier = z.infer<typeof insertSupplierSchema>;
936 % export type Supplier = typeof suppliers.$inferSelect;
937 %
938 % // Inventory Items (Articulos)
939 % export type UnitType = "unidad" | "kg" | "g" | "litro" | "ml" | "caja" | "paquete" | "docena";
940 %
941 % export const inventoryItems = pgTable("inventory_items", {
942 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
943 %   sku: text("sku").unique(),
944 %   name: text("name").notNull(),
945 %   description: text("description"),
946 %   categoryId: varchar("category_id"),
947 %   supplierId: varchar("supplier_id"),
948 %   unit: text("unit").$type<UnitType>().notNull().default("unidad"),
949 %   costPrice: decimal("cost_price", { precision: 10, scale: 2 }).default("0"),
950 %   minStock: integer("min_stock").default(0),
951 %   maxStock: integer("max_stock"),
952 %   currentStock: integer("current_stock").default(0),
953 %   location: text("location"),
954 %   isActive: text("is_active").default("true"),
955 % });
956 %
957 % export const insertInventoryItemSchema = createInsertSchema(inventoryItems).omit({ id: true });
958 % export type InsertInventoryItem = z.infer<typeof insertInventoryItemSchema>;
959 % export type InventoryItem = typeof inventoryItems.$inferSelect;
960 %
961 % export type InventoryItemWithDetails = InventoryItem & {
962 %   category?: ItemCategory;
963 %   supplier?: Supplier;
964 % };
965 %
966 % // Stock Movements (Movimientos de Stock)
967 % export type MovementType = "entrada" | "salida" | "ajuste" | "transferencia" | "consumo";
968 %
969 % export const stockMovements = pgTable("stock_movements", {
970 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
971 %   itemId: varchar("item_id").notNull(),
972 %   movementType: text("movement_type").$type<MovementType>().notNull(),
973 %   quantity: integer("quantity").notNull(),
974 %   previousStock: integer("previous_stock").notNull(),
975 %   newStock: integer("new_stock").notNull(),
976 %   unitCost: decimal("unit_cost", { precision: 10, scale: 2 }),
977 %   reference: text("reference"),
978 %   notes: text("notes"),
979 %   createdAt: text("created_at").notNull(),
980 %   createdBy: text("created_by"),
981 % });
982 %
983 % export const insertStockMovementSchema = createInsertSchema(stockMovements).omit({ id: true });
984 % export type InsertStockMovement = z.infer<typeof insertStockMovementSchema>;
985 % export type StockMovement = typeof stockMovements.$inferSelect;
986 %
987 % export type StockMovementWithItem = StockMovement & {
988 %   item: InventoryItem;
989 % };
990 %
991 % // Purchase Orders (Ordenes de Compra)
992 % export type PurchaseOrderStatus = "draft" | "sent" | "partial" | "received" | "cancelled";
993 %
994 % export const purchaseOrders = pgTable("purchase_orders", {
995 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
996 %   orderNumber: text("order_number").notNull(),
997 %   supplierId: varchar("supplier_id").notNull(),
998 %   status: text("status").$type<PurchaseOrderStatus>().notNull().default("draft"),
999 %   subtotal: decimal("subtotal", { precision: 12, scale: 2 }).default("0"),
1000 %   tax: decimal("tax", { precision: 12, scale: 2 }).default("0"),
1001 %   total: decimal("total", { precision: 12, scale: 2 }).default("0"),
1002 %   notes: text("notes"),
1003 %   createdAt: text("created_at").notNull(),
1004 %   expectedDate: text("expected_date"),
1005 %   receivedAt: text("received_at"),
1006 % });
1007 %
1008 % export const insertPurchaseOrderSchema = createInsertSchema(purchaseOrders).omit({ id: true });
1009 % export type InsertPurchaseOrder = z.infer<typeof insertPurchaseOrderSchema>;
1010 % export type PurchaseOrder = typeof purchaseOrders.$inferSelect;
1011 %
1012 % // Purchase Order Items
1013 % export const purchaseOrderItems = pgTable("purchase_order_items", {
1014 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1015 %   purchaseOrderId: varchar("purchase_order_id").notNull(),
1016 %   itemId: varchar("item_id").notNull(),
1017 %   quantity: integer("quantity").notNull(),
1018 %   unitCost: decimal("unit_cost", { precision: 10, scale: 2 }).notNull(),
1019 %   subtotal: decimal("subtotal", { precision: 10, scale: 2 }).notNull(),
1020 %   receivedQuantity: integer("received_quantity").default(0),

```

shared/schema.ts (cont.)

```

1021 % });
1022 %
1023 % export const insertPurchaseOrderItemSchema = createInsertSchema(purchaseOrderItems).omit({ id: true });
1024 % export type InsertPurchaseOrderItem = z.infer<typeof insertPurchaseOrderItemSchema>;
1025 % export type PurchaseOrderItem = typeof purchaseOrderItems.$inferSelect;
1026 %
1027 % export type PurchaseOrderWithDetails = PurchaseOrder & {
1028 %   supplier: Supplier;
1029 %   items: (PurchaseOrderItem & { item: InventoryItem })[];
1030 % };
1031 %
1032 % // =====
1033 % // SPA MODULE
1034 % // =====
1035 %
1036 % // SPA Cabins (Gabinetes)
1037 % export const spaCabins = pgTable("spa_cabins", {
1038 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1039 %   name: text("name").notNull(),
1040 %   description: text("description"),
1041 %   isActive: text("is_active").default("true"),
1042 % });
1043 %
1044 % export const insertSpaCabinSchema = createInsertSchema(spaCabins).omit({ id: true });
1045 % export type InsertSpaCabin = z.infer<typeof insertSpaCabinSchema>;
1046 % export type SpaCabin = typeof spaCabins.$inferSelect;
1047 %
1048 % // SPA Treatment Categories
1049 % export const spaTreatmentCategories = pgTable("spa_treatment_categories", {
1050 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1051 %   name: text("name").notNull(),
1052 %   description: text("description"),
1053 %   sortOrder: integer("sort_order").default(0),
1054 % });
1055 %
1056 % export const insertSpaTreatmentCategorySchema = createInsertSchema(spaTreatmentCategories).omit({ id: true });
1057 % export type InsertSpaTreatmentCategory = z.infer<typeof insertSpaTreatmentCategorySchema>;
1058 % export type SpaTreatmentCategory = typeof spaTreatmentCategories.$inferSelect;
1059 %
1060 % // SPA Treatments
1061 % export const spaTreatments = pgTable("spa_treatments", {
1062 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1063 %   categoryId: varchar("category_id"),
1064 %   name: text("name").notNull(),
1065 %   description: text("description"),
1066 %   durationMinutes: integer("duration_minutes").notNull().default(60),
1067 %   price: decimal("price", { precision: 10, scale: 2 }).notNull().default("0"),
1068 %   isActive: text("is_active").default("true"),
1069 % });
1070 %
1071 % export const insertSpaTreatmentSchema = createInsertSchema(spaTreatments).omit({ id: true });
1072 % export type InsertSpaTreatment = z.infer<typeof insertSpaTreatmentSchema>;
1073 % export type SpaTreatment = typeof spaTreatments.$inferSelect;
1074 %
1075 % // SPA Appointments (Turnos)
1076 % export type SpaAppointmentStatus = "pending" | "confirmed" | "in_progress" | "completed" | "cancelled" | "no_show";
1077 %
1078 % export const spaAppointments = pgTable("spa_appointments", {
1079 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1080 %   cabinId: varchar("cabin_id").notNull(),
1081 %   treatmentId: varchar("treatment_id").notNull(),
1082 %   guestName: text("guest_name").notNull(),
1083 %   guestLastName: text("guest_last_name"),
1084 %   guestPhone: text("guest_phone"),
1085 %   guestEmail: text("guest_email"),
1086 %   reservationId: varchar("reservation_id"),
1087 %   appointmentDate: text("appointment_date").notNull(),
1088 %   startTime: text("start_time").notNull(),
1089 %   endTime: text("end_time").notNull(),
1090 %   status: text("status").$type<SpaAppointmentStatus>().notNull().default("pending"),
1091 %   notes: text("notes"),
1092 %   createdAt: text("created_at").notNull(),
1093 % });
1094 %
1095 % export const insertSpaAppointmentSchema = createInsertSchema(spaAppointments).omit({ id: true });
1096 % export type InsertSpaAppointment = z.infer<typeof insertSpaAppointmentSchema>;
1097 % export type SpaAppointment = typeof spaAppointments.$inferSelect;
1098 %
1099 % export type SpaAppointmentWithDetails = SpaAppointment & {
1100 %   cabin: SpaCabin;
1101 %   treatment: SpaTreatment;
1102 % };
1103 %
1104 % // SPA Account (Cuenta SPA - similar to restaurant orders)
1105 % export type SpaAccountStatus = "open" | "closed" | "cancelled";
1106 %
1107 % export const spaAccounts = pgTable("spa_accounts", {
1108 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1109 %   appointmentId: varchar("appointment_id").notNull(),
1110 %   guestName: text("guest_name").notNull(),
1111 %   reservationId: varchar("reservation_id"),
1112 %   status: text("status").$type<SpaAccountStatus>().notNull().default("open"),
1113 %   subtotal: decimal("subtotal", { precision: 10, scale: 2 }).default("0"),

```

shared/schema.ts (cont.)

```

1114 %   total: decimal("total", { precision: 10, scale: 2 }).default("0"),
1115 %   totalPaid: decimal("total_paid", { precision: 10, scale: 2 }).default("0"),
1116 %   receiptType: text("receipt_type"),
1117 %   notes: text("notes"),
1118 %   openedAt: text("opened_at").notNull(),
1119 %   closedAt: text("closed_at"),
1120 %   closedBy: text("closed_by"),
1121 %   chargedTo: text("charged_to"),
1122 % });
1123 %
1124 % export const insertSpaAccountSchema = createInsertSchema(spaAccounts).omit({ id: true });
1125 % export type InsertSpaAccount = z.infer<typeof insertSpaAccountSchema>;
1126 % export type SpaAccount = typeof spaAccounts.$inferSelect;
1127 %
1128 % // SPA Account Items (Cargos en cuenta SPA)
1129 % export const spaAccountItems = pgTable("spa_account_items", {
1130 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1131 %   accountId: varchar("account_id").notNull(),
1132 %   description: text("description").notNull(),
1133 %   quantity: integer("quantity").notNull().default(1),
1134 %   unitPrice: decimal("unit_price", { precision: 10, scale: 2 }).notNull(),
1135 %   subtotal: decimal("subtotal", { precision: 10, scale: 2 }).notNull(),
1136 %   itemType: text("item_type").notNull().default("treatment"),
1137 %   notes: text("notes"),
1138 %   createdAt: text("created_at").notNull(),
1139 % });
1140 %
1141 % export const insertSpaAccountItemSchema = createInsertSchema(spaAccountItems).omit({ id: true });
1142 % export type InsertSpaAccountItem = z.infer<typeof insertSpaAccountItemSchema>;
1143 % export type SpaAccountItem = typeof spaAccountItems.$inferSelect;
1144 %
1145 % export type SpaPaymentMethod = "cash" | "debit_card" | "credit_card" | "transfer" | "mercadopago" | "room_charge";
1146 %
1147 % export const spaPayments = pgTable("spa_payments", {
1148 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1149 %   accountId: varchar("account_id").notNull(),
1150 %   amount: decimal("amount", { precision: 10, scale: 2 }).notNull(),
1151 %   method: text("method").$type<SpaPaymentMethod>().notNull(),
1152 %   isAdvance: text("is_advance").default("false"),
1153 %   appointmentId: varchar("appointment_id"),
1154 %   reservationId: varchar("reservation_id"),
1155 %   notes: text("notes"),
1156 %   createdAt: text("created_at").notNull(),
1157 % });
1158 %
1159 % export const insertSpaPaymentSchema = createInsertSchema(spaPayments).omit({ id: true });
1160 % export type InsertSpaPayment = z.infer<typeof insertSpaPaymentSchema>;
1161 % export type SpaPayment = typeof spaPayments.$inferSelect;
1162 %
1163 % export type SpaAccountWithItems = SpaAccount & {
1164 %   items: SpaAccountItem[];
1165 %   payments: SpaPayment[];
1166 %   appointment?: SpaAppointmentWithDetails;
1167 % };
1168 %
1169 % // ===== MAINTENANCE MODULE =====
1170 %
1171 % // Maintenance Staff (Personal de Mantenimiento)
1172 % export const maintenanceStaff = pgTable("maintenance_staff", {
1173 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1174 %   name: text("name").notNull(),
1175 %   phone: text("phone"),
1176 %   email: text("email"),
1177 %   specialty: text("specialty"),
1178 %   isActive: text("is_active").default("true"),
1179 % });
1180 %
1181 % export const insertMaintenanceStaffSchema = createInsertSchema(maintenanceStaff).omit({ id: true });
1182 % export type InsertMaintenanceStaff = z.infer<typeof insertMaintenanceStaffSchema>;
1183 % export type MaintenanceStaff = typeof maintenanceStaff.$inferSelect;
1184 %
1185 % // Work Order types
1186 % export type WorkOrderPriority = "low" | "medium" | "high" | "urgent";
1187 % export type WorkOrderStatus = "pending" | "assigned" | "in_progress" | "completed" | "cancelled";
1188 % export type WorkOrderCategory = "plumbing" | "electrical" | "hvac" | "furniture" | "cleaning" | "appliances" | "struc...
1189 %
1190 % // Work Orders (Ordenes de Trabajo)
1191 % export const workOrders = pgTable("work_orders", {
1192 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1193 %   orderCode: text("order_code").notNull(),
1194 %   title: text("title").notNull(),
1195 %   description: text("description"),
1196 %   roomId: varchar("room_id"),
1197 %   location: text("location"),
1198 %   category: text("category").$type<WorkOrderCategory>().notNull().default("general"),
1199 %   priority: text("priority").$type<WorkOrderPriority>().notNull().default("medium"),
1200 %   status: text("status").$type<WorkOrderStatus>().notNull().default("pending"),
1201 %   assignedToId: varchar("assigned_to_id"),
1202 %   reportedBy: text("reported_by"),
1203 %   reportedAt: text("reported_at").notNull(),
1204 %   scheduledDate: text("scheduled_date"),
1205 %   completedAt: text("completed_at"),
1206 %   completedBy: text("completed_by"),

```

shared/schema.ts (cont.)

```
1207 %     estimatedCost: decimal("estimated_cost", { precision: 10, scale: 2 })),
1208 %     actualCost: decimal("actual_cost", { precision: 10, scale: 2 })),
1209 %     notes: text("notes"),
1210 %   });
1211 %
1212 % export const insertWorkOrderSchema = createInsertSchema(workOrders).omit({ id: true });
1213 % export type InsertWorkOrder = z.infer<typeof insertWorkOrderSchema>;
1214 % export type WorkOrder = typeof workOrders.$inferSelect;
1215 %
1216 % export type WorkOrderWithDetails = WorkOrder & {
1217 %   room?: Room;
1218 %   assignedTo?: MaintenanceStaff;
1219 % };
1220 %
1221 % // ===== ADMINISTRATION MODULE =====
1222 %
1223 % // System User Roles (extends existing UserRole with admin roles)
1224 % export type SystemUserRole = "admin" | "manager" | "reception" | "housekeeping" | "maintenance" | "restaurant" | "spa...";
1225 %
1226 % // System Users (Usuarios del Sistema)
1227 % export const systemUsers = pgTable("system_users", {
1228 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1229 %   username: text("username").notNull().unique(),
1230 %   email: text("email").notNull(),
1231 %   fullName: text("full_name").notNull(),
1232 %   role: text("role").$type<SystemUserRole>().notNull().default("reception"),
1233 %   department: text("department"),
1234 %   phone: text("phone"),
1235 %   isActive: text("is_active").default("true"),
1236 %   lastLogin: text("last_login"),
1237 %   createdAt: text("created_at").notNull(),
1238 % });
1239 %
1240 % export const insertSystemUserSchema = createInsertSchema(systemUsers).omit({ id: true });
1241 % export type InsertSystemUser = z.infer<typeof insertSystemUserSchema>;
1242 % export type SystemUser = typeof systemUsers.$inferSelect;
1243 %
1244 % // System Settings (Configuracion del Sistema)
1245 % export const systemSettings = pgTable("system_settings", {
1246 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1247 %   key: text("key").notNull().unique(),
1248 %   value: text("value").notNull(),
1249 %   category: text("category").notNull().default("general"),
1250 %   description: text("description"),
1251 %   updatedAt: text("updated_at").notNull(),
1252 %   updatedBy: text("updated_by"),
1253 % });
1254 %
1255 % export const insertSystemSettingSchema = createInsertSchema(systemSettings).omit({ id: true });
1256 % export type InsertSystemSetting = z.infer<typeof insertSystemSettingSchema>;
1257 % export type SystemSetting = typeof systemSettings.$inferSelect;
1258 %
1259 % // Audit Logs (Registro de Auditoria)
1260 % export type AuditAction = "create" | "update" | "delete" | "login" | "logout" | "view" | "export";
1261 %
1262 % export const auditLogs = pgTable("audit_logs", {
1263 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1264 %   userId: varchar("user_id"),
1265 %   userName: text("user_name"),
1266 %   action: text("action").$type<AuditAction>().notNull(),
1267 %   module: text("module").notNull(),
1268 %   entityType: text("entity_type"),
1269 %   entityId: varchar("entity_id"),
1270 %   description: text("description").notNull(),
1271 %   details: text("details"),
1272 %   ipAddress: text("ip_address"),
1273 %   timestamp: text("timestamp").notNull(),
1274 % });
1275 %
1276 % export const insertAuditLogSchema = createInsertSchema(auditLogs).omit({ id: true });
1277 % export type InsertAuditLog = z.infer<typeof insertAuditLogSchema>;
1278 % export type AuditLog = typeof auditLogs.$inferSelect;
1279 %
1280 % // Packages (Paquetes Turisticos)
1281 % export type PackageStatus = "active" | "inactive" | "expired";
1282 %
1283 % export const packages = pgTable("packages", {
1284 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1285 %   code: text("code").notNull().unique(),
1286 %   name: text("name").notNull(),
1287 %   description: text("description"),
1288 %   roomTypeId: varchar("room_type_id"),
1289 %   nights: integer("nights").notNull().default(1),
1290 %   basePrice: decimal("base_price", { precision: 12, scale: 2 }).notNull(),
1291 %   discountPercent: decimal("discount_percent", { precision: 5, scale: 2 }),
1292 %   validFrom: text("valid_from"),
1293 %   validUntil: text("valid_until"),
1294 %   status: text("status").$type<PackageStatus>().notNull().default("active"),
1295 %   includedServices: text("included_services").array(),
1296 %   terms: text("terms"),
1297 %   createdAt: text("created_at").notNull(),
1298 % });
1299 %
```

shared/schema.ts (cont.)

```
1300 % export const insertPackageSchema = createInsertSchema(packages).omit({ id: true });
1301 % export type InsertPackage = z.infer<typeof insertPackageSchema>;
1302 % export type Package = typeof packages.$inferSelect;
1303 %
1304 % // Package Included Items (Items incluidos en paquetes)
1305 % export type PackageItemType = "accommodation" | "breakfast" | "dinner" | "spa" | "restaurant" | "event" | "transfer" ...
1306 %
1307 % export const packageItems = pgTable("package_items", {
1308 %   id: varchar("id").primaryKey().default(sql`gen_random_uuid()`),
1309 %   packageId: varchar("package_id").notNull(),
1310 %   itemType: text("item_type").$type<PackageItemType>().notNull(),
1311 %   description: text("description").notNull(),
1312 %   quantity: integer("quantity").notNull().default(1),
1313 %   unitValue: decimal("unit_value", { precision: 10, scale: 2 }),
1314 % });
1315 %
1316 % export const insertPackageItemSchema = createInsertSchema(packageItems).omit({ id: true });
1317 % export type InsertPackageItem = z.infer<typeof insertPackageItemSchema>;
1318 % export type PackageItem = typeof packageItems.$inferSelect;
1319 %
1320 % // Package with details
1321 % export type PackageWithDetails = Package & {
1322 %   roomType?: RoomType;
1323 %   items: PackageItem[];
1324 % };
1325 %
1326 % // System Notifications (base for chatbot + web check-in)
1327 % export type NotificationType = "web_checkin" | "chatbot_request" | "chatbot_housekeeping" | "chatbot_maintenance" | "...
1328 % export type NotificationArea = "reception" | "housekeeping" | "maintenance" | "restaurant" | "spa" | "all";
1329 % export type NotificationPriority = "low" | "normal" | "high" | "urgent";
1330 %
1331 % export const systemNotifications = pgTable("system_notifications", {
1332 %   id: serial("id").primaryKey(),
1333 %   type: text("type").$type<NotificationType>().notNull(),
1334 %   title: text("title").notNull(),
1335 %   message: text("message").notNull(),
1336 %   targetArea: text("target_area").$type<NotificationArea>().notNull(),
1337 %   relatedEntityType: text("related_entity_type"),
1338 %   relatedEntityId: text("related_entity_id"),
1339 %   isRead: boolean("is_read").default(false),
1340 %   readAt: timestamp("read_at"),
1341 %   readBy: text("read_by"),
1342 %   priority: text("priority").$type<NotificationPriority>().notNull().default("normal"),
1343 %   createdAt: timestamp("created_at").defaultNow(),
1344 % });
1345 %
1346 % export const insertSystemNotificationSchema = createInsertSchema(systemNotifications).omit({ id: true, createdAt: tru...
1347 % export type InsertSystemNotification = z.infer<typeof insertSystemNotificationSchema>;
1348 % export type SystemNotification = typeof systemNotifications.$inferSelect;
1349 %
1350 % // Web Check-in
1351 % export type WebCheckinStatus = "pending" | "completed" | "expired";
1352 %
1353 % export const webCheckins = pgTable("web_checkins", {
1354 %   id: serial("id").primaryKey(),
1355 %   reservationId: varchar("reservation_id").notNull(),
1356 %   token: text("token").notNull().unique(),
1357 %   status: text("status").$type<WebCheckinStatus>().notNull().default("pending"),
1358 %   confirmedFirstName: text("confirmed_first_name"),
1359 %   confirmedLastName: text("confirmed_last_name"),
1360 %   confirmedDocumentType: text("confirmed_document_type"),
1361 %   confirmedDocumentNumber: text("confirmed_document_number"),
1362 %   confirmedNationality: text("confirmed_nationality"),
1363 %   confirmedPhone: text("confirmed_phone"),
1364 %   confirmedEmail: text("confirmed_email"),
1365 %   documentPhotoUrl: text("document_photo_url"),
1366 %   estimatedArrivalTime: text("estimated_arrival_time"),
1367 %   requestEarlyCheckIn: boolean("request_early_check_in").default(false),
1368 %   earlyCheckInTime: text("early_check_in_time"),
1369 %   termsAccepted: boolean("terms_accepted").default(false),
1370 %   termsAcceptedAt: timestamp("terms_accepted_at"),
1371 %   ipAddress: text("ip_address"),
1372 %   completedAt: timestamp("completed_at"),
1373 %   expiresAt: timestamp("expires_at"),
1374 %   createdAt: timestamp("created_at").defaultNow(),
1375 % });
1376 %
1377 % export const insertWebCheckinSchema = createInsertSchema(webCheckins).omit({ id: true, createdAt: true });
1378 % export type InsertWebCheckin = z.infer<typeof insertWebCheckinSchema>;
1379 % export type WebCheckin = typeof webCheckins.$inferSelect;
1380 %
1381 % // Hospitality Module - Guest Preferences CRM
1382 % export type PreferenceCategory = "habitacion" | "alimentacion" | "amenities" | "servicio" | "fecha_especial" | "motiv...
1383 % export type PreferencePriority = "low" | "normal" | "high" | "critical";
1384 %
1385 % export const guestPreferences = pgTable("guest_preferences", {
1386 %   id: serial("id").primaryKey(),
1387 %   guestId: varchar("guest_id").notNull(),
1388 %   category: text("category").$type<PreferenceCategory>().notNull(),
1389 %   subcategory: text("subcategory"),
1390 %   title: text("title").notNull(),
1391 %   description: text("description"),
1392 %   isActive: boolean("is_active").notNull().default(true),
```


shared/schema.ts (cont.)

```
1393 %   priority: text("priority").$type<PreferencePriority>().notNull().default("normal"),
1394 %   visibleTo: text("visible_to").array().notNull().default(["all"]),
1395 %   recordedBy: text("recorded_by"),
1396 %   sourceStay: varchar("source_stay"),
1397 %   createdAt: timestamp("created_at").defaultNow(),
1398 %   updatedAt: timestamp("updated_at").defaultNow(),
1399 % });
1400 %
1401 % export const insertGuestPreferenceSchema = createInsertSchema(guestPreferences).omit({ id: true, createdAt: true, upd...
1402 % export type InsertGuestPreference = z.infer<typeof insertGuestPreferenceSchema>;
1403 % export type GuestPreference = typeof guestPreferences.$inferSelect;
1404 %
1405 % export const stayNotes = pgTable("stay_notes", {
1406 %   id: serial("id").primaryKey(),
1407 %   reservationId: varchar("reservation_id").notNull(),
1408 %   guestId: varchar("guest_id"),
1409 %   category: text("category").$type<PreferenceCategory>().notNull(),
1410 %   title: text("title").notNull(),
1411 %   description: text("description"),
1412 %   priority: text("priority").$type<PreferencePriority>().notNull().default("normal"),
1413 %   visibleTo: text("visible_to").array().notNull().default(["all"]),
1414 %   isResolved: boolean("is_resolved").default(false),
1415 %   resolvedAt: timestamp("resolved_at"),
1416 %   resolvedBy: text("resolved_by"),
1417 %   recordedBy: text("recorded_by"),
1418 %   createdAt: timestamp("created_at").defaultNow(),
1419 % });
1420 %
1421 % export const insertStayNoteSchema = createInsertSchema(stayNotes).omit({ id: true, createdAt: true, isResolved: true,...
1422 % export type InsertStayNote = z.infer<typeof insertStayNoteSchema>;
1423 % export type StayNote = typeof stayNotes.$inferSelect;
1424 %
1425 % export const hospitalityAlerts = pgTable("hospitality_alerts", {
1426 %   id: serial("id").primaryKey(),
1427 %   reservationId: varchar("reservation_id").notNull(),
1428 %   guestId: varchar("guest_id").notNull(),
1429 %   preferenceId: integer("preference_id"),
1430 %   alertMessage: text("alert_message").notNull(),
1431 %   targetArea: text("target_area").notNull(),
1432 %   priority: text("priority").$type<PreferencePriority>().notNull().default("normal"),
1433 %   isAcknowledged: boolean("is_acknowledged").default(false),
1434 %   acknowledgedAt: timestamp("acknowledged_at"),
1435 %   acknowledgedBy: text("acknowledged_by"),
1436 %   createdAt: timestamp("created_at").defaultNow(),
1437 % });
1438 %
1439 % export const insertHospitalityAlertSchema = createInsertSchema(hospitalityAlerts).omit({ id: true, createdAt: true, i...
1440 % export type InsertHospitalityAlert = z.infer<typeof insertHospitalityAlertSchema>;
1441 % export type HospitalityAlert = typeof hospitalityAlerts.$inferSelect;
1442 %
```

```

1 % import express, { type Request, Response, NextFunction } from "express";
2 % import { registerRoutes } from "../routes";
3 % import { serveStatic } from "../static";
4 % import { createServer } from "http";
5 %
6 % const app = express();
7 % const httpServer = createServer(app);
8 %
9 % declare module "http" {
10 %   interface IncomingMessage {
11 %     rawBody: unknown;
12 %   }
13 % }
14 %
15 % app.use("/api/public/web-checkin", express.json({ limit: "5mb" }));
16 % app.use(
17 %   express.json({
18 %     limit: "1mb",
19 %     verify: (req, _res, buf) => {
20 %       req.rawBody = buf;
21 %     },
22 %   }),
23 % );
24 %
25 % app.use(express.urlencoded({ extended: false }));
26 %
27 % export function log(message: string, source = "express") {
28 %   const formattedTime = new Date().toLocaleTimeString("en-US", {
29 %     hour: "numeric",
30 %     minute: "2-digit",
31 %     second: "2-digit",
32 %     hour12: true,
33 %   });
34 %
35 %   console.log(`${formattedTime} [${source}] ${message}`);
36 % }
37 %
38 % app.use((req, res, next) => {
39 %   const start = Date.now();
40 %   const path = req.path;
41 %   let capturedJsonResponse: Record<string, any> | undefined = undefined;
42 %
43 %   const originalResJson = res.json;
44 %   res.json = function (bodyJson, ...args) {
45 %     capturedJsonResponse = bodyJson;
46 %     return originalResJson.apply(res, [bodyJson, ...args]);
47 %   };
48 %
49 %   res.on("finish", () => {
50 %     const duration = Date.now() - start;
51 %     if (path.startsWith("/api")) {
52 %       let logLine = `${req.method} ${path} ${res.statusCode} in ${duration}ms`;
53 %       if (capturedJsonResponse) {
54 %         logLine += ` :: ${JSON.stringify(capturedJsonResponse)}`;
55 %       }
56 %
57 %       log(logLine);
58 %     }
59 %   });
60 %
61 %   next();
62 % });
63 %
64 % (async () => {
65 %   await registerRoutes(httpServer, app);
66 %
67 %   app.use((err: any, _req: Request, res: Response, _next: NextFunction) => {
68 %     const status = err.status || err.statusCode || 500;
69 %     const message = err.message || "Internal Server Error";
70 %
71 %     res.status(status).json({ message });
72 %     throw err;
73 %   });
74 %
75 %   // importantly only setup vite in development and after
76 %   // setting up all the other routes so the catch-all route
77 %   // doesn't interfere with the other routes
78 %   if (process.env.NODE_ENV === "production") {
79 %     serveStatic(app);
80 %   } else {
81 %     const { setupVite } = await import("../vite");
82 %     await setupVite(httpServer, app);
83 %   }
84 %
85 %   // ALWAYS serve the app on the port specified in the environment variable PORT
86 %   // Other ports are firewalled. Default to 5000 if not specified.
87 %   // this serves both the API and the client.
88 %   // It is the only port that is not firewalled.
89 %   const port = parseInt(process.env.PORT || "5000", 10);
90 %   httpServer.listen(

```

server/index.ts (cont.)

```
91 %     {
92 %       port,
93 %       host: "0.0.0.0",
94 %       reusePort: true,
95 %     },
96 %     () => {
97 %       log(`serving on port ${port}`);
98 %     },
99 %   );
100 % }());
101 %
```

```

1 % import type { Express } from "express";
2 % import { createServer, type Server } from "http";
3 % import OpenAI from "openai";
4 % import { randomUUID } from "crypto";
5 % import { storage } from "../storage";
6 % import { insertGuestReviewSchema } from "@shared/schema";
7 %
8 % function timeToMinutes(time: string): number {
9 %   const [h, m] = time.split(":").map(Number);
10 %   return h * 60 + m;
11 % }
12 %
13 % const openai = new OpenAI({
14 %   apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
15 %   baseURL: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
16 % });
17 %
18 % export async function registerRoutes(
19 %   httpServer: Server,
20 %   app: Express
21 % ): Promise<Server> {
22 %   app.get("/api/download/source-code", async (_req, res) => {
23 %     const fs = await import("fs");
24 %     const path = await import("path");
25 %     const filePath = path.resolve("MARAN_SUITE_CODIGO_COMPLETO.txt");
26 %     if (fs.existsSync(filePath)) {
27 %       res.setHeader("Content-Type", "text/plain; charset=utf-8");
28 %       res.setHeader("Content-Disposition", "attachment; filename=MARAN_SUITE_CODIGO_COMPLETO.txt");
29 %       fs.createReadStream(filePath).pipe(res);
30 %     } else {
31 %       res.status(404).send("Archivo no encontrado");
32 %     }
33 %   });
34 %
35 %   app.get("/api/download/source-code-zip", async (_req, res) => {
36 %     const { execSync } = await import("child_process");
37 %     const fs = await import("fs");
38 %     const zipPath = "/tmp/maran-suite-source.zip";
39 %     try {
40 %       if (fs.existsSync(zipPath)) fs.unlinkSync(zipPath);
41 %       execSync(
42 %         `cd /home/runner/workspace && zip -r ${zipPath} ` +
43 %         `shared/ server/index.ts server/routes.ts server/storage.ts server/vite.ts server/static.ts ` +
44 %         `client/index.html client/src/ ` +
45 %         `package.json tsconfig.json vite.config.ts tailwind.config.ts drizzle.config.ts components.json ` +
46 %         `script/ replit.md MARAN_SUITE_CODIGO_COMPLETO.txt ` +
47 %         `-x "**/node_modules/" -x "**/.git/" -x "**/dist/"`,
48 %         { timeout: 30000 }
49 %       );
50 %       res.setHeader("Content-Type", "application/zip");
51 %       res.setHeader("Content-Disposition", "attachment; filename=maran-suite-source.zip");
52 %       fs.createReadStream(zipPath).pipe(res);
53 %     } catch (err) {
54 %       res.status(500).send("Error generando ZIP");
55 %     }
56 %   });
57 %
58 %   // Dashboard
59 %   app.get("/api/dashboard/stats", async (req, res) => {
60 %     try {
61 %       const stats = await storage.getDashboardStats();
62 %       res.json(stats);
63 %     } catch (error) {
64 %       res.status(500).json({ error: "Error fetching dashboard stats" });
65 %     }
66 %   });
67 %
68 %   // Today's arrivals (check-ins scheduled for today)
69 %   app.get("/api/dashboard/arrivals", async (req, res) => {
70 %     try {
71 %       const arrivals = await storage.getReservationsForCheckIn();
72 %       res.json(arrivals);
73 %     } catch (error) {
74 %       res.status(500).json({ error: "Error fetching arrivals" });
75 %     }
76 %   });
77 %
78 %   // Today's departures (check-outs scheduled for today)
79 %   app.get("/api/dashboard/departures", async (req, res) => {
80 %     try {
81 %       const departures = await storage.getReservationsForCheckOut();
82 %       res.json(departures);
83 %     } catch (error) {
84 %       res.status(500).json({ error: "Error fetching departures" });
85 %     }
86 %   });
87 %
88 %   // Planning
89 %   app.get("/api/planning", async (req, res) => {
90 %     try {

```

server/routes.ts (cont.)

```
91 %         const startDate = req.query.start as string;
92 %         const endDate = req.query.end as string;
93 %
94 %         if (!startDate || !endDate) {
95 %             return res.status(400).json({ error: "Start and end dates are required" });
96 %         }
97 %
98 %         const planningData = await storage.getPlanningData(startDate, endDate);
99 %         res.json(planningData);
100 %     } catch (error) {
101 %         res.status(500).json({ error: "Error fetching planning data" });
102 %     }
103 % });
104 %
105 % // Room Types
106 % app.get("/api/room-types", async (req, res) => {
107 %     try {
108 %         const roomTypes = await storage.getRoomTypes();
109 %         res.json(roomTypes);
110 %     } catch (error) {
111 %         res.status(500).json({ error: "Error fetching room types" });
112 %     }
113 % });
114 %
115 % app.post("/api/room-types", async (req, res) => {
116 %     try {
117 %         const roomType = await storage.createRoomType(req.body);
118 %         res.status(201).json(roomType);
119 %     } catch (error) {
120 %         res.status(500).json({ error: "Error creating room type" });
121 %     }
122 % });
123 %
124 % app.patch("/api/room-types/:id", async (req, res) => {
125 %     try {
126 %         const roomType = await storage.updateRoomType(req.params.id, req.body);
127 %         if (!roomType) {
128 %             return res.status(404).json({ error: "Room type not found" });
129 %         }
130 %         res.json(roomType);
131 %     } catch (error) {
132 %         res.status(500).json({ error: "Error updating room type" });
133 %     }
134 % });
135 %
136 % app.delete("/api/room-types/:id", async (req, res) => {
137 %     try {
138 %         const deleted = await storage.deleteRoomType(req.params.id);
139 %         if (!deleted) {
140 %             return res.status(404).json({ error: "Room type not found" });
141 %         }
142 %         res.status(204).send();
143 %     } catch (error) {
144 %         res.status(500).json({ error: "Error deleting room type" });
145 %     }
146 % });
147 %
148 % // Rate Plans
149 % app.get("/api/rate-plans", async (req, res) => {
150 %     try {
151 %         const ratePlans = await storage.getRatePlans();
152 %         res.json(ratePlans);
153 %     } catch (error) {
154 %         res.status(500).json({ error: "Error fetching rate plans" });
155 %     }
156 % });
157 %
158 % app.get("/api/rate-plans/by-room-type/:roomId", async (req, res) => {
159 %     try {
160 %         const ratePlans = await storage.getRatePlansByRoomType(req.params.roomId);
161 %         res.json(ratePlans);
162 %     } catch (error) {
163 %         res.status(500).json({ error: "Error fetching rate plans by room type" });
164 %     }
165 % });
166 %
167 % app.get("/api/rate-plans/:id", async (req, res) => {
168 %     try {
169 %         const ratePlan = await storage.getRatePlan(req.params.id);
170 %         if (!ratePlan) {
171 %             return res.status(404).json({ error: "Rate plan not found" });
172 %         }
173 %         res.json(ratePlan);
174 %     } catch (error) {
175 %         res.status(500).json({ error: "Error fetching rate plan" });
176 %     }
177 % });
178 %
179 % app.post("/api/rate-plans", async (req, res) => {
180 %     try {
181 %         const ratePlan = await storage.createRatePlan(req.body);
182 %         res.status(201).json(ratePlan);
183 %     } catch (error) {
```

server/routes.ts (cont.)

```
184 %     res.status(500).json({ error: "Error creating rate plan" });
185 %   }
186 % });
187 %
188 % app.patch("/api/rate-plans/:id", async (req, res) => {
189 %   try {
190 %     const ratePlan = await storage.updateRatePlan(req.params.id, req.body);
191 %     if (!ratePlan) {
192 %       return res.status(404).json({ error: "Rate plan not found" });
193 %     }
194 %     res.json(ratePlan);
195 %   } catch (error) {
196 %     res.status(500).json({ error: "Error updating rate plan" });
197 %   }
198 % });
199 %
200 % app.delete("/api/rate-plans/:id", async (req, res) => {
201 %   try {
202 %     const deleted = await storage.deleteRatePlan(req.params.id);
203 %     if (!deleted) {
204 %       return res.status(404).json({ error: "Rate plan not found" });
205 %     }
206 %     res.status(204).send();
207 %   } catch (error) {
208 %     res.status(500).json({ error: "Error deleting rate plan" });
209 %   }
210 % });
211 %
212 % // Rooms
213 % app.get("/api/rooms", async (req, res) => {
214 %   try {
215 %     const rooms = await storage.getRooms();
216 %     res.json(rooms);
217 %   } catch (error) {
218 %     res.status(500).json({ error: "Error fetching rooms" });
219 %   }
220 % });
221 %
222 % app.get("/api/rooms/:id", async (req, res) => {
223 %   try {
224 %     const room = await storage.getRoom(req.params.id);
225 %     if (!room) {
226 %       return res.status(404).json({ error: "Room not found" });
227 %     }
228 %     res.json(room);
229 %   } catch (error) {
230 %     res.status(500).json({ error: "Error fetching room" });
231 %   }
232 % });
233 %
234 % app.post("/api/rooms", async (req, res) => {
235 %   try {
236 %     const room = await storage.createRoom(req.body);
237 %     res.status(201).json(room);
238 %   } catch (error) {
239 %     res.status(500).json({ error: "Error creating room" });
240 %   }
241 % });
242 %
243 % app.patch("/api/rooms/:id", async (req, res) => {
244 %   try {
245 %     const room = await storage.updateRoom(req.params.id, req.body);
246 %     if (!room) {
247 %       return res.status(404).json({ error: "Room not found" });
248 %     }
249 %     res.json(room);
250 %   } catch (error) {
251 %     res.status(500).json({ error: "Error updating room" });
252 %   }
253 % });
254 %
255 % app.delete("/api/rooms/:id", async (req, res) => {
256 %   try {
257 %     const deleted = await storage.deleteRoom(req.params.id);
258 %     if (!deleted) {
259 %       return res.status(404).json({ error: "Room not found" });
260 %     }
261 %     res.status(204).send();
262 %   } catch (error) {
263 %     res.status(500).json({ error: "Error deleting room" });
264 %   }
265 % });
266 %
267 % // Companies
268 % app.get("/api/companies", async (req, res) => {
269 %   try {
270 %     const companies = await storage.getCompanies();
271 %     res.json(companies);
272 %   } catch (error) {
273 %     res.status(500).json({ error: "Error fetching companies" });
274 %   }
275 % });
276 %
```

server/routes.ts (cont.)

```
277 %   app.get("/api/companies/search", async (req, res) => {
278 %       try {
279 %         const query = req.query.q as string || "";
280 %         const companies = await storage.searchCompanies(query);
281 %         res.json(companies);
282 %       } catch (error) {
283 %         res.status(500).json({ error: "Error searching companies" });
284 %       }
285 %     });
286 %
287 %   app.get("/api/companies/:id", async (req, res) => {
288 %       try {
289 %         const company = await storage.getCompany(req.params.id);
290 %         if (!company) {
291 %           return res.status(404).json({ error: "Company not found" });
292 %         }
293 %         res.json(company);
294 %       } catch (error) {
295 %         res.status(500).json({ error: "Error fetching company" });
296 %       }
297 %     });
298 %
299 %   app.post("/api/companies", async (req, res) => {
300 %       try {
301 %         const company = await storage.createCompany(req.body);
302 %         res.status(201).json(company);
303 %       } catch (error) {
304 %         res.status(500).json({ error: "Error creating company" });
305 %       }
306 %     });
307 %
308 %   app.patch("/api/companies/:id", async (req, res) => {
309 %       try {
310 %         const company = await storage.updateCompany(req.params.id, req.body);
311 %         if (!company) {
312 %           return res.status(404).json({ error: "Company not found" });
313 %         }
314 %         res.json(company);
315 %       } catch (error) {
316 %         res.status(500).json({ error: "Error updating company" });
317 %       }
318 %     });
319 %
320 %   app.delete("/api/companies/:id", async (req, res) => {
321 %       try {
322 %         const deleted = await storage.deleteCompany(req.params.id);
323 %         if (!deleted) {
324 %           return res.status(404).json({ error: "Company not found" });
325 %         }
326 %         res.status(204).send();
327 %       } catch (error) {
328 %         res.status(500).json({ error: "Error deleting company" });
329 %       }
330 %     });
331 %
332 %   // Guests
333 %   app.get("/api/guests", async (req, res) => {
334 %       try {
335 %         const guests = await storage.getGuests();
336 %         res.json(guests);
337 %       } catch (error) {
338 %         res.status(500).json({ error: "Error fetching guests" });
339 %       }
340 %     });
341 %
342 %   app.get("/api/guests/search", async (req, res) => {
343 %       try {
344 %         const query = req.query.q as string || "";
345 %         const guests = await storage.searchGuests(query);
346 %         res.json(guests);
347 %       } catch (error) {
348 %         res.status(500).json({ error: "Error searching guests" });
349 %       }
350 %     });
351 %
352 %   app.get("/api/guests/:id", async (req, res) => {
353 %       try {
354 %         const guest = await storage.getGuest(req.params.id);
355 %         if (!guest) {
356 %           return res.status(404).json({ error: "Guest not found" });
357 %         }
358 %         res.json(guest);
359 %       } catch (error) {
360 %         res.status(500).json({ error: "Error fetching guest" });
361 %       }
362 %     });
363 %
364 %   app.post("/api/guests", async (req, res) => {
365 %       try {
366 %         const guest = await storage.createGuest(req.body);
367 %         res.status(201).json(guest);
368 %       } catch (error) {
369 %         res.status(500).json({ error: "Error creating guest" });
370 %       }
371 %     });
372 % }
```

server/routes.ts (cont.)

```
370 %     }
371 %   });
372 %
373 %   app.patch("/api/guests/:id", async (req, res) => {
374 %     try {
375 %       const guest = await storage.updateGuest(req.params.id, req.body);
376 %       if (!guest) {
377 %         return res.status(404).json({ error: "Guest not found" });
378 %       }
379 %       res.json(guest);
380 %     } catch (error) {
381 %       res.status(500).json({ error: "Error updating guest" });
382 %     }
383 %   });
384 %
385 %   app.delete("/api/guests/:id", async (req, res) => {
386 %     try {
387 %       const deleted = await storage.deleteGuest(req.params.id);
388 %       if (!deleted) {
389 %         return res.status(404).json({ error: "Guest not found" });
390 %       }
391 %       res.status(204).send();
392 %     } catch (error) {
393 %       res.status(500).json({ error: "Error deleting guest" });
394 %     }
395 %   });
396 %
397 %   // Bed Types
398 %   app.get("/api/bed-types", async (req, res) => {
399 %     try {
400 %       const bedTypes = await storage.getBedTypes();
401 %       res.json(bedTypes);
402 %     } catch (error) {
403 %       res.status(500).json({ error: "Error fetching bed types" });
404 %     }
405 %   });
406 %
407 %   app.post("/api/bed-types", async (req, res) => {
408 %     try {
409 %       const bedType = await storage.createBedType(req.body);
410 %       res.status(201).json(bedType);
411 %     } catch (error) {
412 %       res.status(500).json({ error: "Error creating bed type" });
413 %     }
414 %   });
415 %
416 %   app.patch("/api/bed-types/:id", async (req, res) => {
417 %     try {
418 %       const id = parseInt(req.params.id);
419 %       const bedType = await storage.updateBedType(id, req.body);
420 %       if (!bedType) {
421 %         return res.status(404).json({ error: "Bed type not found" });
422 %       }
423 %       res.json(bedType);
424 %     } catch (error) {
425 %       res.status(500).json({ error: "Error updating bed type" });
426 %     }
427 %   });
428 %
429 %   app.delete("/api/bed-types/:id", async (req, res) => {
430 %     try {
431 %       const id = parseInt(req.params.id);
432 %       const result = await storage.deleteBedType(id);
433 %       if (!result) {
434 %         return res.status(404).json({ error: "Bed type not found" });
435 %       }
436 %       res.json({ success: true });
437 %     } catch (error) {
438 %       res.status(500).json({ error: "Error deleting bed type" });
439 %     }
440 %   });
441 %
442 %   // Reservations
443 %   app.get("/api/reservations", async (req, res) => {
444 %     try {
445 %       const reservations = await storage.getReservations();
446 %       res.json(reservations);
447 %     } catch (error) {
448 %       res.status(500).json({ error: "Error fetching reservations" });
449 %     }
450 %   });
451 %
452 %   app.get("/api/reservations/recent", async (req, res) => {
453 %     try {
454 %       const reservations = await storage.getRecentReservations(5);
455 %       res.json(reservations);
456 %     } catch (error) {
457 %       res.status(500).json({ error: "Error fetching recent reservations" });
458 %     }
459 %   });
460 %
461 %   app.get("/api/reservations/check-in", async (req, res) => {
462 %     try {
```


server/routes.ts (cont.)

```

463 %     const reservations = await storage.getReservationsForCheckIn();
464 %     res.json(reservations);
465 %   } catch (error) {
466 %     res.status(500).json({ error: "Error fetching check-in reservations" });
467 %   }
468 % });
469 %
470 % app.get("/api/reservations/check-ins-by-date", async (req, res) => {
471 %   try {
472 %     const { date } = req.query;
473 %     if (!date || typeof date !== "string") {
474 %       return res.status(400).json({ error: "Date parameter required" });
475 %     }
476 %     const reservations = await storage.getCheckInsByDate(date);
477 %     res.json(reservations);
478 %   } catch (error) {
479 %     res.status(500).json({ error: "Error fetching check-ins by date" });
480 %   }
481 % });
482 %
483 % app.get("/api/reservations/check-out", async (req, res) => {
484 %   try {
485 %     const reservations = await storage.getReservationsForCheckOut();
486 %     res.json(reservations);
487 %   } catch (error) {
488 %     res.status(500).json({ error: "Error fetching check-out reservations" });
489 %   }
490 % });
491 %
492 % app.get("/api/reservations/:id", async (req, res) => {
493 %   try {
494 %     const reservation = await storage.getReservation(req.params.id);
495 %     if (!reservation) {
496 %       return res.status(404).json({ error: "Reservation not found" });
497 %     }
498 %     res.json(reservation);
499 %   } catch (error) {
500 %     res.status(500).json({ error: "Error fetching reservation" });
501 %   }
502 % });
503 %
504 % app.post("/api/reservations", async (req, res) => {
505 %   try {
506 %     const data = {
507 %       ...req.body,
508 %       reservationCode: req.body.reservationCode || storage.generateReservationCode(),
509 %       createdAt: req.body.createdAt || new Date().toISOString(),
510 %     };
511 %     const reservation = await storage.createReservation(data);
512 %
513 %     const today = new Date().toISOString().split("T")[0];
514 %     if (data.earlyCheckIn && data.earlyCheckInCharge && parseFloat(data.earlyCheckInCharge) > 0) {
515 %       await storage.createCharge({
516 %         reservationId: reservation.id,
517 %         description: `Early Check-in ${data.earlyCheckInTime || ""}`.trim(),
518 %         amount: data.earlyCheckInCharge,
519 %         date: today,
520 %         category: "otros",
521 %       });
522 %     }
523 %     if (data.lateCheckOut && data.lateCheckOutCharge && parseFloat(data.lateCheckOutCharge) > 0) {
524 %       await storage.createCharge({
525 %         reservationId: reservation.id,
526 %         description: `Late Check-out ${data.lateCheckOutTime || ""}`.trim(),
527 %         amount: data.lateCheckOutCharge,
528 %         date: today,
529 %         category: "otros",
530 %       });
531 %     }
532 %
533 %     res.status(201).json(reservation);
534 %   } catch (error) {
535 %     res.status(500).json({ error: "Error creating reservation" });
536 %   }
537 % });
538 %
539 % app.patch("/api/reservations/:id", async (req, res) => {
540 %   try {
541 %     const existing = await storage.getReservation(req.params.id);
542 %     if (!existing) {
543 %       return res.status(404).json({ error: "Reservation not found" });
544 %     }
545 %     const reservation = await storage.updateReservation(req.params.id, req.body);
546 %     if (!reservation) {
547 %       return res.status(404).json({ error: "Reservation not found" });
548 %     }
549 %
550 %     const today = new Date().toISOString().split("T")[0];
551 %     if (req.body.earlyCheckIn && req.body.earlyCheckInCharge && parseFloat(req.body.earlyCheckInCharge) > 0 && !exi...
552 %       await storage.createCharge({
553 %         reservationId: reservation.id,
554 %         description: `Early Check-in ${req.body.earlyCheckInTime || ""}`.trim(),
555 %         amount: req.body.earlyCheckInCharge,

```

server/routes.ts (cont.)

```
556 %         date: today,
557 %         category: "otros",
558 %     });
559 % }
560 % if (req.body.lateCheckOut && req.body.lateCheckOutCharge && parseFloat(req.body.lateCheckOutCharge) > 0 && !exi...
561 %     await storage.createCharge({
562 %         reservationId: reservation.id,
563 %         description: `Late Check-out ${req.body.lateCheckOutTime || ""}`.trim(),
564 %         amount: req.body.lateCheckOutCharge,
565 %         date: today,
566 %         category: "otros",
567 %     });
568 % }
569 %
570 % res.json(reservation);
571 % } catch (error) {
572 %     res.status(500).json({ error: "Error updating reservation" });
573 % }
574 % });
575 %
576 % // Duplicate reservation endpoint
577 % app.post("/api/reservations/:id/duplicate", async (req, res) => {
578 %     try {
579 %         const original = await storage.getReservation(req.params.id);
580 %         if (!original) {
581 %             return res.status(404).json({ error: "Reservation not found" });
582 %         }
583 %
584 %         const { checkInDate, checkOutDate, roomId } = req.body;
585 %
586 %         if (!checkInDate || !checkOutDate) {
587 %             return res.status(400).json({ error: "Check-in and check-out dates are required" });
588 %         }
589 %
590 %         // Normalize dates to date-only strings (YYYY-MM-DD) to avoid timezone issues
591 %         const normalizeDate = (dateStr: string): string => {
592 %             // If already in YYYY-MM-DD format, use as-is
593 %             if (/^\d{4}-\d{2}-\d{2}$/.test(dateStr)) return dateStr;
594 %             // Otherwise parse and extract date part
595 %             const d = new Date(dateStr);
596 %             return d.toISOString().split('T')[0];
597 %         };
598 %
599 %         const normalizedCheckIn = normalizeDate(checkInDate);
600 %         const normalizedCheckOut = normalizeDate(checkOutDate);
601 %
602 %         // Parse normalized dates for comparison (UTC midnight)
603 %         const checkIn = new Date(normalizedCheckIn + 'T00:00:00Z');
604 %         const checkOut = new Date(normalizedCheckOut + 'T00:00:00Z');
605 %
606 %         if (checkOut <= checkIn) {
607 %             return res.status(400).json({ error: "La fecha de salida debe ser posterior a la entrada" });
608 %         }
609 %
610 %         const nights = Math.round(
611 %             (checkOut.getTime() - checkIn.getTime()) / (1000 * 60 * 60 * 24)
612 %         );
613 %
614 %         if (nights <= 0) {
615 %             return res.status(400).json({ error: "La reserva debe tener al menos 1 noche" });
616 %         }
617 %
618 %         const finalRoomId = roomId || original.roomId;
619 %         const room = await storage.getRoom(finalRoomId);
620 %
621 %         // Check for overlapping reservations on the same room
622 %         // Active statuses that block the room
623 %         const blockingStatuses = ["tentative", "pending", "confirmed", "checked_in"];
624 %         const allReservations = await storage.getReservations();
625 %         const overlapping = allReservations.filter(r => {
626 %             // Exclude the original reservation from overlap check
627 %             if (r.id === original.id) return false;
628 %             if (r.roomId !== finalRoomId) return false;
629 %             if (!blockingStatuses.includes(r.status)) return false;
630 %
631 %             const rCheckIn = new Date(normalizeDate(r.checkInDate) + 'T00:00:00Z');
632 %             const rCheckOut = new Date(normalizeDate(r.checkOutDate) + 'T00:00:00Z');
633 %
634 %             // Check for date overlap
635 %             return !(checkOut <= rCheckIn || checkIn >= rCheckOut);
636 %         });
637 %
638 %         if (overlapping.length > 0) {
639 %             return res.status(400).json({
640 %                 error: `La habitacion ${room?.roomNumber || finalRoomId} ya tiene una reserva en esas fechas`
641 %             });
642 %         }
643 %
644 %         const newCode = storage.generateReservationCode();
645 %
646 %         const duplicated = await storage.createReservation({
647 %             reservationCode: newCode,
648 %             guestId: original.guestId,
```

server/routes.ts (cont.)

```

649 %         companyId: original.companyId || null,
650 %         roomId: room?.roomId || original.roomId,
651 %         ratePlanId: original.ratePlanId,
652 %         checkInDate: normalizedCheckIn,
653 %         checkOutDate: normalizedCheckOut,
654 %         nights,
655 %         baseRatePerNight: original.baseRatePerNight,
656 %         discountType: original.discountType,
657 %         discountValue: original.discountValue,
658 %         finalRatePerNight: original.finalRatePerNight,
659 %         totalRoomAmount: (parseFloat(original.finalRatePerNight || "0") * nights).toFixed(2),
660 %         status: "confirmed",
661 %         source: original.source,
662 %         otaChannelId: original.otaChannelId,
663 %         externalReservationId: null,
664 %         numberOfGuests: original.numberofGuests,
665 %         notes: `Duplicada de ${original.reservationCode}`,
666 %         createdAt: new Date().toISOString(),
667 %         lastModifiedBy: null,
668 %     });
669 %
670 %
671 %     res.status(201).json(duplicated);
672 % } catch (error) {
673 %     res.status(500).json({ error: "Error duplicating reservation" });
674 % }
675 % });
676 %
677 % app.delete("/api/reservations/:id", async (req, res) => {
678 %     try {
679 %         const deleted = await storage.deleteReservation(req.params.id);
680 %         if (!deleted) {
681 %             return res.status(404).json({ error: "Reservation not found" });
682 %         }
683 %         res.status(204).send();
684 %     } catch (error) {
685 %         res.status(500).json({ error: "Error deleting reservation" });
686 %     }
687 % });
688 %
689 % // Check-in endpoint
690 % app.post("/api/reservations/:id/check-in", async (req, res) => {
691 %     try {
692 %         const reservation = await storage.getReservation(req.params.id);
693 %         if (!reservation) {
694 %             return res.status(404).json({ error: "Reservation not found" });
695 %         }
696 %
697 %         // Validate room is available (clean) before check-in
698 %         const room = await storage.getRoom(reservation.roomId);
699 %         if (!room) {
700 %             return res.status(400).json({ error: "Habitación no encontrada" });
701 %         }
702 %
703 %         if (room.status !== "available") {
704 %             const statusMessages: Record<string, string> = {
705 %                 occupied: "La habitación está ocupada",
706 %                 cleaning: "La habitación está en limpieza",
707 %                 maintenance: "La habitación está en mantenimiento",
708 %                 out_of_service: "La habitación está fuera de servicio",
709 %             };
710 %             const message = statusMessages[room.status] || `La habitación no está disponible (estado: ${room.status})`;
711 %             return res.status(400).json({ error: message });
712 %         }
713 %
714 %         // Update reservation status
715 %         await storage.updateReservation(req.params.id, { status: "checked_in" });
716 %
717 %         // Update room status to occupied
718 %         await storage.updateRoom(reservation.roomId, { status: "occupied" });
719 %
720 %         // Generate hospitality alerts from guest preferences
721 %         if (reservation.guestId) {
722 %             const preferences = await storage.getActiveGuestPreferences(reservation.guestId);
723 %             for (const pref of preferences) {
724 %                 const areaMap: Record<string, string[]> = {
725 %                     alimentacion: ["restaurant", "reception"],
726 %                     habitacion: ["housekeeping", "reception"],
727 %                     amenities: ["housekeeping"],
728 %                     servicio: ["reception"],
729 %                     fecha_especial: ["reception"],
730 %                     motivo_viaje: ["reception"],
731 %                     nota_interna: ["reception"],
732 %                     otro: ["reception"],
733 %                 };
734 %                 const targetAreas = areaMap[pref.category] || ["reception"];
735 %                 for (const area of targetAreas) {
736 %                     await storage.createHospitalityAlert({
737 %                         reservationId: req.params.id,
738 %                         guestId: reservation.guestId,
739 %                         preferenceId: pref.id,
740 %                         alertMessage: `${pref.title}: ${pref.description || pref.title}`,
741 %                         targetArea: area,

```

server/routes.ts (cont.)

```
742 %         priority: pref.priority as any,
743 %     });
744 % }
745 % if (pref.priority === "critical" || pref.priority === "high") {
746 %     await storage.createNotification({
747 %         type: "hospitality_alert",
748 %         title: `& b Alerta de hospitalidad - Hab. ${room.roomNumber}`,
749 %         message: `${pref.title}: ${pref.description || ""}`,
750 %         targetArea: "all" as any,
751 %         relatedEntityType: "reservation",
752 %         relatedEntityId: req.params.id,
753 %         priority: pref.priority === "critical" ? "urgent" : "high",
754 %     });
755 % }
756 % }
757 % }
758 %
759 % res.json({ success: true });
760 % } catch (error) {
761 %     res.status(500).json({ error: "Error processing check-in" });
762 % }
763 % });
764 %
765 % // Get reservation folio (charges summary)
766 % app.get("/api/reservations/:id/folio", async (req, res) => {
767 %     try {
768 %         const reservation = await storage.getReservation(req.params.id);
769 %         if (!reservation) {
770 %             return res.status(404).json({ error: "Reservation not found" });
771 %         }
772 %
773 %         const charges = await storage.getCharges(req.params.id);
774 %         const payments = await storage.getPayments(req.params.id);
775 %         const totalCharges = charges.reduce((sum, c) => sum + parseFloat(c.amount), 0);
776 %         const totalPayments = payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
777 %         const roomTotal = parseFloat(reservation.totalRoomAmount || "0");
778 %         const grandTotal = roomTotal + totalCharges;
779 %         const balance = grandTotal - totalPayments;
780 %
781 %         res.json({
782 %             reservationCode: reservation.reservationCode,
783 %             guestName: `${reservation.guest?.firstName} ${reservation.guest?.lastName}`,
784 %             roomNumber: reservation.room?.roomNumber,
785 %             checkInDate: reservation.checkInDate,
786 %             checkOutDate: reservation.checkOutDate,
787 %             nights: reservation.nights,
788 %             roomRate: reservation.finalRatePerNight,
789 %             roomTotal,
790 %             charges,
791 %             totalCharges,
792 %             payments,
793 %             totalPayments,
794 %             grandTotal,
795 %             balance,
796 %         });
797 %     } catch (error) {
798 %         res.status(500).json({ error: "Error fetching folio" });
799 %     }
800 % });
801 %
802 % // Check-out endpoint
803 % app.post("/api/reservations/:id/check-out", async (req, res) => {
804 %     try {
805 %         const reservation = await storage.getReservation(req.params.id);
806 %         if (!reservation) {
807 %             return res.status(404).json({ error: "Reservation not found" });
808 %         }
809 %
810 %         if (reservation.status !== "checked_in") {
811 %             return res.status(400).json({ error: "Solo se puede hacer check-out de reservas con estado checked_in" });
812 %         }
813 %
814 %         // Get balance - if forceCheckout is true, skip balance check
815 %         const forceCheckout = req.body.forceCheckout === true;
816 %         if (!forceCheckout) {
817 %             const chargesTotal = await storage.getChargesTotal(req.params.id);
818 %             const paymentsTotal = await storage.getPaymentsTotal(req.params.id);
819 %             const roomTotal = parseFloat(reservation.totalRoomAmount || "0");
820 %             const balance = roomTotal + chargesTotal - paymentsTotal;
821 %
822 %             if (balance > 0.01) {
823 %                 return res.status(400).json({
824 %                     error: "Saldo pendiente",
825 %                     message: `La reserva tiene un saldo pendiente de $${balance.toFixed(2)}. Liquide antes de hacer check-out.`,
826 %                     balance
827 %                 });
828 %             }
829 %         }
830 %
831 %         // Update reservation status
832 %         await storage.updateReservation(req.params.id, { status: "checked_out" });
833 %
834 %         // Update room status to cleaning
```

server/routes.ts (cont.)

```
835 %         await storage.updateRoom(reservation.roomId, { status: "cleaning" });
836 %
837 %         // Create housekeeping task for the room
838 %         await storage.createCheckoutCleaningTask(reservation.roomId);
839 %
840 %         res.json({ success: true });
841 %     } catch (error) {
842 %         res.status(500).json({ error: "Error processing check-out" });
843 %     }
844 % });
845 %
846 % // Cancel reservation endpoint (logs to CancelledReservationLog)
847 % app.post("/api/reservations/:id/cancel", async (req, res) => {
848 %     try {
849 %         const reservation = await storage.getReservation(req.params.id);
850 %         if (!reservation) {
851 %             return res.status(404).json({ error: "Reservation not found" });
852 %         }
853 %
854 %         // Log the cancellation
855 %         await storage.createCancelledReservationLog({
856 %             reservationCode: reservation.reservationCode,
857 %             guestName: `${reservation.guest?.firstName} ${reservation.guest?.lastName}`,
858 %             roomNumber: reservation.room?.roomNumber || "",
859 %             checkInDate: reservation.checkInDate,
860 %             checkOutDate: reservation.checkOutDate,
861 %             cancellationDate: new Date().toISOString(),
862 %             cancelledBy: req.body.cancelledBy || null,
863 %             reason: req.body.reason || null,
864 %         });
865 %
866 %         // Update reservation status
867 %         await storage.updateReservation(req.params.id, { status: "cancelled" });
868 %
869 %         // If room was occupied, set to dirty
870 %         if (reservation.room?.status === "occupied") {
871 %             await storage.updateRoom(reservation.roomId, { status: "dirty" });
872 %         }
873 %
874 %         res.json({ success: true });
875 %     } catch (error) {
876 %         res.status(500).json({ error: "Error cancelling reservation" });
877 %     }
878 % });
879 %
880 % // Check overbooking
881 % app.get("/api/reservations/check-overbooking", async (req, res) => {
882 %     try {
883 %         const { roomId, checkInDate, checkOutDate, excludeReservationId } = req.query;
884 %         if (!roomId || !checkInDate || !checkOutDate) {
885 %             return res.status(400).json({ error: "Missing required parameters" });
886 %         }
887 %         const hasConflict = await storage.checkOverbooking(
888 %             roomId as string,
889 %             checkInDate as string,
890 %             checkOutDate as string,
891 %             excludeReservationId as string | undefined
892 %         );
893 %         res.json({ hasConflict });
894 %     } catch (error) {
895 %         res.status(500).json({ error: "Error checking overbooking" });
896 %     }
897 % });
898 %
899 % // Cancelled reservation logs
900 % app.get("/api/cancelled-reservations", async (req, res) => {
901 %     try {
902 %         const logs = await storage.getCancelledReservationLogs();
903 %         res.json(logs);
904 %     } catch (error) {
905 %         res.status(500).json({ error: "Error fetching cancelled reservation logs" });
906 %     }
907 % });
908 %
909 % // Generate reservation code
910 % app.get("/api/reservations/generate-code", async (req, res) => {
911 %     try {
912 %         const code = storage.generateReservationCode();
913 %         res.json({ code });
914 %     } catch (error) {
915 %         res.status(500).json({ error: "Error generating reservation code" });
916 %     }
917 % });
918 %
919 % // Get reservations by guest
920 % app.get("/api/guests/:guestId/reservations", async (req, res) => {
921 %     try {
922 %         const reservations = await storage.getReservationsByGuest(req.params.guestId);
923 %         res.json(reservations);
924 %     } catch (error) {
925 %         res.status(500).json({ error: "Error fetching guest reservations" });
926 %     }
927 % });
```

server/routes.ts (cont.)

```
928 %
929 % // Charges
930 % app.get("/api/reservations/:reservationId/charges", async (req, res) => {
931 %   try {
932 %     const charges = await storage.getCharges(req.params.reservationId);
933 %     res.json(charges);
934 %   } catch (error) {
935 %     res.status(500).json({ error: "Error fetching charges" });
936 %   }
937 % });
938 %
939 % app.get("/api/reservations/:reservationId/charges/total", async (req, res) => {
940 %   try {
941 %     const total = await storage.getChargesTotal(req.params.reservationId);
942 %     res.json({ total });
943 %   } catch (error) {
944 %     res.status(500).json({ error: "Error fetching charges total" });
945 %   }
946 % });
947 %
948 % app.post("/api/charges", async (req, res) => {
949 %   try {
950 %     const charge = await storage.createCharge(req.body);
951 %     res.status(201).json(charge);
952 %   } catch (error) {
953 %     res.status(500).json({ error: "Error creating charge" });
954 %   }
955 % });
956 %
957 % app.patch("/api/charges/:id", async (req, res) => {
958 %   try {
959 %     const charge = await storage.updateCharge(req.params.id, req.body);
960 %     if (!charge) {
961 %       return res.status(404).json({ error: "Charge not found" });
962 %     }
963 %     res.json(charge);
964 %   } catch (error) {
965 %     res.status(500).json({ error: "Error updating charge" });
966 %   }
967 % });
968 %
969 % app.delete("/api/charges/:id", async (req, res) => {
970 %   try {
971 %     const deleted = await storage.deleteCharge(req.params.id);
972 %     if (!deleted) {
973 %       return res.status(404).json({ error: "Charge not found" });
974 %     }
975 %     res.status(204).send();
976 %   } catch (error) {
977 %     res.status(500).json({ error: "Error deleting charge" });
978 %   }
979 % });
980 %
981 % // Transfer charge to another reservation
982 % app.post("/api/charges/:id/transfer", async (req, res) => {
983 %   try {
984 %     const { targetReservationId } = req.body;
985 %
986 %     // Validate input - only targetReservationId is accepted
987 %     if (!targetReservationId || typeof targetReservationId !== "string") {
988 %       return res.status(400).json({ error: "Target reservation ID is required" });
989 %     }
990 %
991 %     // Verify the charge exists
992 %     const charge = await storage.getCharge(req.params.id);
993 %     if (!charge) {
994 %       return res.status(404).json({ error: "Charge not found" });
995 %     }
996 %
997 %     // Prevent transferring to same reservation
998 %     if (charge.reservationId === targetReservationId) {
999 %       return res.status(400).json({ error: "Cannot transfer to the same reservation" });
1000 %     }
1001 %
1002 %     // Verify the target reservation exists and is in transferable state
1003 %     const targetReservation = await storage.getReservation(targetReservationId);
1004 %     if (!targetReservation) {
1005 %       return res.status(404).json({ error: "Target reservation not found" });
1006 %     }
1007 %
1008 %     // Only allow transfer to active reservations (checked_in or confirmed)
1009 %     if (targetReservation.status !== "checked_in" && targetReservation.status !== "confirmed") {
1010 %       return res.status(400).json({ error: "Target reservation must be active (checked-in or confirmed)" });
1011 %     }
1012 %
1013 %     // Update ONLY the reservationId field - explicitly whitelist
1014 %     const updatedCharge = await storage.updateCharge(req.params.id, {
1015 %       reservationId: targetReservationId,
1016 %     });
1017 %
1018 %     res.json(updatedCharge);
1019 %   } catch (error) {
1020 %     res.status(500).json({ error: "Error transferring charge" });
1021 %   }
1022 % }
```

server/routes.ts (cont.)

```
1021 %     }
1022 %   });
1023 %
1024 %   // Payments
1025 %   app.get("/api/reservations/:reservationId/payments", async (req, res) => {
1026 %     try {
1027 %       const payments = await storage.getPayments(req.params.reservationId);
1028 %       res.json(payments);
1029 %     } catch (error) {
1030 %       res.status(500).json({ error: "Error fetching payments" });
1031 %     }
1032 %   });
1033 %
1034 %   app.get("/api/reservations/:reservationId/payments/total", async (req, res) => {
1035 %     try {
1036 %       const total = await storage.getPaymentsTotal(req.params.reservationId);
1037 %       res.json({ total });
1038 %     } catch (error) {
1039 %       res.status(500).json({ error: "Error fetching payments total" });
1040 %     }
1041 %   });
1042 %
1043 %   app.post("/api/payments", async (req, res) => {
1044 %     try {
1045 %       const payment = await storage.createPayment(req.body);
1046 %       res.status(201).json(payment);
1047 %     } catch (error) {
1048 %       res.status(500).json({ error: "Error creating payment" });
1049 %     }
1050 %   });
1051 %
1052 %   app.patch("/api/payments/:id", async (req, res) => {
1053 %     try {
1054 %       const payment = await storage.updatePayment(req.params.id, req.body);
1055 %       if (!payment) {
1056 %         return res.status(404).json({ error: "Payment not found" });
1057 %       }
1058 %       res.json(payment);
1059 %     } catch (error) {
1060 %       res.status(500).json({ error: "Error updating payment" });
1061 %     }
1062 %   });
1063 %
1064 %   app.delete("/api/payments/:id", async (req, res) => {
1065 %     try {
1066 %       const deleted = await storage.deletePayment(req.params.id);
1067 %       if (!deleted) {
1068 %         return res.status(404).json({ error: "Payment not found" });
1069 %       }
1070 %       res.status(204).send();
1071 %     } catch (error) {
1072 %       res.status(500).json({ error: "Error deleting payment" });
1073 %     }
1074 %   });
1075 %
1076 %   // OTA Channels
1077 %   app.get("/api/ota-channels", async (req, res) => {
1078 %     try {
1079 %       const channels = await storage.getOTAChannels();
1080 %       res.json(channels);
1081 %     } catch (error) {
1082 %       res.status(500).json({ error: "Error fetching OTA channels" });
1083 %     }
1084 %   });
1085 %
1086 %   app.get("/api/ota-channels/:id", async (req, res) => {
1087 %     try {
1088 %       const channel = await storage.getOTAChannel(req.params.id);
1089 %       if (!channel) {
1090 %         return res.status(404).json({ error: "OTA channel not found" });
1091 %       }
1092 %       res.json(channel);
1093 %     } catch (error) {
1094 %       res.status(500).json({ error: "Error fetching OTA channel" });
1095 %     }
1096 %   });
1097 %
1098 %   app.post("/api/ota-channels", async (req, res) => {
1099 %     try {
1100 %       const channel = await storage.createOTAChannel({
1101 %         ...req.body,
1102 %         createdAt: new Date().toISOString(),
1103 %       });
1104 %       res.status(201).json(channel);
1105 %     } catch (error) {
1106 %       res.status(500).json({ error: "Error creating OTA channel" });
1107 %     }
1108 %   });
1109 %
1110 %   app.patch("/api/ota-channels/:id", async (req, res) => {
1111 %     try {
1112 %       const channel = await storage.updateOTAChannel(req.params.id, req.body);
1113 %       if (!channel) {
```

server/routes.ts (cont.)

```
1114 %         return res.status(404).json({ error: "OTA channel not found" });
1115 %     }
1116 %     res.json(channel);
1117 % } catch (error) {
1118 %     res.status(500).json({ error: "Error updating OTA channel" });
1119 % }
1120 % });
1121 %
1122 % app.delete("/api/ota-channels/:id", async (req, res) => {
1123 %     try {
1124 %         const deleted = await storage.deleteOTAChannel(req.params.id);
1125 %         if (!deleted) {
1126 %             return res.status(404).json({ error: "OTA channel not found" });
1127 %         }
1128 %         res.status(204).send();
1129 %     } catch (error) {
1130 %         res.status(500).json({ error: "Error deleting OTA channel" });
1131 %     }
1132 % });
1133 %
1134 % // OTA Reservation Logs
1135 % app.get("/api/ota-reservations", async (req, res) => {
1136 %     try {
1137 %         const channelId = req.query.channelId as string | undefined;
1138 %         const logs = await storage.getOTAReservationLogs(channelId);
1139 %         res.json(logs);
1140 %     } catch (error) {
1141 %         res.status(500).json({ error: "Error fetching OTA reservation logs" });
1142 %     }
1143 % });
1144 %
1145 % app.get("/api/ota-reservations/:id", async (req, res) => {
1146 %     try {
1147 %         const log = await storage.getOTAReservationLog(req.params.id);
1148 %         if (!log) {
1149 %             return res.status(404).json({ error: "OTA reservation log not found" });
1150 %         }
1151 %         res.json(log);
1152 %     } catch (error) {
1153 %         res.status(500).json({ error: "Error fetching OTA reservation log" });
1154 %     }
1155 % });
1156 %
1157 % app.post("/api/ota-reservations", async (req, res) => {
1158 %     try {
1159 %         const log = await storage.createOTAReservationLog({
1160 %             ...req.body,
1161 %             createdAt: new Date().toISOString(),
1162 %         });
1163 %         res.status(201).json(log);
1164 %     } catch (error) {
1165 %         res.status(500).json({ error: "Error creating OTA reservation log" });
1166 %     }
1167 % });
1168 %
1169 % app.post("/api/ota-reservations/:id/sync", async (req, res) => {
1170 %     try {
1171 %         const reservation = await storage.syncOTAReservation(req.params.id);
1172 %         if (!reservation) {
1173 %             return res.status(400).json({ error: "Could not sync reservation - no available rooms or already synced" });
1174 %         }
1175 %         res.json({ success: true, reservation });
1176 %     } catch (error) {
1177 %         res.status(500).json({ error: "Error syncing OTA reservation" });
1178 %     }
1179 % });
1180 %
1181 % // Simulate OTA reservation import (for testing)
1182 % app.post("/api/ota-channels/:id/simulate-import", async (req, res) => {
1183 %     try {
1184 %         const channel = await storage.getOTAChannel(req.params.id);
1185 %         if (!channel) {
1186 %             return res.status(404).json({ error: "OTA channel not found" });
1187 %         }
1188 %
1189 %         const today = new Date();
1190 %         const checkIn = new Date(today.getTime() + (Math.floor(Math.random() * 7) + 1) * 86400000);
1191 %         const nights = Math.floor(Math.random() * 5) + 1;
1192 %         const checkOut = new Date(checkIn.getTime() + nights * 86400000);
1193 %         const totalAmount = (Math.floor(Math.random() * 200) + 50) * nights;
1194 %         const commission = totalAmount * parseFloat(channel.commissionPercent || "15") / 100;
1195 %
1196 %         const guestNames = ["Maria Rodriguez", "John Smith", "Carlos Gonzalez", "Sophie Martin", "Hans Mueller", "Yuki ..."];
1197 %         const roomTypes = ["Standard", "Doble", "Suite", "Familiar"];
1198 %
1199 %         const log = await storage.createOTAReservationLog({
1200 %             channelId: channel.id,
1201 %             externalReservationId: `${channel.channelType.toUpperCase()}-${Date.now()}`,
1202 %             guestName: guestNames[Math.floor(Math.random() * guestNames.length)],
1203 %             checkInDate: checkIn.toISOString().split("T")[0],
1204 %             checkOutDate: checkOut.toISOString().split("T")[0],
1205 %             roomTypeName: roomTypes[Math.floor(Math.random() * roomTypes.length)],
1206 %             totalAmount: totalAmount.toFixed(2),
```


server/routes.ts (cont.)

```
1207 %         commission: commission.toFixed(2),
1208 %         netAmount: (totalAmount - commission).toFixed(2),
1209 %         status: "pending",
1210 %         rawData: null,
1211 %         syncedAt: null,
1212 %         createdAt: new Date().toISOString(),
1213 %     });
1214 %
1215 %     res.status(201).json(log);
1216 % } catch (error) {
1217 %     res.status(500).json({ error: "Error simulating OTA import" });
1218 % }
1219 % });
1220 %
1221 % // Groups
1222 % app.get("/api/groups", async (req, res) => {
1223 %     try {
1224 %         const groups = await storage.getGroups();
1225 %         res.json(groups);
1226 %     } catch (error) {
1227 %         res.status(500).json({ error: "Error fetching groups" });
1228 %     }
1229 % });
1230 %
1231 % app.get("/api/groups/:id", async (req, res) => {
1232 %     try {
1233 %         const group = await storage.getGroup(req.params.id);
1234 %         if (!group) {
1235 %             return res.status(404).json({ error: "Group not found" });
1236 %         }
1237 %         res.json(group);
1238 %     } catch (error) {
1239 %         res.status(500).json({ error: "Error fetching group" });
1240 %     }
1241 % });
1242 %
1243 % app.post("/api/groups", async (req, res) => {
1244 %     try {
1245 %         const { name, contactName, contactPhone, contactEmail, eventDate, checkInDate, checkOutDate, status, releaseDat...
1246 %
1247 %         if (!name || !checkInDate || !checkOutDate) {
1248 %             return res.status(400).json({ error: "Name, checkInDate, and checkOutDate are required" });
1249 %         }
1250 %
1251 %         const groupCode = storage.generateGroupCode();
1252 %         const group = await storage.createGroup({
1253 %             groupCode,
1254 %             name,
1255 %             contactName: contactName || null,
1256 %             contactPhone: contactPhone || null,
1257 %             contactEmail: contactEmail || null,
1258 %             eventDate: eventDate || null,
1259 %             checkInDate,
1260 %             checkOutDate,
1261 %             status: status || "tentative",
1262 %             releaseDate: releaseDate || null,
1263 %             notes: notes || null,
1264 %             createdAt: new Date().toISOString(),
1265 %             createdBy: null,
1266 %         });
1267 %         res.status(201).json(group);
1268 %     } catch (error) {
1269 %         res.status(500).json({ error: "Error creating group" });
1270 %     }
1271 % });
1272 %
1273 % app.patch("/api/groups/:id", async (req, res) => {
1274 %     try {
1275 %         const { name, contactName, contactPhone, contactEmail, eventDate, checkInDate, checkOutDate, status, releaseDat...
1276 %         const updateData: Record<string, unknown> = {};
1277 %
1278 %         if (name !== undefined) updateData.name = name;
1279 %         if (contactName !== undefined) updateData.contactName = contactName;
1280 %         if (contactPhone !== undefined) updateData.contactPhone = contactPhone;
1281 %         if (contactEmail !== undefined) updateData.contactEmail = contactEmail;
1282 %         if (eventDate !== undefined) updateData.eventDate = eventDate;
1283 %         if (checkInDate !== undefined) updateData.checkInDate = checkInDate;
1284 %         if (checkOutDate !== undefined) updateData.checkOutDate = checkOutDate;
1285 %         if (status !== undefined) updateData.status = status;
1286 %         if (releaseDate !== undefined) updateData.releaseDate = releaseDate;
1287 %         if (notes !== undefined) updateData.notes = notes;
1288 %
1289 %         const group = await storage.updateGroup(req.params.id, updateData);
1290 %         if (!group) {
1291 %             return res.status(404).json({ error: "Group not found" });
1292 %         }
1293 %         res.json(group);
1294 %     } catch (error) {
1295 %         res.status(500).json({ error: "Error updating group" });
1296 %     }
1297 % });
1298 %
1299 % app.delete("/api/groups/:id", async (req, res) => {
```

server/routes.ts (cont.)

```
1300 %     try {
1301 %         const deleted = await storage.deleteGroup(req.params.id);
1302 %         if (!deleted) {
1303 %             return res.status(404).json({ error: "Group not found" });
1304 %         }
1305 %         res.status(204).send();
1306 %     } catch (error) {
1307 %         res.status(500).json({ error: "Error deleting group" });
1308 %     }
1309 % });
1310 %
1311 % // Group Room Blocks
1312 % app.get("/api/groups/:groupId/blocks", async (req, res) => {
1313 %     try {
1314 %         const blocks = await storage.getGroupBlocks(req.params.groupId);
1315 %         res.json(blocks);
1316 %     } catch (error) {
1317 %         res.status(500).json({ error: "Error fetching group blocks" });
1318 %     }
1319 % });
1320 %
1321 % app.post("/api/groups/:groupId/blocks", async (req, res) => {
1322 %     try {
1323 %         const { roomId, quantity, ratePlanId, agreedRate, blockCheckInDate, blockCheckOutDate } = req.body;
1324 %
1325 %         if (!roomId || quantity === undefined) {
1326 %             return res.status(400).json({ error: "roomId and quantity are required" });
1327 %         }
1328 %
1329 %         const block = await storage.createGroupBlock({
1330 %             groupId: req.params.groupId,
1331 %             roomId,
1332 %             quantity: typeof quantity === 'number' ? quantity : parseInt(quantity, 10),
1333 %             ratePlanId: ratePlanId || null,
1334 %             agreedRate: agreedRate ? String(agreedRate) : null,
1335 %             blockCheckInDate: blockCheckInDate || null,
1336 %             blockCheckOutDate: blockCheckOutDate || null,
1337 %         });
1338 %         res.status(201).json(block);
1339 %     } catch (error) {
1340 %         res.status(500).json({ error: "Error creating group block" });
1341 %     }
1342 % });
1343 %
1344 % app.patch("/api/group-blocks/:id", async (req, res) => {
1345 %     try {
1346 %         const block = await storage.updateGroupBlock(req.params.id, req.body);
1347 %         if (!block) {
1348 %             return res.status(404).json({ error: "Group block not found" });
1349 %         }
1350 %         res.json(block);
1351 %     } catch (error) {
1352 %         res.status(500).json({ error: "Error updating group block" });
1353 %     }
1354 % });
1355 %
1356 % app.delete("/api/group-blocks/:id", async (req, res) => {
1357 %     try {
1358 %         const deleted = await storage.deleteGroupBlock(req.params.id);
1359 %         if (!deleted) {
1360 %             return res.status(404).json({ error: "Group block not found" });
1361 %         }
1362 %         res.status(204).send();
1363 %     } catch (error) {
1364 %         res.status(500).json({ error: "Error deleting group block" });
1365 %     }
1366 % });
1367 %
1368 % // Group Room Assignment
1369 % app.post("/api/groups/:groupId/assign-room", async (req, res) => {
1370 %     try {
1371 %         const { roomId, guestFirstName, guestLastName, checkInDate, checkOutDate, agreedRate, ratePlanId } = req.body;
1372 %         if (!roomId || !guestFirstName || !guestLastName) {
1373 %             return res.status(400).json({ error: "Room ID, guest first name, and guest last name are required" });
1374 %         }
1375 %         const reservation = await storage.assignRoomToGroup(
1376 %             req.params.groupId,
1377 %             roomId,
1378 %             guestFirstName,
1379 %             guestLastName,
1380 %             {
1381 %                 checkInDate: checkInDate || undefined,
1382 %                 checkOutDate: checkOutDate || undefined,
1383 %                 agreedRate: agreedRate ? String(agreedRate) : undefined,
1384 %                 ratePlanId: ratePlanId !== undefined ? ratePlanId : undefined,
1385 %             }
1386 %         );
1387 %         if (!reservation) {
1388 %             return res.status(400).json({ error: "Could not assign room to group" });
1389 %         }
1390 %         res.status(201).json(reservation);
1391 %     } catch (error) {
1392 %         res.status(500).json({ error: "Error assigning room to group" });
1393 %     }
1394 % });
```

server/routes.ts (cont.)

```
1393 %     }
1394 %   });
1395 %
1396 % // Group Mass Actions - Check-in all group reservations
1397 % app.post("/api/groups/:groupId/check-in-all", async (req, res) => {
1398 %   try {
1399 %     const group = await storage.getGroup(req.params.groupId);
1400 %     if (!group) {
1401 %       return res.status(404).json({ error: "Group not found" });
1402 %     }
1403 %
1404 %     const results = { success: 0, failed: 0, errors: [] as string[] };
1405 %
1406 %     for (const reservation of group.reservations) {
1407 %       if (reservation.status === "confirmed") {
1408 %         // Verify room is available for check-in
1409 %         const room = await storage.getRoom(reservation.roomId);
1410 %         if (room && room.status === "available") {
1411 %           await storage.updateReservation(reservation.id, { status: "checked_in" });
1412 %           await storage.updateRoom(reservation.roomId, { status: "occupied" });
1413 %           results.success++;
1414 %         } else {
1415 %           results.failed++;
1416 %           results.errors.push(`Hab. ${room?.roomNumber || reservation.roomId}: no disponible para check-in`);
1417 %         }
1418 %       } else if (reservation.status === "checked_in") {
1419 %         // Already checked in, count as success
1420 %         results.success++;
1421 %       }
1422 %     }
1423 %
1424 %     // Update group status to inhouse if any successful check-ins
1425 %     if (results.success > 0) {
1426 %       await storage.updateGroup(req.params.groupId, { status: "inhouse" });
1427 %     }
1428 %
1429 %     res.json(results);
1430 %   } catch (error) {
1431 %     res.status(500).json({ error: "Error processing group check-in" });
1432 %   }
1433 % });
1434 %
1435 % // Group Mass Actions - Check-out all group reservations
1436 % app.post("/api/groups/:groupId/check-out-all", async (req, res) => {
1437 %   try {
1438 %     const group = await storage.getGroup(req.params.groupId);
1439 %     if (!group) {
1440 %       return res.status(404).json({ error: "Group not found" });
1441 %     }
1442 %
1443 %     const results = { success: 0, failed: 0, errors: [] as string[] };
1444 %
1445 %     for (const reservation of group.reservations) {
1446 %       if (reservation.status === "checked_in") {
1447 %         const charges = await storage.getCharges(reservation.id);
1448 %         const payments = await storage.getPayments(reservation.id);
1449 %         const totalCharges = charges.reduce((sum: number, c) => sum + parseFloat(c.amount), 0);
1450 %         const totalPayments = payments.reduce((sum: number, p) => sum + parseFloat(p.amount), 0);
1451 %         const roomTotal = parseFloat(reservation.totalRoomAmount || "0");
1452 %         const grandTotal = roomTotal + totalCharges;
1453 %         const balance = grandTotal - totalPayments;
1454 %
1455 %         if (balance > 0.01) {
1456 %           results.failed++;
1457 %           results.errors.push(`Hab. ${reservation.room?.roomNumber}: saldo pendiente ${balance.toFixed(2)}`);
1458 %         } else {
1459 %           await storage.updateReservation(reservation.id, { status: "checked_out" });
1460 %           await storage.updateRoom(reservation.roomId, { status: "cleaning" });
1461 %           try {
1462 %             await storage.createHousekeepingTask({
1463 %               roomId: reservation.roomId,
1464 %               type: "checkout",
1465 %               status: "pending",
1466 %               priority: "high",
1467 %               notes: `Check-out grupal - ${group.name}`,
1468 %             });
1469 %           } catch {}
1470 %           results.success++;
1471 %         }
1472 %       } else if (reservation.status === "checked_out") {
1473 %         results.success++;
1474 %       }
1475 %     }
1476 %
1477 %     // Update group status to finished if all checked out
1478 %     const allCheckedOut = group.reservations.every(r =>
1479 %       r.status === "checked_out" || r.status === "cancelled"
1480 %     ) || (results.success === group.reservations.length);
1481 %
1482 %     if (allCheckedOut && group.reservations.length > 0) {
1483 %       await storage.updateGroup(req.params.groupId, { status: "finished" });
1484 %     }
1485 %   }
```

server/routes.ts (cont.)

```
1486 %         res.json(results);
1487 %     } catch (error) {
1488 %         res.status(500).json({ error: "Error processing group check-out" });
1489 %     }
1490 % });
1491 %
1492 % // Group Invoice - Get consolidated invoice data for the group
1493 % app.get("/api/groups/:groupId/invoice", async (req, res) => {
1494 %     try {
1495 %         const group = await storage.getGroup(req.params.groupId);
1496 %         if (!group) {
1497 %             return res.status(404).json({ error: "Group not found" });
1498 %         }
1499 %
1500 %         const invoiceData = {
1501 %             group: {
1502 %                 code: group.groupCode,
1503 %                 name: group.name,
1504 %                 contactName: group.contactName,
1505 %                 contactPhone: group.contactPhone,
1506 %                 contactEmail: group.contactEmail,
1507 %                 checkInDate: group.checkInDate,
1508 %                 checkOutDate: group.checkOutDate,
1509 %             },
1510 %             reservations: [] as any[],
1511 %             totals: {
1512 %                 accommodation: 0,
1513 %                 charges: 0,
1514 %                 payments: 0,
1515 %                 balance: 0,
1516 %             }
1517 %         };
1518 %
1519 %         for (const reservation of group.reservations) {
1520 %             const charges = await storage.getCharges(reservation.id);
1521 %             const payments = await storage.getPayments(reservation.id);
1522 %
1523 %             // Calculate nights and accommodation cost
1524 %             const checkIn = new Date(reservation.checkInDate);
1525 %             const checkOut = new Date(reservation.checkOutDate);
1526 %             const nights = Math.ceil((checkOut.getTime() - checkIn.getTime()) / (1000 * 60 * 60 * 24));
1527 %             const rate = parseFloat(reservation.finalRatePerNight || reservation.baseRatePerNight || "0");
1528 %             const accommodationTotal = nights * rate;
1529 %
1530 %             const chargesTotal = charges.reduce((sum: number, c) => sum + parseFloat(c.amount), 0);
1531 %             const paymentsTotal = payments.reduce((sum: number, p) => sum + parseFloat(p.amount), 0);
1532 %             const totalCost = accommodationTotal + chargesTotal;
1533 %
1534 %             invoiceData.reservations.push({
1535 %                 reservationCode: reservation.reservationCode,
1536 %                 guest: `${reservation.guest?.firstName} ${reservation.guest?.lastName}`,
1537 %                 room: reservation.room?.roomNumber,
1538 %                 nights,
1539 %                 ratePerNight: rate,
1540 %                 accommodationTotal,
1541 %                 charges: charges.map(c => ({
1542 %                     description: c.description,
1543 %                     amount: parseFloat(c.amount),
1544 %                     category: c.category,
1545 %                     date: c.date,
1546 %                 })),
1547 %                 chargesTotal,
1548 %                 payments: payments.map(p => ({
1549 %                     method: p.method,
1550 %                     amount: parseFloat(p.amount),
1551 %                     date: p.date,
1552 %                     reference: p.reference,
1553 %                 })),
1554 %                 paymentsTotal,
1555 %                 balance: totalCost - paymentsTotal,
1556 %             });
1557 %
1558 %             invoiceData.totals.accommodation += accommodationTotal;
1559 %             invoiceData.totals.charges += chargesTotal;
1560 %             invoiceData.totals.payments += paymentsTotal;
1561 %         }
1562 %
1563 %         invoiceData.totals.balance = invoiceData.totals.accommodation + invoiceData.totals.charges - invoiceData.totals...
1564 %
1565 %         res.json(invoiceData);
1566 %     } catch (error) {
1567 %         res.status(500).json({ error: "Error generating group invoice" });
1568 %     }
1569 % });
1570 %
1571 % app.post("/api/groups/:groupId/payment", async (req, res) => {
1572 %     try {
1573 %         const group = await storage.getGroup(req.params.groupId);
1574 %         if (!group) {
1575 %             return res.status(404).json({ error: "Group not found" });
1576 %         }
1577 %
1578 %         const { amount, method, reference, receiptType, distribution } = req.body;
```

server/routes.ts (cont.)

```

1579 %         if (!amount || !method) {
1580 %             return res.status(400).json({ error: "amount and method are required" });
1581 %         }
1582 %
1583 %         const totalAmount = parseFloat(amount);
1584 %         if (totalAmount <= 0) {
1585 %             return res.status(400).json({ error: "Amount must be positive" });
1586 %         }
1587 %
1588 %         const checkedInReservations = group.reservations.filter(
1589 %             r => r.status === "checked_in"
1590 %         );
1591 %
1592 %         if (checkedInReservations.length === 0) {
1593 %             return res.status(400).json({ error: "No hay reservas en casa para registrar pagos" });
1594 %         }
1595 %
1596 %         if (distribution === "equal") {
1597 %             const perRoom = totalAmount / checkedInReservations.length;
1598 %             for (const reservation of checkedInReservations) {
1599 %                 await storage.createPayment({
1600 %                     reservationId: reservation.id,
1601 %                     amount: perRoom.toFixed(2),
1602 %                     method,
1603 %                     reference: reference || `Pago grupal - ${group.name}`,
1604 %                     date: new Date().toISOString().split("T")[0],
1605 %                 });
1606 %             }
1607 %         } else if (distribution === "proportional") {
1608 %             let totalCost = 0;
1609 %             const costs: { id: string; cost: number }[] = [];
1610 %             for (const reservation of checkedInReservations) {
1611 %                 const charges = await storage.getCharges(reservation.id);
1612 %                 const chargesTotal = charges.reduce((sum: number, c) => sum + parseFloat(c.amount), 0);
1613 %                 const roomTotal = parseFloat(reservation.totalRoomAmount || "0");
1614 %                 const cost = roomTotal + chargesTotal;
1615 %                 costs.push({ id: reservation.id, cost });
1616 %                 totalCost += cost;
1617 %             }
1618 %             for (const item of costs) {
1619 %                 const proportion = totalCost > 0 ? item.cost / totalCost : 1 / costs.length;
1620 %                 const paymentAmount = (totalAmount * proportion).toFixed(2);
1621 %                 await storage.createPayment({
1622 %                     reservationId: item.id,
1623 %                     amount: paymentAmount,
1624 %                     method,
1625 %                     reference: reference || `Pago grupal - ${group.name}`,
1626 %                     date: new Date().toISOString().split("T")[0],
1627 %                 });
1628 %             }
1629 %         } else {
1630 %             const perRoom = totalAmount / checkedInReservations.length;
1631 %             for (const reservation of checkedInReservations) {
1632 %                 await storage.createPayment({
1633 %                     reservationId: reservation.id,
1634 %                     amount: perRoom.toFixed(2),
1635 %                     method,
1636 %                     reference: reference || `Pago grupal - ${group.name}`,
1637 %                     date: new Date().toISOString().split("T")[0],
1638 %                 });
1639 %             }
1640 %         }
1641 %
1642 %         res.json({ success: true, distributed: checkedInReservations.length });
1643 %     } catch (error) {
1644 %         res.status(500).json({ error: "Error processing group payment" });
1645 %     }
1646 % });
1647 %
1648 % // Guest Reviews
1649 % app.get("/api/reviews", async (req, res) => {
1650 %     try {
1651 %         const reviews = await storage.getGuestReviews();
1652 %         res.json(reviews);
1653 %     } catch (error) {
1654 %         res.status(500).json({ error: "Error fetching reviews" });
1655 %     }
1656 % });
1657 %
1658 % app.get("/api/reviews/analytics", async (req, res) => {
1659 %     try {
1660 %         const analytics = await storage.getReviewAnalyticsSummary();
1661 %         res.json(analytics);
1662 %     } catch (error) {
1663 %         res.status(500).json({ error: "Error fetching review analytics" });
1664 %     }
1665 % });
1666 %
1667 % app.get("/api/reviews/:id", async (req, res) => {
1668 %     try {
1669 %         const review = await storage.getGuestReview(req.params.id);
1670 %         if (!review) {
1671 %             return res.status(404).json({ error: "Review not found" });

```

server/routes.ts (cont.)

```
1672 %     }
1673 %     res.json(review);
1674 %   } catch (error) {
1675 %     res.status(500).json({ error: "Error fetching review" });
1676 %   }
1677 % });
1678 %
1679 % app.post("/api/reviews", async (req, res) => {
1680 %   try {
1681 %     const validationResult = insertGuestReviewSchema.safeParse(req.body);
1682 %     if (!validationResult.success) {
1683 %       return res.status(400).json({
1684 %         error: "Invalid review data",
1685 %         details: validationResult.error.errors
1686 %       });
1687 %     }
1688 %     const review = await storage.createGuestReview(validationResult.data);
1689 %     const enrichedReview = await storage.getGuestReview(review.id);
1690 %     res.status(201).json(enrichedReview);
1691 %   } catch (error) {
1692 %     res.status(500).json({ error: "Error creating review" });
1693 %   }
1694 % });
1695 %
1696 % app.patch("/api/reviews/:id", async (req, res) => {
1697 %   try {
1698 %     const partialSchema = insertGuestReviewSchema.partial();
1699 %     const validationResult = partialSchema.safeParse(req.body);
1700 %     if (!validationResult.success) {
1701 %       return res.status(400).json({
1702 %         error: "Invalid review data",
1703 %         details: validationResult.error.errors
1704 %       });
1705 %     }
1706 %     const review = await storage.updateGuestReview(req.params.id, validationResult.data);
1707 %     if (!review) {
1708 %       return res.status(404).json({ error: "Review not found" });
1709 %     }
1710 %     const enrichedReview = await storage.getGuestReview(review.id);
1711 %     res.json(enrichedReview);
1712 %   } catch (error) {
1713 %     res.status(500).json({ error: "Error updating review" });
1714 %   }
1715 % });
1716 %
1717 % app.delete("/api/reviews/:id", async (req, res) => {
1718 %   try {
1719 %     const deleted = await storage.deleteGuestReview(req.params.id);
1720 %     if (!deleted) {
1721 %       return res.status(404).json({ error: "Review not found" });
1722 %     }
1723 %     res.status(204).send();
1724 %   } catch (error) {
1725 %     res.status(500).json({ error: "Error deleting review" });
1726 %   }
1727 % });
1728 %
1729 % // Sentiment Analysis Endpoint
1730 % app.post("/api/reviews/:id/analyze", async (req, res) => {
1731 %   try {
1732 %     if (!process.env.AI_INTEGRATIONS_OPENAI_API_KEY) {
1733 %       return res.status(503).json({ error: "AI service not configured" });
1734 %     }
1735 %
1736 %     const review = await storage.getGuestReview(req.params.id);
1737 %     if (!review) {
1738 %       return res.status(404).json({ error: "Review not found" });
1739 %     }
1740 %
1741 %     const prompt = `Analyze the following hotel guest review and provide sentiment analysis in JSON format.
1742 %
1743 % Review Title: ${review.title || "No title"}
1744 % Review Content: ${review.content}
1745 % Rating: ${review.rating}/5
1746 %
1747 % Respond with a JSON object containing:
1748 % {
1749 %   "sentiment": "positive" | "neutral" | "negative",
1750 %   "sentimentScore": number between 0 and 1 (0 = very negative, 1 = very positive),
1751 %   "categories": array of categories mentioned (from: "service", "cleanliness", "location", "amenities", "value", "foo...
1752 %   "keyPhrases": array of key phrases extracted from the review (max 5),
1753 %   "improvementSuggestions": array of specific improvement suggestions based on any negative aspects (max 3, empty if ...
1754 % }
1755 %
1756 % Only respond with the JSON object, no additional text.`;
1757 %
1758 %     const completion = await openai.chat.completions.create({
1759 %       model: "gpt-4o-mini",
1760 %       messages: [{ role: "user", content: prompt }],
1761 %       max_tokens: 500,
1762 %       temperature: 0.3,
1763 %     });
1764 %   }

```

server/routes.ts (cont.)

```
1765 %     const analysisText = completion.choices[0]?.message?.content || "{}";
1766 %     let analysis;
1767 %     try {
1768 %       analysis = JSON.parse(analysisText);
1769 %     } catch {
1770 %       analysis = {
1771 %         sentiment: review.rating >= 4 ? "positive" : review.rating >= 3 ? "neutral" : "negative",
1772 %         sentimentScore: review.rating / 5,
1773 %         categories: ["general"],
1774 %         keyPhrases: [],
1775 %         improvementSuggestions: [],
1776 %       };
1777 %     }
1778 %
1779 %     const updatedReview = await storage.updateGuestReview(req.params.id, {
1780 %       sentiment: analysis.sentiment,
1781 %       sentimentScore: String(analysis.sentimentScore),
1782 %       categories: analysis.categories,
1783 %       keyPhrases: analysis.keyPhrases,
1784 %       improvementSuggestions: analysis.improvementSuggestions,
1785 %       analyzedAt: new Date().toISOString(),
1786 %     });
1787 %
1788 %     res.json(updatedReview);
1789 %   } catch (error) {
1790 %     console.error("Sentiment analysis error:", error);
1791 %     res.status(500).json({ error: "Error analyzing review sentiment" });
1792 %   }
1793 % });
1794 %
1795 % // Batch analyze all unanalyzed reviews
1796 % app.post("/api/reviews/analyze-all", async (req, res) => {
1797 %   try {
1798 %     if (!process.env.AI_INTEGRATIONS_OPENAI_API_KEY) {
1799 %       return res.status(503).json({ error: "AI service not configured" });
1800 %     }
1801 %
1802 %     const reviews = await storage.getGuestReviews();
1803 %     const unanalyzed = reviews.filter(r => !r.analyzedAt);
1804 %
1805 %     const results = { analyzed: 0, errors: 0 };
1806 %
1807 %     for (const review of unanalyzed) {
1808 %       try {
1809 %         const prompt = `Analyze the following hotel guest review and provide sentiment analysis in JSON format.
1810 %
1811 % Review Title: ${review.title || "No title"}
1812 % Review Content: ${review.content}
1813 % Rating: ${review.rating}/5
1814 %
1815 % Respond with a JSON object containing:
1816 % {
1817 %   "sentiment": "positive" | "neutral" | "negative",
1818 %   "sentimentScore": number between 0 and 1,
1819 %   "categories": array of categories (from: "service", "cleanliness", "location", "amenities", "value", "food", "staff...
1820 %   "keyPhrases": array of key phrases (max 5),
1821 %   "improvementSuggestions": array of improvement suggestions (max 3)
1822 % }
1823 %
1824 % Only respond with the JSON object.`;
1825 %
1826 %         const completion = await openai.chat.completions.create({
1827 %           model: "gpt-4o-mini",
1828 %           messages: [{ role: "user", content: prompt }],
1829 %           max_tokens: 500,
1830 %           temperature: 0.3,
1831 %         });
1832 %
1833 %         const analysisText = completion.choices[0]?.message?.content || "{}";
1834 %         const analysis = JSON.parse(analysisText);
1835 %
1836 %         await storage.updateGuestReview(review.id, {
1837 %           sentiment: analysis.sentiment,
1838 %           sentimentScore: String(analysis.sentimentScore),
1839 %           categories: analysis.categories,
1840 %           keyPhrases: analysis.keyPhrases,
1841 %           improvementSuggestions: analysis.improvementSuggestions,
1842 %           analyzedAt: new Date().toISOString(),
1843 %         });
1844 %         results.analyzed++;
1845 %       } catch {
1846 %         results.errors++;
1847 %       }
1848 %     }
1849 %
1850 %     res.json({ message: `Analyzed ${results.analyzed} reviews, ${results.errors} errors`, ...results });
1851 %   } catch (error) {
1852 %     res.status(500).json({ error: "Error batch analyzing reviews" });
1853 %   }
1854 % });
1855 %
1856 % // Housekeeping Tasks
1857 % app.get("/api/housekeeping", async (req, res) => {
```

server/routes.ts (cont.)

```
1858 %     try {
1859 %         const date = req.query.date as string | undefined;
1860 %         const tasks = await storage.getHousekeepingTasks(date);
1861 %         res.json(tasks);
1862 %     } catch (error) {
1863 %         res.status(500).json({ error: "Error fetching housekeeping tasks" });
1864 %     }
1865 % });
1866 %
1867 % app.get("/api/housekeeping/:id", async (req, res) => {
1868 %     try {
1869 %         const task = await storage.getHousekeepingTask(req.params.id);
1870 %         if (!task) {
1871 %             return res.status(404).json({ error: "Task not found" });
1872 %         }
1873 %         res.json(task);
1874 %     } catch (error) {
1875 %         res.status(500).json({ error: "Error fetching task" });
1876 %     }
1877 % });
1878 %
1879 % app.post("/api/housekeeping", async (req, res) => {
1880 %     try {
1881 %         const task = await storage.createHousekeepingTask({
1882 %             ...req.body,
1883 %             createdAt: new Date().toISOString(),
1884 %         });
1885 %         res.status(201).json(task);
1886 %     } catch (error) {
1887 %         res.status(500).json({ error: "Error creating housekeeping task" });
1888 %     }
1889 % });
1890 %
1891 % app.patch("/api/housekeeping/:id", async (req, res) => {
1892 %     try {
1893 %         const task = await storage.updateHousekeepingTask(req.params.id, req.body);
1894 %         if (!task) {
1895 %             return res.status(404).json({ error: "Task not found" });
1896 %         }
1897 %         res.json(task);
1898 %     } catch (error) {
1899 %         res.status(500).json({ error: "Error updating task" });
1900 %     }
1901 % });
1902 %
1903 % app.delete("/api/housekeeping/:id", async (req, res) => {
1904 %     try {
1905 %         const deleted = await storage.deleteHousekeepingTask(req.params.id);
1906 %         if (!deleted) {
1907 %             return res.status(404).json({ error: "Task not found" });
1908 %         }
1909 %         res.status(204).send();
1910 %     } catch (error) {
1911 %         res.status(500).json({ error: "Error deleting task" });
1912 %     }
1913 % });
1914 %
1915 % // Update room status (for housekeeping)
1916 % app.patch("/api/housekeeping/room/:roomId/status", async (req, res) => {
1917 %     try {
1918 %         const { status } = req.body;
1919 %         const room = await storage.updateRoom(req.params.roomId, { status });
1920 %         if (!room) {
1921 %             return res.status(404).json({ error: "Room not found" });
1922 %         }
1923 %         res.json(room);
1924 %     } catch (error) {
1925 %         res.status(500).json({ error: "Error updating room status" });
1926 %     }
1927 % });
1928 %
1929 % // Start a task (change status to in_progress)
1930 % app.post("/api/housekeeping/:id/start", async (req, res) => {
1931 %     try {
1932 %         const task = await storage.updateHousekeepingTask(req.params.id, {
1933 %             status: "in_progress",
1934 %             startedAt: new Date().toISOString(),
1935 %         });
1936 %         if (!task) {
1937 %             return res.status(404).json({ error: "Task not found" });
1938 %         }
1939 %         // Also update room status to cleaning
1940 %         await storage.updateRoom(task.roomId, { status: "cleaning" });
1941 %         res.json(task);
1942 %     } catch (error) {
1943 %         res.status(500).json({ error: "Error starting task" });
1944 %     }
1945 % });
1946 %
1947 % // Complete a task (change status to completed)
1948 % app.post("/api/housekeeping/:id/complete", async (req, res) => {
1949 %     try {
1950 %         const task = await storage.updateHousekeepingTask(req.params.id, {
```


server/routes.ts (cont.)

```
1951 %         status: "completed",
1952 %         completedAt: new Date().toISOString(),
1953 %     });
1954 %     if (!task) {
1955 %         return res.status(404).json({ error: "Task not found" });
1956 %     }
1957 %     // Mark room as available
1958 %     await storage.updateRoom(task.roomId, { status: "available" });
1959 %     res.json(task);
1960 % } catch (error) {
1961 %     res.status(500).json({ error: "Error completing task" });
1962 % }
1963 % });
1964 %
1965 % // Inspect a task (for supervisor)
1966 % app.post("/api/housekeeping/:id/inspect", async (req, res) => {
1967 %     try {
1968 %         const { inspectedBy } = req.body;
1969 %         const task = await storage.updateHousekeepingTask(req.params.id, {
1970 %             status: "inspected",
1971 %             inspectedBy,
1972 %             inspectedAt: new Date().toISOString(),
1973 %         });
1974 %         if (!task) {
1975 %             return res.status(404).json({ error: "Task not found" });
1976 %         }
1977 %         res.json(task);
1978 %     } catch (error) {
1979 %         res.status(500).json({ error: "Error inspecting task" });
1980 %     }
1981 % });
1982 %
1983 % // ===== RESTAURANT MODULE =====
1984 %
1985 % // Restaurant Areas
1986 % app.get("/api/restaurant/areas", async (req, res) => {
1987 %     try {
1988 %         const areas = await storage.getRestaurantAreas();
1989 %         res.json(areas);
1990 %     } catch (error) {
1991 %         res.status(500).json({ error: "Error fetching areas" });
1992 %     }
1993 % });
1994 %
1995 % app.post("/api/restaurant/areas", async (req, res) => {
1996 %     try {
1997 %         const area = await storage.createRestaurantArea(req.body);
1998 %         res.status(201).json(area);
1999 %     } catch (error) {
2000 %         res.status(500).json({ error: "Error creating area" });
2001 %     }
2002 % });
2003 %
2004 % app.patch("/api/restaurant/areas/:id", async (req, res) => {
2005 %     try {
2006 %         const area = await storage.updateRestaurantArea(req.params.id, req.body);
2007 %         if (!area) return res.status(404).json({ error: "Area not found" });
2008 %         res.json(area);
2009 %     } catch (error) {
2010 %         res.status(500).json({ error: "Error updating area" });
2011 %     }
2012 % });
2013 %
2014 % app.delete("/api/restaurant/areas/:id", async (req, res) => {
2015 %     try {
2016 %         await storage.deleteRestaurantArea(req.params.id);
2017 %         res.status(204).send();
2018 %     } catch (error) {
2019 %         res.status(500).json({ error: "Error deleting area" });
2020 %     }
2021 % });
2022 %
2023 % // Restaurant Tables
2024 % app.get("/api/restaurant/tables", async (req, res) => {
2025 %     try {
2026 %         const tables = await storage.getRestaurantTables();
2027 %         res.json(tables);
2028 %     } catch (error) {
2029 %         res.status(500).json({ error: "Error fetching tables" });
2030 %     }
2031 % });
2032 %
2033 % app.post("/api/restaurant/tables", async (req, res) => {
2034 %     try {
2035 %         const table = await storage.createRestaurantTable(req.body);
2036 %         res.status(201).json(table);
2037 %     } catch (error) {
2038 %         res.status(500).json({ error: "Error creating table" });
2039 %     }
2040 % });
2041 %
2042 % app.patch("/api/restaurant/tables/:id", async (req, res) => {
2043 %     try {
```

server/routes.ts (cont.)

```
2044 %     const table = await storage.updateRestaurantTable(req.params.id, req.body);
2045 %     if (!table) return res.status(404).json({ error: "Table not found" });
2046 %     res.json(table);
2047 %   } catch (error) {
2048 %     res.status(500).json({ error: "Error updating table" });
2049 %   }
2050 % });
2051 %
2052 % app.delete("/api/restaurant/tables/:id", async (req, res) => {
2053 %   try {
2054 %     await storage.deleteRestaurantTable(req.params.id);
2055 %     res.status(204).send();
2056 %   } catch (error) {
2057 %     res.status(500).json({ error: "Error deleting table" });
2058 %   }
2059 % });
2060 %
2061 % // Menu Categories
2062 % app.get("/api/restaurant/menu/categories", async (req, res) => {
2063 %   try {
2064 %     const categories = await storage.getMenuCategories();
2065 %     res.json(categories);
2066 %   } catch (error) {
2067 %     res.status(500).json({ error: "Error fetching menu categories" });
2068 %   }
2069 % });
2070 %
2071 % app.post("/api/restaurant/menu/categories", async (req, res) => {
2072 %   try {
2073 %     const category = await storage.createMenuCategory(req.body);
2074 %     res.status(201).json(category);
2075 %   } catch (error) {
2076 %     res.status(500).json({ error: "Error creating category" });
2077 %   }
2078 % });
2079 %
2080 % app.patch("/api/restaurant/menu/categories/:id", async (req, res) => {
2081 %   try {
2082 %     const category = await storage.updateMenuCategory(req.params.id, req.body);
2083 %     if (!category) return res.status(404).json({ error: "Category not found" });
2084 %     res.json(category);
2085 %   } catch (error) {
2086 %     res.status(500).json({ error: "Error updating category" });
2087 %   }
2088 % });
2089 %
2090 % app.delete("/api/restaurant/menu/categories/:id", async (req, res) => {
2091 %   try {
2092 %     await storage.deleteMenuCategory(req.params.id);
2093 %     res.status(204).send();
2094 %   } catch (error) {
2095 %     res.status(500).json({ error: "Error deleting category" });
2096 %   }
2097 % });
2098 %
2099 % // Menu Items
2100 % app.get("/api/restaurant/menu/items", async (req, res) => {
2101 %   try {
2102 %     const items = await storage.getMenuItems();
2103 %     res.json(items);
2104 %   } catch (error) {
2105 %     res.status(500).json({ error: "Error fetching menu items" });
2106 %   }
2107 % });
2108 %
2109 % app.post("/api/restaurant/menu/items", async (req, res) => {
2110 %   try {
2111 %     const item = await storage.createMenuItem(req.body);
2112 %     res.status(201).json(item);
2113 %   } catch (error) {
2114 %     res.status(500).json({ error: "Error creating menu item" });
2115 %   }
2116 % });
2117 %
2118 % app.patch("/api/restaurant/menu/items/:id", async (req, res) => {
2119 %   try {
2120 %     const item = await storage.updateMenuItem(req.params.id, req.body);
2121 %     if (!item) return res.status(404).json({ error: "Item not found" });
2122 %     res.json(item);
2123 %   } catch (error) {
2124 %     res.status(500).json({ error: "Error updating menu item" });
2125 %   }
2126 % });
2127 %
2128 % app.delete("/api/restaurant/menu/items/:id", async (req, res) => {
2129 %   try {
2130 %     await storage.deleteMenuItem(req.params.id);
2131 %     res.status(204).send();
2132 %   } catch (error) {
2133 %     res.status(500).json({ error: "Error deleting menu item" });
2134 %   }
2135 % });
2136 %
```

server/routes.ts (cont.)

```

2137 % // Restaurant Orders
2138 % app.get("/api/restaurant/orders", async (req, res) => {
2139 %   try {
2140 %     const status = req.query.status as string | undefined;
2141 %     const orders = await storage.getRestaurantOrders(status as any);
2142 %     const ordersWithSplits = await Promise.all(
2143 %       orders.map(async (order: any) => {
2144 %         const splits = await storage.getOrderSplits(order.id);
2145 %         return { ...order, splits };
2146 %       })
2147 %     );
2148 %     res.json(ordersWithSplits);
2149 %   } catch (error) {
2150 %     res.status(500).json({ error: "Error fetching orders" });
2151 %   }
2152 % });
2153 %
2154 % app.get("/api/restaurant/orders/:id", async (req, res) => {
2155 %   try {
2156 %     const order = await storage.getRestaurantOrder(req.params.id);
2157 %     if (!order) return res.status(404).json({ error: "Order not found" });
2158 %     res.json(order);
2159 %   } catch (error) {
2160 %     res.status(500).json({ error: "Error fetching order" });
2161 %   }
2162 % });
2163 %
2164 % app.post("/api/restaurant/orders", async (req, res) => {
2165 %   try {
2166 %     const { waiterName, tableId, areaId, orderLabel } = req.body;
2167 %     if (!waiterName || !waiterName.trim()) {
2168 %       return res.status(400).json({ error: "Mozo es requerido" });
2169 %     }
2170 %     if (!tableId && !areaId) {
2171 %       return res.status(400).json({ error: "Se requiere mesa o area" });
2172 %     }
2173 %     if (!tableId && (!orderLabel || !orderLabel.trim())) {
2174 %       return res.status(400).json({ error: "Etiqueta de orden es requerida para areas sin mesas" });
2175 %     }
2176 %     const orderNumber = storage.generateOrderNumber();
2177 %     const order = await storage.createRestaurantOrder({
2178 %       ...req.body,
2179 %       orderNumber,
2180 %       openedAt: new Date().toISOString(),
2181 %     });
2182 %     if (order.tableId) {
2183 %       await storage.updateRestaurantTable(order.tableId, { status: "occupied" });
2184 %     }
2185 %     res.status(201).json(order);
2186 %   } catch (error) {
2187 %     res.status(500).json({ error: "Error creating order" });
2188 %   }
2189 % });
2190 %
2191 % app.patch("/api/restaurant/orders/:id", async (req, res) => {
2192 %   try {
2193 %     const order = await storage.updateRestaurantOrder(req.params.id, req.body);
2194 %     if (!order) return res.status(404).json({ error: "Order not found" });
2195 %     res.json(order);
2196 %   } catch (error) {
2197 %     res.status(500).json({ error: "Error updating order" });
2198 %   }
2199 % });
2200 %
2201 % // Close order and optionally charge to room
2202 % app.post("/api/restaurant/orders/:id/close", async (req, res) => {
2203 %   try {
2204 %     const order = await storage.getRestaurantOrder(req.params.id);
2205 %     if (!order) return res.status(404).json({ error: "Order not found" });
2206 %
2207 %     const { chargeToRoom, roomNumber, reservationId, receiptType, paymentMethod } = req.body;
2208 %
2209 %     // Update order as closed
2210 %     const updatedOrder = await storage.updateRestaurantOrder(req.params.id, {
2211 %       status: "closed",
2212 %       closedAt: new Date().toISOString(),
2213 %       chargedToRoom: chargeToRoom ? "true" : "false",
2214 %       roomNumber: roomNumber || null,
2215 %       receiptType: receiptType || null,
2216 %       paymentMethod: paymentMethod || null,
2217 %     });
2218 %
2219 %     // If charging to room, create a charge on the reservation
2220 %     if (chargeToRoom && reservationId) {
2221 %       await storage.createCharge({
2222 %         reservationId,
2223 %         description: `Restaurante - Pedido ${order.orderNumber}`,
2224 %         amount: order.total || "0",
2225 %         category: "restaurant",
2226 %         date: new Date().toISOString(),
2227 %       });
2228 %     }
2229 %   }

```

server/routes.ts (cont.)

```
2230 % // Free up the table
2231 % if (order.tableId) {
2232 %   await storage.updateRestaurantTable(order.tableId, { status: "available" });
2233 % }
2234 %
2235 % res.json(updatedOrder);
2236 % } catch (error) {
2237 %   res.status(500).json({ error: "Error closing order" });
2238 % }
2239 % });
2240 %
2241 % // Order Items
2242 % app.post("/api/restaurant/orders/:orderId/items", async (req, res) => {
2243 %   try {
2244 %     const { menuItemId, quantity, notes, course } = req.body;
2245 %     const menuItem = await storage.getMenuItem(menuItemId);
2246 %     if (!menuItem) return res.status(404).json({ error: "Menu item not found" });
2247 %
2248 %     const order = await storage.getRestaurantOrder(req.params.orderId);
2249 %     if (!order) return res.status(404).json({ error: "Order not found" });
2250 %
2251 %     const unitPrice = menuItem.price;
2252 %     const subtotal = (parseFloat(unitPrice) * (quantity || 1)).toFixed(2);
2253 %     const itemCourse = course || 1;
2254 %     const activeCourse = order.activeCourse || 1;
2255 %     const itemStatus = itemCourse <= activeCourse ? "pending" : "waiting_course";
2256 %
2257 %     const item = await storage.createOrderItem({
2258 %       orderId: req.params.orderId,
2259 %       menuItemId,
2260 %       quantity: quantity || 1,
2261 %       unitPrice,
2262 %       subtotal,
2263 %       notes,
2264 %       course: itemCourse,
2265 %       status: itemStatus,
2266 %     });
2267 %
2268 %     // Update order totals
2269 %     const orderItems = await storage.getOrderItems(req.params.orderId);
2270 %     const newSubtotal = orderItems.reduce((sum, i) => sum + parseFloat(i.subtotal), 0);
2271 %     const tax = newSubtotal * 0.21; // 21% IVA
2272 %     await storage.updateRestaurantOrder(req.params.orderId, {
2273 %       subtotal: newSubtotal.toFixed(2),
2274 %       tax: tax.toFixed(2),
2275 %       total: (newSubtotal + tax).toFixed(2),
2276 %       status: "in_progress",
2277 %     });
2278 %
2279 %     res.status(201).json(item);
2280 %   } catch (error) {
2281 %     res.status(500).json({ error: "Error adding item to order" });
2282 %   }
2283 % });
2284 %
2285 % // Advance course
2286 % app.post("/api/restaurant/orders/:id/advance-course", async (req, res) => {
2287 %   try {
2288 %     const order = await storage.getRestaurantOrder(req.params.id);
2289 %     if (!order) return res.status(404).json({ error: "Order not found" });
2290 %
2291 %     const currentCourse = order.activeCourse || 1;
2292 %     if (currentCourse >= 3) return res.status(400).json({ error: "Ya se alcanzó el último curso" });
2293 %
2294 %     const newCourse = currentCourse + 1;
2295 %     await storage.updateRestaurantOrder(req.params.id, { activeCourse: newCourse });
2296 %
2297 %     const orderItems = await storage.getOrderItems(req.params.id);
2298 %     let activated = 0;
2299 %     for (const item of orderItems) {
2300 %       if (item.course === newCourse && item.status === "waiting_course") {
2301 %         await storage.updateOrderItem(item.id, { status: "pending" });
2302 %         activated++;
2303 %       }
2304 %     }
2305 %
2306 %     res.json({ activeCourse: newCourse, activatedItems: activated });
2307 %   } catch (error) {
2308 %     res.status(500).json({ error: "Error advancing course" });
2309 %   }
2310 % });
2311 %
2312 % app.delete("/api/restaurant/orders/:orderId/items/:itemId", async (req, res) => {
2313 %   try {
2314 %     await storage.deleteOrderItem(req.params.itemId);
2315 %
2316 %     // Recalculate order totals
2317 %     const orderItems = await storage.getOrderItems(req.params.orderId);
2318 %     const newSubtotal = orderItems.reduce((sum, i) => sum + parseFloat(i.subtotal), 0);
2319 %     const tax = newSubtotal * 0.21;
2320 %     await storage.updateRestaurantOrder(req.params.orderId, {
2321 %       subtotal: newSubtotal.toFixed(2),
2322 %       tax: tax.toFixed(2),
```

server/routes.ts (cont.)

```
2323 %         total: (newSubtotal + tax).toFixed(2),
2324 %     });
2325 %
2326 %     res.status(204).send();
2327 % } catch (error) {
2328 %     res.status(500).json({ error: "Error removing item from order" });
2329 % }
2330 % });
2331 %
2332 % // Table Reservations
2333 % app.get("/api/restaurant/table-reservations", async (req, res) => {
2334 %     try {
2335 %         const { date } = req.query;
2336 %         if (date && typeof date === "string") {
2337 %             const reservations = await storage.getTableReservationsByDate(date);
2338 %             return res.json(reservations);
2339 %         }
2340 %         const reservations = await storage.getTableReservations();
2341 %         res.json(reservations);
2342 %     } catch (error) {
2343 %         res.status(500).json({ error: "Error fetching table reservations" });
2344 %     }
2345 % });
2346 %
2347 % app.get("/api/restaurant/table-reservations/:id", async (req, res) => {
2348 %     try {
2349 %         const reservation = await storage.getTableReservation(req.params.id);
2350 %         if (!reservation) return res.status(404).json({ error: "Reservation not found" });
2351 %         res.json(reservation);
2352 %     } catch (error) {
2353 %         res.status(500).json({ error: "Error fetching table reservation" });
2354 %     }
2355 % });
2356 %
2357 % app.post("/api/restaurant/table-reservations", async (req, res) => {
2358 %     try {
2359 %         const reservation = await storage.createTableReservation({
2360 %             ...req.body,
2361 %             createdAt: new Date().toISOString(),
2362 %         });
2363 %         res.status(201).json(reservation);
2364 %     } catch (error) {
2365 %         res.status(500).json({ error: "Error creating table reservation" });
2366 %     }
2367 % });
2368 %
2369 % app.patch("/api/restaurant/table-reservations/:id", async (req, res) => {
2370 %     try {
2371 %         const reservation = await storage.updateTableReservation(req.params.id, req.body);
2372 %         if (!reservation) return res.status(404).json({ error: "Reservation not found" });
2373 %         res.json(reservation);
2374 %     } catch (error) {
2375 %         res.status(500).json({ error: "Error updating table reservation" });
2376 %     }
2377 % });
2378 %
2379 % app.delete("/api/restaurant/table-reservations/:id", async (req, res) => {
2380 %     try {
2381 %         await storage.deleteTableReservation(req.params.id);
2382 %         res.status(204).send();
2383 %     } catch (error) {
2384 %         res.status(500).json({ error: "Error deleting table reservation" });
2385 %     }
2386 % });
2387 %
2388 % // Restaurant Time Slots
2389 % app.get("/api/restaurant/time-slots", async (req, res) => {
2390 %     try {
2391 %         const slots = await storage.getRestaurantTimeSlots();
2392 %         res.json(slots);
2393 %     } catch (error) {
2394 %         res.status(500).json({ error: "Error fetching time slots" });
2395 %     }
2396 % });
2397 %
2398 % app.post("/api/restaurant/time-slots", async (req, res) => {
2399 %     try {
2400 %         const slot = await storage.createRestaurantTimeSlot(req.body);
2401 %         res.status(201).json(slot);
2402 %     } catch (error) {
2403 %         res.status(500).json({ error: "Error creating time slot" });
2404 %     }
2405 % });
2406 %
2407 % app.patch("/api/restaurant/time-slots/:id", async (req, res) => {
2408 %     try {
2409 %         const slot = await storage.updateRestaurantTimeSlot(req.params.id, req.body);
2410 %         if (!slot) return res.status(404).json({ error: "Time slot not found" });
2411 %         res.json(slot);
2412 %     } catch (error) {
2413 %         res.status(500).json({ error: "Error updating time slot" });
2414 %     }
2415 % });
```

server/routes.ts (cont.)

```
2416 %
2417 % app.delete("/api/restaurant/time-slots/:id", async (req, res) => {
2418 %   try {
2419 %     await storage.deleteRestaurantTimeSlot(req.params.id);
2420 %     res.status(204).send();
2421 %   } catch (error) {
2422 %     res.status(500).json({ error: "Error deleting time slot" });
2423 %   }
2424 % });
2425 %
2426 % // Order Splits
2427 % app.post("/api/restaurant/orders/:id/split", async (req, res) => {
2428 %   try {
2429 %     const order = await storage.getRestaurantOrder(req.params.id);
2430 %     if (!order) return res.status(404).json({ error: "Order not found" });
2431 %     if (order.status === "closed") return res.status(400).json({ error: "La orden ya está cerrada" });
2432 %
2433 %     const existingSplits = await storage.getOrderSplits(req.params.id);
2434 %     if (existingSplits.length > 0) return res.status(400).json({ error: "La orden ya tiene una división activa" });
2435 %
2436 %     const { parts } = req.body;
2437 %     if (!parts || parts < 2) return res.status(400).json({ error: "Se requieren al menos 2 partes" });
2438 %
2439 %     const total = parseFloat(order.total || "0");
2440 %     const baseAmount = Math.floor(total / parts * 100) / 100;
2441 %     const remainder = total - baseAmount * parts;
2442 %
2443 %     const splits = [];
2444 %     for (let i = 1; i <= parts; i++) {
2445 %       const amount = i === parts ? (baseAmount + remainder).toFixed(2) : baseAmount.toFixed(2);
2446 %       const split = await storage.createOrderSplit({
2447 %         orderId: req.params.id,
2448 %         splitNumber: i,
2449 %         amount,
2450 %         createdAt: new Date().toISOString(),
2451 %       });
2452 %       splits.push(split);
2453 %     }
2454 %
2455 %     res.status(201).json(splits);
2456 %   } catch (error) {
2457 %     res.status(500).json({ error: "Error splitting order" });
2458 %   }
2459 % });
2460 %
2461 % app.get("/api/restaurant/orders/:id/split", async (req, res) => {
2462 %   try {
2463 %     const splits = await storage.getOrderSplits(req.params.id);
2464 %     res.json(splits);
2465 %   } catch (error) {
2466 %     res.status(500).json({ error: "Error fetching splits" });
2467 %   }
2468 % });
2469 %
2470 % app.patch("/api/restaurant/orders/:id/split/:splitId", async (req, res) => {
2471 %   try {
2472 %     const { method, receiptType } = req.body;
2473 %     if (!method) return res.status(400).json({ error: "Método de pago requerido" });
2474 %
2475 %     const split = await storage.updateOrderSplit(req.params.splitId, {
2476 %       method,
2477 %       receiptType: receiptType || null,
2478 %       isPaid: "true",
2479 %       paidAt: new Date().toISOString(),
2480 %     });
2481 %     if (!split) return res.status(404).json({ error: "Split not found" });
2482 %
2483 %     const allSplits = await storage.getOrderSplits(req.params.id);
2484 %     const allPaid = allSplits.every(s => s.isPaid === "true");
2485 %
2486 %     if (allPaid) {
2487 %       const order = await storage.getRestaurantOrder(req.params.id);
2488 %       await storage.updateRestaurantOrder(req.params.id, {
2489 %         status: "closed",
2490 %         closedAt: new Date().toISOString(),
2491 %         paymentMethod: method,
2492 %         receiptType: receiptType || null,
2493 %       });
2494 %       if (order?.tableId) {
2495 %         await storage.updateRestaurantTable(order.tableId, { status: "available" });
2496 %       }
2497 %     }
2498 %
2499 %     res.json({ split, allPaid });
2500 %   } catch (error) {
2501 %     res.status(500).json({ error: "Error paying split" });
2502 %   }
2503 % });
2504 %
2505 % app.delete("/api/restaurant/orders/:id/split", async (req, res) => {
2506 %   try {
2507 %     await storage.deleteOrderSplitsByOrder(req.params.id);
2508 %     res.status(204).send();
```

server/routes.ts (cont.)

```
2509 %     } catch (error) {
2510 %       res.status(500).json({ error: "Error cancelling split" });
2511 %     }
2512 %   });
2513 %
2514 %   // Recipes
2515 %   app.get("/api/restaurant/recipes", async (req, res) => {
2516 %     try {
2517 %       const recipes = await storage.getRecipes();
2518 %       res.json(recipes);
2519 %     } catch (error) {
2520 %       res.status(500).json({ error: "Error fetching recipes" });
2521 %     }
2522 %   });
2523 %
2524 %   app.get("/api/restaurant/recipes/:id", async (req, res) => {
2525 %     try {
2526 %       const recipe = await storage.getRecipe(req.params.id);
2527 %       if (!recipe) return res.status(404).json({ error: "Recipe not found" });
2528 %       res.json(recipe);
2529 %     } catch (error) {
2530 %       res.status(500).json({ error: "Error fetching recipe" });
2531 %     }
2532 %   });
2533 %
2534 %   app.get("/api/restaurant/recipes/by-menu-item/:menuItemId", async (req, res) => {
2535 %     try {
2536 %       const recipe = await storage.getRecipeByMenuItem(req.params.menuItemId);
2537 %       res.json(recipe || null);
2538 %     } catch (error) {
2539 %       res.status(500).json({ error: "Error fetching recipe" });
2540 %     }
2541 %   });
2542 %
2543 %   app.post("/api/restaurant/recipes", async (req, res) => {
2544 %     try {
2545 %       const recipe = await storage.createRecipe(req.body);
2546 %       res.status(201).json(recipe);
2547 %     } catch (error) {
2548 %       res.status(500).json({ error: "Error creating recipe" });
2549 %     }
2550 %   });
2551 %
2552 %   app.patch("/api/restaurant/recipes/:id", async (req, res) => {
2553 %     try {
2554 %       const recipe = await storage.updateRecipe(req.params.id, req.body);
2555 %       if (!recipe) return res.status(404).json({ error: "Recipe not found" });
2556 %       res.json(recipe);
2557 %     } catch (error) {
2558 %       res.status(500).json({ error: "Error updating recipe" });
2559 %     }
2560 %   });
2561 %
2562 %   app.delete("/api/restaurant/recipes/:id", async (req, res) => {
2563 %     try {
2564 %       await storage.deleteRecipe(req.params.id);
2565 %       res.status(204).send();
2566 %     } catch (error) {
2567 %       res.status(500).json({ error: "Error deleting recipe" });
2568 %     }
2569 %   });
2570 %
2571 %   // Recipe Ingredients
2572 %   app.get("/api/restaurant/recipes/:recipeId/ingredients", async (req, res) => {
2573 %     try {
2574 %       const ingredients = await storage.getRecipeIngredients(req.params.recipeId);
2575 %       res.json(ingredients);
2576 %     } catch (error) {
2577 %       res.status(500).json({ error: "Error fetching ingredients" });
2578 %     }
2579 %   });
2580 %
2581 %   app.post("/api/restaurant/recipes/:recipeId/ingredients", async (req, res) => {
2582 %     try {
2583 %       const ingredient = await storage.createRecipeIngredient({
2584 %         ...req.body,
2585 %         recipeId: req.params.recipeId,
2586 %       });
2587 %       res.status(201).json(ingredient);
2588 %     } catch (error) {
2589 %       res.status(500).json({ error: "Error creating ingredient" });
2590 %     }
2591 %   });
2592 %
2593 %   app.patch("/api/restaurant/recipe-ingredients/:id", async (req, res) => {
2594 %     try {
2595 %       const ingredient = await storage.updateRecipeIngredient(req.params.id, req.body);
2596 %       if (!ingredient) return res.status(404).json({ error: "Ingredient not found" });
2597 %       res.json(ingredient);
2598 %     } catch (error) {
2599 %       res.status(500).json({ error: "Error updating ingredient" });
2600 %     }
2601 %   });
```

server/routes.ts (cont.)

```
2602 %
2603 % app.delete("/api/restaurant/recipe-ingredients/:id", async (req, res) => {
2604 %     try {
2605 %         await storage.deleteRecipeIngredient(req.params.id);
2606 %         res.status(204).send();
2607 %     } catch (error) {
2608 %         res.status(500).json({ error: "Error deleting ingredient" });
2609 %     }
2610 % });
2611 %
2612 % // ===== INVENTORY MODULE =====
2613 %
2614 % // Item Categories
2615 % app.get("/api/inventory/categories", async (req, res) => {
2616 %     try {
2617 %         let categories = await storage.getItemCategories();
2618 %         const area = req.query.area as string | undefined;
2619 %         if (area) {
2620 %             categories = categories.filter(c => (c as any).area === area);
2621 %         }
2622 %         res.json(categories);
2623 %     } catch (error) {
2624 %         res.status(500).json({ error: "Error fetching categories" });
2625 %     }
2626 % });
2627 %
2628 % app.post("/api/inventory/categories", async (req, res) => {
2629 %     try {
2630 %         const category = await storage.createItemCategory(req.body);
2631 %         res.status(201).json(category);
2632 %     } catch (error) {
2633 %         res.status(500).json({ error: "Error creating category" });
2634 %     }
2635 % });
2636 %
2637 % app.patch("/api/inventory/categories/:id", async (req, res) => {
2638 %     try {
2639 %         const category = await storage.updateItemCategory(req.params.id, req.body);
2640 %         if (!category) return res.status(404).json({ error: "Category not found" });
2641 %         res.json(category);
2642 %     } catch (error) {
2643 %         res.status(500).json({ error: "Error updating category" });
2644 %     }
2645 % });
2646 %
2647 % app.delete("/api/inventory/categories/:id", async (req, res) => {
2648 %     try {
2649 %         await storage.deleteItemCategory(req.params.id);
2650 %         res.status(204).send();
2651 %     } catch (error) {
2652 %         res.status(500).json({ error: "Error deleting category" });
2653 %     }
2654 % });
2655 %
2656 % // Suppliers
2657 % app.get("/api/inventory/suppliers", async (req, res) => {
2658 %     try {
2659 %         const suppliers = await storage.getSuppliers();
2660 %         res.json(suppliers);
2661 %     } catch (error) {
2662 %         res.status(500).json({ error: "Error fetching suppliers" });
2663 %     }
2664 % });
2665 %
2666 % app.post("/api/inventory/suppliers", async (req, res) => {
2667 %     try {
2668 %         const supplier = await storage.createSupplier(req.body);
2669 %         res.status(201).json(supplier);
2670 %     } catch (error) {
2671 %         res.status(500).json({ error: "Error creating supplier" });
2672 %     }
2673 % });
2674 %
2675 % app.patch("/api/inventory/suppliers/:id", async (req, res) => {
2676 %     try {
2677 %         const supplier = await storage.updateSupplier(req.params.id, req.body);
2678 %         if (!supplier) return res.status(404).json({ error: "Supplier not found" });
2679 %         res.json(supplier);
2680 %     } catch (error) {
2681 %         res.status(500).json({ error: "Error updating supplier" });
2682 %     }
2683 % });
2684 %
2685 % app.delete("/api/inventory/suppliers/:id", async (req, res) => {
2686 %     try {
2687 %         await storage.deleteSupplier(req.params.id);
2688 %         res.status(204).send();
2689 %     } catch (error) {
2690 %         res.status(500).json({ error: "Error deleting supplier" });
2691 %     }
2692 % });
2693 %
2694 % // Inventory Items
```


server/routes.ts (cont.)

```
2695 % app.get("/api/inventory/items", async (req, res) => {
2696 %   try {
2697 %     let items = await storage.getInventoryItems();
2698 %     const area = req.query.area as string | undefined;
2699 %     if (area) {
2700 %       items = items.filter(item => {
2701 %         if (item.category && (item.category as any).area === area) return true;
2702 %         return false;
2703 %       });
2704 %     }
2705 %     res.json(items);
2706 %   } catch (error) {
2707 %     res.status(500).json({ error: "Error fetching inventory items" });
2708 %   }
2709 % });
2710 %
2711 % app.get("/api/inventory/items/low-stock", async (req, res) => {
2712 %   try {
2713 %     const items = await storage.getInventoryItemsBelowMinStock();
2714 %     res.json(items);
2715 %   } catch (error) {
2716 %     res.status(500).json({ error: "Error fetching low stock items" });
2717 %   }
2718 % });
2719 %
2720 % app.post("/api/inventory/items", async (req, res) => {
2721 %   try {
2722 %     const item = await storage.createInventoryItem(req.body);
2723 %     res.status(201).json(item);
2724 %   } catch (error) {
2725 %     res.status(500).json({ error: "Error creating inventory item" });
2726 %   }
2727 % });
2728 %
2729 % app.patch("/api/inventory/items/:id", async (req, res) => {
2730 %   try {
2731 %     const item = await storage.updateInventoryItem(req.params.id, req.body);
2732 %     if (!item) return res.status(404).json({ error: "Item not found" });
2733 %     res.json(item);
2734 %   } catch (error) {
2735 %     res.status(500).json({ error: "Error updating inventory item" });
2736 %   }
2737 % });
2738 %
2739 % app.delete("/api/inventory/items/:id", async (req, res) => {
2740 %   try {
2741 %     await storage.deleteInventoryItem(req.params.id);
2742 %     res.status(204).send();
2743 %   } catch (error) {
2744 %     res.status(500).json({ error: "Error deleting inventory item" });
2745 %   }
2746 % });
2747 %
2748 % // Stock Movements
2749 % app.get("/api/inventory/movements", async (req, res) => {
2750 %   try {
2751 %     const itemId = req.query.itemId as string | undefined;
2752 %     const movements = await storage.getStockMovements(itemId);
2753 %     res.json(movements);
2754 %   } catch (error) {
2755 %     res.status(500).json({ error: "Error fetching stock movements" });
2756 %   }
2757 % });
2758 %
2759 % app.post("/api/inventory/movements", async (req, res) => {
2760 %   try {
2761 %     const { itemId, movementType, quantity, notes } = req.body;
2762 %
2763 %     // Get current stock
2764 %     const item = await storage.getInventoryItem(itemId);
2765 %     if (!item) return res.status(404).json({ error: "Item not found" });
2766 %
2767 %     const previousStock = item.currentStock ?? 0;
2768 %     let newStock = previousStock;
2769 %
2770 %     if (movementType === "entrada") {
2771 %       newStock = previousStock + quantity;
2772 %     } else if (movementType === "salida" || movementType === "consumo") {
2773 %       newStock = previousStock - quantity;
2774 %       if (newStock < 0) {
2775 %         return res.status(400).json({ error: "Stock insuficiente" });
2776 %       }
2777 %     } else if (movementType === "ajuste") {
2778 %       newStock = quantity; // Direct adjustment to specified value
2779 %     }
2780 %
2781 %     const movement = await storage.createStockMovement({
2782 %       itemId,
2783 %       movementType,
2784 %       quantity,
2785 %       previousStock,
2786 %       newStock,
2787 %       notes,
```

server/routes.ts (cont.)

```
2788 %         createdAt: new Date().toISOString(),
2789 %     });
2790 %
2791 %     res.status(201).json(movement);
2792 % } catch (error) {
2793 %     res.status(500).json({ error: "Error creating stock movement" });
2794 % }
2795 % });
2796 %
2797 % // ===== SPA =====
2798 %
2799 % // SPA Cabins
2800 % app.get("/api/spa/cabins", async (req, res) => {
2801 %     try {
2802 %         const cabins = await storage.getSpaCabins();
2803 %         res.json(cabins);
2804 %     } catch (error) {
2805 %         res.status(500).json({ error: "Error fetching spa cabins" });
2806 %     }
2807 % });
2808 %
2809 % app.get("/api/spa/cabins/:id", async (req, res) => {
2810 %     try {
2811 %         const cabin = await storage.getSpaCabin(req.params.id);
2812 %         if (!cabin) return res.status(404).json({ error: "Cabin not found" });
2813 %         res.json(cabin);
2814 %     } catch (error) {
2815 %         res.status(500).json({ error: "Error fetching spa cabin" });
2816 %     }
2817 % });
2818 %
2819 % app.post("/api/spa/cabins", async (req, res) => {
2820 %     try {
2821 %         const cabin = await storage.createSpaCabin(req.body);
2822 %         res.status(201).json(cabin);
2823 %     } catch (error) {
2824 %         res.status(500).json({ error: "Error creating spa cabin" });
2825 %     }
2826 % });
2827 %
2828 % app.patch("/api/spa/cabins/:id", async (req, res) => {
2829 %     try {
2830 %         const cabin = await storage.updateSpaCabin(req.params.id, req.body);
2831 %         if (!cabin) return res.status(404).json({ error: "Cabin not found" });
2832 %         res.json(cabin);
2833 %     } catch (error) {
2834 %         res.status(500).json({ error: "Error updating spa cabin" });
2835 %     }
2836 % });
2837 %
2838 % app.delete("/api/spa/cabins/:id", async (req, res) => {
2839 %     try {
2840 %         await storage.deleteSpaCabin(req.params.id);
2841 %         res.status(204).send();
2842 %     } catch (error) {
2843 %         res.status(500).json({ error: "Error deleting spa cabin" });
2844 %     }
2845 % });
2846 %
2847 % // SPA Treatment Categories
2848 % app.get("/api/spa/treatment-categories", async (req, res) => {
2849 %     try {
2850 %         const categories = await storage.getSpaTreatmentCategories();
2851 %         res.json(categories);
2852 %     } catch (error) {
2853 %         res.status(500).json({ error: "Error fetching treatment categories" });
2854 %     }
2855 % });
2856 %
2857 % app.get("/api/spa/treatment-categories/:id", async (req, res) => {
2858 %     try {
2859 %         const category = await storage.getSpaTreatmentCategory(req.params.id);
2860 %         if (!category) return res.status(404).json({ error: "Category not found" });
2861 %         res.json(category);
2862 %     } catch (error) {
2863 %         res.status(500).json({ error: "Error fetching treatment category" });
2864 %     }
2865 % });
2866 %
2867 % app.post("/api/spa/treatment-categories", async (req, res) => {
2868 %     try {
2869 %         const category = await storage.createSpaTreatmentCategory(req.body);
2870 %         res.status(201).json(category);
2871 %     } catch (error) {
2872 %         res.status(500).json({ error: "Error creating treatment category" });
2873 %     }
2874 % });
2875 %
2876 % app.patch("/api/spa/treatment-categories/:id", async (req, res) => {
2877 %     try {
2878 %         const category = await storage.updateSpaTreatmentCategory(req.params.id, req.body);
2879 %         if (!category) return res.status(404).json({ error: "Category not found" });
2880 %         res.json(category);
```

server/routes.ts (cont.)

```
2881 %     } catch (error) {
2882 %       res.status(500).json({ error: "Error updating treatment category" });
2883 %     }
2884 %   });
2885 %
2886 %   app.delete("/api/spa/treatment-categories/:id", async (req, res) => {
2887 %     try {
2888 %       await storage.deleteSpaTreatmentCategory(req.params.id);
2889 %       res.status(204).send();
2890 %     } catch (error) {
2891 %       res.status(500).json({ error: "Error deleting treatment category" });
2892 %     }
2893 %   });
2894 %
2895 %   // SPA Treatments
2896 %   app.get("/api/spa/treatments", async (req, res) => {
2897 %     try {
2898 %       const treatments = await storage.getSpaTreatments();
2899 %       res.json(treatments);
2900 %     } catch (error) {
2901 %       res.status(500).json({ error: "Error fetching treatments" });
2902 %     }
2903 %   });
2904 %
2905 %   app.get("/api/spa/treatments/:id", async (req, res) => {
2906 %     try {
2907 %       const treatment = await storage.getSpaTreatment(req.params.id);
2908 %       if (!treatment) return res.status(404).json({ error: "Treatment not found" });
2909 %       res.json(treatment);
2910 %     } catch (error) {
2911 %       res.status(500).json({ error: "Error fetching treatment" });
2912 %     }
2913 %   });
2914 %
2915 %   app.get("/api/spa/treatments/by-category/:categoryId", async (req, res) => {
2916 %     try {
2917 %       const treatments = await storage.getSpaTreatmentsByCategory(req.params.categoryId);
2918 %       res.json(treatments);
2919 %     } catch (error) {
2920 %       res.status(500).json({ error: "Error fetching treatments by category" });
2921 %     }
2922 %   });
2923 %
2924 %   app.post("/api/spa/treatments", async (req, res) => {
2925 %     try {
2926 %       const treatment = await storage.createSpaTreatment(req.body);
2927 %       res.status(201).json(treatment);
2928 %     } catch (error) {
2929 %       res.status(500).json({ error: "Error creating treatment" });
2930 %     }
2931 %   });
2932 %
2933 %   app.patch("/api/spa/treatments/:id", async (req, res) => {
2934 %     try {
2935 %       const treatment = await storage.updateSpaTreatment(req.params.id, req.body);
2936 %       if (!treatment) return res.status(404).json({ error: "Treatment not found" });
2937 %       res.json(treatment);
2938 %     } catch (error) {
2939 %       res.status(500).json({ error: "Error updating treatment" });
2940 %     }
2941 %   });
2942 %
2943 %   app.delete("/api/spa/treatments/:id", async (req, res) => {
2944 %     try {
2945 %       await storage.deleteSpaTreatment(req.params.id);
2946 %       res.status(204).send();
2947 %     } catch (error) {
2948 %       res.status(500).json({ error: "Error deleting treatment" });
2949 %     }
2950 %   });
2951 %
2952 %   // SPA Appointments
2953 %   app.get("/api/spa/appointments", async (req, res) => {
2954 %     try {
2955 %       const date = req.query.date as string | undefined;
2956 %       const startDate = req.query.startDate as string | undefined;
2957 %       const endDate = req.query.endDate as string | undefined;
2958 %
2959 %       if (startDate && endDate) {
2960 %         const appointments = await storage.getSpaAppointmentsByDateRange(startDate, endDate);
2961 %         return res.json(appointments);
2962 %       }
2963 %
2964 %       const appointments = await storage.getSpaAppointments(date);
2965 %       res.json(appointments);
2966 %     } catch (error) {
2967 %       res.status(500).json({ error: "Error fetching appointments" });
2968 %     }
2969 %   });
2970 %
2971 %   app.get("/api/spa/appointments/weekly-summary", async (req, res) => {
2972 %     try {
2973 %       const startDate = req.query.startDate as string;
```

server/routes.ts (cont.)

```

2974 %     const endDate = req.query.endDate as string;
2975 %     if (!startDate || !endDate) {
2976 %         return res.status(400).json({ error: "startDate and endDate are required" });
2977 %     }
2978 %     const appointments = await storage.getSpaAppointmentsByDateRange(startDate, endDate);
2979 %     const activeStatuses = ["pending", "confirmed", "in_progress"];
2980 %     const summaryMap = new Map<string, { cabinId: string; date: string; count: number }>();
2981 %
2982 %     for (const apt of appointments) {
2983 %         if (!activeStatuses.includes(apt.status)) continue;
2984 %         const key = `${apt.cabinId}_${apt.appointmentDate}`;
2985 %         const existing = summaryMap.get(key);
2986 %         if (existing) {
2987 %             existing.count++;
2988 %         } else {
2989 %             summaryMap.set(key, { cabinId: apt.cabinId, date: apt.appointmentDate, count: 1 });
2990 %         }
2991 %     }
2992 %
2993 %     res.json(Array.from(summaryMap.values()));
2994 % } catch (error) {
2995 %     res.status(500).json({ error: "Error fetching weekly summary" });
2996 % }
2997 % });
2998 %
2999 % app.get("/api/spa/appointments/:id", async (req, res) => {
3000 %     try {
3001 %         const appointment = await storage.getSpaAppointment(req.params.id);
3002 %         if (!appointment) return res.status(404).json({ error: "Appointment not found" });
3003 %         res.json(appointment);
3004 %     } catch (error) {
3005 %         res.status(500).json({ error: "Error fetching appointment" });
3006 %     }
3007 % });
3008 %
3009 % app.post("/api/spa/appointments", async (req, res) => {
3010 %     try {
3011 %         const { cabinId, treatmentId, guestName, guestLastName, guestPhone, guestEmail, reservationId, appointmentDate, ...
3012 %
3013 %         if (!cabinId || !treatmentId || !guestName || !appointmentDate || !startTime || !endTime) {
3014 %             return res.status(400).json({ error: "cabinId, treatmentId, guestName, appointmentDate, startTime, and endTim...
3015 %         }
3016 %
3017 %         const existingAppointments = await storage.getSpaAppointmentsByCabin(cabinId, appointmentDate);
3018 %         const activeStatuses = ["pending", "confirmed", "in_progress"];
3019 %         const newStart = timeToMinutes(startTime);
3020 %         const newEnd = timeToMinutes(endTime);
3021 %
3022 %         for (const existing of existingAppointments) {
3023 %             if (!activeStatuses.includes(existing.status)) continue;
3024 %             const existStart = timeToMinutes(existing.startTime);
3025 %             const existEnd = timeToMinutes(existing.endTime);
3026 %             if (newStart < existEnd && newEnd > existStart) {
3027 %                 const cabin = await storage.getSpaCabin(cabinId);
3028 %                 return res.status(409).json({
3029 %                     error: "Superposición de turno",
3030 %                     message: `El gabinete '${cabin?.name || cabinId}' ya tiene un turno de ${existing.startTime} a ${existing...
3031 %                 });
3032 %             }
3033 %         }
3034 %
3035 %         const appointment = await storage.createSpaAppointment({
3036 %             cabinId,
3037 %             treatmentId,
3038 %             guestName,
3039 %             guestLastName: guestLastName || null,
3040 %             guestPhone: guestPhone || null,
3041 %             guestEmail: guestEmail || null,
3042 %             reservationId: reservationId || null,
3043 %             appointmentDate,
3044 %             startTime,
3045 %             endTime,
3046 %             status: status || "pending",
3047 %             notes: notes || null,
3048 %             createdAt: new Date().toISOString(),
3049 %         });
3050 %
3051 %         const treatment = await storage.getSpaTreatment(treatmentId);
3052 %         if (!treatment) {
3053 %             return res.status(400).json({ error: "Treatment not found" });
3054 %         }
3055 %
3056 %         const fullName = guestLastName ? `${guestName} ${guestLastName}` : guestName;
3057 %         const treatmentPrice = treatment.price || "0";
3058 %
3059 %         const account = await storage.createSpaAccount({
3060 %             appointmentId: appointment.id,
3061 %             guestName: fullName,
3062 %             reservationId: reservationId || null,
3063 %             status: "open",
3064 %             subtotal: treatmentPrice,
3065 %             total: treatmentPrice,
3066 %             notes: null,

```

server/routes.ts (cont.)

```
3067 %         openedAt: new Date().toISOString(),
3068 %         closedAt: null,
3069 %         closedBy: null,
3070 %         chargedTo: null,
3071 %     });
3072 %
3073 %     await storage.createSpaAccountItem({
3074 %         accountId: account.id,
3075 %         description: treatment.name,
3076 %         quantity: 1,
3077 %         unitPrice: treatmentPrice,
3078 %         subtotal: treatmentPrice,
3079 %         itemType: "treatment",
3080 %         notes: null,
3081 %         createdAt: new Date().toISOString(),
3082 %     });
3083 %
3084 %     res.status(201).json(appointment);
3085 % } catch (error) {
3086 %     res.status(500).json({ error: "Error creating appointment" });
3087 % }
3088 % });
3089 %
3090 % app.patch("/api/spa/appointments/:id", async (req, res) => {
3091 %     try {
3092 %         const current = await storage.getSpaAppointment(req.params.id);
3093 %         if (!current) return res.status(404).json({ error: "Appointment not found" });
3094 %
3095 %         const cabinId = req.body.cabinId || current.cabinId;
3096 %         const appointmentDate = req.body.appointmentDate || current.appointmentDate;
3097 %         const startTime = req.body.startTime || current.startTime;
3098 %         const endTime = req.body.endTime || current.endTime;
3099 %
3100 %         if (req.body.cabinId || req.body.startTime || req.body.endTime || req.body.appointmentDate) {
3101 %             const existingAppointments = await storage.getSpaAppointmentsByCabin(cabinId, appointmentDate);
3102 %             const activeStatuses = ["pending", "confirmed", "in_progress"];
3103 %             const newStart = timeToMinutes(startTime);
3104 %             const newEnd = timeToMinutes(endTime);
3105 %
3106 %             for (const existing of existingAppointments) {
3107 %                 if (existing.id === req.params.id) continue;
3108 %                 if (!activeStatuses.includes(existing.status)) continue;
3109 %                 const existStart = timeToMinutes(existing.startTime);
3110 %                 const existEnd = timeToMinutes(existing.endTime);
3111 %                 if (newStart < existEnd && newEnd > existStart) {
3112 %                     const cabin = await storage.getSpaCabin(cabinId);
3113 %                     return res.status(409).json({
3114 %                         error: "Superposición de turno",
3115 %                         message: `El gabinete '${cabin?.name || cabinId}' ya tiene un turno de ${existing.startTime} a ${existi...
3116 %                     });
3117 %                 }
3118 %             }
3119 %         }
3120 %
3121 %         const appointment = await storage.updateSpaAppointment(req.params.id, req.body);
3122 %         if (!appointment) return res.status(404).json({ error: "Appointment not found" });
3123 %         res.json(appointment);
3124 %     } catch (error) {
3125 %         res.status(500).json({ error: "Error updating appointment" });
3126 %     }
3127 % });
3128 %
3129 % app.delete("/api/spa/appointments/:id", async (req, res) => {
3130 %     try {
3131 %         await storage.deleteSpaAppointment(req.params.id);
3132 %         res.status(204).send();
3133 %     } catch (error) {
3134 %         res.status(500).json({ error: "Error deleting appointment" });
3135 %     }
3136 % });
3137 %
3138 % // SPA Accounts
3139 % app.get("/api/spa/accounts", async (req, res) => {
3140 %     try {
3141 %         const status = req.query.status as string | undefined;
3142 %         const accounts = await storage.getSpaAccounts(status as "open" | "closed" | "cancelled" | undefined);
3143 %         res.json(accounts);
3144 %     } catch (error) {
3145 %         res.status(500).json({ error: "Error fetching spa accounts" });
3146 %     }
3147 % });
3148 %
3149 % app.get("/api/spa/accounts/:id", async (req, res) => {
3150 %     try {
3151 %         const account = await storage.getSpaAccount(req.params.id);
3152 %         if (!account) return res.status(404).json({ error: "Account not found" });
3153 %         res.json(account);
3154 %     } catch (error) {
3155 %         res.status(500).json({ error: "Error fetching spa account" });
3156 %     }
3157 % });
3158 %
3159 % app.get("/api/spa/accounts/by-appointment/:appointmentId", async (req, res) => {
```

server/routes.ts (cont.)

```
3160 %     try {
3161 %         const account = await storage.getSpaAccountByAppointment(req.params.appointmentId);
3162 %         if (!account) return res.status(404).json({ error: "Account not found" });
3163 %         res.json(account);
3164 %     } catch (error) {
3165 %         res.status(500).json({ error: "Error fetching spa account" });
3166 %     }
3167 % });
3168 %
3169 % app.post("/api/spa/accounts", async (req, res) => {
3170 %     try {
3171 %         const { appointmentId, guestName, reservationId, notes } = req.body;
3172 %
3173 %         if (!appointmentId || !guestName) {
3174 %             return res.status(400).json({ error: "appointmentId and guestName are required" });
3175 %         }
3176 %
3177 %         const account = await storage.createSpaAccount({
3178 %             appointmentId,
3179 %             guestName,
3180 %             reservationId: reservationId || null,
3181 %             status: "open",
3182 %             subtotal: "0",
3183 %             total: "0",
3184 %             notes: notes || null,
3185 %             openedAt: new Date().toISOString(),
3186 %             closedAt: null,
3187 %             closedBy: null,
3188 %             chargedTo: null,
3189 %         });
3190 %         res.status(201).json(account);
3191 %     } catch (error) {
3192 %         res.status(500).json({ error: "Error creating spa account" });
3193 %     }
3194 % });
3195 %
3196 % app.patch("/api/spa/accounts/:id", async (req, res) => {
3197 %     try {
3198 %         const account = await storage.updateSpaAccount(req.params.id, req.body);
3199 %         if (!account) return res.status(404).json({ error: "Account not found" });
3200 %         res.json(account);
3201 %     } catch (error) {
3202 %         res.status(500).json({ error: "Error updating spa account" });
3203 %     }
3204 % });
3205 %
3206 % app.post("/api/spa/accounts/:id/close", async (req, res) => {
3207 %     try {
3208 %         const { chargedTo, receiptType } = req.body;
3209 %
3210 %         if (!chargedTo || !receiptType) {
3211 %             return res.status(400).json({ error: "chargedTo and receiptType are required" });
3212 %         }
3213 %
3214 %         const accountData = await storage.getSpaAccount(req.params.id);
3215 %         if (!accountData) return res.status(404).json({ error: "Account not found" });
3216 %
3217 %         const totalAmount = accountData.items.reduce((sum, item) => sum + parseFloat(item.subtotal), 0);
3218 %         const totalPaid = accountData.payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3219 %
3220 %         if (totalPaid < totalAmount) {
3221 %             return res.status(400).json({
3222 %                 error: "Saldo pendiente",
3223 %                 message: `Faltan $$${(totalAmount - totalPaid).toFixed(2)} por cobrar antes de cerrar el folio.`
3224 %             });
3225 %         }
3226 %
3227 %         const account = await storage.closeSpaAccount(req.params.id, chargedTo, receiptType);
3228 %         if (!account) return res.status(404).json({ error: "Account not found" });
3229 %         res.json(account);
3230 %     } catch (error) {
3231 %         res.status(500).json({ error: "Error closing spa account" });
3232 %     }
3233 % });
3234 %
3235 % app.get("/api/spa/accounts/:id/payments", async (req, res) => {
3236 %     try {
3237 %         const payments = await storage.getSpaPayments(req.params.id);
3238 %         res.json(payments);
3239 %     } catch (error) {
3240 %         res.status(500).json({ error: "Error fetching payments" });
3241 %     }
3242 % });
3243 %
3244 % app.post("/api/spa/accounts/:id/payments", async (req, res) => {
3245 %     try {
3246 %         const { amount, method, isAdvance, appointmentId, reservationId, notes } = req.body;
3247 %
3248 %         if (!amount || !method) {
3249 %             return res.status(400).json({ error: "amount and method are required" });
3250 %         }
3251 %
3252 %         const payment = await storage.createSpaPayment({
```

server/routes.ts (cont.)

```
3253 %         accountId: req.params.id,
3254 %         amount,
3255 %         method,
3256 %         isAdvance: isAdvance ? "true" : "false",
3257 %         appointmentId: appointmentId || null,
3258 %         reservationId: reservationId || null,
3259 %         notes: notes || null,
3260 %         createdAt: new Date().toISOString(),
3261 %     });
3262 %
3263 %     if (method === "room_charge" && reservationId) {
3264 %         const charge = {
3265 %             reservationId,
3266 %             category: "spa" as const,
3267 %             description: `SPA - Pago ${isAdvance ? "(Seña)" : ""}`,
3268 %             amount: amount,
3269 %             date: new Date().toISOString().split("T")[0],
3270 %             createdBy: null,
3271 %         };
3272 %         await storage.createCharge(charge);
3273 %     }
3274 %
3275 %     res.status(201).json(payment);
3276 % } catch (error) {
3277 %     res.status(500).json({ error: "Error creating payment" });
3278 % }
3279 % });
3280 %
3281 % app.delete("/api/spa/payments/:id", async (req, res) => {
3282 %     try {
3283 %         await storage.deleteSpaPayment(req.params.id);
3284 %         res.status(204).send();
3285 %     } catch (error) {
3286 %         res.status(500).json({ error: "Error deleting payment" });
3287 %     }
3288 % });
3289 %
3290 % // SPA Account Items
3291 % app.get("/api/spa/accounts/:accountId/items", async (req, res) => {
3292 %     try {
3293 %         const items = await storage.getSpaAccountItems(req.params.accountId);
3294 %         res.json(items);
3295 %     } catch (error) {
3296 %         res.status(500).json({ error: "Error fetching account items" });
3297 %     }
3298 % });
3299 %
3300 % app.post("/api/spa/accounts/:accountId/items", async (req, res) => {
3301 %     try {
3302 %         const { description, quantity, unitPrice, itemType, notes } = req.body;
3303 %
3304 %         if (!description || !unitPrice) {
3305 %             return res.status(400).json({ error: "description and unitPrice are required" });
3306 %         }
3307 %
3308 %         const qty = quantity || 1;
3309 %         const subtotal = (parseFloat(unitPrice) * qty).toFixed(2);
3310 %
3311 %         const item = await storage.createSpaAccountItem({
3312 %             accountId: req.params.accountId,
3313 %             description,
3314 %             quantity: qty,
3315 %             unitPrice,
3316 %             subtotal,
3317 %             itemType: itemType || "treatment",
3318 %             notes: notes || null,
3319 %             createdAt: new Date().toISOString(),
3320 %         });
3321 %         res.status(201).json(item);
3322 %     } catch (error) {
3323 %         res.status(500).json({ error: "Error creating account item" });
3324 %     }
3325 % });
3326 %
3327 % app.patch("/api/spa/account-items/:id", async (req, res) => {
3328 %     try {
3329 %         const item = await storage.updateSpaAccountItem(req.params.id, req.body);
3330 %         if (!item) return res.status(404).json({ error: "Item not found" });
3331 %         res.json(item);
3332 %     } catch (error) {
3333 %         res.status(500).json({ error: "Error updating account item" });
3334 %     }
3335 % });
3336 %
3337 % app.delete("/api/spa/account-items/:id", async (req, res) => {
3338 %     try {
3339 %         await storage.deleteSpaAccountItem(req.params.id);
3340 %         res.status(204).send();
3341 %     } catch (error) {
3342 %         res.status(500).json({ error: "Error deleting account item" });
3343 %     }
3344 % });
3345 %
```

server/routes.ts (cont.)

```
3346 % // ===== EVENTS =====
3347 % // Event Rooms
3348 % app.get("/api/events/rooms", async (req, res) => {
3349 %   try {
3350 %     const rooms = await storage.getEventRooms();
3351 %     res.json(rooms);
3352 %   } catch (error) {
3353 %     res.status(500).json({ error: "Error fetching event rooms" });
3354 %   }
3355 % });
3356 %
3357 % app.get("/api/events/rooms/:id", async (req, res) => {
3358 %   try {
3359 %     const room = await storage.getEventRoom(req.params.id);
3360 %     if (!room) return res.status(404).json({ error: "Event room not found" });
3361 %     res.json(room);
3362 %   } catch (error) {
3363 %     res.status(500).json({ error: "Error fetching event room" });
3364 %   }
3365 % });
3366 %
3367 % app.post("/api/events/rooms", async (req, res) => {
3368 %   try {
3369 %     const room = await storage.createEventRoom(req.body);
3370 %     res.status(201).json(room);
3371 %   } catch (error) {
3372 %     res.status(500).json({ error: "Error creating event room" });
3373 %   }
3374 % });
3375 %
3376 % app.patch("/api/events/rooms/:id", async (req, res) => {
3377 %   try {
3378 %     const room = await storage.updateEventRoom(req.params.id, req.body);
3379 %     if (!room) return res.status(404).json({ error: "Event room not found" });
3380 %     res.json(room);
3381 %   } catch (error) {
3382 %     res.status(500).json({ error: "Error updating event room" });
3383 %   }
3384 % });
3385 %
3386 % app.delete("/api/events/rooms/:id", async (req, res) => {
3387 %   try {
3388 %     await storage.deleteEventRoom(req.params.id);
3389 %     res.status(204).send();
3390 %   } catch (error) {
3391 %     res.status(500).json({ error: "Error deleting event room" });
3392 %   }
3393 % });
3394 %
3395 % // Event Planning
3396 % app.get("/api/events/planning", async (req, res) => {
3397 %   try {
3398 %     const startDate = req.query.start as string;
3399 %     const endDate = req.query.end as string;
3400 %
3401 %     if (!startDate || !endDate) {
3402 %       return res.status(400).json({ error: "Start and end dates are required" });
3403 %     }
3404 %
3405 %     const planningData = await storage.getEventPlanningData(startDate, endDate);
3406 %     res.json(planningData);
3407 %   } catch (error) {
3408 %     res.status(500).json({ error: "Error fetching event planning data" });
3409 %   }
3410 % });
3411 %
3412 % // Event Charge Types - MUST come before /api/events/:id
3413 % app.get("/api/events/charge-types", async (req, res) => {
3414 %   try {
3415 %     const chargeTypes = await storage.getEventChargeTypes();
3416 %     res.json(chargeTypes);
3417 %   } catch (error) {
3418 %     res.status(500).json({ error: "Error fetching event charge types" });
3419 %   }
3420 % });
3421 %
3422 % app.post("/api/events/charge-types", async (req, res) => {
3423 %   try {
3424 %     const chargeType = await storage.createEventChargeType(req.body);
3425 %     res.status(201).json(chargeType);
3426 %   } catch (error) {
3427 %     res.status(500).json({ error: "Error creating event charge type" });
3428 %   }
3429 % });
3430 %
3431 % app.patch("/api/events/charge-types/:id", async (req, res) => {
3432 %   try {
3433 %     const chargeType = await storage.updateEventChargeType(req.params.id, req.body);
3434 %     if (!chargeType) return res.status(404).json({ error: "Event charge type not found" });
3435 %     res.json(chargeType);
3436 %   } catch (error) {
3437 %     res.status(500).json({ error: "Error updating event charge type" });
3438 %   }
3439 % }
```


server/routes.ts (cont.)

```
3439 % });
3440 %
3441 % app.delete("/api/events/charge-types/:id", async (req, res) => {
3442 %   try {
3443 %     await storage.deleteEventChargeType(req.params.id);
3444 %     res.status(204).send();
3445 %   } catch (error) {
3446 %     res.status(500).json({ error: "Error deleting event charge type" });
3447 %   }
3448 % });
3449 %
3450 % // Events
3451 % app.get("/api/events", async (req, res) => {
3452 %   try {
3453 %     const events = await storage.getEvents();
3454 %     res.json(events);
3455 %   } catch (error) {
3456 %     res.status(500).json({ error: "Error fetching events" });
3457 %   }
3458 % });
3459 %
3460 % app.get("/api/events/:id", async (req, res) => {
3461 %   try {
3462 %     const event = await storage.getEvent(req.params.id);
3463 %     if (!event) return res.status(404).json({ error: "Event not found" });
3464 %     res.json(event);
3465 %   } catch (error) {
3466 %     res.status(500).json({ error: "Error fetching event" });
3467 %   }
3468 % });
3469 %
3470 % app.post("/api/events", async (req, res) => {
3471 %   try {
3472 %     const { eventRoomId, startDate, endDate } = req.body;
3473 %     if (eventRoomId && startDate && endDate) {
3474 %       const activeStatuses = ["tentative", "confirmed", "in_progress"];
3475 %       const existingEvents = await storage.getEventsByDateRange(startDate, endDate);
3476 %       for (const existing of existingEvents) {
3477 %         if (existing.eventRoomId !== eventRoomId) continue;
3478 %         if (!activeStatuses.includes(existing.status)) continue;
3479 %         if (existing.startDate <= endDate && existing.endDate >= startDate) {
3480 %           const room = await storage.getEventRoom(eventRoomId);
3481 %           return res.status(409).json({
3482 %             error: "Superposición de evento",
3483 %             message: `El salón '${room?.name || eventRoomId}' ya tiene el evento '${existing.name}' reservado del $...
3484 %           });
3485 %         }
3486 %       }
3487 %     }
3488 %     const eventCode = storage.generateEventCode();
3489 %     const event = await storage.createEvent({
3490 %       ...req.body,
3491 %       eventCode,
3492 %       createdAt: new Date().toISOString(),
3493 %     });
3494 %     res.status(201).json(event);
3495 %   } catch (error) {
3496 %     res.status(500).json({ error: "Error creating event" });
3497 %   }
3498 % });
3499 %
3500 % app.patch("/api/events/:id", async (req, res) => {
3501 %   try {
3502 %     const current = await storage.getEvent(req.params.id);
3503 %     if (!current) return res.status(404).json({ error: "Event not found" });
3504 %
3505 %     const eventRoomId = req.body.eventRoomId || current.eventRoomId;
3506 %     const startDate = req.body.startDate || current.startDate;
3507 %     const endDate = req.body.endDate || current.endDate;
3508 %
3509 %     if (req.body.eventRoomId || req.body.startDate || req.body.endDate) {
3510 %       const activeStatuses = ["tentative", "confirmed", "in_progress"];
3511 %       const existingEvents = await storage.getEventsByDateRange(startDate, endDate);
3512 %       for (const existing of existingEvents) {
3513 %         if (existing.id === req.params.id) continue;
3514 %         if (existing.eventRoomId !== eventRoomId) continue;
3515 %         if (!activeStatuses.includes(existing.status)) continue;
3516 %         if (existing.startDate <= endDate && existing.endDate >= startDate) {
3517 %           const room = await storage.getEventRoom(eventRoomId);
3518 %           return res.status(409).json({
3519 %             error: "Superposición de evento",
3520 %             message: `El salón '${room?.name || eventRoomId}' ya tiene el evento '${existing.name}' reservado del $...
3521 %           });
3522 %         }
3523 %       }
3524 %     }
3525 %
3526 %     const event = await storage.updateEvent(req.params.id, req.body);
3527 %     res.json(event);
3528 %   } catch (error) {
3529 %     res.status(500).json({ error: "Error updating event" });
3530 %   }
3531 % });
```

server/routes.ts (cont.)

```
3532 %
3533 % app.delete("/api/events/:id", async (req, res) => {
3534 %   try {
3535 %     await storage.deleteEvent(req.params.id);
3536 %     res.status(204).send();
3537 %   } catch (error) {
3538 %     res.status(500).json({ error: "Error deleting event" });
3539 %   }
3540 % });
3541 %
3542 % // Event Charges
3543 % app.get("/api/events/:eventId/charges", async (req, res) => {
3544 %   try {
3545 %     const charges = await storage.getEventCharges(req.params.eventId);
3546 %     res.json(charges);
3547 %   } catch (error) {
3548 %     res.status(500).json({ error: "Error fetching event charges" });
3549 %   }
3550 % });
3551 %
3552 % app.post("/api/events/:eventId/charges", async (req, res) => {
3553 %   try {
3554 %     const evt = await storage.getEvent(req.params.eventId);
3555 %     if (!evt) return res.status(404).json({ error: "Event not found" });
3556 %     if (evt.status === "invoiced" || evt.status === "cancelled") {
3557 %       return res.status(400).json({ error: "No se pueden agregar cargos a un evento facturado o cancelado" });
3558 %     }
3559 %     const { chargeTypeId, description, quantity, unitPrice, notes } = req.body;
3560 %
3561 %     if (!description || !unitPrice) {
3562 %       return res.status(400).json({ error: "description and unitPrice are required" });
3563 %     }
3564 %
3565 %     const qty = quantity || 1;
3566 %     const total = (parseFloat(unitPrice) * qty).toFixed(2);
3567 %
3568 %     const charge = await storage.createEventCharge({
3569 %       eventId: req.params.eventId,
3570 %       chargeTypeId: chargeTypeId || null,
3571 %       description,
3572 %       quantity: qty,
3573 %       unitPrice,
3574 %       totalAmount: total,
3575 %       date: new Date().toISOString().split("T")[0],
3576 %       notes: notes || null,
3577 %       createdAt: new Date().toISOString(),
3578 %     });
3579 %     res.status(201).json(charge);
3580 %   } catch (error) {
3581 %     res.status(500).json({ error: "Error creating event charge" });
3582 %   }
3583 % });
3584 %
3585 % app.patch("/api/events/charges/:id", async (req, res) => {
3586 %   try {
3587 %     const charge = await storage.updateEventCharge(req.params.id, req.body);
3588 %     if (!charge) return res.status(404).json({ error: "Event charge not found" });
3589 %     res.json(charge);
3590 %   } catch (error) {
3591 %     res.status(500).json({ error: "Error updating event charge" });
3592 %   }
3593 % });
3594 %
3595 % app.delete("/api/events/charges/:id", async (req, res) => {
3596 %   try {
3597 %     await storage.deleteEventCharge(req.params.id);
3598 %     res.status(204).send();
3599 %   } catch (error) {
3600 %     res.status(500).json({ error: "Error deleting event charge" });
3601 %   }
3602 % });
3603 %
3604 % // Event Payments
3605 % app.get("/api/events/:eventId/payments", async (req, res) => {
3606 %   try {
3607 %     const payments = await storage.getEventPayments(req.params.eventId);
3608 %     res.json(payments);
3609 %   } catch (error) {
3610 %     res.status(500).json({ error: "Error fetching event payments" });
3611 %   }
3612 % });
3613 %
3614 % app.post("/api/events/:eventId/payments", async (req, res) => {
3615 %   try {
3616 %     const existingEvent = await storage.getEvent(req.params.eventId);
3617 %     if (!existingEvent) return res.status(404).json({ error: "Event not found" });
3618 %     if (existingEvent.status === "invoiced") {
3619 %       return res.status(400).json({ error: "No se pueden agregar pagos a un evento facturado" });
3620 %     }
3621 %     const { amount, method, isAdvance, reservationId, notes } = req.body;
3622 %     if (!amount || !method) {
3623 %       return res.status(400).json({ error: "amount and method are required" });
3624 %     }
3625 %   }
3626 % });
```

server/routes.ts (cont.)

```

3625 %         if (method === "room_charge" && !reservationId) {
3626 %             return res.status(400).json({ error: "reservationId es requerido para cargo a habitacion" });
3627 %         }
3628 %         const payment = await storage.createEventPayment({
3629 %             eventId: req.params.eventId,
3630 %             amount,
3631 %             method,
3632 %             isAdvance: isAdvance ? "true" : "false",
3633 %             reservationId: reservationId || null,
3634 %             notes: notes || null,
3635 %             paidAt: new Date().toISOString(),
3636 %             createdAt: new Date().toISOString(),
3637 %         });
3638 %         const refreshedEvent = await storage.getEvent(req.params.eventId);
3639 %         if (refreshedEvent) {
3640 %             const totalPaid = (refreshedEvent.payments || []).reduce((sum, p) => sum + parseFloat(p.amount), 0);
3641 %             await storage.updateEvent(req.params.eventId, { totalPaid: totalPaid.toFixed(2) } as any);
3642 %         }
3643 %         res.status(201).json(payment);
3644 %     } catch (error) {
3645 %         res.status(500).json({ error: "Error creating event payment" });
3646 %     }
3647 % });
3648 %
3649 % app.delete("/api/events/:eventId/payments/:payId", async (req, res) => {
3650 %     try {
3651 %         const event = await storage.getEvent(req.params.eventId);
3652 %         if (event && (event.status === "invoiced")) {
3653 %             return res.status(400).json({ error: "No se pueden eliminar pagos de un evento facturado" });
3654 %         }
3655 %         await storage.deleteEventPayment(req.params.payId);
3656 %         if (event) {
3657 %             const remaining = (event.payments || []).filter(p => p.id !== req.params.payId);
3658 %             const totalPaid = remaining.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3659 %             await storage.updateEvent(req.params.eventId, { totalPaid: totalPaid.toFixed(2) } as any);
3660 %         }
3661 %         res.status(204).send();
3662 %     } catch (error) {
3663 %         res.status(500).json({ error: "Error deleting event payment" });
3664 %     }
3665 % });
3666 %
3667 % app.get("/api/events/:eventId/summary", async (req, res) => {
3668 %     try {
3669 %         const event = await storage.getEvent(req.params.eventId);
3670 %         if (!event) return res.status(404).json({ error: "Event not found" });
3671 %         const charges = event.charges || [];
3672 %         const payments = event.payments || [];
3673 %         const totalCharges = charges.reduce((sum, c) => sum + parseFloat(c.totalAmount), 0);
3674 %         const totalPayments = payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3675 %         res.json({
3676 %             totalCharges,
3677 %             totalPayments,
3678 %             balance: totalCharges - totalPayments,
3679 %             charges,
3680 %             payments,
3681 %         });
3682 %     } catch (error) {
3683 %         res.status(500).json({ error: "Error fetching event summary" });
3684 %     }
3685 % });
3686 %
3687 % app.post("/api/events/:eventId/close", async (req, res) => {
3688 %     try {
3689 %         const { receiptType } = req.body;
3690 %         if (!receiptType) {
3691 %             return res.status(400).json({ error: "receiptType es requerido" });
3692 %         }
3693 %         const event = await storage.getEvent(req.params.eventId);
3694 %         if (!event) return res.status(404).json({ error: "Event not found" });
3695 %
3696 %         const charges = event.charges || [];
3697 %         const payments = event.payments || [];
3698 %         const totalCharges = charges.reduce((sum, c) => sum + parseFloat(c.totalAmount), 0);
3699 %         const totalPayments = payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3700 %         const balance = totalCharges - totalPayments;
3701 %
3702 %         if (balance > 0.01) {
3703 %             return res.status(400).json({
3704 %                 error: "Saldo pendiente",
3705 %                 message: `Hay un saldo pendiente de $$${balance.toFixed(2)}. Registre los pagos antes de cerrar.`
3706 %             });
3707 %         }
3708 %
3709 %         for (const payment of payments) {
3710 %             if (payment.method === "room_charge" && payment.reservationId) {
3711 %                 await storage.createCharge({
3712 %                     reservationId: payment.reservationId,
3713 %                     description: `Eventos - ${event.name}`,
3714 %                     amount: payment.amount,
3715 %                     date: new Date().toISOString().split("T")[0],
3716 %                     category: "events",
3717 %                     createdBy: null,

```

server/routes.ts (cont.)

```
3718 %     });
3719 %   }
3720 % }
3721 %
3722 %   await storage.updateEvent(req.params.eventId, {
3723 %     status: "invoiced",
3724 %     receiptType,
3725 %     closedAt: new Date().toISOString(),
3726 %     totalAmount: totalCharges.toFixed(2),
3727 %     totalPaid: totalPayments.toFixed(2),
3728 %   } as any);
3729 %
3730 %   const updated = await storage.getEvent(req.params.eventId);
3731 %   res.json(updated);
3732 % } catch (error) {
3733 %   res.status(500).json({ error: "Error closing event" });
3734 % }
3735 % });
3736 %
3737 % // Event Tables (Evento por Mesa)
3738 % app.get("/api/events/:eventId/tables", async (req, res) => {
3739 %   try {
3740 %     const tables = await storage.getEventTables(req.params.eventId);
3741 %     res.json(tables);
3742 %   } catch (error) {
3743 %     res.status(500).json({ error: "Error fetching event tables" });
3744 %   }
3745 % });
3746 %
3747 % app.post("/api/events/:eventId/tables", async (req, res) => {
3748 %   try {
3749 %     const { tableNumber, label, seats, reservationId } = req.body;
3750 %     if (!tableNumber) {
3751 %       return res.status(400).json({ error: "tableNumber is required" });
3752 %     }
3753 %     const table = await storage.createEventTable({
3754 %       eventId: req.params.eventId,
3755 %       tableNumber,
3756 %       label: label || null,
3757 %       seats: seats || null,
3758 %       reservationId: reservationId || null,
3759 %       createdAt: new Date().toISOString(),
3760 %     });
3761 %     res.status(201).json(table);
3762 %   } catch (error) {
3763 %     res.status(500).json({ error: "Error creating event table" });
3764 %   }
3765 % });
3766 %
3767 % app.patch("/api/events/:eventId/tables/:tableId", async (req, res) => {
3768 %   try {
3769 %     const table = await storage.updateEventTable(req.params.tableId, req.body);
3770 %     if (!table) return res.status(404).json({ error: "Event table not found" });
3771 %     res.json(table);
3772 %   } catch (error) {
3773 %     res.status(500).json({ error: "Error updating event table" });
3774 %   }
3775 % });
3776 %
3777 % app.delete("/api/events/:eventId/tables/:tableId", async (req, res) => {
3778 %   try {
3779 %     const table = await storage.getEventTable(req.params.tableId);
3780 %     if (!table) return res.status(404).json({ error: "Event table not found" });
3781 %     if (table.charges.length > 0 || table.payments.length > 0) {
3782 %       return res.status(400).json({ error: "No se puede eliminar una mesa con cargos o pagos" });
3783 %     }
3784 %     await storage.deleteEventTable(req.params.tableId);
3785 %     res.status(204).send();
3786 %   } catch (error) {
3787 %     res.status(500).json({ error: "Error deleting event table" });
3788 %   }
3789 % });
3790 %
3791 % app.post("/api/events/:eventId/tables/:tableId/charges", async (req, res) => {
3792 %   try {
3793 %     const table = await storage.getEventTable(req.params.tableId);
3794 %     if (!table) return res.status(404).json({ error: "Table not found" });
3795 %     if (table.status !== "open") {
3796 %       return res.status(400).json({ error: "No se pueden agregar cargos a una mesa cerrada" });
3797 %     }
3798 %     const { description, quantity, unitPrice } = req.body;
3799 %     if (!description || !unitPrice) {
3800 %       return res.status(400).json({ error: "description and unitPrice are required" });
3801 %     }
3802 %     const qty = quantity || 1;
3803 %     const total = (parseFloat(unitPrice) * qty).toFixed(2);
3804 %     const charge = await storage.createEventTableCharge({
3805 %       eventTableId: req.params.tableId,
3806 %       description,
3807 %       quantity: qty,
3808 %       unitPrice,
3809 %       total,
3810 %       createdAt: new Date().toISOString(),
```

server/routes.ts (cont.)

```
3811 %     });
3812 %     res.status(201).json(charge);
3813 %   } catch (error) {
3814 %     res.status(500).json({ error: "Error creating table charge" });
3815 %   }
3816 % });
3817 %
3818 % app.delete("/api/events/:eventId/tables/:tableId/charges/:chargeId", async (req, res) => {
3819 %   try {
3820 %     await storage.deleteEventTableCharge(req.params.chargeId);
3821 %     res.status(204).send();
3822 %   } catch (error) {
3823 %     res.status(500).json({ error: "Error deleting table charge" });
3824 %   }
3825 % });
3826 %
3827 % app.post("/api/events/:eventId/tables/:tableId/payments", async (req, res) => {
3828 %   try {
3829 %     const table = await storage.getEventTable(req.params.tableId);
3830 %     if (!table) return res.status(404).json({ error: "Table not found" });
3831 %     if (table.status !== "open") {
3832 %       return res.status(400).json({ error: "No se pueden agregar pagos a una mesa cerrada" });
3833 %     }
3834 %     const { amount, method, isAdvance, reservationId } = req.body;
3835 %     if (!amount || !method) {
3836 %       return res.status(400).json({ error: "amount and method are required" });
3837 %     }
3838 %     if (method === "room_charge" && !reservationId) {
3839 %       return res.status(400).json({ error: "reservationId es requerido para cargo a habitacion" });
3840 %     }
3841 %     const payment = await storage.createEventTablePayment({
3842 %       eventTableId: req.params.tableId,
3843 %       amount,
3844 %       method,
3845 %       isAdvance: isAdvance ? "true" : "false",
3846 %       reservationId: reservationId || null,
3847 %       paidAt: new Date().toISOString(),
3848 %       createdAt: new Date().toISOString(),
3849 %     });
3850 %     res.status(201).json(payment);
3851 %   } catch (error) {
3852 %     res.status(500).json({ error: "Error creating table payment" });
3853 %   }
3854 % });
3855 %
3856 % app.get("/api/events/:eventId/tables/:tableId/summary", async (req, res) => {
3857 %   try {
3858 %     const table = await storage.getEventTable(req.params.tableId);
3859 %     if (!table) return res.status(404).json({ error: "Event table not found" });
3860 %     const totalCharges = table.charges.reduce((sum, c) => sum + parseFloat(c.total), 0);
3861 %     const totalPayments = table.payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3862 %     res.json({
3863 %       totalCharges,
3864 %       totalPayments,
3865 %       balance: totalCharges - totalPayments,
3866 %       charges: table.charges,
3867 %       payments: table.payments,
3868 %     });
3869 %   } catch (error) {
3870 %     res.status(500).json({ error: "Error fetching table summary" });
3871 %   }
3872 % });
3873 %
3874 % app.post("/api/events/:eventId/tables/:tableId/close", async (req, res) => {
3875 %   try {
3876 %     const { receiptType } = req.body;
3877 %     if (!receiptType) {
3878 %       return res.status(400).json({ error: "receiptType es requerido" });
3879 %     }
3880 %     const table = await storage.getEventTable(req.params.tableId);
3881 %     if (!table) return res.status(404).json({ error: "Event table not found" });
3882 %
3883 %     const totalCharges = table.charges.reduce((sum, c) => sum + parseFloat(c.total), 0);
3884 %     const totalPayments = table.payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3885 %     const balance = totalCharges - totalPayments;
3886 %
3887 %     if (balance > 0.01) {
3888 %       return res.status(400).json({
3889 %         error: "Saldo pendiente",
3890 %         message: `Saldo pendiente de $$${balance.toFixed(2)}. Registre los pagos antes de cerrar.`
3891 %       });
3892 %     }
3893 %
3894 %     for (const payment of table.payments) {
3895 %       if (payment.method === "room_charge" && payment.reservationId) {
3896 %         const event = await storage.getEvent(req.params.eventId);
3897 %         await storage.createCharge({
3898 %           reservationId: payment.reservationId,
3899 %           description: `Eventos Mesa ${table.tableNumber} - ${event?.name || ""}`,
3900 %           amount: payment.amount,
3901 %           date: new Date().toISOString().split("T")[0],
3902 %           category: "events",
3903 %           createdBy: null,
```

server/routes.ts (cont.)

```
3904 %     });
3905 %   }
3906 % }
3907 %
3908 %   await storage.updateEventTable(req.params.tableId, {
3909 %     status: "invoiced",
3910 %     receiptType,
3911 %     closedAt: new Date().toISOString(),
3912 %   });
3913 %
3914 %   const updated = await storage.getEventTable(req.params.tableId);
3915 %   res.json(updated);
3916 % } catch (error) {
3917 %   res.status(500).json({ error: "Error closing table" });
3918 % }
3919 % });
3920 %
3921 % app.get("/api/events/:eventId/tables-summary", async (req, res) => {
3922 %   try {
3923 %     const tables = await storage.getEventTables(req.params.eventId);
3924 %     const tableSummaries = tables.map(table => {
3925 %       const totalCharges = table.charges.reduce((sum, c) => sum + parseFloat(c.total), 0);
3926 %       const totalPayments = table.payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3927 %       return {
3928 %         id: table.id,
3929 %         tableNumber: table.tableNumber,
3930 %         label: table.label,
3931 %         seats: table.seats,
3932 %         status: table.status,
3933 %         totalCharges,
3934 %         totalPayments,
3935 %         balance: totalCharges - totalPayments,
3936 %       };
3937 %     });
3938 %     const totalAll = tableSummaries.reduce((sum, t) => sum + t.totalCharges, 0);
3939 %     const paidAll = tableSummaries.reduce((sum, t) => sum + t.totalPayments, 0);
3940 %     const pendingTables = tableSummaries.filter(t => t.status === "open").length;
3941 %     res.json({
3942 %       tables: tableSummaries,
3943 %       totalCharges: totalAll,
3944 %       totalPayments: paidAll,
3945 %       balance: totalAll - paidAll,
3946 %       pendingTables,
3947 %     });
3948 %   } catch (error) {
3949 %     res.status(500).json({ error: "Error fetching tables summary" });
3950 %   }
3951 % });
3952 %
3953 % // ===== MAINTENANCE =====
3954 %
3955 % // Maintenance Staff
3956 % app.get("/api/maintenance/staff", async (req, res) => {
3957 %   try {
3958 %     const staff = await storage.getMaintenanceStaff();
3959 %     res.json(staff);
3960 %   } catch (error) {
3961 %     res.status(500).json({ error: "Error fetching maintenance staff" });
3962 %   }
3963 % });
3964 %
3965 % app.get("/api/maintenance/staff/:id", async (req, res) => {
3966 %   try {
3967 %     const staff = await storage.getMaintenanceStaffMember(req.params.id);
3968 %     if (!staff) return res.status(404).json({ error: "Staff member not found" });
3969 %     res.json(staff);
3970 %   } catch (error) {
3971 %     res.status(500).json({ error: "Error fetching staff member" });
3972 %   }
3973 % });
3974 %
3975 % app.post("/api/maintenance/staff", async (req, res) => {
3976 %   try {
3977 %     const staff = await storage.createMaintenanceStaff(req.body);
3978 %     res.status(201).json(staff);
3979 %   } catch (error) {
3980 %     res.status(500).json({ error: "Error creating staff member" });
3981 %   }
3982 % });
3983 %
3984 % app.patch("/api/maintenance/staff/:id", async (req, res) => {
3985 %   try {
3986 %     const staff = await storage.updateMaintenanceStaff(req.params.id, req.body);
3987 %     if (!staff) return res.status(404).json({ error: "Staff member not found" });
3988 %     res.json(staff);
3989 %   } catch (error) {
3990 %     res.status(500).json({ error: "Error updating staff member" });
3991 %   }
3992 % });
3993 %
3994 % app.delete("/api/maintenance/staff/:id", async (req, res) => {
3995 %   try {
3996 %     await storage.deleteMaintenanceStaff(req.params.id);
```

server/routes.ts (cont.)

```
3997 %         res.status(204).send();
3998 %     } catch (error) {
3999 %         res.status(500).json({ error: "Error deleting staff member" });
4000 %     }
4001 % });
4002 %
4003 % // Work Orders
4004 % app.get("/api/maintenance/work-orders", async (req, res) => {
4005 %     try {
4006 %         const { status, roomId } = req.query;
4007 %         let orders;
4008 %         if (status) {
4009 %             orders = await storage.getWorkOrdersByStatus(status as any);
4010 %         } else if (roomId) {
4011 %             orders = await storage.getWorkOrdersByRoom(roomId as string);
4012 %         } else {
4013 %             orders = await storage.getWorkOrders();
4014 %         }
4015 %         res.json(orders);
4016 %     } catch (error) {
4017 %         res.status(500).json({ error: "Error fetching work orders" });
4018 %     }
4019 % });
4020 %
4021 % app.get("/api/maintenance/work-orders/:id", async (req, res) => {
4022 %     try {
4023 %         const order = await storage.getWorkOrder(req.params.id);
4024 %         if (!order) return res.status(404).json({ error: "Work order not found" });
4025 %         res.json(order);
4026 %     } catch (error) {
4027 %         res.status(500).json({ error: "Error fetching work order" });
4028 %     }
4029 % });
4030 %
4031 % app.post("/api/maintenance/work-orders", async (req, res) => {
4032 %     try {
4033 %         const orderCode = storage.generateWorkOrderCode();
4034 %         const order = await storage.createWorkOrder({
4035 %             ...req.body,
4036 %             orderCode,
4037 %             reportedAt: new Date().toISOString(),
4038 %         });
4039 %         res.status(201).json(order);
4040 %     } catch (error) {
4041 %         res.status(500).json({ error: "Error creating work order" });
4042 %     }
4043 % });
4044 %
4045 % app.patch("/api/maintenance/work-orders/:id", async (req, res) => {
4046 %     try {
4047 %         const updates = { ...req.body };
4048 %         if (updates.status === "completed" && !updates.completedAt) {
4049 %             updates.completedAt = new Date().toISOString();
4050 %         }
4051 %         const order = await storage.updateWorkOrder(req.params.id, updates);
4052 %         if (!order) return res.status(404).json({ error: "Work order not found" });
4053 %         res.json(order);
4054 %     } catch (error) {
4055 %         res.status(500).json({ error: "Error updating work order" });
4056 %     }
4057 % });
4058 %
4059 % app.delete("/api/maintenance/work-orders/:id", async (req, res) => {
4060 %     try {
4061 %         await storage.deleteWorkOrder(req.params.id);
4062 %         res.status(204).send();
4063 %     } catch (error) {
4064 %         res.status(500).json({ error: "Error deleting work order" });
4065 %     }
4066 % });
4067 %
4068 % // Dashboard stats for maintenance
4069 % app.get("/api/maintenance/dashboard", async (req, res) => {
4070 %     try {
4071 %         const orders = await storage.getWorkOrders();
4072 %         const staff = await storage.getMaintenanceStaff();
4073 %
4074 %         const pending = orders.filter(o => o.status === "pending").length;
4075 %         const inProgress = orders.filter(o => o.status === "in_progress" || o.status === "assigned").length;
4076 %         const completedToday = orders.filter(o => {
4077 %             if (o.status !== "completed" || !o.completedAt) return false;
4078 %             const today = new Date().toISOString().split("T")[0];
4079 %             return o.completedAt.startsWith(today);
4080 %         }).length;
4081 %         const urgent = orders.filter(o => o.priority === "urgent" && o.status !== "completed" && o.status !== "canceled...");
4082 %
4083 %         res.json({
4084 %             pending,
4085 %             inProgress,
4086 %             completedToday,
4087 %             urgent,
4088 %             totalStaff: staff.filter(s => s.isActive === "true").length,
4089 %             recentOrders: orders
```

server/routes.ts (cont.)

```
4090 %         .filter(o => o.status !== "completed" && o.status !== "cancelled")
4091 %         .sort((a, b) => {
4092 %             const priorityOrder = { urgent: 0, high: 1, medium: 2, low: 3 };
4093 %             return (priorityOrder[a.priority as keyof typeof priorityOrder] || 2) -
4094 %                 (priorityOrder[b.priority as keyof typeof priorityOrder] || 2);
4095 %         })
4096 %         .slice(0, 5),
4097 %     });
4098 %     } catch (error) {
4099 %         res.status(500).json({ error: "Error fetching maintenance dashboard" });
4100 %     }
4101 % });
4102 %
4103 % // ===== ADMINISTRATION =====
4104 %
4105 % // Admin Dashboard
4106 % app.get("/api/admin/dashboard", async (req, res) => {
4107 %     try {
4108 %         const stats = await storage.getAdminDashboardStats();
4109 %         res.json(stats);
4110 %     } catch (error) {
4111 %         res.status(500).json({ error: "Error fetching admin dashboard" });
4112 %     }
4113 % });
4114 %
4115 % // System Users
4116 % app.get("/api/admin/users", async (req, res) => {
4117 %     try {
4118 %         const users = await storage.getSystemUsers();
4119 %         res.json(users);
4120 %     } catch (error) {
4121 %         res.status(500).json({ error: "Error fetching users" });
4122 %     }
4123 % });
4124 %
4125 % app.get("/api/admin/users/:id", async (req, res) => {
4126 %     try {
4127 %         const user = await storage.getSystemUser(req.params.id);
4128 %         if (!user) return res.status(404).json({ error: "User not found" });
4129 %         res.json(user);
4130 %     } catch (error) {
4131 %         res.status(500).json({ error: "Error fetching user" });
4132 %     }
4133 % });
4134 %
4135 % app.post("/api/admin/users", async (req, res) => {
4136 %     try {
4137 %         const user = await storage.createSystemUser({
4138 %             ...req.body,
4139 %             createdAt: new Date().toISOString(),
4140 %         });
4141 %         res.status(201).json(user);
4142 %     } catch (error) {
4143 %         res.status(500).json({ error: "Error creating user" });
4144 %     }
4145 % });
4146 %
4147 % app.patch("/api/admin/users/:id", async (req, res) => {
4148 %     try {
4149 %         const user = await storage.updateSystemUser(req.params.id, req.body);
4150 %         if (!user) return res.status(404).json({ error: "User not found" });
4151 %         res.json(user);
4152 %     } catch (error) {
4153 %         res.status(500).json({ error: "Error updating user" });
4154 %     }
4155 % });
4156 %
4157 % app.delete("/api/admin/users/:id", async (req, res) => {
4158 %     try {
4159 %         await storage.deleteSystemUser(req.params.id);
4160 %         res.status(204).send();
4161 %     } catch (error) {
4162 %         res.status(500).json({ error: "Error deleting user" });
4163 %     }
4164 % });
4165 %
4166 % // System Settings
4167 % app.get("/api/admin/settings", async (req, res) => {
4168 %     try {
4169 %         const category = req.query.category as string | undefined;
4170 %         if (category) {
4171 %             const settings = await storage.getSystemSettingsByCategory(category);
4172 %             res.json(settings);
4173 %         } else {
4174 %             const settings = await storage.getSystemSettings();
4175 %             res.json(settings);
4176 %         }
4177 %     } catch (error) {
4178 %         res.status(500).json({ error: "Error fetching settings" });
4179 %     }
4180 % });
4181 %
4182 % app.get("/api/admin/settings/:key", async (req, res) => {
```


server/routes.ts (cont.)

```
4183 %     try {
4184 %         const setting = await storage.getSystemSetting(req.params.key);
4185 %         if (!setting) return res.status(404).json({ error: "Setting not found" });
4186 %         res.json(setting);
4187 %     } catch (error) {
4188 %         res.status(500).json({ error: "Error fetching setting" });
4189 %     }
4190 % });
4191 %
4192 % app.put("/api/admin/settings", async (req, res) => {
4193 %     try {
4194 %         const setting = await storage.upsertSystemSetting({
4195 %             ...req.body,
4196 %             updatedAt: new Date().toISOString(),
4197 %         });
4198 %         res.json(setting);
4199 %     } catch (error) {
4200 %         res.status(500).json({ error: "Error saving setting" });
4201 %     }
4202 % });
4203 %
4204 % app.delete("/api/admin/settings/:key", async (req, res) => {
4205 %     try {
4206 %         await storage.deleteSystemSetting(req.params.key);
4207 %         res.status(204).send();
4208 %     } catch (error) {
4209 %         res.status(500).json({ error: "Error deleting setting" });
4210 %     }
4211 % });
4212 %
4213 % // Audit Logs
4214 % app.get("/api/admin/audit-logs", async (req, res) => {
4215 %     try {
4216 %         const module = req.query.module as string | undefined;
4217 %         const userId = req.query.userId as string | undefined;
4218 %
4219 %         if (module) {
4220 %             const logs = await storage.getAuditLogsByModule(module);
4221 %             res.json(logs);
4222 %         } else if (userId) {
4223 %             const logs = await storage.getAuditLogsByUser(userId);
4224 %             res.json(logs);
4225 %         } else {
4226 %             const logs = await storage.getAuditLogs();
4227 %             res.json(logs);
4228 %         }
4229 %     } catch (error) {
4230 %         res.status(500).json({ error: "Error fetching audit logs" });
4231 %     }
4232 % });
4233 %
4234 % app.post("/api/admin/audit-logs", async (req, res) => {
4235 %     try {
4236 %         const log = await storage.createAuditLog({
4237 %             ...req.body,
4238 %             timestamp: new Date().toISOString(),
4239 %         });
4240 %         res.status(201).json(log);
4241 %     } catch (error) {
4242 %         res.status(500).json({ error: "Error creating audit log" });
4243 %     }
4244 % });
4245 %
4246 % // ===== PACKAGES =====
4247 % app.get("/api/packages", async (req, res) => {
4248 %     try {
4249 %         const packages = await storage.getPackages();
4250 %         res.json(packages);
4251 %     } catch (error) {
4252 %         res.status(500).json({ error: "Error fetching packages" });
4253 %     }
4254 % });
4255 %
4256 % app.get("/api/packages/active", async (req, res) => {
4257 %     try {
4258 %         const packages = await storage.getActivePackages();
4259 %         res.json(packages);
4260 %     } catch (error) {
4261 %         res.status(500).json({ error: "Error fetching active packages" });
4262 %     }
4263 % });
4264 %
4265 % app.get("/api/packages/:id", async (req, res) => {
4266 %     try {
4267 %         const pkg = await storage.getPackage(req.params.id);
4268 %         if (!pkg) return res.status(404).json({ error: "Package not found" });
4269 %         res.json(pkg);
4270 %     } catch (error) {
4271 %         res.status(500).json({ error: "Error fetching package" });
4272 %     }
4273 % });
4274 %
4275 % app.post("/api/packages", async (req, res) => {
```

server/routes.ts (cont.)

```
4276 %     try {
4277 %       const { name, description, roomTypeId, nights, basePrice, discountPercent, validFrom, validUntil, status, inclu...
4278 %       if (!name || !basePrice) {
4279 %         return res.status(400).json({ error: "Name and base price are required" });
4280 %       }
4281 %       const code = storage.generatePackageCode();
4282 %       const pkg = await storage.createPackage({
4283 %         code,
4284 %         name,
4285 %         description,
4286 %         roomTypeId,
4287 %         nights: nights || 1,
4288 %         basePrice: String(basePrice),
4289 %         discountPercent: discountPercent ? String(discountPercent) : null,
4290 %         validFrom,
4291 %         validUntil,
4292 %         status: status || "active",
4293 %         includedServices,
4294 %         terms,
4295 %         createdAt: new Date().toISOString(),
4296 %       });
4297 %       res.status(201).json(pkg);
4298 %     } catch (error) {
4299 %       res.status(500).json({ error: "Error creating package" });
4300 %     }
4301 %   });
4302 %
4303 %   app.patch("/api/packages/:id", async (req, res) => {
4304 %     try {
4305 %       const updates = { ...req.body };
4306 %       if (updates.basePrice) updates.basePrice = String(updates.basePrice);
4307 %       if (updates.discountPercent) updates.discountPercent = String(updates.discountPercent);
4308 %       const pkg = await storage.updatePackage(req.params.id, updates);
4309 %       if (!pkg) return res.status(404).json({ error: "Package not found" });
4310 %       res.json(pkg);
4311 %     } catch (error) {
4312 %       res.status(500).json({ error: "Error updating package" });
4313 %     }
4314 %   });
4315 %
4316 %   app.delete("/api/packages/:id", async (req, res) => {
4317 %     try {
4318 %       await storage.deletePackage(req.params.id);
4319 %       res.status(204).send();
4320 %     } catch (error) {
4321 %       res.status(500).json({ error: "Error deleting package" });
4322 %     }
4323 %   });
4324 %
4325 %   // Package Items
4326 %   app.get("/api/packages/:packageId/items", async (req, res) => {
4327 %     try {
4328 %       const items = await storage.getPackageItems(req.params.packageId);
4329 %       res.json(items);
4330 %     } catch (error) {
4331 %       res.status(500).json({ error: "Error fetching package items" });
4332 %     }
4333 %   });
4334 %
4335 %   app.post("/api/packages/:packageId/items", async (req, res) => {
4336 %     try {
4337 %       const { itemType, description, quantity, unitValue } = req.body;
4338 %       if (!itemType || !description) {
4339 %         return res.status(400).json({ error: "Item type and description are required" });
4340 %       }
4341 %       const item = await storage.createPackageItem({
4342 %         packageId: req.params.packageId,
4343 %         itemType,
4344 %         description,
4345 %         quantity: quantity || 1,
4346 %         unitValue: unitValue ? String(unitValue) : null,
4347 %       });
4348 %       res.status(201).json(item);
4349 %     } catch (error) {
4350 %       res.status(500).json({ error: "Error creating package item" });
4351 %     }
4352 %   });
4353 %
4354 %   app.patch("/api/package-items/:id", async (req, res) => {
4355 %     try {
4356 %       const updates = { ...req.body };
4357 %       if (updates.unitValue) updates.unitValue = String(updates.unitValue);
4358 %       const item = await storage.updatePackageItem(req.params.id, updates);
4359 %       if (!item) return res.status(404).json({ error: "Package item not found" });
4360 %       res.json(item);
4361 %     } catch (error) {
4362 %       res.status(500).json({ error: "Error updating package item" });
4363 %     }
4364 %   });
4365 %
4366 %   app.delete("/api/package-items/:id", async (req, res) => {
4367 %     try {
4368 %       await storage.deletePackageItem(req.params.id);
```

server/routes.ts (cont.)

```
4369 %         res.status(204).send();
4370 %     } catch (error) {
4371 %         res.status(500).json({ error: "Error deleting package item" });
4372 %     }
4373 % });
4374 %
4375 % // ===== SYSTEM NOTIFICATIONS =====
4376 %
4377 % app.get("/api/notifications", async (req, res) => {
4378 %     try {
4379 %         const area = req.query.area as string | undefined;
4380 %         const limit = req.query.limit ? parseInt(req.query.limit as string) : 50;
4381 %         const notifications = await storage.getNotifications(area as any, limit);
4382 %         res.json(notifications);
4383 %     } catch (error) {
4384 %         res.status(500).json({ error: "Error fetching notifications" });
4385 %     }
4386 % });
4387 %
4388 % app.post("/api/notifications", async (req, res) => {
4389 %     try {
4390 %         const notification = await storage.createNotification(req.body);
4391 %         res.status(201).json(notification);
4392 %     } catch (error) {
4393 %         res.status(500).json({ error: "Error creating notification" });
4394 %     }
4395 % });
4396 %
4397 % app.patch("/api/notifications/:id/read", async (req, res) => {
4398 %     try {
4399 %         const id = parseInt(req.params.id);
4400 %         const notification = await storage.markNotificationRead(id);
4401 %         if (!notification) return res.status(404).json({ error: "Notification not found" });
4402 %         res.json(notification);
4403 %     } catch (error) {
4404 %         res.status(500).json({ error: "Error marking notification as read" });
4405 %     }
4406 % });
4407 %
4408 % app.patch("/api/notifications/read-all", async (req, res) => {
4409 %     try {
4410 %         const area = req.query.area as string | undefined;
4411 %         const count = await storage.markAllNotificationsRead(area as any);
4412 %         res.json({ markedRead: count });
4413 %     } catch (error) {
4414 %         res.status(500).json({ error: "Error marking notifications as read" });
4415 %     }
4416 % });
4417 %
4418 % app.get("/api/notifications/unread-count", async (req, res) => {
4419 %     try {
4420 %         const area = req.query.area as string | undefined;
4421 %         const count = await storage.getUnreadNotificationCount(area as any);
4422 %         res.json({ count });
4423 %     } catch (error) {
4424 %         res.status(500).json({ error: "Error getting unread count" });
4425 %     }
4426 % });
4427 %
4428 % // ===== CHATBOT WEBHOOK =====
4429 %
4430 % app.post("/api/webhook/chatbot", async (req, res) => {
4431 %     try {
4432 %         const secret = req.headers["x-chatbot-secret"] as string;
4433 %         const expectedSecret = process.env.CHATBOT_WEBHOOK_SECRET;
4434 %         if (!expectedSecret || secret !== expectedSecret) {
4435 %             return res.status(401).json({ error: "Invalid or missing webhook secret" });
4436 %         }
4437 %
4438 %         const { eventType, area, priority, guestName, roomNumber, reservationId, message, timestamp } = req.body;
4439 %
4440 %         const validAreas = ["housekeeping", "maintenance", "restaurant", "spa", "reception", "all"];
4441 %         const validPriorities = ["normal", "high", "urgent"];
4442 %
4443 %         if (!area || !message) {
4444 %             return res.status(400).json({ error: "area and message are required" });
4445 %         }
4446 %         if (!validAreas.includes(area)) {
4447 %             return res.status(400).json({ error: `Invalid area. Must be one of: ${validAreas.join(", ")}` });
4448 %         }
4449 %         if (priority && !validPriorities.includes(priority)) {
4450 %             return res.status(400).json({ error: `Invalid priority. Must be one of: ${validPriorities.join(", ")}` });
4451 %         }
4452 %
4453 %         const areaLabels: Record<string, string> = {
4454 %             housekeeping: "Housekeeping",
4455 %             maintenance: "Mantenimiento",
4456 %             restaurant: "Restaurante",
4457 %             spa: "SPA",
4458 %             reception: "Recepción",
4459 %             all: "General",
4460 %         };
4461 %     }
4462 % });
```

server/routes.ts (cont.)

```

4462 %     const typeMap: Record<string, string> = {
4463 %         housekeeping: "chatbot_housekeeping",
4464 %         maintenance: "chatbot_maintenance",
4465 %         restaurant: "chatbot_restaurant",
4466 %         spa: "chatbot_spa",
4467 %         reception: "chatbot_request",
4468 %         all: "chatbot_request",
4469 %     };
4470 %
4471 %     const notification = await storage.createNotification({
4472 %         type: (typeMap[area] || "chatbot_request") as any,
4473 %         title: `Solicitud de ${guestName || "Huésped"} - Hab. ${roomNumber || "N/A"}`,
4474 %         message,
4475 %         targetArea: area,
4476 %         relatedEntityType: reservationId ? "reservation" : "room",
4477 %         relatedEntityId: reservationId ? String(reservationId) : roomNumber,
4478 %         priority: priority || "normal",
4479 %     });
4480 %
4481 %     if (area === "housekeeping" && roomNumber) {
4482 %         const rooms = await storage.getRooms();
4483 %         const room = rooms.find(r => r.roomNumber === roomNumber);
4484 %         if (room) {
4485 %             try {
4486 %                 await storage.createHousekeepingTask({
4487 %                     roomId: room.id,
4488 %                     type: "guest_request",
4489 %                     status: "pending",
4490 %                     priority: priority === "urgent" ? "urgent" : "normal",
4491 %                     notes: `Chatbot: ${message} (${guestName || "Huésped"})`,
4492 %                 });
4493 %             } catch {}
4494 %         }
4495 %     }
4496 %
4497 %     res.json({ success: true, notificationId: notification.id });
4498 % } catch (error) {
4499 %     res.status(500).json({ error: "Error processing webhook" });
4500 % }
4501 % });
4502 %
4503 % // ===== WEB CHECK-IN =====
4504 %
4505 % app.post("/api/web-checkin/generate/:reservationId", async (req, res) => {
4506 %     try {
4507 %         const reservation = await storage.getReservation(req.params.reservationId);
4508 %         if (!reservation) {
4509 %             return res.status(404).json({ error: "Reservation not found" });
4510 %         }
4511 %
4512 %         const existing = await storage.getWebCheckinByReservation(req.params.reservationId);
4513 %         if (existing && existing.status !== "expired") {
4514 %             const protocol = req.headers["x-forwarded-proto"] || "https";
4515 %             const host = req.headers.host;
4516 %             const link = `${protocol}://${host}/web-checkin/${existing.token}`;
4517 %             return res.json({ token: existing.token, link, webCheckin: existing });
4518 %         }
4519 %
4520 %         const token = randomUUID();
4521 %         const checkInDate = new Date(reservation.checkInDate);
4522 %         const expiresAt = new Date(checkInDate.getTime() + 24 * 60 * 60 * 1000);
4523 %
4524 %         const webCheckin = await storage.createWebCheckin({
4525 %             reservationId: req.params.reservationId,
4526 %             token,
4527 %             status: "pending",
4528 %             confirmedFirstName: reservation.guest?.firstName || null,
4529 %             confirmedLastName: reservation.guest?.lastName || null,
4530 %             confirmedDocumentType: reservation.guest?.documentType || null,
4531 %             confirmedDocumentNumber: reservation.guest?.documentNumber || null,
4532 %             confirmedNationality: reservation.guest?.nationality || null,
4533 %             confirmedPhone: reservation.guest?.phone || null,
4534 %             confirmedEmail: reservation.guest?.email || null,
4535 %             expiresAt,
4536 %         });
4537 %
4538 %         const protocol = req.headers["x-forwarded-proto"] || "https";
4539 %         const host = req.headers.host;
4540 %         const link = `${protocol}://${host}/web-checkin/${token}`;
4541 %
4542 %         res.status(201).json({ token, link, webCheckin });
4543 %     } catch (error) {
4544 %         res.status(500).json({ error: "Error generating web check-in" });
4545 %     }
4546 % });
4547 %
4548 % app.get("/api/web-checkin/list", async (req, res) => {
4549 %     try {
4550 %         const webCheckins = await storage.listWebCheckins();
4551 %         const enriched = [];
4552 %         for (const wc of webCheckins) {
4553 %             const reservation = await storage.getReservation(wc.reservationId);
4554 %             enriched.push({

```

server/routes.ts (cont.)

```
4555 %         ...wc,
4556 %         reservation: reservation ? {
4557 %             reservationCode: reservation.reservationCode,
4558 %             guestName: `${reservation.guest?.firstName} ${reservation.guest?.lastName}`,
4559 %             roomNumber: reservation.room?.roomNumber,
4560 %             checkInDate: reservation.checkInDate,
4561 %             checkOutDate: reservation.checkOutDate,
4562 %             status: reservation.status,
4563 %         } : null,
4564 %     });
4565 % }
4566 % res.json(enriched);
4567 % } catch (error) {
4568 %     res.status(500).json({ error: "Error listing web check-ins" });
4569 % }
4570 % });
4571 %
4572 % app.get("/api/web-checkin/:reservationId/status", async (req, res) => {
4573 %     try {
4574 %         const webCheckin = await storage.getWebCheckinByReservation(req.params.reservationId);
4575 %         if (!webCheckin) {
4576 %             return res.json({ status: "not_generated" });
4577 %         }
4578 %         res.json(webCheckin);
4579 %     } catch (error) {
4580 %         res.status(500).json({ error: "Error getting web check-in status" });
4581 %     }
4582 % });
4583 %
4584 % // Public endpoints (no auth required)
4585 % app.get("/api/public/web-checkin/:token", async (req, res) => {
4586 %     try {
4587 %         const webCheckin = await storage.getWebCheckinByToken(req.params.token);
4588 %         if (!webCheckin) {
4589 %             return res.status(404).json({ error: "Web check-in no encontrado" });
4590 %         }
4591 %
4592 %         if (webCheckin.status === "completed") {
4593 %             return res.status(400).json({ error: "Este web check-in ya fue completado" });
4594 %         }
4595 %
4596 %         if (webCheckin.expiresAt && new Date() > new Date(webCheckin.expiresAt)) {
4597 %             await storage.updateWebCheckin(webCheckin.id, { status: "expired" } as any);
4598 %             return res.status(400).json({ error: "Este enlace ha expirado" });
4599 %         }
4600 %
4601 %         const reservation = await storage.getReservation(webCheckin.reservationId);
4602 %
4603 %         res.json({
4604 %             webCheckin: {
4605 %                 id: webCheckin.id,
4606 %                 status: webCheckin.status,
4607 %                 confirmedFirstName: webCheckin.confirmedFirstName,
4608 %                 confirmedLastName: webCheckin.confirmedLastName,
4609 %                 confirmedDocumentType: webCheckin.confirmedDocumentType,
4610 %                 confirmedDocumentNumber: webCheckin.confirmedDocumentNumber,
4611 %                 confirmedNationality: webCheckin.confirmedNationality,
4612 %                 confirmedPhone: webCheckin.confirmedPhone,
4613 %                 confirmedEmail: webCheckin.confirmedEmail,
4614 %             },
4615 %             reservation: reservation ? {
4616 %                 checkInDate: reservation.checkInDate,
4617 %                 checkOutDate: reservation.checkOutDate,
4618 %                 roomType: reservation.room?.roomType?.name,
4619 %                 nights: reservation.nights,
4620 %             } : null,
4621 %             hotel: {
4622 %                 name: "Maran Suites & Towers",
4623 %                 address: "Alameda de la Federación 497, Paraná, Entre Ríos",
4624 %                 phone: "+54 343 423-5444",
4625 %             },
4626 %         });
4627 %     } catch (error) {
4628 %         res.status(500).json({ error: "Error loading web check-in" });
4629 %     }
4630 % });
4631 %
4632 % app.post("/api/public/web-checkin/:token", async (req, res) => {
4633 %     try {
4634 %         const webCheckin = await storage.getWebCheckinByToken(req.params.token);
4635 %         if (!webCheckin) {
4636 %             return res.status(404).json({ error: "Web check-in no encontrado" });
4637 %         }
4638 %
4639 %         if (webCheckin.status === "completed") {
4640 %             return res.status(400).json({ error: "Este web check-in ya fue completado" });
4641 %         }
4642 %
4643 %         if (webCheckin.expiresAt && new Date() > new Date(webCheckin.expiresAt)) {
4644 %             await storage.updateWebCheckin(webCheckin.id, { status: "expired" } as any);
4645 %             return res.status(400).json({ error: "Este enlace ha expirado" });
4646 %         }
4647 %     }
```

server/routes.ts (cont.)

```
4648 %     const {
4649 %         confirmedFirstName, confirmedLastName,
4650 %         confirmedDocumentType, confirmedDocumentNumber,
4651 %         confirmedNationality, confirmedPhone, confirmedEmail,
4652 %         documentPhotoUrl, estimatedArrivalTime,
4653 %         requestEarlyCheckIn, earlyCheckInTime,
4654 %         termsAccepted,
4655 %     } = req.body;
4656 %
4657 %     if (!termsAccepted) {
4658 %         return res.status(400).json({ error: "Debe aceptar los términos y condiciones" });
4659 %     }
4660 %
4661 %     if (!confirmedFirstName?.trim() || !confirmedLastName?.trim()) {
4662 %         return res.status(400).json({ error: "Nombre y apellido son obligatorios" });
4663 %     }
4664 %
4665 %     if (documentPhotoUrl && typeof documentPhotoUrl === "string" && documentPhotoUrl.length > 5 * 1024 * 1024) {
4666 %         return res.status(400).json({ error: "La imagen del documento es demasiado grande" });
4667 %     }
4668 %
4669 %     const ipAddress = req.headers["x-forwarded-for"] as string || req.socket.remoteAddress || "";
4670 %
4671 %     await storage.updateWebCheckin(webCheckin.id, {
4672 %         status: "completed",
4673 %         confirmedFirstName,
4674 %         confirmedLastName,
4675 %         confirmedDocumentType,
4676 %         confirmedDocumentNumber,
4677 %         confirmedNationality,
4678 %         confirmedPhone,
4679 %         confirmedEmail,
4680 %         documentPhotoUrl,
4681 %         estimatedArrivalTime,
4682 %         requestEarlyCheckIn: requestEarlyCheckIn || false,
4683 %         earlyCheckInTime: earlyCheckInTime || null,
4684 %         termsAccepted: true,
4685 %         termsAcceptedAt: new Date(),
4686 %         ipAddress,
4687 %         completedAt: new Date(),
4688 %     } as any);
4689 %
4690 %     const reservation = await storage.getReservation(webCheckin.reservationId);
4691 %     if (reservation && reservation.guest) {
4692 %         await storage.updateGuest(reservation.guest.id, {
4693 %             firstName: confirmedFirstName || reservation.guest.firstName,
4694 %             lastName: confirmedLastName || reservation.guest.lastName,
4695 %             documentType: confirmedDocumentType || reservation.guest.documentType,
4696 %             documentNumber: confirmedDocumentNumber || reservation.guest.documentNumber,
4697 %             nationality: confirmedNationality || reservation.guest.nationality,
4698 %             phone: confirmedPhone || reservation.guest.phone,
4699 %             email: confirmedEmail || reservation.guest.email,
4700 %         });
4701 %     }
4702 %
4703 %     if (requestEarlyCheckIn && reservation) {
4704 %         await storage.updateReservation(webCheckin.reservationId, {
4705 %             earlyCheckIn: true,
4706 %             earlyCheckInTime: earlyCheckInTime || null,
4707 %         } as any);
4708 %     }
4709 %
4710 %     await storage.createNotification({
4711 %         type: "web_checkin",
4712 %         title: `Web Check-in completado - ${confirmedFirstName} ${confirmedLastName}`,
4713 %         message: `El huésped completó el web check-in. Llegada estimada: ${estimatedArrivalTime || "No especificada"}...
4714 %         targetArea: "reception",
4715 %         relatedEntityType: "reservation",
4716 %         relatedEntityId: webCheckin.reservationId,
4717 %         priority: requestEarlyCheckIn ? "high" : "normal",
4718 %     });
4719 %
4720 %     res.json({ success: true });
4721 % } catch (error) {
4722 %     res.status(500).json({ error: "Error processing web check-in" });
4723 % }
4724 % });
4725 %
4726 % // ===== HOSPITALITY MODULE =====
4727 %
4728 % // Guest Preferences CRUD
4729 % app.get("/api/guests/:id/preferences", async (req, res) => {
4730 %     try {
4731 %         const activeOnly = req.query.active === "true";
4732 %         const prefs = activeOnly
4733 %             ? await storage.getActiveGuestPreferences(req.params.id)
4734 %             : await storage.getGuestPreferences(req.params.id);
4735 %         res.json(prefs);
4736 %     } catch (error) {
4737 %         res.status(500).json({ error: "Error fetching preferences" });
4738 %     }
4739 % });
4740 %
```

server/routes.ts (cont.)

```
4741 %   app.post("/api/guests/:id/preferences", async (req, res) => {
4742 %     try {
4743 %       const pref = await storage.createGuestPreference({ ...req.body, guestId: req.params.id });
4744 %       res.status(201).json(pref);
4745 %     } catch (error) {
4746 %       res.status(500).json({ error: "Error creating preference" });
4747 %     }
4748 %   });
4749 %
4750 %   app.patch("/api/guests/:guestId/preferences/:prefId", async (req, res) => {
4751 %     try {
4752 %       const pref = await storage.updateGuestPreference(parseInt(req.params.prefId), req.body);
4753 %       if (!pref) return res.status(404).json({ error: "Preference not found" });
4754 %       res.json(pref);
4755 %     } catch (error) {
4756 %       res.status(500).json({ error: "Error updating preference" });
4757 %     }
4758 %   });
4759 %
4760 %   app.patch("/api/guests/:guestId/preferences/:prefId/toggle", async (req, res) => {
4761 %     try {
4762 %       const pref = await storage.toggleGuestPreference(parseInt(req.params.prefId));
4763 %       if (!pref) return res.status(404).json({ error: "Preference not found" });
4764 %       res.json(pref);
4765 %     } catch (error) {
4766 %       res.status(500).json({ error: "Error toggling preference" });
4767 %     }
4768 %   });
4769 %
4770 %   app.delete("/api/guests/:guestId/preferences/:prefId", async (req, res) => {
4771 %     try {
4772 %       const deleted = await storage.deleteGuestPreference(parseInt(req.params.prefId));
4773 %       if (!deleted) return res.status(404).json({ error: "Preference not found" });
4774 %       res.json({ success: true });
4775 %     } catch (error) {
4776 %       res.status(500).json({ error: "Error deleting preference" });
4777 %     }
4778 %   });
4779 %
4780 %   // Stay Notes CRUD
4781 %   app.get("/api/reservations/:id/stay-notes", async (req, res) => {
4782 %     try {
4783 %       const notes = await storage.getStayNotes(req.params.id);
4784 %       res.json(notes);
4785 %     } catch (error) {
4786 %       res.status(500).json({ error: "Error fetching stay notes" });
4787 %     }
4788 %   });
4789 %
4790 %   app.get("/api/hospitality/stay-notes/active", async (req, res) => {
4791 %     try {
4792 %       const includeResolved = req.query.includeResolved === "true";
4793 %       if (includeResolved) {
4794 %         const allNotes = Array.from((await storage.getActiveStayNotes()) || []);
4795 %         const allReservations = await storage.getReservations();
4796 %         const checkedInIds = new Set(allReservations.filter((r) => r.status === "checked_in").map((r) => r.id));
4797 %         const allStayNotes: any[] = [];
4798 %         for (const r of allReservations) {
4799 %           const notes = await storage.getStayNotes(r.id);
4800 %           allStayNotes.push(...notes);
4801 %         }
4802 %         res.json(allStayNotes);
4803 %       } else {
4804 %         const notes = await storage.getActiveStayNotes();
4805 %         res.json(notes);
4806 %       }
4807 %     } catch (error) {
4808 %       res.status(500).json({ error: "Error fetching stay notes" });
4809 %     }
4810 %   });
4811 %
4812 %   app.post("/api/reservations/:id/stay-notes", async (req, res) => {
4813 %     try {
4814 %       const note = await storage.createStayNote({ ...req.body, reservationId: req.params.id });
4815 %       res.status(201).json(note);
4816 %     } catch (error) {
4817 %       res.status(500).json({ error: "Error creating stay note" });
4818 %     }
4819 %   });
4820 %
4821 %   app.patch("/api/hospitality/stay-notes/:noteId", async (req, res) => {
4822 %     try {
4823 %       const note = await storage.updateStayNote(parseInt(req.params.noteId), req.body);
4824 %       if (!note) return res.status(404).json({ error: "Note not found" });
4825 %       res.json(note);
4826 %     } catch (error) {
4827 %       res.status(500).json({ error: "Error updating stay note" });
4828 %     }
4829 %   });
4830 %
4831 %   app.patch("/api/hospitality/stay-notes/:noteId/resolve", async (req, res) => {
4832 %     try {
4833 %       const { resolvedBy } = req.body;
```

server/routes.ts (cont.)

```

4834 %         const note = await storage.resolveStayNote(parseInt(req.params.noteId), resolvedBy || "Sistema");
4835 %         if (!note) return res.status(404).json({ error: "Note not found" });
4836 %         res.json(note);
4837 %     } catch (error) {
4838 %         res.status(500).json({ error: "Error resolving stay note" });
4839 %     }
4840 % });
4841 %
4842 % app.delete("/api/hospitality/stay-notes/:noteId", async (req, res) => {
4843 %     try {
4844 %         const deleted = await storage.deleteStayNote(parseInt(req.params.noteId));
4845 %         if (!deleted) return res.status(404).json({ error: "Note not found" });
4846 %         res.json({ success: true });
4847 %     } catch (error) {
4848 %         res.status(500).json({ error: "Error deleting stay note" });
4849 %     }
4850 % });
4851 %
4852 % // Hospitality Alerts
4853 % app.get("/api/hospitality/alerts", async (req, res) => {
4854 %     try {
4855 %         const area = req.query.area as string | undefined;
4856 %         const alerts = await storage.getHospitalityAlerts(area);
4857 %         res.json(alerts);
4858 %     } catch (error) {
4859 %         res.status(500).json({ error: "Error fetching alerts" });
4860 %     }
4861 % });
4862 %
4863 % app.get("/api/hospitality/alerts/reservation/:id", async (req, res) => {
4864 %     try {
4865 %         const alerts = await storage.getHospitalityAlertsByReservation(req.params.id);
4866 %         res.json(alerts);
4867 %     } catch (error) {
4868 %         res.status(500).json({ error: "Error fetching reservation alerts" });
4869 %     }
4870 % });
4871 %
4872 % app.patch("/api/hospitality/alerts/:alertId/acknowledge", async (req, res) => {
4873 %     try {
4874 %         const { acknowledgedBy } = req.body;
4875 %         const alert = await storage.acknowledgeHospitalityAlert(parseInt(req.params.alertId), acknowledgedBy || "Sistem...
4876 %         if (!alert) return res.status(404).json({ error: "Alert not found" });
4877 %         res.json(alert);
4878 %     } catch (error) {
4879 %         res.status(500).json({ error: "Error acknowledging alert" });
4880 %     }
4881 % });
4882 %
4883 % // Hospitality Dashboard
4884 % app.get("/api/hospitality/dashboard", async (req, res) => {
4885 %     try {
4886 %         const allReservations = await storage.getReservations();
4887 %         const checkedIn = allReservations.filter((r) => r.status === "checked_in");
4888 %
4889 %         const guestsWithPrefs = [];
4890 %         for (const r of checkedIn) {
4891 %             if (r.guestId) {
4892 %                 const guest = await storage.getGuest(r.guestId);
4893 %                 const prefs = await storage.getActiveGuestPreferences(r.guestId);
4894 %                 if (guest) {
4895 %                     guestsWithPrefs.push({
4896 %                         guest,
4897 %                         reservation: r,
4898 %                         preferences: prefs,
4899 %                         hasCritical: prefs.some((p) => p.priority === "critical"),
4900 %                         hasHigh: prefs.some((p) => p.priority === "high"),
4901 %                     });
4902 %                 }
4903 %             }
4904 %         }
4905 %
4906 %         const allAlerts = await storage.getHospitalityAlerts();
4907 %         const pendingAlerts = allAlerts.filter((a) => !a.isAcknowledged);
4908 %
4909 %         const allGuests = await storage.getGuests();
4910 %         const allPrefs = [];
4911 %         for (const g of allGuests) {
4912 %             const prefs = await storage.getActiveGuestPreferences(g.id);
4913 %             const specialDates = prefs.filter((p) => p.category === "fecha_especial");
4914 %             if (specialDates.length > 0) {
4915 %                 allPrefs.push({ guest: g, specialDates });
4916 %             }
4917 %         }
4918 %
4919 %         const criticalPrefs = [];
4920 %         for (const g of allGuests) {
4921 %             const prefs = await storage.getActiveGuestPreferences(g.id);
4922 %             const critical = prefs.filter((p) => p.priority === "critical");
4923 %             if (critical.length > 0) {
4924 %                 criticalPrefs.push({ guest: g, preferences: critical });
4925 %             }
4926 %         }

```


server/routes.ts (cont.)

```
4927 %
4928 %         res.json({
4929 %             inHouseGuests: guestsWithPrefs,
4930 %             pendingAlerts,
4931 %             upcomingSpecialDates: allPrefs,
4932 %             criticalPreferences: criticalPrefs,
4933 %             stats: {
4934 %                 totalInHouseWithPrefs: guestsWithPrefs.filter((g) => g.preferences.length > 0).length,
4935 %                 pendingAlertsCount: pendingAlerts.length,
4936 %                 criticalCount: criticalPrefs.reduce((sum, g) => sum + g.preferences.length, 0),
4937 %             },
4938 %         });
4939 %     } catch (error) {
4940 %         res.status(500).json({ error: "Error fetching dashboard" });
4941 %     }
4942 % });
4943 %
4944 % return httpServer;
4945 % }
4946 %
```

```

1 % import {
2 %   type User,
3 %   type InsertUser,
4 %   type Room,
5 %   type InsertRoom,
6 %   type RoomType,
7 %   type InsertRoomType,
8 %   type RatePlan,
9 %   type InsertRatePlan,
10 %  type RatePlanWithRoomType,
11 %  type Company,
12 %  type InsertCompany,
13 %  type Guest,
14 %  type InsertGuest,
15 %  type BedType,
16 %  type InsertBedType,
17 %  type Reservation,
18 %  type InsertReservation,
19 %  type Charge,
20 %  type InsertCharge,
21 %  type Payment,
22 %  type InsertPayment,
23 %  type CancelledReservationLog,
24 %  type InsertCancelledReservationLog,
25 %  type OTChannel,
26 %  type InsertOTChannel,
27 %  type OTChannelWithStats,
28 %  type OTAReservationLog,
29 %  type InsertOTAReservationLog,
30 %  type OTAReservationLogWithChannel,
31 %  type RoomWithType,
32 %  type ReservationWithDetails,
33 %  type RoomStatus,
34 %  type ReservationStatus,
35 %  type ReservationSource,
36 %  type PlanningData,
37 %  type PlanningCellStatus,
38 %  type Group,
39 %  type InsertGroup,
40 %  type GroupRoomBlock,
41 %  type InsertGroupRoomBlock,
42 %  type GroupReservationLink,
43 %  type InsertGroupReservationLink,
44 %  type GroupWithDetails,
45 %  type GroupRoomBlockWithDetails,
46 %  type GuestReview,
47 %  type InsertGuestReview,
48 %  type GuestReviewWithDetails,
49 %  type SentimentType,
50 %  type HousekeepingTask,
51 %  type InsertHousekeepingTask,
52 %  type HousekeepingTaskWithRoom,
53 %  // Restaurant
54 %  type RestaurantArea,
55 %  type InsertRestaurantArea,
56 %  type RestaurantTable,
57 %  type InsertRestaurantTable,
58 %  type RestaurantTableWithArea,
59 %  type MenuCategory,
60 %  type InsertMenuCategory,
61 %  type MenuItem,
62 %  type InsertMenuItem,
63 %  type MenuItemWithCategory,
64 %  type RestaurantOrder,
65 %  type InsertRestaurantOrder,
66 %  type OrderItem,
67 %  type InsertOrderItem,
68 %  type RestaurantOrderWithDetails,
69 %  type TableStatus,
70 %  type OrderStatus,
71 %  type TableReservation,
72 %  type InsertTableReservation,
73 %  type TableReservationWithTable,
74 %  type RestaurantTimeSlot,
75 %  type InsertRestaurantTimeSlot,
76 %  type OrderSplit,
77 %  type InsertOrderSplit,
78 %  type Recipe,
79 %  type InsertRecipe,
80 %  type RecipeIngredient,
81 %  type InsertRecipeIngredient,
82 %  type RecipeWithIngredients,
83 %  // Inventory
84 %  type ItemCategory,
85 %  type InsertItemCategory,
86 %  type Supplier,
87 %  type InsertSupplier,
88 %  type InventoryItem,
89 %  type InsertInventoryItem,
90 %  type InventoryItemWithDetails,

```

server/storage.ts (cont.)

```

91 %     type StockMovement,
92 %     type InsertStockMovement,
93 %     type StockMovementWithItem,
94 %     type PurchaseOrder,
95 %     type InsertPurchaseOrder,
96 %     type PurchaseOrderItem,
97 %     type InsertPurchaseOrderItem,
98 %     type PurchaseOrderWithDetails,
99 %     // SPA
100 %    type SpaCabin,
101 %    type InsertSpaCabin,
102 %    type SpaTreatmentCategory,
103 %    type InsertSpaTreatmentCategory,
104 %    type SpaTreatment,
105 %    type InsertSpaTreatment,
106 %    type SpaAppointment,
107 %    type InsertSpaAppointment,
108 %    type SpaAppointmentWithDetails,
109 %    type SpaAppointmentStatus,
110 %    type SpaAccount,
111 %    type InsertSpaAccount,
112 %    type SpaAccountStatus,
113 %    type SpaAccountItem,
114 %    type InsertSpaAccountItem,
115 %    type SpaAccountWithItems,
116 %    type SpaPayment,
117 %    type InsertSpaPayment,
118 %    type SpaPaymentMethod,
119 %    // Events
120 %    type EventRoom,
121 %    type InsertEventRoom,
122 %    type EventRoomStatus,
123 %    type Event as HotelEvent,
124 %    type InsertEvent,
125 %    type EventStatus,
126 %    type EventType,
127 %    type EventChargeType,
128 %    type InsertEventChargeType,
129 %    type EventCharge,
130 %    type InsertEventCharge,
131 %    type EventWithDetails,
132 %    type EventChargeWithType,
133 %    type EventPlanningData,
134 %    type EventPlanningCellStatus,
135 %    type EventPayment,
136 %    type InsertEventPayment,
137 %    type EventTable,
138 %    type InsertEventTable,
139 %    type EventTableWithDetails,
140 %    type EventTableCharge,
141 %    type InsertEventTableCharge,
142 %    type EventTablePayment,
143 %    type InsertEventTablePayment,
144 %    // Maintenance
145 %    type MaintenanceStaff,
146 %    type InsertMaintenanceStaff,
147 %    type WorkOrder,
148 %    type InsertWorkOrder,
149 %    type WorkOrderWithDetails,
150 %    type WorkOrderStatus,
151 %    type WorkOrderPriority,
152 %    // Administration
153 %    type SystemUser,
154 %    type InsertSystemUser,
155 %    type SystemUserRole,
156 %    type SystemSetting,
157 %    type InsertSystemSetting,
158 %    type AuditLog,
159 %    type InsertAuditLog,
160 %    type AuditAction,
161 %    type WorkOrderCategory,
162 %    // Packages
163 %    type Package,
164 %    type InsertPackage,
165 %    type PackageItem,
166 %    type InsertPackageItem,
167 %    type PackageWithDetails,
168 %    type PackageStatus,
169 %    // Notifications & Web Check-in
170 %    type SystemNotification,
171 %    type InsertSystemNotification,
172 %    type NotificationArea,
173 %    type WebCheckin,
174 %    type InsertWebCheckin,
175 %    // Hospitality
176 %    type GuestPreference,
177 %    type InsertGuestPreference,
178 %    type StayNote,
179 %    type InsertStayNote,
180 %    type HospitalityAlert,
181 %    type InsertHospitalityAlert,
182 % } from "@shared/schema";
183 % import { randomUUID } from "crypto";

```

server/storage.ts (cont.)

```
184 %
185 % export interface IStorage {
186 %   // Users
187 %   getUser(id: string): Promise<User | undefined>;
188 %   getUserByUsername(username: string): Promise<User | undefined>;
189 %   createUser(user: InsertUser): Promise<User>;
190 %
191 %   // Room Types
192 %   getRoomTypes(): Promise<RoomType[]>;
193 %   getRoomType(id: string): Promise<RoomType | undefined>;
194 %   createRoomType(roomType: InsertRoomType): Promise<RoomType>;
195 %   updateRoomType(id: string, roomType: Partial<InsertRoomType>): Promise<RoomType | undefined>;
196 %   deleteRoomType(id: string): Promise<boolean>;
197 %
198 %   // Rate Plans
199 %   getRatePlans(): Promise<RatePlanWithRoomType[]>;
200 %   getRatePlan(id: string): Promise<RatePlanWithRoomType | undefined>;
201 %   getRatePlansByRoomType(roomTypeId: string): Promise<RatePlan[]>;
202 %   createRatePlan(ratePlan: InsertRatePlan): Promise<RatePlan>;
203 %   updateRatePlan(id: string, ratePlan: Partial<InsertRatePlan>): Promise<RatePlan | undefined>;
204 %   deleteRatePlan(id: string): Promise<boolean>;
205 %
206 %   // Rooms
207 %   getRooms(): Promise<RoomWithType[]>;
208 %   getRoom(id: string): Promise<RoomWithType | undefined>;
209 %   createRoom(room: InsertRoom): Promise<Room>;
210 %   updateRoom(id: string, room: Partial<InsertRoom>): Promise<Room | undefined>;
211 %   deleteRoom(id: string): Promise<boolean>;
212 %
213 %   // Companies
214 %   getCompanies(): Promise<Company[]>;
215 %   getCompany(id: string): Promise<Company | undefined>;
216 %   searchCompanies(query: string): Promise<Company[]>;
217 %   createCompany(company: InsertCompany): Promise<Company>;
218 %   updateCompany(id: string, company: Partial<InsertCompany>): Promise<Company | undefined>;
219 %   deleteCompany(id: string): Promise<boolean>;
220 %
221 %   // Guests
222 %   getGuests(): Promise<Guest[]>;
223 %   getGuest(id: string): Promise<Guest | undefined>;
224 %   searchGuests(query: string): Promise<Guest[]>;
225 %   createGuest(guest: InsertGuest): Promise<Guest>;
226 %   updateGuest(id: string, guest: Partial<InsertGuest>): Promise<Guest | undefined>;
227 %   deleteGuest(id: string): Promise<boolean>;
228 %
229 %   // Bed Types
230 %   getBedTypes(): Promise<BedType[]>;
231 %   getBedType(id: number): Promise<BedType | undefined>;
232 %   createBedType(bedType: InsertBedType): Promise<BedType>;
233 %   updateBedType(id: number, bedType: Partial<InsertBedType>): Promise<BedType | undefined>;
234 %   deleteBedType(id: number): Promise<boolean>;
235 %
236 %   // Reservations
237 %   getReservations(): Promise<ReservationWithDetails[]>;
238 %   getReservation(id: string): Promise<ReservationWithDetails | undefined>;
239 %   getReservationByCode(code: string): Promise<ReservationWithDetails | undefined>;
240 %   getRecentReservations(limit: number): Promise<ReservationWithDetails[]>;
241 %   getReservationsForCheckIn(): Promise<ReservationWithDetails[]>;
242 %   getReservationsForCheckOut(): Promise<ReservationWithDetails[]>;
243 %   getCheckInsByDate(date: string): Promise<ReservationWithDetails[]>;
244 %   getReservationsByGuest(guestId: string): Promise<ReservationWithDetails[]>;
245 %   createReservation(reservation: InsertReservation): Promise<Reservation>;
246 %   updateReservation(id: string, reservation: Partial<InsertReservation>): Promise<Reservation | undefined>;
247 %   deleteReservation(id: string): Promise<boolean>;
248 %   generateReservationCode(): string;
249 %
250 %   // Charges
251 %   getCharges(reservationId: string): Promise<Charge[]>;
252 %   getCharge(id: string): Promise<Charge | undefined>;
253 %   createCharge(charge: InsertCharge): Promise<Charge>;
254 %   updateCharge(id: string, charge: Partial<InsertCharge>): Promise<Charge | undefined>;
255 %   deleteCharge(id: string): Promise<boolean>;
256 %   getChargesTotal(reservationId: string): Promise<number>;
257 %
258 %   // Payments
259 %   getPayments(reservationId: string): Promise<Payment[]>;
260 %   createPayment(payment: InsertPayment): Promise<Payment>;
261 %   updatePayment(id: string, payment: Partial<InsertPayment>): Promise<Payment | undefined>;
262 %   deletePayment(id: string): Promise<boolean>;
263 %   getPaymentsTotal(reservationId: string): Promise<number>;
264 %
265 %   // Cancelled Reservation Logs
266 %   getCancelledReservationLogs(): Promise<CancelledReservationLog[]>;
267 %   createCancelledReservationLog(log: InsertCancelledReservationLog): Promise<CancelledReservationLog>;
268 %
269 %   // Overbooking check
270 %   checkOverbooking(roomId: string, checkInDate: string, checkOutDate: string, excludeReservationId?: string): Promise...
271 %
272 %   // Dashboard
273 %   getDashboardStats(): Promise<{
274 %     totalRooms: number;
275 %     availableRooms: number;
276 %     occupiedRooms: number;
```

server/storage.ts (cont.)

```
277 %     dirtyRooms: number;
278 %     cleaningRooms: number;
279 %     maintenanceRooms: number;
280 %     oosRooms: number;
281 %     todayCheckIns: number;
282 %     todayCheckOuts: number;
283 %     occupancyRate: number;
284 %     totalGuests: number;
285 %     pendingReservations: number;
286 % };
287 %
288 % // Planning
289 % getPlanningData(startDate: string, endDate: string): Promise<PlanningData>;
290 %
291 % // OTA Channels
292 % getOTAChannels(): Promise<OTAChannelWithStats[]>;
293 % getOTAChannel(id: string): Promise<OTAChannel | undefined>;
294 % createOTAChannel(channel: InsertOTAChannel): Promise<OTAChannel>;
295 % updateOTAChannel(id: string, channel: Partial<InsertOTAChannel>): Promise<OTAChannel | undefined>;
296 % deleteOTAChannel(id: string): Promise<boolean>;
297 %
298 % // OTA Reservation Logs
299 % getOTAReservationLogs(channelId?: string): Promise<OTAReservationLogWithChannel[]>;
300 % getOTAReservationLog(id: string): Promise<OTAReservationLogWithChannel | undefined>;
301 % createOTAReservationLog(log: InsertOTAReservationLog): Promise<OTAReservationLog>;
302 % updateOTAReservationLog(id: string, log: Partial<InsertOTAReservationLog>): Promise<OTAReservationLog | undefined>;
303 % syncOTAReservation(logId: string): Promise<Reservation | undefined>;
304 %
305 % // Groups
306 % getGroups(): Promise<GroupWithDetails[]>;
307 % getGroup(id: string): Promise<GroupWithDetails | undefined>;
308 % createGroup(group: InsertGroup): Promise<Group>;
309 % updateGroup(id: string, group: Partial<InsertGroup>): Promise<Group | undefined>;
310 % deleteGroup(id: string): Promise<boolean>;
311 % generateGroupCode(): string;
312 %
313 % // Group Room Blocks
314 % getGroupBlocks(groupId: string): Promise<GroupRoomBlockWithDetails[]>;
315 % createGroupBlock(block: InsertGroupRoomBlock): Promise<GroupRoomBlock>;
316 % updateGroupBlock(id: string, block: Partial<InsertGroupRoomBlock>): Promise<GroupRoomBlock | undefined>;
317 % deleteGroupBlock(id: string): Promise<boolean>;
318 %
319 % // Group Reservation Links
320 % getGroupReservationLinks(groupId: string): Promise<GroupReservationLink[]>;
321 % createGroupReservationLink(link: InsertGroupReservationLink): Promise<GroupReservationLink>;
322 % assignRoomToGroup(
323 %     groupId: string,
324 %     roomId: string,
325 %     guestFirstName: string,
326 %     guestLastName: string,
327 %     options?: {
328 %         checkInDate?: string;
329 %         checkOutDate?: string;
330 %         agreedRate?: string;
331 %         ratePlanId?: string | null;
332 %     }
333 % ): Promise<Reservation | undefined>;
334 %
335 % // Guest Reviews
336 % getGuestReviews(): Promise<GuestReviewWithDetails[]>;
337 % getGuestReview(id: string): Promise<GuestReviewWithDetails | undefined>;
338 % getGuestReviewsByGuest(guestId: string): Promise<GuestReviewWithDetails[]>;
339 % createGuestReview(review: InsertGuestReview): Promise<GuestReview>;
340 % updateGuestReview(id: string, review: Partial<InsertGuestReview>): Promise<GuestReview | undefined>;
341 % deleteGuestReview(id: string): Promise<boolean>;
342 % getReviewAnalyticsSummary(): Promise<{
343 %     totalReviews: number;
344 %     averageRating: number;
345 %     sentimentBreakdown: { positive: number; neutral: number; negative: number };
346 %     topCategories: { category: string; count: number; avgSentiment: number }[];
347 %     recentTrend: { date: string; avgRating: number; count: number }[];
348 %     improvementAreas: string[];
349 % }>;
350 %
351 % // Housekeeping Tasks
352 % getHousekeepingTasks(date?: string): Promise<HousekeepingTaskWithRoom[]>;
353 % getHousekeepingTask(id: string): Promise<HousekeepingTaskWithRoom | undefined>;
354 % getHousekeepingTasksByRoom(roomId: string): Promise<HousekeepingTask[]>;
355 % createHousekeepingTask(task: InsertHousekeepingTask): Promise<HousekeepingTask>;
356 % updateHousekeepingTask(id: string, task: Partial<InsertHousekeepingTask>): Promise<HousekeepingTask | undefined>;
357 % deleteHousekeepingTask(id: string): Promise<boolean>;
358 % createCheckoutCleaningTask(roomId: string): Promise<HousekeepingTask>;
359 %
360 % // ===== RESTAURANT =====
361 % // Restaurant Areas
362 % getRestaurantAreas(): Promise<RestaurantArea[]>;
363 % getRestaurantArea(id: string): Promise<RestaurantArea | undefined>;
364 % createRestaurantArea(area: InsertRestaurantArea): Promise<RestaurantArea>;
365 % updateRestaurantArea(id: string, area: Partial<InsertRestaurantArea>): Promise<RestaurantArea | undefined>;
366 % deleteRestaurantArea(id: string): Promise<boolean>;
367 %
368 % // Restaurant Tables
369 % getRestaurantTables(): Promise<RestaurantTableWithArea[]>;
```

server/storage.ts (cont.)

```

370 %   getRestaurantTable(id: string): Promise<RestaurantTableWithArea | undefined>;
371 %   getTablesByArea(areaId: string): Promise<RestaurantTable[]>;
372 %   createRestaurantTable(table: InsertRestaurantTable): Promise<RestaurantTable>;
373 %   updateRestaurantTable(id: string, table: Partial<InsertRestaurantTable>): Promise<RestaurantTable | undefined>;
374 %   deleteRestaurantTable(id: string): Promise<boolean>;
375 %
376 %   // Menu Categories
377 %   getMenuCategories(): Promise<MenuCategory[]>;
378 %   getMenuCategory(id: string): Promise<MenuCategory | undefined>;
379 %   createMenuCategory(category: InsertMenuCategory): Promise<MenuCategory>;
380 %   updateMenuCategory(id: string, category: Partial<InsertMenuCategory>): Promise<MenuCategory | undefined>;
381 %   deleteMenuCategory(id: string): Promise<boolean>;
382 %
383 %   // Menu Items
384 %   getMenuItems(): Promise<MenuItemWithCategory[]>;
385 %   getMenuItem(id: string): Promise<MenuItemWithCategory | undefined>;
386 %   getMenuItemsByCategory(categoryId: string): Promise<MenuItem[]>;
387 %   createMenuItem(item: InsertMenuItem): Promise<MenuItem>;
388 %   updateMenuItem(id: string, item: Partial<InsertMenuItem>): Promise<MenuItem | undefined>;
389 %   deleteMenuItem(id: string): Promise<boolean>;
390 %
391 %   // Restaurant Orders
392 %   getRestaurantOrders(status?: OrderStatus): Promise<RestaurantOrderWithDetails[]>;
393 %   getRestaurantOrder(id: string): Promise<RestaurantOrderWithDetails | undefined>;
394 %   getOrdersByTable(tableId: string): Promise<RestaurantOrder[]>;
395 %   createRestaurantOrder(order: InsertRestaurantOrder): Promise<RestaurantOrder>;
396 %   updateRestaurantOrder(id: string, order: Partial<InsertRestaurantOrder>): Promise<RestaurantOrder | undefined>;
397 %   deleteRestaurantOrder(id: string): Promise<boolean>;
398 %   generateOrderNumber(): string;
399 %
400 %   // Order Items
401 %   getOrderItems(orderId: string): Promise<OrderItem[]>;
402 %   createOrderItem(item: InsertOrderItem): Promise<OrderItem>;
403 %   updateOrderItem(id: string, item: Partial<InsertOrderItem>): Promise<OrderItem | undefined>;
404 %   deleteOrderItem(id: string): Promise<boolean>;
405 %
406 %   // Table Reservations
407 %   getTableReservations(): Promise<TableReservationWithTable[]>;
408 %   getTableReservation(id: string): Promise<TableReservationWithTable | undefined>;
409 %   getTableReservationsByDate(date: string): Promise<TableReservationWithTable[]>;
410 %   getTableReservationsByTable(tableId: string): Promise<TableReservation[]>;
411 %   createTableReservation(reservation: InsertTableReservation): Promise<TableReservation>;
412 %   updateTableReservation(id: string, reservation: Partial<InsertTableReservation>): Promise<TableReservation | undefi...
413 %   deleteTableReservation(id: string): Promise<boolean>;
414 %
415 %   // Restaurant Time Slots
416 %   getRestaurantTimeSlots(): Promise<RestaurantTimeSlot[]>;
417 %   createRestaurantTimeSlot(slot: InsertRestaurantTimeSlot): Promise<RestaurantTimeSlot>;
418 %   updateRestaurantTimeSlot(id: string, slot: Partial<InsertRestaurantTimeSlot>): Promise<RestaurantTimeSlot | undefin...
419 %   deleteRestaurantTimeSlot(id: string): Promise<boolean>;
420 %
421 %   // Recipes
422 %   getRecipes(): Promise<RecipeWithIngredients[]>;
423 %   getRecipe(id: string): Promise<RecipeWithIngredients | undefined>;
424 %   getRecipeByMenuItem(menuItemId: string): Promise<RecipeWithIngredients | undefined>;
425 %   createRecipe(recipe: InsertRecipe): Promise<Recipe>;
426 %   updateRecipe(id: string, recipe: Partial<InsertRecipe>): Promise<Recipe | undefined>;
427 %   deleteRecipe(id: string): Promise<boolean>;
428 %
429 %   // Recipe Ingredients
430 %   getRecipeIngredients(recipeId: string): Promise<RecipeIngredient[]>;
431 %   createRecipeIngredient(ingredient: InsertRecipeIngredient): Promise<RecipeIngredient>;
432 %   updateRecipeIngredient(id: string, ingredient: Partial<InsertRecipeIngredient>): Promise<RecipeIngredient | undefin...
433 %   deleteRecipeIngredient(id: string): Promise<boolean>;
434 %
435 %   // Order Splits
436 %   getOrderSplits(orderId: string): Promise<OrderSplit[]>;
437 %   createOrderSplit(split: InsertOrderSplit): Promise<OrderSplit>;
438 %   updateOrderSplit(id: string, split: Partial<InsertOrderSplit>): Promise<OrderSplit | undefined>;
439 %   deleteOrderSplitsByOrder(orderId: string): Promise<boolean>;
440 %
441 %   // ===== INVENTORY =====
442 %   // Item Categories
443 %   getItemCategories(): Promise<ItemCategory[]>;
444 %   getItemCategory(id: string): Promise<ItemCategory | undefined>;
445 %   createItemCategory(category: InsertItemCategory): Promise<ItemCategory>;
446 %   updateItemCategory(id: string, category: Partial<InsertItemCategory>): Promise<ItemCategory | undefined>;
447 %   deleteItemCategory(id: string): Promise<boolean>;
448 %
449 %   // Suppliers
450 %   getSuppliers(): Promise<Supplier[]>;
451 %   getSupplier(id: string): Promise<Supplier | undefined>;
452 %   createSupplier(supplier: InsertSupplier): Promise<Supplier>;
453 %   updateSupplier(id: string, supplier: Partial<InsertSupplier>): Promise<Supplier | undefined>;
454 %   deleteSupplier(id: string): Promise<boolean>;
455 %
456 %   // Inventory Items
457 %   getInventoryItems(): Promise<InventoryItemWithDetails[]>;
458 %   getInventoryItem(id: string): Promise<InventoryItemWithDetails | undefined>;
459 %   getInventoryItemsBelowMinStock(): Promise<InventoryItem[]>;
460 %   createInventoryItem(item: InsertInventoryItem): Promise<InventoryItem>;
461 %   updateInventoryItem(id: string, item: Partial<InsertInventoryItem>): Promise<InventoryItem | undefined>;
462 %   deleteInventoryItem(id: string): Promise<boolean>;

```

server/storage.ts (cont.)

```
463 %
464 % // Stock Movements
465 % getStockMovements(itemId?: string): Promise<StockMovementWithItem[]>;
466 % createStockMovement(movement: InsertStockMovement): Promise<StockMovement>;
467 %
468 % // ===== SPA =====
469 % // SPA Cabins
470 % getSpaCabins(): Promise<SpaCabin[]>;
471 % getSpaCabin(id: string): Promise<SpaCabin | undefined>;
472 % createSpaCabin(cabin: InsertSpaCabin): Promise<SpaCabin>;
473 % updateSpaCabin(id: string, cabin: Partial<InsertSpaCabin>): Promise<SpaCabin | undefined>;
474 % deleteSpaCabin(id: string): Promise<boolean>;
475 %
476 % // SPA Treatment Categories
477 % getSpaTreatmentCategories(): Promise<SpaTreatmentCategory[]>;
478 % getSpaTreatmentCategory(id: string): Promise<SpaTreatmentCategory | undefined>;
479 % createSpaTreatmentCategory(category: InsertSpaTreatmentCategory): Promise<SpaTreatmentCategory>;
480 % updateSpaTreatmentCategory(id: string, category: Partial<InsertSpaTreatmentCategory>): Promise<SpaTreatmentCategory...
481 % deleteSpaTreatmentCategory(id: string): Promise<boolean>;
482 %
483 % // SPA Treatments
484 % getSpaTreatments(): Promise<SpaTreatment[]>;
485 % getSpaTreatment(id: string): Promise<SpaTreatment | undefined>;
486 % getSpaTreatmentsByCategory(categoryId: string): Promise<SpaTreatment[]>;
487 % createSpaTreatment(treatment: InsertSpaTreatment): Promise<SpaTreatment>;
488 % updateSpaTreatment(id: string, treatment: Partial<InsertSpaTreatment>): Promise<SpaTreatment | undefined>;
489 % deleteSpaTreatment(id: string): Promise<boolean>;
490 %
491 % // SPA Appointments
492 % getSpaAppointments(date?: string): Promise<SpaAppointmentWithDetails[]>;
493 % getSpaAppointment(id: string): Promise<SpaAppointmentWithDetails | undefined>;
494 % getSpaAppointmentsByCabin(cabinId: string, date: string): Promise<SpaAppointment[]>;
495 % getSpaAppointmentsByDateRange(startDate: string, endDate: string): Promise<SpaAppointmentWithDetails[]>;
496 % createSpaAppointment(appointment: InsertSpaAppointment): Promise<SpaAppointment>;
497 % updateSpaAppointment(id: string, appointment: Partial<InsertSpaAppointment>): Promise<SpaAppointment | undefined>;
498 % deleteSpaAppointment(id: string): Promise<boolean>;
499 %
500 % // SPA Accounts
501 % getSpaAccounts(status?: SpaAccountStatus): Promise<SpaAccountWithItems[]>;
502 % getSpaAccount(id: string): Promise<SpaAccountWithItems | undefined>;
503 % getSpaAccountByAppointment(appointmentId: string): Promise<SpaAccountWithItems | undefined>;
504 % createSpaAccount(account: InsertSpaAccount): Promise<SpaAccount>;
505 % updateSpaAccount(id: string, account: Partial<InsertSpaAccount>): Promise<SpaAccount | undefined>;
506 % closeSpaAccount(id: string, chargedTo: string, receiptType?: string): Promise<SpaAccount | undefined>;
507 %
508 % // SPA Account Items
509 % getSpaAccountItems(accountId: string): Promise<SpaAccountItem[]>;
510 % createSpaAccountItem(item: InsertSpaAccountItem): Promise<SpaAccountItem>;
511 % updateSpaAccountItem(id: string, item: Partial<InsertSpaAccountItem>): Promise<SpaAccountItem | undefined>;
512 % deleteSpaAccountItem(id: string): Promise<boolean>;
513 %
514 % // SPA Payments
515 % getSpaPayments(accountId: string): Promise<SpaPayment[]>;
516 % createSpaPayment(payment: InsertSpaPayment): Promise<SpaPayment>;
517 % deleteSpaPayment(id: string): Promise<boolean>;
518 %
519 % // ===== EVENTS =====
520 % // Event Rooms
521 % getEventRooms(): Promise<EventRoom[]>;
522 % getEventRoom(id: string): Promise<EventRoom | undefined>;
523 % createEventRoom(room: InsertEventRoom): Promise<EventRoom>;
524 % updateEventRoom(id: string, room: Partial<InsertEventRoom>): Promise<EventRoom | undefined>;
525 % deleteEventRoom(id: string): Promise<boolean>;
526 %
527 % // Events
528 % getEvents(): Promise<EventWithDetails[]>;
529 % getEvent(id: string): Promise<EventWithDetails | undefined>;
530 % getEventsByDateRange(startDate: string, endDate: string): Promise<EventWithDetails[]>;
531 % createEvent(event: InsertEvent): Promise<HotelEvent>;
532 % updateEvent(id: string, event: Partial<InsertEvent>): Promise<HotelEvent | undefined>;
533 % deleteEvent(id: string): Promise<boolean>;
534 % generateEventCode(): string;
535 %
536 % // Event Charge Types
537 % getEventChargeTypes(): Promise<EventChargeType[]>;
538 % getEventChargeType(id: string): Promise<EventChargeType | undefined>;
539 % createEventChargeType(chargeType: InsertEventChargeType): Promise<EventChargeType>;
540 % updateEventChargeType(id: string, chargeType: Partial<InsertEventChargeType>): Promise<EventChargeType | undefined>;
541 % deleteEventChargeType(id: string): Promise<boolean>;
542 %
543 % // Event Charges
544 % getEventCharges(eventId: string): Promise<EventChargeWithType[]>;
545 % createEventCharge(charge: InsertEventCharge): Promise<EventCharge>;
546 % updateEventCharge(id: string, charge: Partial<InsertEventCharge>): Promise<EventCharge | undefined>;
547 % deleteEventCharge(id: string): Promise<boolean>;
548 %
549 % // Event Payments
550 % getEventPayments(eventId: string): Promise<EventPayment[]>;
551 % createEventPayment(payment: InsertEventPayment): Promise<EventPayment>;
552 % deleteEventPayment(id: string): Promise<boolean>;
553 %
554 % // Event Tables (Evento por Mesa)
555 % getEventTables(eventId: string): Promise<EventTableWithDetails[]>;
```

server/storage.ts (cont.)

```

556 %   getEventTable(id: string): Promise<EventTableWithDetails | undefined>;
557 %   createEventTable(table: InsertEventTable): Promise<EventTable>;
558 %   updateEventTable(id: string, table: Partial<InsertEventTable>): Promise<EventTable | undefined>;
559 %   deleteEventTable(id: string): Promise<boolean>;
560 %
561 %   // Event Table Charges
562 %   getEventTableCharges(tableId: string): Promise<EventTableCharge[]>;
563 %   createEventTableCharge(charge: InsertEventTableCharge): Promise<EventTableCharge>;
564 %   deleteEventTableCharge(id: string): Promise<boolean>;
565 %
566 %   // Event Table Payments
567 %   getEventTablePayments(tableId: string): Promise<EventTablePayment[]>;
568 %   createEventTablePayment(payment: InsertEventTablePayment): Promise<EventTablePayment>;
569 %   deleteEventTablePayment(id: string): Promise<boolean>;
570 %
571 %   // Event Planning
572 %   getEventPlanningData(startDate: string, endDate: string): Promise<EventPlanningData>;
573 %
574 %   // ===== MAINTENANCE =====
575 %   // Maintenance Staff
576 %   getMaintenanceStaff(): Promise<MaintenanceStaff[]>;
577 %   getMaintenanceStaffMember(id: string): Promise<MaintenanceStaff | undefined>;
578 %   createMaintenanceStaff(staff: InsertMaintenanceStaff): Promise<MaintenanceStaff>;
579 %   updateMaintenanceStaff(id: string, staff: Partial<InsertMaintenanceStaff>): Promise<MaintenanceStaff | undefined>;
580 %   deleteMaintenanceStaff(id: string): Promise<boolean>;
581 %
582 %   // Work Orders
583 %   getWorkOrders(): Promise<WorkOrderWithDetails[]>;
584 %   getWorkOrder(id: string): Promise<WorkOrderWithDetails | undefined>;
585 %   getWorkOrdersByRoom(roomId: string): Promise<WorkOrderWithDetails[]>;
586 %   getWorkOrdersByStatus(status: WorkOrderStatus): Promise<WorkOrderWithDetails[]>;
587 %   createWorkOrder(order: InsertWorkOrder): Promise<WorkOrder>;
588 %   updateWorkOrder(id: string, order: Partial<InsertWorkOrder>): Promise<WorkOrder | undefined>;
589 %   deleteWorkOrder(id: string): Promise<boolean>;
590 %   generateWorkOrderCode(): string;
591 %
592 %   // Administration - System Users
593 %   getSystemUsers(): Promise<SystemUser[]>;
594 %   getSystemUser(id: string): Promise<SystemUser | undefined>;
595 %   getSystemUserByUsername(username: string): Promise<SystemUser | undefined>;
596 %   createSystemUser(user: InsertSystemUser): Promise<SystemUser>;
597 %   updateSystemUser(id: string, user: Partial<InsertSystemUser>): Promise<SystemUser | undefined>;
598 %   deleteSystemUser(id: string): Promise<boolean>;
599 %
600 %   // Administration - System Settings
601 %   getSystemSettings(): Promise<SystemSetting[]>;
602 %   getSystemSetting(key: string): Promise<SystemSetting | undefined>;
603 %   getSystemSettingsByCategory(category: string): Promise<SystemSetting[]>;
604 %   upsertSystemSetting(setting: InsertSystemSetting): Promise<SystemSetting>;
605 %   deleteSystemSetting(key: string): Promise<boolean>;
606 %
607 %   // Administration - Audit Logs
608 %   getAuditLogs(): Promise<AuditLog[]>;
609 %   getAuditLogsByModule(module: string): Promise<AuditLog[]>;
610 %   getAuditLogsByUser(userId: string): Promise<AuditLog[]>;
611 %   createAuditLog(log: InsertAuditLog): Promise<AuditLog>;
612 %   getAdminDashboardStats(): Promise<{
613 %       totalUsers: number;
614 %       activeUsers: number;
615 %       recentLogins: number;
616 %       totalSettings: number;
617 %       recentAuditLogs: AuditLog[];
618 %   }>;
619 %
620 %   // ===== PACKAGES =====
621 %   getPackages(): Promise<PackageWithDetails[]>;
622 %   getPackage(id: string): Promise<PackageWithDetails | undefined>;
623 %   getActivePackages(): Promise<PackageWithDetails[]>;
624 %   createPackage(pkg: InsertPackage): Promise<Package>;
625 %   updatePackage(id: string, pkg: Partial<InsertPackage>): Promise<Package | undefined>;
626 %   deletePackage(id: string): Promise<boolean>;
627 %   generatePackageCode(): string;
628 %
629 %   // Package Items
630 %   getPackageItems(packageId: string): Promise<PackageItem[]>;
631 %   createPackageItem(item: InsertPackageItem): Promise<PackageItem>;
632 %   updatePackageItem(id: string, item: Partial<InsertPackageItem>): Promise<PackageItem | undefined>;
633 %   deletePackageItem(id: string): Promise<boolean>;
634 %
635 %   // System Notifications
636 %   getNotifications(area?: NotificationArea, limit?: number): Promise<SystemNotification[]>;
637 %   createNotification(notification: InsertSystemNotification): Promise<SystemNotification>;
638 %   markNotificationRead(id: number): Promise<SystemNotification | undefined>;
639 %   markAllNotificationsRead(area?: NotificationArea): Promise<number>;
640 %   getUnreadNotificationCount(area?: NotificationArea): Promise<number>;
641 %
642 %   // Web Check-in
643 %   createWebCheckin(data: InsertWebCheckin): Promise<WebCheckin>;
644 %   getWebCheckinByToken(token: string): Promise<WebCheckin | undefined>;
645 %   getWebCheckinByReservation(reservationId: string): Promise<WebCheckin | undefined>;
646 %   updateWebCheckin(id: number, data: Partial<InsertWebCheckin>): Promise<WebCheckin | undefined>;
647 %   listWebCheckins(): Promise<WebCheckin[]>;
648 %

```


server/storage.ts (cont.)

```

649 % // Hospitality - Guest Preferences
650 % getGuestPreferences(guestId: string): Promise<GuestPreference[]>;
651 % getActiveGuestPreferences(guestId: string): Promise<GuestPreference[]>;
652 % getGuestPreference(id: number): Promise<GuestPreference | undefined>;
653 % createGuestPreference(pref: InsertGuestPreference): Promise<GuestPreference>;
654 % updateGuestPreference(id: number, pref: Partial<InsertGuestPreference>): Promise<GuestPreference | undefined>;
655 % toggleGuestPreference(id: number): Promise<GuestPreference | undefined>;
656 % deleteGuestPreference(id: number): Promise<boolean>;
657 %
658 % // Hospitality - Stay Notes
659 % getStayNotes(reservationId: string): Promise<StayNote[]>;
660 % getActiveStayNotes(): Promise<StayNote[]>;
661 % createStayNote(note: InsertStayNote): Promise<StayNote>;
662 % updateStayNote(id: number, note: Partial<InsertStayNote>): Promise<StayNote | undefined>;
663 % resolveStayNote(id: number, resolvedBy: string): Promise<StayNote | undefined>;
664 % deleteStayNote(id: number): Promise<boolean>;
665 %
666 % // Hospitality - Alerts
667 % getHospitalityAlerts(area?: string): Promise<HospitalityAlert[]>;
668 % getHospitalityAlertsByReservation(reservationId: string): Promise<HospitalityAlert[]>;
669 % createHospitalityAlert(alert: InsertHospitalityAlert): Promise<HospitalityAlert>;
670 % acknowledgeHospitalityAlert(id: number, acknowledgedBy: string): Promise<HospitalityAlert | undefined>;
671 % }
672 %
673 % export class MemStorage implements IStorage {
674 %   private users: Map<string, User>;
675 %   private roomTypes: Map<string, RoomType>;
676 %   private ratePlans: Map<string, RatePlan>;
677 %   private rooms: Map<string, Room>;
678 %   private companies: Map<string, Company>;
679 %   private guests: Map<string, Guest>;
680 %   private bedTypesMap: Map<number, BedType>;
681 %   private bedTypeCounter: number;
682 %   private reservations: Map<string, Reservation>;
683 %   private charges: Map<string, Charge>;
684 %   private payments: Map<string, Payment>;
685 %   private cancelledReservationLogs: Map<string, CancelledReservationLog>;
686 %   private otaChannels: Map<string, OTAChannel>;
687 %   private otaReservationLogs: Map<string, OTAReservationLog>;
688 %   private groups: Map<string, Group>;
689 %   private groupRoomBlocks: Map<string, GroupRoomBlock>;
690 %   private groupReservationLinks: Map<string, GroupReservationLink>;
691 %   private guestReviews: Map<string, GuestReview>;
692 %   private housekeepingTasks: Map<string, HousekeepingTask>;
693 %   // Restaurant
694 %   private restaurantAreas: Map<string, RestaurantArea>;
695 %   private restaurantTables: Map<string, RestaurantTable>;
696 %   private menuCategories: Map<string, MenuCategory>;
697 %   private menuItems: Map<string, MenuItem>;
698 %   private restaurantOrders: Map<string, RestaurantOrder>;
699 %   private orderItems: Map<string, OrderItem>;
700 %   private tableReservations: Map<string, TableReservation>;
701 %   private restaurantTimeSlots: Map<string, RestaurantTimeSlot>;
702 %   private orderSplitsMap: Map<string, OrderSplit>;
703 %   private recipesMap: Map<string, Recipe>;
704 %   private recipeIngredientsMap: Map<string, RecipeIngredient>;
705 %   // Inventory
706 %   private itemCategories: Map<string, ItemCategory>;
707 %   private suppliers: Map<string, Supplier>;
708 %   private inventoryItems: Map<string, InventoryItem>;
709 %   private stockMovements: Map<string, StockMovement>;
710 %   // SPA
711 %   private spaCabins: Map<string, SpaCabin>;
712 %   private spaTreatmentCategories: Map<string, SpaTreatmentCategory>;
713 %   private spaTreatments: Map<string, SpaTreatment>;
714 %   private spaAppointments: Map<string, SpaAppointment>;
715 %   private spaAccounts: Map<string, SpaAccount>;
716 %   private spaAccountItems: Map<string, SpaAccountItem>;
717 %   private spaPayments: Map<string, SpaPayment>;
718 %   // Events
719 %   private eventRooms: Map<string, EventRoom>;
720 %   private events: Map<string, HotelEvent>;
721 %   private eventChargeTypes: Map<string, EventChargeType>;
722 %   private eventCharges: Map<string, EventCharge>;
723 %   private eventPayments: Map<string, EventPayment>;
724 %   private eventTablesMap: Map<string, EventTable>;
725 %   private eventTableCharges: Map<string, EventTableCharge>;
726 %   private eventTablePayments: Map<string, EventTablePayment>;
727 %   // Maintenance
728 %   private maintenanceStaff: Map<string, MaintenanceStaff>;
729 %   private workOrders: Map<string, WorkOrder>;
730 %   // Administration
731 %   private systemUsers: Map<string, SystemUser>;
732 %   private systemSettings: Map<string, SystemSetting>;
733 %   private auditLogs: Map<string, AuditLog>;
734 %   // Packages
735 %   private packages: Map<string, Package>;
736 %   private packageItems: Map<string, PackageItem>;
737 %   // Notifications & Web Check-in
738 %   private notificationsMap: Map<number, SystemNotification>;
739 %   private notificationCounter: number;
740 %   private webCheckinsMap: Map<number, WebCheckin>;
741 %   private webCheckinCounter: number;

```

server/storage.ts (cont.)

```
742 % // Hospitality
743 % private guestPreferencesMap: Map<number, GuestPreference>;
744 % private guestPreferenceCounter: number;
745 % private stayNotesMap: Map<number, StayNote>;
746 % private stayNoteCounter: number;
747 % private hospitalityAlertsMap: Map<number, HospitalityAlert>;
748 % private hospitalityAlertCounter: number;
749 % // Counters
750 % private reservationCounter: number;
751 % private guestCounter: number;
752 % private groupCounter: number;
753 % private orderCounter: number;
754 % private eventCounter: number;
755 % private workOrderCounter: number;
756 % private packageCounter: number;
757 %
758 % constructor() {
759 %     this.users = new Map();
760 %     this.roomTypes = new Map();
761 %     this.ratePlans = new Map();
762 %     this.rooms = new Map();
763 %     this.companies = new Map();
764 %     this.guests = new Map();
765 %     this.bedTypesMap = new Map();
766 %     this.bedTypeCounter = 0;
767 %     this.reservations = new Map();
768 %     this.charges = new Map();
769 %     this.payments = new Map();
770 %     this.cancelledReservationLogs = new Map();
771 %     this.otaChannels = new Map();
772 %     this.otaReservationLogs = new Map();
773 %     this.groups = new Map();
774 %     this.groupRoomBlocks = new Map();
775 %     this.groupReservationLinks = new Map();
776 %     this.guestReviews = new Map();
777 %     this.housekeepingTasks = new Map();
778 %     // Restaurant
779 %     this.restaurantAreas = new Map();
780 %     this.restaurantTables = new Map();
781 %     this.menuCategories = new Map();
782 %     this.menuItems = new Map();
783 %     this.restaurantOrders = new Map();
784 %     this.orderItems = new Map();
785 %     this.tableReservations = new Map();
786 %     this.restaurantTimeSlots = new Map();
787 %     this.orderSplitsMap = new Map();
788 %     this.recipesMap = new Map();
789 %     this.recipeIngredientsMap = new Map();
790 %     // Inventory
791 %     this.itemCategories = new Map();
792 %     this.suppliers = new Map();
793 %     this.inventoryItems = new Map();
794 %     this.stockMovements = new Map();
795 %     // SPA
796 %     this.spaCabins = new Map();
797 %     this.spaTreatmentCategories = new Map();
798 %     this.spaTreatments = new Map();
799 %     this.spaAppointments = new Map();
800 %     this.spaAccounts = new Map();
801 %     this.spaAccountItems = new Map();
802 %     this.spaPayments = new Map();
803 %     // Events
804 %     this.eventRooms = new Map();
805 %     this.events = new Map();
806 %     this.eventChargeTypes = new Map();
807 %     this.eventCharges = new Map();
808 %     this.eventPayments = new Map();
809 %     this.eventTablesMap = new Map();
810 %     this.eventTableCharges = new Map();
811 %     this.eventTablePayments = new Map();
812 %     // Maintenance
813 %     this.maintenanceStaff = new Map();
814 %     this.workOrders = new Map();
815 %     // Administration
816 %     this.systemUsers = new Map();
817 %     this.systemSettings = new Map();
818 %     this.auditLogs = new Map();
819 %     // Packages
820 %     this.packages = new Map();
821 %     this.packageItems = new Map();
822 %     // Notifications & Web Check-in
823 %     this.notificationsMap = new Map();
824 %     this.notificationCounter = 0;
825 %     this.webCheckinsMap = new Map();
826 %     this.webCheckinCounter = 0;
827 %     // Hospitality
828 %     this.guestPreferencesMap = new Map();
829 %     this.guestPreferenceCounter = 0;
830 %     this.stayNotesMap = new Map();
831 %     this.stayNoteCounter = 0;
832 %     this.hospitalityAlertsMap = new Map();
833 %     this.hospitalityAlertCounter = 0;
834 %     // Counters
```

server/storage.ts (cont.)

```

835 %     this.reservationCounter = 1000;
836 %     this.guestCounter = 0;
837 %     this.groupCounter = 0;
838 %     this.orderCounter = 1000;
839 %     this.eventCounter = 1000;
840 %     this.workOrderCounter = 1000;
841 %     this.packageCounter = 0;
842 %
843 %     // Seed with demo data
844 %     this.seedData();
845 % }
846 %
847 % private seedData() {
848 %     // Create room types - Real hotel categories
849 %     const roomTypes: RoomType[] = [
850 %         { id: "rt1", code: "EJEC", name: "Ejecutiva", description: "Habitación ejecutiva, ideal para viajeros de negoci...",
851 %           { id: "rt2", code: "PREM", name: "Premium", description: "Habitación premium con amenities superiores", baseOcc...
852 %           { id: "rt3", code: "SPAN", name: "Suite Panorámica", description: "Suite con vistas panorámicas y living separa...
853 %           { id: "rt4", code: "SPRES", name: "Suite Presidencial", description: "La suite mas exclusiva del hotel", baseOc...
854 %     ];
855 %     roomTypes.forEach((rt) => this.roomTypes.set(rt.id, rt));
856 %
857 %     // Create bed types
858 %     const defaultBedTypes: BedType[] = [
859 %         { id: 1, code: "SGL", name: "Simple", description: "Cama simple individual", isActive: true, displayOrder: 1, c...
860 %         { id: 2, code: "DBL", name: "Doble", description: "Cama doble matrimonial", isActive: true, displayOrder: 2, cr...
861 %         { id: 3, code: "TWN", name: "Twin - 2 camas separadas", description: "Dos camas individuales separadas", isActi...
862 %         { id: 4, code: "TPL", name: "Triple", description: "Configuración triple (matrimonial + individual)", isActive:...
863 %         { id: 5, code: "MAT_LIV", name: "Matrimonial con Living", description: "Cama matrimonial con living separado", ...
864 %         { id: 6, code: "STE_SOFA", name: "Suite con Sofá Cama", description: "Suite con sofá cama adicional", isActive:...
865 %     ];
866 %     defaultBedTypes.forEach((bt) => this.bedTypesMap.set(bt.id, bt));
867 %     this.bedTypeCounter = 6;
868 %
869 %     // Create rate plans
870 %     const ratePlans: RatePlan[] = [
871 %         { id: "rp1", name: "BAR (Mejor Tarifa)", roomTypeId: "rt1", baseRate: "85.00", currency: "USD", refundable: "tr...
872 %         { id: "rp2", name: "BAR (Mejor Tarifa)", roomTypeId: "rt2", baseRate: "120.00", currency: "USD", refundable: "t...
873 %         { id: "rp3", name: "BAR (Mejor Tarifa)", roomTypeId: "rt3", baseRate: "180.00", currency: "USD", refundable: "t...
874 %         { id: "rp4", name: "BAR (Mejor Tarifa)", roomTypeId: "rt4", baseRate: "350.00", currency: "USD", refundable: "t...
875 %         { id: "rp5", name: "No Reembolsable", roomTypeId: "rt1", baseRate: "70.00", currency: "USD", refundable: "false...
876 %         { id: "rp6", name: "No Reembolsable", roomTypeId: "rt2", baseRate: "100.00", currency: "USD", refundable: "fals...
877 %         { id: "rp7", name: "Corporativo", roomTypeId: "rt1", baseRate: "75.00", currency: "USD", refundable: "true", ca...
878 %         { id: "rp8", name: "Corporativo", roomTypeId: "rt2", baseRate: "105.00", currency: "USD", refundable: "true", c...
879 %         { id: "rp9", name: "Corporativo", roomTypeId: "rt3", baseRate: "160.00", currency: "USD", refundable: "true", c...
880 %     ];
881 %     ratePlans.forEach((rp) => this.ratePlans.set(rp.id, rp));
882 %
883 %     // Create actual hotel rooms based on document
884 %     const rooms: Room[] = [
885 %         // Floor 2
886 %         { id: "r201", roomNumber: "201", roomTypeId: "rt2", floor: 2, status: "available", bedConfig: "MAT_CC_EXTRA", f...
887 %         { id: "r202", roomNumber: "202", roomTypeId: "rt1", floor: 2, status: "available", bedConfig: "TWIN_CC", featur...
888 %         { id: "r204", roomNumber: "204", roomTypeId: "rt2", floor: 2, status: "available", bedConfig: "MAT_CC", feature...
889 %         { id: "r205", roomNumber: "205", roomTypeId: "rt3", floor: 2, status: "available", bedConfig: "MAT_EXTRA", feat...
890 %         { id: "r206", roomNumber: "206", roomTypeId: "rt3", floor: 2, status: "available", bedConfig: "MAT_CC_EXTRA", f...
891 %         { id: "r207", roomNumber: "207", roomTypeId: "rt2", floor: 2, status: "available", bedConfig: "MAT", features: ...
892 %         // Floor 3
893 %         { id: "r301", roomNumber: "301", roomTypeId: "rt2", floor: 3, status: "available", bedConfig: "MAT_CC_EXTRA", f...
894 %         { id: "r302", roomNumber: "302", roomTypeId: "rt1", floor: 3, status: "available", bedConfig: "TWIN_CC", featur...
895 %         { id: "r303", roomNumber: "303", roomTypeId: "rt1", floor: 3, status: "available", bedConfig: "MAT_CC", feature...
896 %         { id: "r304", roomNumber: "304", roomTypeId: "rt1", floor: 3, status: "available", bedConfig: "MAT", features: ...
897 %         { id: "r305", roomNumber: "305", roomTypeId: "rt3", floor: 3, status: "available", bedConfig: "MAT_EXTRA", feat...
898 %         { id: "r306", roomNumber: "306", roomTypeId: "rt3", floor: 3, status: "available", bedConfig: "MAT_CC_EXTRA", f...
899 %         { id: "r307", roomNumber: "307", roomTypeId: "rt2", floor: 3, status: "available", bedConfig: "MAT_CC", feature...
900 %         // Floor 4
901 %         { id: "r401", roomNumber: "401", roomTypeId: "rt2", floor: 4, status: "available", bedConfig: "MAT_CC_EXTRA", f...
902 %         { id: "r402", roomNumber: "402", roomTypeId: "rt1", floor: 4, status: "available", bedConfig: "TWIN_CC", featur...
903 %         { id: "r403", roomNumber: "403", roomTypeId: "rt1", floor: 4, status: "available", bedConfig: "MAT_CC", feature...
904 %         { id: "r404", roomNumber: "404", roomTypeId: "rt1", floor: 4, status: "available", bedConfig: "MAT", features: ...
905 %         { id: "r405", roomNumber: "405", roomTypeId: "rt3", floor: 4, status: "available", bedConfig: "MAT_EXTRA", feat...
906 %         { id: "r406", roomNumber: "406", roomTypeId: "rt3", floor: 4, status: "available", bedConfig: "MAT_EXTRA", feat...
907 %         { id: "r407", roomNumber: "407", roomTypeId: "rt2", floor: 4, status: "available", bedConfig: "MAT_CC", feature...
908 %         // Floor 5
909 %         { id: "r501", roomNumber: "501", roomTypeId: "rt2", floor: 5, status: "available", bedConfig: "MAT_CC_EXTRA", f...
910 %         { id: "r502", roomNumber: "502", roomTypeId: "rt1", floor: 5, status: "available", bedConfig: "MAT_CC", feature...
911 %         { id: "r503", roomNumber: "503", roomTypeId: "rt1", floor: 5, status: "available", bedConfig: "MAT_CC", feature...
912 %         { id: "r504", roomNumber: "504", roomTypeId: "rt1", floor: 5, status: "available", bedConfig: "MAT", features: ...
913 %         { id: "r505", roomNumber: "505", roomTypeId: "rt3", floor: 5, status: "available", bedConfig: "MAT_CC_EXTRA", f...
914 %         { id: "r506", roomNumber: "506", roomTypeId: "rt3", floor: 5, status: "available", bedConfig: "MAT_EXTRA", feat...
915 %         { id: "r507", roomNumber: "507", roomTypeId: "rt2", floor: 5, status: "available", bedConfig: "MAT_CC", feature...
916 %         // Floor 6
917 %         { id: "r601", roomNumber: "601", roomTypeId: "rt2", floor: 6, status: "available", bedConfig: "TWIN_CC_EXTRA", ...
918 %         { id: "r602", roomNumber: "602", roomTypeId: "rt1", floor: 6, status: "available", bedConfig: "TWIN_CC", featur...
919 %         { id: "r603", roomNumber: "603", roomTypeId: "rt1", floor: 6, status: "available", bedConfig: "MAT_CC", feature...
920 %         { id: "r604", roomNumber: "604", roomTypeId: "rt1", floor: 6, status: "available", bedConfig: "MAT", features: ...
921 %         { id: "r605", roomNumber: "605", roomTypeId: "rt3", floor: 6, status: "available", bedConfig: "MAT_CC_EXTRA", f...
922 %         { id: "r606", roomNumber: "606", roomTypeId: "rt3", floor: 6, status: "available", bedConfig: "MAT_EXTRA", feat...
923 %         { id: "r607", roomNumber: "607", roomTypeId: "rt2", floor: 6, status: "available", bedConfig: "MAT", features: ...
924 %     ];
925 %     rooms.forEach((r) => this.rooms.set(r.id, r));
926 %
927 %     // Create companies

```

server/storage.ts (cont.)

```

928 %     const seedDate = new Date().toISOString().split("T")[0];
929 %     const companies: Company[] = [
930 %       { id: "comp1", razonSocial: "TechCorp Argentina S.A.", nombreFantasia: "TechCorp", direccion: "Av. del Libertad...",
931 %       { id: "comp2", razonSocial: "Consultoría Global S.R.L.", nombreFantasia: "ConsultGlobal", direccion: "Callao 50...",
932 %       { id: "comp3", razonSocial: "Exportadora del Sur S.A.", nombreFantasia: "ExportSur", direccion: "Bv. Oroño 2000...
933 %     ];
934 %     companies.forEach((c) => this.companies.set(c.id, c));
935 %
936 %     // Create guests
937 %     const guestSeedDate = new Date().toISOString().split("T")[0];
938 %     const guests: Guest[] = [
939 %       { id: "g1", codigo: "H-2025-0001", firstName: "Carlos", lastName: "García", email: "carlos.garcia@email.com", p...
940 %       { id: "g2", codigo: "H-2025-0002", firstName: "Maria", lastName: "López", email: "maria.lopez@email.com", phone...
941 %       { id: "g3", codigo: "H-2025-0003", firstName: "John", lastName: "Smith", email: "john.smith@email.com", phone: ...
942 %       { id: "g4", codigo: "H-2025-0004", firstName: "Ana", lastName: "Martínez", email: "ana.martinez@email.com", pho...
943 %       { id: "g5", codigo: "H-2025-0005", firstName: "Roberto", lastName: "Fernández", email: "roberto.f@email.com", p...
944 %       { id: "g6", codigo: "H-2025-0006", firstName: "Laura", lastName: "Pérez", email: "laura.p@email.com", phone: "+...
945 %       { id: "g7", codigo: "H-2025-0007", firstName: "Diego", lastName: "Ramírez", email: "diego.r@email.com", phone: ...
946 %       { id: "g8", codigo: "H-2025-0008", firstName: "Sophie", lastName: "Martin", email: "sophie.m@email.com", phone:...
947 %     ];
948 %     guests.forEach((g) => this.guests.set(g.id, g));
949 %     this.guestCounter = 8;
950 %
951 %     // Create reservations with varied dates using real room IDs
952 %     const today = new Date().toISOString().split("T")[0];
953 %     const tomorrow = new Date(Date.now() + 86400000).toISOString().split("T")[0];
954 %     const dayAfter = new Date(Date.now() + 2 * 86400000).toISOString().split("T")[0];
955 %     const in3Days = new Date(Date.now() + 3 * 86400000).toISOString().split("T")[0];
956 %     const in5Days = new Date(Date.now() + 5 * 86400000).toISOString().split("T")[0];
957 %     const nextWeek = new Date(Date.now() + 7 * 86400000).toISOString().split("T")[0];
958 %     const in10Days = new Date(Date.now() + 10 * 86400000).toISOString().split("T")[0];
959 %
960 %     const reservations: Reservation[] = [
961 %       { id: "res1", reservationCode: "RES-1001", guestId: "g1", companyId: null, roomTypeId: "rt2", roomId: "r201", r...
962 %       { id: "res2", reservationCode: "RES-1002", guestId: "g2", companyId: null, roomTypeId: "rt3", roomId: "r305", r...
963 %       { id: "res3", reservationCode: "RES-1003", guestId: "g3", companyId: null, roomTypeId: "rt3", roomId: "r505", r...
964 %       { id: "res4", reservationCode: "RES-1004", guestId: "g4", companyId: null, roomTypeId: "rt2", roomId: "r401", r...
965 %       { id: "res5", reservationCode: "RES-1005", guestId: "g5", companyId: "comp1", roomTypeId: "rt1", roomId: "r302"...
966 %       { id: "res6", reservationCode: "RES-1006", guestId: "g6", companyId: null, roomTypeId: "rt1", roomId: "r202", r...
967 %       { id: "res7", reservationCode: "RES-1007", guestId: "g7", companyId: null, roomTypeId: "rt2", roomId: "r307", r...
968 %       { id: "res8", reservationCode: "RES-1008", guestId: "g8", companyId: null, roomTypeId: "rt3", roomId: "r405", r...
969 %     ];
970 %     reservations.forEach((r) => this.reservations.set(r.id, r));
971 %
972 %     // Update room statuses for checked-in reservations
973 %     const room201 = this.rooms.get("r201");
974 %     if (room201) this.rooms.set("r201", { ...room201, status: "occupied" });
975 %     const room305 = this.rooms.get("r305");
976 %     if (room305) this.rooms.set("r305", { ...room305, status: "occupied" });
977 %     const room505 = this.rooms.get("r505");
978 %     if (room505) this.rooms.set("r505", { ...room505, status: "occupied" });
979 %     const room401 = this.rooms.get("r401");
980 %     if (room401) this.rooms.set("r401", { ...room401, status: "occupied" });
981 %
982 %     // Create sample charges for checked-in reservations
983 %     const charges: Charge[] = [
984 %       { id: "ch1", reservationId: "res1", description: "Alojamiento - 1 noche", amount: "120.00", date: today, catego...
985 %       { id: "ch2", reservationId: "res2", description: "Alojamiento - 7 noches", amount: "1134.00", date: today, cate...
986 %       { id: "ch2b", reservationId: "res2", description: "Minibar", amount: "25.00", date: today, category: "minibar",...
987 %       { id: "ch3", reservationId: "res3", description: "Alojamiento - 3 noches", amount: "540.00", date: today, cate...
988 %       { id: "ch3b", reservationId: "res3", description: "Restaurante - Cena", amount: "85.00", date: today, category:...
989 %       { id: "ch4", reservationId: "res4", description: "Alojamiento - 2 noches", amount: "240.00", date: today, cate...
990 %     ];
991 %     charges.forEach((c) => this.charges.set(c.id, c));
992 %
993 %     this.reservationCounter = 1008;
994 %
995 %     // Restaurant Areas - Dos secciones del plano + areas sin mesas
996 %     const restaurantAreas: RestaurantArea[] = [
997 %       { id: "area1", name: "Sector Bodega (Mesas 1-18)", areaType: "indoor", capacity: 72, hasTables: "true", isActiv...
998 %       { id: "area2", name: "Sector Moneda (Mesas 19-32)", areaType: "indoor", capacity: 56, hasTables: "true", isActi...
999 %       { id: "area-rs", name: "Room Service", areaType: "private", capacity: 0, hasTables: "false", isActive: "true", ...
1000 %       { id: "area-delivery", name: "Delivery", areaType: "private", capacity: 0, hasTables: "false", isActive: "true"...
1001 %       { id: "area-solarium", name: "Solarium", areaType: "outdoor", capacity: 0, hasTables: "false", isActive: "true"...
1002 %       { id: "area-spa", name: "SPA", areaType: "private", capacity: 0, hasTables: "false", isActive: "true", notes: n...
1003 %     ];
1004 %     restaurantAreas.forEach((a) => this.restaurantAreas.set(a.id, a));
1005 %
1006 %     // Restaurant Tables - Plano Justo - Seccion A (Mesas 1-18)
1007 %     // Layout del plano:
1008 %     // Fila 1: 18, 17, 15
1009 %     // Fila 2: 8, 6
1010 %     // Fila 3: 1, 2, 3
1011 %     // Fila 4: 16, 14 | 4, 5
1012 %     // Fila 5: 7, 9
1013 %     // Fila 6: 10, 13, 12, 11
1014 %     const restaurantTables: RestaurantTable[] = [
1015 %       // Seccion A - Mesas 1-18 (cuadradas)
1016 %       { id: "t1", tableNumber: "1", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 0,...
1017 %       { id: "t2", tableNumber: "2", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 1,...
1018 %       { id: "t3", tableNumber: "3", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 2,...
1019 %       { id: "t4", tableNumber: "4", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 2,...
1020 %       { id: "t5", tableNumber: "5", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 3,...

```

server/storage.ts (cont.)

```
1021 % { id: "t6", tableNumber: "6", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 1,...
1022 % { id: "t7", tableNumber: "7", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 0,...
1023 % { id: "t8", tableNumber: "8", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 0,...
1024 % { id: "t9", tableNumber: "9", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: 1,...
1025 % { id: "t10", tableNumber: "10", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1026 % { id: "t11", tableNumber: "11", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1027 % { id: "t12", tableNumber: "12", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1028 % { id: "t13", tableNumber: "13", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1029 % { id: "t14", tableNumber: "14", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1030 % { id: "t15", tableNumber: "15", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1031 % { id: "t16", tableNumber: "16", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1032 % { id: "t17", tableNumber: "17", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1033 % { id: "t18", tableNumber: "18", areaId: "area1", capacity: 4, shape: "square", status: "available", positionX: ...
1034 % // Seccion B - Mesas 19-32 (redondas)
1035 % // Layout del plano:
1036 % // Fila 1: 20
1037 % // Fila 2: 32, 29, 30, 31
1038 % // Fila 3: 26, 27, 28
1039 % // Fila 4: 23, 24, 25
1040 % // Fila 5: 21, 22
1041 % // Fila 6: 19
1042 % { id: "t19", tableNumber: "19", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 0...
1043 % { id: "t20", tableNumber: "20", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 0...
1044 % { id: "t21", tableNumber: "21", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 0...
1045 % { id: "t22", tableNumber: "22", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 1...
1046 % { id: "t23", tableNumber: "23", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 0...
1047 % { id: "t24", tableNumber: "24", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 1...
1048 % { id: "t25", tableNumber: "25", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 2...
1049 % { id: "t26", tableNumber: "26", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 0...
1050 % { id: "t27", tableNumber: "27", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 1...
1051 % { id: "t28", tableNumber: "28", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 2...
1052 % { id: "t29", tableNumber: "29", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 1...
1053 % { id: "t30", tableNumber: "30", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 2...
1054 % { id: "t31", tableNumber: "31", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 3...
1055 % { id: "t32", tableNumber: "32", areaId: "area2", capacity: 4, shape: "round", status: "available", positionX: 0...
1056 % ];
1057 % restaurantTables.forEach((t) => this.restaurantTables.set(t.id, t));
1058 %
1059 % // Menu Categories (orden: 1-Bebidas sin alcohol, 2-Bebidas con alcohol, 3-Entradas, 4-Principales, 5-Postres)
1060 % const menuCategories: MenuCategory[] = [
1061 %   { id: "mc5", name: "Bebidas sin Alcohol", description: "Aguas, gaseosas y jugos", displayOrder: 1, isActive: "t...
1062 %   { id: "mc6", name: "Bebidas con Alcohol", description: "Vinos, cervezas y cocktails", displayOrder: 2, isActive...
1063 %   { id: "mc1", name: "Entradas", description: "Para comenzar", displayOrder: 3, isActive: "true" },
1064 %   { id: "mc2", name: "Platos Principales", description: "Carnes, pastas y pescados", displayOrder: 4, isActive: "...
1065 %   { id: "mc3", name: "Postres", description: "Dulces y helados", displayOrder: 5, isActive: "true" },
1066 % ];
1067 % menuCategories.forEach((c) => this.menuCategories.set(c.id, c));
1068 %
1069 % // Menu Items
1070 % const menuItems: MenuItem[] = [
1071 %   { id: "mi1", categoryId: "mc1", name: "Empanadas (3 unidades)", description: "Carne cortada a cuchillo", price:...
1072 %   { id: "mi2", categoryId: "mc1", name: "Provolone", description: "Queso provolone a la plancha con oregano", pri...
1073 %   { id: "mi3", categoryId: "mc1", name: "Tabla de Fiambres", description: "Jamon crudo, salamin, quesos", price: ...
1074 %   { id: "mi4", categoryId: "mc2", name: "Bife de Chorizo", description: "400g, con guarnicion a eleccion", price:...
1075 %   { id: "mi5", categoryId: "mc2", name: "Salmon Grille", description: "Con vegetales de estacion", price: "14000...
1076 %   { id: "mi6", categoryId: "mc2", name: "Ravioles de Ricota", description: "Con salsa bolognesa o filetto", price...
1077 %   { id: "mi7", categoryId: "mc2", name: "Pollo a la Parrilla", description: "Medio pollo con ensalada", price: "7...
1078 %   { id: "mi8", categoryId: "mc3", name: "Flan con Dulce de Leche", description: "Casero", price: "2800.00", prepa...
1079 %   { id: "mi9", categoryId: "mc3", name: "Helado (3 bochas)", description: "Sabores a eleccion", price: "3200.00",...
1080 %   { id: "mi10", categoryId: "mc3", name: "Tiramisu", description: "Postre italiano clasico", price: "4500.00", pr...
1081 %   { id: "mi11", categoryId: "mc5", name: "Agua Mineral", description: "Con o sin gas 500ml", price: "1200.00", pr...
1082 %   { id: "mi12", categoryId: "mc5", name: "Gaseosa", description: "Coca-Cola, Sprite, Fanta", price: "1500.00", pr...
1083 %   { id: "mi15", categoryId: "mc5", name: "Jugo de Naranja", description: "Exprimido natural", price: "1800.00", p...
1084 %   { id: "mi13", categoryId: "mc6", name: "Copa de Vino Malbec", description: "Bodega Luigi Bosca", price: "3500.0...
1085 %   { id: "mi14", categoryId: "mc6", name: "Cerveza Artesanal", description: "Pinta 500ml", price: "2800.00", prepa...
1086 %   { id: "mi16", categoryId: "mc6", name: "Fernet con Cola", description: "Branca con Coca-Cola", price: "3000.00"...
1087 % ];
1088 % menuItems.forEach((i) => this.menuItems.set(i.id, i));
1089 %
1090 % // Inventory Categories
1091 % const itemCategories: ItemCategory[] = [
1092 %   { id: "ic1", name: "Alimentos", description: "Productos alimenticios", parentId: null, isActive: "true" },
1093 %   { id: "ic2", name: "Bebidas", description: "Bebidas alcoholicas y sin alcohol", parentId: null, isActive: "true...
1094 %   { id: "ic3", name: "Limpieza", description: "Productos de limpieza", parentId: null, isActive: "true" },
1095 %   { id: "ic4", name: "Amenities", description: "Articulos de tocador para huéspedes", parentId: null, isActive: "...
1096 %   { id: "ic5", name: "Manteleria", description: "Sabananas, toallas, manteles", parentId: null, isActive: "true" },
1097 % ];
1098 % itemCategories.forEach((c) => this.itemCategories.set(c.id, c));
1099 %
1100 % // Suppliers
1101 % const suppliers: Supplier[] = [
1102 %   { id: "sup1", name: "Distribuidora Norte S.A.", contactName: "Juan Perez", phone: "+54 343 456-7890", email: "v...
1103 %   { id: "sup2", name: "Bebidas Premium", contactName: "Maria Garcia", phone: "+54 343 567-8901", email: "pedidos@...
1104 %   { id: "sup3", name: "Limpieza Total S.R.L.", contactName: "Carlos Lopez", phone: "+54 343 678-9012", email: "ve...
1105 % ];
1106 % suppliers.forEach((s) => this.suppliers.set(s.id, s));
1107 %
1108 % // Inventory Items (some items intentionally below minStock to show low stock alerts)
1109 % const inventoryItems: InventoryItem[] = [
1110 %   { id: "inv1", sku: "ALI-001", name: "Cafe en grano", description: "Cafe colombiano premium", categoryId: "ic1",...
1111 %   { id: "inv2", sku: "ALI-002", name: "Azucar", description: "Azucar comun", categoryId: "ic1", supplierId: "sup1...
1112 %   { id: "inv3", sku: "BEB-001", name: "Agua Mineral 500ml", description: "Pack x24", categoryId: "ic2", supplierI...
1113 %   { id: "inv4", sku: "BEB-002", name: "Coca-Cola 500ml", description: "Pack x24", categoryId: "ic2", supplierId: ...
```

server/storage.ts (cont.)

```
1114 %     { id: "inv5", sku: "BEB-003", name: "Vino Malbec Reserva", description: "Bodega Luigi Bosca", categoryId: "ic2"...
1115 %     { id: "inv6", sku: "LIM-001", name: "Detergente Industrial", description: "Bidon 5L", categoryId: "ic3", suppli...
1116 %     { id: "inv7", sku: "LIM-002", name: "Desinfectante", description: "Bidon 5L", categoryId: "ic3", supplierId: "s...
1117 %     { id: "inv8", sku: "AME-001", name: "Shampoo Individual", description: "Sachet 30ml x100", categoryId: "ic4", s...
1118 %     { id: "inv9", sku: "AME-002", name: "Jabon Individual", description: "Pastilla 20g x100", categoryId: "ic4", su...
1119 %     { id: "inv10", sku: "MAN-001", name: "Toallas Blancas", description: "Toalla 70x140cm", categoryId: "ic5", supp...
1120 % ];
1121 % inventoryItems.forEach((i) => this.inventoryItems.set(i.id, i));
1122 %
1123 % // SPA Cabins (Gabinetes)
1124 % const spaCabins: SpaCabin[] = [
1125 %     { id: "cab1", name: "Cabina 1 - Masajes", description: "Cabina para masajes relajantes y terapeuticos", isActiv...
1126 %     { id: "cab2", name: "Cabina 2 - Masajes", description: "Cabina para masajes con aromaterapia", isActive: "true" },
1127 %     { id: "cab3", name: "Cabina 3 - Faciales", description: "Cabina especializada en tratamientos faciales", isActi...
1128 %     { id: "cab4", name: "Cabina 4 - Corporales", description: "Cabina para tratamientos corporales", isActive: "tru...
1129 %     { id: "cab5", name: "Cabina 5 - VIP", description: "Cabina VIP para tratamientos premium", isActive: "true" },
1130 %     { id: "cab6", name: "Cabina 6 - Parejas", description: "Cabina doble para tratamientos en pareja", isActive: "t...
1131 % ];
1132 % spaCabins.forEach((c) => this.spaCabins.set(c.id, c));
1133 %
1134 % // SPA Treatment Categories
1135 % const spaTreatmentCategories: SpaTreatmentCategory[] = [
1136 %     { id: "stc1", name: "Masajes", description: "Masajes relajantes y terapeuticos", sortOrder: 1 },
1137 %     { id: "stc2", name: "Faciales", description: "Tratamientos de limpieza y rejuvenecimiento facial", sortOrder: 2 },
1138 %     { id: "stc3", name: "Corporales", description: "Tratamientos corporales de embellecimiento", sortOrder: 3 },
1139 %     { id: "stc4", name: "Circuitos", description: "Circuitos de aguas termales", sortOrder: 4 },
1140 %     { id: "stc5", name: "Especiales", description: "Tratamientos premium y paquetes especiales", sortOrder: 5 },
1141 % ];
1142 % spaTreatmentCategories.forEach((c) => this.spaTreatmentCategories.set(c.id, c));
1143 %
1144 % // SPA Treatments
1145 % const spaTreatments: SpaTreatment[] = [
1146 %     // Masajes
1147 %     { id: "st1", categoryId: "stc1", name: "Masaje Relajante", description: "Masaje corporal con aceites esenciales...
1148 %     { id: "st2", categoryId: "stc1", name: "Masaje Descontracturante", description: "Masaje profundo para aliviar t...
1149 %     { id: "st3", categoryId: "stc1", name: "Masaje con Piedras Calientes", description: "Terapia con piedras volcan...
1150 %     { id: "st4", categoryId: "stc1", name: "Reflexologia Podal", description: "Masaje de pies con tecnica reflexolo...
1151 %     // Faciales
1152 %     { id: "st5", categoryId: "stc2", name: "Limpieza Facial Profunda", description: "Limpieza e hidratacion profund...
1153 %     { id: "st6", categoryId: "stc2", name: "Tratamiento Antiage", description: "Tratamiento rejuvenecedor con colag...
1154 %     { id: "st7", categoryId: "stc2", name: "Mascara de Oro", description: "Mascara facial premium con particulas de...
1155 %     // Corporales
1156 %     { id: "st8", categoryId: "stc3", name: "Exfoliacion Corporal", description: "Exfoliacion con sales marinas", du...
1157 %     { id: "st9", categoryId: "stc3", name: "Envoltura de Chocolate", description: "Tratamiento hidratante con cacao...
1158 %     { id: "st10", categoryId: "stc3", name: "Reductor Modelador", description: "Tratamiento reductivo con vendas fr...
1159 %     // Circuitos
1160 %     { id: "st11", categoryId: "stc4", name: "Circuito de Aguas", description: "Acceso a piscinas termales, sauna y ...
1161 %     { id: "st12", categoryId: "stc4", name: "Circuito Premium", description: "Circuito de aguas + te y frutas", dur...
1162 %     // Especiales
1163 %     { id: "st13", categoryId: "stc5", name: "Dia de Spa Completo", description: "Circuito + masaje + facial + almue...
1164 %     { id: "st14", categoryId: "stc5", name: "Experiencia en Pareja", description: "Circuito + masaje para dos perso...
1165 % ];
1166 % spaTreatments.forEach((t) => this.spaTreatments.set(t.id, t));
1167 %
1168 % // Event Rooms (5 salones de eventos)
1169 % const eventRooms: EventRoom[] = [
1170 %     { id: "er1", name: "Salon Parana", capacity: 100, status: "available", description: "Salon principal con vista ...
1171 %     { id: "er2", name: "Salon Victoria", capacity: 60, status: "available", description: "Salon ejecutivo para reun...
1172 %     { id: "er3", name: "Salon Diamante", capacity: 40, status: "available", description: "Sala de conferencias", am...
1173 %     { id: "er4", name: "Salon Esmeralda", capacity: 30, status: "available", description: "Sala de reuniones ejecut...
1174 %     { id: "er5", name: "Terraza Eventos", capacity: 150, status: "available", description: "Espacio al aire libre p...
1175 % ];
1176 % eventRooms.forEach((r) => this.eventRooms.set(r.id, r));
1177 %
1178 % // Event Charge Types (tipos de cargo predefinidos)
1179 % const eventChargeTypes: EventChargeType[] = [
1180 %     { id: "ect1", code: "CB1", name: "Coffee Break 1", defaultPrice: "8500.00", isActive: "true" },
1181 %     { id: "ect2", code: "CB2", name: "Coffee Break 2", defaultPrice: "12000.00", isActive: "true" },
1182 %     { id: "ect3", code: "CB3", name: "Coffee Break 3", defaultPrice: "15000.00", isActive: "true" },
1183 %     { id: "ect4", code: "CENA", name: "Cena Ejecutiva", defaultPrice: "35000.00", isActive: "true" },
1184 %     { id: "ect5", code: "COCKTAIL", name: "Cocktail", defaultPrice: "25000.00", isActive: "true" },
1185 %     { id: "ect6", code: "SOC1", name: "Social 1", defaultPrice: "18000.00", isActive: "true" },
1186 %     { id: "ect7", code: "SOC2", name: "Social 2", defaultPrice: "28000.00", isActive: "true" },
1187 %     { id: "ect8", code: "SOC3", name: "Social 3", defaultPrice: "38000.00", isActive: "true" },
1188 % ];
1189 % eventChargeTypes.forEach((ct) => this.eventChargeTypes.set(ct.id, ct));
1190 %
1191 % // Maintenance Staff (Personal de Mantenimiento)
1192 % const maintenanceStaffData: MaintenanceStaff[] = [
1193 %     { id: "ms1", name: "Carlos Rodriguez", phone: "+54 343 456-7890", email: "carlos.rodriguez@maransuites.com", sp...
1194 %     { id: "ms2", name: "Miguel Fernandez", phone: "+54 343 456-7891", email: "miguel.fernandez@maransuites.com", sp...
1195 %     { id: "ms3", name: "Jorge Martinez", phone: "+54 343 456-7892", email: "jorge.martinez@maransuites.com", specia...
1196 %     { id: "ms4", name: "Roberto Sanchez", phone: "+54 343 456-7893", email: "roberto.sanchez@maransuites.com", spec...
1197 % ];
1198 % maintenanceStaffData.forEach((s) => this.maintenanceStaff.set(s.id, s));
1199 %
1200 % // System Users (Usuarios del Sistema)
1201 % const now = new Date().toISOString();
1202 % const systemUsersData: SystemUser[] = [
1203 %     { id: "su1", username: "admin", email: "admin@maransuites.com", fullName: "Administrador Sistema", role: "admin...
1204 %     { id: "su2", username: "gerencia", email: "gerencia@maransuites.com", fullName: "Gerente General", role: "manag...
1205 %     { id: "su3", username: "recepcion1", email: "recepcion1@maransuites.com", fullName: "Maria Garcia", role: "rece...
1206 %     { id: "su4", username: "housekeeping1", email: "housekeeping@maransuites.com", fullName: "Ana Martinez", role: ...
```

server/storage.ts (cont.)

```

1207 %     { id: "su5", username: "restaurante1", email: "restaurante@maransuites.com", fullName: "Carlos Lopez", role: "r...
1208 %     { id: "su6", username: "spa1", email: "spa@maransuites.com", fullName: "Laura Fernandez", role: "spa" as System...
1209 % ];
1210 % systemUsersData.forEach((u => this.systemUsers.set(u.id, u));
1211 %
1212 % // System Settings (Configuracion del Sistema)
1213 % const systemSettingsData: SystemSetting[] = [
1214 %   { id: "ss1", key: "hotel_name", value: "Maran Suites & Towers", category: "general", description: "Nombre del h...
1215 %   { id: "ss2", key: "hotel_address", value: "Alameda de la Federacion 343, Parana, Entre Rios", category: "genera...
1216 %   { id: "ss3", key: "hotel_phone", value: "+54 343 400-0000", category: "general", description: "Telefono princip...
1217 %   { id: "ss4", key: "hotel_email", value: "info@maransuites.com", category: "general", description: "Email de con...
1218 %   { id: "ss5", key: "check_in_time", value: "15:00", category: "reservations", description: "Hora de check-in", u...
1219 %   { id: "ss6", key: "check_out_time", value: "11:00", category: "reservations", description: "Hora de check-out",...
1220 %   { id: "ss7", key: "default_currency", value: "ARS", category: "billing", description: "Moneda predeterminada", ...
1221 %   { id: "ss8", key: "tax_rate", value: "21", category: "billing", description: "Tasa de IVA (%)", updatedAt: now,...
1222 %   { id: "ss9", key: "breakfast_included", value: "true", category: "amenities", description: "Desayuno incluido p...
1223 %   { id: "ss10", key: "wifi_password", value: "MaranGuest2025", category: "amenities", description: "Contraseña Wi...
1224 % ];
1225 % systemSettingsData.forEach((s => this.systemSettings.set(s.id, s));
1226 %
1227 % // Sample Audit Logs
1228 % const auditLogsData: AuditLog[] = [
1229 %   { id: "al1", userId: "su1", userName: "Administrador Sistema", action: "login" as AuditAction, module: "system"...
1230 %   { id: "al2", userId: "su3", userName: "Maria Garcia", action: "create" as AuditAction, module: "reservations", ...
1231 % ];
1232 % auditLogsData.forEach((l => this.auditLogs.set(l.id, l));
1233 %
1234 % // Seed chatbot notifications
1235 % const seedNow = new Date();
1236 % const chatbotNotifications: SystemNotification[] = [
1237 %   {
1238 %     id: 1, type: "chatbot_housekeeping" as any, title: "Solicitud de Carlos García - Hab. 201",
1239 %     message: "Solicita toallas adicionales para la habitación", targetArea: "housekeeping" as any,
1240 %     relatedEntityType: "room", relatedEntityId: "201", isRead: false, readAt: null, readBy: null,
1241 %     priority: "normal" as any, createdAt: new Date(seedNow.getTime() - 45 * 60000),
1242 %   },
1243 %   {
1244 %     id: 2, type: "chatbot_restaurant" as any, title: "Solicitud de María López - Hab. 305",
1245 %     message: "Consulta horario de desayuno para mañana y si tienen opciones sin gluten",
1246 %     targetArea: "restaurant" as any, relatedEntityType: "room", relatedEntityId: "305",
1247 %     isRead: false, readAt: null, readBy: null, priority: "normal" as any,
1248 %     createdAt: new Date(seedNow.getTime() - 30 * 60000),
1249 %   },
1250 %   {
1251 %     id: 3, type: "chatbot_maintenance" as any, title: "Solicitud de John Smith - Hab. 505",
1252 %     message: "El aire acondicionado no enfría correctamente, la habitación está muy calurosa",
1253 %     targetArea: "maintenance" as any, relatedEntityType: "room", relatedEntityId: "505",
1254 %     isRead: false, readAt: null, readBy: null, priority: "urgent" as any,
1255 %     createdAt: new Date(seedNow.getTime() - 15 * 60000),
1256 %   },
1257 %   {
1258 %     id: 4, type: "chatbot_spa" as any, title: "Solicitud de Ana Martínez - Hab. 401",
1259 %     message: "Quiere reservar un masaje relajante para las 16:00 de hoy",
1260 %     targetArea: "spa" as any, relatedEntityType: "room", relatedEntityId: "401",
1261 %     isRead: false, readAt: null, readBy: null, priority: "normal" as any,
1262 %     createdAt: new Date(seedNow.getTime() - 10 * 60000),
1263 %   },
1264 %   {
1265 %     id: 5, type: "chatbot_housekeeping" as any, title: "Solicitud de Diego Ramírez - Hab. 307",
1266 %     message: "Necesita almohada extra y una manta adicional por favor",
1267 %     targetArea: "housekeeping" as any, relatedEntityType: "room", relatedEntityId: "307",
1268 %     isRead: true, readAt: new Date(seedNow.getTime() - 60 * 60000), readBy: "reception",
1269 %     priority: "normal" as any, createdAt: new Date(seedNow.getTime() - 120 * 60000),
1270 %   },
1271 %   {
1272 %     id: 6, type: "chatbot_request" as any, title: "Solicitud de Sophie Martin - Hab. 405",
1273 %     message: "Consulta sobre el horario de check-out y si es posible late check-out",
1274 %     targetArea: "reception" as any, relatedEntityType: "room", relatedEntityId: "405",
1275 %     isRead: true, readAt: new Date(seedNow.getTime() - 90 * 60000), readBy: "reception",
1276 %     priority: "normal" as any, createdAt: new Date(seedNow.getTime() - 180 * 60000),
1277 %   },
1278 % ];
1279 % chatbotNotifications.forEach((n => this.notificationsMap.set(n.id, n));
1280 % this.notificationCounter = chatbotNotifications.length;
1281 %
1282 % const prefNow = new Date();
1283 % const seedPreferences: GuestPreference[] = [
1284 %   { id: 1, guestId: "g1", category: "alimentacion", subcategory: "alergias", title: "Alergia al maní", descriptio...
1285 %   { id: 2, guestId: "g1", category: "habitacion", subcategory: "ubicacion", title: "Piso alto", description: "Pre...
1286 %   { id: 3, guestId: "g1", category: "amenities", subcategory: "almohadas", title: "Almohadas extra", description:...
1287 %   { id: 4, guestId: "g2", category: "fecha_especial", subcategory: "cumpleaños", title: "Cumpleaños 22 de julio",...
1288 %   { id: 5, guestId: "g2", category: "alimentacion", subcategory: "dieta", title: "Vegetariana", description: "Die...
1289 %   { id: 6, guestId: "g3", category: "habitacion", subcategory: "almohadas", title: "Almohada hipoalérgica", des...
1290 %   { id: 7, guestId: "g3", category: "servicio", subcategory: "idioma", title: "Idioma inglés", description: "Pref...
1291 %   { id: 8, guestId: "g4", category: "alimentacion", subcategory: "alergias", title: "Intolerancia a lactosa", des...
1292 % ];
1293 % seedPreferences.forEach((p => this.guestPreferencesMap.set(p.id, p));
1294 % this.guestPreferenceCounter = seedPreferences.length;
1295 % }
1296 %
1297 % // Users
1298 % async getUser(id: string): Promise<User | undefined> {
1299 %   return this.users.get(id);

```

server/storage.ts (cont.)

```
1300 % }
1301 %
1302 % async getUserByUsername(username: string): Promise<User | undefined> {
1303 %   return Array.from(this.users.values()).find((user) => user.username === username);
1304 % }
1305 %
1306 % async createUser(insertUser: InsertUser): Promise<User> {
1307 %   const id = randomUUID();
1308 %   const user: User = {
1309 %     id,
1310 %     username: insertUser.username,
1311 %     password: insertUser.password,
1312 %     firstName: insertUser.firstName ?? null,
1313 %     lastName: insertUser.lastName ?? null,
1314 %     role: (insertUser.role ?? "reception") as "reception" | "housekeeping" | "management" | "director" | "restauran...
1315 %   };
1316 %   this.users.set(id, user);
1317 %   return user;
1318 % }
1319 %
1320 % // Room Types
1321 % async getRoomTypes(): Promise<RoomType[]> {
1322 %   return Array.from(this.roomTypes.values());
1323 % }
1324 %
1325 % async getRoomType(id: string): Promise<RoomType | undefined> {
1326 %   return this.roomTypes.get(id);
1327 % }
1328 %
1329 % async createRoomType(roomType: InsertRoomType): Promise<RoomType> {
1330 %   const id = randomUUID();
1331 %   const newRoomType: RoomType = {
1332 %     id,
1333 %     code: roomType.code,
1334 %     name: roomType.name,
1335 %     description: roomType.description ?? null,
1336 %     baseOccupancy: roomType.baseOccupancy ?? 2,
1337 %     maxOccupancy: roomType.maxOccupancy ?? 4,
1338 %   };
1339 %   this.roomTypes.set(id, newRoomType);
1340 %   return newRoomType;
1341 % }
1342 %
1343 % async updateRoomType(id: string, updates: Partial<InsertRoomType>): Promise<RoomType | undefined> {
1344 %   const roomType = this.roomTypes.get(id);
1345 %   if (!roomType) return undefined;
1346 %   const updatedRoomType: RoomType = { ...roomType, ...updates };
1347 %   this.roomTypes.set(id, updatedRoomType);
1348 %   return updatedRoomType;
1349 % }
1350 %
1351 % async deleteRoomType(id: string): Promise<boolean> {
1352 %   return this.roomTypes.delete(id);
1353 % }
1354 %
1355 % // Rate Plans
1356 % async getRatePlans(): Promise<RatePlanWithRoomType[]> {
1357 %   const ratePlans = Array.from(this.ratePlans.values());
1358 %   return ratePlans.map((rp) => ({
1359 %     ...rp,
1360 %     roomType: this.roomTypes.get(rp.roomTypeId)!,
1361 %   }));
1362 % }
1363 %
1364 % async getRatePlan(id: string): Promise<RatePlanWithRoomType | undefined> {
1365 %   const ratePlan = this.ratePlans.get(id);
1366 %   if (!ratePlan) return undefined;
1367 %   return {
1368 %     ...ratePlan,
1369 %     roomType: this.roomTypes.get(ratePlan.roomTypeId)!,
1370 %   };
1371 % }
1372 %
1373 % async getRatePlansByRoomType(roomTypeId: string): Promise<RatePlan[]> {
1374 %   return Array.from(this.ratePlans.values()).filter((rp) => rp.roomTypeId === roomTypeId);
1375 % }
1376 %
1377 % async createRatePlan(ratePlan: InsertRatePlan): Promise<RatePlan> {
1378 %   const id = randomUUID();
1379 %   const newRatePlan: RatePlan = {
1380 %     id,
1381 %     name: ratePlan.name,
1382 %     roomTypeId: ratePlan.roomTypeId,
1383 %     baseRate: ratePlan.baseRate,
1384 %     currency: ratePlan.currency ?? "ARS",
1385 %     refundable: ratePlan.refundable ?? "true",
1386 %     cancellationPolicy: ratePlan.cancellationPolicy ?? null,
1387 %   };
1388 %   this.ratePlans.set(id, newRatePlan);
1389 %   return newRatePlan;
1390 % }
1391 %
1392 % async updateRatePlan(id: string, updates: Partial<InsertRatePlan>): Promise<RatePlan | undefined> {
```


server/storage.ts (cont.)

```
1393 %     const ratePlan = this.ratePlans.get(id);
1394 %     if (!ratePlan) return undefined;
1395 %     const updatedRatePlan: RatePlan = { ...ratePlan, ...updates };
1396 %     this.ratePlans.set(id, updatedRatePlan);
1397 %     return updatedRatePlan;
1398 % }
1399 %
1400 % async deleteRatePlan(id: string): Promise<boolean> {
1401 %     return this.ratePlans.delete(id);
1402 % }
1403 %
1404 % // Rooms
1405 % async getRooms(): Promise<RoomWithType[]> {
1406 %     const rooms = Array.from(this.rooms.values());
1407 %     return rooms.map((room) => ({
1408 %         ...room,
1409 %         roomType: this.roomTypes.get(room.roomTypeId)!,
1410 %     }));
1411 % }
1412 %
1413 % async getRoom(id: string): Promise<RoomWithType | undefined> {
1414 %     const room = this.rooms.get(id);
1415 %     if (!room) return undefined;
1416 %     return {
1417 %         ...room,
1418 %         roomType: this.roomTypes.get(room.roomTypeId)!,
1419 %     };
1420 % }
1421 %
1422 % async createRoom(insertRoom: InsertRoom): Promise<Room> {
1423 %     const id = randomUUID();
1424 %     const room: Room = {
1425 %         id,
1426 %         roomNumber: insertRoom.roomNumber,
1427 %         roomTypeId: insertRoom.roomTypeId,
1428 %         floor: insertRoom.floor ?? 1,
1429 %         status: (insertRoom.status ?? "available") as RoomStatus,
1430 %         bedConfig: insertRoom.bedConfig ?? null,
1431 %         features: insertRoom.features ?? null,
1432 %         maxOccupancy: insertRoom.maxOccupancy ?? 2,
1433 %         notes: insertRoom.notes ?? null,
1434 %     };
1435 %     this.rooms.set(id, room);
1436 %     return room;
1437 % }
1438 %
1439 % async updateRoom(id: string, updates: Partial<InsertRoom>): Promise<Room | undefined> {
1440 %     const room = this.rooms.get(id);
1441 %     if (!room) return undefined;
1442 %     const updatedRoom: Room = {
1443 %         ...room,
1444 %         ...updates,
1445 %         status: (updates.status ?? room.status) as RoomStatus,
1446 %     };
1447 %     this.rooms.set(id, updatedRoom);
1448 %     return updatedRoom;
1449 % }
1450 %
1451 % async deleteRoom(id: string): Promise<boolean> {
1452 %     return this.rooms.delete(id);
1453 % }
1454 %
1455 % // Companies
1456 % async getCompanies(): Promise<Company[]> {
1457 %     return Array.from(this.companies.values()).filter((c) => c.isActive === "true");
1458 % }
1459 %
1460 % async getCompany(id: string): Promise<Company | undefined> {
1461 %     return this.companies.get(id);
1462 % }
1463 %
1464 % async searchCompanies(query: string): Promise<Company[]> {
1465 %     const lowerQuery = query.toLowerCase();
1466 %     return Array.from(this.companies.values()).filter((c) =>
1467 %         c.isActive === "true" && (
1468 %             c.razonSocial.toLowerCase().includes(lowerQuery) ||
1469 %             c.nombreFantasia?.toLowerCase().includes(lowerQuery) ||
1470 %             c.cuilCuit.toLowerCase().includes(lowerQuery)
1471 %         )
1472 %     );
1473 % }
1474 %
1475 % async createCompany(insertCompany: InsertCompany): Promise<Company> {
1476 %     const id = randomUUID();
1477 %     const company: Company = {
1478 %         id,
1479 %         razonSocial: insertCompany.razonSocial,
1480 %         nombreFantasia: insertCompany.nombreFantasia ?? null,
1481 %         direccion: insertCompany.direccion ?? null,
1482 %         pais: insertCompany.pais ?? "Argentina",
1483 %         codigoPostal: insertCompany.codigoPostal ?? null,
1484 %         localidad: insertCompany.localidad ?? null,
1485 %         provincia: insertCompany.provincia ?? null,
```

server/storage.ts (cont.)

```

1486 %         telefono: insertCompany.telefono ?? null,
1487 %         email: insertCompany.email ?? null,
1488 %         cuilCuit: insertCompany.cuilCuit,
1489 %         numeroFiscal: insertCompany.numeroFiscal ?? null,
1490 %         condicionIva: (insertCompany.condicionIva ?? "responsable_inscripto") as "responsable_inscripto" | "monotributo...",
1491 %         inscripcionNacional: insertCompany.inscripcionNacional ?? null,
1492 %         inscripcionProvincial: insertCompany.inscripcionProvincial ?? null,
1493 %         contactName: insertCompany.contactName ?? null,
1494 %         contactEmail: insertCompany.contactEmail ?? null,
1495 %         contactPhone: insertCompany.contactPhone ?? null,
1496 %         creditLimit: insertCompany.creditLimit ?? "0",
1497 %         paymentTermDays: insertCompany.paymentTermDays ?? 30,
1498 %         notes: insertCompany.notes ?? null,
1499 %         isActive: insertCompany.isActive ?? "true",
1500 %         createdAt: new Date().toISOString().split("T")[0],
1501 %     };
1502 %     this.companies.set(id, company);
1503 %     return company;
1504 % }
1505 %
1506 % async updateCompany(id: string, updates: Partial<InsertCompany>): Promise<Company | undefined> {
1507 %     const company = this.companies.get(id);
1508 %     if (!company) return undefined;
1509 %     const updatedCompany: Company = {
1510 %         ...company,
1511 %         ...updates,
1512 %         condicionIva: (updates.condicionIva ?? company.condicionIva) as "responsable_inscripto" | "monotributo" | "exen...",
1513 %     };
1514 %     this.companies.set(id, updatedCompany);
1515 %     return updatedCompany;
1516 % }
1517 %
1518 % async deleteCompany(id: string): Promise<boolean> {
1519 %     const company = this.companies.get(id);
1520 %     if (!company) return false;
1521 %     company.isActive = "false";
1522 %     this.companies.set(id, company);
1523 %     return true;
1524 % }
1525 %
1526 % // Guests
1527 % async getGuests(): Promise<Guest[]> {
1528 %     return Array.from(this.guests.values());
1529 % }
1530 %
1531 % async getGuest(id: string): Promise<Guest | undefined> {
1532 %     return this.guests.get(id);
1533 % }
1534 %
1535 % async searchGuests(query: string): Promise<Guest[]> {
1536 %     const lowerQuery = query.toLowerCase();
1537 %     return Array.from(this.guests.values()).filter((g) =>
1538 %         g.firstName.toLowerCase().includes(lowerQuery) ||
1539 %         g.lastName.toLowerCase().includes(lowerQuery) ||
1540 %         g.email?.toLowerCase().includes(lowerQuery) ||
1541 %         g.documentNumber?.toLowerCase().includes(lowerQuery)
1542 %     );
1543 % }
1544 %
1545 % private generateGuestCode(): string {
1546 %     this.guestCounter++;
1547 %     const year = new Date().getFullYear();
1548 %     return `H-${year}-${this.guestCounter.toString().padStart(4, "0")}`;
1549 % }
1550 %
1551 % async createGuest(insertGuest: InsertGuest): Promise<Guest> {
1552 %     const id = randomUUID();
1553 %     const guest: Guest = {
1554 %         id,
1555 %         codigo: this.generateGuestCode(),
1556 %         firstName: insertGuest.firstName,
1557 %         lastName: insertGuest.lastName,
1558 %         email: insertGuest.email ?? null,
1559 %         phone: insertGuest.phone ?? null,
1560 %         documentType: insertGuest.documentType ?? null,
1561 %         documentNumber: insertGuest.documentNumber ?? null,
1562 %         nationality: insertGuest.nationality ?? null,
1563 %         direccion: insertGuest.direccion ?? null,
1564 %         localidad: insertGuest.localidad ?? null,
1565 %         codigoPostal: insertGuest.codigoPostal ?? null,
1566 %         fechaNacimiento: insertGuest.fechaNacimiento ?? null,
1567 %         sexo: (insertGuest.sexo ?? "no_especifica") as "masculino" | "femenino" | "otro" | "no_especifica",
1568 %         segment: (insertGuest.segment ?? "LEISURE") as "LEISURE" | "CORP" | "SPORT" | "CONGRESS" | "OTHER",
1569 %         cuilCuit: insertGuest.cuilCuit ?? null,
1570 %         companyId: insertGuest.companyId ?? null,
1571 %         fechaAlta: new Date().toISOString().split("T")[0],
1572 %         vehiculoPatente: insertGuest.vehiculoPatente ?? null,
1573 %         vehiculoMarca: insertGuest.vehiculoMarca ?? null,
1574 %         vehiculoModelo: insertGuest.vehiculoModelo ?? null,
1575 %         vehiculoColor: insertGuest.vehiculoColor ?? null,
1576 %     };
1577 %     this.guests.set(id, guest);
1578 %     return guest;

```

server/storage.ts (cont.)

```
1579 %     }
1580 %
1581 %     async updateGuest(id: string, updates: Partial<InsertGuest>): Promise<Guest | undefined> {
1582 %         const guest = this.guests.get(id);
1583 %         if (!guest) return undefined;
1584 %         const updatedGuest: Guest = {
1585 %             ...guest,
1586 %             ...updates,
1587 %             sexo: (updates.sexo ?? guest.sexo) as "masculino" | "femenino" | "otro" | "no_especifica",
1588 %             segment: (updates.segment ?? guest.segment) as "LEISURE" | "CORP" | "SPORT" | "CONGRESS" | "OTHER",
1589 %         };
1590 %         this.guests.set(id, updatedGuest);
1591 %         return updatedGuest;
1592 %     }
1593 %
1594 %     async deleteGuest(id: string): Promise<boolean> {
1595 %         return this.guests.delete(id);
1596 %     }
1597 %
1598 %     // Bed Types
1599 %     async getBedTypes(): Promise<BedType[]> {
1600 %         return Array.from(this.bedTypesMap.values()).sort((a, b) => a.displayOrder - b.displayOrder);
1601 %     }
1602 %
1603 %     async getBedType(id: number): Promise<BedType | undefined> {
1604 %         return this.bedTypesMap.get(id);
1605 %     }
1606 %
1607 %     async createBedType(bedType: InsertBedType): Promise<BedType> {
1608 %         this.bedTypeCounter++;
1609 %         const newBedType: BedType = {
1610 %             id: this.bedTypeCounter,
1611 %             code: bedType.code,
1612 %             name: bedType.name,
1613 %             description: bedType.description ?? null,
1614 %             isActive: bedType.isActive ?? true,
1615 %             displayOrder: bedType.displayOrder ?? 0,
1616 %             createdAt: new Date(),
1617 %         };
1618 %         this.bedTypesMap.set(newBedType.id, newBedType);
1619 %         return newBedType;
1620 %     }
1621 %
1622 %     async updateBedType(id: number, bedType: Partial<InsertBedType>): Promise<BedType | undefined> {
1623 %         const existing = this.bedTypesMap.get(id);
1624 %         if (!existing) return undefined;
1625 %         const updated = { ...existing, ...bedType };
1626 %         this.bedTypesMap.set(id, updated);
1627 %         return updated;
1628 %     }
1629 %
1630 %     async deleteBedType(id: number): Promise<boolean> {
1631 %         const existing = this.bedTypesMap.get(id);
1632 %         if (!existing) return false;
1633 %         const updated = { ...existing, isActive: false };
1634 %         this.bedTypesMap.set(id, updated);
1635 %         return true;
1636 %     }
1637 %
1638 %     // Reservations
1639 %     private enrichReservation(reservation: Reservation): ReservationWithDetails {
1640 %         const guest = this.guests.get(reservation.guestId);
1641 %         const room = this.rooms.get(reservation.roomId);
1642 %         const roomType = room ? this.roomTypes.get(room.roomTypeId) : undefined;
1643 %         const ratePlan = reservation.ratePlanId ? this.ratePlans.get(reservation.ratePlanId) : undefined;
1644 %         const charges = Array.from(this.charges.values()).filter((c) => c.reservationId === reservation.id);
1645 %
1646 %         return {
1647 %             ...reservation,
1648 %             guest: guest!,
1649 %             room: room ? { ...room, roomType } : undefined as any,
1650 %             ratePlan,
1651 %             charges,
1652 %         };
1653 %     }
1654 %
1655 %     generateReservationCode(): string {
1656 %         this.reservationCounter++;
1657 %         return `RES-${this.reservationCounter}`;
1658 %     }
1659 %
1660 %     async getReservations(): Promise<ReservationWithDetails[]> {
1661 %         const reservations = Array.from(this.reservations.values());
1662 %         return reservations.map((r) => this.enrichReservation(r)).sort((a, b) =>
1663 %             new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()
1664 %         );
1665 %     }
1666 %
1667 %     async getReservation(id: string): Promise<ReservationWithDetails | undefined> {
1668 %         const reservation = this.reservations.get(id);
1669 %         if (!reservation) return undefined;
1670 %         return this.enrichReservation(reservation);
1671 %     }
1671 % }
```

server/storage.ts (cont.)

```

1672 %
1673 % async getReservationByCode(code: string): Promise<ReservationWithDetails | undefined> {
1674 %   const reservation = Array.from(this.reservations.values()).find((r) => r.reservationCode === code);
1675 %   if (!reservation) return undefined;
1676 %   return this.enrichReservation(reservation);
1677 % }
1678 %
1679 % async getRecentReservations(limit: number): Promise<ReservationWithDetails[]> {
1680 %   const reservations = await this.getReservations();
1681 %   return reservations.slice(0, limit);
1682 % }
1683 %
1684 % async getReservationsForCheckIn(): Promise<ReservationWithDetails[]> {
1685 %   const reservations = await this.getReservations();
1686 %   return reservations.filter((r) => r.status === "confirmed");
1687 % }
1688 %
1689 % async getReservationsForCheckOut(): Promise<ReservationWithDetails[]> {
1690 %   const reservations = await this.getReservations();
1691 %   return reservations.filter((r) => r.status === "checked_in");
1692 % }
1693 %
1694 % async getCheckInsByDate(date: string): Promise<ReservationWithDetails[]> {
1695 %   const reservations = await this.getReservations();
1696 %   return reservations.filter((r) => r.checkInDate === date && r.status === "checked_in");
1697 % }
1698 %
1699 % async getReservationsByGuest(guestId: string): Promise<ReservationWithDetails[]> {
1700 %   const reservations = await this.getReservations();
1701 %   return reservations.filter((r) => r.guestId === guestId);
1702 % }
1703 %
1704 % async createReservation(insertReservation: InsertReservation): Promise<Reservation> {
1705 %   const id = randomUUID();
1706 %   const reservation: Reservation = {
1707 %     id,
1708 %     reservationCode: insertReservation.reservationCode,
1709 %     guestId: insertReservation.guestId,
1710 %     companyId: insertReservation.companyId ?? null,
1711 %     roomTypeId: insertReservation.roomTypeId,
1712 %     roomId: insertReservation.roomId,
1713 %     ratePlanId: insertReservation.ratePlanId ?? null,
1714 %     checkInDate: insertReservation.checkInDate,
1715 %     checkOutDate: insertReservation.checkOutDate,
1716 %     nights: insertReservation.nights ?? 1,
1717 %     baseRatePerNight: insertReservation.baseRatePerNight ?? null,
1718 %     discountType: (insertReservation.discountType ?? "none") as "none" | "percent" | "fixed",
1719 %     discountValue: insertReservation.discountValue ?? "0",
1720 %     finalRatePerNight: insertReservation.finalRatePerNight ?? null,
1721 %     totalRoomAmount: insertReservation.totalRoomAmount ?? null,
1722 %     status: (insertReservation.status ?? "pending") as ReservationStatus,
1723 %     source: (insertReservation.source ?? "directo") as any,
1724 %     otaChannelId: insertReservation.otaChannelId ?? null,
1725 %     externalReservationId: insertReservation.externalReservationId ?? null,
1726 %     numberOfGuests: insertReservation.numberOfGuests ?? 1,
1727 %     notes: insertReservation.notes ?? null,
1728 %     createdAt: insertReservation.createdAt || new Date().toISOString(),
1729 %     lastModifiedBy: insertReservation.lastModifiedBy ?? null,
1730 %   };
1731 %   this.reservations.set(id, reservation);
1732 %   return reservation;
1733 % }
1734 %
1735 % async updateReservation(id: string, updates: Partial<InsertReservation>): Promise<Reservation | undefined> {
1736 %   const reservation = this.reservations.get(id);
1737 %   if (!reservation) return undefined;
1738 %   const updatedReservation: Reservation = {
1739 %     ...reservation,
1740 %     ...updates,
1741 %     status: (updates.status ?? reservation.status) as ReservationStatus,
1742 %     discountType: (updates.discountType ?? reservation.discountType) as "none" | "percent" | "fixed",
1743 %     source: (updates.source ?? reservation.source) as any,
1744 %     otaChannelId: updates.otaChannelId !== undefined ? updates.otaChannelId : reservation.otaChannelId,
1745 %     externalReservationId: updates.externalReservationId !== undefined ? updates.externalReservationId : reservatio...
1746 %   };
1747 %   this.reservations.set(id, updatedReservation);
1748 %   return updatedReservation;
1749 % }
1750 %
1751 % async deleteReservation(id: string): Promise<boolean> {
1752 %   return this.reservations.delete(id);
1753 % }
1754 %
1755 % // Charges
1756 % async getCharges(reservationId: string): Promise<Charge[]> {
1757 %   return Array.from(this.charges.values()).filter((c) => c.reservationId === reservationId);
1758 % }
1759 %
1760 % async getCharge(id: string): Promise<Charge | undefined> {
1761 %   return this.charges.get(id);
1762 % }
1763 %
1764 % async createCharge(charge: InsertCharge): Promise<Charge> {

```

server/storage.ts (cont.)

```

1765 %     const id = randomUUID();
1766 %     const newCharge: Charge = {
1767 %         id,
1768 %         reservationId: charge.reservationId,
1769 %         description: charge.description,
1770 %         amount: charge.amount,
1771 %         date: charge.date,
1772 %         category: (charge.category ?? "otros") as "room" | "restaurant" | "spa" | "minibar" | "otros" | "adjustment",
1773 %         createdBy: charge.createdBy ?? null,
1774 %     };
1775 %     this.charges.set(id, newCharge);
1776 %     return newCharge;
1777 % }
1778 %
1779 % async updateCharge(id: string, updates: Partial<InsertCharge>): Promise<Charge | undefined> {
1780 %     const charge = this.charges.get(id);
1781 %     if (!charge) return undefined;
1782 %     const updatedCharge: Charge = {
1783 %         ...charge,
1784 %         ...updates,
1785 %         category: (updates.category ?? charge.category) as "room" | "restaurant" | "spa" | "minibar" | "otros" | "adju...
1786 %     };
1787 %     this.charges.set(id, updatedCharge);
1788 %     return updatedCharge;
1789 % }
1790 %
1791 % async deleteCharge(id: string): Promise<boolean> {
1792 %     return this.charges.delete(id);
1793 % }
1794 %
1795 % async getChargesTotal(reservationId: string): Promise<number> {
1796 %     const charges = await this.getCharges(reservationId);
1797 %     return charges.reduce((sum, c) => sum + parseFloat(c.amount), 0);
1798 % }
1799 %
1800 % // Payments
1801 % async getPayments(reservationId: string): Promise<Payment[]> {
1802 %     return Array.from(this.payments.values())
1803 %         .filter((p) => p.reservationId === reservationId)
1804 %         .sort((a, b) => new Date(b.date).getTime() - new Date(a.date).getTime());
1805 % }
1806 %
1807 % async createPayment(payment: InsertPayment): Promise<Payment> {
1808 %     const id = randomUUID();
1809 %     const newPayment: Payment = {
1810 %         id,
1811 %         reservationId: payment.reservationId,
1812 %         amount: payment.amount,
1813 %         method: payment.method as Payment["method"],
1814 %         date: payment.date,
1815 %         reference: payment.reference ?? null,
1816 %         receivedBy: payment.receivedBy ?? null,
1817 %         notes: payment.notes ?? null,
1818 %     };
1819 %     this.payments.set(id, newPayment);
1820 %     return newPayment;
1821 % }
1822 %
1823 % async updatePayment(id: string, payment: Partial<InsertPayment>): Promise<Payment | undefined> {
1824 %     const existing = this.payments.get(id);
1825 %     if (!existing) return undefined;
1826 %     const updatedPayment: Payment = {
1827 %         ...existing,
1828 %         ...payment,
1829 %         method: (payment.method ?? existing.method) as Payment["method"],
1830 %     };
1831 %     this.payments.set(id, updatedPayment);
1832 %     return updatedPayment;
1833 % }
1834 %
1835 % async deletePayment(id: string): Promise<boolean> {
1836 %     return this.payments.delete(id);
1837 % }
1838 %
1839 % async getPaymentsTotal(reservationId: string): Promise<number> {
1840 %     const payments = await this.getPayments(reservationId);
1841 %     return payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
1842 % }
1843 %
1844 % // Cancelled Reservation Logs
1845 % async getCancelledReservationLogs(): Promise<CancelledReservationLog[]> {
1846 %     return Array.from(this.cancelledReservationLogs.values()).sort((a, b) =>
1847 %         new Date(b.cancellationDate).getTime() - new Date(a.cancellationDate).getTime()
1848 %     );
1849 % }
1850 %
1851 % async createCancelledReservationLog(log: InsertCancelledReservationLog): Promise<CancelledReservationLog> {
1852 %     const id = randomUUID();
1853 %     const newLog: CancelledReservationLog = {
1854 %         id,
1855 %         reservationCode: log.reservationCode,
1856 %         guestName: log.guestName,
1857 %         roomNumber: log.roomNumber,

```

server/storage.ts (cont.)

```

1858 %         checkInDate: log.checkInDate,
1859 %         checkOutDate: log.checkOutDate,
1860 %         cancellationDate: log.cancellationDate,
1861 %         cancelledBy: log.cancelledBy ?? null,
1862 %         reason: log.reason ?? null,
1863 %     };
1864 %     this.cancelledReservationLogs.set(id, newLog);
1865 %     return newLog;
1866 % }
1867 %
1868 % // Overbooking check - returns true if there is a conflict
1869 % async checkOverbooking(roomId: string, checkInDate: string, checkOutDate: string, excludeReservationId?: string): P...
1870 %     const reservations = Array.from(this.reservations.values());
1871 %     const activeStatuses: ReservationStatus[] = ["tentative", "pending", "confirmed", "checked_in"];
1872 %
1873 %     for (const res of reservations) {
1874 %         if (res.roomId !== roomId) continue;
1875 %         if (excludeReservationId && res.id === excludeReservationId) continue;
1876 %         if (!activeStatuses.includes(res.status)) continue;
1877 %
1878 %         // Check for date overlap
1879 %         // Reservation A conflicts with B if A.checkIn < B.checkOut AND A.checkOut > B.checkIn
1880 %         if (checkInDate < res.checkOutDate && checkOutDate > res.checkInDate) {
1881 %             return true; // Conflict found
1882 %         }
1883 %     }
1884 %     return false; // No conflict
1885 % }
1886 %
1887 % // Dashboard Stats
1888 % async getDashboardStats() {
1889 %     const rooms = Array.from(this.rooms.values());
1890 %     const reservations = Array.from(this.reservations.values());
1891 %     const guests = Array.from(this.guests.values());
1892 %     const today = new Date().toISOString().split("T")[0];
1893 %
1894 %     const totalRooms = rooms.length;
1895 %     const availableRooms = rooms.filter((r) => r.status === "available").length;
1896 %     const occupiedRooms = rooms.filter((r) => r.status === "occupied").length;
1897 %     const dirtyRooms = rooms.filter((r) => r.status === "dirty").length;
1898 %     const cleaningRooms = rooms.filter((r) => r.status === "cleaning").length;
1899 %     const maintenanceRooms = rooms.filter((r) => r.status === "maintenance").length;
1900 %     const oosRooms = rooms.filter((r) => r.status === "oos").length;
1901 %
1902 %     const todayCheckIns = reservations.filter(
1903 %         (r) => r.checkInDate === today && (r.status === "confirmed" || r.status === "pending" || r.status === "tentative")
1904 %     ).length;
1905 %     const todayCheckOuts = reservations.filter(
1906 %         (r) => r.checkOutDate === today && r.status === "checked_in"
1907 %     ).length;
1908 %
1909 %     const occupancyRate = totalRooms > 0 ? Math.round((occupiedRooms / totalRooms) * 100) : 0;
1910 %     const totalGuests = guests.length;
1911 %     const pendingReservations = reservations.filter((r) => r.status === "pending" || r.status === "tentative").length;
1912 %
1913 %     return {
1914 %         totalRooms,
1915 %         availableRooms,
1916 %         occupiedRooms,
1917 %         dirtyRooms,
1918 %         cleaningRooms,
1919 %         maintenanceRooms,
1920 %         oosRooms,
1921 %         todayCheckIns,
1922 %         todayCheckOuts,
1923 %         occupancyRate,
1924 %         totalGuests,
1925 %         pendingReservations,
1926 %     };
1927 % }
1928 %
1929 % // Planning
1930 % async getPlanningData(startDate: string, endDate: string): Promise<PlanningData> {
1931 %     const rooms = await this.getRooms();
1932 %     const allReservations = Array.from(this.reservations.values());
1933 %     const allGroupLinks = Array.from(this.groupReservationLinks.values());
1934 %
1935 %     // Generate array of days between start and end
1936 %     const days: string[] = [];
1937 %     const start = new Date(startDate);
1938 %     const end = new Date(endDate);
1939 %     for (let d = new Date(start); d <= end; d.setDate(d.getDate() + 1)) {
1940 %         days.push(d.toISOString().split("T")[0]);
1941 %     }
1942 %
1943 %     // Build set of reservation IDs linked to groups
1944 %     const groupReservationIds = new Set(allGroupLinks.map(l => l.reservationId));
1945 %
1946 %     // Build occupancy map and cell reservations
1947 %     const occupancy: Record<string, PlanningCellStatus[]> = {};
1948 %     const cellReservations: Record<string, Record<string, string>> = {};
1949 %     const cellGroupBlocks: Record<string, Record<string, string>> = {};
1950 %     const reservationsMap: Record<string, { id: string; guestName: string; checkIn: string; checkOut: string; status:...

```

server/storage.ts (cont.)

```

1951 %     const groupBlocksMap: Record<string, { id: string; groupName: string; groupCode: string; checkIn: string; checkOu...
1952 %
1953 %     // Filter active reservations (not cancelled or checked_out)
1954 %     const activeReservations = allReservations.filter(r =>
1955 %         r.status !== "cancelled" && r.status !== "checked_out"
1956 %     );
1957 %
1958 %     // Build reservation info map
1959 %     for (const res of activeReservations) {
1960 %         const guest = this.guests.get(res.guestId);
1961 %         if (guest) {
1962 %             const isGroupReservation = groupReservationIds.has(res.id);
1963 %             let groupName: string | undefined;
1964 %
1965 %             if (isGroupReservation) {
1966 %                 const link = allGroupLinks.find(l => l.reservationId === res.id);
1967 %                 if (link) {
1968 %                     const group = this.groups.get(link.groupId);
1969 %                     if (group) {
1970 %                         groupName = group.name;
1971 %                     }
1972 %                 }
1973 %             }
1974 %
1975 %             reservationsMap[res.id] = {
1976 %                 id: res.id,
1977 %                 guestName: `${guest.firstName} ${guest.lastName}`,
1978 %                 checkIn: res.checkInDate,
1979 %                 checkOut: res.checkOutDate,
1980 %                 status: res.status as ReservationStatus,
1981 %                 source: res.source as ReservationSource,
1982 %                 isGroup: isGroupReservation,
1983 %                 groupName,
1984 %                 earlyCheckIn: res.earlyCheckIn ?? false,
1985 %                 earlyCheckInTime: res.earlyCheckInTime ?? null,
1986 %                 lateCheckOut: res.lateCheckOut ?? false,
1987 %                 lateCheckOutTime: res.lateCheckOutTime ?? null,
1988 %             };
1989 %         }
1990 %     }
1991 %
1992 %     // Calculate occupancy for each room
1993 %     for (const room of rooms) {
1994 %         occupancy[room.id] = [];
1995 %         cellReservations[room.id] = {};
1996 %         cellGroupBlocks[room.id] = {};
1997 %
1998 %         for (const day of days) {
1999 %             // Check room status first
2000 %             if (room.status === "maintenance") {
2001 %                 occupancy[room.id].push("maintenance");
2002 %                 continue;
2003 %             }
2004 %             if (room.status === "cleaning") {
2005 %                 occupancy[room.id].push("cleaning");
2006 %                 continue;
2007 %             }
2008 %
2009 %             // Find reservation for this room on this day
2010 %             const reservation = activeReservations.find(r => {
2011 %                 if (r.roomId !== room.id) return false;
2012 %                 const checkIn = r.checkInDate;
2013 %                 const checkOut = r.checkOutDate;
2014 %                 return day >= checkIn && day < checkOut;
2015 %             });
2016 %
2017 %             if (reservation) {
2018 %                 cellReservations[room.id][day] = reservation.id;
2019 %                 const isGroupRes = groupReservationIds.has(reservation.id);
2020 %
2021 %                 if (isGroupRes) {
2022 %                     occupancy[room.id].push("group_blocked");
2023 %                 } else if (reservation.status === "checked_in") {
2024 %                     if (day === reservation.checkOutDate) {
2025 %                         occupancy[room.id].push("checkout_today");
2026 %                     } else {
2027 %                         occupancy[room.id].push("checked_in");
2028 %                     }
2029 %                 } else {
2030 %                     occupancy[room.id].push("booked");
2031 %                 }
2032 %             } else {
2033 %                 occupancy[room.id].push("available");
2034 %             }
2035 %         }
2036 %     }
2037 %
2038 %     return {
2039 %         rooms,
2040 %         days,
2041 %         occupancy,
2042 %         groupBlocks: groupBlocksMap,
2043 %         cellGroupBlocks,

```

server/storage.ts (cont.)

```

2044 %         reservations: reservationsMap,
2045 %         cellReservations,
2046 %     };
2047 % }
2048 %
2049 % // OTA Channels
2050 % async getOTACHannels(): Promise<OTACHannelWithStats[]> {
2051 %     const channels = Array.from(this.otaChannels.values());
2052 %     return channels.map((channel) => {
2053 %         const logs = Array.from(this.otaReservationLogs.values()).filter(
2054 %             (log) => log.channelId === channel.id
2055 %         );
2056 %         const totalReservations = logs.length;
2057 %         const pendingSync = logs.filter((log) => log.status === "pending").length;
2058 %         const totalRevenue = logs.reduce((sum, log) => sum + parseFloat(log.totalAmount || "0"), 0);
2059 %         const totalCommission = logs.reduce((sum, log) => sum + parseFloat(log.commission || "0"), 0);
2060 %         return {
2061 %             ...channel,
2062 %             totalReservations,
2063 %             pendingSync,
2064 %             totalRevenue,
2065 %             totalCommission,
2066 %         };
2067 %     });
2068 % }
2069 %
2070 % async getOTACHannel(id: string): Promise<OTACHannel | undefined> {
2071 %     return this.otaChannels.get(id);
2072 % }
2073 %
2074 % async createOTACHannel(channel: InsertOTACHannel): Promise<OTACHannel> {
2075 %     const id = randomUUID();
2076 %     const newChannel: OTACHannel = {
2077 %         id,
2078 %         name: channel.name,
2079 %         channelType: channel.channelType as "booking" | "expedia" | "airbnb" | "despegar" | "hotelbeds" | "agoda" | "tr...",
2080 %         status: (channel.status || "inactive") as "active" | "inactive" | "pending" | "error",
2081 %         apiKey: channel.apiKey ?? null,
2082 %         apiSecret: channel.apiSecret ?? null,
2083 %         hotelCode: channel.hotelCode ?? null,
2084 %         commissionPercent: channel.commissionPercent ?? "15.00",
2085 %         syncEnabled: channel.syncEnabled ?? "false",
2086 %         lastSyncAt: channel.lastSyncAt ?? null,
2087 %         createdAt: channel.createdAt,
2088 %     };
2089 %     this.otaChannels.set(id, newChannel);
2090 %     return newChannel;
2091 % }
2092 %
2093 % async updateOTACHannel(id: string, channel: Partial<InsertOTACHannel>): Promise<OTACHannel | undefined> {
2094 %     const existing = this.otaChannels.get(id);
2095 %     if (!existing) return undefined;
2096 %     const updated: OTACHannel = { ...existing, ...channel } as OTACHannel;
2097 %     this.otaChannels.set(id, updated);
2098 %     return updated;
2099 % }
2100 %
2101 % async deleteOTACHannel(id: string): Promise<boolean> {
2102 %     return this.otaChannels.delete(id);
2103 % }
2104 %
2105 % // OTA Reservation Logs
2106 % async getOTAReservationLogs(channelId?: string): Promise<OTAReservationLogWithChannel[]> {
2107 %     let logs = Array.from(this.otaReservationLogs.values());
2108 %     if (channelId) {
2109 %         logs = logs.filter((log) => log.channelId === channelId);
2110 %     }
2111 %     return logs.map((log) => ({
2112 %         ...log,
2113 %         channel: this.otaChannels.get(log.channelId)!,
2114 %     })).sort((a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime());
2115 % }
2116 %
2117 % async getOTAReservationLog(id: string): Promise<OTAReservationLogWithChannel | undefined> {
2118 %     const log = this.otaReservationLogs.get(id);
2119 %     if (!log) return undefined;
2120 %     return {
2121 %         ...log,
2122 %         channel: this.otaChannels.get(log.channelId)!,
2123 %     };
2124 % }
2125 %
2126 % async createOTAReservationLog(log: InsertOTAReservationLog): Promise<OTAReservationLog> {
2127 %     const id = randomUUID();
2128 %     const newLog: OTAReservationLog = {
2129 %         id,
2130 %         channelId: log.channelId,
2131 %         externalReservationId: log.externalReservationId,
2132 %         internalReservationId: log.internalReservationId ?? null,
2133 %         guestName: log.guestName,
2134 %         checkInDate: log.checkInDate,
2135 %         checkOutDate: log.checkOutDate,
2136 %         roomTypeName: log.roomTypeName ?? null,

```


server/storage.ts (cont.)

```

2137 %         totalAmount: log.totalAmount ?? null,
2138 %         commission: log.commission ?? null,
2139 %         netAmount: log.netAmount ?? null,
2140 %         status: (log.status || "pending") as "pending" | "synced" | "failed" | "cancelled",
2141 %         rawData: log.rawData ?? null,
2142 %         syncedAt: log.syncedAt ?? null,
2143 %         createdAt: log.createdAt,
2144 %     };
2145 %     this.otaReservationLogs.set(id, newLog);
2146 %     return newLog;
2147 % }
2148 %
2149 % async updateOTAReservationLog(id: string, log: Partial<InsertOTAReservationLog>): Promise<OTAReservationLog | undef...
2150 %     const existing = this.otaReservationLogs.get(id);
2151 %     if (!existing) return undefined;
2152 %     const updated: OTAReservationLog = { ...existing, ...log } as OTAReservationLog;
2153 %     this.otaReservationLogs.set(id, updated);
2154 %     return updated;
2155 % }
2156 %
2157 % async syncOTAReservation(logId: string): Promise<Reservation | undefined> {
2158 %     const log = this.otaReservationLogs.get(logId);
2159 %     if (!log || log.status === "synced") return undefined;
2160 %
2161 %     const channel = this.otaChannels.get(log.channelId);
2162 %     if (!channel) return undefined;
2163 %
2164 %     // Parse guest name
2165 %     const nameParts = log.guestName.split(" ");
2166 %     const firstName = nameParts[0] || "OTA";
2167 %     const lastName = nameParts.slice(1).join(" ") || "Guest";
2168 %
2169 %     // Create or find guest
2170 %     let guest = Array.from(this.guests.values()).find(
2171 %         (g) => g.firstName === firstName && g.lastName === lastName
2172 %     );
2173 %     if (!guest) {
2174 %         guest = await this.createGuest({
2175 %             firstName,
2176 %             lastName,
2177 %             email: null,
2178 %             phone: null,
2179 %             documentType: null,
2180 %             documentNumber: null,
2181 %             nationality: null,
2182 %         });
2183 %     }
2184 %
2185 %     // Find room type by name or use first available
2186 %     let roomType = Array.from(this.roomTypes.values()).find(
2187 %         (rt) => rt.name === log.roomTypeName
2188 %     );
2189 %     if (!roomType) {
2190 %         roomType = Array.from(this.roomTypes.values())[0];
2191 %     }
2192 %
2193 %     // Find available room
2194 %     const availableRoom = Array.from(this.rooms.values()).find(
2195 %         (r) => r.roomTypeId === roomType!.id && r.status === "available"
2196 %     );
2197 %     if (!availableRoom) return undefined;
2198 %
2199 %     // Calculate nights
2200 %     const checkIn = new Date(log.checkInDate);
2201 %     const checkOut = new Date(log.checkOutDate);
2202 %     const nights = Math.ceil((checkOut.getTime() - checkIn.getTime()) / (1000 * 60 * 60 * 24));
2203 %
2204 %     // Create reservation
2205 %     const reservation = await this.createReservation({
2206 %         reservationCode: this.generateReservationCode(),
2207 %         guestId: guest.id,
2208 %         roomTypeId: roomType!.id,
2209 %         roomId: availableRoom.id,
2210 %         ratePlanId: null,
2211 %         checkInDate: log.checkInDate,
2212 %         checkOutDate: log.checkOutDate,
2213 %         nights,
2214 %         baseRatePerNight: log.netAmount ? (parseFloat(log.netAmount) / nights).toFixed(2) : null,
2215 %         discountType: "none",
2216 %         discountValue: "0",
2217 %         finalRatePerNight: log.netAmount ? (parseFloat(log.netAmount) / nights).toFixed(2) : null,
2218 %         totalRoomAmount: log.netAmount,
2219 %         status: "confirmed",
2220 %         source: channel.channelType as any,
2221 %         otaChannelId: channel.id,
2222 %         externalReservationId: log.externalReservationId,
2223 %         numberOfGuests: 1,
2224 %         notes: `Reserva importada de ${channel.name}`,
2225 %         createdAt: new Date().toISOString(),
2226 %         lastModifiedBy: null,
2227 %     });
2228 %
2229 %     // Update log

```

server/storage.ts (cont.)

```

2230 %     await this.updateOTAReservationLog(logId, {
2231 %         status: "synced",
2232 %         internalReservationId: reservation.id,
2233 %         syncedAt: new Date().toISOString(),
2234 %     });
2235 %
2236 %     return reservation;
2237 % }
2238 %
2239 % // Groups
2240 % generateGroupCode(): string {
2241 %     this.groupCounter++;
2242 %     return `GRP-${this.groupCounter.toString().padStart(4, "0")}`;
2243 % }
2244 %
2245 % private async buildGroupWithDetails(group: Group): Promise<GroupWithDetails> {
2246 %     const blocks = await this.getGroupBlocks(group.id);
2247 %     const links = await this.getGroupReservationLinks(group.id);
2248 %     const reservations: ReservationWithDetails[] = [];
2249 %
2250 %     for (const link of links) {
2251 %         const res = await this.getReservation(link.reservationId);
2252 %         if (res) reservations.push(res);
2253 %     }
2254 %
2255 %     const totalRooms = blocks.reduce((sum, b) => sum + b.quantity, 0);
2256 %     const assignedRooms = reservations.length;
2257 %
2258 %     return {
2259 %         ...group,
2260 %         blocks,
2261 %         reservations,
2262 %         totalRooms,
2263 %         assignedRooms,
2264 %     };
2265 % }
2266 %
2267 % async getGroups(): Promise<GroupWithDetails[]> {
2268 %     const groups = Array.from(this.groups.values());
2269 %     const result: GroupWithDetails[] = [];
2270 %     for (const group of groups) {
2271 %         result.push(await this.buildGroupWithDetails(group));
2272 %     }
2273 %     return result.sort((a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime());
2274 % }
2275 %
2276 % async getGroup(id: string): Promise<GroupWithDetails | undefined> {
2277 %     const group = this.groups.get(id);
2278 %     if (!group) return undefined;
2279 %     return this.buildGroupWithDetails(group);
2280 % }
2281 %
2282 % async createGroup(group: InsertGroup): Promise<Group> {
2283 %     const id = randomUUID();
2284 %     const newGroup: Group = {
2285 %         id,
2286 %         groupCode: group.groupCode,
2287 %         name: group.name,
2288 %         contactName: group.contactName ?? null,
2289 %         contactPhone: group.contactPhone ?? null,
2290 %         contactEmail: group.contactEmail ?? null,
2291 %         eventDate: group.eventDate ?? null,
2292 %         checkInDate: group.checkInDate,
2293 %         checkOutDate: group.checkOutDate,
2294 %         status: (group.status ?? "tentative") as Group["status"],
2295 %         releaseDate: group.releaseDate ?? null,
2296 %         notes: group.notes ?? null,
2297 %         createdAt: group.createdAt,
2298 %         createdBy: group.createdBy ?? null,
2299 %     };
2300 %     this.groups.set(id, newGroup);
2301 %     return newGroup;
2302 % }
2303 %
2304 % async updateGroup(id: string, group: Partial<InsertGroup>): Promise<Group | undefined> {
2305 %     const existing = this.groups.get(id);
2306 %     if (!existing) return undefined;
2307 %     const updated: Group = { ...existing, ...group } as Group;
2308 %     this.groups.set(id, updated);
2309 %     return updated;
2310 % }
2311 %
2312 % async deleteGroup(id: string): Promise<boolean> {
2313 %     // Delete all associated blocks and links
2314 %     Array.from(this.groupRoomBlocks.values())
2315 %         .filter(b => b.groupId === id)
2316 %         .forEach(b => this.groupRoomBlocks.delete(b.id));
2317 %     Array.from(this.groupReservationLinks.values())
2318 %         .filter(l => l.groupId === id)
2319 %         .forEach(l => this.groupReservationLinks.delete(l.id));
2320 %     return this.groups.delete(id);
2321 % }
2322 %

```

server/storage.ts (cont.)

```

2323 % // Group Room Blocks
2324 % async getGroupBlocks(groupId: string): Promise<GroupRoomBlockWithDetails[]> {
2325 %   const blocks = Array.from(this.groupRoomBlocks.values()).filter(b => b.groupId === groupId);
2326 %   return blocks.map(block => {
2327 %     const roomType = this.roomTypes.get(block.roomTypeId);
2328 %     const ratePlan = block.ratePlanId ? this.ratePlans.get(block.ratePlanId) : undefined;
2329 %     return {
2330 %       ...block,
2331 %       roomType: roomType!,
2332 %       ratePlan,
2333 %     };
2334 %   });
2335 % }
2336 %
2337 % async createGroupBlock(block: InsertGroupRoomBlock): Promise<GroupRoomBlock> {
2338 %   const id = randomUUID();
2339 %   const newBlock: GroupRoomBlock = {
2340 %     id,
2341 %     groupId: block.groupId,
2342 %     roomTypeId: block.roomTypeId,
2343 %     quantity: block.quantity,
2344 %     ratePlanId: block.ratePlanId ?? null,
2345 %     agreedRate: block.agreedRate ?? null,
2346 %     blockCheckInDate: block.blockCheckInDate ?? null,
2347 %     blockCheckOutDate: block.blockCheckOutDate ?? null,
2348 %   };
2349 %   this.groupRoomBlocks.set(id, newBlock);
2350 %   return newBlock;
2351 % }
2352 %
2353 % async updateGroupBlock(id: string, block: Partial<InsertGroupRoomBlock>): Promise<GroupRoomBlock | undefined> {
2354 %   const existing = this.groupRoomBlocks.get(id);
2355 %   if (!existing) return undefined;
2356 %   const updated: GroupRoomBlock = { ...existing, ...block } as GroupRoomBlock;
2357 %   this.groupRoomBlocks.set(id, updated);
2358 %   return updated;
2359 % }
2360 %
2361 % async deleteGroupBlock(id: string): Promise<boolean> {
2362 %   return this.groupRoomBlocks.delete(id);
2363 % }
2364 %
2365 % // Group Reservation Links
2366 % async getGroupReservationLinks(groupId: string): Promise<GroupReservationLink[]> {
2367 %   return Array.from(this.groupReservationLinks.values()).filter(l => l.groupId === groupId);
2368 % }
2369 %
2370 % async createGroupReservationLink(link: InsertGroupReservationLink): Promise<GroupReservationLink> {
2371 %   const id = randomUUID();
2372 %   const newLink: GroupReservationLink = {
2373 %     id,
2374 %     groupId: link.groupId,
2375 %     reservationId: link.reservationId,
2376 %   };
2377 %   this.groupReservationLinks.set(id, newLink);
2378 %   return newLink;
2379 % }
2380 %
2381 % async assignRoomToGroup(
2382 %   groupId: string,
2383 %   roomId: string,
2384 %   guestFirstName: string,
2385 %   guestLastName: string,
2386 %   options?: {
2387 %     checkInDate?: string;
2388 %     checkOutDate?: string;
2389 %     agreedRate?: string;
2390 %     ratePlanId?: string | null;
2391 %   }
2392 % ): Promise<Reservation | undefined> {
2393 %   const group = this.groups.get(groupId);
2394 %   if (!group) return undefined;
2395 %
2396 %   const room = this.rooms.get(roomId);
2397 %   if (!room) return undefined;
2398 %
2399 %   // Create guest for this room assignment
2400 %   const guest = await this.createGuest({
2401 %     firstName: guestFirstName,
2402 %     lastName: guestLastName,
2403 %     email: null,
2404 %     phone: null,
2405 %     documentType: null,
2406 %     documentNumber: null,
2407 %     nationality: null,
2408 %   });
2409 %
2410 %   // Find matching block for defaults
2411 %   const blocks = await this.getGroupBlocks(groupId);
2412 %   const matchingBlock = blocks.find(b => b.roomTypeId === room.roomTypeId);
2413 %
2414 %   // Use provided values or fall back to block values or group values
2415 %   const checkInDate = options?.checkInDate || matchingBlock?.blockCheckInDate || group.checkInDate;

```

server/storage.ts (cont.)

```

2416 %     const checkOutDate = options?.checkOutDate || matchingBlock?.blockCheckOutDate || group.checkOutDate;
2417 %     const agreedRate = options?.agreedRate || matchingBlock?.agreedRate || "0";
2418 %     const ratePlanId = options?.ratePlanId !== undefined ? options.ratePlanId : (matchingBlock?.ratePlanId || null);
2419 %
2420 %     // Calculate nights
2421 %     const checkIn = new Date(checkInDate);
2422 %     const checkOut = new Date(checkOutDate);
2423 %     const nights = Math.ceil((checkOut.getTime() - checkIn.getTime()) / (1000 * 60 * 60 * 24));
2424 %
2425 %     // Create reservation
2426 %     const reservation = await this.createReservation({
2427 %         reservationCode: `G${group.groupCode}-${room.roomNumber}`,
2428 %         guestId: guest.id,
2429 %         roomTypeId: room.roomTypeId,
2430 %         roomId: room.id,
2431 %         ratePlanId,
2432 %         checkInDate,
2433 %         checkOutDate,
2434 %         nights,
2435 %         baseRatePerNight: agreedRate,
2436 %         discountType: "none",
2437 %         discountValue: "0",
2438 %         finalRatePerNight: agreedRate,
2439 %         totalRoomAmount: (parseFloat(agreedRate) * nights).toFixed(2),
2440 %         status: "confirmed",
2441 %         source: "empresa",
2442 %         otaChannelId: null,
2443 %         externalReservationId: null,
2444 %         numberOfGuests: 1,
2445 %         notes: `Grupo: ${group.name}`,
2446 %         createdAt: new Date().toISOString(),
2447 %         lastModifiedBy: null,
2448 %     });
2449 %
2450 %     // Create link
2451 %     await this.createGroupReservationLink({
2452 %         groupId,
2453 %         reservationId: reservation.id,
2454 %     });
2455 %
2456 %     return reservation;
2457 % }
2458 %
2459 % // Guest Reviews
2460 % private enrichReview(review: GuestReview): GuestReviewWithDetails {
2461 %     const guest = this.guests.get(review.guestId);
2462 %     const room = review.roomId ? this.rooms.get(review.roomId) : undefined;
2463 %     const reservation = review.reservationId ? this.reservations.get(review.reservationId) : undefined;
2464 %     return {
2465 %         ...review,
2466 %         guest: guest!,
2467 %         room,
2468 %         reservation,
2469 %     };
2470 % }
2471 %
2472 % async getGuestReviews(): Promise<GuestReviewWithDetails[]> {
2473 %     return Array.from(this.guestReviews.values())
2474 %         .map(r => this.enrichReview(r))
2475 %         .sort((a, b) => new Date(b.reviewDate).getTime() - new Date(a.reviewDate).getTime());
2476 % }
2477 %
2478 % async getGuestReview(id: string): Promise<GuestReviewWithDetails | undefined> {
2479 %     const review = this.guestReviews.get(id);
2480 %     return review ? this.enrichReview(review) : undefined;
2481 % }
2482 %
2483 % async getGuestReviewsByGuest(guestId: string): Promise<GuestReviewWithDetails[]> {
2484 %     return Array.from(this.guestReviews.values())
2485 %         .filter(r => r.guestId === guestId)
2486 %         .map(r => this.enrichReview(r))
2487 %         .sort((a, b) => new Date(b.reviewDate).getTime() - new Date(a.reviewDate).getTime());
2488 % }
2489 %
2490 % async createGuestReview(review: InsertGuestReview): Promise<GuestReview> {
2491 %     const id = randomUUID();
2492 %     const newReview: GuestReview = {
2493 %         id,
2494 %         reservationId: review.reservationId || null,
2495 %         guestId: review.guestId,
2496 %         roomId: review.roomId || null,
2497 %         reviewDate: review.reviewDate,
2498 %         source: review.source || "direct",
2499 %         rating: review.rating,
2500 %         title: review.title || null,
2501 %         content: review.content,
2502 %         sentiment: (review.sentiment as SentimentType) || null,
2503 %         sentimentScore: review.sentimentScore || null,
2504 %         categories: review.categories || null,
2505 %         categoryScores: review.categoryScores || null,
2506 %         keyPhrases: review.keyPhrases || null,
2507 %         improvementSuggestions: review.improvementSuggestions || null,
2508 %         analyzedAt: review.analyzedAt || null,

```

server/storage.ts (cont.)

```

2509 %         isPublished: review.isPublished || "false",
2510 %         staffResponse: review.staffResponse || null,
2511 %         respondedAt: review.respondedAt || null,
2512 %         respondedBy: review.respondedBy || null,
2513 %     };
2514 %     this.guestReviews.set(id, newReview);
2515 %     return newReview;
2516 % }
2517 %
2518 % async updateGuestReview(id: string, review: Partial<InsertGuestReview>): Promise<GuestReview | undefined> {
2519 %     const existing = this.guestReviews.get(id);
2520 %     if (!existing) return undefined;
2521 %     const updated: GuestReview = {
2522 %         ...existing,
2523 %         ...review,
2524 %         sentiment: review.sentiment !== undefined ? (review.sentiment as SentimentType) : existing.sentiment,
2525 %     };
2526 %     this.guestReviews.set(id, updated);
2527 %     return updated;
2528 % }
2529 %
2530 % async deleteGuestReview(id: string): Promise<boolean> {
2531 %     return this.guestReviews.delete(id);
2532 % }
2533 %
2534 % async getReviewAnalyticsSummary(): Promise<{
2535 %     totalReviews: number;
2536 %     averageRating: number;
2537 %     sentimentBreakdown: { positive: number; neutral: number; negative: number };
2538 %     topCategories: { category: string; count: number; avgSentiment: number }[];
2539 %     recentTrend: { date: string; avgRating: number; count: number }[];
2540 %     improvementAreas: string[];
2541 % }> {
2542 %     const reviews = Array.from(this.guestReviews.values());
2543 %     const totalReviews = reviews.length;
2544 %     const averageRating = totalReviews > 0
2545 %         ? reviews.reduce((sum, r) => sum + r.rating, 0) / totalReviews
2546 %         : 0;
2547 %
2548 %     const sentimentBreakdown = {
2549 %         positive: reviews.filter(r => r.sentiment === "positive").length,
2550 %         neutral: reviews.filter(r => r.sentiment === "neutral").length,
2551 %         negative: reviews.filter(r => r.sentiment === "negative").length,
2552 %     };
2553 %
2554 %     // Count categories
2555 %     const categoryMap = new Map<string, { count: number; totalScore: number }>();
2556 %     reviews.forEach(r => {
2557 %         if (r.categories) {
2558 %             r.categories.forEach(cat => {
2559 %                 const existing = categoryMap.get(cat) || { count: 0, totalScore: 0 };
2560 %                 const score = r.sentimentScore ? parseFloat(r.sentimentScore) : 0.5;
2561 %                 categoryMap.set(cat, { count: existing.count + 1, totalScore: existing.totalScore + score });
2562 %             });
2563 %         }
2564 %     });
2565 %
2566 %     const topCategories = Array.from(categoryMap.entries())
2567 %         .map(([category, data]) => ({
2568 %             category,
2569 %             count: data.count,
2570 %             avgSentiment: data.totalScore / data.count,
2571 %         }))
2572 %         .sort((a, b) => b.count - a.count)
2573 %         .slice(0, 5);
2574 %
2575 %     // Recent trend (last 7 days)
2576 %     const dateMap = new Map<string, { total: number; count: number }>();
2577 %     reviews.forEach(r => {
2578 %         const date = r.reviewDate.split("T")[0];
2579 %         const existing = dateMap.get(date) || { total: 0, count: 0 };
2580 %         dateMap.set(date, { total: existing.total + r.rating, count: existing.count + 1 });
2581 %     });
2582 %
2583 %     const recentTrend = Array.from(dateMap.entries())
2584 %         .map(([date, data]) => ({
2585 %             date,
2586 %             avgRating: data.total / data.count,
2587 %             count: data.count,
2588 %         }))
2589 %         .sort((a, b) => a.date.localeCompare(b.date))
2590 %         .slice(-7);
2591 %
2592 %     // Collect improvement suggestions from negative reviews
2593 %     const improvementAreas: string[] = [];
2594 %     reviews
2595 %         .filter(r => r.sentiment === "negative" && r.improvementSuggestions)
2596 %         .forEach(r => {
2597 %             if (r.improvementSuggestions) {
2598 %                 improvementAreas.push(...r.improvementSuggestions);
2599 %             }
2600 %         });
2601 % }

```

server/storage.ts (cont.)

```
2602 %     return {
2603 %         totalReviews,
2604 %         averageRating: Math.round(averageRating * 10) / 10,
2605 %         sentimentBreakdown,
2606 %         topCategories,
2607 %         recentTrend,
2608 %         improvementAreas: Array.from(new Set(improvementAreas)).slice(0, 10),
2609 %     };
2610 % }
2611 %
2612 % // Housekeeping Tasks
2613 % async getHousekeepingTasks(date?: string): Promise<HousekeepingTaskWithRoom[]> {
2614 %     const tasks = Array.from(this.housekeepingTasks.values());
2615 %     const filteredTasks = date
2616 %         ? tasks.filter(t => t.scheduledDate === date)
2617 %         : tasks;
2618 %
2619 %     return filteredTasks.map(task => {
2620 %         const room = this.rooms.get(task.roomId);
2621 %         const roomType = room ? this.roomTypes.get(room.roomTypeId) : undefined;
2622 %         return {
2623 %             ...task,
2624 %             room: { ...room!, roomType },
2625 %         };
2626 %     }).sort((a, b) => {
2627 %         const priorityOrder = { urgent: 0, high: 1, normal: 2, low: 3 };
2628 %         return priorityOrder[a.priority as keyof typeof priorityOrder] - priorityOrder[b.priority as keyof typeof prior...
2629 %     });
2630 % }
2631 %
2632 % async getHousekeepingTask(id: string): Promise<HousekeepingTaskWithRoom | undefined> {
2633 %     const task = this.housekeepingTasks.get(id);
2634 %     if (!task) return undefined;
2635 %
2636 %     const room = this.rooms.get(task.roomId);
2637 %     const roomType = room ? this.roomTypes.get(room.roomTypeId) : undefined;
2638 %     return {
2639 %         ...task,
2640 %         room: { ...room!, roomType },
2641 %     };
2642 % }
2643 %
2644 % async getHousekeepingTasksByRoom(roomId: string): Promise<HousekeepingTask[]> {
2645 %     return Array.from(this.housekeepingTasks.values())
2646 %         .filter(t => t.roomId === roomId);
2647 % }
2648 %
2649 % async createHousekeepingTask(task: InsertHousekeepingTask): Promise<HousekeepingTask> {
2650 %     const id = randomUUID();
2651 %     const newTask: HousekeepingTask = {
2652 %         id,
2653 %         roomId: task.roomId,
2654 %         taskType: (task.taskType || "checkout_clean") as "checkout_clean" | "stayover_clean" | "deep_clean" | "inspecti...
2655 %         status: (task.status || "pending") as "pending" | "in_progress" | "completed" | "inspected",
2656 %         priority: (task.priority || "normal") as "low" | "normal" | "high" | "urgent",
2657 %         assignedTo: task.assignedTo ?? null,
2658 %         notes: task.notes ?? null,
2659 %         scheduledDate: task.scheduledDate,
2660 %         startedAt: task.startedAt ?? null,
2661 %         completedAt: task.completedAt ?? null,
2662 %         inspectedBy: task.inspectedBy ?? null,
2663 %         inspectedAt: task.inspectedAt ?? null,
2664 %         createdAt: task.createdAt,
2665 %     };
2666 %     this.housekeepingTasks.set(id, newTask);
2667 %     return newTask;
2668 % }
2669 %
2670 % async updateHousekeepingTask(id: string, task: Partial<InsertHousekeepingTask>): Promise<HousekeepingTask | undefin...
2671 %     const existing = this.housekeepingTasks.get(id);
2672 %     if (!existing) return undefined;
2673 %
2674 %     const updated: HousekeepingTask = {
2675 %         ...existing,
2676 %         ...task,
2677 %         status: (task.status ?? existing.status) as "pending" | "in_progress" | "completed" | "inspected",
2678 %         taskType: (task.taskType ?? existing.taskType) as "checkout_clean" | "stayover_clean" | "deep_clean" | "inspect...
2679 %         priority: (task.priority ?? existing.priority) as "low" | "normal" | "high" | "urgent",
2680 %     };
2681 %     this.housekeepingTasks.set(id, updated);
2682 %     return updated;
2683 % }
2684 %
2685 % async deleteHousekeepingTask(id: string): Promise<boolean> {
2686 %     return this.housekeepingTasks.delete(id);
2687 % }
2688 %
2689 % async createCheckoutCleaningTask(roomId: string): Promise<HousekeepingTask> {
2690 %     const today = new Date().toISOString().split("T")[0];
2691 %     return this.createHousekeepingTask({
2692 %         roomId,
2693 %         taskType: "checkout_clean",
2694 %         status: "pending",
```

server/storage.ts (cont.)

```
2695 %         priority: "high",
2696 %         scheduledDate: today,
2697 %         createdAt: new Date().toISOString(),
2698 %     });
2699 % }
2700 %
2701 % // ===== RESTAURANT =====
2702 % async getRestaurantAreas(): Promise<RestaurantArea[]> {
2703 %     return Array.from(this.restaurantAreas.values()).filter(a => a.isActive === "true");
2704 % }
2705 %
2706 % async getRestaurantArea(id: string): Promise<RestaurantArea | undefined> {
2707 %     return this.restaurantAreas.get(id);
2708 % }
2709 %
2710 % async createRestaurantArea(area: InsertRestaurantArea): Promise<RestaurantArea> {
2711 %     const id = randomUUID();
2712 %     const newArea: RestaurantArea = {
2713 %         id,
2714 %         name: area.name,
2715 %         areaType: (area.areaType ?? "indoor") as "indoor" | "outdoor" | "terrace" | "bar" | "private",
2716 %         capacity: area.capacity ?? 20,
2717 %         isActive: area.isActive ?? "true",
2718 %         notes: area.notes ?? null,
2719 %     };
2720 %     this.restaurantAreas.set(id, newArea);
2721 %     return newArea;
2722 % }
2723 %
2724 % async updateRestaurantArea(id: string, area: Partial<InsertRestaurantArea>): Promise<RestaurantArea | undefined> {
2725 %     const existing = this.restaurantAreas.get(id);
2726 %     if (!existing) return undefined;
2727 %     const updated: RestaurantArea = {
2728 %         ...existing,
2729 %         ...area,
2730 %         areaType: (area.areaType ?? existing.areaType) as "indoor" | "outdoor" | "terrace" | "bar" | "private",
2731 %     };
2732 %     this.restaurantAreas.set(id, updated);
2733 %     return updated;
2734 % }
2735 %
2736 % async deleteRestaurantArea(id: string): Promise<boolean> {
2737 %     return this.restaurantAreas.delete(id);
2738 % }
2739 %
2740 % async getRestaurantTables(): Promise<RestaurantTableWithArea[]> {
2741 %     return Array.from(this.restaurantTables.values())
2742 %         .filter(t => t.isActive === "true")
2743 %         .map(table => ({
2744 %             ...table,
2745 %             area: this.restaurantAreas.get(table.areaId)!,
2746 %         }));
2747 % }
2748 %
2749 % async getRestaurantTable(id: string): Promise<RestaurantTableWithArea | undefined> {
2750 %     const table = this.restaurantTables.get(id);
2751 %     if (!table) return undefined;
2752 %     return { ...table, area: this.restaurantAreas.get(table.areaId)! };
2753 % }
2754 %
2755 % async getTablesByArea(areaId: string): Promise<RestaurantTable[]> {
2756 %     return Array.from(this.restaurantTables.values()).filter(t => t.areaId === areaId);
2757 % }
2758 %
2759 % async createRestaurantTable(table: InsertRestaurantTable): Promise<RestaurantTable> {
2760 %     const id = randomUUID();
2761 %     const newTable: RestaurantTable = {
2762 %         id,
2763 %         tableNumber: table.tableNumber,
2764 %         areaId: table.areaId,
2765 %         capacity: table.capacity ?? 4,
2766 %         shape: (table.shape ?? "square") as "square" | "round" | "rectangular",
2767 %         status: (table.status ?? "available") as "available" | "occupied" | "reserved" | "cleaning" | "blocked",
2768 %         positionX: table.positionX ?? 0,
2769 %         positionY: table.positionY ?? 0,
2770 %         isActive: table.isActive ?? "true",
2771 %     };
2772 %     this.restaurantTables.set(id, newTable);
2773 %     return newTable;
2774 % }
2775 %
2776 % async updateRestaurantTable(id: string, table: Partial<InsertRestaurantTable>): Promise<RestaurantTable | undefined> {
2777 %     const existing = this.restaurantTables.get(id);
2778 %     if (!existing) return undefined;
2779 %     const updated: RestaurantTable = {
2780 %         ...existing,
2781 %         ...table,
2782 %         status: (table.status ?? existing.status) as "available" | "occupied" | "reserved" | "cleaning" | "blocked",
2783 %         shape: (table.shape ?? existing.shape) as "square" | "round" | "rectangular",
2784 %     };
2785 %     this.restaurantTables.set(id, updated);
2786 %     return updated;
2787 % }
```

server/storage.ts (cont.)

```

2788 %
2789 %   async deleteRestaurantTable(id: string): Promise<boolean> {
2790 %       return this.restaurantTables.delete(id);
2791 %   }
2792 %
2793 %   async getMenuCategories(): Promise<MenuCategory[]> {
2794 %       return Array.from(this.menuCategories.values())
2795 %           .filter(c => c.isActive === "true")
2796 %           .sort((a, b) => (a.displayOrder ?? 0) - (b.displayOrder ?? 0));
2797 %   }
2798 %
2799 %   async getMenuCategory(id: string): Promise<MenuCategory | undefined> {
2800 %       return this.menuCategories.get(id);
2801 %   }
2802 %
2803 %   async createMenuCategory(category: InsertMenuCategory): Promise<MenuCategory> {
2804 %       const id = randomUUID();
2805 %       const newCategory: MenuCategory = {
2806 %           id,
2807 %           name: category.name,
2808 %           description: category.description ?? null,
2809 %           displayOrder: category.displayOrder ?? 0,
2810 %           isActive: category.isActive ?? "true",
2811 %       };
2812 %       this.menuCategories.set(id, newCategory);
2813 %       return newCategory;
2814 %   }
2815 %
2816 %   async updateMenuCategory(id: string, category: Partial<InsertMenuCategory>): Promise<MenuCategory | undefined> {
2817 %       const existing = this.menuCategories.get(id);
2818 %       if (!existing) return undefined;
2819 %       const updated = { ...existing, ...category };
2820 %       this.menuCategories.set(id, updated);
2821 %       return updated;
2822 %   }
2823 %
2824 %   async deleteMenuCategory(id: string): Promise<boolean> {
2825 %       return this.menuCategories.delete(id);
2826 %   }
2827 %
2828 %   async getMenuItems(): Promise<MenuItemWithCategory[]> {
2829 %       return Array.from(this.menuItems.values())
2830 %           .filter(i => i.isActive === "true")
2831 %           .map(item => ({
2832 %               ...item,
2833 %               category: this.menuCategories.get(item.categoryId)!,
2834 %           }))
2835 %           .sort((a, b) => (a.displayOrder ?? 0) - (b.displayOrder ?? 0));
2836 %   }
2837 %
2838 %   async getItem(id: string): Promise<MenuItemWithCategory | undefined> {
2839 %       const item = this.menuItems.get(id);
2840 %       if (!item) return undefined;
2841 %       return { ...item, category: this.menuCategories.get(item.categoryId)! };
2842 %   }
2843 %
2844 %   async getMenuItemsByCategory(categoryId: string): Promise<MenuItem[]> {
2845 %       return Array.from(this.menuItems.values()).filter(i => i.categoryId === categoryId);
2846 %   }
2847 %
2848 %   async createMenuItem(item: InsertMenuItem): Promise<MenuItem> {
2849 %       const id = randomUUID();
2850 %       const newItem: MenuItem = {
2851 %           id,
2852 %           categoryId: item.categoryId,
2853 %           name: item.name,
2854 %           description: item.description ?? null,
2855 %           price: item.price,
2856 %           preparationTime: item.preparationTime ?? null,
2857 %           isAvailable: item.isAvailable ?? "true",
2858 %           isActive: item.isActive ?? "true",
2859 %           allergens: item.allergens ?? null,
2860 %           displayOrder: item.displayOrder ?? 0,
2861 %       };
2862 %       this.menuItems.set(id, newItem);
2863 %       return newItem;
2864 %   }
2865 %
2866 %   async updateMenuItem(id: string, item: Partial<InsertMenuItem>): Promise<MenuItem | undefined> {
2867 %       const existing = this.menuItems.get(id);
2868 %       if (!existing) return undefined;
2869 %       const updated = { ...existing, ...item };
2870 %       this.menuItems.set(id, updated);
2871 %       return updated;
2872 %   }
2873 %
2874 %   async deleteMenuItem(id: string): Promise<boolean> {
2875 %       return this.menuItems.delete(id);
2876 %   }
2877 %
2878 %   async getRestaurantOrders(status?: OrderStatus): Promise<RestaurantOrderWithDetails[]> {
2879 %       let orders = Array.from(this.restaurantOrders.values());
2880 %       if (status) {

```


server/storage.ts (cont.)

```

2881 %         orders = orders.filter(o => o.status === status);
2882 %     }
2883 %     return orders.map(order => {
2884 %         const table = order.tableId ? this.restaurantTables.get(order.tableId) : undefined;
2885 %         const tableArea = table ? this.restaurantAreas.get(table.areaId) : undefined;
2886 %         const area = order.areaId ? this.restaurantAreas.get(order.areaId) : tableArea;
2887 %         const guest = order.guestId ? this.guests.get(order.guestId) : undefined;
2888 %         const items = Array.from(this.orderItems.values())
2889 %             .filter(i => i.orderId === order.id)
2890 %             .map(item => ({
2891 %                 ...item,
2892 %                 menuItem: this.menuItems.get(item.menuItemId)!,
2893 %             }));
2894 %         return {
2895 %             ...order,
2896 %             table: table ? { ...table, area: tableArea! } : undefined,
2897 %             area,
2898 %             guest,
2899 %             items,
2900 %         };
2901 %     }).sort((a, b) => b.openedAt.localeCompare(a.openedAt));
2902 % }
2903 %
2904 % async getRestaurantOrder(id: string): Promise<RestaurantOrderWithDetails | undefined> {
2905 %     const order = this.restaurantOrders.get(id);
2906 %     if (!order) return undefined;
2907 %     const table = order.tableId ? this.restaurantTables.get(order.tableId) : undefined;
2908 %     const tableArea = table ? this.restaurantAreas.get(table.areaId) : undefined;
2909 %     const area = order.areaId ? this.restaurantAreas.get(order.areaId) : tableArea;
2910 %     const guest = order.guestId ? this.guests.get(order.guestId) : undefined;
2911 %     const items = Array.from(this.orderItems.values())
2912 %         .filter(i => i.orderId === order.id)
2913 %         .map(item => ({
2914 %             ...item,
2915 %             menuItem: this.menuItems.get(item.menuItemId)!,
2916 %         }));
2917 %     return {
2918 %         ...order,
2919 %         table: table ? { ...table, area: tableArea! } : undefined,
2920 %         area,
2921 %         guest,
2922 %         items,
2923 %     };
2924 % }
2925 %
2926 % async getOrdersByTable(tableId: string): Promise<RestaurantOrder[]> {
2927 %     return Array.from(this.restaurantOrders.values()).filter(o => o.tableId === tableId);
2928 % }
2929 %
2930 % async createRestaurantOrder(order: InsertRestaurantOrder): Promise<RestaurantOrder> {
2931 %     const id = randomUUID();
2932 %     const newOrder: RestaurantOrder = {
2933 %         id,
2934 %         orderNumber: order.orderNumber,
2935 %         tableId: order.tableId ?? null,
2936 %         areaId: order.areaId ?? null,
2937 %         reservationId: order.reservationId ?? null,
2938 %         guestId: order.guestId ?? null,
2939 %         orderType: (order.orderType ?? "dine_in") as "dine_in" | "room_service" | "takeaway",
2940 %         status: (order.status ?? "open") as "open" | "in_progress" | "served" | "closed" | "cancelled",
2941 %         covers: order.covers ?? 1,
2942 %         waiterName: order.waiterName ?? null,
2943 %         orderLabel: order.orderLabel ?? null,
2944 %         activeCourse: order.activeCourse ?? 1,
2945 %         subtotal: order.subtotal ?? "0",
2946 %         tax: order.tax ?? "0",
2947 %         total: order.total ?? "0",
2948 %         notes: order.notes ?? null,
2949 %         openedAt: order.openedAt,
2950 %         closedAt: order.closedAt ?? null,
2951 %         chargedToRoom: order.chargedToRoom ?? "false",
2952 %         roomNumber: order.roomNumber ?? null,
2953 %         receiptType: order.receiptType ?? null,
2954 %         paymentMethod: order.paymentMethod ?? null,
2955 %     };
2956 %     this.restaurantOrders.set(id, newOrder);
2957 %     return newOrder;
2958 % }
2959 %
2960 % async updateRestaurantOrder(id: string, order: Partial<InsertRestaurantOrder>): Promise<RestaurantOrder | undefined> {
2961 %     const existing = this.restaurantOrders.get(id);
2962 %     if (!existing) return undefined;
2963 %     const updated: RestaurantOrder = {
2964 %         ...existing,
2965 %         ...order,
2966 %         status: (order.status ?? existing.status) as "open" | "in_progress" | "served" | "closed" | "cancelled",
2967 %         orderType: (order.orderType ?? existing.orderType) as "dine_in" | "room_service" | "takeaway",
2968 %     };
2969 %     this.restaurantOrders.set(id, updated);
2970 %     return updated;
2971 % }
2972 %
2973 % async deleteRestaurantOrder(id: string): Promise<boolean> {

```

server/storage.ts (cont.)

```
2974 %     return this.restaurantOrders.delete(id);
2975 %   }
2976 %
2977 %   generateOrderNumber(): string {
2978 %     this.orderCounter++;
2979 %     const today = new Date();
2980 %     const dateStr = `${today.getFullYear()}${String(today.getMonth() + 1).padStart(2, "0")}${String(today.getDate())...
2981 %     return `ORD-${dateStr}-${this.orderCounter}`;
2982 %   }
2983 %
2984 %   async getOrderItems(orderId: string): Promise<OrderItem[]> {
2985 %     return Array.from(this.orderItems.values()).filter(i => i.orderId === orderId);
2986 %   }
2987 %
2988 %   async createOrderItem(item: InsertOrderItem): Promise<OrderItem> {
2989 %     const id = randomUUID();
2990 %     const newItem: OrderItem = {
2991 %       id,
2992 %       orderId: item.orderId,
2993 %       menuItemId: item.menuItemId,
2994 %       quantity: item.quantity ?? 1,
2995 %       unitPrice: item.unitPrice,
2996 %       subtotal: item.subtotal,
2997 %       status: (item.status ?? "pending") as "pending" | "preparing" | "ready" | "served" | "cancelled" | "waiting_cou...
2998 %       course: item.course ?? 1,
2999 %       notes: item.notes ?? null,
3000 %       sentAt: item.sentAt ?? null,
3001 %     };
3002 %     this.orderItems.set(id, newItem);
3003 %     return newItem;
3004 %   }
3005 %
3006 %   async updateOrderItem(id: string, item: Partial<InsertOrderItem>): Promise<OrderItem | undefined> {
3007 %     const existing = this.orderItems.get(id);
3008 %     if (!existing) return undefined;
3009 %     const updated: OrderItem = {
3010 %       ...existing,
3011 %       ...item,
3012 %       status: (item.status ?? existing.status) as "pending" | "preparing" | "ready" | "served" | "cancelled" | "waiti...
3013 %     };
3014 %     this.orderItems.set(id, updated);
3015 %     return updated;
3016 %   }
3017 %
3018 %   async deleteOrderItem(id: string): Promise<boolean> {
3019 %     return this.orderItems.delete(id);
3020 %   }
3021 %
3022 %   // Table Reservations
3023 %   async getTableReservations(): Promise<TableReservationWithTable[]> {
3024 %     const reservations = Array.from(this.tableReservations.values());
3025 %     return reservations.map(r => ({
3026 %       ...r,
3027 %       table: this.restaurantTables.get(r.tableId)!,
3028 %     })).filter(r => r.table);
3029 %   }
3030 %
3031 %   async getTableReservation(id: string): Promise<TableReservationWithTable | undefined> {
3032 %     const reservation = this.tableReservations.get(id);
3033 %     if (!reservation) return undefined;
3034 %     const table = this.restaurantTables.get(reservation.tableId);
3035 %     if (!table) return undefined;
3036 %     return { ...reservation, table };
3037 %   }
3038 %
3039 %   async getTableReservationsByDate(date: string): Promise<TableReservationWithTable[]> {
3040 %     const reservations = Array.from(this.tableReservations.values())
3041 %       .filter(r => r.reservationDate === date && r.status !== "cancelled");
3042 %     return reservations.map(r => ({
3043 %       ...r,
3044 %       table: this.restaurantTables.get(r.tableId)!,
3045 %     })).filter(r => r.table);
3046 %   }
3047 %
3048 %   async getTableReservationsByTable(tableId: string): Promise<TableReservation[]> {
3049 %     return Array.from(this.tableReservations.values())
3050 %       .filter(r => r.tableId === tableId);
3051 %   }
3052 %
3053 %   async createTableReservation(reservation: InsertTableReservation): Promise<TableReservation> {
3054 %     const id = randomUUID();
3055 %     const newReservation: TableReservation = {
3056 %       id,
3057 %       tableId: reservation.tableId,
3058 %       guestName: reservation.guestName,
3059 %       guestPhone: reservation.guestPhone ?? null,
3060 %       guestEmail: reservation.guestEmail ?? null,
3061 %       partySize: reservation.partySize ?? 2,
3062 %       reservationDate: reservation.reservationDate,
3063 %       reservationTime: reservation.reservationTime,
3064 %       status: (reservation.status ?? "pending") as "pending" | "confirmed" | "seated" | "completed" | "cancelled" | "...
3065 %       notes: reservation.notes ?? null,
3066 %       createdAt: reservation.createdAt,
```

server/storage.ts (cont.)

```

3067 %     };
3068 %     this.tableReservations.set(id, newReservation);
3069 %     return newReservation;
3070 % }
3071 %
3072 % async updateTableReservation(id: string, reservation: Partial<InsertTableReservation>): Promise<TableReservation | ...
3073 %     const existing = this.tableReservations.get(id);
3074 %     if (!existing) return undefined;
3075 %     const updated = { ...existing, ...reservation } as TableReservation;
3076 %     this.tableReservations.set(id, updated);
3077 %     return updated;
3078 % }
3079 %
3080 % async deleteTableReservation(id: string): Promise<boolean> {
3081 %     return this.tableReservations.delete(id);
3082 % }
3083 %
3084 % // Restaurant Time Slots
3085 % async getRestaurantTimeSlots(): Promise<RestaurantTimeSlot[]> {
3086 %     return Array.from(this.restaurantTimeSlots.values())
3087 %         .filter(s => s.isActive === "true")
3088 %         .sort((a, b) => (a.displayOrder ?? 0) - (b.displayOrder ?? 0));
3089 % }
3090 %
3091 % async createRestaurantTimeSlot(slot: InsertRestaurantTimeSlot): Promise<RestaurantTimeSlot> {
3092 %     const id = randomUUID();
3093 %     const newSlot: RestaurantTimeSlot = {
3094 %         id,
3095 %         time: slot.time,
3096 %         label: slot.label ?? null,
3097 %         isActive: slot.isActive ?? "true",
3098 %         displayOrder: slot.displayOrder ?? 0,
3099 %     };
3100 %     this.restaurantTimeSlots.set(id, newSlot);
3101 %     return newSlot;
3102 % }
3103 %
3104 % async updateRestaurantTimeSlot(id: string, slot: Partial<InsertRestaurantTimeSlot>): Promise<RestaurantTimeSlot | u...
3105 %     const existing = this.restaurantTimeSlots.get(id);
3106 %     if (!existing) return undefined;
3107 %     const updated = { ...existing, ...slot } as RestaurantTimeSlot;
3108 %     this.restaurantTimeSlots.set(id, updated);
3109 %     return updated;
3110 % }
3111 %
3112 % async deleteRestaurantTimeSlot(id: string): Promise<boolean> {
3113 %     return this.restaurantTimeSlots.delete(id);
3114 % }
3115 %
3116 % // Recipes
3117 % async getRecipes(): Promise<RecipeWithIngredients[]> {
3118 %     return Array.from(this.recipesMap.values()).map(recipe => {
3119 %         const menuItem = this.menuItems.get(recipe.menuItemId);
3120 %         const ingredients = Array.from(this.recipeIngredientsMap.values())
3121 %             .filter(i => i.recipeId === recipe.id);
3122 %         return { ...recipe, menuItem, ingredients };
3123 %     });
3124 % }
3125 %
3126 % async getRecipe(id: string): Promise<RecipeWithIngredients | undefined> {
3127 %     const recipe = this.recipesMap.get(id);
3128 %     if (!recipe) return undefined;
3129 %     const menuItem = this.menuItems.get(recipe.menuItemId);
3130 %     const ingredients = Array.from(this.recipeIngredientsMap.values())
3131 %         .filter(i => i.recipeId === recipe.id);
3132 %     return { ...recipe, menuItem, ingredients };
3133 % }
3134 %
3135 % async getRecipeByMenuItem(menuItemId: string): Promise<RecipeWithIngredients | undefined> {
3136 %     const recipe = Array.from(this.recipesMap.values()).find(r => r.menuItemId === menuItemId);
3137 %     if (!recipe) return undefined;
3138 %     const menuItem = this.menuItems.get(recipe.menuItemId);
3139 %     const ingredients = Array.from(this.recipeIngredientsMap.values())
3140 %         .filter(i => i.recipeId === recipe.id);
3141 %     return { ...recipe, menuItem, ingredients };
3142 % }
3143 %
3144 % async createRecipe(recipe: InsertRecipe): Promise<Recipe> {
3145 %     const id = randomUUID();
3146 %     const newRecipe: Recipe = {
3147 %         id,
3148 %         menuItemId: recipe.menuItemId,
3149 %         notes: recipe.notes ?? null,
3150 %     };
3151 %     this.recipesMap.set(id, newRecipe);
3152 %     return newRecipe;
3153 % }
3154 %
3155 % async updateRecipe(id: string, recipe: Partial<InsertRecipe>): Promise<Recipe | undefined> {
3156 %     const existing = this.recipesMap.get(id);
3157 %     if (!existing) return undefined;
3158 %     const updated = { ...existing, ...recipe } as Recipe;
3159 %     this.recipesMap.set(id, updated);

```

server/storage.ts (cont.)

```
3160 %     return updated;
3161 % }
3162 %
3163 % async deleteRecipe(id: string): Promise<boolean> {
3164 %     Array.from(this.recipeIngredientsMap.entries())
3165 %         .filter(([, i]) => i.recipeId === id)
3166 %         .forEach(([, key]) => this.recipeIngredientsMap.delete(key));
3167 %     return this.recipesMap.delete(id);
3168 % }
3169 %
3170 % // Recipe Ingredients
3171 % async getRecipeIngredients(recipeId: string): Promise<RecipeIngredient[]> {
3172 %     return Array.from(this.recipeIngredientsMap.values())
3173 %         .filter(i => i.recipeId === recipeId);
3174 % }
3175 %
3176 % async createRecipeIngredient(ingredient: InsertRecipeIngredient): Promise<RecipeIngredient> {
3177 %     const id = randomUUID();
3178 %     const newIngredient: RecipeIngredient = {
3179 %         id,
3180 %         recipeId: ingredient.recipeId,
3181 %         inventoryItemId: ingredient.inventoryItemId ?? null,
3182 %         ingredientName: ingredient.ingredientName,
3183 %         quantity: ingredient.quantity,
3184 %         unit: ingredient.unit,
3185 %         unitCost: ingredient.unitCost ?? "0",
3186 %     };
3187 %     this.recipeIngredientsMap.set(id, newIngredient);
3188 %     return newIngredient;
3189 % }
3190 %
3191 % async updateRecipeIngredient(id: string, ingredient: Partial<InsertRecipeIngredient>): Promise<RecipeIngredient | u...
3192 %     const existing = this.recipeIngredientsMap.get(id);
3193 %     if (!existing) return undefined;
3194 %     const updated = { ...existing, ...ingredient } as RecipeIngredient;
3195 %     this.recipeIngredientsMap.set(id, updated);
3196 %     return updated;
3197 % }
3198 %
3199 % async deleteRecipeIngredient(id: string): Promise<boolean> {
3200 %     return this.recipeIngredientsMap.delete(id);
3201 % }
3202 %
3203 % // Order Splits
3204 % async getOrderSplits(orderId: string): Promise<OrderSplit[]> {
3205 %     return Array.from(this.orderSplitsMap.values())
3206 %         .filter(s => s.orderId === orderId)
3207 %         .sort((a, b) => a.splitNumber - b.splitNumber);
3208 % }
3209 %
3210 % async createOrderSplit(split: InsertOrderSplit): Promise<OrderSplit> {
3211 %     const id = randomUUID();
3212 %     const newSplit: OrderSplit = {
3213 %         id,
3214 %         orderId: split.orderId,
3215 %         splitNumber: split.splitNumber,
3216 %         amount: split.amount,
3217 %         method: split.method ?? null,
3218 %         receiptType: split.receiptType ?? null,
3219 %         isPaid: split.isPaid ?? "false",
3220 %         paidAt: split.paidAt ?? null,
3221 %         createdAt: split.createdAt ?? new Date().toISOString(),
3222 %     };
3223 %     this.orderSplitsMap.set(id, newSplit);
3224 %     return newSplit;
3225 % }
3226 %
3227 % async updateOrderSplit(id: string, split: Partial<InsertOrderSplit>): Promise<OrderSplit | undefined> {
3228 %     const existing = this.orderSplitsMap.get(id);
3229 %     if (!existing) return undefined;
3230 %     const updated: OrderSplit = { ...existing, ...split };
3231 %     this.orderSplitsMap.set(id, updated);
3232 %     return updated;
3233 % }
3234 %
3235 % async deleteOrderSplitsByOrder(orderId: string): Promise<boolean> {
3236 %     const toDelete = Array.from(this.orderSplitsMap.values()).filter(s => s.orderId === orderId);
3237 %     toDelete.forEach(s => this.orderSplitsMap.delete(s.id));
3238 %     return true;
3239 % }
3240 %
3241 % // ===== INVENTORY =====
3242 % async getItemCategories(): Promise<ItemCategory[]> {
3243 %     return Array.from(this.itemCategories.values()).filter(c => c.isActive === "true");
3244 % }
3245 %
3246 % async getItemCategory(id: string): Promise<ItemCategory | undefined> {
3247 %     return this.itemCategories.get(id);
3248 % }
3249 %
3250 % async createItemCategory(category: InsertItemCategory): Promise<ItemCategory> {
3251 %     const id = randomUUID();
3252 %     const newCategory: ItemCategory = {
```

server/storage.ts (cont.)

```

3253 %         id,
3254 %         name: category.name,
3255 %         description: category.description ?? null,
3256 %         parentId: category.parentId ?? null,
3257 %         isActive: category.isActive ?? "true",
3258 %     };
3259 %     this.itemCategories.set(id, newCategory);
3260 %     return newCategory;
3261 % }
3262 %
3263 % async updateItemCategory(id: string, category: Partial<InsertItemCategory>): Promise<ItemCategory | undefined> {
3264 %     const existing = this.itemCategories.get(id);
3265 %     if (!existing) return undefined;
3266 %     const updated = { ...existing, ...category };
3267 %     this.itemCategories.set(id, updated);
3268 %     return updated;
3269 % }
3270 %
3271 % async deleteItemCategory(id: string): Promise<boolean> {
3272 %     return this.itemCategories.delete(id);
3273 % }
3274 %
3275 % async getSuppliers(): Promise<Supplier[]> {
3276 %     return Array.from(this.suppliers.values()).filter(s => s.isActive === "true");
3277 % }
3278 %
3279 % async getSupplier(id: string): Promise<Supplier | undefined> {
3280 %     return this.suppliers.get(id);
3281 % }
3282 %
3283 % async createSupplier(supplier: InsertSupplier): Promise<Supplier> {
3284 %     const id = randomUUID();
3285 %     const newSupplier: Supplier = {
3286 %         id,
3287 %         name: supplier.name,
3288 %         contactName: supplier.contactName ?? null,
3289 %         phone: supplier.phone ?? null,
3290 %         email: supplier.email ?? null,
3291 %         address: supplier.address ?? null,
3292 %         cuit: supplier.cuit ?? null,
3293 %         paymentTermDays: supplier.paymentTermDays ?? 30,
3294 %         notes: supplier.notes ?? null,
3295 %         isActive: supplier.isActive ?? "true",
3296 %     };
3297 %     this.suppliers.set(id, newSupplier);
3298 %     return newSupplier;
3299 % }
3300 %
3301 % async updateSupplier(id: string, supplier: Partial<InsertSupplier>): Promise<Supplier | undefined> {
3302 %     const existing = this.suppliers.get(id);
3303 %     if (!existing) return undefined;
3304 %     const updated = { ...existing, ...supplier };
3305 %     this.suppliers.set(id, updated);
3306 %     return updated;
3307 % }
3308 %
3309 % async deleteSupplier(id: string): Promise<boolean> {
3310 %     return this.suppliers.delete(id);
3311 % }
3312 %
3313 % async getInventoryItems(): Promise<InventoryItemWithDetails[]> {
3314 %     return Array.from(this.inventoryItems.values())
3315 %         .filter(i => i.isActive === "true")
3316 %         .map(item => ({
3317 %             ...item,
3318 %             category: item.categoryId ? this.itemCategories.get(item.categoryId) : undefined,
3319 %             supplier: item.supplierId ? this.suppliers.get(item.supplierId) : undefined,
3320 %         }));
3321 % }
3322 %
3323 % async getInventoryItem(id: string): Promise<InventoryItemWithDetails | undefined> {
3324 %     const item = this.inventoryItems.get(id);
3325 %     if (!item) return undefined;
3326 %     return {
3327 %         ...item,
3328 %         category: item.categoryId ? this.itemCategories.get(item.categoryId) : undefined,
3329 %         supplier: item.supplierId ? this.suppliers.get(item.supplierId) : undefined,
3330 %     };
3331 % }
3332 %
3333 % async getInventoryItemsBelowMinStock(): Promise<InventoryItem[]> {
3334 %     return Array.from(this.inventoryItems.values())
3335 %         .filter(i => i.isActive === "true" && (i.currentStock ?? 0) < (i.minStock ?? 0));
3336 % }
3337 %
3338 % async createInventoryItem(item: InsertInventoryItem): Promise<InventoryItem> {
3339 %     const id = randomUUID();
3340 %     const newItem: InventoryItem = {
3341 %         id,
3342 %         sku: item.sku ?? null,
3343 %         name: item.name,
3344 %         description: item.description ?? null,
3345 %         categoryId: item.categoryId ?? null,

```

server/storage.ts (cont.)

```

3346 %         supplierId: item.supplierId ?? null,
3347 %         unit: (item.unit ?? "unidad") as "unidad" | "kg" | "g" | "litro" | "ml" | "caja" | "paquete" | "docena",
3348 %         costPrice: item.costPrice ?? "0",
3349 %         minStock: item.minStock ?? 0,
3350 %         maxStock: item.maxStock ?? null,
3351 %         currentStock: item.currentStock ?? 0,
3352 %         location: item.location ?? null,
3353 %         isActive: item.isActive ?? "true",
3354 %     };
3355 %     this.inventoryItems.set(id, newItem);
3356 %     return newItem;
3357 % }
3358 %
3359 % async updateInventoryItem(id: string, item: Partial<InsertInventoryItem>): Promise<InventoryItem | undefined> {
3360 %     const existing = this.inventoryItems.get(id);
3361 %     if (!existing) return undefined;
3362 %     const updated: InventoryItem = {
3363 %         ...existing,
3364 %         ...item,
3365 %         unit: (item.unit ?? existing.unit) as "unidad" | "kg" | "g" | "litro" | "ml" | "caja" | "paquete" | "docena",
3366 %     };
3367 %     this.inventoryItems.set(id, updated);
3368 %     return updated;
3369 % }
3370 %
3371 % async deleteInventoryItem(id: string): Promise<boolean> {
3372 %     return this.inventoryItems.delete(id);
3373 % }
3374 %
3375 % async getStockMovements(itemId?: string): Promise<StockMovementWithItem[]> {
3376 %     let movements = Array.from(this.stockMovements.values());
3377 %     if (itemId) {
3378 %         movements = movements.filter(m => m.itemId === itemId);
3379 %     }
3380 %     return movements
3381 %         .map(movement => ({
3382 %             ...movement,
3383 %             item: this.inventoryItems.get(movement.itemId)!,
3384 %         }))
3385 %         .sort((a, b) => b.createdAt.localeCompare(a.createdAt));
3386 % }
3387 %
3388 % async createStockMovement(movement: InsertStockMovement): Promise<StockMovement> {
3389 %     const id = randomUUID();
3390 %     const newMovement: StockMovement = {
3391 %         id,
3392 %         itemId: movement.itemId,
3393 %         movementType: movement.movementType as "entrada" | "salida" | "ajuste" | "transferencia" | "consumo",
3394 %         quantity: movement.quantity,
3395 %         previousStock: movement.previousStock,
3396 %         newStock: movement.newStock,
3397 %         unitCost: movement.unitCost ?? null,
3398 %         reference: movement.reference ?? null,
3399 %         notes: movement.notes ?? null,
3400 %         createdAt: movement.createdAt,
3401 %         createdBy: movement.createdBy ?? null,
3402 %     };
3403 %     this.stockMovements.set(id, newMovement);
3404 %
3405 %     // Update current stock in the inventory item
3406 %     const item = this.inventoryItems.get(movement.itemId);
3407 %     if (item) {
3408 %         item.currentStock = movement.newStock;
3409 %         this.inventoryItems.set(movement.itemId, item);
3410 %     }
3411 %
3412 %     return newMovement;
3413 % }
3414 %
3415 % // ===== SPA METHODS =====
3416 %
3417 % async getSpaCabins(): Promise<SpaCabin[]> {
3418 %     return Array.from(this.spaCabins.values()).filter(c => c.isActive === "true");
3419 % }
3420 %
3421 % async getSpaCabin(id: string): Promise<SpaCabin | undefined> {
3422 %     return this.spaCabins.get(id);
3423 % }
3424 %
3425 % async createSpaCabin(cabin: InsertSpaCabin): Promise<SpaCabin> {
3426 %     const id = randomUUID();
3427 %     const newCabin: SpaCabin = {
3428 %         id,
3429 %         name: cabin.name,
3430 %         description: cabin.description ?? null,
3431 %         isActive: cabin.isActive ?? "true",
3432 %     };
3433 %     this.spaCabins.set(id, newCabin);
3434 %     return newCabin;
3435 % }
3436 %
3437 % async updateSpaCabin(id: string, cabin: Partial<InsertSpaCabin>): Promise<SpaCabin | undefined> {
3438 %     const existing = this.spaCabins.get(id);

```

server/storage.ts (cont.)

```
3439 %     if (!existing) return undefined;
3440 %     const updated = { ...existing, ...cabin };
3441 %     this.spaCabins.set(id, updated);
3442 %     return updated;
3443 % }
3444 %
3445 % async deleteSpaCabin(id: string): Promise<boolean> {
3446 %     return this.spaCabins.delete(id);
3447 % }
3448 %
3449 % async getSpaTreatmentCategories(): Promise<SpaTreatmentCategory[]> {
3450 %     return Array.from(this.spaTreatmentCategories.values()).sort((a, b) => (a.sortOrder ?? 0) - (b.sortOrder ?? 0));
3451 % }
3452 %
3453 % async getSpaTreatmentCategory(id: string): Promise<SpaTreatmentCategory | undefined> {
3454 %     return this.spaTreatmentCategories.get(id);
3455 % }
3456 %
3457 % async createSpaTreatmentCategory(category: InsertSpaTreatmentCategory): Promise<SpaTreatmentCategory> {
3458 %     const id = randomUUID();
3459 %     const newCategory: SpaTreatmentCategory = {
3460 %         id,
3461 %         name: category.name,
3462 %         description: category.description ?? null,
3463 %         sortOrder: category.sortOrder ?? 0,
3464 %     };
3465 %     this.spaTreatmentCategories.set(id, newCategory);
3466 %     return newCategory;
3467 % }
3468 %
3469 % async updateSpaTreatmentCategory(id: string, category: Partial<InsertSpaTreatmentCategory>): Promise<SpaTreatmentCa...
3470 %     const existing = this.spaTreatmentCategories.get(id);
3471 %     if (!existing) return undefined;
3472 %     const updated = { ...existing, ...category };
3473 %     this.spaTreatmentCategories.set(id, updated);
3474 %     return updated;
3475 % }
3476 %
3477 % async deleteSpaTreatmentCategory(id: string): Promise<boolean> {
3478 %     return this.spaTreatmentCategories.delete(id);
3479 % }
3480 %
3481 % async getSpaTreatments(): Promise<SpaTreatment[]> {
3482 %     return Array.from(this.spaTreatments.values()).filter(t => t.isActive === "true");
3483 % }
3484 %
3485 % async getSpaTreatment(id: string): Promise<SpaTreatment | undefined> {
3486 %     return this.spaTreatments.get(id);
3487 % }
3488 %
3489 % async getSpaTreatmentsByCategory(categoryId: string): Promise<SpaTreatment[]> {
3490 %     return Array.from(this.spaTreatments.values()).filter(t => t.categoryId === categoryId && t.isActive === "true");
3491 % }
3492 %
3493 % async createSpaTreatment(treatment: InsertSpaTreatment): Promise<SpaTreatment> {
3494 %     const id = randomUUID();
3495 %     const newTreatment: SpaTreatment = {
3496 %         id,
3497 %         categoryId: treatment.categoryId ?? null,
3498 %         name: treatment.name,
3499 %         description: treatment.description ?? null,
3500 %         durationMinutes: treatment.durationMinutes ?? 60,
3501 %         price: treatment.price ?? "0",
3502 %         isActive: treatment.isActive ?? "true",
3503 %     };
3504 %     this.spaTreatments.set(id, newTreatment);
3505 %     return newTreatment;
3506 % }
3507 %
3508 % async updateSpaTreatment(id: string, treatment: Partial<InsertSpaTreatment>): Promise<SpaTreatment | undefined> {
3509 %     const existing = this.spaTreatments.get(id);
3510 %     if (!existing) return undefined;
3511 %     const updated = { ...existing, ...treatment };
3512 %     this.spaTreatments.set(id, updated);
3513 %     return updated;
3514 % }
3515 %
3516 % async deleteSpaTreatment(id: string): Promise<boolean> {
3517 %     return this.spaTreatments.delete(id);
3518 % }
3519 %
3520 % async getSpaAppointments(date?: string): Promise<SpaAppointmentWithDetails[]> {
3521 %     let appointments = Array.from(this.spaAppointments.values());
3522 %     if (date) {
3523 %         appointments = appointments.filter(a => a.appointmentDate === date);
3524 %     }
3525 %     return appointments.map(a => ({
3526 %         ...a,
3527 %         cabin: this.spaCabins.get(a.cabinId)!,
3528 %         treatment: this.spaTreatments.get(a.treatmentId)!,
3529 %     })).sort((a, b) => a.startTime.localeCompare(b.startTime));
3530 % }
3531 %
```

server/storage.ts (cont.)

```

3532 %   async getSpaAppointment(id: string): Promise<SpaAppointmentWithDetails | undefined> {
3533 %       const appointment = this.spaAppointments.get(id);
3534 %       if (!appointment) return undefined;
3535 %       return {
3536 %           ...appointment,
3537 %           cabin: this.spaCabins.get(appointment.cabinId)!,
3538 %           treatment: this.spaTreatments.get(appointment.treatmentId)!,
3539 %       };
3540 %   }
3541 %
3542 %   async getSpaAppointmentsByCabin(cabinId: string, date: string): Promise<SpaAppointment[]> {
3543 %       return Array.from(this.spaAppointments.values())
3544 %           .filter(a => a.cabinId === cabinId && a.appointmentDate === date);
3545 %   }
3546 %
3547 %   async getSpaAppointmentsByDateRange(startDate: string, endDate: string): Promise<SpaAppointmentWithDetails[]> {
3548 %       return Array.from(this.spaAppointments.values())
3549 %           .filter(a => a.appointmentDate >= startDate && a.appointmentDate <= endDate)
3550 %           .map(a => ({
3551 %               ...a,
3552 %               cabin: this.spaCabins.get(a.cabinId)!,
3553 %               treatment: this.spaTreatments.get(a.treatmentId)!,
3554 %           })))
3555 %           .sort((a, b) => {
3556 %               if (a.appointmentDate !== b.appointmentDate) {
3557 %                   return a.appointmentDate.localeCompare(b.appointmentDate);
3558 %               }
3559 %               return a.startTime.localeCompare(b.startTime);
3560 %           });
3561 %   }
3562 %
3563 %   async createSpaAppointment(appointment: InsertSpaAppointment): Promise<SpaAppointment> {
3564 %       const id = randomUUID();
3565 %       const newAppointment: SpaAppointment = {
3566 %           id,
3567 %           cabinId: appointment.cabinId,
3568 %           treatmentId: appointment.treatmentId,
3569 %           guestName: appointment.guestName,
3570 %           guestLastName: appointment.guestLastName ?? null,
3571 %           guestPhone: appointment.guestPhone ?? null,
3572 %           guestEmail: appointment.guestEmail ?? null,
3573 %           reservationId: appointment.reservationId ?? null,
3574 %           appointmentDate: appointment.appointmentDate,
3575 %           startTime: appointment.startTime,
3576 %           endTime: appointment.endTime,
3577 %           status: (appointment.status ?? "pending") as SpaAppointmentStatus,
3578 %           notes: appointment.notes ?? null,
3579 %           createdAt: appointment.createdAt,
3580 %       };
3581 %       this.spaAppointments.set(id, newAppointment);
3582 %       return newAppointment;
3583 %   }
3584 %
3585 %   async updateSpaAppointment(id: string, appointment: Partial<InsertSpaAppointment>): Promise<SpaAppointment | undefi...
3586 %       const existing = this.spaAppointments.get(id);
3587 %       if (!existing) return undefined;
3588 %       const updated: SpaAppointment = {
3589 %           ...existing,
3590 %           ...appointment,
3591 %           status: (appointment.status ?? existing.status) as SpaAppointmentStatus,
3592 %       };
3593 %       this.spaAppointments.set(id, updated);
3594 %       return updated;
3595 %   }
3596 %
3597 %   async deleteSpaAppointment(id: string): Promise<boolean> {
3598 %       return this.spaAppointments.delete(id);
3599 %   }
3600 %
3601 %   async getSpaAccounts(status?: SpaAccountStatus): Promise<SpaAccountWithItems[]> {
3602 %       let accounts = Array.from(this.spaAccounts.values());
3603 %       if (status) {
3604 %           accounts = accounts.filter(a => a.status === status);
3605 %       }
3606 %       return accounts.map(account => {
3607 %           const items = Array.from(this.spaAccountItems.values()).filter(i => i.accountId === account.id);
3608 %           const payments = Array.from(this.spaPayments.values()).filter(p => p.accountId === account.id);
3609 %           const appointment = this.spaAppointments.get(account.appointmentId);
3610 %           return {
3611 %               ...account,
3612 %               items,
3613 %               payments,
3614 %               appointment: appointment ? {
3615 %                   ...appointment,
3616 %                   cabin: this.spaCabins.get(appointment.cabinId)!,
3617 %                   treatment: this.spaTreatments.get(appointment.treatmentId)!,
3618 %               } : undefined,
3619 %           };
3620 %       });
3621 %   }
3622 %
3623 %   async getSpaAccount(id: string): Promise<SpaAccountWithItems | undefined> {
3624 %       const account = this.spaAccounts.get(id);

```


server/storage.ts (cont.)

```

3625 %     if (!account) return undefined;
3626 %     const items = Array.from(this.spaAccountItems.values()).filter(i => i.accountId === id);
3627 %     const payments = Array.from(this.spaPayments.values()).filter(p => p.accountId === id);
3628 %     const appointment = this.spaAppointments.get(account.appointmentId);
3629 %     return {
3630 %         ...account,
3631 %         items,
3632 %         payments,
3633 %         appointment: appointment ? {
3634 %             ...appointment,
3635 %             cabin: this.spaCabins.get(appointment.cabinId)!,
3636 %             treatment: this.spaTreatments.get(appointment.treatmentId)!,
3637 %         } : undefined,
3638 %     };
3639 % }
3640 %
3641 % async getSpaAccountByAppointment(appointmentId: string): Promise<SpaAccountWithItems | undefined> {
3642 %     const account = Array.from(this.spaAccounts.values()).find(a => a.appointmentId === appointmentId);
3643 %     if (!account) return undefined;
3644 %     return this.getSpaAccount(account.id);
3645 % }
3646 %
3647 % async createSpaAccount(account: InsertSpaAccount): Promise<SpaAccount> {
3648 %     const id = randomUUID();
3649 %     const newAccount: SpaAccount = {
3650 %         id,
3651 %         appointmentId: account.appointmentId,
3652 %         guestName: account.guestName,
3653 %         reservationId: account.reservationId ?? null,
3654 %         status: (account.status ?? "open") as SpaAccountStatus,
3655 %         subtotal: account.subtotal ?? "0",
3656 %         total: account.total ?? "0",
3657 %         totalPaid: account.totalPaid ?? "0",
3658 %         receiptType: account.receiptType ?? null,
3659 %         notes: account.notes ?? null,
3660 %         openedAt: account.openedAt,
3661 %         closedAt: account.closedAt ?? null,
3662 %         closedBy: account.closedBy ?? null,
3663 %         chargedTo: account.chargedTo ?? null,
3664 %     };
3665 %     this.spaAccounts.set(id, newAccount);
3666 %     return newAccount;
3667 % }
3668 %
3669 % async updateSpaAccount(id: string, account: Partial<InsertSpaAccount>): Promise<SpaAccount | undefined> {
3670 %     const existing = this.spaAccounts.get(id);
3671 %     if (!existing) return undefined;
3672 %     const updated: SpaAccount = {
3673 %         ...existing,
3674 %         ...account,
3675 %         status: (account.status ?? existing.status) as SpaAccountStatus,
3676 %     };
3677 %     this.spaAccounts.set(id, updated);
3678 %     return updated;
3679 % }
3680 %
3681 % async closeSpaAccount(id: string, chargedTo: string, receiptType?: string): Promise<SpaAccount | undefined> {
3682 %     const account = this.spaAccounts.get(id);
3683 %     if (!account) return undefined;
3684 %     const items = Array.from(this.spaAccountItems.values()).filter(i => i.accountId === id);
3685 %     const total = items.reduce((sum, item) => sum + parseFloat(item.subtotal), 0);
3686 %     const payments = Array.from(this.spaPayments.values()).filter(p => p.accountId === id);
3687 %     const totalPaid = payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3688 %
3689 %     const updated: SpaAccount = {
3690 %         ...account,
3691 %         status: "closed",
3692 %         subtotal: total.toFixed(2),
3693 %         total: total.toFixed(2),
3694 %         totalPaid: totalPaid.toFixed(2),
3695 %         receiptType: receiptType ?? null,
3696 %         closedAt: new Date().toISOString(),
3697 %         chargedTo,
3698 %     };
3699 %     this.spaAccounts.set(id, updated);
3700 %
3701 %     // If charged to room, create a charge on the reservation
3702 %     if (chargedTo.startsWith("room:")) {
3703 %         const reservationId = chargedTo.replace("room:", "");
3704 %         const reservation = this.reservations.get(reservationId);
3705 %         if (reservation) {
3706 %             const chargeId = randomUUID();
3707 %             const charge: Charge = {
3708 %                 id: chargeId,
3709 %                 reservationId,
3710 %                 category: "spa",
3711 %                 description: "Servicios SPA",
3712 %                 amount: total.toFixed(2),
3713 %                 date: new Date().toISOString().split("T")[0],
3714 %                 createdBy: null,
3715 %             };
3716 %             this.charges.set(chargeId, charge);
3717 %         }

```

server/storage.ts (cont.)

```
3718 %     }
3719 %
3720 %     return updated;
3721 % }
3722 %
3723 % async getSpaAccountItems(accountId: string): Promise<SpaAccountItem[]> {
3724 %     return Array.from(this.spaAccountItems.values()).filter(i => i.accountId === accountId);
3725 % }
3726 %
3727 % async createSpaAccountItem(item: InsertSpaAccountItem): Promise<SpaAccountItem> {
3728 %     const id = randomUUID();
3729 %     const newItem: SpaAccountItem = {
3730 %         id,
3731 %         accountId: item.accountId,
3732 %         description: item.description,
3733 %         quantity: item.quantity ?? 1,
3734 %         unitPrice: item.unitPrice,
3735 %         subtotal: item.subtotal,
3736 %         itemType: item.itemType ?? "treatment",
3737 %         notes: item.notes ?? null,
3738 %         createdAt: item.createdAt,
3739 %     };
3740 %     this.spaAccountItems.set(id, newItem);
3741 %
3742 %     // Update account totals
3743 %     const account = this.spaAccounts.get(item.accountId);
3744 %     if (account) {
3745 %         const items = Array.from(this.spaAccountItems.values()).filter(i => i.accountId === item.accountId);
3746 %         const total = items.reduce((sum, i) => sum + parseFloat(i.subtotal), 0);
3747 %         account.subtotal = total.toFixed(2);
3748 %         account.total = total.toFixed(2);
3749 %         this.spaAccounts.set(item.accountId, account);
3750 %     }
3751 %
3752 %     return newItem;
3753 % }
3754 %
3755 % async updateSpaAccountItem(id: string, item: Partial<InsertSpaAccountItem>): Promise<SpaAccountItem | undefined> {
3756 %     const existing = this.spaAccountItems.get(id);
3757 %     if (!existing) return undefined;
3758 %     const updated = { ...existing, ...item };
3759 %     this.spaAccountItems.set(id, updated);
3760 %
3761 %     // Update account totals
3762 %     const account = this.spaAccounts.get(existing.accountId);
3763 %     if (account) {
3764 %         const items = Array.from(this.spaAccountItems.values()).filter(i => i.accountId === existing.accountId);
3765 %         const total = items.reduce((sum, i) => sum + parseFloat(i.subtotal), 0);
3766 %         account.subtotal = total.toFixed(2);
3767 %         account.total = total.toFixed(2);
3768 %         this.spaAccounts.set(existing.accountId, account);
3769 %     }
3770 %
3771 %     return updated;
3772 % }
3773 %
3774 % async deleteSpaAccountItem(id: string): Promise<boolean> {
3775 %     const item = this.spaAccountItems.get(id);
3776 %     if (!item) return false;
3777 %     const accountId = item.accountId;
3778 %     const deleted = this.spaAccountItems.delete(id);
3779 %
3780 %     // Update account totals
3781 %     if (deleted) {
3782 %         const account = this.spaAccounts.get(accountId);
3783 %         if (account) {
3784 %             const items = Array.from(this.spaAccountItems.values()).filter(i => i.accountId === accountId);
3785 %             const total = items.reduce((sum, i) => sum + parseFloat(i.subtotal), 0);
3786 %             account.subtotal = total.toFixed(2);
3787 %             account.total = total.toFixed(2);
3788 %             this.spaAccounts.set(accountId, account);
3789 %         }
3790 %     }
3791 %
3792 %     return deleted;
3793 % }
3794 %
3795 % async getSpaPayments(accountId: string): Promise<SpaPayment[]> {
3796 %     return Array.from(this.spaPayments.values()).filter(p => p.accountId === accountId);
3797 % }
3798 %
3799 % async createSpaPayment(payment: InsertSpaPayment): Promise<SpaPayment> {
3800 %     const id = randomUUID();
3801 %     const newPayment: SpaPayment = {
3802 %         id,
3803 %         accountId: payment.accountId,
3804 %         amount: payment.amount,
3805 %         method: payment.method as SpaPaymentMethod,
3806 %         isAdvance: payment.isAdvance ?? "false",
3807 %         appointmentId: payment.appointmentId ?? null,
3808 %         reservationId: payment.reservationId ?? null,
3809 %         notes: payment.notes ?? null,
3810 %         createdAt: payment.createdAt,
```

server/storage.ts (cont.)

```

3811 %     };
3812 %     this.spaPayments.set(id, newPayment);
3813 %
3814 %     const account = this.spaAccounts.get(payment.accountId);
3815 %     if (account) {
3816 %         const allPayments = Array.from(this.spaPayments.values()).filter(p => p.accountId === payment.accountId);
3817 %         const totalPaid = allPayments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3818 %         account.totalPaid = totalPaid.toFixed(2);
3819 %         this.spaAccounts.set(payment.accountId, account);
3820 %     }
3821 %
3822 %     return newPayment;
3823 % }
3824 %
3825 % async deleteSpaPayment(id: string): Promise<boolean> {
3826 %     const payment = this.spaPayments.get(id);
3827 %     if (!payment) return false;
3828 %     const accountId = payment.accountId;
3829 %     const deleted = this.spaPayments.delete(id);
3830 %
3831 %     if (deleted) {
3832 %         const account = this.spaAccounts.get(accountId);
3833 %         if (account) {
3834 %             const allPayments = Array.from(this.spaPayments.values()).filter(p => p.accountId === accountId);
3835 %             const totalPaid = allPayments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
3836 %             account.totalPaid = totalPaid.toFixed(2);
3837 %             this.spaAccounts.set(accountId, account);
3838 %         }
3839 %     }
3840 %
3841 %     return deleted;
3842 % }
3843 %
3844 % // ===== EVENTS =====
3845 % // Event Rooms
3846 % async getEventRooms(): Promise<EventRoom[]> {
3847 %     return Array.from(this.eventRooms.values());
3848 % }
3849 %
3850 % async getEventRoom(id: string): Promise<EventRoom | undefined> {
3851 %     return this.eventRooms.get(id);
3852 % }
3853 %
3854 % async createEventRoom(room: InsertEventRoom): Promise<EventRoom> {
3855 %     const id = randomUUID();
3856 %     const newRoom: EventRoom = {
3857 %         id,
3858 %         name: room.name,
3859 %         capacity: room.capacity ?? 50,
3860 %         status: (room.status ?? "available") as EventRoomStatus,
3861 %         description: room.description ?? null,
3862 %         amenities: room.amenities ?? null,
3863 %         isActive: room.isActive ?? "true",
3864 %     };
3865 %     this.eventRooms.set(id, newRoom);
3866 %     return newRoom;
3867 % }
3868 %
3869 % async updateEventRoom(id: string, room: Partial<InsertEventRoom>): Promise<EventRoom | undefined> {
3870 %     const existing = this.eventRooms.get(id);
3871 %     if (!existing) return undefined;
3872 %     const updated: EventRoom = {
3873 %         ...existing,
3874 %         ...room,
3875 %         status: (room.status ?? existing.status) as EventRoomStatus,
3876 %     };
3877 %     this.eventRooms.set(id, updated);
3878 %     return updated;
3879 % }
3880 %
3881 % async deleteEventRoom(id: string): Promise<boolean> {
3882 %     return this.eventRooms.delete(id);
3883 % }
3884 %
3885 % // Events
3886 % async getEvents(): Promise<EventWithDetails[]> {
3887 %     return Array.from(this.events.values()).map(event => this.enrichEvent(event));
3888 % }
3889 %
3890 % async getEvent(id: string): Promise<EventWithDetails | undefined> {
3891 %     const event = this.events.get(id);
3892 %     if (!event) return undefined;
3893 %     return this.enrichEvent(event);
3894 % }
3895 %
3896 % async getEventsByDateRange(startDate: string, endDate: string): Promise<EventWithDetails[]> {
3897 %     const events = Array.from(this.events.values()).filter(event => {
3898 %         return event.startDate <= endDate && event.endDate >= startDate;
3899 %     });
3900 %     return events.map(event => this.enrichEvent(event));
3901 % }
3902 %
3903 % private enrichEvent(event: HotelEvent): EventWithDetails {

```

server/storage.ts (cont.)

```

3904 %     const eventRoom = this.eventRooms.get(event.eventRoomId!);
3905 %     const company = event.companyId ? this.companies.get(event.companyId) : undefined;
3906 %     const charges = Array.from(this.eventCharges.values())
3907 %       .filter(c => c.eventId === event.id)
3908 %       .map(charge => {
3909 %         const chargeType = charge.chargeTypeId ? this.eventChargeTypes.get(charge.chargeTypeId) : undefined;
3910 %         return { ...charge, chargeType };
3911 %       });
3912 %     const payments = Array.from(this.eventPayments.values())
3913 %       .filter(p => p.eventId === event.id);
3914 %     return { ...event, eventRoom, company, charges, payments };
3915 %   }
3916 %
3917 %   async createEvent(event: InsertEvent): Promise<HotelEvent> {
3918 %     const id = randomUUID();
3919 %     const newEvent: HotelEvent = {
3920 %       id,
3921 %       eventCode: event.eventCode,
3922 %       name: event.name,
3923 %       eventRoomId: event.eventRoomId,
3924 %       eventType: (event.eventType ?? "corporate") as EventType,
3925 %       contactName: event.contactName,
3926 %       contactPhone: event.contactPhone ?? null,
3927 %       contactEmail: event.contactEmail ?? null,
3928 %       companyId: event.companyId ?? null,
3929 %       startDate: event.startDate,
3930 %       endDate: event.endDate,
3931 %       startTime: event.startTime ?? null,
3932 %       endTime: event.endTime ?? null,
3933 %       attendees: event.attendees ?? 10,
3934 %       status: (event.status ?? "tentative") as EventStatus,
3935 %       notes: event.notes ?? null,
3936 %       receiptType: event.receiptType ?? null,
3937 %       closedAt: event.closedAt ?? null,
3938 %       totalAmount: event.totalAmount ?? null,
3939 %       totalPaid: event.totalPaid ?? null,
3940 %       createdAt: event.createdAt,
3941 %     };
3942 %     this.events.set(id, newEvent);
3943 %     return newEvent;
3944 %   }
3945 %
3946 %   async updateEvent(id: string, event: Partial<InsertEvent>): Promise<HotelEvent | undefined> {
3947 %     const existing = this.events.get(id);
3948 %     if (!existing) return undefined;
3949 %     const updated: HotelEvent = {
3950 %       ...existing,
3951 %       ...event,
3952 %       eventType: (event.eventType ?? existing.eventType) as EventType,
3953 %       status: (event.status ?? existing.status) as EventStatus,
3954 %     };
3955 %     this.events.set(id, updated);
3956 %     return updated;
3957 %   }
3958 %
3959 %   async deleteEvent(id: string): Promise<boolean> {
3960 %     // Also delete associated charges
3961 %     const chargesToDelete = Array.from(this.eventCharges.values()).filter(c => c.eventId === id);
3962 %     chargesToDelete.forEach(c => this.eventCharges.delete(c.id));
3963 %     return this.events.delete(id);
3964 %   }
3965 %
3966 %   generateEventCode(): string {
3967 %     this.eventCounter++;
3968 %     const year = new Date().getFullYear();
3969 %     return `EVT-${year}-${this.eventCounter.toString().padStart(4, "0")}`;
3970 %   }
3971 %
3972 %   // Event Charge Types
3973 %   async getEventChargeTypes(): Promise<EventChargeType[]> {
3974 %     return Array.from(this.eventChargeTypes.values());
3975 %   }
3976 %
3977 %   async getEventChargeType(id: string): Promise<EventChargeType | undefined> {
3978 %     return this.eventChargeTypes.get(id);
3979 %   }
3980 %
3981 %   async createEventChargeType(chargeType: InsertEventChargeType): Promise<EventChargeType> {
3982 %     const id = randomUUID();
3983 %     const newChargeType: EventChargeType = {
3984 %       id,
3985 %       code: chargeType.code,
3986 %       name: chargeType.name,
3987 %       defaultPrice: chargeType.defaultPrice ?? null,
3988 %       isActive: chargeType.isActive ?? "true",
3989 %     };
3990 %     this.eventChargeTypes.set(id, newChargeType);
3991 %     return newChargeType;
3992 %   }
3993 %
3994 %   async updateEventChargeType(id: string, chargeType: Partial<InsertEventChargeType>): Promise<EventChargeType | unde...
3995 %     const existing = this.eventChargeTypes.get(id);
3996 %     if (!existing) return undefined;

```

server/storage.ts (cont.)

```
3997 %     const updated: EventChargeType = { ...existing, ...chargeType };
3998 %     this.eventChargeTypes.set(id, updated);
3999 %     return updated;
4000 % }
4001 %
4002 % async deleteEventChargeType(id: string): Promise<boolean> {
4003 %     return this.eventChargeTypes.delete(id);
4004 % }
4005 %
4006 % // Event Charges
4007 % async getEventCharges(eventId: string): Promise<EventChargeWithType[]> {
4008 %     const charges = Array.from(this.eventCharges.values()).filter(c => c.eventId === eventId);
4009 %     return charges.map(charge => {
4010 %         const chargeType = charge.chargeTypeId ? this.eventChargeTypes.get(charge.chargeTypeId) : undefined;
4011 %         return { ...charge, chargeType };
4012 %     });
4013 % }
4014 %
4015 % async createEventCharge(charge: InsertEventCharge): Promise<EventCharge> {
4016 %     const id = randomUUID();
4017 %     const newCharge: EventCharge = {
4018 %         id,
4019 %         eventId: charge.eventId,
4020 %         chargeTypeId: charge.chargeTypeId ?? null,
4021 %         description: charge.description,
4022 %         quantity: charge.quantity ?? 1,
4023 %         unitPrice: charge.unitPrice,
4024 %         totalAmount: charge.totalAmount,
4025 %         date: charge.date,
4026 %         notes: charge.notes ?? null,
4027 %         createdAt: charge.createdAt,
4028 %     };
4029 %     this.eventCharges.set(id, newCharge);
4030 %     return newCharge;
4031 % }
4032 %
4033 % async updateEventCharge(id: string, charge: Partial<InsertEventCharge>): Promise<EventCharge | undefined> {
4034 %     const existing = this.eventCharges.get(id);
4035 %     if (!existing) return undefined;
4036 %     const updated: EventCharge = { ...existing, ...charge };
4037 %     this.eventCharges.set(id, updated);
4038 %     return updated;
4039 % }
4040 %
4041 % async deleteEventCharge(id: string): Promise<boolean> {
4042 %     return this.eventCharges.delete(id);
4043 % }
4044 %
4045 % // Event Planning
4046 % async getEventPlanningData(startDate: string, endDate: string): Promise<EventPlanningData> {
4047 %     const rooms = Array.from(this.eventRooms.values()).filter(r => r.isActive === "true");
4048 %     const days: string[] = [];
4049 %     const currentDate = new Date(startDate);
4050 %     const end = new Date(endDate);
4051 %     while (currentDate <= end) {
4052 %         days.push(currentDate.toISOString().split("T")[0]);
4053 %         currentDate.setDate(currentDate.getDate() + 1);
4054 %     }
4055 %
4056 %     const eventsInRange = Array.from(this.events.values()).filter(event => {
4057 %         return event.startDate <= endDate && event.endDate >= startDate && event.status !== "cancelled";
4058 %     });
4059 %
4060 %     const occupancy: Record<string, EventPlanningCellStatus[]> = {};
4061 %     const eventsMap: Record<string, { id: string; name: string; contactName: string; startDate: string; endDate: stri...
4062 %     const cellEvents: Record<string, Record<string, string>> = {};
4063 %
4064 %     rooms.forEach(room => {
4065 %         occupancy[room.id] = [];
4066 %         cellEvents[room.id] = {};
4067 %
4068 %         days.forEach(day => {
4069 %             const event = eventsInRange.find(e => e.eventRoomId === room.id && e.startDate <= day && e.endDate >= day);
4070 %             if (event) {
4071 %                 occupancy[room.id].push("event");
4072 %                 cellEvents[room.id][day] = event.id;
4073 %                 if (!eventsMap[event.id]) {
4074 %                     eventsMap[event.id] = {
4075 %                         id: event.id,
4076 %                         name: event.name,
4077 %                         contactName: event.contactName,
4078 %                         startDate: event.startDate,
4079 %                         endDate: event.endDate,
4080 %                         status: event.status as EventStatus,
4081 %                         eventType: event.eventType as EventType,
4082 %                     };
4083 %                 }
4084 %             } else if (room.status === "maintenance") {
4085 %                 occupancy[room.id].push("maintenance");
4086 %             } else {
4087 %                 occupancy[room.id].push("available");
4088 %             }
4089 %         });
4090 %     });
```

server/storage.ts (cont.)

```
4090 %     });
4091 %
4092 %     return { rooms, days, occupancy, events: eventsMap, cellEvents };
4093 % }
4094 %
4095 % // Event Payments
4096 % async getEventPayments(eventId: string): Promise<EventPayment[]> {
4097 %     return Array.from(this.eventPayments.values()).filter(p => p.eventId === eventId);
4098 % }
4099 %
4100 % async createEventPayment(payment: InsertEventPayment): Promise<EventPayment> {
4101 %     const id = randomUUID();
4102 %     const newPayment: EventPayment = {
4103 %         id,
4104 %         eventId: payment.eventId,
4105 %         amount: payment.amount,
4106 %         method: payment.method,
4107 %         isAdvance: payment.isAdvance ?? "false",
4108 %         reservationId: payment.reservationId ?? null,
4109 %         notes: payment.notes ?? null,
4110 %         paidAt: payment.paidAt ?? new Date().toISOString(),
4111 %         createdAt: payment.createdAt ?? new Date().toISOString(),
4112 %     };
4113 %     this.eventPayments.set(id, newPayment);
4114 %     return newPayment;
4115 % }
4116 %
4117 % async deleteEventPayment(id: string): Promise<boolean> {
4118 %     return this.eventPayments.delete(id);
4119 % }
4120 %
4121 % // Event Tables
4122 % async getEventTables(eventId: string): Promise<EventTableWithDetails[]> {
4123 %     return Array.from(this.eventTablesMap.values())
4124 %         .filter(t => t.eventId === eventId)
4125 %         .map(table => this.enrichEventTable(table));
4126 % }
4127 %
4128 % async getEventTable(id: string): Promise<EventTableWithDetails | undefined> {
4129 %     const table = this.eventTablesMap.get(id);
4130 %     if (!table) return undefined;
4131 %     return this.enrichEventTable(table);
4132 % }
4133 %
4134 % private enrichEventTable(table: EventTable): EventTableWithDetails {
4135 %     const charges = Array.from(this.eventTableCharges.values()).filter(c => c.eventTableId === table.id);
4136 %     const payments = Array.from(this.eventTablePayments.values()).filter(p => p.eventTableId === table.id);
4137 %     return { ...table, charges, payments };
4138 % }
4139 %
4140 % async createEventTable(table: InsertEventTable): Promise<EventTable> {
4141 %     const id = randomUUID();
4142 %     const newTable: EventTable = {
4143 %         id,
4144 %         eventId: table.eventId,
4145 %         tableNumber: table.tableNumber,
4146 %         label: table.label ?? null,
4147 %         seats: table.seats ?? null,
4148 %         status: table.status ?? "open",
4149 %         reservationId: table.reservationId ?? null,
4150 %         receiptType: table.receiptType ?? null,
4151 %         closedAt: table.closedAt ?? null,
4152 %         createdAt: table.createdAt ?? new Date().toISOString(),
4153 %     };
4154 %     this.eventTablesMap.set(id, newTable);
4155 %     return newTable;
4156 % }
4157 %
4158 % async updateEventTable(id: string, table: Partial<InsertEventTable>): Promise<EventTable | undefined> {
4159 %     const existing = this.eventTablesMap.get(id);
4160 %     if (!existing) return undefined;
4161 %     const updated: EventTable = { ...existing, ...table };
4162 %     this.eventTablesMap.set(id, updated);
4163 %     return updated;
4164 % }
4165 %
4166 % async deleteEventTable(id: string): Promise<boolean> {
4167 %     const charges = Array.from(this.eventTableCharges.values()).filter(c => c.eventTableId === id);
4168 %     charges.forEach(c => this.eventTableCharges.delete(c.id));
4169 %     const payments = Array.from(this.eventTablePayments.values()).filter(p => p.eventTableId === id);
4170 %     payments.forEach(p => this.eventTablePayments.delete(p.id));
4171 %     return this.eventTablesMap.delete(id);
4172 % }
4173 %
4174 % // Event Table Charges
4175 % async getEventTableCharges(tableId: string): Promise<EventTableCharge[]> {
4176 %     return Array.from(this.eventTableCharges.values()).filter(c => c.eventTableId === tableId);
4177 % }
4178 %
4179 % async createEventTableCharge(charge: InsertEventTableCharge): Promise<EventTableCharge> {
4180 %     const id = randomUUID();
4181 %     const newCharge: EventTableCharge = {
4182 %         id,
```

server/storage.ts (cont.)

```

4183 %         eventTableId: charge.eventTableId,
4184 %         description: charge.description,
4185 %         quantity: charge.quantity ?? 1,
4186 %         unitPrice: charge.unitPrice,
4187 %         total: charge.total,
4188 %         createdAt: charge.createdAt ?? new Date().toISOString(),
4189 %     };
4190 %     this.eventTableCharges.set(id, newCharge);
4191 %     return newCharge;
4192 % }
4193 %
4194 % async deleteEventTableCharge(id: string): Promise<boolean> {
4195 %     return this.eventTableCharges.delete(id);
4196 % }
4197 %
4198 % // Event Table Payments
4199 % async getEventTablePayments(tableId: string): Promise<EventTablePayment[]> {
4200 %     return Array.from(this.eventTablePayments.values()).filter(p => p.eventTableId === tableId);
4201 % }
4202 %
4203 % async createEventTablePayment(payment: InsertEventTablePayment): Promise<EventTablePayment> {
4204 %     const id = randomUUID();
4205 %     const newPayment: EventTablePayment = {
4206 %         id,
4207 %         eventTableId: payment.eventTableId,
4208 %         amount: payment.amount,
4209 %         method: payment.method,
4210 %         isAdvance: payment.isAdvance ?? "false",
4211 %         reservationId: payment.reservationId ?? null,
4212 %         receiptType: payment.receiptType ?? null,
4213 %         paidAt: payment.paidAt ?? new Date().toISOString(),
4214 %         createdAt: payment.createdAt ?? new Date().toISOString(),
4215 %     };
4216 %     this.eventTablePayments.set(id, newPayment);
4217 %     return newPayment;
4218 % }
4219 %
4220 % async deleteEventTablePayment(id: string): Promise<boolean> {
4221 %     return this.eventTablePayments.delete(id);
4222 % }
4223 %
4224 % // ===== MAINTENANCE =====
4225 %
4226 % // Maintenance Staff
4227 % async getMaintenanceStaff(): Promise<MaintenanceStaff[]> {
4228 %     return Array.from(this.maintenanceStaff.values());
4229 % }
4230 %
4231 % async getMaintenanceStaffMember(id: string): Promise<MaintenanceStaff | undefined> {
4232 %     return this.maintenanceStaff.get(id);
4233 % }
4234 %
4235 % async createMaintenanceStaff(staff: InsertMaintenanceStaff): Promise<MaintenanceStaff> {
4236 %     const id = randomUUID();
4237 %     const newStaff: MaintenanceStaff = {
4238 %         id,
4239 %         name: staff.name,
4240 %         phone: staff.phone ?? null,
4241 %         email: staff.email ?? null,
4242 %         specialty: staff.specialty ?? null,
4243 %         isActive: staff.isActive ?? "true",
4244 %     };
4245 %     this.maintenanceStaff.set(id, newStaff);
4246 %     return newStaff;
4247 % }
4248 %
4249 % async updateMaintenanceStaff(id: string, staff: Partial<InsertMaintenanceStaff>): Promise<MaintenanceStaff | undefi...
4250 %     const existing = this.maintenanceStaff.get(id);
4251 %     if (!existing) return undefined;
4252 %     const updated: MaintenanceStaff = { ...existing, ...staff };
4253 %     this.maintenanceStaff.set(id, updated);
4254 %     return updated;
4255 % }
4256 %
4257 % async deleteMaintenanceStaff(id: string): Promise<boolean> {
4258 %     return this.maintenanceStaff.delete(id);
4259 % }
4260 %
4261 % // Work Orders
4262 % private enrichWorkOrder(order: WorkOrder): WorkOrderWithDetails {
4263 %     const room = order.roomId ? this.rooms.get(order.roomId) : undefined;
4264 %     const assignedTo = order.assignedToId ? this.maintenanceStaff.get(order.assignedToId) : undefined;
4265 %     return { ...order, room, assignedTo };
4266 % }
4267 %
4268 % async getWorkOrders(): Promise<WorkOrderWithDetails[]> {
4269 %     return Array.from(this.workOrders.values()).map(order => this.enrichWorkOrder(order));
4270 % }
4271 %
4272 % async getWorkOrder(id: string): Promise<WorkOrderWithDetails | undefined> {
4273 %     const order = this.workOrders.get(id);
4274 %     if (!order) return undefined;
4275 %     return this.enrichWorkOrder(order);

```

server/storage.ts (cont.)

```
4276 %     }
4277 %
4278 %     async getWorkOrdersByRoom(roomId: string): Promise<WorkOrderWithDetails[]> {
4279 %         return Array.from(this.workOrders.values())
4280 %             .filter(order => order.roomId === roomId)
4281 %             .map(order => this.enrichWorkOrder(order));
4282 %     }
4283 %
4284 %     async getWorkOrdersByStatus(status: WorkOrderStatus): Promise<WorkOrderWithDetails[]> {
4285 %         return Array.from(this.workOrders.values())
4286 %             .filter(order => order.status === status)
4287 %             .map(order => this.enrichWorkOrder(order));
4288 %     }
4289 %
4290 %     async createWorkOrder(order: InsertWorkOrder): Promise<WorkOrder> {
4291 %         const id = randomUUID();
4292 %         const newOrder: WorkOrder = {
4293 %             id,
4294 %             orderCode: order.orderCode,
4295 %             title: order.title,
4296 %             description: order.description ?? null,
4297 %             roomId: order.roomId ?? null,
4298 %             location: order.location ?? null,
4299 %             category: (order.category ?? "general") as WorkOrderCategory,
4300 %             priority: (order.priority ?? "medium") as WorkOrderPriority,
4301 %             status: (order.status ?? "pending") as WorkOrderStatus,
4302 %             assignedToId: order.assignedToId ?? null,
4303 %             reportedBy: order.reportedBy ?? null,
4304 %             reportedAt: order.reportedAt,
4305 %             scheduledDate: order.scheduledDate ?? null,
4306 %             completedAt: order.completedAt ?? null,
4307 %             completedBy: order.completedBy ?? null,
4308 %             estimatedCost: order.estimatedCost ?? null,
4309 %             actualCost: order.actualCost ?? null,
4310 %             notes: order.notes ?? null,
4311 %         };
4312 %         this.workOrders.set(id, newOrder);
4313 %         return newOrder;
4314 %     }
4315 %
4316 %     async updateWorkOrder(id: string, order: Partial<InsertWorkOrder>): Promise<WorkOrder | undefined> {
4317 %         const existing = this.workOrders.get(id);
4318 %         if (!existing) return undefined;
4319 %         const updated: WorkOrder = {
4320 %             ...existing,
4321 %             ...order,
4322 %             category: (order.category ?? existing.category) as WorkOrderCategory,
4323 %             priority: (order.priority ?? existing.priority) as WorkOrderPriority,
4324 %             status: (order.status ?? existing.status) as WorkOrderStatus,
4325 %         };
4326 %         this.workOrders.set(id, updated);
4327 %         return updated;
4328 %     }
4329 %
4330 %     async deleteWorkOrder(id: string): Promise<boolean> {
4331 %         return this.workOrders.delete(id);
4332 %     }
4333 %
4334 %     generateWorkOrderCode(): string {
4335 %         this.workOrderCounter++;
4336 %         return `OT-${this.workOrderCounter}`;
4337 %     }
4338 %
4339 %     // ===== ADMINISTRATION =====
4340 %
4341 %     // System Users
4342 %     async getSystemUsers(): Promise<SystemUser[]> {
4343 %         return Array.from(this.systemUsers.values());
4344 %     }
4345 %
4346 %     async getSystemUser(id: string): Promise<SystemUser | undefined> {
4347 %         return this.systemUsers.get(id);
4348 %     }
4349 %
4350 %     async getSystemUserByUsername(username: string): Promise<SystemUser | undefined> {
4351 %         return Array.from(this.systemUsers.values()).find((u) => u.username === username);
4352 %     }
4353 %
4354 %     async createSystemUser(user: InsertSystemUser): Promise<SystemUser> {
4355 %         const id = randomUUID();
4356 %         const newUser: SystemUser = {
4357 %             id,
4358 %             username: user.username,
4359 %             email: user.email,
4360 %             fullName: user.fullName,
4361 %             role: (user.role ?? "reception") as SystemUserRole,
4362 %             department: user.department ?? null,
4363 %             phone: user.phone ?? null,
4364 %             isActive: user.isActive ?? "true",
4365 %             lastLogin: user.lastLogin ?? null,
4366 %             createdAt: user.createdAt,
4367 %         };
4368 %         this.systemUsers.set(id, newUser);
```


server/storage.ts (cont.)

```

4369 %     return newUser;
4370 % }
4371 %
4372 % async updateSystemUser(id: string, user: Partial<InsertSystemUser>): Promise<SystemUser | undefined> {
4373 %     const existing = this.systemUsers.get(id);
4374 %     if (!existing) return undefined;
4375 %     const updated: SystemUser = {
4376 %         ...existing,
4377 %         ...user,
4378 %         role: (user.role ?? existing.role) as SystemUserRole,
4379 %     };
4380 %     this.systemUsers.set(id, updated);
4381 %     return updated;
4382 % }
4383 %
4384 % async deleteSystemUser(id: string): Promise<boolean> {
4385 %     return this.systemUsers.delete(id);
4386 % }
4387 %
4388 % // System Settings
4389 % async getSystemSettings(): Promise<SystemSetting[]> {
4390 %     return Array.from(this.systemSettings.values());
4391 % }
4392 %
4393 % async getSystemSetting(key: string): Promise<SystemSetting | undefined> {
4394 %     return Array.from(this.systemSettings.values()).find((s) => s.key === key);
4395 % }
4396 %
4397 % async getSystemSettingsByCategory(category: string): Promise<SystemSetting[]> {
4398 %     return Array.from(this.systemSettings.values()).filter((s) => s.category === category);
4399 % }
4400 %
4401 % async upsertSystemSetting(setting: InsertSystemSetting): Promise<SystemSetting> {
4402 %     const existing = Array.from(this.systemSettings.values()).find((s) => s.key === setting.key);
4403 %     if (existing) {
4404 %         const updated: SystemSetting = {
4405 %             ...existing,
4406 %             value: setting.value,
4407 %             description: setting.description ?? existing.description,
4408 %             updatedAt: setting.updatedAt,
4409 %             updatedBy: setting.updatedBy ?? existing.updatedBy,
4410 %         };
4411 %         this.systemSettings.set(existing.id, updated);
4412 %         return updated;
4413 %     }
4414 %     const id = randomUUID();
4415 %     const newSetting: SystemSetting = {
4416 %         id,
4417 %         key: setting.key,
4418 %         value: setting.value,
4419 %         category: setting.category ?? "general",
4420 %         description: setting.description ?? null,
4421 %         updatedAt: setting.updatedAt,
4422 %         updatedBy: setting.updatedBy ?? null,
4423 %     };
4424 %     this.systemSettings.set(id, newSetting);
4425 %     return newSetting;
4426 % }
4427 %
4428 % async deleteSystemSetting(key: string): Promise<boolean> {
4429 %     const setting = Array.from(this.systemSettings.values()).find((s) => s.key === key);
4430 %     if (!setting) return false;
4431 %     return this.systemSettings.delete(setting.id);
4432 % }
4433 %
4434 % // Audit Logs
4435 % async getAuditLogs(): Promise<AuditLog[]> {
4436 %     return Array.from(this.auditLogs.values()).sort((a, b) =>
4437 %         new Date(b.timestamp).getTime() - new Date(a.timestamp).getTime()
4438 %     );
4439 % }
4440 %
4441 % async getAuditLogsByModule(module: string): Promise<AuditLog[]> {
4442 %     return Array.from(this.auditLogs.values())
4443 %         .filter((l) => l.module === module)
4444 %         .sort((a, b) => new Date(b.timestamp).getTime() - new Date(a.timestamp).getTime());
4445 % }
4446 %
4447 % async getAuditLogsByUser(userId: string): Promise<AuditLog[]> {
4448 %     return Array.from(this.auditLogs.values())
4449 %         .filter((l) => l.userId === userId)
4450 %         .sort((a, b) => new Date(b.timestamp).getTime() - new Date(a.timestamp).getTime());
4451 % }
4452 %
4453 % async createAuditLog(log: InsertAuditLog): Promise<AuditLog> {
4454 %     const id = randomUUID();
4455 %     const newLog: AuditLog = {
4456 %         id,
4457 %         userId: log.userId ?? null,
4458 %         userName: log.userName ?? null,
4459 %         action: log.action as AuditAction,
4460 %         module: log.module,
4461 %         entityType: log.entityType ?? null,

```

server/storage.ts (cont.)

```

4462 %         entityId: log.entityId ?? null,
4463 %         description: log.description,
4464 %         details: log.details ?? null,
4465 %         ipAddress: log.ipAddress ?? null,
4466 %         timestamp: log.timestamp,
4467 %     };
4468 %     this.auditLogs.set(id, newLog);
4469 %     return newLog;
4470 % }
4471 %
4472 % async getAdminDashboardStats(): Promise<{
4473 %     totalUsers: number;
4474 %     activeUsers: number;
4475 %     recentLogins: number;
4476 %     totalSettings: number;
4477 %     recentAuditLogs: AuditLog[];
4478 % }> {
4479 %     const users = Array.from(this.systemUsers.values());
4480 %     const logs = Array.from(this.auditLogs.values());
4481 %     const today = new Date().toISOString().split("T")[0];
4482 %
4483 %     const recentLogins = users.filter((u) => {
4484 %         if (!u.lastLogin) return false;
4485 %         return u.lastLogin.startsWith(today);
4486 %     }).length;
4487 %
4488 %     return {
4489 %         totalUsers: users.length,
4490 %         activeUsers: users.filter((u) => u.isActive === "true").length,
4491 %         recentLogins,
4492 %         totalSettings: this.systemSettings.size,
4493 %         recentAuditLogs: logs
4494 %             .sort((a, b) => new Date(b.timestamp).getTime() - new Date(a.timestamp).getTime())
4495 %             .slice(0, 10),
4496 %     };
4497 % }
4498 %
4499 % // ===== PACKAGES =====
4500 % private enrichPackage(pkg: Package): PackageWithDetails {
4501 %     const roomType = pkg.roomTypeId ? this.roomTypes.get(pkg.roomTypeId) : undefined;
4502 %     const items = Array.from(this.packageItems.values()).filter(i => i.packageId === pkg.id);
4503 %     return { ...pkg, roomType, items };
4504 % }
4505 %
4506 % async getPackages(): Promise<PackageWithDetails[]> {
4507 %     return Array.from(this.packages.values()).map(p => this.enrichPackage(p));
4508 % }
4509 %
4510 % async getPackage(id: string): Promise<PackageWithDetails | undefined> {
4511 %     const pkg = this.packages.get(id);
4512 %     return pkg ? this.enrichPackage(pkg) : undefined;
4513 % }
4514 %
4515 % async getActivePackages(): Promise<PackageWithDetails[]> {
4516 %     const today = new Date().toISOString().split("T")[0];
4517 %     return Array.from(this.packages.values())
4518 %         .filter(p => {
4519 %             if (p.status !== "active") return false;
4520 %             if (p.validFrom && p.validFrom > today) return false;
4521 %             if (p.validUntil && p.validUntil < today) return false;
4522 %             return true;
4523 %         })
4524 %         .map(p => this.enrichPackage(p));
4525 % }
4526 %
4527 % async createPackage(pkg: InsertPackage): Promise<Package> {
4528 %     const id = randomUUID();
4529 %     const newPkg: Package = {
4530 %         id,
4531 %         code: pkg.code,
4532 %         name: pkg.name,
4533 %         description: pkg.description ?? null,
4534 %         roomTypeId: pkg.roomTypeId ?? null,
4535 %         nights: pkg.nights ?? 1,
4536 %         basePrice: pkg.basePrice,
4537 %         discountPercent: pkg.discountPercent ?? null,
4538 %         validFrom: pkg.validFrom ?? null,
4539 %         validUntil: pkg.validUntil ?? null,
4540 %         status: (pkg.status as PackageStatus) ?? "active",
4541 %         includedServices: pkg.includedServices ?? null,
4542 %         terms: pkg.terms ?? null,
4543 %         createdAt: pkg.createdAt,
4544 %     };
4545 %     this.packages.set(id, newPkg);
4546 %     return newPkg;
4547 % }
4548 %
4549 % async updatePackage(id: string, pkg: Partial<InsertPackage>): Promise<Package | undefined> {
4550 %     const existing = this.packages.get(id);
4551 %     if (!existing) return undefined;
4552 %     const updated: Package = { ...existing, ...pkg } as Package;
4553 %     this.packages.set(id, updated);
4554 %     return updated;

```

server/storage.ts (cont.)

```
4555 % }
4556 %
4557 % async deletePackage(id: string): Promise<boolean> {
4558 %     // Delete package items first
4559 %     const itemsToDelete = Array.from(this.packageItems.values()).filter(i => i.packageId === id);
4560 %     itemsToDelete.forEach(i => this.packageItems.delete(i.id));
4561 %     return this.packages.delete(id);
4562 % }
4563 %
4564 % generatePackageCode(): string {
4565 %     this.packageCounter++;
4566 %     return `PKG-${this.packageCounter.toString().padStart(4, "0")}`;
4567 % }
4568 %
4569 % async getPackageItems(packageId: string): Promise<PackageItem[]> {
4570 %     return Array.from(this.packageItems.values()).filter(i => i.packageId === packageId);
4571 % }
4572 %
4573 % async createPackageItem(item: InsertPackageItem): Promise<PackageItem> {
4574 %     const id = randomUUID();
4575 %     const newItem: PackageItem = {
4576 %         id,
4577 %         packageId: item.packageId,
4578 %         itemType: item.itemType as any,
4579 %         description: item.description,
4580 %         quantity: item.quantity ?? 1,
4581 %         unitValue: item.unitValue ?? null,
4582 %     };
4583 %     this.packageItems.set(id, newItem);
4584 %     return newItem;
4585 % }
4586 %
4587 % async updatePackageItem(id: string, item: Partial<InsertPackageItem>): Promise<PackageItem | undefined> {
4588 %     const existing = this.packageItems.get(id);
4589 %     if (!existing) return undefined;
4590 %     const updated: PackageItem = { ...existing, ...item } as PackageItem;
4591 %     this.packageItems.set(id, updated);
4592 %     return updated;
4593 % }
4594 %
4595 % async deletePackageItem(id: string): Promise<boolean> {
4596 %     return this.packageItems.delete(id);
4597 % }
4598 %
4599 % // System Notifications
4600 % async getNotifications(area?: NotificationArea, limit?: number): Promise<SystemNotification[]> {
4601 %     let notifications = Array.from(this.notificationsMap.values());
4602 %     if (area) {
4603 %         notifications = notifications.filter(n => n.targetArea === area || n.targetArea === "all");
4604 %     }
4605 %     notifications.sort((a, b) => {
4606 %         if (a.isRead !== b.isRead) return a.isRead ? 1 : -1;
4607 %         return new Date(b.createdAt!).getTime() - new Date(a.createdAt!).getTime();
4608 %     });
4609 %     if (limit) {
4610 %         notifications = notifications.slice(0, limit);
4611 %     }
4612 %     return notifications;
4613 % }
4614 %
4615 % async createNotification(notification: InsertSystemNotification): Promise<SystemNotification> {
4616 %     const id = ++this.notificationCounter;
4617 %     const newNotification: SystemNotification = {
4618 %         id,
4619 %         type: notification.type,
4620 %         title: notification.title,
4621 %         message: notification.message,
4622 %         targetArea: notification.targetArea,
4623 %         relatedEntityType: notification.relatedEntityType ?? null,
4624 %         relatedEntityId: notification.relatedEntityId ?? null,
4625 %         isRead: false,
4626 %         readAt: null,
4627 %         readBy: null,
4628 %         priority: notification.priority ?? "normal",
4629 %         createdAt: new Date(),
4630 %     };
4631 %     this.notificationsMap.set(id, newNotification);
4632 %     return newNotification;
4633 % }
4634 %
4635 % async markNotificationRead(id: number): Promise<SystemNotification | undefined> {
4636 %     const notification = this.notificationsMap.get(id);
4637 %     if (!notification) return undefined;
4638 %     const updated = { ...notification, isRead: true, readAt: new Date() };
4639 %     this.notificationsMap.set(id, updated);
4640 %     return updated;
4641 % }
4642 %
4643 % async markAllNotificationsRead(area?: NotificationArea): Promise<number> {
4644 %     let count = 0;
4645 %     for (const [id, notification] of this.notificationsMap) {
4646 %         if (!notification.isRead) {
4647 %             if (!area || notification.targetArea === area || notification.targetArea === "all") {
```

server/storage.ts (cont.)

```
4648 %         this.notificationsMap.set(id, { ...notification, isRead: true, readAt: new Date() });
4649 %         count++;
4650 %     }
4651 % }
4652 % }
4653 % return count;
4654 % }
4655 %
4656 % async getUnreadNotificationCount(area?: NotificationArea): Promise<number> {
4657 %     let count = 0;
4658 %     for (const notification of this.notificationsMap.values()) {
4659 %         if (!notification.isRead) {
4660 %             if (!area || notification.targetArea === area || notification.targetArea === "all") {
4661 %                 count++;
4662 %             }
4663 %         }
4664 %     }
4665 %     return count;
4666 % }
4667 %
4668 % // Web Check-in
4669 % async createWebCheckin(data: InsertWebCheckin): Promise<WebCheckin> {
4670 %     const id = ++this.webCheckinCounter;
4671 %     const newCheckin: WebCheckin = {
4672 %         id,
4673 %         reservationId: data.reservationId,
4674 %         token: data.token,
4675 %         status: data.status ?? "pending",
4676 %         confirmedFirstName: data.confirmedFirstName ?? null,
4677 %         confirmedLastName: data.confirmedLastName ?? null,
4678 %         confirmedDocumentType: data.confirmedDocumentType ?? null,
4679 %         confirmedDocumentNumber: data.confirmedDocumentNumber ?? null,
4680 %         confirmedNationality: data.confirmedNationality ?? null,
4681 %         confirmedPhone: data.confirmedPhone ?? null,
4682 %         confirmedEmail: data.confirmedEmail ?? null,
4683 %         documentPhotoUrl: data.documentPhotoUrl ?? null,
4684 %         estimatedArrivalTime: data.estimatedArrivalTime ?? null,
4685 %         requestEarlyCheckIn: data.requestEarlyCheckIn ?? false,
4686 %         earlyCheckInTime: data.earlyCheckInTime ?? null,
4687 %         termsAccepted: data.termsAccepted ?? false,
4688 %         termsAcceptedAt: data.termsAcceptedAt ?? null,
4689 %         ipAddress: data.ipAddress ?? null,
4690 %         completedAt: data.completedAt ?? null,
4691 %         expiresAt: data.expiresAt ?? null,
4692 %         createdAt: new Date(),
4693 %     };
4694 %     this.webCheckinsMap.set(id, newCheckin);
4695 %     return newCheckin;
4696 % }
4697 %
4698 % async getWebCheckinByToken(token: string): Promise<WebCheckin | undefined> {
4699 %     for (const checkin of this.webCheckinsMap.values()) {
4700 %         if (checkin.token === token) return checkin;
4701 %     }
4702 %     return undefined;
4703 % }
4704 %
4705 % async getWebCheckinByReservation(reservationId: string): Promise<WebCheckin | undefined> {
4706 %     for (const checkin of this.webCheckinsMap.values()) {
4707 %         if (checkin.reservationId === reservationId) return checkin;
4708 %     }
4709 %     return undefined;
4710 % }
4711 %
4712 % async updateWebCheckin(id: number, data: Partial<InsertWebCheckin>): Promise<WebCheckin | undefined> {
4713 %     const existing = this.webCheckinsMap.get(id);
4714 %     if (!existing) return undefined;
4715 %     const updated = { ...existing, ...data } as WebCheckin;
4716 %     this.webCheckinsMap.set(id, updated);
4717 %     return updated;
4718 % }
4719 %
4720 % async listWebCheckins(): Promise<WebCheckin[]> {
4721 %     return Array.from(this.webCheckinsMap.values()).sort(
4722 %         (a, b) => new Date(b.createdAt!).getTime() - new Date(a.createdAt!).getTime()
4723 %     );
4724 % }
4725 %
4726 % // ===== HOSPITALITY - GUEST PREFERENCES =====
4727 % async getGuestPreferences(guestId: string): Promise<GuestPreference[]> {
4728 %     return Array.from(this.guestPreferencesMap.values())
4729 %         .filter((p) => p.guestId === guestId)
4730 %         .sort((a, b) => new Date(b.createdAt!).getTime() - new Date(a.createdAt!).getTime());
4731 % }
4732 %
4733 % async getActiveGuestPreferences(guestId: string): Promise<GuestPreference[]> {
4734 %     return Array.from(this.guestPreferencesMap.values())
4735 %         .filter((p) => p.guestId === guestId && p.isActive)
4736 %         .sort((a, b) => {
4737 %             const priorityOrder = { critical: 0, high: 1, normal: 2, low: 3 };
4738 %             return (priorityOrder[a.priority as keyof typeof priorityOrder] || 2) - (priorityOrder[b.priority as keyof ty...
4739 %         });
4740 % }
```

server/storage.ts (cont.)

```
4741 %
4742 %   async getGuestPreference(id: number): Promise<GuestPreference | undefined> {
4743 %       return this.guestPreferencesMap.get(id);
4744 %   }
4745 %
4746 %   async createGuestPreference(pref: InsertGuestPreference): Promise<GuestPreference> {
4747 %       const id = ++this.guestPreferenceCounter;
4748 %       const now = new Date();
4749 %       const preference: GuestPreference = {
4750 %           id,
4751 %           guestId: pref.guestId,
4752 %           category: pref.category,
4753 %           subcategory: pref.subcategory ?? null,
4754 %           title: pref.title,
4755 %           description: pref.description ?? null,
4756 %           isActive: pref.isActive ?? true,
4757 %           priority: pref.priority ?? "normal",
4758 %           visibleTo: pref.visibleTo ?? ["all"],
4759 %           recordedBy: pref.recordedBy ?? null,
4760 %           sourceStay: pref.sourceStay ?? null,
4761 %           createdAt: now,
4762 %           updatedAt: now,
4763 %       };
4764 %       this.guestPreferencesMap.set(id, preference);
4765 %       return preference;
4766 %   }
4767 %
4768 %   async updateGuestPreference(id: number, pref: Partial<InsertGuestPreference>): Promise<GuestPreference | undefined> {
4769 %       const existing = this.guestPreferencesMap.get(id);
4770 %       if (!existing) return undefined;
4771 %       const updated = { ...existing, ...pref, updatedAt: new Date() } as GuestPreference;
4772 %       this.guestPreferencesMap.set(id, updated);
4773 %       return updated;
4774 %   }
4775 %
4776 %   async toggleGuestPreference(id: number): Promise<GuestPreference | undefined> {
4777 %       const existing = this.guestPreferencesMap.get(id);
4778 %       if (!existing) return undefined;
4779 %       const updated = { ...existing, isActive: !existing.isActive, updatedAt: new Date() };
4780 %       this.guestPreferencesMap.set(id, updated);
4781 %       return updated;
4782 %   }
4783 %
4784 %   async deleteGuestPreference(id: number): Promise<boolean> {
4785 %       return this.guestPreferencesMap.delete(id);
4786 %   }
4787 %
4788 %   // ===== HOSPITALITY - STAY NOTES =====
4789 %   async getStayNotes(reservationId: string): Promise<StayNote[]> {
4790 %       return Array.from(this.stayNotesMap.values())
4791 %           .filter((n) => n.reservationId === reservationId)
4792 %           .sort((a, b) => new Date(b.createdAt!).getTime() - new Date(a.createdAt!).getTime());
4793 %   }
4794 %
4795 %   async getActiveStayNotes(): Promise<StayNote[]> {
4796 %       return Array.from(this.stayNotesMap.values())
4797 %           .filter((n) => !n.isResolved)
4798 %           .sort((a, b) => new Date(b.createdAt!).getTime() - new Date(a.createdAt!).getTime());
4799 %   }
4800 %
4801 %   async createStayNote(note: InsertStayNote): Promise<StayNote> {
4802 %       const id = ++this.stayNoteCounter;
4803 %       const stayNote: StayNote = {
4804 %           id,
4805 %           reservationId: note.reservationId,
4806 %           guestId: note.guestId ?? null,
4807 %           category: note.category,
4808 %           title: note.title,
4809 %           description: note.description ?? null,
4810 %           priority: note.priority ?? "normal",
4811 %           visibleTo: note.visibleTo ?? ["all"],
4812 %           isResolved: false,
4813 %           resolvedAt: null,
4814 %           resolvedBy: null,
4815 %           recordedBy: note.recordedBy ?? null,
4816 %           createdAt: new Date(),
4817 %       };
4818 %       this.stayNotesMap.set(id, stayNote);
4819 %       return stayNote;
4820 %   }
4821 %
4822 %   async updateStayNote(id: number, note: Partial<InsertStayNote>): Promise<StayNote | undefined> {
4823 %       const existing = this.stayNotesMap.get(id);
4824 %       if (!existing) return undefined;
4825 %       const updated = { ...existing, ...note } as StayNote;
4826 %       this.stayNotesMap.set(id, updated);
4827 %       return updated;
4828 %   }
4829 %
4830 %   async resolveStayNote(id: number, resolvedBy: string): Promise<StayNote | undefined> {
4831 %       const existing = this.stayNotesMap.get(id);
4832 %       if (!existing) return undefined;
4833 %       const updated = { ...existing, isResolved: true, resolvedAt: new Date(), resolvedBy };

```

server/storage.ts (cont.)

```
4834 %     this.stayNotesMap.set(id, updated);
4835 %     return updated;
4836 % }
4837 %
4838 % async deleteStayNote(id: number): Promise<boolean> {
4839 %     return this.stayNotesMap.delete(id);
4840 % }
4841 %
4842 % // ===== HOSPITALITY - ALERTS =====
4843 % async getHospitalityAlerts(area?: string): Promise<HospitalityAlert[]> {
4844 %     let alerts = Array.from(this.hospitalityAlertsMap.values());
4845 %     if (area && area !== "all") {
4846 %         alerts = alerts.filter((a) => a.targetArea === area || a.targetArea === "all");
4847 %     }
4848 %     return alerts.sort((a, b) => new Date(b.createdAt!).getTime() - new Date(a.createdAt!).getTime());
4849 % }
4850 %
4851 % async getHospitalityAlertsByReservation(reservationId: string): Promise<HospitalityAlert[]> {
4852 %     return Array.from(this.hospitalityAlertsMap.values())
4853 %         .filter((a) => a.reservationId === reservationId)
4854 %         .sort((a, b) => new Date(b.createdAt!).getTime() - new Date(a.createdAt!).getTime());
4855 % }
4856 %
4857 % async createHospitalityAlert(alert: InsertHospitalityAlert): Promise<HospitalityAlert> {
4858 %     const id = ++this.hospitalityAlertCounter;
4859 %     const hospitalityAlert: HospitalityAlert = {
4860 %         id,
4861 %         reservationId: alert.reservationId,
4862 %         guestId: alert.guestId,
4863 %         preferenceId: alert.preferenceId ?? null,
4864 %         alertMessage: alert.alertMessage,
4865 %         targetArea: alert.targetArea,
4866 %         priority: alert.priority ?? "normal",
4867 %         isAcknowledged: false,
4868 %         acknowledgedAt: null,
4869 %         acknowledgedBy: null,
4870 %         createdAt: new Date(),
4871 %     };
4872 %     this.hospitalityAlertsMap.set(id, hospitalityAlert);
4873 %     return hospitalityAlert;
4874 % }
4875 %
4876 % async acknowledgeHospitalityAlert(id: number, acknowledgedBy: string): Promise<HospitalityAlert | undefined> {
4877 %     const existing = this.hospitalityAlertsMap.get(id);
4878 %     if (!existing) return undefined;
4879 %     const updated = { ...existing, isAcknowledged: true, acknowledgedAt: new Date(), acknowledgedBy };
4880 %     this.hospitalityAlertsMap.set(id, updated);
4881 %     return updated;
4882 % }
4883 % }
4884 %
4885 % export const storage = new MemStorage();
4886 %
```

```

1 % import { type Express } from "express";
2 % import { createServer as createViteServer, createLogger } from "vite";
3 % import { type Server } from "http";
4 % import viteConfig from "../vite.config";
5 % import fs from "fs";
6 % import path from "path";
7 % import { nanoid } from "nanoid";
8 %
9 % const viteLogger = createLogger();
10 %
11 % export async function setupVite(server: Server, app: Express) {
12 %   const serverOptions = {
13 %     middlewareMode: true,
14 %     hmr: { server, path: "/vite-hmr" },
15 %     allowedHosts: true as const,
16 %   };
17 %
18 %   const vite = await createViteServer({
19 %     ...viteConfig,
20 %     configFile: false,
21 %     customLogger: {
22 %       ...viteLogger,
23 %       error: (msg, options) => {
24 %         viteLogger.error(msg, options);
25 %         process.exit(1);
26 %       },
27 %     },
28 %     server: serverOptions,
29 %     appType: "custom",
30 %   });
31 %
32 %   app.use(vite.middlewares);
33 %
34 %   app.use("{}", async (req, res, next) => {
35 %     const url = req.originalUrl;
36 %
37 %     try {
38 %       const clientTemplate = path.resolve(
39 %         import.meta.dirname,
40 %         "..",
41 %         "client",
42 %         "index.html",
43 %       );
44 %
45 %       // always reload the index.html file from disk incase it changes
46 %       let template = await fs.promises.readFile(clientTemplate, "utf-8");
47 %       template = template.replace(
48 %         `src="/src/main.tsx"`,
49 %         `src="/src/main.tsx?v=${nanoid()}"`,
50 %       );
51 %       const page = await vite.transformIndexHtml(url, template);
52 %       res.status(200).set({ "Content-Type": "text/html" }).end(page);
53 %     } catch (e) {
54 %       vite.ssrFixStacktrace(e as Error);
55 %       next(e);
56 %     }
57 %   });
58 % }
59 %

```

```
1 % import express, { type Express } from "express";
2 % import fs from "fs";
3 % import path from "path";
4 %
5 % export function serveStatic(app: Express) {
6 %   const distPath = path.resolve(__dirname, "public");
7 %   if (!fs.existsSync(distPath)) {
8 %     throw new Error(
9 %       `Could not find the build directory: ${distPath}, make sure to build the client first`,
10 %    );
11 %   }
12 %
13 %   app.use(express.static(distPath, {
14 %     maxAge: '1h',
15 %     setHeaders: (res, filePath) => {
16 %       if (filePath.endsWith('.html')) {
17 %         res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
18 %       }
19 %     }
20 %   }));
21 %
22 %   app.use("/*", (_req, res) => {
23 %     res.setHeader('Cache-Control', 'no-cache, no-store, must-revalidate');
24 %     res.sendFile(path.resolve(distPath, "index.html"));
25 %   });
26 % }
27 %
```



```
1 % import { build as esbuild } from "esbuild";
2 % import { build as viteBuild } from "vite";
3 % import { rm, readFile } from "fs/promises";
4 %
5 % // server deps to bundle to reduce openat(2) syscalls
6 % // which helps cold start times
7 % const allowlist = [
8 %   "@google/generative-ai",
9 %   "axios",
10 %   "connect-pg-simple",
11 %   "cors",
12 %   "date-fns",
13 %   "drizzle-orm",
14 %   "drizzle-zod",
15 %   "express",
16 %   "express-rate-limit",
17 %   "express-session",
18 %   "jsonwebtoken",
19 %   "memorystore",
20 %   "multer",
21 %   "nanoid",
22 %   "nodemailer",
23 %   "openai",
24 %   "passport",
25 %   "passport-local",
26 %   "pg",
27 %   "stripe",
28 %   "uuid",
29 %   "ws",
30 %   "xlsx",
31 %   "zod",
32 %   "zod-validation-error",
33 % ];
34 %
35 % async function buildAll() {
36 %   await rm("dist", { recursive: true, force: true });
37 %
38 %   console.log("building client...");
39 %   await viteBuild();
40 %
41 %   console.log("building server...");
42 %   const pkg = JSON.parse(await readFile("package.json", "utf-8"));
43 %   const allDeps = [
44 %     ...Object.keys(pkg.dependencies || {}),
45 %     ...Object.keys(pkg.devDependencies || {}),
46 %   ];
47 %   const externals = allDeps.filter((dep) => !allowlist.includes(dep));
48 %
49 %   await esbuild({
50 %     entryPoints: ["server/index.ts"],
51 %     platform: "node",
52 %     bundle: true,
53 %     format: "cjs",
54 %     outfile: "dist/index.cjs",
55 %     define: {
56 %       "process.env.NODE_ENV": '"production"',
57 %     },
58 %     minify: true,
59 %     external: externals,
60 %     logLevel: "info",
61 %   });
62 % }
63 %
64 % buildAll().catch((err) => {
65 %   console.error(err);
66 %   process.exit(1);
67 % });
68 %
```

```
1 % <!DOCTYPE html>
2 % <html lang="es">
3 %   <head>
4 %     <meta charset="UTF-8" />
5 %     <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1" />
6 %     <meta name="description" content="HotelPro - Sistema de gestión hotelera. Administra reservas, habitaciones, hués...
7 %     <title>HotelPro - Sistema de Gestión Hotelera</title>
8 %     <link rel="icon" type="image/png" href="/favicon.png" />
9 %     <link rel="preconnect" href="https://fonts.googleapis.com">
10 %    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
11 %    <link href="https://fonts.googleapis.com/css2?family=Architects+Daughter&family=DM+Sans:ital,opsz,wght@0,9..40,10...
12 %  </head>
13 %  <body>
14 %    <div id="root"></div>
15 %    <script type="module" src="/src/main.tsx"></script>
16 %  </body>
17 % </html>
```

```
1 % import { createRoot } from "react-dom/client";
2 % import App from "../App";
3 % import "../index.css";
4 %
5 % createRoot(document.getElementById("root")!).render(<App />);
6 %
```

```
1 % import { Switch, Route, useRoute } from "wouter";
2 % import { queryClient } from "../lib/queryClient";
3 % import { QueryClientProvider } from "@tanstack/react-query";
4 % import { Toaster } from "@components/ui/toaster";
5 % import { TooltipProvider } from "@components/ui/tooltip";
6 % import { ThemeProvider } from "@components/theme-provider";
7 % import { ThemeToggle } from "@components/theme-toggle";
8 % import { SidebarProvider, SidebarTrigger } from "@components/ui/sidebar";
9 % import { AppSidebar } from "@components/app-sidebar";
10 % import NotFound from "@pages/not-found";
11 % import Dashboard from "@pages/dashboard";
12 % import PlanningPage from "@pages/planning";
13 % import RoomsPage from "@pages/rooms";
14 % import ReservationsPage from "@pages/reservations";
15 % import GuestsPage from "@pages/guests";
16 % import CheckInPage from "@pages/check-in";
17 % import CheckOutPage from "@pages/check-out";
18 % import RatePlansPage from "@pages/rate-plans";
19 % import OTACHannelsPage from "@pages/ota-channels";
20 % import NewReservationPage from "@pages/new-reservation";
21 % import GroupsPage from "@pages/groups";
22 % import GroupDetailPage from "@pages/group-detail";
23 % import ReviewsPage from "@pages/reviews";
24 % import HousekeepingPage from "@pages/housekeeping";
25 % import RestaurantPage from "@pages/restaurant";
26 % import InventoryPage from "@pages/inventory";
27 % import SpaPage from "@pages/spa";
28 % import EventsPage from "@pages/events";
29 % import MaintenancePage from "@pages/maintenance";
30 % import AdministrationPage from "@pages/administration";
31 % import PackagesPage from "@pages/packages";
32 % import CompaniesPage from "@pages/companies";
33 % import WebCheckinPublicPage from "@pages/web-checkin-public";
34 % import ChatbotDashboardPage from "@pages/chatbot-dashboard";
35 % import HospitalityPage from "@pages/hospitality";
36 %
37 % function Router() {
38 %   return (
39 %     <Switch>
40 %       <Route path="/" component={Dashboard} />
41 %       <Route path="/planning" component={PlanningPage} />
42 %       <Route path="/rooms" component={RoomsPage} />
43 %       <Route path="/reservations" component={ReservationsPage} />
44 %       <Route path="/new-reservation" component={NewReservationPage} />
45 %       <Route path="/guests" component={GuestsPage} />
46 %       <Route path="/groups" component={GroupsPage} />
47 %       <Route path="/groups/:id" component={GroupDetailPage} />
48 %       <Route path="/check-in" component={CheckInPage} />
49 %       <Route path="/check-out" component={CheckOutPage} />
50 %       <Route path="/rate-plans" component={RatePlansPage} />
51 %       <Route path="/ota-channels" component={OTACHannelsPage} />
52 %       <Route path="/reviews" component={ReviewsPage} />
53 %       <Route path="/housekeeping" component={HousekeepingPage} />
54 %       <Route path="/restaurant" component={RestaurantPage} />
55 %       <Route path="/inventory" component={InventoryPage} />
56 %       <Route path="/spa" component={SpaPage} />
57 %       <Route path="/events" component={EventsPage} />
58 %       <Route path="/maintenance" component={MaintenancePage} />
59 %       <Route path="/administration" component={AdministrationPage} />
60 %       <Route path="/packages" component={PackagesPage} />
61 %       <Route path="/companies" component={CompaniesPage} />
62 %       <Route path="/chatbot" component={ChatbotDashboardPage} />
63 %       <Route path="/hospitality" component={HospitalityPage} />
64 %       <Route component={NotFound} />
65 %     </Switch>
66 %   );
67 % }
68 %
69 % function AppLayout() {
70 %   const sidebarStyle = {
71 %     "--sidebar-width": "16rem",
72 %     "--sidebar-width-icon": "3rem",
73 %   };
74 %
75 %   return (
76 %     <SidebarProvider style={sidebarStyle as React.CSSProperties}>
77 %       <div className="flex h-screen w-full">
78 %         <AppSidebar />
79 %         <div className="flex flex-col flex-1 overflow-hidden">
80 %           <header className="flex items-center justify-between gap-4 px-4 py-2 border-b bg-background/95 backdrop-blur...">
81 %             <SidebarTrigger data-testid="button-sidebar-toggle" />
82 %             <ThemeToggle />
83 %           </header>
84 %           <main className="flex-1 overflow-auto">
85 %             <Router />
86 %           </main>
87 %         </div>
88 %       </div>
89 %     </SidebarProvider>
90 %   );
91 % }
```

client/src/App.tsx (cont.)

```
91 % }
92 %
93 % function App() {
94 %   const [isWebCheckin] = useRoute("/web-checkin/:token");
95 %
96 %   return (
97 %     <QueryClientProvider client={queryClient}>
98 %       <ThemeProvider defaultTheme="light" storageKey="hotel-ui-theme">
99 %         <TooltipProvider>
100 %           {isWebCheckin ? (
101 %             <WebCheckinPublicPage />
102 %           ) : (
103 %             <AppLayout />
104 %           )}
105 %         <Toaster />
106 %       </TooltipProvider>
107 %     </ThemeProvider>
108 %   </QueryClientProvider>
109 % );
110 % }
111 %
112 % export default App;
113 %
```

```
1 % @tailwind base;
2 % @tailwind components;
3 % @tailwind utilities;
4 %
5 % /* LIGHT MODE */
6 % :root {
7 %   --button-outline: rgba(0,0,0, .10);
8 %   --badge-outline: rgba(0,0,0, .05);
9 %   --opaque-button-border-intensity: -8;
10 %   --elevate-1: rgba(0,0,0, .03);
11 %   --elevate-2: rgba(0,0,0, .08);
12 %   --background: 210 5% 98%;
13 %   --foreground: 210 6% 12%;
14 %   --border: 210 5% 90%;
15 %   --card: 210 5% 96%;
16 %   --card-foreground: 210 6% 14%;
17 %   --card-border: 210 5% 88%;
18 %   --sidebar: 210 5% 94%;
19 %   --sidebar-foreground: 210 6% 16%;
20 %   --sidebar-border: 210 5% 86%;
21 %   --sidebar-primary: 210 85% 45%;
22 %   --sidebar-primary-foreground: 210 85% 98%;
23 %   --sidebar-accent: 210 12% 88%;
24 %   --sidebar-accent-foreground: 210 6% 18%;
25 %   --sidebar-ring: 210 85% 45%;
26 %   --popover: 210 5% 92%;
27 %   --popover-foreground: 210 6% 18%;
28 %   --popover-border: 210 5% 84%;
29 %   --primary: 210 85% 32%;
30 %   --primary-foreground: 210 85% 98%;
31 %   --secondary: 210 8% 86%;
32 %   --secondary-foreground: 210 6% 20%;
33 %   --muted: 210 10% 90%;
34 %   --muted-foreground: 210 6% 35%;
35 %   --accent: 210 12% 92%;
36 %   --accent-foreground: 210 6% 22%;
37 %   --destructive: 0 72% 35%;
38 %   --destructive-foreground: 0 72% 98%;
39 %   --input: 210 5% 75%;
40 %   --ring: 210 85% 45%;
41 %   --chart-1: 210 75% 28%;
42 %   --chart-2: 195 68% 32%;
43 %   --chart-3: 175 55% 35%;
44 %   --chart-4: 160 48% 38%;
45 %   --chart-5: 145 42% 40%;
46 %   --font-sans: Inter, system-ui, sans-serif;
47 %   --font-serif: Georgia, serif;
48 %   --font-mono: 'JetBrains Mono', monospace;
49 %   --radius: .5rem;
50 %   --shadow-2xs: 0px 2px 0px 0px hsl(210 5% 20% / 0.00);
51 %   --shadow-xs: 0px 2px 0px 0px hsl(210 5% 20% / 0.00);
52 %   --shadow-sm: 0px 2px 0px 0px hsl(210 5% 20% / 0.00), 0px 1px 2px -1px hsl(210 5% 20% / 0.00);
53 %   --shadow: 0px 2px 0px 0px hsl(210 5% 20% / 0.00), 0px 1px 2px -1px hsl(210 5% 20% / 0.00);
54 %   --shadow-md: 0px 2px 0px 0px hsl(210 5% 20% / 0.00), 0px 2px 4px -1px hsl(210 5% 20% / 0.00);
55 %   --shadow-lg: 0px 2px 0px 0px hsl(210 5% 20% / 0.00), 0px 4px 6px -1px hsl(210 5% 20% / 0.00);
56 %   --shadow-xl: 0px 2px 0px 0px hsl(210 5% 20% / 0.00), 0px 8px 10px -1px hsl(210 5% 20% / 0.00);
57 %   --shadow-2xl: 0px 2px 0px 0px hsl(210 5% 20% / 0.00);
58 %   --tracking-normal: 0em;
59 %   --spacing: 0.25rem;
60 %
61 % /* Fallback for older browsers */
62 %   --sidebar-primary-border: hsl(var(--sidebar-primary));
63 %   --sidebar-primary-border: hsl(from hsl(var(--sidebar-primary)) h s calc(1 + var(--opaque-button-border-intensity)) ...
64 %
65 % /* Fallback for older browsers */
66 %   --sidebar-accent-border: hsl(var(--sidebar-accent));
67 %   --sidebar-accent-border: hsl(from hsl(var(--sidebar-accent)) h s calc(1 + var(--opaque-button-border-intensity)) / ...
68 %
69 % /* Fallback for older browsers */
70 %   --primary-border: hsl(var(--primary));
71 %   --primary-border: hsl(from hsl(var(--primary)) h s calc(1 + var(--opaque-button-border-intensity)) / alpha);
72 %
73 % /* Fallback for older browsers */
74 %   --secondary-border: hsl(var(--secondary));
75 %   --secondary-border: hsl(from hsl(var(--secondary)) h s calc(1 + var(--opaque-button-border-intensity)) / alpha);
76 %
77 % /* Fallback for older browsers */
78 %   --muted-border: hsl(var(--muted));
79 %   --muted-border: hsl(from hsl(var(--muted)) h s calc(1 + var(--opaque-button-border-intensity)) / alpha);
80 %
81 % /* Fallback for older browsers */
82 %   --accent-border: hsl(var(--accent));
83 %   --accent-border: hsl(from hsl(var(--accent)) h s calc(1 + var(--opaque-button-border-intensity)) / alpha);
84 %
85 % /* Fallback for older browsers */
86 %   --destructive-border: hsl(var(--destructive));
87 %   --destructive-border: hsl(from hsl(var(--destructive)) h s calc(1 + var(--opaque-button-border-intensity)) / alpha);
88 % }
89 %
90 % .dark {
```

client/src/index.css (cont.)

```
91 % --button-outline: rgba(255,255,255, .10);
92 % --badge-outline: rgba(255,255,255, .05);
93 % --opaque-button-border-intensity: 9;
94 % --elevate-1: rgba(255,255,255, .04);
95 % --elevate-2: rgba(255,255,255, .09);
96 % --background: 210 5% 8%;
97 % --foreground: 210 5% 92%;
98 % --border: 210 5% 18%;
99 % --card: 210 5% 10%;
100 % --card-foreground: 210 5% 90%;
101 % --card-border: 210 5% 14%;
102 % --sidebar: 210 5% 12%;
103 % --sidebar-foreground: 210 5% 88%;
104 % --sidebar-border: 210 5% 16%;
105 % --sidebar-primary: 210 85% 45%;
106 % --sidebar-primary-foreground: 210 85% 98%;
107 % --sidebar-accent: 210 12% 18%;
108 % --sidebar-accent-foreground: 210 5% 88%;
109 % --sidebar-ring: 210 85% 55%;
110 % --popover: 210 5% 14%;
111 % --popover-foreground: 210 5% 86%;
112 % --popover-border: 210 5% 20%;
113 % --primary: 210 85% 32%;
114 % --primary-foreground: 210 85% 98%;
115 % --secondary: 210 8% 20%;
116 % --secondary-foreground: 210 5% 84%;
117 % --muted: 210 10% 16%;
118 % --muted-foreground: 210 5% 65%;
119 % --accent: 210 12% 14%;
120 % --accent-foreground: 210 5% 82%;
121 % --destructive: 0 72% 35%;
122 % --destructive-foreground: 0 72% 98%;
123 % --input: 210 5% 30%;
124 % --ring: 210 85% 55%;
125 % --chart-1: 210 75% 65%;
126 % --chart-2: 195 68% 62%;
127 % --chart-3: 175 55% 58%;
128 % --chart-4: 160 48% 55%;
129 % --chart-5: 145 42% 52%;
130 % --shadow-2xs: 0px 2px 0px 0px hsl(210 5% 5% / 0.00);
131 % --shadow-xs: 0px 2px 0px 0px hsl(210 5% 5% / 0.00);
132 % --shadow-sm: 0px 2px 0px 0px hsl(210 5% 5% / 0.00), 0px 1px 2px -1px hsl(210 5% 5% / 0.00);
133 % --shadow: 0px 2px 0px 0px hsl(210 5% 5% / 0.00), 0px 1px 2px -1px hsl(210 5% 5% / 0.00);
134 % --shadow-md: 0px 2px 0px 0px hsl(210 5% 5% / 0.00), 0px 2px 4px -1px hsl(210 5% 5% / 0.00);
135 % --shadow-lg: 0px 2px 0px 0px hsl(210 5% 5% / 0.00), 0px 4px 6px -1px hsl(210 5% 5% / 0.00);
136 % --shadow-xl: 0px 2px 0px 0px hsl(210 5% 5% / 0.00), 0px 8px 10px -1px hsl(210 5% 5% / 0.00);
137 % --shadow-2xl: 0px 2px 0px 0px hsl(210 5% 5% / 0.00);
138 %
139 % /* Fallback for older browsers */
140 % --sidebar-primary-border: hsl(var(--sidebar-primary));
141 % --sidebar-primary-border: hsl(from hsl(var(--sidebar-primary)) h s calc(l + var(--opaque-button-border-intensity)) ...
142 %
143 % /* Fallback for older browsers */
144 % --sidebar-accent-border: hsl(var(--sidebar-accent));
145 % --sidebar-accent-border: hsl(from hsl(var(--sidebar-accent)) h s calc(l + var(--opaque-button-border-intensity)) / ...
146 %
147 % /* Fallback for older browsers */
148 % --primary-border: hsl(var(--primary));
149 % --primary-border: hsl(from hsl(var(--primary)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);
150 %
151 % /* Fallback for older browsers */
152 % --secondary-border: hsl(var(--secondary));
153 % --secondary-border: hsl(from hsl(var(--secondary)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);
154 %
155 % /* Fallback for older browsers */
156 % --muted-border: hsl(var(--muted));
157 % --muted-border: hsl(from hsl(var(--muted)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);
158 %
159 % /* Fallback for older browsers */
160 % --accent-border: hsl(var(--accent));
161 % --accent-border: hsl(from hsl(var(--accent)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);
162 %
163 % /* Fallback for older browsers */
164 % --destructive-border: hsl(var(--destructive));
165 % --destructive-border: hsl(from hsl(var(--destructive)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);
166 % }
167 %
168 % @layer base {
169 %   * {
170 %     @apply border-border;
171 %   }
172 %
173 %   body {
174 %     @apply font-sans antialiased bg-background text-foreground;
175 %   }
176 % }
177 %
178 % /**
179 %  * Using the elevate system.
180 %  * Automatic contrast adjustment.
181 %  *
182 %  * <element className="hover-elevate" />
183 %  * <element className="active-elevate-2" />
```

client/src/index.css (cont.)

```
184 % *
185 % * // Using the tailwind utility when a data attribute is "on"
186 % * <element className="toggle-elevate data-[state=on]:toggle-elevated" />
187 % * // Or manually controlling the toggle state
188 % * <element className="toggle-elevate toggle-elevated" />
189 % *
190 % * Elevation systems have to handle many states.
191 % * - not-hovered, vs. hovered vs. active (three mutually exclusive states)
192 % * - toggled or not
193 % * - focused or not (this is not handled with these utilities)
194 % *
195 % * Even without handling focused or not, this is six possible combinations that
196 % * need to be distinguished from eachother visually.
197 % */
198 % @layer utilities {
199 %
200 % /* Hide ugly search cancel button in Chrome until we can style it properly */
201 % input[type="search"]::-webkit-search-cancel-button {
202 %   @apply hidden;
203 % }
204 %
205 % /* Placeholder styling for contentEditable div */
206 % [contenteditable][data-placeholder]:empty::before {
207 %   content: attr(data-placeholder);
208 %   color: hsl(var(--muted-foreground));
209 %   pointer-events: none;
210 % }
211 %
212 % /* .no-default-hover-elevate/no-default-active-elevate is an escape hatch so consumers of
213 % * buttons/badges can remove the automatic brightness adjustment on interactions
214 % * and program their own. */
215 % .no-default-hover-elevate {}
216 %
217 % .no-default-active-elevate {}
218 %
219 %
220 % /**
221 %  * Toggleable backgrounds go behind the content. Hoverable/active goes on top.
222 %  * This way they can stack/compound. Both will overlap the parent's borders!
223 %  * So borders will be automatically adjusted both on toggle, and hover/active,
224 %  * and they will be compounded.
225 %  */
226 % .toggle-elevate::before,
227 % .toggle-elevate-2::before {
228 %   content: "";
229 %   pointer-events: none;
230 %   position: absolute;
231 %   inset: 0px;
232 %   /*border-radius: inherit; match rounded corners */
233 %   border-radius: inherit;
234 %   z-index: -1;
235 %   /* sits behind content but above backdrop */
236 % }
237 %
238 % .toggle-elevate.toggle-elevated::before {
239 %   background-color: var(--elevate-2);
240 % }
241 %
242 % /* If there's a 1px border, adjust the inset so that it covers that parent's border */
243 % .border.toggle-elevate::before {
244 %   inset: -1px;
245 % }
246 %
247 % /* Does not work on elements with overflow:hidden! */
248 % .hover-elevate:not(.no-default-hover-elevate),
249 % .active-elevate:not(.no-default-active-elevate),
250 % .hover-elevate-2:not(.no-default-hover-elevate),
251 % .active-elevate-2:not(.no-default-active-elevate) {
252 %   position: relative;
253 %   z-index: 0;
254 % }
255 %
256 % .hover-elevate:not(.no-default-hover-elevate)::after,
257 % .active-elevate:not(.no-default-active-elevate)::after,
258 % .hover-elevate-2:not(.no-default-hover-elevate)::after,
259 % .active-elevate-2:not(.no-default-active-elevate)::after {
260 %   content: "";
261 %   pointer-events: none;
262 %   position: absolute;
263 %   inset: 0px;
264 %   /*border-radius: inherit; match rounded corners */
265 %   border-radius: inherit;
266 %   z-index: 999;
267 %   /* sits in front of content */
268 % }
269 %
270 % .hover-elevate:hover:not(.no-default-hover-elevate)::after,
271 % .active-elevate:active:not(.no-default-active-elevate)::after {
272 %   background-color: var(--elevate-1);
273 % }
274 %
275 % .hover-elevate-2:hover:not(.no-default-hover-elevate)::after,
276 % .active-elevate-2:active:not(.no-default-active-elevate)::after {
```


client/src/index.css (cont.)

```
277 %     background-color: var(--elevate-2);
278 % }
279 %
280 % /* If there's a 1px border, adjust the inset so that it covers that parent's border */
281 % .border.hover-elevate:not(.no-hover-interaction-elevate)::after,
282 % .border.active-elevate:not(.no-active-interaction-elevate)::after,
283 % .border.hover-elevate-2:not(.no-hover-interaction-elevate)::after,
284 % .border.active-elevate-2:not(.no-active-interaction-elevate)::after,
285 % .border.hover-elevate:not(.no-hover-interaction-elevate)::after {
286 %     inset: -1px;
287 % }
288 % }
289 %
```

```

1 % import { QueryClient, QueryFunction } from "@tanstack/react-query";
2 %
3 % async function throwIfResNotOk(res: Response) {
4 %   if (!res.ok) {
5 %     const text = (await res.text()) || res.statusText;
6 %     throw new Error(`${res.status}: ${text}`);
7 %   }
8 % }
9 %
10 % export async function apiRequest(
11 %   method: string,
12 %   url: string,
13 %   data?: unknown | undefined,
14 % ): Promise<Response> {
15 %   const res = await fetch(url, {
16 %     method,
17 %     headers: data ? { "Content-Type": "application/json" } : {},
18 %     body: data ? JSON.stringify(data) : undefined,
19 %     credentials: "include",
20 %   });
21 %
22 %   await throwIfResNotOk(res);
23 %   return res;
24 % }
25 %
26 % type UnauthorizedBehavior = "returnNull" | "throw";
27 % export const getQueryFn: <T>(options: {
28 %   on401: UnauthorizedBehavior;
29 % }) => QueryFunction<T> =
30 %   ({ on401: unauthorizedBehavior }) =>
31 %     async ({ queryKey }) => {
32 %       const res = await fetch(queryKey.join("/") as string, {
33 %         credentials: "include",
34 %       });
35 %
36 %       if (unauthorizedBehavior === "returnNull" && res.status === 401) {
37 %         return null;
38 %       }
39 %
40 %       await throwIfResNotOk(res);
41 %       return await res.json();
42 %     };
43 %
44 % export const queryClient = new QueryClient({
45 %   defaultOptions: {
46 %     queries: {
47 %       queryFn: getQueryFn({ on401: "throw" }),
48 %       refetchInterval: false,
49 %       refetchOnWindowFocus: false,
50 %       staleTime: Infinity,
51 %       retry: false,
52 %     },
53 %     mutations: {
54 %       retry: false,
55 %     },
56 %   },
57 % });
58 %

```

```
1 % import { clsx, type ClassValue } from "clsx"
2 % import { twMerge } from "tailwind-merge"
3 %
4 % export function cn(...inputs: ClassValue[]) {
5 %   return twMerge(clsx(inputs))
6 % }
7 %
```

```

1 % import * as React from "react"
2 %
3 % import type {
4 %   ToastActionElement,
5 %   ToastProps,
6 % } from "@components/ui/toast"
7 %
8 % const TOAST_LIMIT = 1
9 % const TOAST_REMOVE_DELAY = 1000000
10 %
11 % type ToasterToast = ToastProps & {
12 %   id: string
13 %   title?: React.ReactNode
14 %   description?: React.ReactNode
15 %   action?: ToastActionElement
16 % }
17 %
18 % const actionTypes = {
19 %   ADD_TOAST: "ADD_TOAST",
20 %   UPDATE_TOAST: "UPDATE_TOAST",
21 %   DISMISS_TOAST: "DISMISS_TOAST",
22 %   REMOVE_TOAST: "REMOVE_TOAST",
23 % } as const
24 %
25 % let count = 0
26 %
27 % function genId() {
28 %   count = (count + 1) % Number.MAX_SAFE_INTEGER
29 %   return count.toString()
30 % }
31 %
32 % type ActionType = typeof actionTypes
33 %
34 % type Action =
35 %   | {
36 %     type: ActionType["ADD_TOAST"]
37 %     toast: ToasterToast
38 %   }
39 %   | {
40 %     type: ActionType["UPDATE_TOAST"]
41 %     toast: Partial<ToasterToast>
42 %   }
43 %   | {
44 %     type: ActionType["DISMISS_TOAST"]
45 %     toastId?: ToasterToast["id"]
46 %   }
47 %   | {
48 %     type: ActionType["REMOVE_TOAST"]
49 %     toastId?: ToasterToast["id"]
50 %   }
51 %
52 % interface State {
53 %   toasts: ToasterToast[]
54 % }
55 %
56 % const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>()
57 %
58 % const addToRemoveQueue = (toastId: string) => {
59 %   if (toastTimeouts.has(toastId)) {
60 %     return
61 %   }
62 %
63 %   const timeout = setTimeout(() => {
64 %     toastTimeouts.delete(toastId)
65 %     dispatch({
66 %       type: "REMOVE_TOAST",
67 %       toastId: toastId,
68 %     })
69 %   }, TOAST_REMOVE_DELAY)
70 %
71 %   toastTimeouts.set(toastId, timeout)
72 % }
73 %
74 % export const reducer = (state: State, action: Action): State => {
75 %   switch (action.type) {
76 %     case "ADD_TOAST":
77 %       return {
78 %         ...state,
79 %         toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
80 %       }
81 %
82 %     case "UPDATE_TOAST":
83 %       return {
84 %         ...state,
85 %         toasts: state.toasts.map((t) =>
86 %           t.id === action.toast.id ? { ...t, ...action.toast } : t
87 %         ),
88 %       }
89 %
90 %     case "DISMISS_TOAST": {

```

client/src/hooks/use-toast.ts (cont.)

```
91 %     const { toastId } = action
92 %
93 %     // ! Side effects ! - This could be extracted into a dismissToast() action,
94 %     // but I'll keep it here for simplicity
95 %     if (toastId) {
96 %       addToRemoveQueue(toastId)
97 %     } else {
98 %       state.toasts.forEach((toast) => {
99 %         addToRemoveQueue(toast.id)
100 %       })
101 %     }
102 %
103 %     return {
104 %       ...state,
105 %       toasts: state.toasts.map((t) =>
106 %         t.id === toastId || toastId === undefined
107 %           ? {
108 %             ...t,
109 %             open: false,
110 %           }
111 %           : t
112 %       ),
113 %     }
114 %   }
115 %   case "REMOVE_TOAST":
116 %     if (action.toastId === undefined) {
117 %       return {
118 %         ...state,
119 %         toasts: [],
120 %       }
121 %     }
122 %     return {
123 %       ...state,
124 %       toasts: state.toasts.filter((t) => t.id !== action.toastId),
125 %     }
126 %   }
127 % }
128 %
129 % const listeners: Array<(state: State) => void> = []
130 %
131 % let memoryState: State = { toasts: [] }
132 %
133 % function dispatch(action: Action) {
134 %   memoryState = reducer(memoryState, action)
135 %   listeners.forEach((listener) => {
136 %     listener(memoryState)
137 %   })
138 % }
139 %
140 % type Toast = Omit<ToasterToast, "id">
141 %
142 % function toast({ ...props }: Toast) {
143 %   const id = genId()
144 %
145 %   const update = (props: ToasterToast) =>
146 %     dispatch({
147 %       type: "UPDATE_TOAST",
148 %       toast: { ...props, id },
149 %     })
150 %   const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id })
151 %
152 %   dispatch({
153 %     type: "ADD_TOAST",
154 %     toast: {
155 %       ...props,
156 %       id,
157 %       open: true,
158 %       onOpenChange: (open) => {
159 %         if (!open) dismiss()
160 %       },
161 %     },
162 %   })
163 %
164 %   return {
165 %     id: id,
166 %     dismiss,
167 %     update,
168 %   }
169 % }
170 %
171 % function useToast() {
172 %   const [state, setState] = React.useState<State>(memoryState)
173 %
174 %   React.useEffect(() => {
175 %     listeners.push(setState)
176 %     return () => {
177 %       const index = listeners.indexOf(setState)
178 %       if (index > -1) {
179 %         listeners.splice(index, 1)
180 %       }
181 %     }
182 %   }, [state])
183 % }
```

client/src/hooks/use-toast.ts (cont.)

```
184 %   return {
185 %     ...state,
186 %     toast,
187 %     dismiss: (toastId?: string) => dispatch({ type: "DISMISS_TOAST", toastId }),
188 %   }
189 % }
190 %
191 % export { useToast, toast }
192 %
```

```
1 % import * as React from "react"
2 %
3 % const MOBILE_BREAKPOINT = 768
4 %
5 % export function useIsMobile() {
6 %   const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined)
7 %
8 %   React.useEffect(() => {
9 %     const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
10 %     const onChange = () => {
11 %       setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
12 %     }
13 %     mql.addEventListener("change", onChange)
14 %     setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
15 %     return () => mql.removeEventListener("change", onChange)
16 %   }, [])
17 %
18 %   return !!isMobile
19 % }
20 %
```

```

1 % import { useState, useEffect, useCallback } from "react";
2 % import { useLocation, Link } from "wouter";
3 % import { useQuery, useMutation } from "@tanstack/react-query";
4 % import {
5 %   LayoutDashboard,
6 %   CalendarCheck,
7 %   DoorOpen,
8 %   Users,
9 %   LogIn,
10 %  LogOut,
11 %  Settings,
12 %  Hotel,
13 %  CalendarDays,
14 %  DollarSign,
15 %  Globe,
16 %  CalendarPlus,
17 %  Users2,
18 %  MessageSquare,
19 %  Sparkles,
20 %  UtensilsCrossed,
21 %  Package,
22 %  Flower2,
23 %  PartyPopper,
24 %  Wrench,
25 %  Shield,
26 %  Gift,
27 %  Building2,
28 %  ChevronDown,
29 %  Boxes,
30 %  Bell,
31 %  CheckCheck,
32 %  AlertTriangle,
33 %  Smartphone,
34 %  Bot,
35 %  Heart,
36 % } from "lucide-react";
37 % import {
38 %   Sidebar,
39 %   SidebarContent,
40 %   SidebarGroup,
41 %   SidebarGroupContent,
42 %   SidebarGroupLabel,
43 %   SidebarHeader,
44 %   SidebarMenu,
45 %   SidebarMenuButton,
46 %   SidebarMenuItem,
47 %   SidebarMenuSub,
48 %   SidebarMenuSubItem,
49 %   SidebarMenuSubButton,
50 %   SidebarFooter,
51 % } from "@components/ui/sidebar";
52 % import {
53 %   Collapsible,
54 %   CollapsibleContent,
55 %   CollapsibleTrigger,
56 % } from "@components/ui/collapsible";
57 % import {
58 %   Popover,
59 %   PopoverContent,
60 %   PopoverTrigger,
61 % } from "@components/ui/popover";
62 % import { Button } from "@components/ui/button";
63 % import { Badge } from "@components/ui/badge";
64 % import { ScrollArea } from "@components/ui/scroll-area";
65 % import { queryClient, apiRequest } from "@lib/queryClient";
66 % import type { SystemNotification } from "@shared/schema";
67 %
68 % const hotelSubItems = [
69 %   { title: "Planning", url: "/planning", icon: CalendarDays },
70 %   { title: "Reservas", url: "/reservations", icon: CalendarCheck },
71 %   { title: "Reserva Rápida", url: "/new-reservation", icon: CalendarPlus },
72 %   { title: "Check-in", url: "/check-in", icon: LogIn },
73 %   { title: "Check-out", url: "/check-out", icon: LogOut },
74 %   { title: "Habitaciones", url: "/rooms", icon: DoorOpen },
75 %   { title: "Tarifas", url: "/rate-plans", icon: DollarSign },
76 %   { title: "Paquetes", url: "/packages", icon: Gift },
77 %   { title: "Canales OTA", url: "/ota-channels", icon: Globe },
78 %   { title: "Grupos", url: "/groups", icon: Users2 },
79 %   { title: "Reseñas", url: "/reviews", icon: MessageSquare },
80 % ];
81 %
82 % const hotelPaths = hotelSubItems.map((i) => i.url);
83 %
84 % function formatTimeAgo(date: Date | string): string {
85 %   const now = new Date();
86 %   const d = typeof date === "string" ? new Date(date) : date;
87 %   const diffMs = now.getTime() - d.getTime();
88 %   const mins = Math.floor(diffMs / 60000);
89 %   if (mins < 1) return "ahora";
90 %   if (mins < 60) return `hace ${mins} min`;

```


client/src/components/app-sidebar.tsx (cont.)

```
91 %   const hours = Math.floor(mins / 60);
92 %   if (hours < 24) return `hace ${hours}h`;
93 %   const days = Math.floor(hours / 24);
94 %   return `hace ${days}d`;
95 % }
96 %
97 % function getNotificationIcon(type: string) {
98 %   if (type === "web_checkin") return <Smartphone className="h-4 w-4 text-green-500" />;
99 %   if (type.startsWith("chatbot_")) return <Bot className="h-4 w-4 text-blue-500" />;
100 %   if (type === "hospitality_alert") return <Heart className="h-4 w-4 text-red-500" />;
101 %   return <Bell className="h-4 w-4 text-muted-foreground" />;
102 % }
103 %
104 % function getAreaLabel(area: string): string {
105 %   const labels: Record<string, string> = {
106 %     reception: "Recepción",
107 %     housekeeping: "Housekeeping",
108 %     maintenance: "Mantenimiento",
109 %     restaurant: "Restaurante",
110 %     spa: "SPA",
111 %     all: "General",
112 %   };
113 %   return labels[area] || area;
114 % }
115 %
116 % function NotificationBell() {
117 %   const [open, setOpen] = useState(false);
118 %   const [, navigate] = useLocation();
119 %
120 %   const { data: countData } = useQuery<{ count: number }>({
121 %     queryKey: ["/api/notifications/unread-count"],
122 %     refetchInterval: 30000,
123 %   });
124 %
125 %   const { data: notifications } = useQuery<SystemNotification[]>({
126 %     queryKey: ["/api/notifications"],
127 %     queryFn: async () => {
128 %       const res = await apiRequest("GET", "/api/notifications?limit=20");
129 %       return res.json();
130 %     },
131 %     enabled: open,
132 %   });
133 %
134 %   const markReadMutation = useMutation({
135 %     mutationFn: async (id: number) => {
136 %       await apiRequest("PATCH", `/api/notifications/${id}/read`);
137 %     },
138 %     onSuccess: () => {
139 %       queryClient.invalidateQueries({ queryKey: ["/api/notifications"] });
140 %       queryClient.invalidateQueries({ queryKey: ["/api/notifications/unread-count"] });
141 %     },
142 %   });
143 %
144 %   const markAllReadMutation = useMutation({
145 %     mutationFn: async () => {
146 %       await apiRequest("PATCH", "/api/notifications/read-all");
147 %     },
148 %     onSuccess: () => {
149 %       queryClient.invalidateQueries({ queryKey: ["/api/notifications"] });
150 %       queryClient.invalidateQueries({ queryKey: ["/api/notifications/unread-count"] });
151 %     },
152 %   });
153 %
154 %   const unreadCount = countData?.count || 0;
155 %
156 %   return (
157 %     <Popover open={open} onOpenChange={setOpen}>
158 %       <PopoverTrigger asChild>
159 %         <button
160 %           className="relative p-1.5 rounded-md hover:bg-sidebar-accent transition-colors"
161 %           data-testid="button-notifications"
162 %         >
163 %           <Bell className="h-5 w-5 text-sidebar-foreground" />
164 %           {unreadCount > 0 && (
165 %             <span className="absolute -top-0.5 -right-0.5 flex h-4 min-w-[16px] items-center justify-center rounded-f...
166 %               {unreadCount > 99 ? "99+" : unreadCount}
167 %             </span>
168 %           )}
169 %         </button>
170 %       </PopoverTrigger>
171 %       <PopoverContent className="w-80 p-0" side="right" align="start">
172 %         <div className="flex items-center justify-between border-b px-4 py-3">
173 %           <h4 className="font-semibold text-sm">Notificaciones</h4>
174 %           {unreadCount > 0 && (
175 %             <Button
176 %               variant="ghost"
177 %               size="sm"
178 %               className="h-7 text-xs"
179 %               onClick={() => markAllReadMutation.mutate()}
180 %               disabled={markAllReadMutation.isPending}
181 %               data-testid="button-mark-all-read"
182 %             >
183 %               <CheckCheck className="h-3 w-3 mr-1" />
```

client/src/components/app-sidebar.tsx (cont.)

```
184 %         Marcar todas como leídas
185 %     </Button>
186 %   })
187 % </div>
188 % <ScrollArea className="max-h-[360px]">
189 %   {notifications && notifications.length > 0 ? (
190 %     <div className="divide-y">
191 %       {notifications.map((n) => (
192 %         <button
193 %           key={n.id}
194 %           className={`w-full text-left px-4 py-3 hover:bg-muted/50 transition-colors ${
195 %             !n.isRead ? "bg-primary/5" : ""
196 %           }}
197 %           onClick={() => {
198 %             if (!n.isRead) markReadMutation.mutate(n.id);
199 %             if (n.type.startsWith("chatbot_")) {
200 %               setOpen(false);
201 %               navigate("/chatbot");
202 %             }
203 %           }}
204 %           data-testid={`notification-item-${n.id}`}
205 %         >
206 %           <div className="flex items-start gap-2.5">
207 %             <div className="mt-0.5">{getNotificationIcon(n.type)}</div>
208 %             <div className="flex-1 min-w-0">
209 %               <div className="flex items-center gap-2 mb-0.5">
210 %                 <span className={`text-sm font-medium truncate ${!n.isRead ? "text-foreground" : "text-muted-..."}>
211 %                   {n.title}
212 %                 </span>
213 %                 {n.priority === "urgent" && (
214 %                   <AlertTriangle className="h-3 w-3 text-destructive shrink-0" />
215 %                 )}
216 %               </div>
217 %               <p className="text-xs text-muted-foreground line-clamp-2">{n.message}</p>
218 %               <div className="flex items-center gap-2 mt-1">
219 %                 <Badge variant="outline" className="text-[10px] px-1.5 py-0 h-4">
220 %                   {getAreaLabel(n.targetArea)}
221 %                 </Badge>
222 %                 <span className="text-[10px] text-muted-foreground">
223 %                   {n.createdAt ? formatTimeAgo(n.createdAt) : ""}
224 %                 </span>
225 %               </div>
226 %             </div>
227 %             {!n.isRead && (
228 %               <div className="h-2 w-2 rounded-full bg-primary shrink-0 mt-1.5" />
229 %             )}
230 %           </div>
231 %         </button>
232 %       )]}
233 %     </div>
234 %   ) : (
235 %     <div className="flex flex-col items-center justify-center py-8 text-center">
236 %       <Bell className="h-8 w-8 text-muted-foreground/40 mb-2" />
237 %       <p className="text-sm text-muted-foreground">Sin notificaciones</p>
238 %     </div>
239 %   )}
240 % </ScrollArea>
241 % </PopoverContent>
242 % </Popover>
243 % );
244 % }
245 %
246 % export function AppSidebar() {
247 %   const [location] = useLocation();
248 %   const [hotelOpen, setHotelOpen] = useState(
249 %     hotelPaths.includes(location) || location.startsWith("/groups/")
250 %   );
251 %
252 %   const isHotelActive = hotelPaths.includes(location) || location.startsWith("/groups/");
253 %
254 %   return (
255 %     <Sidebar>
256 %       <SidebarHeader className="p-4">
257 %         <div className="flex items-center gap-3">
258 %           <div className="flex h-10 w-10 items-center justify-center rounded-md bg-primary text-primary-foreground">
259 %             <Hotel className="h-6 w-6" />
260 %           </div>
261 %           <div className="flex flex-col flex-1">
262 %             <span className="text-lg font-semibold text-sidebar-foreground">Maran Suites</span>
263 %             <span className="text-xs text-muted-foreground">Sistema de Gestión</span>
264 %           </div>
265 %         </div>
266 %       </SidebarHeader>
267 %       <SidebarContent>
268 %         <SidebarGroup>
269 %           <SidebarGroupContent>
270 %             <SidebarMenu>
271 %               <SidebarMenuItem>
272 %                 <SidebarMenuButton
273 %                   asChild
274 %                   isActive={location === "/"}
275 %                   data-testid="nav-dashboard"
276 %                 />
```

client/src/components/app-sidebar.tsx (cont.)

```
277 %         >
278 %         <Link href="/">
279 %           <LayoutDashboard className="h-5 w-5" />
280 %           <span>Dashboard</span>
281 %         </Link>
282 %       </SidebarMenuButton>
283 %     </SidebarMenuItem>
284 %
285 %     <Collapsible open={hotelOpen} onOpenChange={setHotelOpen} className="group/collapsible">
286 %       <SidebarMenuItem>
287 %         <CollapsibleTrigger asChild>
288 %           <SidebarMenuButton
289 %             isActive={isHotelActive}
290 %             data-testid="nav-hotel"
291 %           >
292 %             <Hotel className="h-5 w-5" />
293 %             <span>Hotel</span>
294 %             <ChevronDown className="ml-auto h-4 w-4 transition-transform group-data-[state=open]/collapsibl...
295 %           </SidebarMenuButton>
296 %         </CollapsibleTrigger>
297 %         <CollapsibleContent>
298 %           <SidebarMenuSub>
299 %             {hotelSubItems.map((item) => (
300 %               <SidebarMenuSubItem key={item.url}>
301 %                 <SidebarMenuSubButton
302 %                   asChild
303 %                   isActive={location === item.url || (item.url === "/groups" && location.startsWith("/group...
304 %                   data-testid={`nav-${item.url.replace("/", "")}`}
305 %                 >
306 %                   <Link href={item.url}>
307 %                     <item.icon className="h-4 w-4" />
308 %                     <span>{item.title}</span>
309 %                   </Link>
310 %                 </SidebarMenuSubButton>
311 %               </SidebarMenuSubItem>
312 %             )})
313 %           </SidebarMenuSub>
314 %         </CollapsibleContent>
315 %       </SidebarMenuItem>
316 %     </Collapsible>
317 %   </SidebarMenu>
318 % </SidebarGroupContent>
319 % </SidebarGroup>
320 %
321 % <SidebarGroup>
322 %   <SidebarGroupLabel>Base de Datos</SidebarGroupLabel>
323 %   <SidebarGroupContent>
324 %     <SidebarMenu>
325 %       <SidebarMenuItem>
326 %         <SidebarMenuButton
327 %           asChild
328 %           isActive={location === "/guests"}
329 %           data-testid="nav-guests"
330 %         >
331 %           <Link href="/guests">
332 %             <Users className="h-5 w-5" />
333 %             <span>Huéspedes</span>
334 %           </Link>
335 %         </SidebarMenuButton>
336 %       </SidebarMenuItem>
337 %       <SidebarMenuItem>
338 %         <SidebarMenuButton
339 %           asChild
340 %           isActive={location === "/companies"}
341 %           data-testid="nav-companies"
342 %         >
343 %           <Link href="/companies">
344 %             <Building2 className="h-5 w-5" />
345 %             <span>Empresas</span>
346 %           </Link>
347 %         </SidebarMenuButton>
348 %       </SidebarMenuItem>
349 %     </SidebarMenu>
350 %   </SidebarGroupContent>
351 % </SidebarGroup>
352 %
353 % <SidebarGroup>
354 %   <SidebarGroupLabel>Servicios</SidebarGroupLabel>
355 %   <SidebarGroupContent>
356 %     <SidebarMenu>
357 %       <SidebarMenuItem>
358 %         <SidebarMenuButton
359 %           asChild
360 %           isActive={location === "/restaurant"}
361 %           data-testid="nav-restaurant"
362 %         >
363 %           <Link href="/restaurant">
364 %             <UtensilsCrossed className="h-5 w-5" />
365 %             <span>Restaurante</span>
366 %           </Link>
367 %         </SidebarMenuButton>
368 %       </SidebarMenuItem>
369 %     </SidebarMenu>
```

client/src/components/app-sidebar.tsx (cont.)

```
370 %         <SidebarMenuButton
371 %           asChild
372 %           isActive={location === "/spa"}
373 %           data-testid="nav-spa"
374 %         >
375 %           <Link href="/spa">
376 %             <Flower2 className="h-5 w-5" />
377 %             <span>SPA</span>
378 %           </Link>
379 %         </SidebarMenuButton>
380 %       </SidebarMenuItem>
381 %     <SidebarMenuItem>
382 %       <SidebarMenuButton
383 %         asChild
384 %         isActive={location === "/events"}
385 %         data-testid="nav-events"
386 %       >
387 %         <Link href="/events">
388 %           <PartyPopper className="h-5 w-5" />
389 %           <span>Eventos</span>
390 %         </Link>
391 %       </SidebarMenuButton>
392 %     </SidebarMenuItem>
393 %   </SidebarMenu>
394 </SidebarGroupContent>
395 </SidebarGroup>
396 %
397 <SidebarGroup>
398 <SidebarGroupLabel>Operaciones</SidebarGroupLabel>
399 <SidebarGroupContent>
400 <SidebarMenu>
401 <SidebarMenuItem>
402 <SidebarMenuButton
403   asChild
404   isActive={location === "/housekeeping"}
405   data-testid="nav-housekeeping"
406 >
407   <Link href="/housekeeping">
408     <Sparkles className="h-5 w-5" />
409     <span>Housekeeping</span>
410   </Link>
411 </SidebarMenuButton>
412 </SidebarMenuItem>
413 <SidebarMenuItem>
414 <SidebarMenuButton
415   asChild
416   isActive={location === "/maintenance"}
417   data-testid="nav-maintenance"
418 >
419   <Link href="/maintenance">
420     <Wrench className="h-5 w-5" />
421     <span>Mantenimiento</span>
422   </Link>
423 </SidebarMenuButton>
424 </SidebarMenuItem>
425 <SidebarMenuItem>
426 <SidebarMenuButton
427   asChild
428   isActive={location === "/inventory"}
429   data-testid="nav-inventory"
430 >
431   <Link href="/inventory">
432     <Boxes className="h-5 w-5" />
433     <span>Inventario</span>
434   </Link>
435 </SidebarMenuButton>
436 </SidebarMenuItem>
437 </SidebarMenu>
438 </SidebarGroupContent>
439 </SidebarGroup>
440 %
441 <SidebarGroup>
442 <SidebarGroupLabel>Experiencia del Huésped</SidebarGroupLabel>
443 <SidebarGroupContent>
444 <SidebarMenu>
445 <SidebarMenuItem>
446 <SidebarMenuButton
447   asChild
448   isActive={location === "/hospitality"}
449   data-testid="nav-hospitality"
450 >
451   <Link href="/hospitality">
452     <Heart className="h-5 w-5" />
453     <span>Hospitalidad</span>
454   </Link>
455 </SidebarMenuButton>
456 </SidebarMenuItem>
457 </SidebarMenu>
458 </SidebarGroupContent>
459 </SidebarGroup>
460 %
461 <SidebarGroup>
462 <SidebarGroupLabel>Comunicaciones</SidebarGroupLabel>
```

client/src/components/app-sidebar.tsx (cont.)

```
463 %         <SidebarGroupContent>
464 %         <SidebarMenu>
465 %           <SidebarMenuItem>
466 %             <SidebarMenuButton
467 %               asChild
468 %               isActive={location === "/chatbot"}
469 %               data-testid="nav-chatbot"
470 %             >
471 %               <Link href="/chatbot">
472 %                 <Bot className="h-5 w-5" />
473 %                 <span>MARA Chatbot</span>
474 %               </Link>
475 %             </SidebarMenuButton>
476 %           </SidebarMenuItem>
477 %         </SidebarMenu>
478 %       </SidebarGroupContent>
479 %     </SidebarGroup>
480 %
481 %     <SidebarGroup>
482 %       <SidebarGroupLabel>Administración</SidebarGroupLabel>
483 %       <SidebarGroupContent>
484 %         <SidebarMenu>
485 %           <SidebarMenuItem>
486 %             <SidebarMenuButton
487 %               asChild
488 %               isActive={location === "/administration"}
489 %               data-testid="nav-administration"
490 %             >
491 %               <Link href="/administration">
492 %                 <Shield className="h-5 w-5" />
493 %                 <span>Administración</span>
494 %               </Link>
495 %             </SidebarMenuButton>
496 %           </SidebarMenuItem>
497 %         </SidebarMenu>
498 %       </SidebarGroupContent>
499 %     </SidebarGroup>
500 %   </SidebarContent>
501 %   <SidebarFooter className="p-4">
502 %     <SidebarMenu>
503 %       <SidebarMenuItem>
504 %         <SidebarMenuButton asChild data-testid="nav-settings">
505 %           <Link href="/settings">
506 %             <Settings className="h-5 w-5" />
507 %             <span>Configuración</span>
508 %           </Link>
509 %         </SidebarMenuButton>
510 %       </SidebarMenuItem>
511 %     </SidebarMenu>
512 %   </SidebarFooter>
513 % </Sidebar>
514 % );
515 % }
516 %
```

```
1 % import { createContext, useContext, useEffect, useState } from "react";
2 %
3 % type Theme = "dark" | "light" | "system";
4 %
5 % type ThemeProviderProps = {
6 %   children: React.ReactNode;
7 %   defaultTheme?: Theme;
8 %   storageKey?: string;
9 % };
10 %
11 % type ThemeProviderState = {
12 %   theme: Theme;
13 %   setTheme: (theme: Theme) => void;
14 % };
15 %
16 % const initialState: ThemeProviderState = {
17 %   theme: "system",
18 %   setTheme: () => null,
19 % };
20 %
21 % const ThemeProviderContext = createContext<ThemeProviderState>(initialState);
22 %
23 % export function ThemeProvider({
24 %   children,
25 %   defaultTheme = "system",
26 %   storageKey = "hotel-ui-theme",
27 %   ...props
28 % }: ThemeProviderProps) {
29 %   const [theme, setTheme] = useState<Theme>(
30 %     () => (localStorage.getItem(storageKey) as Theme) || defaultTheme
31 %   );
32 %
33 %   useEffect(() => {
34 %     const root = window.document.documentElement;
35 %
36 %     root.classList.remove("light", "dark");
37 %
38 %     if (theme === "system") {
39 %       const systemTheme = window.matchMedia("(prefers-color-scheme: dark)")
40 %         .matches
41 %         ? "dark"
42 %         : "light";
43 %
44 %       root.classList.add(systemTheme);
45 %       return;
46 %     }
47 %
48 %     root.classList.add(theme);
49 %   }, [theme]);
50 %
51 %   const value = {
52 %     theme,
53 %     setTheme: (theme: Theme) => {
54 %       localStorage.setItem(storageKey, theme);
55 %       setTheme(theme);
56 %     },
57 %   };
58 %
59 %   return (
60 %     <ThemeProviderContext.Provider {...props} value={value}>
61 %       {children}
62 %     </ThemeProviderContext.Provider>
63 %   );
64 % }
65 %
66 % export const useTheme = () => {
67 %   const context = useContext(ThemeProviderContext);
68 %
69 %   if (context === undefined)
70 %     throw new Error("useTheme must be used within a ThemeProvider");
71 %
72 %   return context;
73 % };
74 %
```

```
1 % import { Moon, Sun } from "lucide-react";
2 % import { Button } from "@components/ui/button";
3 % import {
4 %   DropdownMenu,
5 %   DropdownMenuContent,
6 %   DropdownMenuItem,
7 %   DropdownMenuTrigger,
8 % } from "@components/ui/dropdown-menu";
9 % import { useTheme } from "@components/theme-provider";
10 %
11 % export function ThemeToggle() {
12 %   const { setTheme } = useTheme();
13 %
14 %   return (
15 %     <DropdownMenu>
16 %       <DropdownMenuTrigger asChild>
17 %         <Button variant="ghost" size="icon" data-testid="button-theme-toggle">
18 %           <Sun className="h-5 w-5 rotate-0 scale-100 transition-all dark:-rotate-90 dark:scale-0" />
19 %           <Moon className="absolute h-5 w-5 rotate-90 scale-0 transition-all dark:rotate-0 dark:scale-100" />
20 %           <span className="sr-only">Toggle theme</span>
21 %         </Button>
22 %       </DropdownMenuTrigger>
23 %       <DropdownMenuContent align="end">
24 %         <DropdownMenuItem onClick={() => setTheme("light")} data-testid="menu-item-light">
25 %           Claro
26 %         </DropdownMenuItem>
27 %         <DropdownMenuItem onClick={() => setTheme("dark")} data-testid="menu-item-dark">
28 %           Oscuro
29 %         </DropdownMenuItem>
30 %         <DropdownMenuItem onClick={() => setTheme("system")} data-testid="menu-item-system">
31 %           Sistema
32 %         </DropdownMenuItem>
33 %       </DropdownMenuContent>
34 %     </DropdownMenu>
35 %   );
36 % }
37 %
```

```

1 % import { useState, useEffect } from "react";
2 % import { useQuery } from "@tanstack/react-query";
3 % import { Input } from "@components/ui/input";
4 % import { Button } from "@components/ui/button";
5 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
6 % import { Badge } from "@components/ui/badge";
7 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
8 % import { Label } from "@components/ui/label";
9 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
10 % import { Search, User, Building2, Plus, X, Check } from "lucide-react";
11 % import type { Guest, Company, InsertGuest, InsertCompany } from "@shared/schema";
12 %
13 % interface GuestSelectorProps {
14 %   onSelect: (guest: Guest) => void;
15 %   onCreateNew: (guest: InsertGuest) => void;
16 %   selectedGuest?: Guest | null;
17 %   onClear?: () => void;
18 % }
19 %
20 % export function GuestSelector({ onSelect, onCreateNew, selectedGuest, onClear }: GuestSelectorProps) {
21 %   const [searchQuery, setSearchQuery] = useState("");
22 %   const [mode, setMode] = useState<"search" | "create">("search");
23 %   const [debouncedQuery, setDebouncedQuery] = useState("");
24 %
25 %   const [newGuest, setNewGuest] = useState({
26 %     firstName: "",
27 %     lastName: "",
28 %     email: "",
29 %     phone: "",
30 %     documentType: "dni" as "dni" | "passport" | "cedula" | "other" | null,
31 %     documentNumber: "",
32 %     nationality: "Argentina",
33 %     direccion: "",
34 %     localidad: "",
35 %     codigoPostal: "",
36 %     fechaNacimiento: "",
37 %     sexo: "no_especifica" as "masculino" | "femenino" | "otro" | "no_especifica",
38 %     cuilCuit: "",
39 %   });
40 %
41 %   useEffect(() => {
42 %     const timer = setTimeout(() => setDebouncedQuery(searchQuery), 300);
43 %     return () => clearTimeout(timer);
44 %   }, [searchQuery]);
45 %
46 %   const { data: searchResults = [], isLoading } = useQuery<Guest[]>({
47 %     queryKey: ["/api/guests/search", debouncedQuery],
48 %     queryFn: async () => {
49 %       if (!debouncedQuery || debouncedQuery.length < 2) return [];
50 %       const res = await fetch(`/api/guests/search?q=${encodeURIComponent(debouncedQuery)}`);
51 %       return res.json();
52 %     },
53 %     enabled: debouncedQuery.length >= 2,
54 %   });
55 %
56 %   const handleCreateGuest = () => {
57 %     if (!newGuest.firstName || !newGuest.lastName) return;
58 %     onCreateNew({
59 %       firstName: newGuest.firstName,
60 %       lastName: newGuest.lastName,
61 %       email: newGuest.email || null,
62 %       phone: newGuest.phone || null,
63 %       documentType: newGuest.documentType,
64 %       documentNumber: newGuest.documentNumber || null,
65 %       nationality: newGuest.nationality || null,
66 %       direccion: newGuest.direccion || null,
67 %       localidad: newGuest.localidad || null,
68 %       codigoPostal: newGuest.codigoPostal || null,
69 %       fechaNacimiento: newGuest.fechaNacimiento || null,
70 %       sexo: newGuest.sexo,
71 %       cuilCuit: newGuest.cuilCuit || null,
72 %       companyId: null,
73 %     });
74 %     setNewGuest({
75 %       firstName: "",
76 %       lastName: "",
77 %       email: "",
78 %       phone: "",
79 %       documentType: "dni",
80 %       documentNumber: "",
81 %       nationality: "Argentina",
82 %       direccion: "",
83 %       localidad: "",
84 %       codigoPostal: "",
85 %       fechaNacimiento: "",
86 %       sexo: "no_especifica",
87 %       cuilCuit: "",
88 %     });
89 %     setMode("search");
90 %   };

```



```

91 %
92 %   if (selectedGuest) {
93 %       return (
94 %           <Card className="bg-accent/30">
95 %               <CardContent className="pt-4 pb-4">
96 %                   <div className="flex items-center justify-between gap-2">
97 %                       <div className="flex items-center gap-2 flex-wrap">
98 %                           <User className="h-4 w-4 text-muted-foreground" />
99 %                           <span className="font-medium">
100 %                               {selectedGuest.firstName} {selectedGuest.lastName}
101 %                           </span>
102 %                           {selectedGuest.codigo && (
103 %                               <Badge variant="secondary" className="text-xs">
104 %                                   {selectedGuest.codigo}
105 %                               </Badge>
106 %                           )}
107 %                           <Badge variant="outline" className="text-xs">
108 %                               {selectedGuest.documentType?.toUpperCase()} {selectedGuest.documentNumber}
109 %                           </Badge>
110 %                       </div>
111 %                       {onClear && (
112 %                           <Button size="icon" variant="ghost" onClick={onClear} data-testid="button-clear-guest">
113 %                               <X className="h-4 w-4" />
114 %                           </Button>
115 %                       )}
116 %                   </div>
117 %                   {selectedGuest.email && (
118 %                       <p className="text-sm text-muted-foreground mt-1">{selectedGuest.email}</p>
119 %                   )}
120 %               </CardContent>
121 %           </Card>
122 %       );
123 %   }
124 %
125 %   return (
126 %       <Card>
127 %           <CardHeader className="pb-3">
128 %               <CardTitle className="text-base flex items-center gap-2">
129 %                   <User className="h-4 w-4" />
130 %                   Seleccionar Huesped
131 %               </CardTitle>
132 %           </CardHeader>
133 %           <CardContent>
134 %               <Tabs value={mode} onValueChange={(v) => setMode(v as "search" | "create")}>
135 %                   <TabsList className="grid w-full grid-cols-2 mb-4">
136 %                       <TabsTrigger value="search" data-testid="tab-search-guest">
137 %                           <Search className="h-4 w-4 mr-2" />
138 %                           Buscar Existente
139 %                       </TabsTrigger>
140 %                       <TabsTrigger value="create" data-testid="tab-create-guest">
141 %                           <Plus className="h-4 w-4 mr-2" />
142 %                           Crear Nuevo
143 %                       </TabsTrigger>
144 %                   </TabsList>
145 %
146 %                   <TabsContent value="search" className="space-y-4">
147 %                       <div className="relative">
148 %                           <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
149 %                           <Input
150 %                               placeholder="Buscar por nombre, email o documento..."
151 %                               value={searchQuery}
152 %                               onChange={(e) => setSearchQuery(e.target.value)}
153 %                               className="pl-10"
154 %                               data-testid="input-search-guest"
155 %                           />
156 %                       </div>
157 %
158 %                       {isLoading && (
159 %                           <p className="text-sm text-muted-foreground text-center py-4">Buscando...</p>
160 %                       )}
161 %
162 %                       {searchResults.length > 0 && (
163 %                           <div className="space-y-2 max-h-60 overflow-y-auto">
164 %                               {searchResults.map((guest) => (
165 %                                   <div
166 %                                       key={guest.id}
167 %                                       className="p-3 rounded-md border hover-elevate active-elevate-2 cursor-pointer"
168 %                                       onClick={() => onSelect(guest)}
169 %                                       data-testid={`guest-result-${guest.id}`}
170 %                                   >
171 %                                       <div className="flex items-center justify-between gap-2">
172 %                                           <div>
173 %                                               <p className="font-medium">
174 %                                                   {guest.firstName} {guest.lastName}
175 %                                                   {guest.codigo && <span className="text-muted-foreground ml-2 text-sm">{guest.codigo}</span>}
176 %                                               </p>
177 %                                               <p className="text-sm text-muted-foreground">
178 %                                                   {guest.documentType?.toUpperCase()} {guest.documentNumber}
179 %                                               </p>
180 %                                           </div>
181 %                                           <Check className="h-4 w-4 text-muted-foreground invisible group-hover:visible" />
182 %                                       </div>
183 %                                   </div>
184 %                               )}
185 %                           </div>
186 %                       )}
187 %                   </TabsContent>
188 %               </Tabs>
189 %           </CardContent>
190 %       </Card>
191 %   );

```

client/src/components/entity-selector.tsx (cont.)

```
184 %         )}}
185 %       </div>
186 %     )}
187 %
188 %     {debouncedQuery.length >= 2 && searchResults.length === 0 && !isLoading && (
189 %       <p className="text-sm text-muted-foreground text-center py-4">
190 %         No se encontraron huéspedes.
191 %         <Button variant="ghost" className="p-0 ml-1 h-auto text-primary underline" onClick={() => setMode("cr...
192 %           Crear nuevo
193 %         </Button>
194 %       </p>
195 %     )}
196 %   </TabsContent>
197 %
198 %   <TabsContent value="create" className="space-y-4">
199 %     <div className="grid grid-cols-2 gap-4">
200 %       <div className="space-y-2">
201 %         <Label htmlFor="firstName">Nombre *</Label>
202 %         <Input
203 %           id="firstName"
204 %           value={newGuest.firstName}
205 %           onChange={(e) => setNewGuest({ ...newGuest, firstName: e.target.value })}
206 %           placeholder="Nombre"
207 %           data-testid="input-guest-firstname"
208 %         />
209 %       </div>
210 %       <div className="space-y-2">
211 %         <Label htmlFor="lastName">Apellido *</Label>
212 %         <Input
213 %           id="lastName"
214 %           value={newGuest.lastName}
215 %           onChange={(e) => setNewGuest({ ...newGuest, lastName: e.target.value })}
216 %           placeholder="Apellido"
217 %           data-testid="input-guest-lastname"
218 %         />
219 %       </div>
220 %     </div>
221 %
222 %     <div className="grid grid-cols-2 gap-4">
223 %       <div className="space-y-2">
224 %         <Label htmlFor="documentType">Tipo Doc.</Label>
225 %         <Select
226 %           value={newGuest.documentType || "dni"}
227 %           onValueChange={(v) => setNewGuest({ ...newGuest, documentType: v as any })}
228 %         >
229 %           <SelectTrigger data-testid="select-document-type">
230 %             <SelectValue />
231 %           </SelectTrigger>
232 %           <SelectContent>
233 %             <SelectItem value="dni">DNI</SelectItem>
234 %             <SelectItem value="passport">Pasaporte</SelectItem>
235 %             <SelectItem value="cedula">Cedula</SelectItem>
236 %             <SelectItem value="other">Otro</SelectItem>
237 %           </SelectContent>
238 %         </Select>
239 %       </div>
240 %       <div className="space-y-2">
241 %         <Label htmlFor="documentNumber">Numero Doc.</Label>
242 %         <Input
243 %           id="documentNumber"
244 %           value={newGuest.documentNumber}
245 %           onChange={(e) => setNewGuest({ ...newGuest, documentNumber: e.target.value })}
246 %           placeholder="12345678"
247 %           data-testid="input-document-number"
248 %         />
249 %       </div>
250 %     </div>
251 %
252 %     <div className="grid grid-cols-2 gap-4">
253 %       <div className="space-y-2">
254 %         <Label htmlFor="email">Email</Label>
255 %         <Input
256 %           id="email"
257 %           type="email"
258 %           value={newGuest.email}
259 %           onChange={(e) => setNewGuest({ ...newGuest, email: e.target.value })}
260 %           placeholder="correo@email.com"
261 %           data-testid="input-guest-email"
262 %         />
263 %       </div>
264 %       <div className="space-y-2">
265 %         <Label htmlFor="phone">Telefono</Label>
266 %         <Input
267 %           id="phone"
268 %           value={newGuest.phone}
269 %           onChange={(e) => setNewGuest({ ...newGuest, phone: e.target.value })}
270 %           placeholder="+54 11 1234-5678"
271 %           data-testid="input-guest-phone"
272 %         />
273 %       </div>
274 %     </div>
275 %
276 %   <div className="grid grid-cols-2 gap-4">
```

client/src/components/entity-selector.tsx (cont.)

```

277 %         <div className="space-y-2">
278 %             <Label htmlFor="nationality">Nacionalidad</Label>
279 %             <Input
280 %                 id="nationality"
281 %                 value={newGuest.nationality}
282 %                 onChange={(e) => setNewGuest({ ...newGuest, nationality: e.target.value })}
283 %                 placeholder="Argentina"
284 %                 data-testid="input-guest-nationality"
285 %             />
286 %         </div>
287 %         <div className="space-y-2">
288 %             <Label htmlFor="sexo">Sexo</Label>
289 %             <Select
290 %                 value={newGuest.sexo}
291 %                 onValueChange={(v) => setNewGuest({ ...newGuest, sexo: v as any })}
292 %             >
293 %                 <SelectTrigger data-testid="select-guest-sexo">
294 %                     <SelectValue />
295 %                 </SelectTrigger>
296 %                 <SelectContent>
297 %                     <SelectItem value="masculino">Masculino</SelectItem>
298 %                     <SelectItem value="femenino">Femenino</SelectItem>
299 %                     <SelectItem value="otro">Otro</SelectItem>
300 %                     <SelectItem value="no_especifica">No especifica</SelectItem>
301 %                 </SelectContent>
302 %             </Select>
303 %         </div>
304 %     </div>
305 %
306 %     <div className="grid grid-cols-2 gap-4">
307 %         <div className="space-y-2">
308 %             <Label htmlFor="fechaNacimiento">Fecha Nacimiento</Label>
309 %             <Input
310 %                 id="fechaNacimiento"
311 %                 type="date"
312 %                 value={newGuest.fechaNacimiento}
313 %                 onChange={(e) => setNewGuest({ ...newGuest, fechaNacimiento: e.target.value })}
314 %                 data-testid="input-guest-fechanacimiento"
315 %             />
316 %         </div>
317 %         <div className="space-y-2">
318 %             <Label htmlFor="cuilCuit">CUIL/CUIT</Label>
319 %             <Input
320 %                 id="cuilCuit"
321 %                 value={newGuest.cuilCuit}
322 %                 onChange={(e) => setNewGuest({ ...newGuest, cuilCuit: e.target.value })}
323 %                 placeholder="20-12345678-9"
324 %                 data-testid="input-guest-cuilcuit"
325 %             />
326 %         </div>
327 %     </div>
328 %
329 %     <div className="space-y-2">
330 %         <Label htmlFor="direccion">Direccion</Label>
331 %         <Input
332 %             id="direccion"
333 %             value={newGuest.direccion}
334 %             onChange={(e) => setNewGuest({ ...newGuest, direccion: e.target.value })}
335 %             placeholder="Av. Corrientes 1234"
336 %             data-testid="input-guest-direccion"
337 %         />
338 %     </div>
339 %
340 %     <div className="grid grid-cols-2 gap-4">
341 %         <div className="space-y-2">
342 %             <Label htmlFor="localidad">Localidad</Label>
343 %             <Input
344 %                 id="localidad"
345 %                 value={newGuest.localidad}
346 %                 onChange={(e) => setNewGuest({ ...newGuest, localidad: e.target.value })}
347 %                 placeholder="CABA"
348 %                 data-testid="input-guest-localidad"
349 %             />
350 %         </div>
351 %         <div className="space-y-2">
352 %             <Label htmlFor="codigoPostal">Codigo Postal</Label>
353 %             <Input
354 %                 id="codigoPostal"
355 %                 value={newGuest.codigoPostal}
356 %                 onChange={(e) => setNewGuest({ ...newGuest, codigoPostal: e.target.value })}
357 %                 placeholder="1000"
358 %                 data-testid="input-guest-codigopostal"
359 %             />
360 %         </div>
361 %     </div>
362 %
363 %     <Button
364 %         onClick={handleCreateGuest}
365 %         disabled={!newGuest.firstName || !newGuest.lastName}
366 %         className="w-full"
367 %         data-testid="button-create-guest"
368 %     >
369 %         <Plus className="h-4 w-4 mr-2" />

```

client/src/components/entity-selector.tsx (cont.)

```
370 %         Crear Huesped
371 %     </Button>
372 % </TabsContent>
373 % </Tabs>
374 % </CardContent>
375 % </Card>
376 % );
377 % }
378 %
379 % interface CompanySelectorProps {
380 %     onSelect: (company: Company) => void;
381 %     onCreateNew: (company: InsertCompany) => void;
382 %     selectedCompany?: Company | null;
383 %     onClear?: () => void;
384 % }
385 %
386 % export function CompanySelector({ onSelect, onCreateNew, selectedCompany, onClear }: CompanySelectorProps) {
387 %     const [searchQuery, setSearchQuery] = useState("");
388 %     const [mode, setMode] = useState<"search" | "create">("search");
389 %     const [debouncedQuery, setDebouncedQuery] = useState("");
390 %
391 %     const [newCompany, setNewCompany] = useState({
392 %         razonSocial: "",
393 %         nombreFantasia: "",
394 %         cuilCuit: "",
395 %         condicionIva: "responsable_inscripto" as "responsable_inscripto" | "monotributo" | "exento" | "consumidor_final" ...
396 %         direccion: "",
397 %         localidad: "",
398 %         provincia: "",
399 %         codigoPostal: "",
400 %         pais: "Argentina",
401 %         telefono: "",
402 %         email: "",
403 %         numeroFiscal: "",
404 %         inscripcionNacional: "",
405 %         inscripcionProvincial: "",
406 %         contactName: "",
407 %         contactEmail: "",
408 %         contactPhone: "",
409 %         creditLimit: "0",
410 %         paymentTermDays: 30,
411 %         notes: "",
412 %     });
413 %
414 %     useEffect(() => {
415 %         const timer = setTimeout(() => setDebouncedQuery(searchQuery), 300);
416 %         return () => clearTimeout(timer);
417 %     }, [searchQuery]);
418 %
419 %     const { data: searchResults = [], isLoading } = useQuery<Company[]>({
420 %         queryKey: ["/api/companies/search", debouncedQuery],
421 %         queryFn: async () => {
422 %             if (!debouncedQuery || debouncedQuery.length < 2) return [];
423 %             const res = await fetch(`/api/companies/search?q=${encodeURIComponent(debouncedQuery)}`);
424 %             return res.json();
425 %         },
426 %         enabled: debouncedQuery.length >= 2,
427 %     });
428 %
429 %     const handleCreateCompany = () => {
430 %         if (!newCompany.razonSocial || !newCompany.cuilCuit) return;
431 %         onCreateNew({
432 %             razonSocial: newCompany.razonSocial,
433 %             cuilCuit: newCompany.cuilCuit,
434 %             nombreFantasia: newCompany.nombreFantasia || null,
435 %             condicionIva: newCompany.condicionIva,
436 %             direccion: newCompany.direccion || null,
437 %             localidad: newCompany.localidad || null,
438 %             provincia: newCompany.provincia || null,
439 %             codigoPostal: newCompany.codigoPostal || null,
440 %             pais: newCompany.pais || "Argentina",
441 %             telefono: newCompany.telefono || null,
442 %             email: newCompany.email || null,
443 %             numeroFiscal: newCompany.numeroFiscal || null,
444 %             inscripcionNacional: newCompany.inscripcionNacional || null,
445 %             inscripcionProvincial: newCompany.inscripcionProvincial || null,
446 %             contactName: newCompany.contactName || null,
447 %             contactEmail: newCompany.contactEmail || null,
448 %             contactPhone: newCompany.contactPhone || null,
449 %             creditLimit: newCompany.creditLimit || "0",
450 %             paymentTermDays: newCompany.paymentTermDays || 30,
451 %             notes: newCompany.notes || null,
452 %             isActive: "true",
453 %         });
454 %         setNewCompany({
455 %             razonSocial: "",
456 %             nombreFantasia: "",
457 %             cuilCuit: "",
458 %             condicionIva: "responsable_inscripto",
459 %             direccion: "",
460 %             localidad: "",
461 %             provincia: "",
462 %             codigoPostal: "",
```


client/src/components/entity-selector.tsx (cont.)

```
556 %         {company.nombreFantasia || company.razonSocial}
557 %     </p>
558 %     <p className="text-sm text-muted-foreground">
559 %         CUIT {company.cuilCuit}
560 %     </p>
561 % </div>
562 % <Check className="h-4 w-4 text-muted-foreground invisible group-hover:visible" />
563 % </div>
564 % </div>
565 %     )}}
566 % </div>
567 % }}
568 %
569 % {debouncedQuery.length >= 2 && searchResults.length === 0 && !isLoading && (
570 %   <p className="text-sm text-muted-foreground text-center py-4">
571 %     No se encontraron empresas.
572 %     <Button variant="ghost" className="p-0 ml-1 h-auto text-primary underline" onClick={() => setMode("cr...
573 %       Crear nueva
574 %     </Button>
575 %   </p>
576 % )}
577 % </TabsContent>
578 %
579 % <TabsContent value="create" className="space-y-4">
580 %   <div className="grid grid-cols-2 gap-4">
581 %     <div className="space-y-2">
582 %       <Label htmlFor="razonSocial">Razon Social *</Label>
583 %       <Input
584 %         id="razonSocial"
585 %         value={newCompany.razonSocial}
586 %         onChange={(e) => setNewCompany({ ...newCompany, razonSocial: e.target.value })}
587 %         placeholder="Empresa S.A."
588 %         data-testid="input-company-razonsocial"
589 %       />
590 %     </div>
591 %     <div className="space-y-2">
592 %       <Label htmlFor="nombreFantasia">Nombre Fantasia</Label>
593 %       <Input
594 %         id="nombreFantasia"
595 %         value={newCompany.nombreFantasia}
596 %         onChange={(e) => setNewCompany({ ...newCompany, nombreFantasia: e.target.value })}
597 %         placeholder="Nombre Comercial"
598 %         data-testid="input-company-nombrefantasia"
599 %       />
600 %     </div>
601 %   </div>
602 %
603 %   <div className="grid grid-cols-2 gap-4">
604 %     <div className="space-y-2">
605 %       <Label htmlFor="cuilCuit">CUIT *</Label>
606 %       <Input
607 %         id="cuilCuit"
608 %         value={newCompany.cuilCuit}
609 %         onChange={(e) => setNewCompany({ ...newCompany, cuilCuit: e.target.value })}
610 %         placeholder="30-12345678-9"
611 %         data-testid="input-company-cuilcuit"
612 %       />
613 %     </div>
614 %     <div className="space-y-2">
615 %       <Label htmlFor="condicionIva">Condicion IVA</Label>
616 %       <Select
617 %         value={newCompany.condicionIva}
618 %         onValueChange={(v) => setNewCompany({ ...newCompany, condicionIva: v as any })}
619 %       >
620 %         <SelectTrigger data-testid="select-condicion-iva">
621 %           <SelectValue />
622 %         </SelectTrigger>
623 %         <SelectContent>
624 %           <SelectItem value="responsable_inscripto">Responsable Inscripto</SelectItem>
625 %           <SelectItem value="monotributo">Monotributo</SelectItem>
626 %           <SelectItem value="exento">Exento</SelectItem>
627 %           <SelectItem value="consumidor_final">Consumidor Final</SelectItem>
628 %           <SelectItem value="no_responsable">No Responsable</SelectItem>
629 %         </SelectContent>
630 %       </Select>
631 %     </div>
632 %   </div>
633 %
634 %   <div className="space-y-2">
635 %     <Label htmlFor="direccion">Direccion</Label>
636 %     <Input
637 %       id="direccion"
638 %       value={newCompany.direccion}
639 %       onChange={(e) => setNewCompany({ ...newCompany, direccion: e.target.value })}
640 %       placeholder="Av. del Libertador 1000"
641 %       data-testid="input-company-direccion"
642 %     />
643 %   </div>
644 %
645 %   <div className="grid grid-cols-3 gap-4">
646 %     <div className="space-y-2">
647 %       <Label htmlFor="localidad">Localidad</Label>
648 %       <Input
```

client/src/components/entity-selector.tsx (cont.)

```
649 %         id="localidad"
650 %         value={newCompany.localidad}
651 %         onChange={(e) => setNewCompany({ ...newCompany, localidad: e.target.value })}
652 %         placeholder="CABA"
653 %         data-testid="input-company-localidad"
654 %       />
655 %     </div>
656 %     <div className="space-y-2">
657 %       <Label htmlFor="provincia">Provincia</Label>
658 %       <Input
659 %         id="provincia"
660 %         value={newCompany.provincia}
661 %         onChange={(e) => setNewCompany({ ...newCompany, provincia: e.target.value })}
662 %         placeholder="Buenos Aires"
663 %         data-testid="input-company-provincia"
664 %       />
665 %     </div>
666 %     <div className="space-y-2">
667 %       <Label htmlFor="codigoPostal">CP</Label>
668 %       <Input
669 %         id="codigoPostal"
670 %         value={newCompany.codigoPostal}
671 %         onChange={(e) => setNewCompany({ ...newCompany, codigoPostal: e.target.value })}
672 %         placeholder="1000"
673 %         data-testid="input-company-codigopostal"
674 %       />
675 %     </div>
676 %   </div>
677 %
678 %   <div className="grid grid-cols-2 gap-4">
679 %     <div className="space-y-2">
680 %       <Label htmlFor="companyEmail">Email</Label>
681 %       <Input
682 %         id="companyEmail"
683 %         type="email"
684 %         value={newCompany.email}
685 %         onChange={(e) => setNewCompany({ ...newCompany, email: e.target.value })}
686 %         placeholder="contacto@empresa.com"
687 %         data-testid="input-company-email"
688 %       />
689 %     </div>
690 %     <div className="space-y-2">
691 %       <Label htmlFor="companyPhone">Telefono</Label>
692 %       <Input
693 %         id="companyPhone"
694 %         value={newCompany.telefono}
695 %         onChange={(e) => setNewCompany({ ...newCompany, telefono: e.target.value })}
696 %         placeholder="+54 11 4000-1234"
697 %         data-testid="input-company-telefono"
698 %       />
699 %     </div>
700 %   </div>
701 %
702 %   <div className="space-y-2">
703 %     <Label htmlFor="contactName">Nombre de Contacto</Label>
704 %     <Input
705 %       id="contactName"
706 %       value={newCompany.contactName}
707 %       onChange={(e) => setNewCompany({ ...newCompany, contactName: e.target.value })}
708 %       placeholder="Juan Perez"
709 %       data-testid="input-company-contact"
710 %     />
711 %   </div>
712 %
713 %   <Button
714 %     onClick={handleCreateCompany}
715 %     disabled={!newCompany.razonSocial || !newCompany.cuilCuit}
716 %     className="w-full"
717 %     data-testid="button-create-company"
718 %   >
719 %     <Plus className="h-4 w-4 mr-2" />
720 %     Crear Empresa
721 %   </Button>
722 % </TabsContent>
723 % </Tabs>
724 % </CardContent>
725 % </Card>
726 % );
727 % }
728 %
```

```

1 % import { useQuery, useMutation } from "@tanstack/react-query";
2 % import {
3 %   DoorOpen,
4 %   Users,
5 %   CalendarCheck,
6 %   TrendingUp,
7 %   ArrowUpRight,
8 %   ArrowDownRight,
9 %   Clock,
10 %   AlertCircle,
11 %   LogIn,
12 %   LogOut,
13 % } from "lucide-react";
14 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
15 % import { Badge } from "@components/ui/badge";
16 % import { Button } from "@components/ui/button";
17 % import { Skeleton } from "@components/ui/skeleton";
18 % import { Link } from "wouter";
19 % import { useToast } from "@hooks/use-toast";
20 % import { queryClient, apiRequest } from "@lib/queryClient";
21 % import type { ReservationWithDetails, RoomWithType } from "@shared/schema";
22 %
23 % type DashboardStats = {
24 %   totalRooms: number;
25 %   availableRooms: number;
26 %   occupiedRooms: number;
27 %   dirtyRooms: number;
28 %   cleaningRooms: number;
29 %   maintenanceRooms: number;
30 %   oosRooms: number;
31 %   todayCheckIns: number;
32 %   todayCheckOuts: number;
33 %   occupancyRate: number;
34 %   totalGuests: number;
35 %   pendingReservations: number;
36 % };
37 %
38 % function StatCard({
39 %   title,
40 %   value,
41 %   description,
42 %   icon: Icon,
43 %   trend,
44 %   trendValue,
45 %   testId,
46 % }: {
47 %   title: string;
48 %   value: string | number;
49 %   description?: string;
50 %   icon: React.ElementType;
51 %   trend?: "up" | "down";
52 %   trendValue?: string;
53 %   testId?: string;
54 % }) {
55 %   return (
56 %     <Card data-testid={testId}>
57 %       <CardHeader className="flex flex-row items-center justify-between gap-2 space-y-0 pb-2">
58 %         <CardTitle className="text-sm font-medium text-muted-foreground">{title}</CardTitle>
59 %         <Icon className="h-5 w-5 text-muted-foreground" />
60 %       </CardHeader>
61 %       <CardContent>
62 %         <div className="text-3xl font-bold">{value}</div>
63 %         {description && (
64 %           <p className="text-sm text-muted-foreground mt-1">{description}</p>
65 %         )}
66 %         {trend && trendValue && (
67 %           <div className="flex items-center gap-1 mt-2">
68 %             {trend === "up" ? (
69 %               <ArrowUpRight className="h-4 w-4 text-green-600 dark:text-green-400" />
70 %             ) : (
71 %               <ArrowDownRight className="h-4 w-4 text-red-600 dark:text-red-400" />
72 %             )}
73 %             <span
74 %               className={`text-sm font-medium ${
75 %                 trend === "up"
76 %                   ? "text-green-600 dark:text-green-400"
77 %                   : "text-red-600 dark:text-red-400"
78 %               }`}
79 %             >
80 %               {trendValue}
81 %             </span>
82 %           </div>
83 %         )}
84 %       </CardContent>
85 %     </Card>
86 %   );
87 % }
88 %
89 % function QuickActionCard({
90 %   title,

```


client/src/pages/dashboard.tsx (cont.)

```
91 %   description,
92 %   icon: Icon,
93 %   href,
94 %   variant = "default",
95 %   testId,
96 % } : {
97 %   title: string;
98 %   description: string;
99 %   icon: React.ElementType;
100 %   href: string;
101 %   variant?: "default" | "primary";
102 %   testId: string;
103 % }) {
104 %   return (
105 %     <Link href={href} data-testid={testId}>
106 %       <Card className={`hover-elevate active-elevate-2 cursor-pointer transition-colors ${
107 %         variant === "primary" ? "bg-primary text-primary-foreground" : ""
108 %       }`} >
109 %         <CardContent className="flex items-center gap-4 p-4">
110 %           <div className={`flex h-12 w-12 items-center justify-center rounded-md ${
111 %             variant === "primary"
112 %               ? "bg-primary-foreground/20"
113 %               : "bg-muted"
114 %             }`} >
115 %             <Icon className="h-6 w-6" />
116 %           </div>
117 %           <div>
118 %             <h3 className="font-semibold">{title}</h3>
119 %             <p className={`text-sm ${
120 %               variant === "primary" ? "text-primary-foreground/80" : "text-muted-foreground"
121 %             }`} >{description}</p>
122 %           </div>
123 %         </CardContent>
124 %       </Card>
125 %     </Link>
126 %   );
127 % }
128 %
129 % function ReservationStatusBadge({ status }: { status: string }) {
130 %   const statusConfig: Record<string, { label: string; variant: "default" | "secondary" | "destructive" | "outline" }>...
131 %   tentative: { label: "Tentativa", variant: "outline" },
132 %   pending: { label: "Pendiente", variant: "secondary" },
133 %   confirmed: { label: "Confirmada", variant: "default" },
134 %   checked_in: { label: "Check-in", variant: "default" },
135 %   checked_out: { label: "Check-out", variant: "outline" },
136 %   cancelled: { label: "Cancelada", variant: "destructive" },
137 % };
138 %
139 %   const config = statusConfig[status] || { label: status, variant: "secondary" as const };
140 %
141 %   return <Badge variant={config.variant}>{config.label}</Badge>;
142 % }
143 %
144 % function RoomStatusBadge({ status }: { status: string }) {
145 %   const statusConfig: Record<string, { label: string; className: string }> = {
146 %     available: { label: "Disponibile", className: "bg-green-100 text-green-800 dark:bg-green-900/30 dark:text-green-400...
147 %     occupied: { label: "Ocupada", className: "bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-400" },
148 %     cleaning: { label: "Limpieza", className: "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yellow-400...
149 %     maintenance: { label: "Mantenimiento", className: "bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-400" },
150 %   };
151 %
152 %   const config = statusConfig[status] || { label: status, className: "" };
153 %
154 %   return <Badge className={config.className}>{config.label}</Badge>;
155 % }
156 %
157 % export default function Dashboard() {
158 %   const { toast } = useToast();
159 %
160 %   const { data: stats, isLoading: statsLoading } = useQuery<DashboardStats>({
161 %     queryKey: ["/api/dashboard/stats"],
162 %   });
163 %
164 %   const { data: recentReservations, isLoading: reservationsLoading } = useQuery<ReservationWithDetails[]>({
165 %     queryKey: ["/api/reservations/recent"],
166 %   });
167 %
168 %   const { data: roomsOverview, isLoading: roomsLoading } = useQuery<RoomWithType[]>({
169 %     queryKey: ["/api/rooms"],
170 %   });
171 %
172 %   const { data: arrivals = [], isLoading: arrivalsLoading } = useQuery<ReservationWithDetails[]>({
173 %     queryKey: ["/api/dashboard/arrivals"],
174 %   });
175 %
176 %   const { data: departures = [], isLoading: departuresLoading } = useQuery<ReservationWithDetails[]>({
177 %     queryKey: ["/api/dashboard/departures"],
178 %   });
179 %
180 %   const checkInMutation = useMutation({
181 %     mutationFn: async (id: string) => {
182 %       return apiRequest("POST", `/api/reservations/${id}/check-in`, {});
183 %     },
184 %   });
```

client/src/pages/dashboard.tsx (cont.)

```
184 %     onSuccess: () => {
185 %       queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
186 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
187 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/arrivals"] });
188 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/departures"] });
189 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
190 %       toast({ title: "Check-in realizado", description: "El huesped ha sido registrado." });
191 %     },
192 %     onError: (error: any) => {
193 %       const message = error?.data?.error || error?.message || "No se pudo realizar el check-in.";
194 %       toast({ title: "Check-in no permitido", description: message, variant: "destructive" });
195 %     },
196 %   });
197 %
198 %   const checkOutMutation = useMutation({
199 %     mutationFn: async (id: string) => {
200 %       return apiRequest("POST", `/api/reservations/${id}/check-out`, {});
201 %     },
202 %     onSuccess: () => {
203 %       queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
204 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
205 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/arrivals"] });
206 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/departures"] });
207 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
208 %       toast({ title: "Check-out realizado", description: "El huesped ha sido despedido." });
209 %     },
210 %     onError: (error: any) => {
211 %       const message = error?.data?.error || error?.message || "No se pudo realizar el check-out.";
212 %       if (message.includes("saldo pendiente") || message.includes("balance")) {
213 %         toast({ title: "Saldo Pendiente", description: message, variant: "destructive" });
214 %       } else {
215 %         toast({ title: "Error", description: message, variant: "destructive" });
216 %       }
217 %     },
218 %   });
219 %
220 %   const today = new Date().toLocaleDateString("es-ES", {
221 %     weekday: "long",
222 %     year: "numeric",
223 %     month: "long",
224 %     day: "numeric",
225 %   });
226 %
227 %   return (
228 %     <div className="flex flex-col gap-6 p-6">
229 %       <div className="flex flex-col gap-1">
230 %         <h1 className="text-3xl font-bold tracking-tight" data-testid="text-dashboard-title">Dashboard</h1>
231 %         <p className="text-muted-foreground capitalize">{today}</p>
232 %       </div>
233 %
234 %       { /* Stats Grid */ }
235 %       <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-4">
236 %         {statsLoading ? (
237 %           <>
238 %             {[...Array(4)].map((_, i) => (
239 %               <Card key={i}>
240 %                 <CardHeader className="pb-2">
241 %                   <Skeleton className="h-4 w-24" />
242 %                 </CardHeader>
243 %                 <CardContent>
244 %                   <Skeleton className="h-8 w-16" />
245 %                   <Skeleton className="h-4 w-32 mt-2" />
246 %                 </CardContent>
247 %               </Card>
248 %             )]}
249 %           </>
250 %         ) : stats ? (
251 %           <>
252 %             <StatCard
253 %               title="Habitaciones Disponibles"
254 %               value={stats.availableRooms}
255 %               description={`de ${stats.totalRooms} habitaciones`}
256 %               icon={DoorOpen}
257 %               testId="stat-available-rooms"
258 %             />
259 %             <StatCard
260 %               title="Ocupación"
261 %               value={` ${stats.occupancyRate}%`}
262 %               description={` ${stats.occupiedRooms} habitaciones ocupadas`}
263 %               icon={TrendingUp}
264 %               trend={stats.occupancyRate > 70 ? "up" : "down"}
265 %               trendValue={stats.occupancyRate > 70 ? "Alta demanda" : "Disponibilidad"}
266 %               testId="stat-occupancy"
267 %             />
268 %             <StatCard
269 %               title="Check-ins Hoy"
270 %               value={stats.todayCheckIns}
271 %               description="llegadas programadas"
272 %               icon={CalendarCheck}
273 %               testId="stat-checkins-today"
274 %             />
275 %             <StatCard
276 %               title="Huéspedes Activos"
```

client/src/pages/dashboard.tsx (cont.)

```
277 %         value={stats.totalGuests}
278 %         description="registrados en el sistema"
279 %         icon={Users}
280 %         testId="stat-active-guests"
281 %     />
282 % </>
283 % ) : null}
284 % </div>
285 %
286 % { /* Quick Actions */ }
287 % <div>
288 %   <h2 className="text-xl font-semibold mb-4">Acciones Rápidas</h2>
289 %   <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-4">
290 %     <QuickActionCard
291 %       title="Nueva Reserva"
292 %       description="Crear una nueva reservación"
293 %       icon={CalendarCheck}
294 %       href="/new-reservation"
295 %       variant="primary"
296 %       testId="action-new-reservation"
297 %     />
298 %     <QuickActionCard
299 %       title="Check-in"
300 %       description="Registrar llegada de huésped"
301 %       icon={ArrowUpRight}
302 %       href="/check-in"
303 %       testId="action-check-in"
304 %     />
305 %     <QuickActionCard
306 %       title="Check-out"
307 %       description="Registrar salida de huésped"
308 %       icon={ArrowDownRight}
309 %       href="/check-out"
310 %       testId="action-check-out"
311 %     />
312 %     <QuickActionCard
313 %       title="Ver Habitaciones"
314 %       description="Estado de todas las habitaciones"
315 %       icon={DoorOpen}
316 %       href="/rooms"
317 %       testId="action-view-rooms"
318 %     />
319 %   </div>
320 % </div>
321 %
322 % { /* Arrivals & Departures */ }
323 % <div className="grid gap-6 lg:grid-cols-2">
324 %   { /* Today's Arrivals */ }
325 %   <Card>
326 %     <CardHeader className="flex flex-row items-center justify-between gap-2">
327 %       <div>
328 %         <CardTitle className="flex items-center gap-2">
329 %           <LogIn className="h-5 w-5 text-green-600" />
330 %           Llegadas del Día
331 %         </CardTitle>
332 %         <CardDescription>{arrivals.length} check-ins pendientes</CardDescription>
333 %       </div>
334 %     </CardHeader>
335 %     <CardContent>
336 %       {arrivalsLoading ? (
337 %         <div className="space-y-3">
338 %           {[...Array(3)].map((_, i) => (
339 %             <Skeleton key={i} className="h-16 w-full" />
340 %           ))}
341 %         </div>
342 %       ) : arrivals.length > 0 ? (
343 %         <div className="space-y-3">
344 %           {arrivals.slice(0, 5).map((reservation) => {
345 %             const roomStatus = reservation.room?.status;
346 %             const isRoomReady = roomStatus === "available";
347 %             return (
348 %               <div
349 %                 key={reservation.id}
350 %                 className="flex items-center justify-between gap-3 p-3 rounded-md border bg-green-50/50 dark:bg...
351 %                 data-testid={`arrival-${reservation.id}`}
352 %               >
353 %                 <div className="flex-1 min-w-0">
354 %                   <p className="font-medium truncate">
355 %                     {reservation.guest?.firstName} {reservation.guest?.lastName}
356 %                   </p>
357 %                   <div className="flex items-center gap-2 text-sm text-muted-foreground">
358 %                     <span>Hab. {reservation.room?.roomNumber}</span>
359 %                     {!isRoomReady && (
360 %                       <Badge variant="secondary" className="text-xs">
361 %                         {roomStatus === "cleaning" ? "En limpieza" :
362 %                           roomStatus === "maintenance" ? "Mantenimiento" :
363 %                           roomStatus === "occupied" ? "Ocupada" : roomStatus}
364 %                       </Badge>
365 %                     )}
366 %                   </div>
367 %                 </div>
368 %                 <Button
369 %                   size="sm"
```

client/src/pages/dashboard.tsx (cont.)

```
370 %         onClick={() => checkInMutation.mutate(reservation.id)}
371 %         disabled={checkInMutation.isPending || !isRoomReady}
372 %         variant={isRoomReady ? "default" : "secondary"}
373 %         data-testid={`checkin-btn-${reservation.id}`}
374 %       >
375 %         <LogIn className="h-4 w-4 mr-1" />
376 %         {isRoomReady ? "Check-in" : "No lista"}
377 %       </Button>
378 %     </div>
379 %   );
380 %   }}}
381 % </div>
382 % ) : (
383 %   <div className="flex flex-col items-center justify-center py-6 text-center">
384 %     <LogIn className="h-10 w-10 text-muted-foreground/30 mb-2" />
385 %     <p className="text-muted-foreground text-sm">No hay llegadas programadas</p>
386 %   </div>
387 % )}
388 % </CardContent>
389 % </Card>
390 %
391 % {/* Today's Departures */}
392 % <Card>
393 %   <CardHeader className="flex flex-row items-center justify-between gap-2">
394 %     <div>
395 %       <CardTitle className="flex items-center gap-2">
396 %         <LogOut className="h-5 w-5 text-orange-600" />
397 %         Salidas del Dia
398 %       </CardTitle>
399 %       <CardDescription>{departures.length} check-outs pendientes</CardDescription>
400 %     </div>
401 %   </CardHeader>
402 %   <CardContent>
403 %     {departuresLoading ? (
404 %       <div className="space-y-3">
405 %         {[...Array(3)].map((_, i) => (
406 %           <Skeleton key={i} className="h-16 w-full" />
407 %         ))}
408 %       </div>
409 %     ) : departures.length > 0 ? (
410 %       <div className="space-y-3">
411 %         {departures.slice(0, 5).map((reservation) => (
412 %           <div
413 %             key={reservation.id}
414 %             className="flex items-center justify-between gap-3 p-3 rounded-md border bg-orange-50/50 dark:bg-...
415 %             data-testid={`departure-${reservation.id}`}
416 %           >
417 %             <div className="flex-1 min-w-0">
418 %               <p className="font-medium truncate">
419 %                 {reservation.guest?.firstName} {reservation.guest?.lastName}
420 %               </p>
421 %               <p className="text-sm text-muted-foreground">
422 %                 Hab. {reservation.room?.roomNumber} | ${reservation.totalRoomAmount}
423 %               </p>
424 %             </div>
425 %             <Button
426 %               size="sm"
427 %               variant="secondary"
428 %               onClick={() => checkOutMutation.mutate(reservation.id)}
429 %               disabled={checkOutMutation.isPending}
430 %               data-testid={`checkout-btn-${reservation.id}`}
431 %             >
432 %               <LogOut className="h-4 w-4 mr-1" />
433 %               Check-out
434 %             </Button>
435 %           </div>
436 %         ))}
437 %       </div>
438 %     ) : (
439 %       <div className="flex flex-col items-center justify-center py-6 text-center">
440 %         <LogOut className="h-10 w-10 text-muted-foreground/30 mb-2" />
441 %         <p className="text-muted-foreground text-sm">No hay salidas programadas</p>
442 %       </div>
443 %     )}
444 %   </CardContent>
445 % </Card>
446 % </div>
447 %
448 % {/* Main Content Grid */}
449 % <div className="grid gap-6 lg:grid-cols-2">
450 %   {/* Recent Reservations */}
451 %   <Card>
452 %     <CardHeader className="flex flex-row items-center justify-between gap-2">
453 %       <div>
454 %         <CardTitle>Reservas Recientes</CardTitle>
455 %         <CardDescription>Ultimas reservaciones registradas</CardDescription>
456 %       </div>
457 %       <Button variant="outline" size="sm" asChild>
458 %         <Link href="/reservations">Ver todas</Link>
459 %       </Button>
460 %     </CardHeader>
461 %     <CardContent>
462 %       {reservationsLoading ? (
```

client/src/pages/dashboard.tsx (cont.)

```
463 %         <div className="space-y-4">
464 %           {...Array(3)].map((_, i) => (
465 %             <div key={i} className="flex items-center gap-4">
466 %               <Skeleton className="h-10 w-10 rounded-full" />
467 %               <div className="flex-1 space-y-2">
468 %                 <Skeleton className="h-4 w-32" />
469 %                 <Skeleton className="h-3 w-24" />
470 %               </div>
471 %               <Skeleton className="h-6 w-20" />
472 %             </div>
473 %           )})
474 %         </div>
475 %       ) : recentReservations && recentReservations.length > 0 ? (
476 %         <div className="space-y-4">
477 %           {recentReservations.slice(0, 5).map((reservation) => (
478 %             <div
479 %               key={reservation.id}
480 %               className="flex items-center justify-between gap-4 p-3 rounded-md bg-muted/50"
481 %               data-testid={`reservation-item-${reservation.id}`}
482 %             >
483 %               <div className="flex items-center gap-3">
484 %                 <div className="flex h-10 w-10 items-center justify-center rounded-full bg-primary/10 text-prim...
485 %                   {reservation.guest?.firstName?.[0]}{reservation.guest?.lastName?.[0]}
486 %                 </div>
487 %                 <div>
488 %                   <p className="font-medium">
489 %                     {reservation.guest?.firstName} {reservation.guest?.lastName}
490 %                   </p>
491 %                   <p className="text-sm text-muted-foreground">
492 %                     Hab. {reservation.room?.roomNumber} | {reservation.checkInDate} - {reservation.checkOutDate}
493 %                   </p>
494 %                 </div>
495 %               </div>
496 %               <ReservationStatusBadge status={reservation.status} />
497 %             </div>
498 %           )})
499 %         </div>
500 %       ) : (
501 %         <div className="flex flex-col items-center justify-center py-8 text-center">
502 %           <CalendarCheck className="h-12 w-12 text-muted-foreground/50 mb-3" />
503 %           <p className="text-muted-foreground">No hay reservas recientes</p>
504 %           <Button variant="outline" size="sm" className="mt-4" asChild>
505 %             <Link href="/reservations/new">Crear primera reserva</Link>
506 %           </Button>
507 %         </div>
508 %       )})
509 %     </CardContent>
510 %   </Card>
511 %
512 %   </* Room Status Overview */>
513 %   <Card>
514 %     <CardHeader className="flex flex-row items-center justify-between gap-2">
515 %       <div>
516 %         <CardTitle>Estado de Habitaciones</CardTitle>
517 %         <CardDescription>Vista rápida del hotel</CardDescription>
518 %       </div>
519 %       <Button variant="outline" size="sm" asChild>
520 %         <Link href="/rooms">Ver todas</Link>
521 %       </Button>
522 %     </CardHeader>
523 %     <CardContent>
524 %       {roomsLoading ? (
525 %         <div className="grid grid-cols-4 gap-2">
526 %           {...Array(8)].map((_, i) => (
527 %             <Skeleton key={i} className="h-16 rounded-md" />
528 %           )})
529 %         </div>
530 %       ) : roomsOverview && roomsOverview.length > 0 ? (
531 %         <div className="grid grid-cols-4 gap-2">
532 %           {roomsOverview.slice(0, 12).map((room) => (
533 %             <div
534 %               key={room.id}
535 %               className={`flex flex-col items-center justify-center p-3 rounded-md border text-center ${
536 %                 room.status === "available"
537 %                   ? "bg-green-50 border-green-200 dark:bg-green-900/20 dark:border-green-800"
538 %                   : room.status === "occupied"
539 %                   ? "bg-blue-50 border-blue-200 dark:bg-blue-900/20 dark:border-blue-800"
540 %                   : room.status === "cleaning"
541 %                   ? "bg-yellow-50 border-yellow-200 dark:bg-yellow-900/20 dark:border-yellow-800"
542 %                   : "bg-red-50 border-red-200 dark:bg-red-900/20 dark:border-red-800"
543 %               }}
544 %               data-testid={`room-tile-${room.id}`}
545 %             >
546 %               <span className="text-lg font-bold">{room.roomNumber}</span>
547 %               <span className="text-xs text-muted-foreground capitalize">
548 %                 {room.status === "available" ? "Libre" :
549 %                   room.status === "occupied" ? "Ocupada" :
550 %                   room.status === "cleaning" ? "Limpieza" : "Mant."}
551 %               </span>
552 %             </div>
553 %           )})
554 %         </div>
555 %       ) : (
```

client/src/pages/dashboard.tsx (cont.)

```
556 %         <div className="flex flex-col items-center justify-center py-8 text-center">
557 %             <DoorOpen className="h-12 w-12 text-muted-foreground/50 mb-3" />
558 %             <p className="text-muted-foreground">No hay habitaciones registradas</p>
559 %             <Button variant="outline" size="sm" className="mt-4" asChild>
560 %                 <Link href="/rooms">Agregar habitaciones</Link>
561 %             </Button>
562 %         </div>
563 %     )}
564 %
565 %     {/* Room Status Legend */}
566 %     {roomsOverview && roomsOverview.length > 0 && (
567 %         <div className="flex flex-wrap items-center gap-4 mt-4 pt-4 border-t">
568 %             <div className="flex items-center gap-2">
569 %                 <div className="h-3 w-3 rounded-full bg-green-500" />
570 %                 <span className="text-sm text-muted-foreground">Disponible</span>
571 %             </div>
572 %             <div className="flex items-center gap-2">
573 %                 <div className="h-3 w-3 rounded-full bg-blue-500" />
574 %                 <span className="text-sm text-muted-foreground">Ocupada</span>
575 %             </div>
576 %             <div className="flex items-center gap-2">
577 %                 <div className="h-3 w-3 rounded-full bg-yellow-500" />
578 %                 <span className="text-sm text-muted-foreground">Limpieza</span>
579 %             </div>
580 %             <div className="flex items-center gap-2">
581 %                 <div className="h-3 w-3 rounded-full bg-red-500" />
582 %                 <span className="text-sm text-muted-foreground">Mantenimiento</span>
583 %             </div>
584 %         </div>
585 %     )}
586 %     </CardContent>
587 % </Card>
588 % </div>
589 %
590 %     {/* Alerts Section */}
591 %     {stats && (stats.maintenanceRooms > 0 || stats.pendingReservations > 0) && (
592 %         <Card className="border-yellow-200 bg-yellow-50 dark:border-yellow-800 dark:bg-yellow-900/20">
593 %             <CardHeader className="pb-2">
594 %                 <CardTitle className="flex items-center gap-2 text-yellow-800 dark:text-yellow-200">
595 %                     <AlertCircle className="h-5 w-5" />
596 %                     Alertas del Sistema
597 %                 </CardTitle>
598 %             </CardHeader>
599 %             <CardContent>
600 %                 <div className="space-y-2">
601 %                     {stats.maintenanceRooms > 0 && (
602 %                         <div className="flex items-center gap-2 text-yellow-800 dark:text-yellow-200">
603 %                             <Clock className="h-4 w-4" />
604 %                             <span>{stats.maintenanceRooms} habitación(es) en mantenimiento</span>
605 %                         </div>
606 %                     )}
607 %                     {stats.pendingReservations > 0 && (
608 %                         <div className="flex items-center gap-2 text-yellow-800 dark:text-yellow-200">
609 %                             <CalendarCheck className="h-4 w-4" />
610 %                             <span>{stats.pendingReservations} reserva(s) pendientes de confirmar</span>
611 %                         </div>
612 %                     )}
613 %                 </div>
614 %             </CardContent>
615 %         </Card>
616 %     )}
617 % </div>
618 % );
619 % }
620 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import {
4 %   DoorOpen,
5 %   Plus,
6 %   Search,
7 %   Filter,
8 %   MoreHorizontal,
9 %   Pencil,
10 %   Trash2,
11 %   Wrench,
12 %   Sparkles,
13 %   CheckCircle,
14 %   Accessibility,
15 %   Fence,
16 %   BedDouble,
17 %   Sofa,
18 %   ShowerHead,
19 %   BedSingle,
20 %   Users,
21 %   Home,
22 % } from "lucide-react";
23 % import { Tooltip, TooltipContent, TooltipTrigger } from "@components/ui/tooltip";
24 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
25 % import { Button } from "@components/ui/button";
26 % import { Input } from "@components/ui/input";
27 % import { Badge } from "@components/ui/badge";
28 % import { Skeleton } from "@components/ui/skeleton";
29 % import {
30 %   Dialog,
31 %   DialogContent,
32 %   DialogDescription,
33 %   DialogFooter,
34 %   DialogHeader,
35 %   DialogTitle,
36 %   DialogTrigger,
37 % } from "@components/ui/dialog";
38 % import {
39 %   DropdownMenu,
40 %   DropdownMenuContent,
41 %   DropdownMenuItem,
42 %   DropdownMenuSeparator,
43 %   DropdownMenuTrigger,
44 % } from "@components/ui/dropdown-menu";
45 % import {
46 %   Select,
47 %   SelectContent,
48 %   SelectItem,
49 %   SelectTrigger,
50 %   SelectValue,
51 % } from "@components/ui/select";
52 % import {
53 %   Table,
54 %   TableBody,
55 %   TableCell,
56 %   TableHead,
57 %   TableHeader,
58 %   TableRow,
59 % } from "@components/ui/table";
60 % import { Label } from "@components/ui/label";
61 % import { Textarea } from "@components/ui/textarea";
62 % import { useToast } from "@hooks/use-toast";
63 % import { queryClient, apiRequest } from "@lib/queryClient";
64 % import type { RoomWithType, RoomType, InsertRoom, RoomStatus } from "@shared/schema";
65 %
66 % function RoomStatusBadge({ status }: { status: RoomStatus }) {
67 %   const statusConfig: Record<RoomStatus, { label: string; className: string }> = {
68 %     available: { label: "Disponible", className: "bg-green-100 text-green-800 dark:bg-green-900/30 dark:text-green-400..."},
69 %     occupied: { label: "Ocupada", className: "bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-400" },
70 %     dirty: { label: "Dirty", className: "bg-orange-100 text-orange-800 dark:bg-orange-900/30 dark:text-orange-400" },
71 %     cleaning: { label: "Limpieza", className: "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yellow-400..."},
72 %     maintenance: { label: "Mantenimiento", className: "bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-400" },
73 %     oos: { label: "Fuera de Servicio", className: "bg-gray-100 text-gray-800 dark:bg-gray-900/30 dark:text-gray-400" },
74 %   };
75 %
76 %   const config = statusConfig[status];
77 %
78 %   return <Badge className={config.className}>{config.label}</Badge>;
79 % }
80 %
81 % const featureConfig: Record<string, { icon: typeof Accessibility; label: string }> = {
82 %   accessible: { icon: Accessibility, label: "Accesible" },
83 %   balcony: { icon: Fence, label: "Balcon" },
84 %   separable_bed: { icon: BedDouble, label: "Cama separable" },
85 %   sofa_bed: { icon: Sofa, label: "Sofa cama" },
86 %   shower_only: { icon: ShowerHead, label: "Solo ducha" },
87 %   extra_bed: { icon: BedSingle, label: "Cama extra" },
88 %   twin_config: { icon: BedSingle, label: "Config. Twin" },
89 %   living_room: { icon: Home, label: "Living" },
90 % };
```

client/src/pages/rooms.tsx (cont.)

```
91 %
92 % function RoomFeatures({ features, maxOccupancy }: { features?: string[] | null; maxOccupancy?: number | null }) {
93 %   if (!features || features.length === 0) {
94 %     return maxOccupancy ? (
95 %       <div className="flex items-center gap-1 text-muted-foreground">
96 %         <Users className="h-3 w-3" />
97 %         <span className="text-xs">{maxOccupancy}</span>
98 %       </div>
99 %     ) : null;
100 %   }
101 %
102 %   return (
103 %     <div className="flex items-center gap-1 flex-wrap">
104 %       {maxOccupancy && (
105 %         <Tooltip>
106 %           <TooltipTrigger asChild>
107 %             <div className="flex items-center gap-0.5 text-muted-foreground">
108 %               <Users className="h-3 w-3" />
109 %               <span className="text-xs">{maxOccupancy}</span>
110 %             </div>
111 %           </TooltipTrigger>
112 %           <TooltipContent>Max. {maxOccupancy} pax</TooltipContent>
113 %         </Tooltip>
114 %       )}
115 %       {features.map((feature) => {
116 %         const config = featureConfig[feature];
117 %         if (!config) return null;
118 %         const Icon = config.icon;
119 %         return (
120 %           <Tooltip key={feature}>
121 %             <TooltipTrigger asChild>
122 %               <div className="text-muted-foreground">
123 %                 <Icon className="h-3.5 w-3.5" />
124 %               </div>
125 %             </TooltipTrigger>
126 %             <TooltipContent>{config.label}</TooltipContent>
127 %           </Tooltip>
128 %         );
129 %       })}
130 %     </div>
131 %   );
132 % }
133 %
134 % function RoomFormDialog({
135 %   room,
136 %   roomTypes,
137 %   open,
138 %   onOpenChange,
139 %   onSuccess,
140 % }: {
141 %   room?: RoomWithType;
142 %   roomTypes: RoomType[];
143 %   open: boolean;
144 %   onOpenChange: (open: boolean) => void;
145 %   onSuccess: () => void;
146 % }) {
147 %   const { toast } = useToast();
148 %   const isEditing = !!room;
149 %
150 %   const [formData, setFormData] = useState<Partial<InsertRoom>>({
151 %     roomNumber: room?.roomNumber || "",
152 %     roomTypeId: room?.roomTypeId || "",
153 %     floor: room?.floor || 1,
154 %     status: room?.status || "available",
155 %     notes: room?.notes || "",
156 %   });
157 %
158 %   const mutation = useMutation({
159 %     mutationFn: async (data: Partial<InsertRoom>) => {
160 %       if (isEditing) {
161 %         return apiRequest("PATCH", `/api/rooms/${room.id}`, data);
162 %       }
163 %       return apiRequest("POST", "/api/rooms", data);
164 %     },
165 %     onSuccess: () => {
166 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
167 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
168 %       toast({
169 %         title: isEditing ? "Habitación actualizada" : "Habitación creada",
170 %         description: `La habitación ${formData.roomNumber} ha sido ${isEditing ? "actualizada" : "creada"} exitosamente...
171 %       });
172 %       onSuccess();
173 %       onOpenChange(false);
174 %     },
175 %     onError: () => {
176 %       toast({
177 %         title: "Error",
178 %         description: "No se pudo guardar la habitación. Intente nuevamente.",
179 %         variant: "destructive",
180 %       });
181 %     },
182 %   });
183 % }
```


client/src/pages/rooms.tsx (cont.)

```
184 %   const handleSubmit = (e: React.FormEvent) => {
185 %       e.preventDefault();
186 %       mutation.mutate(formData);
187 %   };
188 %
189 %   return (
190 %       <Dialog open={open} onOpenChange={onOpenChange}>
191 %         <DialogContent className="sm:max-w-[425px]">
192 %           <DialogHeader>
193 %             <DialogTitle>{isEditing ? "Editar Habitación" : "Nueva Habitación"}</DialogTitle>
194 %             <DialogDescription>
195 %               {isEditing
196 %                 ? "Modifica los detalles de la habitación."
197 %                 : "Ingresa los datos para crear una nueva habitación."}
198 %             </DialogDescription>
199 %           </DialogHeader>
200 %           <form onSubmit={handleSubmit}>
201 %             <div className="grid gap-4 py-4">
202 %               <div className="grid gap-2">
203 %                 <Label htmlFor="roomNumber">Número de Habitación</Label>
204 %                 <Input
205 %                   id="roomNumber"
206 %                   value={formData.roomNumber}
207 %                   onChange={(e) => setFormData({ ...formData, roomNumber: e.target.value })}
208 %                   placeholder="101"
209 %                   required
210 %                   data-testid="input-room-number"
211 %                 />
212 %               </div>
213 %               <div className="grid gap-2">
214 %                 <Label htmlFor="roomType">Tipo de Habitación</Label>
215 %                 <Select
216 %                   value={formData.roomTypeId}
217 %                   onValueChange={(value) => setFormData({ ...formData, roomTypeId: value })}
218 %                 >
219 %                   <SelectTrigger data-testid="select-room-type">
220 %                     <SelectValue placeholder="Seleccionar tipo" />
221 %                   </SelectTrigger>
222 %                   <SelectContent>
223 %                     {roomTypes.map((type) => (
224 %                       <SelectItem key={type.id} value={type.id}>
225 %                         {type.name}
226 %                       </SelectItem>
227 %                     ))}
228 %                   </SelectContent>
229 %                 </Select>
230 %               </div>
231 %               <div className="grid gap-2">
232 %                 <Label htmlFor="floor">Piso</Label>
233 %                 <Input
234 %                   id="floor"
235 %                   type="number"
236 %                   min={1}
237 %                   value={formData.floor}
238 %                   onChange={(e) => setFormData({ ...formData, floor: parseInt(e.target.value) })}
239 %                   data-testid="input-floor"
240 %                 />
241 %               </div>
242 %               <div className="grid gap-2">
243 %                 <Label htmlFor="status">Estado</Label>
244 %                 <Select
245 %                   value={formData.status}
246 %                   onValueChange={(value) => setFormData({ ...formData, status: value as RoomStatus })}
247 %                 >
248 %                   <SelectTrigger data-testid="select-status">
249 %                     <SelectValue />
250 %                   </SelectTrigger>
251 %                   <SelectContent>
252 %                     <SelectItem value="available">Disponible</SelectItem>
253 %                     <SelectItem value="occupied">Ocupada</SelectItem>
254 %                     <SelectItem value="dirty">Dirty</SelectItem>
255 %                     <SelectItem value="cleaning">Limpieza</SelectItem>
256 %                     <SelectItem value="maintenance">Mantenimiento</SelectItem>
257 %                     <SelectItem value="oos">Fuera de Servicio</SelectItem>
258 %                   </SelectContent>
259 %                 </Select>
260 %               </div>
261 %               <div className="grid gap-2">
262 %                 <Label htmlFor="notes">Notas</Label>
263 %                 <Textarea
264 %                   id="notes"
265 %                   value={formData.notes || ""}
266 %                   onChange={(e) => setFormData({ ...formData, notes: e.target.value })}
267 %                   placeholder="Notas adicionales..."
268 %                   data-testid="input-notes"
269 %                 />
270 %               </div>
271 %             </div>
272 %           <DialogFooter>
273 %             <Button type="button" variant="outline" onClick={() => onOpenChange(false)}>
274 %               Cancelar
275 %             </Button>
276 %             <Button type="submit" disabled={mutation.isPending} data-testid="button-submit-room">
```

client/src/pages/rooms.tsx (cont.)

```
277 %         {mutation.isPending ? "Guardando..." : isEditing ? "Guardar Cambios" : "Crear Habitación"}
278 %       </Button>
279 %     </DialogFooter>
280 %   </form>
281 % </DialogContent>
282 % </Dialog>
283 % );
284 % }
285 %
286 % export default function RoomsPage() {
287 %   const { toast } = useToast();
288 %   const [searchQuery, setSearchQuery] = useState("");
289 %   const [statusFilter, setStatusFilter] = useState<string>("all");
290 %   const [typeFilter, setTypeFilter] = useState<string>("all");
291 %   const [viewMode, setViewMode] = useState<"grid" | "list">("grid");
292 %   const [dialogOpen, setDialogOpen] = useState(false);
293 %   const [selectedRoom, setSelectedRoom] = useState<RoomWithType | undefined>();
294 %
295 %   const { data: rooms, isLoading } = useQuery<RoomWithType[]>({
296 %     queryKey: ["/api/rooms"],
297 %   });
298 %
299 %   const { data: roomTypes } = useQuery<RoomType[]>({
300 %     queryKey: ["/api/room-types"],
301 %   });
302 %
303 %   const updateStatusMutation = useMutation({
304 %     mutationFn: async ({ id, status }: { id: string; status: RoomStatus }) => {
305 %       return apiRequest("PATCH", `/api/rooms/${id}`, { status });
306 %     },
307 %     onSuccess: () => {
308 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
309 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
310 %       toast({ title: "Estado actualizado", description: "El estado de la habitación ha sido actualizado." });
311 %     },
312 %   });
313 %
314 %   const deleteMutation = useMutation({
315 %     mutationFn: async (id: string) => {
316 %       return apiRequest("DELETE", `/api/rooms/${id}`, undefined);
317 %     },
318 %     onSuccess: () => {
319 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
320 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
321 %       toast({ title: "Habitación eliminada", description: "La habitación ha sido eliminada del sistema." });
322 %     },
323 %   });
324 %
325 %   const filteredRooms = rooms?.filter((room) => {
326 %     const matchesSearch = room.roomNumber.toLowerCase().includes(searchQuery.toLowerCase());
327 %     const matchesStatus = statusFilter === "all" || room.status === statusFilter;
328 %     const matchesType = typeFilter === "all" || room.roomTypeId === typeFilter;
329 %     return matchesSearch && matchesStatus && matchesType;
330 %   });
331 %
332 %   const handleEditRoom = (room: RoomWithType) => {
333 %     setSelectedRoom(room);
334 %     setDialogOpen(true);
335 %   };
336 %
337 %   const handleNewRoom = () => {
338 %     setSelectedRoom(undefined);
339 %     setDialogOpen(true);
340 %   };
341 %
342 %   const statusCounts = {
343 %     all: rooms?.length || 0,
344 %     available: rooms?.filter((r) => r.status === "available").length || 0,
345 %     occupied: rooms?.filter((r) => r.status === "occupied").length || 0,
346 %     dirty: rooms?.filter((r) => r.status === "dirty").length || 0,
347 %     cleaning: rooms?.filter((r) => r.status === "cleaning").length || 0,
348 %     maintenance: rooms?.filter((r) => r.status === "maintenance").length || 0,
349 %     oos: rooms?.filter((r) => r.status === "oos").length || 0,
350 %   };
351 %
352 %   return (
353 %     <div className="flex flex-col gap-6 p-6">
354 %       <div className="flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-between">
355 %         <div>
356 %           <h1 className="text-3xl font-bold tracking-tight" data-testid="text-rooms-title">
357 %             Habitaciones
358 %           </h1>
359 %           <p className="text-muted-foreground">Gestiona el inventario de habitaciones del hotel</p>
360 %         </div>
361 %         <Button onClick={handleNewRoom} data-testid="button-new-room">
362 %           <Plus className="mr-2 h-4 w-4" />
363 %           Nueva Habitación
364 %         </Button>
365 %       </div>
366 %
367 %       </* Status Summary Cards */>
368 %       <div className="grid gap-4 md:grid-cols-5">
369 %         <[
```

client/src/pages/rooms.tsx (cont.)

```
370 %     { key: "all", label: "Total", color: "bg-muted" },
371 %     { key: "available", label: "Disponibles", color: "bg-green-100 dark:bg-green-900/30" },
372 %     { key: "occupied", label: "Ocupadas", color: "bg-blue-100 dark:bg-blue-900/30" },
373 %     { key: "cleaning", label: "Limpieza", color: "bg-yellow-100 dark:bg-yellow-900/30" },
374 %     { key: "maintenance", label: "Mantenimiento", color: "bg-red-100 dark:bg-red-900/30" },
375 %   ].map(({ key, label, color }) => (
376 %     <Card
377 %       key={key}
378 %       className={`cursor-pointer hover-elevate ${statusFilter === key ? "ring-2 ring-primary" : ""}`}
379 %       onClick={() => setStatusFilter(key)}
380 %     >
381 %       <CardContent className={`p-4 ${color} rounded-lg`} >
382 %         <div className="text-2xl font-bold">{statusCounts[key as keyof typeof statusCounts]}</div>
383 %         <div className="text-sm text-muted-foreground">{label}</div>
384 %       </CardContent>
385 %     </Card>
386 %   ))
387 % </div>
388 %
389 % /* Filters */
390 % <Card>
391 %   <CardContent className="p-4">
392 %     <div className="flex flex-col gap-4 sm:flex-row sm:items-center">
393 %       <div className="relative flex-1">
394 %         <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
395 %         <Input
396 %           placeholder="Buscar por número de habitación..."
397 %           value={searchQuery}
398 %           onChange={(e) => setSearchQuery(e.target.value)}
399 %           className="pl-10"
400 %           data-testid="input-search-rooms"
401 %         />
402 %       </div>
403 %       <Select value={typeFilter} onValueChange={setTypeFilter}>
404 %         <SelectTrigger className="w-[180px]" data-testid="select-filter-type">
405 %           <Filter className="mr-2 h-4 w-4" />
406 %           <SelectValue placeholder="Tipo" />
407 %         </SelectTrigger>
408 %         <SelectContent>
409 %           <SelectItem value="all">Todos los tipos</SelectItem>
410 %           {roomTypes?.map((type) => (
411 %             <SelectItem key={type.id} value={type.id}>
412 %               {type.name}
413 %             </SelectItem>
414 %           ))}
415 %         </SelectContent>
416 %       </Select>
417 %       <div className="flex gap-1">
418 %         <Button
419 %           variant={viewMode === "grid" ? "default" : "outline"}
420 %           size="icon"
421 %           onClick={() => setViewMode("grid")}
422 %           data-testid="button-view-grid"
423 %         >
424 %           <DoorOpen className="h-4 w-4" />
425 %         </Button>
426 %         <Button
427 %           variant={viewMode === "list" ? "default" : "outline"}
428 %           size="icon"
429 %           onClick={() => setViewMode("list")}
430 %           data-testid="button-view-list"
431 %         >
432 %           <MoreHorizontal className="h-4 w-4" />
433 %         </Button>
434 %       </div>
435 %     </div>
436 %   </CardContent>
437 % </Card>
438 %
439 % /* Rooms Display */
440 % {isLoading ? (
441 %   <div className={viewMode === "grid" ? "grid gap-4 md:grid-cols-3 lg:grid-cols-4" : ""}>
442 %     {[...Array(8)].map((_, i) => (
443 %       <Skeleton key={i} className={viewMode === "grid" ? "h-32" : "h-16 mb-2"} />
444 %     ))}
445 %   </div>
446 % ) : filteredRooms && filteredRooms.length > 0 ? (
447 %   viewMode === "grid" ? (
448 %     <div className="grid gap-4 md:grid-cols-3 lg:grid-cols-4">
449 %       {filteredRooms.map((room) => (
450 %         <Card
451 %           key={room.id}
452 %           className={`hover-elevate cursor-pointer ${
453 %             room.status === "available"
454 %               ? "border-green-200 dark:border-green-800"
455 %               : room.status === "occupied"
456 %               ? "border-blue-200 dark:border-blue-800"
457 %               : room.status === "cleaning"
458 %               ? "border-yellow-200 dark:border-yellow-800"
459 %               : "border-red-200 dark:border-red-800"
460 %             }}
461 %           data-testid={`room-card-${room.id}`}
462 %         >
```

client/src/pages/rooms.tsx (cont.)

```
463 %         <CardHeader className="pb-2">
464 %           <div className="flex items-center justify-between gap-2">
465 %             <CardTitle className="text-2xl">{room.roomNumber}</CardTitle>
466 %             <DropdownMenu>
467 %               <DropdownMenuTrigger asChild>
468 %                 <Button variant="ghost" size="icon">
469 %                   <MoreHorizontal className="h-4 w-4" />
470 %                 </Button>
471 %               </DropdownMenuTrigger>
472 %               <DropdownMenuContent align="end">
473 %                 <DropdownMenuItem onClick={() => handleEditRoom(room)}>
474 %                   <Pencil className="mr-2 h-4 w-4" />
475 %                   Editar
476 %                 </DropdownMenuItem>
477 %                 <DropdownMenuSeparator />
478 %                 <DropdownMenuItem
479 %                   onClick={() => updateStatusMutation.mutate({ id: room.id, status: "available" })}
480 %                 >
481 %                   <CheckCircle className="mr-2 h-4 w-4 text-green-600" />
482 %                   Marcar Disponible
483 %                 </DropdownMenuItem>
484 %                 <DropdownMenuItem
485 %                   onClick={() => updateStatusMutation.mutate({ id: room.id, status: "cleaning" })}
486 %                 >
487 %                   <Sparkles className="mr-2 h-4 w-4 text-yellow-600" />
488 %                   Enviar a Limpieza
489 %                 </DropdownMenuItem>
490 %                 <DropdownMenuItem
491 %                   onClick={() => updateStatusMutation.mutate({ id: room.id, status: "maintenance" })}
492 %                 >
493 %                   <Wrench className="mr-2 h-4 w-4 text-red-600" />
494 %                   Enviar a Mantenimiento
495 %                 </DropdownMenuItem>
496 %                 <DropdownMenuSeparator />
497 %                 <DropdownMenuItem
498 %                   className="text-destructive"
499 %                   onClick={() => deleteMutation.mutate(room.id)}
500 %                 >
501 %                   <Trash2 className="mr-2 h-4 w-4" />
502 %                   Eliminar
503 %                 </DropdownMenuItem>
504 %               </DropdownMenuContent>
505 %             </DropdownMenu>
506 %           </div>
507 %           <CardDescription>Piso {room.floor}</CardDescription>
508 %         </CardHeader>
509 %         <CardContent>
510 %           <div className="flex flex-col gap-2">
511 %             <div className="flex items-center gap-2 flex-wrap">
512 %               <RoomStatusBadge status={room.status} />
513 %               <span className="text-sm text-muted-foreground">
514 %                 {room.roomType?.name || "Sin tipo"}
515 %               </span>
516 %             </div>
517 %             <RoomFeatures features={room.features} maxOccupancy={room.maxOccupancy} />
518 %           </div>
519 %         </CardContent>
520 %       </Card>
521 %     )}
522 %   </div>
523 % ) : (
524 %   <Card>
525 %     <Table>
526 %       <TableHeader>
527 %         <TableRow>
528 %           <TableHead>Numero</TableHead>
529 %           <TableHead>Tipo</TableHead>
530 %           <TableHead>Piso</TableHead>
531 %           <TableHead>Pax</TableHead>
532 %           <TableHead>Caracteristicas</TableHead>
533 %           <TableHead>Estado</TableHead>
534 %           <TableHead className="w-[50px]"></TableHead>
535 %         </TableRow>
536 %       </TableHeader>
537 %       <TableBody>
538 %         {filteredRooms.map((room) => (
539 %           <TableRow key={room.id} data-testid={`room-row-${room.id}`}>
540 %             <TableCell className="font-medium">{room.roomNumber}</TableCell>
541 %             <TableCell>{room.roomType?.name || "Sin tipo"></TableCell>
542 %             <TableCell>{room.floor}</TableCell>
543 %             <TableCell>{room.maxOccupancy || 2}</TableCell>
544 %             <TableCell>
545 %               <RoomFeatures features={room.features} />
546 %             </TableCell>
547 %             <TableCell>
548 %               <RoomStatusBadge status={room.status} />
549 %             </TableCell>
550 %             <TableCell>
551 %               <DropdownMenu>
552 %                 <DropdownMenuTrigger asChild>
553 %                   <Button variant="ghost" size="icon">
554 %                     <MoreHorizontal className="h-4 w-4" />
555 %                   </Button>
```

client/src/pages/rooms.tsx (cont.)

```
556 %         </DropdownMenuTrigger>
557 %     <DropdownMenuContent align="end">
558 %         <DropdownMenuItem onClick={() => handleEditRoom(room)}>
559 %             <Pencil className="mr-2 h-4 w-4" />
560 %             Editar
561 %         </DropdownMenuItem>
562 %         <DropdownMenuSeparator />
563 %         <DropdownMenuItem
564 %             onClick={() => updateStatusMutation.mutate({ id: room.id, status: "available" })}
565 %         >
566 %             <CheckCircle className="mr-2 h-4 w-4 text-green-600" />
567 %             Marcar Disponible
568 %         </DropdownMenuItem>
569 %         <DropdownMenuItem
570 %             onClick={() => updateStatusMutation.mutate({ id: room.id, status: "cleaning" })}
571 %         >
572 %             <Sparkles className="mr-2 h-4 w-4 text-yellow-600" />
573 %             Enviar a Limpieza
574 %         </DropdownMenuItem>
575 %         <DropdownMenuItem
576 %             onClick={() => updateStatusMutation.mutate({ id: room.id, status: "maintenance" })}
577 %         >
578 %             <Wrench className="mr-2 h-4 w-4 text-red-600" />
579 %             Enviar a Mantenimiento
580 %         </DropdownMenuItem>
581 %         <DropdownMenuSeparator />
582 %         <DropdownMenuItem
583 %             className="text-destructive"
584 %             onClick={() => deleteMutation.mutate(room.id)}
585 %         >
586 %             <Trash2 className="mr-2 h-4 w-4" />
587 %             Eliminar
588 %         </DropdownMenuItem>
589 %     </DropdownMenuContent>
590 % </DropdownMenu>
591 % </TableCell>
592 % </TableRow>
593 %     )}
594 % </TableBody>
595 % </Table>
596 % </Card>
597 % )
598 % ) : (
599 %     <Card>
600 %         <CardContent className="flex flex-col items-center justify-center py-12 text-center">
601 %             <DoorOpen className="h-16 w-16 text-muted-foreground/50 mb-4" />
602 %             <h3 className="text-lg font-semibold mb-2">No hay habitaciones</h3>
603 %             <p className="text-muted-foreground mb-4">
604 %                 {searchQuery || statusFilter !== "all" || typeFilter !== "all"
605 %                     ? "No se encontraron habitaciones con los filtros aplicados."
606 %                     : "Comienza agregando la primera habitación del hotel."}
607 %             </p>
608 %             {!searchQuery && statusFilter === "all" && typeFilter === "all" && (
609 %                 <Button onClick={handleNewRoom}>
610 %                     <Plus className="mr-2 h-4 w-4" />
611 %                     Agregar Primera Habitación
612 %                 </Button>
613 %             )}
614 %         </CardContent>
615 %     </Card>
616 % )}
617 %
618 % {/* Room Form Dialog */}
619 % <RoomFormDialog
620 %     room={selectedRoom}
621 %     roomTypes={roomTypes || []}
622 %     open={dialogOpen}
623 %     onOpenChange={setDialogOpen}
624 %     onSuccess={() => setSelectedRoom(undefined)}
625 % />
626 % </div>
627 % );
628 % }
629 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import {
4 %   Users,
5 %   Plus,
6 %   Search,
7 %   MoreHorizontal,
8 %   Pencil,
9 %   Trash2,
10 %   Eye,
11 %   Mail,
12 %   Phone,
13 %   MapPin,
14 %   FileText,
15 %   Car,
16 %   Heart,
17 %   Calendar,
18 % } from "lucide-react";
19 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
20 % import { Button } from "@components/ui/button";
21 % import { Input } from "@components/ui/input";
22 % import { Skeleton } from "@components/ui/skeleton";
23 % import {
24 %   Dialog,
25 %   DialogContent,
26 %   DialogDescription,
27 %   DialogFooter,
28 %   DialogHeader,
29 %   DialogTitle,
30 % } from "@components/ui/dialog";
31 % import {
32 %   DropdownMenu,
33 %   DropdownMenuContent,
34 %   DropdownMenuItem,
35 %   DropdownMenuSeparator,
36 %   DropdownMenuTrigger,
37 % } from "@components/ui/dropdown-menu";
38 % import {
39 %   Select,
40 %   SelectContent,
41 %   SelectItem,
42 %   SelectTrigger,
43 %   SelectValue,
44 % } from "@components/ui/select";
45 % import {
46 %   Table,
47 %   TableBody,
48 %   TableCell,
49 %   TableHead,
50 %   TableHeader,
51 %   TableRow,
52 % } from "@components/ui/table";
53 % import { Label } from "@components/ui/label";
54 % import { useToast } from "@hooks/use-toast";
55 % import { queryClient, apiRequest } from "@lib/queryClient";
56 % import type { Guest, InsertGuest, ReservationWithDetails, ReservationStatus, GuestPreference } from "@shared/schema";
57 % import { Badge } from "@components/ui/badge";
58 %
59 % function GuestFormDialog({
60 %   guest,
61 %   open,
62 %   onOpenChange,
63 %   onSuccess,
64 % }: {
65 %   guest?: Guest;
66 %   open: boolean;
67 %   onOpenChange: (open: boolean) => void;
68 %   onSuccess: () => void;
69 % }) {
70 %   const { toast } = useToast();
71 %   const isEditing = !!guest;
72 %
73 %   const [formData, setFormData] = useState<Partial<InsertGuest>>({
74 %     firstName: guest?.firstName || "",
75 %     lastName: guest?.lastName || "",
76 %     email: guest?.email || "",
77 %     phone: guest?.phone || "",
78 %     documentType: guest?.documentType || "dni",
79 %     documentNumber: guest?.documentNumber || "",
80 %     nationality: guest?.nationality || "",
81 %     localidad: guest?.localidad || "",
82 %     vehiculoPatente: guest?.vehiculoPatente || "",
83 %     vehiculoMarca: guest?.vehiculoMarca || "",
84 %     vehiculoModelo: guest?.vehiculoModelo || "",
85 %     vehiculoColor: guest?.vehiculoColor || "",
86 %   });
87 %
88 %   const mutation = useMutation({
89 %     mutationFn: async (data: Partial<InsertGuest>) => {
90 %       if (isEditing) {
```

client/src/pages/guests.tsx (cont.)

```
91 %         return apiRequest("PATCH", `/api/guests/${guest.id}`, data);
92 %     }
93 %     return apiRequest("POST", "/api/guests", data);
94 % },
95 % onSuccess: () => {
96 %     queryClient.invalidateQueries({ queryKey: ["/api/guests"] });
97 %     queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
98 %     toast({
99 %         title: isEditing ? "Huésped actualizado" : "Huésped registrado",
100 %         description: `${formData.firstName} ${formData.lastName} ha sido ${isEditing ? "actualizado" : "registrado"} ...
101 %     });
102 %     onSuccess();
103 %     onOpenChange(false);
104 % },
105 % onError: () => {
106 %     toast({
107 %         title: "Error",
108 %         description: "No se pudo guardar el huésped. Intente nuevamente.",
109 %         variant: "destructive",
110 %     });
111 % },
112 % });
113 %
114 % const handleSubmit = (e: React.FormEvent) => {
115 %     e.preventDefault();
116 %     mutation.mutate(formData);
117 % };
118 %
119 % return (
120 %     <Dialog open={open} onOpenChange={onOpenChange}>
121 %         <DialogContent className="sm:max-w-[500px] max-h-[90vh] overflow-y-auto">
122 %             <DialogHeader>
123 %                 <DialogTitle>{isEditing ? "Editar Huésped" : "Nuevo Huésped"}</DialogTitle>
124 %                 <DialogDescription>
125 %                     {isEditing ? "Modifica los datos del huésped." : "Ingresa los datos para registrar un nuevo huésped."}
126 %                 </DialogDescription>
127 %             </DialogHeader>
128 %             <form onSubmit={handleSubmit}>
129 %                 <div className="grid gap-4 py-4">
130 %                     <div className="grid grid-cols-2 gap-4">
131 %                         <div className="grid gap-2">
132 %                             <Label htmlFor="firstName">Nombre</Label>
133 %                             <Input
134 %                                 id="firstName"
135 %                                 value={formData.firstName}
136 %                                 onChange={(e) => setFormData({ ...formData, firstName: e.target.value })}
137 %                                 placeholder="Juan"
138 %                                 required
139 %                                 data-testid="input-first-name"
140 %                             />
141 %                         </div>
142 %                         <div className="grid gap-2">
143 %                             <Label htmlFor="lastName">Apellido</Label>
144 %                             <Input
145 %                                 id="lastName"
146 %                                 value={formData.lastName}
147 %                                 onChange={(e) => setFormData({ ...formData, lastName: e.target.value })}
148 %                                 placeholder="Pérez"
149 %                                 required
150 %                                 data-testid="input-last-name"
151 %                             />
152 %                         </div>
153 %                     </div>
154 %                     <div className="grid grid-cols-2 gap-4">
155 %                         <div className="grid gap-2">
156 %                             <Label htmlFor="email">Email</Label>
157 %                             <Input
158 %                                 id="email"
159 %                                 type="email"
160 %                                 value={formData.email || ""}
161 %                                 onChange={(e) => setFormData({ ...formData, email: e.target.value })}
162 %                                 placeholder="juan@email.com"
163 %                                 data-testid="input-email"
164 %                             />
165 %                         </div>
166 %                         <div className="grid gap-2">
167 %                             <Label htmlFor="phone">Teléfono</Label>
168 %                             <Input
169 %                                 id="phone"
170 %                                 value={formData.phone || ""}
171 %                                 onChange={(e) => setFormData({ ...formData, phone: e.target.value })}
172 %                                 placeholder="+54 11 1234-5678"
173 %                                 data-testid="input-phone"
174 %                             />
175 %                         </div>
176 %                     </div>
177 %                     <div className="grid grid-cols-2 gap-4">
178 %                         <div className="grid gap-2">
179 %                             <Label htmlFor="documentType">Tipo de Documento</Label>
180 %                             <Select
181 %                                 value={formData.documentType || "dni"}
182 %                                 onValueChange={(value) => setFormData({ ...formData, documentType: value })}
183 %                             />
184 %                         </div>
185 %                     </div>
186 %                 </div>
187 %             </form>
188 %         </DialogContent>
189 %     </Dialog>
190 % );
```

```

184 %         <SelectTrigger data-testid="select-document-type">
185 %             <SelectValue />
186 %         </SelectTrigger>
187 %         <SelectContent>
188 %             <SelectItem value="dni">DNI</SelectItem>
189 %             <SelectItem value="passport">Pasaporte</SelectItem>
190 %             <SelectItem value="cedula">Cédula</SelectItem>
191 %             <SelectItem value="other">Otro</SelectItem>
192 %         </SelectContent>
193 %     </Select>
194 % </div>
195 % <div className="grid gap-2">
196 %     <Label htmlFor="documentNumber">Número de Documento</Label>
197 %     <Input
198 %         id="documentNumber"
199 %         value={formData.documentNumber || ""}
200 %         onChange={(e) => setFormData({ ...formData, documentNumber: e.target.value })}
201 %         placeholder="12345678"
202 %         data-testid="input-document-number"
203 %     />
204 % </div>
205 % </div>
206 % <div className="grid gap-2">
207 %     <Label htmlFor="nationality">Nacionalidad</Label>
208 %     <Input
209 %         id="nationality"
210 %         value={formData.nationality || ""}
211 %         onChange={(e) => setFormData({ ...formData, nationality: e.target.value })}
212 %         placeholder="Argentina"
213 %         data-testid="input-nationality"
214 %     />
215 % </div>
216 % <div className="grid gap-2">
217 %     <Label htmlFor="localidad">Ciudad / Localidad</Label>
218 %     <Input
219 %         id="localidad"
220 %         value={formData.localidad || ""}
221 %         onChange={(e) => setFormData({ ...formData, localidad: e.target.value })}
222 %         placeholder="Buenos Aires"
223 %         data-testid="input-localidad"
224 %     />
225 % </div>
226 %
227 % <div className="pt-2 border-t">
228 %     <Label className="text-sm font-medium text-muted-foreground">Datos del Vehículo (opcional)</Label>
229 % </div>
230 % <div className="grid grid-cols-2 gap-4">
231 %     <div className="grid gap-2">
232 %         <Label htmlFor="vehiculoPatente">Patente</Label>
233 %         <Input
234 %             id="vehiculoPatente"
235 %             name="vehiculoPatente"
236 %             value={formData.vehiculoPatente || ""}
237 %             onChange={(e) => setFormData({ ...formData, vehiculoPatente: e.target.value.toUpperCase() })}
238 %             placeholder="ABC 123"
239 %             data-testid="input-vehiculo-patente"
240 %         />
241 %     </div>
242 %     <div className="grid gap-2">
243 %         <Label htmlFor="vehiculoMarca">Marca</Label>
244 %         <Input
245 %             id="vehiculoMarca"
246 %             name="vehiculoMarca"
247 %             value={formData.vehiculoMarca || ""}
248 %             onChange={(e) => setFormData({ ...formData, vehiculoMarca: e.target.value })}
249 %             placeholder="Toyota"
250 %             data-testid="input-vehiculo-marca"
251 %         />
252 %     </div>
253 % </div>
254 % <div className="grid grid-cols-2 gap-4">
255 %     <div className="grid gap-2">
256 %         <Label htmlFor="vehiculoModelo">Modelo</Label>
257 %         <Input
258 %             id="vehiculoModelo"
259 %             name="vehiculoModelo"
260 %             value={formData.vehiculoModelo || ""}
261 %             onChange={(e) => setFormData({ ...formData, vehiculoModelo: e.target.value })}
262 %             placeholder="Corolla"
263 %             data-testid="input-vehiculo-modelo"
264 %         />
265 %     </div>
266 %     <div className="grid gap-2">
267 %         <Label htmlFor="vehiculoColor">Color</Label>
268 %         <Input
269 %             id="vehiculoColor"
270 %             name="vehiculoColor"
271 %             value={formData.vehiculoColor || ""}
272 %             onChange={(e) => setFormData({ ...formData, vehiculoColor: e.target.value })}
273 %             placeholder="Blanco"
274 %             data-testid="input-vehiculo-color"
275 %         />
276 %     </div>

```


client/src/pages/guests.tsx (cont.)

```
277 %         </div>
278 %     </div>
279 %     <DialogFooter>
280 %         <Button type="button" variant="outline" onClick={() => onOpenChange(false)}>
281 %             Cancelar
282 %         </Button>
283 %         <Button type="submit" disabled={mutation.isPending} data-testid="button-submit-guest">
284 %             {mutation.isPending ? "Guardando..." : isEditing ? "Guardar Cambios" : "Registrar Huésped"}
285 %         </Button>
286 %     </DialogFooter>
287 % </form>
288 % </DialogContent>
289 % </Dialog>
290 % );
291 % }
292 %
293 % function ReservationStatusBadge({ status }: { status: ReservationStatus }) {
294 %     const statusConfig: Record<ReservationStatus, { label: string; variant: "default" | "secondary" | "destructive" | "...
295 %     pending: { label: "Pendiente", variant: "secondary" },
296 %     tentative: { label: "Tentativa", variant: "secondary" },
297 %     confirmed: { label: "Confirmada", variant: "default" },
298 %     checked_in: { label: "Hospedado", variant: "outline" },
299 %     checked_out: { label: "Finalizada", variant: "secondary" },
300 %     cancelled: { label: "Cancelada", variant: "destructive" },
301 % };
302 % const config = statusConfig[status];
303 % return <Badge variant={config.variant}>{config.label}</Badge>;
304 % }
305 %
306 % function GuestDetailDialog({
307 %     guest,
308 %     open,
309 %     onOpenChange,
310 % }): {
311 %     guest: Guest;
312 %     open: boolean;
313 %     onOpenChange: (open: boolean) => void;
314 % } {
315 %     const { data: allReservations } = useQuery<ReservationWithDetails[]>({
316 %         queryKey: ["/api/reservations"],
317 %     });
318 %
319 %     const { data: preferences = [] } = useQuery<GuestPreference[]>({
320 %         queryKey: ["/api/guests", guest.id, "preferences"],
321 %         enabled: open,
322 %     });
323 %
324 %     const activePreferences = preferences.filter((p) => p.isActive);
325 %
326 %     const guestReservations = allReservations?.filter(r => r.guestId === guest.id) || [];
327 %     const totalStays = guestReservations.filter(r => r.status === "checked_out").length;
328 %     const totalNights = guestReservations.reduce((sum, r) => sum + (r.nights || 0), 0);
329 %     const totalSpent = guestReservations.reduce((sum, r) => sum + parseFloat(r.totalRoomAmount || "0"), 0);
330 %
331 %     return (
332 %         <Dialog open={open} onOpenChange={onOpenChange}>
333 %             <DialogContent className="sm:max-w-[550px] max-h-[90vh] overflow-y-auto">
334 %                 <DialogHeader>
335 %                     <DialogTitle>Detalle del Huésped</DialogTitle>
336 %                     <DialogDescription>Información completa y historial de reservaciones.</DialogDescription>
337 %                 </DialogHeader>
338 %                 <div className="grid gap-4 py-4">
339 %                     <div className="flex items-center gap-4 p-4 bg-muted rounded-lg">
340 %                         <div className="flex h-14 w-14 items-center justify-center rounded-full bg-primary/10 text-primary font-b...
341 %                             {guest.firstName?.[0]}{guest.lastName?.[0]}
342 %                         </div>
343 %                         <div className="flex-1">
344 %                             <p className="font-semibold text-xl">
345 %                                 {guest.firstName} {guest.lastName}
346 %                             </p>
347 %                             <p className="text-sm text-muted-foreground">{guest.nationality || "Sin nacionalidad"}</p>
348 %                         </div>
349 %                     </div>
350 %
351 %                     <div className="grid grid-cols-3 gap-3 text-center">
352 %                         <div className="p-3 border rounded-lg">
353 %                             <p className="text-2xl font-bold text-primary">{totalStays}</p>
354 %                             <p className="text-sm text-muted-foreground">Estancias</p>
355 %                         </div>
356 %                         <div className="p-3 border rounded-lg">
357 %                             <p className="text-2xl font-bold text-primary">{totalNights}</p>
358 %                             <p className="text-sm text-muted-foreground">Noches</p>
359 %                         </div>
360 %                         <div className="p-3 border rounded-lg">
361 %                             <p className="text-2xl font-bold text-primary">${totalSpent.toFixed(0)}</p>
362 %                             <p className="text-sm text-muted-foreground">Gastado</p>
363 %                         </div>
364 %                     </div>
365 %
366 %                     <div className="space-y-3">
367 %                         <div className="grid grid-cols-2 gap-3">
368 %                             {guest.email} && (
369 %                                 <div className="flex items-center gap-3 p-3 border rounded-lg">
```

```

370 %         <Mail className="h-4 w-4 text-muted-foreground" />
371 %         <div className="overflow-hidden">
372 %           <p className="text-xs text-muted-foreground">Email</p>
373 %           <p className="font-medium text-sm truncate">{guest.email}</p>
374 %         </div>
375 %       </div>
376 %     )}
377 %     {guest.phone && (
378 %       <div className="flex items-center gap-3 p-3 border rounded-lg">
379 %         <Phone className="h-4 w-4 text-muted-foreground" />
380 %         <div>
381 %           <p className="text-xs text-muted-foreground">Teléfono</p>
382 %           <p className="font-medium text-sm">{guest.phone}</p>
383 %         </div>
384 %       </div>
385 %     )}
386 %   </div>
387 %   <div className="grid grid-cols-2 gap-3">
388 %     {guest.documentNumber && (
389 %       <div className="flex items-center gap-3 p-3 border rounded-lg">
390 %         <FileText className="h-4 w-4 text-muted-foreground" />
391 %         <div>
392 %           <p className="text-xs text-muted-foreground">
393 %             {guest.documentType?.toUpperCase() || "Documento"}
394 %           </p>
395 %           <p className="font-medium text-sm">{guest.documentNumber}</p>
396 %         </div>
397 %       </div>
398 %     )}
399 %     {guest.localidad && (
400 %       <div className="flex items-center gap-3 p-3 border rounded-lg">
401 %         <MapPin className="h-4 w-4 text-muted-foreground" />
402 %         <div className="overflow-hidden">
403 %           <p className="text-xs text-muted-foreground">Ciudad</p>
404 %           <p className="font-medium text-sm truncate">{guest.localidad}</p>
405 %         </div>
406 %       </div>
407 %     )}
408 %   </div>
409 % </div>
410 %
411 % {guest.vehiculoPatente && (
412 %   <div className="border rounded-lg p-3">
413 %     <div className="flex items-center gap-2 mb-2">
414 %       <Car className="h-4 w-4 text-muted-foreground" />
415 %       <span className="font-medium text-sm">Vehículo</span>
416 %     </div>
417 %     <div className="grid grid-cols-2 gap-2 text-sm">
418 %       <div>
419 %         <p className="text-xs text-muted-foreground">Patente</p>
420 %         <p className="font-medium">{guest.vehiculoPatente}</p>
421 %       </div>
422 %       {guest.vehiculoMarca && (
423 %         <div>
424 %           <p className="text-xs text-muted-foreground">Marca</p>
425 %           <p className="font-medium">{guest.vehiculoMarca}</p>
426 %         </div>
427 %       )}
428 %       {guest.vehiculoModelo && (
429 %         <div>
430 %           <p className="text-xs text-muted-foreground">Modelo</p>
431 %           <p className="font-medium">{guest.vehiculoModelo}</p>
432 %         </div>
433 %       )}
434 %       {guest.vehiculoColor && (
435 %         <div>
436 %           <p className="text-xs text-muted-foreground">Color</p>
437 %           <p className="font-medium">{guest.vehiculoColor}</p>
438 %         </div>
439 %       )}
440 %     </div>
441 %   </div>
442 % )}
443 %
444 % {activePreferences.length > 0 && (
445 %   <div className="border rounded-lg" data-testid="guest-preferences-section">
446 %     <div className="p-3 border-b bg-muted/50 flex items-center gap-2">
447 %       <Heart className="h-4 w-4 text-primary" />
448 %       <h4 className="font-semibold">Preferencias de Hospitalidad</h4>
449 %       <Badge variant="outline" className="text-xs">{activePreferences.length}</Badge>
450 %     </div>
451 %     <div className="divide-y max-h-[200px] overflow-y-auto">
452 %       {activePreferences.map((pref) => (
453 %         <div key={pref.id} className="flex items-center justify-between p-3 text-sm" data-testid={`guest-pr...`}>
454 %           <div>
455 %             <p className="font-medium">{pref.title}</p>
456 %             {pref.description && <p className="text-xs text-muted-foreground">{pref.description}</p>}
457 %           </div>
458 %           <Badge
459 %             variant="outline"
460 %             className={ `text-xs ${
461 %               pref.priority === "critical" ? "border-red-400 text-red-700 dark:text-red-300" :
462 %               pref.priority === "high" ? "border-orange-400 text-orange-700 dark:text-orange-300" :

```

client/src/pages/guests.tsx (cont.)

```
463 %         ""
464 %       }`}`
465 %     >
466 %     {pref.priority === "critical" ? "Crítica" : pref.priority === "high" ? "Alta" : pref.priority =...
467 %   </Badge>
468 % </div>
469 %   )}}
470 % </div>
471 % </div>
472 % })
473 %
474 % <div className="border rounded-lg">
475 %   <div className="p-3 border-b bg-muted/50">
476 %     <h4 className="font-semibold">Historial de Reservaciones</h4>
477 %   </div>
478 %   <div className="divide-y max-h-[200px] overflow-y-auto">
479 %     {guestReservations.length > 0 ? (
480 %       guestReservations.map((reservation) => (
481 %         <div key={reservation.id} className="flex items-center justify-between p-3 text-sm" data-testid={`g...
482 %           <div className="flex items-center gap-2">
483 %             <Calendar className="h-4 w-4 text-muted-foreground" />
484 %             <div>
485 %               <p className="font-medium">Hab. {reservation.room?.roomNumber}</p>
486 %               <p className="text-muted-foreground text-xs">
487 %                 {reservation.checkInDate} - {reservation.checkOutDate}
488 %               </p>
489 %             </div>
490 %           </div>
491 %           <div className="flex items-center gap-2">
492 %             <span className="font-medium">${reservation.totalRoomAmount || 0}</span>
493 %             <ReservationStatusBadge status={reservation.status} />
494 %           </div>
495 %         </div>
496 %       )
497 %     ) : (
498 %       <div className="p-4 text-center text-muted-foreground text-sm">
499 %         No hay reservaciones registradas
500 %       </div>
501 %     )
502 %   </div>
503 % </div>
504 % </div>
505 % <DialogFooter>
506 %   <Button variant="outline" onClick={() => onOpenChange(false)}>
507 %     Cerrar
508 %   </Button>
509 % </DialogFooter>
510 % </DialogContent>
511 % </Dialog>
512 % );
513 % }
514 %
515 % export default function GuestsPage() {
516 %   const { toast } = useToast();
517 %   const [searchQuery, setSearchQuery] = useState("");
518 %   const [dialogOpen, setDialogOpen] = useState(false);
519 %   const [detailDialogOpen, setDetailDialogOpen] = useState(false);
520 %   const [selectedGuest, setSelectedGuest] = useState<Guest | undefined>();
521 %
522 %   const { data: guests, isLoading } = useQuery<Guest[]>({
523 %     queryKey: ["/api/guests"],
524 %   });
525 %
526 %   const deleteMutation = useMutation({
527 %     mutationFn: async (id: string) => {
528 %       return apiRequest("DELETE", `/api/guests/${id}`, undefined);
529 %     },
530 %     onSuccess: () => {
531 %       queryClient.invalidateQueries({ queryKey: ["/api/guests"] });
532 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
533 %       toast({ title: "Huésped eliminado", description: "El huésped ha sido eliminado del sistema." });
534 %     },
535 %   });
536 %
537 %   const filteredGuests = guests?.filter((guest) => {
538 %     const fullName = `${guest.firstName} ${guest.lastName}`.toLowerCase();
539 %     const matchesSearch =
540 %       fullName.includes(searchQuery.toLowerCase()) ||
541 %       guest.email?.toLowerCase().includes(searchQuery.toLowerCase()) ||
542 %       guest.documentNumber?.includes(searchQuery);
543 %     return matchesSearch;
544 %   });
545 %
546 %   const handleEditGuest = (guest: Guest) => {
547 %     setSelectedGuest(guest);
548 %     setDialogOpen(true);
549 %   };
550 %
551 %   const handleViewGuest = (guest: Guest) => {
552 %     setSelectedGuest(guest);
553 %     setDetailDialogOpen(true);
554 %   };
555 % }
```


client/src/pages/guests.tsx (cont.)

```
649 %           <DropdownMenuItem onClick={() => handleEditGuest(guest)}>
650 %             <Pencil className="mr-2 h-4 w-4" />
651 %             Editar
652 %           </DropdownMenuItem>
653 %           <DropdownMenuSeparator />
654 %           <DropdownMenuItem
655 %             className="text-destructive"
656 %             onClick={() => deleteMutation.mutate(guest.id)}
657 %           >
658 %             <Trash2 className="mr-2 h-4 w-4" />
659 %             Eliminar
660 %           </DropdownMenuItem>
661 %         </DropdownMenuContent>
662 %       </DropdownMenu>
663 %     </TableCell>
664 %   </TableRow>
665 % ))}
666 % </TableBody>
667 % </Table>
668 % </Card>
669 % ) : (
670 %   <Card>
671 %     <CardContent className="flex flex-col items-center justify-center py-12 text-center">
672 %       <Users className="h-16 w-16 text-muted-foreground/50 mb-4" />
673 %       <h3 className="text-lg font-semibold mb-2">No hay huéspedes</h3>
674 %       <p className="text-muted-foreground mb-4">
675 %         {searchQuery
676 %           ? "No se encontraron huéspedes con los criterios de búsqueda."
677 %           : "Comienza registrando el primer huésped del hotel."}
678 %       </p>
679 %       {!searchQuery && (
680 %         <Button onClick={handleNewGuest}>
681 %           <Plus className="mr-2 h-4 w-4" />
682 %           Registrar Primer Huésped
683 %         </Button>
684 %       )}
685 %     </CardContent>
686 %   </Card>
687 % )}
688 %
689 % {/* Guest Form Dialog */}
690 % <GuestFormDialog
691 %   guest={selectedGuest}
692 %   open={dialogOpen}
693 %   onOpenChange={setDialogOpen}
694 %   onSuccess={() => setSelectedGuest(undefined)}
695 % />
696 %
697 % {/* Guest Detail Dialog */}
698 % {selectedGuest && (
699 %   <GuestDetailDialog
700 %     guest={selectedGuest}
701 %     open={detailDialogOpen}
702 %     onOpenChange={setDetailDialogOpen}
703 %   />
704 % )}
705 % </div>
706 % );
707 % }
708 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { useForm } from "react-hook-form";
5 % import { zodResolver } from "@hookform/resolvers/zod";
6 % import { z } from "zod";
7 % import { Card, CardContent } from "@components/ui/card";
8 % import { Button } from "@components/ui/button";
9 % import { Badge } from "@components/ui/badge";
10 % import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter, DialogDescription } from "@components/ui/di...
11 % import { Input } from "@components/ui/input";
12 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
13 % import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
14 % import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@components/ui/table";
15 % import { useToast } from "@hooks/use-toast";
16 % import { Plus, Search, Building2, Pencil, Loader2, Trash2 } from "lucide-react";
17 % import { insertCompanySchema, type Company } from "@shared/schema";
18 %
19 % const companyFormSchema = insertCompanySchema.extend({
20 %   razonSocial: z.string().min(1, "Razón social requerida"),
21 %   cuilCuit: z.string().min(1, "CUIT requerido"),
22 % });
23 %
24 % type CompanyFormData = z.infer<typeof companyFormSchema>;
25 %
26 % export default function CompaniesPage() {
27 %   const { toast } = useToast();
28 %   const [showForm, setShowForm] = useState(false);
29 %   const [editingCompany, setEditingCompany] = useState<Company | null>(null);
30 %   const [searchTerm, setSearchTerm] = useState("");
31 %
32 %   const { data: companies = [], isLoading } = useQuery<Company[]>({
33 %     queryKey: ["/api/companies"],
34 %   });
35 %
36 %   const form = useForm<CompanyFormData>({
37 %     resolver: zodResolver(companyFormSchema),
38 %     defaultValues: {
39 %       razonSocial: "",
40 %       nombreFantasia: "",
41 %       cuilCuit: "",
42 %       condicionIva: "responsable_inscripto",
43 %       direccion: "",
44 %       localidad: "",
45 %       provincia: "",
46 %       codigoPostal: "",
47 %       telefono: "",
48 %       email: "",
49 %       contactName: "",
50 %       contactEmail: "",
51 %       contactPhone: "",
52 %       creditLimit: "0",
53 %       paymentTermDays: 30,
54 %       notes: "",
55 %     },
56 %   });
57 %
58 %   const createMutation = useMutation({
59 %     mutationFn: async (data: CompanyFormData) => {
60 %       const res = await apiRequest("POST", "/api/companies", data);
61 %       return res.json();
62 %     },
63 %     onSuccess: () => {
64 %       queryClient.invalidateQueries({ queryKey: ["/api/companies"] });
65 %       toast({ title: "Empresa creada correctamente" });
66 %       setShowForm(false);
67 %       form.reset();
68 %     },
69 %     onError: () => {
70 %       toast({ title: "Error al crear empresa", variant: "destructive" });
71 %     },
72 %   });
73 %
74 %   const updateMutation = useMutation({
75 %     mutationFn: async ({ id, data }: { id: string; data: CompanyFormData }) => {
76 %       const res = await apiRequest("PATCH", `/api/companies/${id}`, data);
77 %       return res.json();
78 %     },
79 %     onSuccess: () => {
80 %       queryClient.invalidateQueries({ queryKey: ["/api/companies"] });
81 %       toast({ title: "Empresa actualizada correctamente" });
82 %       setShowForm(false);
83 %       setEditingCompany(null);
84 %       form.reset();
85 %     },
86 %     onError: () => {
87 %       toast({ title: "Error al actualizar empresa", variant: "destructive" });
88 %     },
89 %   });
90 % }
```

client/src/pages/companies.tsx (cont.)

```
91 %   const deleteMutation = useMutation({
92 %     mutationFn: async (id: string) => {
93 %       await apiRequest("DELETE", `/api/companies/${id}`);
94 %     },
95 %     onSuccess: () => {
96 %       queryClient.invalidateQueries({ queryKey: ["/api/companies"] });
97 %       toast({ title: "Empresa eliminada correctamente" });
98 %     },
99 %     onError: () => {
100 %       toast({ title: "Error al eliminar empresa", variant: "destructive" });
101 %     },
102 %   });
103 %
104 %   const handleDelete = (company: Company) => {
105 %     if (window.confirm(`¿Estás seguro de eliminar "${company.razonSocial}"?`)) {
106 %       deleteMutation.mutate(company.id);
107 %     }
108 %   };
109 %
110 %   const filteredCompanies = companies.filter((c) =>
111 %     c.razonSocial.toLowerCase().includes(searchTerm.toLowerCase()) ||
112 %     c.nombreFantasia?.toLowerCase().includes(searchTerm.toLowerCase()) ||
113 %     c.cuilCuit?.toLowerCase().includes(searchTerm.toLowerCase()) ||
114 %     c.contactName?.toLowerCase().includes(searchTerm.toLowerCase())
115 %   );
116 %
117 %   const handleEdit = (company: Company) => {
118 %     setEditingCompany(company);
119 %     form.reset({
120 %       razonSocial: company.razonSocial,
121 %       nombreFantasia: company.nombreFantasia || "",
122 %       cuilCuit: company.cuilCuit || "",
123 %       condicionIva: company.condicionIva || "responsable_inscripto",
124 %       direccion: company.direccion || "",
125 %       localidad: company.localidad || "",
126 %       provincia: company.provincia || "",
127 %       codigoPostal: company.codigoPostal || "",
128 %       telefono: company.telefono || "",
129 %       email: company.email || "",
130 %       contactName: company.contactName || "",
131 %       contactEmail: company.contactEmail || "",
132 %       contactPhone: company.contactPhone || "",
133 %       creditLimit: company.creditLimit || "0",
134 %       paymentTermDays: company.paymentTermDays || 30,
135 %       notes: company.notes || "",
136 %     });
137 %     setShowForm(true);
138 %   };
139 %
140 %   const handleSubmit = (data: CompanyFormData) => {
141 %     if (editingCompany) {
142 %       updateMutation.mutate({ id: editingCompany.id, data });
143 %     } else {
144 %       createMutation.mutate(data);
145 %     }
146 %   };
147 %
148 %   const handleOpenNew = () => {
149 %     setEditingCompany(null);
150 %     form.reset({
151 %       razonSocial: "",
152 %       nombreFantasia: "",
153 %       cuilCuit: "",
154 %       condicionIva: "responsable_inscripto",
155 %       direccion: "",
156 %       localidad: "",
157 %       provincia: "",
158 %       codigoPostal: "",
159 %       telefono: "",
160 %       email: "",
161 %       contactName: "",
162 %       contactEmail: "",
163 %       contactPhone: "",
164 %       creditLimit: "0",
165 %       paymentTermDays: 30,
166 %       notes: "",
167 %     });
168 %     setShowForm(true);
169 %   };
170 %
171 %   if (isLoading) {
172 %     return (
173 %       <div className="flex items-center justify-center h-64">
174 %         <Loader2 className="h-8 w-8 animate-spin text-muted-foreground" />
175 %       </div>
176 %     );
177 %   }
178 %
179 %   return (
180 %     <div className="p-6 space-y-6" data-testid="page-companies">
181 %       <div className="flex items-center justify-between">
182 %         <div>
183 %           <h1 className="text-3xl font-bold" data-testid="text-page-title">Empresas</h1>
```

```

184 %         <p className="text-muted-foreground">Gestión de empresas y corporativos</p>
185 %     </div>
186 %     <Button onClick={handleOpenNew} data-testid="button-add-company">
187 %         <Plus className="h-4 w-4 mr-2" />
188 %         Nueva Empresa
189 %     </Button>
190 % </div>
191 %
192 % <div className="flex items-center gap-4">
193 %     <div className="relative flex-1 max-w-sm">
194 %         <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
195 %         <Input
196 %             placeholder="Buscar por nombre, CUIT o contacto..."
197 %             value={searchTerm}
198 %             onChange={(e) => setSearchTerm(e.target.value)}
199 %             className="pl-10"
200 %             data-testid="input-search-company"
201 %         />
202 %     </div>
203 %     <Badge variant="outline">{companies.length} empresas</Badge>
204 % </div>
205 %
206 % <Card>
207 %     <CardContent className="p-0">
208 %         <Table>
209 %             <TableHeader>
210 %                 <TableRow>
211 %                     <TableHead>Razón Social</TableHead>
212 %                     <TableHead>Nombre Fantasia</TableHead>
213 %                     <TableHead>CUIT</TableHead>
214 %                     <TableHead>Cond. IVA</TableHead>
215 %                     <TableHead>Contacto</TableHead>
216 %                     <TableHead>Teléfono</TableHead>
217 %                     <TableHead className="w-[100px]">Acciones</TableHead>
218 %                 </TableRow>
219 %             </TableHeader>
220 %             <TableBody>
221 %                 {filteredCompanies.length === 0 ? (
222 %                     <TableRow>
223 %                         <TableCell colspan={7} className="text-center py-8 text-muted-foreground">
224 %                             {searchTerm ? "No se encontraron empresas" : "No hay empresas registradas"}
225 %                         </TableCell>
226 %                     </TableRow>
227 %                 ) : (
228 %                     filteredCompanies.map((company) => (
229 %                         <TableRow key={company.id} data-testid={`row-company-${company.id}`}>
230 %                             <TableCell className="font-medium">
231 %                                 <div className="flex items-center gap-2">
232 %                                     <Building2 className="h-4 w-4 text-muted-foreground" />
233 %                                     {company.razonSocial}
234 %                                 </div>
235 %                             </TableCell>
236 %                             <TableCell>{company.nombreFantasia || "-"}</TableCell>
237 %                             <TableCell>{company.cuilCuit || "-"}</TableCell>
238 %                             <TableCell>
239 %                                 <Badge variant="outline" className="text-xs">
240 %                                     {company.condicionIva === "responsable_inscripto" ? "Resp. Inscripto" :
241 %                                     company.condicionIva === "monotributista" ? "Monotributo" :
242 %                                     company.condicionIva === "exento" ? "Exento" :
243 %                                     company.condicionIva || "-"}
244 %                                 </Badge>
245 %                             </TableCell>
246 %                             <TableCell>{company.contactName || "-"}</TableCell>
247 %                             <TableCell>{company.telefono || "-"}</TableCell>
248 %                             <TableCell>
249 %                                 <div className="flex items-center gap-1">
250 %                                     <Button
251 %                                         variant="ghost"
252 %                                         size="icon"
253 %                                         onClick={() => handleEdit(company)}
254 %                                         data-testid={`button-edit-company-${company.id}`}
255 %                                     >
256 %                                         <Pencil className="h-4 w-4" />
257 %                                     </Button>
258 %                                     <Button
259 %                                         variant="ghost"
260 %                                         size="icon"
261 %                                         onClick={() => handleDelete(company)}
262 %                                         data-testid={`button-delete-company-${company.id}`}
263 %                                     >
264 %                                         <Trash2 className="h-4 w-4 text-destructive" />
265 %                                     </Button>
266 %                                 </div>
267 %                             </TableCell>
268 %                         </TableRow>
269 %                     ))
270 %                 )}
271 %             </TableBody>
272 %         </Table>
273 %     </CardContent>
274 % </Card>
275 %
276 % <Dialog open={showForm} onOpenChange={(open) => { if (!open) { setShowForm(false); setEditingCompany(null); } }}>

```



```

277 %         <DialogContent className="max-w-2xl max-h-[90vh] overflow-y-auto">
278 %             <DialogHeader>
279 %                 <DialogTitle>{editingCompany ? "Editar Empresa" : "Nueva Empresa"}</DialogTitle>
280 %                 <DialogDescription>
281 %                     {editingCompany ? "Modificá los datos de la empresa" : "Completá los datos para registrar una nueva emp...
282 %                 </DialogDescription>
283 %             </DialogHeader>
284 %             <Form {...form}>
285 %                 <form onSubmit={form.handleSubmit(handleSubmit)} className="space-y-4">
286 %                     <div className="grid grid-cols-2 gap-4">
287 %                         <FormField control={form.control} name="razonSocial" render={({ field }) => (
288 %                             <FormItem className="col-span-2">
289 %                                 <FormLabel>Razón Social </FormLabel>
290 %                                 <FormControl><Input {...field} data-testid="input-company-razon-social" /></FormControl>
291 %                                 <FormMessage />
292 %                             </FormItem>
293 %                         )} />
294 %                         <FormField control={form.control} name="nombreFantasia" render={({ field }) => (
295 %                             <FormItem>
296 %                                 <FormLabel>Nombre Fantasia</FormLabel>
297 %                                 <FormControl><Input {...field} data-testid="input-company-nombre-fantasia" /></FormControl>
298 %                                 <FormMessage />
299 %                             </FormItem>
300 %                         )} />
301 %                         <FormField control={form.control} name="cuitCuit" render={({ field }) => (
302 %                             <FormItem>
303 %                                 <FormLabel>CUIT *</FormLabel>
304 %                                 <FormControl><Input {...field} placeholder="XX-XXXXXXX-X" data-testid="input-company-cuit" /></F...
305 %                                 <FormMessage />
306 %                             </FormItem>
307 %                         )} />
308 %                         <FormField control={form.control} name="condicionIva" render={({ field }) => (
309 %                             <FormItem>
310 %                                 <FormLabel>Condición IVA</FormLabel>
311 %                                 <Select onChange={field.onChange} value={field.value}>
312 %                                     <FormControl>
313 %                                         <SelectTrigger data-testid="select-company-iva">
314 %                                             <SelectValue />
315 %                                         </SelectTrigger>
316 %                                     </FormControl>
317 %                                     <SelectContent>
318 %                                         <SelectItem value="responsable_inscripto">Responsable Inscripto</SelectItem>
319 %                                         <SelectItem value="monotributista">Monotributista</SelectItem>
320 %                                         <SelectItem value="exento">Exento</SelectItem>
321 %                                         <SelectItem value="consumidor_final">Consumidor Final</SelectItem>
322 %                                     </SelectContent>
323 %                                 </Select>
324 %                                 <FormMessage />
325 %                             </FormItem>
326 %                         )} />
327 %                         <FormField control={form.control} name="direccion" render={({ field }) => (
328 %                             <FormItem>
329 %                                 <FormLabel>Dirección</FormLabel>
330 %                                 <FormControl><Input {...field} data-testid="input-company-direccion" /></FormControl>
331 %                                 <FormMessage />
332 %                             </FormItem>
333 %                         )} />
334 %                         <FormField control={form.control} name="localidad" render={({ field }) => (
335 %                             <FormItem>
336 %                                 <FormLabel>Localidad</FormLabel>
337 %                                 <FormControl><Input {...field} data-testid="input-company-localidad" /></FormControl>
338 %                                 <FormMessage />
339 %                             </FormItem>
340 %                         )} />
341 %                         <FormField control={form.control} name="provincia" render={({ field }) => (
342 %                             <FormItem>
343 %                                 <FormLabel>Provincia</FormLabel>
344 %                                 <FormControl><Input {...field} data-testid="input-company-provincia" /></FormControl>
345 %                                 <FormMessage />
346 %                             </FormItem>
347 %                         )} />
348 %                         <FormField control={form.control} name="codigoPostal" render={({ field }) => (
349 %                             <FormItem>
350 %                                 <FormLabel>Código Postal</FormLabel>
351 %                                 <FormControl><Input {...field} data-testid="input-company-cp" /></FormControl>
352 %                                 <FormMessage />
353 %                             </FormItem>
354 %                         )} />
355 %                         <FormField control={form.control} name="telefono" render={({ field }) => (
356 %                             <FormItem>
357 %                                 <FormLabel>Teléfono</FormLabel>
358 %                                 <FormControl><Input {...field} data-testid="input-company-telefono" /></FormControl>
359 %                                 <FormMessage />
360 %                             </FormItem>
361 %                         )} />
362 %                         <FormField control={form.control} name="email" render={({ field }) => (
363 %                             <FormItem>
364 %                                 <FormLabel>Email</FormLabel>
365 %                                 <FormControl><Input {...field} type="email" data-testid="input-company-email" /></FormControl>
366 %                                 <FormMessage />
367 %                             </FormItem>
368 %                         )} />
369 %                     </div>

```

```

370 %
371 %
372 %     <div className="border-t pt-4">
373 %         <h3 className="text-sm font-medium mb-3">Persona de Contacto</h3>
374 %         <div className="grid grid-cols-3 gap-4">
375 %             <FormField control={form.control} name="contactName" render={({ field }) => (
376 %                 <FormItem>
377 %                     <FormLabel>Nombre</FormLabel>
378 %                     <FormControl><Input {...field} data-testid="input-company-contact-name" /></FormControl>
379 %                     <FormMessage />
380 %                 </FormItem>
381 %             )} />
382 %             <FormField control={form.control} name="contactEmail" render={({ field }) => (
383 %                 <FormItem>
384 %                     <FormLabel>Email</FormLabel>
385 %                     <FormControl><Input {...field} data-testid="input-company-contact-email" /></FormControl>
386 %                     <FormMessage />
387 %                 </FormItem>
388 %             )} />
389 %             <FormField control={form.control} name="contactPhone" render={({ field }) => (
390 %                 <FormItem>
391 %                     <FormLabel>Teléfono</FormLabel>
392 %                     <FormControl><Input {...field} data-testid="input-company-contact-phone" /></FormControl>
393 %                     <FormMessage />
394 %                 </FormItem>
395 %             )} />
396 %         </div>
397 %     </div>
398 %
399 %     <div className="border-t pt-4">
400 %         <h3 className="text-sm font-medium mb-3">Condiciones Comerciales</h3>
401 %         <div className="grid grid-cols-2 gap-4">
402 %             <FormField control={form.control} name="creditLimit" render={({ field }) => (
403 %                 <FormItem>
404 %                     <FormLabel>Límite de Crédito</FormLabel>
405 %                     <FormControl><Input {...field} type="number" data-testid="input-company-credit-limit" /></FormC...
406 %                     <FormMessage />
407 %                 </FormItem>
408 %             )} />
409 %             <FormField control={form.control} name="paymentTermDays" render={({ field }) => (
410 %                 <FormItem>
411 %                     <FormLabel>Plazo de Pago (días)</FormLabel>
412 %                     <FormControl><Input {...field} type="number" data-testid="input-company-payment-term" /></FormC...
413 %                     <FormMessage />
414 %                 </FormItem>
415 %             )} />
416 %         </div>
417 %
418 %         <FormField control={form.control} name="notes" render={({ field }) => (
419 %             <FormItem>
420 %                 <FormLabel>Notas</FormLabel>
421 %                 <FormControl><Input {...field} data-testid="input-company-notes" /></FormControl>
422 %                 <FormMessage />
423 %             </FormItem>
424 %         )} />
425 %
426 %         <DialogFooter>
427 %             <Button type="button" variant="outline" onClick={() => { setShowForm(false); setEditingCompany(null);...
428 %                 Cancelar
429 %             </Button>
430 %             <Button type="submit" disabled={createMutation.isPending || updateMutation.isPending} data-testid="bu...
431 %                 {(createMutation.isPending || updateMutation.isPending) && <Loader2 className="h-4 w-4 mr-2 animate...
432 %                 {editingCompany ? "Guardar Cambios" : "Crear Empresa"}
433 %             </Button>
434 %         </DialogFooter>
435 %     </form>
436 % </Form>
437 % </DialogContent>
438 % </Dialog>
439 % </div>
440 % );
441 % }
442 %

```

```
1 % import { useState, useEffect } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { useLocation, useSearch } from "wouter";
4 % import {
5 %   CalendarCheck,
6 %   Plus,
7 %   Search,
8 %   Filter,
9 %   MoreHorizontal,
10 %   Pencil,
11 %   Trash2,
12 %   Eye,
13 %   Calendar,
14 %   User,
15 %   DoorOpen,
16 %   X,
17 %   Check,
18 %   LogIn,
19 %   LogOut,
20 %   Copy,
21 %   ArrowRightLeft,
22 %   Sunrise,
23 %   Sunset,
24 % } from "lucide-react";
25 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
26 % import { Button } from "@components/ui/button";
27 % import { Input } from "@components/ui/input";
28 % import { Badge } from "@components/ui/badge";
29 % import { Skeleton } from "@components/ui/skeleton";
30 % import {
31 %   Dialog,
32 %   DialogContent,
33 %   DialogDescription,
34 %   DialogFooter,
35 %   DialogHeader,
36 %   DialogTitle,
37 % } from "@components/ui/dialog";
38 % import {
39 %   DropdownMenu,
40 %   DropdownMenuContent,
41 %   DropdownMenuItem,
42 %   DropdownMenuSeparator,
43 %   DropdownMenuTrigger,
44 % } from "@components/ui/dropdown-menu";
45 % import {
46 %   Select,
47 %   SelectContent,
48 %   SelectItem,
49 %   SelectTrigger,
50 %   SelectValue,
51 % } from "@components/ui/select";
52 % import {
53 %   Table,
54 %   TableBody,
55 %   TableCell,
56 %   TableHead,
57 %   TableHeader,
58 %   TableRow,
59 % } from "@components/ui/table";
60 % import { Label } from "@components/ui/label";
61 % import { Textarea } from "@components/ui/textarea";
62 % import { Switch } from "@components/ui/switch";
63 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
64 % import { useToast } from "@hooks/use-toast";
65 % import { queryClient, apiRequest } from "@lib/queryClient";
66 % import { GuestSelector, CompanySelector } from "@components/entity-selector";
67 % import type { ReservationWithDetails, Guest, Company, RoomWithType, RoomType, RatePlan, InsertReservation, InsertGues...
68 %
69 % function ReservationStatusBadge({ status }: { status: ReservationStatus }) {
70 %   const statusConfig: Record<ReservationStatus, { label: string; variant: "default" | "secondary" | "destructive" | "...
71 %     tentative: { label: "Tentativa", variant: "outline" },
72 %     pending: { label: "Pendiente", variant: "secondary" },
73 %     confirmed: { label: "Confirmada", variant: "default" },
74 %     checked_in: { label: "Check-in", variant: "default" },
75 %     checked_out: { label: "Check-out", variant: "outline" },
76 %     cancelled: { label: "Cancelada", variant: "destructive" },
77 %   };
78 %
79 %   const config = statusConfig[status];
80 %
81 %   return <Badge variant={config.variant}>{config.label}</Badge>;
82 % }
83 %
84 % function ReservationFormDialog({
85 %   reservation,
86 %   guests,
87 %   rooms,
88 %   roomTypes,
89 %   open,
90 %   onOpenChange,
```

client/src/pages/reservations.tsx (cont.)

```
91 %     onSuccess,
92 %   }: {
93 %     reservation?: ReservationWithDetails;
94 %     guests: Guest[];
95 %     rooms: RoomWithType[];
96 %     roomTypes: RoomType[];
97 %     open: boolean;
98 %     onOpenChange: (open: boolean) => void;
99 %     onSuccess: () => void;
100 %   }) {
101 %     const { toast } = useToast();
102 %     const isEditing = !!reservation;
103 %
104 %     const today = new Date().toISOString().split("T")[0];
105 %     const tomorrow = new Date(Date.now() + 86400000).toISOString().split("T")[0];
106 %
107 %     const [selectedGuest, setSelectedGuest] = useState<Guest | null>(
108 %       reservation?.guest || null
109 %     );
110 %     const [selectedCompany, setSelectedCompany] = useState<Company | null>(
111 %       reservation?.company || null
112 %     );
113 %
114 %     const [selectedRoomTypeId, setSelectedRoomTypeId] = useState<string>(reservation?.roomTypeId || "");
115 %
116 %     const [formData, setFormData] = useState<Partial<InsertReservation>>({
117 %       reservationCode: reservation?.reservationCode || "",
118 %       guestId: reservation?.guestId || "",
119 %       companyId: reservation?.companyId || "",
120 %       roomTypeId: reservation?.roomTypeId || "",
121 %       roomId: reservation?.roomId || "",
122 %       ratePlanId: reservation?.ratePlanId || "",
123 %       checkInDate: reservation?.checkInDate || today,
124 %       checkOutDate: reservation?.checkOutDate || tomorrow,
125 %       nights: reservation?.nights || 1,
126 %       numberOfGuests: reservation?.numberOfGuests || 1,
127 %       status: reservation?.status || "pending",
128 %       source: reservation?.source || "directo",
129 %       discountType: reservation?.discountType || "none",
130 %       discountValue: reservation?.discountValue || "0",
131 %       baseRatePerNight: reservation?.baseRatePerNight || "",
132 %       finalRatePerNight: reservation?.finalRatePerNight || "",
133 %       totalRoomAmount: reservation?.totalRoomAmount || "",
134 %       bedTypeId: reservation?.bedTypeId || null,
135 %       bedTypeNotes: reservation?.bedTypeNotes || "",
136 %       earlyCheckIn: reservation?.earlyCheckIn || false,
137 %       earlyCheckInTime: reservation?.earlyCheckInTime || "",
138 %       earlyCheckInCharge: reservation?.earlyCheckInCharge || "",
139 %       lateCheckOut: reservation?.lateCheckOut || false,
140 %       lateCheckOutTime: reservation?.lateCheckOutTime || "",
141 %       lateCheckOutCharge: reservation?.lateCheckOutCharge || "",
142 %       notes: reservation?.notes || "",
143 %       createdAt: reservation?.createdAt || new Date().toISOString(),
144 %     });
145 %
146 %     const createGuestMutation = useMutation({
147 %       mutationFn: async (guest: InsertGuest): Promise<Guest> => {
148 %         const res = await apiRequest("POST", "/api/guests", guest);
149 %         return res.json();
150 %       },
151 %       onSuccess: (newGuest: Guest) => {
152 %         queryClient.invalidateQueries({ queryKey: ["/api/guests"] });
153 %         setSelectedGuest(newGuest);
154 %         setFormData((prev) => ({ ...prev, guestId: newGuest.id }));
155 %         toast({
156 %           title: "Huesped creado",
157 %           description: `${newGuest.firstName} ${newGuest.lastName} ha sido registrado.`,
158 %         });
159 %       },
160 %       onError: () => {
161 %         toast({
162 %           title: "Error",
163 %           description: "No se pudo crear el huesped.",
164 %           variant: "destructive",
165 %         });
166 %       },
167 %     });
168 %
169 %     const createCompanyMutation = useMutation({
170 %       mutationFn: async (company: InsertCompany): Promise<Company> => {
171 %         const res = await apiRequest("POST", "/api/companies", company);
172 %         return res.json();
173 %       },
174 %       onSuccess: (newCompany: Company) => {
175 %         queryClient.invalidateQueries({ queryKey: ["/api/companies"] });
176 %         setSelectedCompany(newCompany);
177 %         setFormData((prev) => ({ ...prev, companyId: newCompany.id }));
178 %         toast({
179 %           title: "Empresa creada",
180 %           description: `${newCompany.razonSocial} ha sido registrada.`,
181 %         });
182 %       },
183 %       onError: () => {
```

client/src/pages/reservations.tsx (cont.)

```
184 %         toast({
185 %             title: "Error",
186 %             description: "No se pudo crear la empresa.",
187 %             variant: "destructive",
188 %         });
189 %     },
190 % });
191 %
192 % const { data: ratePlans } = useQuery<RatePlan[]>({
193 %     queryKey: ["/api/rate-plans/by-room-type", selectedRoomTypeId],
194 %     queryFn: async () => {
195 %         if (!selectedRoomTypeId) return [];
196 %         const res = await fetch(`/api/rate-plans/by-room-type/${selectedRoomTypeId}`);
197 %         return res.json();
198 %     },
199 %     enabled: !!selectedRoomTypeId,
200 % });
201 %
202 % const { data: bedTypes } = useQuery<BedType[]>({
203 %     queryKey: ["/api/bed-types"],
204 % });
205 %
206 % const { data: generatedCode } = useQuery<{ code: string }>({
207 %     queryKey: ["/api/reservations/generate-code"],
208 %     enabled: !isEditing && !formData.reservationCode,
209 % });
210 %
211 % const calculateNights = (checkIn: string, checkOut: string) => {
212 %     const start = new Date(checkIn);
213 %     const end = new Date(checkOut);
214 %     const diff = Math.ceil((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24));
215 %     return Math.max(1, diff);
216 % };
217 %
218 % const calculateTotals = (baseRate: string, discountType: DiscountType, discountValue: string, nights: number) => {
219 %     const base = parseFloat(baseRate) || 0;
220 %     const discount = parseFloat(discountValue) || 0;
221 %
222 %     let finalRate = base;
223 %     if (discountType === "percent") {
224 %         finalRate = base - (base * discount / 100);
225 %     } else if (discountType === "fixed") {
226 %         finalRate = base - discount;
227 %     }
228 %     finalRate = Math.max(0, finalRate);
229 %
230 %     return {
231 %         finalRatePerNight: finalRate.toFixed(2),
232 %         totalRoomAmount: (finalRate * nights).toFixed(2),
233 %     };
234 % };
235 %
236 % const handleRoomTypeChange = (roomTypeId: string) => {
237 %     setSelectedRoomTypeId(roomTypeId);
238 %     setFormData({
239 %         ...formData,
240 %         roomTypeId,
241 %         roomId: "",
242 %         ratePlanId: "",
243 %         baseRatePerNight: "",
244 %         finalRatePerNight: "",
245 %         totalRoomAmount: "",
246 %     });
247 % };
248 %
249 % const handleRatePlanChange = (ratePlanId: string) => {
250 %     const plan = ratePlans?.find(p => p.id === ratePlanId);
251 %     if (plan) {
252 %         const nights = calculateNights(formData.checkInDate || today, formData.checkOutDate || tomorrow);
253 %         const totals = calculateTotals(plan.baseRate, formData.discountType as DiscountType, formData.discountValue || ...
254 %         setFormData({
255 %             ...formData,
256 %             ratePlanId,
257 %             baseRatePerNight: plan.baseRate,
258 %             ...totals,
259 %         });
260 %     }
261 % };
262 %
263 % const handleDateChange = (field: "checkInDate" | "checkOutDate", value: string) => {
264 %     const newData = { ...formData, [field]: value };
265 %     const nights = calculateNights(
266 %         field === "checkInDate" ? value : formData.checkInDate || today,
267 %         field === "checkOutDate" ? value : formData.checkOutDate || tomorrow
268 %     );
269 %     const totals = calculateTotals(
270 %         formData.baseRatePerNight || "0",
271 %         formData.discountType as DiscountType,
272 %         formData.discountValue || "0",
273 %         nights
274 %     );
275 %     setFormData({ ...newData, nights, ...totals });
276 % };
```

```

277 %
278 % const handleDiscountChange = (discountType?: DiscountType, discountValue?: string) => {
279 %   const nights = calculateNights(formData.checkInDate || today, formData.checkOutDate || tomorrow);
280 %   const totals = calculateTotals(
281 %     formData.baseRatePerNight || "0",
282 %     discountType || formData.discountType as DiscountType,
283 %     discountValue !== undefined ? discountValue : formData.discountValue || "0",
284 %     nights
285 %   );
286 %   setFormData({
287 %     ...formData,
288 %     ...(discountType !== undefined && { discountType }),
289 %     ...(discountValue !== undefined && { discountValue }),
290 %     ...totals,
291 %   });
292 % };
293 %
294 % const handleBaseRateChange = (newBaseRate: string) => {
295 %   const nights = calculateNights(formData.checkInDate || today, formData.checkOutDate || tomorrow);
296 %   const totals = calculateTotals(
297 %     newBaseRate,
298 %     formData.discountType as DiscountType,
299 %     formData.discountValue || "0",
300 %     nights
301 %   );
302 %   setFormData({
303 %     ...formData,
304 %     baseRatePerNight: newBaseRate,
305 %     ...totals,
306 %   });
307 % };
308 %
309 % const mutation = useMutation({
310 %   mutationFn: async (data: Partial<InsertReservation>) => {
311 %     if (isEditing) {
312 %       return apiRequest("PATCH", `/api/reservations/${reservation.id}`, data);
313 %     }
314 %     return apiRequest("POST", "/api/reservations", {
315 %       ...data,
316 %       reservationCode: data.reservationCode || generatedCode?.code || `RES-${Date.now()}`,
317 %     });
318 %   },
319 %   onSuccess: () => {
320 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
321 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
322 %     queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
323 %     toast({
324 %       title: isEditing ? "Reserva actualizada" : "Reserva creada",
325 %       description: `La reserva ha sido ${isEditing ? "actualizada" : "creada"} exitosamente.`,
326 %     });
327 %     onSuccess();
328 %     onOpenChange(false);
329 %   },
330 %   onError: () => {
331 %     toast({
332 %       title: "Error",
333 %       description: "No se pudo guardar la reserva. Intente nuevamente.",
334 %       variant: "destructive",
335 %     });
336 %   },
337 % });
338 %
339 % const handleSubmit = (e: React.FormEvent) => {
340 %   e.preventDefault();
341 %   mutation.mutate(formData);
342 % };
343 %
344 % const availableRooms = rooms.filter((r) =>
345 %   (r.status === "available" || r.id === reservation?.roomId) &&
346 %   r.roomTypeId === selectedRoomTypeId
347 % );
348 %
349 % return (
350 %   <Dialog open={open} onOpenChange={onOpenChange}>
351 %     <DialogContent className="sm:max-w-[600px] max-h-[90vh] overflow-y-auto">
352 %       <DialogHeader>
353 %         <DialogTitle>{isEditing ? "Editar Reserva" : "Nueva Reserva"}</DialogTitle>
354 %         <DialogDescription>
355 %           {isEditing ? "Modifica los detalles de la reserva." : "Ingresa los datos para crear una nueva reserva."}
356 %           {!isEditing && generatedCode && (
357 %             <Badge variant="outline" className="ml-2">
358 %               Código: {generatedCode.code}
359 %             </Badge>
360 %           )}
361 %         </DialogDescription>
362 %       </DialogHeader>
363 %       <form onSubmit={handleSubmit}>
364 %         <div className="grid gap-4 py-4">
365 %           <GuestSelector
366 %             selectedGuest={selectedGuest}
367 %             onSelect={(guest) => {
368 %               setSelectedGuest(guest);
369 %               setFormData((prev) => ({ ...prev, guestId: guest.id }));

```

client/src/pages/reservations.tsx (cont.)

```
370 %         }}
371 %         onCreateNew={() => createGuestMutation.mutate(guest)}
372 %         onClear={() => {
373 %             setSelectedGuest(null);
374 %             setFormData((prev) => ({ ...prev, guestId: "" }));
375 %         }}
376 %     />
377 %
378 %     <CompanySelector
379 %         selectedCompany={selectedCompany}
380 %         onSelect={() => {
381 %             setSelectedCompany(company);
382 %             setFormData((prev) => ({ ...prev, companyId: company.id }));
383 %         }}
384 %         onCreateNew={() => createCompanyMutation.mutate(company)}
385 %         onClear={() => {
386 %             setSelectedCompany(null);
387 %             setFormData((prev) => ({ ...prev, companyId: "" }));
388 %         }}
389 %     />
390 %
391 %     <div className="grid gap-2">
392 %         <Label htmlFor="source">Origen</Label>
393 %         <Select
394 %             value={formData.source}
395 %             onChange={(value) => setFormData({ ...formData, source: value as ReservationSource })}
396 %         >
397 %             <SelectTrigger data-testid="select-source">
398 %                 <SelectValue placeholder="Origen" />
399 %             </SelectTrigger>
400 %             <SelectContent>
401 %                 <SelectItem value="directo">Directo</SelectItem>
402 %                 <SelectItem value="web">Web</SelectItem>
403 %                 <SelectItem value="ota">OTA (Booking, etc.)</SelectItem>
404 %                 <SelectItem value="empresa">Empresa</SelectItem>
405 %                 <SelectItem value="telefono">Teléfono</SelectItem>
406 %             </SelectContent>
407 %         </Select>
408 %     </div>
409 %
410 %     <div className="grid grid-cols-2 gap-4">
411 %         <div className="grid gap-2">
412 %             <Label htmlFor="roomType">Tipo de Habitación</Label>
413 %             <Select
414 %                 value={selectedRoomTypeId}
415 %                 onChange={handleRoomTypeChange}
416 %             >
417 %                 <SelectTrigger data-testid="select-room-type">
418 %                     <SelectValue placeholder="Seleccionar tipo" />
419 %                 </SelectTrigger>
420 %                 <SelectContent>
421 %                     {roomTypes.map((type) => (
422 %                         <SelectItem key={type.id} value={type.id}>
423 %                             {type.name}
424 %                         </SelectItem>
425 %                     ))}
426 %                 </SelectContent>
427 %             </Select>
428 %         </div>
429 %         <div className="grid gap-2">
430 %             <Label htmlFor="room">Habitación</Label>
431 %             <Select
432 %                 value={formData.roomId}
433 %                 onChange={(value) => setFormData({ ...formData, roomId: value })}
434 %                 disabled={!selectedRoomTypeId}
435 %             >
436 %                 <SelectTrigger data-testid="select-room">
437 %                     <SelectValue placeholder="Seleccionar habitación" />
438 %                 </SelectTrigger>
439 %                 <SelectContent>
440 %                     {availableRooms.map((room) => (
441 %                         <SelectItem key={room.id} value={room.id}>
442 %                             Hab. {room.roomNumber}
443 %                         </SelectItem>
444 %                     ))}
445 %                 </SelectContent>
446 %             </Select>
447 %         </div>
448 %     </div>
449 %
450 %     <div className="grid gap-2">
451 %         <Label htmlFor="ratePlan">Plan Tarifario</Label>
452 %         <Select
453 %             value={formData.ratePlanId || ""}
454 %             onChange={handleRatePlanChange}
455 %             disabled={!selectedRoomTypeId}
456 %         >
457 %             <SelectTrigger data-testid="select-rate-plan">
458 %                 <SelectValue placeholder="Seleccionar plan tarifario" />
459 %             </SelectTrigger>
460 %             <SelectContent>
461 %                 {ratePlans?.map((plan) => (
462 %                     <SelectItem key={plan.id} value={plan.id}>
```

client/src/pages/reservations.tsx (cont.)

```
463 %         {plan.name} - ${plan.baseRate}/noche
464 %     </SelectedItem>
465 %     )))
466 % </SelectContent>
467 % </Select>
468 % </div>
469 %
470 % <div className="grid grid-cols-2 gap-4">
471 %   <div className="grid gap-2">
472 %     <Label htmlFor="checkIn">Fecha Check-in</Label>
473 %     <Input
474 %       id="checkIn"
475 %       type="date"
476 %       value={formData.checkInDate}
477 %       onChange={(e) => handleDateChange("checkInDate", e.target.value)}
478 %       required
479 %       data-testid="input-check-in"
480 %     />
481 %   </div>
482 %   <div className="grid gap-2">
483 %     <Label htmlFor="checkOut">Fecha Check-out</Label>
484 %     <Input
485 %       id="checkOut"
486 %       type="date"
487 %       value={formData.checkOutDate}
488 %       onChange={(e) => handleDateChange("checkOutDate", e.target.value)}
489 %       required
490 %       data-testid="input-check-out"
491 %     />
492 %   </div>
493 % </div>
494 %
495 % <div className="grid grid-cols-3 gap-4">
496 %   <div className="grid gap-2">
497 %     <Label htmlFor="nights">Noches</Label>
498 %     <Input
499 %       id="nights"
500 %       type="number"
501 %       value={formData.nights}
502 %       readOnly
503 %       className="bg-muted"
504 %       data-testid="input-nights"
505 %     />
506 %   </div>
507 %   <div className="grid gap-2">
508 %     <Label htmlFor="guests">Huéspedes</Label>
509 %     <Input
510 %       id="guests"
511 %       type="number"
512 %       min={1}
513 %       value={formData.numberOfGuests}
514 %       onChange={(e) => setFormData({ ...formData, numberOfGuests: parseInt(e.target.value) })}
515 %       data-testid="input-num-guests"
516 %     />
517 %   </div>
518 %   <div className="grid gap-2">
519 %     <Label htmlFor="status">Estado</Label>
520 %     <Select
521 %       value={formData.status}
522 %       onValueChange={(value) => setFormData({ ...formData, status: value as ReservationStatus })}
523 %     >
524 %       <SelectTrigger data-testid="select-reservation-status">
525 %         <SelectValue />
526 %       </SelectTrigger>
527 %       <SelectContent>
528 %         <SelectedItem value="pending">Pendiente</SelectedItem>
529 %         <SelectedItem value="confirmed">Confirmada</SelectedItem>
530 %         <SelectedItem value="checked_in">Check-in</SelectedItem>
531 %         <SelectedItem value="checked_out">Check-out</SelectedItem>
532 %         <SelectedItem value="cancelled">Cancelada</SelectedItem>
533 %       </SelectContent>
534 %     </Select>
535 %   </div>
536 % </div>
537 %
538 % <div className="grid grid-cols-3 gap-4">
539 %   <div className="grid gap-2">
540 %     <Label htmlFor="discountType">Tipo Descuento</Label>
541 %     <Select
542 %       value={formData.discountType}
543 %       onValueChange={(value) => handleDiscountChange(value as DiscountType, undefined)}
544 %     >
545 %       <SelectTrigger data-testid="select-discount-type">
546 %         <SelectValue />
547 %       </SelectTrigger>
548 %       <SelectContent>
549 %         <SelectedItem value="none">Sin descuento</SelectedItem>
550 %         <SelectedItem value="percent">Porcentaje (%)</SelectedItem>
551 %         <SelectedItem value="fixed">Monto Fijo ($)</SelectedItem>
552 %       </SelectContent>
553 %     </Select>
554 %   </div>
555 %   <div className="grid gap-2">
```



```

556 %         <Label htmlFor="discountValue">Valor Descuento</Label>
557 %         <Input
558 %             id="discountValue"
559 %             type="number"
560 %             min={0}
561 %             step="0.01"
562 %             value={formData.discountValue || ""}
563 %             onChange={(e) => handleDiscountChange(undefined, e.target.value)}
564 %             disabled={formData.discountType === "none"}
565 %             data-testid="input-discount-value"
566 %         />
567 %     </div>
568 %     <div className="grid gap-2">
569 %         <Label htmlFor="baseRate">Tarifa Base/Noche</Label>
570 %         <div className="relative">
571 %             <span className="absolute left-3 top-1/2 -translate-y-1/2 text-muted-foreground">$/</span>
572 %             <Input
573 %                 id="baseRate"
574 %                 type="number"
575 %                 min={0}
576 %                 step="0.01"
577 %                 value={formData.baseRatePerNight || ""}
578 %                 onChange={(e) => handleBaseRateChange(e.target.value)}
579 %                 placeholder="0.00"
580 %                 className="pl-7"
581 %                 data-testid="input-base-rate"
582 %             />
583 %         </div>
584 %     </div>
585 % </div>
586 %
587 % <div className="p-4 bg-muted rounded-lg">
588 %     <div className="flex justify-between items-center">
589 %         <div>
590 %             <p className="text-sm text-muted-foreground">Tarifa Final/Noche</p>
591 %             <p className="text-lg font-semibold">${formData.finalRatePerNight || "0.00"}</p>
592 %         </div>
593 %         <div className="text-right">
594 %             <p className="text-sm text-muted-foreground">Total Habitación</p>
595 %             <p className="text-2xl font-bold text-primary">${formData.totalRoomAmount || "0.00"}</p>
596 %         </div>
597 %     </div>
598 % </div>
599 %
600 % <div className="grid gap-4">
601 %     <Label className="text-sm font-semibold">Preferencias de camaje</Label>
602 %     <div className="grid grid-cols-2 gap-4">
603 %         <div className="grid gap-2">
604 %             <Label htmlFor="bedType">Tipo de camaje</Label>
605 %             <Select
606 %                 value={formData.bedTypeId ? String(formData.bedTypeId) : ""}
607 %                 onChange={(value) => setFormData({ ...formData, bedTypeId: parseInt(value) })}
608 %             >
609 %                 <SelectTrigger data-testid="select-bed-type">
610 %                     <SelectValue placeholder="Seleccionar tipo de camaje" />
611 %                 </SelectTrigger>
612 %                 <SelectContent>
613 %                     {bedTypes?.filter(bt => bt.isActive).map((bt) => (
614 %                         <SelectItem key={bt.id} value={String(bt.id)}>
615 %                             {bt.name}
616 %                         </SelectItem>
617 %                     ))}
618 %                 </SelectContent>
619 %             </Select>
620 %         </div>
621 %         <div className="grid gap-2">
622 %             <Label htmlFor="bedTypeNotes">Notas de camaje</Label>
623 %             <Input
624 %                 id="bedTypeNotes"
625 %                 value={formData.bedTypeNotes || ""}
626 %                 onChange={(e) => setFormData({ ...formData, bedTypeNotes: e.target.value })}
627 %                 placeholder="Preferencias especiales..."
628 %                 data-testid="input-bed-type-notes"
629 %             />
630 %         </div>
631 %     </div>
632 % </div>
633 %
634 % <div className="grid gap-3 border rounded-lg p-3">
635 %     <Label className="text-sm font-semibold">Servicios especiales</Label>
636 %     <div className="grid gap-3">
637 %         <div className="flex items-center justify-between">
638 %             <div className="flex items-center gap-2">
639 %                 <Sunrise className="h-4 w-4 text-orange-500" />
640 %                 <Label htmlFor="earlyCheckIn" className="cursor-pointer">Early Check-in</Label>
641 %                 <span className="text-xs text-muted-foreground">(Estándar: 12:00 hs)</span>
642 %             </div>
643 %             <Switch
644 %                 id="earlyCheckIn"
645 %                 checked={!!formData.earlyCheckIn}
646 %                 onCheckedChange={(checked) => setFormData({ ...formData, earlyCheckIn: checked, ...(checked && {...
647 %                     data-testid="switch-early-checkin"
648 %                 />

```

```

649 %         </div>
650 %         {formData.earlyCheckIn && (
651 %           <div className="grid grid-cols-2 gap-2 pl-6">
652 %             <div>
653 %               <Label htmlFor="earlyCheckInTime" className="text-xs">Hora acordada</Label>
654 %               <Input
655 %                 id="earlyCheckInTime"
656 %                 type="time"
657 %                 value={formData.earlyCheckInTime || ""}
658 %                 onChange={(e) => setFormData({ ...formData, earlyCheckInTime: e.target.value })}
659 %                 data-testid="input-early-checkin-time"
660 %               />
661 %             </div>
662 %             <div>
663 %               <Label htmlFor="earlyCheckInCharge" className="text-xs">Cargo (vacío = cortesía)</Label>
664 %               <Input
665 %                 id="earlyCheckInCharge"
666 %                 type="number"
667 %                 step="0.01"
668 %                 min="0"
669 %                 placeholder="0.00"
670 %                 value={formData.earlyCheckInCharge || ""}
671 %                 onChange={(e) => setFormData({ ...formData, earlyCheckInCharge: e.target.value })}
672 %                 data-testid="input-early-checkin-charge"
673 %               />
674 %             </div>
675 %           </div>
676 %         )}
677 %
678 %         <div className="flex items-center justify-between">
679 %           <div className="flex items-center gap-2">
680 %             <Sunset className="h-4 w-4 text-purple-500" />
681 %             <Label htmlFor="lateCheckOut" className="cursor-pointer">Late Check-out</Label>
682 %             <span className="text-xs text-muted-foreground">(Estándar: 11:00 hs)</span>
683 %           </div>
684 %           <Switch
685 %             id="lateCheckOut"
686 %             checked={!formData.lateCheckOut}
687 %             onChange={(checked) => setFormData({ ...formData, lateCheckOut: checked, ...(checked && {...
688 %               data-testid="switch-late-checkout"
689 %             })}
690 %           />
691 %         </div>
692 %         {formData.lateCheckOut && (
693 %           <div className="grid grid-cols-2 gap-2 pl-6">
694 %             <div>
695 %               <Label htmlFor="lateCheckOutTime" className="text-xs">Hora acordada</Label>
696 %               <Input
697 %                 id="lateCheckOutTime"
698 %                 type="time"
699 %                 value={formData.lateCheckOutTime || ""}
700 %                 onChange={(e) => setFormData({ ...formData, lateCheckOutTime: e.target.value })}
701 %                 data-testid="input-late-checkout-time"
702 %               />
703 %             </div>
704 %             <div>
705 %               <Label htmlFor="lateCheckOutCharge" className="text-xs">Cargo (vacío = cortesía)</Label>
706 %               <Input
707 %                 id="lateCheckOutCharge"
708 %                 type="number"
709 %                 step="0.01"
710 %                 min="0"
711 %                 placeholder="0.00"
712 %                 value={formData.lateCheckOutCharge || ""}
713 %                 onChange={(e) => setFormData({ ...formData, lateCheckOutCharge: e.target.value })}
714 %                 data-testid="input-late-checkout-charge"
715 %               />
716 %             </div>
717 %           </div>
718 %         )}
719 %       </div>
720 %
721 %       <div className="grid gap-2">
722 %         <Label htmlFor="notes">Notas</Label>
723 %         <Textarea
724 %           id="notes"
725 %           value={formData.notes || ""}
726 %           onChange={(e) => setFormData({ ...formData, notes: e.target.value })}
727 %           placeholder="Notas adicionales sobre la reserva..."
728 %           data-testid="input-reservation-notes"
729 %         />
730 %       </div>
731 %     </div>
732 %     <DialogFooter>
733 %       <Button type="button" variant="outline" onClick={() => onOpenChange(false)}>
734 %         Volver
735 %       </Button>
736 %       <Button type="submit" disabled={mutation.isPending} data-testid="button-submit-reservation">
737 %         {mutation.isPending ? "Guardando..." : isEditing ? "Guardar Cambios" : "Crear Reserva"}
738 %       </Button>
739 %     </DialogFooter>
740 %   </form>
741 % </DialogContent>

```

client/src/pages/reservations.tsx (cont.)

```
742 %     </Dialog>
743 %   );
744 % }
745 %
746 % const paymentMethodLabels: Record<PaymentMethod, string> = {
747 %   efectivo: "Efectivo",
748 %   tarjeta_debito: "Tarjeta Débito",
749 %   tarjeta_credito: "Tarjeta Crédito",
750 %   transferencia: "Transferencia",
751 %   mercadopago: "MercadoPago",
752 %   cuenta_corriente: "Cuenta Corriente",
753 % };
754 %
755 % function ReservationDetailDialog({
756 %   reservation,
757 %   open,
758 %   onOpenChange,
759 %   onCancel,
760 % }: {
761 %   reservation: ReservationWithDetails;
762 %   open: boolean;
763 %   onOpenChange: (open: boolean) => void;
764 %   onCancel: () => void;
765 % }) {
766 %   const { toast } = useToast();
767 %   const [showAddCharge, setShowAddCharge] = useState(false);
768 %   const [showAddPayment, setShowAddPayment] = useState(false);
769 %   const [transferringChargeId, setTransferringChargeId] = useState<string | null>(null);
770 %   const [targetReservationId, setTargetReservationId] = useState<string>("");
771 %   const [newCharge, setNewCharge] = useState({
772 %     description: "",
773 %     amount: "",
774 %     category: "otros" as "room" | "restaurant" | "spa" | "minibar" | "otros" | "adjustment",
775 %   });
776 %   const [newPayment, setNewPayment] = useState({
777 %     amount: "",
778 %     method: "efectivo" as PaymentMethod,
779 %     reference: "",
780 %     notes: "",
781 %   });
782 %
783 %   // Fetch active reservations for transfer target selection
784 %   const { data: activeReservations, isError: isActiveReservationsError, isLoading: isActiveReservationsLoading } = us...
785 %   const queryKey: ["/api/reservations", "transfer-targets"],
786 %   const queryFn: async () => {
787 %     const res = await fetch("/api/reservations");
788 %     if (!res.ok) throw new Error("Failed to fetch reservations");
789 %     const all = await res.json();
790 %     // Filter to only show checked_in or confirmed reservations, excluding current
791 %     return all.filter((r: ReservationWithDetails) =>
792 %       (r.status === "checked_in" || r.status === "confirmed") && r.id !== reservation.id
793 %     );
794 %   },
795 %   enabled: transferringChargeId !== null,
796 % });
797 %
798 % const { data: charges, refetch: refetchCharges } = useQuery<Charge[]>({
799 %   queryKey: ["/api/reservations", reservation.id, "charges"],
800 %   queryFn: async () => {
801 %     const res = await fetch(`/api/reservations/${reservation.id}/charges`);
802 %     return res.json();
803 %   },
804 % });
805 %
806 % const { data: payments, refetch: refetchPayments } = useQuery<Payment[]>({
807 %   queryKey: ["/api/reservations", reservation.id, "payments"],
808 %   queryFn: async () => {
809 %     const res = await fetch(`/api/reservations/${reservation.id}/payments`);
810 %     return res.json();
811 %   },
812 % });
813 %
814 % const addChargeMutation = useMutation({
815 %   mutationFn: async (chargeData: { description: string; amount: string; category: string; reservationId: string; da...
816 %     return apiRequest("POST", "/api/charges", chargeData);
817 %   },
818 %   onSuccess: () => {
819 %     refetchCharges();
820 %     setShowAddCharge(false);
821 %     setNewCharge({ description: "", amount: "", category: "otros" });
822 %     toast({ title: "Cargo agregado", description: "El cargo ha sido registrado en el folio." });
823 %   },
824 % });
825 %
826 % const deleteChargeMutation = useMutation({
827 %   mutationFn: async (chargeId: string) => {
828 %     return apiRequest("DELETE", `/api/charges/${chargeId}`, undefined);
829 %   },
830 %   onSuccess: () => {
831 %     refetchCharges();
832 %     toast({ title: "Cargo eliminado", description: "El cargo ha sido eliminado del folio." });
833 %   },
834 % });
```

```

835 %
836 % const addPaymentMutation = useMutation({
837 %   mutationFn: async (paymentData: { amount: string; method: PaymentMethod; reference?: string; notes?: string; rese...
838 %     return apiRequest("POST", "/api/payments", paymentData);
839 %   },
840 %   onSuccess: () => {
841 %     refetchPayments();
842 %     setShowAddPayment(false);
843 %     setNewPayment({ amount: "", method: "efectivo", reference: "", notes: "" });
844 %     toast({ title: "Pago registrado", description: "El pago ha sido registrado exitosamente." });
845 %   },
846 % });
847 %
848 % const deletePaymentMutation = useMutation({
849 %   mutationFn: async (paymentId: string) => {
850 %     return apiRequest("DELETE", `/api/payments/${paymentId}`, undefined);
851 %   },
852 %   onSuccess: () => {
853 %     refetchPayments();
854 %     toast({ title: "Pago eliminado", description: "El pago ha sido eliminado del registro." });
855 %   },
856 % });
857 %
858 % const transferChargeMutation = useMutation({
859 %   mutationFn: async ({ chargeId, targetReservationId }: { chargeId: string; targetReservationId: string }) => {
860 %     return apiRequest("POST", `/api/charges/${chargeId}/transfer`, { targetReservationId });
861 %   },
862 %   onSuccess: () => {
863 %     refetchCharges();
864 %     setTransferringChargeId(null);
865 %     setTargetReservationId("");
866 %     toast({ title: "Cargo transferido", description: "El cargo ha sido transferido a la otra habitación." });
867 %   },
868 %   onError: () => {
869 %     toast({ title: "Error", description: "No se pudo transferir el cargo.", variant: "destructive" });
870 %   },
871 % });
872 %
873 % const handleTransferCharge = () => {
874 %   if (!transferringChargeId || !targetReservationId) return;
875 %   transferChargeMutation.mutate({ chargeId: transferringChargeId, targetReservationId });
876 % };
877 %
878 % const handleAddCharge = () => {
879 %   if (!newCharge.description || !newCharge.amount) return;
880 %   addChargeMutation.mutate({
881 %     ...newCharge,
882 %     reservationId: reservation.id,
883 %     date: new Date().toISOString().split("T")[0],
884 %   });
885 % };
886 %
887 % const handleAddPayment = () => {
888 %   if (!newPayment.amount) return;
889 %   addPaymentMutation.mutate({
890 %     ...newPayment,
891 %     reservationId: reservation.id,
892 %     date: new Date().toISOString().split("T")[0],
893 %   });
894 % };
895 %
896 % const categoryLabels: Record<string, string> = {
897 %   room: "Habitación",
898 %   restaurant: "Restaurante",
899 %   spa: "Spa",
900 %   minibar: "Minibar",
901 %   otros: "Otros",
902 %   adjustment: "Ajuste",
903 % };
904 %
905 % const consumptionCharges = charges?.filter((c) => c.category !== "payment") || [];
906 % const totalConsumptions = consumptionCharges.reduce((sum, c) => sum + parseFloat(c.amount), 0);
907 % const totalPayments = payments?.reduce((sum, p) => sum + parseFloat(p.amount), 0) || 0;
908 % const subtotalRoom = parseFloat(reservation.totalRoomAmount || "0");
909 % const totalToPay = subtotalRoom + totalConsumptions;
910 % const balance = totalToPay - totalPayments;
911 %
912 % return (
913 %   <Dialog open={open} onOpenChange={onOpenChange}>
914 %     <DialogContent className="sm:max-w-[650px] max-h-[90vh] overflow-y-auto">
915 %       <DialogHeader>
916 %         <DialogTitle className="flex items-center gap-2">
917 %           Reserva {reservation.reservationCode}
918 %           <ReservationStatusBadge status={reservation.status} />
919 %         </DialogTitle>
920 %         <DialogDescription>Detalle de la reservación</DialogDescription>
921 %       </DialogHeader>
922 %
923 %       <Tabs defaultValue="datos" className="w-full">
924 %         <TabList className="grid w-full grid-cols-2">
925 %           <TabTrigger value="datos" data-testid="tab-datos">Datos</TabTrigger>
926 %           <TabTrigger value="folio" data-testid="tab-folio">Folio</TabTrigger>
927 %         </TabList>

```

```

928 %
929 %
930 %
931 %
932 %
933 %
934 %
935 %
936 %
937 %
938 %
939 %
940 %
941 %
942 %
943 %
944 %
945 %
946 %
947 %
948 %
949 %
950 %
951 %
952 %
953 %
954 %
955 %
956 %
957 %
958 %
959 %
960 %
961 %
962 %
963 %
964 %
965 %
966 %
967 %
968 %
969 %
970 %
971 %
972 %
973 %
974 %
975 %
976 %
977 %
978 %
979 %
980 %
981 %
982 %
983 %
984 %
985 %
986 %
987 %
988 %
989 %
990 %
991 %
992 %
993 %
994 %
995 %
996 %
997 %
998 %
999 %
1000 %
1001 %
1002 %
1003 %
1004 %
1005 %
1006 %
1007 %
1008 %
1009 %
1010 %
1011 %
1012 %
1013 %
1014 %
1015 %
1016 %
1017 %
1018 %
1019 %
1020 %

<TabsContent value="datos" className="space-y-4 mt-4">
  <div className="flex items-center gap-4 p-4 bg-muted rounded-lg">
    <div className="flex h-12 w-12 items-center justify-center rounded-full bg-primary/10 text-primary font...
      {reservation.guest?.firstName?.[0]}{reservation.guest?.lastName?.[0]}
    </div>
    <div className="flex-1">
      <p className="font-semibold text-lg" data-testid="text-guest-name">
        {reservation.guest?.firstName} {reservation.guest?.lastName}
      </p>
      <p className="text-sm text-muted-foreground">{reservation.guest?.email}</p>
      {reservation.guest?.phone && (
        <p className="text-sm text-muted-foreground">{reservation.guest?.phone}</p>
      )}
    </div>
    <Badge variant="outline">{reservation.source}</Badge>
  </div>

  {reservation.guest?.localidad && (
    <div className="p-3 border rounded-lg">
      <p className="text-sm text-muted-foreground mb-1">Ciudad</p>
      <p className="text-sm">
        {reservation.guest.localidad}
        {reservation.guest.nationality && ` - ${reservation.guest.nationality}`}
      </p>
    </div>
  )}

  <div className="grid grid-cols-4 gap-3 text-center">
    <div className="p-3 border rounded-lg">
      <DoorOpen className="h-4 w-4 mx-auto mb-1 text-muted-foreground" />
      <p className="text-sm text-muted-foreground">Hab.</p>
      <p className="font-semibold" data-testid="text-room-number">{reservation.room?.roomNumber}</p>
    </div>
    <div className="p-3 border rounded-lg">
      <Calendar className="h-4 w-4 mx-auto mb-1 text-muted-foreground" />
      <p className="text-sm text-muted-foreground">Check-in</p>
      <p className="font-semibold text-sm" data-testid="text-checkin">{reservation.checkInDate}</p>
    </div>
    <div className="p-3 border rounded-lg">
      <Calendar className="h-4 w-4 mx-auto mb-1 text-muted-foreground" />
      <p className="text-sm text-muted-foreground">Check-out</p>
      <p className="font-semibold text-sm" data-testid="text-checkout">{reservation.checkOutDate}</p>
    </div>
    <div className="p-3 border rounded-lg">
      <User className="h-4 w-4 mx-auto mb-1 text-muted-foreground" />
      <p className="text-sm text-muted-foreground">Noches</p>
      <p className="font-semibold" data-testid="text-nights">{reservation.nights}</p>
    </div>
  </div>

  <div className="grid grid-cols-2 gap-3">
    <div className="p-3 border rounded-lg">
      <p className="text-sm text-muted-foreground mb-1">Tipo de Habitación</p>
      <p className="font-medium">{reservation.room?.roomType?.name || "-"}</p>
    </div>
    <div className="p-3 border rounded-lg">
      <p className="text-sm text-muted-foreground mb-1">Huéspedes</p>
      <p className="font-medium">{reservation.numberOfGuests} persona(s)</p>
    </div>
  </div>

  {reservation.notes && (
    <div className="p-3 border rounded-lg">
      <p className="text-sm text-muted-foreground mb-1">Notas</p>
      <p className="text-sm">{reservation.notes}</p>
    </div>
  )}
</TabsContent>

<TabsContent value="folio" className="space-y-4 mt-4">
  <div className="grid grid-cols-2 gap-3">
    <div className="p-3 border rounded-lg">
      <p className="text-sm text-muted-foreground mb-1">Plan Tarifario</p>
      <p className="font-medium">{reservation.ratePlan?.name || "Tarifa Base"}</p>
    </div>
    <div className="p-3 border rounded-lg">
      <p className="text-sm text-muted-foreground mb-1">Estado</p>
      <ReservationStatusBadge status={reservation.status} />
    </div>
  </div>

  <div className="border rounded-lg">
    <div className="p-3 border-b bg-muted/50">
      <h4 className="font-semibold">Resumen de Alojamiento</h4>
    </div>
    <div className="p-3">
      <div className="flex justify-between text-sm mb-2">
        <span>Tarifa por noche</span>
        <span>${reservation.finalRatePerNight || 0}</span>
      </div>
      {reservation.discountType !== "none" && (
        <div className="flex justify-between text-sm mb-2 text-muted-foreground">

```

client/src/pages/reservations.tsx (cont.)

```
1021 %         <span>Descuento aplicado ({reservation.discountType === "percent" ? `${reservation.discountValue}...
1022 %         <span>~</span>
1023 %       </div>
1024 %     )}
1025 %     <div className="flex justify-between text-sm mb-2">
1026 %       <span>Cantidad de noches</span>
1027 %       <span>x {reservation.nights}</span>
1028 %     </div>
1029 %     <div className="flex justify-between font-semibold pt-2 border-t">
1030 %       <span>Subtotal Alojamiento</span>
1031 %       <span data-testid="text-subtotal-room">${subtotalRoom.toFixed(2)}</span>
1032 %     </div>
1033 %   </div>
1034 % </div>
1035 %
1036 % <div className="border rounded-lg">
1037 %   <div className="flex items-center justify-between p-3 border-b bg-muted/50">
1038 %     <h4 className="font-semibold">Consumos / Cargos Adicionales</h4>
1039 %     <Button size="sm" variant="outline" onClick={() => setShowAddCharge(!showAddCharge)} data-testid="but...
1040 %     <Plus className="h-4 w-4 mr-1" />
1041 %     Agregar
1042 %   </Button>
1043 % </div>
1044 %
1045 % {showAddCharge && (
1046 %   <div className="p-3 border-b bg-muted/30">
1047 %     <div className="grid grid-cols-4 gap-2">
1048 %       <Input
1049 %         placeholder="Descripción"
1050 %         value={newCharge.description}
1051 %         onChange={(e) => setNewCharge({ ...newCharge, description: e.target.value })}
1052 %         className="col-span-2"
1053 %         data-testid="input-charge-description"
1054 %       />
1055 %       <Input
1056 %         type="number"
1057 %         placeholder="Monto"
1058 %         value={newCharge.amount}
1059 %         onChange={(e) => setNewCharge({ ...newCharge, amount: e.target.value })}
1060 %         data-testid="input-charge-amount"
1061 %       />
1062 %       <Select
1063 %         value={newCharge.category}
1064 %         onValueChange={(value) => setNewCharge({ ...newCharge, category: value as typeof newCharge.cate...
1065 %       >
1066 %         <SelectTrigger data-testid="select-charge-category">
1067 %           <SelectValue />
1068 %         </SelectTrigger>
1069 %         <SelectContent>
1070 %           <SelectItem value="restaurant">Restaurante</SelectItem>
1071 %           <SelectItem value="spa">Spa</SelectItem>
1072 %           <SelectItem value="minibar">Minibar</SelectItem>
1073 %           <SelectItem value="otros">Otros</SelectItem>
1074 %           <SelectItem value="adjustment">Ajuste</SelectItem>
1075 %         </SelectContent>
1076 %       </Select>
1077 %     </div>
1078 %     <div className="flex justify-end gap-2 mt-2">
1079 %       <Button size="sm" variant="ghost" onClick={() => setShowAddCharge(false)}>
1080 %         Cancelar
1081 %       </Button>
1082 %       <Button
1083 %         size="sm"
1084 %         onClick={handleAddCharge}
1085 %         disabled={addChargeMutation.isPending}
1086 %         data-testid="button-confirm-charge"
1087 %       >
1088 %         Confirmar
1089 %       </Button>
1090 %     </div>
1091 %   </div>
1092 % )}
1093 %
1094 % <div className="divide-y max-h-[150px] overflow-y-auto">
1095 %   {consumptionCharges.map((charge) => (
1096 %     <div key={charge.id} className="flex items-center justify-between p-3 text-sm" data-testid={`charge...
1097 %     <div className="flex items-center gap-2 flex-wrap">
1098 %       <Badge variant="outline" className="text-xs">{categoryLabels[charge.category]}</Badge>
1099 %       <span>{charge.description}</span>
1100 %       <span className="text-muted-foreground text-xs">({charge.date})</span>
1101 %     </div>
1102 %     <div className="flex items-center gap-2">
1103 %       <span className="font-medium">${charge.amount}</span>
1104 %       <Button
1105 %         size="icon"
1106 %         variant="ghost"
1107 %         className="h-6 w-6"
1108 %         onClick={() => setTransferringChargeId(charge.id)}
1109 %         title="Transferir a otra habitación"
1110 %         data-testid={`button-transfer-charge-${charge.id}`}
1111 %       >
1112 %         <ArrowRightLeft className="h-3 w-3 text-blue-600" />
1113 %     </Button>
```

client/src/pages/reservations.tsx (cont.)

```
1114 %         <Button
1115 %             size="icon"
1116 %             variant="ghost"
1117 %             className="h-6 w-6"
1118 %             onClick={() => deleteChargeMutation.mutate(charge.id)}
1119 %             data-testid={`button-delete-charge-${charge.id}`}
1120 %         >
1121 %             <Trash2 className="h-3 w-3 text-destructive" />
1122 %         </Button>
1123 %     </div>
1124 % </div>
1125 % ))}
1126 % {consumptionCharges.length === 0 && (
1127 %     <div className="p-3 text-sm text-muted-foreground text-center">
1128 %         Sin consumos adicionales
1129 %     </div>
1130 % )}
1131 % </div>
1132 % <div className="flex justify-between p-3 border-t text-sm font-medium">
1133 %     <span>Total Consumos</span>
1134 %     <span data-testid="text-total-consumptions">${totalConsumptions.toFixed(2)}</span>
1135 % </div>
1136 % </div>
1137 %
1138 % <div className="border rounded-lg">
1139 %     <div className="flex items-center justify-between p-3 border-b bg-muted/50">
1140 %         <h4 className="font-semibold">Pagos / Anticipos</h4>
1141 %         <Button size="sm" variant="outline" onClick={() => setShowAddPayment(!showAddPayment)} data-testid="b...
1142 %             <Plus className="h-4 w-4 mr-1" />
1143 %             Registrar Pago
1144 %         </Button>
1145 %     </div>
1146 %
1147 %     {showAddPayment && (
1148 %         <div className="p-3 border-b bg-muted/30">
1149 %             <div className="grid grid-cols-2 gap-2 mb-2">
1150 %                 <div className="relative">
1151 %                     <span className="absolute left-3 top-1/2 -translate-y-1/2 text-muted-foreground">$/span>
1152 %                     <Input
1153 %                         type="number"
1154 %                         placeholder="Monto"
1155 %                         value={newPayment.amount}
1156 %                         onChange={(e) => setNewPayment({ ...newPayment, amount: e.target.value })}
1157 %                         className="pl-7"
1158 %                         data-testid="input-payment-amount"
1159 %                     />
1160 %                 </div>
1161 %                 <Select
1162 %                     value={newPayment.method}
1163 %                     onValueChange={(value) => setNewPayment({ ...newPayment, method: value as PaymentMethod })}
1164 %                 >
1165 %                     <SelectTrigger data-testid="select-payment-method">
1166 %                         <SelectValue />
1167 %                     </SelectTrigger>
1168 %                     <SelectContent>
1169 %                         <SelectItem value="efectivo">Efectivo</SelectItem>
1170 %                         <SelectItem value="tarjeta_debito">Tarjeta Débito</SelectItem>
1171 %                         <SelectItem value="tarjeta_credito">Tarjeta Crédito</SelectItem>
1172 %                         <SelectItem value="transferencia">Transferencia</SelectItem>
1173 %                         <SelectItem value="mercadopago">MercadoPago</SelectItem>
1174 %                         <SelectItem value="cuenta_corriente">Cuenta Corriente</SelectItem>
1175 %                     </SelectContent>
1176 %                 </Select>
1177 %             </div>
1178 %             <Input
1179 %                 placeholder="Referencia (Nº comprobante, tarjeta, etc.)"
1180 %                 value={newPayment.reference}
1181 %                 onChange={(e) => setNewPayment({ ...newPayment, reference: e.target.value })}
1182 %                 className="mb-2"
1183 %                 data-testid="input-payment-reference"
1184 %             />
1185 %             <div className="flex justify-end gap-2">
1186 %                 <Button size="sm" variant="ghost" onClick={() => setShowAddPayment(false)}>
1187 %                     Cancelar
1188 %                 </Button>
1189 %                 <Button
1190 %                     size="sm"
1191 %                     onClick={handleAddPayment}
1192 %                     disabled={addPaymentMutation.isPending}
1193 %                     data-testid="button-confirm-payment"
1194 %                 >
1195 %                     Confirmar Pago
1196 %                 </Button>
1197 %             </div>
1198 %         </div>
1199 %     )}
1200 %
1201 % <div className="divide-y max-h-[120px] overflow-y-auto">
1202 %     {payments?.map((payment) => (
1203 %         <div key={payment.id} className="flex items-center justify-between p-3 text-sm" data-testid={`payme...
1204 %             <div className="flex items-center gap-2 flex-wrap">
1205 %                 <Badge variant="outline" className="text-xs">{paymentMethodLabels[payment.method]}</Badge>
1206 %                 {payment.reference && <span className="text-muted-foreground">{payment.reference}</span>}
```

client/src/pages/reservations.tsx (cont.)

```
1207 %         <span className="text-muted-foreground text-xs">{payment.date}</span>
1208 %       </div>
1209 %       <div className="flex items-center gap-2">
1210 %         <span className="font-medium text-green-600">${parseFloat(payment.amount).toFixed(2)}</span>
1211 %         <Button
1212 %           size="icon"
1213 %           variant="ghost"
1214 %           className="h-6 w-6"
1215 %           onClick={() => deletePaymentMutation.mutate(payment.id)}
1216 %           data-testid={`button-delete-payment-${payment.id}`}
1217 %         >
1218 %           <Trash2 className="h-3 w-3 text-destructive" />
1219 %         </Button>
1220 %       </div>
1221 %     </div>
1222 %   )}}
1223 %   {!!payments || payments.length === 0} && (
1224 %     <div className="p-3 text-sm text-muted-foreground text-center">
1225 %       Sin pagos registrados
1226 %     </div>
1227 %   )}
1228 % </div>
1229 % <div className="flex justify-between p-3 border-t text-sm font-medium">
1230 %   <span>Total Pagado</span>
1231 %   <span className="text-green-600" data-testid="text-total-payments">${totalPayments.toFixed(2)}</span>
1232 % </div>
1233 % </div>
1234 %
1235 % <div className="border rounded-lg bg-primary/5">
1236 %   <div className="p-4">
1237 %     <div className="flex justify-between text-sm mb-1">
1238 %       <span>Subtotal Alojamiento</span>
1239 %       <span>${subtotalRoom.toFixed(2)}</span>
1240 %     </div>
1241 %     <div className="flex justify-between text-sm mb-1">
1242 %       <span>+ Consumos</span>
1243 %       <span>${totalConsumptions.toFixed(2)}</span>
1244 %     </div>
1245 %     <div className="flex justify-between text-sm mb-2 border-b pb-2">
1246 %       <span>- Pagos/Anticipos</span>
1247 %       <span className="text-green-600">-${totalPayments.toFixed(2)}</span>
1248 %     </div>
1249 %     <div className="flex justify-between font-bold text-lg">
1250 %       <span>SALDO PENDIENTE</span>
1251 %       <span className={balance > 0 ? "text-destructive" : "text-green-600"} data-testid="text-balance">
1252 %         ${balance.toFixed(2)}
1253 %       </span>
1254 %     </div>
1255 %   </div>
1256 % </div>
1257 % </TabsContent>
1258 % </Tabs>
1259 %
1260 % <DialogFooter className="gap-2 sm:justify-between">
1261 %   <div>
1262 %     {reservation.status !== "cancelled" && reservation.status !== "checked_out" && (
1263 %       <Button
1264 %         variant="destructive"
1265 %         onClick={() => {
1266 %           onChange(false);
1267 %           onCancel();
1268 %         }}
1269 %         data-testid="button-cancel-from-detail"
1270 %       >
1271 %         <X className="mr-2 h-4 w-4" />
1272 %         Anular Reserva
1273 %       </Button>
1274 %     )}
1275 %   </div>
1276 %   <Button variant="outline" onClick={() => onChange(false)}>
1277 %     Cerrar
1278 %   </Button>
1279 % </DialogFooter>
1280 % </DialogContent>
1281 %
1282 % /* Transfer Charge Dialog */
1283 % <Dialog open={transferringChargeId !== null} onChange={open => {
1284 %   if (!open) {
1285 %     setTransferringChargeId(null);
1286 %     setTargetReservationId("");
1287 %   }
1288 % }}>
1289 %   <DialogContent className="sm:max-w-[450px]">
1290 %     <DialogHeader>
1291 %       <DialogTitle className="flex items-center gap-2">
1292 %         <ArrowRightLeft className="h-5 w-5" />
1293 %         Transferir Cargo
1294 %       </DialogTitle>
1295 %       <DialogDescription>
1296 %         Seleccione la habitación destino para transferir este cargo
1297 %       </DialogDescription>
1298 %     </DialogHeader>
1299 %   </DialogContent>
```


client/src/pages/reservations.tsx (cont.)

```
1300 %         <div className="space-y-4 py-4">
1301 %             {transferringChargeId && (() => {
1302 %                 const chargeToTransfer = consumptionCharges.find(c => c.id === transferringChargeId);
1303 %                 if (!chargeToTransfer) return null;
1304 %                 return (
1305 %                     <div className="p-3 bg-muted rounded-lg">
1306 %                         <p className="text-sm font-medium mb-1">Cargo a transferir:</p>
1307 %                         <div className="flex justify-between text-sm">
1308 %                             <span>{chargeToTransfer.description}</span>
1309 %                             <span className="font-semibold">${chargeToTransfer.amount}</span>
1310 %                         </div>
1311 %                     </div>
1312 %                 );
1313 %             })()
1314 %
1315 %         <div className="space-y-2">
1316 %             <label className="text-sm font-medium">Habitación destino</label>
1317 %             {isActiveReservationsError ? (
1318 %                 <div className="p-3 border border-destructive/50 bg-destructive/10 rounded-lg text-sm text-destructive">
1319 %                     Error al cargar las habitaciones disponibles. Por favor, intente nuevamente.
1320 %                 </div>
1321 %             ) : (
1322 %                 <Select value={targetReservationId} onChange={setTargetReservationId} disabled={isActiveReservat...
1323 %                     <SelectTrigger data-testid="select-target-reservation">
1324 %                         <SelectValue placeholder={isActiveReservationsLoading ? "Cargando..." : "Seleccionar habitación...."}
1325 %                     </SelectTrigger>
1326 %                     <SelectContent>
1327 %                         {activeReservations?.map((r) => (
1328 %                             <SelectItem key={r.id} value={r.id}>
1329 %                                 Hab. {r.room?.roomNumber} - {r.guest?.firstName} {r.guest?.lastName}
1330 %                             </SelectItem>
1331 %                         ))}
1332 %                         {(!activeReservations || activeReservations.length === 0) && !isActiveReservationsLoading && (
1333 %                             <div className="p-2 text-sm text-muted-foreground text-center">
1334 %                                 No hay otras habitaciones activas
1335 %                             </div>
1336 %                         )}
1337 %                     </SelectContent>
1338 %                 </Select>
1339 %             )}
1340 %         </div>
1341 %     </div>
1342 %
1343 %     <DialogFooter>
1344 %         <Button variant="outline" onClick={() => {
1345 %             setTransferringChargeId(null);
1346 %             setTargetReservationId("");
1347 %         }}>
1348 %             Cancelar
1349 %         </Button>
1350 %         <Button
1351 %             onClick={handleTransferCharge}
1352 %             disabled={!targetReservationId || transferChargeMutation.isPending}
1353 %             data-testid="button-confirm-transfer"
1354 %             >
1355 %             {transferChargeMutation.isPending ? "Transfiriendo..." : "Transferir Cargo"}
1356 %         </Button>
1357 %     </DialogFooter>
1358 % </DialogContent>
1359 % </Dialog>
1360 % </Dialog>
1361 % );
1362 % }
1363 %
1364 % function CancelReservationDialog({
1365 %     reservation,
1366 %     open,
1367 %     onOpenChange,
1368 %     onSuccess,
1369 % }: {
1370 %     reservation: ReservationWithDetails;
1371 %     open: boolean;
1372 %     onOpenChange: (open: boolean) => void;
1373 %     onSuccess: () => void;
1374 % }) {
1375 %     const { toast } = useToast();
1376 %     const [reason, setReason] = useState("");
1377 %     const [cancelledBy, setCancelledBy] = useState("");
1378 %
1379 %     const cancelMutation = useMutation({
1380 %         mutationFn: async () => {
1381 %             return apiRequest("POST", `/api/reservations/${reservation.id}/cancel`, {
1382 %                 reason,
1383 %                 cancelledBy: cancelledBy || "Usuario del Sistema",
1384 %             });
1385 %         },
1386 %         onSuccess: () => {
1387 %             queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
1388 %             queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
1389 %             queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
1390 %             queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
1391 %             queryClient.invalidateQueries({ queryKey: ["/api/cancelled-reservations"], exact: false });
1392 %             toast({
```

client/src/pages/reservations.tsx (cont.)

```
1393 %         title: "Reserva anulada",
1394 %         description: "La reserva ha sido anulada y registrada en el log de cancelaciones.",
1395 %     });
1396 %     setReason("");
1397 %     setCancelledBy("");
1398 %     onOpenChange(false);
1399 %     onSuccess();
1400 % },
1401 % onError: () => {
1402 %     toast({
1403 %         title: "Error",
1404 %         description: "No se pudo anular la reserva.",
1405 %         variant: "destructive",
1406 %     });
1407 % },
1408 % });
1409 %
1410 % const handleCancel = () => {
1411 %     if (!reason.trim()) {
1412 %         toast({
1413 %             title: "Motivo requerido",
1414 %             description: "Por favor, ingrese el motivo de la anulación.",
1415 %             variant: "destructive",
1416 %         });
1417 %         return;
1418 %     }
1419 %     cancelMutation.mutate();
1420 % };
1421 %
1422 % return (
1423 %     <Dialog open={open} onOpenChange={onOpenChange}>
1424 %         <DialogContent className="sm:max-w-[500px]">
1425 %             <DialogHeader>
1426 %                 <DialogTitle className="flex items-center gap-2 text-destructive">
1427 %                     <X className="h-5 w-5" />
1428 %                     Anular Reserva
1429 %                 </DialogTitle>
1430 %                 <DialogDescription>
1431 %                     <strong>Anular</strong> cambia el estado a "Cancelada" y registra el motivo en el historial.
1432 %                     La reserva NO se elimina del sistema.
1433 %                 </DialogDescription>
1434 %             </DialogHeader>
1435 %
1436 %             <div className="space-y-4 py-4">
1437 %                 <div className="p-3 bg-muted rounded-lg space-y-2">
1438 %                     <div className="flex items-center justify-between">
1439 %                         <span className="text-sm text-muted-foreground">Código:</span>
1440 %                         <span className="font-medium">{reservation.reservationCode}</span>
1441 %                     </div>
1442 %                     <div className="flex items-center justify-between">
1443 %                         <span className="text-sm text-muted-foreground">Huésped:</span>
1444 %                         <span className="font-medium">
1445 %                             {reservation.guest?.firstName} {reservation.guest?.lastName}
1446 %                         </span>
1447 %                     </div>
1448 %                     <div className="flex items-center justify-between">
1449 %                         <span className="text-sm text-muted-foreground">Habitación:</span>
1450 %                         <span className="font-medium">{reservation.room?.roomNumber}</span>
1451 %                     </div>
1452 %                     <div className="flex items-center justify-between">
1453 %                         <span className="text-sm text-muted-foreground">Fechas:</span>
1454 %                         <span className="font-medium">
1455 %                             {reservation.checkInDate} - {reservation.checkOutDate}
1456 %                         </span>
1457 %                     </div>
1458 %                 </div>
1459 %
1460 %                 <div className="space-y-2">
1461 %                     <Label htmlFor="cancelledBy">Anulado por</Label>
1462 %                     <Input
1463 %                         id="cancelledBy"
1464 %                         placeholder="Nombre del usuario que anula"
1465 %                         value={cancelledBy}
1466 %                         onChange={(e) => setCancelledBy(e.target.value)}
1467 %                         data-testid="input-cancelled-by"
1468 %                     />
1469 %                 </div>
1470 %
1471 %                 <div className="space-y-2">
1472 %                     <Label htmlFor="reason">Motivo de anulación *</Label>
1473 %                     <Textarea
1474 %                         id="reason"
1475 %                         placeholder="Ingrese el motivo de la anulación..."
1476 %                         value={reason}
1477 %                         onChange={(e) => setReason(e.target.value)}
1478 %                         rows={3}
1479 %                         data-testid="input-cancel-reason"
1480 %                     />
1481 %                 </div>
1482 %             </div>
1483 %
1484 %             <DialogFooter className="gap-2">
1485 %                 <Button variant="outline" onClick={() => onOpenChange(false)}>
```

client/src/pages/reservations.tsx (cont.)

```
1486 %         Volver
1487 %       </Button>
1488 %       <Button
1489 %         variant="destructive"
1490 %         onClick={handleCancel}
1491 %         disabled={cancelMutation.isPending}
1492 %         data-testid="button-confirm-cancel"
1493 %       >
1494 %         {cancelMutation.isPending ? "Anulando..." : "Confirmar Anulación"}
1495 %       </Button>
1496 %     </DialogFooter>
1497 %   </DialogContent>
1498 % </Dialog>
1499 % );
1500 % }
1501 %
1502 % export default function ReservationsPage() {
1503 %   const { toast } = useToast();
1504 %   const [, navigate] = useLocation();
1505 %   const searchParams = useSearch();
1506 %   const [searchQuery, setSearchQuery] = useState("");
1507 %   const [statusFilter, setStatusFilter] = useState<string>("all");
1508 %   const [dialogOpen, setDialogOpen] = useState(false);
1509 %   const [detailDialogOpen, setDetailDialogOpen] = useState(false);
1510 %   const [cancelDialogOpen, setCancelDialogOpen] = useState(false);
1511 %   const [duplicateDialogOpen, setDuplicateDialogOpen] = useState(false);
1512 %   const [duplicateCheckIn, setDuplicateCheckIn] = useState("");
1513 %   const [duplicateCheckOut, setDuplicateCheckOut] = useState("");
1514 %   const [selectedReservation, setSelectedReservation] = useState<ReservationWithDetails | undefined>();
1515 %
1516 %   const { data: reservations, isLoading } = useQuery<ReservationWithDetails[]>({
1517 %     queryKey: ["/api/reservations"],
1518 %   });
1519 %
1520 %   useEffect(() => {
1521 %     const params = new URLSearchParams(searchParams);
1522 %     const viewId = params.get("view");
1523 %     if (viewId && reservations) {
1524 %       const reservation = reservations.find(r => r.id === viewId);
1525 %       if (reservation) {
1526 %         setSelectedReservation(reservation);
1527 %         setDetailDialogOpen(true);
1528 %         navigate("/reservations", { replace: true });
1529 %       }
1530 %     }
1531 %   }, [searchParams, reservations, navigate]);
1532 %
1533 %   const { data: guests } = useQuery<Guest[]>({
1534 %     queryKey: ["/api/guests"],
1535 %   });
1536 %
1537 %   const { data: rooms } = useQuery<RoomWithType[]>({
1538 %     queryKey: ["/api/rooms"],
1539 %   });
1540 %
1541 %   const { data: roomTypes } = useQuery<RoomType[]>({
1542 %     queryKey: ["/api/room-types"],
1543 %   });
1544 %
1545 %   const updateStatusMutation = useMutation({
1546 %     mutationFn: async ({ id, status }: { id: string; status: ReservationStatus }) => {
1547 %       return apiRequest("PATCH", `/api/reservations/${id}`, { status });
1548 %     },
1549 %     onSuccess: () => {
1550 %       queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
1551 %       queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
1552 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
1553 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
1554 %       toast({ title: "Estado actualizado", description: "El estado de la reserva ha sido actualizado." });
1555 %     },
1556 %   });
1557 %
1558 %   const deleteMutation = useMutation({
1559 %     mutationFn: async (id: string) => {
1560 %       return apiRequest("DELETE", `/api/reservations/${id}`, undefined);
1561 %     },
1562 %     onSuccess: () => {
1563 %       queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
1564 %       queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
1565 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
1566 %       toast({ title: "Reserva eliminada", description: "La reserva ha sido eliminada del sistema." });
1567 %     },
1568 %   });
1569 %
1570 %   const duplicateMutation = useMutation({
1571 %     mutationFn: async ({ id, checkInDate, checkOutDate }: { id: string; checkInDate: string; checkOutDate: string }) ...
1572 %     return apiRequest("POST", `/api/reservations/${id}/duplicate`, { checkInDate, checkOutDate });
1573 %   },
1574 %   onSuccess: () => {
1575 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
1576 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
1577 %     queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
1578 %     setDuplicateDialogOpen(false);
1579 %   }
1580 % });
```

client/src/pages/reservations.tsx (cont.)

```
1579 %         setDuplicateCheckIn("");
1580 %         setDuplicateCheckOut("");
1581 %         toast({ title: "Reserva duplicada", description: "Se ha creado una nueva reserva con los datos copiados." });
1582 %     },
1583 %     onError: (error: any) => {
1584 %         toast({ title: "Error", description: error.message || "No se pudo duplicar la reserva", variant: "destructive" });
1585 %     },
1586 % });
1587 %
1588 % const filteredReservations = reservations?.filter((res) => {
1589 %     const guestName = `${res.guest?.firstName} ${res.guest?.lastName}`.toLowerCase();
1590 %     const matchesSearch =
1591 %         guestName.includes(searchQuery.toLowerCase()) ||
1592 %         res.room?.roomNumber?.toLowerCase().includes(searchQuery.toLowerCase());
1593 %     const matchesStatus = statusFilter === "all" || res.status === statusFilter;
1594 %     return matchesSearch && matchesStatus;
1595 % });
1596 %
1597 % const handleEditReservation = (reservation: ReservationWithDetails) => {
1598 %     setSelectedReservation(reservation);
1599 %     setDialogOpen(true);
1600 % };
1601 %
1602 % const handleViewReservation = (reservation: ReservationWithDetails) => {
1603 %     setSelectedReservation(reservation);
1604 %     setDetailDialogOpen(true);
1605 % };
1606 %
1607 % const handleDuplicateReservation = (reservation: ReservationWithDetails) => {
1608 %     setSelectedReservation(reservation);
1609 %     setDuplicateCheckIn("");
1610 %     setDuplicateCheckOut("");
1611 %     setDuplicateDialogOpen(true);
1612 % };
1613 %
1614 % const handleNewReservation = () => {
1615 %     setSelectedReservation(undefined);
1616 %     setDialogOpen(true);
1617 % };
1618 %
1619 % const handleCancelReservation = (reservation: ReservationWithDetails) => {
1620 %     setSelectedReservation(reservation);
1621 %     setCancelDialogOpen(true);
1622 % };
1623 %
1624 % return (
1625 %     <div className="flex flex-col gap-6 p-6">
1626 %         <div className="flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-between">
1627 %             <div>
1628 %                 <h1 className="text-3xl font-bold tracking-tight" data-testid="text-reservations-title">
1629 %                     Reservas
1630 %                 </h1>
1631 %                 <p className="text-muted-foreground">Gestiona todas las reservaciones del hotel</p>
1632 %             </div>
1633 %             <Button onClick={handleNewReservation} data-testid="button-new-reservation">
1634 %                 <Plus className="mr-2 h-4 w-4" />
1635 %                 Nueva Reserva
1636 %             </Button>
1637 %         </div>
1638 %
1639 %         {/* Filters */}
1640 %         <Card>
1641 %             <CardContent className="p-4">
1642 %                 <div className="flex flex-col gap-4 sm:flex-row sm:items-center">
1643 %                     <div className="relative flex-1">
1644 %                         <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
1645 %                         <Input
1646 %                             placeholder="Buscar por huésped o habitación..."
1647 %                             value={searchQuery}
1648 %                             onChange={(e) => setSearchQuery(e.target.value)}
1649 %                             className="pl-10"
1650 %                             data-testid="input-search-reservations"
1651 %                         />
1652 %                     </div>
1653 %                     <Select value={statusFilter} onChange={setStatusFilter}>
1654 %                         <SelectTrigger className="w-[180px]" data-testid="select-filter-status">
1655 %                             <Filter className="mr-2 h-4 w-4" />
1656 %                             <SelectValue placeholder="Estado" />
1657 %                         </SelectTrigger>
1658 %                         <SelectContent>
1659 %                             <SelectItem value="all">Todos</SelectItem>
1660 %                             <SelectItem value="pending">Pendientes</SelectItem>
1661 %                             <SelectItem value="confirmed">Confirmadas</SelectItem>
1662 %                             <SelectItem value="checked_in">Check-in</SelectItem>
1663 %                             <SelectItem value="checked_out">Check-out</SelectItem>
1664 %                             <SelectItem value="cancelled">Canceladas</SelectItem>
1665 %                         </SelectContent>
1666 %                     </Select>
1667 %                 </div>
1668 %             </CardContent>
1669 %         </Card>
1670 %
1671 %         {/* Reservations Table */}
```

```

1672 % {isLoading ? (
1673 %   <Card>
1674 %     <CardContent className="p-6">
1675 %       <div className="space-y-4">
1676 %         {[...Array(5)].map((_, i) => (
1677 %           <Skeleton key={i} className="h-16" />
1678 %         ))}
1679 %       </div>
1680 %     </CardContent>
1681 %   </Card>
1682 % ) : filteredReservations && filteredReservations.length > 0 ? (
1683 %   <Card>
1684 %     <Table>
1685 %       <TableHeader>
1686 %         <TableRow>
1687 %           <TableHead>Huésped</TableHead>
1688 %           <TableHead>Habitación</TableHead>
1689 %           <TableHead>Check-in</TableHead>
1690 %           <TableHead>Check-out</TableHead>
1691 %           <TableHead>Huéspedes</TableHead>
1692 %           <TableHead>Estado</TableHead>
1693 %           <TableHead>Total</TableHead>
1694 %           <TableHead className="w-[50px]"></TableHead>
1695 %         </TableRow>
1696 %       </TableHeader>
1697 %       <TableBody>
1698 %         {filteredReservations.map((reservation) => (
1699 %           <TableRow>
1700 %             key={reservation.id}
1701 %             data-testid={`reservation-row-${reservation.id}`}
1702 %             className="cursor-pointer hover-elevate"
1703 %             onClick={() => handleViewReservation(reservation)}
1704 %           >
1705 %             <TableCell>
1706 %               <div className="flex items-center gap-3">
1707 %                 <div className="flex h-8 w-8 items-center justify-center rounded-full bg-primary/10 text-primar...
1708 %                   {reservation.guest?.firstName?.[0]}{reservation.guest?.lastName?.[0]}
1709 %                 </div>
1710 %                 <div>
1711 %                   <p className="font-medium">
1712 %                     {reservation.guest?.firstName} {reservation.guest?.lastName}
1713 %                   </p>
1714 %                   <p className="text-sm text-muted-foreground">{reservation.guest?.email}</p>
1715 %                 </div>
1716 %               </div>
1717 %             </TableCell>
1718 %             <TableCell>{reservation.room?.roomNumber}</TableCell>
1719 %             <TableCell>{reservation.checkInDate}</TableCell>
1720 %             <TableCell>{reservation.checkOutDate}</TableCell>
1721 %             <TableCell>{reservation.numberOfGuests}</TableCell>
1722 %             <TableCell>
1723 %               <ReservationStatusBadge status={reservation.status} />
1724 %             </TableCell>
1725 %             <TableCell className="font-medium">${reservation.totalRoomAmount || 0}</TableCell>
1726 %             <TableCell onClick={(e) => e.stopPropagation()}>
1727 %               <DropdownMenu>
1728 %                 <DropdownMenuTrigger asChild>
1729 %                   <Button variant="ghost" size="icon">
1730 %                     <MoreHorizontal className="h-4 w-4" />
1731 %                   </Button>
1732 %                 </DropdownMenuTrigger>
1733 %                 <DropdownMenuContent align="end">
1734 %                   <DropdownMenuItem onClick={() => handleViewReservation(reservation)}>
1735 %                     <Eye className="mr-2 h-4 w-4" />
1736 %                     Ver Detalle
1737 %                   </DropdownMenuItem>
1738 %                   <DropdownMenuItem onClick={() => handleEditReservation(reservation)}>
1739 %                     <Pencil className="mr-2 h-4 w-4" />
1740 %                     Editar
1741 %                   </DropdownMenuItem>
1742 %                   <DropdownMenuItem>
1743 %                     onClick={() => handleDuplicateReservation(reservation)}
1744 %                     data-testid={`duplicate-reservation-${reservation.id}`}
1745 %                   >
1746 %                     <Copy className="mr-2 h-4 w-4" />
1747 %                     Duplicar
1748 %                   </DropdownMenuItem>
1749 %                   <DropdownMenuSeparator />
1750 %                   {reservation.status === "pending" && (
1751 %                     <DropdownMenuItem>
1752 %                       onClick={() => updateStatusMutation.mutate({ id: reservation.id, status: "confirmed" })}
1753 %                     >
1754 %                       <Check className="mr-2 h-4 w-4 text-green-600" />
1755 %                       Confirmar
1756 %                     </DropdownMenuItem>
1757 %                   )}
1758 %                 </DropdownMenuContent>
1759 %               </DropdownMenu>
1760 %             </TableCell>
1761 %             >
1762 %               <LogIn className="mr-2 h-4 w-4 text-blue-600" />
1763 %               Hacer Check-in
1764 %             </DropdownMenuItem>

```

client/src/pages/reservations.tsx (cont.)

```
1765 %         })
1766 %       {reservation.status === "checked_in" && (
1767 %         <DropdownMenuItem
1768 %           onClick={() => updateStatusMutation.mutate({ id: reservation.id, status: "checked_out" })}
1769 %         >
1770 %           <LogOut className="mr-2 h-4 w-4 text-orange-600" />
1771 %           Hacer Check-out
1772 %         </DropdownMenuItem>
1773 %       )}
1774 %     {reservation.status !== "cancelled" && reservation.status !== "checked_out" && (
1775 %       <DropdownMenuItem
1776 %         onClick={() => handleCancelReservation(reservation)}
1777 %       >
1778 %         <X className="mr-2 h-4 w-4 text-red-600" />
1779 %         Anular Reserva
1780 %       </DropdownMenuItem>
1781 %     )}
1782 %     <DropdownMenuSeparator />
1783 %     <DropdownMenuItem
1784 %       className="text-destructive"
1785 %       onClick={() => deleteMutation.mutate(reservation.id)}
1786 %     >
1787 %       <Trash2 className="mr-2 h-4 w-4" />
1788 %       Eliminar
1789 %     </DropdownMenuItem>
1790 %   </DropdownMenuContent>
1791 % </DropdownMenu>
1792 % </TableCell>
1793 % </TableRow>
1794 %   )}
1795 % </TableBody>
1796 % </Table>
1797 % </Card>
1798 % ) : (
1799 %   <Card>
1800 %     <CardContent className="flex flex-col items-center justify-center py-12 text-center">
1801 %       <CalendarCheck className="h-16 w-16 text-muted-foreground/50 mb-4" />
1802 %       <h3 className="text-lg font-semibold mb-2">No hay reservas</h3>
1803 %       <p className="text-muted-foreground mb-4">
1804 %         {searchQuery || statusFilter !== "all"
1805 %           ? "No se encontraron reservas con los filtros aplicados."
1806 %           : "Comienza creando la primera reserva del hotel."}
1807 %       </p>
1808 %       {!searchQuery && statusFilter === "all" && (
1809 %         <Button onClick={handleNewReservation}>
1810 %           <Plus className="mr-2 h-4 w-4" />
1811 %           Crear Primera Reserva
1812 %         </Button>
1813 %       )}
1814 %     </CardContent>
1815 %   </Card>
1816 % )}
1817 %
1818 % {/* Reservation Form Dialog */}
1819 % <ReservationFormDialog
1820 %   reservation={selectedReservation}
1821 %   guests={guests || []}
1822 %   rooms={rooms || []}
1823 %   roomTypes={roomTypes || []}
1824 %   open={dialogOpen}
1825 %   onOpenChange={setDialogOpen}
1826 %   onSuccess={() => setSelectedReservation(undefined)}
1827 % />
1828 %
1829 % {/* Reservation Detail Dialog */}
1830 % {selectedReservation && (
1831 %   <ReservationDetailDialog
1832 %     reservation={selectedReservation}
1833 %     open={detailDialogOpen}
1834 %     onOpenChange={setDetailDialogOpen}
1835 %     onCancel={() => setCancelDialogOpen(true)}
1836 %   />
1837 % )}
1838 %
1839 % {/* Cancel Reservation Dialog */}
1840 % {selectedReservation && (
1841 %   <CancelReservationDialog
1842 %     reservation={selectedReservation}
1843 %     open={cancelDialogOpen}
1844 %     onOpenChange={setCancelDialogOpen}
1845 %     onSuccess={() => setSelectedReservation(undefined)}
1846 %   />
1847 % )}
1848 %
1849 % {/* Duplicate Reservation Dialog */}
1850 % <Dialog open={duplicateDialogOpen} onOpenChange={setDuplicateDialogOpen}>
1851 %   <DialogContent>
1852 %     <DialogHeader>
1853 %       <DialogTitle>Duplicar Reserva</DialogTitle>
1854 %       <DialogDescription>
1855 %         Crea una nueva reserva con los mismos datos del huésped y habitación.
1856 %         Selecciona las nuevas fechas para la reserva duplicada.
1857 %       </DialogDescription>
```

client/src/pages/reservations.tsx (cont.)

```
1858 %         </DialogHeader>
1859 %     {selectedReservation && (() => {
1860 %         const checkInDate = duplicateCheckIn ? new Date(duplicateCheckIn) : null;
1861 %         const checkOutDate = duplicateCheckOut ? new Date(duplicateCheckOut) : null;
1862 %         const today = new Date();
1863 %         today.setHours(0, 0, 0, 0);
1864 %
1865 %         const isCheckInPast = checkInDate && checkInDate < today;
1866 %         const isCheckOutBeforeCheckIn = checkInDate && checkOutDate <= checkInDate;
1867 %         const nights = checkInDate && checkOutDate && !isCheckOutBeforeCheckIn
1868 %             ? Math.ceil((checkOutDate.getTime() - checkInDate.getTime()) / (1000 * 60 * 60 * 24))
1869 %             : 0;
1870 %         const isValidDates = duplicateCheckIn && duplicateCheckOut && !isCheckInPast && !isCheckOutBeforeCheckIn...
1871 %
1872 %         return (
1873 %             <div className="space-y-4">
1874 %                 <div className="p-3 rounded-md bg-muted">
1875 %                     <p className="text-sm font-medium">Reserva original:</p>
1876 %                     <p className="text-sm text-muted-foreground">
1877 %                         {selectedReservation.guest?.firstName} {selectedReservation.guest?.lastName} - Hab. {selectedReservation.nights}
1878 %                     </p>
1879 %                 </div>
1880 %                 <div className="grid gap-4 sm:grid-cols-2">
1881 %                     <div className="space-y-2">
1882 %                         <Label htmlFor="duplicate-checkin">Nueva Fecha Entrada</Label>
1883 %                         <Input
1884 %                             id="duplicate-checkin"
1885 %                             type="date"
1886 %                             min={today.toISOString().split('T')[0]}
1887 %                             value={duplicateCheckIn}
1888 %                             onChange={(e) => setDuplicateCheckIn(e.target.value)}
1889 %                             data-testid="input-duplicate-checkin"
1890 %                         />
1891 %                         {isCheckInPast && (
1892 %                             <p className="text-xs text-destructive">La fecha no puede ser en el pasado</p>
1893 %                         )}
1894 %                     </div>
1895 %                     <div className="space-y-2">
1896 %                         <Label htmlFor="duplicate-checkout">Nueva Fecha Salida</Label>
1897 %                         <Input
1898 %                             id="duplicate-checkout"
1899 %                             type="date"
1900 %                             min={duplicateCheckIn || today.toISOString().split('T')[0]}
1901 %                             value={duplicateCheckOut}
1902 %                             onChange={(e) => setDuplicateCheckOut(e.target.value)}
1903 %                             data-testid="input-duplicate-checkout"
1904 %                         />
1905 %                         {isCheckOutBeforeCheckIn && (
1906 %                             <p className="text-xs text-destructive">La salida debe ser posterior a la entrada</p>
1907 %                         )}
1908 %                     </div>
1909 %                 </div>
1910 %                 {nights > 0 && (
1911 %                     <p className="text-sm text-muted-foreground">
1912 %                         Estancia: {nights} noche(s)
1913 %                     </p>
1914 %                 )}
1915 %             </div>
1916 %         );
1917 %     })());
1918 %     <DialogFooter className="gap-2">
1919 %         <Button variant="outline" onClick={() => setDuplicateDialogOpen(false)}>
1920 %             Cancelar
1921 %         </Button>
1922 %         <Button
1923 %             onClick={() => {
1924 %                 if (selectedReservation && duplicateCheckIn && duplicateCheckOut) {
1925 %                     const checkIn = new Date(duplicateCheckIn);
1926 %                     const checkOut = new Date(duplicateCheckOut);
1927 %                     const today = new Date();
1928 %                     today.setHours(0, 0, 0, 0);
1929 %
1930 %                     if (checkIn < today) {
1931 %                         toast({ title: "Error", description: "La fecha de entrada no puede ser en el pasado", variant: "d...
1932 %                     });
1933 %                     return;
1934 %                 }
1935 %                 if (checkOut <= checkIn) {
1936 %                     toast({ title: "Error", description: "La fecha de salida debe ser posterior a la entrada", varian...
1937 %                 });
1938 %                 return;
1939 %             }
1940 %             duplicateMutation.mutate({
1941 %                 id: selectedReservation.id,
1942 %                 checkInDate: duplicateCheckIn,
1943 %                 checkOutDate: duplicateCheckOut,
1944 %             });
1945 %         }
1946 %     }
1947 %     disabled={!duplicateCheckIn || !duplicateCheckOut || duplicateMutation.isPending}
1948 %     data-testid="button-confirm-duplicate"
1949 % >
1950 %     {duplicateMutation.isPending ? "Duplicando..." : "Crear Reserva"}
```

client/src/pages/reservations.tsx (cont.)

```
1951 %         </DialogFooter>
1952 %         </DialogContent>
1953 %       </Dialog>
1954 %     </div>
1955 %   );
1956 % }
1957 %
```



```
1 % import { useState, useMemo } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { useLocation } from "wouter";
4 % import {
5 %   CalendarPlus,
6 %   User,
7 %   Building2,
8 %   DoorOpen,
9 %   Calendar,
10 %   DollarSign,
11 %   Check,
12 %   Sunrise,
13 %   Sunset,
14 % } from "lucide-react";
15 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
16 % import { Button } from "@components/ui/button";
17 % import { Input } from "@components/ui/input";
18 % import { Badge } from "@components/ui/badge";
19 % import { Label } from "@components/ui/label";
20 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
21 % import { Textarea } from "@components/ui/textarea";
22 % import { Switch } from "@components/ui/switch";
23 % import { useToast } from "@hooks/use-toast";
24 % import { queryClient, apiRequest } from "@lib/queryClient";
25 % import { GuestSelector, CompanySelector } from "@components/entity-selector";
26 % import type { Guest, Company, RoomType, RoomWithType, RatePlan, InsertGuest, InsertCompany, BedType } from "@shared/s...
27 %
28 % export default function NewReservationPage() {
29 %   const { toast } = useToast();
30 %   const [, setLocation] = useLocation();
31 %
32 %   const [selectedGuest, setSelectedGuest] = useState<Guest | null>(null);
33 %   const [selectedCompany, setSelectedCompany] = useState<Company | null>(null);
34 %   const [selectedRoomTypeId, setSelectedRoomTypeId] = useState<string>("");
35 %   const [selectedRoomId, setSelectedRoomId] = useState<string>("");
36 %   const [selectedRatePlanId, setSelectedRatePlanId] = useState<string>("");
37 %   const [checkInDate, setCheckInDate] = useState<string>(() => {
38 %     const today = new Date();
39 %     return today.toISOString().split("T")[0];
40 %   });
41 %   const [checkOutDate, setCheckOutDate] = useState<string>(() => {
42 %     const tomorrow = new Date();
43 %     tomorrow.setDate(tomorrow.getDate() + 1);
44 %     return tomorrow.toISOString().split("T")[0];
45 %   });
46 %   const [numberOfGuests, setNumberOfGuests] = useState<number>(1);
47 %   const [notes, setNotes] = useState<string>("");
48 %   const [discountType, setDiscountType] = useState<string>("none");
49 %   const [discountValue, setDiscountValue] = useState<string>("0");
50 %   const [bedTypeId, setBedTypeId] = useState<number | null>(null);
51 %   const [bedTypeNotes, setBedTypeNotes] = useState<string>("");
52 %   const [earlyCheckIn, setEarlyCheckIn] = useState(false);
53 %   const [earlyCheckInTime, setEarlyCheckInTime] = useState("");
54 %   const [earlyCheckInCharge, setEarlyCheckInCharge] = useState("");
55 %   const [lateCheckOut, setLateCheckOut] = useState(false);
56 %   const [lateCheckOutTime, setLateCheckOutTime] = useState("");
57 %   const [lateCheckOutCharge, setLateCheckOutCharge] = useState("");
58 %
59 %   const { data: roomTypes } = useQuery<RoomType[]>({
60 %     queryKey: ["/api/room-types"],
61 %   });
62 %
63 %   const { data: rooms } = useQuery<RoomWithType[]>({
64 %     queryKey: ["/api/rooms"],
65 %   });
66 %
67 %   const { data: ratePlans } = useQuery<RatePlan[]>({
68 %     queryKey: ["/api/rate-plans"],
69 %   });
70 %
71 %   const { data: bedTypes } = useQuery<BedType[]>({
72 %     queryKey: ["/api/bed-types"],
73 %   });
74 %
75 %   const nights = useMemo(() => {
76 %     if (!checkInDate || !checkOutDate) return 0;
77 %     const start = new Date(checkInDate);
78 %     const end = new Date(checkOutDate);
79 %     const diff = Math.ceil((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24));
80 %     return diff > 0 ? diff : 0;
81 %   }, [checkInDate, checkOutDate]);
82 %
83 %   const availableRooms = rooms?.filter((room) =>
84 %     room.status === "available" &&
85 %     (selectedRoomTypeId ? room.roomTypeId === selectedRoomTypeId : true)
86 %   );
87 %
88 %   const applicableRatePlans = ratePlans?.filter((rp) =>
89 %     rp.roomTypeId === selectedRoomTypeId
90 %   );
```

```

91 %
92 %   const selectedRatePlan = ratePlans?.find((rp) => rp.id === selectedRatePlanId);
93 %
94 %   const baseRate = selectedRatePlan ? parseFloat(selectedRatePlan.baseRate) : 0;
95 %   const finalRate = useMemo(() => {
96 %     if (!baseRate) return 0;
97 %     if (discountType === "percent") {
98 %       return baseRate * (1 - parseFloat(discountValue) / 100);
99 %     } else if (discountType === "fixed") {
100 %       return Math.max(0, baseRate - parseFloat(discountValue));
101 %     }
102 %     return baseRate;
103 %   }, [baseRate, discountType, discountValue]);
104 %
105 %   const totalAmount = (finalRate * nights).toFixed(2);
106 %
107 %   const createGuestMutation = useMutation({
108 %     mutationFn: async (guest: InsertGuest): Promise<Guest> => {
109 %       const res = await apiRequest("POST", "/api/guests", guest);
110 %       return res.json();
111 %     },
112 %     onSuccess: (newGuest: Guest) => {
113 %       queryClient.invalidateQueries({ queryKey: ["/api/guests"] });
114 %       setSelectedGuest(newGuest);
115 %       toast({
116 %         title: "Huesped creado",
117 %         description: `${newGuest.firstName} ${newGuest.lastName} ha sido registrado.`,
118 %       });
119 %     },
120 %     onError: () => {
121 %       toast({
122 %         title: "Error",
123 %         description: "No se pudo crear el huesped.",
124 %         variant: "destructive",
125 %       });
126 %     },
127 %   });
128 %
129 %   const createCompanyMutation = useMutation({
130 %     mutationFn: async (company: InsertCompany): Promise<Company> => {
131 %       const res = await apiRequest("POST", "/api/companies", company);
132 %       return res.json();
133 %     },
134 %     onSuccess: (newCompany: Company) => {
135 %       queryClient.invalidateQueries({ queryKey: ["/api/companies"] });
136 %       setSelectedCompany(newCompany);
137 %       toast({
138 %         title: "Empresa creada",
139 %         description: `${newCompany.razonSocial} ha sido registrada.`,
140 %       });
141 %     },
142 %     onError: () => {
143 %       toast({
144 %         title: "Error",
145 %         description: "No se pudo crear la empresa.",
146 %         variant: "destructive",
147 %       });
148 %     },
149 %   });
150 %
151 %   const createReservationMutation = useMutation({
152 %     mutationFn: async () => {
153 %       const res = await apiRequest("POST", "/api/reservations", {
154 %         reservationCode: "",
155 %         guestId: selectedGuest!.id,
156 %         companyId: selectedCompany?.id || null,
157 %         roomTypeId: selectedRoomTypeId,
158 %         roomId: selectedRoomId,
159 %         ratePlanId: selectedRatePlanId || null,
160 %         checkInDate,
161 %         checkOutDate,
162 %         nights,
163 %         baseRatePerNight: baseRate.toFixed(2),
164 %         discountType,
165 %         discountValue,
166 %         finalRatePerNight: finalRate.toFixed(2),
167 %         totalRoomAmount: totalAmount,
168 %         status: "confirmed",
169 %         source: selectedCompany ? "empresa" : "directo",
170 %         numberOfGuests,
171 %         notes: notes || null,
172 %         bedTypeId: bedTypeId || null,
173 %         bedTypeNotes: bedTypeNotes || null,
174 %         earlyCheckIn,
175 %         earlyCheckInTime: earlyCheckInTime || null,
176 %         earlyCheckInCharge: earlyCheckInCharge || null,
177 %         lateCheckOut,
178 %         lateCheckOutTime: lateCheckOutTime || null,
179 %         lateCheckOutCharge: lateCheckOutCharge || null,
180 %       });
181 %       return res.json();
182 %     },
183 %     onSuccess: () => {

```

client/src/pages/new-reservation.tsx (cont.)

```
184 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
185 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations/check-in"] });
186 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
187 %     queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
188 %     toast({
189 %       title: "Reserva creada",
190 %       description: "La reserva ha sido creada exitosamente.",
191 %     });
192 %     setLocation("/reservations");
193 %   },
194 %   onError: () => {
195 %     toast({
196 %       title: "Error",
197 %       description: "No se pudo crear la reserva. Intente nuevamente.",
198 %       variant: "destructive",
199 %     });
200 %   },
201 % });
202 %
203 % const canSubmit = selectedGuest && selectedRoomTypeId && selectedRoomId && selectedRatePlanId && nights > 0 && chec...
204 %
205 % return (
206 %   <div className="flex flex-col gap-6 p-6">
207 %     <div className="flex flex-col gap-1">
208 %       <h1 className="text-3xl font-bold tracking-tight" data-testid="text-new-reservation-title">
209 %         Nueva Reserva
210 %       </h1>
211 %       <p className="text-muted-foreground">Complete los datos para crear una nueva reserva</p>
212 %     </div>
213 %
214 %     <Card className="bg-primary/5 border-primary/20">
215 %       <CardContent className="flex items-center gap-4 p-4">
216 %         <div className="flex h-12 w-12 items-center justify-center rounded-full bg-primary/10">
217 %           <CalendarPlus className="h-6 w-6 text-primary" />
218 %         </div>
219 %         <div>
220 %           <h3 className="font-semibold">Crear Reserva</h3>
221 %           <p className="text-sm text-muted-foreground">
222 %             Complete todos los campos requeridos para registrar la reserva
223 %           </p>
224 %         </div>
225 %       </CardContent>
226 %     </Card>
227 %
228 %     <div className="grid gap-6 lg:grid-cols-2">
229 %       <div className="space-y-6">
230 %         <GuestSelector
231 %           selectedGuest={selectedGuest}
232 %           onSelect={setSelectedGuest}
233 %           onCreateNew={(guest) => createGuestMutation.mutate(guest)}
234 %           onClear={() => setSelectedGuest(null)}
235 %         />
236 %
237 %         <CompanySelector
238 %           selectedCompany={selectedCompany}
239 %           onSelect={setSelectedCompany}
240 %           onCreateNew={(company) => createCompanyMutation.mutate(company)}
241 %           onClear={() => setSelectedCompany(null)}
242 %         />
243 %       </div>
244 %       <Card>
245 %         <CardHeader className="pb-3">
246 %           <CardTitle className="text-base flex items-center gap-2">
247 %             <Calendar className="h-4 w-4" />
248 %             Fechas
249 %           </CardTitle>
250 %         </CardHeader>
251 %         <CardContent className="space-y-4">
252 %           <div className="grid grid-cols-2 gap-4">
253 %             <div className="space-y-2">
254 %               <Label>Fecha de Entrada</Label>
255 %               <Input
256 %                 type="date"
257 %                 value={checkInDate}
258 %                 onChange={(e) => setCheckInDate(e.target.value)}
259 %                 data-testid="input-checkin-date"
260 %               />
261 %             </div>
262 %             <div className="space-y-2">
263 %               <Label>Fecha de Salida</Label>
264 %               <Input
265 %                 type="date"
266 %                 value={checkOutDate}
267 %                 onChange={(e) => setCheckOutDate(e.target.value)}
268 %                 min={checkInDate}
269 %                 data-testid="input-checkout-date"
270 %               />
271 %             </div>
272 %           </div>
273 %           {nights > 0 && (
274 %             <div className="flex items-center gap-2 text-sm text-muted-foreground">
275 %               <Badge variant="secondary">{nights} noche(s)</Badge>
276 %             </div>
277 %           )
278 %         </CardContent>
279 %       </Card>
280 %     </div>
281 %   </div>
282 % );
```

client/src/pages/new-reservation.tsx (cont.)

```
277 %     })
278 %   </CardContent>
279 % </Card>
280 % </div>
281 %
282 % <div className="space-y-6">
283 %   <Card>
284 %     <CardHeader className="pb-3">
285 %       <CardTitle className="text-base flex items-center gap-2">
286 %         <DoorOpen className="h-4 w-4" />
287 %         Habitación
288 %       </CardTitle>
289 %     </CardHeader>
290 %     <CardContent className="space-y-4">
291 %       <div className="space-y-2">
292 %         <Label>Tipo de Habitación</Label>
293 %         <Select
294 %           value={selectedRoomTypeId}
295 %           onChange={(v) => {
296 %             setSelectedRoomTypeId(v);
297 %             setSelectedRoomId("");
298 %             setSelectedRatePlanId("");
299 %           }}
300 %         >
301 %           <SelectTrigger data-testid="select-room-type">
302 %             <SelectValue placeholder="Seleccionar tipo..." />
303 %           </SelectTrigger>
304 %           <SelectContent>
305 %             {roomTypes?.map((rt) => (
306 %               <SelectItem key={rt.id} value={rt.id}>
307 %                 {rt.name} - {rt.maxOccupancy} pax
308 %               </SelectItem>
309 %             ))}
310 %           </SelectContent>
311 %         </Select>
312 %       </div>
313 %
314 %       <div className="space-y-2">
315 %         <Label>Habitación Disponible</Label>
316 %         <Select
317 %           value={selectedRoomId}
318 %           onChange={setSelectedRoomId}
319 %           disabled={!selectedRoomTypeId}
320 %         >
321 %           <SelectTrigger data-testid="select-room">
322 %             <SelectValue placeholder={selectedRoomTypeId ? "Seleccionar habitación..." : "Primero seleccione ..."} />
323 %           </SelectTrigger>
324 %           <SelectContent>
325 %             {availableRooms?.map((room) => (
326 %               <SelectItem key={room.id} value={room.id}>
327 %                 Hab. {room.roomNumber} - Piso {room.floor}
328 %               </SelectItem>
329 %             ))}
330 %           </SelectContent>
331 %         </Select>
332 %         {selectedRoomTypeId && availableRooms?.length === 0 && (
333 %           <p className="text-sm text-destructive">No hay habitaciones disponibles de este tipo</p>
334 %         )}
335 %       </div>
336 %
337 %       <div className="space-y-2">
338 %         <Label>Cantidad de Huespedes</Label>
339 %         <Input
340 %           type="number"
341 %           min="1"
342 %           value={numberOfGuests}
343 %           onChange={(e) => setNumberOfGuests(parseInt(e.target.value) || 1)}
344 %           data-testid="input-guests"
345 %         />
346 %       </div>
347 %
348 %       <div className="space-y-2">
349 %         <Label>Tipo de camaje</Label>
350 %         <Select
351 %           value={bedTypeId ? String(bedTypeId) : ""}
352 %           onChange={(v) => setBedTypeId(parseInt(v))}
353 %         >
354 %           <SelectTrigger data-testid="select-bed-type">
355 %             <SelectValue placeholder="Seleccionar tipo de camaje..." />
356 %           </SelectTrigger>
357 %           <SelectContent>
358 %             {bedTypes?.filter(bt => bt.isActive).map((bt) => (
359 %               <SelectItem key={bt.id} value={String(bt.id)}>
360 %                 {bt.name}
361 %               </SelectItem>
362 %             ))}
363 %           </SelectContent>
364 %         </Select>
365 %       </div>
366 %
367 %       <div className="space-y-2">
368 %         <Label>Notas de camaje</Label>
369 %         <Input
```

client/src/pages/new-reservation.tsx (cont.)

```
370 %         value={bedTypeNotes}
371 %         onChange={(e) => setBedTypeNotes(e.target.value)}
372 %         placeholder="Preferencias especiales..."
373 %         data-testid="input-bed-type-notes"
374 %     />
375 % </div>
376 % </CardContent>
377 % </Card>
378 %
379 % <Card>
380 %   <CardHeader className="pb-3">
381 %     <CardTitle className="text-base flex items-center gap-2">
382 %       <DollarSign className="h-4 w-4" />
383 %       Tarifa y Descuentos
384 %     </CardTitle>
385 %   </CardHeader>
386 %   <CardContent className="space-y-4">
387 %     <div className="space-y-2">
388 %       <Label>Plan de Tarifa</Label>
389 %       <Select
390 %         value={selectedRatePlanId}
391 %         onValueChange={setSelectedRatePlanId}
392 %         disabled={!selectedRoomTypeId}
393 %       >
394 %         <SelectTrigger data-testid="select-rate-plan">
395 %           <SelectValue placeholder={selectedRoomTypeId ? "Seleccionar tarifa..." : "Primero selecciona tipo..."} />
396 %         </SelectTrigger>
397 %         <SelectContent>
398 %           {applicableRatePlans?.map((rp) => (
399 %             <SelectItem key={rp.id} value={rp.id}>
400 %               {rp.name} - ${rp.baseRate}/noche
401 %             </SelectItem>
402 %           ))}
403 %         </SelectContent>
404 %       </Select>
405 %     </div>
406 %
407 %     <div className="grid grid-cols-2 gap-4">
408 %       <div className="space-y-2">
409 %         <Label>Tipo de Descuento</Label>
410 %         <Select value={discountType} onValueChange={setDiscountType}>
411 %           <SelectTrigger data-testid="select-discount-type">
412 %             <SelectValue />
413 %           </SelectTrigger>
414 %           <SelectContent>
415 %             <SelectItem value="none">Sin descuento</SelectItem>
416 %             <SelectItem value="percent">Porcentaje (%)</SelectItem>
417 %             <SelectItem value="fixed">Monto fijo ($)</SelectItem>
418 %           </SelectContent>
419 %         </Select>
420 %       </div>
421 %       {discountType !== "none" && (
422 %         <div className="space-y-2">
423 %           <Label>Valor</Label>
424 %           <Input
425 %             type="number"
426 %             min="0"
427 %             value={discountValue}
428 %             onChange={(e) => setDiscountValue(e.target.value)}
429 %             data-testid="input-discount-value"
430 %           />
431 %         </div>
432 %       )}
433 %     </div>
434 %   </CardContent>
435 % </Card>
436 %
437 % <Card>
438 %   <CardHeader className="pb-3">
439 %     <CardTitle className="text-base">Servicios especiales</CardTitle>
440 %   </CardHeader>
441 %   <CardContent className="grid gap-3">
442 %     <div className="flex items-center justify-between">
443 %       <div className="flex items-center gap-2">
444 %         <Sunrise className="h-4 w-4 text-orange-500" />
445 %         <Label htmlFor="nr-earlyCheckIn" className="cursor-pointer">Early Check-in</Label>
446 %         <span className="text-xs text-muted-foreground">(Estándar: 12:00 hs)</span>
447 %       </div>
448 %       <Switch
449 %         id="nr-earlyCheckIn"
450 %         checked={earlyCheckIn}
451 %         onCheckedChange={(checked) => { setEarlyCheckIn(checked); if (!checked) { setEarlyCheckInTime(""); }; ...
452 %         data-testid="switch-early-checkin"
453 %       />
454 %     </div>
455 %     {earlyCheckIn && (
456 %       <div className="grid grid-cols-2 gap-2 pl-6">
457 %         <div>
458 %           <Label className="text-xs">Hora acordada</Label>
459 %           <Input type="time" value={earlyCheckInTime} onChange={(e) => setEarlyCheckInTime(e.target.value)}...
460 %         </div>
461 %         <div>
462 %           <Label className="text-xs">Cargo (vacío = cortesía)</Label>
```

client/src/pages/new-reservation.tsx (cont.)

```
463 %         <Input type="number" step="0.01" min="0" placeholder="0.00" value={earlyCheckInCharge} onChange={...
464 %       </div>
465 %     </div>
466 %   )}
467 %   <div className="flex items-center justify-between">
468 %     <div className="flex items-center gap-2">
469 %       <Sunset className="h-4 w-4 text-purple-500" />
470 %       <Label htmlFor="nr-lateCheckOut" className="cursor-pointer">Late Check-out</Label>
471 %       <span className="text-xs text-muted-foreground">(Estándar: 11:00 hs)</span>
472 %     </div>
473 %     <Switch
474 %       id="nr-lateCheckOut"
475 %       checked={lateCheckOut}
476 %       onCheckedChange={(checked) => { setLateCheckOut(checked); if (!checked) { setLateCheckOutTime(""); ...
477 %         data-testid="switch-late-checkout"
478 %       } />
479 %   </div>
480 %   {lateCheckOut && (
481 %     <div className="grid grid-cols-2 gap-2 pl-6">
482 %       <div>
483 %         <Label className="text-xs">Hora acordada</Label>
484 %         <Input type="time" value={lateCheckOutTime} onChange={(e) => setLateCheckOutTime(e.target.value)}...
485 %       </div>
486 %       <div>
487 %         <Label className="text-xs">Cargo (vacío = cortesía)</Label>
488 %         <Input type="number" step="0.01" min="0" placeholder="0.00" value={lateCheckOutCharge} onChange={...
489 %       </div>
490 %     </div>
491 %   )}
492 % </CardContent>
493 % </Card>
494 %
495 % <Card>
496 %   <CardHeader className="pb-3">
497 %     <CardTitle className="text-base">Notas</CardTitle>
498 %   </CardHeader>
499 %   <CardContent>
500 %     <Textarea
501 %       placeholder="Notas adicionales sobre la reserva..."
502 %       value={notes}
503 %       onChange={(e) => setNotes(e.target.value)}
504 %       data-testid="input-notes"
505 %     />
506 %   </CardContent>
507 % </Card>
508 % </div>
509 % </div>
510 %
511 % <Card className="bg-muted/50">
512 %   <CardHeader className="pb-3">
513 %     <CardTitle className="text-base">Resumen de Reserva</CardTitle>
514 %   </CardHeader>
515 %   <CardContent>
516 %     <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
517 %       <div>
518 %         <p className="text-sm text-muted-foreground">Huesped</p>
519 %         <p className="font-medium">
520 %           {selectedGuest ? `${selectedGuest.firstName} ${selectedGuest.lastName}` : "-"}
521 %         </p>
522 %       </div>
523 %       <div>
524 %         <p className="text-sm text-muted-foreground">Habitacion</p>
525 %         <p className="font-medium">
526 %           {selectedRoomId ? rooms?.find(r => r.id === selectedRoomId)?.roomNumber : "-"}
527 %         </p>
528 %       </div>
529 %       <div>
530 %         <p className="text-sm text-muted-foreground">Noches</p>
531 %         <p className="font-medium">{nights}</p>
532 %       </div>
533 %       <div>
534 %         <p className="text-sm text-muted-foreground">Total</p>
535 %         <p className="font-medium text-lg">${totalAmount}</p>
536 %       </div>
537 %     </div>
538 %   <div className="mt-4 flex justify-end gap-2">
539 %     <Button variant="outline" onClick={() => setLocation("/reservations")} data-testid="button-cancel">
540 %       Volver
541 %     </Button>
542 %     <Button
543 %       onClick={() => createReservationMutation.mutate()}
544 %       disabled={!canSubmit || createReservationMutation.isPending}
545 %       data-testid="button-create-reservation"
546 %     >
547 %       {createReservationMutation.isPending ? (
548 %         "Creando..."
549 %       ) : (
550 %         <>
551 %           <Check className="mr-2 h-4 w-4" />
552 %           Crear Reserva
553 %         </>
554 %       )}
555 %     </Button>
```

client/src/pages/new-reservation.tsx (cont.)

```
556 %         </div>
557 %         </CardContent>
558 %       </Card>
559 %     </div>
560 %   );
561 % }
562 %
```

```
1 % import { useState, useEffect, Fragment } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { useLocation } from "wouter";
4 % import { ChevronLeft, ChevronRight, Info, Plus, LogIn, LogOut, ExternalLink, Calendar, User, DollarSign, Bed, Users, ...
5 % import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
6 % import { Calendar as CalendarPicker } from "@components/ui/calendar";
7 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
8 % import { Button } from "@components/ui/button";
9 % import { Badge } from "@components/ui/badge";
10 % import { Skeleton } from "@components/ui/skeleton";
11 % import { ScrollArea, ScrollBar } from "@components/ui/scroll-area";
12 % import { Input } from "@components/ui/input";
13 % import { Label } from "@components/ui/label";
14 % import { Separator } from "@components/ui/separator";
15 % import {
16 %   Dialog,
17 %   DialogContent,
18 %   DialogDescription,
19 %   DialogFooter,
20 %   DialogHeader,
21 %   DialogTitle,
22 % } from "@components/ui/dialog";
23 % import {
24 %   Select,
25 %   SelectContent,
26 %   SelectItem,
27 %   SelectTrigger,
28 %   SelectValue,
29 % } from "@components/ui/select";
30 % import {
31 %   Tooltip,
32 %   TooltipContent,
33 %   TooltipTrigger,
34 % } from "@components/ui/tooltip";
35 % import { useToast } from "@hooks/use-toast";
36 % import { queryClient, apiRequest } from "@lib/queryClient";
37 % import type { PlanningData, PlanningCellStatus, Guest, RoomWithType, ReservationWithDetails, ReservationStatus, Reser...
38 %
39 % function formatDate(dateStr: string) {
40 %   const date = new Date(dateStr + "T12:00:00");
41 %   return {
42 %     dayName: date.toLocaleDateString("es-ES", { weekday: "short" }),
43 %     dayNumber: date.getDate(),
44 %     monthName: date.toLocaleDateString("es-ES", { month: "short" }),
45 %     isToday: dateStr === new Date().toISOString().split("T")[0],
46 %     isWeekend: date.getDay() === 0 || date.getDay() === 6,
47 %   };
48 % }
49 %
50 % function formatDateReadable(dateStr: string) {
51 %   const date = new Date(dateStr + "T12:00:00");
52 %   return date.toLocaleDateString("es-ES", { weekday: "long", day: "numeric", month: "long" });
53 % }
54 %
55 % function getStatusColor(status: PlanningCellStatus): string {
56 %   switch (status) {
57 %     case "available":
58 %       return "bg-green-100 dark:bg-green-900/30 border-green-200 dark:border-green-800";
59 %     case "booked":
60 %       return "bg-blue-100 dark:bg-blue-900/40 border-blue-200 dark:border-blue-800";
61 %     case "checked_in":
62 %       return "bg-amber-100 dark:bg-amber-900/40 border-amber-200 dark:border-amber-800";
63 %     case "checkout_today":
64 %       return "bg-orange-100 dark:bg-orange-900/40 border-orange-200 dark:border-orange-800";
65 %     case "maintenance":
66 %       return "bg-red-100 dark:bg-red-900/40 border-red-200 dark:border-red-800";
67 %     case "cleaning":
68 %       return "bg-purple-100 dark:bg-purple-900/40 border-purple-200 dark:border-purple-800";
69 %     case "group_blocked":
70 %       return "bg-indigo-100 dark:bg-indigo-900/40 border-indigo-200 dark:border-indigo-800";
71 %     default:
72 %       return "bg-muted";
73 %   }
74 % }
75 %
76 % function getStatusLabel(status: PlanningCellStatus): string {
77 %   switch (status) {
78 %     case "available":
79 %       return "Disponible";
80 %     case "booked":
81 %       return "Reservado";
82 %     case "checked_in":
83 %       return "Ocupado";
84 %     case "checkout_today":
85 %       return "Check-out hoy";
86 %     case "maintenance":
87 %       return "Mantenimiento";
88 %     case "cleaning":
89 %       return "Limpieza";
90 %     case "group_blocked":
```


client/src/pages/planning.tsx (cont.)

```
91 %         return "Grupo";
92 %     default:
93 %         return status;
94 % }
95 % }
96 %
97 % function getSourceColor(source: ReservationSource): string {
98 %     switch (source) {
99 %         case "directo":
100 %             return "bg-blue-200 dark:bg-blue-800/60 border-blue-300 dark:border-blue-700";
101 %         case "telefono":
102 %             return "bg-sky-200 dark:bg-sky-800/60 border-sky-300 dark:border-sky-700";
103 %         case "web":
104 %             return "bg-cyan-200 dark:bg-cyan-800/60 border-cyan-300 dark:border-cyan-700";
105 %         case "booking":
106 %             return "bg-indigo-200 dark:bg-indigo-800/60 border-indigo-300 dark:border-indigo-700";
107 %         case "expedia":
108 %             return "bg-yellow-200 dark:bg-yellow-800/60 border-yellow-300 dark:border-yellow-700";
109 %         case "airbnb":
110 %             return "bg-rose-200 dark:bg-rose-800/60 border-rose-300 dark:border-rose-700";
111 %         case "despegar":
112 %             return "bg-orange-200 dark:bg-orange-800/60 border-orange-300 dark:border-orange-700";
113 %         case "hotelbeds":
114 %             return "bg-purple-200 dark:bg-purple-800/60 border-purple-300 dark:border-purple-700";
115 %         case "agoda":
116 %             return "bg-red-200 dark:bg-red-800/60 border-red-300 dark:border-red-700";
117 %         case "ota":
118 %             return "bg-violet-200 dark:bg-violet-800/60 border-violet-300 dark:border-violet-700";
119 %         case "empresa":
120 %             return "bg-emerald-200 dark:bg-emerald-800/60 border-emerald-300 dark:border-emerald-700";
121 %         default:
122 %             return "bg-gray-200 dark:bg-gray-800/60 border-gray-300 dark:border-gray-700";
123 %     }
124 % }
125 %
126 % function getSourceLabel(source: ReservationSource): string {
127 %     switch (source) {
128 %         case "directo":
129 %             return "Directo";
130 %         case "telefono":
131 %             return "Teléfono";
132 %         case "web":
133 %             return "Web";
134 %         case "booking":
135 %             return "Booking";
136 %         case "expedia":
137 %             return "Expedia";
138 %         case "airbnb":
139 %             return "Airbnb";
140 %         case "despegar":
141 %             return "Despegar";
142 %         case "hotelbeds":
143 %             return "Hotelbeds";
144 %         case "agoda":
145 %             return "Agoda";
146 %         case "ota":
147 %             return "OTA";
148 %         case "empresa":
149 %             return "Empresa";
150 %         default:
151 %             return source;
152 %     }
153 % }
154 %
155 % const featureIconMap: Record<string, { icon: typeof Accessibility; label: string }> = {
156 %     accessible: { icon: Accessibility, label: "Accesible" },
157 %     balcony: { icon: Mountain, label: "Balcón" },
158 %     sofa_bed: { icon: Sofa, label: "Sofá cama" },
159 %     living_room: { icon: Armchair, label: "Living" },
160 %     twin_config: { icon: BedDouble, label: "Config. twin" },
161 %     separable_bed: { icon: ArrowLeftRight, label: "Camas separables" },
162 %     extra_bed: { icon: BedSingle, label: "Cama extra" },
163 %     shower_only: { icon: Droplets, label: "Solo ducha" },
164 % };
165 %
166 % const bedConfigLabels: Record<string, string> = {
167 %     MAT: "Matrimonial",
168 %     TWIN: "Twin",
169 %     MAT_CC: "Matrimonial + Cama cucheta",
170 %     TWIN_CC: "Twin + Cama cucheta",
171 %     MAT_EXTRA: "Matrimonial + Extra",
172 %     MAT_CC_EXTRA: "Matrimonial + CC + Extra",
173 % };
174 %
175 % function Legend() {
176 %     const statusItems: { status: PlanningCellStatus; label: string }[] = [
177 %         { status: "available", label: "Disponible" },
178 %         { status: "cleaning", label: "Limpieza" },
179 %         { status: "maintenance", label: "Mantenimiento" },
180 %         { status: "group_blocked", label: "Grupo bloq." },
181 %     ];
182 %
183 %     const sourceItems: { source: ReservationSource; label: string }[] = [
```

client/src/pages/planning.tsx (cont.)

```
184 %     { source: "directo", label: "Directo" },
185 %     { source: "telefono", label: "Teléfono" },
186 %     { source: "web", label: "Web" },
187 %     { source: "booking", label: "Booking" },
188 %     { source: "expedia", label: "Expedia" },
189 %     { source: "airbnb", label: "Airbnb" },
190 %     { source: "despegar", label: "Despegar" },
191 %     { source: "hotelbeds", label: "Hotelbeds" },
192 %     { source: "agoda", label: "Agoda" },
193 %     { source: "ota", label: "OTA" },
194 %     { source: "empresa", label: "Empresa" },
195 % ];
196 %
197 % return (
198 %   <div className="flex flex-col gap-2">
199 %     <div className="flex flex-wrap items-center gap-3">
200 %       <span className="text-xs font-medium text-muted-foreground">Estado:</span>
201 %       {statusItems.map(({ status, label }) => (
202 %         <div key={status} className="flex items-center gap-1.5">
203 %           <div className={`w-4 h-4 rounded border ${getStatusColor(status)}`} />
204 %           <span className="text-xs text-muted-foreground">{label}</span>
205 %         </div>
206 %       )
207 %     )}
208 %   </div>
209 %   <div className="flex flex-wrap items-center gap-3">
210 %     <span className="text-xs font-medium text-muted-foreground">Origen:</span>
211 %     {sourceItems.map(({ source, label }) => (
212 %       <div key={source} className="flex items-center gap-1.5">
213 %         <div className={`w-4 h-4 rounded border ${getSourceColor(source)}`} />
214 %         <span className="text-xs text-muted-foreground">{label}</span>
215 %       </div>
216 %     )
217 %   )}
218 % </div>
219 % );
220 %
221 % type QuickReservationData = {
222 %   roomId: string;
223 %   roomNumber: string;
224 %   roomTypeName: string;
225 %   checkInDate: string;
226 % };
227 %
228 % function QuickReservationDialog({
229 %   open,
230 %   onOpenChange,
231 %   reservationData,
232 %   guests,
233 % }: {
234 %   open: boolean;
235 %   onOpenChange: (open: boolean) => void;
236 %   reservationData: QuickReservationData | null;
237 %   guests: Guest[];
238 % }) {
239 %   const { toast } = useToast();
240 %   const [guestId, setGuestId] = useState("");
241 %   const [numberOfGuests, setNumberOfGuests] = useState(1);
242 %   const [checkOutDate, setCheckOutDate] = useState("");
243 %
244 %   useEffect(() => {
245 %     if (reservationData) {
246 %       const nextDay = new Date(reservationData.checkInDate + "T12:00:00");
247 %       nextDay.setDate(nextDay.getDate() + 1);
248 %       setCheckOutDate(nextDay.toISOString().split("T")[0]);
249 %     }
250 %   }, [reservationData]);
251 %
252 %   const mutation = useMutation({
253 %     mutationFn: async (data: {
254 %       guestId: string;
255 %       roomId: string;
256 %       checkInDate: string;
257 %       checkOutDate: string;
258 %       numberOfGuests: number;
259 %       status: string;
260 %       totalAmount: string;
261 %       createdAt: string;
262 %     }) => {
263 %       return apiRequest("POST", "/api/reservations", data);
264 %     },
265 %     onSuccess: () => {
266 %       queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
267 %       queryClient.invalidateQueries({ predicate: (query) =>
268 %         Array.isArray(query.queryKey) && query.queryKey[0] === "/api/planning"
269 %       });
270 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
271 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/arrivals"] });
272 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/departures"] });
273 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
274 %       toast({
275 %         title: "Reserva creada",
276 %         description: "La reserva ha sido creada exitosamente desde el planning.",
```

client/src/pages/planning.tsx (cont.)

```
277 %     });
278 %     onOpenChange(false);
279 %     setGuestId("");
280 %     setCheckOutDate("");
281 %     setNumberOfGuests(1);
282 %   },
283 %   onError: () => {
284 %     toast({
285 %       title: "Error",
286 %       description: "No se pudo crear la reserva. Intente nuevamente.",
287 %       variant: "destructive",
288 %     });
289 %   },
290 % });
291 %
292 % const handleSubmit = () => {
293 %   if (!reservationData || !guestId || !checkOutDate) {
294 %     toast({
295 %       title: "Datos incompletos",
296 %       description: "Por favor complete todos los campos obligatorios.",
297 %       variant: "destructive",
298 %     });
299 %     return;
300 %   }
301 %
302 %   if (checkOutDate <= reservationData.checkInDate) {
303 %     toast({
304 %       title: "Fechas inválidas",
305 %       description: "La fecha de check-out debe ser posterior al check-in.",
306 %       variant: "destructive",
307 %     });
308 %     return;
309 %   }
310 %
311 %   mutation.mutate({
312 %     guestId,
313 %     roomId: reservationData.roomId,
314 %     checkInDate: reservationData.checkInDate,
315 %     checkOutDate,
316 %     numberOfGuests,
317 %     status: "confirmed",
318 %     totalAmount: "0",
319 %     createdAt: new Date().toISOString(),
320 %   });
321 % };
322 %
323 % if (!reservationData) return null;
324 %
325 % return (
326 %   <Dialog open={open} onOpenChange={onOpenChange}>
327 %     <DialogContent className="sm:max-w-[425px]">
328 %       <DialogHeader>
329 %         <DialogTitle>Nueva Reserva Rapida</DialogTitle>
330 %         <DialogDescription>
331 %           Habitación {reservationData.roomNumber} ({reservationData.roomTypeName}) - Check-in: {formatDateReadable(...
332 %         </DialogDescription>
333 %       </DialogHeader>
334 %       <div className="grid gap-4 py-4">
335 %         <div className="grid gap-2">
336 %           <Label htmlFor="guest">Huésped *</Label>
337 %           <Select value={guestId} onValueChange={setGuestId}>
338 %             <SelectTrigger data-testid="select-guest-quick">
339 %               <SelectValue placeholder="Seleccionar huésped" />
340 %             </SelectTrigger>
341 %             <SelectContent>
342 %               {guests.map((guest) => (
343 %                 <SelectItem key={guest.id} value={guest.id}>
344 %                   {guest.firstName} {guest.lastName}
345 %                 </SelectItem>
346 %               ))}
347 %             </SelectContent>
348 %           </Select>
349 %         </div>
350 %         <div className="grid gap-2">
351 %           <Label htmlFor="checkIn">Check-in</Label>
352 %           <Input
353 %             id="checkIn"
354 %             type="date"
355 %             value={reservationData.checkInDate}
356 %             disabled
357 %             className="bg-muted"
358 %           />
359 %         </div>
360 %         <div className="grid gap-2">
361 %           <Label htmlFor="checkOut">Check-out *</Label>
362 %           <Input
363 %             id="checkOut"
364 %             type="date"
365 %             value={checkOutDate}
366 %             onChange={(e) => setCheckOutDate(e.target.value)}
367 %             min={reservationData.checkInDate}
368 %             data-testid="input-checkout-quick"
369 %           />
369 %       </div>
369 %     </DialogContent>
369 %   </Dialog>
369 % );
```

client/src/pages/planning.tsx (cont.)

```
370 %         </div>
371 %         <div className="grid gap-2">
372 %             <Label htmlFor="guests">Cantidad de huéspedes</Label>
373 %             <Input
374 %                 id="guests"
375 %                 type="number"
376 %                 min={1}
377 %                 max={10}
378 %                 value={numberOfGuests}
379 %                 onChange={(e) => setNumberOfGuests(parseInt(e.target.value) || 1)}
380 %                 data-testid="input-guests-quick"
381 %             />
382 %         </div>
383 %     </div>
384 %     <DialogFooter>
385 %         <Button variant="outline" onClick={() => onOpenChange(false)} data-testid="button-cancel-quick">
386 %             Volver
387 %         </Button>
388 %         <Button onClick={handleSubmit} disabled={mutation.isPending} data-testid="button-create-quick">
389 %             {mutation.isPending ? "Creando..." : "Crear Reserva"}
390 %         </Button>
391 %     </DialogFooter>
392 % </DialogContent>
393 % </Dialog>
394 % );
395 % }
396 %
397 % function getStatusBadge(status: ReservationStatus) {
398 %     const config: Record<ReservationStatus, { label: string; variant: "default" | "secondary" | "destructive" | "outlin...
399 %     tentative: { label: "Tentativa", variant: "outline" },
400 %     pending: { label: "Pendiente", variant: "secondary" },
401 %     confirmed: { label: "Confirmada", variant: "default" },
402 %     checked_in: { label: "Check-in", variant: "default" },
403 %     checked_out: { label: "Check-out", variant: "outline" },
404 %     cancelled: { label: "Cancelada", variant: "destructive" },
405 % };
406 % return config[status] || { label: status, variant: "outline" };
407 % }
408 %
409 % function ReservationDetailModal({
410 %     open,
411 %     onOpenChange,
412 %     reservationId,
413 %     onNavigate,
414 % }): {
415 %     open: boolean;
416 %     onOpenChange: (open: boolean) => void;
417 %     reservationId: string | null;
418 %     onNavigate: (path: string) => void;
419 % } {
420 %     const { toast } = useToast();
421 %
422 %     const { data: reservation, isLoading } = useQuery<ReservationWithDetails>({
423 %         queryKey: ["/api/reservations", reservationId],
424 %         queryFn: async () => {
425 %             const res = await fetch(`/api/reservations/${reservationId}`);
426 %             if (!res.ok) throw new Error("Failed to fetch reservation");
427 %             return res.json();
428 %         },
429 %         enabled: !!reservationId && open,
430 %     });
431 %
432 %     const checkInMutation = useMutation({
433 %         mutationFn: async () => {
434 %             return apiRequest("POST", `/api/reservations/${reservationId}/check-in`, {});
435 %         },
436 %         onSuccess: () => {
437 %             queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
438 %             queryClient.invalidateQueries({ predicate: (query) =>
439 %                 Array.isArray(query.queryKey) && query.queryKey[0] === "/api/planning"
440 %             });
441 %             queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
442 %             queryClient.invalidateQueries({ queryKey: ["/api/dashboard/arrivals"] });
443 %             queryClient.invalidateQueries({ queryKey: ["/api/dashboard/departures"] });
444 %             queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
445 %             toast({ title: "Check-in realizado", description: "El huesped ha sido registrado." });
446 %             onOpenChange(false);
447 %         },
448 %         onError: () => {
449 %             toast({ title: "Error", description: "No se pudo realizar el check-in.", variant: "destructive" });
450 %         },
451 %     });
452 %
453 %     const checkOutMutation = useMutation({
454 %         mutationFn: async () => {
455 %             return apiRequest("POST", `/api/reservations/${reservationId}/check-out`, {});
456 %         },
457 %         onSuccess: () => {
458 %             queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
459 %             queryClient.invalidateQueries({ predicate: (query) =>
460 %                 Array.isArray(query.queryKey) && query.queryKey[0] === "/api/planning"
461 %             });
462 %             queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
```

client/src/pages/planning.tsx (cont.)

```
463 %     queryClient.invalidateQueries({ queryKey: ["/api/dashboard/arrivals"] });
464 %     queryClient.invalidateQueries({ queryKey: ["/api/dashboard/departures"] });
465 %     queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
466 %     toast({ title: "Check-out realizado", description: "El huesped ha sido despedido." });
467 %     onOpenChange(false);
468 %   },
469 %   onError: (error: any) => {
470 %     const message = error?.data?.error || error?.message || "No se pudo realizar el check-out.";
471 %     if (message.includes("saldo pendiente") || message.includes("balance")) {
472 %       toast({ title: "Saldo Pendiente", description: message, variant: "destructive" });
473 %     } else {
474 %       toast({ title: "Error", description: message, variant: "destructive" });
475 %     }
476 %   },
477 % });
478 %
479 % if (!reservationId) return null;
480 %
481 % const statusBadge = reservation ? getStatusBadge(reservation.status) : null;
482 % const canCheckIn = reservation?.status === "confirmed" || reservation?.status === "pending";
483 % const canCheckOut = reservation?.status === "checked_in";
484 % const totalCharges = reservation?.charges?.reduce((sum, c) => sum + parseFloat(c.amount), 0) || 0;
485 %
486 % return (
487 %   <Dialog open={open} onOpenChange={onOpenChange}>
488 %     <DialogContent className="sm:max-w-[500px]">
489 %       <DialogHeader>
490 %         <DialogTitle className="flex items-center gap-2">
491 %           <Calendar className="h-5 w-5" />
492 %           Detalle de Reserva
493 %         </DialogTitle>
494 %         {reservation && (
495 %           <DialogDescription>
496 %             Codigo: {reservation.reservationCode}
497 %           </DialogDescription>
498 %         )}
499 %       </DialogHeader>
500 %
501 %       {isLoading ? (
502 %         <div className="space-y-3 py-4">
503 %           <Skeleton className="h-6 w-full" />
504 %           <Skeleton className="h-6 w-3/4" />
505 %           <Skeleton className="h-6 w-1/2" />
506 %         </div>
507 %       ) : reservation ? (
508 %         <div className="space-y-4 py-2">
509 %           <div className="flex items-center justify-between">
510 %             <div className="flex items-center gap-2">
511 %               <User className="h-4 w-4 text-muted-foreground" />
512 %               <span className="font-medium">
513 %                 {reservation.guest?.firstName} {reservation.guest?.lastName}
514 %               </span>
515 %             </div>
516 %             <Badge variant={statusBadge?.variant}>{statusBadge?.label}</Badge>
517 %           </div>
518 %
519 %           <Separator />
520 %
521 %           <div className="grid grid-cols-2 gap-4 text-sm">
522 %             <div className="space-y-1">
523 %               <div className="text-muted-foreground">Habitacion</div>
524 %               <div className="font-medium flex items-center gap-1">
525 %                 <Bed className="h-4 w-4" />
526 %                 {reservation.room?.roomNumber} - {reservation.room?.roomType?.name}
527 %               </div>
528 %             </div>
529 %             <div className="space-y-1">
530 %               <div className="text-muted-foreground">Huespedes</div>
531 %               <div className="font-medium flex items-center gap-1">
532 %                 <Users className="h-4 w-4" />
533 %                 {reservation.numberOfGuests} persona(s)
534 %               </div>
535 %             </div>
536 %             <div className="space-y-1">
537 %               <div className="text-muted-foreground">Check-in</div>
538 %               <div className="font-medium">{formatDateReadable(reservation.checkInDate)}</div>
539 %             </div>
540 %             <div className="space-y-1">
541 %               <div className="text-muted-foreground">Check-out</div>
542 %               <div className="font-medium">{formatDateReadable(reservation.checkOutDate)}</div>
543 %             </div>
544 %             <div className="space-y-1">
545 %               <div className="text-muted-foreground">Noches</div>
546 %               <div className="font-medium">{reservation.nights}</div>
547 %             </div>
548 %             <div className="space-y-1">
549 %               <div className="text-muted-foreground">Tarifa/noche</div>
550 %               <div className="font-medium">${reservation.finalRatePerNight}</div>
551 %             </div>
552 %           </div>
553 %
554 %           <Separator />
555 %
```

client/src/pages/planning.tsx (cont.)

```
556 %         <div className="flex items-center justify-between bg-muted/50 rounded-md p-3">
557 %           <div className="flex items-center gap-2">
558 %             <DollarSign className="h-5 w-5 text-muted-foreground" />
559 %             <div>
560 %               <div className="text-sm text-muted-foreground">Total Habitacion</div>
561 %               <div className="font-semibold">${reservation.totalRoomAmount}</div>
562 %             </div>
563 %           </div>
564 %           {totalCharges > 0 && (
565 %             <div className="text-right">
566 %               <div className="text-sm text-muted-foreground">Cargos extras</div>
567 %               <div className="font-semibold">${totalCharges.toFixed(2)}</div>
568 %             </div>
569 %           )}
570 %         </div>
571 %
572 %         {reservation.notes && (
573 %           <div className="text-sm bg-muted/30 rounded-md p-3">
574 %             <div className="text-muted-foreground mb-1">Notas:</div>
575 %             <div>{reservation.notes}</div>
576 %           </div>
577 %         )}
578 %       </div>
579 %     ) : null}
580 %
581 %   <DialogFooter className="flex-col gap-2 sm:flex-row">
582 %     {canCheckIn && (
583 %       <Button
584 %         onClick={() => checkInMutation.mutate()}
585 %         disabled={checkInMutation.isPending}
586 %         className="w-full sm:w-auto"
587 %         data-testid="button-checkin-quick"
588 %       >
589 %         <LogIn className="h-4 w-4 mr-2" />
590 %         {checkInMutation.isPending ? "Procesando..." : "Check-in"}
591 %       </Button>
592 %     )}
593 %     {canCheckOut && (
594 %       <Button
595 %         onClick={() => checkOutMutation.mutate()}
596 %         disabled={checkOutMutation.isPending}
597 %         variant="secondary"
598 %         className="w-full sm:w-auto"
599 %         data-testid="button-checkout-quick"
600 %       >
601 %         <LogOut className="h-4 w-4 mr-2" />
602 %         {checkOutMutation.isPending ? "Procesando..." : "Check-out"}
603 %       </Button>
604 %     )}
605 %     <Button
606 %       variant="outline"
607 %       onClick={() => {
608 %         onOpenChange(false);
609 %         onNavigate(`/reservations?view=${reservationId}`);
610 %       }}
611 %       className="w-full sm:w-auto"
612 %       data-testid="button-view-full"
613 %     >
614 %       <ExternalLink className="h-4 w-4 mr-2" />
615 %       Ver Completo
616 %     </Button>
617 %   </DialogFooter>
618 % </DialogContent>
619 % </Dialog>
620 % );
621 % }
622 %
623 % export default function PlanningPage() {
624 %   const [, navigate] = useLocation();
625 %   const [dateRange, setDateRange] = useState(() => {
626 %     const today = new Date();
627 %     const start = new Date(today);
628 %     start.setDate(start.getDate() - 1);
629 %     const end = new Date(today);
630 %     end.setDate(end.getDate() + 14);
631 %     return {
632 %       start: start.toISOString().split("T")[0],
633 %       end: end.toISOString().split("T")[0],
634 %     };
635 %   });
636 %
637 %   const [quickReservationOpen, setQuickReservationOpen] = useState(false);
638 %   const [selectedCell, setSelectedCell] = useState<QuickReservationData | null>(null);
639 %   const [reservationDetailOpen, setReservationDetailOpen] = useState(false);
640 %   const [selectedReservationId, setSelectedReservationId] = useState<string | null>(null);
641 %
642 %   const { data, isLoading } = useQuery<PlanningData>({
643 %     queryKey: ["/api/planning", dateRange.start, dateRange.end],
644 %     queryFn: async () => {
645 %       const res = await fetch(`/api/planning?start=${dateRange.start}&end=${dateRange.end}`);
646 %       if (!res.ok) throw new Error("Failed to fetch planning data");
647 %       return res.json();
648 %     },
649 %   });
```

client/src/pages/planning.tsx (cont.)

```
649 % });
650 %
651 % const { data: guests = [] } = useQuery<Guest[]>({
652 %   queryKey: ["/api/guests"],
653 %   queryFn: async () => {
654 %     const res = await fetch("/api/guests");
655 %     if (!res.ok) throw new Error("Failed to fetch guests");
656 %     return res.json();
657 %   },
658 % });
659 %
660 % const navigateDays = (direction: "prev" | "next") => {
661 %   const days = direction === "prev" ? -7 : 7;
662 %   const newStart = new Date(dateRange.start);
663 %   const newEnd = new Date(dateRange.end);
664 %   newStart.setDate(newStart.getDate() + days);
665 %   newEnd.setDate(newEnd.getDate() + days);
666 %   setDateRange({
667 %     start: newStart.toISOString().split("T")[0],
668 %     end: newEnd.toISOString().split("T")[0],
669 %   });
670 % };
671 %
672 % const goToToday = () => {
673 %   const today = new Date();
674 %   const start = new Date(today);
675 %   start.setDate(start.getDate() - 1);
676 %   const end = new Date(today);
677 %   end.setDate(end.getDate() + 14);
678 %   setDateRange({
679 %     start: start.toISOString().split("T")[0],
680 %     end: end.toISOString().split("T")[0],
681 %   });
682 % };
683 %
684 % const goToDate = (date: Date | undefined) => {
685 %   if (!date) return;
686 %   const start = new Date(date);
687 %   start.setDate(start.getDate() - 1);
688 %   const end = new Date(date);
689 %   end.setDate(end.getDate() + 14);
690 %   setDateRange({
691 %     start: start.toISOString().split("T")[0],
692 %     end: end.toISOString().split("T")[0],
693 %   });
694 % };
695 %
696 % const handleCellClick = (room: RoomWithType, day: string, status: PlanningCellStatus, reservationId?: string) => {
697 %   if (status === "available") {
698 %     setSelectedCell({
699 %       roomId: room.id,
700 %       roomNumber: room.roomNumber,
701 %       roomTypeName: room.roomType.name,
702 %       checkInDate: day,
703 %     });
704 %     setQuickReservationOpen(true);
705 %   } else if (reservationId) {
706 %     setSelectedReservationId(reservationId);
707 %     setReservationDetailOpen(true);
708 %   }
709 % };
710 %
711 % const groupedRooms = data?.rooms.reduce((acc, room) => {
712 %   const floor = room.floor;
713 %   if (!acc[floor]) acc[floor] = [];
714 %   acc[floor].push(room);
715 %   return acc;
716 % }, {} as Record<number, typeof data.rooms>) || {};
717 %
718 % const floors = Object.keys(groupedRooms).map(Number).sort((a, b) => a - b);
719 %
720 % return (
721 %   <div className="flex flex-col gap-4 p-6 h-full">
722 %     <div className="flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-between">
723 %       <div>
724 %         <h1 className="text-3xl font-bold tracking-tight" data-testid="text-planning-title">
725 %           Planning de Ocupación
726 %         </h1>
727 %         <p className="text-muted-foreground">Vista de disponibilidad por habitación y fecha (15 días)</p>
728 %       </div>
729 %       <div className="flex items-center gap-2 flex-wrap">
730 %         <Button
731 %           variant="outline"
732 %           size="icon"
733 %           onClick={() => navigateDays("prev")}
734 %           data-testid="button-prev-week"
735 %         >
736 %           <ChevronLeft className="h-4 w-4" />
737 %         </Button>
738 %         <Button
739 %           variant="outline"
740 %           onClick={goToToday}
741 %           data-testid="button-today">
```

client/src/pages/planning.tsx (cont.)

```
742 %         >
743 %         Hoy
744 %       </Button>
745 %     <Popover>
746 %       <PopoverTrigger asChild>
747 %         <Button variant="outline" data-testid="button-date-picker">
748 %           <CalendarSearch className="h-4 w-4 mr-2" />
749 %           Ir a fecha
750 %         </Button>
751 %       </PopoverTrigger>
752 %       <PopoverContent className="w-auto p-0" align="end">
753 %         <CalendarPicker
754 %           mode="single"
755 %           selected={new Date(dateRange.start)}
756 %           onSelect={goToDate}
757 %           initialFocus
758 %         />
759 %       </PopoverContent>
760 %     </Popover>
761 %     <Button
762 %       variant="outline"
763 %       size="icon"
764 %       onClick={() => navigateDays("next")}
765 %       data-testid="button-next-week"
766 %     >
767 %       <ChevronRight className="h-4 w-4" />
768 %     </Button>
769 %   </div>
770 </div>
771
772 <Legend />
773
774 <Card className="flex-1 overflow-hidden">
775   <CardHeader className="py-3 px-4 border-b">
776     <CardTitle className="text-base font-medium flex items-center gap-2">
777       <Info className="h-4 w-4 text-muted-foreground" />
778       <span>
779         {data ? `${data.rooms.length} habitaciones` : "Cargando..."} | {" "}
780         {dateRange.start} – {dateRange.end}
781       </span>
782     </CardTitle>
783   </CardHeader>
784   <CardContent className="p-0 h-[calc(100%-60px)]">
785     {isLoading ? (
786       <div className="p-4 space-y-2">
787         {[...Array(10)].map((_, i) => (
788           <Skeleton key={i} className="h-10 w-full" />
789         ))}
790       </div>
791     ) : data ? (
792       <ScrollArea className="h-full">
793         <div className="min-w-max">
794           <table className="w-full border-collapse">
795             <thead className="sticky top-0 z-20 bg-background">
796               <tr>
797                 <th className="sticky left-0 z-30 bg-muted px-3 py-2 text-left text-sm font-medium w-24 border-...>
798                   Hab.
799                 </th>
800                 {data.days.map((day) => {
801                   const info = formatDate(day);
802                   return (
803                     <th
804                       key={day}
805                       className={`px-1 py-2 text-center text-xs font-medium border-b min-w-[60px] ${
806                         info.isToday ? "bg-primary/10" : info.isWeekend ? "bg-muted/50" : ""
807                       }`}
808                     >
809                       <div className="flex flex-col items-center gap-0.5">
810                         <span className="text-muted-foreground capitalize">{info.dayName}</span>
811                         <span className={`text-sm font-semibold ${info.isToday ? "text-primary" : ""}`}>
812                           {info.dayNumber}
813                         </span>
814                         <span className="text-muted-foreground capitalize">{info.monthName}</span>
815                       </div>
816                     </th>
817                   );
818                 })}
819               </tr>
820             </thead>
821             <tbody>
822               {floors.map((floor) => (
823                 <Fragment key={`floor-${floor}`}>
824                   <tr className="bg-muted/30">
825                     <td
826                       colspan={data.days.length + 1}
827                       className="px-3 py-1.5 text-xs font-semibold text-muted-foreground sticky left-0 bg-muted...>
828                       >
829                       Piso {floor}
830                     </td>
831                   </tr>
832                   {groupedRooms[floor]?.map((room) => (
833                     <tr key={room.id} className="hover:bg-muted/20" data-testid={`row-room-${room.id}`}>
834                       <td className="sticky left-0 z-10 bg-background px-3 py-1.5 border-r">
```



```

835 %      <Tooltip>
836 %      <TooltipTrigger asChild>
837 %      <div className="flex flex-col cursor-default" data-testid={`room-header-${room.id}`}>
838 %      <span className="font-medium text-sm">{room.roomNumber}</span>
839 %      <span className="text-xs text-muted-foreground">{room.roomType.name}</span>
840 %      {room.bedConfig && (
841 %      <span className="text-[10px] text-muted-foreground">{room.bedConfig}</span>
842 %      )}
843 %      {room.features && room.features.length > 0 && (
844 %      <div className="flex flex-row items-center gap-0.5 mt-0.5">
845 %      {room.features.map((feature) => {
846 %      const mapped = featureIconMap[feature];
847 %      if (!mapped) return null;
848 %      const IconComp = mapped.icon;
849 %      return <span key={feature} title={mapped.label}><IconComp className="h-3 w-...
850 %      )}}
851 %      </div>
852 %      )}
853 %    </div>
854 %  </TooltipTrigger>
855 %  <TooltipContent side="right" className="max-w-[220px]">
856 %    <div className="text-xs space-y-1">
857 %    <div className="font-semibold">{room.roomNumber} - {room.roomType.name}</div>
858 %    <div>Piso: {room.floor}</div>
859 %    {room.bedConfig && (
860 %    <div>Camaje: {bedConfigLabels[room.bedConfig] || room.bedConfig}</div>
861 %    )}
862 %    {room.maxOccupancy && (
863 %    <div>Ocupación máx: {room.maxOccupancy} personas</div>
864 %    )}
865 %    {room.features && room.features.length > 0 && (
866 %    <div>
867 %    <span className="font-medium">Características:</span>
868 %    <ul className="list-disc pl-3 mt-0.5">
869 %    {room.features.map((f) => (
870 %    <li key={f}>{featureIconMap[f]?.label || f}</li>
871 %    )}}
872 %    </ul>
873 %    </div>
874 %    )}
875 %    {room.notes && (
876 %    <div className="border-t pt-1 mt-1 text-muted-foreground">{room.notes}</div>
877 %    )}
878 %    </div>
879 %  </TooltipContent>
880 % </Tooltip>
881 % </td>
882 % {data.days.map((day, dayIndex) => {
883 %   const status = data.occupancy[room.id]?.[dayIndex] || "available";
884 %   const reservationId = data.cellReservations[room.id]?.[day];
885 %   const reservation = reservationId ? data.reservations[reservationId] : null;
886 %   const info = formatDate(day);
887 %   const isClickable = status === "available";
888 %
889 %   return (
890 %     <td
891 %       key={day}
892 %       className={`p-0.5 border-b ${info.isToday ? "bg-primary/5" : ""}`}
893 %     >
894 %       <Tooltip>
895 %       <TooltipTrigger asChild>
896 %       <div
897 %         onClick={() => handleCellClick(room, day, status, reservationId)}
898 %         className={`h-8 rounded border flex items-center justify-center transition-al...
899 %         reservation ? getSourceColor(reservation.source) : getStatusColor(status)
900 %       } ${
901 %         isClickable || reservationId
902 %         ? "cursor-pointer hover:ring-2 hover:ring-primary/50 hover:scale-105"
903 %         : "cursor-default"
904 %       }`
905 %       data-testid={`cell-${room.id}-${day}`}>
906 %       >
907 %       {reservation ? (
908 %         <span className="text-[10px] font-medium truncate px-1 max-w-[56px] inline-...
909 %         {reservation.earlyCheckIn && day === reservation.checkIn && (
910 %         <Sunrise className="h-3 w-3 text-orange-400 flex-shrink-0" data-testid=...
911 %         )}
912 %         {reservation.isGroup ? "GRP" : reservation.guestName.split(" ")[0]}
913 %         {reservation.lateCheckOut && day === reservation.checkOut && (
914 %         <Sunset className="h-3 w-3 text-purple-400 flex-shrink-0" data-testid=...
915 %         )}
916 %         </span>
917 %       ) : isClickable ? (
918 %         <Plus className="h-3 w-3 text-green-600 dark:text-green-400 opacity-0 group...
919 %       ) : null}
920 %     </div>
921 %   </TooltipTrigger>
922 %   <TooltipContent side="top" className="max-w-[200px]">
923 %     <div className="text-xs space-y-1">
924 %     <div className="font-semibold">{room.roomNumber} - {room.roomType.name}</div>
925 %     <div>Estado: {getStatusLabel(status)}</div>
926 %     {reservation ? (
927 %     <div className="border-t pt-1 mt-1">

```

client/src/pages/planning.tsx (cont.)

```
928 % {reservation.isGroup && reservation.groupName && (
929 %   <div className="font-semibold text-indigo-600 dark:text-indigo-400">
930 %     Grupo: {reservation.groupName}
931 %   </div>
932 % )}
933 % <div className="font-medium">{reservation.guestName}</div>
934 % <div className="text-muted-foreground">
935 %   {reservation.checkIn} a {reservation.checkOut}
936 % </div>
937 % <div className="text-muted-foreground">
938 %   Origen: {getSourceLabel(reservation.source)}
939 % </div>
940 % {reservation.earlyCheckIn && (
941 %   <div className="text-orange-400 font-medium">
942 %     Early Check-in: {reservation.earlyCheckInTime || "--"} hs
943 %   </div>
944 % )}
945 % {reservation.lateCheckOut && (
946 %   <div className="text-purple-400 font-medium">
947 %     Late Check-out: {reservation.lateCheckOutTime || "--"} hs
948 %   </div>
949 % )}
950 % <div className="border-t pt-1 mt-1 text-primary">
951 %   Clic para ver detalle
952 % </div>
953 % </div>
954 % ) : isClickable ? (
955 %   <div className="border-t pt-1 mt-1 text-primary">
956 %     Clic para crear reserva
957 %   </div>
958 %   ) : null}
959 % </div>
960 % </TooltipContent>
961 % </Tooltip>
962 % </td>
963 % );
964 % }}
965 % </tr>
966 % )}
967 % </Fragment>
968 % )}
969 % </tbody>
970 % </table>
971 % </div>
972 % <ScrollBar orientation="horizontal" />
973 % </ScrollArea>
974 % ) : null}
975 % </CardContent>
976 % </Card>
977 %
978 % <QuickReservationDialog
979 %   open={quickReservationOpen}
980 %   onOpenChange={setQuickReservationOpen}
981 %   reservationData={selectedCell}
982 %   guests={guests}
983 % />
984 %
985 % <ReservationDetailModal
986 %   open={reservationDetailOpen}
987 %   onOpenChange={setReservationDetailOpen}
988 %   reservationId={selectedReservationId}
989 %   onNavigate={navigate}
990 % />
991 % </div>
992 % );
993 % }
994 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import {
4 %   Login,
5 %   Search,
6 %   Calendar,
7 %   User,
8 %   DoorOpen,
9 %   Check,
10 %   Clock,
11 %   Plus,
12 %   UserPlus,
13 %   CalendarDays,
14 %   Smartphone,
15 %   Link2,
16 %   Copy,
17 %   ExternalLink,
18 %   CheckCircle2,
19 %   Send,
20 %   FileText,
21 %   Image,
22 %   AlertCircle,
23 %   Heart,
24 %   AlertTriangle,
25 % } from "lucide-react";
26 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
27 % import { Button } from "@components/ui/button";
28 % import { Input } from "@components/ui/input";
29 % import { Badge } from "@components/ui/badge";
30 % import { Skeleton } from "@components/ui/skeleton";
31 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
32 % import { Label } from "@components/ui/label";
33 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
34 % import { Textarea } from "@components/ui/textarea";
35 % import {
36 %   AlertDialog,
37 %   AlertDialogAction,
38 %   AlertDialogCancel,
39 %   AlertDialogContent,
40 %   AlertDialogDescription,
41 %   AlertDialogFooter,
42 %   AlertDialogHeader,
43 %   AlertDialogTitle,
44 % } from "@components/ui/alert-dialog";
45 % import {
46 %   Dialog,
47 %   DialogContent,
48 %   DialogHeader,
49 %   DialogTitle,
50 % } from "@components/ui/dialog";
51 % import { useToast } from "@hooks/use-toast";
52 % import { queryClient, apiRequest } from "@lib/queryClient";
53 % import { GuestSelector, CompanySelector } from "@components/entity-selector";
54 % import type { ReservationWithDetails, Guest, Company, RoomType, RoomWithType, RatePlan, InsertGuest, InsertCompany, W...
55 %
56 % interface WebCheckinListItem extends WebCheckin {
57 %   reservation?: {
58 %     reservationCode: string;
59 %     guestName: string;
60 %     roomNumber: string;
61 %     checkInDate: string;
62 %     checkOutDate: string;
63 %     status: string;
64 %   } | null;
65 % }
66 %
67 % export default function CheckInPage() {
68 %   const [toast] = useToast();
69 %   const [activeTab, setActiveTab] = useState<"reservations" | "walkin" | "webcheckin" | "history">("reservations");
70 %   const [searchQuery, setSearchQuery] = useState("");
71 %   const [selectedReservation, setSelectedReservation] = useState<ReservationWithDetails | null>(null);
72 %   const [confirmDialogOpen, setConfirmDialogOpen] = useState(false);
73 %
74 %   const [selectedGuest, setSelectedGuest] = useState<Guest | null>(null);
75 %   const [selectedCompany, setSelectedCompany] = useState<Company | null>(null);
76 %   const [selectedRoomTypeId, setSelectedRoomTypeId] = useState<string>("");
77 %   const [selectedRoomId, setSelectedRoomId] = useState<string>("");
78 %   const [selectedRatePlanId, setSelectedRatePlanId] = useState<string>("");
79 %   const [nights, setNights] = useState<number>(1);
80 %   const [numberOfGuests, setNumberOfGuests] = useState<number>(1);
81 %   const [walkInNotes, setWalkInNotes] = useState<string>("");
82 %   const [checkInNotes, setCheckInNotes] = useState<string>("");
83 %
84 %   const [historyDate, setHistoryDate] = useState<string>(() => {
85 %     return new Date().toISOString().split("T")[0];
86 %   });
87 %
88 %   const [webCheckinDialogOpen, setWebCheckinDialogOpen] = useState(false);
89 %   const [webCheckinReservation, setWebCheckinReservation] = useState<ReservationWithDetails | null>(null);
90 %   const [generatedLink, setGeneratedLink] = useState<string | null>(null);
```

client/src/pages/check-in.tsx (cont.)

```
91 %   const [webCheckinDetailId, setWebCheckinDetailId] = useState<string | null>(null);
92 %
93 %   const { data: reservations, isLoading } = useQuery<ReservationWithDetails[]>({
94 %     queryKey: ["/api/reservations/check-in"],
95 %   });
96 %
97 %   const { data: checkInsByDate, isLoading: isLoadingHistory } = useQuery<ReservationWithDetails[]>({
98 %     queryKey: ["/api/reservations/check-ins-by-date", historyDate],
99 %     queryFn: async () => {
100 %       const res = await apiRequest("GET", `/api/reservations/check-ins-by-date?date=${historyDate}`, undefined);
101 %       return res.json();
102 %     },
103 %     enabled: activeTab === "history",
104 %   });
105 %
106 %   const { data: roomTypes } = useQuery<RoomType[]>({
107 %     queryKey: ["/api/room-types"],
108 %   });
109 %
110 %   const { data: rooms } = useQuery<RoomWithType[]>({
111 %     queryKey: ["/api/rooms"],
112 %   });
113 %
114 %   const { data: ratePlans } = useQuery<RatePlan[]>({
115 %     queryKey: ["/api/rate-plans"],
116 %   });
117 %
118 %   const { data: allReservations } = useQuery<ReservationWithDetails[]>({
119 %     queryKey: ["/api/reservations"],
120 %     enabled: activeTab === "webcheckin",
121 %   });
122 %
123 %   const webCheckinReservations = allReservations?.filter(
124 %     (r) => r.status === "confirmed" || r.status === "pending"
125 %   );
126 %
127 %   const { data: webCheckinList } = useQuery<WebCheckinListItem[]>({
128 %     queryKey: ["/api/web-checkin/list"],
129 %     queryFn: async () => {
130 %       const res = await apiRequest("GET", "/api/web-checkin/list");
131 %       return res.json();
132 %     },
133 %     enabled: activeTab === "webcheckin",
134 %   });
135 %
136 %   const generateLinkMutation = useMutation({
137 %     mutationFn: async (reservationId: string) => {
138 %       const res = await apiRequest("POST", `/api/web-checkin/generate/${reservationId}`);
139 %       return res.json();
140 %     },
141 %     onSuccess: (data) => {
142 %       setGeneratedLink(data.link);
143 %       queryClient.invalidateQueries({ queryKey: ["/api/web-checkin/list"] });
144 %       toast({
145 %         title: "Link generado",
146 %         description: "El enlace de web check-in ha sido generado exitosamente.",
147 %       });
148 %     },
149 %     onError: () => {
150 %       toast({
151 %         title: "Error",
152 %         description: "No se pudo generar el enlace de web check-in.",
153 %         variant: "destructive",
154 %       });
155 %     },
156 %   });
157 %
158 %   const availableRooms = rooms?.filter((room) =>
159 %     room.status === "available" &&
160 %     (selectedRoomTypeId ? room.roomTypeId === selectedRoomTypeId : true)
161 %   );
162 %
163 %   const applicableRatePlans = ratePlans?.filter((rp) =>
164 %     rp.roomTypeId === selectedRoomTypeId
165 %   );
166 %
167 %   const { data: guestPreferences = [] } = useQuery<GuestPreference[]>({
168 %     queryKey: ["/api/guests", selectedReservation?.guestId, "preferences"],
169 %     enabled: !!selectedReservation?.guestId && confirmDialogOpen,
170 %   });
171 %
172 %   const activePrefs = guestPreferences.filter((p) => p.isActive);
173 %   const criticalPrefs = activePrefs.filter((p) => p.priority === "critical" || p.priority === "high");
174 %
175 %   const selectedRatePlan = ratePlans?.find((rp) => rp.id === selectedRatePlanId);
176 %   const totalAmount = selectedRatePlan ? (parseFloat(selectedRatePlan.baseRate) * nights).toFixed(2) : "0.00";
177 %
178 %   const checkInMutation = useMutation({
179 %     mutationFn: async (id: string) => {
180 %       return apiRequest("POST", `/api/reservations/${id}/check-in`, undefined);
181 %     },
182 %     onSuccess: () => {
183 %       queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
```

client/src/pages/check-in.tsx (cont.)

```
184 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations/check-in"] });
185 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations/check-ins-by-date"] });
186 %     queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
187 %     queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
188 %     queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
189 %     toast({
190 %       title: "Check-in realizado",
191 %       description: `El huesped ${selectedReservation?.guest?.firstName} ${selectedReservation?.guest?.lastName} ha ...
192 %     });
193 %     setConfirmDialogOpen(false);
194 %     setSelectedReservation(null);
195 %   },
196 %   onError: (error: any) => {
197 %     const message = error?.data?.error || error?.message || "No se pudo realizar el check-in. Intente nuevamente.";
198 %     toast({
199 %       title: "Check-in no permitido",
200 %       description: message,
201 %       variant: "destructive",
202 %     });
203 %   },
204 % });
205 %
206 % const createGuestMutation = useMutation({
207 %   mutationFn: async (guest: InsertGuest): Promise<Guest> => {
208 %     const res = await apiRequest("POST", "/api/guests", guest);
209 %     return res.json();
210 %   },
211 %   onSuccess: (newGuest: Guest) => {
212 %     queryClient.invalidateQueries({ queryKey: ["/api/guests"] });
213 %     setSelectedGuest(newGuest);
214 %     toast({
215 %       title: "Huesped creado",
216 %       description: `${newGuest.firstName} ${newGuest.lastName} ha sido registrado.`,
217 %     });
218 %   },
219 %   onError: () => {
220 %     toast({
221 %       title: "Error",
222 %       description: "No se pudo crear el huesped.",
223 %       variant: "destructive",
224 %     });
225 %   },
226 % });
227 %
228 % const createCompanyMutation = useMutation({
229 %   mutationFn: async (company: InsertCompany): Promise<Company> => {
230 %     const res = await apiRequest("POST", "/api/companies", company);
231 %     return res.json();
232 %   },
233 %   onSuccess: (newCompany: Company) => {
234 %     queryClient.invalidateQueries({ queryKey: ["/api/companies"] });
235 %     setSelectedCompany(newCompany);
236 %     toast({
237 %       title: "Empresa creada",
238 %       description: `${newCompany.razonSocial} ha sido registrada.`,
239 %     });
240 %   },
241 %   onError: () => {
242 %     toast({
243 %       title: "Error",
244 %       description: "No se pudo crear la empresa.",
245 %       variant: "destructive",
246 %     });
247 %   },
248 % });
249 %
250 % const walkInMutation = useMutation({
251 %   mutationFn: async () => {
252 %     const today = new Date().toISOString().split("T")[0];
253 %     const checkOutDate = new Date(Date.now() + nights * 86400000).toISOString().split("T")[0];
254 %
255 %     const res = await apiRequest("POST", "/api/reservations", {
256 %       reservationCode: "",
257 %       guestId: selectedGuest!.id,
258 %       companyId: selectedCompany?.id || null,
259 %       roomTypeId: selectedRoomTypeId,
260 %       roomId: selectedRoomId,
261 %       ratePlanId: selectedRatePlanId || null,
262 %       checkInDate: today,
263 %       checkOutDate,
264 %       nights,
265 %       baseRatePerNight: selectedRatePlan?.baseRate || "0",
266 %       discountType: "none",
267 %       discountValue: "0",
268 %       finalRatePerNight: selectedRatePlan?.baseRate || "0",
269 %       totalRoomAmount: totalAmount,
270 %       status: "confirmed",
271 %       source: selectedCompany ? "empresa" : "directo",
272 %       numberOfGuests,
273 %       notes: walkInNotes ? `Walk-in: ${walkInNotes}` : "Walk-in",
274 %     });
275 %     const reservation = await res.json();
276 %   }
```

client/src/pages/check-in.tsx (cont.)

```
277 %         await apiRequest("POST", `/api/reservations/${reservation.id}/check-in`, undefined);
278 %         return reservation;
279 %     },
280 %     onSuccess: () => {
281 %         queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
282 %         queryClient.invalidateQueries({ queryKey: ["/api/reservations/check-in"] });
283 %         queryClient.invalidateQueries({ queryKey: ["/api/reservations/check-ins-by-date"] });
284 %         queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
285 %         queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
286 %         queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
287 %         toast({
288 %             title: "Walk-in exitoso",
289 %             description: `${selectedGuest?.firstName} ${selectedGuest?.lastName} ha sido registrado en la habitacion.`,
290 %         });
291 %         resetWalkInForm();
292 %         setActiveTab("reservations");
293 %     },
294 %     onError: () => {
295 %         toast({
296 %             title: "Error",
297 %             description: "No se pudo completar el walk-in. Intente nuevamente.",
298 %             variant: "destructive",
299 %         });
300 %     },
301 % });
302 %
303 % const resetWalkInForm = () => {
304 %     setSelectedGuest(null);
305 %     setSelectedCompany(null);
306 %     setSelectedRoomTypeId("");
307 %     setSelectedRoomId("");
308 %     setSelectedRatePlanId("");
309 %     setNights(1);
310 %     setNumberOfGuests(1);
311 %     setWalkInNotes("");
312 % };
313 %
314 % const filteredReservations = reservations?.filter((res) => {
315 %     const guestName = `${res.guest?.firstName} ${res.guest?.lastName}`.toLowerCase();
316 %     return (
317 %         guestName.includes(searchQuery.toLowerCase()) ||
318 %         res.room?.roomNumber?.toLowerCase().includes(searchQuery.toLowerCase())
319 %     );
320 % });
321 %
322 % const handleCheckIn = (reservation: ReservationWithDetails) => {
323 %     setSelectedReservation(reservation);
324 %     setCheckInNotes(reservation.notes || "");
325 %     setConfirmDialogOpen(true);
326 % };
327 %
328 % const performCheckIn = async () => {
329 %     if (!selectedReservation) return;
330 %
331 %     if (checkInNotes !== (selectedReservation.notes || "")) {
332 %         await apiRequest("PATCH", `/api/reservations/${selectedReservation.id}`, {
333 %             notes: checkInNotes,
334 %         });
335 %     }
336 %
337 %     checkInMutation.mutate(selectedReservation.id);
338 % };
339 %
340 % const canSubmitWalkIn = selectedGuest && selectedRoomTypeId && selectedRoomId && nights > 0;
341 %
342 % const today = new Date().toLocaleDateString("es-ES", {
343 %     weekday: "long",
344 %     year: "numeric",
345 %     month: "long",
346 %     day: "numeric",
347 % });
348 %
349 % return (
350 %     <div className="flex flex-col gap-6 p-6">
351 %         <div className="flex flex-col gap-1">
352 %             <h1 className="text-3xl font-bold tracking-tight" data-testid="text-checkin-title">
353 %                 Check-in
354 %             </h1>
355 %             <p className="text-muted-foreground capitalize">{today}</p>
356 %         </div>
357 %
358 %         <Tabs value={activeTab} onValueChange={(v) => setActiveTab(v as "reservations" | "walkin" | "webcheckin" | "his...
359 %         <TabsList className="grid w-full max-w-3xl grid-cols-4">
360 %             <TabsTrigger value="reservations" data-testid="tab-reservations">
361 %                 <LogIn className="h-4 w-4 mr-2" />
362 %                 Reservas
363 %             </TabsTrigger>
364 %             <TabsTrigger value="walkin" data-testid="tab-walkin">
365 %                 <UserPlus className="h-4 w-4 mr-2" />
366 %                 Walk-in
367 %             </TabsTrigger>
368 %             <TabsTrigger value="webcheckin" data-testid="tab-webcheckin">
369 %                 <Smartphone className="h-4 w-4 mr-2" />
```

client/src/pages/check-in.tsx (cont.)

```
370 %         Web Check-in
371 %       </TabsTrigger>
372 %       <TabsTrigger value="history" data-testid="tab-history">
373 %         <CalendarDays className="h-4 w-4 mr-2" />
374 %         Historial
375 %       </TabsTrigger>
376 %     </TabsList>
377 %
378 %     <TabsContent value="reservations" className="space-y-6 mt-6">
379 %       <Card className="bg-primary/5 border-primary/20">
380 %         <CardContent className="flex items-center gap-4 p-4">
381 %           <div className="flex h-12 w-12 items-center justify-center rounded-full bg-primary/10">
382 %             <LogIn className="h-6 w-6 text-primary" />
383 %           </div>
384 %           <div>
385 %             <h3 className="font-semibold">Registro de Llegadas</h3>
386 %             <p className="text-sm text-muted-foreground">
387 %               Reservas confirmadas listas para check-in
388 %             </p>
389 %           </div>
390 %           <Badge className="ml-auto" variant="secondary">
391 %             {reservations?.length || 0} pendientes
392 %           </Badge>
393 %         </CardContent>
394 %       </Card>
395 %
396 %       <Card>
397 %         <CardContent className="p-4">
398 %           <div className="relative">
399 %             <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
400 %             <Input
401 %               placeholder="Buscar por nombre de huesped o numero de habitacion..."
402 %               value={searchQuery}
403 %               onChange={(e) => setSearchQuery(e.target.value)}
404 %               className="pl-10"
405 %               data-testid="input-search-checkin"
406 %             />
407 %           </div>
408 %         </CardContent>
409 %       </Card>
410 %
411 %       {isLoading ? (
412 %         <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
413 %           {[...Array(3)].map((_, i) => (
414 %             <Skeleton key={i} className="h-48" />
415 %           ))}
416 %         </div>
417 %       ) : filteredReservations && filteredReservations.length > 0 ? (
418 %         <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
419 %           {filteredReservations.map((reservation) => (
420 %             <Card
421 %               key={reservation.id}
422 %               className="hover-elevate"
423 %               data-testid={`checkin-card-${reservation.id}`}
424 %             >
425 %               <CardHeader className="pb-3">
426 %                 <div className="flex items-start justify-between gap-2">
427 %                   <div className="flex items-center gap-3">
428 %                     <div className="flex h-10 w-10 items-center justify-center rounded-full bg-primary/10 text-pr...
429 %                       {reservation.guest?.firstName?.[0]}{reservation.guest?.lastName?.[0]}
430 %                     </div>
431 %                     <div>
432 %                       <CardTitle className="text-lg">
433 %                         {reservation.guest?.firstName} {reservation.guest?.lastName}
434 %                       </CardTitle>
435 %                       <CardDescription>{reservation.guest?.email}</CardDescription>
436 %                     </div>
437 %                   </div>
438 %                   <Badge variant="secondary">Confirmada</Badge>
439 %                 </div>
440 %               </CardHeader>
441 %               <CardContent className="space-y-3">
442 %                 <div className="grid grid-cols-2 gap-3">
443 %                   <div className="flex items-center gap-2 text-sm">
444 %                     <DoorOpen className="h-4 w-4 text-muted-foreground" />
445 %                     <span>Hab. {reservation.room?.roomNumber}</span>
446 %                   </div>
447 %                   <div className="flex items-center gap-2 text-sm">
448 %                     <User className="h-4 w-4 text-muted-foreground" />
449 %                     <span>{reservation.numberofGuests} huesped(es)</span>
450 %                   </div>
451 %                 </div>
452 %                 <div className="grid grid-cols-2 gap-3">
453 %                   <div className="flex items-center gap-2 text-sm">
454 %                     <Calendar className="h-4 w-4 text-muted-foreground" />
455 %                     <span>{reservation.checkInDate}</span>
456 %                   </div>
457 %                   <div className="flex items-center gap-2 text-sm">
458 %                     <Clock className="h-4 w-4 text-muted-foreground" />
459 %                     <span>{reservation.checkOutDate}</span>
460 %                   </div>
461 %                 </div>
462 %               </div>
463 %             </div>
464 %           )}
465 %         </div>
466 %       ) : null
467 %     </div>
468 %   </div>
469 % </div>
470 % </div>
471 % </div>
472 % </div>
473 % </div>
474 % </div>
475 % </div>
476 % </div>
477 % </div>
478 % </div>
479 % </div>
480 % </div>
481 % </div>
482 % </div>
483 % </div>
484 % </div>
485 % </div>
486 % </div>
487 % </div>
488 % </div>
489 % </div>
490 % </div>
491 % </div>
492 % </div>
493 % </div>
494 % </div>
495 % </div>
496 % </div>
497 % </div>
498 % </div>
499 % </div>
500 % </div>
501 % </div>
502 % </div>
503 % </div>
504 % </div>
505 % </div>
506 % </div>
507 % </div>
508 % </div>
509 % </div>
510 % </div>
511 % </div>
512 % </div>
513 % </div>
514 % </div>
515 % </div>
516 % </div>
517 % </div>
518 % </div>
519 % </div>
520 % </div>
521 % </div>
522 % </div>
523 % </div>
524 % </div>
525 % </div>
526 % </div>
527 % </div>
528 % </div>
529 % </div>
530 % </div>
531 % </div>
532 % </div>
533 % </div>
534 % </div>
535 % </div>
536 % </div>
537 % </div>
538 % </div>
539 % </div>
540 % </div>
541 % </div>
542 % </div>
543 % </div>
544 % </div>
545 % </div>
546 % </div>
547 % </div>
548 % </div>
549 % </div>
550 % </div>
551 % </div>
552 % </div>
553 % </div>
554 % </div>
555 % </div>
556 % </div>
557 % </div>
558 % </div>
559 % </div>
560 % </div>
561 % </div>
562 % </div>
563 % </div>
564 % </div>
565 % </div>
566 % </div>
567 % </div>
568 % </div>
569 % </div>
570 % </div>
571 % </div>
572 % </div>
573 % </div>
574 % </div>
575 % </div>
576 % </div>
577 % </div>
578 % </div>
579 % </div>
580 % </div>
581 % </div>
582 % </div>
583 % </div>
584 % </div>
585 % </div>
586 % </div>
587 % </div>
588 % </div>
589 % </div>
590 % </div>
591 % </div>
592 % </div>
593 % </div>
594 % </div>
595 % </div>
596 % </div>
597 % </div>
598 % </div>
599 % </div>
600 % </div>
601 % </div>
602 % </div>
603 % </div>
604 % </div>
605 % </div>
606 % </div>
607 % </div>
608 % </div>
609 % </div>
610 % </div>
611 % </div>
612 % </div>
613 % </div>
614 % </div>
615 % </div>
616 % </div>
617 % </div>
618 % </div>
619 % </div>
620 % </div>
621 % </div>
622 % </div>
623 % </div>
624 % </div>
625 % </div>
626 % </div>
627 % </div>
628 % </div>
629 % </div>
630 % </div>
631 % </div>
632 % </div>
633 % </div>
634 % </div>
635 % </div>
636 % </div>
637 % </div>
638 % </div>
639 % </div>
640 % </div>
641 % </div>
642 % </div>
643 % </div>
644 % </div>
645 % </div>
646 % </div>
647 % </div>
648 % </div>
649 % </div>
650 % </div>
651 % </div>
652 % </div>
653 % </div>
654 % </div>
655 % </div>
656 % </div>
657 % </div>
658 % </div>
659 % </div>
660 % </div>
661 % </div>
662 % </div>
663 % </div>
664 % </div>
665 % </div>
666 % </div>
667 % </div>
668 % </div>
669 % </div>
670 % </div>
671 % </div>
672 % </div>
673 % </div>
674 % </div>
675 % </div>
676 % </div>
677 % </div>
678 % </div>
679 % </div>
680 % </div>
681 % </div>
682 % </div>
683 % </div>
684 % </div>
685 % </div>
686 % </div>
687 % </div>
688 % </div>
689 % </div>
690 % </div>
691 % </div>
692 % </div>
693 % </div>
694 % </div>
695 % </div>
696 % </div>
697 % </div>
698 % </div>
699 % </div>
700 % </div>
701 % </div>
702 % </div>
703 % </div>
704 % </div>
705 % </div>
706 % </div>
707 % </div>
708 % </div>
709 % </div>
710 % </div>
711 % </div>
712 % </div>
713 % </div>
714 % </div>
715 % </div>
716 % </div>
717 % </div>
718 % </div>
719 % </div>
720 % </div>
721 % </div>
722 % </div>
723 % </div>
724 % </div>
725 % </div>
726 % </div>
727 % </div>
728 % </div>
729 % </div>
730 % </div>
731 % </div>
732 % </div>
733 % </div>
734 % </div>
735 % </div>
736 % </div>
737 % </div>
738 % </div>
739 % </div>
740 % </div>
741 % </div>
742 % </div>
743 % </div>
744 % </div>
745 % </div>
746 % </div>
747 % </div>
748 % </div>
749 % </div>
750 % </div>
751 % </div>
752 % </div>
753 % </div>
754 % </div>
755 % </div>
756 % </div>
757 % </div>
758 % </div>
759 % </div>
760 % </div>
761 % </div>
762 % </div>
763 % </div>
764 % </div>
765 % </div>
766 % </div>
767 % </div>
768 % </div>
769 % </div>
770 % </div>
771 % </div>
772 % </div>
773 % </div>
774 % </div>
775 % </div>
776 % </div>
777 % </div>
778 % </div>
779 % </div>
780 % </div>
781 % </div>
782 % </div>
783 % </div>
784 % </div>
785 % </div>
786 % </div>
787 % </div>
788 % </div>
789 % </div>
790 % </div>
791 % </div>
792 % </div>
793 % </div>
794 % </div>
795 % </div>
796 % </div>
797 % </div>
798 % </div>
799 % </div>
800 % </div>
801 % </div>
802 % </div>
803 % </div>
804 % </div>
805 % </div>
806 % </div>
807 % </div>
808 % </div>
809 % </div>
810 % </div>
811 % </div>
812 % </div>
813 % </div>
814 % </div>
815 % </div>
816 % </div>
817 % </div>
818 % </div>
819 % </div>
820 % </div>
821 % </div>
822 % </div>
823 % </div>
824 % </div>
825 % </div>
826 % </div>
827 % </div>
828 % </div>
829 % </div>
830 % </div>
831 % </div>
832 % </div>
833 % </div>
834 % </div>
835 % </div>
836 % </div>
837 % </div>
838 % </div>
839 % </div>
840 % </div>
841 % </div>
842 % </div>
843 % </div>
844 % </div>
845 % </div>
846 % </div>
847 % </div>
848 % </div>
849 % </div>
850 % </div>
851 % </div>
852 % </div>
853 % </div>
854 % </div>
855 % </div>
856 % </div>
857 % </div>
858 % </div>
859 % </div>
860 % </div>
861 % </div>
862 % </div>
863 % </div>
864 % </div>
865 % </div>
866 % </div>
867 % </div>
868 % </div>
869 % </div>
870 % </div>
871 % </div>
872 % </div>
873 % </div>
874 % </div>
875 % </div>
876 % </div>
877 % </div>
878 % </div>
879 % </div>
880 % </div>
881 % </div>
882 % </div>
883 % </div>
884 % </div>
885 % </div>
886 % </div>
887 % </div>
888 % </div>
889 % </div>
890 % </div>
891 % </div>
892 % </div>
893 % </div>
894 % </div>
895 % </div>
896 % </div>
897 % </div>
898 % </div>
899 % </div>
900 % </div>
901 % </div>
902 % </div>
903 % </div>
904 % </div>
905 % </div>
906 % </div>
907 % </div>
908 % </div>
909 % </div>
910 % </div>
911 % </div>
912 % </div>
913 % </div>
914 % </div>
915 % </div>
916 % </div>
917 % </div>
918 % </div>
919 % </div>
920 % </div>
921 % </div>
922 % </div>
923 % </div>
924 % </div>
925 % </div>
926 % </div>
927 % </div>
928 % </div>
929 % </div>
930 % </div>
931 % </div>
932 % </div>
933 % </div>
934 % </div>
935 % </div>
936 % </div>
937 % </div>
938 % </div>
939 % </div>
940 % </div>
941 % </div>
942 % </div>
943 % </div>
944 % </div>
945 % </div>
946 % </div>
947 % </div>
948 % </div>
949 % </div>
950 % </div>
951 % </div>
952 % </div>
953 % </div>
954 % </div>
955 % </div>
956 % </div>
957 % </div>
958 % </div>
959 % </div>
960 % </div>
961 % </div>
962 % </div>
963 % </div>
964 % </div>
965 % </div>
966 % </div>
967 % </div>
968 % </div>
969 % </div>
970 % </div>
971 % </div>
972 % </div>
973 % </div>
974 % </div>
975 % </div>
976 % </div>
977 % </div>
978 % </div>
979 % </div>
980 % </div>
981 % </div>
982 % </div>
983 % </div>
984 % </div>
985 % </div>
986 % </div>
987 % </div>
988 % </div>
989 % </div>
990 % </div>
991 % </div>
992 % </div>
993 % </div>
994 % </div>
995 % </div>
996 % </div>
997 % </div>
998 % </div>
999 % </div>
1000 % </div>
```

client/src/pages/check-in.tsx (cont.)

```
463 %         <Button
464 %           className="w-full"
465 %           onClick={() => handleCheckIn(reservation)}
466 %           data-testid={`button-checkin-${reservation.id}`}
467 %         >
468 %           <Check className="mr-2 h-4 w-4" />
469 %           Realizar Check-in
470 %         </Button>
471 %       </div>
472 %     </CardContent>
473 %   </Card>
474 % ))}
475 % </div>
476 % ) : (
477 %   <Card>
478 %     <CardContent className="flex flex-col items-center justify-center py-12 text-center">
479 %       <LogIn className="h-16 w-16 text-muted-foreground/50 mb-4" />
480 %       <h3 className="text-lg font-semibold mb-2">No hay check-ins pendientes</h3>
481 %       <p className="text-muted-foreground mb-4">
482 %         {searchQuery
483 %           ? "No se encontraron reservas con los criterios de búsqueda."
484 %           : "Todas las reservas confirmadas ya han realizado check-in."}
485 %       </p>
486 %       <Button variant="outline" onClick={() => setActiveTab("walkin")} data-testid="button-goto-walkin">
487 %         <Plus className="h-4 w-4 mr-2" />
488 %         Registrar Walk-in
489 %       </Button>
490 %     </CardContent>
491 %   </Card>
492 % )}
493 % </TabsContent>
494 %
495 % <TabsContent value="walkin" className="space-y-6 mt-6">
496 %   <Card className="bg-accent/30 border-accent">
497 %     <CardContent className="flex items-center gap-4 p-4">
498 %       <div className="flex h-12 w-12 items-center justify-center rounded-full bg-accent">
499 %         <UserPlus className="h-6 w-6" />
500 %       </div>
501 %       <div>
502 %         <h3 className="font-semibold">Walk-in (Sin Reserva)</h3>
503 %         <p className="text-sm text-muted-foreground">
504 %           Registrar un huesped que llega sin reserva previa
505 %         </p>
506 %       </div>
507 %     </CardContent>
508 %   </Card>
509 %
510 %   <div className="grid gap-6 lg:grid-cols-2">
511 %     <div className="space-y-6">
512 %       <GuestSelector
513 %         selectedGuest={selectedGuest}
514 %         onSelect={setSelectedGuest}
515 %         onCreateNew={(guest) => createGuestMutation.mutate(guest)}
516 %         onClear={() => setSelectedGuest(null)}
517 %       />
518 %
519 %       <CompanySelector
520 %         selectedCompany={selectedCompany}
521 %         onSelect={setSelectedCompany}
522 %         onCreateNew={(company) => createCompanyMutation.mutate(company)}
523 %         onClear={() => setSelectedCompany(null)}
524 %       />
525 %     </div>
526 %
527 %     <div className="space-y-6">
528 %       <Card>
529 %         <CardHeader className="pb-3">
530 %           <CardTitle className="text-base flex items-center gap-2">
531 %             <DoorOpen className="h-4 w-4" />
532 %             Habitación
533 %           </CardTitle>
534 %         </CardHeader>
535 %         <CardContent className="space-y-4">
536 %           <div className="space-y-2">
537 %             <Label>Tipo de Habitación</Label>
538 %             <Select
539 %               value={selectedRoomTypeId}
540 %               onValueChange={(v) => {
541 %                 setSelectedRoomTypeId(v);
542 %                 setSelectedRoomId("");
543 %                 setSelectedRatePlanId("");
544 %               }}
545 %             >
546 %               <SelectTrigger data-testid="select-room-type">
547 %                 <SelectValue placeholder="Seleccionar tipo..." />
548 %               </SelectTrigger>
549 %               <SelectContent>
550 %                 {roomTypes?.map((rt) => (
551 %                   <SelectItem key={rt.id} value={rt.id}>
552 %                     {rt.name} - {rt.maxOccupancy} pax
553 %                   </SelectItem>
554 %                 ))}
555 %               </SelectContent>
```



```

556 %         </Select>
557 %     </div>
558 %
559 %     {selectedRoomTypeId && (
560 %         <div className="space-y-2">
561 %             <Label>Habitacion Disponible</Label>
562 %             <Select value={selectedRoomId} onValueChange={setSelectedRoomId}>
563 %                 <SelectTrigger data-testid="select-room">
564 %                     <SelectValue placeholder="Seleccionar habitacion..." />
565 %                 </SelectTrigger>
566 %                 <SelectContent>
567 %                     {availableRooms?.map((room) => (
568 %                         <SelectItem key={room.id} value={room.id}>
569 %                             Hab. {room.roomNumber} - Piso {room.floor}
570 %                         </SelectItem>
571 %                     ))}
572 %                 </SelectContent>
573 %             </Select>
574 %             {availableRooms?.length === 0 && (
575 %                 <p className="text-sm text-destructive">
576 %                     No hay habitaciones disponibles de este tipo
577 %                 </p>
578 %             )}
579 %         </div>
580 %     )}
581 %
582 %     {selectedRoomTypeId && applicableRatePlans && applicableRatePlans.length > 0 && (
583 %         <div className="space-y-2">
584 %             <Label>Plan de Tarifa</Label>
585 %             <Select value={selectedRatePlanId} onValueChange={setSelectedRatePlanId}>
586 %                 <SelectTrigger data-testid="select-rate-plan">
587 %                     <SelectValue placeholder="Seleccionar tarifa..." />
588 %                 </SelectTrigger>
589 %                 <SelectContent>
590 %                     {applicableRatePlans.map((rp) => (
591 %                         <SelectItem key={rp.id} value={rp.id}>
592 %                             {rp.name} - ${rp.baseRate}/noche
593 %                         </SelectItem>
594 %                     ))}
595 %                 </SelectContent>
596 %             </Select>
597 %         </div>
598 %     )}
599 %
600 %     <div className="grid grid-cols-2 gap-4">
601 %         <div className="space-y-2">
602 %             <Label htmlFor="nights">Noches</Label>
603 %             <Input
604 %                 id="nights"
605 %                 type="number"
606 %                 min={1}
607 %                 value={nights}
608 %                 onChange={(e) => setNights(parseInt(e.target.value) || 1)}
609 %                 data-testid="input-nights"
610 %             />
611 %         </div>
612 %         <div className="space-y-2">
613 %             <Label htmlFor="numGuests">Huespedes</Label>
614 %             <Input
615 %                 id="numGuests"
616 %                 type="number"
617 %                 min={1}
618 %                 value={numberOfGuests}
619 %                 onChange={(e) => setNumberOfGuests(parseInt(e.target.value) || 1)}
620 %                 data-testid="input-num-guests"
621 %             />
622 %         </div>
623 %     </div>
624 % </CardContent>
625 % </Card>
626 %
627 % <Card>
628 %     <CardHeader className="pb-3">
629 %         <CardTitle className="text-base">Resumen</CardTitle>
630 %     </CardHeader>
631 %     <CardContent className="space-y-3">
632 %         <div className="flex justify-between text-sm">
633 %             <span className="text-muted-foreground">Huesped: </span>
634 %             <span className="font-medium">
635 %                 {selectedGuest ? `${selectedGuest.firstName} ${selectedGuest.lastName}` : "-"}
636 %             </span>
637 %         </div>
638 %         {selectedCompany && (
639 %             <div className="flex justify-between text-sm">
640 %                 <span className="text-muted-foreground">Empresa: </span>
641 %                 <span className="font-medium">{selectedCompany.nombreFantasia || selectedCompany.razonSocial}</...
642 %             </div>
643 %         )}
644 %         <div className="flex justify-between text-sm">
645 %             <span className="text-muted-foreground">Habitacion: </span>
646 %             <span className="font-medium">
647 %                 {selectedRoomId ? rooms?.find((r) => r.id === selectedRoomId)?.roomNumber : "-"}
648 %             </span>

```

client/src/pages/check-in.tsx (cont.)

```
649 %         </div>
650 %         <div className="flex justify-between text-sm">
651 %           <span className="text-muted-foreground">Noches:</span>
652 %           <span className="font-medium">{nights}</span>
653 %         </div>
654 %         <div className="flex justify-between text-sm">
655 %           <span className="text-muted-foreground">Tarifa:</span>
656 %           <span className="font-medium">
657 %             {selectedRatePlan ? `$$${selectedRatePlan.baseRate}/noche` : "-"}
658 %           </span>
659 %         </div>
660 %         <div className="border-t pt-3 mt-3">
661 %           <div className="flex justify-between">
662 %             <span className="font-semibold">Total:</span>
663 %             <span className="font-bold text-lg">${totalAmount}</span>
664 %           </div>
665 %         </div>
666 %
667 %         <div className="space-y-2 mt-4">
668 %           <Label htmlFor="walkInNotes">Comentarios / Notas</Label>
669 %           <Textarea
670 %             id="walkInNotes"
671 %             placeholder="Agregar comentarios para recepción..."
672 %             value={walkInNotes}
673 %             onChange={(e) => setWalkInNotes(e.target.value)}
674 %             className="min-h-[60px]"
675 %             data-testid="input-walkin-notes"
676 %           />
677 %         </div>
678 %
679 %         <Button
680 %           className="w-full mt-4"
681 %           onClick={() => walkInMutation.mutate()}
682 %           disabled={!canSubmitWalkIn || walkInMutation.isPending}
683 %           data-testid="button-complete-walkin"
684 %         >
685 %           {walkInMutation.isPending ? (
686 %             "Procesando..."
687 %           ) : (
688 %             <div>
689 %               <Check className="h-4 w-4 mr-2" />
690 %               Completar Walk-in
691 %             </div>
692 %           )}
693 %         </Button>
694 %       </CardContent>
695 %     </Card>
696 %   </div>
697 % </div>
698 % </TabsContent>
699 %
700 % <TabsContent value="webcheckin" className="space-y-6 mt-6">
701 %   <Card className="bg-green-50 dark:bg-green-950/30 border-green-200 dark:border-green-800">
702 %     <CardContent className="flex items-center gap-4 p-4">
703 %       <div className="flex h-12 w-12 items-center justify-center rounded-full bg-green-100 dark:bg-green-900">
704 %         <Smartphone className="h-6 w-6 text-green-600 dark:text-green-400" />
705 %       </div>
706 %       <div>
707 %         <h3 className="font-semibold">Web Check-in</h3>
708 %         <p className="text-sm text-muted-foreground">
709 %           Genera links para que los huéspedes completen su check-in desde el celular
710 %         </p>
711 %       </div>
712 %     </CardContent>
713 %   </Card>
714 %
715 %   <Card>
716 %     <CardHeader>
717 %       <CardTitle className="text-lg">Generar Link de Web Check-in</CardTitle>
718 %       <CardDescription>Seleccioná una reserva confirmada para generar el enlace</CardDescription>
719 %     </CardHeader>
720 %     <CardContent>
721 %       {webCheckinReservations && webCheckinReservations.length > 0 ? (
722 %         <div className="space-y-2">
723 %           {webCheckinReservations.map((res) => (
724 %             <div>
725 %               key={res.id}
726 %               className="flex items-center justify-between p-3 border rounded-lg hover:bg-muted/50 transition...
727 %               data-testid={`webcheckin-res-${res.id}`}
728 %             >
729 %               <div className="flex items-center gap-3">
730 %                 <div className="flex h-9 w-9 items-center justify-center rounded-full bg-primary/10 text-prim...
731 %                   {res.guest?.firstName?.[0]}{res.guest?.lastName?.[0]}
732 %                 </div>
733 %                 <div>
734 %                   <p className="font-medium text-sm">{res.guest?.firstName} {res.guest?.lastName}</p>
735 %                   <p className="text-xs text-muted-foreground">
736 %                     Hab. {res.room?.roomNumber} | {res.checkInDate} - {res.checkOutDate}
737 %                   </p>
738 %                 </div>
739 %               </div>
740 %               <div className="flex items-center gap-2">
741 %                 <Badge variant={res.status === "confirmed" ? "default" : "secondary"} className="text-xs">
```

```

742 %         {res.status === "confirmed" ? "Confirmada" : "Pendiente"}
743 %     </Badge>
744 %     <Button
745 %         size="sm"
746 %         variant="outline"
747 %         onClick={() => {
748 %             setWebCheckinReservation(res);
749 %             setGeneratedLink(null);
750 %             setWebCheckinDialogOpen(true);
751 %             generateLinkMutation.mutate(res.id);
752 %         }}
753 %         disabled={generateLinkMutation.isPending}
754 %         data-testid={`button-generate-link-${res.id}`}
755 %     >
756 %         <Link2 className="h-4 w-4 mr-1" />
757 %         Generar Link
758 %     </Button>
759 % </div>
760 % </div>
761 %     )}}
762 % </div>
763 % ) : (
764 %     <p className="text-sm text-muted-foreground text-center py-6">
765 %         No hay reservas pendientes o confirmadas para generar web check-in
766 %     </p>
767 % )}
768 % </CardContent>
769 % </Card>
770 %
771 % <Card>
772 %     <CardHeader>
773 %         <CardTitle className="text-lg">Web Check-ins Enviados</CardTitle>
774 %         <CardDescription>Estado de los web check-ins generados</CardDescription>
775 %     </CardHeader>
776 %     <CardContent>
777 %         {webCheckinList && webCheckinList.length > 0 ? (
778 %             <div className="space-y-2">
779 %                 {webCheckinList.map((wc) => (
780 %                     <div
781 %                         key={wc.id}
782 %                         className="flex items-center justify-between p-3 border rounded-lg"
783 %                         data-testid={`webcheckin-item-${wc.id}`}
784 %                     >
785 %                         <div className="flex items-center gap-3">
786 %                             <div className={`h-9 w-9 rounded-full flex items-center justify-center ${
787 %                                 wc.status === "completed" ? "bg-green-100 dark:bg-green-900" :
788 %                                 wc.status === "expired" ? "bg-red-100 dark:bg-red-900" :
789 %                                 "bg-amber-100 dark:bg-amber-900"
790 %                             }`}>
791 %                                 {wc.status === "completed" ? (
792 %                                     <CheckCircle2 className="h-4 w-4 text-green-600 dark:text-green-400" />
793 %                                 ) : wc.status === "expired" ? (
794 %                                     <Clock className="h-4 w-4 text-red-600 dark:text-red-400" />
795 %                                 ) : (
796 %                                     <Send className="h-4 w-4 text-amber-600 dark:text-amber-400" />
797 %                                 )}
798 %                             </div>
799 %                             <div>
800 %                                 <p className="font-medium text-sm">{wc.reservation?.guestName || "Huésped"}</p>
801 %                                 <p className="text-xs text-muted-foreground">
802 %                                     {wc.reservation?.roomNumber ? `Hab. ${wc.reservation.roomNumber} | ` : ""}
803 %                                     {wc.reservation?.checkInDate || ""} - {wc.reservation?.checkOutDate || ""}
804 %                                 </p>
805 %                             </div>
806 %                         </div>
807 %                         <div className="flex items-center gap-2">
808 %                             <Badge variant={
809 %                                 wc.status === "completed" ? "default" :
810 %                                 wc.status === "expired" ? "destructive" :
811 %                                 "secondary"
812 %                             } data-testid={`badge-wc-status-${wc.id}`}>
813 %                                 {wc.status === "completed" ? "Completado" :
814 %                                 wc.status === "expired" ? "Expirado" :
815 %                                 "Pendiente"}
816 %                             </Badge>
817 %                             {wc.status === "completed" && (
818 %                                 <Button
819 %                                     variant="ghost"
820 %                                     size="sm"
821 %                                     onClick={() => setWebCheckinDetailId(webCheckinDetailId === String(wc.id) ? null : String...
822 %                                     data-testid={`button-wc-detail-${wc.id}`}
823 %                                 >
824 %                                     <FileText className="h-4 w-4" />
825 %                                 </Button>
826 %                             )}
827 %                         </div>
828 %                     </div>
829 %                 )}}
830 %                 {webCheckinList.filter(wc => wc.status === "completed" && webCheckinDetailId === String(wc.id)).map...
831 %                 <Card key={`detail-${wc.id}`} className="border-green-200 dark:border-green-800 bg-green-50/50 da...
832 %                 <CardContent className="p-4 space-y-3">
833 %                     <h4 className="font-semibold text-sm flex items-center gap-2">
834 %                         <CheckCircle2 className="h-4 w-4 text-green-600" />

```

```

835 %         Datos confirmados por el huésped
836 %     </h4>
837 %     <div className="grid grid-cols-2 gap-x-4 gap-y-2 text-sm">
838 %         <div><span className="text-muted-foreground">Nombre:</span> {wc.confirmedFirstName} {wc.con...
839 %         <div><span className="text-muted-foreground">Documento:</span> {wc.confirmedDocumentType} {...
840 %         <div><span className="text-muted-foreground">Nacionalidad:</span> {wc.confirmedNationality} ...
841 %         <div><span className="text-muted-foreground">Teléfono:</span> {wc.confirmedPhone || "-"}</div>
842 %         <div><span className="text-muted-foreground">Email:</span> {wc.confirmedEmail || "-"}</div>
843 %         <div><span className="text-muted-foreground">Llegada:</span> {wc.estimatedArrivalTime || "N...
844 %         {wc.requestEarlyCheckIn && (
845 %             <div className="col-span-2">
846 %                 <Badge variant="outline" className="text-amber-600 border-amber-300">
847 %                     Early check-in solicitado{wc.earlyCheckInTime ? `: ${wc.earlyCheckInTime}` : ""}
848 %                 </Badge>
849 %             </div>
850 %         )}
851 %     </div>
852 %     {wc.documentPhotoUrl && (
853 %         <div>
854 %             <p className="text-xs text-muted-foreground mb-1 flex items-center gap-1">
855 %                 <Image className="h-3 w-3" /> Foto del documento
856 %             </p>
857 %             <img src={wc.documentPhotoUrl} alt="Documento" className="max-w-xs rounded border" data-t...
858 %             </div>
859 %         )}
860 %     </CardContent>
861 % </Card>
862 %     )}
863 % </div>
864 % ) : (
865 %     <p className="text-sm text-muted-foreground text-center py-6">
866 %         No se han generado web check-ins aún
867 %     </p>
868 % )}
869 % </CardContent>
870 % </Card>
871 % </TabsContent>
872 %
873 % <TabsContent value="history" className="space-y-6 mt-6">
874 %     <Card className="bg-accent/30 border-accent">
875 %         <CardContent className="flex items-center gap-4 p-4">
876 %             <div className="flex h-12 w-12 items-center justify-center rounded-full bg-accent">
877 %                 <CalendarDays className="h-6 w-6" />
878 %             </div>
879 %             <div className="flex-1">
880 %                 <h3 className="font-semibold">Check-ins por Fecha</h3>
881 %                 <p className="text-sm text-muted-foreground">
882 %                     Ver huéspedes que hicieron check-in en una fecha específica
883 %                 </p>
884 %             </div>
885 %             <div className="flex items-center gap-2">
886 %                 <Label className="text-sm">Fecha:</Label>
887 %                 <Input
888 %                     type="date"
889 %                     value={historyDate}
890 %                     onChange={(e) => setHistoryDate(e.target.value)}
891 %                     className="w-auto"
892 %                     data-testid="input-history-date"
893 %                 />
894 %             </div>
895 %         </CardContent>
896 %     </Card>
897 %
898 %     {isLoadingHistory ? (
899 %         <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
900 %             {[...Array(3)].map((_, i) => (
901 %                 <Skeleton key={i} className="h-40" />
902 %             ))}
903 %         </div>
904 %     ) : checkInsByDate && checkInsByDate.length > 0 ? (
905 %         <>
906 %             <div className="flex items-center gap-2">
907 %                 <Badge variant="secondary" className="text-sm">
908 %                     {checkInsByDate.length} check-in(s) registrado(s)
909 %                 </Badge>
910 %             </div>
911 %             <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
912 %                 {checkInsByDate.map((reservation) => (
913 %                     <Card key={reservation.id} data-testid={`history-card-${reservation.id}`}>
914 %                         <CardHeader className="pb-3">
915 %                             <div className="flex items-start justify-between gap-2">
916 %                                 <div className="flex items-center gap-3">
917 %                                     <div className="flex h-10 w-10 items-center justify-center rounded-full bg-green-500/10 tex...
918 %                                         {reservation.guest?.firstName?.[0]}{reservation.guest?.lastName?.[0]}
919 %                                     </div>
920 %                                     <div>
921 %                                         <CardTitle className="text-lg">
922 %                                             {reservation.guest?.firstName} {reservation.guest?.lastName}
923 %                                         </CardTitle>
924 %                                         <CardDescription>{reservation.reservationCode}</CardDescription>
925 %                                     </div>
926 %                                 </div>
927 %                             </div>
928 %                         </div>
929 %                         <Badge variant="default" className="bg-green-500">Check-in</Badge>

```

client/src/pages/check-in.tsx (cont.)

```
928 %         </div>
929 %       </CardHeader>
930 %     <CardContent className="space-y-3">
931 %       <div className="grid grid-cols-2 gap-3">
932 %         <div className="flex items-center gap-2 text-sm">
933 %           <DoorOpen className="h-4 w-4 text-muted-foreground" />
934 %           <span>Hab. {reservation.room?.roomNumber}</span>
935 %         </div>
936 %         <div className="flex items-center gap-2 text-sm">
937 %           <User className="h-4 w-4 text-muted-foreground" />
938 %           <span>{reservation.numberOfGuests} huesped(es)</span>
939 %         </div>
940 %       </div>
941 %       <div className="grid grid-cols-2 gap-3">
942 %         <div className="flex items-center gap-2 text-sm">
943 %           <Calendar className="h-4 w-4 text-muted-foreground" />
944 %           <span>Entrada: {reservation.checkInDate}</span>
945 %         </div>
946 %         <div className="flex items-center gap-2 text-sm">
947 %           <Clock className="h-4 w-4 text-muted-foreground" />
948 %           <span>Salida: {reservation.checkOutDate}</span>
949 %         </div>
950 %       </div>
951 %     </CardContent>
952 %   </Card>
953 %   )))
954 % </div>
955 % </>
956 % ) : (
957 %   <Card>
958 %     <CardContent className="flex flex-col items-center justify-center py-12 text-center">
959 %       <CalendarDays className="h-16 w-16 text-muted-foreground/50 mb-4" />
960 %       <h3 className="text-lg font-semibold mb-2">No hay check-ins para esta fecha</h3>
961 %       <p className="text-muted-foreground">
962 %         No se encontraron registros de check-in para el {new Date(historyDate + "T12:00:00").toLocaleDateSt...
963 %         weekday: "long",
964 %         day: "numeric",
965 %         month: "long",
966 %         year: "numeric"
967 %       </p>
968 %     </CardContent>
969 %   </Card>
970 % )
971 % </TabsContent>
972 % </Tabs>
973 %
974 %
975 % <Dialog open={webCheckinDialogOpen} onOpenChange={setWebCheckinDialogOpen}>
976 %   <DialogContent>
977 %     <DialogHeader>
978 %       <DialogTitle className="flex items-center gap-2">
979 %         <Smartphone className="h-5 w-5 text-green-600" />
980 %         Web Check-in
981 %       </DialogTitle>
982 %     </DialogHeader>
983 %     {webCheckinReservation && (
984 %       <div className="space-y-4">
985 %         <div className="bg-muted/50 rounded-lg p-3 text-sm">
986 %           <p className="font-medium">{webCheckinReservation.guest?.firstName} {webCheckinReservation.guest?.las...
987 %           <p className="text-muted-foreground">
988 %             Hab. {webCheckinReservation.room?.roomNumber} | {webCheckinReservation.checkInDate} - {webCheckinRe...
989 %           </p>
990 %         </div>
991 %         {generateLinkMutation.isSuccess ? (
992 %           <div className="space-y-3 text-center py-4">
993 %             <AlertCircle className="h-10 w-10 text-destructive mx-auto" />
994 %             <p className="text-sm text-destructive">No se pudo generar el enlace</p>
995 %             <Button
996 %               variant="outline"
997 %               onClick={() => generateLinkMutation.mutate(webCheckinReservation!.id)}
998 %               data-testid="button-retry-generate"
999 %             >
1000 %               Reintentar
1001 %             </Button>
1002 %           </div>
1003 %         ) : generatedLink ? (
1004 %           <div className="space-y-3">
1005 %             <Label>Enlace generado</Label>
1006 %             <div className="flex gap-2">
1007 %               <Input value={generatedLink} readOnly className="text-xs" data-testid="input-generated-link" />
1008 %               <Button
1009 %                 variant="outline"
1010 %                 size="icon"
1011 %                 onClick={() => {
1012 %                   navigator.clipboard.writeText(generatedLink);
1013 %                   toast({ title: "Copiado", description: "Link copiado al portapapeles" });
1014 %                 }}
1015 %                 data-testid="button-copy-link"
1016 %               >
1017 %                 <Copy className="h-4 w-4" />
1018 %               </Button>
1019 %             </div>
1020 %           </div>
1021 %         )
1022 %       </div>
1023 %     )
1024 %   </DialogContent>
1025 % </Dialog>
1026 % </>
```

client/src/pages/check-in.tsx (cont.)

```
1021 %         {webCheckinReservation.guest?.phone && (
1022 %             <Button
1023 %                 className="flex-1 bg-green-600 hover:bg-green-700 text-white"
1024 %                 onClick={() => {
1025 %                     const phone = webCheckinReservation.guest?.phone?.replace(/\\D/g, "") || "";
1026 %                     const msg = encodeURIComponent(
1027 %                         `Hola ${webCheckinReservation.guest?.firstName}! Desde Maran Suites & Towers te invitamos...
1028 %                     );
1029 %                     window.open(`https://wa.me/${phone}?text=${msg}`, "_blank");
1030 %                 }}
1031 %                 data-testid="button-whatsapp"
1032 %             >
1033 %                 <ExternalLink className="h-4 w-4 mr-2" />
1034 %                 Abrir WhatsApp
1035 %             </Button>
1036 %         )}
1037 %         <Button
1038 %             variant="outline"
1039 %             className="flex-1"
1040 %             onClick={() => window.open(generatedLink, "_blank")}
1041 %             data-testid="button-open-link"
1042 %         >
1043 %             <ExternalLink className="h-4 w-4 mr-2" />
1044 %             Abrir Link
1045 %         </Button>
1046 %     </div>
1047 % </div>
1048 % ) : (
1049 %     <div className="flex items-center justify-center py-6">
1050 %         <div className="animate-spin h-6 w-6 border-2 border-primary border-t-transparent rounded-full" />
1051 %     </div>
1052 % )}
1053 % </div>
1054 % )}
1055 % </DialogContent>
1056 % </Dialog>
1057 %
1058 % <AlertDialog open={confirmDialogOpen} onOpenChange={setConfirmDialogOpen}>
1059 %     <AlertDialogContent>
1060 %         <AlertDialogHeader>
1061 %             <AlertDialogTitle>Confirmar Check-in</AlertDialogTitle>
1062 %             <AlertDialogDescription asChild>
1063 %                 <div>
1064 %                     {selectedReservation && (
1065 %                         <>
1066 %                             <p className="mb-2">
1067 %                                 Vas a registrar la llegada de{" "}
1068 %                                 <strong>
1069 %                                     {selectedReservation.guest?.firstName} {selectedReservation.guest?.lastName}
1070 %                                 </strong>{" "}
1071 %                                 a la habitacion <strong>{selectedReservation.room?.roomNumber}</strong>.
1072 %                             </p>
1073 %                             <p className="mb-4 text-muted-foreground">Esto marcara la habitacion como ocupada.</p>
1074 %                         </>
1075 %                     )}
1076 %                     <div className={`mb-4 p-3 rounded-lg border ${criticalPrefs.length > 0 ? "border-red-300 bg-red-50" : ""}`>
1077 %                         <div className="flex items-center gap-2 mb-2">
1078 %                             {criticalPrefs.length > 0 ? (
1079 %                                 <AlertTriangle className="h-4 w-4 text-red-500" />
1080 %                             ) : (
1081 %                                 <Heart className="h-4 w-4 text-orange-500" />
1082 %                             )}
1083 %                             <span className="font-medium text-sm text-foreground">
1084 %                                 Preferencias del huésped ({activePrefs.length})
1085 %                             </span>
1086 %                         </div>
1087 %                         <div className="space-y-1.5">
1088 %                             {activePrefs.map((pref) => (
1089 %                                 <div key={pref.id} className="flex items-center gap-2 text-sm">
1090 %                                     <Badge
1091 %                                         variant="outline"
1092 %                                         className={`text-xs ${
1093 %                                             pref.priority === "critical" ? "border-red-400 text-red-700 dark:text-red-300" :
1094 %                                             pref.priority === "high" ? "border-orange-400 text-orange-700 dark:text-orange-300" :
1095 %                                             ""
1096 %                                         }`}
1097 %                                     >
1098 %                                         {pref.priority === "critical" ? "Crítica" : pref.priority === "high" ? "Alta" : pref.priority === "normal" ? "Normal" : ""}
1099 %                                     </Badge>
1100 %                                     <span className="text-foreground">{pref.title}</span>
1101 %                                 </div>
1102 %                             )]}
1103 %                         </div>
1104 %                     </div>
1105 %                 </div>
1106 %             </AlertDialogDescription>
1107 %         </AlertDialogHeader>
1108 %         <AlertDialogContent>
1109 %             <Label htmlFor="checkInNotes" className="text-foreground">Comentarios / Notas</Label>
1110 %             <Textarea
1111 %                 id="checkInNotes"
1112 %                 placeholder="Agregar comentarios para recepción..."
1113 %                 value={checkInNotes}
1114 %                 onChange={(e) => setCheckInNotes(e.target.value)}
1115 %             />
1116 %         </AlertDialogContent>
1117 %     </AlertDialog>
1118 % </div>
```

client/src/pages/check-in.tsx (cont.)

```
1114 %             className="min-h-[60px]"
1115 %             data-testid="input-checkin-notes"
1116 %         />
1117 %     </div>
1118 % </>
1119 %     })
1120 % </div>
1121 % </AlertDialogDescription>
1122 % </AlertDialogHeader>
1123 % <AlertDialogFooter className="gap-2">
1124 %   <AlertDialogCancel data-testid="button-cancel-checkin">Volver</AlertDialogCancel>
1125 %   <AlertDialogAction
1126 %     onClick={performCheckIn}
1127 %     disabled={checkInMutation.isPending}
1128 %     data-testid="button-confirm-checkin"
1129 %   >
1130 %     {checkInMutation.isPending ? "Procesando..." : "Confirmar Check-in"}
1131 %   </AlertDialogAction>
1132 % </AlertDialogFooter>
1133 % </AlertDialogContent>
1134 % </AlertDialog>
1135 % </div>
1136 % );
1137 % }
1138 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { useLocation } from "wouter";
4 % import {
5 %   Logout,
6 %   Search,
7 %   Calendar,
8 %   User,
9 %   DoorOpen,
10 %   Check,
11 %   Clock,
12 %   CreditCard,
13 %   Plus,
14 %   ChevronRight,
15 %   ChevronLeft,
16 %   CircleCheck,
17 %   Trash2,
18 %   ArrowLeft,
19 % } from "lucide-react";
20 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@/components/ui/card";
21 % import { Button } from "@/components/ui/button";
22 % import { Input } from "@/components/ui/input";
23 % import { Badge } from "@/components/ui/badge";
24 % import { Skeleton } from "@/components/ui/skeleton";
25 % import { Label } from "@/components/ui/label";
26 % import {
27 %   Select,
28 %   SelectContent,
29 %   SelectItem,
30 %   SelectTrigger,
31 %   SelectValue,
32 % } from "@/components/ui/select";
33 % import {
34 %   Dialog,
35 %   DialogContent,
36 %   DialogDescription,
37 %   DialogFooter,
38 %   DialogHeader,
39 %   DialogTitle,
40 % } from "@/components/ui/dialog";
41 % import {
42 %   Table,
43 %   TableBody,
44 %   TableCell,
45 %   TableHead,
46 %   TableHeader,
47 %   TableRow,
48 % } from "@/components/ui/table";
49 % import { useToast } from "@/hooks/use-toast";
50 % import { queryClient, apiRequest } from "@/lib/queryClient";
51 % import type { ReservationWithDetails, Charge, Payment, PaymentMethod } from "@shared/schema";
52 %
53 % const paymentMethodLabels: Record<PaymentMethod, string> = {
54 %   efectivo: "Efectivo",
55 %   tarjeta_debito: "Tarjeta Débito",
56 %   tarjeta_credito: "Tarjeta Crédito",
57 %   transferencia: "Transferencia",
58 %   mercadopago: "MercadoPago",
59 %   cuenta_corriente: "Cuenta Corriente",
60 % };
61 %
62 % interface FolioData {
63 %   reservationCode: string;
64 %   guestName: string;
65 %   roomNumber: string;
66 %   checkInDate: string;
67 %   checkOutDate: string;
68 %   nights: number;
69 %   roomRate: string;
70 %   roomTotal: number;
71 %   charges: Charge[];
72 %   totalCharges: number;
73 %   payments: Payment[];
74 %   totalPayments: number;
75 %   grandTotal: number;
76 %   balance: number;
77 % }
78 %
79 % export default function CheckOutPage() {
80 %   const { toast } = useToast();
81 %   const [, setLocation] = useLocation();
82 %   const [searchQuery, setSearchQuery] = useState("");
83 %   const [selectedReservation, setSelectedReservation] = useState<ReservationWithDetails | null>(null);
84 %   const [wizardStep, setWizardStep] = useState(0);
85 %   const [addChargeOpen, setAddChargeOpen] = useState(false);
86 %   const [newChargeDesc, setNewChargeDesc] = useState("");
87 %   const [newChargeAmount, setNewChargeAmount] = useState("");
88 %   const [paymentAmount, setPaymentAmount] = useState("");
89 %   const [paymentMethod, setPaymentMethod] = useState<PaymentMethod>("efectivo");
90 %   const [paymentReference, setPaymentReference] = useState("");
```


client/src/pages/check-out.tsx (cont.)

```
91 %   const [checkoutComplete, setCheckoutComplete] = useState(false);
92 %   const [finalSummary, setFinalSummary] = useState<{ guestName: string; roomNumber: string; checkOutDate: string; tot...
93 %
94 %   const { data: reservations, isLoading } = useQuery<ReservationWithDetails[]>({
95 %     queryKey: ["/api/reservations/check-out"],
96 %   });
97 %
98 %   const { data: folio, refetch: refetchFolio } = useQuery<FolioData>({
99 %     queryKey: ["/api/reservations", selectedReservation?.id, "folio"],
100 %    queryFn: async () => {
101 %      const res = await fetch(`/api/reservations/${selectedReservation!.id}/folio`);
102 %      return res.json();
103 %    },
104 %    enabled: !!selectedReservation,
105 %  });
106 %
107 %  const addChargeMutation = useMutation({
108 %    mutationFn: async (data: { description: string; amount: string }) => {
109 %      return apiRequest("POST", "/api/charges", {
110 %        reservationId: selectedReservation!.id,
111 %        description: data.description,
112 %        amount: data.amount,
113 %        date: new Date().toISOString().split("T")[0],
114 %        category: "otros",
115 %      });
116 %    },
117 %    onSuccess: () => {
118 %      refetchFolio();
119 %      setAddChargeOpen(false);
120 %      setNewChargeDesc("");
121 %      setNewChargeAmount("");
122 %      toast({ title: "Cargo agregado" });
123 %    },
124 %  });
125 %
126 %  const addPaymentMutation = useMutation({
127 %    mutationFn: async (data: { amount: string; method: PaymentMethod; reference: string }) => {
128 %      return apiRequest("POST", "/api/payments", {
129 %        reservationId: selectedReservation!.id,
130 %        amount: data.amount,
131 %        method: data.method,
132 %        date: new Date().toISOString().split("T")[0],
133 %        reference: data.reference || null,
134 %      });
135 %    },
136 %    onSuccess: () => {
137 %      refetchFolio();
138 %      setPaymentAmount("");
139 %      setPaymentReference("");
140 %      toast({ title: "Pago registrado" });
141 %    },
142 %  });
143 %
144 %  const checkOutMutation = useMutation({
145 %    mutationFn: async (id: string) => {
146 %      return apiRequest("POST", `/api/reservations/${id}/check-out`, {});
147 %    },
148 %    onSuccess: () => {
149 %      queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
150 %      queryClient.invalidateQueries({ queryKey: ["/api/reservations/check-out"] });
151 %      queryClient.invalidateQueries({ queryKey: ["/api/reservations/recent"] });
152 %      queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
153 %      queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
154 %      queryClient.invalidateQueries({ queryKey: ["/api/planning"] });
155 %      queryClient.invalidateQueries({ queryKey: ["/api/housekeeping"] });
156 %
157 %      const methods = folio?.payments?.map(p => paymentMethodLabels[p.method as PaymentMethod] || p.method) || [];
158 %      setFinalSummary({
159 %        guestName: `${selectedReservation?.guest?.firstName} ${selectedReservation?.guest?.lastName}`,
160 %        roomNumber: selectedReservation?.room?.roomNumber || "",
161 %        checkOutDate: new Date().toLocaleDateString("es-AR"),
162 %        totalPaid: folio?.totalPayments || 0,
163 %        methods: [...new Set(methods)],
164 %      });
165 %      setCheckoutComplete(true);
166 %      setWizardStep(3);
167 %    },
168 %    onError: async (error: any) => {
169 %      let message = "No se pudo realizar el check-out. Intente nuevamente.";
170 %      try {
171 %        if (error?.message) message = error.message;
172 %      } catch {}
173 %      toast({
174 %        title: "Error",
175 %        description: message,
176 %        variant: "destructive",
177 %      });
178 %    },
179 %  });
180 %
181 %  const filteredReservations = reservations?.filter((res) => {
182 %    const guestName = `${res.guest?.firstName} ${res.guest?.lastName}`.toLowerCase();
183 %    return (
```

client/src/pages/check-out.tsx (cont.)

```
184 %         guestName.includes(searchQuery.toLowerCase()) ||
185 %         res.room?.roomNumber?.toLowerCase().includes(searchQuery.toLowerCase())
186 %     );
187 % });
188 %
189 % const startCheckout = (reservation: ReservationWithDetails) => {
190 %     setSelectedReservation(reservation);
191 %     setWizardStep(1);
192 %     setCheckoutComplete(false);
193 %     setFinalSummary(null);
194 % };
195 %
196 % const cancelWizard = () => {
197 %     setSelectedReservation(null);
198 %     setWizardStep(0);
199 %     setCheckoutComplete(false);
200 %     setFinalSummary(null);
201 % };
202 %
203 % const today = new Date().toLocaleDateString("es-ES", {
204 %     weekday: "long",
205 %     year: "numeric",
206 %     month: "long",
207 %     day: "numeric",
208 % });
209 %
210 % const balance = folio?.balance || 0;
211 %
212 % if (wizardStep > 0 && selectedReservation) {
213 %     return (
214 %         <div className="flex flex-col gap-6 p-6">
215 %             <div className="flex items-center gap-4">
216 %                 <Button variant="ghost" size="icon" onClick={cancelWizard} data-testid="button-cancel-wizard">
217 %                     <ArrowLeft className="h-5 w-5" />
218 %                 </Button>
219 %                 <div>
220 %                     <h1 className="text-2xl font-bold tracking-tight" data-testid="text-wizard-title">
221 %                         Check-out - {selectedReservation.guest?.firstName} {selectedReservation.guest?.lastName}
222 %                     </h1>
223 %                     <p className="text-muted-foreground">Hab. {selectedReservation.room?.roomNumber}</p>
224 %                 </div>
225 %             </div>
226 %
227 %             <div className="flex items-center gap-2 mb-2">
228 %                 {[1, 2, 3].map((step) => (
229 %                     <div key={step} className="flex items-center gap-2">
230 %                         <div>
231 %                             className={`flex h-8 w-8 items-center justify-center rounded-full text-sm font-semibold ${
232 %                                 wizardStep === step
233 %                                     ? "bg-primary text-primary-foreground"
234 %                                     : wizardStep > step
235 %                                     ? "bg-green-500 text-white"
236 %                                     : "bg-muted text-muted-foreground"
237 %                             }`}
238 %                             data-testid={`step-indicator-${step}`}
239 %                         >
240 %                             {wizardStep > step ? <Check className="h-4 w-4" /> : step}
241 %                         </div>
242 %                         <span className={`text-sm ${wizardStep === step ? "font-semibold" : "text-muted-foreground"}`} >
243 %                             {step === 1 ? "Resumen" : step === 2 ? "Pago" : "Confirmación"}
244 %                         </span>
245 %                         {step < 3 && <ChevronRight className="h-4 w-4 text-muted-foreground" />}
246 %                     </div>
247 %                 )))
248 %             </div>
249 %
250 %             {wizardStep === 1 && (
251 %                 <div className="grid gap-4">
252 %                     <Card>
253 %                         <CardHeader className="pb-3">
254 %                             <div className="flex items-center justify-between">
255 %                                 <CardTitle className="text-base">Cargos</CardTitle>
256 %                                 <Button
257 %                                     variant="outline"
258 %                                     size="sm"
259 %                                     onClick={() => setAddChargeOpen(true)}
260 %                                     data-testid="button-add-charge"
261 %                                 >
262 %                                     <Plus className="h-4 w-4 mr-1" /> Agregar cargo
263 %                                 </Button>
264 %                             </div>
265 %                         </CardHeader>
266 %                         <CardContent>
267 %                             <Table>
268 %                                 <TableHeader>
269 %                                     <TableRow>
270 %                                         <TableHead>Descripción</TableHead>
271 %                                         <TableHead>Fecha</TableHead>
272 %                                         <TableHead className="text-right">Monto</TableHead>
273 %                                     </TableRow>
274 %                                 </TableHeader>
275 %                                 <TableBody>
276 %                                     <TableRow className="bg-muted/50">
```

client/src/pages/check-out.tsx (cont.)

```
277 %         <TableCell className="font-medium">Alojamiento {{folio?.nights || selectedReservation.nights} n...
278 %         <TableCell>{{selectedReservation.checkInDate}} ' {{selectedReservation.checkOutDate}}</TableCell>
279 %         <TableCell className="text-right font-medium">${{folio?.roomTotal || parseFloat(selectedReserva...
280 %     </TableRow>
281 %     {{folio?.charges?.map((charge) => (
282 %         <TableRow key={charge.id} data-testid={`charge-row-${charge.id}`}>
283 %             <TableCell>{{charge.description}}</TableCell>
284 %             <TableCell>{{charge.date}}</TableCell>
285 %             <TableCell className="text-right">${parseFloat(charge.amount).toFixed(2)}</TableCell>
286 %         </TableRow>
287 %     )}}
288 % </TableBody>
289 % </Table>
290 % <div className="flex justify-end mt-3 pt-3 border-t">
291 %     <div className="text-right">
292 %         <p className="text-sm text-muted-foreground">Total cargos</p>
293 %         <p className="text-lg font-bold" data-testid="text-total-charges">${(folio?.grandTotal || 0).toFi...
294 %     </div>
295 % </div>
296 % </CardContent>
297 % </Card>
298 %
299 % <Card>
300 %     <CardHeader className="pb-3">
301 %         <CardTitle className="text-base">Pagos registrados</CardTitle>
302 %     </CardHeader>
303 %     <CardContent>
304 %         {{folio?.payments && folio.payments.length > 0 ? (
305 %             <Table>
306 %                 <TableHeader>
307 %                     <TableRow>
308 %                         <TableHead>Fecha</TableHead>
309 %                         <TableHead>Método</TableHead>
310 %                         <TableHead className="text-right">Monto</TableHead>
311 %                     </TableRow>
312 %                 </TableHeader>
313 %                 <TableBody>
314 %                     {{folio.payments.map((payment) => (
315 %                         <TableRow key={payment.id} data-testid={`payment-row-${payment.id}`}>
316 %                             <TableCell>{{payment.date}}</TableCell>
317 %                             <TableCell>{{paymentMethodLabels[payment.method as PaymentMethod] || payment.method}}</TableC...
318 %                             <TableCell className="text-right">${parseFloat(payment.amount).toFixed(2)}</TableCell>
319 %                         </TableRow>
320 %                     )}}
321 %                 </TableBody>
322 %             </Table>
323 %         ) : (
324 %             <p className="text-sm text-muted-foreground text-center py-4">Sin pagos registrados</p>
325 %         )}
326 %         <div className="flex justify-end mt-3 pt-3 border-t">
327 %             <div className="text-right">
328 %                 <p className="text-sm text-muted-foreground">Total pagado</p>
329 %                 <p className="text-lg font-bold" data-testid="text-total-payments">${(folio?.totalPayments || 0)...
330 %             </div>
331 %         </div>
332 %     </CardContent>
333 % </Card>
334 %
335 % <Card className={{balance > 0.01 ? "border-red-300 bg-red-50 dark:bg-red-900/10 dark:border-red-800" : "bo...
336 %     <CardContent className="flex items-center justify-between p-4">
337 %         <div>
338 %             <p className="text-sm font-medium">Saldo pendiente</p>
339 %             <p className={{text-2xl font-bold ${balance > 0.01 ? "text-red-600 dark:text-red-400" : "text-green...
340 %                 ${balance.toFixed(2)}}
341 %             </p>
342 %         </div>
343 %         {{balance <= 0.01 && <CircleCheck className="h-8 w-8 text-green-500" />}}
344 %     </CardContent>
345 % </Card>
346 %
347 % <div className="flex justify-end">
348 %     <Button onClick={() => setWizardStep(2)} data-testid="button-continue-to-payment">
349 %         Continuar al pago <ChevronRight className="h-4 w-4 ml-1" />
350 %     </Button>
351 % </div>
352 %
353 % <Dialog open={{addChargeOpen}} onOpenChange={{setAddChargeOpen}}>
354 %     <DialogContent>
355 %         <DialogHeader>
356 %             <DialogTitle>Agregar cargo</DialogTitle>
357 %             <DialogDescription>Agregar un cargo de último momento al folio.</DialogDescription>
358 %         </DialogHeader>
359 %         <div className="grid gap-4 py-4">
360 %             <div>
361 %                 <Label>Descripción</Label>
362 %                 <Input
363 %                     value={{newChargeDesc}}
364 %                     onChange={{(e) => setNewChargeDesc(e.target.value)}}
365 %                     placeholder="Ej: Minibar, Lavandería..."
366 %                     data-testid="input-charge-description"
367 %                 />
368 %             </div>
369 %         </div>
```

client/src/pages/check-out.tsx (cont.)

```
370 %         <Label>Monto</Label>
371 %         <Input
372 %             type="number"
373 %             step="0.01"
374 %             min="0"
375 %             value={newChargeAmount}
376 %             onChange={(e) => setNewChargeAmount(e.target.value)}
377 %             placeholder="0.00"
378 %             data-testid="input-charge-amount"
379 %         />
380 %     </div>
381 </div>
382 <DialogFooter>
383 %     <Button variant="outline" onClick={() => setAddChargeOpen(false)}>Cancelar</Button>
384 %     <Button
385 %         onClick={() => addChargeMutation.mutate({ description: newChargeDesc, amount: newChargeAmount })}
386 %         disabled={!newChargeDesc || !newChargeAmount || addChargeMutation.isPending}
387 %         data-testid="button-confirm-charge"
388 %     >
389 %         {addChargeMutation.isPending ? "Guardando..." : "Agregar"}
390 %     </Button>
391 </DialogFooter>
392 </DialogContent>
393 </Dialog>
394 </div>
395 }}
396
397 {wizardStep === 2 && (
398     <div className="grid gap-4">
399         {balance <= 0.01 ? (
400             <Card className="border-green-300 bg-green-50 dark:bg-green-900/10 dark:border-green-800">
401                 <CardContent className="flex flex-col items-center py-8 gap-3">
402                     <CircleCheck className="h-12 w-12 text-green-500" />
403                     <h3 className="text-lg font-semibold text-green-700 dark:text-green-400" data-testid="text-account-...">
404                         <p className="text-sm text-muted-foreground">El huésped no tiene saldo pendiente.</p>
405                     </CardContent>
406                 </Card>
407             ) : (
408                 <Card>
409                     <CardHeader className="pb-3">
410                         <CardTitle className="text-base">Registrar pago</CardTitle>
411                         <CardDescription>
412                             Saldo pendiente: <span className="font-bold text-red-600 dark:text-red-400">${balance.toFixed(2...)}</span>
413                         </CardDescription>
414                     </CardHeader>
415                     <CardContent className="grid gap-4">
416                         <div className="grid grid-cols-2 gap-4">
417                             <div>
418                                 <Label>Monto</Label>
419                                 <Input
420                                     type="number"
421                                     step="0.01"
422                                     min="0"
423                                     value={paymentAmount}
424                                     onChange={(e) => setPaymentAmount(e.target.value)}
425                                     placeholder={balance.toFixed(2)}
426                                     data-testid="input-payment-amount"
427                                 />
428                             </div>
429                         </div>
430                         <div>
431                             <Label>Método de pago</Label>
432                             <Select value={paymentMethod} onChange={(v) => setPaymentMethod(v as PaymentMethod)}>
433                                 <SelectTrigger data-testid="select-payment-method">
434                                     <SelectValue />
435                                 </SelectTrigger>
436                                 <SelectContent>
437                                     {Object.entries(paymentMethodLabels).map(([value, label]) => (
438                                         <SelectItem key={value} value={value}>{label}</SelectItem>
439                                     ))}
440                                 </SelectContent>
441                             </Select>
442                         </div>
443                     </div>
444                     <div>
445                         <Label>Referencia (opcional)</Label>
446                         <Input
447                             value={paymentReference}
448                             onChange={(e) => setPaymentReference(e.target.value)}
449                             placeholder="N° de comprobante, autorización..."
450                             data-testid="input-payment-reference"
451                         />
452                     </div>
453                     <Button
454                         onClick={() => addPaymentMutation.mutate({
455                             amount: paymentAmount || balance.toFixed(2),
456                             method: paymentMethod,
457                             reference: paymentReference,
458                         })}
459                         disabled={addPaymentMutation.isPending}
460                         data-testid="button-register-payment"
461                     >
462                         {addPaymentMutation.isPending ? "Registrando..." : "Registrar pago"}
```

client/src/pages/check-out.tsx (cont.)

```
463 %         </Button>
464 %       </CardContent>
465 %     </Card>
466 %
467 %     {folio?.payments && folio.payments.length > 0 && (
468 %       <Card>
469 %         <CardHeader className="pb-3">
470 %           <CardTitle className="text-base">Pagos registrados</CardTitle>
471 %         </CardHeader>
472 %         <CardContent>
473 %           <Table>
474 %             <TableHeader>
475 %               <TableRow>
476 %                 <TableHead>Método</TableHead>
477 %                 <TableHead className="text-right">Monto</TableHead>
478 %               </TableRow>
479 %             </TableHeader>
480 %             <TableBody>
481 %               {folio.payments.map((p) => (
482 %                 <TableRow key={p.id}>
483 %                   <TableCell>{paymentMethodLabels[p.method as PaymentMethod] || p.method}</TableCell>
484 %                   <TableCell className="text-right">${parseFloat(p.amount).toFixed(2)}</TableCell>
485 %                 </TableRow>
486 %               )
487 %             </TableBody>
488 %           </Table>
489 %         </CardContent>
490 %       </Card>
491 %     )}
492 %
493 %     <Card className="border-red-300 bg-red-50 dark:bg-red-900/10 dark:border-red-800">
494 %       <CardContent className="flex items-center justify-between p-4">
495 %         <p className="font-medium">Saldo restante</p>
496 %         <p className="text-xl font-bold text-red-600 dark:text-red-400" data-testid="text-remaining-balan...
497 %       </CardContent>
498 %     </Card>
499 %   </>
500 % )}
501 %
502 %   <div className="flex justify-between">
503 %     <Button variant="outline" onClick={() => setWizardStep(1)} data-testid="button-back-to-summary">
504 %       <ChevronLeft className="h-4 w-4 mr-1" /> Volver al resumen
505 %     </Button>
506 %     <Button
507 %       onClick={() => selectedReservation && checkOutMutation.mutate(selectedReservation.id)}
508 %       disabled={balance > 0.01 || checkOutMutation.isPending}
509 %       data-testid="button-confirm-checkout"
510 %     >
511 %       {checkOutMutation.isPending ? "Procesando..." : "Confirmar Check-out"}
512 %     </Button>
513 %   </div>
514 % </div>
515 % )}
516 %
517 % {wizardStep === 3 && checkoutComplete && finalSummary && (
518 %   <div className="grid gap-4">
519 %     <Card className="border-green-300 bg-green-50 dark:bg-green-900/10 dark:border-green-800">
520 %       <CardContent className="flex flex-col items-center py-8 gap-4">
521 %         <div className="flex h-16 w-16 items-center justify-center rounded-full bg-green-100 dark:bg-green-90...
522 %           <CircleCheck className="h-10 w-10 text-green-600 dark:text-green-400" />
523 %         </div>
524 %         <h2 className="text-xl font-bold text-green-700 dark:text-green-400" data-testid="text-checkout-compl...
525 %           Check-out completado
526 %         </h2>
527 %       </CardContent>
528 %     </Card>
529 %
530 %     <Card>
531 %       <CardHeader className="pb-3">
532 %         <CardTitle className="text-base">Resumen final</CardTitle>
533 %       </CardHeader>
534 %       <CardContent className="grid gap-3">
535 %         <div className="grid grid-cols-2 gap-4">
536 %           <div>
537 %             <p className="text-sm text-muted-foreground">Huésped</p>
538 %             <p className="font-medium" data-testid="text-summary-guest">{finalSummary.guestName}</p>
539 %           </div>
540 %           <div>
541 %             <p className="text-sm text-muted-foreground">Habitación</p>
542 %             <p className="font-medium" data-testid="text-summary-room">{finalSummary.roomNumber}</p>
543 %           </div>
544 %           <div>
545 %             <p className="text-sm text-muted-foreground">Fecha de salida</p>
546 %             <p className="font-medium" data-testid="text-summary-date">{finalSummary.checkOutDate}</p>
547 %           </div>
548 %           <div>
549 %             <p className="text-sm text-muted-foreground">Total cobrado</p>
550 %             <p className="font-medium" data-testid="text-summary-total">${finalSummary.totalPaid.toFixed(2)}</p>
551 %           </div>
552 %         </div>
553 %         {finalSummary.methods.length > 0 && (
554 %           <div>
555 %             <p className="text-sm text-muted-foreground">Método(s) de pago</p>
```

```

556 %         <div className="flex gap-2 mt-1">
557 %             {finalSummary.methods.map((m) => (
558 %                 <Badge key={m} variant="secondary">{m}</Badge>
559 %             ))}
560 %         </div>
561 %     </div>
562 % })
563 % </CardContent>
564 % </Card>
565 %
566 % <Card className="bg-blue-50 border-blue-200 dark:bg-blue-900/10 dark:border-blue-800">
567 %     <CardContent className="flex items-center gap-3 p-4">
568 %         <DoorOpen className="h-5 w-5 text-blue-600 dark:text-blue-400" />
569 %         <p className="text-sm" data-testid="text-housekeeping-notice">
570 %             La habitación <strong>{finalSummary.roomNumber}</strong> fue enviada a Housekeeping para limpieza.
571 %         </p>
572 %     </CardContent>
573 % </Card>
574 %
575 %     <div className="flex justify-center">
576 %         <Button onClick={() => setLocation("/planning")} data-testid="button-back-to-planning">
577 %             Volver al Planning
578 %         </Button>
579 %     </div>
580 % </div>
581 % })
582 % </div>
583 % );
584 % }
585 %
586 % return (
587 %     <div className="flex flex-col gap-6 p-6">
588 %         <div className="flex flex-col gap-1">
589 %             <h1 className="text-3xl font-bold tracking-tight" data-testid="text-checkout-title">
590 %                 Check-out
591 %             </h1>
592 %             <p className="text-muted-foreground capitalize">{today}</p>
593 %         </div>
594 %
595 %         <Card className="bg-orange-50 border-orange-200 dark:bg-orange-900/20 dark:border-orange-800">
596 %             <CardContent className="flex items-center gap-4 p-4">
597 %                 <div className="flex h-12 w-12 items-center justify-center rounded-full bg-orange-100 dark:bg-orange-900/40">
598 %                     <Logout className="h-6 w-6 text-orange-600 dark:text-orange-400" />
599 %                 </div>
600 %                 <div>
601 %                     <h3 className="font-semibold">Registro de Salidas</h3>
602 %                     <p className="text-sm text-muted-foreground">
603 %                         Huéspedes activos listos para check-out
604 %                     </p>
605 %                 </div>
606 %                 <Badge className="ml-auto bg-orange-100 text-orange-800 dark:bg-orange-900/40 dark:text-orange-400" data-te...
607 %                     {reservations?.length || 0} activos
608 %                 </Badge>
609 %             </CardContent>
610 %         </Card>
611 %
612 %         <Card>
613 %             <CardContent className="p-4">
614 %                 <div className="relative">
615 %                     <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
616 %                     <Input
617 %                         placeholder="Buscar por nombre de huésped o número de habitación..."
618 %                         value={searchQuery}
619 %                         onChange={(e) => setSearchQuery(e.target.value)}
620 %                         className="pl-10"
621 %                         data-testid="input-search-checkout"
622 %                     />
623 %                 </div>
624 %             </CardContent>
625 %         </Card>
626 %
627 %         {isLoading ? (
628 %             <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
629 %                 {[...Array(3)].map((_, i) => (
630 %                     <Skeleton key={i} className="h-48" />
631 %                 ))}
632 %             </div>
633 %         ) : filteredReservations && filteredReservations.length > 0 ? (
634 %             <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
635 %                 {filteredReservations.map((reservation) => (
636 %                     <Card
637 %                         key={reservation.id}
638 %                         className="hover-elevate"
639 %                         data-testid={`checkout-card-${reservation.id}`}
640 %                     >
641 %                         <CardHeader className="pb-3">
642 %                             <div className="flex items-start justify-between gap-2">
643 %                                 <div className="flex items-center gap-3">
644 %                                     <div className="flex h-10 w-10 items-center justify-center rounded-full bg-blue-100 text-blue-600...
645 %                                         {reservation.guest?.firstName?.[0]}{reservation.guest?.lastName?.[0]}
646 %                                     </div>
647 %                                     <div>
648 %                                         <CardTitle className="text-lg">

```

client/src/pages/check-out.tsx (cont.)

```
649 %         {reservation.guest?.firstName} {reservation.guest?.lastName}
650 %       </CardTitle>
651 %       <CardDescription>{reservation.guest?.phone || reservation.guest?.email}</CardDescription>
652 %     </div>
653 %   </div>
654 %   <Badge className="bg-blue-100 text-blue-800 dark:bg-blue-900/40 dark:text-blue-400">
655 %     Alojado
656 %   </Badge>
657 % </div>
658 % </CardHeader>
659 % <CardContent className="space-y-3">
660 %   <div className="grid grid-cols-2 gap-3">
661 %     <div className="flex items-center gap-2 text-sm">
662 %       <DoorOpen className="h-4 w-4 text-muted-foreground" />
663 %       <span>Hab. {reservation.room?.roomNumber}</span>
664 %     </div>
665 %     <div className="flex items-center gap-2 text-sm">
666 %       <User className="h-4 w-4 text-muted-foreground" />
667 %       <span>{reservation.numberOfGuests} huéspedes(es)</span>
668 %     </div>
669 %   </div>
670 %   <div className="grid grid-cols-2 gap-3">
671 %     <div className="flex items-center gap-2 text-sm">
672 %       <Calendar className="h-4 w-4 text-muted-foreground" />
673 %       <span>Desde: {reservation.checkInDate}</span>
674 %     </div>
675 %     <div className="flex items-center gap-2 text-sm">
676 %       <Clock className="h-4 w-4 text-muted-foreground" />
677 %       <span>Hasta: {reservation.checkOutDate}</span>
678 %     </div>
679 %   </div>
680 %   <div className="flex items-center justify-between p-3 bg-muted rounded-lg">
681 %     <div className="flex items-center gap-2">
682 %       <CreditCard className="h-4 w-4 text-muted-foreground" />
683 %       <span className="text-sm">Total habitación</span>
684 %     </div>
685 %     <span className="text-lg font-bold">${parseFloat(reservation.totalRoomAmount || "0").toFixed(2)}</s...
686 %   </div>
687 %   <div className="pt-2">
688 %     <Button
689 %       variant="outline"
690 %       className="w-full border-orange-300 text-orange-700 hover:bg-orange-50 dark:border-orange-700 dar...
691 %       onClick={() => startCheckout(reservation)}
692 %       data-testid={`button-checkout-${reservation.id}`}
693 %     >
694 %       <Logout className="mr-2 h-4 w-4" />
695 %       Realizar Check-out
696 %     </Button>
697 %   </div>
698 % </CardContent>
699 % </Card>
700 %   )}
701 % </div>
702 % ) : (
703 %   <Card>
704 %     <CardContent className="flex flex-col items-center justify-center py-12 text-center">
705 %       <Logout className="h-16 w-16 text-muted-foreground/50 mb-4" />
706 %       <h3 className="text-lg font-semibold mb-2">No hay check-outs pendientes</h3>
707 %       <p className="text-muted-foreground">
708 %         {searchQuery
709 %           ? "No se encontraron huéspedes con los criterios de búsqueda."
710 %           : "No hay huéspedes alojados actualmente."}
711 %       </p>
712 %     </CardContent>
713 %   </Card>
714 %   )}
715 % </div>
716 % );
717 % }
718 %
```

```
1 % import { useState, useCallback, useRef } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { useForm } from "react-hook-form";
5 % import { zodResolver } from "hookform/resolvers/zod";
6 % import { z } from "zod";
7 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
8 % import { Button } from "@components/ui/button";
9 % import { Badge } from "@components/ui/badge";
10 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
11 % import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter, DialogDescription } from "@components/ui/di...
12 % import { Input } from "@components/ui/input";
13 % import { Label } from "@components/ui/label";
14 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
15 % import { Textarea } from "@components/ui/textarea";
16 % import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
17 % import { Switch } from "@components/ui/switch";
18 % import { useToast } from "@hooks/use-toast";
19 % import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@components/ui/table";
20 % import {
21 %   Plus,
22 %   UtensilsCrossed,
23 %   Users,
24 %   CircleDollarSign,
25 %   Clock,
26 %   MapPin,
27 %   Square,
28 %   Circle,
29 %   RectangleHorizontal,
30 %   X,
31 %   CreditCard,
32 %   Loader2,
33 %   CalendarDays,
34 %   Phone,
35 %   Trash2,
36 %   Check,
37 %   XCircle,
38 %   GripVertical,
39 %   Settings,
40 %   Edit,
41 %   Eye,
42 %   Search,
43 %   BookOpen,
44 %   Receipt,
45 %   ChefHat,
46 %   Banknote,
47 %   ArrowUpDown,
48 % } from "lucide-react";
49 %
50 % type RestaurantArea = {
51 %   id: string;
52 %   name: string;
53 %   areaType: "indoor" | "outdoor" | "terrace" | "bar" | "private";
54 %   capacity: number;
55 %   hasTables: string | null;
56 %   isActive: string;
57 %   notes: string | null;
58 % };
59 %
60 % type RestaurantTable = {
61 %   id: string;
62 %   tableNumber: string;
63 %   areaId: string;
64 %   capacity: number;
65 %   shape: "square" | "round" | "rectangular";
66 %   status: "available" | "occupied" | "reserved" | "cleaning" | "blocked";
67 %   positionX: number;
68 %   positionY: number;
69 %   hasWindow: string | null;
70 %   isActive: string;
71 %   area?: RestaurantArea;
72 % };
73 %
74 % type MenuCategory = {
75 %   id: string;
76 %   name: string;
77 %   description: string | null;
78 %   displayOrder: number | null;
79 %   isActive: string | null;
80 % };
81 %
82 % type MenuItem = {
83 %   id: string;
84 %   categoryId: string;
85 %   name: string;
86 %   description: string | null;
87 %   price: string;
88 %   preparationTime: number | null;
89 %   isAvailable: string | null;
90 %   isActive: string | null;
```


client/src/pages/restaurant.tsx (cont.)

```
91 %   allergens: string | null;
92 %   displayOrder: number | null;
93 %   category?: MenuCategory;
94 % };
95 %
96 % type OrderSplit = {
97 %   id: string;
98 %   orderId: string;
99 %   splitNumber: number;
100 %   amount: string;
101 %   method: string | null;
102 %   receiptType: string | null;
103 %   isPaid: string | null;
104 %   paidAt: string | null;
105 %   createdAt: string | null;
106 % };
107 %
108 % type RestaurantOrder = {
109 %   id: string;
110 %   orderNumber: string;
111 %   tableId: string | null;
112 %   areaId: string | null;
113 %   status: "open" | "in_progress" | "served" | "closed" | "cancelled";
114 %   covers: number;
115 %   waiterName: string | null;
116 %   orderLabel: string | null;
117 %   activeCourse: number | null;
118 %   subtotal: string;
119 %   tax: string;
120 %   total: string;
121 %   openedAt: string;
122 %   receiptType: string | null;
123 %   paymentMethod: string | null;
124 %   table?: RestaurantTable;
125 %   area?: RestaurantArea;
126 %   items?: Array<{
127 %     id: string;
128 %     menuItemId: string;
129 %     quantity: number;
130 %     unitPrice: string;
131 %     subtotal: string;
132 %     menuItem?: MenuItem;
133 %     notes?: string | null;
134 %     status?: string;
135 %     course?: number | null;
136 %   }>;
137 % };
138 %
139 % type TableReservation = {
140 %   id: string;
141 %   tableId: string;
142 %   guestName: string;
143 %   guestPhone: string | null;
144 %   guestEmail: string | null;
145 %   partySize: number;
146 %   reservationDate: string;
147 %   reservationTime: string;
148 %   status: "pending" | "confirmed" | "seated" | "completed" | "cancelled" | "no_show";
149 %   notes: string | null;
150 %   createdAt: string;
151 %   table?: RestaurantTable;
152 % };
153 %
154 % type TimeSlot = {
155 %   id: string;
156 %   time: string;
157 %   label: string | null;
158 %   isActive: string | null;
159 %   displayOrder: number | null;
160 % };
161 %
162 % type RecipeIngredient = {
163 %   id: string;
164 %   recipeId: string;
165 %   inventoryItemId: string | null;
166 %   ingredientName: string;
167 %   quantity: string;
168 %   unit: string;
169 %   unitCost: string;
170 % };
171 %
172 % type Recipe = {
173 %   id: string;
174 %   menuItemId: string;
175 %   notes: string | null;
176 %   menuItem?: MenuItem;
177 %   ingredients: RecipeIngredient[];
178 % };
179 %
180 % const reservationStatusColors: Record<string, string> = {
181 %   pending: "bg-yellow-500/20 text-yellow-700 dark:text-yellow-400",
182 %   confirmed: "bg-blue-500/20 text-blue-700 dark:text-blue-400",
183 %   seated: "bg-green-500/20 text-green-700 dark:text-green-400",
```

client/src/pages/restaurant.tsx (cont.)

```
184 %   completed: "bg-gray-500/20 text-gray-700 dark:text-gray-400",
185 %   cancelled: "bg-red-500/20 text-red-700 dark:text-red-400",
186 %   no_show: "bg-orange-500/20 text-orange-700 dark:text-orange-400",
187 % };
188 %
189 % const reservationStatusLabels: Record<string, string> = {
190 %   pending: "Pendiente",
191 %   confirmed: "Confirmada",
192 %   seated: "Sentado",
193 %   completed: "Completada",
194 %   cancelled: "Cancelada",
195 %   no_show: "No se presento",
196 % };
197 %
198 % const reservationFormSchema = z.object({
199 %   tableId: z.string().min(1, "Debe seleccionar una mesa"),
200 %   guestName: z.string().min(1, "El nombre es requerido"),
201 %   guestPhone: z.string().optional(),
202 %   guestEmail: z.string().email("Email invalido").optional().or(z.literal("")),
203 %   partySize: z.coerce.number().min(1, "Minimo 1 persona"),
204 %   reservationDate: z.string().min(1, "La fecha es requerida"),
205 %   reservationTime: z.string().min(1, "La hora es requerida"),
206 %   notes: z.string().optional(),
207 % });
208 %
209 % type ReservationFormValues = z.infer<typeof reservationFormSchema>;
210 %
211 % const tableStatusColors: Record<string, string> = {
212 %   available: "bg-green-500/20 text-green-700 dark:text-green-400 border-green-500/40",
213 %   occupied: "bg-red-500/20 text-red-700 dark:text-red-400 border-red-500/40",
214 %   reserved: "bg-blue-500/20 text-blue-700 dark:text-blue-400 border-blue-500/40",
215 %   cleaning: "bg-yellow-500/20 text-yellow-700 dark:text-yellow-400 border-yellow-500/40",
216 %   blocked: "bg-gray-500/20 text-gray-700 dark:text-gray-400 border-gray-500/40",
217 % };
218 %
219 % const tableStatusLabels: Record<string, string> = {
220 %   available: "Disponible",
221 %   occupied: "Ocupada",
222 %   reserved: "Reservada",
223 %   cleaning: "Limpieza",
224 %   blocked: "Bloqueada",
225 % };
226 %
227 % const receiptTypeLabels: Record<string, string> = {
228 %   ticket: "Ticket",
229 %   factura_a: "Factura A",
230 %   factura_b: "Factura B",
231 %   factura_c: "Factura C",
232 %   nota_credito: "Nota de Credito",
233 % };
234 %
235 % const paymentMethodLabels: Record<string, string> = {
236 %   efectivo: "Efectivo",
237 %   tarjeta_debito: "Tarjeta Debito",
238 %   tarjeta_credito: "Tarjeta Credito",
239 %   transferencia: "Transferencia",
240 %   cuenta_habitacion: "Cuenta Habitacion",
241 %   mercadopago: "MercadoPago",
242 % };
243 %
244 % const menuItemFormSchema = z.object({
245 %   name: z.string().min(1, "El nombre es requerido"),
246 %   categoryId: z.string().min(1, "La categoria es requerida"),
247 %   description: z.string().optional(),
248 %   price: z.coerce.number().min(0, "El precio debe ser positivo"),
249 %   preparationTime: z.coerce.number().min(0).optional(),
250 %   isAvailable: z.string().default("true"),
251 % });
252 %
253 % type MenuItemFormValues = z.infer<typeof menuItemFormSchema>;
254 %
255 % const categoryFormSchema = z.object({
256 %   name: z.string().min(1, "El nombre es requerido"),
257 %   description: z.string().optional(),
258 %   displayOrder: z.coerce.number().default(0),
259 % });
260 %
261 % type CategoryFormValues = z.infer<typeof categoryFormSchema>;
262 %
263 % export default function RestaurantPage() {
264 %   const { toast } = useToast();
265 %   const [activeTab, setActiveTab] = useState("floor");
266 %   const [selectedArea, setSelectedArea] = useState<string>("all");
267 %   const [selectedTable, setSelectedTable] = useState<RestaurantTable | null>(null);
268 %   const [isNewOrderDialogOpen, setIsNewOrderDialogOpen] = useState(false);
269 %   const [isOrderDialogOpen, setIsOrderDialogOpen] = useState(false);
270 %   const [currentOrder, setCurrentOrder] = useState<RestaurantOrder | null>(null);
271 %   const [newCovers, setNewCovers] = useState(2);
272 %   const [orderView, setOrderView] = useState<"folio" | "menu" | "delete">("menu");
273 %   const [selectedCategory, setSelectedCategory] = useState<string | null>(null);
274 %   const [pendingItem, setPendingItem] = useState<MenuItem | null>(null);
275 %   const [itemNotes, setItemNotes] = useState("");
276 %   const [isCloseDialogOpen, setIsCloseDialogOpen] = useState(false);
```

client/src/pages/restaurant.tsx (cont.)

```
277 % const [isReservationDialogOpen, setIsReservationDialogOpen] = useState(false);
278 % const [isDailyReservationsOpen, setIsDailyReservationsOpen] = useState(false);
279 % const [reservationDate, setReservationDate] = useState(new Date().toISOString().split("T")[0]);
280 % const [editingReservation, setEditingReservation] = useState<TableReservation | null>(null);
281 % const [isEditMode, setIsEditMode] = useState(false);
282 % const [draggedTable, setDraggedTable] = useState<RestaurantTable | null>(null);
283 % const [isAddTableDialogOpen, setIsAddTableDialogOpen] = useState(false);
284 % const [isEditTableDialogOpen, setIsEditTableDialogOpen] = useState(false);
285 % const [editingTable, setEditingTable] = useState<RestaurantTable | null>(null);
286 % const [editTableCapacity, setEditTableCapacity] = useState(4);
287 % const [editTableShape, setEditTableShape] = useState("square");
288 % const [editTableWindow, setEditTableWindow] = useState(false);
289 % const [reservationSortBy, setReservationSortBy] = useState<"name" | "time">("name");
290 % const [isMenuItemDialogOpen, setIsMenuItemDialogOpen] = useState(false);
291 % const [editingMenuItem, setEditingMenuItem] = useState<MenuItem | null>(null);
292 % const [isCategoryDialogOpen, setIsCategoryDialogOpen] = useState(false);
293 % const [editingCategory, setEditingCategory] = useState<MenuCategory | null>(null);
294 % const [reservationSearch, setReservationSearch] = useState("");
295 % const [closeReceiptType, setCloseReceiptType] = useState("ticket");
296 % const [closePaymentMethod, setClosePaymentMethod] = useState("efectivo");
297 % const [isTimeSlotsDialogOpen, setIsTimeSlotsDialogOpen] = useState(false);
298 % const [newTimeSlot, setNewTimeSlot] = useState("");
299 % const [isRecipeDialogOpen, setIsRecipeDialogOpen] = useState(false);
300 % const [selectedRecipeItem, setSelectedRecipeItem] = useState<MenuItem | null>(null);
301 % const [newIngredientName, setNewIngredientName] = useState("");
302 % const [newIngredientQty, setNewIngredientQty] = useState("");
303 % const [newIngredientUnit, setNewIngredientUnit] = useState("g");
304 % const [newIngredientCost, setNewIngredientCost] = useState("");
305 % const [newTableNumber, setNewTableNumber] = useState("");
306 % const [newTableCapacity, setNewTableCapacity] = useState(4);
307 % const [newTableShape, setNewTableShape] = useState("square");
308 % const [newTableArea, setNewTableArea] = useState("");
309 % const [newTableWindow, setNewTableWindow] = useState(false);
310 % const [newWaiterName, setNewWaiterName] = useState("");
311 % const [newOrderLabel, setNewOrderLabel] = useState("");
312 % const [isDirectOrderDialogOpen, setIsDirectOrderDialogOpen] = useState(false);
313 % const [directOrderAreaId, setDirectOrderAreaId] = useState("");
314 % const [itemCourse, setItemCourse] = useState(1);
315 % const [isSplitMode, setIsSplitMode] = useState(false);
316 % const [splitParts, setSplitParts] = useState(2);
317 % const [splitReceiptType, setSplitReceiptType] = useState("ticket");
318 % const [splitPayMethod, setSplitPayMethod] = useState("efectivo");
319 % const [splitPayMethods, setSplitPayMethods] = useState<Record<string, string>>({});
320 % const [splitReceiptTypes, setSplitReceiptTypes] = useState<Record<string, string>>({});
321 %
322 % const courseLabels: Record<number, string> = { 1: "Entradas", 2: "Platos Principales", 3: "Postres" };
323 %
324 % const reservationForm = useForm<ReservationFormValues>({
325 %   resolver: zodResolver(reservationFormSchema),
326 %   defaultValues: {
327 %     tableId: "",
328 %     guestName: "",
329 %     guestPhone: "",
330 %     guestEmail: "",
331 %     partySize: 2,
332 %     reservationDate: new Date().toISOString().split("T")[0],
333 %     reservationTime: "20:00",
334 %     notes: "",
335 %   },
336 % });
337 %
338 % const menuItemForm = useForm<MenuItemFormValues>({
339 %   resolver: zodResolver(menuItemFormSchema),
340 %   defaultValues: {
341 %     name: "",
342 %     categoryId: "",
343 %     description: "",
344 %     price: 0,
345 %     preparationTime: 0,
346 %     isAvailable: "true",
347 %   },
348 % });
349 %
350 % const categoryForm = useForm<CategoryFormValues>({
351 %   resolver: zodResolver(categoryFormSchema),
352 %   defaultValues: { name: "", description: "", displayOrder: 0 },
353 % });
354 %
355 % const { data: areas = [], isLoading: areasLoading } = useQuery<RestaurantArea[]>({
356 %   queryKey: ["/api/restaurant/areas"],
357 % });
358 %
359 % const { data: tables = [], isLoading: tablesLoading } = useQuery<RestaurantTable[]>({
360 %   queryKey: ["/api/restaurant/tables"],
361 % });
362 %
363 % const { data: menuCategories = [] } = useQuery<MenuCategory[]>({
364 %   queryKey: ["/api/restaurant/menu/categories"],
365 % });
366 %
367 % const { data: menuItems = [] } = useQuery<MenuItem[]>({
368 %   queryKey: ["/api/restaurant/menu/items"],
369 % });
```

client/src/pages/restaurant.tsx (cont.)

```
370 %
371 %   const { data: orders = [], isLoading: ordersLoading } = useQuery<RestaurantOrder[]>({
372 %     queryKey: ["/api/restaurant/orders"],
373 %   });
374 %
375 %   const { data: reservations = [] } = useQuery<TableReservation[]>({
376 %     queryKey: ["/api/restaurant/table-reservations"],
377 %   });
378 %
379 %   const { data: timeSlots = [] } = useQuery<TimeSlot[]>({
380 %     queryKey: ["/api/restaurant/time-slots"],
381 %   });
382 %
383 %   const { data: recipes = [] } = useQuery<Recipe[]>({
384 %     queryKey: ["/api/restaurant/recipes"],
385 %   });
386 %
387 %   const todayReservations = reservations.filter(r =>
388 %     r.reservationDate === new Date().toISOString().split("T")[0] &&
389 %     r.status !== "cancelled" && r.status !== "completed"
390 %   );
391 %
392 %   const sortedCategories = [...menuCategories].sort((a, b) => (a.displayOrder ?? 0) - (b.displayOrder ?? 0));
393 %
394 %   const createReservationMutation = useMutation({
395 %     mutationFn: async (data: ReservationFormValues) => {
396 %       const res = await apiRequest("POST", "/api/restaurant/table-reservations", data);
397 %       return res.json();
398 %     },
399 %     onSuccess: () => {
400 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/table-reservations"] });
401 %       setIsReservationDialogOpen(false);
402 %       reservationForm.reset();
403 %       toast({ title: "Reserva creada", description: "La reserva ha sido registrada" });
404 %     },
405 %   });
406 %
407 %   const updateReservationMutation = useMutation({
408 %     mutationFn: async ({ id, data }: { id: string; data: Partial<TableReservation> }) => {
409 %       const res = await apiRequest("PATCH", `/api/restaurant/table-reservations/${id}`, data);
410 %       return res.json();
411 %     },
412 %     onSuccess: () => {
413 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/table-reservations"] });
414 %       setEditingReservation(null);
415 %       toast({ title: "Reserva actualizada" });
416 %     },
417 %   });
418 %
419 %   const deleteReservationMutation = useMutation({
420 %     mutationFn: async (id: string) => {
421 %       await apiRequest("DELETE", `/api/restaurant/table-reservations/${id}`);
422 %     },
423 %     onSuccess: () => {
424 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/table-reservations"] });
425 %       toast({ title: "Reserva eliminada" });
426 %     },
427 %   });
428 %
429 %   const createOrderMutation = useMutation({
430 %     mutationFn: async (data: { tableId?: string; areaId?: string; covers: number; waiterName: string; orderLabel?: st...
431 %       const res = await apiRequest("POST", "/api/restaurant/orders", data);
432 %       return res.json();
433 %     },
434 %     onSuccess: (order: RestaurantOrder) => {
435 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
436 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/tables"] });
437 %       setCurrentOrder(order);
438 %       setIsNewOrderDialogOpen(false);
439 %       setIsDirectOrderDialogOpen(false);
440 %       setNewWaiterName("");
441 %       setNewOrderLabel("");
442 %       setOrderView("menu");
443 %       setSelectedCategory(null);
444 %       setIsOrderDialogOpen(true);
445 %       toast({ title: "Pedido creado", description: `Pedido ${order.orderNumber} iniciado` });
446 %     },
447 %   });
448 %
449 %   const addItemMutation = useMutation({
450 %     mutationFn: async (data: { orderId: string; menuItemId: string; quantity: number; notes?: string; course?: number...
451 %       const res = await apiRequest("POST", `/api/restaurant/orders/${data.orderId}/items`, {
452 %         menuItemId: data.menuItemId,
453 %         quantity: data.quantity,
454 %         notes: data.notes,
455 %         course: data.course || 1,
456 %       });
457 %       return res.json();
458 %     },
459 %     onSuccess: () => {
460 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
461 %       setPendingItem(null);
462 %       setItemNotes("");
```

client/src/pages/restaurant.tsx (cont.)

```
463 %         toast({ title: "Item agregado" });
464 %     },
465 % });
466 %
467 % const advanceCourseMutation = useMutation({
468 %     mutationFn: async (orderId: string) => {
469 %         const res = await apiRequest("POST", `/api/restaurant/orders/${orderId}/advance-course`);
470 %         return res.json();
471 %     },
472 %     onSuccess: (data: { activeCourse: number; activatedItems: number }) => {
473 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
474 %         toast({ title: `Curso activado: ${courseLabels[data.activeCourse]}`, description: `${data.activatedItems} items...` });
475 %     },
476 % });
477 %
478 % const createSplitMutation = useMutation({
479 %     mutationFn: async (data: { orderId: string; parts: number }) => {
480 %         const res = await apiRequest("POST", `/api/restaurant/orders/${data.orderId}/split`, { parts: data.parts });
481 %         return res.json();
482 %     },
483 %     onSuccess: () => {
484 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
485 %         toast({ title: "Cuenta dividida" });
486 %     },
487 % });
488 %
489 % const paySplitMutation = useMutation({
490 %     mutationFn: async (data: { orderId: string; splitId: string; method: string; receiptType: string }) => {
491 %         const res = await apiRequest("PATCH", `/api/restaurant/orders/${data.orderId}/split/${data.splitId}`, {
492 %             method: data.method,
493 %             receiptType: data.receiptType,
494 %         });
495 %         return res.json();
496 %     },
497 %     onSuccess: (data: { split: OrderSplit; allPaid: boolean }) => {
498 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
499 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/tables"] });
500 %         if (data.allPaid) {
501 %             setCurrentOrder(null);
502 %             setIsCloseDialogOpen(false);
503 %             setIsSplitMode(false);
504 %             toast({ title: "Todas las partes pagadas – mesa cerrada" });
505 %         } else {
506 %             toast({ title: "Parte cobrada" });
507 %         }
508 %     },
509 % });
510 %
511 % const cancelSplitMutation = useMutation({
512 %     mutationFn: async (orderId: string) => {
513 %         await apiRequest("DELETE", `/api/restaurant/orders/${orderId}/split`);
514 %     },
515 %     onSuccess: () => {
516 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
517 %         setIsSplitMode(false);
518 %         toast({ title: "División cancelada" });
519 %     },
520 % });
521 %
522 % const deleteItemMutation = useMutation({
523 %     mutationFn: async (data: { orderId: string; itemId: string }) => {
524 %         const res = await apiRequest("DELETE", `/api/restaurant/orders/${data.orderId}/items/${data.itemId}`);
525 %         return res.json();
526 %     },
527 %     onSuccess: () => {
528 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
529 %         toast({ title: "Item eliminado" });
530 %     },
531 % });
532 %
533 % const closeOrderMutation = useMutation({
534 %     mutationFn: async (data: { orderId: string; receiptType: string; paymentMethod: string }) => {
535 %         const res = await apiRequest("POST", `/api/restaurant/orders/${data.orderId}/close`, {
536 %             chargeToRoom: data.paymentMethod === "cuenta_habitacion",
537 %             receiptType: data.receiptType,
538 %             paymentMethod: data.paymentMethod,
539 %         });
540 %         return res.json();
541 %     },
542 %     onSuccess: () => {
543 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/orders"] });
544 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/tables"] });
545 %         setCurrentOrder(null);
546 %         setIsCloseDialogOpen(false);
547 %         toast({ title: "Pedido cerrado" });
548 %     },
549 % });
550 %
551 % const updateTableMutation = useMutation({
552 %     mutationFn: async ({ id, data }: { id: string; data: Record<string, unknown> }) => {
553 %         const res = await apiRequest("PATCH", `/api/restaurant/tables/${id}`, data);
554 %         return res.json();
555 %     },
```

client/src/pages/restaurant.tsx (cont.)

```
556 %     onSuccess: () => {
557 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/tables"] });
558 %     },
559 %   });
560 %
561 %   const createTableMutation = useMutation({
562 %     mutationFn: async (data: Record<string, unknown>) => {
563 %       const res = await apiRequest("POST", "/api/restaurant/tables", data);
564 %       return res.json();
565 %     },
566 %     onSuccess: () => {
567 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/tables"] });
568 %       setIsAddTableDialogOpen(false);
569 %       setNewTableNumber("");
570 %       setNewTableCapacity(4);
571 %       setNewTableShape("square");
572 %       setNewTableWindow(false);
573 %       toast({ title: "Mesa creada" });
574 %     },
575 %   });
576 %
577 %   const deleteTableMutation = useMutation({
578 %     mutationFn: async (id: string) => {
579 %       await apiRequest("DELETE", `/api/restaurant/tables/${id}`);
580 %     },
581 %     onSuccess: () => {
582 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/tables"] });
583 %       toast({ title: "Mesa eliminada" });
584 %     },
585 %   });
586 %
587 %   const createMenuItemMutation = useMutation({
588 %     mutationFn: async (data: MenuItemFormValues) => {
589 %       const res = await apiRequest("POST", "/api/restaurant/menu/items", {
590 %         ...data,
591 %         price: data.price.toString(),
592 %       });
593 %       return res.json();
594 %     },
595 %     onSuccess: () => {
596 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/menu/items"] });
597 %       setIsMenuItemDialogOpen(false);
598 %       menuItemForm.reset();
599 %       toast({ title: "Plato creado" });
600 %     },
601 %   });
602 %
603 %   const updateMenuItemMutation = useMutation({
604 %     mutationFn: async ({ id, data }: { id: string; data: MenuItemFormValues }) => {
605 %       const res = await apiRequest("PATCH", `/api/restaurant/menu/items/${id}`, {
606 %         ...data,
607 %         price: data.price.toString(),
608 %       });
609 %       return res.json();
610 %     },
611 %     onSuccess: () => {
612 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/menu/items"] });
613 %       setIsMenuItemDialogOpen(false);
614 %       setEditingMenuItem(null);
615 %       menuItemForm.reset();
616 %       toast({ title: "Plato actualizado" });
617 %     },
618 %   });
619 %
620 %   const deleteMenuItemMutation = useMutation({
621 %     mutationFn: async (id: string) => {
622 %       await apiRequest("DELETE", `/api/restaurant/menu/items/${id}`);
623 %     },
624 %     onSuccess: () => {
625 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/menu/items"] });
626 %       toast({ title: "Plato eliminado" });
627 %     },
628 %   });
629 %
630 %   const createCategoryMutation = useMutation({
631 %     mutationFn: async (data: CategoryFormValues) => {
632 %       const res = await apiRequest("POST", "/api/restaurant/menu/categories", data);
633 %       return res.json();
634 %     },
635 %     onSuccess: () => {
636 %       queryClient.invalidateQueries({ queryKey: ["/api/restaurant/menu/categories"] });
637 %       setIsCategoryDialogOpen(false);
638 %       categoryForm.reset();
639 %       toast({ title: "Categoria creada" });
640 %     },
641 %   });
642 %
643 %   const updateCategoryMutation = useMutation({
644 %     mutationFn: async ({ id, data }: { id: string; data: CategoryFormValues }) => {
645 %       const res = await apiRequest("PATCH", `/api/restaurant/menu/categories/${id}`, data);
646 %       return res.json();
647 %     },
648 %     onSuccess: () => {
```

client/src/pages/restaurant.tsx (cont.)

```
649 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/menu/categories"] });
650 %         setIsCategoryDialogOpen(false);
651 %         setEditingCategory(null);
652 %         categoryForm.reset();
653 %         toast({ title: "Categoria actualizada" });
654 %     },
655 % });
656 %
657 % const deleteCategoryMutation = useMutation({
658 %     mutationFn: async (id: string) => {
659 %         await apiRequest("DELETE", `/api/restaurant/menu/categories/${id}`);
660 %     },
661 %     onSuccess: () => {
662 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/menu/categories"] });
663 %         toast({ title: "Categoria eliminada" });
664 %     },
665 % });
666 %
667 % const createTimeSlotMutation = useMutation({
668 %     mutationFn: async (time: string) => {
669 %         const res = await apiRequest("POST", "/api/restaurant/time-slots", { time });
670 %         return res.json();
671 %     },
672 %     onSuccess: () => {
673 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/time-slots"] });
674 %         setNewTimeSlot("");
675 %         toast({ title: "Horario agregado" });
676 %     },
677 % });
678 %
679 % const deleteTimeSlotMutation = useMutation({
680 %     mutationFn: async (id: string) => {
681 %         await apiRequest("DELETE", `/api/restaurant/time-slots/${id}`);
682 %     },
683 %     onSuccess: () => {
684 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/time-slots"] });
685 %         toast({ title: "Horario eliminado" });
686 %     },
687 % });
688 %
689 % const createRecipeMutation = useMutation({
690 %     mutationFn: async (menuItem: string) => {
691 %         const res = await apiRequest("POST", "/api/restaurant/recipes", { menuItem });
692 %         return res.json();
693 %     },
694 %     onSuccess: () => {
695 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/recipes"] });
696 %     },
697 % });
698 %
699 % const addIngredientMutation = useMutation({
700 %     mutationFn: async (data: { recipeId: string; ingredientName: string; quantity: string; unit: string; unitCost: st...
701 %     const res = await apiRequest("POST", `/api/restaurant/recipes/${data.recipeId}/ingredients`, data);
702 %     return res.json();
703 % },
704 %     onSuccess: () => {
705 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/recipes"] });
706 %         setNewIngredientName("");
707 %         setNewIngredientQty("");
708 %         setNewIngredientUnit("g");
709 %         setNewIngredientCost("");
710 %         toast({ title: "Ingrediente agregado" });
711 %     },
712 % });
713 %
714 % const deleteIngredientMutation = useMutation({
715 %     mutationFn: async (id: string) => {
716 %         await apiRequest("DELETE", `/api/restaurant/recipe-ingredients/${id}`);
717 %     },
718 %     onSuccess: () => {
719 %         queryClient.invalidateQueries({ queryKey: ["/api/restaurant/recipes"] });
720 %         toast({ title: "Ingrediente eliminado" });
721 %     },
722 % });
723 %
724 % const filteredTables = selectedArea === "all"
725 %     ? tables
726 %     : tables.filter((t) => t.areaId === selectedArea);
727 %
728 % const activeOrders = orders.filter((o) => o.status !== "closed" && o.status !== "cancelled");
729 %
730 % const handleTableClick = (table: RestaurantTable) => {
731 %     if (isEditMode) return;
732 %     setSelectedTable(table);
733 %     if (table.status === "available") {
734 %         setNewCovers(table.capacity);
735 %         setNewWaiterName("");
736 %         setIsNewOrderDialogOpen(true);
737 %     } else if (table.status === "occupied") {
738 %         const tableOrder = orders.find((o) => o.tableId === table.id && o.status !== "closed" && o.status !== "cancelle...
739 %         if (tableOrder) {
740 %             setCurrentOrder(tableOrder);
741 %             setOrderView("menu");
```

client/src/pages/restaurant.tsx (cont.)

```
742 %         setSelectedCategory(null);
743 %         setIsOrderDialogOpen(true);
744 %     }
745 % }
746 % };
747 %
748 % const handleConfirmItem = () => {
749 %     if (currentOrder && pendingItem) {
750 %         addItemMutation.mutate({
751 %             orderId: currentOrder.id,
752 %             menuItemId: pendingItem.id,
753 %             quantity: 1,
754 %             notes: itemNotes || undefined,
755 %             course: itemCourse,
756 %         });
757 %     }
758 % };
759 %
760 % const handleCancelItem = () => {
761 %     setPendingItem(null);
762 %     setItemNotes("");
763 % };
764 %
765 % const getUpdatedOrder = () => {
766 %     if (!currentOrder) return null;
767 %     return orders.find(o => o.id === currentOrder.id) || currentOrder;
768 % };
769 %
770 % const getOrderItems = () => {
771 %     const order = getUpdatedOrder();
772 %     return order?.items || [];
773 % };
774 %
775 % const handleDragStart = (table: RestaurantTable) => {
776 %     if (!isEditMode) return;
777 %     setDraggedTable(table);
778 % };
779 %
780 % const handleDragOver = (e: React.DragEvent) => {
781 %     e.preventDefault();
782 % };
783 %
784 % const handleDrop = (x: number, y: number, areaId: string) => {
785 %     if (!draggedTable || !isEditMode) return;
786 %     const existingTable = tables.find(t => t.positionX === x && t.positionY === y && t.areaId === areaId && t.id !== ...
787 %     if (existingTable) return;
788 %     updateTableMutation.mutate({
789 %         id: draggedTable.id,
790 %         data: { positionX: x, positionY: y },
791 %     });
792 %     setDraggedTable(null);
793 % };
794 %
795 % const filteredReservations = reservations
796 %     .filter(r => r.reservationDate === reservationDate)
797 %     .filter(r => {
798 %         if (!reservationSearch) return true;
799 %         return r.guestName.toLowerCase().includes(reservationSearch.toLowerCase());
800 %     })
801 %     .sort((a, b) => {
802 %         if (reservationSortBy === "time") return a.reservationTime.localeCompare(b.reservationTime);
803 %         return a.guestName.localeCompare(b.guestName);
804 %     });
805 %
806 % const openMenuItemDialog = (item?: MenuItem) => {
807 %     if (item) {
808 %         setEditingMenuItem(item);
809 %         menuItemForm.reset({
810 %             name: item.name,
811 %             categoryId: item.categoryId,
812 %             description: item.description || "",
813 %             price: parseFloat(item.price),
814 %             preparationTime: item.preparationTime || 0,
815 %             isAvailable: item.isAvailable || "true",
816 %         });
817 %     } else {
818 %         setEditingMenuItem(null);
819 %         menuItemForm.reset({
820 %             name: "",
821 %             categoryId: sortedCategories[0]?.id || "",
822 %             description: "",
823 %             price: 0,
824 %             preparationTime: 0,
825 %             isAvailable: "true",
826 %         });
827 %     }
828 %     setIsMenuItemDialogOpen(true);
829 % };
830 %
831 % const openCategoryDialog = (cat?: MenuCategory) => {
832 %     if (cat) {
833 %         setEditingCategory(cat);
834 %         categoryForm.reset({
```


client/src/pages/restaurant.tsx (cont.)

```
835 %         name: cat.name,
836 %         description: cat.description || "",
837 %         displayOrder: cat.displayOrder || 0,
838 %     });
839 % } else {
840 %     setEditingCategory(null);
841 %     categoryForm.reset({ name: "", description: "", displayOrder: 0 });
842 % }
843 % setIsCategoryDialogOpen(true);
844 % };
845 %
846 % const openRecipeDialog = async (item: MenuItem) => {
847 %     setSelectedRecipeItem(item);
848 %     const existing = recipes.find(r => r.menuItemId === item.id);
849 %     if (!existing) {
850 %         await createRecipeMutation.mutateAsync(item.id);
851 %     }
852 %     setIsRecipeDialogOpen(true);
853 % };
854 %
855 % const currentRecipe = selectedRecipeItem ? recipes.find(r => r.menuItemId === selectedRecipeItem.id) : null;
856 % const recipeCost = currentRecipe?.ingredients.reduce((sum, ing) => {
857 %     return sum + parseFloat(ing.quantity) * parseFloat(ing.unitCost || "0");
858 % }, 0) || 0;
859 %
860 % if (areasLoading || tablesLoading) {
861 %     return (
862 %         <div className="flex items-center justify-center h-full">
863 %             <Loader2 className="h-8 w-8 animate-spin text-muted-foreground" />
864 %         </div>
865 %     );
866 % }
867 %
868 % return (
869 %     <div className="p-6 space-y-6">
870 %         <div className="flex items-center justify-between gap-4 flex-wrap">
871 %             <div>
872 %                 <h1 className="text-2xl font-bold" data-testid="text-page-title">Restaurante</h1>
873 %                 <p className="text-muted-foreground">Gestiona mesas, pedidos y menu</p>
874 %             </div>
875 %             <div className="flex items-center gap-2 flex-wrap">
876 %                 <Button
877 %                     variant="outline"
878 %                     onClick={() => setIsDailyReservationsOpen(true)}
879 %                     data-testid="button-daily-reservations"
880 %                 >
881 %                     <CalendarDays className="h-4 w-4 mr-2" />
882 %                     Reservas del dia
883 %                     {todayReservations.length > 0 && (
884 %                         <Badge variant="secondary" className="ml-2">{todayReservations.length}</Badge>
885 %                     )}
886 %                 </Button>
887 %                 <Badge variant="outline" className="gap-1">
888 %                     <UtensilsCrossed className="h-3 w-3" />
889 %                     {activeOrders.length} pedidos activos
890 %                 </Badge>
891 %             </div>
892 %         </div>
893 %
894 %         <Tabs value={activeTab} onValueChange={setActiveTab}>
895 %             <TabsList>
896 %                 <TabsTrigger value="floor" data-testid="tab-floor">
897 %                     <MapPin className="h-4 w-4 mr-2" />
898 %                     Plano de Mesas
899 %                 </TabsTrigger>
900 %                 <TabsTrigger value="orders" data-testid="tab-orders">
901 %                     <Clock className="h-4 w-4 mr-2" />
902 %                     Pedidos
903 %                 </TabsTrigger>
904 %                 <TabsTrigger value="menu" data-testid="tab-menu">
905 %                     <UtensilsCrossed className="h-4 w-4 mr-2" />
906 %                     Menu
907 %                 </TabsTrigger>
908 %                 <TabsTrigger value="reservations" data-testid="tab-reservations">
909 %                     <CalendarDays className="h-4 w-4 mr-2" />
910 %                     Reservas
911 %                 </TabsTrigger>
912 %                 <TabsTrigger value="recipes" data-testid="tab-recipes">
913 %                     <ChefHat className="h-4 w-4 mr-2" />
914 %                     Recetas y Costos
915 %                 </TabsTrigger>
916 %             </TabsList>
917 %
918 %             {/* ===== FLOOR PLAN TAB ===== */}
919 %             <TabsContent value="floor" className="space-y-4">
920 %                 <div className="flex items-center gap-2 flex-wrap">
921 %                     <Select value={selectedArea} onValueChange={setSelectedArea}>
922 %                         <SelectTrigger className="w-48" data-testid="select-area">
923 %                             <SelectValue placeholder="Filtrar por area" />
924 %                         </SelectTrigger>
925 %                         <SelectContent>
926 %                             <SelectItem value="all">Todas las areas</SelectItem>
927 %                             {areas.map((area) => (
```

client/src/pages/restaurant.tsx (cont.)

```
928 %         <SelectItem key={area.id} value={area.id}>
929 %             {area.name}
930 %         </SelectItem>
931 %     )}}
932 % </SelectContent>
933 % </Select>
934 % <div className="flex gap-2 text-sm text-muted-foreground">
935 %     {Object.entries(tableStatusLabels).map(([key, label]) => (
936 %         <div key={key} className="flex items-center gap-1">
937 %             <div className={`w-3 h-3 rounded-full ${tableStatusColors[key]} `} />
938 %             <span>{label}</span>
939 %         </div>
940 %     )}}
941 % </div>
942 % <div className="ml-auto flex items-center gap-2">
943 %     <Button
944 %         variant={isEditMode ? "default" : "outline"}
945 %         size="sm"
946 %         onClick={() => setIsEditMode(!isEditMode)}
947 %         data-testid="button-edit-layout"
948 %     >
949 %         {isEditMode ? <Check className="h-4 w-4 mr-2" /> : <Settings className="h-4 w-4 mr-2" />}
950 %         {isEditMode ? "Guardar Layout" : "Editar Layout"}
951 %     </Button>
952 %     {isEditMode && (
953 %         <Button
954 %             size="sm"
955 %             onClick={() => {
956 %                 if (areas.length > 0) {
957 %                     setNewTableArea(areas[0].id);
958 %                     setIsAddTableDialogOpen(true);
959 %                 }
960 %             }}
961 %             data-testid="button-add-table"
962 %         >
963 %             <Plus className="h-4 w-4 mr-2" />
964 %             Agregar Mesa
965 %         </Button>
966 %     )}
967 % </div>
968 % </div>
969 %
970 % {areas.length === 0 ? (
971 %     <Card>
972 %         <CardContent className="flex flex-col items-center justify-center py-12 text-center">
973 %             <MapPin className="h-12 w-12 text-muted-foreground mb-4" />
974 %             <h3 className="text-lg font-semibold mb-2">Sin areas configuradas</h3>
975 %             <p className="text-muted-foreground mb-4">Agrega areas y mesas para comenzar</p>
976 %         </CardContent>
977 %     </Card>
978 % ) : (
979 %     <div className="grid gap-6">
980 %         {(selectedArea === "all" ? areas : areas.filter((a) => a.id === selectedArea)).map((area) => {
981 %             if (area.hasTables === "false") {
982 %                 const areaOrders = activeOrders.filter((o) => o.areaId === area.id);
983 %                 return (
984 %                     <Card key={area.id}>
985 %                         <CardHeader className="pb-3">
986 %                             <div className="flex items-center justify-between">
987 %                                 <CardTitle className="text-base">{area.name}</CardTitle>
988 %                                 <Button
989 %                                     size="sm"
990 %                                     onClick={() => {
991 %                                         setDirectOrderAreaId(area.id);
992 %                                         setNewWaiterName("");
993 %                                         setNewOrderLabel("");
994 %                                         setNewCovers(1);
995 %                                         setIsDirectOrderDialogOpen(true);
996 %                                     }}
997 %                                     data-testid={`button-new-direct-order-${area.id}`}
998 %                                 >
999 %                                     <Plus className="h-4 w-4 mr-2" />
1000 %                                     Nueva Orden
1001 %                                 </Button>
1002 %                             </div>
1003 %                         </CardHeader>
1004 %                         <CardContent>
1005 %                             {areaOrders.length === 0 ? (
1006 %                                 <p className="text-sm text-muted-foreground text-center py-4">Sin órdenes activas</p>
1007 %                             ) : (
1008 %                                 <div className="space-y-2">
1009 %                                     {areaOrders.map((order) => (
1010 %                                         <button
1011 %                                             key={order.id}
1012 %                                             className="w-full flex items-center justify-between p-3 border rounded-md hover-eleva...
1013 %                                             onClick={() => {
1014 %                                                 setCurrentOrder(order);
1015 %                                                 setOrderView("menu");
1016 %                                                 setSelectedCategory(null);
1017 %                                                 setIsOrderDialogOpen(true);
1018 %                                             }}
1019 %                                             data-testid={`direct-order-${order.id}`}
1020 %                                         >
```

```

1021 %         <div className="flex-1">
1022 %             <div className="flex items-center gap-2">
1023 %                 <span className="font-medium">{order.orderLabel || order.orderNumber}</span>
1024 %                 <Badge variant="outline" className="text-xs">{order.orderNumber}</Badge>
1025 %             </div>
1026 %             <div className="text-xs text-muted-foreground mt-0.5">
1027 %                 Mozo: {order.waiterName || "-"} | {new Date(order.openedAt).toLocaleTimeString("e...
1028 %                 {order.items && order.items.length > 0 && ` | ${order.items.length} items`}
1029 %             </div>
1030 %         </div>
1031 %         <div className="text-right">
1032 %             <span className="font-semibold">${parseFloat(order.total).toLocaleString("es-AR", {...
1033 %             <div className="text-xs">
1034 %                 <Badge variant={order.status === "open" ? "default" : order.status === "in_progre...
1035 %                 {order.status === "open" ? "Abierto" : order.status === "in_progress" ? "En cur...
1036 %             </div>
1037 %         </div>
1038 %     </div>
1039 %     </button>
1040 %   )}}
1041 % </div>
1042 %   )}
1043 % </CardContent>
1044 % </Card>
1045 % );
1046 % }
1047 %
1048 % const areaTables = filteredTables.filter((t) => t.areaId === area.id);
1049 % const gridCols = 8;
1050 % const gridRows = 6;
1051 %
1052 % return (
1053 %   <Card key={area.id}>
1054 %     <CardHeader className="pb-3">
1055 %       <div className="flex items-center justify-between gap-4 flex-wrap">
1056 %         <CardTitle className="text-lg flex items-center gap-2">
1057 %           {area.name}
1058 %         </CardTitle>
1059 %         <span className="text-sm text-muted-foreground">
1060 %           {areaTables.length} mesas | Capacidad: {area.capacity}
1061 %         </span>
1062 %       </div>
1063 %     </CardHeader>
1064 %     <CardContent>
1065 %       <div
1066 %         className="grid gap-1 p-4 bg-muted/30 rounded-lg relative"
1067 %         style={{
1068 %           gridTemplateColumns: `repeat(${gridCols}, minmax(70px, 1fr))`,
1069 %           gridTemplateRows: `repeat(${gridRows}, 75px)`,
1070 %         }}
1071 %       >
1072 %         {Array.from({ length: gridCols * gridRows }).map((_, idx) => {
1073 %           const x = idx % gridCols;
1074 %           const y = Math.floor(idx / gridCols);
1075 %           const table = areaTables.find(t => t.positionX === x && t.positionY === y);
1076 %
1077 %           if (table) {
1078 %             return (
1079 %               <button
1080 %                 key={table.id}
1081 %                 draggable={isEditMode}
1082 %                 onDragStart={() => handleDragStart(table)}
1083 %                 onDragOver={handleDragOver}
1084 %                 onDrop={() => handleDrop(x, y, area.id)}
1085 %                 onClick={() => handleTableClick(table)}
1086 %                 className={`p-2 border-2 transition-all flex flex-col items-center justify-center gap...
1087 %                 tableStatusColors[table.status]
1088 %                 } ${table.shape === "round" ? "rounded-full" : "rounded-md"} ${
1089 %                   isEditMode ? "cursor-grab active:cursor-grabbing ring-2 ring-primary/30" : "hover-e...
1090 %                 } ${draggedTable?.id === table.id ? "opacity-50" : ""}`
1091 %                 style={{
1092 %                   gridColumn: x + 1,
1093 %                   gridRow: y + 1,
1094 %                 }}
1095 %                 data-testid={`table-${table.tableNumber}`}
1096 %               >
1097 %                 {table.hasWindow === "true" && (
1098 %                   <div className="absolute -top-1 -right-1 w-4 h-4 bg-sky-500 rounded-full flex items...
1099 %                   <Eye className="h-2.5 w-2.5 text-white" />
1100 %                 </div>
1101 %                 )}
1102 %                 <span className="font-bold text-sm">{table.tableNumber}</span>
1103 %                 <div className="flex items-center gap-0.5 text-[10px]">
1104 %                   <Users className="h-2.5 w-2.5" />
1105 %                   {table.capacity}
1106 %                 </div>
1107 %                 {table.status === "occupied" && (() => {
1108 %                   const tableOrder = activeOrders.find(o => o.tableId === table.id);
1109 %                   return tableOrder?.waiterName ? (
1110 %                     <span className="text-[9px] truncate max-w-full opacity-80">{tableOrder.waiterNam...
1111 %                   ) : null;
1112 %                 })()}
1113 %                 {isEditMode && (

```

client/src/pages/restaurant.tsx (cont.)

```
1114 %             <GripVertical className="h-3 w-3 opacity-50" />
1115 %         }}
1116 %     </button>
1117 % );
1118 % }
1119 %
1120 %     return (
1121 %     <div
1122 %         key={`empty-${x}-${y}`}
1123 %         className={`rounded-md transition-all ${
1124 %             isEditMode
1125 %             ? "border-2 border-dashed border-muted-foreground/20 hover:border-primary/40 hover:...
1126 %             : "opacity-0"
1127 %         }`}
1128 %         onDragOver={handleDragOver}
1129 %         onDrop={() => handleDrop(x, y, area.id)}
1130 %         style={{
1131 %             gridColumn: x + 1,
1132 %             gridRow: y + 1,
1133 %         }}
1134 %     />
1135 %     );
1136 % }}
1137 % </div>
1138 % {isEditMode && areaTables.length > 0 && (
1139 %     <div className="mt-3 flex flex-wrap gap-2">
1140 %         {areaTables.map(t => (
1141 %             <div key={t.id} className="flex items-center gap-1 text-xs border rounded px-2 py-1">
1142 %                 <span>Mesa {t.tableNumber}</span>
1143 %                 {t.hasWindow === "true" && <Eye className="h-3 w-3 text-sky-500" />}
1144 %                 <Button
1145 %                     variant="ghost"
1146 %                     size="icon"
1147 %                     className="h-5 w-5"
1148 %                     onClick={() => {
1149 %                         setEditingTable(t);
1150 %                         setEditTableCapacity(t.capacity);
1151 %                         setEditTableShape(t.shape || "square");
1152 %                         setEditTableWindow(t.hasWindow === "true");
1153 %                         setIsEditTableDialogOpen(true);
1154 %                     }}
1155 %                     data-testid={`button-edit-table-${t.tableNumber}`}
1156 %                 >
1157 %                     <Edit className="h-3 w-3" />
1158 %                 </Button>
1159 %                 <Button
1160 %                     variant="ghost"
1161 %                     size="icon"
1162 %                     className="h-5 w-5"
1163 %                     onClick={() => deleteTableMutation.mutate(t.id)}
1164 %                     data-testid={`button-delete-table-${t.tableNumber}`}
1165 %                 >
1166 %                     <Trash2 className="h-3 w-3 text-destructive" />
1167 %                 </Button>
1168 %             </div>
1169 %         )})
1170 %     </div>
1171 % )}
1172 % </CardContent>
1173 % </Card>
1174 % );
1175 % }}
1176 % </div>
1177 % }}
1178 % </TabsContent>
1179 %
1180 % {/* ===== ORDERS TAB ===== */}
1181 % <TabsContent value="orders" className="space-y-4">
1182 %     {ordersLoading ? (
1183 %         <div className="flex items-center justify-center py-12">
1184 %             <Loader2 className="h-8 w-8 animate-spin text-muted-foreground" />
1185 %         </div>
1186 %     ) : activeOrders.length === 0 ? (
1187 %         <Card>
1188 %             <CardContent className="flex flex-col items-center justify-center py-12 text-center">
1189 %                 <UtensilsCrossed className="h-12 w-12 text-muted-foreground mb-4" />
1190 %                 <h3 className="text-lg font-semibold mb-2">Sin pedidos activos</h3>
1191 %                 <p className="text-muted-foreground">Los pedidos apareceran aqui cuando las mesas esten ocupadas</p>
1192 %             </CardContent>
1193 %         </Card>
1194 %     ) : (
1195 %         <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
1196 %             {activeOrders.map((order) => (
1197 %                 <Card key={order.id} data-testid={`order-card-${order.orderNumber}`}>
1198 %                     <CardHeader className="pb-2">
1199 %                         <div className="flex items-center justify-between gap-2">
1200 %                             <CardTitle className="text-base">{order.orderLabel || order.orderNumber}</CardTitle>
1201 %                             <Badge variant={order.status === "open" ? "default" : "secondary"}>
1202 %                                 {order.status === "open" ? "Abierto" : order.status === "in_progress" ? "En Proceso" : "Servi...
1203 %                             </Badge>
1204 %                         </div>
1205 %                     </CardHeader>
1206 %                     <CardContent className="space-y-3">
```

client/src/pages/restaurant.tsx (cont.)

```
1207 %         {order.table && (
1208 %           <div className="flex items-center gap-2 text-sm">
1209 %             <MapPin className="h-4 w-4 text-muted-foreground" />
1210 %             Mesa {order.table.tableNumber}
1211 %           </div>
1212 %         )}
1213 %         {order.tableId && order.areaId && (
1214 %           <div className="flex items-center gap-2 text-sm">
1215 %             <MapPin className="h-4 w-4 text-muted-foreground" />
1216 %             {areas.find(a => a.id === order.areaId)?.name}
1217 %           </div>
1218 %         )}
1219 %         {order.waiterName && (
1220 %           <div className="flex items-center gap-2 text-sm">
1221 %             <Users className="h-4 w-4 text-muted-foreground" />
1222 %             Mozo: {order.waiterName}
1223 %           </div>
1224 %         )}
1225 %         <div className="flex items-center gap-2 text-sm">
1226 %           <Users className="h-4 w-4 text-muted-foreground" />
1227 %           {order.covers} comensales
1228 %         </div>
1229 %         <div className="flex items-center gap-2 text-sm">
1230 %           <Clock className="h-4 w-4 text-muted-foreground" />
1231 %           Abierto: {new Date(order.openedAt).toLocaleTimeString("es-AR", { hour: "2-digit", minute: "2-di...
1232 %         </div>
1233 %         <div className="pt-2 border-t flex items-center justify-between">
1234 %           <span className="font-semibold">Total:</span>
1235 %           <span className="text-lg font-bold">
1236 %             ${parseFloat(order.total).toLocaleString("es-AR", { minimumFractionDigits: 2 })}
1237 %           </span>
1238 %         </div>
1239 %         <div className="flex gap-2">
1240 %           <Button
1241 %             size="sm"
1242 %             variant="outline"
1243 %             className="flex-1"
1244 %             onClick={() => {
1245 %               setCurrentOrder(order);
1246 %               setOrderView("menu");
1247 %               setSelectedCategory(null);
1248 %               setIsOrderDialogOpen(true);
1249 %             }}
1250 %             data-testid={`button-add-items-${order.orderNumber}`}
1251 %           >
1252 %             <Plus className="h-4 w-4 mr-1" />
1253 %             Items
1254 %           </Button>
1255 %           <Button
1256 %             size="sm"
1257 %             className="flex-1"
1258 %             onClick={() => {
1259 %               setCurrentOrder(order);
1260 %               setCloseReceiptType("ticket");
1261 %               setClosePaymentMethod("efectivo");
1262 %               setIsCloseDialogOpen(true);
1263 %             }}
1264 %             data-testid={`button-close-${order.orderNumber}`}
1265 %           >
1266 %             <CreditCard className="h-4 w-4 mr-1" />
1267 %             Cerrar
1268 %           </Button>
1269 %         </div>
1270 %       </CardContent>
1271 %     </Card>
1272 %   )}
1273 % </div>
1274 % )}
1275 % </TabsContent>
1276 %
1277 % {/* ===== MENU TAB ===== */}
1278 % <TabsContent value="menu" className="space-y-4">
1279 %   <div className="flex items-center justify-between gap-4 flex-wrap">
1280 %     <h2 className="text-lg font-semibold">Carta del Restaurante</h2>
1281 %     <div className="flex items-center gap-2">
1282 %       <Button variant="outline" onClick={() => openCategoryDialog()} data-testid="button-add-category">
1283 %         <Plus className="h-4 w-4 mr-2" />
1284 %         Nueva Categoría
1285 %       </Button>
1286 %       <Button onClick={() => openMenuItemDialog()} data-testid="button-add-menu-item">
1287 %         <Plus className="h-4 w-4 mr-2" />
1288 %         Agregar Plato
1289 %       </Button>
1290 %     </div>
1291 %   </div>
1292 %   {sortedCategories.length === 0 ? (
1293 %     <Card>
1294 %       <CardContent className="flex flex-col items-center justify-center py-12 text-center">
1295 %         <UtensilsCrossed className="h-12 w-12 text-muted-foreground mb-4" />
1296 %         <h3 className="text-lg font-semibold mb-2">Menu vacío</h3>
1297 %         <p className="text-muted-foreground mb-4">Agrega categorías y platos al menú</p>
1298 %       </CardContent>
1299 %     </Card>
```

client/src/pages/restaurant.tsx (cont.)

```
1300 %         ) : (
1301 %             <div className="grid gap-6">
1302 %                 {sortedCategories.map((category) => (
1303 %                     <Card key={category.id}>
1304 %                         <CardHeader>
1305 %                             <div className="flex items-center justify-between gap-2">
1306 %                                 <div>
1307 %                                     <CardTitle className="flex items-center gap-2">
1308 %                                         {category.name}
1309 %                                         <Badge variant="outline" className="text-xs">Orden: {category.displayOrder ?? 0}</Badge>
1310 %                                     </CardTitle>
1311 %                                     {category.description && (
1312 %                                         <p className="text-sm text-muted-foreground mt-1">{category.description}</p>
1313 %                                     )}
1314 %                                 </div>
1315 %                                 <div className="flex items-center gap-1">
1316 %                                     <Button variant="ghost" size="icon" onClick={() => openCategoryDialog(category)} data-testid=...
1317 %                                     <Edit className="h-4 w-4" />
1318 %                                 </Button>
1319 %                                 <Button variant="ghost" size="icon" onClick={() => deleteCategoryMutation.mutate(category.id)...
1320 %                                 <Trash2 className="h-4 w-4 text-destructive" />
1321 %                                 </Button>
1322 %                             </div>
1323 %                         </CardHeader>
1324 %                         <CardContent>
1325 %                             <div className="grid gap-3 sm:grid-cols-2 lg:grid-cols-3">
1326 %                                 {menuItems
1327 %                                     .filter((item) => item.categoryId === category.id)
1328 %                                     .map((item) => (
1329 %                                         <div
1330 %                                             key={item.id}
1331 %                                             className="p-3 border rounded-md flex items-start justify-between gap-2"
1332 %                                             data-testid={ ` menu-item-${item.id} ` }
1333 %                                         >
1334 %                                             <div className="flex-1">
1335 %                                                 <div className="font-medium">{item.name}</div>
1336 %                                                 {item.description && (
1337 %                                                     <p className="text-sm text-muted-foreground line-clamp-2">{item.description}</p>
1338 %                                                 )}
1339 %                                                 {item.preparationTime && (
1340 %                                                     <div className="flex items-center gap-1 text-xs text-muted-foreground mt-1">
1341 %                                                         <Clock className="h-3 w-3" />
1342 %                                                         {item.preparationTime} min
1343 %                                                     </div>
1344 %                                                 )}
1345 %                                             </div>
1346 %                                             <div className="text-right flex flex-col items-end gap-1">
1347 %                                                 <div className="font-semibold">
1348 %                                                     ${parseFloat(item.price).toLocaleString("es-AR", { minimumFractionDigits: 2 })}
1349 %                                                 </div>
1350 %                                                 {item.isAvailable === "false" && (
1351 %                                                     <Badge variant="destructive" className="text-xs">No disponible</Badge>
1352 %                                                 )}
1353 %                                             </div>
1354 %                                             <div className="flex items-center gap-1">
1355 %                                                 <Button variant="ghost" size="icon" className="h-6 w-6" onClick={() => openRecipeDial...
1356 %                                                 <ChefHat className="h-3 w-3" />
1357 %                                             </Button>
1358 %                                                 <Button variant="ghost" size="icon" className="h-6 w-6" onClick={() => openMenuItemDi...
1359 %                                                 <Edit className="h-3 w-3" />
1360 %                                             </Button>
1361 %                                                 <Button variant="ghost" size="icon" className="h-6 w-6" onClick={() => deleteMenuItem...
1362 %                                                 <Trash2 className="h-3 w-3 text-destructive" />
1363 %                                             </Button>
1364 %                                         </div>
1365 %                                     </div>
1366 %                                 </div>
1367 %                             </div>
1368 %                         </CardContent>
1369 %                     </Card>
1370 %                 )}
1371 %             </div>
1372 %         )}
1373 %     </TabsContent>
1374 %
1375 %     { /* ===== RESERVATIONS TAB ===== */ }
1376 %     <TabsContent value="reservations" className="space-y-4">
1377 %         <div className="flex items-center justify-between gap-4 flex-wrap">
1378 %             <h2 className="text-lg font-semibold">Reservas de Mesa</h2>
1379 %             <div className="flex items-center gap-2 flex-wrap">
1380 %                 <div className="relative">
1381 %                     <Search className="absolute left-2 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
1382 %                     <Input
1383 %                         placeholder="Buscar por nombre..."
1384 %                         value={reservationSearch}
1385 %                         onChange={(e) => setReservationSearch(e.target.value)}
1386 %                         className="pl-8 w-48"
1387 %                         data-testid="input-reservation-search"
1388 %                     />
1389 %                 </div>
1390 %             </div>
1391 %             <Input
1392 %                 type="date"
```

client/src/pages/restaurant.tsx (cont.)

```
1393 %         value={reservationDate}
1394 %         onChange={(e) => setReservationDate(e.target.value)}
1395 %         className="w-40"
1396 %         data-testid="input-reservation-date-filter"
1397 %     />
1398 %     <Button
1399 %         variant="outline"
1400 %         size="sm"
1401 %         onClick={() => setReservationSortBy(reservationSortBy === "name" ? "time" : "name")}
1402 %         data-testid="button-sort-reservations"
1403 %     >
1404 %         <ArrowUpDown className="h-4 w-4 mr-1" />
1405 %         {reservationSortBy === "name" ? "A-Z" : "Hora"}
1406 %     </Button>
1407 %     <Button variant="outline" size="icon" onClick={() => setIsTimeSlotsDialogOpen(true)} data-testid="butto...
1408 %         <Settings className="h-4 w-4" />
1409 %     </Button>
1410 %     <Button onClick={() => setIsReservationDialogOpen(true)} data-testid="button-new-reservation">
1411 %         <Plus className="h-4 w-4 mr-2" />
1412 %         Nueva Reserva
1413 %     </Button>
1414 % </div>
1415 % </div>
1416 %
1417 % {filteredReservations.length === 0 ? (
1418 %     <Card>
1419 %         <CardContent className="flex flex-col items-center justify-center py-12 text-center">
1420 %             <CalendarDays className="h-12 w-12 text-muted-foreground mb-4" />
1421 %             <h3 className="text-lg font-semibold mb-2">Sin reservas para esta fecha</h3>
1422 %             <p className="text-muted-foreground mb-4">No hay reservas programadas</p>
1423 %             <Button onClick={() => setIsReservationDialogOpen(true)} data-testid="button-add-first-reservation">
1424 %                 <Plus className="h-4 w-4 mr-2" />
1425 %                 Crear Reserva
1426 %             </Button>
1427 %         </CardContent>
1428 %     </Card>
1429 % ) : (
1430 %     <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
1431 %         {filteredReservations.map((reservation) => {
1432 %             const table = tables.find(t => t.id === reservation.tableId);
1433 %             return (
1434 %                 <Card key={reservation.id} data-testid={`reservation-card-${reservation.id}`}>
1435 %                     <CardHeader className="pb-3">
1436 %                         <div className="flex items-center justify-between gap-2">
1437 %                             <CardTitle className="text-base">{reservation.guestName}</CardTitle>
1438 %                             <Badge className={reservationStatusColors[reservation.status]}>
1439 %                                 {reservationStatusLabel[reservation.status]}
1440 %                             </Badge>
1441 %                         </div>
1442 %                     </CardHeader>
1443 %                     <CardContent className="space-y-2">
1444 %                         <div className="flex items-center gap-2 text-sm">
1445 %                             <Clock className="h-4 w-4 text-muted-foreground" />
1446 %                             <span>{reservation.reservationTime}</span>
1447 %                         </div>
1448 %                         <div className="flex items-center gap-2 text-sm">
1449 %                             <MapPin className="h-4 w-4 text-muted-foreground" />
1450 %                             <span>Mesa {table?.tableNumber} || { "?"}</span>
1451 %                             {table?.hasWindow === "true" && (
1452 %                                 <Badge variant="outline" className="text-xs gap-1">
1453 %                                     <Eye className="h-3 w-3" /> Ventana
1454 %                                 </Badge>
1455 %                             )}
1456 %                         </div>
1457 %                         <div className="flex items-center gap-2 text-sm">
1458 %                             <Users className="h-4 w-4 text-muted-foreground" />
1459 %                             <span>{reservation.partySize} personas</span>
1460 %                         </div>
1461 %                         {reservation.guestPhone && (
1462 %                             <div className="flex items-center gap-2 text-sm text-muted-foreground">
1463 %                                 <Phone className="h-4 w-4" />
1464 %                                 <span>{reservation.guestPhone}</span>
1465 %                             </div>
1466 %                         )}
1467 %                         {reservation.notes && (
1468 %                             <p className="text-sm text-muted-foreground mt-2">{reservation.notes}</p>
1469 %                         )}
1470 %                     <div className="flex items-center gap-2 pt-2">
1471 %                         {reservation.status === "pending" && (
1472 %                             <Button
1473 %                                 size="sm"
1474 %                                 onClick={() => updateReservationMutation.mutate({ id: reservation.id, data: { status: "co...
1475 %                                 data-testid={`button-confirm-${reservation.id}`}
1476 %                             >
1477 %                                 <Check className="h-4 w-4 mr-1" />
1478 %                                 Confirmar
1479 %                             </Button>
1480 %                         )}
1481 %                     </div>
1482 %                     {(reservation.status === "pending" || reservation.status === "confirmed") && (
1483 %                         <Button
1484 %                             size="sm"
1485 %                             variant="outline"
1486 %                             onClick={() => updateReservationMutation.mutate({ id: reservation.id, data: { status: "se...
```

client/src/pages/restaurant.tsx (cont.)

```
1486 %         data-testid={`button-seat-${reservation.id}`}
1487 %     >
1488 %         Sentar
1489 %     </Button>
1490 % })
1491 % {reservation.status !== "cancelled" && reservation.status !== "completed" && (
1492 %     <Button
1493 %         size="sm"
1494 %         variant="ghost"
1495 %         onClick={() => updateReservationMutation.mutate({ id: reservation.id, data: { status: "ca...
1496 %         data-testid={`button-cancel-${reservation.id}`}
1497 %     >
1498 %         <XCircle className="h-4 w-4" />
1499 %     </Button>
1500 % })
1501 % <Button
1502 %     size="icon"
1503 %     variant="ghost"
1504 %     onClick={() => deleteReservationMutation.mutate(reservation.id)}
1505 %     data-testid={`button-delete-${reservation.id}`}
1506 % >
1507 %     <Trash2 className="h-4 w-4" />
1508 % </Button>
1509 % </div>
1510 % </CardContent>
1511 % </Card>
1512 % );
1513 % }}}
1514 % </div>
1515 % }}
1516 % </TabsContent>
1517 %
1518 % {/ * ===== RECIPES & COSTS TAB ===== */}
1519 % <TabsContent value="recipes" className="space-y-4">
1520 %     <div className="flex items-center justify-between gap-4 flex-wrap">
1521 %         <div>
1522 %             <h2 className="text-lg font-semibold">Recetas y Costos</h2>
1523 %             <p className="text-sm text-muted-foreground">Carga recetas por plato para calcular costos y margenes</p>
1524 %         </div>
1525 %     </div>
1526 %
1527 %     {menuItems.length === 0 ? (
1528 %         <Card>
1529 %             <CardContent className="flex flex-col items-center justify-center py-12 text-center">
1530 %                 <ChefHat className="h-12 w-12 text-muted-foreground mb-4" />
1531 %                 <h3 className="text-lg font-semibold mb-2">Sin platos configurados</h3>
1532 %                 <p className="text-muted-foreground">Primero crea platos en la seccion Menu</p>
1533 %             </CardContent>
1534 %         </Card>
1535 %     ) : (
1536 %         <Card>
1537 %             <CardContent className="pt-6">
1538 %                 <Table>
1539 %                     <TableHeader>
1540 %                         <TableRow>
1541 %                             <TableHead>Plato</TableHead>
1542 %                             <TableHead>Categoria</TableHead>
1543 %                             <TableHead className="text-right">Precio Venta</TableHead>
1544 %                             <TableHead className="text-right">Costo</TableHead>
1545 %                             <TableHead className="text-right">Margen</TableHead>
1546 %                             <TableHead className="text-right">% Margen</TableHead>
1547 %                             <TableHead className="w-20"></TableHead>
1548 %                         </TableRow>
1549 %                     </TableHeader>
1550 %                     <TableBody>
1551 %                         {menuItems.map((item) => {
1552 %                             const recipe = recipes.find(r => r.menuItemId === item.id);
1553 %                             const cost = recipe?.ingredients.reduce((sum, ing) => sum + parseFloat(ing.quantity) * parseFlo...
1554 %                             const price = parseFloat(item.price);
1555 %                             const margin = price - cost;
1556 %                             const marginPct = price > 0 ? (margin / price) * 100 : 0;
1557 %                             const cat = menuCategories.find(c => c.id === item.categoryId);
1558 %
1559 %                             return (
1560 %                                 <TableRow key={item.id} data-testid={`recipe-row-${item.id}`}>
1561 %                                     <TableCell className="font-medium">{item.name}</TableCell>
1562 %                                     <TableCell>{cat?.name || "-"}</TableCell>
1563 %                                     <TableCell className="text-right">
1564 %                                         ${price.toLocaleString("es-AR", { minimumFractionDigits: 2 })}
1565 %                                     </TableCell>
1566 %                                     <TableCell className="text-right">
1567 %                                         {recipe ? `${cost.toLocaleString("es-AR", { minimumFractionDigits: 2 })}` : "-"}
1568 %                                     </TableCell>
1569 %                                     <TableCell className="text-right">
1570 %                                         {recipe ? (
1571 %                                             <span className={margin >= 0 ? "text-green-600" : "text-red-600"}>
1572 %                                                 ${margin.toLocaleString("es-AR", { minimumFractionDigits: 2 })}
1573 %                                             </span>
1574 %                                         ) : "-"}
1575 %                                     </TableCell>
1576 %                                     <TableCell className="text-right">
1577 %                                         {recipe ? (
1578 %                                             <Badge variant={marginPct >= 50 ? "default" : marginPct >= 30 ? "secondary" : "destruct...
```


client/src/pages/restaurant.tsx (cont.)

```
1579 %             {marginPct.toFixed(1)}%
1580 %         </Badge>
1581 %         ) : "-"}
1582 %     </TableCell>
1583 %     <TableCell>
1584 %         <Button variant="outline" size="sm" onClick={() => openRecipeDialog(item)} data-testid={`...
1585 %         <BookOpen className="h-4 w-4 mr-1" />
1586 %         Receta
1587 %     </Button>
1588 % </TableCell>
1589 % </TableRow>
1590 % </Table>
1591 % </TableBody>
1592 % </Table>
1593 % </CardContent>
1594 % </Card>
1595 % </Card>
1596 % </Card>
1597 % </TabsContent>
1598 % </Tabs>
1599 %
1600 % { /* ===== DIALOGS ===== */}
1601 %
1602 % { /* New Order Dialog (table-based) */}
1603 % <Dialog open={isNewOrderDialogOpen} onOpenChange={setIsNewOrderDialogOpen}>
1604 %   <DialogContent>
1605 %     <DialogHeader>
1606 %       <DialogTitle>Nuevo Pedido - Mesa {selectedTable?.tableNumber}</DialogTitle>
1607 %     </DialogHeader>
1608 %     <div className="space-y-4">
1609 %       <div className="space-y-2">
1610 %         <Label htmlFor="waiter-name">Mozo *</Label>
1611 %         <Input
1612 %           id="waiter-name"
1613 %           value={newWaiterName}
1614 %           onChange={(e) => setNewWaiterName(e.target.value)}
1615 %           placeholder="Nombre del mozo"
1616 %           data-testid="input-waiter-name"
1617 %         />
1618 %       </div>
1619 %       <div className="space-y-2">
1620 %         <Label htmlFor="covers">Cantidad de comensales</Label>
1621 %         <Input
1622 %           id="covers"
1623 %           type="number"
1624 %           min={1}
1625 %           value={newCovers}
1626 %           onChange={(e) => setNewCovers(parseInt(e.target.value) || 1)}
1627 %           data-testid="input-covers"
1628 %         />
1629 %         <p className="text-xs text-muted-foreground">Capacidad de la mesa: {selectedTable?.capacity}</p>
1630 %       </div>
1631 %     </div>
1632 %     <DialogFooter>
1633 %       <Button variant="outline" onClick={() => setIsNewOrderDialogOpen(false)}>
1634 %         Cancelar
1635 %       </Button>
1636 %       <Button
1637 %         onClick={() => {
1638 %           if (selectedTable && newWaiterName.trim()) {
1639 %             createOrderMutation.mutate({ tableId: selectedTable.id, covers: newCovers, waiterName: newWaiterNam...
1640 %           } else {
1641 %             toast({ title: "Mozo requerido", description: "Ingrese el nombre del mozo", variant: "destructive" });
1642 %           }
1643 %         }}
1644 %         disabled={createOrderMutation.isPending}
1645 %         data-testid="button-create-order"
1646 %       >
1647 %         {createOrderMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1648 %         Crear Pedido
1649 %       </Button>
1650 %     </DialogFooter>
1651 %   </DialogContent>
1652 % </Dialog>
1653 %
1654 % { /* Direct Order Dialog (tableless areas) */}
1655 % <Dialog open={isDirectOrderDialogOpen} onOpenChange={setIsDirectOrderDialogOpen}>
1656 %   <DialogContent>
1657 %     <DialogHeader>
1658 %       <DialogTitle>Nueva Orden - {areas.find(a => a.id === directOrderAreaId)?.name}</DialogTitle>
1659 %     </DialogHeader>
1660 %     <div className="space-y-4">
1661 %       <div className="space-y-2">
1662 %         <Label htmlFor="order-label">Etiqueta de orden *</Label>
1663 %         <Input
1664 %           id="order-label"
1665 %           value={newOrderLabel}
1666 %           onChange={(e) => setNewOrderLabel(e.target.value)}
1667 %           placeholder="Ej: Hab. 305, Mesa Solarium 2"
1668 %           data-testid="input-order-label"
1669 %         />
1670 %       </div>
1671 %       <div className="space-y-2">
```

client/src/pages/restaurant.tsx (cont.)

```
1672 %         <Label htmlFor="direct-waiter">Mozo *</Label>
1673 %         <Input
1674 %           id="direct-waiter"
1675 %           value={newWaiterName}
1676 %           onChange={(e) => setNewWaiterName(e.target.value)}
1677 %           placeholder="Nombre del mozo"
1678 %           data-testid="input-direct-waiter"
1679 %         />
1680 %       </div>
1681 %       <div className="space-y-2">
1682 %         <Label htmlFor="direct-covers">Comensales (opcional)</Label>
1683 %         <Input
1684 %           id="direct-covers"
1685 %           type="number"
1686 %           min={1}
1687 %           value={newCovers}
1688 %           onChange={(e) => setNewCovers(parseInt(e.target.value) || 1)}
1689 %           data-testid="input-direct-covers"
1690 %         />
1691 %       </div>
1692 %     </div>
1693 %     <DialogFooter>
1694 %       <Button variant="outline" onClick={() => setIsDirectOrderDialogOpen(false)}>
1695 %         Cancelar
1696 %       </Button>
1697 %       <Button
1698 %         onClick={() => {
1699 %           if (newOrderLabel.trim() && newWaiterName.trim()) {
1700 %             createOrderMutation.mutate({
1701 %               areaId: directOrderAreaId,
1702 %               covers: newCovers,
1703 %               waiterName: newWaiterName.trim(),
1704 %               orderLabel: newOrderLabel.trim(),
1705 %             });
1706 %           } else {
1707 %             toast({ title: "Campos requeridos", description: "Ingreso etiqueta y mozo", variant: "destructive" });
1708 %           }
1709 %         }}
1710 %         disabled={createOrderMutation.isPending}
1711 %         data-testid="button-create-direct-order"
1712 %       >
1713 %         {createOrderMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1714 %       Crear Orden
1715 %     </Button>
1716 %   </DialogFooter>
1717 % </DialogContent>
1718 % </Dialog>
1719 %
1720 %   /* Order Dialog (menu/folio/delete views) */
1721 %   <Dialog open={isOrderDialogOpen} onOpenChange={(open) => {
1722 %     setIsOrderDialogOpen(open);
1723 %     if (!open) {
1724 %       setPendingItem(null);
1725 %       setItemNotes("");
1726 %       setSelectedCategory(null);
1727 %     }
1728 %   }}>
1729 %     <DialogContent className="max-w-3xl max-h-[90vh] flex flex-col">
1730 %       <DialogHeader>
1731 %         <div className="flex items-center justify-between gap-4">
1732 %           <div>
1733 %             <DialogTitle>
1734 %               {getUpdatedOrder()?.orderLabel || getUpdatedOrder()?.orderNumber} - {getUpdatedOrder()?.tableId ? `...` : ` `}
1735 %             </DialogTitle>
1736 %             <p className="text-xs text-muted-foreground mt-1">
1737 %               Mozo: {getUpdatedOrder()?.waiterName || "-"} | Abierto: {currentOrder ? new Date(currentOrder.opene...
1738 %               {(getUpdatedOrder()?.activeCourse || 1) > 1 && ` | Curso: ${courseLabels[getUpdatedOrder()?.activeC...
1739 %             </p>
1740 %           </div>
1741 %           <div className="flex items-center gap-1">
1742 %             {getUpdatedOrder() && (getUpdatedOrder()?.activeCourse || 1) < 3 && getOrderItems().some(i => i.cours...
1743 %             <Button
1744 %               variant="outline"
1745 %               size="sm"
1746 %               onClick={() => {
1747 %                 if (currentOrder) advanceCourseMutation.mutate(currentOrder.id);
1748 %               }}
1749 %               disabled={advanceCourseMutation.isPending}
1750 %               data-testid="button-advance-course"
1751 %             >
1752 %               {advanceCourseMutation.isPending ? <Loader2 className="h-4 w-4 animate-spin" /> : <ArrowUpDown cl...
1753 %               Sig. Curso
1754 %             </Button>
1755 %           </div>
1756 %           <Button
1757 %             variant="ghost"
1758 %             size="sm"
1759 %             onClick={() => setOrderView("folio")}
1760 %             className={orderView === "folio" ? "bg-muted" : ""}
1761 %             data-testid="button-view-folio"
1762 %           >
1763 %             <CircleDollarSign className="h-4 w-4 mr-1" />
1764 %             Folio
```

client/src/pages/restaurant.tsx (cont.)

```
1765 %         </Button>
1766 %     </div>
1767 % </div>
1768 % </DialogHeader>
1769 %
1770 % {orderView === "folio" && (
1771 %   <div className="flex-1 overflow-y-auto space-y-4">
1772 %     <h3 className="font-semibold text-lg">Resumen de Consumos</h3>
1773 %     {getOrderItems().length === 0 ? (
1774 %       <p className="text-muted-foreground text-center py-8">No hay items en este pedido</p>
1775 %     ) : (
1776 %       <div className="space-y-2">
1777 %         {[1, 2, 3].map(course => {
1778 %           const courseItems = getOrderItems().filter(i => (i.course || 1) === course);
1779 %           if (courseItems.length === 0) return null;
1780 %           return (
1781 %             <div key={course}>
1782 %               <div className="flex items-center gap-2 mb-1">
1783 %                 <span className="text-xs font-semibold uppercase text-muted-foreground">{courseLabels[cours...
1784 %                 {course === (getUpdatedOrder()?.activeCourse || 1) && (
1785 %                   <Badge variant="default" className="text-[10px] h-4">Activo</Badge>
1786 %                 )}
1787 %               </div>
1788 %               {courseItems.map((item) => (
1789 %                 <div key={item.id} className={`flex items-center justify-between p-3 border rounded-md mb-1...
1790 %                 <div className="flex items-center gap-2">
1791 %                   <span className="font-medium">{item.menuItem?.name || "Item"}</span>
1792 %                   <span className="text-muted-foreground">x{item.quantity}</span>
1793 %                   {item.status === "waiting_course" && <Badge variant="outline" className="text-[10px]">E...
1794 %                 </div>
1795 %                 <span className="font-semibold">
1796 %                   ${parseFloat(item.subtotal).toLocaleString("es-AR", { minimumFractionDigits: 2 })}
1797 %                 </span>
1798 %               </div>
1799 %             )}
1800 %           </div>
1801 %         )}
1802 %       </div>
1803 %     <div className="pt-4 border-t flex items-center justify-between text-lg font-bold">
1804 %       <span>Total:</span>
1805 %       <span>
1806 %         ${parseFloat(getUpdatedOrder()?.total || "0").toLocaleString("es-AR", { minimumFractionDigits: ...
1807 %       </span>
1808 %     </div>
1809 %   </div>
1810 % )}
1811 %   <Button
1812 %     variant="outline"
1813 %     onClick={() => setOrderView("menu")}
1814 %     className="w-full"
1815 %     data-testid="button-back-to-menu"
1816 %   >
1817 %     Volver al Menu
1818 %   </Button>
1819 % </div>
1820 % )}
1821 %
1822 % {orderView === "delete" && (
1823 %   <div className="flex-1 overflow-y-auto space-y-4">
1824 %     <h3 className="font-semibold text-lg">Eliminar Items</h3>
1825 %     {getOrderItems().length === 0 ? (
1826 %       <p className="text-muted-foreground text-center py-8">No hay items para eliminar</p>
1827 %     ) : (
1828 %       <div className="space-y-2">
1829 %         {getOrderItems().map((item) => (
1830 %           <div key={item.id} className="flex items-center justify-between p-3 border rounded-md">
1831 %             <div>
1832 %               <span className="font-medium">{item.menuItem?.name || "Item"}</span>
1833 %               <span className="text-muted-foreground ml-2">x{item.quantity}</span>
1834 %             </div>
1835 %             <Button
1836 %               size="sm"
1837 %               variant="destructive"
1838 %               onClick={() => {
1839 %                 if (currentOrder) {
1840 %                   deleteItemMutation.mutate({ orderId: currentOrder.id, itemId: item.id });
1841 %                 }
1842 %               }}
1843 %               disabled={deleteItemMutation.isPending}
1844 %               data-testid={`button-delete-item-${item.id}`}
1845 %             >
1846 %               <X className="h-4 w-4" />
1847 %             </Button>
1848 %           </div>
1849 %         )}
1850 %       </div>
1851 %     <Button
1852 %       variant="outline"
1853 %       onClick={() => setOrderView("menu")}
1854 %       className="w-full"
1855 %       data-testid="button-back-from-delete"
1856 %     >
1857 %       </div>
```

client/src/pages/restaurant.tsx (cont.)

```
1858 %         Volver al Menu
1859 %       </Button>
1860 %     </div>
1861 %   )}
1862 %
1863 %   {orderView === "menu" && !pendingItem && (
1864 %     <div className="flex-1 overflow-y-auto space-y-4">
1865 %       <div className="flex flex-wrap gap-2">
1866 %         {sortedCategories.map((cat) => (
1867 %           <Button
1868 %             key={cat.id}
1869 %             variant={selectedCategory === cat.id ? "default" : "outline"}
1870 %             size="sm"
1871 %             onClick={() => setSelectedCategory(selectedCategory === cat.id ? null : cat.id)}
1872 %             data-testid={`button-category-${cat.id}`}
1873 %           >
1874 %             {cat.name}
1875 %           </Button>
1876 %         )}}
1877 %       <Button
1878 %         variant="outline"
1879 %         size="sm"
1880 %         onClick={() => setOrderView("delete")}
1881 %         className="text-destructive border-destructive"
1882 %         data-testid="button-delete-mode"
1883 %       >
1884 %         <X className="h-4 w-4 mr-1" />
1885 %         Borrar Item
1886 %       </Button>
1887 %     </div>
1888 %
1889 %     {selectedCategory ? (
1890 %       <div className="grid gap-2 sm:grid-cols-2">
1891 %         {menuItems
1892 %           .filter((item) => item.categoryId === selectedCategory && item.isAvailable !== "false")
1893 %           .map((item) => (
1894 %             <button
1895 %               key={item.id}
1896 %               className="p-3 border rounded-md text-left hover-elevate flex items-center justify-between"
1897 %               onClick={() => setPendingItem(item)}
1898 %               data-testid={`select-item-${item.id}`}
1899 %             >
1900 %               <span className="font-medium">{item.name}</span>
1901 %               <span className="text-muted-foreground">
1902 %                 ${parseFloat(item.price).toLocaleString("es-AR", { minimumFractionDigits: 2 })}
1903 %               </span>
1904 %             </button>
1905 %           )}}
1906 %       </div>
1907 %     ) : (
1908 %       <div className="text-center py-8 text-muted-foreground">
1909 %         Selecciona una categoria para ver los platos
1910 %       </div>
1911 %     )}
1912 %   </div>
1913 % )}
1914 %
1915 %   {orderView === "menu" && pendingItem && (
1916 %     <div className="flex-1 space-y-4">
1917 %       <div className="p-4 border rounded-md bg-muted/30">
1918 %         <h3 className="font-semibold text-lg mb-1">{pendingItem.name}</h3>
1919 %         <p className="text-muted-foreground">
1920 %           ${parseFloat(pendingItem.price).toLocaleString("es-AR", { minimumFractionDigits: 2 })}
1921 %         </p>
1922 %         {pendingItem.description && (
1923 %           <p className="text-sm text-muted-foreground mt-2">{pendingItem.description}</p>
1924 %         )}
1925 %       </div>
1926 %       <div className="space-y-2">
1927 %         <Label>Curso</Label>
1928 %         <div className="flex gap-2">
1929 %           {[1, 2, 3].map(c => (
1930 %             <Button
1931 %               key={c}
1932 %               variant={itemCourse === c ? "default" : "outline"}
1933 %               size="sm"
1934 %               onClick={() => setItemCourse(c)}
1935 %               data-testid={`button-course-${c}`}
1936 %             >
1937 %               {courseLabels[c]}
1938 %             </Button>
1939 %           )}}
1940 %         </div>
1941 %       </div>
1942 %       <div className="space-y-2">
1943 %         <Label htmlFor="item-notes">Observaciones (opcional)</Label>
1944 %         <Textarea
1945 %           id="item-notes"
1946 %           value={itemNotes}
1947 %           onChange={(e) => setItemNotes(e.target.value)}
1948 %           placeholder="Ej: sin sal, termino medio, etc."
1949 %           rows={2}
1950 %           data-testid="input-item-notes"
```

client/src/pages/restaurant.tsx (cont.)

```
1951 %         />
1952 %     </div>
1953 %     <p className="font-medium">Agregar este item?</p>
1954 %     <div className="flex gap-2">
1955 %         <Button
1956 %             className="flex-1"
1957 %             onClick={handleConfirmItem}
1958 %             disabled={addItemMutation.isPending}
1959 %             data-testid="button-confirm-item"
1960 %         >
1961 %             {addItemMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1962 %             Si
1963 %         </Button>
1964 %         <Button
1965 %             variant="outline"
1966 %             className="flex-1"
1967 %             onClick={handleCancelItem}
1968 %             data-testid="button-cancel-item"
1969 %         >
1970 %             No
1971 %         </Button>
1972 %     </div>
1973 % </div>
1974 % }}
1975 %
1976 % <DialogFooter className="flex-col sm:flex-row gap-2 border-t pt-4">
1977 %     <Button
1978 %         variant="destructive"
1979 %         size="lg"
1980 %         className="w-full sm:w-auto"
1981 %         onClick={() => {
1982 %             setIsOrderDialogOpen(false);
1983 %             setCloseReceiptType("ticket");
1984 %             setClosePaymentMethod("efectivo");
1985 %             setIsCloseDialogOpen(true);
1986 %         }}
1987 %         data-testid="button-close-table"
1988 %     >
1989 %         <CreditCard className="h-5 w-5 mr-2" />
1990 %         Cerrar Mesa
1991 %     </Button>
1992 %     <Button
1993 %         onClick={() => setIsOrderDialogOpen(false)}
1994 %         className="w-full sm:w-auto"
1995 %         data-testid="button-done"
1996 %     >
1997 %         Listo
1998 %     </Button>
1999 % </DialogFooter>
2000 % </DialogContent>
2001 % </Dialog>
2002 %
2003 % /* Close Order Dialog with Receipt Type, Payment Method, and Split */
2004 % <Dialog open={isCloseDialogOpen} onChange={(open) => { setIsCloseDialogOpen(open); if (!open) setIsSplitMod...
2005 %     <DialogContent className="max-w-lg">
2006 %         <DialogHeader>
2007 %             <DialogTitle className="flex items-center gap-2">
2008 %                 <Receipt className="h-5 w-5" />
2009 %                 Cerrar - {getUpdatedOrder()?.orderLabel || currentOrder?.orderNumber}
2010 %             </DialogTitle>
2011 %         </DialogHeader>
2012 %         <div className="space-y-4">
2013 %             <h3 className="font-semibold">Resumen de Consumos</h3>
2014 %             {getOrderItems().length === 0 ? (
2015 %                 <p className="text-muted-foreground text-center py-4">No hay items en este pedido</p>
2016 %             ) : (
2017 %                 <div className="space-y-2 max-h-40 overflow-y-auto">
2018 %                     {getOrderItems().map((item) => (
2019 %                         <div key={item.id} className="flex items-center justify-between py-2 border-b">
2020 %                             <div>
2021 %                                 <span>{item.menuItem?.name || "Item"}</span>
2022 %                                 <span className="text-muted-foreground ml-2">x{item.quantity}</span>
2023 %                                 {item.course && item.course > 1 && <Badge variant="outline" className="ml-1 text-[10px]">{cours...
2024 %                             </div>
2025 %                             <span>
2026 %                                 ${parseFloat(item.subtotal).toLocaleString("es-AR", { minimumFractionDigits: 2 })}
2027 %                             </span>
2028 %                         </div>
2029 %                     ))}
2030 %                 </div>
2031 %             )}
2032 %         </div>
2033 %         <div className="pt-2 border-t space-y-1">
2034 %             <div className="flex justify-between text-sm">
2035 %                 <span>Subtotal:</span>
2036 %                 <span>${parseFloat(getUpdatedOrder()?.subtotal || "0").toLocaleString("es-AR", { minimumFractionDigit...
2037 %             </div>
2038 %             <div className="flex justify-between text-sm">
2039 %                 <span>IVA (21%):</span>
2040 %                 <span>${parseFloat(getUpdatedOrder()?.tax || "0").toLocaleString("es-AR", { minimumFractionDigits: 2 ...
2041 %             </div>
2042 %             <div className="flex justify-between text-xl font-bold pt-2">
2043 %                 <span>Total:</span>
2044 %                 <span>${parseFloat(getUpdatedOrder()?.total || "0").toLocaleString("es-AR", { minimumFractionDigits: ...
```

client/src/pages/restaurant.tsx (cont.)

```
2044 %         </div>
2045 %     </div>
2046 %
2047 %     {!isSplitMode ? (
2048 %     <
2049 %         <div className="grid grid-cols-2 gap-4 pt-4 border-t">
2050 %             <div className="space-y-2">
2051 %                 <Label>Tipo de Comprobante</Label>
2052 %                 <Select value={closeReceiptType} onValueChange={setCloseReceiptType}>
2053 %                     <SelectTrigger data-testid="select-receipt-type">
2054 %                         <SelectValue />
2055 %                     </SelectTrigger>
2056 %                     <SelectContent>
2057 %                         {Object.entries(receiptTypeLabels).map(([value, label]) => (
2058 %                             <SelectItem key={value} value={value}>{label}</SelectItem>
2059 %                         ))}
2060 %                     </SelectContent>
2061 %                 </Select>
2062 %             </div>
2063 %             <div className="space-y-2">
2064 %                 <Label>Forma de Pago</Label>
2065 %                 <Select value={closePaymentMethod} onValueChange={setClosePaymentMethod}>
2066 %                     <SelectTrigger data-testid="select-payment-method">
2067 %                         <SelectValue />
2068 %                     </SelectTrigger>
2069 %                     <SelectContent>
2070 %                         {Object.entries(paymentMethodLabels).map(([value, label]) => (
2071 %                             <SelectItem key={value} value={value}>{label}</SelectItem>
2072 %                         ))}
2073 %                     </SelectContent>
2074 %                 </Select>
2075 %             </div>
2076 %         </div>
2077 %
2078 %         {closePaymentMethod === "cuenta_habitacion" && (
2079 %             <div className="p-3 bg-blue-500/10 border border-blue-500/30 rounded-md text-sm text-blue-700 dark:...
2080 %                 El consumo se cargara a la cuenta de la habitacion del huesped
2081 %             </div>
2082 %         )}
2083 %
2084 %         <div className="pt-2 border-t">
2085 %             <Button
2086 %                 variant="outline"
2087 %                 size="sm"
2088 %                 className="w-full"
2089 %                 onClick={() => setIsSplitMode(true)}
2090 %                 data-testid="button-split-bill"
2091 %             >
2092 %                 <Banknote className="h-4 w-4 mr-2" />
2093 %                 Dividir Cuenta
2094 %             </Button>
2095 %         </div>
2096 %     </>
2097 % ) : (
2098 %     <div className="pt-4 border-t space-y-4">
2099 %         <h4 className="font-semibold flex items-center gap-2">
2100 %             <Banknote className="h-4 w-4" />
2101 %             Dividir Cuenta
2102 %         </h4>
2103 %         {{{() => {
2104 %             const updatedOrder = orders?.find((o: RestaurantOrder) => o.id === currentOrder?.id);
2105 %             const splits: OrderSplit[] = (updatedOrder as any)?.splits || [];
2106 %             if (splits.length === 0) {
2107 %                 return (
2108 %                     <div className="space-y-3">
2109 %                         <div className="space-y-2">
2110 %                             <Label>Cantidad de partes</Label>
2111 %                             <div className="flex gap-2">
2112 %                                 {[2, 3, 4].map(n => (
2113 %                                     <Button
2114 %                                         key={n}
2115 %                                         variant={splitParts === n ? "default" : "outline"}
2116 %                                         size="sm"
2117 %                                         onClick={() => setSplitParts(n)}
2118 %                                         data-testid={`button-split-${n}`}
2119 %                                     >
2120 %                                         {n} partes
2121 %                                     </Button>
2122 %                                 ))}
2123 %                             <Input
2124 %                                 type="number"
2125 %                                 min={2}
2126 %                                 max={10}
2127 %                                 value={splitParts}
2128 %                                 onChange={(e) => setSplitParts(Math.max(2, parseInt(e.target.value) || 2)}}
2129 %                                 className="w-20"
2130 %                                 data-testid="input-split-parts"
2131 %                             </div>
2132 %                         </div>
2133 %                     </div>
2134 %                     <div className="p-3 bg-muted rounded-md text-sm">
2135 %                         Cada parte: <strong>${(parseFloat(getUpdatedOrder()?.total || "0") / splitParts).toLocaleSt...
2136 %                     </div>
```

client/src/pages/restaurant.tsx (cont.)

```
2137 %         <div className="flex gap-2">
2138 %           <Button
2139 %             variant="outline"
2140 %             onClick={() => setIsSplitMode(false)}
2141 %             className="flex-1"
2142 %           >
2143 %             Cancelar
2144 %           </Button>
2145 %           <Button
2146 %             onClick={() => {
2147 %               if (currentOrder) createSplitMutation.mutate({ orderId: currentOrder.id, parts: splitPa...
2148 %             }}
2149 %             disabled={createSplitMutation.isPending}
2150 %             className="flex-1"
2151 %             data-testid="button-confirm-split"
2152 %           >
2153 %             {createSplitMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
2154 %             Dividir
2155 %           </Button>
2156 %         </div>
2157 %       </div>
2158 %     );
2159 %   }
2160 %   return (
2161 %     <div className="space-y-3">
2162 %       {splits.map((split) => (
2163 %         <div key={split.id} className={`p-3 border rounded-md ${split.isPaid === "true" ? "bg-green-5...
2164 %         <div className="flex items-center justify-between mb-2">
2165 %           <span className="font-medium">Parte {split.splitNumber}</span>
2166 %           <span className="font-bold">${parseFloat(split.amount).toLocaleString("es-AR", { minimumF...
2167 %         </div>
2168 %         {split.isPaid === "true" ? (
2169 %           <Badge variant="default" className="bg-green-600">Pagado - {paymentMethodLabels[split.met...
2170 %         ) : (
2171 %           <div className="flex gap-2 items-end">
2172 %             <div className="flex-1 space-y-1">
2173 %               <Select value={splitPayMethods[split.id] || "efectivo"} onChange={(v) => setSpli...
2174 %               <SelectTrigger className="h-8" data-testid={`select-split-method-${split.splitNumb...
2175 %                 <SelectValue />
2176 %               </SelectTrigger>
2177 %               <SelectContent>
2178 %                 {Object.entries(paymentMethodLabels).map(([v, l]) => (
2179 %                   <SelectItem key={v} value={v}>{l}</SelectItem>
2180 %                 ))}
2181 %               </SelectContent>
2182 %             </Select>
2183 %           </div>
2184 %           <div className="flex-1 space-y-1">
2185 %             <Select value={splitReceiptTypes[split.id] || "ticket"} onChange={(v) => setSpli...
2186 %             <SelectTrigger className="h-8" data-testid={`select-split-receipt-${split.splitNumb...
2187 %               <SelectValue />
2188 %             </SelectTrigger>
2189 %             <SelectContent>
2190 %               {Object.entries(receiptTypeLabels).map(([v, l]) => (
2191 %                 <SelectItem key={v} value={v}>{l}</SelectItem>
2192 %               ))}
2193 %             </SelectContent>
2194 %           </Select>
2195 %         </div>
2196 %         <Button
2197 %           size="sm"
2198 %           onClick={() => {
2199 %             if (currentOrder) {
2200 %               paySplitMutation.mutate({
2201 %                 orderId: currentOrder.id,
2202 %                 splitId: split.id,
2203 %                 method: splitPayMethods[split.id] || "efectivo",
2204 %                 receiptType: splitReceiptTypes[split.id] || "ticket",
2205 %               });
2206 %             }
2207 %           }}
2208 %         disabled={paySplitMutation.isPending}
2209 %         data-testid={`button-pay-split-${split.splitNumber}`}
2210 %       >
2211 %         Cobrar
2212 %       </Button>
2213 %     </div>
2214 %   )}
2215 % </div>
2216 % ))}
2217 % <div className="flex gap-2">
2218 %   <Button
2219 %     variant="outline"
2220 %     size="sm"
2221 %     onClick={() => {
2222 %       if (currentOrder) cancelSplitMutation.mutate(currentOrder.id);
2223 %     }}
2224 %     disabled={cancelSplitMutation.isPending || splits.some(s => s.isPaid === "true")}
2225 %     data-testid="button-cancel-split"
2226 %   >
2227 %     Cancelar División
2228 %   </Button>
2229 % </div>
```

client/src/pages/restaurant.tsx (cont.)

```
2230 %         </div>
2231 %     );
2232 % }}())}
2233 % </div>
2234 % }}
2235 % </div>
2236 % <DialogFooter className="flex-col sm:flex-row gap-2">
2237 %     <Button
2238 %         variant="outline"
2239 %         onClick={() => { setIsCloseDialogOpen(false); setIsSplitMode(false); }}
2240 %         className="w-full sm:w-auto"
2241 %     >
2242 %         Volver
2243 %     </Button>
2244 %     {!isSplitMode && (() => {
2245 %         const updatedOrder = orders?.find((o: RestaurantOrder) => o.id === currentOrder?.id);
2246 %         const hasSplits = ((updatedOrder as any)?.splits || []).length > 0;
2247 %         return !hasSplits ? (
2248 %             <Button
2249 %                 variant="destructive"
2250 %                 onClick={() => {
2251 %                     if (currentOrder) {
2252 %                         closeOrderMutation.mutate({
2253 %                             orderId: currentOrder.id,
2254 %                             receiptType: closeReceiptType,
2255 %                             paymentMethod: closePaymentMethod,
2256 %                         });
2257 %                     }
2258 %                 }}
2259 %                 disabled={closeOrderMutation.isPending}
2260 %                 className="w-full sm:w-auto"
2261 %                 data-testid="button-confirm-close"
2262 %             >
2263 %                 {closeOrderMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
2264 %                 Confirmar Cierre
2265 %             </Button>
2266 %         ) : (
2267 %             <Button
2268 %                 variant="outline"
2269 %                 onClick={() => setIsSplitMode(true)}
2270 %                 className="w-full sm:w-auto"
2271 %                 data-testid="button-view-splits"
2272 %             >
2273 %                 Ver División (((updatedOrder as any)?.splits || []).filter((s: OrderSplit) => s.isPaid === "true")...
2274 %             </Button>
2275 %         );
2276 %     }}())}
2277 % </DialogFooter>
2278 % </DialogContent>
2279 % </Dialog>
2280 %
2281 % /* Reservation Dialog */
2282 % <Dialog open={isReservationDialogOpen} onOpenChange={setIsReservationDialogOpen}>
2283 %     <DialogContent className="max-w-md">
2284 %         <DialogHeader>
2285 %             <DialogTitle>Nueva Reserva</DialogTitle>
2286 %         </DialogHeader>
2287 %         <Form {...reservationForm}>
2288 %             <form onSubmit={reservationForm.handleSubmit((data) => createReservationMutation.mutate(data))} className...
2289 %                 <FormField
2290 %                     control={reservationForm.control}
2291 %                     name="guestName"
2292 %                     render={({ field }) => (
2293 %                         <FormItem>
2294 %                             <FormLabel>Nombre del huesped *</FormLabel>
2295 %                             <FormControl>
2296 %                                 <Input {...field} placeholder="Nombre completo" data-testid="input-guest-name" />
2297 %                             </FormControl>
2298 %                             <FormMessage />
2299 %                         </FormItem>
2300 %                     )}
2301 %                 />
2302 %                 <div className="grid grid-cols-2 gap-4">
2303 %                     <FormField
2304 %                         control={reservationForm.control}
2305 %                         name="guestPhone"
2306 %                         render={({ field }) => (
2307 %                             <FormItem>
2308 %                                 <FormLabel>Telefono</FormLabel>
2309 %                                 <FormControl>
2310 %                                     <Input {...field} placeholder="+54 11 xxxx-xxxx" data-testid="input-guest-phone" />
2311 %                                 </FormControl>
2312 %                                 <FormMessage />
2313 %                             </FormItem>
2314 %                         )}
2315 %                     />
2316 %                     <FormField
2317 %                         control={reservationForm.control}
2318 %                         name="guestEmail"
2319 %                         render={({ field }) => (
2320 %                             <FormItem>
2321 %                                 <FormLabel>Email</FormLabel>
2322 %                                 <FormControl>
```


client/src/pages/restaurant.tsx (cont.)

```
2323 %         <Input {...field} type="email" placeholder="email@ejemplo.com" data-testid="input-guest-email..."
2324 %       </FormControl>
2325 %     <FormMessage />
2326 %   </FormItem>
2327 % }
2328 % />
2329 % </div>
2330 % <div className="grid grid-cols-2 gap-4">
2331 %   <FormField
2332 %     control={reservationForm.control}
2333 %     name="reservationDate"
2334 %     render={({ field }) => (
2335 %       <FormItem>
2336 %         <FormLabel>Fecha *</FormLabel>
2337 %         <FormControl>
2338 %           <Input {...field} type="date" data-testid="input-reservation-date" />
2339 %         </FormControl>
2340 %         <FormMessage />
2341 %       </FormItem>
2342 %     )}
2343 %   />
2344 %   <FormField
2345 %     control={reservationForm.control}
2346 %     name="reservationTime"
2347 %     render={({ field }) => (
2348 %       <FormItem>
2349 %         <FormLabel>Hora *</FormLabel>
2350 %         {timeSlots.length > 0 ? (
2351 %           <Select onValueChange={field.onChange} value={field.value}>
2352 %             <FormControl>
2353 %               <SelectTrigger data-testid="select-reservation-time">
2354 %                 <SelectValue placeholder="Seleccionar turno" />
2355 %               </SelectTrigger>
2356 %             </FormControl>
2357 %             <SelectContent>
2358 %               {timeSlots.map((slot) => (
2359 %                 <SelectItem key={slot.id} value={slot.time}>
2360 %                   {slot.time} {slot.label ? `(${slot.label})` : ""}
2361 %                 </SelectItem>
2362 %               ))}
2363 %             </SelectContent>
2364 %           </Select>
2365 %         ) : (
2366 %           <FormControl>
2367 %             <Input {...field} type="time" data-testid="input-reservation-time" />
2368 %           </FormControl>
2369 %         )}
2370 %         <FormMessage />
2371 %       </FormItem>
2372 %     )}
2373 %   />
2374 % </div>
2375 % <div className="grid grid-cols-2 gap-4">
2376 %   <FormField
2377 %     control={reservationForm.control}
2378 %     name="tableId"
2379 %     render={({ field }) => (
2380 %       <FormItem>
2381 %         <FormLabel>Mesa *</FormLabel>
2382 %         <Select onValueChange={field.onChange} value={field.value}>
2383 %           <FormControl>
2384 %             <SelectTrigger data-testid="select-table">
2385 %               <SelectValue placeholder="Seleccionar mesa" />
2386 %             </SelectTrigger>
2387 %           </FormControl>
2388 %           <SelectContent>
2389 %             {tables.filter(t => t.isActive === "true").map((table) => (
2390 %               <SelectItem key={table.id} value={table.id}>
2391 %                 Mesa {table.tableNumber} ({table.capacity} pers.)
2392 %                 {table.hasWindow === "true" ? " - Ventana" : ""}
2393 %               </SelectItem>
2394 %             ))}
2395 %           </SelectContent>
2396 %         </Select>
2397 %         <FormMessage />
2398 %       </FormItem>
2399 %     )}
2400 %   />
2401 %   <FormField
2402 %     control={reservationForm.control}
2403 %     name="partySize"
2404 %     render={({ field }) => (
2405 %       <FormItem>
2406 %         <FormLabel>Personas *</FormLabel>
2407 %         <FormControl>
2408 %           <Input {...field} type="number" min={1} data-testid="input-party-size" />
2409 %         </FormControl>
2410 %         <FormMessage />
2411 %       </FormItem>
2412 %     )}
2413 %   />
2414 % </div>
2415 % </FormField>
```

client/src/pages/restaurant.tsx (cont.)

```
2416 %         control={reservationForm.control}
2417 %         name="notes"
2418 %         render={({ field }) => (
2419 %           <FormItem>
2420 %             <FormLabel>Notas</FormLabel>
2421 %             <FormControl>
2422 %               <Textarea {...field} placeholder="Preferencias, alergias, ocasion especial..." data-testid="inp...
2423 %             </FormControl>
2424 %             <FormMessage />
2425 %           </FormItem>
2426 %         )}
2427 %       />
2428 %     <DialogFooter>
2429 %       <Button type="button" variant="outline" onClick={() => setIsReservationDialogOpen(false)}>
2430 %         Cancelar
2431 %       </Button>
2432 %       <Button
2433 %         type="submit"
2434 %         disabled={createReservationMutation.isPending}
2435 %         data-testid="button-save-reservation"
2436 %       >
2437 %         {createReservationMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
2438 %         Guardar Reserva
2439 %       </Button>
2440 %     </DialogFooter>
2441 %   </form>
2442 % </Form>
2443 % </DialogContent>
2444 % </Dialog>
2445 %
2446 % /* Daily Reservations Dialog */
2447 % <Dialog open={isDailyReservationsOpen} onOpenChange={setIsDailyReservationsOpen}>
2448 %   <DialogContent className="max-w-2xl">
2449 %     <DialogHeader>
2450 %       <DialogTitle>Reservas del Dia - {new Date().toLocaleDateString("es-AR", { weekday: "long", day: "numeric"...
2451 %     </DialogHeader>
2452 %     <div className="space-y-4">
2453 %       {todayReservations.length === 0 ? (
2454 %         <div className="text-center py-8 text-muted-foreground">
2455 %           <CalendarDays className="h-12 w-12 mx-auto mb-4 opacity-50" />
2456 %           <p>No hay reservas para hoy</p>
2457 %         </div>
2458 %       ) : (
2459 %         <div className="space-y-3 max-h-96 overflow-y-auto">
2460 %           {todayReservations
2461 %             .sort((a, b) => a.reservationTime.localeCompare(b.reservationTime))
2462 %             .map((reservation) => {
2463 %               const table = tables.find(t => t.id === reservation.tableId);
2464 %               return (
2465 %                 <div
2466 %                   key={reservation.id}
2467 %                   className="flex items-center justify-between p-3 border rounded-md gap-4"
2468 %                   data-testid={`daily-reservation-${reservation.id}`}
2469 %                 >
2470 %                   <div className="flex items-center gap-4">
2471 %                     <div className="text-lg font-bold">{reservation.reservationTime}</div>
2472 %                     <div>
2473 %                       <div className="font-medium">{reservation.guestName}</div>
2474 %                       <div className="text-sm text-muted-foreground">
2475 %                         Mesa {table?.tableNumber || "?"} - {reservation.partySize} personas
2476 %                         {table?.hasWindow === "true" && " (Ventana)"}
2477 %                       </div>
2478 %                     </div>
2479 %                   </div>
2480 %                   <div className="flex items-center gap-2">
2481 %                     <Badge className={reservationStatusColors[reservation.status]}>
2482 %                       {reservationStatusLabel[reservation.status]}
2483 %                     </Badge>
2484 %                     {reservation.status === "pending" && (
2485 %                       <Button
2486 %                         size="sm"
2487 %                         onClick={() => updateReservationMutation.mutate({ id: reservation.id, data: { status: "...
2488 %                       >
2489 %                         <Check className="h-4 w-4" />
2490 %                       </Button>
2491 %                     )}
2492 %                     {(reservation.status === "pending" || reservation.status === "confirmed") && (
2493 %                       <Button
2494 %                         size="sm"
2495 %                         variant="outline"
2496 %                         onClick={() => updateReservationMutation.mutate({ id: reservation.id, data: { status: "...
2497 %                       >
2498 %                         Sentar
2499 %                       </Button>
2500 %                     )}
2501 %                   </div>
2502 %                 </div>
2503 %               )};
2504 %             )}
2505 %           </div>
2506 %         )}
2507 %       </div>
2508 %     <DialogFooter>
```

client/src/pages/restaurant.tsx (cont.)

```
2509 %         <Button onClick={() => setIsDailyReservationsOpen(false)}>
2510 %             Cerrar
2511 %         </Button>
2512 %     </DialogFooter>
2513 % </DialogContent>
2514 % </Dialog>
2515 %
2516 % { /* Add Table Dialog */ }
2517 % <Dialog open={isAddTableDialogOpen} onOpenChange={setIsAddTableDialogOpen}>
2518 %     <DialogContent>
2519 %         <DialogHeader>
2520 %             <DialogTitle>Agregar Mesa</DialogTitle>
2521 %         </DialogHeader>
2522 %         <div className="space-y-4">
2523 %             <div className="grid grid-cols-2 gap-4">
2524 %                 <div className="space-y-2">
2525 %                     <Label>Numero de Mesa *</Label>
2526 %                     <Input
2527 %                         value={newTableNumber}
2528 %                         onChange={(e) => setNewTableNumber(e.target.value)}
2529 %                         placeholder="Ej: 1, 2, A1"
2530 %                         data-testid="input-new-table-number"
2531 %                     />
2532 %                 </div>
2533 %                 <div className="space-y-2">
2534 %                     <Label>Capacidad</Label>
2535 %                     <Input
2536 %                         type="number"
2537 %                         min={1}
2538 %                         value={newTableCapacity}
2539 %                         onChange={(e) => setNewTableCapacity(parseInt(e.target.value) || 1)}
2540 %                         data-testid="input-new-table-capacity"
2541 %                     />
2542 %                 </div>
2543 %             </div>
2544 %             <div className="grid grid-cols-2 gap-4">
2545 %                 <div className="space-y-2">
2546 %                     <Label>Forma</Label>
2547 %                     <Select value={newTableShape} onValueChange={setNewTableShape}>
2548 %                         <SelectTrigger data-testid="select-new-table-shape">
2549 %                             <SelectValue />
2550 %                         </SelectTrigger>
2551 %                         <SelectContent>
2552 %                             <SelectItem value="square">Cuadrada</SelectItem>
2553 %                             <SelectItem value="round">Redonda</SelectItem>
2554 %                             <SelectItem value="rectangular">Rectangular</SelectItem>
2555 %                         </SelectContent>
2556 %                     </Select>
2557 %                 </div>
2558 %                 <div className="space-y-2">
2559 %                     <Label>Area</Label>
2560 %                     <Select value={newTableArea} onValueChange={setNewTableArea}>
2561 %                         <SelectTrigger data-testid="select-new-table-area">
2562 %                             <SelectValue />
2563 %                         </SelectTrigger>
2564 %                         <SelectContent>
2565 %                             {areas.map((area) => (
2566 %                                 <SelectItem key={area.id} value={area.id}>{area.name}</SelectItem>
2567 %                             ))}
2568 %                         </SelectContent>
2569 %                     </Select>
2570 %                 </div>
2571 %             </div>
2572 %             <div className="flex items-center gap-3 p-3 border rounded-md">
2573 %                 <Switch
2574 %                     checked={newTableWindow}
2575 %                     onCheckedChange={setNewTableWindow}
2576 %                     data-testid="switch-new-table-window"
2577 %                 />
2578 %                 <div>
2579 %                     <Label className="cursor-pointer">Mesa con ventana</Label>
2580 %                     <p className="text-xs text-muted-foreground">Visible al hacer reservas</p>
2581 %                 </div>
2582 %             </div>
2583 %         </DialogContent>
2584 %         <DialogFooter>
2585 %             <Button variant="outline" onClick={() => setIsAddTableDialogOpen(false)}>Cancelar</Button>
2586 %             <Button
2587 %                 onClick={() => {
2588 %                     if (!newTableNumber || !newTableArea) return;
2589 %                     const areaTbIs = tables.filter(t => t.areaId === newTableArea);
2590 %                     let px = 0, py = 0;
2591 %                     if (areaTbIs.length > 0) {
2592 %                         const maxX = Math.max(...areaTbIs.map(t => t.positionX));
2593 %                         px = maxX + 1;
2594 %                         if (px >= 8) {
2595 %                             px = 0;
2596 %                             py = Math.max(...areaTbIs.map(t => t.positionY)) + 1;
2597 %                         }
2598 %                     }
2599 %                     createTableMutation.mutate({
2600 %                         tableNumber: newTableNumber,
2601 %                         areaId: newTableArea,
```

client/src/pages/restaurant.tsx (cont.)

```
2602 %         capacity: newTableCapacity,
2603 %         shape: newTableShape,
2604 %         hasWindow: newTableWindow ? "true" : "false",
2605 %         positionX: px,
2606 %         positionY: py,
2607 %     });
2608 % }
2609 % disabled={createTableMutation.isPending}
2610 % data-testid="button-save-table"
2611 % >
2612 %     {createTableMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
2613 %     Crear Mesa
2614 % </Button>
2615 % </DialogFooter>
2616 % </DialogContent>
2617 % </Dialog>
2618 %
2619 % {/* Edit Table Dialog */}
2620 % <Dialog open={isEditTableDialogOpen} onOpenChange={setIsEditTableDialogOpen}>
2621 %     <DialogContent>
2622 %         <DialogHeader>
2623 %             <DialogTitle>Editar Mesa {editingTable?.tableNumber}</DialogTitle>
2624 %         </DialogHeader>
2625 %         <div className="space-y-4">
2626 %             <div className="grid grid-cols-2 gap-4">
2627 %                 <div className="space-y-2">
2628 %                     <Label>Capacidad</Label>
2629 %                     <Input
2630 %                         type="number"
2631 %                         min={1}
2632 %                         value={editTableCapacity}
2633 %                         onChange={(e) => setEditTableCapacity(parseInt(e.target.value) || 1)}
2634 %                         data-testid="input-edit-table-capacity"
2635 %                     />
2636 %                 </div>
2637 %                 <div className="space-y-2">
2638 %                     <Label>Forma</Label>
2639 %                     <Select value={editTableShape} onValueChange={setEditTableShape}>
2640 %                         <SelectTrigger data-testid="select-edit-table-shape">
2641 %                             <SelectValue />
2642 %                         </SelectTrigger>
2643 %                         <SelectContent>
2644 %                             <SelectItem value="square">Cuadrada</SelectItem>
2645 %                             <SelectItem value="round">Redonda</SelectItem>
2646 %                             <SelectItem value="rectangular">Rectangular</SelectItem>
2647 %                         </SelectContent>
2648 %                     </Select>
2649 %                 </div>
2650 %             </div>
2651 %             <div className="flex items-center gap-3 p-3 border rounded-md">
2652 %                 <Switch
2653 %                     checked={editTableWindow}
2654 %                     onCheckedChange={setEditTableWindow}
2655 %                     data-testid="switch-edit-table-window"
2656 %                 />
2657 %                 <div>
2658 %                     <Label className="cursor-pointer">Mesa con ventana</Label>
2659 %                     <p className="text-xs text-muted-foreground">Visible al hacer reservas</p>
2660 %                 </div>
2661 %             </div>
2662 %         </DialogContent>
2663 %         <DialogFooter>
2664 %             <Button variant="outline" onClick={() => setIsEditTableDialogOpen(false)}>Cancelar</Button>
2665 %             <Button
2666 %                 onClick={() => {
2667 %                     if (!editingTable) return;
2668 %                     updateTableMutation.mutate({
2669 %                         id: editingTable.id,
2670 %                         data: {
2671 %                             capacity: editTableCapacity,
2672 %                             shape: editTableShape,
2673 %                             hasWindow: editTableWindow ? "true" : "false",
2674 %                         },
2675 %                     });
2676 %                     setIsEditTableDialogOpen(false);
2677 %                     toast({ title: "Mesa actualizada" });
2678 %                 }}
2679 %                 disabled={updateTableMutation.isPending}
2680 %                 data-testid="button-save-edit-table"
2681 %             >
2682 %                 Guardar Cambios
2683 %             </Button>
2684 %         </DialogFooter>
2685 %     </DialogContent>
2686 % </Dialog>
2687 %
2688 % {/* Menu Item Dialog (Create/Edit) */}
2689 % <Dialog open={isMenuItemDialogOpen} onOpenChange={setIsMenuItemDialogOpen}>
2690 %     <DialogContent>
2691 %         <DialogHeader>
2692 %             <DialogTitle>{editingMenuItem ? "Editar Plato" : "Nuevo Plato"</DialogTitle>
2693 %         </DialogHeader>
2694 %         <Form {...menuItemForm}>
```

client/src/pages/restaurant.tsx (cont.)

```
2695 %         <form
2696 %             onSubmit={menuItemForm.handleSubmit((data) => {
2697 %                 if (editingMenuItem) {
2698 %                     updateMenuItemMutation.mutate({ id: editingMenuItem.id, data });
2699 %                 } else {
2700 %                     createMenuItemMutation.mutate(data);
2701 %                 }
2702 %             })}
2703 %             className="space-y-4"
2704 %         >
2705 %             <FormField
2706 %                 control={menuItemForm.control}
2707 %                 name="name"
2708 %                 render={({ field }) => (
2709 %                     <FormItem>
2710 %                         <FormLabel>Nombre *</FormLabel>
2711 %                         <FormControl>
2712 %                             <Input {...field} placeholder="Nombre del plato" data-testid="input-menu-item-name" />
2713 %                         </FormControl>
2714 %                         <FormMessage />
2715 %                     </FormItem>
2716 %                 )}
2717 %             />
2718 %             <FormField
2719 %                 control={menuItemForm.control}
2720 %                 name="categoryId"
2721 %                 render={({ field }) => (
2722 %                     <FormItem>
2723 %                         <FormLabel>Categoria *</FormLabel>
2724 %                         <Select onChange={field.onChange} value={field.value}>
2725 %                             <FormControl>
2726 %                                 <SelectTrigger data-testid="select-menu-item-category">
2727 %                                     <SelectValue placeholder="Seleccionar categoria" />
2728 %                                 </SelectTrigger>
2729 %                             </FormControl>
2730 %                             <SelectContent>
2731 %                                 {sortedCategories.map((cat) => (
2732 %                                     <SelectItem key={cat.id} value={cat.id}>{cat.name}</SelectItem>
2733 %                                 ))}
2734 %                             </SelectContent>
2735 %                         </Select>
2736 %                         <FormMessage />
2737 %                     </FormItem>
2738 %                 )}
2739 %             />
2740 %             <FormField
2741 %                 control={menuItemForm.control}
2742 %                 name="description"
2743 %                 render={({ field }) => (
2744 %                     <FormItem>
2745 %                         <FormLabel>Descripcion</FormLabel>
2746 %                         <FormControl>
2747 %                             <Textarea {...field} placeholder="Descripcion del plato" data-testid="input-menu-item-descripti...
2748 %                         </FormControl>
2749 %                         <FormMessage />
2750 %                     </FormItem>
2751 %                 )}
2752 %             />
2753 %             <div className="grid grid-cols-2 gap-4">
2754 %                 <FormField
2755 %                     control={menuItemForm.control}
2756 %                     name="price"
2757 %                     render={({ field }) => (
2758 %                         <FormItem>
2759 %                             <FormLabel>Precio *</FormLabel>
2760 %                             <FormControl>
2761 %                                 <Input {...field} type="number" step="0.01" min={0} data-testid="input-menu-item-price" />
2762 %                             </FormControl>
2763 %                             <FormMessage />
2764 %                         </FormItem>
2765 %                     )}
2766 %                 />
2767 %                 <FormField
2768 %                     control={menuItemForm.control}
2769 %                     name="preparationTime"
2770 %                     render={({ field }) => (
2771 %                         <FormItem>
2772 %                             <FormLabel>Tiempo prep. (min)</FormLabel>
2773 %                             <FormControl>
2774 %                                 <Input {...field} type="number" min={0} data-testid="input-menu-item-prep-time" />
2775 %                             </FormControl>
2776 %                             <FormMessage />
2777 %                         </FormItem>
2778 %                     )}
2779 %                 />
2780 %             </div>
2781 %             <FormField
2782 %                 control={menuItemForm.control}
2783 %                 name="isAvailable"
2784 %                 render={({ field }) => (
2785 %                     <FormItem className="flex items-center gap-3 p-3 border rounded-md">
2786 %                         <FormControl>
2787 %                             <Switch
```

client/src/pages/restaurant.tsx (cont.)

```
2788 %             checked={field.value === "true"}
2789 %             onChange={() => field.onChange(checked ? "true" : "false")}
2790 %             data-testid="switch-menu-item-available"
2791 %         />
2792 %     </FormControl>
2793 %     <FormLabel className="cursor-pointer !mt-0">Disponible</FormLabel>
2794 % </FormItem>
2795 % })
2796 % />
2797 % <DialogFooter>
2798 %   <Button type="button" variant="outline" onClick={() => setIsMenuItemDialogOpen(false)}>Cancelar</Button>
2799 %   <Button
2800 %     type="submit"
2801 %     disabled={createMenuItemMutation.isPending || updateMenuItemMutation.isPending}
2802 %     data-testid="button-save-menu-item"
2803 %   >
2804 %     {(createMenuItemMutation.isPending || updateMenuItemMutation.isPending) && (
2805 %       <Loader2 className="h-4 w-4 mr-2 animate-spin" />
2806 %     )}
2807 %     {editingMenuItem ? "Actualizar" : "Crear Plato"}
2808 %   </Button>
2809 % </DialogFooter>
2810 % </form>
2811 % </Form>
2812 % </DialogContent>
2813 % </Dialog>
2814 %
2815 % { /* Category Dialog (Create/Edit) */ }
2816 % <Dialog open={isCategoryDialogOpen} onOpenChange={setIsCategoryDialogOpen}>
2817 %   <DialogContent>
2818 %     <DialogHeader>
2819 %       <DialogTitle>{editingCategory ? "Editar Categoria" : "Nueva Categoria"}</DialogTitle>
2820 %     </DialogHeader>
2821 %     <Form {...categoryForm}>
2822 %       <form
2823 %         onSubmit={categoryForm.handleSubmit((data) => {
2824 %           if (editingCategory) {
2825 %             updateCategoryMutation.mutate({ id: editingCategory.id, data });
2826 %           } else {
2827 %             createCategoryMutation.mutate(data);
2828 %           }
2829 %         })}
2830 %         className="space-y-4"
2831 %       >
2832 %         <FormField
2833 %           control={categoryForm.control}
2834 %           name="name"
2835 %           render={({ field }) => (
2836 %             <FormItem>
2837 %               <FormLabel>Nombre *</FormLabel>
2838 %               <FormControl>
2839 %                 <Input {...field} placeholder="Ej: Bebidas con alcohol, Entradas, etc." data-testid="input-cate..." />
2840 %               </FormControl>
2841 %               <FormMessage />
2842 %             </FormItem>
2843 %           )}
2844 %         />
2845 %         <FormField
2846 %           control={categoryForm.control}
2847 %           name="description"
2848 %           render={({ field }) => (
2849 %             <FormItem>
2850 %               <FormLabel>Descripcion</FormLabel>
2851 %               <FormControl>
2852 %                 <Textarea {...field} placeholder="Descripcion de la categoria" data-testid="input-category-desc..." />
2853 %               </FormControl>
2854 %               <FormMessage />
2855 %             </FormItem>
2856 %           )}
2857 %         />
2858 %         <FormField
2859 %           control={categoryForm.control}
2860 %           name="displayOrder"
2861 %           render={({ field }) => (
2862 %             <FormItem>
2863 %               <FormLabel>Orden de Visualizacion</FormLabel>
2864 %               <FormControl>
2865 %                 <Input {...field} type="number" min={0} data-testid="input-category-order" />
2866 %               </FormControl>
2867 %               <FormMessage />
2868 %               <p className="text-xs text-muted-foreground">Orden sugerido: 1) Bebidas 2) Entradas 3) Principale...</p>
2869 %             </FormItem>
2870 %           )}
2871 %         />
2872 %       <DialogFooter>
2873 %         <Button type="button" variant="outline" onClick={() => setIsCategoryDialogOpen(false)}>Cancelar</Button>
2874 %         <Button
2875 %           type="submit"
2876 %           disabled={createCategoryMutation.isPending || updateCategoryMutation.isPending}
2877 %           data-testid="button-save-category"
2878 %         >
2879 %           {(createCategoryMutation.isPending || updateCategoryMutation.isPending) && (
2880 %             <Loader2 className="h-4 w-4 mr-2 animate-spin" />
```

client/src/pages/restaurant.tsx (cont.)

```
2881 %         )}
2882 %         {editingCategory ? "Actualizar" : "Crear Categoria"}
2883 %       </Button>
2884 %     </DialogFooter>
2885 %   </form>
2886 % </Form>
2887 % </DialogContent>
2888 % </Dialog>
2889 %
2890 % { /* Time Slots Configuration Dialog */ }
2891 % <Dialog open={isTimeSlotsDialogOpen} onOpenChange={setIsTimeSlotsDialogOpen}>
2892 %   <DialogContent>
2893 %     <DialogHeader>
2894 %       <DialogTitle className="flex items-center gap-2">
2895 %         <Clock className="h-5 w-5" />
2896 %         Configurar Turnos de Reserva
2897 %       </DialogTitle>
2898 %       <DialogDescription>
2899 %         Define los horarios disponibles para reservas
2900 %       </DialogDescription>
2901 %     </DialogHeader>
2902 %     <div className="space-y-4">
2903 %       <div className="flex items-center gap-2">
2904 %         <Input
2905 %           type="time"
2906 %           value={newTimeSlot}
2907 %           onChange={(e) => setNewTimeSlot(e.target.value)}
2908 %           className="w-32"
2909 %           data-testid="input-new-time-slot"
2910 %         />
2911 %         <Button
2912 %           size="sm"
2913 %           onClick={() => {
2914 %             if (newTimeSlot) createTimeSlotMutation.mutate(newTimeSlot);
2915 %           }}
2916 %           disabled={!newTimeSlot || createTimeSlotMutation.isPending}
2917 %           data-testid="button-add-time-slot"
2918 %         >
2919 %           <Plus className="h-4 w-4 mr-1" />
2920 %           Agregar
2921 %         </Button>
2922 %       </div>
2923 %       {timeSlots.length === 0 ? (
2924 %         <p className="text-sm text-muted-foreground text-center py-4">
2925 %           Sin turnos configurados. Las reservas usaran horario libre.
2926 %         </p>
2927 %       ) : (
2928 %         <div className="flex flex-wrap gap-2">
2929 %           {timeSlots.map((slot) => (
2930 %             <Badge key={slot.id} variant="secondary" className="gap-1 text-sm py-1.5 px-3">
2931 %               {slot.time}
2932 %               <button
2933 %                 onClick={() => deleteTimeSlotMutation.mutate(slot.id)}
2934 %                 className="ml-1 hover:text-destructive"
2935 %                 data-testid={`button-delete-slot-${slot.id}`}
2936 %               >
2937 %                 <X className="h-3 w-3" />
2938 %               </button>
2939 %             </Badge>
2940 %           ))}
2941 %         </div>
2942 %       )}
2943 %     </div>
2944 %     <DialogFooter>
2945 %       <Button onClick={() => setIsTimeSlotsDialogOpen(false)}>Cerrar</Button>
2946 %     </DialogFooter>
2947 %   </DialogContent>
2948 % </Dialog>
2949 %
2950 % { /* Recipe Dialog */ }
2951 % <Dialog open={isRecipeDialogOpen} onOpenChange={setIsRecipeDialogOpen}>
2952 %   <DialogContent className="max-w-2xl">
2953 %     <DialogHeader>
2954 %       <DialogTitle className="flex items-center gap-2">
2955 %         <ChefHat className="h-5 w-5" />
2956 %         Receta - {selectedRecipeItem?.name}
2957 %       </DialogTitle>
2958 %       <DialogDescription>
2959 %         {selectedRecipeItem && (
2960 %           <span>Precio de venta: ${parseFloat(selectedRecipeItem.price).toLocaleString("es-AR", { minimumFracti...
2961 %         )}
2962 %       </DialogDescription>
2963 %     </DialogHeader>
2964 %     <div className="space-y-4">
2965 %       {currentRecipe && currentRecipe.ingredients.length > 0 && (
2966 %         <Table>
2967 %           <TableHeader>
2968 %             <TableRow>
2969 %               <TableHead>Ingrediente</TableHead>
2970 %               <TableHead className="text-right">Cantidad</TableHead>
2971 %               <TableHead>Unidad</TableHead>
2972 %               <TableHead className="text-right">Costo Unit.</TableHead>
2973 %               <TableHead className="text-right">Subtotal</TableHead>
```

client/src/pages/restaurant.tsx (cont.)

```
2974 %         <TableHead className="w-10"></TableHead>
2975 %       </TableRow>
2976 %     </TableHeader>
2977 %     <TableBody>
2978 %       {currentRecipe.ingredients.map((ing) => (
2979 %         <TableRow key={ing.id}>
2980 %           <TableCell className="font-medium">{ing.ingredientName}</TableCell>
2981 %           <TableCell className="text-right">{ing.quantity}</TableCell>
2982 %           <TableCell>{ing.unit}</TableCell>
2983 %           <TableCell className="text-right">${parseFloat(ing.unitCost).toLocaleString("es-AR", { minimumFrac...
2984 %           <TableCell className="text-right">
2985 %             ${((parseFloat(ing.quantity) * parseFloat(ing.unitCost))).toLocaleString("es-AR", { minimumFrac...
2986 %           </TableCell>
2987 %           <TableCell>
2988 %             <Button variant="ghost" size="icon" className="h-6 w-6" onClick={() => deleteIngredientMutati...
2989 %               <Trash2 className="h-3 w-3 text-destructive" />
2990 %             </Button>
2991 %           </TableCell>
2992 %         </TableRow>
2993 %       )})
2994 %     <TableRow className="font-bold">
2995 %       <TableCell colspan={4} className="text-right">Costo Total:</TableCell>
2996 %       <TableCell className="text-right">
2997 %         ${recipeCost.toLocaleString("es-AR", { minimumFractionDigits: 2 })}
2998 %       </TableCell>
2999 %       <TableCell></TableCell>
3000 %     </TableRow>
3001 %   </TableBody>
3002 % </Table>
3003 % )}
3004 %
3005 % {currentRecipe && selectedRecipeItem && (
3006 %   <div className="flex items-center gap-4 p-3 bg-muted rounded-md text-sm">
3007 %     <div>Costo: <strong>${recipeCost.toLocaleString("es-AR", { minimumFractionDigits: 2 })}</strong></div>
3008 %     <div>Precio: <strong>${parseFloat(selectedRecipeItem.price).toLocaleString("es-AR", { minimumFraction...
3009 %     <div>
3010 %       Margen:{" "}
3011 %       <strong className={parseFloat(selectedRecipeItem.price) - recipeCost >= 0 ? "text-green-600" : "tex...
3012 %         ${parseFloat(selectedRecipeItem.price) - recipeCost}.toLocaleString("es-AR", { minimumFractionDi...
3013 %         {" "})(parseFloat(selectedRecipeItem.price) > 0 ? ((parseFloat(selectedRecipeItem.price) - recip...
3014 %       </strong>
3015 %     </div>
3016 %   </div>
3017 % )}
3018 %
3019 % <div className="border-t pt-4">
3020 %   <Label className="mb-2 block">Agregar Ingrediente</Label>
3021 %   <div className="grid grid-cols-5 gap-2">
3022 %     <Input
3023 %       placeholder="Nombre"
3024 %       value={newIngredientName}
3025 %       onChange={(e) => setNewIngredientName(e.target.value)}
3026 %       className="col-span-2"
3027 %       data-testid="input-ingredient-name"
3028 %     />
3029 %     <Input
3030 %       type="number"
3031 %       placeholder="Cant."
3032 %       step="0.001"
3033 %       value={newIngredientQty}
3034 %       onChange={(e) => setNewIngredientQty(e.target.value)}
3035 %       data-testid="input-ingredient-qty"
3036 %     />
3037 %     <Select value={newIngredientUnit} onValueChange={setNewIngredientUnit}>
3038 %       <SelectTrigger data-testid="select-ingredient-unit">
3039 %         <SelectValue />
3040 %       </SelectTrigger>
3041 %       <SelectContent>
3042 %         <SelectItem value="g">g</SelectItem>
3043 %         <SelectItem value="kg">kg</SelectItem>
3044 %         <SelectItem value="ml">ml</SelectItem>
3045 %         <SelectItem value="l">l</SelectItem>
3046 %         <SelectItem value="unidad">unidad</SelectItem>
3047 %         <SelectItem value="porcion">porcion</SelectItem>
3048 %       </SelectContent>
3049 %     </Select>
3050 %     <Input
3051 %       type="number"
3052 %       placeholder="$/u"
3053 %       step="0.01"
3054 %       value={newIngredientCost}
3055 %       onChange={(e) => setNewIngredientCost(e.target.value)}
3056 %       data-testid="input-ingredient-cost"
3057 %     />
3058 %   </div>
3059 %   <Button
3060 %     size="sm"
3061 %     className="mt-2"
3062 %     onClick={() => {
3063 %       if (!currentRecipe || !newIngredientName || !newIngredientQty) return;
3064 %       addIngredientMutation.mutate({
3065 %         recipeId: currentRecipe.id,
3066 %         ingredientName: newIngredientName,
```


client/src/pages/restaurant.tsx (cont.)

```
3067 %             quantity: newIngredientQty,  
3068 %             unit: newIngredientUnit,  
3069 %             unitCost: newIngredientCost || "0",  
3070 %         });  
3071 %     }}  
3072 %     disabled={addIngredientMutation.isPending || !newIngredientName || !newIngredientQty}  
3073 %     data-testid="button-add-ingredient"  
3074 %     >  
3075 %         {addIngredientMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}  
3076 %         <Plus className="h-4 w-4 mr-1" />  
3077 %         Agregar Ingrediente  
3078 %     </Button>  
3079 % </div>  
3080 % </div>  
3081 % <DialogFooter>  
3082 %     <Button onClick={() => setIsRecipeDialogOpen(false)}>Cerrar</Button>  
3083 % </DialogFooter>  
3084 % </DialogContent>  
3085 % </Dialog>  
3086 % </div>  
3087 % );  
3088 % }  
3089 %
```

```
1 % import { useState, useMemo } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { useForm } from "react-hook-form";
5 % import { zodResolver } from "hookform/resolvers/zod";
6 % import { z } from "zod";
7 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
8 % import { Button } from "@components/ui/button";
9 % import { Badge } from "@components/ui/badge";
10 % import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@components/ui/dialog";
11 % import { Input } from "@components/ui/input";
12 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
13 % import { Textarea } from "@components/ui/textarea";
14 % import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
15 % import { ScrollArea, ScrollBar } from "@components/ui/scroll-area";
16 % import { useToast } from "@hooks/use-toast";
17 % import { format, addDays, startOfDay, parseISO, isSameDay, startOfWeek, addWeeks } from "date-fns";
18 % import { es } from "date-fns/locale";
19 % import {
20 %   Plus,
21 %   ChevronLeft,
22 %   ChevronRight,
23 %   Clock,
24 %   User,
25 %   Phone,
26 %   Loader2,
27 %   Calendar,
28 %   Sparkles,
29 %   CreditCard,
30 %   Home,
31 %   Receipt,
32 %   Pencil,
33 %   X,
34 %   DollarSign,
35 %   Trash2,
36 %   CalendarDays,
37 %   LayoutGrid,
38 %   AlertTriangle,
39 %   Package,
40 % } from "lucide-react";
41 %
42 % type SpaCabin = {
43 %   id: string;
44 %   name: string;
45 %   description: string | null;
46 %   isActive: string | null;
47 % };
48 %
49 % type SpaTreatment = {
50 %   id: string;
51 %   categoryId: string | null;
52 %   name: string;
53 %   description: string | null;
54 %   durationMinutes: number;
55 %   price: string;
56 %   isActive: string | null;
57 % };
58 %
59 % type SpaAppointment = {
60 %   id: string;
61 %   cabinId: string;
62 %   treatmentId: string;
63 %   guestName: string;
64 %   guestLastName: string | null;
65 %   guestPhone: string | null;
66 %   guestEmail: string | null;
67 %   reservationId: string | null;
68 %   appointmentDate: string;
69 %   startTime: string;
70 %   endTime: string;
71 %   status: "pending" | "confirmed" | "in_progress" | "completed" | "cancelled" | "no_show";
72 %   notes: string | null;
73 %   createdAt: string;
74 %   cabin?: SpaCabin;
75 %   treatment?: SpaTreatment;
76 % };
77 %
78 % type SpaPayment = {
79 %   id: string;
80 %   accountId: string;
81 %   amount: string;
82 %   method: string;
83 %   isAdvance: string | null;
84 %   appointmentId: string | null;
85 %   reservationId: string | null;
86 %   notes: string | null;
87 %   createdAt: string;
88 % };
89 %
90 % type SpaAccountItem = {
```

client/src/pages/spa.tsx (cont.)

```
91 %   id: string;
92 %   accountId: string;
93 %   description: string;
94 %   quantity: number;
95 %   unitPrice: string;
96 %   subtotal: string;
97 %   itemType: string;
98 %   notes: string | null;
99 %   createdAt: string;
100 % };
101 %
102 % type SpaAccount = {
103 %   id: string;
104 %   appointmentId: string;
105 %   guestName: string;
106 %   reservationId: string | null;
107 %   status: "open" | "closed" | "cancelled";
108 %   subtotal: string;
109 %   total: string;
110 %   totalPaid: string;
111 %   receiptType: string | null;
112 %   notes: string | null;
113 %   openedAt: string;
114 %   closedAt: string | null;
115 %   closedBy: string | null;
116 %   chargedTo: string | null;
117 %   items: SpaAccountItem[];
118 %   payments: SpaPayment[];
119 % };
120 %
121 % type Reservation = {
122 %   id: string;
123 %   guestId: string;
124 %   roomId: string;
125 %   status: string;
126 %   guest?: { firstName: string; lastName: string };
127 %   room?: { roomNumber: string };
128 % };
129 %
130 % type WeeklySummaryItem = {
131 %   cabinId: string;
132 %   date: string;
133 %   count: number;
134 % };
135 %
136 % type InventoryItemWithDetails = {
137 %   id: string;
138 %   name: string;
139 %   currentStock: number;
140 %   unit: string;
141 %   costPrice: string;
142 %   category?: { name: string; area: string };
143 % };
144 %
145 % const appointmentStatusColors: Record<string, string> = {
146 %   pending: "bg-yellow-500/20 text-yellow-700 dark:text-yellow-400 border-yellow-500/30",
147 %   confirmed: "bg-blue-500/20 text-blue-700 dark:text-blue-400 border-blue-500/30",
148 %   in_progress: "bg-green-500/20 text-green-700 dark:text-green-400 border-green-500/30",
149 %   completed: "bg-gray-500/20 text-gray-700 dark:text-gray-400 border-gray-500/30",
150 %   cancelled: "bg-red-500/20 text-red-700 dark:text-red-400 border-red-500/30",
151 %   no_show: "bg-orange-500/20 text-orange-700 dark:text-orange-400 border-orange-500/30",
152 % };
153 %
154 % const appointmentStatusLabels: Record<string, string> = {
155 %   pending: "Pendiente",
156 %   confirmed: "Confirmado",
157 %   in_progress: "En Curso",
158 %   completed: "Completado",
159 %   cancelled: "Cancelado",
160 %   no_show: "No Show",
161 % };
162 %
163 % const paymentMethodLabels: Record<string, string> = {
164 %   cash: "Efectivo",
165 %   debit_card: "Tarjeta Débito",
166 %   credit_card: "Tarjeta Crédito",
167 %   transfer: "Transferencia",
168 %   mercadopago: "MercadoPago",
169 %   room_charge: "Cargo a Habitación",
170 % };
171 %
172 % const TIME_SLOTS: string[] = [];
173 % for (let h = 8; h < 22; h++) {
174 %   TIME_SLOTS.push(`${h.toString().padStart(2, "0")}:00`);
175 %   TIME_SLOTS.push(`${h.toString().padStart(2, "0")}:30`);
176 % }
177 % TIME_SLOTS.push("22:00");
178 %
179 % const appointmentFormSchema = z.object({
180 %   cabinId: z.string().min(1, "Seleccione un gabinete"),
181 %   treatmentId: z.string().min(1, "Seleccione un tratamiento"),
182 %   guestName: z.string().min(1, "El nombre es requerido"),
183 %   guestLastName: z.string().optional(),
```

client/src/pages/spa.tsx (cont.)

```
184 %   guestPhone: z.string().optional(),
185 %   guestEmail: z.string().email("Email invalido").optional().or(z.literal("")),
186 %   appointmentDate: z.string().min(1, "La fecha es requerida"),
187 %   startTime: z.string().min(1, "La hora de inicio es requerida"),
188 %   reservationId: z.string().optional(),
189 %   notes: z.string().optional(),
190 % });
191 %
192 % type AppointmentFormValues = z.infer<typeof appointmentFormSchema>;
193 %
194 % function timeToMinutes(time: string): number {
195 %   const [h, m] = time.split(":").map(Number);
196 %   return h * 60 + m;
197 % }
198 %
199 % type ViewMode = "daily" | "weekly";
200 % type SpaTab = "agenda" | "insumos";
201 %
202 % export default function SpaPage() {
203 %   const [selectedDate, setSelectedDate] = useState(startOfDay(new Date()));
204 %   const [viewMode, setViewMode] = useState<ViewMode>("daily");
205 %   const [activeTab, setActiveTab] = useState<SpaTab>("agenda");
206 %   const [selectedAppointment, setSelectedAppointment] = useState<SpaAppointment | null>(null);
207 %   const [isNewDialogOpen, setIsNewDialogOpen] = useState(false);
208 %   const [isEditMode, setIsEditMode] = useState(false);
209 %   const [editingAppointmentId, setEditingAppointmentId] = useState<string | null>(null);
210 %   const [isCancelConfirmOpen, setIsCancelConfirmOpen] = useState(false);
211 %   const [isFolioOpen, setIsFolioOpen] = useState(false);
212 %   const [isAddChargeOpen, setIsAddChargeOpen] = useState(false);
213 %   const [isAddPaymentOpen, setIsAddPaymentOpen] = useState(false);
214 %   const [paymentMethod, setPaymentMethod] = useState("");
215 %   const [paymentAmount, setPaymentAmount] = useState("");
216 %   const [paymentReservationId, setPaymentReservationId] = useState("");
217 %   const [isPaymentAdvance, setIsPaymentAdvance] = useState(false);
218 %   const [chargeDescription, setChargeDescription] = useState("");
219 %   const [chargePrice, setChargePrice] = useState("");
220 %   const [chargeQuantity, setChargeQuantity] = useState("1");
221 %   const [chargeType, setChargeType] = useState("service");
222 %   const [receiptType, setReceiptType] = useState("");
223 %   const { toast } = useToast();
224 %
225 %   const { data: cabins = [], isLoading: cabinsLoading } = useQuery<SpaCabin[]>({
226 %     queryKey: ["/api/spa/cabins"],
227 %   });
228 %
229 %   const { data: treatments = [] } = useQuery<SpaTreatment[]>({
230 %     queryKey: ["/api/spa/treatments"],
231 %   });
232 %
233 %   const { data: checkedInReservations = [] } = useQuery<Reservation[]>({
234 %     queryKey: ["/api/reservations"],
235 %     select: (data) => data.filter((r) => r.status === "checked_in"),
236 %   });
237 %
238 %   const dateStr = format(selectedDate, "yyyy-MM-dd");
239 %   const { data: appointments = [], isLoading: appointmentsLoading } = useQuery<SpaAppointment[]>({
240 %     queryKey: ["/api/spa/appointments", dateStr, dateStr],
241 %     queryFn: async () => {
242 %       const response = await fetch(
243 %         `/api/spa/appointments?startDate=${dateStr}&endDate=${dateStr}`
244 %       );
245 %       if (!response.ok) throw new Error("Error fetching appointments");
246 %       return response.json();
247 %     },
248 %   });
249 %
250 %   const weekStart = startOfWeek(selectedDate, { weekStartsOn: 1 });
251 %   const weekEnd = addDays(weekStart, 6);
252 %   const weekStartStr = format(weekStart, "yyyy-MM-dd");
253 %   const weekEndStr = format(weekEnd, "yyyy-MM-dd");
254 %
255 %   const { data: weeklySummary = [] } = useQuery<WeeklySummaryItem[]>({
256 %     queryKey: ["/api/spa/appointments/weekly-summary", weekStartStr, weekEndStr],
257 %     queryFn: async () => {
258 %       const response = await fetch(
259 %         `/api/spa/appointments/weekly-summary?startDate=${weekStartStr}&endDate=${weekEndStr}`
260 %       );
261 %       if (!response.ok) throw new Error("Error");
262 %       return response.json();
263 %     },
264 %     enabled: viewMode === "weekly",
265 %   });
266 %
267 %   const { data: selectedAccount, refetch: refetchAccount } = useQuery<SpaAccount>({
268 %     queryKey: ["/api/spa/accounts/by-appointment", selectedAppointment?.id],
269 %     queryFn: async () => {
270 %       if (!selectedAppointment) throw new Error("No appointment selected");
271 %       const response = await fetch(`/api/spa/accounts/by-appointment/${selectedAppointment.id}`);
272 %       if (!response.ok) throw new Error("Account not found");
273 %       return response.json();
274 %     },
275 %     enabled: !!selectedAppointment,
276 %     staleTime: 0,
```

client/src/pages/spa.tsx (cont.)

```
277 % });
278 %
279 % const { data: spaInventoryItems = [] } = useQuery<InventoryItemWithDetails[]>({
280 %   queryKey: ["/api/inventory/items", "spa"],
281 %   queryFn: async () => {
282 %     const response = await fetch("/api/inventory/items?area=spa");
283 %     if (!response.ok) throw new Error("Error");
284 %     return response.json();
285 %   },
286 %   enabled: activeTab === "insumos",
287 % });
288 %
289 % const activeCabins = cabins.filter((c) => c.isActive === "true");
290 %
291 % const form = useForm<AppointmentFormValues>({
292 %   resolver: zodResolver(appointmentFormSchema),
293 %   defaultValues: {
294 %     cabinId: "",
295 %     treatmentId: "",
296 %     guestName: "",
297 %     guestLastName: "",
298 %     guestPhone: "",
299 %     guestEmail: "",
300 %     appointmentDate: dateStr,
301 %     startTime: "",
302 %     reservationId: "",
303 %     notes: "",
304 %   },
305 % });
306 %
307 % const createAppointmentMutation = useMutation({
308 %   mutationFn: async (data: AppointmentFormValues) => {
309 %     const treatment = treatments.find((t) => t.id === data.treatmentId);
310 %     const durationMinutes = treatment?.durationMinutes ?? 60;
311 %
312 %     const startMinutes = timeToMinutes(data.startTime);
313 %     const endMinutes = startMinutes + durationMinutes;
314 %     const endHours = Math.floor(endMinutes / 60);
315 %     const endMins = endMinutes % 60;
316 %     const endTime = `${endHours.toString().padStart(2, "0")}:${endMins.toString().padStart(2, "0")}`;
317 %
318 %     const res = await fetch("/api/spa/appointments", {
319 %       method: "POST",
320 %       headers: { "Content-Type": "application/json" },
321 %       body: JSON.stringify({
322 %         ...data,
323 %         endTime,
324 %         status: "confirmed",
325 %       }),
326 %     });
327 %
328 %     if (!res.ok) {
329 %       const err = await res.json();
330 %       throw new Error(err.message || err.error || "Error al crear turno");
331 %     }
332 %     return res.json();
333 %   },
334 %   onSuccess: () => {
335 %     queryClient.invalidateQueries({ queryKey: ["/api/spa/appointments"] });
336 %     toast({ title: "Turno creado correctamente" });
337 %     setIsNewDialogOpen(false);
338 %     form.reset({
339 %       cabinId: "", treatmentId: "", guestName: "", guestLastName: "",
340 %       guestPhone: "", guestEmail: "", appointmentDate: dateStr,
341 %       startTime: "", reservationId: "", notes: "",
342 %     });
343 %   },
344 %   onError: (error: Error) => {
345 %     toast({ title: error.message, variant: "destructive" });
346 %   },
347 % });
348 %
349 % const editAppointmentMutation = useMutation({
350 %   mutationFn: async (data: AppointmentFormValues & { id: string }) => {
351 %     const treatment = treatments.find((t) => t.id === data.treatmentId);
352 %     const durationMinutes = treatment?.durationMinutes ?? 60;
353 %     const startMinutes = timeToMinutes(data.startTime);
354 %     const endMinutes = startMinutes + durationMinutes;
355 %     const endHours = Math.floor(endMinutes / 60);
356 %     const endMins = endMinutes % 60;
357 %     const endTime = `${endHours.toString().padStart(2, "0")}:${endMins.toString().padStart(2, "0")}`;
358 %
359 %     const res = await fetch(`/api/spa/appointments/${data.id}`, {
360 %       method: "PATCH",
361 %       headers: { "Content-Type": "application/json" },
362 %       body: JSON.stringify({
363 %         cabinId: data.cabinId,
364 %         treatmentId: data.treatmentId,
365 %         guestName: data.guestName,
366 %         guestLastName: data.guestLastName || null,
367 %         guestPhone: data.guestPhone || null,
368 %         guestEmail: data.guestEmail || null,
369 %         appointmentDate: data.appointmentDate,
```

client/src/pages/spa.tsx (cont.)

```
370 %         startTime: data.startTime,
371 %         endTime,
372 %         reservationId: data.reservationId || null,
373 %         notes: data.notes || null,
374 %     }},
375 % });
376 % if (!res.ok) {
377 %     const err = await res.json();
378 %     throw new Error(err.message || err.error || "Error al editar turno");
379 % }
380 % return res.json();
381 % },
382 % onSuccess: () => {
383 %     queryClient.invalidateQueries({ queryKey: ["/api/spa/appointments"] });
384 %     toast({ title: "Turno actualizado correctamente" });
385 %     setIsNewDialogOpen(false);
386 %     setIsEditMode(false);
387 %     setEditingAppointmentId(null);
388 %     setSelectedAppointment(null);
389 % },
390 % onError: (error: Error) => {
391 %     toast({ title: error.message, variant: "destructive" });
392 % },
393 % });
394 %
395 % const updateAppointmentMutation = useMutation({
396 %     mutationFn: async ({ id, status }: { id: string; status: string }) => {
397 %         return apiRequest("PATCH", `/api/spa/appointments/${id}`, { status });
398 %     },
399 %     onSuccess: () => {
400 %         queryClient.invalidateQueries({ queryKey: ["/api/spa/appointments"] });
401 %         toast({ title: "Turno actualizado" });
402 %     },
403 %     onError: () => {
404 %         toast({ title: "Error al actualizar el turno", variant: "destructive" });
405 %     },
406 % });
407 %
408 % const addChargeMutation = useMutation({
409 %     mutationFn: async ({ accountId, description, quantity, unitPrice, itemType }: {
410 %         accountId: string; description: string; quantity: number; unitPrice: string; itemType: string;
411 %     }) => {
412 %         return apiRequest("POST", `/api/spa/accounts/${accountId}/items`, {
413 %             description, quantity, unitPrice, itemType,
414 %         });
415 %     },
416 %     onSuccess: () => {
417 %         queryClient.invalidateQueries({ queryKey: ["/api/spa/accounts"] });
418 %         refetchAccount();
419 %         toast({ title: "Cargo agregado" });
420 %         setIsAddChargeOpen(false);
421 %         setChargeDescription("");
422 %         setChargePrice("");
423 %         setChargeQuantity("1");
424 %         setChargeType("service");
425 %     },
426 % });
427 %
428 % const deleteChargeMutation = useMutation({
429 %     mutationFn: async (itemId: string) => {
430 %         return apiRequest("DELETE", `/api/spa/account-items/${itemId}`);
431 %     },
432 %     onSuccess: () => {
433 %         refetchAccount();
434 %         toast({ title: "Cargo eliminado" });
435 %     },
436 % });
437 %
438 % const addPaymentMutation = useMutation({
439 %     mutationFn: async ({ accountId, amount, method, isAdvance, reservationId }: {
440 %         accountId: string; amount: string; method: string; isAdvance: boolean; reservationId?: string;
441 %     }) => {
442 %         return apiRequest("POST", `/api/spa/accounts/${accountId}/payments`, {
443 %             amount, method, isAdvance, reservationId: reservationId || null,
444 %         });
445 %     },
446 %     onSuccess: () => {
447 %         refetchAccount();
448 %         toast({ title: "Pago registrado" });
449 %         setIsAddPaymentOpen(false);
450 %         setPaymentAmount("");
451 %         setPaymentMethod("");
452 %         setPaymentReservationId("");
453 %         setIsPaymentAdvance(false);
454 %     },
455 % });
456 %
457 % const closeAccountMutation = useMutation({
458 %     mutationFn: async ({ accountId, receiptType }: { accountId: string; receiptType: string }) => {
459 %         const res = await fetch(`/api/spa/accounts/${accountId}/close`, {
460 %             method: "POST",
461 %             headers: { "Content-Type": "application/json" },
462 %             body: JSON.stringify({ chargedTo: "direct", receiptType }),
```

client/src/pages/spa.tsx (cont.)

```
463 %     });
464 %     if (!res.ok) {
465 %         const err = await res.json();
466 %         throw new Error(err.message || err.error || "Error al cerrar");
467 %     }
468 %     return res.json();
469 % },
470 % onSuccess: async () => {
471 %     if (selectedAppointment) {
472 %         await updateAppointmentMutation.mutateAsync({ id: selectedAppointment.id, status: "completed" });
473 %     }
474 %     queryClient.invalidateQueries({ queryKey: ["/api/spa/accounts"] });
475 %     queryClient.invalidateQueries({ queryKey: ["/api/spa/appointments"] });
476 %     toast({ title: "Folio cerrado correctamente" });
477 %     setIsFolioOpen(false);
478 %     setSelectedAppointment(null);
479 %     setReceiptType("");
480 % },
481 % onError: (error: Error) => {
482 %     toast({ title: error.message, variant: "destructive" });
483 % },
484 % });
485 %
486 % const handlePreviousDay = () => setSelectedDate(addDays(selectedDate, -1));
487 % const handleNextDay = () => setSelectedDate(addDays(selectedDate, 1));
488 % const handleToday = () => setSelectedDate(startOfDay(new Date()));
489 % const handlePreviousWeek = () => setSelectedDate(addWeeks(selectedDate, -1));
490 % const handleNextWeek = () => setSelectedDate(addWeeks(selectedDate, 1));
491 %
492 % const handleCellClick = (cabinId: string, time: string) => {
493 %     setIsEditMode(false);
494 %     setEditingAppointmentId(null);
495 %     form.reset({
496 %         cabinId, treatmentId: "", guestName: "", guestLastName: "",
497 %         guestPhone: "", guestEmail: "", appointmentDate: dateStr,
498 %         startTime: time, reservationId: "", notes: "",
499 %     });
500 %     setIsNewDialogOpen(true);
501 % };
502 %
503 % const handleEditAppointment = (apt: SpaAppointment) => {
504 %     setIsEditMode(true);
505 %     setEditingAppointmentId(apt.id);
506 %     form.reset({
507 %         cabinId: apt.cabinId,
508 %         treatmentId: apt.treatmentId,
509 %         guestName: apt.guestName,
510 %         guestLastName: apt.guestLastName || "",
511 %         guestPhone: apt.guestPhone || "",
512 %         guestEmail: apt.guestEmail || "",
513 %         appointmentDate: apt.appointmentDate,
514 %         startTime: apt.startTime,
515 %         reservationId: apt.reservationId || "",
516 %         notes: apt.notes || "",
517 %     });
518 %     setSelectedAppointment(null);
519 %     setIsNewDialogOpen(true);
520 % };
521 %
522 % const handleCancelAppointment = () => {
523 %     if (selectedAppointment) {
524 %         updateAppointmentMutation.mutate({ id: selectedAppointment.id, status: "cancelled" });
525 %         setIsCancelConfirmOpen(false);
526 %         setSelectedAppointment(null);
527 %     }
528 % };
529 %
530 % const handleOpenFolio = () => {
531 %     setSelectedAppointment(selectedAppointment);
532 %     setIsFolioOpen(true);
533 % };
534 %
535 % const getAppointmentForSlot = (cabinId: string, slotTime: string): SpaAppointment | null => {
536 %     const slotMinutes = timeToMinutes(slotTime);
537 %     for (const apt of appointments) {
538 %         if (apt.cabinId !== cabinId) continue;
539 %         const aptDate = parseISO(apt.appointmentDate);
540 %         if (!isSameDay(aptDate, selectedDate)) continue;
541 %         const startMinutes = timeToMinutes(apt.startTime);
542 %         const endMinutes = timeToMinutes(apt.endTime);
543 %         if (slotMinutes >= startMinutes && slotMinutes < endMinutes) return apt;
544 %     }
545 %     return null;
546 % };
547 %
548 % const isSlotStart = (cabinId: string, slotTime: string): boolean => {
549 %     return appointments.some(apt =>
550 %         apt.cabinId === cabinId &&
551 %         apt.startTime === slotTime &&
552 %         isSameDay(parseISO(apt.appointmentDate), selectedDate)
553 %     );
554 % };
555 %
```

client/src/pages/spa.tsx (cont.)

```
556 %   const getAppointmentColSpan = (appointment: SpaAppointment): number => {
557 %       const startMinutes = timeToMinutes(appointment.startTime);
558 %       const endMinutes = timeToMinutes(appointment.endTime);
559 %       return Math.ceil((endMinutes - startMinutes) / 30);
560 %   };
561 %
562 %   const onSubmit = (data: AppointmentFormValues) => {
563 %       if (isEditMode && editingAppointmentId) {
564 %           editAppointmentMutation.mutate({ ...data, id: editingAppointmentId });
565 %       } else {
566 %           createAppointmentMutation.mutate(data);
567 %       }
568 %   };
569 %
570 %   const hasAdvancePayment = (aptId: string) => {
571 %       if (!selectedAccount) return false;
572 %       return selectedAccount.payments.some(p => p.isAdvance === "true");
573 %   };
574 %
575 %   const accountTotal = useMemo(() => {
576 %       if (!selectedAccount) return 0;
577 %       return selectedAccount.items.reduce((sum, item) => sum + parseFloat(item.subtotal), 0);
578 %   }, [selectedAccount]);
579 %
580 %   const accountPaid = useMemo(() => {
581 %       if (!selectedAccount) return 0;
582 %       return selectedAccount.payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
583 %   }, [selectedAccount]);
584 %
585 %   const accountBalance = accountTotal - accountPaid;
586 %
587 %   const weekDays = useMemo(() => {
588 %       const days = [];
589 %       for (let i = 0; i < 7; i++) {
590 %           days.push(addDays(weekStart, i));
591 %       }
592 %       return days;
593 %   }, [weekStart]);
594 %
595 %   const getWeeklyCount = (cabinId: string, date: Date): number => {
596 %       const dateStr = format(date, "yyyy-MM-dd");
597 %       const item = weeklySummary.find(s => s.cabinId === cabinId && s.date === dateStr);
598 %       return item?.count ?? 0;
599 %   };
600 %
601 %   const getOccupancyColor = (count: number): string => {
602 %       if (count === 0) return "bg-gray-50 dark:bg-gray-800/30";
603 %       if (count <= 2) return "bg-green-100 dark:bg-green-900/30 text-green-800 dark:text-green-300";
604 %       if (count <= 4) return "bg-amber-100 dark:bg-amber-900/30 text-amber-800 dark:text-amber-300";
605 %       return "bg-red-100 dark:bg-red-900/30 text-red-800 dark:text-red-300";
606 %   };
607 %
608 %   if (cabinsLoading || appointmentsLoading) {
609 %       return (
610 %           <div className="flex items-center justify-center h-full">
611 %               <Loader2 className="h-8 w-8 animate-spin text-muted-foreground" />
612 %           </div>
613 %       );
614 %   }
615 %
616 %   return (
617 %       <div className="flex flex-col h-full p-4 gap-4">
618 %           <div className="flex items-center justify-between gap-4 flex-wrap">
619 %               <div className="flex items-center gap-3">
620 %                   <div className="flex h-10 w-10 items-center justify-center rounded-md bg-primary/10">
621 %                       <Sparkles className="h-5 w-5 text-primary" />
622 %                   </div>
623 %                   <div>
624 %                       <h1 className="text-2xl font-bold" data-testid="text-page-title">SPA</h1>
625 %                       <p className="text-sm text-muted-foreground">Gestión de turnos y tratamientos</p>
626 %                   </div>
627 %               </div>
628 %
629 %               <div className="flex items-center gap-1 bg-muted rounded-lg p-1">
630 %                   <Button
631 %                       variant={activeTab === "agenda" ? "default" : "ghost"}
632 %                       size="sm"
633 %                       onClick={() => setActiveTab("agenda")}
634 %                       data-testid="tab-agenda"
635 %                   >
636 %                       <CalendarDays className="h-4 w-4 mr-1" /> Agenda
637 %                   </Button>
638 %                   <Button
639 %                       variant={activeTab === "insumos" ? "default" : "ghost"}
640 %                       size="sm"
641 %                       onClick={() => setActiveTab("insumos")}
642 %                       data-testid="tab-insumos"
643 %                   >
644 %                       <Package className="h-4 w-4 mr-1" /> Insumos
645 %                   </Button>
646 %               </div>
647 %           </div>
648 %       );

```



```

649 %     {activeTab === "agenda" && (
650 %         <>
651 %             <div className="flex items-center justify-between gap-4 flex-wrap">
652 %                 <div className="flex items-center gap-2">
653 %                     <div className="flex items-center gap-1 bg-muted rounded-lg p-1">
654 %                         <Button
655 %                             variant={viewMode === "daily" ? "default" : "ghost"}
656 %                             size="sm"
657 %                             onClick={() => setViewMode("daily")}
658 %                             data-testid="view-daily"
659 %                         >
660 %                             <LayoutGrid className="h-4 w-4 mr-1" /> Diaria
661 %                         </Button>
662 %                         <Button
663 %                             variant={viewMode === "weekly" ? "default" : "ghost"}
664 %                             size="sm"
665 %                             onClick={() => setViewMode("weekly")}
666 %                             data-testid="view-weekly"
667 %                         >
668 %                             <CalendarDays className="h-4 w-4 mr-1" /> Semanal
669 %                         </Button>
670 %                     </div>
671 %                 </div>
672 %
673 %                 <div className="flex items-center gap-2">
674 %                     <Button variant="outline" size="icon" onClick={viewMode === "daily" ? handlePreviousDay : handlePreviou...
675 %                         <ChevronLeft className="h-4 w-4" />
676 %                     </Button>
677 %                     <Button variant="outline" onClick={handleToday} data-testid="button-today">Hoy</Button>
678 %                     <Button variant="outline" size="icon" onClick={viewMode === "daily" ? handleNextDay : handleNextWeek} d...
679 %                         <ChevronRight className="h-4 w-4" />
680 %                     </Button>
681 %                     <span className="text-sm font-medium ml-2">
682 %                         {viewMode === "daily"
683 %                             ? format(selectedDate, "EEEE d 'de' MMMM yyyy", { locale: es })
684 %                             : `${format(weekStart, "d MMM", { locale: es })} - ${format(weekEnd, "d MMM yyyy", { locale: es })`}
685 %                         }
686 %                     </span>
687 %                 </div>
688 %
689 %                 {viewMode === "daily" && (
690 %                     <Button onClick={() => {
691 %                         setIsEditMode(false);
692 %                         setEditingAppointmentId(null);
693 %                         form.reset({
694 %                             cabinId: "", treatmentId: "", guestName: "", guestLastName: "",
695 %                             guestPhone: "", guestEmail: "", appointmentDate: dateStr,
696 %                             startTime: "", reservationId: "", notes: "",
697 %                         });
698 %                         setIsNewDialogOpen(true);
699 %                     }} data-testid="button-new-appointment">
700 %                         <Plus className="h-4 w-4 mr-2" /> Nuevo Turno
701 %                     </Button>
702 %                 )}
703 %             </div>
704 %
705 %             {viewMode === "daily" ? (
706 %                 <Card className="flex-1 flex flex-col overflow-hidden">
707 %                     <CardHeader className="py-3 px-4">
708 %                         <CardTitle className="text-base">Planning del Día - Gabinetes</CardTitle>
709 %                     </CardHeader>
710 %                     <CardContent className="flex-1 p-0 overflow-hidden">
711 %                         <ScrollArea className="h-full">
712 %                             <div className="min-w-[1800px]">
713 %                                 <table className="w-full border-collapse table-fixed">
714 %                                     <thead className="sticky top-0 z-10 bg-background">
715 %                                         <tr>
716 %                                             <th className="border-b border-r p-2 text-left text-xs font-medium text-muted-foreground w-...
717 %                                                 Gabinete
718 %                                             </th>
719 %                                             {TIME_SLOTS.map((time) => (
720 %                                                 <th key={time} className="border-b border-r p-1 text-center text-[10px] font-medium text-...
721 %                                                     {time}
722 %                                                 </th>
723 %                                             ))}
724 %                                         </tr>
725 %                                     </thead>
726 %                                     <tbody>
727 %                                         {activeCabins.map((cabin) => {
728 %                                             const skipSlots = new Set<number>();
729 %                                             return (
730 %                                                 <tr key={cabin.id} className="h-14">
731 %                                                     <td className="border-b border-r p-2 text-sm font-medium sticky left-0 bg-background z-...
732 %                                                         <div className="truncate" title={cabin.name}></div>
733 %                                                         <div className="text-[10px] text-muted-foreground truncate">{cabin.description}</div>
734 %                                                     </td>
735 %                                                     {TIME_SLOTS.map((time, slotIdx) => {
736 %                                                         if (skipSlots.has(slotIdx)) return null;
737 %                                                         const appointment = getAppointmentForSlot(cabin.id, time);
738 %                                                         const isStart = isSlotStart(cabin.id, time);
739 %                                                         if (appointment && isStart) {
740 %                                                             const colSpan = getAppointmentColSpan(appointment);
741 %

```

```

742 %         for (let i = 1; i < colSpan; i++) skipSlots.add(slotIdx + i);
743 %         const isCancelled = appointment.status === "cancelled";
744 %
745 %         return (
746 %             <td key={slotIdx} colSpan={colSpan} className="border-b border-r p-0.5 h-14">
747 %                 <div
748 %                     className={`h-full rounded px-2 py-1 cursor-pointer flex flex-col justify-cen...
749 %                     onClick={() => setSelectedAppointment(appointment)}
750 %                     title={` ${appointment.guestName} ${appointment.guestLastName} || ""} - ${treat...
751 %                     data-testid={`appointment-${appointment.id}`}
752 %                 >
753 %                     <div className="text-xs font-medium truncate">
754 %                         {appointment.guestName} {appointment.guestLastName} || ""
755 %                     </div>
756 %                     <div className="text-[10px] truncate opacity-75">
757 %                         {treatments.find(t => t.id === appointment.treatmentId)?.name}
758 %                     </div>
759 %                     <div className="text-[10px] opacity-60">
760 %                         {appointment.startTime} - {appointment.endTime}
761 %                     </div>
762 %                 </div>
763 %             </td>
764 %         );
765 %     }
766 %     if (appointment && !isStart) return null;
767 %     return (
768 %         <td key={slotIdx} className="border-b border-r p-0.5 h-14 cursor-pointer hover:bg-m...
769 %     );
770 %     }}}
771 % </tr>
772 % );
773 % }}}
774 % </tbody>
775 % </table>
776 % </div>
777 % <ScrollBar orientation="horizontal" />
778 % </ScrollArea>
779 % </CardContent>
780 % </Card>
781 % ) : (
782 % <Card className="flex-1 flex flex-col overflow-hidden">
783 %   <CardHeader className="py-3 px-4">
784 %     <CardTitle className="text-base">Vista Semanal - Gabinetes</CardTitle>
785 %   </CardHeader>
786 %   <CardContent className="flex-1 p-0 overflow-auto">
787 %     <table className="w-full border-collapse">
788 %       <thead className="sticky top-0 z-10 bg-background">
789 %         <tr>
790 %           <th className="border-b border-r p-3 text-left text-xs font-medium text-muted-foreground w-40">
791 %             Gabinete
792 %           </th>
793 %           {weekDays.map((day) => (
794 %             <th key={day.toISOString()} className="border-b border-r p-3 text-center text-xs font-medium ...
795 %               <div>{format(day, "EEE", { locale: es })}</div>
796 %               <div className="text-sm font-semibold">{format(day, "d/M")}</div>
797 %             </th>
798 %           ))}
799 %         </tr>
800 %       </thead>
801 %       <tbody>
802 %         {activeCabins.map((cabin) => (
803 %           <tr key={cabin.id}>
804 %             <td className="border-b border-r p-3 text-sm font-medium">
805 %               <div className="truncate">{cabin.name}</div>
806 %             </td>
807 %             {weekDays.map((day) => {
808 %               const count = getWeeklyCount(cabin.id, day);
809 %               return (
810 %                 <td
811 %                   key={day.toISOString()}
812 %                   className={`border-b border-r p-3 text-center cursor-pointer hover:opacity-80 transitio...
813 %                   onClick={() => {
814 %                     setSelectedDate(startOfDay(day));
815 %                     setViewMode("daily");
816 %                   }}
817 %                   data-testid={`weekly-cell-${cabin.id}-${format(day, "yyyy-MM-dd")}`}
818 %                 >
819 %                   <span className="text-lg font-semibold">{count}</span>
820 %                   <div className="text-[10px] opacity-70">{count === 1 ? "turno" : "turnos"}</div>
821 %                 </td>
822 %               );
823 %             })}
824 %           </tr>
825 %         ))}
826 %       </tbody>
827 %     </table>
828 %     <div className="flex items-center gap-4 p-3 text-xs text-muted-foreground border-t">
829 %       <span className="font-medium">Leyenda:</span>
830 %       <div className="flex items-center gap-1"><div className="w-3 h-3 rounded bg-gray-100 dark:bg-gray-8...
831 %         <div className="flex items-center gap-1"><div className="w-3 h-3 rounded bg-green-100 dark:bg-green...
832 %         <div className="flex items-center gap-1"><div className="w-3 h-3 rounded bg-amber-100 dark:bg-amber...
833 %         <div className="flex items-center gap-1"><div className="w-3 h-3 rounded bg-red-100 dark:bg-red-900...
834 %       </div>

```

client/src/pages/spa.tsx (cont.)

```
835 %         </CardContent>
836 %     </Card>
837 % }}
838 % </>
839 % }}
840 %
841 % {activeTab === "insumos" && (
842 %   <Card className="flex-1">
843 %     <CardHeader className="py-3 px-4">
844 %       <CardTitle className="text-base">Insumos del SPA</CardTitle>
845 %     </CardHeader>
846 %     <CardContent>
847 %       {spaInventoryItems.length === 0 ? (
848 %         <div className="text-center py-12 text-muted-foreground">
849 %           <Package className="h-12 w-12 mx-auto mb-3 opacity-40" />
850 %           <p className="text-sm">No hay insumos asignados al área SPA.</p>
851 %           <p className="text-xs mt-1">Asigná categorías con área "SPA" desde el módulo de Inventario.</p>
852 %         </div>
853 %       ) : (
854 %         <div className="space-y-2">
855 %           {spaInventoryItems.map((item) => (
856 %             <div key={item.id} className="flex items-center justify-between p-3 border rounded-lg" data-testid=...
857 %               <div>
858 %                 <p className="text-sm font-medium">{item.name}</p>
859 %                 <p className="text-xs text-muted-foreground">{item.category?.name}</p>
860 %               </div>
861 %               <div className="flex items-center gap-4">
862 %                 <div className="text-right">
863 %                   <p className="text-sm font-medium ${item.currentStock <= 0 ? "text-red-600" : ""}`">
864 %                     {item.currentStock} {item.unit}
865 %                   </p>
866 %                   <p className="text-xs text-muted-foreground">${parseFloat(item.costPrice).toLocaleString()}/u...
867 %                 </div>
868 %               </div>
869 %             </div>
870 %           )})
871 %         </div>
872 %       )
873 %     </CardContent>
874 %   </Card>
875 % }}
876 %
877 % {/* New / Edit Appointment Dialog */}
878 % <Dialog open={isNewDialogOpen} onOpenChange={(open) => { if (!open) { setIsNewDialogOpen(false); setIsEditMode(...
879 %   <DialogContent className="max-w-md">
880 %     <DialogHeader>
881 %       <DialogTitle>{isEditMode ? "Editar Turno" : "Nuevo Turno SPA"}</DialogTitle>
882 %     </DialogHeader>
883 %     <Form {...form}>
884 %       <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4">
885 %         <FormField control={form.control} name="appointmentDate" render={({ field }) => (
886 %           <FormItem>
887 %             <FormLabel>Fecha</FormLabel>
888 %             <FormControl><Input type="date" {...field} data-testid="input-appointment-date" /></FormControl>
889 %             <FormMessage />
890 %           </FormItem>
891 %         )} />
892 %       </form>
893 %       <div className="grid grid-cols-2 gap-4">
894 %         <FormField control={form.control} name="cabinId" render={({ field }) => (
895 %           <FormItem>
896 %             <FormLabel>Gabinete</FormLabel>
897 %             <Select onValueChange={field.onChange} value={field.value}>
898 %               <FormControl><SelectTrigger data-testid="select-cabin"><SelectValue placeholder="Seleccionar" /...
899 %               <SelectContent>
900 %                 {activeCabins.map((cabin) => (
901 %                   <SelectItem key={cabin.id} value={cabin.id}>{cabin.name}</SelectItem>
902 %                 )})
903 %               </SelectContent>
904 %             </Select>
905 %             <FormMessage />
906 %           </FormItem>
907 %         )} />
908 %         <FormField control={form.control} name="startTime" render={({ field }) => (
909 %           <FormItem>
910 %             <FormLabel>Hora</FormLabel>
911 %             <Select onValueChange={field.onChange} value={field.value}>
912 %               <FormControl><SelectTrigger data-testid="select-start-time"><SelectValue placeholder="Seleccion...
913 %               <SelectContent>
914 %                 {TIME_SLOTS.slice(0, -1).map((time) => (
915 %                   <SelectItem key={time} value={time}>{time}</SelectItem>
916 %                 )})
917 %               </SelectContent>
918 %             </Select>
919 %             <FormMessage />
920 %           </FormItem>
921 %         )} />
922 %       </div>
923 %       <FormField control={form.control} name="treatmentId" render={({ field }) => (
924 %         <FormItem>
925 %           <FormLabel>Tratamiento</FormLabel>
926 %           <Select onValueChange={field.onChange} value={field.value}>
```

client/src/pages/spa.tsx (cont.)

```
928 %         <FormControl><SelectTrigger data-testid="select-treatment"><SelectValue placeholder="Seleccionar ...
929 %         <SelectContent>
930 %             {treatments.filter(t => t.isActive === "true").map((treatment) => (
931 %                 <SelectItem key={treatment.id} value={treatment.id}>
932 %                     {treatment.name} - ${parseFloat(treatment.price).toLocaleString()} ({treatment.durationMinu...
933 %                 </SelectItem>
934 %             )}}
935 %         </SelectContent>
936 %     </Select>
937 %     <FormMessage />
938 % </FormItem>
939 % }) />
940 %
941 % <FormField control={form.control} name="reservationId" render={({ field }) => (
942 %     <FormItem>
943 %         <FormLabel>Huésped del Hotel (opcional)</FormLabel>
944 %         <Select onValueChange={field.onChange} value={field.value || undefined}>
945 %             <FormControl><SelectTrigger data-testid="select-reservation"><SelectValue placeholder="Sin asocia...
946 %             <SelectContent>
947 %                 {checkedInReservations.map((res) => (
948 %                     <SelectItem key={res.id} value={res.id}>
949 %                         Hab. {res.room?.roomNumber} - {res.guest?.firstName} {res.guest?.lastName}
950 %                     </SelectItem>
951 %                 )}}
952 %             </SelectContent>
953 %         </Select>
954 %         <FormMessage />
955 %     </FormItem>
956 % }) />
957 %
958 % <div className="grid grid-cols-2 gap-4">
959 %     <FormField control={form.control} name="guestName" render={({ field }) => (
960 %         <FormItem>
961 %             <FormLabel>Nombre</FormLabel>
962 %             <FormControl><Input {...field} data-testid="input-guest-name" /></FormControl>
963 %             <FormMessage />
964 %         </FormItem>
965 %     )} />
966 %     <FormField control={form.control} name="guestLastName" render={({ field }) => (
967 %         <FormItem>
968 %             <FormLabel>Apellido</FormLabel>
969 %             <FormControl><Input {...field} data-testid="input-guest-lastname" /></FormControl>
970 %             <FormMessage />
971 %         </FormItem>
972 %     )} />
973 % </div>
974 %
975 % <div className="grid grid-cols-2 gap-4">
976 %     <FormField control={form.control} name="guestPhone" render={({ field }) => (
977 %         <FormItem>
978 %             <FormLabel>Teléfono</FormLabel>
979 %             <FormControl><Input {...field} data-testid="input-guest-phone" /></FormControl>
980 %             <FormMessage />
981 %         </FormItem>
982 %     )} />
983 %     <FormField control={form.control} name="guestEmail" render={({ field }) => (
984 %         <FormItem>
985 %             <FormLabel>Email</FormLabel>
986 %             <FormControl><Input type="email" {...field} data-testid="input-guest-email" /></FormControl>
987 %             <FormMessage />
988 %         </FormItem>
989 %     )} />
990 % </div>
991 %
992 % <FormField control={form.control} name="notes" render={({ field }) => (
993 %     <FormItem>
994 %         <FormLabel>Notas</FormLabel>
995 %         <FormControl><Textarea {...field} data-testid="input-notes" /></FormControl>
996 %         <FormMessage />
997 %     </FormItem>
998 % }) />
999 %
1000 % <DialogFooter>
1001 %     <Button type="button" variant="outline" onClick={() => { setIsNewDialogOpen(false); setIsEditMode(fal...
1002 %     <Button type="submit" disabled={createAppointmentMutation.isPending || editAppointmentMutation.isPend...
1003 %         {(createAppointmentMutation.isPending || editAppointmentMutation.isPending) && <Loader2 className="...
1004 %         {isEditMode ? "Guardar Cambios" : "Crear Turno"}
1005 %     </Button>
1006 % </DialogFooter>
1007 % </form>
1008 % </DialogContent>
1009 % </Dialog>
1010 %
1011 %
1012 % /* Appointment Detail Dialog */
1013 % <Dialog open={!selectedAppointment && !isFolioOpen} onOpenChange={() => setSelectedAppointment(null)}>
1014 %     <DialogContent className="max-w-lg">
1015 %         <DialogHeader>
1016 %             <DialogTitle>Detalle del Turno</DialogTitle>
1017 %         </DialogHeader>
1018 %         {selectedAppointment && (
1019 %             <div className="space-y-4">
1020 %                 <div className="flex items-center gap-2">
```

client/src/pages/spa.tsx (cont.)

```
1021 %         <Badge className={appointmentStatusColors[selectedAppointment.status]}>
1022 %           {appointmentStatusLabels[selectedAppointment.status]}
1023 %         </Badge>
1024 %       {selectedAccount && selectedAccount.payments.some(p => p.isAdvance === "true") && (
1025 %         <Badge variant="secondary" className="bg-purple-100 text-purple-700 dark:bg-purple-900/30 dark:text...
1026 %         <DollarSign className="h-3 w-3 mr-1" /> SEÑA
1027 %       </Badge>
1028 %     )}
1029 %   </div>
1030 %
1031 %   <div className="grid grid-cols-2 gap-4">
1032 %     <div className="space-y-2">
1033 %       <div className="flex items-center gap-2 text-sm">
1034 %         <Calendar className="h-4 w-4 text-muted-foreground" />
1035 %         <span>{format(parseISO(selectedAppointment.appointmentDate), "EEEE d 'de' MMMM", { locale: es })}...
1036 %       </div>
1037 %       <div className="flex items-center gap-2 text-sm">
1038 %         <Clock className="h-4 w-4 text-muted-foreground" />
1039 %         <span>{selectedAppointment.startTime} - {selectedAppointment.endTime}</span>
1040 %       </div>
1041 %       <div className="flex items-center gap-2 text-sm">
1042 %         <User className="h-4 w-4 text-muted-foreground" />
1043 %         <span>{selectedAppointment.guestName} {selectedAppointment.guestLastName || ""}</span>
1044 %       </div>
1045 %       {selectedAppointment.guestPhone && (
1046 %         <div className="flex items-center gap-2 text-sm">
1047 %           <Phone className="h-4 w-4 text-muted-foreground" />
1048 %           <span>{selectedAppointment.guestPhone}</span>
1049 %         </div>
1050 %       )}
1051 %     </div>
1052 %     <div className="space-y-2">
1053 %       <h4 className="text-sm font-medium">Tratamiento</h4>
1054 %       <p className="text-sm text-muted-foreground">
1055 %         {treatments.find(t => t.id === selectedAppointment.treatmentId)?.name || "N/A"}
1056 %       </p>
1057 %       <h4 className="text-sm font-medium">Gabinete</h4>
1058 %       <p className="text-sm text-muted-foreground">
1059 %         {cabins.find(c => c.id === selectedAppointment.cabinId)?.name || "N/A"}
1060 %       </p>
1061 %     </div>
1062 %   </div>
1063 %
1064 %   {selectedAccount && selectedAccount.items.length > 0 && (
1065 %     <div className="border-t pt-4">
1066 %       <h4 className="text-sm font-medium mb-2 flex items-center gap-2">
1067 %         <Receipt className="h-4 w-4" /> Cuenta
1068 %       </h4>
1069 %       <div className="space-y-1 text-sm">
1070 %         {selectedAccount.items.map((item) => (
1071 %           <div key={item.id} className="flex justify-between">
1072 %             <span>{item.description} x{item.quantity}</span>
1073 %             <span>${parseFloat(item.subtotal).toLocaleString()}</span>
1074 %           </div>
1075 %         ))}
1076 %       <div className="flex justify-between font-bold pt-2 border-t">
1077 %         <span>Total</span>
1078 %         <span>${accountTotal.toLocaleString()}</span>
1079 %       </div>
1080 %       {accountPaid > 0 && (
1081 %         <div className="flex justify-between text-green-600">
1082 %           <span>Pagado</span>
1083 %           <span>${accountPaid.toLocaleString()}</span>
1084 %         </div>
1085 %       )}
1086 %     </div>
1087 %     {selectedAccount.status === "closed" && (
1088 %       <Badge className="mt-2" variant="secondary">Folio cerrado</Badge>
1089 %     )}
1090 %   </div>
1091 % })
1092 %
1093 %   {selectedAppointment.notes && (
1094 %     <div className="border-t pt-4">
1095 %       <h4 className="text-sm font-medium mb-2">Notas</h4>
1096 %       <p className="text-sm text-muted-foreground">{selectedAppointment.notes}</p>
1097 %     </div>
1098 %   )}
1099 %
1100 %   <DialogFooter className="flex-wrap gap-2">
1101 %     [{"pending", "confirmed"].includes(selectedAppointment.status) && (
1102 %       <Button variant="outline" size="sm" onClick={() => handleEditAppointment(selectedAppointment)} data...
1103 %       <Pencil className="h-4 w-4 mr-1" /> Editar
1104 %     </Button>
1105 %   )}
1106 %   {selectedAppointment.status === "confirmed" && (
1107 %     <Button variant="default" size="sm" onClick={() => {
1108 %       updateAppointmentMutation.mutate({ id: selectedAppointment.id, status: "in_progress" });
1109 %       setSelectedAppointment(null);
1110 %     }} disabled={updateAppointmentMutation.isPending} data-testid="button-start-appointment">
1111 %     Iniciar
1112 %   </Button>
1113 % )}
```

```

1114 %         [{"in_progress", "confirmed"].includes(selectedAppointment.status) && selectedAccount && selectedAcco...
1115 %         <Button variant="default" size="sm" onClick={handleOpenFolio} data-testid="button-open-folio">
1116 %             <CreditCard className="h-4 w-4 mr-1" /> Folio
1117 %         </Button>
1118 %     )}
1119 %     {selectedAppointment.status === "pending" && (
1120 %         <Button variant="default" size="sm" onClick={() => {
1121 %             updateAppointmentMutation.mutate({ id: selectedAppointment.id, status: "confirmed" });
1122 %             setSelectedAppointment(null);
1123 %         }} disabled={updateAppointmentMutation.isPending} data-testid="button-confirm-appointment">
1124 %             Confirmar
1125 %         </Button>
1126 %     )}
1127 %     [{"pending", "confirmed"].includes(selectedAppointment.status) && (
1128 %         <>
1129 %             <Button variant="outline" size="sm" onClick={() => {
1130 %                 updateAppointmentMutation.mutate({ id: selectedAppointment.id, status: "no_show" });
1131 %                 setSelectedAppointment(null);
1132 %             }} disabled={updateAppointmentMutation.isPending} data-testid="button-noshow-appointment">
1133 %                 No Show
1134 %             </Button>
1135 %             <Button variant="destructive" size="sm" onClick={() => setIsCancelConfirmOpen(true)} data-testid=...
1136 %                 <X className="h-4 w-4 mr-1" /> Cancelar
1137 %             </Button>
1138 %         </>
1139 %     )}
1140 % </DialogFooter>
1141 % </div>
1142 % )}
1143 % </DialogContent>
1144 % </Dialog>
1145 %
1146 % {/ * Cancel Confirmation Dialog */}
1147 % <Dialog open={isCancelConfirmOpen} onOpenChange={setIsCancelConfirmOpen}>
1148 %     <DialogContent className="max-w-sm">
1149 %         <DialogHeader>
1150 %             <DialogTitle className="flex items-center gap-2">
1151 %                 <AlertTriangle className="h-5 w-5 text-destructive" />
1152 %                 Confirmar Cancelación
1153 %             </DialogTitle>
1154 %         </DialogHeader>
1155 %         {selectedAppointment && (
1156 %             <div className="space-y-4">
1157 %                 <p className="text-sm text-muted-foreground">
1158 %                     ¿Estás seguro que querés cancelar el turno de <strong>{selectedAppointment.guestName} {selectedAppoin...
1159 %                 </p>
1160 %                 <DialogFooter>
1161 %                     <Button variant="outline" onClick={() => setIsCancelConfirmOpen(false)}>Volver</Button>
1162 %                     <Button variant="destructive" onClick={handleCancelAppointment} disabled={updateAppointmentMutation.i...
1163 %                         Si, cancelar turno
1164 %                     </Button>
1165 %                 </DialogFooter>
1166 %             </div>
1167 %         )}
1168 %     </DialogContent>
1169 % </Dialog>
1170 %
1171 % {/ * Folio Dialog */}
1172 % <Dialog open={isFolioOpen} onOpenChange={(open) => { if (!open) { setIsFolioOpen(false); } }}>
1173 %     <DialogContent className="max-w-3xl max-h-[85vh] overflow-y-auto">
1174 %         <DialogHeader>
1175 %             <DialogTitle className="flex items-center gap-2">
1176 %                 <Receipt className="h-5 w-5" />
1177 %                 Folio SPA - {selectedAppointment?.guestName} {selectedAppointment?.guestLastName || ""}
1178 %             </DialogTitle>
1179 %         </DialogHeader>
1180 %         {selectedAccount && selectedAppointment && (
1181 %             <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
1182 %                 {/ * Left: Charges */}
1183 %                 <div className="space-y-4">
1184 %                     <div className="flex items-center justify-between">
1185 %                         <h3 className="font-medium text-sm">Cargos</h3>
1186 %                         {selectedAccount.status === "open" && (
1187 %                             <Button size="sm" variant="outline" onClick={() => setIsAddChargeOpen(true)} data-testid="button-...
1188 %                             <Plus className="h-3 w-3 mr-1" /> Agregar
1189 %                         </Button>
1190 %                     )}
1191 %                 </div>
1192 %                 <div className="space-y-2">
1193 %                     {selectedAccount.items.map((item) => (
1194 %                         <div key={item.id} className="flex items-center justify-between p-2 border rounded text-sm" data-...
1195 %                             <div className="flex-1">
1196 %                                 <div className="flex items-center gap-2">
1197 %                                     <Badge variant="secondary" className="text-[10px]">{item.itemType}</Badge>
1198 %                                     <span>{item.description}</span>
1199 %                                 </div>
1200 %                                 <div className="text-xs text-muted-foreground mt-0.5">
1201 %                                     {item.quantity} x ${parseFloat(item.unitPrice).toLocaleString()}
1202 %                                 </div>
1203 %                             </div>
1204 %                             <div className="flex items-center gap-2">
1205 %                                 <span className="font-medium">${parseFloat(item.subtotal).toLocaleString()}</span>
1206 %                                 {selectedAccount.status === "open" && item.itemType !== "treatment" && (

```

client/src/pages/spa.tsx (cont.)

```
1207 %           <Button size="icon" variant="ghost" className="h-6 w-6" onClick={() => deleteChargeMutation...
1208 %             <Trash2 className="h-3 w-3 text-muted-foreground" />
1209 %           </Button>
1210 %         )}
1211 %       </div>
1212 %     </div>
1213 %   )}}
1214 % </div>
1215 % <div className="border-t pt-3 space-y-1">
1216 %   <div className="flex justify-between text-lg font-bold">
1217 %     <span>Total</span>
1218 %     <span>${accountTotal.toLocaleString()}</span>
1219 %   </div>
1220 %   <div className="flex justify-between text-sm text-green-600">
1221 %     <span>Pagado</span>
1222 %     <span>${accountPaid.toLocaleString()}</span>
1223 %   </div>
1224 %   <div className={`flex justify-between text-sm font-medium ${accountBalance > 0 ? "text-red-600" : "...
1225 %     <span>Saldo</span>
1226 %     <span>${accountBalance.toLocaleString()}</span>
1227 %   </div>
1228 % </div>
1229 % </div>
1230 %
1231 % /* Right: Payments */
1232 % <div className="space-y-4">
1233 %   <div className="flex items-center justify-between">
1234 %     <h3 className="font-medium text-sm">Pagos</h3>
1235 %     {selectedAccount.status === "open" && (
1236 %       <Button size="sm" variant="outline" onClick={() => setIsAddPaymentOpen(true)} data-testid="button...
1237 %       <DollarSign className="h-3 w-3 mr-1" /> Registrar Pago
1238 %     </Button>
1239 %   )}
1240 % </div>
1241 % <div className="space-y-2">
1242 %   {selectedAccount.payments.length === 0 ? (
1243 %     <p className="text-sm text-muted-foreground text-center py-4">Sin pagos registrados</p>
1244 %   ) : (
1245 %     selectedAccount.payments.map((payment) => (
1246 %       <div key={payment.id} className="flex items-center justify-between p-2 border rounded text-sm" ...
1247 %         <div>
1248 %           <div className="flex items-center gap-2">
1249 %             <span>{paymentMethodLabels[payment.method] || payment.method}</span>
1250 %             {payment.isAdvance === "true" && (
1251 %               <Badge variant="secondary" className="text-[10px] bg-purple-100 text-purple-700 dark:bg...
1252 %             )}
1253 %           </div>
1254 %           {payment.notes && <p className="text-xs text-muted-foreground">{payment.notes}</p>}
1255 %         </div>
1256 %         <span className="font-medium text-green-600">${parseFloat(payment.amount).toLocaleString()}</...
1257 %       </div>
1258 %     )}
1259 %   )}
1260 % </div>
1261 %
1262 % {selectedAccount.status === "open" && (
1263 %   <div className="border-t pt-4 space-y-3">
1264 %     <div>
1265 %       <label className="text-sm font-medium">Comprobante</label>
1266 %       <Select value={receiptType} onChange={setReceiptType}>
1267 %         <SelectTrigger data-testid="select-receipt-type">
1268 %           <SelectValue placeholder="Seleccionar comprobante" />
1269 %         </SelectTrigger>
1270 %         <SelectContent>
1271 %           <SelectItem value="ticket">Ticket</SelectItem>
1272 %           <SelectItem value="factura_a">Factura A</SelectItem>
1273 %           <SelectItem value="factura_b">Factura B</SelectItem>
1274 %           <SelectItem value="factura_c">Factura C</SelectItem>
1275 %           <SelectItem value="nota_credito">Nota de Crédito</SelectItem>
1276 %         </SelectContent>
1277 %       </Select>
1278 %     </div>
1279 %     <Button
1280 %       className="w-full"
1281 %       disabled={accountBalance > 0 || !receiptType || closeAccountMutation.isPending}
1282 %       onClick={() => closeAccountMutation.mutate({ accountId: selectedAccount.id, receiptType })}
1283 %       data-testid="button-close-folio"
1284 %     >
1285 %       {closeAccountMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1286 %       <Receipt className="h-4 w-4 mr-2" />
1287 %       Cerrar Folio
1288 %     </Button>
1289 %     {accountBalance > 0 && (
1290 %       <p className="text-xs text-red-500 text-center">Saldo pendiente: ${accountBalance.toLocaleStrin...
1291 %     )}
1292 %   </div>
1293 % )}
1294 % </div>
1295 % </div>
1296 % )}
1297 % </DialogContent>
1298 % </Dialog>
1299 %
```

client/src/pages/spa.tsx (cont.)

```
1300 %      {/* Add Charge Dialog */}
1301 %      <Dialog open={isAddChargeOpen} onOpenChange={setIsAddChargeOpen}>
1302 %        <DialogContent className="max-w-sm">
1303 %          <DialogHeader>
1304 %            <DialogTitle>Agregar Cargo</DialogTitle>
1305 %          </DialogHeader>
1306 %          <div className="space-y-4">
1307 %            <div>
1308 %              <label className="text-sm font-medium">Tipo</label>
1309 %              <Select value={chargeType} onValueChange={setChargeType}>
1310 %                <SelectTrigger data-testid="select-charge-type">
1311 %                  <SelectValue />
1312 %                </SelectTrigger>
1313 %                <SelectContent>
1314 %                  <SelectItem value="product">Producto</SelectItem>
1315 %                  <SelectItem value="service">Servicio Extra</SelectItem>
1316 %                  <SelectItem value="extra">Consumo (pileta, toalla, etc.)</SelectItem>
1317 %                </SelectContent>
1318 %              </Select>
1319 %            </div>
1320 %            <div>
1321 %              <label className="text-sm font-medium">Descripción</label>
1322 %              <Input value={chargeDescription} onChange={(e) => setChargeDescription(e.target.value)} placeholder="Ej...
1323 %            </div>
1324 %            <div className="grid grid-cols-2 gap-4">
1325 %              <div>
1326 %                <label className="text-sm font-medium">Precio Unit.</label>
1327 %                <Input type="number" step="0.01" value={chargePrice} onChange={(e) => setChargePrice(e.target.value)}...
1328 %              </div>
1329 %              <div>
1330 %                <label className="text-sm font-medium">Cantidad</label>
1331 %                <Input type="number" min="1" value={chargeQuantity} onChange={(e) => setChargeQuantity(e.target.value...
1332 %              </div>
1333 %            </div>
1334 %            <DialogFooter>
1335 %              <Button variant="outline" onClick={() => setIsAddChargeOpen(false)}>Cancelar</Button>
1336 %              <Button
1337 %                disabled={!chargeDescription || !chargePrice || addChargeMutation.isPending}
1338 %                onClick={() => {
1339 %                  if (selectedAccount) {
1340 %                    addChargeMutation.mutate({
1341 %                      accountId: selectedAccount.id,
1342 %                      description: chargeDescription,
1343 %                      quantity: parseInt(chargeQuantity) || 1,
1344 %                      unitPrice: chargePrice,
1345 %                      itemType: chargeType,
1346 %                    });
1347 %                  }
1348 %                }}
1349 %                data-testid="button-submit-charge"
1350 %              >
1351 %                {addChargeMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1352 %                Agregar
1353 %              </Button>
1354 %            </DialogFooter>
1355 %          </div>
1356 %        </DialogContent>
1357 %      </Dialog>
1358 %
1359 %      {/* Add Payment Dialog */}
1360 %      <Dialog open={isAddPaymentOpen} onOpenChange={setIsAddPaymentOpen}>
1361 %        <DialogContent className="max-w-sm">
1362 %          <DialogHeader>
1363 %            <DialogTitle>Registrar Pago</DialogTitle>
1364 %          </DialogHeader>
1365 %          <div className="space-y-4">
1366 %            <div>
1367 %              <label className="text-sm font-medium">Método de Pago</label>
1368 %              <Select value={paymentMethod} onValueChange={(v) => { setPaymentMethod(v); if (v !== "room_charge") set...
1369 %                <SelectTrigger data-testid="select-payment-method">
1370 %                  <SelectValue placeholder="Seleccionar" />
1371 %                </SelectTrigger>
1372 %                <SelectContent>
1373 %                  <SelectItem value="cash">Efectivo</SelectItem>
1374 %                  <SelectItem value="debit_card">Tarjeta Débito</SelectItem>
1375 %                  <SelectItem value="credit_card">Tarjeta Crédito</SelectItem>
1376 %                  <SelectItem value="transfer">Transferencia</SelectItem>
1377 %                  <SelectItem value="mercadopago">MercadoPago</SelectItem>
1378 %                  <SelectItem value="room_charge">Cargo a Habitación</SelectItem>
1379 %                </SelectContent>
1380 %              </Select>
1381 %            </div>
1382 %
1383 %            {paymentMethod === "room_charge" && (
1384 %              <div>
1385 %                <label className="text-sm font-medium">Habitación</label>
1386 %                <Select value={paymentReservationId} onValueChange={setPaymentReservationId}>
1387 %                  <SelectTrigger data-testid="select-payment-room">
1388 %                    <SelectValue placeholder="Seleccionar habitación" />
1389 %                  </SelectTrigger>
1390 %                  <SelectContent>
1391 %                    {checkedInReservations.map((res) => (
1392 %                      <SelectItem key={res.id} value={res.id}>
```


client/src/pages/spa.tsx (cont.)

```
1393 %           Hab. {res.room?.roomNumber} - {res.guest?.firstName} {res.guest?.lastName}
1394 %           </SelectedItem>
1395 %         )}}
1396 %       </SelectContent>
1397 %     </Select>
1398 %   </div>
1399 % })
1400 %
1401 % <div>
1402 %   <label className="text-sm font-medium">Monto</label>
1403 %   <input type="number" step="0.01" value={paymentAmount} onChange={(e) => setPaymentAmount(e.target.value...
1404 % </div>
1405 %
1406 % <div className="flex items-center gap-2">
1407 %   <input type="checkbox" id="is-advance" checked={isPaymentAdvance} onChange={(e) => setIsPaymentAdvance(...
1408 %   <label htmlFor="is-advance" className="text-sm">Es seña / anticipo</label>
1409 % </div>
1410 %
1411 % <DialogFooter>
1412 %   <Button variant="outline" onClick={() => setIsAddPaymentOpen(false)}>Cancelar</Button>
1413 %   <Button
1414 %     disabled={!paymentAmount || !paymentMethod || addPaymentMutation.isPending || (paymentMethod === "roo...
1415 %     onClick={() => {
1416 %       if (selectedAccount) {
1417 %         addPaymentMutation.mutate({
1418 %           accountId: selectedAccount.id,
1419 %           amount: paymentAmount,
1420 %           method: paymentMethod,
1421 %           isAdvance: isPaymentAdvance,
1422 %           reservationId: paymentMethod === "room_charge" ? paymentReservationId : undefined,
1423 %         });
1424 %       }
1425 %     }}
1426 %     data-testid="button-submit-payment"
1427 %   >
1428 %     {addPaymentMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1429 %     Registrar
1430 %   </Button>
1431 % </DialogFooter>
1432 % </div>
1433 % </DialogContent>
1434 % </Dialog>
1435 % </div>
1436 % );
1437 % }
1438 %
```

```
1 % import { useState, useMemo } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { useForm } from "react-hook-form";
5 % import { zodResolver } from "hookform/resolvers/zod";
6 % import { z } from "zod";
7 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
8 % import { Button } from "@components/ui/button";
9 % import { Badge } from "@components/ui/badge";
10 % import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter, DialogDescription } from "@components/ui/di...
11 % import { Input } from "@components/ui/input";
12 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
13 % import { Textarea } from "@components/ui/textarea";
14 % import { Label } from "@components/ui/label";
15 % import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
16 % import { ScrollArea, ScrollBar } from "@components/ui/scroll-area";
17 % import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@components/ui/table";
18 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
19 % import { AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent, AlertDialogDescription, AlertDialogFo...
20 % import { useToast } from "@hooks/use-toast";
21 % import { format, addDays, startOfDay, parseISO, isSameDay, addWeeks, subWeeks, isValid } from "date-fns";
22 % import { es } from "date-fns/locale";
23 % import {
24 %   Plus,
25 %   ChevronLeft,
26 %   ChevronRight,
27 %   Calendar,
28 %   User,
29 %   Phone,
30 %   Mail,
31 %   Loader2,
32 %   Building2,
33 %   Users,
34 %   Clock,
35 %   CreditCard,
36 %   Trash2,
37 %   Edit,
38 %   Receipt,
39 %   DollarSign,
40 %   Info,
41 %   UtensilsCrossed,
42 % } from "lucide-react";
43 %
44 % type EventRoom = {
45 %   id: string;
46 %   name: string;
47 %   capacity: number;
48 %   description: string | null;
49 %   isActive: boolean;
50 % };
51 %
52 % type EventChargeType = {
53 %   id: string;
54 %   name: string;
55 %   category: string;
56 %   defaultPrice: string;
57 %   description: string | null;
58 %   isActive: boolean;
59 % };
60 %
61 % type EventCharge = {
62 %   id: string;
63 %   eventId: string;
64 %   chargeTypeId: string | null;
65 %   description: string;
66 %   quantity: number;
67 %   unitPrice: string;
68 %   totalAmount: string;
69 %   date: string;
70 %   notes: string | null;
71 %   createdAt: string;
72 % };
73 %
74 % type EventPayment = {
75 %   id: string;
76 %   eventId: string;
77 %   amount: string;
78 %   method: string;
79 %   isAdvance: string;
80 %   reservationId: string | null;
81 %   notes: string | null;
82 %   paidAt: string;
83 %   createdAt: string;
84 % };
85 %
86 % type EventTableCharge = {
87 %   id: string;
88 %   eventTableId: string;
89 %   description: string;
90 %   quantity: number;
```

client/src/pages/events.tsx (cont.)

```
91 %     unitPrice: string;
92 %     total: string;
93 %     createdAt: string;
94 % };
95 %
96 % type EventTablePayment = {
97 %     id: string;
98 %     eventId: string;
99 %     amount: string;
100 %    method: string;
101 %    isAdvance: string;
102 %    reservationId: string | null;
103 %    receiptType: string | null;
104 %    paidAt: string;
105 %    createdAt: string;
106 % };
107 %
108 % type EventTableType = {
109 %     id: string;
110 %     eventId: string;
111 %     tableNumber: number;
112 %     label: string | null;
113 %     seats: number | null;
114 %     status: string;
115 %     reservationId: string | null;
116 %     receiptType: string | null;
117 %     closedAt: string | null;
118 %     createdAt: string;
119 %     charges: EventTableCharge[];
120 %     payments: EventTablePayment[];
121 % };
122 %
123 % type HotelEvent = {
124 %     id: string;
125 %     eventCode: string;
126 %     name: string;
127 %     eventRoomId: string;
128 %     eventType: "corporate" | "social" | "wedding" | "conference" | "meeting" | "table_event" | "other";
129 %     contactName: string;
130 %     contactPhone: string | null;
131 %     contactEmail: string | null;
132 %     companyId: string | null;
133 %     startDate: string;
134 %     endDate: string;
135 %     startTime: string | null;
136 %     endTime: string | null;
137 %     attendees: number;
138 %     status: "tentative" | "confirmed" | "in_progress" | "completed" | "cancelled" | "invoiced";
139 %     notes: string | null;
140 %     receiptType: string | null;
141 %     closedAt: string | null;
142 %     totalAmount: string | null;
143 %     totalPaid: string | null;
144 %     createdAt: string;
145 %     eventRoom?: EventRoom;
146 %     charges?: EventCharge[];
147 %     payments?: EventPayment[];
148 % };
149 %
150 % type Reservation = {
151 %     id: string;
152 %     reservationCode: string;
153 %     guestId: string;
154 %     roomId: string;
155 %     status: string;
156 %     guest?: { firstName: string; lastName: string };
157 %     room?: { roomNumber: string };
158 % };
159 %
160 % type Company = {
161 %     id: string;
162 %     name: string;
163 % };
164 %
165 % const eventStatusColors: Record<string, string> = {
166 %     tentative: "bg-yellow-500/20 text-yellow-700 dark:text-yellow-400 border-yellow-500/30",
167 %     confirmed: "bg-blue-500/20 text-blue-700 dark:text-blue-400 border-blue-500/30",
168 %     in_progress: "bg-green-500/20 text-green-700 dark:text-green-400 border-green-500/30",
169 %     completed: "bg-gray-500/20 text-gray-700 dark:text-gray-400 border-gray-500/30",
170 %     cancelled: "bg-red-500/20 text-red-700 dark:text-red-400 border-red-500/30",
171 %     invoiced: "bg-purple-500/20 text-purple-700 dark:text-purple-400 border-purple-500/30",
172 % };
173 %
174 % const eventStatusLabels: Record<string, string> = {
175 %     tentative: "Tentativo",
176 %     confirmed: "Confirmado",
177 %     in_progress: "En Curso",
178 %     completed: "Completado",
179 %     cancelled: "Cancelado",
180 %     invoiced: "Facturado",
181 % };
182 %
183 % const eventTypeLabels: Record<string, string> = {
```

client/src/pages/events.tsx (cont.)

```
184 %     corporate: "Corporativo",
185 %     social: "Social",
186 %     wedding: "Boda",
187 %     conference: "Conferencia",
188 %     meeting: "Reunion",
189 %     table_event: "Evento por Mesa",
190 %     other: "Otro",
191 % };
192 %
193 % const paymentMethodLabels: Record<string, string> = {
194 %     efectivo: "Efectivo",
195 %     tarjeta_debito: "Tarjeta Debito",
196 %     tarjeta_credito: "Tarjeta Credito",
197 %     transferencia: "Transferencia",
198 %     mercadopago: "MercadoPago",
199 %     room_charge: "Cargo a Habitacion",
200 %     cuenta_corriente: "Cuenta Corriente",
201 % };
202 %
203 % const receiptTypes = ["Ticket", "Factura A", "Factura B", "Factura C", "Nota Credito"];
204 %
205 % function safeFormatDate(dateStr: string, fmt: string, opts?: any): string {
206 %     try {
207 %         const d = parseISO(dateStr);
208 %         if (!isValid(d)) return dateStr;
209 %         return format(d, fmt, opts);
210 %     } catch {
211 %         return dateStr;
212 %     }
213 % }
214 %
215 % const eventFormSchema = z.object({
216 %     eventRoomId: z.string().min(1, "Seleccione un salon"),
217 %     name: z.string().min(1, "El nombre del evento es requerido"),
218 %     eventType: z.enum(["corporate", "social", "wedding", "conference", "meeting", "table_event", "other"]),
219 %     contactName: z.string().min(1, "El nombre de contacto es requerido"),
220 %     contactPhone: z.string().optional(),
221 %     contactEmail: z.string().email("Email invalido").optional().or(z.literal("")),
222 %     startDate: z.string().min(1, "La fecha de inicio es requerida"),
223 %     endDate: z.string().min(1, "La fecha de fin es requerida"),
224 %     startTime: z.string().optional(),
225 %     endTime: z.string().optional(),
226 %     attendees: z.number().min(1, "Debe tener al menos 1 asistente"),
227 %     notes: z.string().optional(),
228 % });
229 %
230 % type EventFormValues = z.infer<typeof eventFormSchema>;
231 %
232 % const chargeFormSchema = z.object({
233 %     chargeTypeId: z.string().optional(),
234 %     description: z.string().min(1, "La descripcion es requerida"),
235 %     quantity: z.number().min(1, "La cantidad debe ser al menos 1"),
236 %     unitPrice: z.string().min(1, "El precio es requerido"),
237 %     notes: z.string().optional(),
238 % });
239 %
240 % type ChargeFormValues = z.infer<typeof chargeFormSchema>;
241 %
242 % type PlanningEvent = {
243 %     id: string;
244 %     name: string;
245 %     contactName: string;
246 %     startDate: string;
247 %     endDate: string;
248 %     status: "tentative" | "confirmed" | "in_progress" | "completed" | "cancelled" | "invoiced";
249 %     eventType: "corporate" | "social" | "wedding" | "conference" | "meeting" | "table_event" | "other";
250 % };
251 %
252 % type EventPlanningResponse = {
253 %     rooms: EventRoom[];
254 %     days: string[];
255 %     events: Record<string, PlanningEvent>;
256 %     cellEvents: Record<string, Record<string, string>>;
257 % };
258 %
259 % export default function EventsPage() {
260 %     const [weekStart, setWeekStart] = useState(startOfDay(new Date()));
261 %     const [selectedEvent, setSelectedEvent] = useState<HotelEvent | null>(null);
262 %     const [isNewDialogOpen, setIsNewDialogOpen] = useState(false);
263 %     const [isChargeDialogOpen, setIsChargeDialogOpen] = useState(false);
264 %     const [prefilledRoomId, setPrefilledRoomId] = useState<string>("");
265 %     const [prefilledDate, setPrefilledDate] = useState<string>("");
266 %     const [cancelConfirmOpen, setCancelConfirmOpen] = useState(false);
267 %     const [folioReceiptType, setFolioReceiptType] = useState("");
268 %     const [activeTab, setActiveTab] = useState("details");
269 %     const [isTableFolioOpen, setIsTableFolioOpen] = useState(false);
270 %     const [selectedTable, setSelectedTable] = useState<EventTableType | null>(null);
271 %     const [tableFolioReceiptType, setTableFolioReceiptType] = useState("");
272 %     const [isAddTableOpen, setIsAddTableOpen] = useState(false);
273 %     const [newTableNumber, setNewTableNumber] = useState(1);
274 %     const [newTableLabel, setNewTableLabel] = useState("");
275 %     const [newTableSeats, setNewTableSeats] = useState(4);
276 %     const [paymentAmount, setPaymentAmount] = useState("");
```

client/src/pages/events.tsx (cont.)

```
277 %   const [paymentMethod, setPaymentMethod] = useState("efectivo");
278 %   const [paymentIsAdvance, setPaymentIsAdvance] = useState(false);
279 %   const [paymentReservationId, setPaymentReservationId] = useState("");
280 %   const [paymentNotes, setPaymentNotes] = useState("");
281 %   const [tableChargeDesc, setTableChargeDesc] = useState("");
282 %   const [tableChargeQty, setTableChargeQty] = useState(1);
283 %   const [tableChargePrice, setTableChargePrice] = useState("");
284 %   const [tablePayAmount, setTablePayAmount] = useState("");
285 %   const [tablePayMethod, setTablePayMethod] = useState("efectivo");
286 %   const [tablePayAdvance, setTablePayAdvance] = useState(false);
287 %   const [tablePayResId, setTablePayResId] = useState("");
288 %   const { toast } = useToast();
289 %
290 %   const weekDays = useMemo(() => {
291 %     return Array.from({ length: 7 }, (_, i) => addDays(weekStart, i));
292 %   }, [weekStart]);
293 %
294 %   const startDateStr = format(weekStart, "yyyy-MM-dd");
295 %   const endDateStr = format(addDays(weekStart, 6), "yyyy-MM-dd");
296 %
297 %   const { data: eventRooms = [], isLoading: roomsLoading } = useQuery<EventRoom[]>({
298 %     queryKey: ["/api/events/rooms"],
299 %   });
300 %
301 %   const { data: chargeTypes = [] } = useQuery<EventChargeType[]>({
302 %     queryKey: ["/api/events/charge-types"],
303 %   });
304 %
305 %   const { data: planningData, isLoading: planningLoading } = useQuery<EventPlanningResponse>({
306 %     queryKey: ["/api/events/planning", startDateStr, endDateStr],
307 %     queryFn: () =>
308 %       fetch(`/api/events/planning?start=${startDateStr}&end=${endDateStr}`)
309 %         .then(res => res.json()),
310 %   });
311 %
312 %   const { data: reservations = [] } = useQuery<Reservation[]>({
313 %     queryKey: ["/api/reservations"],
314 %     queryFn: () => fetch("/api/reservations").then(r => r.json()),
315 %   });
316 %
317 %   const activeReservations = reservations.filter(r => r.status === "checked_in" || r.status === "confirmed");
318 %
319 %   const { data: eventTables = [], refetch: refetchTables } = useQuery<EventTableType[]>({
320 %     queryKey: ["/api/events", selectedEvent?.id, "tables"],
321 %     queryFn: () => fetch(`/api/events/${selectedEvent!.id}/tables`).then(r => r.json()),
322 %     enabled: !!selectedEvent && selectedEvent.eventType === "table_event",
323 %   });
324 %
325 %   const eventsMap = planningData?.events || {};
326 %   const cellEventsMap = planningData?.cellEvents || {};
327 %
328 %   const eventForm = useForm<EventFormValues>({
329 %     resolver: zodResolver(eventFormSchema),
330 %     defaultValues: {
331 %       eventRoomId: "",
332 %       name: "",
333 %       eventType: "corporate",
334 %       contactName: "",
335 %       contactPhone: "",
336 %       contactEmail: "",
337 %       startDate: "",
338 %       endDate: "",
339 %       startTime: "",
340 %       endTime: "",
341 %       attendees: 10,
342 %       notes: "",
343 %     },
344 %   });
345 %
346 %   const chargeForm = useForm<ChargeFormValues>({
347 %     resolver: zodResolver(chargeFormSchema),
348 %     defaultValues: {
349 %       chargeTypeId: "",
350 %       description: "",
351 %       quantity: 1,
352 %       unitPrice: "",
353 %       notes: "",
354 %     },
355 %   });
356 %
357 %   const createEventMutation = useMutation({
358 %     mutationFn: (data: EventFormValues) => apiRequest("POST", "/api/events", data),
359 %     onSuccess: () => {
360 %       queryClient.invalidateQueries({ queryKey: ["/api/events/planning"] });
361 %       queryClient.invalidateQueries({ queryKey: ["/api/events"] });
362 %       setIsNewDialogOpen(false);
363 %       eventForm.reset();
364 %       toast({ title: "Evento creado exitosamente" });
365 %     },
366 %     onError: async (error: any) => {
367 %       try {
368 %         const msg = error?.message || "Error al crear el evento";
369 %         if (msg.includes("Superposición") || msg.includes("409")) {
```

client/src/pages/events.tsx (cont.)

```
370 %         toast({ title: "Conflicto de horario", description: msg, variant: "destructive" });
371 %     } else {
372 %         toast({ title: msg, variant: "destructive" });
373 %     }
374 % } catch {
375 %     toast({ title: "Error al crear el evento", variant: "destructive" });
376 % }
377 % },
378 % });
379 %
380 % const updateEventMutation = useMutation({
381 %     mutationFn: ({ id, data }: { id: string; data: Partial<EventFormValues & { status: string }> }) =>
382 %         apiRequest("PATCH", `/api/events/${id}`, data),
383 %     onSuccess: () => {
384 %         queryClient.invalidateQueries({ queryKey: ["/api/events/planning"] });
385 %         queryClient.invalidateQueries({ queryKey: ["/api/events"] });
386 %         toast({ title: "Evento actualizado" });
387 %     },
388 %     onError: async (error: any) => {
389 %         const msg = error?.message || "Error al actualizar el evento";
390 %         toast({ title: msg, variant: "destructive" });
391 %     },
392 % });
393 %
394 % const createChargeMutation = useMutation({
395 %     mutationFn: ({ eventId, data }: { eventId: string; data: ChargeFormValues }) =>
396 %         apiRequest("POST", `/api/events/${eventId}/charges`, data),
397 %     onSuccess: async () => {
398 %         queryClient.invalidateQueries({ queryKey: ["/api/events/planning"] });
399 %         if (selectedEvent) {
400 %             const response = await fetch(`/api/events/${selectedEvent.id}`);
401 %             if (response.ok) {
402 %                 const updatedEvent = await response.json();
403 %                 setSelectedEvent(updatedEvent);
404 %             }
405 %         }
406 %         setIsChargeDialogOpen(false);
407 %         chargeForm.reset();
408 %         toast({ title: "Cargo agregado" });
409 %     },
410 %     onError: () => {
411 %         toast({ title: "Error al agregar el cargo", variant: "destructive" });
412 %     },
413 % });
414 %
415 % const deleteChargeMutation = useMutation({
416 %     mutationFn: (chargeId: string) => apiRequest("DELETE", `/api/events/charges/${chargeId}`),
417 %     onSuccess: async () => {
418 %         queryClient.invalidateQueries({ queryKey: ["/api/events/planning"] });
419 %         if (selectedEvent) {
420 %             const response = await fetch(`/api/events/${selectedEvent.id}`);
421 %             if (response.ok) {
422 %                 const updatedEvent = await response.json();
423 %                 setSelectedEvent(updatedEvent);
424 %             }
425 %         }
426 %         toast({ title: "Cargo eliminado" });
427 %     },
428 %     onError: () => {
429 %         toast({ title: "Error al eliminar el cargo", variant: "destructive" });
430 %     },
431 % });
432 %
433 % const refreshSelectedEvent = async () => {
434 %     if (!selectedEvent) return;
435 %     const response = await fetch(`/api/events/${selectedEvent.id}`);
436 %     if (response.ok) {
437 %         const updatedEvent = await response.json();
438 %         setSelectedEvent(updatedEvent);
439 %     }
440 % };
441 %
442 % const addPaymentMutation = useMutation({
443 %     mutationFn: ({ eventId, data }: { eventId: string; data: any }) =>
444 %         apiRequest("POST", `/api/events/${eventId}/payments`, data),
445 %     onSuccess: async () => {
446 %         await refreshSelectedEvent();
447 %         setPaymentAmount("");
448 %         setPaymentMethod("efectivo");
449 %         setPaymentIsAdvance(false);
450 %         setPaymentReservationId("");
451 %         setPaymentNotes("");
452 %         toast({ title: "Pago registrado" });
453 %     },
454 %     onError: () => {
455 %         toast({ title: "Error al registrar el pago", variant: "destructive" });
456 %     },
457 % });
458 %
459 % const deletePaymentMutation = useMutation({
460 %     mutationFn: ({ eventId, payId }: { eventId: string; payId: string }) =>
461 %         apiRequest("DELETE", `/api/events/${eventId}/payments/${payId}`),
462 %     onSuccess: async () => {
```

client/src/pages/events.tsx (cont.)

```
463 %         await refreshSelectedEvent();
464 %         toast({ title: "Pago eliminado" });
465 %     },
466 %     onError: (error: any) => {
467 %         toast({ title: error?.message || "Error al eliminar el pago", variant: "destructive" });
468 %     },
469 % });
470 %
471 % const closeEventMutation = useMutation({
472 %     mutationFn: ({ eventId, receiptType }: { eventId: string; receiptType: string }) =>
473 %         apiRequest("POST", `/api/events/${eventId}/close`, { receiptType }),
474 %     onSuccess: async () => {
475 %         await refreshSelectedEvent();
476 %         queryClient.invalidateQueries({ queryKey: ["/api/events/planning"] });
477 %         toast({ title: "Evento facturado exitosamente" });
478 %     },
479 %     onError: (error: any) => {
480 %         toast({ title: error?.message || "Error al cerrar el evento", variant: "destructive" });
481 %     },
482 % });
483 %
484 % const addTableChargeMutation = useMutation({
485 %     mutationFn: ({ eventId, tableId, data }: { eventId: string; tableId: string; data: any }) =>
486 %         apiRequest("POST", `/api/events/${eventId}/tables/${tableId}/charges`, data),
487 %     onSuccess: async () => {
488 %         await refetchTables();
489 %         if (selectedTable) {
490 %             const resp = await fetch(`/api/events/${selectedEvent!.id}/tables`);
491 %             const tables: EventTableType[] = await resp.json();
492 %             const updated = tables.find(t => t.id === selectedTable.id);
493 %             if (updated) setSelectedTable(updated);
494 %         }
495 %         setTableChargeDesc("");
496 %         setTableChargeQty(1);
497 %         setTableChargePrice("");
498 %         toast({ title: "Cargo agregado a la mesa" });
499 %     },
500 %     onError: () => {
501 %         toast({ title: "Error al agregar cargo", variant: "destructive" });
502 %     },
503 % });
504 %
505 % const deleteTableChargeMutation = useMutation({
506 %     mutationFn: ({ eventId, tableId, chargeId }: { eventId: string; tableId: string; chargeId: string }) =>
507 %         apiRequest("DELETE", `/api/events/${eventId}/tables/${tableId}/charges/${chargeId}`),
508 %     onSuccess: async () => {
509 %         await refetchTables();
510 %         if (selectedTable) {
511 %             const resp = await fetch(`/api/events/${selectedEvent!.id}/tables`);
512 %             const tables: EventTableType[] = await resp.json();
513 %             const updated = tables.find(t => t.id === selectedTable.id);
514 %             if (updated) setSelectedTable(updated);
515 %         }
516 %         toast({ title: "Cargo eliminado" });
517 %     },
518 %     onError: () => {
519 %         toast({ title: "Error al eliminar cargo", variant: "destructive" });
520 %     },
521 % });
522 %
523 % const addTablePaymentMutation = useMutation({
524 %     mutationFn: ({ eventId, tableId, data }: { eventId: string; tableId: string; data: any }) =>
525 %         apiRequest("POST", `/api/events/${eventId}/tables/${tableId}/payments`, data),
526 %     onSuccess: async () => {
527 %         await refetchTables();
528 %         if (selectedTable) {
529 %             const resp = await fetch(`/api/events/${selectedEvent!.id}/tables`);
530 %             const tables: EventTableType[] = await resp.json();
531 %             const updated = tables.find(t => t.id === selectedTable.id);
532 %             if (updated) setSelectedTable(updated);
533 %         }
534 %         setTablePayAmount("");
535 %         setTablePayMethod("efectivo");
536 %         setTablePayAdvance(false);
537 %         setTablePayResId("");
538 %         toast({ title: "Pago registrado en mesa" });
539 %     },
540 %     onError: () => {
541 %         toast({ title: "Error al registrar pago", variant: "destructive" });
542 %     },
543 % });
544 %
545 % const closeTableMutation = useMutation({
546 %     mutationFn: ({ eventId, tableId, receiptType }: { eventId: string; tableId: string; receiptType: string }) =>
547 %         apiRequest("POST", `/api/events/${eventId}/tables/${tableId}/close`, { receiptType }),
548 %     onSuccess: async () => {
549 %         await refetchTables();
550 %         setIsTableFolioOpen(false);
551 %         setSelectedTable(null);
552 %         toast({ title: "Mesa cerrada exitosamente" });
553 %     },
554 %     onError: (error: any) => {
555 %         toast({ title: error?.message || "Error al cerrar mesa", variant: "destructive" });
```

client/src/pages/events.tsx (cont.)

```
556 %     },
557 %   });
558 %
559 %   const createTableMutation = useMutation({
560 %     mutationFn: ({ eventId, data }: { eventId: string; data: any }) =>
561 %       apiRequest("POST", `/api/events/${eventId}/tables`, data),
562 %     onSuccess: async () => {
563 %       await refetchTables();
564 %       setIsAddTableOpen(false);
565 %       setNewTableNumber(n => n + 1);
566 %       setNewTableLabel("");
567 %       toast({ title: "Mesa agregada" });
568 %     },
569 %     onError: () => {
570 %       toast({ title: "Error al agregar mesa", variant: "destructive" });
571 %     },
572 %   });
573 %
574 %   const deleteTableMutation = useMutation({
575 %     mutationFn: ({ eventId, tableId }: { eventId: string; tableId: string }) =>
576 %       apiRequest("DELETE", `/api/events/${eventId}/tables/${tableId}`),
577 %     onSuccess: async () => {
578 %       await refetchTables();
579 %       toast({ title: "Mesa eliminada" });
580 %     },
581 %     onError: (error: any) => {
582 %       toast({ title: error?.message || "Error al eliminar mesa", variant: "destructive" });
583 %     },
584 %   });
585 %
586 %   const handleCellClick = (roomId: string, date: Date) => {
587 %     const dateStr = format(date, "yyyy-MM-dd");
588 %     setPrefilledRoomId(roomId);
589 %     setPrefilledDate(dateStr);
590 %     eventForm.reset({
591 %       eventId: roomId,
592 %       name: "",
593 %       eventType: "corporate",
594 %       contactName: "",
595 %       contactPhone: "",
596 %       contactEmail: "",
597 %       startDate: dateStr,
598 %       endDate: dateStr,
599 %       startTime: "",
600 %       endTime: "",
601 %       attendees: 10,
602 %       notes: "",
603 %     });
604 %     setIsNewDialogOpen(true);
605 %   };
606 %
607 %   const handleEventClick = async (eventId: string) => {
608 %     try {
609 %       const response = await fetch(`/api/events/${eventId}`);
610 %       if (response.ok) {
611 %         const fullEvent = await response.json();
612 %         setSelectedEvent(fullEvent);
613 %         setActiveTab("details");
614 %       }
615 %     } catch (error) {
616 %       toast({ title: "Error al cargar el evento", variant: "destructive" });
617 %     }
618 %   };
619 %
620 %   const onSubmitEvent = (data: EventFormValues) => {
621 %     createEventMutation.mutate(data);
622 %   };
623 %
624 %   const onSubmitCharge = (data: ChargeFormValues) => {
625 %     if (!selectedEvent) return;
626 %     createChargeMutation.mutate({ eventId: selectedEvent.id, data });
627 %   };
628 %
629 %   const handleChargeTypeSelect = (chargeTypeId: string) => {
630 %     const chargeType = chargeTypes.find(ct => ct.id === chargeTypeId);
631 %     if (chargeType) {
632 %       chargeForm.setValue("description", chargeType.name);
633 %       chargeForm.setValue("unitPrice", chargeType.defaultPrice);
634 %     }
635 %   };
636 %
637 %   const getEventsForCell = (roomId: string, date: Date): PlanningEvent[] => {
638 %     const dateStr = format(date, "yyyy-MM-dd");
639 %     const eventId = cellEventsMap[roomId]?.[dateStr];
640 %     if (eventId && eventsMap[eventId]) {
641 %       return [eventsMap[eventId]];
642 %     }
643 %     return [];
644 %   };
645 %
646 %   const calculateEventTotal = (event: HotelEvent): number => {
647 %     if (!event.charges) return 0;
648 %     return event.charges.reduce((sum, charge) => sum + parseFloat(charge.totalAmount), 0);
```


client/src/pages/events.tsx (cont.)

```
649 % };
650 %
651 % const calculateEventPaid = (event: HotelEvent): number => {
652 %   if (!event.payments) return 0;
653 %   return event.payments.reduce((sum, p) => sum + parseFloat(p.amount), 0);
654 % };
655 %
656 % const handleAddPayment = () => {
657 %   if (!selectedEvent || !paymentAmount || parseFloat(paymentAmount) <= 0) return;
658 %   addPaymentMutation.mutate({
659 %     eventId: selectedEvent.id,
660 %     data: {
661 %       amount: paymentAmount,
662 %       method: paymentMethod,
663 %       isAdvance: paymentIsAdvance,
664 %       reservationId: paymentMethod === "room_charge" ? paymentReservationId : null,
665 %       notes: paymentNotes || null,
666 %     },
667 %   });
668 % };
669 %
670 % const handleCloseEvent = () => {
671 %   if (!selectedEvent || !folioReceiptType) return;
672 %   closeEventMutation.mutate({ eventId: selectedEvent.id, receiptType: folioReceiptType });
673 % };
674 %
675 % const handleAddTableCharge = () => {
676 %   if (!selectedEvent || !selectedTable || !tableChargeDesc || !tableChargePrice) return;
677 %   addTableChargeMutation.mutate({
678 %     eventId: selectedEvent.id,
679 %     tableId: selectedTable.id,
680 %     data: { description: tableChargeDesc, quantity: tableChargeQty, unitPrice: tableChargePrice },
681 %   });
682 % };
683 %
684 % const handleAddTablePayment = () => {
685 %   if (!selectedEvent || !selectedTable || !tablePayAmount || parseFloat(tablePayAmount) <= 0) return;
686 %   addTablePaymentMutation.mutate({
687 %     eventId: selectedEvent.id,
688 %     tableId: selectedTable.id,
689 %     data: {
690 %       amount: tablePayAmount,
691 %       method: tablePayMethod,
692 %       isAdvance: tablePayAdvance,
693 %       reservationId: tablePayMethod === "room_charge" ? tablePayResId : null,
694 %     },
695 %   });
696 % };
697 %
698 % const handleCloseTable = () => {
699 %   if (!selectedEvent || !selectedTable || !tableFolioReceiptType) return;
700 %   closeTableMutation.mutate({
701 %     eventId: selectedEvent.id,
702 %     tableId: selectedTable.id,
703 %     receiptType: tableFolioReceiptType,
704 %   });
705 % };
706 %
707 % const canShowFolio = (event: HotelEvent) => {
708 %   return ["confirmed", "in_progress", "completed"].includes(event.status);
709 % };
710 %
711 % if (roomsLoading) {
712 %   return (
713 %     <div className="flex items-center justify-center h-full">
714 %       <Loader2 className="h-8 w-8 animate-spin" />
715 %     </div>
716 %   );
717 % }
718 %
719 % const detailTabCount = selectedEvent?.eventType === "table_event" ? 4 : 3;
720 %
721 % return (
722 %   <div className="flex flex-col h-full p-4 gap-4">
723 %     <Card>
724 %       <CardHeader className="pb-4">
725 %         <div className="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-4">
726 %           <CardTitle className="flex items-center gap-2">
727 %             <Calendar className="h-5 w-5" />
728 %             Eventos - Planificacion Semanal
729 %           </CardTitle>
730 %           <div className="flex flex-wrap items-center gap-2">
731 %             <div className="flex items-center gap-1">
732 %               <Button
733 %                 variant="outline"
734 %                 size="icon"
735 %                 onClick={() => setWeekStart(subWeeks(weekStart, 1))}
736 %                 data-testid="button-prev-week"
737 %               >
738 %                 <ChevronLeft className="h-4 w-4" />
739 %               </Button>
740 %               <span className="text-sm font-medium min-w-[180px] text-center">
741 %                 {format(weekStart, "d MMM", { locale: es })} - {format(addDays(weekStart, 6), "d MMM yyyy", { local...
```

```

742 %         </span>
743 %         <Button
744 %             variant="outline"
745 %             size="icon"
746 %             onClick={() => setWeekStart(addWeeks(weekStart, 1))}
747 %             data-testid="button-next-week"
748 %         >
749 %             <ChevronRight className="h-4 w-4" />
750 %         </Button>
751 %     </div>
752 %     <Button onClick={() => setWeekStart(startOfDay(new Date()))} variant="outline" data-testid="button-today">
753 %         Hoy
754 %     </Button>
755 %     <Button
756 %         onClick={() => {
757 %             eventForm.reset();
758 %             setPrefilledRoomId("");
759 %             setPrefilledDate("");
760 %             setIsNewDialogOpen(true);
761 %         }}
762 %         data-testid="button-new-event"
763 %     >
764 %         <Plus className="h-4 w-4 mr-2" />
765 %         Nuevo Evento
766 %     </Button>
767 % </div>
768 % </div>
769 % </CardHeader>
770 % <CardContent>
771 %     <ScrollArea className="w-full">
772 %         <div className="min-w-[900px]">
773 %             <Table>
774 %                 <TableHeader>
775 %                     <TableRow>
776 %                         <TableHead className="w-[150px] sticky left-0 bg-background z-10">Salon</TableHead>
777 %                         {weekDays.map((day) => (
778 %                             <TableHead key={day.toISOString()} className="text-center min-w-[120px]">
779 %                                 <div className="flex flex-col">
780 %                                     <span className="text-xs text-muted-foreground">
781 %                                         {format(day, "EEE", { locale: es })}
782 %                                     </span>
783 %                                     <span className={isSameDay(day, new Date()) ? "font-bold text-primary" : ""}>
784 %                                         {format(day, "d")}
785 %                                     </span>
786 %                                 </div>
787 %                             </TableHead>
788 %                         ))}
789 %                     </TableRow>
790 %                 </TableHeader>
791 %                 <TableBody>
792 %                     {eventRooms.filter(r => r.isActive).map((room) => (
793 %                         <TableRow key={room.id}>
794 %                             <TableCell className="font-medium sticky left-0 bg-background z-10">
795 %                                 <div className="flex flex-col">
796 %                                     <span>{room.name}</span>
797 %                                     <span className="text-xs text-muted-foreground">
798 %                                         Cap: {room.capacity}
799 %                                     </span>
800 %                                 </div>
801 %                             </TableCell>
802 %                             {weekDays.map((day) => {
803 %                                 const cellEvents = getEventsForCell(room.id, day);
804 %                                 return (
805 %                                     <TableCell
806 %                                         key={day.toISOString()}
807 %                                         className="p-1 align-top cursor-pointer hover-elevate min-h-[80px]"
808 %                                         onClick={() => cellEvents.length === 0 && handleCellClick(room.id, day)}
809 %                                         data-testid={`cell-${room.id}-${format(day, "yyyy-MM-dd")}`}
810 %                                     >
811 %                                         <div className="flex flex-col gap-1">
812 %                                             {cellEvents.map((event) => (
813 %                                                 <div
814 %                                                     key={event.id}
815 %                                                     className={`p-1.5 rounded-md text-xs cursor-pointer ${eventStatusColors[event.status]}
816 %                                                     onClick={(e) => {
817 %                                                         e.stopPropagation();
818 %                                                         handleEventClick(event.id);
819 %                                                     }}
820 %                                                     data-testid={`event-${event.id}`}
821 %                                                 >
822 %                                                     <div className="font-medium truncate">{event.name}</div>
823 %                                                     <div className="text-xs opacity-80 truncate">{event.contactName}</div>
824 %                                                 </div>
825 %                                             ))}
826 %                                         {cellEvents.length === 0 && (
827 %                                             <div className="h-16 flex items-center justify-center">
828 %                                                 <Plus className="h-4 w-4 text-muted-foreground opacity-0 group-hover:opacity-100 tr...
829 %                                             </div>
830 %                                         )}
831 %                                     </div>
832 %                                 </TableCell>
833 %                             </TableBody>
834 %                         </TableRow>

```

client/src/pages/events.tsx (cont.)

```
835 %         </TableRow>
836 %       )}}
837 %     </TableBody>
838 %   </Table>
839 % </div>
840 %   <ScrollBar orientation="horizontal" />
841 % </ScrollArea>
842 % </CardContent>
843 % </Card>
844 %
845 % {/* New Event Dialog */}
846 % <Dialog open={isNewDialogOpen} onOpenChange={setIsNewDialogOpen}>
847 %   <DialogContent className="max-w-2xl">
848 %     <DialogHeader>
849 %       <DialogTitle>Nuevo Evento</DialogTitle>
850 %       <DialogDescription>
851 %         Complete los datos del evento
852 %       </DialogDescription>
853 %     </DialogHeader>
854 %     <Form {...eventForm}>
855 %       <form onSubmit={eventForm.handleSubmit(onSubmitEvent)} className="space-y-4">
856 %         <div className="grid grid-cols-2 gap-4">
857 %           <FormField
858 %             control={eventForm.control}
859 %             name="name"
860 %             render={({ field }) => (
861 %               <FormItem className="col-span-2">
862 %                 <FormLabel>Nombre del Evento</FormLabel>
863 %                 <FormControl>
864 %                   <Input placeholder="Nombre del evento" {...field} data-testid="input-event-name" />
865 %                 </FormControl>
866 %                 <FormMessage />
867 %               </FormItem>
868 %             )}
869 %           />
870 %
871 %           <FormField
872 %             control={eventForm.control}
873 %             name="eventRoomId"
874 %             render={({ field }) => (
875 %               <FormItem>
876 %                 <FormLabel>Salon</FormLabel>
877 %                 <Select onValueChange={field.onChange} value={field.value}>
878 %                   <FormControl>
879 %                     <SelectTrigger data-testid="select-event-room">
880 %                       <SelectValue placeholder="Seleccionar salon" />
881 %                     </SelectTrigger>
882 %                   </FormControl>
883 %                   <SelectContent>
884 %                     {eventRooms.filter(r => r.isActive).map((room) => (
885 %                       <SelectItem key={room.id} value={room.id}>
886 %                         {room.name} (Cap: {room.capacity})
887 %                       </SelectItem>
888 %                     ))}
889 %                   </SelectContent>
890 %                 </Select>
891 %                 <FormMessage />
892 %               </FormItem>
893 %             )}
894 %           />
895 %
896 %           <FormField
897 %             control={eventForm.control}
898 %             name="eventType"
899 %             render={({ field }) => (
900 %               <FormItem>
901 %                 <FormLabel>Tipo de Evento</FormLabel>
902 %                 <Select onValueChange={field.onChange} value={field.value}>
903 %                   <FormControl>
904 %                     <SelectTrigger data-testid="select-event-type">
905 %                       <SelectValue placeholder="Seleccionar tipo" />
906 %                     </SelectTrigger>
907 %                   </FormControl>
908 %                   <SelectContent>
909 %                     <SelectItem value="corporate">Corporativo</SelectItem>
910 %                     <SelectItem value="social">Social</SelectItem>
911 %                     <SelectItem value="wedding">Boda</SelectItem>
912 %                     <SelectItem value="conference">Conferencia</SelectItem>
913 %                     <SelectItem value="meeting">Reunion</SelectItem>
914 %                     <SelectItem value="table_event">Evento por Mesa</SelectItem>
915 %                     <SelectItem value="other">Otro</SelectItem>
916 %                   </SelectContent>
917 %                 </Select>
918 %                 <FormMessage />
919 %               </FormItem>
920 %             )}
921 %           />
922 %
923 %           {eventForm.watch("eventType") === "table_event" && (
924 %             <div className="col-span-2 flex items-start gap-2 p-3 rounded-md bg-blue-50 dark:bg-blue-950 border...
925 %               <Info className="h-4 w-4 text-blue-600 mt-0.5 flex-shrink-0" />
926 %               <p className="text-sm text-blue-700 dark:text-blue-300">
927 %                 En un Evento por Mesa, los cargos y pagos se gestionan por mesa individual. Los cargos generale...
```

client/src/pages/events.tsx (cont.)

```
928 %         </p>
929 %     </div>
930 % })
931 %
932 % <FormField
933 %   control={eventForm.control}
934 %   name="contactName"
935 %   render={({ field }) => (
936 %     <FormItem>
937 %       <FormLabel>Persona de Contacto</FormLabel>
938 %       <FormControl>
939 %         <Input placeholder="Nombre de contacto" {...field} data-testid="input-contact-name" />
940 %       </FormControl>
941 %       <FormMessage />
942 %     </FormItem>
943 %   )}
944 % />
945 %
946 % <FormField
947 %   control={eventForm.control}
948 %   name="contactPhone"
949 %   render={({ field }) => (
950 %     <FormItem>
951 %       <FormLabel>Telefono</FormLabel>
952 %       <FormControl>
953 %         <Input placeholder="Telefono" {...field} data-testid="input-contact-phone" />
954 %       </FormControl>
955 %       <FormMessage />
956 %     </FormItem>
957 %   )}
958 % />
959 %
960 % <FormField
961 %   control={eventForm.control}
962 %   name="contactEmail"
963 %   render={({ field }) => (
964 %     <FormItem>
965 %       <FormLabel>Email</FormLabel>
966 %       <FormControl>
967 %         <Input type="email" placeholder="Email" {...field} data-testid="input-contact-email" />
968 %       </FormControl>
969 %       <FormMessage />
970 %     </FormItem>
971 %   )}
972 % />
973 %
974 % <FormField
975 %   control={eventForm.control}
976 %   name="attendees"
977 %   render={({ field }) => (
978 %     <FormItem>
979 %       <FormLabel>Asistentes</FormLabel>
980 %       <FormControl>
981 %         <Input
982 %           type="number"
983 %           min={1}
984 %           {...field}
985 %           onChange={(e) => field.onChange(parseInt(e.target.value) || 1)}
986 %           data-testid="input-attendees"
987 %         />
988 %       </FormControl>
989 %       <FormMessage />
990 %     </FormItem>
991 %   )}
992 % />
993 %
994 % <FormField
995 %   control={eventForm.control}
996 %   name="startDate"
997 %   render={({ field }) => (
998 %     <FormItem>
999 %       <FormLabel>Fecha Inicio</FormLabel>
1000 %     <FormControl>
1001 %       <Input type="date" {...field} data-testid="input-start-date" />
1002 %     </FormControl>
1003 %     <FormMessage />
1004 %   </FormItem>
1005 % )}
1006 % />
1007 %
1008 % <FormField
1009 %   control={eventForm.control}
1010 %   name="endDate"
1011 %   render={({ field }) => (
1012 %     <FormItem>
1013 %       <FormLabel>Fecha Fin</FormLabel>
1014 %       <FormControl>
1015 %         <Input type="date" {...field} data-testid="input-end-date" />
1016 %       </FormControl>
1017 %       <FormMessage />
1018 %     </FormItem>
1019 %   )}
1020 % />
```

client/src/pages/events.tsx (cont.)

```
1021 %
1022 %         <FormField
1023 %             control={eventForm.control}
1024 %             name="startTime"
1025 %             render={({ field }) => (
1026 %                 <FormItem>
1027 %                     <FormLabel>Hora Inicio</FormLabel>
1028 %                     <FormControl>
1029 %                         <Input type="time" {...field} data-testid="input-start-time" />
1030 %                     </FormControl>
1031 %                     <FormMessage />
1032 %                 </FormItem>
1033 %             )}
1034 %         />
1035 %
1036 %         <FormField
1037 %             control={eventForm.control}
1038 %             name="endTime"
1039 %             render={({ field }) => (
1040 %                 <FormItem>
1041 %                     <FormLabel>Hora Fin</FormLabel>
1042 %                     <FormControl>
1043 %                         <Input type="time" {...field} data-testid="input-end-time" />
1044 %                     </FormControl>
1045 %                     <FormMessage />
1046 %                 </FormItem>
1047 %             )}
1048 %         />
1049 %
1050 %         <FormField
1051 %             control={eventForm.control}
1052 %             name="notes"
1053 %             render={({ field }) => (
1054 %                 <FormItem className="col-span-2">
1055 %                     <FormLabel>Notas</FormLabel>
1056 %                     <FormControl>
1057 %                         <Textarea placeholder="Notas adicionales" {...field} data-testid="input-notes" />
1058 %                     </FormControl>
1059 %                     <FormMessage />
1060 %                 </FormItem>
1061 %             )}
1062 %         />
1063 %     </div>
1064 %
1065 %     <DialogFooter>
1066 %         <Button type="button" variant="outline" onClick={() => setIsNewDialogOpen(false)}>
1067 %             Cancelar
1068 %         </Button>
1069 %         <Button type="submit" disabled={createEventMutation.isPending} data-testid="button-submit-event">
1070 %             {createEventMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1071 %             Crear Evento
1072 %         </Button>
1073 %     </DialogFooter>
1074 % </form>
1075 % </Form>
1076 % </DialogContent>
1077 % </Dialog>
1078 %
1079 % {/* Event Detail Dialog */}
1080 % <Dialog open={!!selectedEvent} onOpenChange={(open) => !open && setSelectedEvent(null)}>
1081 %     <DialogContent className="max-w-4xl max-h-[90vh] overflow-y-auto">
1082 %         <DialogHeader>
1083 %             <DialogTitle className="flex items-center gap-2">
1084 %                 <span>{selectedEvent?.name}</span>
1085 %                 {selectedEvent && (
1086 %                     <>
1087 %                         <Badge className={eventStatusColors[selectedEvent.status]}>
1088 %                             {eventStatusLabels[selectedEvent.status]}
1089 %                         </Badge>
1090 %                         {selectedEvent.eventType === "table_event" && (
1091 %                             <Badge variant="outline" className="border-orange-500 text-orange-600">
1092 %                                 <UtilsilsCrossed className="h-3 w-3 mr-1" />
1093 %                                 Por Mesa
1094 %                             </Badge>
1095 %                         )}
1096 %                     </>
1097 %                 )}
1098 %             </DialogTitle>
1099 %             <DialogDescription>
1100 %                 Codigo: {selectedEvent?.eventCode}
1101 %             </DialogDescription>
1102 %         </DialogHeader>
1103 %
1104 %         {selectedEvent && (
1105 %             <Tabs value={activeTab} onValueChange={setActiveTab} className="w-full">
1106 %                 <TabsList className={`grid w-full ${detailTabCount === 4 ? "grid-cols-4" : "grid-cols-3"}`}>
1107 %                     <TabsTrigger value="details">Detalles</TabsTrigger>
1108 %                     <TabsTrigger value="charges">Cargos</TabsTrigger>
1109 %                     {selectedEvent.eventType === "table_event" && (
1110 %                         <TabsTrigger value="tables">Mesas</TabsTrigger>
1111 %                     )}
1112 %                     <TabsTrigger value="folio">Folio</TabsTrigger>
1113 %                 </TabsList>
```

```

1114 %
1115 % <TabsContent value="details" className="space-y-4">
1116 %   <div className="grid grid-cols-2 gap-4">
1117 %     <div className="space-y-2">
1118 %       <div className="flex items-center gap-2 text-sm">
1119 %         <Building2 className="h-4 w-4 text-muted-foreground" />
1120 %         <span className="font-medium">Salon:</span>
1121 %         <span>{selectedEvent.eventRoom?.name}</span>
1122 %       </div>
1123 %       <div className="flex items-center gap-2 text-sm">
1124 %         <Calendar className="h-4 w-4 text-muted-foreground" />
1125 %         <span className="font-medium">Fecha:</span>
1126 %         <span>
1127 %           {safeFormatDate(selectedEvent.startDate, "d MMM yyyy", { locale: es })}
1128 %           {selectedEvent.startDate !== selectedEvent.endDate && (
1129 %             <- {safeFormatDate(selectedEvent.endDate, "d MMM yyyy", { locale: es })}</>
1130 %           )}
1131 %         </span>
1132 %       </div>
1133 %       {selectedEvent.startTime && (
1134 %         <div className="flex items-center gap-2 text-sm">
1135 %           <Clock className="h-4 w-4 text-muted-foreground" />
1136 %           <span className="font-medium">Horario:</span>
1137 %           <span>
1138 %             {selectedEvent.startTime} - {selectedEvent.endTime || "..."}
1139 %           </span>
1140 %         </div>
1141 %       )}
1142 %       <div className="flex items-center gap-2 text-sm">
1143 %         <Users className="h-4 w-4 text-muted-foreground" />
1144 %         <span className="font-medium">Asistentes:</span>
1145 %         <span>{selectedEvent.attendees}</span>
1146 %       </div>
1147 %       <div className="flex items-center gap-2 text-sm">
1148 %         <span className="font-medium">Tipo:</span>
1149 %         <span>{eventTypeLabels[selectedEvent.eventType]}</span>
1150 %       </div>
1151 %     </div>
1152 %
1153 %     <div className="space-y-2">
1154 %       <div className="flex items-center gap-2 text-sm">
1155 %         <User className="h-4 w-4 text-muted-foreground" />
1156 %         <span className="font-medium">Contacto:</span>
1157 %         <span>{selectedEvent.contactName}</span>
1158 %       </div>
1159 %       {selectedEvent.contactPhone && (
1160 %         <div className="flex items-center gap-2 text-sm">
1161 %           <Phone className="h-4 w-4 text-muted-foreground" />
1162 %           <span>{selectedEvent.contactPhone}</span>
1163 %         </div>
1164 %       )}
1165 %       {selectedEvent.contactEmail && (
1166 %         <div className="flex items-center gap-2 text-sm">
1167 %           <Mail className="h-4 w-4 text-muted-foreground" />
1168 %           <span>{selectedEvent.contactEmail}</span>
1169 %         </div>
1170 %       )}
1171 %     </div>
1172 %   </div>
1173 %
1174 %   {selectedEvent.notes && (
1175 %     <div className="p-3 rounded-md bg-muted">
1176 %       <p className="text-sm">{selectedEvent.notes}</p>
1177 %     </div>
1178 %   )}
1179 %
1180 %   <div className="flex flex-wrap gap-2 pt-4 border-t">
1181 %     {selectedEvent.status === "tentative" && (
1182 %       <Button
1183 %         onClick={() => {
1184 %           updateEventMutation.mutate({
1185 %             id: selectedEvent.id,
1186 %             data: { status: "confirmed" }
1187 %           });
1188 %           setSelectedEvent({ ...selectedEvent, status: "confirmed" });
1189 %         }}
1190 %         data-testid="button-confirm-event"
1191 %       >
1192 %         Confirmar
1193 %       </Button>
1194 %     )}
1195 %     {selectedEvent.status === "confirmed" && (
1196 %       <Button
1197 %         onClick={() => {
1198 %           updateEventMutation.mutate({
1199 %             id: selectedEvent.id,
1200 %             data: { status: "in_progress" }
1201 %           });
1202 %           setSelectedEvent({ ...selectedEvent, status: "in_progress" });
1203 %         }}
1204 %         data-testid="button-start-event"
1205 %       >
1206 %         Iniciar

```

client/src/pages/events.tsx (cont.)

```
1207 %         </Button>
1208 %     )}
1209 %     {selectedEvent.status === "in_progress" && (
1210 %         <Button
1211 %             onClick={() => {
1212 %                 updateEventMutation.mutate({
1213 %                     id: selectedEvent.id,
1214 %                     data: { status: "completed" }
1215 %                 });
1216 %                 setSelectedEvent({ ...selectedEvent, status: "completed" });
1217 %             }}
1218 %             data-testid="button-complete-event"
1219 %         >
1220 %             Completar
1221 %         </Button>
1222 %     )}
1223 %     {canShowFolio(selectedEvent) && (
1224 %         <Button
1225 %             variant="outline"
1226 %             onClick={() => {
1227 %                 setFolioReceiptType("");
1228 %                 setActiveTab("folio");
1229 %             }}
1230 %             data-testid="button-open-folio"
1231 %         >
1232 %             <Receipt className="h-4 w-4 mr-2" />
1233 %             Folio / Facturar
1234 %         </Button>
1235 %     )}
1236 %     {(selectedEvent.status === "tentative" || selectedEvent.status === "confirmed") && (
1237 %         <Button
1238 %             variant="destructive"
1239 %             onClick={() => setCancelConfirmOpen(true)}
1240 %             data-testid="button-cancel-event"
1241 %         >
1242 %             Cancelar Evento
1243 %         </Button>
1244 %     )}
1245 % </div>
1246 % </TabsContent>
1247 %
1248 % <TabsContent value="charges" className="space-y-4">
1249 %     <div className="flex items-center justify-between">
1250 %         <h4 className="font-medium">Cargos del Evento</h4>
1251 %         {selectedEvent.status !== "invoiced" && selectedEvent.status !== "cancelled" && (
1252 %             <Button
1253 %                 size="sm"
1254 %                 onClick={() => {
1255 %                     chargeForm.reset();
1256 %                     setIsChargeDialogOpen(true);
1257 %                 }}
1258 %                 data-testid="button-add-charge"
1259 %             >
1260 %                 <Plus className="h-4 w-4 mr-2" />
1261 %                 Agregar Cargo
1262 %             </Button>
1263 %         )}
1264 %     </div>
1265 %
1266 %     <Table>
1267 %         <TableHeader>
1268 %             <TableRow>
1269 %                 <TableHead>Descripcion</TableHead>
1270 %                 <TableHead className="text-right">Cantidad</TableHead>
1271 %                 <TableHead className="text-right">Precio Unit.</TableHead>
1272 %                 <TableHead className="text-right">Total</TableHead>
1273 %                 <TableHead className="w-[50px]"></TableHead>
1274 %             </TableRow>
1275 %         </TableHeader>
1276 %         <TableBody>
1277 %             {selectedEvent.charges?.map((charge) => (
1278 %                 <TableRow key={charge.id}>
1279 %                     <TableCell>{charge.description}</TableCell>
1280 %                     <TableCell className="text-right">{charge.quantity}</TableCell>
1281 %                     <TableCell className="text-right">${charge.unitPrice}</TableCell>
1282 %                     <TableCell className="text-right font-medium">${charge.totalAmount}</TableCell>
1283 %                     <TableCell>
1284 %                         {selectedEvent.status !== "invoiced" && (
1285 %                             <Button
1286 %                                 variant="ghost"
1287 %                                 size="icon"
1288 %                                 onClick={() => deleteChargeMutation.mutate(charge.id)}
1289 %                                 data-testid={`button-delete-charge-${charge.id}`}
1290 %                             >
1291 %                                 <Trash2 className="h-4 w-4" />
1292 %                             </Button>
1293 %                         )}
1294 %                     </TableCell>
1295 %                 </TableRow>
1296 %             )}
1297 %         </TableBody>
1298 %     </Table>
1299 %     {(!selectedEvent.charges || selectedEvent.charges.length === 0) && (
1300 %         <Table>
1301 %             <TableBody>
1302 %                 <TableRow>
1303 %                     <TableCell colspan={5} className="text-center text-muted-foreground">
```

client/src/pages/events.tsx (cont.)

```
1300 %           No hay cargos registrados
1301 %         </TableCell>
1302 %       </TableRow>
1303 %     })
1304 %   </TableBody>
1305 % </Table>
1306 %
1307 %   <div className="flex justify-end border-t pt-4">
1308 %     <div className="text-lg font-bold">
1309 %       Total: ${calculateEventTotal(selectedEvent).toFixed(2)}
1310 %     </div>
1311 %   </div>
1312 % </TabsContent>
1313 %
1314 %   { /* Mesas Tab - only for table_event */ }
1315 %   { selectedEvent.eventType === "table_event" && (
1316 %     <TabsContent value="tables" className="space-y-4">
1317 %       <div className="flex items-center justify-between">
1318 %         <h4 className="font-medium">Mesas del Evento</h4>
1319 %         { selectedEvent.status !== "invoiced" && selectedEvent.status !== "cancelled" && (
1320 %           <Button
1321 %             size="sm"
1322 %             onClick={() => {
1323 %               setNewTableNumber(eventTables.length + 1);
1324 %               setNewTableLabel("");
1325 %               setNewTableSeats(4);
1326 %               setIsAddTableOpen(true);
1327 %             }}
1328 %             data-testid="button-add-table"
1329 %           >
1330 %             <Plus className="h-4 w-4 mr-2" />
1331 %             Agregar Mesa
1332 %           </Button>
1333 %         ) }
1334 %       </div>
1335 %
1336 %       { eventTables.length === 0 ? (
1337 %         <div className="text-center py-8 text-muted-foreground">
1338 %           No hay mesas registradas. Agregue mesas para gestionar cargos individuales.
1339 %         </div>
1340 %       ) : (
1341 %         <>
1342 %           <div className="grid grid-cols-2 md:grid-cols-3 gap-3">
1343 %             { eventTables.map((table) => {
1344 %               const tCharges = table.charges.reduce((s, c) => s + parseFloat(c.total), 0);
1345 %               const tPayments = table.payments.reduce((s, p) => s + parseFloat(p.amount), 0);
1346 %               const tBalance = tCharges - tPayments;
1347 %               return (
1348 %                 <Card
1349 %                   key={table.id}
1350 %                   className={ `cursor-pointer hover-elevate ${table.status === "closed" || table.status ==...` }
1351 %                   onClick={() => {
1352 %                     setSelectedTable(table);
1353 %                     setTableFolioReceiptType("");
1354 %                     setIsTableFolioOpen(true);
1355 %                   }}
1356 %                   data-testid={`table-card-${table.id}` }
1357 %                 >
1358 %                   <CardContent className="p-4">
1359 %                     <div className="flex items-center justify-between mb-2">
1360 %                       <span className="font-bold text-lg">Mesa {table.tableNumber}</span>
1361 %                       <Badge variant={table.status === "open" ? "default" : "secondary"}>
1362 %                         {table.status === "open" ? "Abierta" : table.status === "invoiced" ? "Facturada" ...
1363 %                       </Badge>
1364 %                     </div>
1365 %                     {table.label && <p className="text-sm text-muted-foreground mb-1">{table.label}</p>}
1366 %                     {table.seats && <p className="text-xs text-muted-foreground">{table.seats} asientos</p>}
1367 %                     <div className="mt-2 pt-2 border-t space-y-1">
1368 %                       <div className="flex justify-between text-sm">
1369 %                         <span>Cargos:</span>
1370 %                         <span className="font-medium">${tCharges.toFixed(2)}</span>
1371 %                       </div>
1372 %                       <div className="flex justify-between text-sm">
1373 %                         <span>Pagado:</span>
1374 %                         <span className="font-medium">${tPayments.toFixed(2)}</span>
1375 %                       </div>
1376 %                       {tBalance > 0.01 && (
1377 %                         <div className="flex justify-between text-sm text-red-600">
1378 %                           <span>Saldo:</span>
1379 %                           <span className="font-bold">${tBalance.toFixed(2)}</span>
1380 %                         </div>
1381 %                       ) }
1382 %                     </div>
1383 %                   </CardContent>
1384 %                 </Card>
1385 %               );
1386 %             } ) }
1387 %           </div>
1388 %
1389 %           <div className="border-t pt-4">
1390 %             <h5 className="font-medium mb-2">Resumen General</h5>
1391 %             <div className="grid grid-cols-3 gap-4 text-center">
1392 %               <div>
```


client/src/pages/events.tsx (cont.)

```
1393 %         <p className="text-sm text-muted-foreground">Total Cargos</p>
1394 %         <p className="text-xl font-bold">
1395 %           ${eventTables.reduce((s, t) => s + t.charges.reduce((sc, c) => sc + parseFloat(c.total)...
1396 %         </p>
1397 %       </div>
1398 %     <div>
1399 %       <p className="text-sm text-muted-foreground">Total Pagado</p>
1400 %       <p className="text-xl font-bold">
1401 %         ${eventTables.reduce((s, t) => s + t.payments.reduce((sp, p) => sp + parseFloat(p.ammoun...
1402 %       </p>
1403 %     </div>
1404 %   <div>
1405 %     <p className="text-sm text-muted-foreground">Mesas Abiertas</p>
1406 %     <p className="text-xl font-bold">
1407 %       {eventTables.filter(t => t.status === "open").length} / {eventTables.length}
1408 %     </p>
1409 %   </div>
1410 % </div>
1411 % </div>
1412 % </>
1413 %   )}
1414 % </TabsContent>
1415 % }}
1416 %
1417 % { /* Folio Tab */}
1418 % <TabsContent value="folio" className="space-y-4">
1419 %   {selectedEvent.status === "invoiced" ? (
1420 %     <div className="text-center py-6">
1421 %       <Receipt className="h-12 w-12 mx-auto text-purple-500 mb-3" />
1422 %       <h4 className="font-bold text-lg">Evento Facturado</h4>
1423 %       <p className="text-muted-foreground">
1424 %         Comprobante: {selectedEvent.receiptType} | Cerrado: {selectedEvent.closedAt ? safeFormatDate(se...
1425 %       </p>
1426 %       <div className="mt-4 grid grid-cols-2 gap-4 max-w-sm mx-auto">
1427 %         <div>
1428 %           <p className="text-sm text-muted-foreground">Total</p>
1429 %           <p className="text-xl font-bold">${selectedEvent.totalAmount || "0.00"}</p>
1430 %         </div>
1431 %         <div>
1432 %           <p className="text-sm text-muted-foreground">Pagado</p>
1433 %           <p className="text-xl font-bold text-green-600">${selectedEvent.totalPaid || "0.00"}</p>
1434 %         </div>
1435 %       </div>
1436 %     </div>
1437 %   ) : selectedEvent.status === "cancelled" ? (
1438 %     <div className="text-center py-6 text-muted-foreground">
1439 %       Evento cancelado - No se puede facturar
1440 %     </div>
1441 %   ) : !canShowFolio(selectedEvent) ? (
1442 %     <div className="text-center py-6 text-muted-foreground">
1443 %       Confirme el evento para acceder al folio
1444 %     </div>
1445 %   ) : (
1446 %     <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
1447 %       { /* Left: Charges */}
1448 %       <div>
1449 %         <h4 className="font-medium mb-3 flex items-center gap-2">
1450 %           <CreditCard className="h-4 w-4" />
1451 %           Cargos
1452 %         </h4>
1453 %         <div className="space-y-2 max-h-[300px] overflow-y-auto">
1454 %           {selectedEvent.charges?.map((charge) => (
1455 %             <div key={charge.id} className="flex items-center justify-between p-2 rounded border text-sm">
1456 %               <div>
1457 %                 <span className="font-medium">{charge.description}</span>
1458 %                 <span className="text-muted-foreground ml-2">x{charge.quantity}</span>
1459 %               </div>
1460 %               <span className="font-medium">${charge.totalAmount}</span>
1461 %             </div>
1462 %           )]}
1463 %           {(!selectedEvent.charges || selectedEvent.charges.length === 0) && (
1464 %             <p className="text-sm text-muted-foreground text-center py-4">Sin cargos</p>
1465 %           )}
1466 %         </div>
1467 %         <div className="mt-3 pt-3 border-t flex justify-between font-bold">
1468 %           <span>Total Cargos:</span>
1469 %           <span>${calculateEventTotal(selectedEvent).toFixed(2)}</span>
1470 %         </div>
1471 %       </div>
1472 %
1473 %       { /* Right: Payments */}
1474 %       <div>
1475 %         <h4 className="font-medium mb-3 flex items-center gap-2">
1476 %           <DollarSign className="h-4 w-4" />
1477 %           Pagos
1478 %         </h4>
1479 %         <div className="space-y-2 max-h-[200px] overflow-y-auto">
1480 %           {selectedEvent.payments?.map((payment) => (
1481 %             <div key={payment.id} className="flex items-center justify-between p-2 rounded border text-...
1482 %               <div className="flex items-center gap-2">
1483 %                 <span className="font-medium">${payment.amount}</span>
1484 %                 <span className="text-muted-foreground">
1485 %                   {paymentMethodLabels[payment.method] || payment.method}
```

client/src/pages/events.tsx (cont.)

```
1486 %         </span>
1487 %         {payment.isAdvance === "true" && (
1488 %           <Badge variant="outline" className="text-xs border-amber-500 text-amber-600">SEÑA</Ba...
1489 %         )}
1490 %       </div>
1491 %       <Button
1492 %         variant="ghost"
1493 %         size="icon"
1494 %         className="h-6 w-6"
1495 %         onClick={() => deletePaymentMutation.mutate({ eventId: selectedEvent.id, payId: payment...
1496 %         data-testid={`button-delete-payment-${payment.id}`}
1497 %       >
1498 %         <Trash2 className="h-3 w-3" />
1499 %       </Button>
1500 %     </div>
1501 %   )}
1502 %   {(!selectedEvent.payments || selectedEvent.payments.length === 0) && (
1503 %     <p className="text-sm text-muted-foreground text-center py-4">Sin pagos</p>
1504 %   )}
1505 % </div>
1506 %
1507 % <div className="mt-3 pt-3 border-t space-y-3">
1508 %   <div className="flex justify-between font-bold">
1509 %     <span>Total Pagado:</span>
1510 %     <span className="text-green-600">${calculateEventPaid(selectedEvent).toFixed(2)}</span>
1511 %   </div>
1512 %
1513 %   <div className="flex justify-between font-bold text-lg">
1514 %     <span>Saldo:</span>
1515 %     <span className={calculateEventTotal(selectedEvent) - calculateEventPaid(selectedEvent) > 0...
1516 %       ${calculateEventTotal(selectedEvent) - calculateEventPaid(selectedEvent)}.toFixed(2)}
1517 %     </span>
1518 %   </div>
1519 %
1520 %   {/ * Add payment form */}
1521 %   <div className="space-y-2 pt-2 border-t">
1522 %     <p className="text-sm font-medium">Agregar Pago</p>
1523 %     <div className="grid grid-cols-2 gap-2">
1524 %       <Input
1525 %         type="number"
1526 %         step="0.01"
1527 %         placeholder="Monto"
1528 %         value={paymentAmount}
1529 %         onChange={(e) => setPaymentAmount(e.target.value)}
1530 %         data-testid="input-payment-amount"
1531 %       />
1532 %       <Select value={paymentMethod} onValueChange={setPaymentMethod}>
1533 %         <SelectTrigger data-testid="select-payment-method">
1534 %           <SelectValue />
1535 %         </SelectTrigger>
1536 %         <SelectContent>
1537 %           {Object.entries(paymentMethodLabels).map(([k, v]) => (
1538 %             <SelectItem key={k} value={k}>{v}</SelectItem>
1539 %           ))}
1540 %         </SelectContent>
1541 %       </Select>
1542 %     </div>
1543 %     {paymentMethod === "room_charge" && (
1544 %       <Select value={paymentReservationId} onValueChange={setPaymentReservationId}>
1545 %         <SelectTrigger data-testid="select-payment-reservation">
1546 %           <SelectValue placeholder="Seleccionar habitacion" />
1547 %         </SelectTrigger>
1548 %         <SelectContent>
1549 %           {activeReservations.map((r) => (
1550 %             <SelectItem key={r.id} value={r.id}>
1551 %               Hab. {r.room?.roomNumber} || "?" - {r.guest?.firstName} {r.guest?.lastName}
1552 %             </SelectItem>
1553 %           ))}
1554 %         </SelectContent>
1555 %       </Select>
1556 %     )}
1557 %   </div>
1558 %   <div className="flex items-center gap-4">
1559 %     <label className="flex items-center gap-2 text-sm">
1560 %       <input
1561 %         type="checkbox"
1562 %         checked={paymentIsAdvance}
1563 %         onChange={(e) => setPaymentIsAdvance(e.target.checked)}
1564 %         data-testid="checkbox-is-advance"
1565 %       />
1566 %       Señal / Anticipo
1567 %     </label>
1568 %   </div>
1569 %   <Input
1570 %     placeholder="Notas (opcional)"
1571 %     value={paymentNotes}
1572 %     onChange={(e) => setPaymentNotes(e.target.value)}
1573 %     data-testid="input-payment-notes"
1574 %   />
1575 %   <Button
1576 %     size="sm"
1577 %     className="w-full"
1578 %     onClick={handleAddPayment}
1579 %     disabled={addPaymentMutation.isPending || !paymentAmount}
```

client/src/pages/events.tsx (cont.)

```
1579 %         data-testid="button-add-payment"
1580 %     >
1581 %         {addPaymentMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1582 %         Registrar Pago
1583 %     </Button>
1584 % </div>
1585 %
1586 %     {/* Close event */}
1587 %     <div className="space-y-2 pt-2 border-t">
1588 %         <p className="text-sm font-medium">Cerrar Evento</p>
1589 %         <Select value={folioReceiptType} onChange={setFolioReceiptType}>
1590 %             <SelectTrigger data-testid="select-receipt-type">
1591 %                 <SelectValue placeholder="Tipo de comprobante" />
1592 %             </SelectTrigger>
1593 %             <SelectContent>
1594 %                 {receiptTypes.map((rt) => (
1595 %                     <SelectItem key={rt} value={rt}>{rt}</SelectItem>
1596 %                 ))}
1597 %             </SelectContent>
1598 %         </Select>
1599 %         <Button
1600 %             className="w-full"
1601 %             variant="default"
1602 %             disabled={
1603 %                 !folioReceiptType ||
1604 %                 calculateEventTotal(selectedEvent) - calculateEventPaid(selectedEvent) > 0.01 ||
1605 %                 closeEventMutation.isPending
1606 %             }
1607 %             onClick={handleCloseEvent}
1608 %             data-testid="button-close-event"
1609 %         >
1610 %             {closeEventMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1611 %             <Receipt className="h-4 w-4 mr-2" />
1612 %             Facturar y Cerrar
1613 %         </Button>
1614 %         {calculateEventTotal(selectedEvent) - calculateEventPaid(selectedEvent) > 0.01 && (
1615 %             <p className="text-xs text-red-500 text-center">
1616 %                 Debe saldar el balance para facturar
1617 %             </p>
1618 %         )}
1619 %     </div>
1620 % </div>
1621 % </div>
1622 % </div>
1623 %     })
1624 % </TabsContent>
1625 % </Tabs>
1626 % })
1627 % </DialogContent>
1628 % </Dialog>
1629 %
1630 %     {/* Cancel Confirmation Dialog (T001) */}
1631 %     <AlertDialog open={cancelConfirmOpen} onOpenChange={setCancelConfirmOpen}>
1632 %         <AlertDialogContent>
1633 %             <AlertDialogHeader>
1634 %                 <AlertDialogTitle>Cancelar Evento</AlertDialogTitle>
1635 %                 <AlertDialogDescription>
1636 %                     ¿Estas seguro que quieres cancelar el evento '{selectedEvent?.name}'? Esta accion marcara el evento como...
1637 %                 </AlertDialogDescription>
1638 %             </AlertDialogHeader>
1639 %             <AlertDialogFooter>
1640 %                 <AlertDialogCancel data-testid="button-cancel-confirm-back">Volver</AlertDialogCancel>
1641 %                 <AlertDialogAction
1642 %                     className="bg-destructive text-destructive-foreground hover:bg-destructive/90"
1643 %                     onClick={() => {
1644 %                         if (selectedEvent) {
1645 %                             updateEventMutation.mutate({
1646 %                                 id: selectedEvent.id,
1647 %                                 data: { status: "cancelled" }
1648 %                             });
1649 %                             setSelectedEvent({ ...selectedEvent, status: "cancelled" });
1650 %                         }
1651 %                         setCancelConfirmOpen(false);
1652 %                     }}
1653 %                     data-testid="button-cancel-confirm-yes"
1654 %                 >
1655 %                     Si, cancelar evento
1656 %                 </AlertDialogAction>
1657 %             </AlertDialogFooter>
1658 %         </AlertDialogContent>
1659 %     </AlertDialog>
1660 %
1661 %     {/* Add Charge Dialog */}
1662 %     <Dialog open={isChargeDialogOpen} onOpenChange={setIsChargeDialogOpen}>
1663 %         <DialogContent>
1664 %             <DialogHeader>
1665 %                 <DialogTitle>Agregar Cargo</DialogTitle>
1666 %                 <DialogDescription>
1667 %                     Seleccione un tipo predefinido o ingrese manualmente
1668 %                 </DialogDescription>
1669 %             </DialogHeader>
1670 %             <Form {...chargeForm}>
1671 %                 <form onSubmit={chargeForm.handleSubmit(onSubmitCharge)} className="space-y-4">
```

client/src/pages/events.tsx (cont.)

```
1672 %         <div className="space-y-4">
1673 %             <div>
1674 %                 <FormLabel>Tipo Predefinido</FormLabel>
1675 %                 <Select
1676 %                     onChange={(value) => {
1677 %                         chargeForm.setValue("chargeTypeId", value);
1678 %                         handleChargeTypeSelect(value);
1679 %                     }}
1680 %                 >
1681 %                     <SelectTrigger data-testid="select-charge-type">
1682 %                         <SelectValue placeholder="Seleccionar tipo (opcional)" />
1683 %                     </SelectTrigger>
1684 %                     <SelectContent>
1685 %                         {chargeTypes.filter(ct => ct.isActive).map((ct) => (
1686 %                             <SelectItem key={ct.id} value={ct.id}>
1687 %                                 {ct.name} - ${ct.defaultPrice}
1688 %                             </SelectItem>
1689 %                         ))}
1690 %                     </SelectContent>
1691 %                 </Select>
1692 %             </div>
1693 %
1694 %             <FormField
1695 %                 control={chargeForm.control}
1696 %                 name="description"
1697 %                 render={({ field }) => (
1698 %                     <FormItem>
1699 %                         <FormLabel>Descripcion</FormLabel>
1700 %                         <FormControl>
1701 %                             <Input placeholder="Descripcion del cargo" {...field} data-testid="input-charge-description" />
1702 %                         </FormControl>
1703 %                         <FormMessage />
1704 %                     </FormItem>
1705 %                 )}
1706 %             />
1707 %
1708 %             <div className="grid grid-cols-2 gap-4">
1709 %                 <FormField
1710 %                     control={chargeForm.control}
1711 %                     name="quantity"
1712 %                     render={({ field }) => (
1713 %                         <FormItem>
1714 %                             <FormLabel>Cantidad</FormLabel>
1715 %                             <FormControl>
1716 %                                 <Input
1717 %                                     type="number"
1718 %                                     min={1}
1719 %                                     {...field}
1720 %                                     onChange={(e) => field.onChange(parseInt(e.target.value) || 1)}
1721 %                                     data-testid="input-charge-quantity"
1722 %                                 />
1723 %                             </FormControl>
1724 %                             <FormMessage />
1725 %                         </FormItem>
1726 %                     )}
1727 %                 />
1728 %
1729 %                 <FormField
1730 %                     control={chargeForm.control}
1731 %                     name="unitPrice"
1732 %                     render={({ field }) => (
1733 %                         <FormItem>
1734 %                             <FormLabel>Precio Unitario</FormLabel>
1735 %                             <FormControl>
1736 %                                 <Input placeholder="0.00" {...field} data-testid="input-charge-price" />
1737 %                             </FormControl>
1738 %                             <FormMessage />
1739 %                         </FormItem>
1740 %                     )}
1741 %                 />
1742 %             </div>
1743 %
1744 %             <FormField
1745 %                 control={chargeForm.control}
1746 %                 name="notes"
1747 %                 render={({ field }) => (
1748 %                     <FormItem>
1749 %                         <FormLabel>Notas</FormLabel>
1750 %                         <FormControl>
1751 %                             <Textarea placeholder="Notas adicionales" {...field} data-testid="input-charge-notes" />
1752 %                         </FormControl>
1753 %                         <FormMessage />
1754 %                     </FormItem>
1755 %                 )}
1756 %             />
1757 %         </div>
1758 %
1759 %         <DialogFooter>
1760 %             <Button type="button" variant="outline" onClick={() => setIsChargeDialogOpen(false)}>
1761 %                 Cancelar
1762 %             </Button>
1763 %             <Button type="submit" disabled={createChargeMutation.isPending} data-testid="button-submit-charge">
1764 %                 {createChargeMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
```

client/src/pages/events.tsx (cont.)

```
1765 %         Agregar
1766 %       </Button>
1767 %     </DialogFooter>
1768 %   </form>
1769 % </Form>
1770 % </DialogContent>
1771 % </Dialog>
1772 %
1773 % { /* Add Table Dialog */ }
1774 % <Dialog open={isAddTableOpen} onOpenChange={setIsAddTableOpen}>
1775 %   <DialogContent className="max-w-sm">
1776 %     <DialogHeader>
1777 %       <DialogTitle>Agregar Mesa</DialogTitle>
1778 %       <DialogDescription>Defina los datos de la nueva mesa</DialogDescription>
1779 %     </DialogHeader>
1780 %     <div className="space-y-4">
1781 %       <div>
1782 %         <Label>Numero de Mesa</Label>
1783 %         <Input
1784 %           type="number"
1785 %           min={1}
1786 %           value={newTableNumber}
1787 %           onChange={(e) => setNewTableNumber(parseInt(e.target.value) || 1)}
1788 %           data-testid="input-table-number"
1789 %         />
1790 %       </div>
1791 %       <div>
1792 %         <Label>Etiqueta (opcional)</Label>
1793 %         <Input
1794 %           value={newTableLabel}
1795 %           onChange={(e) => setNewTableLabel(e.target.value)}
1796 %           placeholder="Ej: VIP, Terraza..."
1797 %           data-testid="input-table-label"
1798 %         />
1799 %       </div>
1800 %       <div>
1801 %         <Label>Asientos</Label>
1802 %         <Input
1803 %           type="number"
1804 %           min={1}
1805 %           value={newTableSeats}
1806 %           onChange={(e) => setNewTableSeats(parseInt(e.target.value) || 1)}
1807 %           data-testid="input-table-seats"
1808 %         />
1809 %       </div>
1810 %     </div>
1811 %     <DialogFooter>
1812 %       <Button variant="outline" onClick={() => setIsAddTableOpen(false)}>Cancelar</Button>
1813 %       <Button
1814 %         onClick={() => {
1815 %           if (selectedEvent) {
1816 %             createTableMutation.mutate({
1817 %               eventId: selectedEvent.id,
1818 %               data: { tableNumber: newTableNumber, label: newTableLabel || null, seats: newTableSeats },
1819 %             });
1820 %           }
1821 %         }}
1822 %         disabled={createTableMutation.isPending}
1823 %         data-testid="button-submit-table"
1824 %       >
1825 %         {createTableMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1826 %         Agregar
1827 %       </Button>
1828 %     </DialogFooter>
1829 %   </DialogContent>
1830 % </Dialog>
1831 %
1832 % { /* Table Folio Dialog */ }
1833 % <Dialog open={isTableFolioOpen} onOpenChange={(open) => { if (!open) { setIsTableFolioOpen(false); setSelectedT...
1834 %   <DialogContent className="max-w-3xl max-h-[85vh] overflow-y-auto">
1835 %     <DialogHeader>
1836 %       <DialogTitle className="flex items-center gap-2">
1837 %         Mesa {selectedTable?.tableNumber}
1838 %         {selectedTable?.label && <span className="text-muted-foreground font-normal">{selectedTable.label}</s...
1839 %         <Badge variant={selectedTable?.status === "open" ? "default" : "secondary"}>
1840 %           {selectedTable?.status === "open" ? "Abierta" : selectedTable?.status === "invoiced" ? "Facturada" : ...
1841 %         </Badge>
1842 %       </DialogTitle>
1843 %       <DialogDescription>
1844 %         {selectedTable?.seats && `${selectedTable.seats} asientos`}
1845 %       </DialogDescription>
1846 %     </DialogHeader>
1847 %
1848 %     {selectedTable && (
1849 %       <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
1850 %         { /* Left: Charges */ }
1851 %         <div>
1852 %           <h4 className="font-medium mb-3 flex items-center gap-2">
1853 %             <CreditCard className="h-4 w-4" />
1854 %             Cargos
1855 %           </h4>
1856 %           <div className="space-y-2 max-h-[250px] overflow-y-auto">
1857 %             {selectedTable.charges.map((charge) => (
```

client/src/pages/events.tsx (cont.)

```
1858 %         <div key={charge.id} className="flex items-center justify-between p-2 rounded border text-sm">
1859 %             <div>
1860 %                 <span className="font-medium">{charge.description}</span>
1861 %                 <span className="text-muted-foreground ml-2">x{charge.quantity}</span>
1862 %             </div>
1863 %             <div className="flex items-center gap-2">
1864 %                 <span className="font-medium">${charge.total}</span>
1865 %                 {selectedTable.status === "open" && (
1866 %                     <Button
1867 %                         variant="ghost"
1868 %                         size="icon"
1869 %                         className="h-6 w-6"
1870 %                         onClick={() => deleteTableChargeMutation.mutate({
1871 %                             eventId: selectedEvent!.id,
1872 %                             tableId: selectedTable.id,
1873 %                             chargeId: charge.id,
1874 %                         })}
1875 %                         data-testid={`button-delete-table-charge-${charge.id}`}
1876 %                     >
1877 %                         <Trash2 className="h-3 w-3" />
1878 %                     </Button>
1879 %                 )}
1880 %             </div>
1881 %         </div>
1882 %     )}
1883 %     {selectedTable.charges.length === 0 && (
1884 %         <p className="text-sm text-muted-foreground text-center py-4">Sin cargos</p>
1885 %     )}
1886 % </div>
1887 %
1888 % {selectedTable.status === "open" && (
1889 %     <div className="mt-3 pt-3 border-t space-y-2">
1890 %         <p className="text-sm font-medium">Agregar Cargo</p>
1891 %         <Input
1892 %             placeholder="Descripcion"
1893 %             value={tableChargeDesc}
1894 %             onChange={(e) => setTableChargeDesc(e.target.value)}
1895 %             data-testid="input-table-charge-desc"
1896 %         />
1897 %         <div className="grid grid-cols-2 gap-2">
1898 %             <Input
1899 %                 type="number"
1900 %                 min={1}
1901 %                 placeholder="Cant."
1902 %                 value={tableChargeQty}
1903 %                 onChange={(e) => setTableChargeQty(parseInt(e.target.value) || 1)}
1904 %                 data-testid="input-table-charge-qty"
1905 %             />
1906 %             <Input
1907 %                 type="number"
1908 %                 step="0.01"
1909 %                 placeholder="Precio Unit."
1910 %                 value={tableChargePrice}
1911 %                 onChange={(e) => setTableChargePrice(e.target.value)}
1912 %                 data-testid="input-table-charge-price"
1913 %             />
1914 %         </div>
1915 %         <Button
1916 %             size="sm"
1917 %             className="w-full"
1918 %             onClick={handleAddTableCharge}
1919 %             disabled={addTableChargeMutation.isPending || !tableChargeDesc || !tableChargePrice}
1920 %             data-testid="button-add-table-charge"
1921 %         >
1922 %             {addTableChargeMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
1923 %             Agregar Cargo
1924 %         </Button>
1925 %     </div>
1926 % )}
1927 %
1928 % <div className="mt-3 pt-3 border-t flex justify-between font-bold">
1929 %     <span>Total Cargos:</span>
1930 %     <span>${selectedTable.charges.reduce((s, c) => s + parseFloat(c.total), 0).toFixed(2)}</span>
1931 % </div>
1932 % </div>
1933 %
1934 % {/* Right: Payments */}
1935 % <div>
1936 %     <h4 className="font-medium mb-3 flex items-center gap-2">
1937 %         <DollarSign className="h-4 w-4" />
1938 %         Pagos
1939 %     </h4>
1940 %     <div className="space-y-2 max-h-[200px] overflow-y-auto">
1941 %         {selectedTable.payments.map((payment) => (
1942 %             <div key={payment.id} className="flex items-center justify-between p-2 rounded border text-sm">
1943 %                 <div className="flex items-center gap-2">
1944 %                     <span className="font-medium">${payment.amount}</span>
1945 %                     <span className="text-muted-foreground">{paymentMethodLabels[payment.method] || payment.metho...
1946 %                     {payment.isAdvance === "true" && (
1947 %                         <Badge variant="outline" className="text-xs border-amber-500 text-amber-600">SEÑA</Badge>
1948 %                     )}
1949 %                 </div>
1950 %             </div>
1951 %         )}
```

client/src/pages/events.tsx (cont.)

```
1951 %         )})
1952 %     {selectedTable.payments.length === 0 && (
1953 %         <p className="text-sm text-muted-foreground text-center py-4">Sin pagos</p>
1954 %     )}
1955 % </div>
1956 %
1957 % <div className="mt-3 pt-3 border-t space-y-3">
1958 %     <div className="flex justify-between font-bold">
1959 %         <span>Total Pagado:</span>
1960 %         <span className="text-green-600">
1961 %             ${selectedTable.payments.reduce((s, p) => s + parseFloat(p.amount), 0).toFixed(2)}
1962 %         </span>
1963 %     </div>
1964 %     {(() => {
1965 %         const tCharges = selectedTable.charges.reduce((s, c) => s + parseFloat(c.total), 0);
1966 %         const tPaid = selectedTable.payments.reduce((s, p) => s + parseFloat(p.amount), 0);
1967 %         const bal = tCharges - tPaid;
1968 %         return (
1969 %             <div className="flex justify-between font-bold text-lg">
1970 %                 <span>Saldo:</span>
1971 %                 <span className={bal > 0.01 ? "text-red-600" : "text-green-600"}>
1972 %                     ${bal.toFixed(2)}
1973 %                 </span>
1974 %             </div>
1975 %         );
1976 %     })()}
1977 %
1978 %     {selectedTable.status === "open" && (
1979 %         <div className="space-y-2 pt-2 border-t">
1980 %             <p className="text-sm font-medium">Agregar Pago</p>
1981 %             <div className="grid grid-cols-2 gap-2">
1982 %                 <Input
1983 %                     type="number"
1984 %                     step="0.01"
1985 %                     placeholder="Monto"
1986 %                     value={tablePayAmount}
1987 %                     onChange={(e) => setTablePayAmount(e.target.value)}
1988 %                     data-testid="input-table-pay-amount"
1989 %                 />
1990 %                 <Select value={tablePayMethod} onChange={setTablePayMethod}>
1991 %                     <SelectTrigger data-testid="select-table-pay-method">
1992 %                         <SelectValue />
1993 %                     </SelectTrigger>
1994 %                     <SelectContent>
1995 %                         {Object.entries(paymentMethodLabels).map(([k, v]) => (
1996 %                             <SelectItem key={k} value={k}>{v}</SelectItem>
1997 %                         ))}
1998 %                     </SelectContent>
1999 %                 </Select>
2000 %             </div>
2001 %             <div>
2002 %                 {tablePayMethod === "room_charge" && (
2003 %                     <Select value={tablePayResId} onChange={setTablePayResId}>
2004 %                         <SelectTrigger data-testid="select-table-pay-reservation">
2005 %                             <SelectValue placeholder="Seleccionar habitacion" />
2006 %                         </SelectTrigger>
2007 %                         <SelectContent>
2008 %                             {activeReservations.map((r) => (
2009 %                                 <SelectItem key={r.id} value={r.id}>
2010 %                                     Hab. {r.room?.roomNumber || "?"} - {r.guest?.firstName} {r.guest?.lastName}
2011 %                                 </SelectItem>
2012 %                             ))}
2013 %                             </SelectContent>
2014 %                         </Select>
2015 %                     )}
2016 %                 <label className="flex items-center gap-2 text-sm">
2017 %                     <input
2018 %                         type="checkbox"
2019 %                         checked={tablePayAdvance}
2020 %                         onChange={(e) => setTablePayAdvance(e.target.checked)}
2021 %                         data-testid="checkbox-table-advance"
2022 %                     />
2023 %                     Señal / Anticipo
2024 %                 </label>
2025 %                 <Button
2026 %                     size="sm"
2027 %                     className="w-full"
2028 %                     onClick={handleAddTablePayment}
2029 %                     disabled={addTablePaymentMutation.isPending || !tablePayAmount}
2030 %                     data-testid="button-add-table-payment"
2031 %                 >
2032 %                     {addTablePaymentMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
2033 %                     Registrar Pago
2034 %                 </Button>
2035 %             </div>
2036 %
2037 % <div className="space-y-2 pt-2 border-t">
2038 %     <p className="text-sm font-medium">Cerrar Mesa</p>
2039 %     <Select value={tableFolioReceiptType} onChange={setTableFolioReceiptType}>
2040 %         <SelectTrigger data-testid="select-table-receipt-type">
2041 %             <SelectValue placeholder="Tipo de comprobante" />
2042 %         </SelectTrigger>
2043 %         <SelectContent>
```

client/src/pages/events.tsx (cont.)

```
2044 %         {receiptTypes.map((rt) => (  
2045 %             <SelectItem key={rt} value={rt}>{rt}</SelectItem>  
2046 %         ))}  
2047 %     </SelectContent>  
2048 % </Select>  
2049 % <Button  
2050 %     className="w-full"  
2051 %     disabled={  
2052 %         !tableFolioReceiptType ||  
2053 %         selectedTable.charges.reduce((s, c) => s + parseFloat(c.total), 0) -  
2054 %         selectedTable.payments.reduce((s, p) => s + parseFloat(p.amount), 0) > 0.01 ||  
2055 %         closeTableMutation.isPending  
2056 %     }  
2057 %     onClick={handleCloseTable}  
2058 %     data-testid="button-close-table"  
2059 % >  
2060 %     {closeTableMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}  
2061 %     <Receipt className="h-4 w-4 mr-2" />  
2062 %     Cerrar Mesa  
2063 % </Button>  
2064 % </div>  
2065 % </>  
2066 %     })  
2067 % </div>  
2068 % </div>  
2069 % </div>  
2070 % })  
2071 % </DialogContent>  
2072 % </Dialog>  
2073 % </div>  
2074 % );  
2075 % }  
2076 %
```



```

1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import {
4 %   Sparkles,
5 %   CheckCircle,
6 %   Play,
7 %   Clock,
8 %   AlertCircle,
9 %   Filter,
10 %   ClipboardCheck,
11 %   Bed,
12 %   Wrench,
13 %   XCircle,
14 %   MoreHorizontal,
15 %   MessageSquare,
16 % } from "lucide-react";
17 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
18 % import { Button } from "@components/ui/button";
19 % import { Badge } from "@components/ui/badge";
20 % import { Skeleton } from "@components/ui/skeleton";
21 % import {
22 %   Select,
23 %   SelectContent,
24 %   SelectItem,
25 %   SelectTrigger,
26 %   SelectValue,
27 % } from "@components/ui/select";
28 % import {
29 %   Dialog,
30 %   DialogContent,
31 %   DialogDescription,
32 %   DialogFooter,
33 %   DialogHeader,
34 %   DialogTitle,
35 % } from "@components/ui/dialog";
36 % import {
37 %   Popover,
38 %   PopoverContent,
39 %   PopoverTrigger,
40 % } from "@components/ui/popover";
41 % import { Label } from "@components/ui/label";
42 % import { Textarea } from "@components/ui/textarea";
43 % import { useToast } from "@hooks/use-toast";
44 % import { queryClient, apiRequest } from "@lib/queryClient";
45 % import type { RoomWithType, RoomStatus, HousekeepingTaskWithRoom } from "@shared/schema";
46 %
47 % type TaskStatus = "pending" | "in_progress" | "completed" | "inspected";
48 % type TaskType = "checkout_clean" | "stayover_clean" | "deep_clean" | "inspection" | "turndown" | "maintenance_prep";
49 % type Priority = "low" | "normal" | "high" | "urgent";
50 %
51 % const statusConfig: Record<RoomStatus, { label: string; icon: typeof Sparkles; className: string; bgClass: string }> = {
52 %   available: { label: "Disponible", icon: CheckCircle, className: "text-green-600 dark:text-green-400", bgClass: "bg-...
53 %   occupied: { label: "Ocupada", icon: Bed, className: "text-blue-600 dark:text-blue-400", bgClass: "bg-blue-50 dark:b...
54 %   dirty: { label: "Sucia", icon: AlertCircle, className: "text-orange-600 dark:text-orange-400", bgClass: "bg-orange-...
55 %   cleaning: { label: "Limpiando", icon: Sparkles, className: "text-yellow-600 dark:text-yellow-400", bgClass: "bg-yel...
56 %   maintenance: { label: "Mantenimiento", icon: Wrench, className: "text-red-600 dark:text-red-400", bgClass: "bg-red-...
57 %   oos: { label: "Fuera Servicio", icon: XCircle, className: "text-gray-600 dark:text-gray-400", bgClass: "bg-gray-50 ...
58 % };
59 %
60 % const taskStatusConfig: Record<TaskStatus, { label: string; className: string }> = {
61 %   pending: { label: "Pendiente", className: "bg-orange-100 text-orange-800 dark:bg-orange-900/30 dark:text-orange-400...
62 %   in_progress: { label: "En Progreso", className: "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yell...
63 %   completed: { label: "Completada", className: "bg-green-100 text-green-800 dark:bg-green-900/30 dark:text-green-400" },
64 %   inspected: { label: "Inspeccionada", className: "bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-400" },
65 % };
66 %
67 % const taskTypeLabels: Record<TaskType, string> = {
68 %   checkout_clean: "Limpieza Check-out",
69 %   stayover_clean: "Limpieza Estancia",
70 %   deep_clean: "Limpieza Profunda",
71 %   inspection: "Inspeccion",
72 %   turndown: "Turndown",
73 %   maintenance_prep: "Prep. Mantenimiento",
74 % };
75 %
76 % const priorityConfig: Record<Priority, { label: string; className: string }> = {
77 %   low: { label: "Baja", className: "bg-gray-100 text-gray-800 dark:bg-gray-900/30 dark:text-gray-400" },
78 %   normal: { label: "Normal", className: "bg-blue-100 text-blue-800 dark:bg-blue-900/30 dark:text-blue-400" },
79 %   high: { label: "Alta", className: "bg-orange-100 text-orange-800 dark:bg-orange-900/30 dark:text-orange-400" },
80 %   urgent: { label: "Urgente", className: "bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-400" },
81 % };
82 %
83 % function RoomCard({
84 %   room,
85 %   tasks,
86 %   onStartTask,
87 %   onCompleteTask,
88 %   onCreateTask,
89 %   onUpdateStatus,
90 %   onOpenDetails,

```

client/src/pages/housekeeping.tsx (cont.)

```
91 %   }: {
92 %     room: RoomWithType;
93 %     tasks: HousekeepingTaskWithRoom[];
94 %     onStartTask: (taskId: string) => void;
95 %     onCompleteTask: (taskId: string) => void;
96 %     onCreateTask: (roomId: string) => void;
97 %     onUpdateStatus: (roomId: string, status: RoomStatus) => void;
98 %     onOpenDetails: (roomId: string) => void;
99 %   }) {
100 %   const config = statusConfig[room.status];
101 %   const Icon = config.icon;
102 %   const pendingTasks = tasks.filter(t => t.status === "pending" || t.status === "in_progress");
103 %   const hasNotes = tasks.some(t => t.notes);
104 %
105 %   return (
106 %     <Card className={` ${config.bgClass} border transition-all`} >
107 %       <CardContent className="p-3">
108 %         <div className="flex items-center justify-between mb-2">
109 %           <div className="flex items-center gap-2">
110 %             <span className="font-bold text-lg">{room.roomNumber}</span>
111 %             <Icon className={`h-4 w-4 ${config.className}`} />
112 %           </div>
113 %           <div className="flex items-center gap-1">
114 %             {hasNotes && (
115 %               <MessageSquare className="h-3 w-3 text-muted-foreground" />
116 %             )}
117 %             <Badge variant="outline" className="text-xs">
118 %               {room.roomType?.code || "N/A"}
119 %             </Badge>
120 %             <Popover>
121 %               <PopoverTrigger asChild>
122 %                 <Button
123 %                   size="icon"
124 %                   variant="ghost"
125 %                   className="h-6 w-6"
126 %                   data-testid={`button-room-menu-${room.id}`}
127 %                 >
128 %                   <MoreHorizontal className="h-3 w-3" />
129 %                 </Button>
130 %               </PopoverTrigger>
131 %               <PopoverContent className="w-48 p-2" align="end">
132 %                 <div className="space-y-1">
133 %                   <Button
134 %                     size="sm"
135 %                     variant="ghost"
136 %                     className="w-full justify-start h-8 text-xs"
137 %                     onClick={() => onOpenDetails(room.id)}
138 %                     data-testid={`button-details-${room.id}`}
139 %                   >
140 %                     <MessageSquare className="h-3 w-3 mr-2" />
141 %                     Ver Detalles / Notas
142 %                   </Button>
143 %                   <Button
144 %                     size="sm"
145 %                     variant="ghost"
146 %                     className="w-full justify-start h-8 text-xs"
147 %                     onClick={() => onUpdateStatus(room.id, "available")}
148 %                     data-testid={`button-quick-available-${room.id}`}
149 %                   >
150 %                     <CheckCircle className="h-3 w-3 mr-2 text-green-500" />
151 %                     Marcar Disponible
152 %                   </Button>
153 %                   <Button
154 %                     size="sm"
155 %                     variant="ghost"
156 %                     className="w-full justify-start h-8 text-xs"
157 %                     onClick={() => onUpdateStatus(room.id, "dirty")}
158 %                     data-testid={`button-quick-dirty-${room.id}`}
159 %                   >
160 %                     <AlertCircle className="h-3 w-3 mr-2 text-orange-500" />
161 %                     Marcar Sucia
162 %                   </Button>
163 %                   <Button
164 %                     size="sm"
165 %                     variant="ghost"
166 %                     className="w-full justify-start h-8 text-xs"
167 %                     onClick={() => onUpdateStatus(room.id, "maintenance")}
168 %                     data-testid={`button-quick-maintenance-${room.id}`}
169 %                   >
170 %                     <Wrench className="h-3 w-3 mr-2 text-red-500" />
171 %                     Mantenimiento
172 %                   </Button>
173 %                 </div>
174 %               </PopoverContent>
175 %             </Popover>
176 %           </div>
177 %         </div>
178 %
179 %         <div className="text-xs text-muted-foreground mb-2">
180 %           {config.label}
181 %         </div>
182 %
183 %         {pendingTasks.length > 0 ? (
```

client/src/pages/housekeeping.tsx (cont.)

```
184 %         <div className="space-y-1">
185 %             {pendingTasks.map(task => (
186 %                 <div key={task.id} className="flex items-center justify-between gap-1">
187 %                     <Badge className={` ${taskStatusConfig[task.status as TaskStatus].className} text-xs`} >
188 %                         {taskTypeLabels[task.taskType as TaskType] || task.taskType}
189 %                     </Badge>
190 %                     {task.status === "pending" && (
191 %                         <Button
192 %                             size="sm"
193 %                             variant="ghost"
194 %                             className="h-6 px-2"
195 %                             onClick={() => onStartTask(task.id)}
196 %                             data-testid={`button-start-task-${task.id}`}
197 %                         >
198 %                             <Play className="h-3 w-3" />
199 %                         </Button>
200 %                     )}
201 %                     {task.status === "in_progress" && (
202 %                         <Button
203 %                             size="sm"
204 %                             variant="ghost"
205 %                             className="h-6 px-2"
206 %                             onClick={() => onCompleteTask(task.id)}
207 %                             data-testid={`button-complete-task-${task.id}`}
208 %                         >
209 %                             <CheckCircle className="h-3 w-3" />
210 %                         </Button>
211 %                     )}
212 %                 </div>
213 %             )}
214 %         </div>
215 %     ) : (
216 %         <div className="flex gap-1 flex-wrap">
217 %             {room.status === "dirty" && (
218 %                 <>
219 %                     <Button
220 %                         size="sm"
221 %                         variant="outline"
222 %                         className="h-6 px-2 text-xs"
223 %                         onClick={() => onUpdateStatus(room.id, "available")}
224 %                         data-testid={`button-mark-clean-${room.id}`}
225 %                     >
226 %                         <CheckCircle className="h-3 w-3 mr-1" />
227 %                         Marcar Limpia
228 %                     </Button>
229 %                     <Button
230 %                         size="sm"
231 %                         variant="ghost"
232 %                         className="h-6 px-2 text-xs"
233 %                         onClick={() => onCreateTask(room.id)}
234 %                         data-testid={`button-create-task-${room.id}`}
235 %                     >
236 %                         <Sparkles className="h-3 w-3 mr-1" />
237 %                         Tarea
238 %                     </Button>
239 %                 </>
240 %             )}
241 %             {room.status === "cleaning" && (
242 %                 <Button
243 %                     size="sm"
244 %                     variant="outline"
245 %                     className="h-6 px-2 text-xs"
246 %                     onClick={() => onUpdateStatus(room.id, "available")}
247 %                     data-testid={`button-finish-clean-${room.id}`}
248 %                 >
249 %                     <CheckCircle className="h-3 w-3 mr-1" />
250 %                     Marcar Limpia
251 %                 </Button>
252 %             )}
253 %             {room.status === "available" && (
254 %                 <Button
255 %                     size="sm"
256 %                     variant="ghost"
257 %                     className="h-6 px-2 text-xs text-muted-foreground"
258 %                     onClick={() => onUpdateStatus(room.id, "dirty")}
259 %                     data-testid={`button-mark-dirty-${room.id}`}
260 %                 >
261 %                     Marcar Sucia
262 %                 </Button>
263 %             )}
264 %         </div>
265 %     )}
266 % </CardContent>
267 % </Card>
268 % );
269 % }
270 %
271 % export default function Housekeeping() {
272 %     const { toast } = useToast();
273 %     const [floorFilter, setFloorFilter] = useState<string>("all");
274 %     const [statusFilter, setStatusFilter] = useState<string>("all");
275 %     const [createTaskDialogOpen, setCreateTaskDialogOpen] = useState(false);
276 %     const [detailsDialogOpen, setDetailsDialogOpen] = useState(false);
```

client/src/pages/housekeeping.tsx (cont.)

```
277 %   const [selectedRoomId, setSelectedRoomId] = useState<string | null>(null);
278 %   const [taskType, setTaskType] = useState<TaskType>("checkout_clean");
279 %   const [priority, setPriority] = useState<Priority>("normal");
280 %   const [notes, setNotes] = useState("");
281 %   const [detailNotes, setDetailNotes] = useState("");
282 %
283 %   const { data: rooms, isLoading: roomsLoading } = useQuery<RoomWithType[]>({
284 %     queryKey: ["/api/rooms"],
285 %   });
286 %
287 %   const today = new Date().toISOString().split("T")[0];
288 %   const { data: tasks, isLoading: tasksLoading } = useQuery<HousekeepingTaskWithRoom[]>({
289 %     queryKey: ["/api/housekeeping", { date: today }],
290 %   });
291 %
292 %   const startTaskMutation = useMutation({
293 %     mutationFn: (taskId: string) => apiRequest("POST", `/api/housekeeping/${taskId}/start`),
294 %     onSuccess: () => {
295 %       queryClient.invalidateQueries({ queryKey: ["/api/housekeeping"] });
296 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
297 %       toast({ title: "Tarea iniciada", description: "La limpieza ha comenzado." });
298 %     },
299 %     onError: () => {
300 %       toast({ title: "Error", description: "No se pudo iniciar la tarea.", variant: "destructive" });
301 %     },
302 %   });
303 %
304 %   const completeTaskMutation = useMutation({
305 %     mutationFn: (taskId: string) => apiRequest("POST", `/api/housekeeping/${taskId}/complete`),
306 %     onSuccess: () => {
307 %       queryClient.invalidateQueries({ queryKey: ["/api/housekeeping"] });
308 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
309 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
310 %       toast({ title: "Tarea completada", description: "La habitacion esta lista." });
311 %     },
312 %     onError: () => {
313 %       toast({ title: "Error", description: "No se pudo completar la tarea.", variant: "destructive" });
314 %     },
315 %   });
316 %
317 %   const updateRoomStatusMutation = useMutation({
318 %     mutationFn: ({ roomId, status }: { roomId: string; status: RoomStatus }) =>
319 %       apiRequest("PATCH", `/api/housekeeping/room/${roomId}/status`, { status }),
320 %     onSuccess: () => {
321 %       queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
322 %       queryClient.invalidateQueries({ queryKey: ["/api/dashboard/stats"] });
323 %       toast({ title: "Estado actualizado" });
324 %     },
325 %     onError: () => {
326 %       toast({ title: "Error", description: "No se pudo actualizar el estado.", variant: "destructive" });
327 %     },
328 %   });
329 %
330 %   const createTaskMutation = useMutation({
331 %     mutationFn: (data: { roomId: string; taskType: string; priority: string; notes?: string; scheduledDate: string })...
332 %     apiRequest("POST", "/api/housekeeping", data),
333 %     onSuccess: () => {
334 %       queryClient.invalidateQueries({ queryKey: ["/api/housekeeping"] });
335 %       setCreateTaskDialogOpen(false);
336 %       setSelectedRoomId(null);
337 %       setNotes("");
338 %       toast({ title: "Tarea creada", description: "La tarea de limpieza ha sido asignada." });
339 %     },
340 %     onError: () => {
341 %       toast({ title: "Error", description: "No se pudo crear la tarea.", variant: "destructive" });
342 %     },
343 %   });
344 %
345 %   const floors = rooms ? Array.from(new Set(rooms.map(r => r.floor))).sort((a, b) => a - b) : [];
346 %
347 %   const filteredRooms = rooms?.filter(room => {
348 %     if (floorFilter !== "all" && room.floor !== parseInt(floorFilter)) return false;
349 %     if (statusFilter !== "all" && room.status !== statusFilter) return false;
350 %     return true;
351 %   }) || [];
352 %
353 %   const roomsByFloor = filteredRooms.reduce((acc, room) => {
354 %     if (!acc[room.floor]) acc[room.floor] = [];
355 %     acc[room.floor].push(room);
356 %     return acc;
357 %   }, {} as Record<number, RoomWithType[]>);
358 %
359 %   const getTasksForRoom = (roomId: string) =>
360 %     tasks?.filter(t => t.roomId === roomId) || [];
361 %
362 %   const stats = rooms ? {
363 %     available: rooms.filter(r => r.status === "available").length,
364 %     occupied: rooms.filter(r => r.status === "occupied").length,
365 %     dirty: rooms.filter(r => r.status === "dirty").length,
366 %     cleaning: rooms.filter(r => r.status === "cleaning").length,
367 %     maintenance: rooms.filter(r => r.status === "maintenance").length,
368 %   } : { available: 0, occupied: 0, dirty: 0, cleaning: 0, maintenance: 0 };
369 %
```

client/src/pages/housekeeping.tsx (cont.)

```
370 %   const handleCreateTask = (roomId: string) => {
371 %       setSelectedRoomId(roomId);
372 %       setCreateTaskDialogOpen(true);
373 %   };
374 %
375 %   const handleOpenDetails = (roomId: string) => {
376 %       setSelectedRoomId(roomId);
377 %       const roomTasks = getTasksForRoom(roomId);
378 %       const existingNotes = roomTasks.find(t => t.notes)?.notes || "";
379 %       setDetailNotes(existingNotes);
380 %       setDetailsDialogOpen(true);
381 %   };
382 %
383 %   const selectedRoom = rooms?.find(r => r.id === selectedRoomId);
384 %   const selectedRoomTasks = selectedRoomId ? getTasksForRoom(selectedRoomId) : [];
385 %
386 %   const handleSubmitTask = () => {
387 %       if (!selectedRoomId) return;
388 %       createTaskMutation.mutate({
389 %           roomId: selectedRoomId,
390 %           taskType,
391 %           priority,
392 %           notes: notes || undefined,
393 %           scheduledDate: today,
394 %       });
395 %   };
396 %
397 %   if (roomsLoading || tasksLoading) {
398 %       return (
399 %           <div className="p-6 space-y-6">
400 %               <Skeleton className="h-8 w-48" />
401 %               <div className="grid grid-cols-4 gap-4">
402 %                   {[1, 2, 3, 4].map(i => <Skeleton key={i} className="h-24" />)}
403 %               </div>
404 %               <div className="grid grid-cols-7 gap-3">
405 %                   {Array(14).fill(0).map((_, i) => <Skeleton key={i} className="h-32" />)}
406 %               </div>
407 %           </div>
408 %       );
409 %   }
410 %
411 %   return (
412 %       <div className="p-6 space-y-6">
413 %           <div className="flex items-center justify-between flex-wrap gap-4">
414 %               <div className="flex items-center gap-2">
415 %                   <ClipboardCheck className="h-6 w-6 text-muted-foreground" />
416 %                   <h1 className="text-2xl font-bold" data-testid="text-page-title">Housekeeping</h1>
417 %               </div>
418 %
419 %               <div className="flex items-center gap-2">
420 %                   <Filter className="h-4 w-4 text-muted-foreground" />
421 %                   <Select value={floorFilter} onChange={setFloorFilter}>
422 %                       <SelectTrigger className="w-32" data-testid="select-floor-filter">
423 %                           <SelectValue placeholder="Piso" />
424 %                       </SelectTrigger>
425 %                       <SelectContent>
426 %                           <SelectItem value="all">Todos los pisos</SelectItem>
427 %                           {floors.map(floor => (
428 %                               <SelectItem key={floor} value={floor.toString()}>Piso {floor}</SelectItem>
429 %                           ))}
430 %                       </SelectContent>
431 %                   </Select>
432 %
433 %                   <Select value={statusFilter} onChange={setStatusFilter}>
434 %                       <SelectTrigger className="w-40" data-testid="select-status-filter">
435 %                           <SelectValue placeholder="Estado" />
436 %                       </SelectTrigger>
437 %                       <SelectContent>
438 %                           <SelectItem value="all">Todos los estados</SelectItem>
439 %                           <SelectItem value="available">Disponible</SelectItem>
440 %                           <SelectItem value="occupied">Ocupada</SelectItem>
441 %                           <SelectItem value="dirty">Sucia</SelectItem>
442 %                           <SelectItem value="cleaning">Limpiando</SelectItem>
443 %                           <SelectItem value="maintenance">Mantenimiento</SelectItem>
444 %                       </SelectContent>
445 %                   </Select>
446 %               </div>
447 %           </div>
448 %
449 %           <div className="grid grid-cols-2 md:grid-cols-5 gap-4">
450 %               <Card>
451 %                   <CardContent className="p-4">
452 %                       <div className="flex items-center gap-2">
453 %                           <CheckCircle className="h-5 w-5 text-green-500" />
454 %                           <div>
455 %                               <p className="text-2xl font-bold" data-testid="stat-available">{stats.available}</p>
456 %                               <p className="text-xs text-muted-foreground">Disponibles</p>
457 %                           </div>
458 %                       </div>
459 %                   </CardContent>
460 %               </Card>
461 %               <Card>
462 %                   <CardContent className="p-4">
```

client/src/pages/housekeeping.tsx (cont.)

```
463 %         <div className="flex items-center gap-2">
464 %           <Bed className="h-5 w-5 text-blue-500" />
465 %           <div>
466 %             <p className="text-2xl font-bold" data-testid="stat-occupied">{stats.occupied}</p>
467 %             <p className="text-xs text-muted-foreground">Ocupadas</p>
468 %           </div>
469 %         </div>
470 %       </CardContent>
471 %     </Card>
472 %   <Card>
473 %     <CardContent className="p-4">
474 %       <div className="flex items-center gap-2">
475 %         <AlertCircle className="h-5 w-5 text-orange-500" />
476 %         <div>
477 %           <p className="text-2xl font-bold" data-testid="stat-dirty">{stats.dirty}</p>
478 %           <p className="text-xs text-muted-foreground">Sucias</p>
479 %         </div>
480 %       </div>
481 %     </CardContent>
482 %   </Card>
483 %   <Card>
484 %     <CardContent className="p-4">
485 %       <div className="flex items-center gap-2">
486 %         <Sparkles className="h-5 w-5 text-yellow-500" />
487 %         <div>
488 %           <p className="text-2xl font-bold" data-testid="stat-cleaning">{stats.cleaning}</p>
489 %           <p className="text-xs text-muted-foreground">Limpiando</p>
490 %         </div>
491 %       </div>
492 %     </CardContent>
493 %   </Card>
494 %   <Card>
495 %     <CardContent className="p-4">
496 %       <div className="flex items-center gap-2">
497 %         <Wrench className="h-5 w-5 text-red-500" />
498 %         <div>
499 %           <p className="text-2xl font-bold" data-testid="stat-maintenance">{stats.maintenance}</p>
500 %           <p className="text-xs text-muted-foreground">Mantenimiento</p>
501 %         </div>
502 %       </div>
503 %     </CardContent>
504 %   </Card>
505 % </div>
506 %
507 % {Object.entries(roomsByFloor)
508 %   .sort(([a], [b]) => parseInt(a) - parseInt(b))
509 %   .map([floor, floorRooms]) => (
510 %     <div key={floor} className="space-y-3">
511 %       <h2 className="text-lg font-semibold flex items-center gap-2">
512 %         <Clock className="h-4 w-4 text-muted-foreground" />
513 %         Piso {floor}
514 %       </h2>
515 %       <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 lg:grid-cols-7 gap-3">
516 %         {floorRooms
517 %           .sort((a, b) => a.roomNumber.localeCompare(b.roomNumber))
518 %           .map(room => (
519 %             <RoomCard
520 %               key={room.id}
521 %               room={room}
522 %               tasks={getTasksForRoom(room.id)}
523 %               onStartTask={() => startTaskMutation.mutate(taskId)}
524 %               onCompleteTask={() => completeTaskMutation.mutate(taskId)}
525 %               onCreateTask={handleCreateTask}
526 %               onUpdateStatus={() =>
527 %                 updateRoomStatusMutation.mutate({ roomId, status })
528 %               }
529 %               onOpenDetails={handleOpenDetails}
530 %             </RoomCard>
531 %           ))}
532 %       </div>
533 %     </div>
534 %   )}
535 %
536 % <Dialog open={createTaskDialogOpen} onOpenChange={setCreateTaskDialogOpen}>
537 %   <DialogContent>
538 %     <DialogHeader>
539 %       <DialogTitle>Crear Tarea de Limpieza</DialogTitle>
540 %       <DialogDescription>
541 %         Asignar una nueva tarea para la habitacion {rooms?.find(r => r.id === selectedRoomId)?.roomNumber}
542 %       </DialogDescription>
543 %     </DialogHeader>
544 %
545 %     <div className="space-y-4">
546 %       <div className="space-y-2">
547 %         <Label>Tipo de Tarea</Label>
548 %         <Select value={taskType} onValueChange={(v) => setTaskType(v as TaskType)}>
549 %           <SelectTrigger data-testid="select-task-type">
550 %             <SelectValue />
551 %           </SelectTrigger>
552 %           <SelectContent>
553 %             <SelectItem value="checkout_clean">Limpieza Check-out</SelectItem>
554 %             <SelectItem value="stayover_clean">Limpieza Estancia</SelectItem>
555 %             <SelectItem value="deep_clean">Limpieza Profunda</SelectItem>
```

```

556 %         <SelectItem value="inspection">Inspeccion</SelectItem>
557 %         <SelectItem value="turndown">Turndown</SelectItem>
558 %         <SelectItem value="maintenance_prep">Prep. Mantenimiento</SelectItem>
559 %     </SelectContent>
560 % </Select>
561 % </div>
562 %
563 % <div className="space-y-2">
564 %   <Label>Prioridad</Label>
565 %   <Select value={priority} onChange={(v) => setPriority(v as Priority)}>
566 %     <SelectTrigger data-testid="select-priority">
567 %       <SelectValue />
568 %     </SelectTrigger>
569 %     <SelectContent>
570 %       <SelectItem value="low">Baja</SelectItem>
571 %       <SelectItem value="normal">Normal</SelectItem>
572 %       <SelectItem value="high">Alta</SelectItem>
573 %       <SelectItem value="urgent">Urgente</SelectItem>
574 %     </SelectContent>
575 %   </Select>
576 % </div>
577 %
578 % <div className="space-y-2">
579 %   <Label>Notas (opcional)</Label>
580 %   <Textarea
581 %     value={notes}
582 %     onChange={(e) => setNotes(e.target.value)}
583 %     placeholder="Instrucciones adicionales..."
584 %     data-testid="input-notes"
585 %   />
586 % </div>
587 % </div>
588 %
589 % <DialogFooter>
590 %   <Button variant="outline" onClick={() => setCreateTaskDialogOpen(false)}>
591 %     Cancelar
592 %   </Button>
593 %   <Button
594 %     onClick={handleSubmitTask}
595 %     disabled={createTaskMutation.isPending}
596 %     data-testid="button-submit-task"
597 %   >
598 %     {createTaskMutation.isPending ? "Creando..." : "Crear Tarea"}
599 %   </Button>
600 % </DialogFooter>
601 % </DialogContent>
602 % </Dialog>
603 %
604 % <Dialog open={detailsDialogOpen} onOpenChange={setDetailsDialogOpen}>
605 %   <DialogContent className="max-w-md">
606 %     <DialogHeader>
607 %       <DialogTitle>Habitacion {selectedRoom?.roomNumber}</DialogTitle>
608 %       <DialogDescription>
609 %         {selectedRoom?.roomType?.name} - {statusConfig[selectedRoom?.status || "available"].label}
610 %       </DialogDescription>
611 %     </DialogHeader>
612 %
613 %     <div className="space-y-4">
614 %       <div className="space-y-2">
615 %         <Label className="text-sm font-medium">Estado Actual</Label>
616 %         <div className="flex gap-2 flex-wrap">
617 %           <Button
618 %             size="sm"
619 %             variant={selectedRoom?.status === "available" ? "default" : "outline"}
620 %             onClick={() => {
621 %               if (selectedRoomId) {
622 %                 updateRoomStatusMutation.mutate({ roomId: selectedRoomId, status: "available" });
623 %               }
624 %             }}
625 %             data-testid="button-detail-available"
626 %           >
627 %             <CheckCircle className="h-3 w-3 mr-1" />
628 %             Disponible
629 %           </Button>
630 %           <Button
631 %             size="sm"
632 %             variant={selectedRoom?.status === "dirty" ? "default" : "outline"}
633 %             onClick={() => {
634 %               if (selectedRoomId) {
635 %                 updateRoomStatusMutation.mutate({ roomId: selectedRoomId, status: "dirty" });
636 %               }
637 %             }}
638 %             data-testid="button-detail-dirty"
639 %           >
640 %             <AlertCircle className="h-3 w-3 mr-1" />
641 %             Sucia
642 %           </Button>
643 %           <Button
644 %             size="sm"
645 %             variant={selectedRoom?.status === "maintenance" ? "default" : "outline"}
646 %             onClick={() => {
647 %               if (selectedRoomId) {
648 %                 updateRoomStatusMutation.mutate({ roomId: selectedRoomId, status: "maintenance" });

```

client/src/pages/housekeeping.tsx (cont.)

```
649 %         }
650 %     }}
651 %     data-testid="button-detail-maintenance"
652 % >
653 %     <Wrench className="h-3 w-3 mr-1" />
654 %     Mantenimiento
655 % </Button>
656 % </div>
657 % </div>
658 %
659 % {selectedRoomTasks.length > 0 && (
660 %   <div className="space-y-2">
661 %     <Label className="text-sm font-medium">Tareas de Hoy</Label>
662 %     <div className="space-y-2">
663 %       {selectedRoomTasks.map(task => (
664 %         <div key={task.id} className="flex items-center justify-between p-2 rounded-md bg-muted/50">
665 %           <div>
666 %             <Badge className={` ${taskStatusConfig[task.status as TaskStatus].className} text-xs`} >
667 %               {taskTypeLabels[task.taskType as TaskType]}
668 %             </Badge>
669 %             <Badge className={` ${priorityConfig[task.priority as Priority].className} text-xs ml-1`} >
670 %               {priorityConfig[task.priority as Priority].label}
671 %             </Badge>
672 %             {task.notes && (
673 %               <p className="text-xs text-muted-foreground mt-1">{task.notes}</p>
674 %             )}
675 %           </div>
676 %         </div>
677 %       )}}
678 %     </div>
679 %   </div>
680 % )}
681 %
682 % <div className="space-y-2">
683 %   <Label className="text-sm font-medium">Agregar Nota Rapida</Label>
684 %   <Textarea
685 %     value={detailNotes}
686 %     onChange={(e) => setDetailNotes(e.target.value)}
687 %     placeholder="Observaciones, problemas, solicitudes especiales..."
688 %     className="text-sm"
689 %     data-testid="input-detail-notes"
690 %   />
691 %   <Button
692 %     size="sm"
693 %     variant="outline"
694 %     onClick={() => {
695 %       if (selectedRoomId && detailNotes.trim()) {
696 %         createTaskMutation.mutate({
697 %           roomId: selectedRoomId,
698 %           taskType: "inspection",
699 %           priority: "normal",
700 %           notes: detailNotes,
701 %           scheduledDate: today,
702 %         });
703 %         setDetailsDialogOpen(false);
704 %       }
705 %     }}
706 %     disabled={!detailNotes.trim()}
707 %     data-testid="button-save-note"
708 %   >
709 %     <MessageSquare className="h-3 w-3 mr-1" />
710 %     Guardar como Tarea
711 %   </Button>
712 % </div>
713 % </div>
714 %
715 % <DialogFooter>
716 %   <Button variant="outline" onClick={() => setDetailsDialogOpen(false)}>
717 %     Cerrar
718 %   </Button>
719 % </DialogFooter>
720 % </DialogContent>
721 % </Dialog>
722 % </div>
723 % );
724 % }
725 %
```



```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { useForm } from "react-hook-form";
5 % import { zodResolver } from "hookform/resolvers/zod";
6 % import { z } from "zod";
7 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
8 % import { Button } from "@components/ui/button";
9 % import { Badge } from "@components/ui/badge";
10 % import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter, DialogDescription } from "@components/ui/di...
11 % import { Input } from "@components/ui/input";
12 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
13 % import { Textarea } from "@components/ui/textarea";
14 % import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
15 % import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@components/ui/table";
16 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
17 % import { useToast } from "@hooks/use-toast";
18 % import { format, parseISO } from "date-fns";
19 % import { es } from "date-fns/locale";
20 % import {
21 %   Plus,
22 %   Loader2,
23 %   Wrench,
24 %   AlertTriangle,
25 %   Clock,
26 %   CheckCircle,
27 %   User,
28 %   Phone,
29 %   Mail,
30 %   Building2,
31 %   Trash2,
32 %   Edit,
33 %   Play,
34 % } from "lucide-react";
35 %
36 % type Room = {
37 %   id: string;
38 %   roomNumber: string;
39 %   floor: number;
40 % };
41 %
42 % type MaintenanceStaff = {
43 %   id: string;
44 %   name: string;
45 %   phone: string | null;
46 %   email: string | null;
47 %   specialty: string | null;
48 %   isActive: string;
49 % };
50 %
51 % type WorkOrder = {
52 %   id: string;
53 %   orderCode: string;
54 %   title: string;
55 %   description: string | null;
56 %   roomId: string | null;
57 %   location: string | null;
58 %   category: "plumbing" | "electrical" | "hvac" | "furniture" | "cleaning" | "appliances" | "structure" | "general";
59 %   priority: "low" | "medium" | "high" | "urgent";
60 %   status: "pending" | "assigned" | "in_progress" | "completed" | "cancelled";
61 %   assignedToId: string | null;
62 %   reportedBy: string | null;
63 %   reportedAt: string;
64 %   scheduledDate: string | null;
65 %   completedAt: string | null;
66 %   completedBy: string | null;
67 %   estimatedCost: string | null;
68 %   actualCost: string | null;
69 %   notes: string | null;
70 %   room?: Room;
71 %   assignedTo?: MaintenanceStaff;
72 % };
73 %
74 % type DashboardStats = {
75 %   pending: number;
76 %   inProgress: number;
77 %   completedToday: number;
78 %   urgent: number;
79 %   totalStaff: number;
80 %   recentOrders: WorkOrder[];
81 % };
82 %
83 % const priorityColors: Record<string, string> = {
84 %   low: "bg-gray-500/20 text-gray-700 dark:text-gray-400 border-gray-500/30",
85 %   medium: "bg-blue-500/20 text-blue-700 dark:text-blue-400 border-blue-500/30",
86 %   high: "bg-orange-500/20 text-orange-700 dark:text-orange-400 border-orange-500/30",
87 %   urgent: "bg-red-500/20 text-red-700 dark:text-red-400 border-red-500/30",
88 % };
89 %
90 % const priorityLabels: Record<string, string> = {
```

client/src/pages/maintenance.tsx (cont.)

```
91 %   low: "Baja",
92 %   medium: "Media",
93 %   high: "Alta",
94 %   urgent: "Urgente",
95 % };
96 %
97 % const statusColors: Record<string, string> = {
98 %   pending: "bg-yellow-500/20 text-yellow-700 dark:text-yellow-400 border-yellow-500/30",
99 %   assigned: "bg-blue-500/20 text-blue-700 dark:text-blue-400 border-blue-500/30",
100 %   in_progress: "bg-purple-500/20 text-purple-700 dark:text-purple-400 border-purple-500/30",
101 %   completed: "bg-green-500/20 text-green-700 dark:text-green-400 border-green-500/30",
102 %   cancelled: "bg-gray-500/20 text-gray-700 dark:text-gray-400 border-gray-500/30",
103 % };
104 %
105 % const statusLabels: Record<string, string> = {
106 %   pending: "Pendiente",
107 %   assigned: "Asignada",
108 %   in_progress: "En Progreso",
109 %   completed: "Completada",
110 %   cancelled: "Cancelada",
111 % };
112 %
113 % const categoryLabels: Record<string, string> = {
114 %   plumbing: "Plomeria",
115 %   electrical: "Electricidad",
116 %   hvac: "Climatizacion",
117 %   furniture: "Mobiliario",
118 %   cleaning: "Limpieza",
119 %   appliances: "Electrodomesticos",
120 %   structure: "Estructura",
121 %   general: "General",
122 % };
123 %
124 % const workOrderFormSchema = z.object({
125 %   title: z.string().min(1, "El titulo es requerido"),
126 %   description: z.string().optional(),
127 %   roomId: z.string().optional(),
128 %   location: z.string().optional(),
129 %   category: z.enum(["plumbing", "electrical", "hvac", "furniture", "cleaning", "appliances", "structure", "general"]),
130 %   priority: z.enum(["low", "medium", "high", "urgent"]),
131 %   assignedToId: z.string().optional(),
132 %   scheduledDate: z.string().optional(),
133 %   estimatedCost: z.string().optional(),
134 %   notes: z.string().optional(),
135 % });
136 %
137 % type WorkOrderFormValues = z.infer<typeof workOrderFormSchema>;
138 %
139 % const staffFormSchema = z.object({
140 %   name: z.string().min(1, "El nombre es requerido"),
141 %   phone: z.string().optional(),
142 %   email: z.string().email("Email invalido").optional().or(z.literal("")),
143 %   specialty: z.string().optional(),
144 % });
145 %
146 % type StaffFormValues = z.infer<typeof staffFormSchema>;
147 %
148 % export default function MaintenancePage() {
149 %   const [selectedOrder, setSelectedOrder] = useState<WorkOrder | null>(null);
150 %   const [isNewOrderDialogOpen, setIsNewOrderDialogOpen] = useState(false);
151 %   const [isNewStaffDialogOpen, setIsNewStaffDialogOpen] = useState(false);
152 %   const [editingStaff, setEditingStaff] = useState<MaintenanceStaff | null>(null);
153 %   const [statusFilter, setStatusFilter] = useState<string>("active");
154 %   const { toast } = useToast();
155 %
156 %   const { data: dashboardStats, isLoading: isLoadingStats } = useQuery<DashboardStats>({
157 %     queryKey: ["/api/maintenance/dashboard"],
158 %   });
159 %
160 %   const { data: workOrders = [], isLoading: isLoadingOrders } = useQuery<WorkOrder[]>({
161 %     queryKey: ["/api/maintenance/work-orders"],
162 %   });
163 %
164 %   const { data: staff = [], isLoading: isLoadingStaff } = useQuery<MaintenanceStaff[]>({
165 %     queryKey: ["/api/maintenance/staff"],
166 %   });
167 %
168 %   const { data: rooms = [] } = useQuery<Room[]>({
169 %     queryKey: ["/api/rooms"],
170 %   });
171 %
172 %   const orderForm = useForm<WorkOrderFormValues>({
173 %     resolver: zodResolver(workOrderFormSchema),
174 %     defaultValues: {
175 %       title: "",
176 %       description: "",
177 %       roomId: "",
178 %       location: "",
179 %       category: "general",
180 %       priority: "medium",
181 %       assignedToId: "",
182 %       scheduledDate: "",
183 %       estimatedCost: "",
```

client/src/pages/maintenance.tsx (cont.)

```
184 %     notes: "",
185 %   },
186 % });
187 %
188 % const staffForm = useForm<StaffFormValues>({
189 %   resolver: zodResolver(staffFormSchema),
190 %   defaultValues: {
191 %     name: "",
192 %     phone: "",
193 %     email: "",
194 %     specialty: "",
195 %   },
196 % });
197 %
198 % const createOrderMutation = useMutation({
199 %   mutationFn: async (data: WorkOrderFormValues) => {
200 %     return apiRequest("POST", "/api/maintenance/work-orders", {
201 %       ...data,
202 %       roomId: data.roomId && data.roomId !== "none" ? data.roomId : null,
203 %       assignedToId: data.assignedToId && data.assignedToId !== "none" ? data.assignedToId : null,
204 %       status: data.assignedToId && data.assignedToId !== "none" ? "assigned" : "pending",
205 %     });
206 %   },
207 %   onSuccess: () => {
208 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/work-orders"] });
209 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/dashboard"] });
210 %     setIsNewOrderDialogOpen(false);
211 %     orderForm.reset();
212 %     toast({ title: "Orden de trabajo creada", description: "La orden fue creada exitosamente" });
213 %   },
214 %   onError: () => {
215 %     toast({ title: "Error", description: "No se pudo crear la orden de trabajo", variant: "destructive" });
216 %   },
217 % });
218 %
219 % const updateOrderMutation = useMutation({
220 %   mutationFn: async ({ id, data }: { id: string; data: Partial<WorkOrder> }) => {
221 %     return apiRequest("PATCH", `/api/maintenance/work-orders/${id}`, data);
222 %   },
223 %   onSuccess: () => {
224 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/work-orders"] });
225 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/dashboard"] });
226 %     setSelectedOrder(null);
227 %     toast({ title: "Orden actualizada", description: "Los cambios fueron guardados" });
228 %   },
229 %   onError: () => {
230 %     toast({ title: "Error", description: "No se pudo actualizar la orden", variant: "destructive" });
231 %   },
232 % });
233 %
234 % const deleteOrderMutation = useMutation({
235 %   mutationFn: async (id: string) => {
236 %     return apiRequest("DELETE", `/api/maintenance/work-orders/${id}`);
237 %   },
238 %   onSuccess: () => {
239 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/work-orders"] });
240 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/dashboard"] });
241 %     setSelectedOrder(null);
242 %     toast({ title: "Orden eliminada" });
243 %   },
244 %   onError: () => {
245 %     toast({ title: "Error", description: "No se pudo eliminar la orden", variant: "destructive" });
246 %   },
247 % });
248 %
249 % const createStaffMutation = useMutation({
250 %   mutationFn: async (data: StaffFormValues) => {
251 %     return apiRequest("POST", "/api/maintenance/staff", data);
252 %   },
253 %   onSuccess: () => {
254 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/staff"] });
255 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/dashboard"] });
256 %     setIsNewStaffDialogOpen(false);
257 %     staffForm.reset();
258 %     toast({ title: "Personal agregado", description: "El tecnico fue agregado exitosamente" });
259 %   },
260 %   onError: () => {
261 %     toast({ title: "Error", description: "No se pudo agregar el personal", variant: "destructive" });
262 %   },
263 % });
264 %
265 % const updateStaffMutation = useMutation({
266 %   mutationFn: async ({ id, data }: { id: string; data: Partial<MaintenanceStaff> }) => {
267 %     return apiRequest("PATCH", `/api/maintenance/staff/${id}`, data);
268 %   },
269 %   onSuccess: () => {
270 %     queryClient.invalidateQueries({ queryKey: ["/api/maintenance/staff"] });
271 %     setEditingStaff(null);
272 %     staffForm.reset();
273 %     toast({ title: "Personal actualizado" });
274 %   },
275 %   onError: () => {
276 %     toast({ title: "Error", description: "No se pudo actualizar el personal", variant: "destructive" });
```

client/src/pages/maintenance.tsx (cont.)

```
277 %     },
278 %   });
279 %
280 %   const deleteStaffMutation = useMutation({
281 %     mutationFn: async (id: string) => {
282 %       return apiRequest("DELETE", `/api/maintenance/staff/${id}`);
283 %     },
284 %     onSuccess: () => {
285 %       queryClient.invalidateQueries({ queryKey: ["/api/maintenance/staff"] });
286 %       queryClient.invalidateQueries({ queryKey: ["/api/maintenance/dashboard"] });
287 %       toast({ title: "Personal eliminado" });
288 %     },
289 %     onError: () => {
290 %       toast({ title: "Error", description: "No se pudo eliminar el personal", variant: "destructive" });
291 %     },
292 %   });
293 %
294 %   const filteredOrders = workOrders.filter((order) => {
295 %     if (statusFilter === "active") {
296 %       return order.status !== "completed" && order.status !== "cancelled";
297 %     }
298 %     if (statusFilter === "completed") {
299 %       return order.status === "completed";
300 %     }
301 %     return true;
302 %   });
303 %
304 %   const sortedOrders = [...filteredOrders].sort((a, b) => {
305 %     const priorityOrder = { urgent: 0, high: 1, medium: 2, low: 3 };
306 %     return (priorityOrder[a.priority] || 2) - (priorityOrder[b.priority] || 2);
307 %   });
308 %
309 %   const handleStatusChange = (order: WorkOrder, newStatus: string) => {
310 %     updateOrderMutation.mutate({
311 %       id: order.id,
312 %       data: { status: newStatus as WorkOrder["status"] },
313 %     });
314 %   };
315 %
316 %   const handleAssign = (order: WorkOrder, staffId: string) => {
317 %     updateOrderMutation.mutate({
318 %       id: order.id,
319 %       data: {
320 %         assignedToId: staffId,
321 %         status: "assigned",
322 %       },
323 %     });
324 %   };
325 %
326 %   const openEditStaff = (member: MaintenanceStaff) => {
327 %     setEditingStaff(member);
328 %     staffForm.reset({
329 %       name: member.name,
330 %       phone: member.phone || "",
331 %       email: member.email || "",
332 %       specialty: member.specialty || "",
333 %     });
334 %   };
335 %
336 %   if (isLoadingStats || isLoadingOrders || isLoadingStaff) {
337 %     return (
338 %       <div className="flex items-center justify-center h-full">
339 %         <Loader2 className="h-8 w-8 animate-spin text-muted-foreground" />
340 %       </div>
341 %     );
342 %   }
343 %
344 %   return (
345 %     <div className="p-6 space-y-6">
346 %       <div className="flex flex-wrap items-center justify-between gap-4">
347 %         <div>
348 %           <h1 className="text-2xl font-bold" data-testid="text-page-title">Mantenimiento</h1>
349 %           <p className="text-muted-foreground">Gestione ordenes de trabajo y personal de mantenimiento</p>
350 %         </div>
351 %         <Button onClick={() => setIsNewOrderDialogOpen(true)} data-testid="button-new-order">
352 %           <Plus className="mr-2 h-4 w-4" />
353 %           Nueva Orden
354 %         </Button>
355 %       </div>
356 %
357 %       <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-5">
358 %         <Card>
359 %           <CardHeader className="flex flex-row items-center justify-between gap-2 space-y-0 pb-2">
360 %             <CardTitle className="text-sm font-medium">Pendientes</CardTitle>
361 %             <Clock className="h-4 w-4 text-yellow-500" />
362 %           </CardHeader>
363 %           <CardContent>
364 %             <div className="text-2xl font-bold" data-testid="text-pending-count">{dashboardStats?.pending || 0}</div>
365 %           </CardContent>
366 %         </Card>
367 %         <Card>
368 %           <CardHeader className="flex flex-row items-center justify-between gap-2 space-y-0 pb-2">
369 %             <CardTitle className="text-sm font-medium">En Progreso</CardTitle>
```

client/src/pages/maintenance.tsx (cont.)

client/src/pages/maintenance.tsx (cont.)

```
463 %         </Badge>
464 %     </TableCell>
465 %     <TableCell>
466 %         {order.assignedTo ? order.assignedTo.name : (
467 %             <Select onChange={(value) => handleAssign(order, value)}>
468 %                 <SelectTrigger className="w-[140px]" data-testid={`select-assign-${order.id}`}>
469 %                     <SelectValue placeholder="Asignar" />
470 %                 </SelectTrigger>
471 %                 <SelectContent>
472 %                     {staff.filter(s => s.isActive === "true").map((s) => (
473 %                         <SelectItem key={s.id} value={s.id}>{s.name}</SelectItem>
474 %                     ))}
475 %                 </SelectContent>
476 %             </Select>
477 %         )}
478 %     </TableCell>
479 %     <TableCell>
480 %         <div className="flex items-center gap-1">
481 %             <Button
482 %                 size="icon"
483 %                 variant="ghost"
484 %                 onClick={() => setSelectedOrder(order)}
485 %                 data-testid={`button-view-order-${order.id}`}
486 %             >
487 %                 <Edit className="h-4 w-4" />
488 %             </Button>
489 %             {order.status === "assigned" && (
490 %                 <Button
491 %                     size="icon"
492 %                     variant="ghost"
493 %                     onClick={() => handleStatusChange(order, "in_progress")}
494 %                     data-testid={`button-start-order-${order.id}`}
495 %                 >
496 %                     <Play className="h-4 w-4" />
497 %                 </Button>
498 %             )}
499 %             {order.status === "in_progress" && (
500 %                 <Button
501 %                     size="icon"
502 %                     variant="ghost"
503 %                     onClick={() => handleStatusChange(order, "completed")}
504 %                     data-testid={`button-complete-order-${order.id}`}
505 %                 >
506 %                     <CheckCircle className="h-4 w-4" />
507 %                 </Button>
508 %             )}
509 %         </div>
510 %     </TableCell>
511 % </TableRow>
512 % ))
513 % </TableBody>
514 % </Table>
515 % </Card>
516 % </TabsContent>
517 %
518 % <TabsContent value="staff" className="space-y-4">
519 %     <div className="flex justify-end">
520 %         <Button onClick={() => setIsNewStaffDialogOpen(true)} data-testid="button-new-staff">
521 %             <Plus className="mr-2 h-4 w-4" />
522 %             Agregar Tecnico
523 %         </Button>
524 %     </div>
525 %
526 %     <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
527 %         {staff.map((member) => (
528 %             <Card key={member.id} data-testid={`card-staff-${member.id}`}>
529 %                 <CardHeader className="flex flex-row items-start justify-between gap-2 space-y-0">
530 %                     <div>
531 %                         <CardTitle className="text-lg">{member.name}</CardTitle>
532 %                         {member.specialty && (
533 %                             <p className="text-sm text-muted-foreground mt-1">{member.specialty}</p>
534 %                         )}
535 %                     </div>
536 %                     <Badge className={member.isActive === "true" ? "bg-green-500/20 text-green-700" : "bg-gray-500/20 t...}>
537 %                         {member.isActive === "true" ? "Activo" : "Inactivo"}
538 %                     </Badge>
539 %                 </CardHeader>
540 %                 <CardContent className="space-y-2">
541 %                     {member.phone && (
542 %                         <div className="flex items-center gap-2 text-sm text-muted-foreground">
543 %                             <Phone className="h-4 w-4" />
544 %                             {member.phone}
545 %                         </div>
546 %                     )}
547 %                     {member.email && (
548 %                         <div className="flex items-center gap-2 text-sm text-muted-foreground">
549 %                             <Mail className="h-4 w-4" />
550 %                             {member.email}
551 %                         </div>
552 %                     )}
553 %                 </CardContent>
554 %                 <div className="flex items-center gap-2 pt-2">
555 %                     <Button size="sm" variant="outline" onClick={() => openEditStaff(member)} data-testid={`button-ed...>
```

client/src/pages/maintenance.tsx (cont.)

```
556 %         <Edit className="h-4 w-4 mr-1" />
557 %         Editar
558 %       </Button>
559 %       <Button
560 %         size="sm"
561 %         variant="outline"
562 %         onClick={() => {
563 %           if (confirm("Eliminar este tecnico?")) {
564 %             deleteStaffMutation.mutate(member.id);
565 %           }
566 %         }}
567 %         data-testid={`button-delete-staff-${member.id}`}
568 %       >
569 %         <Trash2 className="h-4 w-4 mr-1" />
570 %         Eliminar
571 %       </Button>
572 %     </div>
573 %   </CardContent>
574 % </Card>
575 %   )))
576 % </div>
577 % </TabsContent>
578 % </Tabs>
579 %
580 % <Dialog open={isNewOrderDialogOpen} onOpenChange={setIsNewOrderDialogOpen}>
581 %   <DialogContent className="max-w-lg">
582 %     <DialogHeader>
583 %       <DialogTitle>Nueva Orden de Trabajo</DialogTitle>
584 %       <DialogDescription>Complete los datos para crear una nueva orden de trabajo</DialogDescription>
585 %     </DialogHeader>
586 %     <Form {...orderForm}>
587 %       <form onSubmit={orderForm.handleSubmit((data) => createOrderMutation.mutate(data))} className="space-y-4">
588 %         <FormField
589 %           control={orderForm.control}
590 %           name="title"
591 %           render={({ field }) => (
592 %             <FormItem>
593 %               <FormLabel>Titulo</FormLabel>
594 %               <FormControl>
595 %                 <Input placeholder="Descripcion breve del problema" {...field} data-testid="input-order-title" />
596 %               </FormControl>
597 %               <FormMessage />
598 %             </FormItem>
599 %           )}
600 %         />
601 %         <FormField
602 %           control={orderForm.control}
603 %           name="description"
604 %           render={({ field }) => (
605 %             <FormItem>
606 %               <FormLabel>Descripcion</FormLabel>
607 %               <FormControl>
608 %                 <Textarea placeholder="Detalles adicionales" {...field} data-testid="input-order-description" />
609 %               </FormControl>
610 %               <FormMessage />
611 %             </FormItem>
612 %           )}
613 %         />
614 %         <div className="grid grid-cols-2 gap-4">
615 %           <FormField
616 %             control={orderForm.control}
617 %             name="roomId"
618 %             render={({ field }) => (
619 %               <FormItem>
620 %                 <FormLabel>Habitacion</FormLabel>
621 %                 <Select onValueChange={field.onChange} value={field.value}>
622 %                   <FormControl>
623 %                     <SelectTrigger data-testid="select-order-room">
624 %                       <SelectValue placeholder="Seleccionar" />
625 %                     </SelectTrigger>
626 %                   </FormControl>
627 %                   <SelectContent>
628 %                     <SelectItem value="none">Ninguna</SelectItem>
629 %                     {rooms.map((room) => (
630 %                       <SelectItem key={room.id} value={room.id}>
631 %                         {room.roomNumber} (Piso {room.floor})
632 %                       </SelectItem>
633 %                     ))}
634 %                   </SelectContent>
635 %                 </Select>
636 %                 <FormMessage />
637 %               </FormItem>
638 %             )}
639 %           />
640 %           <FormField
641 %             control={orderForm.control}
642 %             name="location"
643 %             render={({ field }) => (
644 %               <FormItem>
645 %                 <FormLabel>Otra ubicacion</FormLabel>
646 %                 <FormControl>
647 %                   <Input placeholder="Ej: Lobby, Restaurante" {...field} data-testid="input-order-location" />
648 %                 </FormControl>
```

client/src/pages/maintenance.tsx (cont.)

```
649 %         <FormMessage />
650 %     </FormItem>
651 % }}
652 % />
653 % </div>
654 % <div className="grid grid-cols-2 gap-4">
655 %     <FormField
656 %         control={orderForm.control}
657 %         name="category"
658 %         render={({ field }) => (
659 %             <FormItem>
660 %                 <FormLabel>Categoria</FormLabel>
661 %                 <Select onChange={field.onChange} value={field.value}>
662 %                     <FormControl>
663 %                         <SelectTrigger data-testid="select-order-category">
664 %                             <SelectValue placeholder="Seleccionar" />
665 %                         </SelectTrigger>
666 %                     </FormControl>
667 %                     <SelectContent>
668 %                         {Object.entries(categoryLabels).map(([value, label]) => (
669 %                             <SelectItem key={value} value={value}>{label}</SelectItem>
670 %                         ))}
671 %                     </SelectContent>
672 %                 </Select>
673 %                 <FormMessage />
674 %             </FormItem>
675 %         )}
676 %     />
677 %     <FormField
678 %         control={orderForm.control}
679 %         name="priority"
680 %         render={({ field }) => (
681 %             <FormItem>
682 %                 <FormLabel>Prioridad</FormLabel>
683 %                 <Select onChange={field.onChange} value={field.value}>
684 %                     <FormControl>
685 %                         <SelectTrigger data-testid="select-order-priority">
686 %                             <SelectValue placeholder="Seleccionar" />
687 %                         </SelectTrigger>
688 %                     </FormControl>
689 %                     <SelectContent>
690 %                         {Object.entries(priorityLabels).map(([value, label]) => (
691 %                             <SelectItem key={value} value={value}>{label}</SelectItem>
692 %                         ))}
693 %                     </SelectContent>
694 %                 </Select>
695 %                 <FormMessage />
696 %             </FormItem>
697 %         )}
698 %     />
699 % </div>
700 % <FormField
701 %     control={orderForm.control}
702 %     name="assignedToId"
703 %     render={({ field }) => (
704 %         <FormItem>
705 %             <FormLabel>Asignar a</FormLabel>
706 %             <Select onChange={field.onChange} value={field.value}>
707 %                 <FormControl>
708 %                     <SelectTrigger data-testid="select-order-assigned">
709 %                         <SelectValue placeholder="Sin asignar" />
710 %                     </SelectTrigger>
711 %                 </FormControl>
712 %                 <SelectContent>
713 %                     <SelectItem value="none">Sin asignar</SelectItem>
714 %                     {staff.filter(s => s.isActive === "true").map((s) => (
715 %                         <SelectItem key={s.id} value={s.id}>{s.name}</SelectItem>
716 %                     ))}
717 %                 </SelectContent>
718 %             </Select>
719 %             <FormMessage />
720 %         </FormItem>
721 %     )}
722 % />
723 % <FormField
724 %     control={orderForm.control}
725 %     name="notes"
726 %     render={({ field }) => (
727 %         <FormItem>
728 %             <FormLabel>Notas</FormLabel>
729 %             <FormControl>
730 %                 <Textarea placeholder="Notas adicionales" {...field} data-testid="input-order-notes" />
731 %             </FormControl>
732 %             <FormMessage />
733 %         </FormItem>
734 %     )}
735 % />
736 % <DialogFooter>
737 %     <Button type="button" variant="outline" onClick={() => setIsNewOrderDialogOpen(false)}>
738 %         Cancelar
739 %     </Button>
740 %     <Button type="submit" disabled={createOrderMutation.isPending} data-testid="button-submit-order">
741 %         {createOrderMutation.isPending && <Loader2 className="mr-2 h-4 w-4 animate-spin" />
```


client/src/pages/maintenance.tsx (cont.)

```
742 %         Crear Orden
743 %       </Button>
744 %     </DialogFooter>
745 %   </form>
746 % </Form>
747 % </DialogContent>
748 % </Dialog>
749 %
750 % <Dialog open={!selectedOrder} onOpenChange={() => setSelectedOrder(null)}>
751 %   <DialogContent className="max-w-lg">
752 %     <DialogHeader>
753 %       <DialogTitle>Orden {selectedOrder?.orderCode}</DialogTitle>
754 %       <DialogDescription>{selectedOrder?.title}</DialogDescription>
755 %     </DialogHeader>
756 %     {selectedOrder && (
757 %       <div className="space-y-4">
758 %         <div className="grid grid-cols-2 gap-4">
759 %           <div>
760 %             <p className="text-sm text-muted-foreground">Estado</p>
761 %             <Badge className={statusColors[selectedOrder.status]}>
762 %               {statusLabels[selectedOrder.status]}
763 %             </Badge>
764 %           </div>
765 %           <div>
766 %             <p className="text-sm text-muted-foreground">Prioridad</p>
767 %             <Badge className={priorityColors[selectedOrder.priority]}>
768 %               {priorityLabels[selectedOrder.priority]}
769 %             </Badge>
770 %           </div>
771 %         </div>
772 %         <div className="grid grid-cols-2 gap-4">
773 %           <div>
774 %             <p className="text-sm text-muted-foreground">Categoria</p>
775 %             <p className="font-medium">{categoryLabels[selectedOrder.category]}</p>
776 %           </div>
777 %           <div>
778 %             <p className="text-sm text-muted-foreground">Ubicacion</p>
779 %             <p className="font-medium">
780 %               {selectedOrder.room ? `Hab. ${selectedOrder.room.roomNumber}` : selectedOrder.location || "No esp..."}
781 %             </p>
782 %           </div>
783 %         </div>
784 %         {selectedOrder.description && (
785 %           <div>
786 %             <p className="text-sm text-muted-foreground">Descripcion</p>
787 %             <p>{selectedOrder.description}</p>
788 %           </div>
789 %         )}
790 %         <div className="grid grid-cols-2 gap-4">
791 %           <div>
792 %             <p className="text-sm text-muted-foreground">Reportada</p>
793 %             <p className="font-medium">
794 %               {format(parseISO(selectedOrder.reportedAt), "dd/MM/yyyy HH:mm", { locale: es })}
795 %             </p>
796 %           </div>
797 %           <div>
798 %             <p className="text-sm text-muted-foreground">Asignado a</p>
799 %             <p className="font-medium">{selectedOrder.assignedTo?.name || "Sin asignar"}</p>
800 %           </div>
801 %         </div>
802 %         {selectedOrder.completedAt && (
803 %           <div>
804 %             <p className="text-sm text-muted-foreground">Completada</p>
805 %             <p className="font-medium">
806 %               {format(parseISO(selectedOrder.completedAt), "dd/MM/yyyy HH:mm", { locale: es })}
807 %             </p>
808 %           </div>
809 %         )}
810 %         {selectedOrder.notes && (
811 %           <div>
812 %             <p className="text-sm text-muted-foreground">Notas</p>
813 %             <p>{selectedOrder.notes}</p>
814 %           </div>
815 %         )}
816 %       <DialogFooter className="flex flex-wrap gap-2">
817 %         {selectedOrder.status === "pending" && (
818 %           <Button
819 %             variant="outline"
820 %             onClick={() => handleStatusChange(selectedOrder, "cancelled")}
821 %           >
822 %             Cancelar Orden
823 %           </Button>
824 %         )}
825 %         {selectedOrder.status === "assigned" && (
826 %           <Button onClick={() => handleStatusChange(selectedOrder, "in_progress")} data-testid="button-start-...>
827 %             <Play className="mr-2 h-4 w-4" />
828 %             Iniciar Trabajo
829 %           </Button>
830 %         )}
831 %         {selectedOrder.status === "in_progress" && (
832 %           <Button onClick={() => handleStatusChange(selectedOrder, "completed")} data-testid="button-complete-...>
833 %             <CheckCircle className="mr-2 h-4 w-4" />
834 %             Completar
```

client/src/pages/maintenance.tsx (cont.)

```
835 %         </Button>
836 %     })
837 %     <Button
838 %         variant="destructive"
839 %         onClick={() => {
840 %             if (confirm("Eliminar esta orden?")) {
841 %                 deleteOrderMutation.mutate(selectedOrder.id);
842 %             }
843 %         }}
844 %     >
845 %         <Trash2 className="mr-2 h-4 w-4" />
846 %         Eliminar
847 %     </Button>
848 % </DialogFooter>
849 % </div>
850 % })
851 % </DialogContent>
852 % </Dialog>
853 %
854 % <Dialog open={isNewStaffDialogOpen || !!editingStaff} onOpenChange={({open}) => {
855 %     if (!open) {
856 %         setIsNewStaffDialogOpen(false);
857 %         setEditingStaff(null);
858 %         staffForm.reset();
859 %     }
860 % }}>
861 %     <DialogContent className="max-w-md">
862 %         <DialogHeader>
863 %             <DialogTitle>{editingStaff ? "Editar Tecnico" : "Nuevo Tecnico"}</DialogTitle>
864 %             <DialogDescription>Complete los datos del personal de mantenimiento</DialogDescription>
865 %         </DialogHeader>
866 %         <Form {...staffForm}>
867 %             <form onSubmit={staffForm.handleSubmit((data) => {
868 %                 if (editingStaff) {
869 %                     updateStaffMutation.mutate({ id: editingStaff.id, data });
870 %                 } else {
871 %                     createStaffMutation.mutate(data);
872 %                 }
873 %             }} className="space-y-4">
874 %                 <FormField
875 %                     control={staffForm.control}
876 %                     name="name"
877 %                     render={({ field }) => (
878 %                         <FormItem>
879 %                             <FormLabel>Nombre Completo</FormLabel>
880 %                             <FormControl>
881 %                                 <Input placeholder="Nombre y Apellido" {...field} data-testid="input-staff-name" />
882 %                             </FormControl>
883 %                             <FormMessage />
884 %                         </FormItem>
885 %                     )}
886 %                 />
887 %                 <FormField
888 %                     control={staffForm.control}
889 %                     name="specialty"
890 %                     render={({ field }) => (
891 %                         <FormItem>
892 %                             <FormLabel>Especialidad</FormLabel>
893 %                             <FormControl>
894 %                                 <Input placeholder="Ej: Plomeria, Electricidad" {...field} data-testid="input-staff-specialty" />
895 %                             </FormControl>
896 %                             <FormMessage />
897 %                         </FormItem>
898 %                     )}
899 %                 />
900 %                 <FormField
901 %                     control={staffForm.control}
902 %                     name="phone"
903 %                     render={({ field }) => (
904 %                         <FormItem>
905 %                             <FormLabel>Telefono</FormLabel>
906 %                             <FormControl>
907 %                                 <Input placeholder="+54 343 ..." {...field} data-testid="input-staff-phone" />
908 %                             </FormControl>
909 %                             <FormMessage />
910 %                         </FormItem>
911 %                     )}
912 %                 />
913 %                 <FormField
914 %                     control={staffForm.control}
915 %                     name="email"
916 %                     render={({ field }) => (
917 %                         <FormItem>
918 %                             <FormLabel>Email</FormLabel>
919 %                             <FormControl>
920 %                                 <Input type="email" placeholder="email@ejemplo.com" {...field} data-testid="input-staff-email" />
921 %                             </FormControl>
922 %                             <FormMessage />
923 %                         </FormItem>
924 %                     )}
925 %                 />
926 %             <DialogFooter>
927 %                 <Button type="button" variant="outline" onClick={() => {
```

client/src/pages/maintenance.tsx (cont.)

```
928 %         setIsNewStaffDialogOpen(false);
929 %         setEditingStaff(null);
930 %         staffForm.reset();
931 %     }}>
932 %         Cancelar
933 %     </Button>
934 %     <Button type="submit" disabled={createStaffMutation.isPending || updateStaffMutation.isPending} data-...
935 %         {(createStaffMutation.isPending || updateStaffMutation.isPending) && <Loader2 className="mr-2 h-4 w-...
936 %             {editingStaff ? "Guardar" : "Agregar"}
937 %         </Button>
938 %     </DialogFooter>
939 % </form>
940 % </Form>
941 % </DialogContent>
942 % </Dialog>
943 % </div>
944 % );
945 % }
946 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
5 % import { Button } from "@components/ui/button";
6 % import { Badge } from "@components/ui/badge";
7 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
8 % import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@components/ui/dialog";
9 % import { Input } from "@components/ui/input";
10 % import { Label } from "@components/ui/label";
11 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
12 % import { Textarea } from "@components/ui/textarea";
13 % import { useForm } from "react-hook-form";
14 % import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
15 % import { useToast } from "@hooks/use-toast";
16 % import {
17 %   Plus,
18 %   Package,
19 %   AlertTriangle,
20 %   ArrowDownCircle,
21 %   ArrowUpCircle,
22 %   Building2,
23 %   Loader2,
24 %   Search,
25 %   TrendingDown,
26 %   TrendingUp,
27 %   History,
28 % } from "lucide-react";
29 %
30 % type ItemCategory = {
31 %   id: string;
32 %   name: string;
33 %   description: string | null;
34 %   parentId: string | null;
35 %   area: string;
36 %   isActive: string | null;
37 % };
38 %
39 % type Supplier = {
40 %   id: string;
41 %   name: string;
42 %   contactName: string | null;
43 %   phone: string | null;
44 %   email: string | null;
45 %   address: string | null;
46 %   cuit: string | null;
47 %   paymentTermDays: number | null;
48 %   notes: string | null;
49 %   isActive: string | null;
50 % };
51 %
52 % type InventoryItem = {
53 %   id: string;
54 %   sku: string | null;
55 %   name: string;
56 %   description: string | null;
57 %   categoryId: string | null;
58 %   supplierId: string | null;
59 %   unit: "unidad" | "kg" | "g" | "litro" | "ml" | "caja" | "paquete" | "docena";
60 %   costPrice: string;
61 %   minStock: number;
62 %   maxStock: number | null;
63 %   currentStock: number;
64 %   location: string | null;
65 %   isActive: string | null;
66 %   category?: ItemCategory;
67 %   supplier?: Supplier;
68 % };
69 %
70 % type StockMovement = {
71 %   id: string;
72 %   itemId: string;
73 %   movementType: "entrada" | "salida" | "ajuste" | "transferencia" | "consumo";
74 %   quantity: number;
75 %   previousStock: number;
76 %   newStock: number;
77 %   unitCost: string | null;
78 %   reference: string | null;
79 %   notes: string | null;
80 %   createdAt: string;
81 %   createdBy: string | null;
82 %   item?: InventoryItem;
83 % };
84 %
85 % const unitLabels: Record<string, string> = {
86 %   unidad: "Unidades",
87 %   kg: "Kilogramos",
88 %   g: "Gramos",
89 %   litro: "Litros",
90 %   ml: "Mililitros",
```

client/src/pages/inventory.tsx (cont.)

```
91 %   caja: "Cajas",
92 %   paquete: "Paquetes",
93 %   docena: "Docenas",
94 % };
95 %
96 % const movementTypeLabels: Record<string, string> = {
97 %   entrada: "Entrada",
98 %   salida: "Salida",
99 %   ajuste: "Ajuste",
100 %   transferencia: "Transferencia",
101 %   consumo: "Consumo",
102 % };
103 %
104 % export default function InventoryPage() {
105 %   const { toast } = useToast();
106 %   const [activeTab, setActiveTab] = useState("items");
107 %   const [searchQuery, setSearchQuery] = useState("");
108 %   const [isNewItemDialogOpen, setIsNewItemDialogOpen] = useState(false);
109 %   const [isMovementDialogOpen, setIsMovementDialogOpen] = useState(false);
110 %   const [selectedItem, setSelectedItem] = useState<InventoryItem | null>(null);
111 %   const [movementType, setMovementType] = useState<"entrada" | "salida">("entrada");
112 %   const [areaFilter, setAreaFilter] = useState("all");
113 %
114 %   const { data: categories = [] } = useQuery<ItemCategory[]>({
115 %     queryKey: ["/api/inventory/categories"],
116 %   });
117 %
118 %   const { data: suppliers = [] } = useQuery<Supplier[]>({
119 %     queryKey: ["/api/inventory/suppliers"],
120 %   });
121 %
122 %   const { data: items = [], isLoading: itemsLoading } = useQuery<InventoryItem[]>({
123 %     queryKey: ["/api/inventory/items"],
124 %   });
125 %
126 %   const { data: lowStockItems = [] } = useQuery<InventoryItem[]>({
127 %     queryKey: ["/api/inventory/items/low-stock"],
128 %   });
129 %
130 %   const { data: movements = [] } = useQuery<StockMovement[]>({
131 %     queryKey: ["/api/inventory/movements"],
132 %   });
133 %
134 %   const createItemMutation = useMutation({
135 %     mutationFn: async (data: Partial<InventoryItem>) => {
136 %       const res = await apiRequest("POST", "/api/inventory/items", data);
137 %       return res.json();
138 %     },
139 %     onSuccess: () => {
140 %       queryClient.invalidateQueries({ queryKey: ["/api/inventory/items"] });
141 %       setIsNewItemDialogOpen(false);
142 %       toast({ title: "Artículo creado" });
143 %     },
144 %   });
145 %
146 %   const createMovementMutation = useMutation({
147 %     mutationFn: async (data: { itemId: string; movementType: string; quantity: number; notes?: string }) => {
148 %       const res = await apiRequest("POST", "/api/inventory/movements", data);
149 %       return res.json();
150 %     },
151 %     onSuccess: () => {
152 %       queryClient.invalidateQueries({ queryKey: ["/api/inventory/items"] });
153 %       queryClient.invalidateQueries({ queryKey: ["/api/inventory/movements"] });
154 %       queryClient.invalidateQueries({ queryKey: ["/api/inventory/items/low-stock"] });
155 %       setIsMovementDialogOpen(false);
156 %       setSelectedItem(null);
157 %       toast({ title: "Movimiento registrado" });
158 %     },
159 %     onError: (error: any) => {
160 %       toast({
161 %         title: "Error",
162 %         description: error.message || "No se pudo registrar el movimiento",
163 %         variant: "destructive",
164 %       });
165 %     },
166 %   });
167 %
168 %   const filteredItems = items.filter((item) => {
169 %     const matchesSearch = item.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
170 %       item.sku?.toLowerCase().includes(searchQuery.toLowerCase());
171 %     const matchesArea = areaFilter === "all" || (item.category as any)?.area === areaFilter;
172 %     return matchesSearch && matchesArea;
173 %   });
174 %
175 %   const totalValue = items.reduce(
176 %     (sum, item) => sum + (item.currentStock * parseFloat(item.costPrice || "0")),
177 %     0
178 %   );
179 %
180 %   const handleMovement = (item: InventoryItem, type: "entrada" | "salida") => {
181 %     setSelectedItem(item);
182 %     setMovementType(type);
183 %     setIsMovementDialogOpen(true);
```

```

184 %   };
185 %
186 %   const MovementForm = () => {
187 %     const [quantity, setQuantity] = useState(1);
188 %     const [notes, setNotes] = useState("");
189 %
190 %     return (
191 %       <div className="space-y-4">
192 %         <div className="p-3 bg-muted rounded-md">
193 %           <div className="font-medium">{selectedItem?.name}</div>
194 %           <div className="text-sm text-muted-foreground">
195 %             Stock actual: {selectedItem?.currentStock} {selectedItem?.unit}
196 %           </div>
197 %         </div>
198 %         <div className="space-y-2">
199 %           <Label>Cantidad</Label>
200 %           <Input
201 %             type="number"
202 %             min={1}
203 %             value={quantity}
204 %             onChange={(e) => setQuantity(parseInt(e.target.value) || 1)}
205 %             data-testid="input-movement-quantity"
206 %           />
207 %         </div>
208 %         <div className="space-y-2">
209 %           <Label>Notas (opcional)</Label>
210 %           <Textarea
211 %             value={notes}
212 %             onChange={(e) => setNotes(e.target.value)}
213 %             placeholder="Ej: Proveedor XYZ, Factura #123"
214 %             data-testid="input-movement-notes"
215 %           />
216 %         </div>
217 %         <DialogFooter>
218 %           <Button variant="outline" onClick={() => setIsMovementDialogOpen(false)}>
219 %             Cancelar
220 %           </Button>
221 %           <Button
222 %             onClick={() => {
223 %               if (selectedItem) {
224 %                 createMovementMutation.mutate({
225 %                   itemId: selectedItem.id,
226 %                   movementType,
227 %                   quantity,
228 %                   notes: notes || undefined,
229 %                 });
230 %               }
231 %             }}
232 %             disabled={createMovementMutation.isPending}
233 %             data-testid="button-confirm-movement"
234 %           >
235 %             {createMovementMutation.isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
236 %             Confirmar {movementType === "entrada" ? "Entrada" : "Salida"}
237 %           </Button>
238 %         </DialogFooter>
239 %       </div>
240 %     );
241 %   };
242 %
243 %   if (itemsLoading) {
244 %     return (
245 %       <div className="flex items-center justify-center h-full">
246 %         <Loader2 className="h-8 w-8 animate-spin text-muted-foreground" />
247 %       </div>
248 %     );
249 %   }
250 %
251 %   return (
252 %     <div className="p-6 space-y-6">
253 %       <div className="flex items-center justify-between gap-4 flex-wrap">
254 %         <div>
255 %           <h1 className="text-2xl font-bold" data-testid="text-page-title">Inventario</h1>
256 %           <p className="text-muted-foreground">Gestiona stock, proveedores y movimientos</p>
257 %         </div>
258 %         <Button onClick={() => setIsNewItemDialogOpen(true)} data-testid="button-add-item">
259 %           <Plus className="h-4 w-4 mr-2" />
260 %           Nuevo Artículo
261 %         </Button>
262 %       </div>
263 %
264 %       <div className="grid gap-4 md:grid-cols-4">
265 %         <Card>
266 %           <CardHeader className="flex flex-row items-center justify-between gap-4 pb-2">
267 %             <CardTitle className="text-sm font-medium">Total Artículos</CardTitle>
268 %             <Package className="h-4 w-4 text-muted-foreground" />
269 %           </CardHeader>
270 %           <CardContent>
271 %             <div className="text-2xl font-bold">{items.length}</div>
272 %           </CardContent>
273 %         </Card>
274 %         <Card>
275 %           <CardHeader className="flex flex-row items-center justify-between gap-4 pb-2">
276 %             <CardTitle className="text-sm font-medium">Stock Bajo</CardTitle>

```

client/src/pages/inventory.tsx (cont.)

```
277 %         <AlertTriangle className="h-4 w-4 text-yellow-500" />
278 %       </CardHeader>
279 %       <CardContent>
280 %         <div className="text-2xl font-bold text-yellow-600">{lowStockItems.length}</div>
281 %       </CardContent>
282 %     </Card>
283 %     <Card>
284 %       <CardHeader className="flex flex-row items-center justify-between gap-4 pb-2">
285 %         <CardTitle className="text-sm font-medium">Proveedores</CardTitle>
286 %         <Building2 className="h-4 w-4 text-muted-foreground" />
287 %       </CardHeader>
288 %       <CardContent>
289 %         <div className="text-2xl font-bold">{suppliers.length}</div>
290 %       </CardContent>
291 %     </Card>
292 %     <Card>
293 %       <CardHeader className="flex flex-row items-center justify-between gap-4 pb-2">
294 %         <CardTitle className="text-sm font-medium">Valor Total</CardTitle>
295 %         <Package className="h-4 w-4 text-muted-foreground" />
296 %       </CardHeader>
297 %       <CardContent>
298 %         <div className="text-2xl font-bold">
299 %           ${totalValue.toLocaleString("es-AR", { minimumFractionDigits: 2 })}
300 %         </div>
301 %       </CardContent>
302 %     </Card>
303 %   </div>
304 %
305 %   <Tabs value={activeTab} onValueChange={setActiveTab}>
306 %     <TabsList>
307 %       <TabsTrigger value="items" data-testid="tab-items">
308 %         <Package className="h-4 w-4 mr-2" />
309 %         Artículos
310 %       </TabsTrigger>
311 %       <TabsTrigger value="low-stock" data-testid="tab-low-stock">
312 %         <AlertTriangle className="h-4 w-4 mr-2" />
313 %         Stock Bajo
314 %       </TabsTrigger>
315 %       <TabsTrigger value="movements" data-testid="tab-movements">
316 %         <History className="h-4 w-4 mr-2" />
317 %         Movimientos
318 %       </TabsTrigger>
319 %       <TabsTrigger value="suppliers" data-testid="tab-suppliers">
320 %         <Building2 className="h-4 w-4 mr-2" />
321 %         Proveedores
322 %       </TabsTrigger>
323 %     </TabsList>
324 %
325 %     <TabsContent value="items" className="space-y-4">
326 %       <div className="flex items-center gap-4">
327 %         <div className="relative flex-1 max-w-sm">
328 %           <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
329 %           <Input
330 %             placeholder="Buscar artículos..."
331 %             value={searchQuery}
332 %             onChange={(e) => setSearchQuery(e.target.value)}
333 %             className="pl-10"
334 %             data-testid="input-search"
335 %           />
336 %         </div>
337 %         <Select value={areaFilter} onValueChange={setAreaFilter}>
338 %           <SelectTrigger className="w-[180px]" data-testid="select-area-filter">
339 %             <SelectValue placeholder="Todas las áreas" />
340 %           </SelectTrigger>
341 %           <SelectContent>
342 %             <SelectItem value="all">Todas las áreas</SelectItem>
343 %             <SelectItem value="general">General</SelectItem>
344 %             <SelectItem value="spa">SPA</SelectItem>
345 %             <SelectItem value="restaurant">Restaurante</SelectItem>
346 %             <SelectItem value="housekeeping">Housekeeping</SelectItem>
347 %             <SelectItem value="maintenance">Mantenimiento</SelectItem>
348 %             <SelectItem value="admin">Administración</SelectItem>
349 %           </SelectContent>
350 %         </Select>
351 %       </div>
352 %
353 %       {filteredItems.length === 0 ? (
354 %         <Card>
355 %           <CardContent className="flex flex-col items-center justify-center py-12 text-center">
356 %             <Package className="h-12 w-12 text-muted-foreground mb-4" />
357 %             <h3 className="text-lg font-semibold mb-2">Sin artículos</h3>
358 %             <p className="text-muted-foreground mb-4">Agrega artículos al inventario</p>
359 %             <Button onClick={() => setIsNewItemDialogOpen(true)}>
360 %               <Plus className="h-4 w-4 mr-2" />
361 %               Agregar Artículo
362 %             </Button>
363 %           </CardContent>
364 %         </Card>
365 %       ) : (
366 %         <div className="border rounded-md overflow-hidden">
367 %           <table className="w-full">
368 %             <thead className="bg-muted/50">
369 %               <tr className="text-left">
```

```

370 %         <th className="p-3 font-medium">Articulo</th>
371 %         <th className="p-3 font-medium">Categoria</th>
372 %         <th className="p-3 font-medium text-right">Stock</th>
373 %         <th className="p-3 font-medium text-right">Min</th>
374 %         <th className="p-3 font-medium text-right">Costo</th>
375 %         <th className="p-3 font-medium text-center">Acciones</th>
376 %     </tr>
377 % </thead>
378 % <tbody>
379 %     {filteredItems.map((item) => (
380 %         <tr key={item.id} className="border-t" data-testid={`row-item-${item.id}`}>
381 %             <td className="p-3">
382 %                 <div className="font-medium">{item.name}</div>
383 %                 {item.sku && (
384 %                     <div className="text-xs text-muted-foreground">SKU: {item.sku}</div>
385 %                 )}
386 %             </td>
387 %             <td className="p-3">
388 %                 <div>{item.category?.name || "-"}</div>
389 %                 {item.category?.area && item.category.area !== "general" && (
390 %                     <Badge variant="secondary" className="text-[10px] mt-0.5">{(item.category as any).area.toUp...
391 %                 )}
392 %             </td>
393 %             <td className="p-3 text-right">
394 %                 <span className={item.currentStock < item.minStock ? "text-red-600 font-semibold" : ""}>
395 %                     {item.currentStock}
396 %                 </span>
397 %                 <span className="text-muted-foreground text-xs ml-1">{item.unit}</span>
398 %             </td>
399 %             <td className="p-3 text-right text-muted-foreground">
400 %                 {item.minStock}
401 %             </td>
402 %             <td className="p-3 text-right">
403 %                 ${parseFloat(item.costPrice).toLocaleString("es-AR", { minimumFractionDigits: 2 })}
404 %             </td>
405 %             <td className="p-3">
406 %                 <div className="flex items-center justify-center gap-1">
407 %                     <Button
408 %                         size="icon"
409 %                         variant="ghost"
410 %                         onClick={() => handleMovement(item, "entrada")}
411 %                         title="Registrar entrada"
412 %                         data-testid={`button-entry-${item.id}`}
413 %                     >
414 %                         <ArrowDownCircle className="h-4 w-4 text-green-600" />
415 %                     </Button>
416 %                     <Button
417 %                         size="icon"
418 %                         variant="ghost"
419 %                         onClick={() => handleMovement(item, "salida")}
420 %                         title="Registrar salida"
421 %                         data-testid={`button-exit-${item.id}`}
422 %                     >
423 %                         <ArrowUpCircle className="h-4 w-4 text-red-600" />
424 %                     </Button>
425 %                 </div>
426 %             </td>
427 %         </tr>
428 %     )})}
429 % </tbody>
430 % </table>
431 % </div>
432 % )}
433 % </TabsContent>
434 %
435 % <TabsContent value="low-stock" className="space-y-4">
436 %     {lowStockItems.length === 0 ? (
437 %         <Card>
438 %             <CardContent className="flex flex-col items-center justify-center py-12 text-center">
439 %                 <TrendingUp className="h-12 w-12 text-green-500 mb-4" />
440 %                 <h3 className="text-lg font-semibold mb-2">Stock OK</h3>
441 %                 <p className="text-muted-foreground">Todos los articulos tienen stock suficiente</p>
442 %             </CardContent>
443 %         </Card>
444 %     ) : (
445 %         <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
446 %             {lowStockItems.map((item) => (
447 %                 <Card key={item.id} className="border-yellow-500/50" data-testid={`low-stock-${item.id}`}>
448 %                     <CardHeader className="pb-2">
449 %                         <div className="flex items-start justify-between gap-2">
450 %                             <CardTitle className="text-base">{item.name}</CardTitle>
451 %                             <Badge variant="destructive">
452 %                                 <TrendingDown className="h-3 w-3 mr-1" />
453 %                                 Bajo
454 %                             </Badge>
455 %                         </div>
456 %                     </CardHeader>
457 %                     <CardContent className="space-y-3">
458 %                         <div className="flex justify-between text-sm">
459 %                             <span className="text-muted-foreground">Stock actual:</span>
460 %                             <span className="font-semibold text-red-600">{item.currentStock} {item.unit}</span>
461 %                         </div>
462 %                         <div className="flex justify-between text-sm">

```


client/src/pages/inventory.tsx (cont.)

```
463 %         <span className="text-muted-foreground">Stock minimo:</span>
464 %         <span>{item.minStock} {item.unit}</span>
465 %     </div>
466 %     <div className="flex justify-between text-sm">
467 %         <span className="text-muted-foreground">Faltante:</span>
468 %         <span className="font-semibold">{item.minStock - item.currentStock} {item.unit}</span>
469 %     </div>
470 %     <Button
471 %       size="sm"
472 %       className="w-full"
473 %       onClick={() => handleMovement(item, "entrada")}
474 %       data-testid={`button-restock-${item.id}`}
475 %     >
476 %       <ArrowDownCircle className="h-4 w-4 mr-2" />
477 %       Registrar Entrada
478 %     </Button>
479 %   </CardContent>
480 % </Card>
481 %   )}
482 % </div>
483 % }}
484 % </TabsContent>
485 %
486 % <TabsContent value="movements" className="space-y-4">
487 %   {movements.length === 0 ? (
488 %     <Card>
489 %       <CardContent className="flex flex-col items-center justify-center py-12 text-center">
490 %         <History className="h-12 w-12 text-muted-foreground mb-4" />
491 %         <h3 className="text-lg font-semibold mb-2">Sin movimientos</h3>
492 %         <p className="text-muted-foreground">Los movimientos de stock aparecerán aquí</p>
493 %       </CardContent>
494 %     </Card>
495 %   ) : (
496 %     <div className="border rounded-md overflow-hidden">
497 %       <table className="w-full">
498 %         <thead className="bg-muted/50">
499 %           <tr className="text-left">
500 %             <th className="p-3 font-medium">Fecha</th>
501 %             <th className="p-3 font-medium">Articulo</th>
502 %             <th className="p-3 font-medium">Tipo</th>
503 %             <th className="p-3 font-medium text-right">Cantidad</th>
504 %             <th className="p-3 font-medium text-right">Stock Anterior</th>
505 %             <th className="p-3 font-medium text-right">Stock Nuevo</th>
506 %             <th className="p-3 font-medium">Notas</th>
507 %           </tr>
508 %         </thead>
509 %         <tbody>
510 %           {movements.slice(0, 50).map((movement) => (
511 %             <tr key={movement.id} className="border-t" data-testid={`movement-${movement.id}`}>
512 %               <td className="p-3 text-sm">
513 %                 {new Date(movement.createdAt).toLocaleString("es-AR", {
514 %                   day: "2-digit",
515 %                   month: "2-digit",
516 %                   hour: "2-digit",
517 %                   minute: "2-digit",
518 %                 })}
519 %               </td>
520 %               <td className="p-3">
521 %                 {movement.item?.name || "N/A"}
522 %               </td>
523 %               <td className="p-3">
524 %                 <Badge
525 %                   variant={movement.movementType === "entrada" ? "default" : "secondary"}
526 %                 >
527 %                   {movementTypeLabels[movement.movementType]}
528 %                 </Badge>
529 %               </td>
530 %               <td className="p-3 text-right font-mono">
531 %                 <span className={movement.movementType === "entrada" ? "text-green-600" : "text-red-600"}>
532 %                   {movement.movementType === "entrada" ? "+" : "-"}{movement.quantity}
533 %                 </span>
534 %               </td>
535 %               <td className="p-3 text-right font-mono text-muted-foreground">
536 %                 {movement.previousStock}
537 %               </td>
538 %               <td className="p-3 text-right font-mono">
539 %                 {movement.newStock}
540 %               </td>
541 %               <td className="p-3 text-sm text-muted-foreground truncate max-w-48">
542 %                 {movement.notes || "-"}
543 %               </td>
544 %             </tr>
545 %           )}
546 %         </tbody>
547 %       </table>
548 %     </div>
549 %   )}
550 % </TabsContent>
551 %
552 % <TabsContent value="suppliers" className="space-y-4">
553 %   <div className="flex items-center justify-end">
554 %     <Button data-testid="button-add-supplier">
555 %       <Plus className="h-4 w-4 mr-2" />
```

client/src/pages/inventory.tsx (cont.)

```
556 %         Nuevo Proveedor
557 %       </Button>
558 %     </div>
559 %     {suppliers.length === 0 ? (
560 %       <Card>
561 %         <CardContent className="flex flex-col items-center justify-center py-12 text-center">
562 %           <Building2 className="h-12 w-12 text-muted-foreground mb-4" />
563 %           <h3 className="text-lg font-semibold mb-2">Sin proveedores</h3>
564 %           <p className="text-muted-foreground mb-4">Agrega proveedores para gestionar compras</p>
565 %         </CardContent>
566 %       </Card>
567 %     ) : (
568 %       <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
569 %         {suppliers.map((supplier) => (
570 %           <Card key={supplier.id} data-testid={`supplier-${supplier.id}`}>
571 %             <CardHeader>
572 %               <CardTitle className="text-base">{supplier.name}</CardTitle>
573 %             </CardHeader>
574 %             <CardContent className="space-y-2 text-sm">
575 %               {supplier.contactName && (
576 %                 <div className="flex justify-between">
577 %                   <span className="text-muted-foreground">Contacto:</span>
578 %                   <span>{supplier.contactName}</span>
579 %                 </div>
580 %               )}
581 %               {supplier.phone && (
582 %                 <div className="flex justify-between">
583 %                   <span className="text-muted-foreground">Telefono:</span>
584 %                   <span>{supplier.phone}</span>
585 %                 </div>
586 %               )}
587 %               {supplier.email && (
588 %                 <div className="flex justify-between">
589 %                   <span className="text-muted-foreground">Email:</span>
590 %                   <span className="truncate ml-2">{supplier.email}</span>
591 %                 </div>
592 %               )}
593 %               {supplier.cuit && (
594 %                 <div className="flex justify-between">
595 %                   <span className="text-muted-foreground">CUIT:</span>
596 %                   <span>{supplier.cuit}</span>
597 %                 </div>
598 %               )}
599 %             </CardContent>
600 %           </Card>
601 %         )]}
602 %       </div>
603 %     )}
604 %   </TabsContent>
605 % </Tabs>
606 %
607 %   <Dialog open={isNewItemDialogOpen} onOpenChange={setIsNewItemDialogOpen}>
608 %     <DialogContent>
609 %       <DialogHeader>
610 %         <DialogTitle>Nuevo Articulo</DialogTitle>
611 %       </DialogHeader>
612 %       <NewItemForm
613 %         categories={categories}
614 %         suppliers={suppliers}
615 %         onSubmit={(data) => createItemMutation.mutate(data)}
616 %         isPending={createItemMutation.isPending}
617 %         onCancel={() => setIsNewItemDialogOpen(false)}
618 %       />
619 %     </DialogContent>
620 %   </Dialog>
621 %
622 %   <Dialog open={isMovementDialogOpen} onOpenChange={setIsMovementDialogOpen}>
623 %     <DialogContent>
624 %       <DialogHeader>
625 %         <DialogTitle>
626 %           Registrar {movementType === "entrada" ? "Entrada" : "Salida"} de Stock
627 %         </DialogTitle>
628 %       </DialogHeader>
629 %       <MovementForm />
630 %     </DialogContent>
631 %   </Dialog>
632 % </div>
633 % );
634 % }
635 %
636 % function NewItemForm({
637 %   categories,
638 %   suppliers,
639 %   onSubmit,
640 %   isPending,
641 %   onCancel,
642 % }: {
643 %   categories: ItemCategory[];
644 %   suppliers: Supplier[];
645 %   onSubmit: (data: Partial<InventoryItem>) => void;
646 %   isPending: boolean;
647 %   onCancel: () => void;
648 % }) {
```

```

649 %   const [name, setName] = useState("");
650 %   const [sku, setSku] = useState("");
651 %   const [categoryId, setCategoryId] = useState("");
652 %   const [supplierId, setSupplierId] = useState("");
653 %   const [unit, setUnit] = useState<string>("unidad");
654 %   const [costPrice, setCostPrice] = useState("0");
655 %   const [minStock, setMinStock] = useState(0);
656 %   const [currentStock, setCurrentStock] = useState(0);
657 %
658 %   return (
659 %     <div className="space-y-4">
660 %       <div className="grid gap-4 sm:grid-cols-2">
661 %         <div className="space-y-2">
662 %           <Label>Nombre</Label>
663 %           <Input
664 %             value={name}
665 %             onChange={(e) => setName(e.target.value)}
666 %             placeholder="Nombre del articulo"
667 %             data-testid="input-item-name"
668 %           />
669 %         </div>
670 %         <div className="space-y-2">
671 %           <Label>SKU (opcional)</Label>
672 %           <Input
673 %             value={sku}
674 %             onChange={(e) => setSku(e.target.value)}
675 %             placeholder="Codigo SKU"
676 %             data-testid="input-item-sku"
677 %           />
678 %         </div>
679 %       </div>
680 %       <div className="grid gap-4 sm:grid-cols-2">
681 %         <div className="space-y-2">
682 %           <Label>Categoria</Label>
683 %           <Select value={categoryId} onValueChange={setCategoryId}>
684 %             <SelectTrigger data-testid="select-category">
685 %               <SelectValue placeholder="Seleccionar categoria" />
686 %             </SelectTrigger>
687 %             <SelectContent>
688 %               {categories.map((cat) => (
689 %                 <SelectItem key={cat.id} value={cat.id}>
690 %                   {cat.name} {cat.area !== "general" ? `(${cat.area.toUpperCase()})` : ""}
691 %                 </SelectItem>
692 %               ))}
693 %             </SelectContent>
694 %           </Select>
695 %         </div>
696 %         <div className="space-y-2">
697 %           <Label>Proveedor</Label>
698 %           <Select value={supplierId} onValueChange={setSupplierId}>
699 %             <SelectTrigger data-testid="select-supplier">
700 %               <SelectValue placeholder="Seleccionar proveedor" />
701 %             </SelectTrigger>
702 %             <SelectContent>
703 %               {suppliers.map((sup) => (
704 %                 <SelectItem key={sup.id} value={sup.id}>
705 %                   {sup.name}
706 %                 </SelectItem>
707 %               ))}
708 %             </SelectContent>
709 %           </Select>
710 %         </div>
711 %       </div>
712 %       <div className="grid gap-4 sm:grid-cols-3">
713 %         <div className="space-y-2">
714 %           <Label>Unidad</Label>
715 %           <Select value={unit} onValueChange={setUnit}>
716 %             <SelectTrigger data-testid="select-unit">
717 %               <SelectValue />
718 %             </SelectTrigger>
719 %             <SelectContent>
720 %               {Object.entries(unitLabels).map(([key, label]) => (
721 %                 <SelectItem key={key} value={key}>
722 %                   {label}
723 %                 </SelectItem>
724 %               ))}
725 %             </SelectContent>
726 %           </Select>
727 %         </div>
728 %         <div className="space-y-2">
729 %           <Label>Costo Unitario</Label>
730 %           <Input
731 %             type="number"
732 %             min={0}
733 %             step="0.01"
734 %             value={costPrice}
735 %             onChange={(e) => setCostPrice(e.target.value)}
736 %             data-testid="input-cost"
737 %           />
738 %         </div>
739 %         <div className="space-y-2">
740 %           <Label>Stock Minimo</Label>
741 %           <Input

```

client/src/pages/inventory.tsx (cont.)

```
742 %         type="number"
743 %         min={0}
744 %         value={minStock}
745 %         onChange={(e) => setMinStock(parseInt(e.target.value) || 0)}
746 %         data-testid="input-min-stock"
747 %       />
748 %     </div>
749 %   </div>
750 %   <div className="space-y-2">
751 %     <Label>Stock Inicial</Label>
752 %     <Input
753 %       type="number"
754 %       min={0}
755 %       value={currentStock}
756 %       onChange={(e) => setCurrentStock(parseInt(e.target.value) || 0)}
757 %       data-testid="input-current-stock"
758 %     />
759 %   </div>
760 %   <DialogFooter>
761 %     <Button variant="outline" onClick={onCancel}>
762 %       Cancelar
763 %     </Button>
764 %     <Button
765 %       onClick={() => {
766 %         onSubmit({
767 %           name,
768 %           sku: sku || undefined,
769 %           categoryId: categoryId || undefined,
770 %           supplierId: supplierId || undefined,
771 %           unit: unit as any,
772 %           costPrice,
773 %           minStock,
774 %           currentStock,
775 %         });
776 %       }}
777 %       disabled={isPending || !name}
778 %       data-testid="button-save-item"
779 %     >
780 %       {isPending && <Loader2 className="h-4 w-4 mr-2 animate-spin" />}
781 %       Guardar
782 %     </Button>
783 %   </DialogFooter>
784 % </div>
785 % );
786 % }
787 %
```

```

1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { useForm } from "react-hook-form";
4 % import { zodResolver } from "@hookform/resolvers/zod";
5 % import { z } from "zod";
6 % import { format } from "date-fns";
7 % import { es } from "date-fns/locale";
8 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
9 % import { Button } from "@components/ui/button";
10 % import { Input } from "@components/ui/input";
11 % import { Label } from "@components/ui/label";
12 % import { Badge } from "@components/ui/badge";
13 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
14 % import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@components/ui/table";
15 % import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@components/ui/dialog";
16 % import { Form, FormControl, FormField, FormItem, FormLabel, FormMessage } from "@components/ui/form";
17 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
18 % import { Skeleton } from "@components/ui/skeleton";
19 % import { useToast } from "@hooks/use-toast";
20 % import { apiRequest, queryClient } from "@lib/queryClient";
21 % import {
22 %   Users,
23 %   Settings,
24 %   Shield,
25 %   Activity,
26 %   Plus,
27 %   Pencil,
28 %   Trash2,
29 %   UserCheck,
30 %   Clock,
31 %   Save,
32 %   History,
33 % } from "lucide-react";
34 % import { Bed, Check } from "lucide-react";
35 % import type { SystemUser, SystemSetting, AuditLog, SystemUserRole, BedType } from "@shared/schema";
36 %
37 % type DashboardStats = {
38 %   totalUsers: number;
39 %   activeUsers: number;
40 %   recentLogins: number;
41 %   totalSettings: number;
42 %   recentAuditLogs: AuditLog[];
43 % };
44 %
45 % const userFormSchema = z.object({
46 %   username: z.string().min(3, "Mínimo 3 caracteres"),
47 %   email: z.string().email("Email inválido"),
48 %   fullName: z.string().min(2, "Nombre requerido"),
49 %   role: z.string(),
50 %   department: z.string().optional(),
51 %   phone: z.string().optional(),
52 %   isActive: z.string().default("true"),
53 % });
54 %
55 % type UserFormValues = z.infer<typeof userFormSchema>;
56 %
57 % const settingFormSchema = z.object({
58 %   key: z.string().min(1, "Clave requerida"),
59 %   value: z.string().min(1, "Valor requerido"),
60 %   category: z.string().default("general"),
61 %   description: z.string().optional(),
62 % });
63 %
64 % type SettingFormValues = z.infer<typeof settingFormSchema>;
65 %
66 % const roleLabels: Record<string, string> = {
67 %   admin: "Administrador",
68 %   manager: "Gerente",
69 %   reception: "Recepcion",
70 %   housekeeping: "Housekeeping",
71 %   maintenance: "Mantenimiento",
72 %   restaurant: "Restaurante",
73 %   spa: "SPA",
74 %   events: "Eventos",
75 % };
76 %
77 % const actionLabels: Record<string, string> = {
78 %   create: "Crear",
79 %   update: "Actualizar",
80 %   delete: "Eliminar",
81 %   login: "Iniciar sesion",
82 %   logout: "Cerrar sesion",
83 %   view: "Ver",
84 %   export: "Exportar",
85 % };
86 %
87 % const categoryLabels: Record<string, string> = {
88 %   general: "General",
89 %   reservations: "Reservas",
90 %   billing: "Facturacion",

```

client/src/pages/administration.tsx (cont.)

```
91 %     amenities: "Amenidades",
92 %   };
93 %
94 %   export default function AdministrationPage() {
95 %     const { toast } = useToast();
96 %     const [activeTab, setActiveTab] = useState("dashboard");
97 %     const [isUserDialogOpen, setIsUserDialogOpen] = useState(false);
98 %     const [isSettingDialogOpen, setIsSettingDialogOpen] = useState(false);
99 %     const [editingUser, setEditingUser] = useState<SystemUser | null>(null);
100 %    const [editingSetting, setEditingSetting] = useState<SystemSetting | null>(null);
101 %    const [filterModule, setFilterModule] = useState<string>("all");
102 %    const [isBedTypeDialogOpen, setIsBedTypeDialogOpen] = useState(false);
103 %    const [editingBedType, setEditingBedType] = useState<BedType | null>(null);
104 %    const [bedTypeForm, setBedTypeForm] = useState({ code: "", name: "", description: "" });
105 %
106 %    const { data: dashboardStats, isLoading: loadingStats } = useQuery<DashboardStats>({
107 %      queryKey: ["/api/admin/dashboard"],
108 %    });
109 %
110 %    const { data: users = [], isLoading: loadingUsers } = useQuery<SystemUser[]>({
111 %      queryKey: ["/api/admin/users"],
112 %    });
113 %
114 %    const { data: settings = [], isLoading: loadingSettings } = useQuery<SystemSetting[]>({
115 %      queryKey: ["/api/admin/settings"],
116 %    });
117 %
118 %    const { data: auditLogs = [], isLoading: loadingLogs } = useQuery<AuditLog[]>({
119 %      queryKey: ["/api/admin/audit-logs"],
120 %    });
121 %
122 %    const { data: bedTypesData = [], isLoading: loadingBedTypes } = useQuery<BedType[]>({
123 %      queryKey: ["/api/bed-types"],
124 %    });
125 %
126 %    const createBedTypeMutation = useMutation({
127 %      mutationFn: async (data: { code: string; name: string; description: string }) => {
128 %        return apiRequest("POST", "/api/bed-types", data);
129 %      },
130 %      onSuccess: () => {
131 %        queryClient.invalidateQueries({ queryKey: ["/api/bed-types"] });
132 %        setIsBedTypeDialogOpen(false);
133 %        setBedTypeForm({ code: "", name: "", description: "" });
134 %        toast({ title: "Tipo de camaje creado correctamente" });
135 %      },
136 %      onError: () => {
137 %        toast({ title: "Error al crear tipo de camaje", variant: "destructive" });
138 %      },
139 %    });
140 %
141 %    const updateBedTypeMutation = useMutation({
142 %      mutationFn: async (data: { id: number; code: string; name: string; description: string }) => {
143 %        const { id, ...rest } = data;
144 %        return apiRequest("PATCH", `/api/bed-types/${id}`, rest);
145 %      },
146 %      onSuccess: () => {
147 %        queryClient.invalidateQueries({ queryKey: ["/api/bed-types"] });
148 %        setIsBedTypeDialogOpen(false);
149 %        setEditingBedType(null);
150 %        setBedTypeForm({ code: "", name: "", description: "" });
151 %        toast({ title: "Tipo de camaje actualizado correctamente" });
152 %      },
153 %      onError: () => {
154 %        toast({ title: "Error al actualizar tipo de camaje", variant: "destructive" });
155 %      },
156 %    });
157 %
158 %    const toggleBedTypeMutation = useMutation({
159 %      mutationFn: async (id: number) => {
160 %        return apiRequest("DELETE", `/api/bed-types/${id}`);
161 %      },
162 %      onSuccess: () => {
163 %        queryClient.invalidateQueries({ queryKey: ["/api/bed-types"] });
164 %        toast({ title: "Estado del tipo de camaje actualizado" });
165 %      },
166 %      onError: () => {
167 %        toast({ title: "Error al cambiar estado", variant: "destructive" });
168 %      },
169 %    });
170 %
171 %    const userForm = useForm<UserFormValues>({
172 %      resolver: zodResolver(userFormSchema),
173 %      defaultValues: {
174 %        username: "",
175 %        email: "",
176 %        fullName: "",
177 %        role: "reception",
178 %        department: "",
179 %        phone: "",
180 %        isActive: "true",
181 %      },
182 %    });
183 %  }
```

client/src/pages/administration.tsx (cont.)

```
184 %   const settingForm = useForm<SettingFormValues>({
185 %     resolver: zodResolver(settingFormSchema),
186 %     defaultValues: {
187 %       key: "",
188 %       value: "",
189 %       category: "general",
190 %       description: "",
191 %     },
192 %   });
193 %
194 %   const createUserMutation = useMutation({
195 %     mutationFn: async (data: UserFormValues) => {
196 %       return apiRequest("POST", "/api/admin/users", data);
197 %     },
198 %     onSuccess: () => {
199 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/users"] });
200 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/dashboard"] });
201 %       setIsUserDialogOpen(false);
202 %       userForm.reset();
203 %       toast({ title: "Usuario creado correctamente" });
204 %     },
205 %     onError: () => {
206 %       toast({ title: "Error al crear usuario", variant: "destructive" });
207 %     },
208 %   });
209 %
210 %   const updateUserMutation = useMutation({
211 %     mutationFn: async (data: UserFormValues & { id: string }) => {
212 %       const { id, ...userData } = data;
213 %       return apiRequest("PATCH", `/api/admin/users/${id}`, userData);
214 %     },
215 %     onSuccess: () => {
216 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/users"] });
217 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/dashboard"] });
218 %       setIsUserDialogOpen(false);
219 %       setEditingUser(null);
220 %       userForm.reset();
221 %       toast({ title: "Usuario actualizado correctamente" });
222 %     },
223 %     onError: () => {
224 %       toast({ title: "Error al actualizar usuario", variant: "destructive" });
225 %     },
226 %   });
227 %
228 %   const deleteUserMutation = useMutation({
229 %     mutationFn: async (id: string) => {
230 %       return apiRequest("DELETE", `/api/admin/users/${id}`);
231 %     },
232 %     onSuccess: () => {
233 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/users"] });
234 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/dashboard"] });
235 %       toast({ title: "Usuario eliminado correctamente" });
236 %     },
237 %     onError: () => {
238 %       toast({ title: "Error al eliminar usuario", variant: "destructive" });
239 %     },
240 %   });
241 %
242 %   const upsertSettingMutation = useMutation({
243 %     mutationFn: async (data: SettingFormValues) => {
244 %       return apiRequest("PUT", "/api/admin/settings", data);
245 %     },
246 %     onSuccess: () => {
247 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/settings"] });
248 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/dashboard"] });
249 %       setIsSettingDialogOpen(false);
250 %       setEditingSetting(null);
251 %       settingForm.reset();
252 %       toast({ title: "Configuracion guardada correctamente" });
253 %     },
254 %     onError: () => {
255 %       toast({ title: "Error al guardar configuracion", variant: "destructive" });
256 %     },
257 %   });
258 %
259 %   const deleteSettingMutation = useMutation({
260 %     mutationFn: async (key: string) => {
261 %       return apiRequest("DELETE", `/api/admin/settings/${key}`);
262 %     },
263 %     onSuccess: () => {
264 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/settings"] });
265 %       queryClient.invalidateQueries({ queryKey: ["/api/admin/dashboard"] });
266 %       toast({ title: "Configuracion eliminada correctamente" });
267 %     },
268 %     onError: () => {
269 %       toast({ title: "Error al eliminar configuracion", variant: "destructive" });
270 %     },
271 %   });
272 %
273 %   const openNewUserDialog = () => {
274 %     setEditingUser(null);
275 %     userForm.reset({
276 %       username: "",
```

client/src/pages/administration.tsx (cont.)

```
277 %         email: "",
278 %         fullName: "",
279 %         role: "reception",
280 %         department: "",
281 %         phone: "",
282 %         isActive: "true",
283 %     });
284 %     setIsUserDialogOpen(true);
285 % };
286 %
287 % const openEditUserDialog = (user: SystemUser) => {
288 %     setEditingUser(user);
289 %     userForm.reset({
290 %         username: user.username,
291 %         email: user.email,
292 %         fullName: user.fullName,
293 %         role: user.role,
294 %         department: user.department || "",
295 %         phone: user.phone || "",
296 %         isActive: user.isActive || "true",
297 %     });
298 %     setIsUserDialogOpen(true);
299 % };
300 %
301 % const openNewSettingDialog = () => {
302 %     setEditingSetting(null);
303 %     settingForm.reset({
304 %         key: "",
305 %         value: "",
306 %         category: "general",
307 %         description: "",
308 %     });
309 %     setIsSettingDialogOpen(true);
310 % };
311 %
312 % const openEditSettingDialog = (setting: SystemSetting) => {
313 %     setEditingSetting(setting);
314 %     settingForm.reset({
315 %         key: setting.key,
316 %         value: setting.value,
317 %         category: setting.category,
318 %         description: setting.description || "",
319 %     });
320 %     setIsSettingDialogOpen(true);
321 % };
322 %
323 % const handleUserSubmit = (data: UserFormValues) => {
324 %     if (editingUser) {
325 %         updateUserMutation.mutate({ ...data, id: editingUser.id });
326 %     } else {
327 %         createUserMutation.mutate(data);
328 %     }
329 % };
330 %
331 % const handleSettingSubmit = (data: SettingFormValues) => {
332 %     upsertSettingMutation.mutate(data);
333 % };
334 %
335 % const filteredLogs = filterModule === "all"
336 %   ? auditLogs
337 %   : auditLogs.filter((log) => log.module === filterModule);
338 %
339 % const uniqueModules = Array.from(new Set(auditLogs.map((l) => l.module)));
340 %
341 % const groupedSettings = settings.reduce((acc, setting) => {
342 %     const cat = setting.category || "general";
343 %     if (!acc[cat]) acc[cat] = [];
344 %     acc[cat].push(setting);
345 %     return acc;
346 % }, {} as Record<string, SystemSetting[]>);
347 %
348 % return (
349 %     <div className="h-full overflow-auto p-6">
350 %         <div className="flex items-center justify-between gap-4 mb-6 flex-wrap">
351 %             <div>
352 %                 <h1 className="text-2xl font-bold" data-testid="text-admin-title">Administracion</h1>
353 %                 <p className="text-muted-foreground">Gestion del sistema, usuarios y configuracion</p>
354 %             </div>
355 %         </div>
356 %
357 %         <Tabs value={activeTab} onValueChange={setActiveTab} className="space-y-4">
358 %             <TabsList className="flex-wrap">
359 %                 <TabsTrigger value="dashboard" data-testid="tab-admin-dashboard">
360 %                     <Activity className="w-4 h-4 mr-2" />
361 %                     Panel
362 %                 </TabsTrigger>
363 %                 <TabsTrigger value="users" data-testid="tab-admin-users">
364 %                     <Users className="w-4 h-4 mr-2" />
365 %                     Usuarios
366 %                 </TabsTrigger>
367 %                 <TabsTrigger value="settings" data-testid="tab-admin-settings">
368 %                     <Settings className="w-4 h-4 mr-2" />
369 %                     Configuracion
```



```

370 %         </TabsTrigger>
371 %     </TabsTrigger value="bed-types" data-testid="tab-admin-bed-types">
372 %         <Bed className="w-4 h-4 mr-2" />
373 %         Tipos de Camaje
374 %     </TabsTrigger>
375 %     <TabsTrigger value="audit" data-testid="tab-admin-audit">
376 %         <History className="w-4 h-4 mr-2" />
377 %         Auditoria
378 %     </TabsTrigger>
379 % </TabsList>
380 %
381 % <TabsContent value="dashboard" className="space-y-4">
382 %     <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-4">
383 %         <Card>
384 %             <CardHeader className="flex flex-row items-center justify-between gap-2 space-y-0 pb-2">
385 %                 <CardTitle className="text-sm font-medium">Total Usuarios</CardTitle>
386 %                 <Users className="h-4 w-4 text-muted-foreground" />
387 %             </CardHeader>
388 %             <CardContent>
389 %                 {loadingStats ? (
390 %                     <Skeleton className="h-8 w-16" />
391 %                 ) : (
392 %                     <div className="text-2xl font-bold" data-testid="text-total-users">
393 %                         {dashboardStats?.totalUsers || 0}
394 %                     </div>
395 %                 )}
396 %             </CardContent>
397 %         </Card>
398 %
399 %         <Card>
400 %             <CardHeader className="flex flex-row items-center justify-between gap-2 space-y-0 pb-2">
401 %                 <CardTitle className="text-sm font-medium">Usuarios Activos</CardTitle>
402 %                 <UserCheck className="h-4 w-4 text-muted-foreground" />
403 %             </CardHeader>
404 %             <CardContent>
405 %                 {loadingStats ? (
406 %                     <Skeleton className="h-8 w-16" />
407 %                 ) : (
408 %                     <div className="text-2xl font-bold text-green-600" data-testid="text-active-users">
409 %                         {dashboardStats?.activeUsers || 0}
410 %                     </div>
411 %                 )}
412 %             </CardContent>
413 %         </Card>
414 %
415 %         <Card>
416 %             <CardHeader className="flex flex-row items-center justify-between gap-2 space-y-0 pb-2">
417 %                 <CardTitle className="text-sm font-medium">Sesiones Hoy</CardTitle>
418 %                 <Clock className="h-4 w-4 text-muted-foreground" />
419 %             </CardHeader>
420 %             <CardContent>
421 %                 {loadingStats ? (
422 %                     <Skeleton className="h-8 w-16" />
423 %                 ) : (
424 %                     <div className="text-2xl font-bold" data-testid="text-recent-logins">
425 %                         {dashboardStats?.recentLogins || 0}
426 %                     </div>
427 %                 )}
428 %             </CardContent>
429 %         </Card>
430 %
431 %         <Card>
432 %             <CardHeader className="flex flex-row items-center justify-between gap-2 space-y-0 pb-2">
433 %                 <CardTitle className="text-sm font-medium">Configuraciones</CardTitle>
434 %                 <Settings className="h-4 w-4 text-muted-foreground" />
435 %             </CardHeader>
436 %             <CardContent>
437 %                 {loadingStats ? (
438 %                     <Skeleton className="h-8 w-16" />
439 %                 ) : (
440 %                     <div className="text-2xl font-bold" data-testid="text-total-settings">
441 %                         {dashboardStats?.totalSettings || 0}
442 %                     </div>
443 %                 )}
444 %             </CardContent>
445 %         </Card>
446 %     </div>
447 %
448 %     <Card>
449 %         <CardHeader>
450 %             <CardTitle>Actividad Reciente</CardTitle>
451 %             <CardDescription>Ultimas acciones en el sistema</CardDescription>
452 %         </CardHeader>
453 %         <CardContent>
454 %             {loadingStats ? (
455 %                 <div className="space-y-2">
456 %                     {[1, 2, 3].map((i) => (
457 %                         <Skeleton key={i} className="h-12 w-full" />
458 %                     ))}
459 %                 </div>
460 %             ) : (
461 %                 <Table>
462 %                     <TableHeader>

```

client/src/pages/administration.tsx (cont.)

```
463 %         <TableRow>
464 %           <TableHead>Usuario</TableHead>
465 %           <TableHead>Accion</TableHead>
466 %           <TableHead>Modulo</TableHead>
467 %           <TableHead>Descripcion</TableHead>
468 %           <TableHead>Fecha</TableHead>
469 %         </TableRow>
470 %       </TableHeader>
471 %       <TableBody>
472 %         {(dashboardStats?.recentAuditLogs || []).slice(0, 5).map((log) => (
473 %           <TableRow key={log.id}>
474 %             <TableCell className="font-medium">{log.userName || "Sistema"}</TableCell>
475 %             <TableCell>
476 %               <Badge variant="outline">{actionLabels[log.action] || log.action}</Badge>
477 %             </TableCell>
478 %             <TableCell className="capitalize">{log.module}</TableCell>
479 %             <TableCell className="text-muted-foreground">{log.description}</TableCell>
480 %             <TableCell className="text-muted-foreground">
481 %               {format(new Date(log.timestamp), "dd/MM HH:mm", { locale: es })}
482 %             </TableCell>
483 %           </TableRow>
484 %         )]}
485 %         {(!dashboardStats?.recentAuditLogs || dashboardStats.recentAuditLogs.length === 0) && (
486 %           <TableRow>
487 %             <TableCell colspan={5} className="text-center text-muted-foreground py-8">
488 %               No hay actividad reciente
489 %             </TableCell>
490 %           </TableRow>
491 %         )}
492 %       </TableBody>
493 %     </Table>
494 %   )}
495 % </CardContent>
496 % </Card>
497 % </TabsContent>
498 %
499 % <TabsContent value="users" className="space-y-4">
500 %   <div className="flex items-center justify-between gap-4 flex-wrap">
501 %     <h2 className="text-lg font-semibold">Usuarios del Sistema</h2>
502 %     <Button onClick={openNewUserDialog} data-testid="button-new-user">
503 %       <Plus className="w-4 h-4 mr-2" />
504 %       Nuevo Usuario
505 %     </Button>
506 %   </div>
507 %
508 %   <Card>
509 %     <CardContent className="pt-6">
510 %       {loadingUsers ? (
511 %         <div className="space-y-2">
512 %           {[1, 2, 3].map((i) => (
513 %             <Skeleton key={i} className="h-12 w-full" />
514 %           ))}
515 %         </div>
516 %       ) : (
517 %         <Table>
518 %           <TableHeader>
519 %             <TableRow>
520 %               <TableHead>Usuario</TableHead>
521 %               <TableHead>Nombre</TableHead>
522 %               <TableHead>Email</TableHead>
523 %               <TableHead>Rol</TableHead>
524 %               <TableHead>Departamento</TableHead>
525 %               <TableHead>Estado</TableHead>
526 %               <TableHead>Ultimo Acceso</TableHead>
527 %               <TableHead className="text-right">Acciones</TableHead>
528 %             </TableRow>
529 %           </TableHeader>
530 %           <TableBody>
531 %             {users.map((user) => (
532 %               <TableRow key={user.id} data-testid={`row-user-${user.id}`}>
533 %                 <TableCell className="font-medium">{user.username}</TableCell>
534 %                 <TableCell>{user.fullName}</TableCell>
535 %                 <TableCell className="text-muted-foreground">{user.email}</TableCell>
536 %                 <TableCell>
537 %                   <Badge variant={user.role === "admin" ? "default" : "secondary"}>
538 %                     {roleLabels[user.role] || user.role}
539 %                   </Badge>
540 %                 </TableCell>
541 %                 <TableCell>{user.department || "-"}</TableCell>
542 %                 <TableCell>
543 %                   <Badge variant={user.isActive === "true" ? "default" : "secondary"}>
544 %                     {user.isActive === "true" ? "Activo" : "Inactivo"}
545 %                   </Badge>
546 %                 </TableCell>
547 %                 <TableCell className="text-muted-foreground">
548 %                   {user.lastLogin
549 %                     ? format(new Date(user.lastLogin), "dd/MM/yyyy HH:mm", { locale: es })
550 %                     : "Nunca"}
551 %                 </TableCell>
552 %                 <TableCell className="text-right">
553 %                   <div className="flex justify-end gap-1">
554 %                     <Button
555 %                       size="icon"
```


client/src/pages/administration.tsx (cont.)

```
649 %         </Button>
650 %     </div>
651 % </TableCell>
652 % </TableRow>
653 %     )})
654 % </TableBody>
655 % </Table>
656 % </CardContent>
657 % </Card>
658 % ))
659 % })
660 % </TabsContent>
661 %
662 % <TabsContent value="bed-types" className="space-y-4">
663 %   <div className="flex items-center justify-between gap-4 flex-wrap">
664 %     <h2 className="text-lg font-semibold">Tipos de Camaje</h2>
665 %     <Button
666 %       onClick={() => {
667 %         setEditingBedType(null);
668 %         setBedTypeForm({ code: "", name: "", description: "" });
669 %         setIsBedTypeDialogOpen(true);
670 %       }}
671 %       data-testid="button-new-bed-type"
672 %     >
673 %       <Plus className="w-4 h-4 mr-2" />
674 %       Nuevo Tipo de Camaje
675 %     </Button>
676 %   </div>
677 %
678 %   <Card>
679 %     <CardContent className="pt-6">
680 %       {loadingBedTypes ? (
681 %         <div className="space-y-2">
682 %           {[1, 2, 3].map((i) => (
683 %             <Skeleton key={i} className="h-12 w-full" />
684 %           ))}
685 %         </div>
686 %       ) : (
687 %         <Table>
688 %           <TableHeader>
689 %             <TableRow>
690 %               <TableHead>Codigo</TableHead>
691 %               <TableHead>Nombre</TableHead>
692 %               <TableHead>Descripcion</TableHead>
693 %               <TableHead>Estado</TableHead>
694 %               <TableHead className="text-right">Acciones</TableHead>
695 %             </TableRow>
696 %           </TableHeader>
697 %           <TableBody>
698 %             {bedTypesData.map((bt) => (
699 %               <TableRow key={bt.id} data-testid={`row-bed-type-${bt.id}`}>
700 %                 <TableCell className="font-mono text-sm">{bt.code}</TableCell>
701 %                 <TableCell className="font-medium">{bt.name}</TableCell>
702 %                 <TableCell className="text-muted-foreground">{bt.description || "-"}</TableCell>
703 %                 <TableCell>
704 %                   <Badge variant={bt.isActive ? "default" : "secondary"}>
705 %                     {bt.isActive ? "Activo" : "Inactivo"}
706 %                   </Badge>
707 %                 </TableCell>
708 %                 <TableCell className="text-right">
709 %                   <div className="flex justify-end gap-1">
710 %                     <Button
711 %                       size="icon"
712 %                       variant="ghost"
713 %                       onClick={() => {
714 %                         setEditingBedType(bt);
715 %                         setBedTypeForm({
716 %                           code: bt.code,
717 %                           name: bt.name,
718 %                           description: bt.description || "",
719 %                         });
720 %                         setIsBedTypeDialogOpen(true);
721 %                       }}
722 %                       data-testid={`button-edit-bed-type-${bt.id}`}
723 %                     >
724 %                       <Pencil className="w-4 h-4" />
725 %                     </Button>
726 %                     <Button
727 %                       size="icon"
728 %                       variant="ghost"
729 %                       onClick={() => toggleBedTypeMutation.mutate(bt.id)}
730 %                       data-testid={`button-toggle-bed-type-${bt.id}`}
731 %                     >
732 %                       {bt.isActive ? <Trash2 className="w-4 h-4" /> : <Check className="w-4 h-4" />}
733 %                     </Button>
734 %                   </div>
735 %                 </TableCell>
736 %               </TableRow>
737 %             )})
738 %             {bedTypesData.length === 0 && (
739 %               <TableRow>
740 %                 <TableCell colspan={5} className="text-center text-muted-foreground py-8">
741 %                   No hay tipos de camaje registrados
```

client/src/pages/administration.tsx (cont.)

```
742 %         </TableCell>
743 %       </TableRow>
744 %     })
745 %   </TableBody>
746 % </Table>
747 % })
748 % </CardContent>
749 % </Card>
750 % </TabsContent>
751 %
752 % <TabsContent value="audit" className="space-y-4">
753 %   <div className="flex items-center justify-between gap-4 flex-wrap">
754 %     <h2 className="text-lg font-semibold">Registro de Auditoria</h2>
755 %     <Select value={filterModule} onChange={setFilterModule}>
756 %       <SelectTrigger className="w-48" data-testid="select-audit-filter">
757 %         <SelectValue placeholder="Filtrar por modulo" />
758 %       </SelectTrigger>
759 %       <SelectContent>
760 %         <SelectItem value="all">Todos los modulos</SelectItem>
761 %         {uniqueModules.map((mod) => (
762 %           <SelectItem key={mod} value={mod}>
763 %             {mod.charAt(0).toUpperCase() + mod.slice(1)}
764 %           </SelectItem>
765 %         ))}
766 %       </SelectContent>
767 %     </Select>
768 %   </div>
769 %
770 %   <Card>
771 %     <CardContent className="pt-6">
772 %       {loadingLogs ? (
773 %         <div className="space-y-2">
774 %           {[1, 2, 3, 4, 5].map((i) => (
775 %             <Skeleton key={i} className="h-12 w-full" />
776 %           ))}
777 %         </div>
778 %       ) : (
779 %         <Table>
780 %           <TableHeader>
781 %             <TableRow>
782 %               <TableHead>Fecha/Hora</TableHead>
783 %               <TableHead>Usuario</TableHead>
784 %               <TableHead>Accion</TableHead>
785 %               <TableHead>Modulo</TableHead>
786 %               <TableHead>Descripcion</TableHead>
787 %               <TableHead>Detalles</TableHead>
788 %               <TableHead>IP</TableHead>
789 %             </TableRow>
790 %           </TableHeader>
791 %           <TableBody>
792 %             {filteredLogs.map((log) => (
793 %               <TableRow key={log.id} data-testid={`row-audit-${log.id}`}>
794 %                 <TableCell className="text-muted-foreground whitespace-nowrap">
795 %                   {format(new Date(log.timestamp), "dd/MM/yyyy HH:mm:ss", { locale: es })}
796 %                 </TableCell>
797 %                 <TableCell className="font-medium">{log.userName} || "Sistema"</TableCell>
798 %                 <TableCell>
799 %                   <Badge variant="outline">{actionLabels[log.action]} || log.action</Badge>
800 %                 </TableCell>
801 %                 <TableCell className="capitalize">{log.module}</TableCell>
802 %                 <TableCell>{log.description}</TableCell>
803 %                 <TableCell className="text-muted-foreground max-w-xs truncate">
804 %                   {log.details || "-"}
805 %                 </TableCell>
806 %                 <TableCell className="text-muted-foreground font-mono text-xs">
807 %                   {log.ipAddress || "-"}
808 %                 </TableCell>
809 %               </TableRow>
810 %             ))}
811 %             {filteredLogs.length === 0 && (
812 %               <TableRow>
813 %                 <TableCell colspan={7} className="text-center text-muted-foreground py-8">
814 %                   No hay registros de auditoria
815 %                 </TableCell>
816 %               </TableRow>
817 %             )}
818 %           </TableBody>
819 %         </Table>
820 %       )}
821 %     </CardContent>
822 %   </Card>
823 % </TabsContent>
824 % </Tabs>
825 %
826 % <Dialog open={isUserDialogOpen} onOpenChange={setIsUserDialogOpen}>
827 %   <DialogContent className="max-w-md">
828 %     <DialogHeader>
829 %       <DialogTitle>{editingUser ? "Editar Usuario" : "Nuevo Usuario"}</DialogTitle>
830 %     </DialogHeader>
831 %     <Form {...userForm}>
832 %       <form onSubmit={userForm.handleSubmit(handleUserSubmit)} className="space-y-4">
833 %         <FormField>
834 %           control={userForm.control}
```

client/src/pages/administration.tsx (cont.)

```
835 %         name="username"
836 %         render={({ field }) => (
837 %             <FormItem>
838 %                 <FormLabel>Usuario</FormLabel>
839 %                 <FormControl>
840 %                     <Input
841 %                         {...field}
842 %                         placeholder="nombre.usuario"
843 %                         disabled={!editingUser}
844 %                         data-testid="input-user-username"
845 %                     />
846 %                 </FormControl>
847 %                 <FormMessage />
848 %             </FormItem>
849 %         )}
850 %     />
851 %
852 %     <FormField
853 %         control={userForm.control}
854 %         name="fullName"
855 %         render={({ field }) => (
856 %             <FormItem>
857 %                 <FormLabel>Nombre Completo</FormLabel>
858 %                 <FormControl>
859 %                     <Input {...field} placeholder="Juan Perez" data-testid="input-user-fullname" />
860 %                 </FormControl>
861 %                 <FormMessage />
862 %             </FormItem>
863 %         )}
864 %     />
865 %
866 %     <FormField
867 %         control={userForm.control}
868 %         name="email"
869 %         render={({ field }) => (
870 %             <FormItem>
871 %                 <FormLabel>Email</FormLabel>
872 %                 <FormControl>
873 %                     <Input
874 %                         {...field}
875 %                         type="email"
876 %                         placeholder="usuario@maransuites.com"
877 %                         data-testid="input-user-email"
878 %                     />
879 %                 </FormControl>
880 %                 <FormMessage />
881 %             </FormItem>
882 %         )}
883 %     />
884 %
885 %     <div className="grid grid-cols-2 gap-4">
886 %         <FormField
887 %             control={userForm.control}
888 %             name="role"
889 %             render={({ field }) => (
890 %                 <FormItem>
891 %                     <FormLabel>Rol</FormLabel>
892 %                     <Select onValueChange={field.onChange} value={field.value}>
893 %                         <FormControl>
894 %                             <SelectTrigger data-testid="select-user-role">
895 %                                 <SelectValue placeholder="Seleccionar" />
896 %                             </SelectTrigger>
897 %                         </FormControl>
898 %                         <SelectContent>
899 %                             {Object.entries(roleLabels).map(([value, label]) => (
900 %                                 <SelectItem key={value} value={value}>
901 %                                     {label}
902 %                                 </SelectItem>
903 %                             ))}
904 %                         </SelectContent>
905 %                     </Select>
906 %                     <FormMessage />
907 %                 </FormItem>
908 %             )}
909 %         />
910 %
911 %         <FormField
912 %             control={userForm.control}
913 %             name="isActive"
914 %             render={({ field }) => (
915 %                 <FormItem>
916 %                     <FormLabel>Estado</FormLabel>
917 %                     <Select onValueChange={field.onChange} value={field.value}>
918 %                         <FormControl>
919 %                             <SelectTrigger data-testid="select-user-status">
920 %                                 <SelectValue placeholder="Seleccionar" />
921 %                             </SelectTrigger>
922 %                         </FormControl>
923 %                         <SelectContent>
924 %                             <SelectItem value="true">Activo</SelectItem>
925 %                             <SelectItem value="false">Inactivo</SelectItem>
926 %                         </SelectContent>
927 %                     </Select>
```

client/src/pages/administration.tsx (cont.)

```
928 %         <FormMessage />
929 %     </FormItem>
930 % }}
931 % />
932 % </div>
933 %
934 % <FormField
935 %   control={userForm.control}
936 %   name="department"
937 %   render={({ field }) => (
938 %     <FormItem>
939 %       <FormLabel>Departamento</FormLabel>
940 %       <FormControl>
941 %         <Input {...field} placeholder="Recepcion" data-testid="input-user-department" />
942 %       </FormControl>
943 %       <FormMessage />
944 %     </FormItem>
945 %   )}
946 % />
947 %
948 % <FormField
949 %   control={userForm.control}
950 %   name="phone"
951 %   render={({ field }) => (
952 %     <FormItem>
953 %       <FormLabel>Telefono</FormLabel>
954 %       <FormControl>
955 %         <Input {...field} placeholder="+54 343 400-0000" data-testid="input-user-phone" />
956 %       </FormControl>
957 %       <FormMessage />
958 %     </FormItem>
959 %   )}
960 % />
961 %
962 % <DialogFooter>
963 %   <Button
964 %     type="button"
965 %     variant="outline"
966 %     onClick={() => setIsUserDialogOpen(false)}
967 %   >
968 %     Cancelar
969 %   </Button>
970 %   <Button
971 %     type="submit"
972 %     disabled={createUserMutation.isPending || updateUserMutation.isPending}
973 %     data-testid="button-save-user"
974 %   >
975 %     <Save className="w-4 h-4 mr-2" />
976 %     {editingUser ? "Guardar Cambios" : "Crear Usuario"}
977 %   </Button>
978 % </DialogFooter>
979 % </form>
980 % </Form>
981 % </DialogContent>
982 % </Dialog>
983 %
984 % <Dialog open={isSettingDialogOpen} onOpenChange={setIsSettingDialogOpen}>
985 %   <DialogContent className="max-w-md">
986 %     <DialogHeader>
987 %       <DialogTitle>
988 %         {editingSetting ? "Editar Configuracion" : "Nueva Configuracion"}
989 %       </DialogTitle>
990 %     </DialogHeader>
991 %     <Form {...settingForm}>
992 %       <form onSubmit={settingForm.handleSubmit(handleSettingSubmit)} className="space-y-4">
993 %         <FormField
994 %           control={settingForm.control}
995 %           name="key"
996 %           render={({ field }) => (
997 %             <FormItem>
998 %               <FormLabel>Clave</FormLabel>
999 %               <FormControl>
1000 %                 <Input
1001 %                   {...field}
1002 %                   placeholder="nombre_configuracion"
1003 %                   disabled={!editingSetting}
1004 %                   data-testid="input-setting-key"
1005 %                 />
1006 %               </FormControl>
1007 %               <FormMessage />
1008 %             </FormItem>
1009 %           )}
1010 %         </Form>
1011 %       </form>
1012 %       <FormField
1013 %         control={settingForm.control}
1014 %         name="value"
1015 %         render={({ field }) => (
1016 %           <FormItem>
1017 %             <FormLabel>Valor</FormLabel>
1018 %             <FormControl>
1019 %               <Input {...field} placeholder="valor" data-testid="input-setting-value" />
1020 %             </FormControl>
```

client/src/pages/administration.tsx (cont.)

```
1021 %         <FormMessage />
1022 %       </FormItem>
1023 %     })
1024 %   />
1025 %
1026 %   <FormField
1027 %     control={settingForm.control}
1028 %     name="category"
1029 %     render={({ field }) => (
1030 %       <FormItem>
1031 %         <FormLabel>Categoria</FormLabel>
1032 %         <Select onChange={field.onChange} value={field.value}>
1033 %           <FormControl>
1034 %             <SelectTrigger data-testid="select-setting-category">
1035 %               <SelectValue placeholder="Seleccionar" />
1036 %             </SelectTrigger>
1037 %           </FormControl>
1038 %           <SelectContent>
1039 %             {Object.entries(categoryLabels).map(([value, label]) => (
1040 %               <SelectItem key={value} value={value}>
1041 %                 {label}
1042 %               </SelectItem>
1043 %             ))}
1044 %           </SelectContent>
1045 %         </Select>
1046 %       <FormMessage />
1047 %     </FormItem>
1048 %   })
1049 % />
1050 %
1051 %   <FormField
1052 %     control={settingForm.control}
1053 %     name="description"
1054 %     render={({ field }) => (
1055 %       <FormItem>
1056 %         <FormLabel>Descripcion</FormLabel>
1057 %         <FormControl>
1058 %           <Input
1059 %             {...field}
1060 %             placeholder="Descripcion de la configuracion"
1061 %             data-testid="input-setting-description"
1062 %           />
1063 %         </FormControl>
1064 %         <FormMessage />
1065 %       </FormItem>
1066 %     })
1067 % />
1068 %
1069 %   <DialogFooter>
1070 %     <Button
1071 %       type="button"
1072 %       variant="outline"
1073 %       onClick={() => setIsSettingDialogOpen(false)}
1074 %     >
1075 %       Cancelar
1076 %     </Button>
1077 %     <Button
1078 %       type="submit"
1079 %       disabled={upsertSettingMutation.isPending}
1080 %       data-testid="button-save-setting"
1081 %     >
1082 %       <Save className="w-4 h-4 mr-2" />
1083 %       Guardar
1084 %     </Button>
1085 %   </DialogFooter>
1086 % </form>
1087 % </Form>
1088 % </DialogContent>
1089 % </Dialog>
1090 %
1091 % <Dialog open={isBedTypeDialogOpen} onOpenChange={setIsBedTypeDialogOpen}>
1092 %   <DialogContent className="max-w-md">
1093 %     <DialogHeader>
1094 %       <DialogTitle>
1095 %         {editingBedType ? "Editar Tipo de Camaje" : "Nuevo Tipo de Camaje"}
1096 %       </DialogTitle>
1097 %     </DialogHeader>
1098 %     <form
1099 %       onSubmit={(e) => {
1100 %         e.preventDefault();
1101 %         if (editingBedType) {
1102 %           updateBedTypeMutation.mutate({ id: editingBedType.id, ...bedTypeForm });
1103 %         } else {
1104 %           createBedTypeMutation.mutate(bedTypeForm);
1105 %         }
1106 %       }}
1107 %     >
1108 %       <div className="space-y-2">
1109 %         <Label>Codigo</Label>
1110 %         <Input
1111 %           value={bedTypeForm.code}
1112 %           onChange={(e) => setBedTypeForm({ ...bedTypeForm, code: e.target.value })}
1113 %         />
```


client/src/pages/administration.tsx (cont.)

```
1114 %         placeholder="MAT, TWIN, etc."
1115 %         data-testid="input-bed-type-code"
1116 %     />
1117 % </div>
1118 % <div className="space-y-2">
1119 %   <Label>Nombre</Label>
1120 %   <Input
1121 %     value={bedTypeForm.name}
1122 %     onChange={(e) => setBedTypeForm({ ...bedTypeForm, name: e.target.value })}
1123 %     placeholder="Matrimonial, Twin, etc."
1124 %     data-testid="input-bed-type-name"
1125 %   />
1126 % </div>
1127 % <div className="space-y-2">
1128 %   <Label>Descripcion</Label>
1129 %   <Input
1130 %     value={bedTypeForm.description}
1131 %     onChange={(e) => setBedTypeForm({ ...bedTypeForm, description: e.target.value })}
1132 %     placeholder="Descripcion del tipo de camaje"
1133 %     data-testid="input-bed-type-description"
1134 %   />
1135 % </div>
1136 % <DialogFooter>
1137 %   <Button
1138 %     type="button"
1139 %     variant="outline"
1140 %     onClick={() => setIsBedTypeDialogOpen(false)}
1141 %   >
1142 %     Cancelar
1143 %   </Button>
1144 %   <Button
1145 %     type="submit"
1146 %     disabled={createBedTypeMutation.isPending || updateBedTypeMutation.isPending}
1147 %     data-testid="button-save-bed-type"
1148 %   >
1149 %     <Save className="w-4 h-4 mr-2" />
1150 %     {editingBedType ? "Guardar Cambios" : "Crear"}
1151 %   </Button>
1152 % </DialogFooter>
1153 % </form>
1154 % </DialogContent>
1155 % </Dialog>
1156 % </div>
1157 % );
1158 % }
1159 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { useToast } from "@hooks/use-toast";
5 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
6 % import { Button } from "@components/ui/button";
7 % import { Input } from "@components/ui/input";
8 % import { Badge } from "@components/ui/badge";
9 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
10 % import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@components/ui/dialog";
11 % import { Label } from "@components/ui/label";
12 % import { Textarea } from "@components/ui/textarea";
13 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
14 % import { Switch } from "@components/ui/switch";
15 % import { ScrollArea } from "@components/ui/scroll-area";
16 % import { Separator } from "@components/ui/separator";
17 % import {
18 %   Heart,
19 %   AlertTriangle,
20 %   Search,
21 %   Plus,
22 %   Edit,
23 %   Trash2,
24 %   Check,
25 %   CheckCircle,
26 %   Clock,
27 %   User,
28 %   UtensilsCrossed,
29 %   Sparkles,
30 %   DoorOpen,
31 %   Star,
32 %   ShieldAlert,
33 %   Gift,
34 %   Briefcase,
35 %   FileText,
36 %   ToggleLeft,
37 %   ToggleRight,
38 %   Eye,
39 % } from "lucide-react";
40 % import type { GuestPreference, Guest, HospitalityAlert } from "@shared/schema";
41 %
42 % const CATEGORY_OPTIONS = [
43 %   { value: "habitacion", label: "Habitación", icon: DoorOpen },
44 %   { value: "alimentacion", label: "Alimentación", icon: UtensilsCrossed },
45 %   { value: "amenities", label: "Amenities", icon: Star },
46 %   { value: "servicio", label: "Servicio", icon: Sparkles },
47 %   { value: "fecha_especial", label: "Fecha Especial", icon: Gift },
48 %   { value: "motivo_viaje", label: "Motivo de Viaje", icon: Briefcase },
49 %   { value: "nota_interna", label: "Nota Interna", icon: FileText },
50 %   { value: "otro", label: "Otro", icon: Heart },
51 % ];
52 %
53 % const SUBCATEGORY_MAP: Record<string, string[]> = {
54 %   habitacion: ["ubicacion", "temperatura", "almohadas", "cama", "vista", "piso", "otro"],
55 %   alimentacion: ["alergias", "dieta", "bebidas", "desayuno", "otro"],
56 %   amenities: ["almohadas", "toallas", "minibar", "aromaterapia", "otro"],
57 %   servicio: ["idioma", "horario", "transporte", "otro"],
58 %   fecha_especial: ["cumpleanos", "aniversario", "otro"],
59 %   motivo_viaje: ["negocios", "vacaciones", "evento", "otro"],
60 %   nota_interna: [],
61 %   otro: [],
62 % };
63 %
64 % const PRIORITY_OPTIONS = [
65 %   { value: "low", label: "Baja", color: "bg-gray-100 text-gray-700 dark:bg-gray-800 dark:text-gray-300" },
66 %   { value: "normal", label: "Normal", color: "bg-blue-100 text-blue-700 dark:bg-blue-900 dark:text-blue-300" },
67 %   { value: "high", label: "Alta", color: "bg-orange-100 text-orange-700 dark:bg-orange-900 dark:text-orange-300" },
68 %   { value: "critical", label: "Crítica", color: "bg-red-100 text-red-700 dark:bg-red-900 dark:text-red-300" },
69 % ];
70 %
71 % const AREA_OPTIONS = [
72 %   { value: "all", label: "Todas las áreas" },
73 %   { value: "reception", label: "Recepción" },
74 %   { value: "housekeeping", label: "Housekeeping" },
75 %   { value: "restaurant", label: "Restaurante" },
76 %   { value: "spa", label: "SPA" },
77 %   { value: "maintenance", label: "Mantenimiento" },
78 % ];
79 %
80 % function getPriorityBadge(priority: string) {
81 %   const opt = PRIORITY_OPTIONS.find((p) => p.value === priority);
82 %   return <Badge className={opt?.color || ""}>{opt?.label || priority}</Badge>;
83 % }
84 %
85 % function getCategoryIcon(category: string) {
86 %   const cat = CATEGORY_OPTIONS.find((c) => c.value === category);
87 %   if (!cat) return <Heart className="h-4 w-4" />;
88 %   const Icon = cat.icon;
89 %   return <Icon className="h-4 w-4" />;
90 % }
```

client/src/pages/hospitality.tsx (cont.)

```
91 %
92 % function getCategoryLabel(category: string) {
93 %   return CATEGORY_OPTIONS.find((c) => c.value === category)?.label || category;
94 % }
95 %
96 % // ===== DASHBOARD TAB =====
97 % function DashboardTab() {
98 %   const { data: dashboard, isLoading } = useQuery<any>({
99 %     queryKey: ["/api/hospitality/dashboard"],
100 %   });
101 %
102 %   if (isLoading) {
103 %     return (
104 %       <div className="flex items-center justify-center py-12">
105 %         <div className="animate-spin h-8 w-8 border-4 border-primary border-t-transparent rounded-full" />
106 %       </div>
107 %     );
108 %   }
109 %
110 %   const stats = dashboard?.stats || {};
111 %   const inHouseGuests = dashboard?.inHouseGuests || [];
112 %   const pendingAlerts = dashboard?.pendingAlerts || [];
113 %   const criticalPreferences = dashboard?.criticalPreferences || [];
114 %
115 %   return (
116 %     <div className="space-y-6">
117 %       <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
118 %         <Card data-testid="stat-inhouse-prefs">
119 %           <CardContent className="p-4 flex items-center gap-4">
120 %             <div className="p-3 rounded-full bg-blue-100 dark:bg-blue-900">
121 %               <User className="h-5 w-5 text-blue-600 dark:text-blue-400" />
122 %             </div>
123 %             <div>
124 %               <p className="text-2xl font-bold">{stats.totalInHouseWithPrefs || 0}</p>
125 %               <p className="text-sm text-muted-foreground">Huéspedes con preferencias</p>
126 %             </div>
127 %           </CardContent>
128 %         </Card>
129 %         <Card data-testid="stat-pending-alerts">
130 %           <CardContent className="p-4 flex items-center gap-4">
131 %             <div className="p-3 rounded-full bg-orange-100 dark:bg-orange-900">
132 %               <AlertTriangle className="h-5 w-5 text-orange-600 dark:text-orange-400" />
133 %             </div>
134 %             <div>
135 %               <p className="text-2xl font-bold">{stats.pendingAlertsCount || 0}</p>
136 %               <p className="text-sm text-muted-foreground">Alertas pendientes</p>
137 %             </div>
138 %           </CardContent>
139 %         </Card>
140 %         <Card data-testid="stat-critical">
141 %           <CardContent className="p-4 flex items-center gap-4">
142 %             <div className="p-3 rounded-full bg-red-100 dark:bg-red-900">
143 %               <ShieldAlert className="h-5 w-5 text-red-600 dark:text-red-400" />
144 %             </div>
145 %             <div>
146 %               <p className="text-2xl font-bold">{stats.criticalCount || 0}</p>
147 %               <p className="text-sm text-muted-foreground">Preferencias críticas</p>
148 %             </div>
149 %           </CardContent>
150 %         </Card>
151 %       </div>
152 %
153 %       {criticalPreferences.length > 0 && (
154 %         <Card className="border-red-200 dark:border-red-800">
155 %           <CardHeader className="pb-3">
156 %             <CardTitle className="text-base flex items-center gap-2 text-red-600 dark:text-red-400">
157 %               <ShieldAlert className="h-5 w-5" />
158 %               Preferencias Críticas (requieren atención inmediata)
159 %             </CardTitle>
160 %           </CardHeader>
161 %           <CardContent>
162 %             <div className="space-y-3">
163 %               {criticalPreferences.map((item: any, idx: number) => (
164 %                 <div key={idx} className="flex items-start gap-3 p-3 bg-red-50 dark:bg-red-950/30 rounded-lg data-te...>
165 %                   <AlertTriangle className="h-5 w-5 text-red-500 mt-0.5 flex-shrink-0" />
166 %                   <div>
167 %                     <p className="font-medium">{item.guest?.firstName} {item.guest?.lastName}</p>
168 %                     {item.preferences?.map((p: GuestPreference) => (
169 %                       <p key={p.id} className="text-sm text-red-700 dark:text-red-300">{p.title}: {p.description}</p>
170 %                     ))}
171 %                   </div>
172 %                 </div>
173 %               ))}
174 %             </div>
175 %           </CardContent>
176 %         </Card>
177 %       )}
178 %
179 %       {pendingAlerts.length > 0 && (
180 %         <Card>
181 %           <CardHeader className="pb-3">
182 %             <CardTitle className="text-base flex items-center gap-2">
183 %               <AlertTriangle className="h-5 w-5 text-orange-500" />
```

client/src/pages/hospitality.tsx (cont.)

```
184 %         Alertas Pendientes ({pendingAlerts.length})
185 %       </CardTitle>
186 %     </CardHeader>
187 %     <CardContent>
188 %       <AlertsList alerts={pendingAlerts} />
189 %     </CardContent>
190 %   </Card>
191 % })
192 %
193 % <Card>
194 %   <CardHeader className="pb-3">
195 %     <CardTitle className="text-base flex items-center gap-2">
196 %       <User className="h-5 w-5" />
197 %       Huéspedes In-House con Preferencias
198 %     </CardTitle>
199 %   </CardHeader>
200 %   <CardContent>
201 %     {inHouseGuests.length === 0 ? (
202 %       <p className="text-muted-foreground text-sm text-center py-6">No hay huéspedes registrados con preferenci...
203 %     ) : (
204 %       <div className="space-y-3">
205 %         {inHouseGuests.map((item: any, idx: number) => (
206 %           <div key={idx} className="flex items-start justify-between p-3 border rounded-lg" data-testid={`inhou...
207 %             <div className="flex items-start gap-3">
208 %               <div className="w-10 h-10 rounded-full bg-primary/10 flex items-center justify-center text-primar...
209 %                 {item.guest?.firstName?.[0]}{item.guest?.lastName?.[0]}
210 %               </div>
211 %               <div>
212 %                 <p className="font-medium">{item.guest?.firstName} {item.guest?.lastName}</p>
213 %                 <p className="text-xs text-muted-foreground">Hab. {item.reservation?.roomId}</p>
214 %                 <div className="flex flex-wrap gap-1 mt-1">
215 %                   {item.preferences?.map((p: GuestPreference) => (
216 %                     <Badge key={p.id} variant="outline" className="text-xs">
217 %                       {getCategoryIcon(p.category)}
218 %                       <span className="ml-1">{p.title}</span>
219 %                     </Badge>
220 %                   )
221 %                 </div>
222 %               </div>
223 %             </div>
224 %             <div className="flex gap-1">
225 %               {item.hasCritical && <Badge className="bg-red-100 text-red-700 dark:bg-red-900 dark:text-red-300"...
226 %               {item.hasHigh && !item.hasCritical && <Badge className="bg-orange-100 text-orange-700 dark:bg-ora...
227 %             </div>
228 %           </div>
229 %         )
230 %       </div>
231 %     )
232 %   </CardContent>
233 % </Card>
234 % </div>
235 % );
236 % }
237 %
238 % // ===== ALERTS LIST COMPONENT =====
239 % function AlertsList({ alerts }: { alerts: HospitalityAlert[] }) {
240 %   const { toast } = useToast();
241 %   const acknowledgeMutation = useMutation({
242 %     mutationFn: async (alertId: number) => {
243 %       await apiRequest("PATCH", `/api/hospitality/alerts/${alertId}/acknowledge`, { acknowledgedBy: "Recepción" });
244 %     },
245 %     onSuccess: () => {
246 %       queryClient.invalidateQueries({ queryKey: ["/api/hospitality/dashboard"] });
247 %       queryClient.invalidateQueries({ queryKey: ["/api/hospitality/alerts"] });
248 %       toast({ title: "Alerta confirmada" });
249 %     },
250 %   });
251 %
252 %   if (alerts.length === 0) {
253 %     return <p className="text-muted-foreground text-sm text-center py-4">Sin alertas pendientes</p>;
254 %   }
255 %
256 %   return (
257 %     <div className="space-y-2">
258 %       {alerts.map((alert) => (
259 %         <div key={alert.id} className="flex items-start justify-between gap-3 p-3 border rounded-lg" data-testid={`al...
260 %           <div className="flex items-start gap-2">
261 %             {getPriorityBadge(alert.priority)}
262 %             <div>
263 %               <p className="text-sm">{alert.alertMessage}</p>
264 %               <p className="text-xs text-muted-foreground mt-1">
265 %                 Área: {AREA_OPTIONS.find((a) => a.value === alert.targetArea)?.label || alert.targetArea}
266 %               </p>
267 %             </div>
268 %           </div>
269 %           {!alert.isAcknowledged && (
270 %             <Button
271 %               size="sm"
272 %               variant="outline"
273 %               onClick={() => acknowledgeMutation.mutate(alert.id)}
274 %               disabled={acknowledgeMutation.isPending}
275 %               data-testid={`btn-acknowledge-${alert.id}`}
276 %             >
```

client/src/pages/hospitality.tsx (cont.)

```
277 %         <Check className="h-4 w-4 mr-1" />
278 %         Entendido
279 %       </Button>
280 %     )}
281 %     {alert.isAcknowledged && (
282 %       <Badge variant="outline" className="text-green-600">
283 %         <CheckCircle className="h-3 w-3 mr-1" />
284 %         Confirmada
285 %       </Badge>
286 %     )}
287 %   </div>
288 % )}}
289 % </div>
290 % );
291 % }
292 %
293 % // ===== PREFERENCE FORM DIALOG =====
294 % function PreferenceFormDialog({
295 %   open,
296 %   onClose,
297 %   guestId,
298 %   editPref,
299 % }: {
300 %   open: boolean;
301 %   onClose: () => void;
302 %   guestId: string;
303 %   editPref?: GuestPreference | null;
304 % }) {
305 %   const { toast } = useToast();
306 %   const [category, setCategory] = useState(editPref?.category || "");
307 %   const [subcategory, setSubcategory] = useState(editPref?.subcategory || "");
308 %   const [title, setTitle] = useState(editPref?.title || "");
309 %   const [description, setDescription] = useState(editPref?.description || "");
310 %   const [priority, setPriority] = useState(editPref?.priority || "normal");
311 %   const [visibleTo, setVisibleTo] = useState<string[]>(editPref?.visibleTo || ["all"]);
312 %
313 %   const subcategories = SUBCATEGORY_MAP[category] || [];
314 %
315 %   const createMutation = useMutation({
316 %     mutationFn: async (data: any) => {
317 %       if (editPref) {
318 %         await apiRequest("PATCH", `/api/guests/${guestId}/preferences/${editPref.id}`, data);
319 %       } else {
320 %         await apiRequest("POST", `/api/guests/${guestId}/preferences`, data);
321 %       }
322 %     },
323 %     onSuccess: () => {
324 %       queryClient.invalidateQueries({ queryKey: ["/api/guests", guestId, "preferences"] });
325 %       queryClient.invalidateQueries({ queryKey: ["/api/hospitality/dashboard"] });
326 %       toast({ title: editPref ? "Preferencia actualizada" : "Preferencia creada" });
327 %       onClose();
328 %     },
329 %   });
330 %
331 %   const handleSubmit = () => {
332 %     if (!category || !title) {
333 %       toast({ title: "Completa los campos requeridos", variant: "destructive" });
334 %       return;
335 %     }
336 %     createMutation.mutate({
337 %       category,
338 %       subcategory: subcategory || null,
339 %       title,
340 %       description: description || null,
341 %       priority,
342 %       visibleTo,
343 %       recordedBy: "Recepción",
344 %     });
345 %   };
346 %
347 %   const toggleArea = (area: string) => {
348 %     if (area === "all") {
349 %       setVisibleTo(["all"]);
350 %     } else {
351 %       const filtered = visibleTo.filter((a) => a !== "all");
352 %       if (filtered.includes(area)) {
353 %         const result = filtered.filter((a) => a !== area);
354 %         setVisibleTo(result.length === 0 ? ["all"] : result);
355 %       } else {
356 %         setVisibleTo([...filtered, area]);
357 %       }
358 %     }
359 %   };
360 %
361 %   return (
362 %     <Dialog open={open} onOpenChange={onClose}>
363 %       <DialogContent className="max-w-lg">
364 %         <DialogHeader>
365 %           <DialogTitle>{editPref ? "Editar Preferencia" : "Nueva Preferencia"}</DialogTitle>
366 %         </DialogHeader>
367 %         <div className="space-y-4">
368 %           <div className="grid grid-cols-2 gap-4">
369 %             <div>
```

client/src/pages/hospitality.tsx (cont.)

```
370 %         <Label>Categoría *</Label>
371 %         <Select value={category} onChange={(v) => { setCategory(v); setSubcategory(""); }}>
372 %           <SelectTrigger data-testid="select-category">
373 %             <SelectValue placeholder="Seleccionar..." />
374 %           </SelectTrigger>
375 %           <SelectContent>
376 %             {CATEGORY_OPTIONS.map((c) => (
377 %               <SelectItem key={c.value} value={c.value}>{c.label}</SelectItem>
378 %             ))}
379 %           </SelectContent>
380 %         </Select>
381 %       </div>
382 %     <div>
383 %       <Label>Subcategoría</Label>
384 %       <Select value={subcategory} onChange={setSubcategory} disabled={subcategories.length === 0}>
385 %         <SelectTrigger data-testid="select-subcategory">
386 %           <SelectValue placeholder="Opcional..." />
387 %         </SelectTrigger>
388 %         <SelectContent>
389 %           {subcategories.map((s) => (
390 %             <SelectItem key={s} value={s}>{s.charAt(0).toUpperCase() + s.slice(1)}</SelectItem>
391 %           ))}
392 %         </SelectContent>
393 %       </Select>
394 %     </div>
395 %   </div>
396 %   <div>
397 %     <Label>Título *</Label>
398 %     <Input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="Ej: Alergia al maní" data-te...
399 %   </div>
400 %   <div>
401 %     <Label>Descripción</Label>
402 %     <Textarea value={description} onChange={(e) => setDescription(e.target.value)} placeholder="Detalles adic...
403 %   </div>
404 %   <div>
405 %     <Label>Prioridad</Label>
406 %     <Select value={priority} onChange={setPriority}>
407 %       <SelectTrigger data-testid="select-priority">
408 %         <SelectValue />
409 %       </SelectTrigger>
410 %       <SelectContent>
411 %         {PRIORITY_OPTIONS.map((p) => (
412 %           <SelectItem key={p.value} value={p.value}>{p.label}</SelectItem>
413 %         ))}
414 %       </SelectContent>
415 %     </Select>
416 %   </div>
417 %   <div>
418 %     <Label>Visible para</Label>
419 %     <div className="flex flex-wrap gap-2 mt-1">
420 %       {AREA_OPTIONS.map((a) => (
421 %         <Badge
422 %           key={a.value}
423 %           variant={visibleTo.includes(a.value) ? "default" : "outline"}
424 %           className="cursor-pointer"
425 %           onClick={() => toggleArea(a.value)}
426 %           data-testid={`toggle-area-${a.value}`}
427 %         >
428 %           {a.label}
429 %         </Badge>
430 %       ))}
431 %     </div>
432 %   </div>
433 % </div>
434 % <DialogFooter>
435 %   <Button variant="outline" onClick={onClose}>Cancelar</Button>
436 %   <Button onClick={handleSubmit} disabled={createMutation.isPending} data-testid="btn-save-preference">
437 %     {createMutation.isPending ? "Guardando..." : editPref ? "Actualizar" : "Crear"}
438 %   </Button>
439 % </DialogFooter>
440 % </DialogContent>
441 % </Dialog>
442 % );
443 % }
444 %
445 % // ===== PREFERENCES TAB =====
446 % function PreferencesTab() {
447 %   const { toast } = useToast();
448 %   const [search, setSearch] = useState("");
449 %   const [selectedGuest, setSelectedGuest] = useState<Guest | null>(null);
450 %   const [showForm, setShowForm] = useState(false);
451 %   const [editPref, setEditPref] = useState<GuestPreference | null>(null);
452 %
453 %   const { data: guests = [] } = useQuery<Guest[]>({ queryKey: ["api/guests"] });
454 %   const { data: preferences = [], isLoading: prefsLoading } = useQuery<GuestPreference[]>({
455 %     queryKey: ["api/guests", selectedGuest?.id, "preferences"],
456 %     enabled: !!selectedGuest,
457 %   });
458 %
459 %   const toggleMutation = useMutation({
460 %     mutationFn: async (prefId: number) => {
461 %       await apiRequest("PATCH", `/api/guests/${selectedGuest?.id}/preferences/${prefId}/toggle`);
462 %     },
```

client/src/pages/hospitality.tsx (cont.)

```
463 %     onSuccess: () => {
464 %       queryClient.invalidateQueries({ queryKey: ["/api/guests", selectedGuest?.id, "preferences"] });
465 %       toast({ title: "Preferencia actualizada" });
466 %     },
467 %   });
468 %
469 %   const deleteMutation = useMutation({
470 %     mutationFn: async (prefId: number) => {
471 %       await apiRequest("DELETE", `/api/guests/${selectedGuest?.id}/preferences/${prefId}`);
472 %     },
473 %     onSuccess: () => {
474 %       queryClient.invalidateQueries({ queryKey: ["/api/guests", selectedGuest?.id, "preferences"] });
475 %       toast({ title: "Preferencia eliminada" });
476 %     },
477 %   });
478 %
479 %   const filteredGuests = guests.filter((g) => {
480 %     const q = search.toLowerCase();
481 %     return (
482 %       g.firstName.toLowerCase().includes(q) ||
483 %       g.lastName.toLowerCase().includes(q) ||
484 %       (g.documentNumber && g.documentNumber.toLowerCase().includes(q))
485 %     );
486 %   });
487 %
488 %   const groupedPrefs = preferences.reduce((acc: Record<string, GuestPreference[]>, pref) => {
489 %     const cat = pref.category;
490 %     if (!acc[cat]) acc[cat] = [];
491 %     acc[cat].push(pref);
492 %     return acc;
493 %   }, {});
494 %
495 %   return (
496 %     <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
497 %       <Card className="lg:col-span-1">
498 %         <CardHeader className="pb-3">
499 %           <CardTitle className="text-base">Buscar Huésped</CardTitle>
500 %         </CardHeader>
501 %         <CardContent>
502 %           <div className="relative mb-3">
503 %             <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
504 %             <Input
505 %               placeholder="Nombre, apellido o documento..."
506 %               value={search}
507 %               onChange={(e) => setSearch(e.target.value)}
508 %               className="pl-9"
509 %               data-testid="input-search-guest"
510 %             />
511 %           </div>
512 %           <ScrollArea className="h-[400px]">
513 %             <div className="space-y-1">
514 %               {filteredGuests.map((guest) => (
515 %                 <button
516 %                   key={guest.id}
517 %                   className={`w-full text-left p-3 rounded-lg transition-colors ${
518 %                     selectedGuest?.id === guest.id
519 %                       ? "bg-primary/10 border border-primary/30"
520 %                       : "hover:bg-muted"
521 %                   }}`
522 %                   onClick={() => setSelectedGuest(guest)}
523 %                   data-testid={`btn-select-guest-${guest.id}`}
524 %                 >
525 %                   <p className="font-medium text-sm">{guest.firstName} {guest.lastName}</p>
526 %                   <p className="text-xs text-muted-foreground">{guest.documentType?.toUpperCase()} {guest.documentNum...
527 %                 </button>
528 %               )})
529 %             </div>
530 %           </ScrollArea>
531 %         </CardContent>
532 %       </Card>
533 %
534 %       <Card className="lg:col-span-2">
535 %         <CardHeader className="pb-3 flex flex-row items-center justify-between">
536 %           <CardTitle className="text-base">
537 %             {selectedGuest
538 %               ? `Preferencias de ${selectedGuest.firstName} ${selectedGuest.lastName}`
539 %               : "Selecciona un huésped"}
540 %           </CardTitle>
541 %           {selectedGuest && (
542 %             <Button size="sm" onClick={() => { setEditPref(null); setShowForm(true); }} data-testid="btn-add-preferen...
543 %               <Plus className="h-4 w-4 mr-1" />
544 %               Agregar
545 %             </Button>
546 %           )}
547 %         </CardHeader>
548 %         <CardContent>
549 %           {!selectedGuest ? (
550 %             <div className="text-center py-12 text-muted-foreground">
551 %               <Heart className="h-12 w-12 mx-auto mb-3 opacity-30" />
552 %               <p>Selecciona un huésped para ver y gestionar sus preferencias</p>
553 %             </div>
554 %           ) : prefsLoading ? (
555 %             <div className="flex justify-center py-8">
```

client/src/pages/hospitality.tsx (cont.)

```
556 %         <div className="animate-spin h-6 w-6 border-3 border-primary border-t-transparent rounded-full" />
557 %       </div>
558 %     ) : preferences.length === 0 ? (
559 %       <div className="text-center py-12 text-muted-foreground">
560 %         <p>Este huésped no tiene preferencias registradas</p>
561 %         <Button size="sm" className="mt-3" onClick={() => { setEditPref(null); setShowForm(true); }}>
562 %           <Plus className="h-4 w-4 mr-1" />
563 %           Agregar primera preferencia
564 %         </Button>
565 %       </div>
566 %     ) : (
567 %       <ScrollArea className="h-[400px]">
568 %         <div className="space-y-4">
569 %           {Object.entries(groupedPrefs).map(([cat, prefs]) => (
570 %             <div key={cat}>
571 %               <div className="flex items-center gap-2 mb-2">
572 %                 {getCategoryIcon(cat)}
573 %                 <h4 className="font-medium text-sm">{getCategoryLabel(cat)}</h4>
574 %                 <Badge variant="outline" className="text-xs">{prefs.length}</Badge>
575 %               </div>
576 %               <div className="space-y-2 ml-6">
577 %                 {prefs.map((pref) => (
578 %                   <div
579 %                     key={pref.id}
580 %                     className={`flex items-start justify-between p-3 border rounded-lg ${
581 %                       !pref.isActive ? "opacity-50" : ""
582 %                     }`}
583 %                     data-testid={`pref-item-${pref.id}`}
584 %                   >
585 %                     <div className="flex-1">
586 %                       <div className="flex items-center gap-2">
587 %                         <p className="font-medium text-sm">{pref.title}</p>
588 %                         {getPriorityBadge(pref.priority)}
589 %                         {!pref.isActive && <Badge variant="outline" className="text-xs">Inactiva</Badge>}
590 %                       </div>
591 %                       {pref.description && <p className="text-xs text-muted-foreground mt-1">{pref.description}>...
592 %                     <div className="flex gap-1 mt-1">
593 %                       {pref.visibleTo?.map((area) => (
594 %                         <Badge key={area} variant="outline" className="text-xs">{area}</Badge>
595 %                       ))}
596 %                     </div>
597 %                   </div>
598 %                   <div className="flex gap-1 ml-2">
599 %                     <Button
600 %                       size="icon"
601 %                       variant="ghost"
602 %                       className="h-7 w-7"
603 %                       onClick={() => toggleMutation.mutate(pref.id)}
604 %                       data-testid={`btn-toggle-${pref.id}`}
605 %                     >
606 %                       {pref.isActive ? <ToggleRight className="h-4 w-4 text-green-500" /> : <ToggleLeft class...
607 %                     </Button>
608 %                     <Button
609 %                       size="icon"
610 %                       variant="ghost"
611 %                       className="h-7 w-7"
612 %                       onClick={() => { setEditPref(pref); setShowForm(true); }}
613 %                       data-testid={`btn-edit-${pref.id}`}
614 %                     >
615 %                       <Edit className="h-4 w-4" />
616 %                     </Button>
617 %                     <Button
618 %                       size="icon"
619 %                       variant="ghost"
620 %                       className="h-7 w-7 text-destructive"
621 %                       onClick={() => deleteMutation.mutate(pref.id)}
622 %                       data-testid={`btn-delete-${pref.id}`}
623 %                     >
624 %                       <Trash2 className="h-4 w-4" />
625 %                     </Button>
626 %                   </div>
627 %                 </div>
628 %               )})
629 %             </div>
630 %           </div>
631 %         </div>
632 %       </ScrollArea>
633 %     )}
634 %   </CardContent>
635 % </Card>
636 %
637 % {showForm && selectedGuest && (
638 %   <PreferenceFormDialog
639 %     open={showForm}
640 %     onClose={() => { setShowForm(false); setEditPref(null); }}
641 %     guestId={selectedGuest.id}
642 %     editPref={editPref}
643 %   />
644 % )}
645 % </div>
646 % );
647 % }
648 % }
```


client/src/pages/hospitality.tsx (cont.)

```
649 %
650 % // ===== STAY NOTES TAB =====
651 % function StayNotesTab() {
652 %   const { toast } = useToast();
653 %   const [filterArea, setFilterArea] = useState("all");
654 %   const [showResolved, setShowResolved] = useState(false);
655 %
656 %   const { data: notes = [], isLoading } = useQuery<any[]>({
657 %     queryKey: ["/api/hospitality/stay-notes/active", showResolved],
658 %     queryFn: async () => {
659 %       const url = showResolved
660 %         ? "/api/hospitality/stay-notes/active?includeResolved=true"
661 %         : "/api/hospitality/stay-notes/active";
662 %       const res = await fetch(url);
663 %       if (!res.ok) throw new Error("Failed to fetch");
664 %       return res.json();
665 %     },
666 %   });
667 %
668 %   const { data: allReservations = [] } = useQuery<any[]>({
669 %     queryKey: ["/api/reservations"],
670 %   });
671 %
672 %   const { data: guests = [] } = useQuery<Guest[]>({
673 %     queryKey: ["/api/guests"],
674 %   });
675 %
676 %   const [showNoteForm, setShowNoteForm] = useState(false);
677 %   const [selectedReservation, setSelectedReservation] = useState("");
678 %   const [noteCategory, setNoteCategory] = useState("");
679 %   const [noteTitle, setNoteTitle] = useState("");
680 %   const [noteDescription, setNoteDescription] = useState("");
681 %   const [notePriority, setNotePriority] = useState("normal");
682 %
683 %   const checkedInReservations = allReservations.filter((r) => r.status === "checked_in");
684 %
685 %   const resolveMutation = useMutation({
686 %     mutationFn: async (noteId: number) => {
687 %       await apiRequest("PATCH", `/api/hospitality/stay-notes/${noteId}/resolve`, { resolvedBy: "Recepción" });
688 %     },
689 %     onSuccess: () => {
690 %       queryClient.invalidateQueries({ queryKey: ["/api/hospitality/stay-notes/active"] });
691 %       toast({ title: "Nota resuelta" });
692 %     },
693 %   });
694 %
695 %   const createNoteMutation = useMutation({
696 %     mutationFn: async (data: any) => {
697 %       await apiRequest("POST", `/api/reservations/${data.reservationId}/stay-notes`, data);
698 %     },
699 %     onSuccess: () => {
700 %       queryClient.invalidateQueries({ queryKey: ["/api/hospitality/stay-notes/active"] });
701 %       toast({ title: "Nota creada" });
702 %       setShowNoteForm(false);
703 %       setNoteTitle("");
704 %       setNoteDescription("");
705 %       setNoteCategory("");
706 %       setSelectedReservation("");
707 %     },
708 %   });
709 %
710 %   const filteredNotes = notes.filter((n) => {
711 %     if (!showResolved && n.isResolved) return false;
712 %     if (filterArea !== "all" && !n.visibleTo?.includes(filterArea) && !n.visibleTo?.includes("all")) return false;
713 %     return true;
714 %   });
715 %
716 %   const getGuestName = (guestId: string) => {
717 %     const g = guests.find((g) => g.id === guestId);
718 %     return g ? `${g.firstName} ${g.lastName}` : "Desconocido";
719 %   };
720 %
721 %   return (
722 %     <div className="space-y-4">
723 %       <div className="flex items-center justify-between gap-4">
724 %         <div className="flex items-center gap-3">
725 %           <Select value={filterArea} onChange={setFilterArea}>
726 %             <SelectTrigger className="w-48" data-testid="select-filter-area">
727 %               <SelectValue />
728 %             </SelectTrigger>
729 %             <SelectContent>
730 %               {AREA_OPTIONS.map((a) => (
731 %                 <SelectItem key={a.value} value={a.value}>{a.label}</SelectItem>
732 %               ))}
733 %             </SelectContent>
734 %           </Select>
735 %           <div className="flex items-center gap-2">
736 %             <Switch checked={showResolved} onCheckedChange={setShowResolved} data-testid="switch-show-resolved" />
737 %             <Label className="text-sm">Mostrar resueltas</Label>
738 %           </div>
739 %         </div>
740 %         <Button onClick={() => setShowNoteForm(true)} data-testid="btn-add-note">
741 %           <Plus className="h-4 w-4 mr-1" />
```

```

742 %         Nueva Nota
743 %       </Button>
744 %     </div>
745 %
746 %     {isLoading ? (
747 %       <div className="flex justify-center py-8">
748 %         <div className="animate-spin h-6 w-6 border-3 border-primary border-t-transparent rounded-full" />
749 %       </div>
750 %     ) : filteredNotes.length === 0 ? (
751 %       <Card>
752 %         <CardContent className="text-center py-12 text-muted-foreground">
753 %           <FileText className="h-12 w-12 mx-auto mb-3 opacity-30" />
754 %           <p>No hay notas de estadia activas</p>
755 %         </CardContent>
756 %       </Card>
757 %     ) : (
758 %       <div className="space-y-3">
759 %         {filteredNotes.map((note) => (
760 %           <Card key={note.id} className={note.isResolved ? "opacity-60" : ""} data-testid={`note-${note.id}`}>
761 %             <CardContent className="p-4">
762 %               <div className="flex items-start justify-between">
763 %                 <div className="flex items-start gap-3">
764 %                   {getCategoryIcon(note.category)}
765 %                   <div>
766 %                     <div className="flex items-center gap-2">
767 %                       <p className="font-medium text-sm">{note.title}</p>
768 %                       {getPriorityBadge(note.priority)}
769 %                       {note.isResolved && (
770 %                         <Badge variant="outline" className="text-green-600 text-xs">
771 %                           <CheckCircle className="h-3 w-3 mr-1" />
772 %                           Resuelta
773 %                         </Badge>
774 %                       )}
775 %                     </div>
776 %                     {note.description && <p className="text-xs text-muted-foreground mt-1">{note.description}</p>}
777 %                     <p className="text-xs text-muted-foreground mt-1">
778 %                       {note.guestId && <span>Huésped: {getGuestName(note.guestId)} . </span>}
779 %                       Reserva: {note.reservationId}
780 %                     </p>
781 %                   </div>
782 %                 </div>
783 %                 {!note.isResolved && (
784 %                   <Button
785 %                     size="sm"
786 %                     variant="outline"
787 %                     onClick={() => resolveMutation.mutate(note.id)}
788 %                     disabled={resolveMutation.isPending}
789 %                     data-testid={`btn-resolve-${note.id}`}
790 %                   >
791 %                     <Check className="h-4 w-4 mr-1" />
792 %                     Resolver
793 %                   </Button>
794 %                 )}
795 %               </div>
796 %             </CardContent>
797 %           </Card>
798 %         )}
799 %       </div>
800 %     )}
801 %
802 %     <Dialog open={showNoteForm} onOpenChange={setShowNoteForm}>
803 %       <DialogContent className="max-w-lg">
804 %         <DialogHeader>
805 %           <DialogTitle>Nueva Nota de Estadia</DialogTitle>
806 %         </DialogHeader>
807 %         <div className="space-y-4">
808 %           <div>
809 %             <Label>Reserva (check-in activo) *</Label>
810 %             <Select value={selectedReservation} onValueChange={setSelectedReservation}>
811 %               <SelectTrigger data-testid="select-reservation-note">
812 %                 <SelectValue placeholder="Seleccionar reserva..." />
813 %               </SelectTrigger>
814 %               <SelectContent>
815 %                 {checkedInReservations.map((r) => (
816 %                   <SelectItem key={r.id} value={r.id}>
817 %                     {r.guestId ? getGuestName(r.guestId) : "Sin huésped"} - Hab. {r.roomId}
818 %                   </SelectItem>
819 %                 ))}
820 %               </SelectContent>
821 %             </Select>
822 %           </div>
823 %           <div>
824 %             <Label>Categoría *</Label>
825 %             <Select value={noteCategory} onValueChange={setNoteCategory}>
826 %               <SelectTrigger data-testid="select-note-category">
827 %                 <SelectValue placeholder="Seleccionar..." />
828 %               </SelectTrigger>
829 %               <SelectContent>
830 %                 {CATEGORY_OPTIONS.map((c) => (
831 %                   <SelectItem key={c.value} value={c.value}>{c.label}</SelectItem>
832 %                 ))}
833 %               </SelectContent>
834 %             </Select>

```

client/src/pages/hospitality.tsx (cont.)

```
835 %         </div>
836 %     <div>
837 %         <Label>Título *</Label>
838 %         <Input value={noteTitle} onChange={(e) => setNoteTitle(e.target.value)} placeholder="Descripción breve"...
839 %     </div>
840 %     <div>
841 %         <Label>Descripción</Label>
842 %         <Textarea value={noteDescription} onChange={(e) => setNoteDescription(e.target.value)} placeholder="Det...
843 %     </div>
844 %     <div>
845 %         <Label>Prioridad</Label>
846 %         <Select value={notePriority} onValueChange={setNotePriority}>
847 %             <SelectTrigger data-testid="select-note-priority">
848 %                 <SelectValue />
849 %             </SelectTrigger>
850 %             <SelectContent>
851 %                 {PRIORITY_OPTIONS.map((p) => (
852 %                     <SelectItem key={p.value} value={p.value}>{p.label}</SelectItem>
853 %                 ))}
854 %             </SelectContent>
855 %         </Select>
856 %     </div>
857 % </div>
858 % <DialogFooter>
859 %     <Button variant="outline" onClick={() => setShowNoteForm(false)}>Cancelar</Button>
860 %     <Button
861 %         onClick={() => {
862 %             if (!selectedReservation || !noteCategory || !noteTitle) {
863 %                 toast({ title: "Completa los campos requeridos", variant: "destructive" });
864 %                 return;
865 %             }
866 %             const reservation = checkedInReservations.find((r) => r.id === selectedReservation);
867 %             createNoteMutation.mutate({
868 %                 reservationId: selectedReservation,
869 %                 guestId: reservation?.guestId || null,
870 %                 category: noteCategory,
871 %                 title: noteTitle,
872 %                 description: noteDescription || null,
873 %                 priority: notePriority,
874 %                 visibleTo: ["all"],
875 %                 recordedBy: "Recepción",
876 %             });
877 %         }}
878 %         disabled={createNoteMutation.isPending}
879 %         data-testid="btn-save-note"
880 %     >
881 %         {createNoteMutation.isPending ? "Guardando..." : "Crear Nota"}
882 %     </Button>
883 % </DialogFooter>
884 % </DialogContent>
885 % </Dialog>
886 % </div>
887 % );
888 % }
889 %
890 % // ===== HISTORY TAB =====
891 % function HistoryTab() {
892 %     const [search, setSearch] = useState("");
893 %     const { data: guests = [] } = useQuery<Guest[]>({ queryKey: ["api/guests"] });
894 %     const [selectedGuestId, setSelectedGuestId] = useState<string | null>(null);
895 %
896 %     const { data: preferences = [] } = useQuery<GuestPreference[]>({
897 %         queryKey: ["api/guests", selectedGuestId, "preferences"],
898 %         enabled: !!selectedGuestId,
899 %     });
900 %
901 %     const { data: alerts = [] } = useQuery<HospitalityAlert[]>({
902 %         queryKey: ["api/hospitality/alerts"],
903 %     });
904 %
905 %     const filteredGuests = guests.filter((g) => {
906 %         const q = search.toLowerCase();
907 %         return g.firstName.toLowerCase().includes(q) || g.lastName.toLowerCase().includes(q);
908 %     });
909 %
910 %     return (
911 %         <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
912 %             <Card>
913 %                 <CardHeader className="pb-3">
914 %                     <CardTitle className="text-base">Buscar Huésped</CardTitle>
915 %                 </CardHeader>
916 %                 <CardContent>
917 %                     <div className="relative mb-3">
918 %                         <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
919 %                         <Input
920 %                             placeholder="Buscar..."
921 %                             value={search}
922 %                             onChange={(e) => setSearch(e.target.value)}
923 %                             className="pl-9"
924 %                             data-testid="input-search-history"
925 %                         />
926 %                     </div>
927 %                     <ScrollArea className="h-[400px]">
```

client/src/pages/hospitality.tsx (cont.)

```
928 %         <div className="space-y-1">
929 %             {filteredGuests.map((g) => (
930 %                 <button
931 %                     key={g.id}
932 %                     className={`w-full text-left p-3 rounded-lg transition-colors ${
933 %                         selectedGuestId === g.id ? "bg-primary/10 border border-primary/30" : "hover:bg-muted"
934 %                     }`}
935 %                     onClick={() => setSelectedGuestId(g.id)}
936 %                     data-testid={`btn-history-guest-${g.id}`}
937 %                 >
938 %                     <p className="font-medium text-sm">{g.firstName} {g.lastName}</p>
939 %                 </button>
940 %             )]}
941 %         </div>
942 %     </ScrollArea>
943 % </CardContent>
944 % </Card>
945 %
946 % <Card className="lg:col-span-2">
947 %   <CardHeader className="pb-3">
948 %     <CardTitle className="text-base">
949 %       {selectedGuestId ? "Historial de Preferencias y Alertas" : "Selecciona un huésped"}
950 %     </CardTitle>
951 %   </CardHeader>
952 %   <CardContent>
953 %     {!selectedGuestId ? (
954 %       <div className="text-center py-12 text-muted-foreground">
955 %         <Clock className="h-12 w-12 mx-auto mb-3 opacity-30" />
956 %         <p>Selecciona un huésped para ver su historial</p>
957 %       </div>
958 %     ) : (
959 %       <ScrollArea className="h-[400px]">
960 %         <div className="space-y-4">
961 %           {preferences.length > 0 && (
962 %             <div>
963 %               <h4 className="font-medium text-sm mb-2 flex items-center gap-2">
964 %                 <Heart className="h-4 w-4" />
965 %                 Preferencias ({preferences.length})
966 %               </h4>
967 %               <div className="space-y-2 ml-6">
968 %                 {preferences.map((p) => (
969 %                   <div key={p.id} className="p-3 border rounded-lg" data-testid={`history-pref-${p.id}`}>
970 %                     <div className="flex items-center gap-2">
971 %                       {getCategoryIcon(p.category)}
972 %                       <span className="font-medium text-sm">{p.title}</span>
973 %                       {getPriorityBadge(p.priority)}
974 %                       <Badge variant={p.isActive ? "default" : "outline"} className="text-xs">
975 %                         {p.isActive ? "Activa" : "Inactiva"}
976 %                       </Badge>
977 %                     </div>
978 %                     {p.description && <p className="text-xs text-muted-foreground mt-1 ml-6">{p.description}</p>}
979 %                     <p className="text-xs text-muted-foreground mt-1 ml-6">
980 %                       Registrada por: {p.recordedBy || "-"} · {p.createdAt ? new Date(p.createdAt).toLocaleDate...
981 %                     </p>
982 %                   </div>
983 %                 )]}
984 %               </div>
985 %             </div>
986 %           )}
987 %         </div>
988 %       )}
989 %     {(() => {
990 %       const guestAlerts = alerts.filter((a) => a.guestId === selectedGuestId);
991 %       if (guestAlerts.length === 0) return null;
992 %       return (
993 %         <div>
994 %           <Separator className="my-4" />
995 %           <h4 className="font-medium text-sm mb-2 flex items-center gap-2">
996 %             <AlertTriangle className="h-4 w-4" />
997 %             Alertas generadas ({guestAlerts.length})
998 %           </h4>
999 %           <div className="space-y-2 ml-6">
1000 %             {guestAlerts.map((a) => (
1001 %               <div key={a.id} className="p-3 border rounded-lg" data-testid={`history-alert-${a.id}`}>
1002 %                 <div className="flex items-center gap-2">
1003 %                   {getPriorityBadge(a.priority)}
1004 %                   <span className="text-sm">{a.alertMessage}</span>
1005 %                   {a.isAcknowledged && (
1006 %                     <Badge variant="outline" className="text-green-600 text-xs">
1007 %                       <CheckCircle className="h-3 w-3 mr-1" />
1008 %                       Confirmada
1009 %                     </Badge>
1010 %                   )}
1011 %                 </div>
1012 %                 <p className="text-xs text-muted-foreground mt-1">
1013 %                   Área: {a.targetArea} · Reserva: {a.reservationId}
1014 %                 </p>
1015 %               </div>
1016 %             )]}
1017 %           </div>
1018 %         </div>
1019 %       )}()
1020 %     )}()
```

client/src/pages/hospitality.tsx (cont.)

```
1021 %             {preferences.length === 0 && alerts.filter((a) => a.guestId === selectedGuestId).length === 0 && (
1022 %                 <p className="text-center text-muted-foreground py-8">Sin historial registrado</p>
1023 %             )}
1024 %         </div>
1025 %     </ScrollArea>
1026 %   }}
1027 % </CardContent>
1028 % </Card>
1029 % </div>
1030 % );
1031 % }
1032 %
1033 % // ===== MAIN PAGE =====
1034 % export default function HospitalityPage() {
1035 %   return (
1036 %     <div className="p-6 space-y-6">
1037 %       <div className="flex items-center gap-3">
1038 %         <Heart className="h-7 w-7 text-primary" />
1039 %         <div>
1040 %           <h1 className="text-2xl font-bold" data-testid="text-page-title">Hospitalidad</h1>
1041 %           <p className="text-sm text-muted-foreground">Gestión de preferencias y experiencia del huésped</p>
1042 %         </div>
1043 %       </div>
1044 %
1045 %       <Tabs defaultValue="dashboard">
1046 %         <TabsList className="grid w-full max-w-2xl grid-cols-4" data-testid="hospitality-tabs">
1047 %           <TabsTrigger value="dashboard" data-testid="tab-dashboard">Dashboard</TabsTrigger>
1048 %           <TabsTrigger value="preferences" data-testid="tab-preferences">Preferencias</TabsTrigger>
1049 %           <TabsTrigger value="stay-notes" data-testid="tab-stay-notes">Notas de Estadia</TabsTrigger>
1050 %           <TabsTrigger value="history" data-testid="tab-history">Historial</TabsTrigger>
1051 %         </TabsList>
1052 %
1053 %         <TabsContent value="dashboard" className="mt-6">
1054 %           <DashboardTab />
1055 %         </TabsContent>
1056 %
1057 %         <TabsContent value="preferences" className="mt-6">
1058 %           <PreferencesTab />
1059 %         </TabsContent>
1060 %
1061 %         <TabsContent value="stay-notes" className="mt-6">
1062 %           <StayNotesTab />
1063 %         </TabsContent>
1064 %
1065 %         <TabsContent value="history" className="mt-6">
1066 %           <HistoryTab />
1067 %         </TabsContent>
1068 %       </Tabs>
1069 %     </div>
1070 %   );
1071 % }
1072 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { useToast } from "@hooks/use-toast";
5 % import type { SystemNotification } from "@shared/schema";
6 % import {
7 %   Bot,
8 %   Bell,
9 %   CheckCheck,
10 %   Filter,
11 %   Smartphone,
12 %   AlertTriangle,
13 %   ArrowUpCircle,
14 %   Clock,
15 %   DoorOpen,
16 %   UtensilsCrossed,
17 %   Flower2,
18 %   Wrench,
19 %   Sparkles,
20 %   Search,
21 %   RefreshCw,
22 %   Eye,
23 %   EyeOff,
24 %   MessageCircle,
25 %   User,
26 %   Hash,
27 % } from "lucide-react";
28 % import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@components/ui/card";
29 % import { Button } from "@components/ui/button";
30 % import { Badge } from "@components/ui/badge";
31 % import { Input } from "@components/ui/input";
32 % import { ScrollArea } from "@components/ui/scroll-area";
33 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
34 % import {
35 %   Select,
36 %   SelectContent,
37 %   SelectItem,
38 %   SelectTrigger,
39 %   SelectValue,
40 % } from "@components/ui/select";
41 % import { Separator } from "@components/ui/separator";
42 % import {
43 %   Tooltip,
44 %   TooltipContent,
45 %   TooltipTrigger,
46 % } from "@components/ui/tooltip";
47 %
48 % function formatTimeAgo(date: Date | string): string {
49 %   const now = new Date();
50 %   const d = typeof date === "string" ? new Date(date) : date;
51 %   const diffMs = now.getTime() - d.getTime();
52 %   const mins = Math.floor(diffMs / 60000);
53 %   if (mins < 1) return "ahora";
54 %   if (mins < 60) return `hace ${mins} min`;
55 %   const hours = Math.floor(mins / 60);
56 %   if (hours < 24) return `hace ${hours}h`;
57 %   const days = Math.floor(hours / 24);
58 %   return `hace ${days}d`;
59 % }
60 %
61 % function formatDate(date: Date | string): string {
62 %   const d = typeof date === "string" ? new Date(date) : date;
63 %   return d.toLocaleDateString("es-AR", {
64 %     day: "2-digit",
65 %     month: "2-digit",
66 %     year: "numeric",
67 %     hour: "2-digit",
68 %     minute: "2-digit",
69 %   });
70 % }
71 %
72 % function getAreaIcon(area: string) {
73 %   switch (area) {
74 %     case "housekeeping": return <Sparkles className="h-4 w-4" />;
75 %     case "maintenance": return <Wrench className="h-4 w-4" />;
76 %     case "restaurant": return <UtensilsCrossed className="h-4 w-4" />;
77 %     case "spa": return <Flower2 className="h-4 w-4" />;
78 %     case "reception": return <DoorOpen className="h-4 w-4" />;
79 %     default: return <Bell className="h-4 w-4" />;
80 %   }
81 % }
82 %
83 % function getAreaLabel(area: string): string {
84 %   const labels: Record<string, string> = {
85 %     reception: "Recepción",
86 %     housekeeping: "Housekeeping",
87 %     maintenance: "Mantenimiento",
88 %     restaurant: "Restaurante",
89 %     spa: "SPA",
90 %     all: "General",
```

client/src/pages/chatbot-dashboard.tsx (cont.)

```
91 % };
92 % return labels[area] || area;
93 % }
94 %
95 % function getAreaColor(area: string): string {
96 %   switch (area) {
97 %     case "housekeeping": return "bg-purple-100 text-purple-800 dark:bg-purple-900 dark:text-purple-200";
98 %     case "maintenance": return "bg-orange-100 text-orange-800 dark:bg-orange-900 dark:text-orange-200";
99 %     case "restaurant": return "bg-amber-100 text-amber-800 dark:bg-amber-900 dark:text-amber-200";
100 %     case "spa": return "bg-pink-100 text-pink-800 dark:bg-pink-900 dark:text-pink-200";
101 %     case "reception": return "bg-blue-100 text-blue-800 dark:bg-blue-900 dark:text-blue-200";
102 %     default: return "bg-gray-100 text-gray-800 dark:bg-gray-900 dark:text-gray-200";
103 %   }
104 % }
105 %
106 % function getPriorityBadge(priority: string) {
107 %   switch (priority) {
108 %     case "urgent":
109 %       return (
110 %         <Badge variant="destructive" className="text-xs gap-1">
111 %           <AlertTriangle className="h-3 w-3" />
112 %           Urgente
113 %         </Badge>
114 %       );
115 %     case "high":
116 %       return (
117 %         <Badge className="text-xs gap-1 bg-orange-500 hover:bg-orange-600">
118 %           <ArrowUpCircle className="h-3 w-3" />
119 %           Alta
120 %         </Badge>
121 %       );
122 %     default:
123 %       return null;
124 %   }
125 % }
126 %
127 % export default function ChatbotDashboardPage() {
128 %   const { toast } = useToast();
129 %   const [areaFilter, setAreaFilter] = useState<string>("all");
130 %   const [searchTerm, setSearchTerm] = useState("");
131 %   const [showRead, setShowRead] = useState(false);
132 %   const [selectedMessage, setSelectedMessage] = useState<SystemNotification | null>(null);
133 %
134 %   const { data: notifications = [], isLoading } = useQuery<SystemNotification[]>({
135 %     queryKey: ["/api/notifications", "chatbot"],
136 %     queryFn: async () => {
137 %       const res = await apiRequest("GET", "/api/notifications?limit=200");
138 %       const all: SystemNotification[] = await res.json();
139 %       return all.filter((n) => n.type.startsWith("chatbot_"));
140 %     },
141 %     refetchInterval: 15000,
142 %   });
143 %
144 %   const markReadMutation = useMutation({
145 %     mutationFn: (id: number) => apiRequest("PATCH", `/api/notifications/${id}/read`),
146 %     onSuccess: () => {
147 %       queryClient.invalidateQueries({ queryKey: ["/api/notifications"] });
148 %     },
149 %   });
150 %
151 %   const markAllReadMutation = useMutation({
152 %     mutationFn: () => apiRequest("PATCH", "/api/notifications/read-all"),
153 %     onSuccess: () => {
154 %       queryClient.invalidateQueries({ queryKey: ["/api/notifications"] });
155 %       toast({ title: "Todos los mensajes marcados como leídos" });
156 %     },
157 %   });
158 %
159 %   const filteredMessages = notifications.filter((n) => {
160 %     if (areaFilter !== "all" && n.targetArea !== areaFilter) return false;
161 %     if (!showRead && n.isRead) return false;
162 %     if (searchTerm) {
163 %       const term = searchTerm.toLowerCase();
164 %       return (
165 %         n.title.toLowerCase().includes(term) ||
166 %         n.message.toLowerCase().includes(term) ||
167 %         (n.relatedEntityId && n.relatedEntityId.toLowerCase().includes(term))
168 %       );
169 %     }
170 %     return true;
171 %   });
172 %
173 %   const unreadCount = notifications.filter((n) => !n.isRead).length;
174 %   const unreadByArea = notifications.reduce((acc, n) => {
175 %     if (!n.isRead) {
176 %       acc[n.targetArea] = (acc[n.targetArea] || 0) + 1;
177 %     }
178 %     return acc;
179 %   }, {} as Record<string, number>);
180 %
181 %   const areaStats = notifications.reduce((acc, n) => {
182 %     acc[n.targetArea] = (acc[n.targetArea] || 0) + 1;
183 %     return acc;
```

client/src/pages/chatbot-dashboard.tsx (cont.)

```
184 %   }, {} as Record<string, number>);
185 %
186 %   const handleSelectMessage = (msg: SystemNotification) => {
187 %     setSelectedMessage(msg);
188 %     if (!msg.isRead) {
189 %       markReadMutation.mutate(msg.id);
190 %     }
191 %   };
192 %
193 %   return (
194 %     <div className="p-6 space-y-6 max-w-[1400px] mx-auto" data-testid="chatbot-dashboard">
195 %       <div className="flex items-center justify-between">
196 %         <div className="flex items-center gap-3">
197 %           <div className="flex h-10 w-10 items-center justify-center rounded-lg bg-blue-100 dark:bg-blue-900">
198 %             <Bot className="h-6 w-6 text-blue-600 dark:text-blue-300" />
199 %           </div>
200 %           <div>
201 %             <h1 className="text-2xl font-bold" data-testid="text-page-title">MARA Chatbot</h1>
202 %             <p className="text-sm text-muted-foreground">
203 %               Mensajes y solicitudes recibidas del chatbot
204 %             </p>
205 %           </div>
206 %         </div>
207 %         <div className="flex items-center gap-2">
208 %           {unreadCount > 0 && (
209 %             <Badge variant="secondary" className="text-sm" data-testid="badge-unread-count">
210 %               {unreadCount} sin leer
211 %             </Badge>
212 %           )}
213 %           <Button
214 %             variant="outline"
215 %             size="sm"
216 %             onClick={() => queryClient.invalidateQueries({ queryKey: ["/api/notifications"] })}
217 %             data-testid="button-refresh"
218 %           >
219 %             <RefreshCw className="h-4 w-4 mr-1" />
220 %             Actualizar
221 %           </Button>
222 %           {unreadCount > 0 && (
223 %             <Button
224 %               variant="outline"
225 %               size="sm"
226 %               onClick={() => markAllReadMutation.mutate()}
227 %               disabled={markAllReadMutation.isPending}
228 %               data-testid="button-mark-all-read"
229 %             >
230 %               <CheckCheck className="h-4 w-4 mr-1" />
231 %               Marcar todas leídas
232 %             </Button>
233 %           )}
234 %         </div>
235 %       </div>
236 %
237 %       <div className="grid grid-cols-2 md:grid-cols-4 lg:grid-cols-6 gap-3">
238 %         {[
239 %           { area: "all", label: "Todos", icon: <MessageCircle className="h-4 w-4" /> },
240 %           { area: "housekeeping", label: "Housekeeping", icon: <Sparkles className="h-4 w-4" /> },
241 %           { area: "maintenance", label: "Mantenimiento", icon: <Wrench className="h-4 w-4" /> },
242 %           { area: "restaurant", label: "Restaurante", icon: <UtensilsCrossed className="h-4 w-4" /> },
243 %           { area: "spa", label: "SPA", icon: <Flower2 className="h-4 w-4" /> },
244 %           { area: "reception", label: "Recepción", icon: <DoorOpen className="h-4 w-4" /> },
245 %         ].map((item) => {
246 %           const count = item.area === "all" ? notifications.length : (areaStats[item.area] || 0);
247 %           const unread = item.area === "all" ? unreadCount : (unreadByArea[item.area] || 0);
248 %           return (
249 %             <Card
250 %               key={item.area}
251 %               className={`cursor-pointer transition-all hover:shadow-md ${
252 %                 areaFilter === item.area
253 %                   ? "ring-2 ring-primary border-primary"
254 %                   : ""
255 %               }`}
256 %               onClick={() => setAreaFilter(item.area)}
257 %               data-testid={`filter-area-${item.area}`}
258 %             >
259 %               <CardContent className="p-3 flex items-center gap-2">
260 %                 {item.icon}
261 %                 <div className="flex-1 min-w-0">
262 %                   <p className="text-xs font-medium truncate">{item.label}</p>
263 %                   <p className="text-lg font-bold">{count}</p>
264 %                 </div>
265 %                 {unread > 0 && (
266 %                   <Badge variant="destructive" className="text-xs h-5 min-w-5 flex items-center justify-center">
267 %                     {unread}
268 %                   </Badge>
269 %                 )}
270 %               </CardContent>
271 %             </Card>
272 %           );
273 %         })}
274 %       </div>
275 %
276 %       <div className="flex items-center gap-3">
```


client/src/pages/chatbot-dashboard.tsx (cont.)

```
277 %         <div className="relative flex-1 max-w-sm">
278 %           <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
279 %           <Input
280 %             placeholder="Buscar por huésped, habitación o mensaje..."
281 %             value={searchTerm}
282 %             onChange={(e) => setSearchTerm(e.target.value)}
283 %             className="pl-9"
284 %             data-testid="input-search"
285 %           />
286 %         </div>
287 %         <Button
288 %           variant={showRead ? "secondary" : "outline"}
289 %           size="sm"
290 %           onClick={() => setShowRead(!showRead)}
291 %           data-testid="button-toggle-read"
292 %         >
293 %           {showRead ? <Eye className="h-4 w-4 mr-1" /> : <EyeOff className="h-4 w-4 mr-1" />}
294 %           {showRead ? "Mostrando leídos" : "Ocultar leídos"}
295 %         </Button>
296 %       </div>
297 %
298 %       <div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
299 %         <Card className="lg:col-span-2">
300 %           <CardHeader className="pb-3">
301 %             <CardTitle className="text-lg flex items-center gap-2">
302 %               <MessageCircle className="h-5 w-5" />
303 %               Mensajes
304 %               {filteredMessages.length > 0 && (
305 %                 <Badge variant="outline" className="ml-1">{filteredMessages.length}</Badge>
306 %               )}
307 %             </CardTitle>
308 %             <CardDescription>
309 %               {areaFilter === "all" ? "Todos los mensajes del chatbot" : `Filtrado por: ${getAreaLabel(areaFilter)}`}
310 %             </CardDescription>
311 %           </CardHeader>
312 %           <CardContent className="p-0">
313 %             <ScrollArea className="h-[500px]">
314 %               {isLoading ? (
315 %                 <div className="flex items-center justify-center py-12 text-muted-foreground">
316 %                   <RefreshCw className="h-5 w-5 animate-spin mr-2" />
317 %                   Cargando mensajes...
318 %                 </div>
319 %               ) : filteredMessages.length === 0 ? (
320 %                 <div className="flex flex-col items-center justify-center py-12 text-muted-foreground gap-2">
321 %                   <Bot className="h-10 w-10 opacity-30" />
322 %                   <p className="text-sm">No hay mensajes{areaFilter !== "all" ? ` de ${getAreaLabel(areaFilter)}` : "...`}
323 %                   <p className="text-xs">Los mensajes del chatbot MARA aparecerán aquí automáticamente</p>
324 %                 </div>
325 %               ) : (
326 %                 <div className="divide-y">
327 %                   {filteredMessages
328 %                     .sort((a, b) => new Date(b.createdAt!).getTime() - new Date(a.createdAt!).getTime())
329 %                     .map((msg) => (
330 %                       <div
331 %                         key={msg.id}
332 %                         className={`flex items-start gap-3 p-4 cursor-pointer transition-colors hover:bg-muted/50 ${
333 %                           !msg.isRead ? "bg-blue-50/50 dark:bg-blue-950/20" : ""
334 %                         } ${selectedMessage?.id === msg.id ? "bg-muted" : ""}`}
335 %                         onClick={() => handleSelectMessage(msg)}
336 %                         data-testid={`message-item-${msg.id}`}
337 %                       >
338 %                         <div className={`flex h-9 w-9 shrink-0 items-center justify-center rounded-full ${getAreaColor(msg.targetArea)}`}
339 %                           {getAreaIcon(msg.targetArea)}
340 %                         </div>
341 %                         <div className="flex-1 min-w-0 space-y-1">
342 %                           <div className="flex items-center gap-2">
343 %                             <p className={`text-sm font-medium truncate ${!msg.isRead ? "font-semibold" : ""}`}>
344 %                               {msg.title}
345 %                             </p>
346 %                             {!msg.isRead && (
347 %                               <span className="flex h-2 w-2 shrink-0 rounded-full bg-blue-500" />
348 %                             )}
349 %                           </div>
350 %                           <p className="text-sm text-muted-foreground line-clamp-1">{msg.message}</p>
351 %                         <div className="flex items-center gap-2 flex-wrap">
352 %                           <span className={`inline-flex items-center gap-1 text-xs px-2 py-0.5 rounded-full ${getAreaLabel(msg.targetArea)}`}
353 %                             {getAreaIcon(msg.targetArea)}
354 %                             {getAreaLabel(msg.targetArea)}
355 %                           </span>
356 %                           <span>
357 %                             {getPriorityBadge(msg.priority)}
358 %                             <span className="text-xs text-muted-foreground flex items-center gap-1">
359 %                               <Clock className="h-3 w-3" />
360 %                               {formatTimeAgo(msg.createdAt!)}
361 %                             </span>
362 %                           </div>
363 %                         </div>
364 %                       </div>
365 %                     )}
366 %                   </div>
367 %                 </ScrollArea>
368 %               </CardContent>
369 %             </Card>
```

```

370 %
371 %
372 %     <CardHeader className="pb-3">
373 %         <CardTitle className="text-lg">Detalle del Mensaje</CardTitle>
374 %     </CardHeader>
375 %     <CardContent>
376 %         {selectedMessage ? (
377 %             <div className="space-y-4" data-testid="message-detail">
378 %                 <div className="flex items-center gap-3">
379 %                     <div className={`flex h-12 w-12 items-center justify-center rounded-full ${getAreaColor(selectedMes...
380 %                         {getAreaIcon(selectedMessage.targetArea)}
381 %                     </div>
382 %                     <div className="flex-1">
383 %                         <p className="font-semibold">{selectedMessage.title}</p>
384 %                         <div className="flex items-center gap-2 mt-1">
385 %                             <span className={`inline-flex items-center gap-1 text-xs px-2 py-0.5 rounded-full ${getAreaColo...
386 %                                 {getAreaLabel(selectedMessage.targetArea)}
387 %                             </span>
388 %                             {getPriorityBadge(selectedMessage.priority)}
389 %                         </div>
390 %                     </div>
391 %                 </div>
392 %             </div>
393 %
394 %             <Separator />
395 %
396 %             <div className="space-y-3">
397 %                 <div>
398 %                     <p className="text-xs text-muted-foreground font-medium mb-1">Mensaje</p>
399 %                     <div className="p-3 bg-muted rounded-lg">
400 %                         <p className="text-sm whitespace-pre-wrap">{selectedMessage.message}</p>
401 %                     </div>
402 %
403 %                     {selectedMessage.relatedEntityId && (
404 %                         <div className="flex items-center gap-2 text-sm">
405 %                             {selectedMessage.relatedEntityType === "room" ? (
406 %                                 <>
407 %                                     <DoorOpen className="h-4 w-4 text-muted-foreground" />
408 %                                     <span className="text-muted-foreground">Habitación:</span>
409 %                                     <span className="font-medium">{selectedMessage.relatedEntityId}</span>
410 %                                 </>
411 %                             ) : (
412 %                                 <>
413 %                                     <Hash className="h-4 w-4 text-muted-foreground" />
414 %                                     <span className="text-muted-foreground">Reserva:</span>
415 %                                     <span className="font-medium">{selectedMessage.relatedEntityId}</span>
416 %                                 </>
417 %                             )}
418 %                         </div>
419 %                     )}
420 %
421 %                     <div className="flex items-center gap-2 text-sm">
422 %                         <Clock className="h-4 w-4 text-muted-foreground" />
423 %                         <span className="text-muted-foreground">Recibido:</span>
424 %                         <span className="font-medium">{formatDate(selectedMessage.createdAt!)}</span>
425 %                     </div>
426 %
427 %                     <div className="flex items-center gap-2 text-sm">
428 %                         {selectedMessage.isRead ? (
429 %                             <>
430 %                                 <Eye className="h-4 w-4 text-green-500" />
431 %                                 <span className="text-green-600 dark:text-green-400">Leído</span>
432 %                                 {selectedMessage.readAt && (
433 %                                     <span className="text-xs text-muted-foreground">
434 %                                         {formatDate(selectedMessage.readAt)}
435 %                                     </span>
436 %                                 )}
437 %                             </>
438 %                         ) : (
439 %                             <>
440 %                                 <EyeOff className="h-4 w-4 text-amber-500" />
441 %                                 <span className="text-amber-600 dark:text-amber-400">Sin leer</span>
442 %                             </>
443 %                         )}
444 %                     </div>
445 %                 </div>
446 %
447 %             <Separator />
448 %
449 %             <div className="flex gap-2">
450 %                 {!selectedMessage.isRead && (
451 %                     <Button
452 %                         size="sm"
453 %                         variant="outline"
454 %                         onClick={() => markReadMutation.mutate(selectedMessage.id)}
455 %                         data-testid="button-mark-read"
456 %                     >
457 %                         <CheckCheck className="h-4 w-4 mr-1" />
458 %                         Marcar como leído
459 %                     </Button>
460 %                 )}
461 %             </div>
462 %         </div>

```

client/src/pages/chatbot-dashboard.tsx (cont.)

```
463 %         ) : (
464 %           <div className="flex flex-col items-center justify-center py-12 text-muted-foreground gap-2">
465 %             <MessageCircle className="h-8 w-8 opacity-30" />
466 %             <p className="text-sm text-center">
467 %               Seleccioná un mensaje para ver los detalles
468 %             </p>
469 %           </div>
470 %         )}
471 %       </CardContent>
472 %     </Card>
473 %   </div>
474 %
475 %   <Card>
476 %     <CardHeader className="pb-3">
477 %       <CardTitle className="text-lg flex items-center gap-2">
478 %         <Bot className="h-5 w-5" />
479 %         Configuración del Webhook
480 %       </CardTitle>
481 %       <CardDescription>
482 %         Información para conectar tu chatbot MARA con este sistema
483 %       </CardDescription>
484 %     </CardHeader>
485 %     <CardContent>
486 %       <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
487 %         <div className="space-y-2">
488 %           <p className="text-sm font-medium">Endpoint</p>
489 %           <code className="block p-2 bg-muted rounded text-xs font-mono" data-testid="text-webhook-url">
490 %             POST /api/webhook/chatbot
491 %           </code>
492 %         </div>
493 %         <div className="space-y-2">
494 %           <p className="text-sm font-medium">Header de Autenticación</p>
495 %           <code className="block p-2 bg-muted rounded text-xs font-mono">
496 %             X-Chatbot-Secret: .....
497 %           </code>
498 %         </div>
499 %         <div className="space-y-2 md:col-span-2">
500 %           <p className="text-sm font-medium">Áreas soportadas</p>
501 %           <div className="flex flex-wrap gap-2">
502 %             {["housekeeping", "maintenance", "restaurant", "spa", "reception"].map((area) => (
503 %               <span
504 %                 key={area}
505 %                 className={`inline-flex items-center gap-1.5 text-xs px-3 py-1.5 rounded-full ${getAreaColor(area...
506 %               >
507 %                 {getAreaIcon(area)}
508 %                 {getAreaLabel(area)}
509 %               </span>
510 %             )})}
511 %           </div>
512 %         </div>
513 %         <div className="space-y-2 md:col-span-2">
514 %           <p className="text-sm font-medium">Ejemplo de payload</p>
515 %           <pre className="p-3 bg-muted rounded text-xs font-mono overflow-x-auto" data-testid="text-payload-examp...
516 %         {`{
517 %           "eventType": "guest_request",
518 %           "area": "housekeeping",
519 %           "priority": "normal",
520 %           "guestName": "Carlos García",
521 %           "roomNumber": "305",
522 %           "message": "Solicita toallas adicionales",
523 %           "timestamp": "${new Date().toISOString()}"
524 %         }`}
525 %         </pre>
526 %       </div>
527 %     </div>
528 %   </CardContent>
529 % </Card>
530 % </div>
531 %   );
532 % }
533 %
```

```
1 % import { useState, useRef, useCallback, useEffect } from "react";
2 % import { useRoute } from "wouter";
3 % import { useQuery, useMutation } from "@tanstack/react-query";
4 % import {
5 %   Hotel,
6 %   User,
7 %   FileText,
8 %   Clock,
9 %   CheckCircle2,
10 %   Camera,
11 %   Upload,
12 %   ChevronLeft,
13 %   ChevronRight,
14 %   Loader2,
15 %   AlertCircle,
16 %   CalendarDays,
17 %   Phone,
18 %   Mail,
19 %   MapPin,
20 % } from "lucide-react";
21 % import { Card, CardContent } from "@components/ui/card";
22 % import { Button } from "@components/ui/button";
23 % import { Input } from "@components/ui/input";
24 % import { Label } from "@components/ui/label";
25 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
26 % import { Checkbox } from "@components/ui/checkbox";
27 % import { apiRequest } from "@lib/queryClient";
28 %
29 % const STEPS = [
30 %   { num: 1, label: "Tus datos", icon: User },
31 %   { num: 2, label: "Documento", icon: FileText },
32 %   { num: 3, label: "Llegada", icon: Clock },
33 %   { num: 4, label: "Confirmación", icon: CheckCircle2 },
34 % ];
35 %
36 % function compressImage(file: File, maxWidth: number): Promise<string> {
37 %   return new Promise((resolve, reject) => {
38 %     const img = new Image();
39 %     const reader = new FileReader();
40 %     reader.onload = (e) => {
41 %       img.onload = () => {
42 %         const canvas = document.createElement("canvas");
43 %         let { width, height } = img;
44 %         if (width > maxWidth) {
45 %           height = (height * maxWidth) / width;
46 %           width = maxWidth;
47 %         }
48 %         canvas.width = width;
49 %         canvas.height = height;
50 %         const ctx = canvas.getContext("2d");
51 %         if (!ctx) return reject(new Error("Canvas not supported"));
52 %         ctx.drawImage(img, 0, 0, width, height);
53 %         resolve(canvas.toDataURL("image/jpeg", 0.8));
54 %       };
55 %       img.onerror = reject;
56 %       img.src = e.target?.result as string;
57 %     };
58 %     reader.onerror = reject;
59 %     reader.readAsDataURL(file);
60 %   });
61 % }
62 %
63 % export default function WebCheckinPublicPage() {
64 %   const [, params] = useRoute("/web-checkin/:token");
65 %   const token = params?.token || "";
66 %
67 %   const [step, setStep] = useState(1);
68 %   const [firstName, setFirstName] = useState("");
69 %   const [lastName, setLastName] = useState("");
70 %   const [documentType, setDocumentType] = useState("DNI");
71 %   const [documentNumber, setDocumentNumber] = useState("");
72 %   const [nationality, setNationality] = useState("Argentina");
73 %   const [phone, setPhone] = useState("");
74 %   const [email, setEmail] = useState("");
75 %   const [documentPhoto, setDocumentPhoto] = useState<string | null>(null);
76 %   const [arrivalTime, setArrivalTime] = useState("");
77 %   const [requestEarlyCheckIn, setRequestEarlyCheckIn] = useState(false);
78 %   const [earlyCheckInTime, setEarlyCheckInTime] = useState("");
79 %   const [termsAccepted, setTermsAccepted] = useState(false);
80 %   const fileInputRef = useRef<HTMLInputElement>(null);
81 %
82 %   const { data, isLoading, error } = useQuery({
83 %     queryKey: ["/api/public/web-checkin", token],
84 %     queryFn: async () => {
85 %       const res = await fetch(`/api/public/web-checkin/${token}`);
86 %       if (!res.ok) {
87 %         const err = await res.json();
88 %         throw new Error(err.error || "Error al cargar");
89 %       }
90 %       return res.json();
```

client/src/pages/web-checkin-public.tsx (cont.)

```
91 %     },
92 %     enabled: !!token,
93 %   });
94 %
95 %   useEffect(() => {
96 %     if (data?.webCheckin) {
97 %       const wc = data.webCheckin;
98 %       if (wc.confirmedFirstName) setFirstName(wc.confirmedFirstName);
99 %       if (wc.confirmedLastName) setLastName(wc.confirmedLastName);
100 %       if (wc.confirmedDocumentType) setDocumentType(wc.confirmedDocumentType);
101 %       if (wc.confirmedDocumentNumber) setDocumentNumber(wc.confirmedDocumentNumber);
102 %       if (wc.confirmedNationality) setNationality(wc.confirmedNationality);
103 %       if (wc.confirmedPhone) setPhone(wc.confirmedPhone);
104 %       if (wc.confirmedEmail) setEmail(wc.confirmedEmail);
105 %     }
106 %   }, [data]);
107 %
108 %   const submitMutation = useMutation({
109 %     mutationFn: async () => {
110 %       const res = await fetch(`/api/public/web-checkin/${token}`, {
111 %         method: "POST",
112 %         headers: { "Content-Type": "application/json" },
113 %         body: JSON.stringify({
114 %           confirmedFirstName: firstName,
115 %           confirmedLastName: lastName,
116 %           confirmedDocumentType: documentType,
117 %           confirmedDocumentNumber: documentNumber,
118 %           confirmedNationality: nationality,
119 %           confirmedPhone: phone,
120 %           confirmedEmail: email,
121 %           documentPhotoUrl: documentPhoto,
122 %           estimatedArrivalTime: arrivalTime,
123 %           requestEarlyCheckIn,
124 %           earlyCheckInTime: requestEarlyCheckIn ? earlyCheckInTime : null,
125 %           termsAccepted,
126 %         }),
127 %       });
128 %       if (!res.ok) {
129 %         const err = await res.json();
130 %         throw new Error(err.error || "Error al enviar");
131 %       }
132 %       return res.json();
133 %     },
134 %     onSuccess: () => {
135 %       setStep(4);
136 %     },
137 %   });
138 %
139 %   const handlePhotoCapture = useCallback(async (e: React.ChangeEvent<HTMLInputElement>) => {
140 %     const file = e.target.files?.[0];
141 %     if (!file) return;
142 %     try {
143 %       const compressed = await compressImage(file, 800);
144 %       setDocumentPhoto(compressed);
145 %     } catch {
146 %       alert("Error al procesar la imagen");
147 %     }
148 %   }, []);
149 %
150 %   const canGoNext = (s: number) => {
151 %     if (s === 1) return firstName.trim() && lastName.trim() && documentNumber.trim();
152 %     if (s === 2) return true;
153 %     if (s === 3) return termsAccepted;
154 %     return false;
155 %   };
156 %
157 %   const handleNext = () => {
158 %     if (step === 3) {
159 %       submitMutation.mutate();
160 %       return;
161 %     }
162 %     setStep((s) => Math.min(s + 1, 4));
163 %   };
164 %
165 %   if (isLoading) {
166 %     return (
167 %       <div className="min-h-screen flex items-center justify-center bg-gradient-to-b from-blue-50 to-white dark:from-...
168 %         <Loader2 className="h-8 w-8 animate-spin text-primary" />
169 %       </div>
170 %     );
171 %   }
172 %
173 %   if (error) {
174 %     return (
175 %       <div className="min-h-screen flex items-center justify-center bg-gradient-to-b from-blue-50 to-white dark:from-...
176 %         <Card className="max-w-md w-full">
177 %           <CardContent className="flex flex-col items-center py-12 text-center">
178 %             <AlertCircle className="h-16 w-16 text-destructive mb-4" />
179 %             <h2 className="text-xl font-bold mb-2">No disponible</h2>
180 %             <p className="text-muted-foreground">{(error as Error).message}</p>
181 %           </CardContent>
182 %         </Card>
183 %       </div>
184 %     );
185 %   }
186 % }
```

client/src/pages/web-checkin-public.tsx (cont.)

```
184 %     );
185 %   }
186 %
187 %   const hotel = data?.hotel;
188 %   const reservation = data?.reservation;
189 %
190 %   return (
191 %     <div className="min-h-screen bg-gradient-to-b from-blue-50 to-white dark:from-gray-900 dark:to-gray-950">
192 %       <header className="bg-white dark:bg-gray-900 border-b shadow-sm sticky top-0 z-50">
193 %         <div className="max-w-lg mx-auto px-4 py-3 flex items-center gap-3">
194 %           <div className="flex h-10 w-10 items-center justify-center rounded-lg bg-primary text-primary-foreground">
195 %             <Hotel className="h-6 w-6" />
196 %           </div>
197 %           <div>
198 %             <h1 className="font-bold text-lg">{hotel?.name || "Maran Suites & Towers"}</h1>
199 %             <p className="text-xs text-muted-foreground">Web Check-in</p>
200 %           </div>
201 %         </div>
202 %       </header>
203 %
204 %       {reservation && (
205 %         <div className="max-w-lg mx-auto px-4 mt-4">
206 %           <div className="flex items-center gap-4 text-sm text-muted-foreground bg-white dark:bg-gray-900 rounded-lg ...">
207 %             <div className="flex items-center gap-1.5">
208 %               <CalendarDays className="h-4 w-4" />
209 %               <span>{reservation.checkInDate} - {reservation.checkOutDate}</span>
210 %             </div>
211 %             <span className="text-muted-foreground/50">|</span>
212 %             <span>{reservation.roomType}</span>
213 %             <span className="text-muted-foreground/50">|</span>
214 %             <span>{reservation.nights} noche(s)</span>
215 %           </div>
216 %         </div>
217 %       )}
218 %
219 %       <div className="max-w-lg mx-auto px-4 mt-6 mb-4">
220 %         <div className="flex items-center justify-between">
221 %           {STEPS.map((s, i) => (
222 %             <div key={s.num} className="flex items-center">
223 %               <div className={`flex items-center gap-1.5 ${step >= s.num ? "text-primary" : "text-muted-foreground"}`}>
224 %                 <div className={`flex h-8 w-8 items-center justify-center rounded-full text-sm font-bold ${
225 %                   step > s.num ? "bg-primary text-primary-foreground" :
226 %                   step === s.num ? "bg-primary text-primary-foreground" :
227 %                   "bg-muted text-muted-foreground"
228 %                 }`} />
229 %                 {step > s.num ? <CheckCircle2 className="h-4 w-4" /> : s.num}
230 %               </div>
231 %               <span className="text-xs font-medium hidden sm:inline">{s.label}</span>
232 %             </div>
233 %             {i < STEPS.length - 1 && (
234 %               <div className={`w-8 sm:w-12 h-0.5 mx-1 ${step > s.num ? "bg-primary" : "bg-muted"}`} />
235 %             )}
236 %           </div>
237 %         </div>
238 %       </div>
239 %     </div>
240 %
241 %     <div className="max-w-lg mx-auto px-4 pb-24">
242 %       {step === 1 && (
243 %         <Card>
244 %           <CardContent className="p-5 space-y-4">
245 %             <h2 className="text-lg font-bold">Datos personales</h2>
246 %             <p className="text-sm text-muted-foreground">Por favor, confirmá o completá tus datos.</p>
247 %             <div className="grid grid-cols-2 gap-3">
248 %               <div className="space-y-1.5">
249 %                 <Label htmlFor="firstName">Nombre *</Label>
250 %                 <Input id="firstName" value={firstName} onChange={(e) => setFirstName(e.target.value)} data-testid=...
251 %               </div>
252 %               <div className="space-y-1.5">
253 %                 <Label htmlFor="lastName">Apellido *</Label>
254 %                 <Input id="lastName" value={lastName} onChange={(e) => setLastName(e.target.value)} data-testid="in...
255 %               </div>
256 %             </div>
257 %             <div className="grid grid-cols-2 gap-3">
258 %               <div className="space-y-1.5">
259 %                 <Label>Tipo de Documento</Label>
260 %                 <Select value={documentType} onChange={setDocumentType}>
261 %                   <SelectTrigger data-testid="select-wc-doctype">
262 %                     <SelectValue />
263 %                   </SelectTrigger>
264 %                   <SelectContent>
265 %                     <SelectItem value="DNI">DNI</SelectItem>
266 %                     <SelectItem value="Passport">Pasaporte</SelectItem>
267 %                     <SelectItem value="CUIT/CUIL">CUIT/CUIL</SelectItem>
268 %                     <SelectItem value="CI">CI</SelectItem>
269 %                   </SelectContent>
270 %                 </Select>
271 %               </div>
272 %               <div className="space-y-1.5">
273 %                 <Label htmlFor="docNumber">N° Documento *</Label>
274 %                 <Input id="docNumber" value={documentNumber} onChange={(e) => setDocumentNumber(e.target.value)} da...
275 %               </div>
276 %             </div>
277 %           </CardContent>
278 %         </Card>
279 %       )}
280 %     </div>
281 %   )
282 % }
```

client/src/pages/web-checkin-public.tsx (cont.)

```
277 %         <div className="space-y-1.5">
278 %             <Label htmlFor="nationality">Nacionalidad</Label>
279 %             <Input id="nationality" value={nationality} onChange={(e) => setNationality(e.target.value)} data-tes...
280 %         </div>
281 %         <div className="grid grid-cols-2 gap-3">
282 %             <div className="space-y-1.5">
283 %                 <Label htmlFor="phone">Teléfono</Label>
284 %                 <Input id="phone" type="tel" value={phone} onChange={(e) => setPhone(e.target.value)} data-testid="...
285 %             </div>
286 %             <div className="space-y-1.5">
287 %                 <Label htmlFor="email">Email</Label>
288 %                 <Input id="email" type="email" value={email} onChange={(e) => setEmail(e.target.value)} data-testid="...
289 %             </div>
290 %         </div>
291 %     </CardContent>
292 % </Card>
293 % }}
294 %
295 % {step === 2 && (
296 %     <Card>
297 %         <CardContent className="p-5 space-y-4">
298 %             <h2 className="text-lg font-bold">Foto del documento</h2>
299 %             <p className="text-sm text-muted-foreground">Tomá una foto del frente de tu documento de identidad. Est...
300 %             <input
301 %                 ref={fileInputRef}
302 %                 type="file"
303 %                 accept="image/*"
304 %                 capture="environment"
305 %                 className="hidden"
306 %                 onChange={handlePhotoCapture}
307 %             />
308 %             {documentPhoto ? (
309 %                 <div className="space-y-3">
310 %                     <div className="border rounded-lg overflow-hidden">
311 %                         <img src={documentPhoto} alt="Documento" className="w-full h-auto" data-testid="img-document-prev...
312 %                     </div>
313 %                     <Button
314 %                         variant="outline"
315 %                         className="w-full"
316 %                         onClick={() => {
317 %                             setDocumentPhoto(null);
318 %                             fileInputRef.current?.click();
319 %                         }}
320 %                         data-testid="button-retake-photo"
321 %                     >
322 %                         <Camera className="h-4 w-4 mr-2" />
323 %                         Tomar otra foto
324 %                     </Button>
325 %                 </div>
326 %             ) : (
327 %                 <div className="flex flex-col gap-3">
328 %                     <Button
329 %                         variant="outline"
330 %                         className="h-32 border-dashed flex flex-col gap-2"
331 %                         onClick={() => fileInputRef.current?.click()}
332 %                         data-testid="button-take-photo"
333 %                     >
334 %                         <Camera className="h-8 w-8 text-muted-foreground" />
335 %                         <span className="text-sm text-muted-foreground">Tomar foto o seleccionar imagen</span>
336 %                     </Button>
337 %                 </div>
338 %             )}
339 %         </CardContent>
340 %     </Card>
341 % }}
342 %
343 % {step === 3 && (
344 %     <Card>
345 %         <CardContent className="p-5 space-y-4">
346 %             <h2 className="text-lg font-bold">Detalles de llegada</h2>
347 %             <div className="space-y-1.5">
348 %                 <Label htmlFor="arrivalTime">Hora estimada de llegada</Label>
349 %                 <Input
350 %                     id="arrivalTime"
351 %                     type="time"
352 %                     value={arrivalTime}
353 %                     onChange={(e) => setArrivalTime(e.target.value)}
354 %                     data-testid="input-wc-arrival-time"
355 %                 />
356 %             </div>
357 %             <div className="flex items-center space-x-2 p-3 bg-muted/50 rounded-lg">
358 %                 <Checkbox
359 %                     id="earlyCheckin"
360 %                     checked={requestEarlyCheckIn}
361 %                     onChange={(c) => setRequestEarlyCheckIn(!c)}
362 %                     data-testid="checkbox-early-checkin"
363 %                 />
364 %                 <Label htmlFor="earlyCheckin" className="text-sm cursor-pointer">
365 %                     Solicitar early check-in (antes de las 15:00)
366 %                 </Label>
367 %             </div>
368 %             {requestEarlyCheckin && (
369 %                 <div className="space-y-1.5">
```

client/src/pages/web-checkin-public.tsx (cont.)

```
370 %         <Label htmlFor="earlyTime">¿A qué hora llegarías?</Label>
371 %         <Input
372 %           id="earlyTime"
373 %           type="time"
374 %           value={earlyCheckInTime}
375 %           onChange={(e) => setEarlyCheckInTime(e.target.value)}
376 %           data-testid="input-wc-early-time"
377 %         />
378 %         <p className="text-xs text-muted-foreground">El early check-in está sujeto a disponibilidad y puede...
379 %       </div>
380 %     )}
381 %     <div className="border-t pt-4 mt-4">
382 %       <div className="flex items-start space-x-2">
383 %         <Checkbox
384 %           id="terms"
385 %           checked={termsAccepted}
386 %           onChange={(c) => setTermsAccepted(!c)}
387 %           data-testid="checkboxbox-terms"
388 %         />
389 %         <Label htmlFor="terms" className="text-sm cursor-pointer leading-relaxed">
390 %           Acepto los términos y condiciones del hotel, incluyendo la política de cancelación y el reglament...
391 %         </Label>
392 %       </div>
393 %     </div>
394 %   </CardContent>
395 % </Card>
396 % }}
397 %
398 % {step === 4 && (
399 %   <Card>
400 %     <CardContent className="p-5 text-center space-y-4">
401 %       <div className="flex justify-center">
402 %         <div className="h-20 w-20 rounded-full bg-green-100 dark:bg-green-900/30 flex items-center justify-ce...
403 %         <CheckCircle2 className="h-10 w-10 text-green-600 dark:text-green-400" />
404 %       </div>
405 %     </div>
406 %     <h2 className="text-2xl font-bold">¡Listo!</h2>
407 %     <p className="text-muted-foreground">
408 %       Tu web check-in ha sido completado exitosamente. Cuando llegues al hotel, solo tenés que acercarte a ...
409 %     </p>
410 %     <div className="bg-muted/50 rounded-lg p-4 text-sm text-left space-y-2">
411 %       <div className="flex items-center gap-2">
412 %         <User className="h-4 w-4 text-muted-foreground" />
413 %         <span>{firstName} {lastName}</span>
414 %       </div>
415 %       {arrivalTime && (
416 %         <div className="flex items-center gap-2">
417 %           <Clock className="h-4 w-4 text-muted-foreground" />
418 %           <span>Llegada estimada: {arrivalTime}</span>
419 %         </div>
420 %       )}
421 %       {requestEarlyCheckIn && (
422 %         <div className="flex items-center gap-2">
423 %           <CalendarDays className="h-4 w-4 text-muted-foreground" />
424 %           <span>Early check-in solicitado{earlyCheckInTime ? `: ${earlyCheckInTime}` : ""}</span>
425 %         </div>
426 %       )}
427 %     </div>
428 %     {hotel && (
429 %       <div className="border-t pt-4 text-sm text-muted-foreground space-y-1">
430 %         <div className="flex items-center gap-2 justify-center">
431 %           <MapPin className="h-4 w-4" />
432 %           <span>{hotel.address}</span>
433 %         </div>
434 %         <div className="flex items-center gap-2 justify-center">
435 %           <Phone className="h-4 w-4" />
436 %           <span>{hotel.phone}</span>
437 %         </div>
438 %       </div>
439 %     )}
440 %   </CardContent>
441 % </Card>
442 % )}
443 % </div>
444 %
445 % {step < 4 && (
446 %   <div className="fixed bottom-0 left-0 right-0 bg-white dark:bg-gray-900 border-t p-4 safe-area-inset-bottom">
447 %     <div className="max-w-lg mx-auto flex gap-3">
448 %       {step > 1 && (
449 %         <Button
450 %           variant="outline"
451 %           onClick={() => setStep((s) => s - 1)}
452 %           className="flex-1"
453 %           data-testid="button-wc-back"
454 %         >
455 %           <ChevronLeft className="h-4 w-4 mr-1" />
456 %           Atrás
457 %         </Button>
458 %       )}
459 %       <Button
460 %         onClick={handleNext}
461 %         disabled={!canGoNext(step) || submitMutation.isPending}
462 %         className="flex-1"
```


client/src/pages/web-checkin-public.tsx (cont.)

```
463 %         data-testid="button-wc-next"
464 %     >
465 %         {submitMutation.isPending ? (
466 %             <>
467 %                 <Loader2 className="h-4 w-4 mr-2 animate-spin" />
468 %                 Enviando...
469 %             </>
470 %         ) : step === 3 ? (
471 %             "Confirmar Check-in"
472 %         ) : (
473 %             <>
474 %                 Siguiente
475 %                 <ChevronRight className="h-4 w-4 ml-1" />
476 %             </>
477 %         )}
478 %     </Button>
479 % </div>
480 % </div>
481 % )}
482 %
483 % {submitMutation.isError && (
484 %     <div className="fixed bottom-20 left-0 right-0 px-4">
485 %         <div className="max-w-lg mx-auto bg-destructive/10 border border-destructive/20 rounded-lg p-3 text-sm text...
486 %             {(submitMutation.error as Error).message}
487 %         </div>
488 %     </div>
489 % )}
490 % </div>
491 % );
492 % }
```

```
1 % import { useState, useEffect } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { useLocation } from "wouter";
4 % import {
5 %   Users2,
6 %   Plus,
7 %   Search,
8 %   Calendar,
9 %   Phone,
10 %   Mail,
11 %   MoreHorizontal,
12 %   Pencil,
13 %   Trash2,
14 %   Eye,
15 %   DoorOpen,
16 %   Hotel,
17 % } from "lucide-react";
18 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
19 % import { Button } from "@components/ui/button";
20 % import { Input } from "@components/ui/input";
21 % import { Badge } from "@components/ui/badge";
22 % import { Skeleton } from "@components/ui/skeleton";
23 % import {
24 %   Dialog,
25 %   DialogContent,
26 %   DialogDescription,
27 %   DialogFooter,
28 %   DialogHeader,
29 %   DialogTitle,
30 % } from "@components/ui/dialog";
31 % import {
32 %   DropdownMenu,
33 %   DropdownMenuContent,
34 %   DropdownMenuItem,
35 %   DropdownMenuSeparator,
36 %   DropdownMenuTrigger,
37 % } from "@components/ui/dropdown-menu";
38 % import {
39 %   Select,
40 %   SelectContent,
41 %   SelectItem,
42 %   SelectTrigger,
43 %   SelectValue,
44 % } from "@components/ui/select";
45 % import {
46 %   Table,
47 %   TableBody,
48 %   TableCell,
49 %   TableHead,
50 %   TableHeader,
51 %   TableRow,
52 % } from "@components/ui/table";
53 % import { Label } from "@components/ui/label";
54 % import { Textarea } from "@components/ui/textarea";
55 % import { useToast } from "@hooks/use-toast";
56 % import { queryClient, apiRequest } from "@lib/queryClient";
57 % import type { GroupWithDetails, GroupStatus, InsertGroup, RoomType } from "@shared/schema";
58 % import { Trash2 as TrashIcon } from "lucide-react";
59 %
60 % interface BlockDraft {
61 %   id: string;
62 %   roomId: string;
63 %   quantity: number;
64 %   agreedRate: string;
65 %   useCustomDates: boolean;
66 %   blockCheckInDate: string;
67 %   blockCheckOutDate: string;
68 %   error?: string;
69 % }
70 %
71 % function GroupStatusBadge({ status }: { status: GroupStatus }) {
72 %   const statusConfig: Record<GroupStatus, { label: string; variant: "default" | "secondary" | "destructive" | "outlin...
73 %     tentative: { label: "Tentativo", variant: "outline" },
74 %     blocked: { label: "Bloqueado", variant: "secondary" },
75 %     confirmed: { label: "Confirmado", variant: "default" },
76 %     inhouse: { label: "En Casa", variant: "default" },
77 %     finished: { label: "Finalizado", variant: "outline" },
78 %     cancelled: { label: "Cancelado", variant: "destructive" },
79 %   };
80 %
81 %   const config = statusConfig[status];
82 %
83 %   return <Badge variant={config.variant}>{config.label}</Badge>;
84 % }
85 %
86 % function GroupFormDialog({
87 %   group,
88 %   open,
89 %   onOpenChange,
90 %   onSuccess,
```

client/src/pages/groups.tsx (cont.)

```
91 %   }: {
92 %     group?: GroupWithDetails;
93 %     open: boolean;
94 %     onOpenChange: (open: boolean) => void;
95 %     onSuccess: () => void;
96 %   } {
97 %     const { toast } = useToast();
98 %     const isEditing = !!group;
99 %
100 %    const today = new Date().toISOString().split("T")[0];
101 %    const tomorrow = new Date(Date.now() + 86400000).toISOString().split("T")[0];
102 %
103 %    const [step, setStep] = useState(1);
104 %    const [formData, setFormData] = useState<Partial<InsertGroup>>>({
105 %      name: group?.name || "",
106 %      contactName: group?.contactName || "",
107 %      contactPhone: group?.contactPhone || "",
108 %      contactEmail: group?.contactEmail || "",
109 %      eventDate: group?.eventDate || "",
110 %      checkInDate: group?.checkInDate || today,
111 %      checkOutDate: group?.checkOutDate || tomorrow,
112 %      status: group?.status || "tentative",
113 %      releaseDate: group?.releaseDate || "",
114 %      notes: group?.notes || "",
115 %    });
116 %
117 %    const [blocks, setBlocks] = useState<BlockDraft[]>([]);
118 %    const [isCreating, setIsCreating] = useState(false);
119 %    const [createdGroupId, setCreatedGroupId] = useState<string | null>(null);
120 %
121 %    // Reset state when dialog opens/closes or mode changes
122 %    useEffect(() => {
123 %      if (open) {
124 %        setStep(1);
125 %        setBlocks([]);
126 %        setCreatedGroupId(null);
127 %        setFormData({
128 %          name: group?.name || "",
129 %          contactName: group?.contactName || "",
130 %          contactPhone: group?.contactPhone || "",
131 %          contactEmail: group?.contactEmail || "",
132 %          eventDate: group?.eventDate || "",
133 %          checkInDate: group?.checkInDate || today,
134 %          checkOutDate: group?.checkOutDate || tomorrow,
135 %          status: group?.status || "tentative",
136 %          releaseDate: group?.releaseDate || "",
137 %          notes: group?.notes || "",
138 %        });
139 %      }
140 %    }, [open, group, today, tomorrow]);
141 %
142 %    const { data: roomTypes } = useQuery<RoomType[]>({
143 %      queryKey: ["/api/room-types"],
144 %    });
145 %
146 %    const updateMutation = useMutation({
147 %      mutationFn: (data: Partial<InsertGroup>) =>
148 %        apiRequest("PATCH", `/api/groups/${group?.id}`, data),
149 %      onSuccess: () => {
150 %        queryClient.invalidateQueries({ queryKey: ["/api/groups"] });
151 %        toast({ title: "Grupo actualizado exitosamente" });
152 %        onSuccess();
153 %        onOpenChange(false);
154 %      },
155 %      onError: () => {
156 %        toast({ title: "Error al actualizar grupo", variant: "destructive" });
157 %      },
158 %    });
159 %
160 %    const handleNext = () => {
161 %      if (!formData.name || !formData.checkInDate || !formData.checkOutDate) {
162 %        toast({ title: "Complete los campos obligatorios", variant: "destructive" });
163 %        return;
164 %      }
165 %      setStep(2);
166 %    };
167 %
168 %    const handleBack = () => {
169 %      setStep(1);
170 %    };
171 %
172 %    const addBlock = () => {
173 %      setBlocks([
174 %        ...blocks,
175 %        {
176 %          id: crypto.randomUUID(),
177 %          roomId: "",
178 %          quantity: 1,
179 %          agreedRate: "",
180 %          useCustomDates: false,
181 %          blockCheckInDate: formData.checkInDate || today,
182 %          blockCheckOutDate: formData.checkOutDate || tomorrow,
183 %        },
```

client/src/pages/groups.tsx (cont.)

```
184 %   });
185 % };
186 %
187 % const updateBlock = (id: string, updates: Partial<BlockDraft>) => {
188 %   setBlocks(blocks.map((b) => (b.id === id ? { ...b, ...updates } : b)));
189 % };
190 %
191 % const removeBlock = (id: string) => {
192 %   setBlocks(blocks.filter((b) => b.id !== id));
193 % };
194 %
195 % const handleCreate = async () => {
196 %   // Clear previous errors
197 %   setBlocks(blocks.map((b) => ({ ...b, error: undefined })));
198 %
199 %   // Validate blocks before submission
200 %   const hasInvalidBlocks = blocks.some((b) => {
201 %     const qty = Number(b.quantity);
202 %     return !b.roomTypeId || !Number.isFinite(qty) || qty < 1;
203 %   });
204 %
205 %   if (hasInvalidBlocks) {
206 %     setBlocks(blocks.map((b) => {
207 %       const qty = Number(b.quantity);
208 %       if (!b.roomTypeId) {
209 %         return { ...b, error: "Seleccione tipo de habitación" };
210 %       }
211 %       if (!Number.isFinite(qty) || qty < 1) {
212 %         return { ...b, error: "Cantidad debe ser mayor a 0" };
213 %       }
214 %       return b;
215 %     }));
216 %     toast({
217 %       title: "Bloques inválidos",
218 %       description: "Corrija los errores marcados en rojo",
219 %       variant: "destructive"
220 %     });
221 %     return;
222 %   }
223 %
224 %   setIsCreating(true);
225 %
226 %   try {
227 %     // Create group if not already created
228 %     let groupId = createdGroupId;
229 %     if (!groupId) {
230 %       const response = await apiRequest("POST", "/api/groups", formData);
231 %       const newGroup = await response.json();
232 %       groupId = newGroup.id;
233 %       setCreatedGroupId(groupId);
234 %     }
235 %
236 %     // Create blocks - track failures
237 %     let failedBlocks = 0;
238 %     let successBlocks = 0;
239 %     const updatedBlocks: BlockDraft[] = [];
240 %
241 %     for (const block of blocks) {
242 %       if (!block.roomTypeId) continue;
243 %
244 %       try {
245 %         await apiRequest("POST", `/api/groups/${groupId}/blocks`, {
246 %           roomTypeId: block.roomTypeId,
247 %           quantity: Number(block.quantity),
248 %           ratePlanId: null,
249 %           agreedRate: block.agreedRate || null,
250 %           blockCheckInDate: block.useCustomDates ? block.blockCheckInDate : null,
251 %           blockCheckOutDate: block.useCustomDates ? block.blockCheckOutDate : null,
252 %         });
253 %         successBlocks++;
254 %         // Don't add successful blocks to updatedBlocks - they're done
255 %       } catch {
256 %         failedBlocks++;
257 %         updatedBlocks.push({ ...block, error: "Error al crear bloque - intente de nuevo" });
258 %       }
259 %     }
260 %
261 %     queryClient.invalidateQueries({ queryKey: ["/api/groups"] });
262 %
263 %     if (failedBlocks > 0) {
264 %       // Keep wizard open with only failed blocks
265 %       setBlocks(updatedBlocks);
266 %       toast({
267 %         title: `${successBlocks} bloque(s) creado(s), ${failedBlocks} fallaron`,
268 %         description: "Corrija los errores o elimine los bloques fallidos",
269 %         variant: "destructive"
270 %       });
271 %       // Keep wizard open for user to retry
272 %     } else {
273 %       // All blocks succeeded - close wizard
274 %       toast({ title: blocks.length > 0 ? `Grupo creado con ${blocks.length} bloque(s)` : "Grupo creado exitosamente...",
275 %         onSuccess();
276 %         onOpenChange(false);
```

client/src/pages/groups.tsx (cont.)

```
277 %         setStep(1);
278 %         setBlocks([]);
279 %         setCreatedGroupId(null);
280 %     }
281 % } catch {
282 %   toast({ title: "Error al crear grupo", variant: "destructive" });
283 % } finally {
284 %   setIsCreating(false);
285 % }
286 % };
287 %
288 % const handleSubmit = (e: React.FormEvent) => {
289 %   e.preventDefault();
290 %   if (isEditing) {
291 %     updateMutation.mutate(formData);
292 %   } else {
293 %     handleCreate();
294 %   }
295 % };
296 %
297 % const isPending = isCreating || updateMutation.isPending;
298 %
299 % return (
300 %   <Dialog open={open} onOpenChange={onOpenChange}>
301 %     <DialogContent className="max-w-3xl max-h-[90vh] overflow-y-auto">
302 %       <DialogHeader>
303 %         <DialogTitle>{isEditing ? "Editar Grupo" : "Nuevo Grupo"}</DialogTitle>
304 %         <DialogDescription>
305 %           {isEditing
306 %             ? "Modifique los datos del grupo"
307 %             : step === 1
308 %               ? "Paso 1: Información del grupo"
309 %               : "Paso 2: Bloques de habitaciones"}
310 %         </DialogDescription>
311 %       </DialogHeader>
312 %
313 %       {!isEditing && (
314 %         <div className="flex gap-2 mb-4">
315 %           <div className={`flex-1 h-2 rounded ${step >= 1 ? "bg-primary" : "bg-muted"}`} />
316 %           <div className={`flex-1 h-2 rounded ${step >= 2 ? "bg-primary" : "bg-muted"}`} />
317 %         </div>
318 %       )}
319 %
320 %       {step === 1 && (
321 %         <div className="space-y-4">
322 %           <div className="grid grid-cols-2 gap-4">
323 %             <div className="col-span-2">
324 %               <Label htmlFor="name">Nombre del Grupo *</Label>
325 %               <Input
326 %                 id="name"
327 %                 value={formData.name}
328 %                 onChange={(e) => setFormData({ ...formData, name: e.target.value })}
329 %                 placeholder="Ej: Congreso Médico 2025"
330 %                 required
331 %                 data-testid="input-group-name"
332 %               />
333 %             </div>
334 %
335 %             <div>
336 %               <Label htmlFor="contactName">Nombre de Contacto</Label>
337 %               <Input
338 %                 id="contactName"
339 %                 value={formData.contactName || ""}
340 %                 onChange={(e) => setFormData({ ...formData, contactName: e.target.value })}
341 %                 placeholder="Nombre del responsable"
342 %                 data-testid="input-group-contact-name"
343 %               />
344 %             </div>
345 %
346 %             <div>
347 %               <Label htmlFor="contactPhone">Teléfono</Label>
348 %               <Input
349 %                 id="contactPhone"
350 %                 value={formData.contactPhone || ""}
351 %                 onChange={(e) => setFormData({ ...formData, contactPhone: e.target.value })}
352 %                 placeholder="+54 11 1234-5678"
353 %                 data-testid="input-group-contact-phone"
354 %               />
355 %             </div>
356 %
357 %             <div className="col-span-2">
358 %               <Label htmlFor="contactEmail">Email</Label>
359 %               <Input
360 %                 id="contactEmail"
361 %                 type="email"
362 %                 value={formData.contactEmail || ""}
363 %                 onChange={(e) => setFormData({ ...formData, contactEmail: e.target.value })}
364 %                 placeholder="contacto@empresa.com"
365 %                 data-testid="input-group-contact-email"
366 %               />
367 %             </div>
368 %
369 %             <div>
```

client/src/pages/groups.tsx (cont.)

```
370 %         <Label htmlFor="checkInDate">Fecha Check-in *</Label>
371 %         <Input
372 %           id="checkInDate"
373 %           type="date"
374 %           value={formData.checkInDate}
375 %           onChange={(e) => setFormData({ ...formData, checkInDate: e.target.value })}
376 %           required
377 %           data-testid="input-group-checkin"
378 %         />
379 %       </div>
380 %
381 %       <div>
382 %         <Label htmlFor="checkOutDate">Fecha Check-out *</Label>
383 %         <Input
384 %           id="checkOutDate"
385 %           type="date"
386 %           value={formData.checkOutDate}
387 %           onChange={(e) => setFormData({ ...formData, checkOutDate: e.target.value })}
388 %           required
389 %           data-testid="input-group-checkout"
390 %         />
391 %       </div>
392 %
393 %       <div>
394 %         <Label htmlFor="eventDate">Fecha del Evento</Label>
395 %         <Input
396 %           id="eventDate"
397 %           type="date"
398 %           value={formData.eventDate || ""}
399 %           onChange={(e) => setFormData({ ...formData, eventDate: e.target.value })}
400 %           data-testid="input-group-event-date"
401 %         />
402 %       </div>
403 %
404 %       <div>
405 %         <Label htmlFor="releaseDate">Fecha de Release</Label>
406 %         <Input
407 %           id="releaseDate"
408 %           type="date"
409 %           value={formData.releaseDate || ""}
410 %           onChange={(e) => setFormData({ ...formData, releaseDate: e.target.value })}
411 %           data-testid="input-group-release-date"
412 %         />
413 %       </div>
414 %
415 %       <div>
416 %         <Label htmlFor="status">Estado</Label>
417 %         <Select
418 %           value={formData.status}
419 %           onChange={(value) => setFormData({ ...formData, status: value as GroupStatus })}
420 %         >
421 %           <SelectTrigger data-testid="select-group-status">
422 %             <SelectValue />
423 %           </SelectTrigger>
424 %           <SelectContent>
425 %             <SelectItem value="tentative">Tentativo</SelectItem>
426 %             <SelectItem value="blocked">Bloqueado</SelectItem>
427 %             <SelectItem value="confirmed">Confirmado</SelectItem>
428 %             <SelectItem value="inhouse">En Casa</SelectItem>
429 %             <SelectItem value="finished">Finalizado</SelectItem>
430 %             <SelectItem value="cancelled">Cancelado</SelectItem>
431 %           </SelectContent>
432 %         </Select>
433 %       </div>
434 %
435 %       <div className="col-span-2">
436 %         <Label htmlFor="notes">Notas</Label>
437 %         <Textarea
438 %           id="notes"
439 %           value={formData.notes || ""}
440 %           onChange={(e) => setFormData({ ...formData, notes: e.target.value })}
441 %           placeholder="Notas adicionales..."
442 %           className="resize-none"
443 %           rows={3}
444 %           data-testid="textarea-group-notes"
445 %         />
446 %       </div>
447 %     </div>
448 %
449 %     <DialogFooter>
450 %       <Button type="button" variant="outline" onClick={() => onOpenChange(false)}>
451 %         Cancelar
452 %       </Button>
453 %       {isEditing ? (
454 %         <Button onClick={() => updateMutation.mutate(formData)} disabled={isPending} data-testid="button-save..."
455 %           {isPending ? "Guardando..." : "Guardar Cambios"}
456 %         </Button>
457 %       ) : (
458 %         <Button type="button" onClick={handleNext} data-testid="button-next-step">
459 %           Siguiente
460 %         </Button>
461 %       )}
462 %     </DialogFooter>
```

client/src/pages/groups.tsx (cont.)

```
463 %         </div>
464 %     )}
465 %
466 %     {step === 2 && !isEditing && (
467 %         <div className="space-y-4">
468 %             <div className="flex items-center justify-between">
469 %                 <p className="text-sm text-muted-foreground">
470 %                     Agregue los bloques de habitaciones para el grupo. Puede configurar fechas y tarifas diferentes para ...
471 %                 </p>
472 %                 <Button type="button" variant="outline" size="sm" onClick={addBlock} data-testid="button-add-block-wiza...
473 %                     <Plus className="mr-2 h-4 w-4" />
474 %                     Agregar Bloque
475 %                 </Button>
476 %             </div>
477 %
478 %             {blocks.length === 0 ? (
479 %                 <div className="rounded-md border border-dashed p-8 text-center">
480 %                     <Hotel className="mx-auto h-10 w-10 text-muted-foreground/50" />
481 %                     <p className="mt-2 text-sm text-muted-foreground">
482 %                         No hay bloques agregados. Puede agregar bloques ahora o después desde el detalle del grupo.
483 %                     </p>
484 %                 </div>
485 %             ) : (
486 %                 <div className="space-y-4">
487 %                     {blocks.map((block, index) => (
488 %                         <div key={block.id} className={`rounded-md border p-4 space-y-3 ${block.error ? "border-destructive...
489 %                             <div className="flex items-center justify-between">
490 %                                 <div className="flex items-center gap-2">
491 %                                     <span className="font-medium text-sm">Bloque {index + 1}</span>
492 %                                     {block.error && <span className="text-xs text-destructive">{block.error}</span>}
493 %                                 </div>
494 %                                 <Button
495 %                                     type="button"
496 %                                     variant="ghost"
497 %                                     size="icon"
498 %                                     onClick={() => removeBlock(block.id)}
499 %                                     data-testid={`button-remove-block-${index}`}
500 %                                 >
501 %                                     <TrashIcon className="h-4 w-4" />
502 %                                 </Button>
503 %                             </div>
504 %
505 %                             <div className="grid grid-cols-3 gap-3">
506 %                                 <div>
507 %                                     <Label className="text-xs">Tipo de Habitación *</Label>
508 %                                     <Select
509 %                                         value={block.roomTypeId}
510 %                                         onChange={(v) => updateBlock(block.id, { roomTypeId: v })}
511 %                                     >
512 %                                         <SelectTrigger data-testid={`select-block-type-${index}`}>
513 %                                             <SelectValue placeholder="Seleccionar" />
514 %                                         </SelectTrigger>
515 %                                         <SelectContent>
516 %                                             {roomTypes?.map((rt) => (
517 %                                                 <SelectItem key={rt.id} value={rt.id}>
518 %                                                     {rt.name}
519 %                                                 </SelectItem>
520 %                                             )]}
521 %                                         </SelectContent>
522 %                                     </Select>
523 %                                 </div>
524 %
525 %                                 <div>
526 %                                     <Label className="text-xs">Cantidad *</Label>
527 %                                     <Input
528 %                                         type="number"
529 %                                         min={1}
530 %                                         value={block.quantity}
531 %                                         onChange={(e) => updateBlock(block.id, { quantity: parseInt(e.target.value) || 1 }}
532 %                                         data-testid={`input-block-qty-${index}`}
533 %                                     />
534 %                                 </div>
535 %
536 %                                 <div>
537 %                                     <Label className="text-xs">Tarifa Acordada</Label>
538 %                                     <Input
539 %                                         type="number"
540 %                                         step="0.01"
541 %                                         min={0}
542 %                                         value={block.agreedRate}
543 %                                         onChange={(e) => updateBlock(block.id, { agreedRate: e.target.value })}
544 %                                         placeholder="0.00"
545 %                                         data-testid={`input-block-rate-${index}`}
546 %                                     />
547 %                                 </div>
548 %                             </div>
549 %
550 %                             <div className="flex items-center gap-2">
551 %                                 <input
552 %                                     type="checkbox"
553 %                                     id={`customDates-${block.id}`}
554 %                                     checked={block.useCustomDates}
555 %                                     onChange={(e) => updateBlock(block.id, { useCustomDates: e.target.checked })}
```

client/src/pages/groups.tsx (cont.)

```
556 %         className="h-4 w-4"
557 %         data-testid={`checkbox-custom-dates-${index}`}
558 %       />
559 %       <Label htmlFor={`customDates-${block.id}`} className="text-xs font-normal">
560 %         Fechas diferentes al grupo
561 %       </Label>
562 %     </div>
563 %
564 %     {block.useCustomDates && (
565 %       <div className="grid grid-cols-2 gap-3 rounded-md bg-muted/30 p-3">
566 %         <div>
567 %           <Label className="text-xs">Check-in Bloque</Label>
568 %           <Input
569 %             type="date"
570 %             value={block.blockCheckInDate}
571 %             onChange={(e) => updateBlock(block.id, { blockCheckInDate: e.target.value })}
572 %             data-testid={`input-block-checkin-${index}`}
573 %           />
574 %         </div>
575 %         <div>
576 %           <Label className="text-xs">Check-out Bloque</Label>
577 %           <Input
578 %             type="date"
579 %             value={block.blockCheckOutDate}
580 %             onChange={(e) => updateBlock(block.id, { blockCheckOutDate: e.target.value })}
581 %             data-testid={`input-block-checkout-${index}`}
582 %           />
583 %         </div>
584 %       </div>
585 %     )}
586 %   </div>
587 % )}}
588 % </div>
589 % )}
590 %
591 % <DialogFooter className="gap-2">
592 %   {!createdGroupId && (
593 %     <Button type="button" variant="outline" onClick={handleBack}>
594 %       Anterior
595 %     </Button>
596 %   )}
597 %   {createdGroupId && (
598 %     <Button
599 %       type="button"
600 %       variant="outline"
601 %       onClick={() => {
602 %         onSuccess();
603 %         onOpenChange(false);
604 %         setStep(1);
605 %         setBlocks([]);
606 %         setCreatedGroupId(null);
607 %       }}
608 %     >
609 %       Terminar sin bloques
610 %     </Button>
611 %   )}
612 %   <Button onClick={handleCreate} disabled={isPending} data-testid="button-create-group">
613 %     {isPending ? "Creando..." : createdGroupId ? "Reintentar Bloques" : "Crear Grupo"}
614 %   </Button>
615 % </DialogFooter>
616 % </div>
617 % )}
618 % </DialogContent>
619 % </Dialog>
620 % );
621 % }
622 %
623 % export default function GroupsPage() {
624 %   const [, navigate] = useLocation();
625 %   const { toast } = useToast();
626 %   const [searchQuery, setSearchQuery] = useState("");
627 %   const [statusFilter, setStatusFilter] = useState<string>("all");
628 %   const [showFormDialog, setShowFormDialog] = useState(false);
629 %   const [editingGroup, setEditingGroup] = useState<GroupWithDetails | undefined>();
630 %   const [deleteConfirmGroup, setDeleteConfirmGroup] = useState<GroupWithDetails | null>(null);
631 %
632 %   const { data: groups, isLoading } = useQuery<GroupWithDetails[]>({
633 %     queryKey: ["/api/groups"],
634 %   });
635 %
636 %   const deleteMutation = useMutation({
637 %     mutationFn: (id: string) => apiRequest("DELETE", `/api/groups/${id}`),
638 %     onSuccess: () => {
639 %       queryClient.invalidateQueries({ queryKey: ["/api/groups"] });
640 %       toast({ title: "Grupo eliminado" });
641 %       setDeleteConfirmGroup(null);
642 %     },
643 %     onError: () => {
644 %       toast({ title: "Error al eliminar grupo", variant: "destructive" });
645 %     },
646 %   });
647 %
648 %   const filteredGroups = groups?.filter((group) => {
```


client/src/pages/groups.tsx (cont.)

```
649 %     const matchesSearch =
650 %       group.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
651 %       group.groupCode.toLowerCase().includes(searchQuery.toLowerCase()) ||
652 %       group.contactName?.toLowerCase().includes(searchQuery.toLowerCase());
653 %     const matchesStatus = statusFilter === "all" || group.status === statusFilter;
654 %     return matchesSearch && matchesStatus;
655 %   });
656 %
657 %   const handleEdit = (group: GroupWithDetails) => {
658 %     setEditingGroup(group);
659 %     setShowFormDialog(true);
660 %   };
661 %
662 %   const handleViewDetail = (group: GroupWithDetails) => {
663 %     navigate(`/groups/${group.id}`);
664 %   };
665 %
666 %   const formatDate = (dateStr: string) => {
667 %     return new Date(dateStr).toLocaleDateString("es-AR", {
668 %       day: "2-digit",
669 %       month: "short",
670 %       year: "numeric",
671 %     });
672 %   };
673 %
674 %   return (
675 %     <div className="flex flex-col gap-6 p-6">
676 %       <div className="flex flex-col gap-1">
677 %         <h1 className="text-3xl font-bold tracking-tight" data-testid="text-groups-title">
678 %           Grupos
679 %         </h1>
680 %         <p className="text-muted-foreground">
681 %           Gestión de reservas grupales y eventos
682 %         </p>
683 %       </div>
684 %
685 %       <div className="flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-between">
686 %         <div className="flex flex-1 items-center gap-4">
687 %           <div className="relative flex-1 max-w-md">
688 %             <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
689 %             <Input
690 %               placeholder="Buscar por nombre, código o contacto..."
691 %               value={searchQuery}
692 %               onChange={(e) => setSearchQuery(e.target.value)}
693 %               className="pl-10"
694 %               data-testid="input-search-groups"
695 %             />
696 %           </div>
697 %           <Select value={statusFilter} onValueChange={setStatusFilter}>
698 %             <SelectTrigger className="w-40" data-testid="select-filter-status">
699 %               <SelectValue placeholder="Estado" />
700 %             </SelectTrigger>
701 %             <SelectContent>
702 %               <SelectItem value="all">Todos</SelectItem>
703 %               <SelectItem value="tentative">Tentativo</SelectItem>
704 %               <SelectItem value="blocked">Bloqueado</SelectItem>
705 %               <SelectItem value="confirmed">Confirmado</SelectItem>
706 %               <SelectItem value="inhouse">En Casa</SelectItem>
707 %               <SelectItem value="finished">Finalizado</SelectItem>
708 %               <SelectItem value="cancelled">Cancelado</SelectItem>
709 %             </SelectContent>
710 %           </Select>
711 %         </div>
712 %         <Button onClick={() => { setEditingGroup(undefined); setShowFormDialog(true); }} data-testid="button-new-group">
713 %           <Plus className="mr-2 h-4 w-4" />
714 %           Nuevo Grupo
715 %         </Button>
716 %       </div>
717 %
718 %       <Card>
719 %         <CardHeader className="pb-3">
720 %           <CardTitle className="flex items-center gap-2">
721 %             <Users2 className="h-5 w-5" />
722 %             Listado de Grupos
723 %           </CardTitle>
724 %         </CardHeader>
725 %         <CardContent>
726 %           {isLoading ? (
727 %             <div className="space-y-2">
728 %               {[...Array(5)].map((_, i) => (
729 %                 <Skeleton key={i} className="h-16 w-full" />
730 %               ))}
731 %             </div>
732 %           ) : filteredGroups && filteredGroups.length > 0 ? (
733 %             <Table>
734 %               <TableHeader>
735 %                 <TableRow>
736 %                   <TableHead>Código</TableHead>
737 %                   <TableHead>Nombre</TableHead>
738 %                   <TableHead>Fechas</TableHead>
739 %                   <TableHead>Contacto</TableHead>
740 %                   <TableHead>Habitaciones</TableHead>
741 %                   <TableHead>Estado</TableHead>
```

client/src/pages/groups.tsx (cont.)

```
742 %         <TableHead className="w-[80px]">Acciones</TableHead>
743 %       </TableRow>
744 %     </TableHeader>
745 %     <TableBody>
746 %       {filteredGroups.map((group) => (
747 %         <TableRow key={group.id} data-testid={`row-group-${group.id}`}>
748 %           <TableCell className="font-mono text-sm">{group.groupCode}</TableCell>
749 %           <TableCell className="font-medium">{group.name}</TableCell>
750 %           <TableCell>
751 %             <div className="flex items-center gap-1 text-sm">
752 %               <Calendar className="h-3.5 w-3.5 text-muted-foreground" />
753 %               {formatDate(group.checkInDate)} - {formatDate(group.checkOutDate)}
754 %             </div>
755 %           </TableCell>
756 %           <TableCell>
757 %             {group.contactName && (
758 %               <div className="flex flex-col gap-0.5 text-sm">
759 %                 <span>{group.contactName}</span>
760 %                 {group.contactPhone && (
761 %                   <span className="flex items-center gap-1 text-muted-foreground">
762 %                     <Phone className="h-3 w-3" />
763 %                     {group.contactPhone}
764 %                   </span>
765 %                 )}
766 %               </div>
767 %             )}
768 %           </TableCell>
769 %           <TableCell>
770 %             <div className="flex items-center gap-2">
771 %               <DoorOpen className="h-4 w-4 text-muted-foreground" />
772 %               <span>{group.assignedRooms} / {group.totalRooms}</span>
773 %             </div>
774 %           </TableCell>
775 %           <TableCell>
776 %             <GroupStatusBadge status={group.status} />
777 %           </TableCell>
778 %           <TableCell>
779 %             <DropdownMenu>
780 %               <DropdownMenuTrigger asChild>
781 %                 <Button variant="ghost" size="icon" data-testid={`button-actions-${group.id}`}>
782 %                   <MoreHorizontal className="h-4 w-4" />
783 %                 </Button>
784 %               </DropdownMenuTrigger>
785 %               <DropdownMenuContent align="end">
786 %                 <DropdownMenuItem onClick={() => handleViewDetail(group)}>
787 %                   <Eye className="mr-2 h-4 w-4" />
788 %                   Ver Detalle
789 %                 </DropdownMenuItem>
790 %                 <DropdownMenuItem onClick={() => handleEdit(group)}>
791 %                   <Pencil className="mr-2 h-4 w-4" />
792 %                   Editar
793 %                 </DropdownMenuItem>
794 %                 <DropdownMenuSeparator />
795 %                 <DropdownMenuItem
796 %                   className="text-destructive"
797 %                   onClick={() => setDeleteConfirmGroup(group)}>
798 %                   >
799 %                     <Trash2 className="mr-2 h-4 w-4" />
800 %                     Eliminar
801 %                   </DropdownMenuItem>
802 %                 </DropdownMenuContent>
803 %               </DropdownMenu>
804 %             </TableCell>
805 %           </TableRow>
806 %         )})
807 %       </TableBody>
808 %     </Table>
809 %   ) : (
810 %     <div className="flex flex-col items-center justify-center py-12 text-center">
811 %       <Users2 className="h-12 w-12 text-muted-foreground/50" />
812 %       <h3 className="mt-4 text-lg font-semibold">No hay grupos</h3>
813 %       <p className="mt-2 text-sm text-muted-foreground">
814 %         {searchQuery || statusFilter !== "all"
815 %           ? "No se encontraron grupos con los filtros aplicados"
816 %           : "Comience creando un nuevo grupo para gestionar reservas grupales"}
817 %       </p>
818 %     </div>
819 %   )}
820 % </CardContent>
821 % </Card>
822 %
823 % <GroupFormDialog
824 %   group={editingGroup}
825 %   open={showFormDialog}
826 %   onOpenChange={setShowFormDialog}
827 %   onSuccess={() => setEditingGroup(undefined)}
828 % />
829 %
830 % <Dialog open={!deleteConfirmGroup} onOpenChange={() => setDeleteConfirmGroup(null)}>
831 %   <DialogContent>
832 %     <DialogHeader>
833 %       <DialogTitle>Confirmar Eliminación</DialogTitle>
834 %       <DialogDescription>
```

client/src/pages/groups.tsx (cont.)

```
835 %           ¿Está seguro que desea eliminar el grupo "{deleteConfirmGroup?.name}"?
836 %           Esta acción eliminará todos los bloques asociados.
837 %         </DialogDescription>
838 %       </DialogHeader>
839 %     <DialogFooter>
840 %       <Button variant="outline" onClick={() => setDeleteConfirmGroup(null)}>
841 %         Cancelar
842 %       </Button>
843 %       <Button
844 %         variant="destructive"
845 %         onClick={() => deleteConfirmGroup && deleteMutation.mutate(deleteConfirmGroup.id)}
846 %         disabled={deleteMutation.isPending}
847 %         data-testid="button-confirm-delete"
848 %       >
849 %         {deleteMutation.isPending ? "Eliminando..." : "Eliminar"}
850 %       </Button>
851 %     </DialogFooter>
852 %   </DialogContent>
853 % </Dialog>
854 % </div>
855 % );
856 % }
857 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { useLocation, useParams } from "wouter";
4 % import {
5 %   ArrowLeft,
6 %   Users2,
7 %   Plus,
8 %   Calendar,
9 %   Phone,
10 %   Mail,
11 %   DoorOpen,
12 %   Trash2,
13 %   User,
14 %   Hotel,
15 %   LogIn,
16 %   LogOut,
17 %   FileText,
18 %   Printer,
19 %   AlertCircle,
20 %   CheckCircle,
21 %   CreditCard,
22 %   DollarSign,
23 % } from "lucide-react";
24 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
25 % import { Button } from "@components/ui/button";
26 % import { Input } from "@components/ui/input";
27 % import { Badge } from "@components/ui/badge";
28 % import { Skeleton } from "@components/ui/skeleton";
29 % import {
30 %   Dialog,
31 %   DialogContent,
32 %   DialogDescription,
33 %   DialogFooter,
34 %   DialogHeader,
35 %   DialogTitle,
36 % } from "@components/ui/dialog";
37 % import {
38 %   Select,
39 %   SelectContent,
40 %   SelectItem,
41 %   SelectTrigger,
42 %   SelectValue,
43 % } from "@components/ui/select";
44 % import {
45 %   Table,
46 %   TableBody,
47 %   TableCell,
48 %   TableHead,
49 %   TableHeader,
50 %   TableRow,
51 % } from "@components/ui/table";
52 % import { Label } from "@components/ui/label";
53 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
54 % import {
55 %   AlertDialog,
56 %   AlertDialogAction,
57 %   AlertDialogCancel,
58 %   AlertDialogContent,
59 %   AlertDialogDescription,
60 %   AlertDialogFooter,
61 %   AlertDialogHeader,
62 %   AlertDialogTitle,
63 % } from "@components/ui/alert-dialog";
64 % import { useToast } from "@hooks/use-toast";
65 % import { queryClient, apiRequest } from "@lib/queryClient";
66 % import type {
67 %   GroupWithDetails,
68 %   GroupStatus,
69 %   GroupRoomBlockWithDetails,
70 %   RoomType,
71 %   RatePlan,
72 %   RoomWithType,
73 %   ReservationWithDetails,
74 % } from "@shared/schema";
75 %
76 % function GroupStatusBadge({ status }: { status: GroupStatus }) {
77 %   const statusConfig: Record<GroupStatus, { label: string; variant: "default" | "secondary" | "destructive" | "outline" }> = {
78 %     tentative: { label: "Tentativo", variant: "outline" },
79 %     blocked: { label: "Bloqueado", variant: "secondary" },
80 %     confirmed: { label: "Confirmado", variant: "default" },
81 %     inhouse: { label: "En Casa", variant: "default" },
82 %     finished: { label: "Finalizado", variant: "outline" },
83 %     cancelled: { label: "Cancelado", variant: "destructive" },
84 %   };
85 %
86 %   const config = statusConfig[status];
87 %   return <Badge variant={config.variant}>{config.label}</Badge>;
88 % }
89 %
90 % function AddBlockDialog({
```

client/src/pages/group-detail.tsx (cont.)

```
91 %   groupId,
92 %   group,
93 %   open,
94 %   onOpenChange,
95 %   onSuccess,
96 % } : {
97 %   groupId: string;
98 %   group: GroupWithDetails;
99 %   open: boolean;
100 %   onOpenChange: (open: boolean) => void;
101 %   onSuccess: () => void;
102 % } ) {
103 %   const { toast } = useToast();
104 %   const [roomId, setRoomTypeId] = useState("");
105 %   const [quantity, setQuantity] = useState(1);
106 %   const [ratePlanId, setRatePlanId] = useState("");
107 %   const [agreedRate, setAgreedRate] = useState("");
108 %   const [useCustomDates, setUseCustomDates] = useState(false);
109 %   const [blockCheckInDate, setBlockCheckInDate] = useState(group.checkInDate);
110 %   const [blockCheckOutDate, setBlockCheckOutDate] = useState(group.checkOutDate);
111 %
112 %   const { data: roomTypes } = useQuery<RoomType[]>({
113 %     queryKey: ["/api/room-types"],
114 %   });
115 %
116 %   const { data: ratePlans } = useQuery<RatePlan[]>({
117 %     queryKey: ["/api/rate-plans/by-room-type", roomId],
118 %     queryFn: async () => {
119 %       if (!roomId) return [];
120 %       const res = await fetch(`/api/rate-plans/by-room-type/${roomId}`);
121 %       return res.json();
122 %     },
123 %     enabled: !!roomId,
124 %   });
125 %
126 %   const createMutation = useMutation({
127 %     mutationFn: () =>
128 %       apiRequest("POST", `/api/groups/${groupId}/blocks`, {
129 %         roomId,
130 %         quantity: Number(quantity),
131 %         ratePlanId: ratePlanId || null,
132 %         agreedRate: agreedRate ? parseFloat(agreedRate).toFixed(2) : null,
133 %         blockCheckInDate: useCustomDates ? blockCheckInDate : null,
134 %         blockCheckOutDate: useCustomDates ? blockCheckOutDate : null,
135 %       }),
136 %     onSuccess: () => {
137 %       queryClient.invalidateQueries({ queryKey: ["/api/groups", groupId] });
138 %       toast({ title: "Bloque agregado exitosamente" });
139 %       onSuccess();
140 %       onOpenChange(false);
141 %       setRoomTypeId("");
142 %       setQuantity(1);
143 %       setRatePlanId("");
144 %       setAgreedRate("");
145 %       setUseCustomDates(false);
146 %       setBlockCheckInDate(group.checkInDate);
147 %       setBlockCheckOutDate(group.checkOutDate);
148 %     },
149 %     onError: () => {
150 %       toast({ title: "Error al agregar bloque", variant: "destructive" });
151 %     },
152 %   });
153 %
154 %   const handleRatePlanChange = (planId: string) => {
155 %     setRatePlanId(planId);
156 %     const plan = ratePlans?.find(p => p.id === planId);
157 %     if (plan) {
158 %       setAgreedRate(plan.baseRate);
159 %     }
160 %   };
161 %
162 %   return (
163 %     <Dialog open={open} onOpenChange={onOpenChange}>
164 %       <DialogContent className="max-w-lg">
165 %         <DialogHeader>
166 %           <DialogTitle>Agregar Bloque de Habitaciones</DialogTitle>
167 %           <DialogDescription>
168 %             Defina el tipo, cantidad y tarifa para este bloque
169 %           </DialogDescription>
170 %         </DialogHeader>
171 %         <div className="space-y-4">
172 %           <div className="grid grid-cols-2 gap-4">
173 %             <div>
174 %               <Label>Tipo de Habitación *</Label>
175 %               <Select value={roomId} onValueChange={setRoomTypeId}>
176 %                 <SelectTrigger data-testid="select-block-room-type">
177 %                   <SelectValue placeholder="Seleccionar tipo" />
178 %                 </SelectTrigger>
179 %                 <SelectContent>
180 %                   {roomTypes?.map((rt) => (
181 %                     <SelectItem key={rt.id} value={rt.id}>
182 %                       {rt.name} {rt.code}
183 %                     </SelectItem>
```

client/src/pages/group-detail.tsx (cont.)

```
184 %         )}}
185 %       </SelectContent>
186 %     </Select>
187 %   </div>
188 %
189 %   <div>
190 %     <Label>Cantidad *</Label>
191 %     <Input
192 %       type="number"
193 %       min={1}
194 %       value={quantity}
195 %       onChange={(e) => setQuantity(parseInt(e.target.value) || 1)}
196 %       data-testid="input-block-quantity"
197 %     />
198 %   </div>
199 % </div>
200 %
201 % <div className="grid grid-cols-2 gap-4">
202 %   <div>
203 %     <Label>Plan Tarifario</Label>
204 %     <Select value={ratePlanId} onValueChange={handleRatePlanChange} disabled={!roomTypeId}>
205 %       <SelectTrigger data-testid="select-block-rate-plan">
206 %         <SelectValue placeholder="Seleccionar plan" />
207 %       </SelectTrigger>
208 %       <SelectContent>
209 %         {ratePlans?.map((rp) => (
210 %           <SelectItem key={rp.id} value={rp.id}>
211 %             {rp.name} - ${rp.baseRate}
212 %           </SelectItem>
213 %         ))}
214 %       </SelectContent>
215 %     </Select>
216 %   </div>
217 %
218 %   <div>
219 %     <Label>Tarifa Acordada</Label>
220 %     <Input
221 %       type="number"
222 %       step="0.01"
223 %       min={0}
224 %       value={agreedRate}
225 %       onChange={(e) => setAgreedRate(e.target.value)}
226 %       placeholder="0.00"
227 %       data-testid="input-block-agreed-rate"
228 %     />
229 %   </div>
230 % </div>
231 %
232 % <div className="flex items-center gap-2 pt-2">
233 %   <input
234 %     type="checkbox"
235 %     id="useCustomDates"
236 %     checked={useCustomDates}
237 %     onChange={(e) => setUseCustomDates(e.target.checked)}
238 %     className="h-4 w-4"
239 %     data-testid="checkbox-custom-dates"
240 %   />
241 %   <Label htmlFor="useCustomDates" className="font-normal">
242 %     Usar fechas diferentes al grupo
243 %   </Label>
244 % </div>
245 %
246 % {useCustomDates && (
247 %   <div className="grid grid-cols-2 gap-4 rounded-md border p-3 bg-muted/30">
248 %     <div>
249 %       <Label>Check-in Bloque</Label>
250 %       <Input
251 %         type="date"
252 %         value={blockCheckInDate}
253 %         onChange={(e) => setBlockCheckInDate(e.target.value)}
254 %         data-testid="input-block-checkin"
255 %       />
256 %     </div>
257 %     <div>
258 %       <Label>Check-out Bloque</Label>
259 %       <Input
260 %         type="date"
261 %         value={blockCheckOutDate}
262 %         onChange={(e) => setBlockCheckOutDate(e.target.value)}
263 %         data-testid="input-block-checkout"
264 %       />
265 %     </div>
266 %   </div>
267 % )}
268 %
269 % <div className="rounded-md bg-muted p-3 text-sm text-muted-foreground">
270 %   <p>Fechas del grupo: {new Date(group.checkInDate).toLocaleDateString("es-AR")} - {new Date(group.checkOut...
271 % </div>
272 % </div>
273 %
274 % <DialogFooter>
275 %   <Button variant="outline" onClick={() => onOpenChange(false)}>
276 %     Cancelar
```

client/src/pages/group-detail.tsx (cont.)

```
277 %         </Button>
278 %         <Button
279 %             onClick={() => createMutation.mutate()}
280 %             disabled={!roomId || createMutation.isPending}
281 %             data-testid="button-save-block"
282 %         >
283 %             {createMutation.isPending ? "Guardando..." : "Agregar Bloque"}
284 %         </Button>
285 %     </DialogFooter>
286 % </DialogContent>
287 % </Dialog>
288 % );
289 % }
290 %
291 % function AssignRoomDialog({
292 %     group,
293 %     block,
294 %     open,
295 %     onOpenChange,
296 %     onSuccess,
297 % }: {
298 %     group: GroupWithDetails;
299 %     block: GroupRoomBlockWithDetails;
300 %     open: boolean;
301 %     onOpenChange: (open: boolean) => void;
302 %     onSuccess: () => void;
303 % }) {
304 %     const { toast } = useToast();
305 %     const [selectedRoomId, setSelectedRoomId] = useState("");
306 %     const [guestFirstName, setGuestFirstName] = useState("Sin Asignar");
307 %     const [guestLastName, setGuestLastName] = useState("");
308 %
309 %     // Get default dates from block or group
310 %     const defaultCheckIn = block.blockCheckInDate || group.checkInDate;
311 %     const defaultCheckOut = block.blockCheckOutDate || group.checkOutDate;
312 %     const defaultRate = block.agreedRate || "";
313 %
314 %     const [useCustomDates, setUseCustomDates] = useState(false);
315 %     const [checkInDate, setCheckInDate] = useState(defaultCheckIn);
316 %     const [checkOutDate, setCheckOutDate] = useState(defaultCheckOut);
317 %     const [useCustomRate, setUseCustomRate] = useState(false);
318 %     const [agreedRate, setAgreedRate] = useState(defaultRate);
319 %
320 %     const { data: rooms } = useQuery<RoomWithType[]>({
321 %         queryKey: ["/api/rooms"],
322 %     });
323 %
324 %     const availableRooms = rooms?.filter(
325 %         (r) => r.roomTypeId === block.roomTypeId && r.status === "available"
326 %     );
327 %
328 %     const assignMutation = useMutation({
329 %         mutationFn: () =>
330 %             apiRequest("POST", `/api/groups/${group.id}/assign-room`, {
331 %                 roomId: selectedRoomId,
332 %                 guestFirstName,
333 %                 guestLastName,
334 %                 checkInDate: useCustomDates ? checkInDate : undefined,
335 %                 checkOutDate: useCustomDates ? checkOutDate : undefined,
336 %                 agreedRate: useCustomRate ? agreedRate : undefined,
337 %                 ratePlanId: block.ratePlanId,
338 %             }),
339 %         onSuccess: () => {
340 %             queryClient.invalidateQueries({ queryKey: ["/api/groups", group.id] });
341 %             queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
342 %             toast({ title: "Habitación asignada exitosamente" });
343 %             onSuccess();
344 %             onOpenChange(false);
345 %             setSelectedRoomId("");
346 %             setGuestFirstName("Sin Asignar");
347 %             setGuestLastName("");
348 %             setUseCustomDates(false);
349 %             setUseCustomRate(false);
350 %         },
351 %         onError: () => {
352 %             toast({ title: "Error al asignar habitación", variant: "destructive" });
353 %         },
354 %     });
355 %
356 %     return (
357 %         <Dialog open={open} onOpenChange={onOpenChange}>
358 %             <DialogContent className="max-w-lg">
359 %                 <DialogHeader>
360 %                     <DialogTitle>Asignar Habitación</DialogTitle>
361 %                     <DialogDescription>
362 %                         Asigne una habitación {block.roomType.name} al grupo
363 %                     </DialogDescription>
364 %                 </DialogHeader>
365 %                 <div className="space-y-4">
366 %                     <div>
367 %                         <Label>Habitación *</Label>
368 %                         <Select value={selectedRoomId} onValueChange={setSelectedRoomId}>
369 %                             <SelectTrigger data-testid="select-assign-room">
```

client/src/pages/group-detail.tsx (cont.)

```
370 %         <SelectValue placeholder="Seleccionar habitación" />
371 %     </SelectTrigger>
372 %     <SelectContent>
373 %         {availableRooms?.length === 0 ? (
374 %             <SelectItem value="" disabled>
375 %                 No hay habitaciones disponibles
376 %             </SelectItem>
377 %         ) : (
378 %             availableRooms?.map((room) => (
379 %                 <SelectItem key={room.id} value={room.id}>
380 %                     {room.roomNumber} - Piso {room.floor}
381 %                 </SelectItem>
382 %             ))
383 %         )}
384 %     </SelectContent>
385 % </Select>
386 % </div>
387 %
388 % <div className="grid grid-cols-2 gap-4">
389 %     <div>
390 %         <Label>Nombre Ocupante</Label>
391 %         <Input
392 %             value={guestFirstName}
393 %             onChange={(e) => setGuestFirstName(e.target.value)}
394 %             placeholder="Nombre"
395 %             data-testid="input-assign-firstname"
396 %         />
397 %     </div>
398 %     <div>
399 %         <Label>Apellido Ocupante</Label>
400 %         <Input
401 %             value={guestLastName}
402 %             onChange={(e) => setGuestLastName(e.target.value)}
403 %             placeholder="Apellido"
404 %             data-testid="input-assign-lastname"
405 %         />
406 %     </div>
407 % </div>
408 %
409 % <div className="flex items-center gap-2 pt-2">
410 %     <input
411 %         type="checkbox"
412 %         id="useCustomDatesAssign"
413 %         checked={useCustomDates}
414 %         onChange={(e) => setUseCustomDates(e.target.checked)}
415 %         className="h-4 w-4"
416 %         data-testid="checkbox-assign-custom-dates"
417 %     />
418 %     <Label htmlFor="useCustomDatesAssign" className="font-normal">
419 %         Usar fechas personalizadas
420 %     </Label>
421 % </div>
422 %
423 % {useCustomDates && (
424 %     <div className="grid grid-cols-2 gap-4 rounded-md border p-3 bg-muted/30">
425 %         <div>
426 %             <Label>Check-in</Label>
427 %             <Input
428 %                 type="date"
429 %                 value={checkInDate}
430 %                 onChange={(e) => setCheckInDate(e.target.value)}
431 %                 data-testid="input-assign-checkin"
432 %             />
433 %         </div>
434 %         <div>
435 %             <Label>Check-out</Label>
436 %             <Input
437 %                 type="date"
438 %                 value={checkOutDate}
439 %                 onChange={(e) => setCheckOutDate(e.target.value)}
440 %                 data-testid="input-assign-checkout"
441 %             />
442 %         </div>
443 %     </div>
444 % )}
445 %
446 % <div className="flex items-center gap-2">
447 %     <input
448 %         type="checkbox"
449 %         id="useCustomRateAssign"
450 %         checked={useCustomRate}
451 %         onChange={(e) => setUseCustomRate(e.target.checked)}
452 %         className="h-4 w-4"
453 %         data-testid="checkbox-assign-custom-rate"
454 %     />
455 %     <Label htmlFor="useCustomRateAssign" className="font-normal">
456 %         Usar tarifa personalizada
457 %     </Label>
458 % </div>
459 %
460 % {useCustomRate && (
461 %     <div className="rounded-md border p-3 bg-muted/30">
462 %         <Label>Tarifa por Noche</Label>
```


client/src/pages/group-detail.tsx (cont.)

client/src/pages/group-detail.tsx (cont.)

```
556 %         variant: "destructive"
557 %     });
558 % }
559 % },
560 % onError: () => {
561 %     toast({ title: "Error en check-in grupal", variant: "destructive" });
562 % },
563 % });
564 %
565 % const checkOutAllMutation = useMutation({
566 %     mutationFn: async () => {
567 %         const res = await apiRequest("POST", `/api/groups/${groupId}/check-out-all`);
568 %         return res.json() as Promise<{ success: number; failed: number; errors: string[] }>;
569 %     },
570 %     onSuccess: (data) => {
571 %         queryClient.invalidateQueries({ queryKey: ["/api/groups", groupId] });
572 %         if (data.success > 0 && data.failed === 0) {
573 %             toast({ title: `Check-out grupal exitoso`, description: `${data.success} habitaciones procesadas` });
574 %         } else if (data.success > 0 && data.failed > 0) {
575 %             toast({
576 %                 title: `Check-out parcial`,
577 %                 description: `${data.success} exitosos, ${data.failed} con saldo pendiente`,
578 %                 variant: "destructive"
579 %             });
580 %         } else if (data.failed > 0) {
581 %             toast({
582 %                 title: `Check-out bloqueado`,
583 %                 description: data.errors.join(", "),
584 %                 variant: "destructive"
585 %             });
586 %         }
587 %     },
588 %     onError: () => {
589 %         toast({ title: "Error en check-out grupal", variant: "destructive" });
590 %     },
591 % });
592 %
593 % const groupPaymentMutation = useMutation({
594 %     mutationFn: async () => {
595 %         return apiRequest("POST", `/api/groups/${groupId}/payment`, {
596 %             amount: groupPaymentAmount,
597 %             method: groupPaymentMethod,
598 %             reference: groupPaymentReference,
599 %             receiptType: groupPaymentReceiptType,
600 %             distribution: groupPaymentDistribution,
601 %         });
602 %     },
603 %     onSuccess: async () => {
604 %         queryClient.invalidateQueries({ queryKey: ["/api/groups", groupId] });
605 %         toast({ title: "Pago grupal registrado exitosamente" });
606 %         setShowGroupPaymentDialog(false);
607 %         setGroupPaymentAmount("");
608 %         setGroupPaymentMethod("");
609 %         setGroupPaymentReference("");
610 %         setGroupPaymentReceiptType("");
611 %         setGroupPaymentDistribution("equal");
612 %         if (showInvoiceDialog) {
613 %             loadInvoice();
614 %         }
615 %     },
616 %     onError: () => {
617 %         toast({ title: "Error al registrar pago grupal", variant: "destructive" });
618 %     },
619 % });
620 %
621 % const loadInvoice = async () => {
622 %     setIsLoadingInvoice(true);
623 %     try {
624 %         const res = await fetch(`/api/groups/${groupId}/invoice`);
625 %         if (res.ok) {
626 %             const data = await res.json();
627 %             setInvoiceData(data);
628 %             setShowInvoiceDialog(true);
629 %         } else {
630 %             toast({ title: "Error al cargar factura", variant: "destructive" });
631 %         }
632 %     } catch {
633 %         toast({ title: "Error al cargar factura", variant: "destructive" });
634 %     } finally {
635 %         setIsLoadingInvoice(false);
636 %     }
637 % };
638 %
639 % const printInvoice = () => {
640 %     window.print();
641 % };
642 %
643 % const formatDate = (dateStr: string) => {
644 %     return new Date(dateStr).toLocaleDateString("es-AR", {
645 %         weekday: "long",
646 %         day: "2-digit",
647 %         month: "long",
648 %         year: "numeric",
```

client/src/pages/group-detail.tsx (cont.)

```
649 %     });
650 %   };
651 %
652 %   if (isLoading) {
653 %     return (
654 %       <div className="p-6 space-y-6">
655 %         <Skeleton className="h-10 w-64" />
656 %         <Skeleton className="h-48 w-full" />
657 %         <Skeleton className="h-64 w-full" />
658 %       </div>
659 %     );
660 %   }
661 %
662 %   if (!group) {
663 %     return (
664 %       <div className="p-6">
665 %         <p>Grupo no encontrado</p>
666 %         <Button onClick={() => navigate("/groups")}>Volver a Grupos</Button>
667 %       </div>
668 %     );
669 %   }
670 %
671 %   return (
672 %     <div className="flex flex-col gap-6 p-6">
673 %       <div className="flex items-center gap-4 flex-wrap">
674 %         <Button variant="ghost" size="icon" onClick={() => navigate("/groups")} data-testid="button-back">
675 %           <ArrowLeft className="h-5 w-5" />
676 %         </Button>
677 %         <div className="flex-1 min-w-[200px]">
678 %           <div className="flex items-center gap-3">
679 %             <h1 className="text-2xl font-bold tracking-tight" data-testid="text-group-name">
680 %               {group.name}
681 %             </h1>
682 %             <GroupStatusBadge status={group.status} />
683 %           </div>
684 %           <p className="text-muted-foreground font-mono">{group.groupCode}</p>
685 %         </div>
686 %
687 %         {/* Mass Action Buttons */}
688 %         {group.reservations.length > 0 && (
689 %           <div className="flex gap-2 flex-wrap">
690 %             {/* Check-in All Button - Show when there are confirmed reservations */}
691 %             {group.reservations.some(r => r.status === "confirmed") && (
692 %               <Button
693 %                 variant="default"
694 %                 onClick={() => setShowCheckInConfirm(true)}
695 %                 disabled={checkInAllMutation.isPending}
696 %                 data-testid="button-check-in-all"
697 %               >
698 %                 <LogIn className="mr-2 h-4 w-4" />
699 %                 {checkInAllMutation.isPending ? "Procesando..." : "Check-in Grupal"}
700 %               </Button>
701 %             )}
702 %
703 %             {group.reservations.some(r => r.status === "checked_in") && (
704 %               <Button
705 %                 variant="secondary"
706 %                 onClick={() => setShowCheckOutConfirm(true)}
707 %                 disabled={checkOutAllMutation.isPending}
708 %                 data-testid="button-check-out-all"
709 %               >
710 %                 <LogOut className="mr-2 h-4 w-4" />
711 %                 {checkOutAllMutation.isPending ? "Procesando..." : "Check-out Grupal"}
712 %               </Button>
713 %             )}
714 %
715 %             {/* Invoice Button */}
716 %             <Button
717 %               variant="outline"
718 %               onClick={loadInvoice}
719 %               disabled={isLoadingInvoice}
720 %               data-testid="button-group-invoice"
721 %             >
722 %               <FileText className="mr-2 h-4 w-4" />
723 %               {isLoadingInvoice ? "Cargando..." : "Factura Grupal"}
724 %             </Button>
725 %
726 %             {/* Rooming List Button */}
727 %             <Button
728 %               variant="outline"
729 %               onClick={() => setShowRoomingListDialog(true)}
730 %               data-testid="button-rooming-list"
731 %             >
732 %               <Printer className="mr-2 h-4 w-4" />
733 %               Rooming List
734 %             </Button>
735 %           </div>
736 %         )}
737 %       </div>
738 %
739 %       <div className="grid gap-4 md:grid-cols-3">
740 %         <Card>
741 %           <CardHeader className="pb-2">
```

client/src/pages/group-detail.tsx (cont.)

```
742 %         <CardTitle className="text-sm font-medium text-muted-foreground flex items-center gap-2">
743 %             <Calendar className="h-4 w-4" />
744 %             Fechas
745 %         </CardTitle>
746 %     </CardHeader>
747 %     <CardContent>
748 %         <p className="text-sm">Check-in: <span className="font-medium">{formatDate(group.checkInDate)}</span></p>
749 %         <p className="text-sm">Check-out: <span className="font-medium">{formatDate(group.checkOutDate)}</span></p>
750 %         {group.eventDate && (
751 %             <p className="text-sm text-muted-foreground mt-1">Evento: {formatDate(group.eventDate)}</p>
752 %         )}
753 %     </CardContent>
754 % </Card>
755 %
756 % <Card>
757 %     <CardHeader className="pb-2">
758 %         <CardTitle className="text-sm font-medium text-muted-foreground flex items-center gap-2">
759 %             <User className="h-4 w-4" />
760 %             Contacto
761 %         </CardTitle>
762 %     </CardHeader>
763 %     <CardContent>
764 %         {group.contactName ? (
765 %             <div>
766 %                 <p className="font-medium">{group.contactName}</p>
767 %                 {group.contactPhone && (
768 %                     <p className="text-sm text-muted-foreground flex items-center gap-1">
769 %                         <Phone className="h-3 w-3" />
770 %                         {group.contactPhone}
771 %                     </p>
772 %                 )}
773 %                 {group.contactEmail && (
774 %                     <p className="text-sm text-muted-foreground flex items-center gap-1">
775 %                         <Mail className="h-3 w-3" />
776 %                         {group.contactEmail}
777 %                     </p>
778 %                 )}
779 %             </div>
780 %         ) : (
781 %             <p className="text-sm text-muted-foreground">Sin contacto definido</p>
782 %         )}
783 %     </CardContent>
784 % </Card>
785 %
786 % <Card>
787 %     <CardHeader className="pb-2">
788 %         <CardTitle className="text-sm font-medium text-muted-foreground flex items-center gap-2">
789 %             <DoorOpen className="h-4 w-4" />
790 %             Habitaciones
791 %         </CardTitle>
792 %     </CardHeader>
793 %     <CardContent>
794 %         <div className="flex items-baseline gap-2">
795 %             <span className="text-3xl font-bold">{group.assignedRooms}</span>
796 %             <span className="text-muted-foreground">/ {group.totalRooms} asignadas</span>
797 %         </div>
798 %         <div className="mt-2 h-2 w-full rounded-full bg-muted overflow-hidden">
799 %             <div
800 %                 className="h-full bg-primary transition-all"
801 %                 style={{ width: `${group.totalRooms > 0 ? (group.assignedRooms / group.totalRooms) * 100 : 0}%` }}
802 %             />
803 %         </div>
804 %     </CardContent>
805 % </Card>
806 % </div>
807 %
808 % {group.notes && (
809 %     <Card>
810 %         <CardContent className="pt-4">
811 %             <p className="text-sm text-muted-foreground">{group.notes}</p>
812 %         </CardContent>
813 %     </Card>
814 % )}
815 %
816 % <Tabs defaultValue="blocks" className="w-full">
817 %     <TabsList>
818 %         <TabsTrigger value="blocks" data-testid="tab-blocks">
819 %             <Hotel className="mr-2 h-4 w-4" />
820 %             Bloques ({group.blocks.length})
821 %         </TabsTrigger>
822 %         <TabsTrigger value="reservations" data-testid="tab-reservations">
823 %             <Users2 className="mr-2 h-4 w-4" />
824 %             Reservas ({group.reservations.length})
825 %         </TabsTrigger>
826 %     </TabsList>
827 %
828 %     <TabsContent value="blocks" className="mt-4">
829 %         <Card>
830 %             <CardHeader className="flex flex-row items-center justify-between gap-4 pb-3">
831 %                 <div>
832 %                     <CardTitle>Bloques de Habitaciones</CardTitle>
833 %                     <CardDescription>Tipos y cantidades reservadas para el grupo</CardDescription>
834 %                 </div>
```

```

835 %         <Button onClick={() => setShowAddBlockDialog(true)} data-testid="button-add-block">
836 %             <Plus className="mr-2 h-4 w-4" />
837 %             Agregar Bloque
838 %         </Button>
839 %     </CardHeader>
840 %     <CardContent>
841 %         {group.blocks.length > 0 ? (
842 %             <Table>
843 %                 <TableHeader>
844 %                     <TableRow>
845 %                         <TableHead>Tipo</TableHead>
846 %                         <TableHead>Cantidad</TableHead>
847 %                         <TableHead>Plan Tarifario</TableHead>
848 %                         <TableHead>Tarifa Acordada</TableHead>
849 %                         <TableHead className="w-[150px]">Acciones</TableHead>
850 %                     </TableRow>
851 %                 </TableHeader>
852 %                 <TableBody>
853 %                     {group.blocks.map((block) => (
854 %                         <TableRow key={block.id} data-testid={`row-block-${block.id}`}>
855 %                             <TableCell className="font-medium">
856 %                                 {block.roomType.name} ({block.roomType.code})
857 %                             </TableCell>
858 %                             <TableCell>{block.quantity}</TableCell>
859 %                             <TableCell>
860 %                                 {block.ratePlan?.name || <span className="text-muted-foreground">-</span>}
861 %                             </TableCell>
862 %                             <TableCell>
863 %                                 {block.agreedRate ? `$$${block.agreedRate}` : <span className="text-muted-foreground">-</span>}
864 %                             </TableCell>
865 %                             <TableCell>
866 %                                 <div className="flex items-center gap-2">
867 %                                     <Button
868 %                                         size="sm"
869 %                                         variant="outline"
870 %                                         onClick={() => setAssigningBlock(block)}
871 %                                         data-testid={`button-assign-${block.id}`}
872 %                                     >
873 %                                         <DoorOpen className="mr-1 h-3 w-3" />
874 %                                         Asignar
875 %                                     </Button>
876 %                                     <Button
877 %                                         size="icon"
878 %                                         variant="ghost"
879 %                                         onClick={() => setDeleteBlockId(block.id)}
880 %                                         data-testid={`button-delete-block-${block.id}`}
881 %                                     >
882 %                                         <Trash2 className="h-4 w-4 text-destructive" />
883 %                                     </Button>
884 %                                 </div>
885 %                             </TableCell>
886 %                         </TableRow>
887 %                     )))
888 %                 </TableBody>
889 %             </Table>
890 %         ) : (
891 %             <div className="flex flex-col items-center justify-center py-8 text-center">
892 %                 <Hotel className="h-10 w-10 text-muted-foreground/50" />
893 %                 <p className="mt-2 text-sm text-muted-foreground">
894 %                     No hay bloques definidos. Agregue bloques para reservar habitaciones.
895 %                 </p>
896 %             </div>
897 %         )}
898 %     </CardContent>
899 % </Card>
900 % </TabsContent>
901 %
902 % <TabsContent value="reservations" className="mt-4">
903 %     <Card>
904 %         <CardHeader>
905 %             <CardTitle>Reservas del Grupo</CardTitle>
906 %             <CardDescription>Habitaciones asignadas individualmente</CardDescription>
907 %         </CardHeader>
908 %         <CardContent>
909 %             {group.reservations.length > 0 ? (
910 %                 <Table>
911 %                     <TableHeader>
912 %                         <TableRow>
913 %                             <TableHead>Código</TableHead>
914 %                             <TableHead>Ocupante</TableHead>
915 %                             <TableHead>Habitación</TableHead>
916 %                             <TableHead>Fechas</TableHead>
917 %                             <TableHead>Estado</TableHead>
918 %                         </TableRow>
919 %                     </TableHeader>
920 %                     <TableBody>
921 %                         {group.reservations.map((res) => (
922 %                             <TableRow key={res.id} data-testid={`row-reservation-${res.id}`}>
923 %                                 <TableCell className="font-mono text-sm">{res.reservationCode}</TableCell>
924 %                                 <TableCell>{res.guest.firstName} {res.guest.lastName}</TableCell>
925 %                                 <TableCell>{res.room.roomNumber}</TableCell>
926 %                                 <TableCell className="text-sm">
927 %                                     {new Date(res.checkInDate).toLocaleDateString("es-AR")} - {new Date(res.checkOutDate).toLoc...

```

client/src/pages/group-detail.tsx (cont.)

```
928 %         </TableCell>
929 %         <TableCell>
930 %             <Badge variant={res.status === "confirmed" ? "default" : "secondary"}>
931 %                 {res.status}
932 %             </Badge>
933 %         </TableCell>
934 %     </TableRow>
935 %   </TableBody>
936 % </Table>
937 %
938 %   ) : (
939 %     <div className="flex flex-col items-center justify-center py-8 text-center">
940 %       <Users2 className="h-10 w-10 text-muted-foreground/50" />
941 %       <p className="mt-2 text-sm text-muted-foreground">
942 %         No hay reservas asignadas. Use los bloques para asignar habitaciones.
943 %       </p>
944 %     </div>
945 %   </CardContent>
946 % </Card>
947 % </TabsContent>
948 % </Tabs>
949 %
950 %
951 % {group && (
952 %   <AddBlockDialog
953 %     groupId={groupId}
954 %     group={group}
955 %     open={showAddBlockDialog}
956 %     onOpenChange={setShowAddBlockDialog}
957 %     onSuccess={() => {}}
958 %   />
959 % )}
960 %
961 % {assigningBlock && (
962 %   <AssignRoomDialog
963 %     group={group}
964 %     block={assigningBlock}
965 %     open={!assigningBlock}
966 %     onOpenChange={(open) => !open && setAssigningBlock(null)}
967 %     onSuccess={() => {}}
968 %   />
969 % )}
970 %
971 % <Dialog open={!deleteBlockId} onOpenChange={() => setDeleteBlockId(null)}>
972 %   <DialogContent>
973 %     <DialogHeader>
974 %       <DialogTitle>Confirmar Eliminación</DialogTitle>
975 %       <DialogDescription>
976 %         ¿Está seguro que desea eliminar este bloque de habitaciones?
977 %       </DialogDescription>
978 %     </DialogHeader>
979 %     <DialogFooter>
980 %       <Button variant="outline" onClick={() => setDeleteBlockId(null)}>
981 %         Cancelar
982 %       </Button>
983 %       <Button
984 %         variant="destructive"
985 %         onClick={() => deleteBlockId && deleteBlockMutation.mutate(deleteBlockId)}
986 %         disabled={deleteBlockMutation.isPending}
987 %       >
988 %         {deleteBlockMutation.isPending ? "Eliminando..." : "Eliminar"}
989 %       </Button>
990 %     </DialogFooter>
991 %   </DialogContent>
992 % </Dialog>
993 %
994 % /* Group Invoice Dialog */
995 % <Dialog open={showInvoiceDialog} onOpenChange={setShowInvoiceDialog}>
996 %   <DialogContent className="max-w-4xl max-h-[90vh] overflow-y-auto print:max-w-none print:max-h-none print:over...>
997 %     <DialogHeader className="print:mb-4">
998 %       <DialogTitle className="flex items-center gap-2">
999 %         <FileText className="h-5 w-5" />
1000 %         Factura Grupal - {invoiceData?.group?.name}
1001 %       </DialogTitle>
1002 %       <DialogDescription>
1003 %         Código: {invoiceData?.group?.code} | {invoiceData?.group?.checkInDate} - {invoiceData?.group?.checkOutDate}
1004 %       </DialogDescription>
1005 %     </DialogHeader>
1006 %
1007 %     {invoiceData && (
1008 %       <div className="space-y-6 print:text-sm" id="invoice-content">
1009 %         /* Group Info */
1010 %         <div className="p-4 bg-muted rounded-lg print:bg-transparent print:border print:p-2">
1011 %           <div className="grid grid-cols-2 gap-4">
1012 %             <div>
1013 %               <p className="text-sm text-muted-foreground">Contacto</p>
1014 %               <p className="font-medium">{invoiceData.group.contactName || "-"}</p>
1015 %               <p className="text-sm">{invoiceData.group.contactPhone}</p>
1016 %               <p className="text-sm">{invoiceData.group.contactEmail}</p>
1017 %             </div>
1018 %             <div className="text-right">
1019 %               <p className="text-sm text-muted-foreground">Período</p>
1020 %               <p className="font-medium">{invoiceData.group.checkInDate} - {invoiceData.group.checkOutDate}</p>

```

```

1021 %         </div>
1022 %     </div>
1023 % </div>
1024 %
1025 %     { /* Reservations Detail */ }
1026 %     <div className="space-y-4">
1027 %         <h3 className="font-semibold">Detalle por Habitación</h3>
1028 %         {invoiceData.reservations.map((res: any, idx: number) => (
1029 %             <Card key={idx} className="print:border print:shadow-none">
1030 %                 <CardHeader className="pb-2">
1031 %                     <div className="flex justify-between items-start">
1032 %                         <div>
1033 %                             <CardTitle className="text-base">Hab. {res.room} - {res.guest}</CardTitle>
1034 %                             <CardDescription>{res.reservationCode} | {res.nights} noches</CardDescription>
1035 %                         </div>
1036 %                         <Badge variant={res.balance > 0 ? "destructive" : "default"}>
1037 %                             {res.balance > 0 ? `Pendiente: ${res.balance.toFixed(2)}` : "Pagado"}
1038 %                         </Badge>
1039 %                     </div>
1040 %                 </CardHeader>
1041 %                 <CardContent className="space-y-2 text-sm">
1042 %                     <div className="flex justify-between">
1043 %                         <span>Alojamiento ({res.nights} x ${res.ratePerNight.toFixed(2)})</span>
1044 %                         <span className="font-medium">${res.accommodationTotal.toFixed(2)}</span>
1045 %                     </div>
1046 %
1047 %                     {res.charges.length > 0 && (
1048 %                         <div className="pl-4 border-l-2 border-muted space-y-1">
1049 %                             <p className="text-muted-foreground">Consumos:</p>
1050 %                             {res.charges.map((c: any, i: number) => (
1051 %                                 <div key={i} className="flex justify-between">
1052 %                                     <span>{c.description}</span>
1053 %                                     <span>${c.amount.toFixed(2)}</span>
1054 %                                 </div>
1055 %                             ))}
1056 %                         <div className="flex justify-between font-medium">
1057 %                             <span>Subtotal consumos</span>
1058 %                             <span>${res.chargesTotal.toFixed(2)}</span>
1059 %                         </div>
1060 %                     </div>
1061 %                 ))}
1062 %
1063 %                 {res.payments.length > 0 && (
1064 %                     <div className="pl-4 border-l-2 border-green-500 space-y-1">
1065 %                         <p className="text-muted-foreground">Pagos:</p>
1066 %                         {res.payments.map((p: any, i: number) => (
1067 %                             <div key={i} className="flex justify-between text-green-600">
1068 %                                 <span>{p.method} {p.reference && ` (${p.reference})`}</span>
1069 %                                 <span>- ${p.amount.toFixed(2)}</span>
1070 %                             </div>
1071 %                         ))}
1072 %                     <div className="flex justify-between font-medium text-green-600">
1073 %                         <span>Total pagos</span>
1074 %                         <span>- ${res.paymentsTotal.toFixed(2)}</span>
1075 %                     </div>
1076 %                 </div>
1077 %             ))}
1078 %         </CardContent>
1079 %     </Card>
1080 %     )))}
1081 % </div>
1082 %
1083 %     { /* Totals */ }
1084 %     <Card className="bg-muted print:bg-transparent print:border-2">
1085 %         <CardContent className="pt-4">
1086 %             <div className="space-y-2 text-sm">
1087 %                 <div className="flex justify-between">
1088 %                     <span>Total Alojamiento</span>
1089 %                     <span className="font-medium">${invoiceData.totals.accommodation.toFixed(2)}</span>
1090 %                 </div>
1091 %                 <div className="flex justify-between">
1092 %                     <span>Total Consumos</span>
1093 %                     <span className="font-medium">${invoiceData.totals.charges.toFixed(2)}</span>
1094 %                 </div>
1095 %                 <div className="flex justify-between text-green-600">
1096 %                     <span>Total Pagos</span>
1097 %                     <span className="font-medium">- ${invoiceData.totals.payments.toFixed(2)}</span>
1098 %                 </div>
1099 %                 <div className="border-t pt-2 mt-2">
1100 %                     <div className="flex justify-between text-lg font-bold">
1101 %                         <span>Saldo Total</span>
1102 %                         <span className={invoiceData.totals.balance > 0 ? "text-destructive" : "text-green-600"}>
1103 %                             ${invoiceData.totals.balance.toFixed(2)}
1104 %                         </span>
1105 %                     </div>
1106 %                 </div>
1107 %             </div>
1108 %         </CardContent>
1109 %     </Card>
1110 % </div>
1111 % ))}
1112 %
1113 % <DialogFooter className="print:hidden gap-2">

```

client/src/pages/group-detail.tsx (cont.)

```
1114 %         <Button variant="outline" onClick={() => setShowInvoiceDialog(false)}>
1115 %             Cerrar
1116 %         </Button>
1117 %         {invoiceData?.totals?.balance > 0.01 && (
1118 %             <Button
1119 %                 variant="default"
1120 %                 onClick={() => {
1121 %                     setGroupPaymentAmount(invoiceData.totals.balance.toFixed(2));
1122 %                     setShowGroupPaymentDialog(true);
1123 %                 }}
1124 %                 data-testid="button-group-payment"
1125 %             >
1126 %                 <CreditCard className="mr-2 h-4 w-4" />
1127 %                 Registrar Pago Grupal
1128 %             </Button>
1129 %         )}
1130 %         <Button onClick={printInvoice} data-testid="button-print-invoice">
1131 %             <Printer className="mr-2 h-4 w-4" />
1132 %             Imprimir
1133 %         </Button>
1134 %     </DialogFooter>
1135 % </DialogContent>
1136 % </Dialog>
1137 %
1138 %     {/* Rooming List Dialog */}
1139 %     <Dialog open={showRoomingListDialog} onOpenChange={setShowRoomingListDialog}>
1140 %         <DialogContent className="max-w-3xl max-h-[90vh] overflow-y-auto print:max-w-none print:max-h-none print:over...
1141 %             <DialogHeader className="print:mb-4">
1142 %                 <DialogTitle className="flex items-center gap-2">
1143 %                     <Users2 className="h-5 w-5" />
1144 %                     Rooming List - {group.name}
1145 %                 </DialogTitle>
1146 %                 <DialogDescription>
1147 %                     {group.groupCode} | {new Date(group.checkInDate).toLocaleDateString("es-AR")} - {new Date(group.checkOu...
1148 %                 </DialogDescription>
1149 %             </DialogHeader>
1150 %
1151 %             <div className="space-y-4 print:text-sm" id="rooming-list-content">
1152 %                 {/* Group Header */}
1153 %                 <div className="grid grid-cols-2 gap-4 p-4 bg-muted rounded-lg print:bg-transparent print:border print:p-2">
1154 %                     <div>
1155 %                         <p className="text-sm text-muted-foreground">Grupo</p>
1156 %                         <p className="font-bold text-lg">{group.name}</p>
1157 %                         <p className="font-mono text-sm">{group.groupCode}</p>
1158 %                     </div>
1159 %                     <div className="text-right">
1160 %                         <p className="text-sm text-muted-foreground">Contacto</p>
1161 %                         <p className="font-medium">{group.contactName || "-"}</p>
1162 %                         <p className="text-sm">{group.contactPhone}</p>
1163 %                     </div>
1164 %                 </div>
1165 %
1166 %                 {/* Summary */}
1167 %                 <div className="flex gap-4 text-sm">
1168 %                     <div className="flex items-center gap-2">
1169 %                         <Calendar className="h-4 w-4 text-muted-foreground" />
1170 %                         <span>Check-in: <strong>{new Date(group.checkInDate).toLocaleDateString("es-AR")}</strong></span>
1171 %                     </div>
1172 %                     <div className="flex items-center gap-2">
1173 %                         <Calendar className="h-4 w-4 text-muted-foreground" />
1174 %                         <span>Check-out: <strong>{new Date(group.checkOutDate).toLocaleDateString("es-AR")}</strong></span>
1175 %                     </div>
1176 %                     <div className="flex items-center gap-2">
1177 %                         <DoorOpen className="h-4 w-4 text-muted-foreground" />
1178 %                         <span><strong>{group.reservations.length}</strong> habitaciones</span>
1179 %                     </div>
1180 %                 </div>
1181 %
1182 %                 {/* Rooming List Table */}
1183 %                 {group.reservations.length > 0 ? (
1184 %                     <Table className="print:text-xs">
1185 %                         <TableHeader>
1186 %                             <TableRow>
1187 %                                 <TableHead className="w-12">#</TableHead>
1188 %                                 <TableHead>Habitación</TableHead>
1189 %                                 <TableHead>Tipo</TableHead>
1190 %                                 <TableHead>Huésped</TableHead>
1191 %                                 <TableHead>Check-in</TableHead>
1192 %                                 <TableHead>Check-out</TableHead>
1193 %                                 <TableHead>Estado</TableHead>
1194 %                             </TableRow>
1195 %                         </TableHeader>
1196 %                         <TableBody>
1197 %                             {group.reservations
1198 %                                 .sort((a, b) => (a.room?.roomNumber || "").localeCompare(b.room?.roomNumber || ""))
1199 %                                 .map((res, idx) => (
1200 %                                     <TableRow key={res.id} data-testid={`row-rooming-${res.id}`}>
1201 %                                         <TableCell className="font-medium">{idx + 1}</TableCell>
1202 %                                         <TableCell className="font-bold">{res.room?.roomNumber}</TableCell>
1203 %                                         <TableCell>{res.room?.roomType?.name || "-"}</TableCell>
1204 %                                         <TableCell>
1205 %                                             <div>
1206 %                                                 <p className="font-medium">{res.guest?.firstName} {res.guest?.lastName}</p>
```


client/src/pages/group-detail.tsx (cont.)

```
1207 %         {res.guest?.documentNumber && (
1208 %           <p className="text-xs text-muted-foreground">
1209 %             {res.guest?.documentType}: {res.guest?.documentNumber}
1210 %           </p>
1211 %         )}
1212 %       </div>
1213 %     </TableCell>
1214 %     <TableCell>{new Date(res.checkInDate).toLocaleDateString("es-AR")}</TableCell>
1215 %     <TableCell>{new Date(res.checkOutDate).toLocaleDateString("es-AR")}</TableCell>
1216 %     <TableCell>
1217 %       <Badge>
1218 %         variant={
1219 %           res.status === "checked_in" ? "default" :
1220 %           res.status === "confirmed" ? "secondary" :
1221 %           res.status === "checked_out" ? "outline" : "destructive"
1222 %         }
1223 %       >
1224 %         {res.status === "checked_in" ? "En Casa" :
1225 %           res.status === "confirmed" ? "Confirmado" :
1226 %           res.status === "checked_out" ? "Salió" : res.status}
1227 %       </Badge>
1228 %     </TableCell>
1229 %   </TableRow>
1230 % )}
1231 % </TableBody>
1232 % </Table>
1233 % ) : (
1234 %   <div className="text-center py-8 text-muted-foreground">
1235 %     No hay habitaciones asignadas en este grupo
1236 %   </div>
1237 % )}
1238 %
1239 % {/* Footer for print */}
1240 % <div className="text-xs text-muted-foreground text-center pt-4 border-t print:mt-8">
1241 %   Generado el {new Date().toLocaleString("es-AR")} | Maran Suites & Towers
1242 % </div>
1243 % </div>
1244 %
1245 % <DialogFooter className="print:hidden gap-2">
1246 %   <Button variant="outline" onClick={() => setShowRoomingListDialog(false)}>
1247 %     Cerrar
1248 %   </Button>
1249 %   <Button onClick={() => window.print()} data-testid="button-print-rooming-list">
1250 %     <Printer className="mr-2 h-4 w-4" />
1251 %     Imprimir
1252 %   </Button>
1253 % </DialogFooter>
1254 % </DialogContent>
1255 % </Dialog>
1256 %
1257 % <AlertDialog open={showCheckInConfirm} onOpenChange={setShowCheckInConfirm}>
1258 %   <AlertDialogContent>
1259 %     <AlertDialogHeader>
1260 %       <AlertDialogTitle>Confirmar Check-in Grupal</AlertDialogTitle>
1261 %       <AlertDialogDescription>
1262 %         Se realizará el check-in de {group.reservations.filter(r => r.status === "confirmed").length} habitació...
1263 %         Las habitaciones pasarán a estado "ocupado".
1264 %       </AlertDialogDescription>
1265 %     </AlertDialogHeader>
1266 %     <AlertDialogFooter>
1267 %       <AlertDialogCancel>Cancelar</AlertDialogCancel>
1268 %       <AlertDialogAction>
1269 %         onClick={() => checkInAllMutation.mutate()}
1270 %         data-testid="button-confirm-check-in-all"
1271 %       >
1272 %         <LogIn className="mr-2 h-4 w-4" />
1273 %         Si, realizar check-in
1274 %       </AlertDialogAction>
1275 %     </AlertDialogFooter>
1276 %   </AlertDialogContent>
1277 % </AlertDialog>
1278 %
1279 % <AlertDialog open={showCheckOutConfirm} onOpenChange={setShowCheckOutConfirm}>
1280 %   <AlertDialogContent>
1281 %     <AlertDialogHeader>
1282 %       <AlertDialogTitle>Confirmar Check-out Grupal</AlertDialogTitle>
1283 %       <AlertDialogDescription>
1284 %         Se realizará el check-out de {group.reservations.filter(r => r.status === "checked_in").length} habitac...
1285 %         Las habitaciones con saldo pendiente no serán procesadas. Las habitaciones irán a limpieza.
1286 %       </AlertDialogDescription>
1287 %     </AlertDialogHeader>
1288 %     <AlertDialogFooter>
1289 %       <AlertDialogCancel>Cancelar</AlertDialogCancel>
1290 %       <AlertDialogAction>
1291 %         onClick={() => checkOutAllMutation.mutate()}
1292 %         data-testid="button-confirm-check-out-all"
1293 %       >
1294 %         <LogOut className="mr-2 h-4 w-4" />
1295 %         Si, realizar check-out
1296 %       </AlertDialogAction>
1297 %     </AlertDialogFooter>
1298 %   </AlertDialogContent>
1299 % </AlertDialog>
```

```

1300 %
1301 % <Dialog open={showGroupPaymentDialog} onOpenChange={setShowGroupPaymentDialog}>
1302 %   <DialogContent className="max-w-lg">
1303 %     <DialogHeader>
1304 %       <DialogTitle className="flex items-center gap-2">
1305 %         <DollarSign className="h-5 w-5" />
1306 %         Pago Grupal
1307 %       </DialogTitle>
1308 %       <DialogDescription>
1309 %         Registre un pago que se distribuirá entre las reservas activas del grupo
1310 %       </DialogDescription>
1311 %     </DialogHeader>
1312 %     <div className="space-y-4">
1313 %       <div>
1314 %         <Label>Monto Total *</Label>
1315 %         <Input
1316 %           type="number"
1317 %           step="0.01"
1318 %           min={0}
1319 %           value={groupPaymentAmount}
1320 %           onChange={(e) => setGroupPaymentAmount(e.target.value)}
1321 %           placeholder="0.00"
1322 %           data-testid="input-group-payment-amount"
1323 %         />
1324 %       </div>
1325 %
1326 %       <div>
1327 %         <Label>Método de Pago *</Label>
1328 %         <Select value={groupPaymentMethod} onValueChange={setGroupPaymentMethod}>
1329 %           <SelectTrigger data-testid="select-group-payment-method">
1330 %             <SelectValue placeholder="Seleccionar método" />
1331 %           </SelectTrigger>
1332 %           <SelectContent>
1333 %             <SelectItem value="efectivo">Efectivo</SelectItem>
1334 %             <SelectItem value="tarjeta_debito">Tarjeta Débito</SelectItem>
1335 %             <SelectItem value="tarjeta_credito">Tarjeta Crédito</SelectItem>
1336 %             <SelectItem value="transferencia">Transferencia</SelectItem>
1337 %             <SelectItem value="mercadopago">MercadoPago</SelectItem>
1338 %             <SelectItem value="cuenta_corriente">Cuenta Corriente</SelectItem>
1339 %           </SelectContent>
1340 %         </Select>
1341 %       </div>
1342 %
1343 %       <div>
1344 %         <Label>Tipo de Comprobante</Label>
1345 %         <Select value={groupPaymentReceiptType} onValueChange={setGroupPaymentReceiptType}>
1346 %           <SelectTrigger data-testid="select-group-payment-receipt">
1347 %             <SelectValue placeholder="Seleccionar comprobante" />
1348 %           </SelectTrigger>
1349 %           <SelectContent>
1350 %             <SelectItem value="ticket">Ticket</SelectItem>
1351 %             <SelectItem value="factura_a">Factura A</SelectItem>
1352 %             <SelectItem value="factura_b">Factura B</SelectItem>
1353 %             <SelectItem value="factura_c">Factura C</SelectItem>
1354 %             <SelectItem value="nota_credito">Nota de Crédito</SelectItem>
1355 %           </SelectContent>
1356 %         </Select>
1357 %       </div>
1358 %
1359 %       <div>
1360 %         <Label>Distribución</Label>
1361 %         <Select value={groupPaymentDistribution} onValueChange={setGroupPaymentDistribution}>
1362 %           <SelectTrigger data-testid="select-group-payment-distribution">
1363 %             <SelectValue />
1364 %           </SelectTrigger>
1365 %           <SelectContent>
1366 %             <SelectItem value="equal">Partes iguales</SelectItem>
1367 %             <SelectItem value="proportional">Proporcional al costo</SelectItem>
1368 %           </SelectContent>
1369 %         </Select>
1370 %         <p className="text-xs text-muted-foreground mt-1">
1371 %           {groupPaymentDistribution === "equal"
1372 %             ? "El monto se divide en partes iguales entre las reservas activas"
1373 %             : "El monto se distribuye proporcionalmente al costo total de cada reserva"}
1374 %         </p>
1375 %       </div>
1376 %
1377 %       <div>
1378 %         <Label>Referencia</Label>
1379 %         <Input
1380 %           value={groupPaymentReference}
1381 %           onChange={(e) => setGroupPaymentReference(e.target.value)}
1382 %           placeholder="Número de comprobante, nota..."
1383 %           data-testid="input-group-payment-reference"
1384 %         />
1385 %       </div>
1386 %     </div>
1387 %
1388 %   <DialogFooter>
1389 %     <Button variant="outline" onClick={() => setShowGroupPaymentDialog(false)}>
1390 %       Cancelar
1391 %     </Button>
1392 %   </DialogFooter>

```

client/src/pages/group-detail.tsx (cont.)

```
1393 %         onClick={() => groupPaymentMutation.mutate()}
1394 %         disabled={!groupPaymentAmount || !groupPaymentMethod || groupPaymentMutation.isPending}
1395 %         data-testid="button-confirm-group-payment"
1396 %       >
1397 %         <CreditCard className="mr-2 h-4 w-4" />
1398 %         {groupPaymentMutation.isPending ? "Procesando..." : "Registrar Pago"}
1399 %       </Button>
1400 %     </DialogFooter>
1401 %   </DialogContent>
1402 % </Dialog>
1403 % </div>
1404 % );
1405 % }
1406 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import {
4 %   Package,
5 %   Plus,
6 %   Edit,
7 %   Trash2,
8 %   Calendar,
9 %   Moon,
10 %   DollarSign,
11 %   Gift,
12 %   Tag,
13 % } from "lucide-react";
14 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
15 % import { Button } from "@components/ui/button";
16 % import { Input } from "@components/ui/input";
17 % import { Badge } from "@components/ui/badge";
18 % import { Skeleton } from "@components/ui/skeleton";
19 % import { Textarea } from "@components/ui/textarea";
20 % import {
21 %   Dialog,
22 %   DialogContent,
23 %   DialogDescription,
24 %   DialogFooter,
25 %   DialogHeader,
26 %   DialogTitle,
27 % } from "@components/ui/dialog";
28 % import {
29 %   Select,
30 %   SelectContent,
31 %   SelectItem,
32 %   SelectTrigger,
33 %   SelectValue,
34 % } from "@components/ui/select";
35 % import {
36 %   Table,
37 %   TableBody,
38 %   TableCell,
39 %   TableHead,
40 %   TableHeader,
41 %   TableRow,
42 % } from "@components/ui/table";
43 % import { Label } from "@components/ui/label";
44 % import { useToast } from "@hooks/use-toast";
45 % import { queryClient, apiRequest } from "@lib/queryClient";
46 % import type { PackageWithDetails, RoomType, PackageStatus, PackageItemType } from "@shared/schema";
47 %
48 % function PackageStatusBadge({ status }: { status: PackageStatus }) {
49 %   const config: Record<PackageStatus, { label: string; variant: "default" | "secondary" | "destructive" }> = {
50 %     active: { label: "Activo", variant: "default" },
51 %     inactive: { label: "Inactivo", variant: "secondary" },
52 %     expired: { label: "Expirado", variant: "destructive" },
53 %   };
54 %   const c = config[status];
55 %   return <Badge variant={c.variant}>{c.label}</Badge>;
56 % }
57 %
58 % const itemTypeLabels: Record<PackageItemType, string> = {
59 %   accommodation: "Alojamiento",
60 %   breakfast: "Desayuno",
61 %   dinner: "Cena",
62 %   spa: "SPA",
63 %   restaurant: "Restaurante",
64 %   event: "Evento",
65 %   transfer: "Traslado",
66 %   other: "Otro",
67 % };
68 %
69 % function PackageFormDialog({
70 %   pkg,
71 %   open,
72 %   onOpenChange,
73 %   onSuccess,
74 % }: {
75 %   pkg?: PackageWithDetails;
76 %   open: boolean;
77 %   onOpenChange: (open: boolean) => void;
78 %   onSuccess: () => void;
79 % }) {
80 %   const { toast } = useToast();
81 %   const isEditing = !!pkg;
82 %
83 %   const [name, setName] = useState(pkg?.name || "");
84 %   const [description, setDescription] = useState(pkg?.description || "");
85 %   const [roomId, setRoomId] = useState(pkg?.roomId || "");
86 %   const [nights, setNights] = useState(pkg?.nights || 1);
87 %   const [basePrice, setBasePrice] = useState(pkg?.basePrice || "");
88 %   const [discountPercent, setDiscountPercent] = useState(pkg?.discountPercent || "");
89 %   const [validFrom, setValidFrom] = useState(pkg?.validFrom || "");
90 %   const [validUntil, setValidUntil] = useState(pkg?.validUntil || "");
```

client/src/pages/packages.tsx (cont.)

```
91 %   const [status, setStatus] = useState<PackageStatus>(pkg?.status || "active");
92 %   const [terms, setTerms] = useState(pkg?.terms || "");
93 %   const [items, setItems] = useState<{ itemType: PackageItemType; description: string; quantity: number }[]>(
94 %     pkg?.items?.map(i => ({ itemType: i.itemType, description: i.description, quantity: i.quantity })) || []
95 %   );
96 %
97 %   const { data: roomTypes } = useQuery<RoomType[]>({ queryKey: ["/api/room-types"] });
98 %
99 %   const createMutation = useMutation({
100 %     mutationFn: async () => {
101 %       const res = await apiRequest("POST", "/api/packages", {
102 %         name,
103 %         description,
104 %         roomId: roomId && roomId !== "any" ? roomId : null,
105 %         nights,
106 %         basePrice: parseFloat(basePrice).toFixed(2),
107 %         discountPercent: discountPercent ? parseFloat(discountPercent).toFixed(2) : null,
108 %         validFrom: validFrom || null,
109 %         validUntil: validUntil || null,
110 %         status,
111 %         terms,
112 %       });
113 %       const newPkg = await res.json();
114 %       for (const item of items) {
115 %         await apiRequest("POST", `/api/packages/${newPkg.id}/items`, item);
116 %       }
117 %       return newPkg;
118 %     },
119 %     onSuccess: () => {
120 %       queryClient.invalidateQueries({ queryKey: ["/api/packages"] });
121 %       toast({ title: "Paquete creado exitosamente" });
122 %       onSuccess();
123 %       setOpenChange(false);
124 %     },
125 %     onError: () => {
126 %       toast({ title: "Error al crear paquete", variant: "destructive" });
127 %     },
128 %   });
129 %
130 %   const updateMutation = useMutation({
131 %     mutationFn: async () => {
132 %       await apiRequest("PATCH", `/api/packages/${pkg!.id}`, {
133 %         name,
134 %         description,
135 %         roomId: roomId && roomId !== "any" ? roomId : null,
136 %         nights,
137 %         basePrice: parseFloat(basePrice).toFixed(2),
138 %         discountPercent: discountPercent ? parseFloat(discountPercent).toFixed(2) : null,
139 %         validFrom: validFrom || null,
140 %         validUntil: validUntil || null,
141 %         status,
142 %         terms,
143 %       });
144 %       for (const existingItem of pkg!.items) {
145 %         await apiRequest("DELETE", `/api/package-items/${existingItem.id}`);
146 %       }
147 %       for (const item of items) {
148 %         await apiRequest("POST", `/api/packages/${pkg!.id}/items`, item);
149 %       }
150 %     },
151 %     onSuccess: () => {
152 %       queryClient.invalidateQueries({ queryKey: ["/api/packages"] });
153 %       toast({ title: "Paquete actualizado exitosamente" });
154 %       onSuccess();
155 %       setOpenChange(false);
156 %     },
157 %     onError: () => {
158 %       toast({ title: "Error al actualizar paquete", variant: "destructive" });
159 %     },
160 %   });
161 %
162 %   const handleSubmit = () => {
163 %     if (!name || !basePrice) {
164 %       toast({ title: "Nombre y precio son requeridos", variant: "destructive" });
165 %       return;
166 %     }
167 %     if (isEditing) {
168 %       updateMutation.mutate();
169 %     } else {
170 %       createMutation.mutate();
171 %     }
172 %   };
173 %
174 %   const addItem = () => {
175 %     setItems([...items, { itemType: "breakfast", description: "", quantity: 1 }]);
176 %   };
177 %
178 %   const updateItem = (idx: number, field: string, value: any) => {
179 %     const newItems = [...items];
180 %     (newItems[idx] as any)[field] = value;
181 %     setItems(newItems);
182 %   };
183 %
```

client/src/pages/packages.tsx (cont.)

```
184 %   const removeItem = (idx: number) => {
185 %       setItems(items.filter((_, i) => i !== idx));
186 %   };
187 %
188 %   return (
189 %       <Dialog open={open} onOpenChange={onOpenChange}>
190 %         <DialogContent className="max-w-2xl max-h-[90vh] overflow-y-auto">
191 %           <DialogHeader>
192 %             <DialogTitle><{isEditing ? "Editar Paquete" : "Nuevo Paquete"}</DialogTitle>
193 %             <DialogDescription>
194 %               {isEditing ? "Modifique los datos del paquete" : "Configure un nuevo paquete turístico"}
195 %             </DialogDescription>
196 %           </DialogHeader>
197 %
198 %           <div className="space-y-4">
199 %             <div className="grid grid-cols-2 gap-4">
200 %               <div className="col-span-2">
201 %                 <Label>Nombre del Paquete *</Label>
202 %                 <Input
203 %                   value={name}
204 %                   onChange={(e) => setName(e.target.value)}
205 %                   placeholder="Ej: Escapada Relax 2 Noches"
206 %                   data-testid="input-package-name"
207 %                 />
208 %               </div>
209 %
210 %               <div className="col-span-2">
211 %                 <Label>Descripción</Label>
212 %                 <Textarea
213 %                   value={description}
214 %                   onChange={(e) => setDescription(e.target.value)}
215 %                   placeholder="Descripción del paquete..."
216 %                   rows={2}
217 %                   data-testid="input-package-description"
218 %                 />
219 %               </div>
220 %
221 %               <div>
222 %                 <Label>Tipo de Habitación</Label>
223 %                 <Select value={roomId} onValueChange={setRoomTypeId}>
224 %                   <SelectTrigger data-testid="select-package-room-type">
225 %                     <SelectValue placeholder="Cualquier tipo" />
226 %                   </SelectTrigger>
227 %                   <SelectContent>
228 %                     <SelectItem value="any">Cualquier tipo</SelectItem>
229 %                     {roomTypes?.map((rt) => (
230 %                       <SelectItem key={rt.id} value={rt.id}>
231 %                         {rt.name}
232 %                       </SelectItem>
233 %                     ))}
234 %                   </SelectContent>
235 %                 </Select>
236 %               </div>
237 %
238 %               <div>
239 %                 <Label>Noches</Label>
240 %                 <Input
241 %                   type="number"
242 %                   min={1}
243 %                   value={nights}
244 %                   onChange={(e) => setNights(parseInt(e.target.value) || 1)}
245 %                   data-testid="input-package-nights"
246 %                 />
247 %               </div>
248 %
249 %               <div>
250 %                 <Label>Precio Base *</Label>
251 %                 <Input
252 %                   type="number"
253 %                   step="0.01"
254 %                   value={basePrice}
255 %                   onChange={(e) => setBasePrice(e.target.value)}
256 %                   placeholder="0.00"
257 %                   data-testid="input-package-price"
258 %                 />
259 %               </div>
260 %
261 %               <div>
262 %                 <Label>Descuento %</Label>
263 %                 <Input
264 %                   type="number"
265 %                   step="0.01"
266 %                   value={discountPercent}
267 %                   onChange={(e) => setDiscountPercent(e.target.value)}
268 %                   placeholder="0.00"
269 %                   data-testid="input-package-discount"
270 %                 />
271 %               </div>
272 %
273 %               <div>
274 %                 <Label>Válido Desde</Label>
275 %                 <Input
276 %                   type="date"
```

client/src/pages/packages.tsx (cont.)

```
277 %         value={validFrom}
278 %         onChange={(e) => setValidFrom(e.target.value)}
279 %         data-testid="input-package-valid-from"
280 %     />
281 % </div>
282 %
283 % <div>
284 %     <Label>Válido Hasta</Label>
285 %     <Input
286 %         type="date"
287 %         value={validUntil}
288 %         onChange={(e) => setValidUntil(e.target.value)}
289 %         data-testid="input-package-valid-until"
290 %     />
291 % </div>
292 %
293 % <div>
294 %     <Label>Estado</Label>
295 %     <Select value={status} onChange={(v) => setStatus(v as PackageStatus)}>
296 %         <SelectTrigger data-testid="select-package-status">
297 %             <SelectValue />
298 %         </SelectTrigger>
299 %         <SelectContent>
300 %             <SelectItem value="active">Activo</SelectItem>
301 %             <SelectItem value="inactive">Inactivo</SelectItem>
302 %             <SelectItem value="expired">Expirado</SelectItem>
303 %         </SelectContent>
304 %     </Select>
305 % </div>
306 % </div>
307 %
308 % <div className="border-t pt-4">
309 %     <div className="flex items-center justify-between mb-3">
310 %         <Label className="text-base font-medium">Servicios Incluidos</Label>
311 %         <Button type="button" variant="outline" size="sm" onClick={addItem} data-testid="button-add-package-item">
312 %             <Plus className="mr-2 h-4 w-4" />
313 %             Agregar
314 %         </Button>
315 %     </div>
316 %
317 %     {items.length > 0 ? (
318 %         <div className="space-y-2">
319 %             {items.map((item, idx) => (
320 %                 <div key={idx} className="flex gap-2 items-center">
321 %                     <Select value={item.itemType} onChange={(v) => updateItem(idx, "itemType", v)}>
322 %                         <SelectTrigger className="w-[140px]" data-testid={`select-item-type-${idx}`}>
323 %                             <SelectValue />
324 %                         </SelectTrigger>
325 %                         <SelectContent>
326 %                             {Object.entries(itemTypeLabels).map(([value, label]) => (
327 %                                 <SelectItem key={value} value={value}>{label}</SelectItem>
328 %                             ))}
329 %                         </SelectContent>
330 %                     </Select>
331 %                     <Input
332 %                         value={item.description}
333 %                         onChange={(e) => updateItem(idx, "description", e.target.value)}
334 %                         placeholder="Descripción del servicio"
335 %                         className="flex-1"
336 %                         data-testid={`input-item-description-${idx}`}
337 %                     />
338 %                     <Input
339 %                         type="number"
340 %                         value={item.quantity}
341 %                         onChange={(e) => updateItem(idx, "quantity", parseInt(e.target.value) || 1)}
342 %                         className="w-16"
343 %                         min={1}
344 %                         data-testid={`input-item-quantity-${idx}`}
345 %                     />
346 %                     <Button type="button" variant="ghost" size="icon" onClick={() => removeItem(idx)}>
347 %                         <Trash2 className="h-4 w-4 text-destructive" />
348 %                     </Button>
349 %                 </div>
350 %             )})
351 %         </div>
352 %     ) : (
353 %         <p className="text-sm text-muted-foreground text-center py-4">
354 %             No hay servicios incluidos. Agregue servicios al paquete.
355 %         </p>
356 %     )}
357 % </div>
358 %
359 % <div className="border-t pt-4">
360 %     <Label>Términos y Condiciones</Label>
361 %     <Textarea
362 %         value={terms}
363 %         onChange={(e) => setTerms(e.target.value)}
364 %         placeholder="Términos, restricciones, notas..."
365 %         rows={2}
366 %         data-testid="input-package-terms"
367 %     />
368 % </div>
369 % </div>
```

client/src/pages/packages.tsx (cont.)

```
370 %
371 %     <DialogFooter>
372 %         <Button variant="outline" onClick={() => onOpenChange(false)}>
373 %             Cancelar
374 %         </Button>
375 %         <Button
376 %             onClick={handleSubmit}
377 %             disabled={createMutation.isPending || updateMutation.isPending}
378 %             data-testid="button-save-package"
379 %         >
380 %             {(createMutation.isPending || updateMutation.isPending) ? "Guardando..." : (isEditing ? "Actualizar" : "C...
381 %         </Button>
382 %     </DialogFooter>
383 % </DialogContent>
384 % </Dialog>
385 % );
386 % }
387 %
388 % export default function PackagesPage() {
389 %     const { toast } = useToast();
390 %     const [showFormDialog, setShowFormDialog] = useState(false);
391 %     const [editingPackage, setEditingPackage] = useState<PackageWithDetails | undefined>();
392 %     const [deletePackageId, setDeletePackageId] = useState<string | null>(null);
393 %
394 %     const { data: packages, isLoading } = useQuery<PackageWithDetails[]>({
395 %         queryKey: ["/api/packages"],
396 %     });
397 %
398 %     const deleteMutation = useMutation({
399 %         mutationFn: (id: string) => apiRequest("DELETE", `/api/packages/${id}`),
400 %         onSuccess: () => {
401 %             queryClient.invalidateQueries({ queryKey: ["/api/packages"] });
402 %             toast({ title: "Paquete eliminado" });
403 %             setDeletePackageId(null);
404 %         },
405 %         onError: () => {
406 %             toast({ title: "Error al eliminar paquete", variant: "destructive" });
407 %         },
408 %     });
409 %
410 %     const handleEdit = (pkg: PackageWithDetails) => {
411 %         setEditingPackage(pkg);
412 %         setShowFormDialog(true);
413 %     };
414 %
415 %     const handleNew = () => {
416 %         setEditingPackage(undefined);
417 %         setShowFormDialog(true);
418 %     };
419 %
420 %     if (isLoading) {
421 %         return (
422 %             <div className="p-6 space-y-6">
423 %                 <Skeleton className="h-10 w-64" />
424 %                 <div className="grid gap-4 md:grid-cols-3">
425 %                     {[1, 2, 3].map((i) => (
426 %                         <Skeleton key={i} className="h-48" />
427 %                     ))}
428 %                 </div>
429 %             </div>
430 %         );
431 %     }
432 %
433 %     return (
434 %         <div className="flex flex-col gap-6 p-6">
435 %             <div className="flex items-center justify-between">
436 %                 <div>
437 %                     <h1 className="text-2xl font-bold tracking-tight" data-testid="text-page-title">
438 %                         Paquetes Turísticos
439 %                     </h1>
440 %                     <p className="text-muted-foreground">
441 %                         Gestione paquetes con noches y servicios incluidos
442 %                     </p>
443 %                 </div>
444 %                 <Button onClick={handleNew} data-testid="button-new-package">
445 %                     <Plus className="mr-2 h-4 w-4" />
446 %                     Nuevo Paquete
447 %                 </Button>
448 %             </div>
449 %
450 %             {packages && packages.length > 0 ? (
451 %                 <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-3">
452 %                     {packages.map((pkg) => (
453 %                         <Card key={pkg.id} className="hover-elevate" data-testid={`card-package-${pkg.id}`}>
454 %                             <CardHeader className="pb-2">
455 %                                 <div className="flex items-start justify-between gap-2">
456 %                                     <div className="flex-1 min-w-0">
457 %                                         <CardTitle className="text-lg truncate">{pkg.name}</CardTitle>
458 %                                         <CardDescription className="font-mono text-xs">{pkg.code}</CardDescription>
459 %                                     </div>
460 %                                     <PackageStatusBadge status={pkg.status} />
461 %                                 </div>
462 %                             </CardHeader>
```


client/src/pages/packages.tsx (cont.)

```
463 %         <CardContent className="space-y-3">
464 %           {pkg.description && (
465 %             <p className="text-sm text-muted-foreground line-clamp-2">{pkg.description}</p>
466 %           )}
467 %
468 %           <div className="flex flex-wrap gap-3 text-sm">
469 %             <div className="flex items-center gap-1">
470 %               <Moon className="h-4 w-4 text-muted-foreground" />
471 %               <span>{pkg.nights} noches</span>
472 %             </div>
473 %             <div className="flex items-center gap-1">
474 %               <DollarSign className="h-4 w-4 text-muted-foreground" />
475 %               <span className="font-semibold">${pkg.basePrice}</span>
476 %               {pkg.discountPercent && parseFloat(pkg.discountPercent) > 0 && (
477 %                 <Badge variant="secondary" className="ml-1">-${pkg.discountPercent}%</Badge>
478 %               )}
479 %             </div>
480 %           </div>
481 %
482 %           {pkg.roomType && (
483 %             <div className="flex items-center gap-1 text-sm">
484 %               <Tag className="h-4 w-4 text-muted-foreground" />
485 %               <span>{pkg.roomType.name}</span>
486 %             </div>
487 %           )}
488 %
489 %           {pkg.items && pkg.items.length > 0 && (
490 %             <div className="flex flex-wrap gap-1">
491 %               {pkg.items.slice(0, 3).map((item, idx) => (
492 %                 <Badge key={idx} variant="outline" className="text-xs">
493 %                   {itemTypeLabels[item.itemType]} x{item.quantity}
494 %                 </Badge>
495 %               )}
496 %               {pkg.items.length > 3 && (
497 %                 <Badge variant="outline" className="text-xs">+{pkg.items.length - 3}</Badge>
498 %               )}
499 %             </div>
500 %           )}
501 %
502 %           {(pkg.validFrom || pkg.validUntil) && (
503 %             <div className="flex items-center gap-1 text-xs text-muted-foreground">
504 %               <Calendar className="h-3 w-3" />
505 %               <span>
506 %                 {pkg.validFrom || "..."} - {pkg.validUntil || "..."}
507 %               </span>
508 %             </div>
509 %           )}
510 %
511 %           <div className="flex gap-2 pt-2 border-t">
512 %             <Button variant="outline" size="sm" onClick={() => handleEdit(pkg)} data-testid={`button-edit-packa...
513 %               <Edit className="mr-2 h-4 w-4" />
514 %               Editar
515 %             </Button>
516 %             <Button variant="ghost" size="sm" onClick={() => setDeletePackageId(pkg.id)} data-testid={`button-d...
517 %               <Trash2 className="h-4 w-4 text-destructive" />
518 %             </Button>
519 %           </div>
520 %         </CardContent>
521 %       </Card>
522 %     )}}
523 %   </div>
524 % ) : (
525 %   <Card>
526 %     <CardContent className="flex flex-col items-center justify-center py-12">
527 %       <Gift className="h-12 w-12 text-muted-foreground/50" />
528 %       <h3 className="mt-4 text-lg font-medium">No hay paquetes</h3>
529 %       <p className="text-muted-foreground text-center max-w-md mt-1">
530 %         Cree paquetes turísticos con noches y servicios incluidos para ofrecer promociones a sus huéspedes.
531 %       </p>
532 %       <Button className="mt-4" onClick={handleNew}>
533 %         <Plus className="mr-2 h-4 w-4" />
534 %         Crear Primer Paquete
535 %       </Button>
536 %     </CardContent>
537 %   </Card>
538 % )}
539 %
540 % <PackageFormDialog
541 %   pkg={editingPackage}
542 %   open={showFormDialog}
543 %   onOpenChange={setShowFormDialog}
544 %   onSuccess={() => {}}
545 % />
546 %
547 % <Dialog open={!deletePackageId} onOpenChange={() => setDeletePackageId(null)}>
548 %   <DialogContent>
549 %     <DialogHeader>
550 %       <DialogTitle>Confirmar Eliminación</DialogTitle>
551 %       <DialogDescription>
552 %         ¿Está seguro que desea eliminar este paquete? Esta acción no se puede deshacer.
553 %       </DialogDescription>
554 %     </DialogHeader>
555 %     <DialogFooter>
```

client/src/pages/packages.tsx (cont.)

```
556 %         <Button variant="outline" onClick={() => setDeletePackageId(null)}>
557 %             Cancelar
558 %         </Button>
559 %         <Button
560 %             variant="destructive"
561 %             onClick={() => deletePackageId && deleteMutation.mutate(deletePackageId)}
562 %             disabled={deleteMutation.isPending}
563 %         >
564 %             {deleteMutation.isPending ? "Eliminando..." : "Eliminar"}
565 %         </Button>
566 %     </DialogFooter>
567 % </DialogContent>
568 % </Dialog>
569 % </div>
570 % );
571 % }
572 %
```

```

1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import {
4 %   DollarSign,
5 %   Plus,
6 %   Search,
7 %   MoreHorizontal,
8 %   Pencil,
9 %   Trash2,
10 %   CheckCircle,
11 %   XCircle,
12 % } from "lucide-react";
13 % import { Card, CardContent, CardHeader, CardTitle, CardDescription } from "@components/ui/card";
14 % import { Button } from "@components/ui/button";
15 % import { Input } from "@components/ui/input";
16 % import { Badge } from "@components/ui/badge";
17 % import { Skeleton } from "@components/ui/skeleton";
18 % import {
19 %   Dialog,
20 %   DialogContent,
21 %   DialogDescription,
22 %   DialogFooter,
23 %   DialogHeader,
24 %   DialogTitle,
25 % } from "@components/ui/dialog";
26 % import {
27 %   DropdownMenu,
28 %   DropdownMenuContent,
29 %   DropdownMenuItem,
30 %   DropdownMenuSeparator,
31 %   DropdownMenuTrigger,
32 % } from "@components/ui/dropdown-menu";
33 % import {
34 %   Select,
35 %   SelectContent,
36 %   SelectItem,
37 %   SelectTrigger,
38 %   SelectValue,
39 % } from "@components/ui/select";
40 % import {
41 %   Table,
42 %   TableBody,
43 %   TableCell,
44 %   TableHead,
45 %   TableHeader,
46 %   TableRow,
47 % } from "@components/ui/table";
48 % import { Label } from "@components/ui/label";
49 % import { Textarea } from "@components/ui/textarea";
50 % import { Switch } from "@components/ui/switch";
51 % import { useToast } from "@hooks/use-toast";
52 % import { queryClient, apiRequest } from "@lib/queryClient";
53 % import type { RatePlanWithRoomType, RoomType, InsertRatePlan } from "@shared/schema";
54 %
55 % function RatePlanFormDialog({
56 %   ratePlan,
57 %   roomTypes,
58 %   open,
59 %   onOpenChange,
60 %   onSuccess,
61 % }: {
62 %   ratePlan?: RatePlanWithRoomType;
63 %   roomTypes: RoomType[];
64 %   open: boolean;
65 %   onOpenChange: (open: boolean) => void;
66 %   onSuccess: () => void;
67 % }) {
68 %   const { toast } = useToast();
69 %   const isEditing = !!ratePlan;
70 %
71 %   const [formData, setFormData] = useState<Partial<InsertRatePlan>>>({
72 %     name: ratePlan?.name || "",
73 %     roomId: ratePlan?.roomId || "",
74 %     baseRate: ratePlan?.baseRate || "",
75 %     currency: ratePlan?.currency || "ARS",
76 %     refundable: ratePlan?.refundable || "true",
77 %     cancellationPolicy: ratePlan?.cancellationPolicy || "",
78 %   });
79 %
80 %   const mutation = useMutation({
81 %     mutationFn: async (data: Partial<InsertRatePlan>) => {
82 %       if (isEditing) {
83 %         return apiRequest("PATCH", `/api/rate-plans/${ratePlan.id}`, data);
84 %       }
85 %       return apiRequest("POST", "/api/rate-plans", data);
86 %     },
87 %     onSuccess: () => {
88 %       queryClient.invalidateQueries({ queryKey: ["/api/rate-plans"] });
89 %       toast({
90 %         title: isEditing ? "Plan tarifario actualizado" : "Plan tarifario creado",

```

client/src/pages/rate-plans.tsx (cont.)

```
91 %         description: `El plan "${formData.name}" ha sido ${isEditing ? "actualizado" : "creado"} exitosamente.`,  
92 %     });  
93 %     onSuccess();  
94 %     onOpenChange(false);  
95 % },  
96 %     onError: () => {  
97 %         toast({  
98 %             title: "Error",  
99 %             description: "No se pudo guardar el plan tarifario. Intente nuevamente.",  
100 %             variant: "destructive",  
101 %         });  
102 %     },  
103 % });  
104 %  
105 % const handleSubmit = (e: React.FormEvent) => {  
106 %     e.preventDefault();  
107 %     mutation.mutate(formData);  
108 % };  
109 %  
110 % return (  
111 %     <Dialog open={open} onOpenChange={onOpenChange}>  
112 %         <DialogContent className="sm:max-w-[500px]">  
113 %             <DialogHeader>  
114 %                 <DialogTitle>{isEditing ? "Editar Plan Tarifario" : "Nuevo Plan Tarifario"}</DialogTitle>  
115 %                 <DialogDescription>  
116 %                     {isEditing  
117 %                         ? "Modifica los detalles del plan tarifario."  
118 %                         : "Crea un nuevo plan de precios para un tipo de habitación."}  
119 %                 </DialogDescription>  
120 %             </DialogHeader>  
121 %             <form onSubmit={handleSubmit}>  
122 %                 <div className="grid gap-4 py-4">  
123 %                     <div className="grid gap-2">  
124 %                         <Label htmlFor="name">Nombre del Plan</Label>  
125 %                         <Input  
126 %                             id="name"  
127 %                             value={formData.name}  
128 %                             onChange={(e) => setFormData({ ...formData, name: e.target.value })}  
129 %                             placeholder="Ej: BAR Standard, No Reembolsable"  
130 %                             required  
131 %                             data-testid="input-rate-plan-name"  
132 %                         />  
133 %                     </div>  
134 %                     <div className="grid gap-2">  
135 %                         <Label htmlFor="roomType">Tipo de Habitación</Label>  
136 %                         <Select  
137 %                             value={formData.roomTypeId}  
138 %                             onChange={(value) => setFormData({ ...formData, roomTypeId: value })}  
139 %                         >  
140 %                             <SelectTrigger data-testid="select-room-type">  
141 %                                 <SelectValue placeholder="Seleccionar tipo de habitación" />  
142 %                             </SelectTrigger>  
143 %                             <SelectContent>  
144 %                                 {roomTypes.map((type) => (  
145 %                                     <SelectItem key={type.id} value={type.id}>  
146 %                                         {type.name}  
147 %                                     </SelectItem>  
148 %                                 ))}  
149 %                             </SelectContent>  
150 %                         </Select>  
151 %                     </div>  
152 %                     <div className="grid grid-cols-2 gap-4">  
153 %                         <div className="grid gap-2">  
154 %                             <Label htmlFor="baseRate">Tarifa Base</Label>  
155 %                             <Input  
156 %                                 id="baseRate"  
157 %                                 type="number"  
158 %                                 min={0}  
159 %                                 step="0.01"  
160 %                                 value={formData.baseRate}  
161 %                                 onChange={(e) => setFormData({ ...formData, baseRate: e.target.value })}  
162 %                                 placeholder="0.00"  
163 %                                 required  
164 %                                 data-testid="input-base-rate"  
165 %                             />  
166 %                         </div>  
167 %                         <div className="grid gap-2">  
168 %                             <Label htmlFor="currency">Moneda</Label>  
169 %                             <Select  
170 %                                 value={formData.currency}  
171 %                                 onChange={(value) => setFormData({ ...formData, currency: value })}  
172 %                             >  
173 %                                 <SelectTrigger data-testid="select-currency">  
174 %                                     <SelectValue />  
175 %                                 </SelectTrigger>  
176 %                                 <SelectContent>  
177 %                                     <SelectItem value="ARS">ARS (Peso Argentino)</SelectItem>  
178 %                                     <SelectItem value="USD">USD (Dólar)</SelectItem>  
179 %                                     <SelectItem value="EUR">EUR (Euro)</SelectItem>  
180 %                                 </SelectContent>  
181 %                             </Select>  
182 %                         </div>  
183 %                     </div>  
184 %                 </form>  
185 %             </DialogContent>  
186 %         </Dialog>  
187 %     );
```

client/src/pages/rate-plans.tsx (cont.)

```
184 %         <div className="flex items-center gap-4">
185 %             <Label htmlFor="refundable">Reembolsable</Label>
186 %             <Switch
187 %                 id="refundable"
188 %                 checked={formData.refundable === "true"}
189 %                 onChange={(checked) =>
190 %                     setFormData({ ...formData, refundable: checked ? "true" : "false" })
191 %                 }
192 %                 data-testid="switch-refundable"
193 %             />
194 %         </div>
195 %         <div className="grid gap-2">
196 %             <Label htmlFor="cancellationPolicy">Política de Cancelación</Label>
197 %             <Textarea
198 %                 id="cancellationPolicy"
199 %                 value={formData.cancellationPolicy || ""}
200 %                 onChange={(e) => setFormData({ ...formData, cancellationPolicy: e.target.value })}
201 %                 placeholder="Ej: Cancelación gratuita hasta 24hs antes..."
202 %                 rows={3}
203 %                 data-testid="input-cancellation-policy"
204 %             />
205 %         </div>
206 %     </div>
207 %     <DialogFooter>
208 %         <Button type="button" variant="outline" onClick={() => onOpenChange(false)}>
209 %             Cancelar
210 %         </Button>
211 %         <Button type="submit" disabled={mutation.isPending} data-testid="button-submit-rate-plan">
212 %             {mutation.isPending ? "Guardando..." : isEditing ? "Guardar Cambios" : "Crear Plan"}
213 %         </Button>
214 %     </DialogFooter>
215 % </form>
216 % </DialogContent>
217 % </Dialog>
218 % );
219 % }
220 %
221 % export default function RatePlansPage() {
222 %     const { toast } = useToast();
223 %     const [searchQuery, setSearchQuery] = useState("");
224 %     const [roomTypeFilter, setRoomTypeFilter] = useState<string>("all");
225 %     const [dialogOpen, setDialogOpen] = useState(false);
226 %     const [selectedRatePlan, setSelectedRatePlan] = useState<RatePlanWithRoomType | undefined>();
227 %
228 %     const { data: ratePlans, isLoading } = useQuery<RatePlanWithRoomType[]>({
229 %         queryKey: ["/api/rate-plans"],
230 %     });
231 %
232 %     const { data: roomTypes } = useQuery<RoomType[]>({
233 %         queryKey: ["/api/room-types"],
234 %     });
235 %
236 %     const deleteMutation = useMutation({
237 %         mutationFn: async (id: string) => {
238 %             return apiRequest("DELETE", `/api/rate-plans/${id}`, undefined);
239 %         },
240 %         onSuccess: () => {
241 %             queryClient.invalidateQueries({ queryKey: ["/api/rate-plans"] });
242 %             toast({
243 %                 title: "Plan eliminado",
244 %                 description: "El plan tarifario ha sido eliminado del sistema."
245 %             });
246 %         },
247 %     });
248 %
249 %     const filteredRatePlans = ratePlans?.filter((plan) => {
250 %         const matchesSearch = plan.name.toLowerCase().includes(searchQuery.toLowerCase());
251 %         const matchesRoomType = roomTypeFilter === "all" || plan.roomTypeId === roomTypeFilter;
252 %         return matchesSearch && matchesRoomType;
253 %     });
254 %
255 %     const handleEditRatePlan = (plan: RatePlanWithRoomType) => {
256 %         setSelectedRatePlan(plan);
257 %         setDialogOpen(true);
258 %     };
259 %
260 %     const handleNewRatePlan = () => {
261 %         setSelectedRatePlan(undefined);
262 %         setDialogOpen(true);
263 %     };
264 %
265 %     const formatCurrency = (amount: string, currency: string) => {
266 %         const num = parseFloat(amount);
267 %         return new Intl.NumberFormat("es-AR", {
268 %             style: "currency",
269 %             currency: currency,
270 %         }).format(num);
271 %     };
272 %
273 %     const groupedByRoomType = roomTypes?.map((roomType) => ({
274 %         roomType,
275 %         plans: ratePlans?.filter((p) => p.roomTypeId === roomType.id) || [],
276 %     }));
```

```

277 %
278 %   return (
279 %     <div className="flex flex-col gap-6 p-6">
280 %       <div className="flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-between">
281 %         <div>
282 %           <h1 className="text-3xl font-bold tracking-tight" data-testid="text-rate-plans-title">
283 %             Planes Tarifarios
284 %           </h1>
285 %           <p className="text-muted-foreground">
286 %             Gestiona los planes de precios para cada tipo de habitación
287 %           </p>
288 %         </div>
289 %         <Button onClick={handleNewRatePlan} data-testid="button-new-rate-plan">
290 %           <Plus className="mr-2 h-4 w-4" />
291 %           Nuevo Plan
292 %         </Button>
293 %       </div>
294 %
295 %     <Card>
296 %       <CardContent className="p-4">
297 %         <div className="flex flex-col gap-4 sm:flex-row sm:items-center">
298 %           <div className="relative flex-1">
299 %             <Search className="absolute left-3 top-1/2 h-4 w-4 -translate-y-1/2 text-muted-foreground" />
300 %             <Input
301 %               placeholder="Buscar planes tarifarios..."
302 %               value={searchQuery}
303 %               onChange={(e) => setSearchQuery(e.target.value)}
304 %               className="pl-10"
305 %               data-testid="input-search-rate-plans"
306 %             />
307 %           </div>
308 %           <Select value={roomTypeFilter} onValueChange={setRoomTypeFilter}>
309 %             <SelectTrigger className="w-[200px]" data-testid="select-filter-room-type">
310 %               <SelectValue placeholder="Tipo de habitación" />
311 %             </SelectTrigger>
312 %             <SelectContent>
313 %               <SelectItem value="all">Todos los tipos</SelectItem>
314 %               {roomTypes?.map((type) => (
315 %                 <SelectItem key={type.id} value={type.id}>
316 %                   {type.name}
317 %                 </SelectItem>
318 %               ))}
319 %             </SelectContent>
320 %           </Select>
321 %         </div>
322 %       </CardContent>
323 %     </Card>
324 %
325 %     {isLoading ? (
326 %       <div className="space-y-4">
327 %         {[...Array(3)].map((_, i) => (
328 %           <Skeleton key={i} className="h-48" />
329 %         ))}
330 %       </div>
331 %     ) : roomTypeFilter === "all" && !searchQuery ? (
332 %       <div className="space-y-6">
333 %         {groupedByRoomType?.map(({ roomType, plans }) => (
334 %           <Card key={roomType.id}>
335 %             <CardHeader>
336 %               <div className="flex items-center justify-between gap-4">
337 %                 <div>
338 %                   <CardTitle className="flex items-center gap-2">
339 %                     <DollarSign className="h-5 w-5 text-primary" />
340 %                     {roomType.name}
341 %                   </CardTitle>
342 %                   <CardDescription>
343 %                     Ocupación: {roomType.baseOccupancy}-{roomType.maxOccupancy} personas
344 %                   </CardDescription>
345 %                 </div>
346 %                 <Badge variant="secondary">{plans.length} planes</Badge>
347 %               </div>
348 %             </CardHeader>
349 %             <CardContent>
350 %               {plans.length > 0 ? (
351 %                 <Table>
352 %                   <TableHeader>
353 %                     <TableRow>
354 %                       <TableHead>Nombre</TableHead>
355 %                       <TableHead>Tarifa</TableHead>
356 %                       <TableHead>Reembolsable</TableHead>
357 %                       <TableHead>Política</TableHead>
358 %                       <TableHead className="w-[50px]"></TableHead>
359 %                     </TableRow>
360 %                   </TableHeader>
361 %                   <TableBody>
362 %                     {plans.map((plan) => (
363 %                       <TableRow key={plan.id} data-testid={`rate-plan-row-${plan.id}`}>
364 %                         <TableCell className="font-medium">{plan.name}</TableCell>
365 %                         <TableCell>
366 %                           <span className="font-semibold text-primary">
367 %                             {formatCurrency(plan.baseRate, plan.currency)}
368 %                           </span>
369 %                           <span className="text-muted-foreground text-sm">/noche</span>

```

client/src/pages/rate-plans.tsx (cont.)

```
370 %         </TableCell>
371 %       <TableCell>
372 %         {plan.refundable === "true" ? (
373 %           <Badge className="bg-green-100 text-green-800 dark:bg-green-900/30 dark:text-green-400">
374 %             <CheckCircle className="mr-1 h-3 w-3" />
375 %             Si
376 %           </Badge>
377 %         ) : (
378 %           <Badge className="bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-400">
379 %             <XCircle className="mr-1 h-3 w-3" />
380 %             No
381 %           </Badge>
382 %         )}
383 %     </TableCell>
384 %     <TableCell className="max-w-[200px] truncate text-muted-foreground">
385 %       {plan.cancellationPolicy || "-"}
386 %     </TableCell>
387 %     <TableCell>
388 %       <DropdownMenu>
389 %         <DropdownMenuTrigger asChild>
390 %           <Button variant="ghost" size="icon">
391 %             <MoreHorizontal className="h-4 w-4" />
392 %           </Button>
393 %         </DropdownMenuTrigger>
394 %         <DropdownMenuContent align="end">
395 %           <DropdownMenuItem onClick={() => handleEditRatePlan(plan)}>
396 %             <Pencil className="mr-2 h-4 w-4" />
397 %             Editar
398 %           </DropdownMenuItem>
399 %           <DropdownMenuSeparator />
400 %           <DropdownMenuItem
401 %             className="text-destructive"
402 %             onClick={() => deleteMutation.mutate(plan.id)}
403 %           >
404 %             <Trash2 className="mr-2 h-4 w-4" />
405 %             Eliminar
406 %           </DropdownMenuItem>
407 %         </DropdownMenuContent>
408 %       </DropdownMenu>
409 %     </TableCell>
410 %   </TableRow>
411 % )}
412 % </TableBody>
413 % </Table>
414 % ) : (
415 %   <div className="text-center py-6 text-muted-foreground">
416 %     No hay planes tarifarios para este tipo de habitación
417 %   </div>
418 % )}
419 % </CardContent>
420 % </Card>
421 % )}
422 % </div>
423 % ) : filteredRatePlans && filteredRatePlans.length > 0 ? (
424 %   <Card>
425 %     <Table>
426 %       <TableHeader>
427 %         <TableRow>
428 %           <TableHead>Nombre</TableHead>
429 %           <TableHead>Tipo de Habitación</TableHead>
430 %           <TableHead>Tarifa</TableHead>
431 %           <TableHead>Reembolsable</TableHead>
432 %           <TableHead>Política</TableHead>
433 %           <TableHead className="w-[50px]"></TableHead>
434 %         </TableRow>
435 %       </TableHeader>
436 %       <TableBody>
437 %         {filteredRatePlans.map((plan) => (
438 %           <TableRow key={plan.id} data-testid={`rate-plan-row-${plan.id}`}>
439 %             <TableCell className="font-medium">{plan.name}</TableCell>
440 %             <TableCell>{plan.roomType?.name || "-"}</TableCell>
441 %             <TableCell>
442 %               <span className="font-semibold text-primary">
443 %                 {formatCurrency(plan.baseRate, plan.currency)}
444 %               </span>
445 %               <span className="text-muted-foreground text-sm">/noche</span>
446 %             </TableCell>
447 %             <TableCell>
448 %               {plan.refundable === "true" ? (
449 %                 <Badge className="bg-green-100 text-green-800 dark:bg-green-900/30 dark:text-green-400">
450 %                   <CheckCircle className="mr-1 h-3 w-3" />
451 %                   Si
452 %                 </Badge>
453 %               ) : (
454 %                 <Badge className="bg-red-100 text-red-800 dark:bg-red-900/30 dark:text-red-400">
455 %                   <XCircle className="mr-1 h-3 w-3" />
456 %                   No
457 %                 </Badge>
458 %               )}
459 %             </TableCell>
460 %             <TableCell className="max-w-[200px] truncate text-muted-foreground">
461 %               {plan.cancellationPolicy || "-"}
462 %             </TableCell>
```

client/src/pages/rate-plans.tsx (cont.)

```
463 %         <TableCell>
464 %             <DropdownMenu>
465 %                 <DropdownMenuTrigger asChild>
466 %                     <Button variant="ghost" size="icon">
467 %                         <MoreHorizontal className="h-4 w-4" />
468 %                     </Button>
469 %                 </DropdownMenuTrigger>
470 %                 <DropdownMenuContent align="end">
471 %                     <DropdownMenuItem onClick={() => handleEditRatePlan(plan)}>
472 %                         <Pencil className="mr-2 h-4 w-4" />
473 %                         Editar
474 %                     </DropdownMenuItem>
475 %                     <DropdownMenuSeparator />
476 %                     <DropdownMenuItem
477 %                         className="text-destructive"
478 %                         onClick={() => deleteMutation.mutate(plan.id)}
479 %                     >
480 %                         <Trash2 className="mr-2 h-4 w-4" />
481 %                         Eliminar
482 %                     </DropdownMenuItem>
483 %                 </DropdownMenuContent>
484 %             </DropdownMenu>
485 %         </TableCell>
486 %     </TableRow>
487 % }}
488 % </TableBody>
489 % </Table>
490 % </Card>
491 % ) : (
492 %     <Card>
493 %         <CardContent className="flex flex-col items-center justify-center py-12 text-center">
494 %             <DollarSign className="h-16 w-16 text-muted-foreground/50 mb-4" />
495 %             <h3 className="text-lg font-semibold mb-2">No hay planes tarifarios</h3>
496 %             <p className="text-muted-foreground mb-4">
497 %                 {searchQuery || roomTypeFilter !== "all"
498 %                 ? "No se encontraron planes con los filtros aplicados."
499 %                 : "Comienza creando el primer plan tarifario."}
500 %             </p>
501 %             {!searchQuery && roomTypeFilter === "all" && (
502 %                 <Button onClick={handleNewRatePlan}>
503 %                     <Plus className="mr-2 h-4 w-4" />
504 %                     Crear Primer Plan
505 %                 </Button>
506 %             )}
507 %         </CardContent>
508 %     </Card>
509 % )}
510 %
511 % <RatePlanFormDialog
512 %     ratePlan={selectedRatePlan}
513 %     roomTypes={roomTypes || []}
514 %     open={dialogOpen}
515 %     onOpenChange={setDialogOpen}
516 %     onSuccess={() => setSelectedRatePlan(undefined)}
517 % />
518 % </div>
519 % );
520 % }
521 %
```



```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import { queryClient, apiRequest } from "@lib/queryClient";
4 % import { useToast } from "@hooks/use-toast";
5 % import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@components/ui/card";
6 % import { Button } from "@components/ui/button";
7 % import { Badge } from "@components/ui/badge";
8 % import { Input } from "@components/ui/input";
9 % import { Label } from "@components/ui/label";
10 % import { Tabs, TabsContent, TabsList, TabsTrigger } from "@components/ui/tabs";
11 % import { Dialog, DialogContent, DialogDescription, DialogHeader, DialogTitle, DialogFooter } from "@components/ui/di...
12 % import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/select";
13 % import { Switch } from "@components/ui/switch";
14 % import { Skeleton } from "@components/ui/skeleton";
15 % import { Plus, Settings, RefreshCw, Check, X, AlertCircle, Clock, Trash2, Play } from "lucide-react";
16 % import type { OTChannelWithStats, OTAReservationLogWithChannel, OTChannelType, OTChannelStatus, OTASyncStatus } fr...
17 %
18 % const channelTypeLabels: Record<OTChannelType, string> = {
19 %   booking: "Booking.com",
20 %   expedia: "Expedia",
21 %   airbnb: "Airbnb",
22 %   despegar: "Despegar",
23 %   hotelbeds: "Hotelbeds",
24 %   agoda: "Agoda",
25 %   trivago: "Trivago",
26 %   manual: "Manual",
27 % };
28 %
29 % const channelTypeColors: Record<OTChannelType, string> = {
30 %   booking: "bg-blue-500",
31 %   expedia: "bg-yellow-500",
32 %   airbnb: "bg-rose-500",
33 %   despegar: "bg-green-500",
34 %   hotelbeds: "bg-orange-500",
35 %   agoda: "bg-purple-500",
36 %   trivago: "bg-cyan-500",
37 %   manual: "bg-slate-500",
38 % };
39 %
40 % const statusBadge: Record<OTChannelStatus, { variant: "default" | "secondary" | "destructive" | "outline"; label: st...
41 %   active: { variant: "default", label: "Activo" },
42 %   inactive: { variant: "secondary", label: "Inactivo" },
43 %   pending: { variant: "outline", label: "Pendiente" },
44 %   error: { variant: "destructive", label: "Error" },
45 % };
46 %
47 % const syncStatusBadge: Record<OTASyncStatus, { variant: "default" | "secondary" | "destructive" | "outline"; label: s...
48 %   pending: { variant: "outline", label: "Pendiente" },
49 %   synced: { variant: "default", label: "Sincronizado" },
50 %   failed: { variant: "destructive", label: "Fallido" },
51 %   cancelled: { variant: "secondary", label: "Cancelado" },
52 % };
53 %
54 % export default function OTChannelsPage() {
55 %   const { toast } = useToast();
56 %   const [isAddDialogOpen, setIsAddDialogOpen] = useState(false);
57 %   const [selectedChannel, setSelectedChannel] = useState<OTChannelWithStats | null>(null);
58 %   const [newChannel, setNewChannel] = useState({
59 %     name: "",
60 %     channelType: "booking" as OTChannelType,
61 %     commissionPercent: "15.00",
62 %     apiKey: "",
63 %     apiSecret: "",
64 %     hotelCode: "",
65 %   });
66 %
67 %   const { data: channels, isLoading: channelsLoading } = useQuery<OTChannelWithStats[]>({
68 %     queryKey: ["/api/ota-channels"],
69 %   });
70 %
71 %   const { data: reservationLogs, isLoading: logsLoading } = useQuery<OTAReservationLogWithChannel[]>({
72 %     queryKey: ["/api/ota-reservations"],
73 %   });
74 %
75 %   const createChannelMutation = useMutation({
76 %     mutationFn: async (data: typeof newChannel) => {
77 %       return apiRequest("POST", "/api/ota-channels", {
78 %         ...data,
79 %         status: "inactive",
80 %         syncEnabled: "false",
81 %       });
82 %     },
83 %     onSuccess: () => {
84 %       queryClient.invalidateQueries({ queryKey: ["/api/ota-channels"] });
85 %       setIsAddDialogOpen(false);
86 %       setNewChannel({
87 %         name: "",
88 %         channelType: "booking",
89 %         commissionPercent: "15.00",
90 %         apiKey: "",
```

client/src/pages/ota-channels.tsx (cont.)

```
91 %         apiSecret: "",
92 %         hotelCode: "",
93 %     });
94 %     toast({ title: "Canal creado exitosamente" });
95 % },
96 %     onError: () => {
97 %         toast({ title: "Error al crear el canal", variant: "destructive" });
98 %     },
99 % });
100 %
101 % const updateChannelMutation = useMutation({
102 %     mutationFn: async ({ id, data }: { id: string; data: Partial<OTACHannelWithStats> }) => {
103 %         return apiRequest("PATCH", `/api/ota-channels/${id}`, data);
104 %     },
105 %     onSuccess: () => {
106 %         queryClient.invalidateQueries({ queryKey: ["/api/ota-channels"] });
107 %         toast({ title: "Canal actualizado" });
108 %     },
109 %     onError: () => {
110 %         toast({ title: "Error al actualizar el canal", variant: "destructive" });
111 %     },
112 % });
113 %
114 % const deleteChannelMutation = useMutation({
115 %     mutationFn: async (id: string) => {
116 %         return apiRequest("DELETE", `/api/ota-channels/${id}`);
117 %     },
118 %     onSuccess: () => {
119 %         queryClient.invalidateQueries({ queryKey: ["/api/ota-channels"] });
120 %         setSelectedChannel(null);
121 %         toast({ title: "Canal eliminado" });
122 %     },
123 %     onError: () => {
124 %         toast({ title: "Error al eliminar el canal", variant: "destructive" });
125 %     },
126 % });
127 %
128 % const simulateImportMutation = useMutation({
129 %     mutationFn: async (channelId: string) => {
130 %         return apiRequest("POST", `/api/ota-channels/${channelId}/simulate-import`);
131 %     },
132 %     onSuccess: () => {
133 %         queryClient.invalidateQueries({ queryKey: ["/api/ota-reservations"] });
134 %         toast({ title: "Reserva simulada importada" });
135 %     },
136 %     onError: () => {
137 %         toast({ title: "Error al simular importación", variant: "destructive" });
138 %     },
139 % });
140 %
141 % const syncReservationMutation = useMutation({
142 %     mutationFn: async (logId: string) => {
143 %         return apiRequest("POST", `/api/ota-reservations/${logId}/sync`);
144 %     },
145 %     onSuccess: () => {
146 %         queryClient.invalidateQueries({ queryKey: ["/api/ota-reservations"] });
147 %         queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
148 %         toast({ title: "Reserva sincronizada al sistema" });
149 %     },
150 %     onError: () => {
151 %         toast({ title: "Error al sincronizar reserva", variant: "destructive" });
152 %     },
153 % });
154 %
155 % const pendingCount = reservationLogs?.filter((log) => log.status === "pending").length || 0;
156 %
157 % return (
158 %     <div className="p-6 space-y-6">
159 %         <div className="flex items-center justify-between gap-4 flex-wrap">
160 %             <div>
161 %                 <h1 className="text-2xl font-semibold" data-testid="text-page-title">Canales OTA</h1>
162 %                 <p className="text-muted-foreground">
163 %                     Gestiona las integraciones con agencias de viajes online
164 %                 </p>
165 %             </div>
166 %             <Button onClick={() => setIsAddDialogOpen(true)} data-testid="button-add-channel">
167 %                 <Plus className="h-4 w-4 mr-2" />
168 %                 Agregar Canal
169 %             </Button>
170 %         </div>
171 %
172 %         <Tabs defaultValue="channels" className="w-full">
173 %             <TabsList>
174 %                 <TabsTrigger value="channels" data-testid="tab-channels">Canales</TabsTrigger>
175 %                 <TabsTrigger value="reservations" data-testid="tab-reservations" className="gap-2">
176 %                     Reservas
177 %                     {pendingCount > 0 && (
178 %                         <Badge variant="secondary" className="ml-1">{pendingCount}</Badge>
179 %                     )}
180 %                 </TabsTrigger>
181 %             </TabsList>
182 %
183 %             <TabsContent value="channels" className="space-y-4 mt-4">
```

client/src/pages/ota-channels.tsx (cont.)

client/src/pages/ota-channels.tsx (cont.)

```
277 %         <div
278 %             key={log.id}
279 %             className="flex items-center justify-between gap-4 p-3 rounded-md bg-muted/50"
280 %             data-testid={`row-ota-reservation-${log.id}`}
281 %         >
282 %             <div className="flex items-center gap-3">
283 %                 <div className={`w-2 h-8 rounded-full ${channelTypeColors[log.channel?.channelType as OTACHan...
284 %                     <div>
285 %                         <p className="font-medium">{log.guestName}</p>
286 %                         <p className="text-sm text-muted-foreground">
287 %                             {log.externalReservationId} - {log.checkInDate} al {log.checkOutDate}
288 %                         </p>
289 %                     </div>
290 %                 </div>
291 %                 <div className="flex items-center gap-3">
292 %                     <div className="text-right">
293 %                         <p className="font-medium">${log.totalAmount}</p>
294 %                         <p className="text-xs text-muted-foreground">
295 %                             Comisión: ${log.commission}
296 %                         </p>
297 %                     </div>
298 %                     <Badge variant={syncStatusBadge[log.status as OTASyncStatus].variant}>
299 %                         {syncStatusBadge[log.status as OTASyncStatus].label}
300 %                     </Badge>
301 %                     {log.status === "pending" && (
302 %                         <Button
303 %                             size="sm"
304 %                             onClick={() => syncReservationMutation.mutate(log.id)}
305 %                             disabled={syncReservationMutation.isPending}
306 %                             data-testid={`button-sync-${log.id}`}
307 %                         >
308 %                             <RefreshCw className={`h-4 w-4 mr-1 ${syncReservationMutation.isPending ? "animate-spin" ...
309 %                                 Sincronizar
310 %                             </Button>
311 %                         </div>
312 %                     </div>
313 %                 </div>
314 %             </div>
315 %         </div>
316 %     </CardContent>
317 % </Card>
318 % ) : (
319 %     <Card>
320 %         <CardContent className="py-10 text-center">
321 %             <p className="text-muted-foreground">No hay reservas de canales OTA</p>
322 %         </CardContent>
323 %     </Card>
324 % )}
325 % </TabsContent>
326 % </Tabs>
327 %
328 % <Dialog open={isAddDialogOpen} onOpenChange={setIsAddDialogOpen}>
329 %     <DialogContent className="sm:max-w-md">
330 %         <DialogHeader>
331 %             <DialogTitle>Agregar Canal OTA</DialogTitle>
332 %             <DialogDescription>
333 %                 Configure una nueva integración con una agencia de viajes online
334 %             </DialogDescription>
335 %         </DialogHeader>
336 %         <div className="space-y-4 py-4">
337 %             <div className="space-y-2">
338 %                 <Label htmlFor="channelType">Tipo de Canal</Label>
339 %                 <Select
340 %                     value={newChannel.channelType}
341 %                     onChange={(value) => setNewChannel({ ...newChannel, channelType: value as OTACHannelType, name: ...
342 %                 >
343 %                     <SelectTrigger data-testid="select-channel-type">
344 %                         <SelectValue placeholder="Seleccionar canal" />
345 %                     </SelectTrigger>
346 %                     <SelectContent>
347 %                         {Object.entries(channelTypeLabels).map(([value, label]) => (
348 %                             <SelectItem key={value} value={value}>{label}</SelectItem>
349 %                         ))}
350 %                     </SelectContent>
351 %                 </Select>
352 %             </div>
353 %             <div className="space-y-2">
354 %                 <Label htmlFor="name">Nombre</Label>
355 %                 <Input
356 %                     id="name"
357 %                     value={newChannel.name}
358 %                     onChange={(e) => setNewChannel({ ...newChannel, name: e.target.value })}
359 %                     placeholder="Nombre del canal"
360 %                     data-testid="input-channel-name"
361 %                 />
362 %             </div>
363 %             <div className="space-y-2">
364 %                 <Label htmlFor="commission">Comisión (%)</Label>
365 %                 <Input
366 %                     id="commission"
367 %                     value={newChannel.commissionPercent}
368 %                     onChange={(e) => setNewChannel({ ...newChannel, commissionPercent: e.target.value })}
369 %                     placeholder="15.00"
```

client/src/pages/ota-channels.tsx (cont.)

```
370 %         data-testid="input-commission"
371 %     />
372 % </div>
373 % <div className="space-y-2">
374 %     <Label htmlFor="hotelCode">Código de Hotel (opcional)</Label>
375 %     <Input
376 %         id="hotelCode"
377 %         value={newChannel.hotelCode}
378 %         onChange={(e) => setNewChannel({ ...newChannel, hotelCode: e.target.value })}
379 %         placeholder="Código en el canal"
380 %         data-testid="input-hotel-code"
381 %     />
382 % </div>
383 % </div>
384 % <DialogFooter>
385 %     <Button variant="outline" onClick={() => setIsAddDialogOpen(false)}>
386 %         Cancelar
387 %     </Button>
388 %     <Button
389 %         onClick={() => createChannelMutation.mutate(newChannel)}
390 %         disabled={!newChannel.name || createChannelMutation.isPending}
391 %         data-testid="button-save-channel"
392 %     >
393 %         Guardar
394 %     </Button>
395 % </DialogFooter>
396 % </DialogContent>
397 % </Dialog>
398 %
399 % <Dialog open={!selectedChannel} onOpenChange={() => setSelectedChannel(null)}>
400 %     <DialogContent className="sm:max-w-md">
401 %         {selectedChannel && (
402 %             <>
403 %                 <DialogHeader>
404 %                     <div className="flex items-center gap-2">
405 %                         <div className={`w-3 h-3 rounded-full ${channelTypeColors[selectedChannel.channelType as OTChannel...]}
406 %                         <DialogTitle>{selectedChannel.name}</DialogTitle>
407 %                     </div>
408 %                     <DialogDescription>
409 %                         {channelTypeLabels[selectedChannel.channelType as OTChannelType]}
410 %                     </DialogDescription>
411 %                 </DialogHeader>
412 %                 <div className="space-y-4 py-4">
413 %                     <div className="flex items-center justify-between">
414 %                         <Label>Estado</Label>
415 %                         <div className="flex items-center gap-2">
416 %                             <Switch
417 %                                 checked={selectedChannel.status === "active"}
418 %                                 onCheckedChange={(checked) => {
419 %                                     updateChannelMutation.mutate({
420 %                                         id: selectedChannel.id,
421 %                                         data: { status: checked ? "active" : "inactive" },
422 %                                     });
423 %                                     setSelectedChannel({ ...selectedChannel, status: checked ? "active" : "inactive" });
424 %                                 }}
425 %                                 data-testid="switch-channel-status"
426 %                             />
427 %                             <span className="text-sm">
428 %                                 {selectedChannel.status === "active" ? "Activo" : "Inactivo"}
429 %                             </span>
430 %                         </div>
431 %                     </div>
432 %                     <div className="grid grid-cols-2 gap-4 text-sm">
433 %                         <div>
434 %                             <span className="text-muted-foreground">Total Reservas</span>
435 %                             <p className="font-medium text-lg">{selectedChannel.totalReservations}</p>
436 %                         </div>
437 %                         <div>
438 %                             <span className="text-muted-foreground">Pendientes</span>
439 %                             <p className="font-medium text-lg">{selectedChannel.pendingSync}</p>
440 %                         </div>
441 %                         <div>
442 %                             <span className="text-muted-foreground">Ingresos Totales</span>
443 %                             <p className="font-medium text-lg">${selectedChannel.totalRevenue.toFixed(2)}</p>
444 %                         </div>
445 %                         <div>
446 %                             <span className="text-muted-foreground">Comisiones Pagadas</span>
447 %                             <p className="font-medium text-lg">${selectedChannel.totalCommission.toFixed(2)}</p>
448 %                         </div>
449 %                     </div>
450 %                     {selectedChannel.lastSyncAt && (
451 %                         <div className="text-sm text-muted-foreground">
452 %                             Última sincronización: {new Date(selectedChannel.lastSyncAt).toLocaleString("es-AR")}
453 %                         </div>
454 %                     )}
455 %                 </div>
456 %                 <DialogFooter className="flex-col sm:flex-row gap-2">
457 %                     <Button
458 %                         variant="outline"
459 %                         onClick={() => simulateImportMutation.mutate(selectedChannel.id)}
460 %                         disabled={simulateImportMutation.isPending}
461 %                         data-testid="button-simulate-import"
462 %                     >
```

client/src/pages/ota-channels.tsx (cont.)

```
463 %           <Play className="h-4 w-4 mr-2" />
464 %           Simular Reserva
465 %         </Button>
466 %         <Button
467 %           variant="destructive"
468 %           onClick={() => deleteChannelMutation.mutate(selectedChannel.id)}
469 %           disabled={deleteChannelMutation.isPending}
470 %           data-testid="button-delete-channel"
471 %         >
472 %           <Trash2 className="h-4 w-4 mr-2" />
473 %           Eliminar
474 %         </Button>
475 %       </DialogFooter>
476 %     </>
477 %   )}
478 % </DialogContent>
479 % </Dialog>
480 % </div>
481 % );
482 % }
483 %
```

```
1 % import { useState } from "react";
2 % import { useQuery, useMutation } from "@tanstack/react-query";
3 % import {
4 %   MessageSquare,
5 %   Plus,
6 %   Search,
7 %   Star,
8 %   TrendingUp,
9 %   TrendingDown,
10 %   Minus,
11 %   RefreshCw,
12 %   Trash2,
13 %   Eye,
14 %   Sparkles,
15 %   ThumbsUp,
16 %   ThumbsDown,
17 %   AlertCircle,
18 % } from "lucide-react";
19 % import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
20 % import { Button } from "@components/ui/button";
21 % import { Input } from "@components/ui/input";
22 % import { Badge } from "@components/ui/badge";
23 % import { Skeleton } from "@components/ui/skeleton";
24 % import { Textarea } from "@components/ui/textarea";
25 % import { Label } from "@components/ui/label";
26 % import {
27 %   Dialog,
28 %   DialogContent,
29 %   DialogDescription,
30 %   DialogFooter,
31 %   DialogHeader,
32 %   DialogTitle,
33 % } from "@components/ui/dialog";
34 % import {
35 %   Select,
36 %   SelectContent,
37 %   SelectItem,
38 %   SelectTrigger,
39 %   SelectValue,
40 % } from "@components/ui/select";
41 % import {
42 %   AlertDialog,
43 %   AlertDialogAction,
44 %   AlertDialogCancel,
45 %   AlertDialogContent,
46 %   AlertDialogDescription,
47 %   AlertDialogFooter,
48 %   AlertDialogHeader,
49 %   AlertDialogTitle,
50 % } from "@components/ui/alert-dialog";
51 % import { useToast } from "@hooks/use-toast";
52 % import { apiRequest, queryClient } from "@lib/queryClient";
53 % import type { GuestReviewWithDetails, Guest, SentimentType } from "@shared/schema";
54 %
55 % type ReviewAnalytics = {
56 %   totalReviews: number;
57 %   averageRating: number;
58 %   sentimentBreakdown: { positive: number; neutral: number; negative: number };
59 %   topCategories: { category: string; count: number; avgSentiment: number }[];
60 %   recentTrend: { date: string; avgRating: number; count: number }[];
61 %   improvementAreas: string[];
62 % };
63 %
64 % function SentimentBadge({ sentiment }: { sentiment: SentimentType | null }) {
65 %   if (!sentiment) {
66 %     return <Badge variant="outline">Sin analizar</Badge>;
67 %   }
68 %
69 %   const config: Record<SentimentType, { label: string; variant: "default" | "secondary" | "destructive"; icon: typeof...
70 %     positive: { label: "Positivo", variant: "default", icon: TrendingUp },
71 %     neutral: { label: "Neutral", variant: "secondary", icon: Minus },
72 %     negative: { label: "Negativo", variant: "destructive", icon: TrendingDown },
73 %   };
74 %
75 %   const { label, variant, icon: Icon } = config[sentiment];
76 %   return (
77 %     <Badge variant={variant} className="gap-1">
78 %       <Icon className="h-3 w-3" />
79 %       {label}
80 %     </Badge>
81 %   );
82 % }
83 %
84 % function StarRating({ rating }: { rating: number }) {
85 %   return (
86 %     <div className="flex items-center gap-0.5">
87 %       {[1, 2, 3, 4, 5].map((star) => (
88 %         <Star
89 %           key={star}
90 %           className={`h-4 w-4 ${star <= rating ? "fill-yellow-500 text-yellow-500" : "text-muted-foreground"}`}
```

client/src/pages/reviews.tsx (cont.)

```
91 %     />
92 %   )))
93 %   <span className="ml-1 text-sm text-muted-foreground">{rating}/5</span>
94 % </div>
95 % );
96 % }
97 %
98 % const categoryLabels: Record<string, string> = {
99 %   service: "Servicio",
100 %   cleanliness: "Limpieza",
101 %   location: "Ubicacion",
102 %   amenities: "Comodidades",
103 %   value: "Relacion Calidad-Precio",
104 %   food: "Gastronomia",
105 %   staff: "Personal",
106 %   general: "General",
107 % };
108 %
109 % function ReviewFormDialog({
110 %   open,
111 %   onOpenChange,
112 %   onSuccess,
113 % }: {
114 %   open: boolean;
115 %   onOpenChange: (open: boolean) => void;
116 %   onSuccess: () => void;
117 % }) {
118 %   const { toast } = useToast();
119 %   const today = new Date().toISOString().split("T")[0];
120 %
121 %   const [formData, setFormData] = useState({
122 %     guestId: "",
123 %     rating: 5,
124 %     title: "",
125 %     content: "",
126 %     source: "direct",
127 %     reviewDate: today,
128 %   });
129 %
130 %   const { data: guests } = useQuery<Guest[]>({
131 %     queryKey: ["/api/guests"],
132 %   });
133 %
134 %   const createMutation = useMutation({
135 %     mutationFn: async (data: typeof formData) => {
136 %       const response = await apiRequest("POST", "/api/reviews", data);
137 %       return response.json();
138 %     },
139 %     onSuccess: () => {
140 %       queryClient.invalidateQueries({ queryKey: ["/api/reviews"] });
141 %       queryClient.invalidateQueries({ queryKey: ["/api/reviews/analytics"] });
142 %       toast({ title: "Resena creada exitosamente" });
143 %       onSuccess();
144 %       onOpenChange(false);
145 %       setFormData({ guestId: "", rating: 5, title: "", content: "", source: "direct", reviewDate: today });
146 %     },
147 %     onError: () => {
148 %       toast({ title: "Error al crear resena", variant: "destructive" });
149 %     },
150 %   });
151 %
152 %   return (
153 %     <Dialog open={open} onOpenChange={onOpenChange}>
154 %       <DialogContent className="max-w-md">
155 %         <DialogHeader>
156 %           <DialogTitle>Nueva Resena</DialogTitle>
157 %           <DialogDescription>Registrar una resena de huesped</DialogDescription>
158 %         </DialogHeader>
159 %
160 %         <form
161 %           onSubmit={(e) => {
162 %             e.preventDefault();
163 %             createMutation.mutate(formData);
164 %           }}
165 %           className="space-y-4"
166 %         >
167 %           <div>
168 %             <Label>Huesped *</Label>
169 %             <Select value={formData.guestId} onValueChange={(v) => setFormData({ ...formData, guestId: v })}>
170 %               <SelectTrigger data-testid="select-guest">
171 %                 <SelectValue placeholder="Seleccionar huesped" />
172 %               </SelectTrigger>
173 %               <SelectContent>
174 %                 {guests?.map((guest) => (
175 %                   <SelectItem key={guest.id} value={guest.id}>
176 %                     {guest.firstName} {guest.lastName}
177 %                   </SelectItem>
178 %                 ))}
179 %               </SelectContent>
180 %             </Select>
181 %           </div>
182 %
183 %           <div>
```


client/src/pages/reviews.tsx (cont.)

```
184 %         <Label>Calificacion *</Label>
185 %         <div className="flex items-center gap-1 mt-1">
186 %             {[1, 2, 3, 4, 5].map((star) => (
187 %                 <button
188 %                     key={star}
189 %                     type="button"
190 %                     onClick={() => setFormData({ ...formData, rating: star })}
191 %                     className="p-1"
192 %                     data-testid={`button-rating-${star}`}
193 %                 >
194 %                     <Star
195 %                         className={`h-6 w-6 ${star <= formData.rating ? "fill-yellow-500 text-yellow-500" : "text-muted-f...
196 %                     />
197 %                 </button>
198 %             )]}
199 %         </div>
200 %     </div>
201 %
202 %     <div>
203 %         <Label>Titulo</Label>
204 %         <Input
205 %             value={formData.title}
206 %             onChange={(e) => setFormData({ ...formData, title: e.target.value })}
207 %             placeholder="Resumen de la experiencia"
208 %             data-testid="input-review-title"
209 %         />
210 %     </div>
211 %
212 %     <div>
213 %         <Label>Contenido *</Label>
214 %         <Textarea
215 %             value={formData.content}
216 %             onChange={(e) => setFormData({ ...formData, content: e.target.value })}
217 %             placeholder="Detalle de la experiencia del huesped..."
218 %             rows={4}
219 %             data-testid="input-review-content"
220 %         />
221 %     </div>
222 %
223 %     <div className="grid grid-cols-2 gap-4">
224 %         <div>
225 %             <Label>Fuente</Label>
226 %             <Select value={formData.source} onValueChange={(v) => setFormData({ ...formData, source: v })}>
227 %                 <SelectTrigger data-testid="select-source">
228 %                     <SelectValue />
229 %                 </SelectTrigger>
230 %                 <SelectContent>
231 %                     <SelectItem value="direct">Directo</SelectItem>
232 %                     <SelectItem value="booking">Booking</SelectItem>
233 %                     <SelectItem value="expedia">Expedia</SelectItem>
234 %                     <SelectItem value="tripadvisor">TripAdvisor</SelectItem>
235 %                     <SelectItem value="google">Google</SelectItem>
236 %                 </SelectContent>
237 %             </Select>
238 %         </div>
239 %         <div>
240 %             <Label>Fecha</Label>
241 %             <Input
242 %                 type="date"
243 %                 value={formData.reviewDate}
244 %                 onChange={(e) => setFormData({ ...formData, reviewDate: e.target.value })}
245 %                 data-testid="input-review-date"
246 %             />
247 %         </div>
248 %     </div>
249 %
250 %     <DialogFooter>
251 %         <Button type="button" variant="outline" onClick={() => onOpenChange(false)}>
252 %             Cancelar
253 %         </Button>
254 %         <Button type="submit" disabled={createMutation.isPending || !formData.guestId || !formData.content} data-...
255 %             {createMutation.isPending ? "Guardando..." : "Guardar"}
256 %         </Button>
257 %     </DialogFooter>
258 % </form>
259 % </DialogContent>
260 % </Dialog>
261 % );
262 % }
263 %
264 % function ReviewDetailDialog({
265 %     review,
266 %     open,
267 %     onOpenChange,
268 % }: {
269 %     review: GuestReviewWithDetails | null;
270 %     open: boolean;
271 %     onOpenChange: (open: boolean) => void;
272 % }) {
273 %     const { toast } = useToast();
274 %
275 %     const analyzeMutation = useMutation({
276 %         mutationFn: async () => {
```

client/src/pages/reviews.tsx (cont.)

```
277 %     const response = await apiRequest("POST", `/api/reviews/${review?.id}/analyze`);
278 %     return response.json();
279 % },
280 % onSuccess: () => {
281 %     queryClient.invalidateQueries({ queryKey: ["/api/reviews"] });
282 %     queryClient.invalidateQueries({ queryKey: ["/api/reviews/analytics"] });
283 %     toast({ title: "Análisis de sentimiento completado" });
284 %     setOpenChange(false);
285 % },
286 % onError: () => {
287 %     toast({ title: "Error al analizar reseña", variant: "destructive" });
288 % },
289 % });
290 %
291 % if (!review) return null;
292 %
293 % return (
294 %     <Dialog open={open} onOpenChange={setOpenChange}>
295 %         <DialogContent className="max-w-lg">
296 %             <DialogHeader>
297 %                 <DialogTitle className="flex items-center gap-2">
298 %                     <MessageSquare className="h-5 w-5" />
299 %                     Detalle de Reseña
300 %                 </DialogTitle>
301 %             </DialogHeader>
302 %
303 %             <div className="space-y-4">
304 %                 <div className="flex items-center justify-between gap-4 flex-wrap">
305 %                     <div>
306 %                         <p className="text-sm text-muted-foreground">Huesped</p>
307 %                         <p className="font-medium">{review.guest?.firstName} {review.guest?.lastName}</p>
308 %                     </div>
309 %                     <div>
310 %                         <p className="text-sm text-muted-foreground">Fecha</p>
311 %                         <p className="font-medium">{new Date(review.reviewDate).toLocaleDateString("es-AR")}</p>
312 %                     </div>
313 %                     <div>
314 %                         <p className="text-sm text-muted-foreground">Fuente</p>
315 %                         <Badge variant="outline">{review.source}</Badge>
316 %                     </div>
317 %                 </div>
318 %
319 %                 <div className="flex items-center justify-between gap-2 flex-wrap">
320 %                     <StarRating rating={review.rating} />
321 %                     <SentimentBadge sentiment={review.sentiment as SentimentType | null} />
322 %                 </div>
323 %
324 %                 {review.title && (
325 %                     <div>
326 %                         <p className="text-sm text-muted-foreground">Titulo</p>
327 %                         <p className="font-medium">{review.title}</p>
328 %                     </div>
329 %                 )}
330 %
331 %                 <div>
332 %                     <p className="text-sm text-muted-foreground">Contenido</p>
333 %                     <p className="mt-1 text-sm">{review.content}</p>
334 %                 </div>
335 %
336 %                 {review.analyzedAt && (
337 %                     <div>
338 %                         {review.categories && review.categories.length > 0 && (
339 %                             <div>
340 %                                 <p className="text-sm text-muted-foreground mb-2">Categorías Detectadas</p>
341 %                                 <div className="flex flex-wrap gap-1">
342 %                                     {review.categories.map((cat) => (
343 %                                         <Badge key={cat} variant="secondary">
344 %                                             {categoryLabels[cat] || cat}
345 %                                         </Badge>
346 %                                     ))}
347 %                                 </div>
348 %                             </div>
349 %                         )}
350 %
351 %                         {review.keyPhrases && review.keyPhrases.length > 0 && (
352 %                             <div>
353 %                                 <p className="text-sm text-muted-foreground mb-2">Frases Clave</p>
354 %                                 <div className="flex flex-wrap gap-1">
355 %                                     {review.keyPhrases.map((phrase, i) => (
356 %                                         <Badge key={i} variant="outline">
357 %                                             {phrase}
358 %                                         </Badge>
359 %                                     ))}
360 %                                 </div>
361 %                             </div>
362 %                         )}
363 %
364 %                         {review.improvementSuggestions && review.improvementSuggestions.length > 0 && (
365 %                             <div className="rounded-md bg-destructive/10 p-3">
366 %                                 <p className="text-sm font-medium text-destructive mb-2 flex items-center gap-1">
367 %                                     <AlertCircle className="h-4 w-4" />
368 %                                     Áreas de Mejora
369 %                                 </p>
370 %                             </div>
371 %                         )}
372 %                     </div>
373 %                 )}
374 %             </DialogContent>
375 %         </Dialog>
376 %     );
377 % }
```

client/src/pages/reviews.tsx (cont.)

```
370 %         <ul className="text-sm space-y-1">
371 %             {review.improvementSuggestions.map((suggestion, i) => (
372 %                 <li key={i}>- {suggestion}</li>
373 %             ))}
374 %         </ul>
375 %     </div>
376 % })
377 %
378 %     <p className="text-xs text-muted-foreground">
379 %         Analizado: {new Date(review.analyzedAt).toLocaleString("es-AR")}
380 %     </p>
381 % </>
382 % })
383 %
384 %     {!review.analyzedAt && (
385 %         <Button
386 %             onClick={() => analyzeMutation.mutate()}
387 %             disabled={analyzeMutation.isPending}
388 %             className="w-full"
389 %             data-testid="button-analyze-review"
390 %         >
391 %             <Sparkles className="h-4 w-4 mr-2" />
392 %             {analyzeMutation.isPending ? "Analizando..." : "Analizar Sentimiento con IA"}
393 %         </Button>
394 %     )}
395 % </div>
396 % </DialogContent>
397 % </Dialog>
398 % );
399 % }
400 %
401 % export default function ReviewsPage() {
402 %     const { toast } = useToast();
403 %     const [searchQuery, setSearchQuery] = useState("");
404 %     const [sentimentFilter, setSentimentFilter] = useState<string>("all");
405 %     const [showFormDialog, setShowFormDialog] = useState(false);
406 %     const [selectedReview, setSelectedReview] = useState<GuestReviewWithDetails | null>(null);
407 %     const [deleteConfirmReview, setDeleteConfirmReview] = useState<GuestReviewWithDetails | null>(null);
408 %
409 %     const { data: reviews, isLoading } = useQuery<GuestReviewWithDetails[]>({
410 %         queryKey: ["/api/reviews"],
411 %     });
412 %
413 %     const { data: analytics } = useQuery<ReviewAnalytics>({
414 %         queryKey: ["/api/reviews/analytics"],
415 %     });
416 %
417 %     const analyzeAllMutation = useMutation({
418 %         mutationFn: () => apiRequest("POST", "/api/reviews/analyze-all"),
419 %         onSuccess: async (response) => {
420 %             const result = await response.json();
421 %             queryClient.invalidateQueries({ queryKey: ["/api/reviews"] });
422 %             queryClient.invalidateQueries({ queryKey: ["/api/reviews/analytics"] });
423 %             toast({ title: result.message });
424 %         },
425 %         onError: () => {
426 %             toast({ title: "Error al analizar resenas", variant: "destructive" });
427 %         },
428 %     });
429 %
430 %     const deleteMutation = useMutation({
431 %         mutationFn: (id: string) => apiRequest("DELETE", `/api/reviews/${id}`),
432 %         onSuccess: () => {
433 %             queryClient.invalidateQueries({ queryKey: ["/api/reviews"] });
434 %             queryClient.invalidateQueries({ queryKey: ["/api/reviews/analytics"] });
435 %             toast({ title: "Resena eliminada" });
436 %             setDeleteConfirmReview(null);
437 %         },
438 %         onError: () => {
439 %             toast({ title: "Error al eliminar resena", variant: "destructive" });
440 %         },
441 %     });
442 %
443 %     const filteredReviews = reviews?.filter((review) => {
444 %         const matchesSearch =
445 %             review.content.toLowerCase().includes(searchQuery.toLowerCase()) ||
446 %             review.title?.toLowerCase().includes(searchQuery.toLowerCase()) ||
447 %             `${review.guest?.firstName} ${review.guest?.lastName}`.toLowerCase().includes(searchQuery.toLowerCase());
448 %
449 %         const matchesSentiment =
450 %             sentimentFilter === "all" ||
451 %             review.sentiment === sentimentFilter ||
452 %             (sentimentFilter === "unanalyzed" && !review.analyzedAt);
453 %
454 %         return matchesSearch && matchesSentiment;
455 %     });
456 %
457 %     const unanalyzedCount = reviews?.filter((r) => !r.analyzedAt).length || 0;
458 %
459 %     return (
460 %         <div className="p-6 space-y-6">
461 %             <div className="flex items-center justify-between gap-4 flex-wrap">
462 %                 <div className="flex items-center gap-2">
```

client/src/pages/reviews.tsx (cont.)

```
463 %         <MessageSquare className="h-6 w-6" />
464 %         <h1 className="text-2xl font-semibold">Resenas de Huespedes</h1>
465 %       </div>
466 %       <div className="flex items-center gap-2 flex-wrap">
467 %         {unanalyzedCount > 0 && (
468 %           <Button
469 %             variant="outline"
470 %             onClick={() => analyzeAllMutation.mutate()}
471 %             disabled={analyzeAllMutation.isPending}
472 %             data-testid="button-analyze-all"
473 %           >
474 %             <Sparkles className="h-4 w-4 mr-2" />
475 %             {analyzeAllMutation.isPending ? "Analizando..." : `Analizar ${unanalyzedCount} pendientes`}
476 %           </Button>
477 %         )}
478 %         <Button onClick={() => setShowFormDialog(true)} data-testid="button-new-review">
479 %           <Plus className="h-4 w-4 mr-2" />
480 %           Nueva Resena
481 %         </Button>
482 %       </div>
483 %     </div>
484 %
485 %     {analytics && (
486 %       <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
487 %         <Card>
488 %           <CardHeader className="pb-2">
489 %             <CardTitle className="text-sm text-muted-foreground">Total Resenas</CardTitle>
490 %           </CardHeader>
491 %           <CardContent>
492 %             <div className="text-2xl font-bold" data-testid="text-total-reviews">{analytics.totalReviews}</div>
493 %           </CardContent>
494 %         </Card>
495 %
496 %         <Card>
497 %           <CardHeader className="pb-2">
498 %             <CardTitle className="text-sm text-muted-foreground">Calificacion Promedio</CardTitle>
499 %           </CardHeader>
500 %           <CardContent>
501 %             <div className="flex items-center gap-2">
502 %               <Star className="h-5 w-5 fill-yellow-500 text-yellow-500" />
503 %               <span className="text-2xl font-bold" data-testid="text-avg-rating">{analytics.averageRating}</span>
504 %               <span className="text-muted-foreground">/ 5</span>
505 %             </div>
506 %           </CardContent>
507 %         </Card>
508 %
509 %         <Card>
510 %           <CardHeader className="pb-2">
511 %             <CardTitle className="text-sm text-muted-foreground">Sentimiento</CardTitle>
512 %           </CardHeader>
513 %           <CardContent>
514 %             <div className="flex items-center gap-3">
515 %               <div className="flex items-center gap-1 text-green-600">
516 %                 <ThumbsUp className="h-4 w-4" />
517 %                 <span className="font-medium" data-testid="text-positive-count">{analytics.sentimentBreakdown.posit...
518 %               </div>
519 %               <div className="flex items-center gap-1 text-muted-foreground">
520 %                 <Minus className="h-4 w-4" />
521 %                 <span className="font-medium" data-testid="text-neutral-count">{analytics.sentimentBreakdown.neutra...
522 %               </div>
523 %               <div className="flex items-center gap-1 text-red-600">
524 %                 <ThumbsDown className="h-4 w-4" />
525 %                 <span className="font-medium" data-testid="text-negative-count">{analytics.sentimentBreakdown.negot...
526 %               </div>
527 %             </div>
528 %           </CardContent>
529 %         </Card>
530 %
531 %         <Card>
532 %           <CardHeader className="pb-2">
533 %             <CardTitle className="text-sm text-muted-foreground">Areas de Mejora</CardTitle>
534 %           </CardHeader>
535 %           <CardContent>
536 %             {analytics.improvementAreas.length > 0 ? (
537 %               <div className="flex flex-wrap gap-1">
538 %                 {analytics.improvementAreas.slice(0, 3).map((area, i) => (
539 %                   <Badge key={i} variant="destructive" className="text-xs">
540 %                     {area.slice(0, 30)}...
541 %                   </Badge>
542 %                 )}
543 %               </div>
544 %             ) : (
545 %               <p className="text-sm text-muted-foreground">Sin sugerencias</p>
546 %             )}
547 %           </CardContent>
548 %         </Card>
549 %       </div>
550 %     )}
551 %
552 %     <div className="flex items-center gap-4 flex-wrap">
553 %       <div className="relative flex-1 min-w-[200px]">
554 %         <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" />
555 %         <Input
```

client/src/pages/reviews.tsx (cont.)

```
556 %         placeholder="Buscar resenas..."
557 %         value={searchQuery}
558 %         onChange={(e) => setSearchQuery(e.target.value)}
559 %         className="pl-9"
560 %         data-testid="input-search-reviews"
561 %       />
562 %     </div>
563 %     <Select value={sentimentFilter} onValueChange={setSentimentFilter}>
564 %       <SelectTrigger className="w-[180px]" data-testid="select-sentiment-filter">
565 %         <SelectValue placeholder="Filtrar por sentimiento" />
566 %       </SelectTrigger>
567 %       <SelectContent>
568 %         <SelectItem value="all">Todos</SelectItem>
569 %         <SelectItem value="positive">Positivos</SelectItem>
570 %         <SelectItem value="neutral">Neutros</SelectItem>
571 %         <SelectItem value="negative">Negativos</SelectItem>
572 %         <SelectItem value="unanalyzed">Sin Analizar</SelectItem>
573 %       </SelectContent>
574 %     </Select>
575 %   </div>
576 %
577 %   {isLoading ? (
578 %     <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
579 %       {[1, 2, 3, 4, 5, 6].map((i) => (
580 %         <Skeleton key={i} className="h-48" />
581 %       ))}
582 %     </div>
583 %   ) : filteredReviews && filteredReviews.length > 0 ? (
584 %     <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
585 %       {filteredReviews.map((review) => (
586 %         <Card key={review.id} className="hover-elevate cursor-pointer" onClick={() => setSelectedReview(review)} ...
587 %           <CardHeader className="pb-2">
588 %             <div className="flex items-center justify-between gap-2">
589 %               <CardTitle className="text-sm font-medium truncate">
590 %                 {review.guest?.firstName} {review.guest?.lastName}
591 %               </CardTitle>
592 %               <SentimentBadge sentiment={review.sentiment as SentimentType | null} />
593 %             </div>
594 %           </CardHeader>
595 %           <CardContent className="space-y-2">
596 %             <div className="flex items-center justify-between gap-2 flex-wrap">
597 %               <StarRating rating={review.rating} />
598 %               <Badge variant="outline" className="text-xs">{review.source}</Badge>
599 %             </div>
600 %             {review.title && <p className="font-medium text-sm truncate">{review.title}</p>}
601 %             <p className="text-sm text-muted-foreground line-clamp-2">{review.content}</p>
602 %             <div className="flex items-center justify-between gap-2 pt-2">
603 %               <span className="text-xs text-muted-foreground">
604 %                 {new Date(review.reviewDate).toLocaleDateString("es-AR")}
605 %               </span>
606 %               <div className="flex items-center gap-1">
607 %                 <Button
608 %                   size="icon"
609 %                   variant="ghost"
610 %                   onClick={(e) => {
611 %                     e.stopPropagation();
612 %                     setSelectedReview(review);
613 %                   }}
614 %                   data-testid={`button-view-review-${review.id}`}
615 %                 >
616 %                   <Eye className="h-4 w-4" />
617 %                 </Button>
618 %                 <Button
619 %                   size="icon"
620 %                   variant="ghost"
621 %                   onClick={(e) => {
622 %                     e.stopPropagation();
623 %                     setDeleteConfirmReview(review);
624 %                   }}
625 %                   data-testid={`button-delete-review-${review.id}`}
626 %                 >
627 %                   <Trash2 className="h-4 w-4" />
628 %                 </Button>
629 %               </div>
630 %             </div>
631 %           </CardContent>
632 %         </Card>
633 %       )}
634 %     </div>
635 %   ) : (
636 %     <Card>
637 %       <CardContent className="flex flex-col items-center justify-center py-12">
638 %         <MessageSquare className="h-12 w-12 text-muted-foreground mb-4" />
639 %         <p className="text-muted-foreground">No hay resenas registradas</p>
640 %         <Button onClick={() => setShowFormDialog(true)} variant="outline" className="mt-4">
641 %           <Plus className="h-4 w-4 mr-2" />
642 %           Agregar primera resena
643 %         </Button>
644 %       </CardContent>
645 %     </Card>
646 %   )}
647 %
648 % </ReviewFormDialog>
```

client/src/pages/reviews.tsx (cont.)

```
649 %         open={showFormDialog}
650 %         onOpenChange={setShowFormDialog}
651 %         onSuccess={() => {
652 %             queryClient.invalidateQueries({ queryKey: ["/api/reviews/analytics"] });
653 %         }}
654 %     />
655 %
656 %     <ReviewDetailDialog
657 %         review={selectedReview}
658 %         open={!selectedReview}
659 %         onOpenChange={(open) => !open && setSelectedReview(null)}
660 %     />
661 %
662 %     <AlertDialog open={!deleteConfirmReview} onOpenChange={(open) => !open && setDeleteConfirmReview(null)}>
663 %         <AlertDialogContent>
664 %             <AlertDialogHeader>
665 %                 <AlertDialogTitle>Eliminar resena</AlertDialogTitle>
666 %                 <AlertDialogDescription>
667 %                     Esta accion no se puede deshacer. Se eliminara permanentemente la resena.
668 %                 </AlertDialogDescription>
669 %             </AlertDialogHeader>
670 %             <AlertDialogFooter>
671 %                 <AlertDialogCancel>Cancelar</AlertDialogCancel>
672 %                 <AlertDialogAction
673 %                     onClick={() => deleteConfirmReview && deleteMutation.mutate(deleteConfirmReview.id)}
674 %                     className="bg-destructive text-destructive-foreground"
675 %                 >
676 %                     Eliminar
677 %                 </AlertDialogAction>
678 %             </AlertDialogFooter>
679 %         </AlertDialogContent>
680 %     </AlertDialog>
681 % </div>
682 % );
683 % }
684 %
```

```
1 % import { Card, CardContent } from "@components/ui/card";
2 % import { AlertCircle } from "lucide-react";
3 %
4 % export default function NotFound() {
5 %   return (
6 %     <div className="min-h-screen w-full flex items-center justify-center bg-gray-50">
7 %       <Card className="w-full max-w-md mx-4">
8 %         <CardContent className="pt-6">
9 %           <div className="flex mb-4 gap-2">
10 %             <AlertCircle className="h-8 w-8 text-red-500" />
11 %             <h1 className="text-2xl font-bold text-gray-900">404 Page Not Found</h1>
12 %           </div>
13 %
14 %           <p className="mt-4 text-sm text-gray-600">
15 %             Did you forget to add the page to the router?
16 %           </p>
17 %         </CardContent>
18 %       </Card>
19 %     </div>
20 %   );
21 % }
22 %
```